ESP32-S31

技术参考手册 预发布 v0.1

PRELIMINARY

ESPRESSIF

关于本文档

ESP32-S31 技术参考手册面向使用 ESP32-S31 系列产品进行底层软件开发的人员，介绍了 ESP32-S31 系列产品中内置的硬件模块，包括概述、功能列表、硬件架构、编程指南、寄存器列表等信息。

本文档中的跳转

在本文档中实现跳转，请参考以下建议：

- 发布进度速览（下一页）罗列了本文档中的所有章节，您可以从这里快速跳转至某个具体章节。
- 您还可以通过文档左侧的**书签**，从文中的任何位置直接跳转至另一个章节。注意，本文档已设置默认打开**书签**功能，但一些 PDF 阅读器或浏览器会忽略此设置。因此，如果您无法找到**书签**功能，请尝试以下方法：
  - 在您的浏览器中安装 PDF 阅读器拓展；
  - 下载本文档，使用本地 PDF 阅读器进行浏览；
  - 配置您的 PDF 阅读器，使其默认打开**书签**功能。
- 大多数 PDF 阅读器均支持跳转功能，允许您借助按钮、菜单选项或快捷键进行跳转（**向上**、**向下**、**向前**、**向后**、**后退**、**前进**及**前往页面**）等。
- 此外，您还可以使用本文档内置的 **GoBack** 按钮（每页右上角）快速后退至跳转之前的位置。注意，本功能仅适用于 Acrobat 系列的 PDF 阅读器（比如 Acrobat Reader 和 Adobe DC）以及内置 Acrobat 系列 PDF 阅读器或拓展的浏览器（比如 Firefox）。

发布进度速览

注意本文档尚未全部完成，具体发布进度见下表：

No.	章节	最新进度
第 I 卷：微处理器和主机		
1	总线架构 [to be added later]	67%
2	高性能处理器 [to be added later]	6%
3	RISC-V 追踪编码器 (TRACE)	已发布
4	低功耗处理器	已发布
5	通用 DMA 控制器 (GDMA-AHB, GDMA-AXI)	已发布
6	2D-DMA 控制器 (2D-DMA)	已发布
第 II 卷：存储器组织结构		
7	系统和存储器 [to be added later]	83%
8	eFuse 控制器 (eFuse) [to be added later]	18%
第 III 卷：系统组件		
9	IO MUX 和 GPIO 交换矩阵 [to be added later]	67%
10	复位和时钟 [to be added later]	53%
11	芯片 Boot 控制 [to be added later]	76%
12	中断矩阵 [to be added later]	17%
13	事件任务矩阵 (ETM) [to be added later]	6%
14	低功耗管理 [to be added later]	6%
15	系统定时器 [to be added later]	27%
16	定时器组 (TIMG) [to be added later]	45%
17	看门狗定时器 (WDT)	已发布
18	实时时钟定时器	已发布
19	权限控制 (PMS) [to be added later]	6%
20	系统寄存器 [to be added later]	6%
21	辅助调试	已发布
22	LP 信箱控制器	已发布
23	电源检测器 (PSDET)	已发布
第 IV 卷：加密和安全组件		
24	AES 加速器 (AES)	已发布
25	ECC 加速器 (ECC)	已发布
26	HMAC 加速器 (HMAC)	已发布
27	RSA 加速器 (RSA)	已发布
28	SHA 加速器 (SHA)	已发布
29	RSA 数字签名外设 (RSA_DS)	已发布
30	ECDSA 数字签名外设 (ECDSA_DS)	已发布
31	安全调试控制器 (SDC) [to be added later]	45%
32	片外存储器加密与解密 (XTS_AES) [to be added later]	17%
33	随机数发生器 (RNG) [to be added later]	94%
34	密钥管理器 [to be added later]	25%
第 V 卷：图像和语音处理		
35	JPEG 图像编解码器	已发布

No.	章节	最新进度
36	像素处理加速器 (PPA)	已发布
37	音频采样率转换器	已发布
38	CORDIC 加速器 (CORDIC) [to be added later]	15%
39	LCD 与 Camera 控制器 (LCD_CAM)	已发布
第 VI 卷：通讯接口		
40	UART 控制器 (UART)	已发布
41	SPI 控制器 (SPI) [to be added later]	27%
42	I2C 控制器 (I2C) [to be added later]	82%
43	I2S 控制器 (I2S) [to be added later]	55%
44	脉冲计数控制器 (PCNT) [to be added later]	64%
45	高速 USB 2.0 OTG [to be added later]	36%
46	USB 串口/JTAG 控制器	已发布
47	以太网介质访问控制器 (EMAC) [to be added later]	12%
48	双线汽车接口 (TWAI) [to be added later]	18%
49	SD/MMC 主机控制器 (SDHOST) [to be added later]	33%
50	LED PWM 控制器 (LEDC)	已发布
51	电机控制脉宽调制器 (MCPWM)	已发布
52	红外遥控 (RMT)	已发布
53	并行 IO 控制器 (PARLIO)	已发布
54	比特调节器 [to be added later]	27%
第 VII 卷：模拟信号处理		
55	触摸传感器 (TOUCH)	已发布
56	温度传感器 (TSENS) [to be added later]	82%
57	SAR ADC 数字控制器 (ADC)	已发布
58	DAC 控制器 (DAC) [to be added later]	75%
59	模拟电压比较器	已发布

说明：

点击链接或扫描二维码确保您使用的是最新版本的文档：
https://www.espressif.com/documentation/esp32-s31_technical_reference_manual_cn.pdf

目录

I 微处理器和主机 30

1 RISC-V 追踪编码器 (TRACE) 31
1.1 术语 31
1.2 介绍 31
1.3 特性 32
1.4 架构概览 34
1.5 功能描述 34
1.5.1 同步 34
1.5.2 地址模式 34
1.5.3 可选边带信号 35
1.5.4 过滤 35
1.5.5 锚定 36
1.5.6 写存储器模式 36
1.5.7 自动重启 36
1.6 编码器输出数据包 36
1.6.1 头部 36
1.6.2 索引 37
1.6.3 有效载荷 37
1.6.3.1 格式 3 37
1.6.3.2 格式 2 39
1.6.3.3 格式 1 39
1.7 中断 41
1.8 编程流程 42
1.8.1 配置编码器选项 42
1.8.2 配置过滤器 42
1.8.3 使能编码器 43
1.8.4 关闭编码器 44
1.8.5 通知 45
1.8.6 解码数据包 45
1.8.7 AHB 配置 45
1.8.8 软件保留 46
1.9 寄存器列表 47
1.10 寄存器 48

2 低功耗处理器 58
2.1 概述 58
2.2 特性 58
2.3 配置与状态寄存器 (CSR) 59
2.3.1 寄存器列表 59
2.3.2 寄存器 60
2.4 中断和异常 70

2.4.1 中断 70
2.4.2 中断处理 71
2.4.3 异常 71
2.5 调试 71
2.5.1 特性 72
2.5.2 功能描述 72
2.5.3 寄存器列表 72
2.5.4 寄存器 72
2.6 硬件触发器 74
2.6.1 特性 74
2.6.2 功能描述 74
2.6.3 触发执行流程 75
2.6.4 寄存器列表 75
2.6.5 寄存器 75
2.7 性能计数器 78
2.8 系统访问 79
2.8.1 存储器访问 79
2.8.2 外设访问 79
2.9 事件任务矩阵功能 79
2.10 睡眠和唤醒流程 79
2.10.1 特性 80
2.10.2 流程 80
2.10.3 唤醒源 81
2.10.4 拒绝睡眠 81
2.11 物理存储器保护 82
2.11.1 特性 82
2.11.2 功能描述 82
2.11.3 寄存器列表 82
2.11.4 寄存器 82

3 通用 DMA 控制器 (GDMA-AHB, GDMA-AXI) 85
3.1 概述 85
3.2 特性 85
3.3 架构 86
3.4 功能描述 87
3.4.1 链表 87
3.4.2 外设到存储器及存储器到外设的数据传输 88
3.4.3 存储器到存储器数据传输 89
3.4.4 启动 GDMA 90
3.4.5 读链表 90
3.4.6 GDMA-AXI 切换链表的中断响应机制 91
3.4.7 数据传输结束标志 91
3.4.8 访问存储器 92
3.4.9 仲裁 92
3.4.10 总线错误响应信息记录 93
3.4.11 自动时钟门控 94

3.5 事件任务矩阵功能 94
3.6 中断 95
3.7 编程流程 96
3.7.1 GDMA TX 通道配置流程 96
3.7.2 GDMA RX 通道配置流程 97
3.7.3 GDMA 存储器到存储器配置流程 97
3.7.4 通道优先级和权重配置流程 97
3.8 寄存器列表 99
3.8.1 GDMA-AHB 寄存器列表 99
3.8.1.1 GDMA-AHB-HP 寄存器列表 99
3.8.1.2 GDMA-AHB-LP 寄存器列表 107
3.8.2 GDMA-AXI 寄存器列表 110
3.9 寄存器 116
3.9.1 GDMA-AHB-HP 寄存器 116
3.9.2 GDMA-AHB-LP 寄存器 149
3.9.3 GDMA-AXI 寄存器 183

4 2D-DMA 控制器 (2D-DMA) 221
4.1 概述 221
4.2 特性 221
4.3 架构 221
4.4 功能描述 222
4.4.1 传输模式 222
4.4.2 链表 223
4.4.3 2D-MOD1 模式和 DSCR-PORT 模式的补数方式 225
4.4.4 外设到存储及存储到外设的数据传输 227
4.4.5 存储到存储数据传输 227
4.4.6 宏块重排功能 227
4.4.6.1 发送方向宏块重排功能 227
4.4.6.2 接收方向宏块重排功能 228
4.4.7 颜色空间转换功能 229
4.4.8 启动 2D-DMA 232
4.4.9 读链表 232
4.4.10 数据传输结束标志 232
4.4.11 访问存储 233
4.4.12 仲裁 233
4.5 事件任务矩阵功能 234
4.6 中断 235
4.7 编程流程 236
4.7.1 2D-DMA 通用配置 236
4.7.2 2D-DMA 模式专属配置 236
4.7.3 2D-DMA TX 通道配置流程 237
4.7.4 2D-DMA RX 通道配置流程 237
4.7.5 2D-DMA 存储器到存储器配置流程 237
4.7.6 通道优先级和权重配置流程 238
4.7.7 工作状态下复位流程 238

4.8 寄存器列表 239
4.9 寄存器 246

II 系统组件 289

5 看门狗定时器 (WDT) 290
5.1 概述 290
5.2 数字看门狗定时器 291
5.2.1 主要特性 291
5.2.2 功能描述 292
5.2.2.1 时钟源与 32 位计数器 292
5.2.2.2 阶段与超时动作 293
5.2.2.3 写保护 293
5.2.2.4 flash 引导保护 294
5.3 超级看门狗定时器 294
5.3.1 主要特性 294
5.3.2 SWD 控制器 294
5.3.2.1 结构 295
5.3.2.2 工作流程 295
5.4 中断 295
5.5 寄存器列表 296
5.6 寄存器 297

6 实时时钟定时器 305
6.1 概述 305
6.2 主要特性 305
6.3 功能描述 305
6.4 事件任务矩阵功能 306
6.5 中断 307
6.6 寄存器列表 308
6.7 寄存器 309

7 辅助调试 317
7.1 概述 317
7.2 主要特性 317
7.3 功能描述 317
7.3.1 区域读写监测 317
7.3.2 栈指针监测 317
7.3.3 PC 和 SP 记录 318
7.3.4 核心锁定监测 318
7.3.5 CPU/DMA 总线访问记录 318
7.4 中断 318
7.5 配置流程 319
7.5.1 区域监测和栈指针监测配置 319
7.5.2 PC 和 SP 记录配置 320
7.5.3 核心锁定监测配置 320

7.5.4 CPU/DMA 总线访问记录配置 321
7.5.4.1 总线访问记录 1 配置 321
7.5.4.2 总线访问记录 2 配置 324
7.6 寄存器列表 327
7.6.1 总线记录配置寄存器列表 327
7.6.2 其他寄存器列表 328
7.7 寄存器 330
7.7.1 总线记录配置寄存器 330
7.7.2 其他寄存器 338

8 LP 信箱控制器 358
8.1 概述 358
8.2 特性 358
8.3 功能描述 359
8.3.1 信息寄存器 359
8.3.2 中断 359
8.3.3 核间通信 360
8.4 寄存器列表 361
8.5 寄存器 362

9 电源检测器 (PSDET) 375
9.1 概述 375
9.2 主要特性 375
9.3 功能描述 375
9.3.1 架构图 375
9.3.2 欠压检测器 376
9.3.3 电压毛刺检测器 377
9.4 中断 378
9.5 寄存器列表 380
9.6 寄存器 381

III 加密和安全组件 388

10 AES 加速器 (AES) 389
10.1 概述 389
10.2 主要特性 389
10.3 时钟和复位 389
10.4 工作模式简介 390
10.5 Typical AES 工作模式 391
10.5.1 密钥、明文、密文 391
10.5.2 字节序 391
10.5.3 Typical AES 工作流程 393
10.6 DMA-AES 工作模式 393
10.6.1 密钥、明文、密文 394
10.6.2 字节序 394
10.6.3 标准增量函数 395

10.6.4 块个数 395
10.6.5 初始向量 395
10.6.6 块工作流程 395
10.6.7 GCM 工作流程 396
10.7 GCM 算法 397
10.7.1 哈希子密钥 398
10.7.2 J_0 398
10.7.3 认证标签 398
10.7.4 AAD 块个数 398
10.7.5 不完整块的有效位数 398
10.8 伪轮次抗 DPA 功能 399
10.9 中断 399
10.10 存储器列表 400
10.11 寄存器列表 400
10.12 寄存器 402

11 ECC 加速器 (ECC) 410
11.1 概述 410
11.2 主要特性 410
11.3 ECC 背景知识 410
11.3.1 椭圆曲线与曲线上的点 410
11.3.2 仿射坐标系与 Jacobian 坐标系 410
11.3.3 内存块 411
11.3.4 数据与数据块 411
11.3.5 数据存储 412
11.3.6 数据读取 412
11.3.7 标准运算与 Jacobian 运算 412
11.4 功能描述 412
11.4.1 曲线模式 412
11.4.2 工作模式 413
11.4.2.1 标准点乘模式 413
11.4.2.2 标准点验证模式 414
11.4.2.3 标准点验证 + 标准点乘模式 414
11.4.2.4 Jacobian 点乘模式 414
11.4.2.5 点加模式 415
11.4.2.6 Jacobian 点验证模式 415
11.4.2.7 标准点验证 + Jacobian 点乘模式 415
11.4.2.8 模加模式 416
11.4.2.9 模减模式 416
11.4.2.10 模乘模式 416
11.4.2.11 模除模式 417
11.4.3 提升抗攻击性能 417
11.4.4 时钟 417
11.4.5 复位 418
11.5 中断 418
11.6 配置流程 418

11.7 寄存器列表 419
11.8 寄存器 420

12 HMAC 加速器 (HMAC) 425
12.1 概述 425
12.2 主要特性 425
12.3 功能描述 425
12.3.1 上行模式 426
12.3.2 下行模式 - JTAG 启动功能 426
12.3.3 下行模式 - RSA 数字签名外设密钥派生功能 426
12.4 HMAC eFuse 配置 427
12.5 配置流程 428
12.5.1 启动 HMAC 模块 428
12.5.2 配置 HMAC 密钥和密钥功能 428
12.5.3 下行模式流程 428
12.5.4 上行模式流程 429
12.6 HMAC 算法细节 429
12.6.1 附加填充位 430
12.6.2 HMAC 算法结构 430
12.6.3 硬件占用 431
12.7 寄存器列表 432
12.8 寄存器 433

13 RSA 加速器 (RSA) 440
13.1 概述 440
13.2 主要特性 440
13.3 功能描述 440
13.3.1 定义与表达 441
13.3.2 大数模幂运算 441
13.3.3 大数模乘运算 442
13.3.4 大数乘法运算 442
13.3.5 控制加速 443
13.4 中断 444
13.5 存储器列表 446
13.6 寄存器列表 446
13.7 寄存器 447

14 SHA 加速器 (SHA) 452
14.1 概述 452
14.2 主要特性 452
14.3 工作模式简介 452
14.4 功能描述 453
14.4.1 信息预处理 453
14.4.1.1 附加填充比特 453
14.4.1.2 信息解析 454
14.4.1.3 设置哈希初始值 454

14.4.2 哈希运算流程 455
14.4.2.1 Typical SHA 模式下的运算流程 455
14.4.2.2 DMA-SHA 模式下的运算流程 457
14.4.3 信息摘要存储 459
14.5 中断 459
14.6 寄存器列表 461
14.7 寄存器 463

15 RSA 数字签名外设 (RSA_DS) 468
15.1 概述 468
15.2 主要特性 468
15.3 功能描述 468
15.3.1 概述 468
15.3.2 私钥运算子 469
15.3.3 软件准备工作 469
15.3.4 硬件工作流程 471
15.3.4.1 硬件占用 471
15.3.5 软件工作流程 472
15.4 存储器列表 474
15.5 寄存器列表 475
15.6 寄存器 476

16 ECDSA 数字签名外设 (ECDSA_DS) 480
16.1 概述 480
16.2 主要特性 480
16.3 ECDSA 背景知识 480
16.3.1 域参数 480
16.3.2 密钥生成 481
16.3.3 签名生成 481
16.3.4 签名验证 482
16.4 功能描述 482
16.4.1 ECDSA_DS 工作模式 482
16.4.2 密钥来源 483
16.4.3 数据和数据块 484
16.4.3.1 数据写入 484
16.4.3.2 数据读取 484
16.4.3.3 消息填充 484
16.4.3.4 解析消息 485
16.4.4 安全功能 485
16.4.4.1 抗攻击性能 485
16.4.4.2 基于状态的寄存器访问控制 485
16.4.4.3 硬件占用 485
16.5 配置流程 486
16.5.1 ECDSA_DS 流程 486
16.5.1.1 IDLE 阶段 487
16.5.1.2 PREP 阶段 488

16.5.1.3 LOAD 阶段 488
16.5.1.4 PROC 阶段 488
16.5.1.5 GAIN 阶段 489
16.5.1.6 POST 阶段 489
16.5.1.7 ECDSA_DS SHA 接口 489
16.5.2 时钟 490
16.5.3 复位 491
16.6 中断 491
16.7 存储器块 493
16.8 寄存器列表 494
16.9 寄存器 496

IV 图像和语音处理 507

17 JPEG 图像编解码器 508
17.1 术语 508
17.2 概述 509
17.3 主要特性 510
17.4 结构概览 511
17.5 功能描述 512
17.5.1 JPEG 编码器 512
17.5.1.1 暂停 512
17.5.1.2 颜色空间转换 513
17.5.1.3 可配置量化系数表 513
17.5.1.4 填充零字节 515
17.5.1.5 EOI 标记 515
17.5.1.6 超时检测 515
17.5.2 JPEG 解码器 515
17.5.2.1 多个分量 515
17.5.2.2 解析 RST 标记 516
17.5.2.3 可配置量化系数表 517
17.5.2.4 可配置哈夫曼表 517
17.5.2.5 超时检测 518
17.6 中断 519
17.7 编程流程 520
17.7.1 JPEG 编码器 520
17.7.2 JPEG 解码器 523
17.7.3 复位 526
17.8 寄存器列表 527
17.9 寄存器 529

18 像素处理加速器 (PPA) 556
18.1 概述 556
18.2 术语 556
18.3 主要特性 556
18.4 结构概览 557

18.5 功能描述 558
18.5.1 PPA 颜色空间 558
18.5.1.1 SRM 颜色空间 559
18.5.1.2 BLEND 颜色空间 561
18.5.2 PPA 2D-DMA 链表配置 562
18.5.2.1 SRM 特殊配置 563
18.5.3 缩放 - 旋转 - 镜像 (SRM) 563
18.5.3.1 基本功能 563
18.5.3.2 像素块重排 564
18.5.4 图层混合 (BLEND) 565
18.5.4.1 基本功能 565
18.5.4.2 填充图像输出 565
18.6 中断 566
18.7 配置流程 567
18.7.1 PPA 时钟复位配置流程 567
18.7.2 SRM 配置流程 567
18.7.3 BLEND CLUT 配置流程 568
18.7.4 BLEND 配置流程 568
18.7.5 BLEND 图像填充配置流程 569
18.7.6 错误处理流程 569
18.8 寄存器列表 570
18.9 寄存器 572

19 音频采样率转换器 595
19.1 概述 595
19.2 术语 595
19.3 主要特性 595
19.4 结构概览 596
19.5 功能描述 597
19.5.1 时钟 597
19.5.2 复位 597
19.5.3 重采样器 597
19.5.4 分数采样率转换器 597
19.5.5 输入/输出数据长度 598
19.5.5.1 数据无剩余 598
19.5.5.2 数据有剩余 598
19.5.6 DMA 接口 599
19.6 中断 599
19.7 配置流程 599
19.7.1 使用限制与提醒 600
19.8 应用示例 601
19.9 寄存器列表 603
19.10 寄存器 604

20 LCD 与 Camera 控制器 (LCD_CAM) 617
20.1 概述 617

20.2 特性 617
20.3 功能描述 617
20.3.1 功能框图 618
20.3.2 信号描述 618
20.3.3 LCD_CAM 模块时钟 619
20.3.3.1 LCD 时钟 619
20.3.3.2 Camera 时钟 620
20.3.4 LCD_CAM 模块复位 621
20.3.5 LCD_CAM 数据格式控制 622
20.3.5.1 LCD 数据格式控制 622
20.3.5.2 Camera 数据格式控制 624
20.3.6 YUV-RGB 数据格式转换 625
20.3.6.1 YUV 数据流格式 626
20.3.6.2 格式转换模块配置流程 626
20.3.7 LCD_CAM 时序 628
20.3.7.1 LCD 时序（RGB 格式） 628
20.3.7.2 LCD 时序（I8080/MOTO6800 格式） 629
20.4 中断 629
20.5 软件配置流程 630
20.5.1 软件配置 LCD（RGB 格式）发送流程 630
20.5.2 软件配置 LCD（I8080/MOTO6800 格式）发送流程 630
20.5.3 软件配置 Camera 接收流程 632
20.6 寄存器列表 633
20.7 寄存器 634

V 通讯接口 653

21 UART 控制器 (UART) 654
21.1 概述 654
21.2 主要特性 654
21.3 UART 架构 655
21.4 功能描述 656
21.4.1 时钟与复位 656
21.4.2 UART FIFO 656
21.4.3 波特率产生与检测 657
21.4.3.1 波特率产生 657
21.4.3.2 波特率检测 657
21.4.4 UART 数据帧 658
21.4.5 AT_CMD 字符格式 659
21.4.6 RS485 659
21.4.6.1 驱动控制 660
21.4.6.2 转换延时 660
21.4.6.3 总线侦听 660
21.4.7 IrDA 660
21.4.8 唤醒 661
21.4.9 流控 662

21.4.9.1 硬件流控 662
21.4.9.2 软件流控 663
21.4.10 GDMA 模式 664
21.5 中断 665
21.6 编程流程 666
21.6.1 寄存器类型 666
21.6.2 具体步骤 667
21.6.2.1 UART*n* 模块初始化 667
21.6.2.2 UART*n* 通信配置 667
21.6.2.3 启动 UART*n* 668
21.7 寄存器列表 669
21.7.1 UART 寄存器列表 669
21.7.2 LP UART 寄存器列表 670
21.7.3 UHCI 寄存器列表 671
21.8 寄存器 673
21.8.1 UART 寄存器 673
21.8.2 LP UART 寄存器 695
21.8.3 UHCI 寄存器 714

22 USB 串口/JTAG 控制器 733
22.1 概述 733
22.2 主要特性 733
22.3 功能描述 735
22.3.1 CDC-ACM USB 接口描述 735
22.3.2 CDC-ACM 固件接口描述 736
22.3.3 USB-JTAG 接口：JTAG 命令处理器 737
22.3.4 USB-JTAG 接口：CMD_REP 使用示例 738
22.3.5 USB-JTAG 接口：响应捕捉单元 738
22.3.6 USB-JTAG 接口：控制传输请求 739
22.4 中断 739
22.5 配置流程 740
22.6 寄存器列表 742
22.7 寄存器 744

23 LED PWM 控制器 (LEDC) 770
23.1 概述 770
23.2 特性 770
23.3 架构概览 770
23.4 功能描述 771
23.4.1 时钟 771
23.4.2 复位 772
23.4.3 定时器 772
23.4.3.1 时钟分频器配置 772
23.4.3.2 20 位计数器 773
23.4.3.3 定时器配置限制 774
23.4.4 PWM 生成器 774

23.4.4.1 PWM 输出信号微调 775
23.4.4.2 应用新配置 775
23.4.5 占空比渐变 776
23.4.5.1 线性占空比渐变 776
23.4.5.2 伽马曲线渐变 776
23.4.5.3 占空比渐变暂停和恢复 777
23.5 事件任务矩阵功能 778
23.6 中断 779
23.7 配置流程 779
23.8 存储空间 781
23.9 寄存器列表 783
23.10 寄存器 785

24 电机控制脉宽调制器 (MCPWM) 798
24.1 概述 798
24.2 主要特性 798
24.3 模块 801
24.3.1 概述 801
24.3.1.1 定时器模块 801
24.3.1.2 操作器模块 801
24.3.1.3 捕获模块 802
24.3.1.4 ETM 模块 802
24.3.2 PWM 定时器模块 802
24.3.2.1 PWM 定时器模块的配置 802
24.3.2.2 PWM 定时器工作模式和定时事件生成 803
24.3.2.3 PWM 定时器影子寄存器 807
24.3.2.4 PWM 定时器同步和锁相 807
24.3.3 PWM 操作器模块 807
24.3.3.1 PWM 生成器模块 808
24.3.3.2 死区生成器模块 819
24.3.3.3 PWM 载波模块 823
24.3.3.4 故障检测模块 826
24.3.4 捕获模块 827
24.3.4.1 介绍 827
24.3.4.2 捕获定时器 827
24.3.4.3 捕获通道 828
24.3.5 ETM 模块 828
24.3.5.1 介绍 828
24.3.5.2 MCPWM 可产生的 ETM 事件 828
24.3.5.3 MCPWM 可接收的 ETM 任务 829
24.4 中断 829
24.5 寄存器列表 831
24.6 寄存器 834

25 红外遥控 (RMT) 880
25.1 概述 880

25.2 主要特性 880
25.3 功能描述 881
25.3.1 RMT 架构 881
25.3.2 RAM 882
25.3.2.1 RAM 结构 882
25.3.2.2 RAM 使用说明 882
25.3.2.3 RAM 访问方式 883
25.3.3 时钟 884
25.3.4 发射器 884
25.3.4.1 普通发送模式 884
25.3.4.2 乒乓发送模式 884
25.3.4.3 发送加载波 885
25.3.4.4 持续发送模式 885
25.3.4.5 多通道同时发送 885
25.3.5 接收器 885
25.3.5.1 普通接收模式 885
25.3.5.2 乒乓接收模式 886
25.3.5.3 接收滤波 886
25.3.5.4 接收去载波 886
25.3.6 配置参数更新 886
25.4 中断 887
25.5 寄存器列表 889
25.6 寄存器 891

26 并行 IO 控制器 (PARLIO) 905
26.1 概况 905
26.2 术语 905
26.3 特性 905
26.4 系统架构 906
26.5 功能描述 907
26.5.1 时钟生成器 907
26.5.2 时钟复位使用限制 908
26.5.3 主从机模式 909
26.5.4 RX 模块接收模式 909
26.5.4.1 电平使能模式 910
26.5.4.2 脉冲使能模式 910
26.5.4.3 软件使能模式 911
26.5.5 RX 模块 GDMA SUC EOF 信号生成 911
26.5.6 TX 模块 EOF 信号生成 912
26.5.7 RX 模块超时 912
26.5.8 TX 模块无限制长度数据传输 912
26.5.9 TX 片选功能 912
26.5.10 TX 模块输出时钟门控 912
26.5.11 TX 模块总线空闲值 913
26.5.12 单帧数据传输 913
26.5.13 字节范围内数据顺序翻转 913

26.6 中断 913
26.7 配置流程 914
26.7.1 数据接收操作流程 914
26.7.2 数据发送操作流程 914
26.8 应用示例 915
26.8.1 与 SPI 进行数据传输 915
26.8.2 与 I2S 进行数据传输 917
26.8.3 与 LCD 进行数据传输 917
26.9 寄存器列表 919
26.10 寄存器 920

VI 模拟和混合信号处理 931

27 触摸传感器 (TOUCH) 932
27.1 术语 932
27.2 特性 932
27.3 系统架构 933
27.3.1 触摸板 933
27.3.2 电容触摸管脚 933
27.3.3 触摸传感器 934
27.4 功能描述 935
27.4.1 Touch FSM 935
27.4.2 采样信号预处理 936
27.4.2.1 测量过程 937
27.4.2.2 测量操作的触发源 938
27.4.2.3 扫描模式 939
27.4.2.4 跳频模式 940
27.4.3 触摸检测 941
27.4.3.1 采样值 941
27.4.3.2 硬件触摸检测 942
27.4.4 接近模式 942
27.4.5 休眠模式 943
27.4.6 防潮功能 943
27.4.7 遇水保护功能 944
27.5 中断 944
27.6 寄存器列表 945
27.6.1 中断与状态寄存器列表 945
27.6.2 配置寄存器列表 946
27.7 寄存器 949
27.7.1 中断与状态寄存器 949
27.7.2 配置寄存器 955

28 SAR ADC 数字控制器 (ADC) 974
28.1 概述 974
28.2 特性 974
28.2.1 ADC 通道 975

28.2.2 ADC 时钟 975
28.3 触发控制 976
28.3.1 特性 976
28.3.2 功能描述 976
28.3.2.1 触发模式选择 976
28.3.2.2 定时器触发 976
28.3.2.3 软件触发与停止 976
28.4 模式表 976
28.4.1 特性 977
28.4.2 功能描述 977
28.4.2.1 模式表结构 977
28.4.2.2 扫描长度与指针 978
28.4.2.3 更新时机 978
28.4.2.4 通道与模式选择 978
28.5 SAR 控制 FSM 979
28.5.1 特性 979
28.5.2 功能描述 979
28.5.2.1 状态机流程 979
28.5.2.2 连续采样模式 979
28.5.2.3 SAR 电源控制 981
28.5.2.4 状态查询 982
28.6 数据处理 982
28.6.1 特性 982
28.6.2 功能描述 982
28.6.2.1 数据仲裁 982
28.6.2.2 IIR 滤波器 982
28.6.2.3 采样数据读取 983
28.7 阈值监控 983
28.7.1 特性 983
28.7.2 功能描述 983
28.7.2.1 阈值配置 983
28.7.2.2 比较逻辑 983
28.7.2.3 全通道监控模式 983
28.8 DMA 控制 984
28.8.1 特性 984
28.8.2 功能描述 984
28.8.2.1 DMA 使能 984
28.8.2.2 EOF 帧管理 984
28.8.2.3 DMA 数据格式 984
28.8.2.4 FIFO 溢出 984
28.9 ETM 事件 984
28.9.1 特性 984
28.9.2 功能描述 985
28.9.2.1 ETM 触发输入 985
28.9.2.2 ETM 事件输出 985
28.9.2.3 ETM 时钟 985

28.10 中断 985
28.11 寄存器列表 986
28.12 寄存器 987

29 模拟电压比较器 1002
29.1 概述 1002
29.2 主要特性 1002
29.3 系统架构 1002
29.4 功能描述 1003
29.4.1 参考电压配置 1003
29.4.2 控制器 1004
29.4.3 时钟 1006
29.4.4 复位 1007
29.5 事件任务矩阵 1007
29.6 中断 1008
29.7 配置流程 1008
29.8 寄存器列表 1009
29.9 寄存器 1010

VII 附录 1022

相关文档和资源 1023

词汇列表 1024
外设相关词汇 1024
寄存器相关缩写 1025
寄存器的访问类型 1026

如何配置寄存器的保留域 1028
概述 1028
如何配置保留域 1028

中断配置寄存器 1029

修订历史 1030

表格

1.3-1 追踪编码器参数 33
1.5-1 可选边带信号 35
1.6-1 头部格式 36
1.6-2 索引格式 37
1.6-3 格式 3 子格式 0 37
1.6-4 格式 3 子格式 1 38
1.6-5 格式 3 子格式 3 38
1.6-6 格式 2 39
1.6-7 格式 1，有地址 40
1.6-8 格式 1，无地址 41
1.7-1 CORE*n*_TRACE 的内部中断源 41
1.8-1 调试模块的触发支持（mcontrol 寄存器的 action 字段） 42
1.8-2 追踪状态 44

2.4-1 LP CPU 中断映射 70
2.4-2 LP CPU 异常原因 71
2.7-1 性能计数器 78
2.10-1 唤醒源 81

3.4-1 GDMA-AHB-HP 配置寄存器与外设选择关系表 88
3.4-2 GDMA-AHB-LP 配置寄存器与外设选择关系表 89
3.4-3 GDMA-AXI 配置寄存器与外设选择关系表 89

4.4-1 描述符字段说明 223
4.4-2 2D-DMA TX 方向配置寄存器与外设选择关系表 227
4.4-3 2D-DMA RX 方向配置寄存器与外设选择关系表 227
4.4-4 2D-DMA 发送方向宏块重排推荐配置表 228
4.4-5 2D-DMA 接收方向宏块重排推荐配置表（不做颜色格式转换） 228
4.4-6 TX 方向颜色格式转换参数配置 230
4.4-7 RX 方向颜色格式转换参数配置 230
4.4-8 RGBB888 的存储格式 231
4.4-9 YUV444 的存储格式 231
4.4-10 YUV422 的存储格式 231

5.2-1 超时动作 293

6.3-1 配置 RTC Timer 记录特定事件的发生时刻 306
6.3-2 RTC Timer 目标时刻配置 306
6.5-1 RTC Timer 的内部中断源 307

7.5-1 HP CPU*n* 包格式 322
7.5-2 DMA 包格式 322
7.5-3 LOST 包格式 322
7.5-4 DMA_0 包格式 325
7.5-5 DMA LOST 包格式 326

8.3-1 LP 信箱控制器的内部中断源 360

9.3-1 电源管脚检测的配置 378
9.4-1 电源检测器的内部中断源 378

10.4-1 工作模式 390
10.4-2 密钥长度和加解密方向 390
10.5-1 状态返回值 391
10.5-2 Typical AES 文本字节序 391
10.5-3 AES-128 密钥字节序 392
10.5-4 AES-256 密钥字节序 392
10.6-1 块模式选择 393
10.6-2 状态返回值 394
10.6-3 TEXT-PADDING 394
10.6-4 DMA-AES 存储字节序 395
10.9-1 AES 的内部中断源 399

11.3-1 ECC 硬件加速器内存块 411
11.4-1 ECC 加速器曲线模式控制 413
11.4-2 ECC 硬件加速器工作模式控制 413

12.4-1 HMAC 功能及配置数值 427

13.3-1 加速效果 444
13.4-1 RSA 加速器的内部中断源 445

14.3-1 SHA 加速器工作模式 453
14.3-2 SHA 哈希算法选择 453
14.4-4 不同运算标准信息摘要的寄存器占用情况 459
14.5-1 SHA 加速器的内部中断源 459

16.4-1 ECDSA_DS 工作模式 482
16.4-2 ECDSA_DS 椭圆曲线选择 483
16.4-3 ECDSA_DS SHA 算法 483
16.4-4 ECDSA_DS 工作状态 483
16.7-1 ECDSA_DS 存储器块 493

17.1-2 标记码描述 509
17.2-1 不同图像格式 MCU 包含的数据单元 509
17.5-1 颜色空间转换支持的图像格式 513
17.5-2 量化系数表中系数的位置编号（示例） 514
17.5-3 推荐用于亮度分量的 8 位精度量化系数表 514
17.5-4 推荐用于色度分量的 8 位精度量化系数表 514
17.5-5 采样因子和图像格式 516
17.7-1 JPEG 编码器 hb 和 vb 配置 521
17.7-2 DMA2D_OUT_MACRO_BLOCK_SIZE_CH*n* 配置 522
17.7-3 JPEG 解码器 JPEG_HA、JPEG_VA 的配置 523
17.7-4 JPEG 解码器 hb 和 vb 配置 524
17.7-5 DMA2D_IN_MACRO_BLOCK_SIZE_CH*n* 配置 524

17.7-6 2D DMA 颜色空间转换配置 525

18.5-1 像素存储顺序 558
18.5-2 PPA SRM Alpha 配置效果 560
18.5-3 BLEND CLUT 地址 561
18.5-4 PPA 链表配置 562
18.5-5 自动补全块大小配置 563
18.5-6 SRM 输出像素块重排数量 564
18.6-1 SRM 参数错误类型 566
18.6-2 BLEND 参数错误类型 566

19.6-1 ASRC 的内部中断源 599

20.3-1 信号描述 618
20.3-2 LCD 数据格式控制 - 阶段一 623
20.3-3 LCD 输出数据重排列功能模式 624
20.3-4 Camera 数据格式控制 625
20.3-5 24 位输出时的转换模式配置 627
20.3-6 非 24 位输出时的转换模式配置 627

21.2-1 UART 和 LP UART 特性区分 654
21.4-1 UART_CHAR_WAKEUP 模式配置 662

22.3-1 标准 CDC-ACM 控制请求 735
22.3-2 CDC-ACM 中 RTS 和 DTR 的设置 736
22.3-3 半字节中的命令 737
22.3-4 USB-JTAG 控制请求 739
22.3-5 JTAG 功能描述符 739
22.5-1 复位芯片进入下载模式 741
22.5-2 复位 SoC 从 flash 启动 741

23.4-1 常用配置频率及精度 774
23.8-1 LEDC 存储空间 781

24.3-1 操作器模块包含的各个子模块功能 802
24.3-2 PWM 生成器中的所有定时事件 809
24.3-3 PWM 定时器递增计数时，定时事件的优先级 810
24.3-4 PWM 定时器递减计数时，定时事件的优先级 810
24.3-5 控制死区时间生成器开关的字段 820
24.3-6 死区生成器的典型操作模式 821
24.3-7 MCPWM 可产生的 ETM 事件 828
24.3-8 MCPWM 可接收的 ETM 任务 829

25.3-1 更新配置参数 886

26.5-1 时钟限制下复位 AXI 时钟域操作步骤 908
26.5-2 时钟限制下 TX 模块从机模式操作要求 909
26.5-3 时钟限制下 RX 模块从机模式操作要求 909
26.6-1 PARLIO 的内部中断源 913

27.3-1 ESP32-S31 电容式触摸传感器的管脚 933
27.3-2 电容式触摸传感器输出信号 935
27.4-1 平滑触摸数据计算 941
27.4-2 基准数据算法 941
27.4-2 基准数据算法 942
27.5-1 触摸传感器的内部中断源 944

28.2-1 SAR ADC 通道 975
28.10-1 SAR ADC 中断源 985

29.3-1 模拟电压比较器 PAD 和 GPIO 之间的映射关系 1003
29.4-1 V_HYS 取值 1004
29.6-1 模拟电压比较器的内部中断源 1008

29.9-4 ENA/RAW/ST 寄存器的配置 1029

插图

1.0-1 追踪编码器概述图 31
1.2-1 指令追踪流程概览 32

2.1-1 LP CPU 概览 58
2.10-1 LP CPU 唤醒和睡眠流程 80

3.3-1 GDMA 控制器架构 86
3.4-1 链表结构图 87
3.4-2 链表关系图 90

4.3-1 2D-DMA 基本架构图 222
4.4-1 链表结构图 223
4.4-2 补齐示意图（整体） 226
4.4-3 补齐示意图（细节） 226
4.4-4 JPEG 解码输出数据宏块排布 228
4.4-5 颜色空间转换示意图 229

5.1-1 看门狗定时器概览 290
5.2-1 ESP32-S31 的数字看门狗定时器 292
5.3-1 SWD 控制器结构 295

8.3-1 LP 信箱控制器系统框图 359

9.3-1 电源检测器架构图 376
9.3-2 欠压检测器处理方式 377
9.3-3 电压毛刺检测器 378

10.7-1 GCM 加密过程 397

12.6-1 HMAC 附加填充位示意图 430
12.6-2 HMAC 结构示意图 430

15.3-1 软件准备工作与硬件工作流程 470

16.5-1 ECDSA_DS 流程 487
16.5-2 clk_ecdsa 时钟图 490
16.5-3 clk_apb_ecdsa 时钟图 490

17.1-1 JPEG 码流格式 509
17.2-1 JPEG 图像编解码器的系统连接框图 510
17.4-1 JPEG 编码器架构 511
17.4-2 JPEG 解码器架构 512

18.1-1 PPA 2D-DMA 连接关系 556
18.4-1 PPA 系统架构 557
18.5-1 PPA SRM 输入像素处理 559
18.5-2 PPA SRM Alpha 配置逻辑 559

18.5-3 2D-DMA 链表 562
18.5-4 SRM 步骤示例 564

19.4-1 ASRC 模块结构 596
19.8-1 采样率转换关系 601

20.3-1 LCD_CAM 功能框图 618
20.3-2 LCD 时钟 620
20.3-3 Camera 时钟 621
20.3-4 LCD 视频帧结构 628
20.3-5 LCD 时序（RGB 格式） 629
20.3-6 LCD 时序（I8080/MOTO6800 格式） 629

21.3-1 UART 基本架构图 655
21.4-1 UART 控制器分频 657
21.4-2 UART 信号下降沿较差时序图 658
21.4-3 UART 数据帧结构 658
21.4-4 AT_CMD 字符格式 659
21.4-5 RS485 模式驱动控制结构图 660
21.4-6 SIR 模式编解码时序图 661
21.4-7 IrDA 编解码结构图 661
21.4-8 硬件流控图 662
21.4-9 硬件流控信号连接图 663
21.4-10 GDMA 模式数据传输 664
21.6-1 UART 编程流程 667

22.2-1 USB 串口/JTAG 控制器高层框图 734
22.2-2 USB 串口/JTAG 控制器框图 735

23.3-1 LED PWM 控制器架构 771
23.3-2 定时器和 PWM 生成器功能框图 771
23.4-1 LEDC_CLK 时钟图 772
23.4-2 LEDC_CLK_DIV 为非整数时的分频 773
23.4-3 计数器和 PWM 信号精度关系 773
23.4-4 LED PWM 输出信号图 775
23.4-5 输出信号占空比线性渐变图 776
23.4-6 输出信号占空比伽马曲线渐变图 777

24.2-1 MCPWM 外设概览 799
24.3-1 定时器模块 801
24.3-2 操作器模块 801
24.3-3 捕获模块 802
24.3-4 ETM 模块 802
24.3-5 递增计数模式波形 803
24.3-6 递减计数模式波形 804
24.3-7 递增递减循环模式波形，同步事件后递减 804
24.3-8 递增递减循环模式波形，同步事件后递增 804
24.3-9 递增模式中生成的 UTEP 和 UTEZ 805

24.3-10 递减模式中生成的 DTEP 和 DTEZ 806
24.3-11 递增递减模式中生成的 DTEP 和 UTEZ 806
24.3-12 PWM 操作器的子模块 808
24.3-13 递增递减模式下的对称波形 811
24.3-14 递增计数模式，单边不对称波形，PWM*x*A 和 PWM*x*B 独立调制–高电平 812
24.3-15 递增计数模式，脉冲位置不对称波形，PWM*x*A 独立调制 813
24.3-16 递增递减循环计数模式，双沿对称波形，在 PWM*x*A 和 PWM*x*B 上独立调制–高电平有效 814
24.3-17 递增递减循环计数模式，双沿对称波形，在 PWM*x*A 和 PWM*x*B 上独立调制–互补 815
24.3-18 递增递减循环计数模式，错误或同步事件，在 PWM*x*A 和 PWM*x*B 上相同调制 816
24.3-19 NCI 在 PWM*x*A 输出上软件强制事件示例 817
24.3-20 CNTU 在 PWM*x*B 输出上软件强制事件示例 818
24.3-21 死区模块的开关拓扑 820
24.3-22 高电平有效互补 (AHC) 死区波形 821
24.3-23 低电平有效互补 (ALC) 死区波形 822
24.3-24 高电平有效 (AH) 死区波形 822
24.3-25 低电平有效 (AL) 死区波形 823
24.3-26 PWM 载波操作的波形示例 824
24.3-27 载波模块的第一个脉冲和之后持续的脉冲示例 825
24.3-28 PWM 载波模块中持续脉冲的 7 种占空比设置 826

25.3-1 RMT 结构框图 881
25.3-2 RAM 中脉冲编码结构 882

26.4-1 PARLIO 系统框图 906
26.5-1 PARLIO 时钟生成 907
26.5-2 主机时钟正向波形 909
26.5-3 主机时钟负向波形 909
26.5-4 RX 模块电平使能模式 910
26.5-5 RX 模块脉冲使能子模式 911
26.5-6 RX 模块软件使能模式 911

27.3-1 触摸传感器系统 933
27.3-2 触摸传感器内部结构 934
27.4-1 Touch FSM 内部结构和工作原理 936
27.4-2 TOUCH_OUT 信号预处理 937
27.4-3 扫描模式时序图 939
27.4-4 触摸传感器跳频时序图 940
27.4-5 感测面积示意图 942

28.1-1 SAR ADC 数字控制器系统框图 974
28.4-1 ADC_SAR1_PATT_TAB1_REG 包含样式 0 ~ 3 977
28.4-2 ADC_SAR1_PATT_TAB2_REG 包含样式 4 ~ 7 977
28.4-3 ADC_SAR1_PATT_TAB3_REG 包含样式 8 ~ 11 977
28.4-4 ADC_SAR1_PATT_TAB4_REG 包含样式 12 ~ 15 977
28.4-5 样式结构（6-bit） 978
28.5-1 连续采样与通道切换时序图 980
28.5-2 连续采样时序图（PATT_TYPE = 1，通道不切换） 980

28.5-3 非连续模式软件触发采样时序图 981

29.3-1 模拟电压比较器系统框图 1003
29.4-1 模拟电压比较器全扫描模式时序图 1006
29.4-2 模拟电压比较器时钟图 1007

第 I 卷

微处理器和主机

该部分介绍构成芯片系统的基本要素，包括高性能 CPU 微处理器架构、用于调试的跟踪编码、直接内存访问 (DMA) 控制器等。

第 1 章

RISC-V 追踪编码器 (TRACE)

ESP32-S31 的高性能 CPU (HP CPU) 通过追踪编码器 (Trace Encoder) 实现了对指令追踪接口的支持。追踪编码器连接 CPU 的指令追踪接口，将信息压缩成更小的数据包，存入系统存储器（详见章节 3 *系统和存储器 [to be added later]*）中。

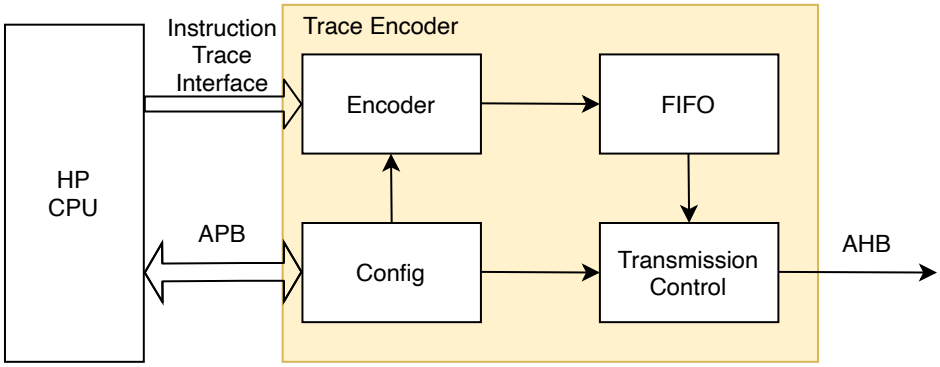


图 1.0-1. 追踪编码器概述图

1.1 术语

为了更好地说明 RISC-V 追踪编码器的功能，本章使用了以下术语。

hart	RISC-V 硬件线程
分支 (branch)	在一定条件下改变执行流程的指令
无法推测的不连续地址 (uninferable discontinuity, or updiscon)	无法仅通过程序二进制文件推测出的程序计数器地址偏移
增量 (delta)	指令在存储器中的存储地址不连续时，程序计数器的地址偏移
陷阱 (trap)	异常或中断，会触发陷阱处理程序
筛选 (qualification)	指令符合筛选条件，即称为通过筛选，才会被追踪
te_inst	编码器发送的数据包的名称
退役 (retire)	机器状态更新时，执行指令的最后一个阶段
EPC	异常程序计数器

1.2 介绍

对于复杂的系统来说，了解程序执行流程并非易事。造成软件不按预期工作的可能有很多种因素，比如与其他核、外设、实时事件的交互，执行不畅，或以上所有因素的结合。

调试器很难实时监测系统运行中的程序执行流程，因为调试器是侵入性的，会破坏系统的运行状态。但是，监测程序执行又很重要。

此时，可使用指令追踪来追踪程序执行。

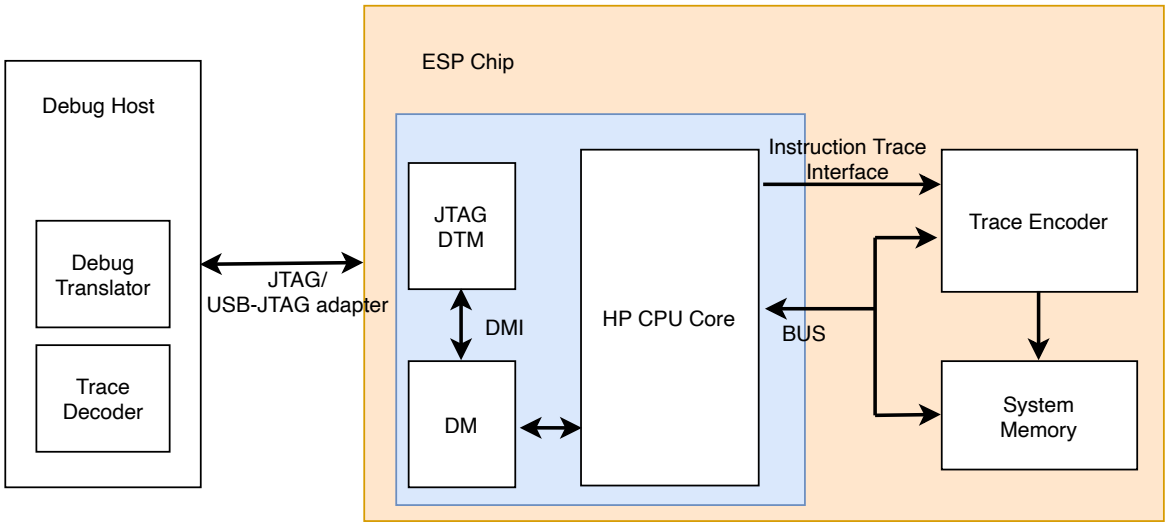


图 1.2-1. 指令追踪流程概览

图 1.2-1 为指令增量追踪的流程。每个 HP 核都有一个单独的编码器，可以独立工作，追踪流程相同：

- HP CPU 核有一个指令追踪接口，可以输出 HP CPU 执行指令的信息，包括指令地址、指令类型等。更多关于 ESP32-S31 HP CPU 指令追踪接口的信息，可参考章节 2 高性能处理器 *[to be added later]*。
- 追踪编码器通过 HP CPU 的指令追踪接口获取相关指令追踪信息，将信息压缩成带宽更小的数据包，存入系统存储器中。
- 调试器（即图中的 Debug Host）可以将系统存储器中的追踪数据包通过 JTAG 或 USB Serial/JTAG 导出，然后用解码器解压缩、重建程序的执行流。追踪解码器通常是外部 PC 上的软件，在获取追踪数据包后，根据在 hart 上运行的程序二进制数据，重建程序的指令流。解码可以离线进行，也可以在 hart 执行时实时进行。

1.3 特性

- 兼容 Efficient Trace for RISC-V Version 2.0（RISC-V 高效追踪规范 v2.0），实现的参数详见表 1.3-1
- 支持增量地址模式和完整地址模式
- 支持过滤器
- 支持通过调试触发器或过滤器报告指令地址
- 支持下列边带信号 (sideband signals) 控制追踪数据流
  - 调试触发器启动或关闭编码器
  - hart 暂停时，编码器在报告最后一个数据包后停止工作
  - hart 复位时，编码器在报告最后一个数据包后停止工作

  - FIFO 即将变满时暂停 hart
- 支持任意有效地址范围存储追踪数据包
- 可配置的同步模式：
  - 同步计数器按包计数
  - 同步计数器按周期计数
  - 关闭同步计数器
- 支持丢包状态标识
- 支持丢包后自动重启
- 写系统存储器时支持循环和非循环模式
- 具有两个中断：
  - 包的大小超过配置的存储器空间时触发中断
  - 丢包时触发中断
- 具有 128 × 8 位 FIFO，用于缓存数据包
- 支持 AHB 突发传输，突发长度可配置

表 1.3-1. 追踪编码器参数

参数名称	值	描述
arch_p	0	追踪器符合 RISC-V 追踪规范初始版本
bpred_size_p	0	不支持分支预测模式
cache_size_p	0	不支持跳转目标缓存模式
call_counter_size_p	0	不支持隐式返回模式
ctype_width_p	0	ctype 总线宽度
context_width_p	0	context 总线宽度
ecause_width_p	6	异常原因宽度
ecause_choice_p	0	不支持多选
f0s_width_p	0	不支持格式 0 数据包
filter_context_p	0	不支持过滤上下文信息
filter_excint_p	1	支持过滤异常原因或中断
filter_privilege_p	1	支持过滤特权
filter_tval_p	1	支持过滤陷阱值
iaddress_lsb_p	1	支持压缩指令
iaddress_width_p	32	指令总线为 32 位
iretire_width_p	1	iretire 总线宽度
ilastsize_width_p	0	ilastsize 总线宽度
itype_width_p	3	itype 总线宽度
nocontext_p	1	在 te_inst 数据包中去掉上下文信息
notime_p	1	在 te_inst 数据包中去掉时间信息
privilege_width_p	2	支持机器和用户、超级用户模式
retires_p	1	每个块退役指令的最大数量
return_stack_size_p	0	不支持隐式返回模式

参数名称	值	描述
sijump_p	0	不支持根据顺序推测跳转
taken_branches_p	1	每个时钟周期仅一条指令退役
impdef_width_p	0	未实现

上述参数的具体描述，详见 Efficient Trace for RISC-V Version 2.0 > 章节 Parameters and Discovery。

1.4 架构概览

本章节主要介绍 ESP32-S31 追踪编码器的具体实现。

如图 1.0-1 所示，追踪编码器包含一个编码器、一个 FIFO、一个寄存器配置模块和一个发送控制模块。

编码器通过指令追踪接口接收 HP CPU 的指令信息，将指令信息压缩成不同的数据包，写入内部 FIFO。

发送控制模块通过 AHB 总线将 FIFO 中的数据写入内部 SRAM。

FIFO 的深度为 128，宽度为 8 位。存储器写入带宽不足时，FIFO 可能会溢出，发生丢包。如果发生丢包，编码器会发送一个包通知丢包情况，在 FIFO 变空前一直停止工作，如果中断已使能还会触发中断。

1.5 功能描述

1.5.1 同步

为保障追踪的有效性，追踪过程中必须定期同步。同步可以通过发送完整的指令地址实现。同步计数器的值达到 TRACE_RESYNC_PROLONGED_REG 寄存器的 TRACE_RESYNC_PROLONGED 字段的值时，编码器会发送一个同步数据包（格式 3 子格式 0，见章节 1.6.3.1）。

同步计数器可通过 TRACE_RESYNC_MODE 配置：

- 0：关闭同步计数器
- 1：无效值，没有作用
- 2：同步计数器按数据包计数
- 3：同步计数器按周期计数

可通过增加 TRACE_RESYNC_PROLONGED 的值，降低发送同步数据包的频率，从而降低同步数据包对带宽的占用。

1.5.2 地址模式

ESP32-S31 支持两种地址模式：增量地址模式和完整地址模式。

- 增量地址模式（默认）：在增量地址模式下，地址会编码为当前指令的实际地址与上一个包含地址的数据包报告的指令实际地址的差值。这种差值编码所需的位比完整地址少，可以令追踪压缩更高效。
- 完整地址模式：在完整地址模式下，追踪的所有地址会编码为绝对地址，而不是差值。这种编码方式相对低效，但是可以辅助软件解码器开发者进行调试。该模式可以通过置位 TRACE_FULL_ADDRESS 使能。

1.5.3 可选边带信号

Efficient Trace for RISC-V Version 2.0 > 章节 Optional Sideband Signals 介绍了用于控制编码器的可选边带信号。ESP32-S31 实现了下列信号：

表 1.5-1. 可选边带信号

信号	功能
trigger[2:0]	• 位 0 的脉冲可以令编码器开始追踪，并持续追踪直到另有通知。追踪时还需要满足其他过滤条件。 • 位 1 的脉冲可以令编码器停止追踪，直到另有通知。 • 位 2 的脉冲可以令编码器报告特定地址。 该功能可通过置位 TRACE_DM_TRIGGER_ENA 使能。触发器地址匹配时，建立 trigger 信号。例如，若 mcontrol 寄存器的 action 字段为 2，则触发器地址匹配时 trigger 信号的位 0 变为高电平；若 action 字段为 3，则触发器地址匹配时 trigger 信号的位 1 变为高电平。更多信息，详见表 1.8-1。
halted	暂停 hart。halted 信号建立时，编码器先输出一个数据包报告 hart 暂停前退役的最后一条指令的地址，然后再输出一个支持数据包指示追踪已暂停。halted 信号失效时，编码器重新开始追踪，从同步数据包开始。该功能可通过置位 TRACE_HALT_ENA 使能。
reset	复位 hart。具体行为同上文 halted 的描述。
stall	请求 hart 暂停。如果某些应用需要无损追踪，便可以在追踪编码器不能输出追踪数据包（内部 FIFO 将满）时使用该信号暂停 hart。该功能通过置位 TRACE_STALL_ENA 使能。

1.5.4 过滤

过滤机制控制编码器追踪的标准。比如，过滤机制可以让编码器在出现下列情况时追踪：

- 指令地址在特定范围内时追踪
- 从一个指令地址开始追踪，到出现第二个指令地址时结束追踪
- 针对一个或多个特定特权等级追踪
- 出现异常及/或特定异常原因触发中断处理程序时追踪

每个过滤器单元包含一个过滤器和一个比较器单元。

每个比较器单元实际包含一对比较器（主比较器和次比较器，简称 P、S），可根据要求设置与比较器单元匹配的范围，并支持：

- 从 iaddr 和 tval 中选择输入
- 每个比较器可以独立选择运算方式（<、>、=、!= 等）
- 次匹配值可以是主比较器的屏蔽值
- 两个比较器有多种一起使用的方式：P、$P\&\&S$、$!(P\&\&S)$、锁存（P 匹配时置 1，S 匹配时清零）
- 每个比较器可以报告特定指令地址

每个过滤器可以指定要过滤的 hart 指令，并具有以下功能：

- 选择比较器单元进行匹配
- 可以选择多个特权等级和异常原因作为输入进行匹配
- 选择匹配中断

1.5.5 锚定

由于数据包的长度不定，为在写入存储器时分清数据包之间的边界，ESP32-S31 编码器在数据包之间插入 0：

- 数据包的长度最大为 13 字节，因此正常数据包中不能出现连续 14 个值为 0 的字节。连续 14 个及以上值为 0 的字节后，第一个不为 0 的字节肯定是数据包的第一个字节。
- 每发送 128 个数据包，编码器会向存储器写入 14 个值为 0 的字节作为锚定符。

1.5.6 写存储器模式

写系统存储器时，追踪数据包的大小可能会超过系统存储器配置的容量。此时可配置写存储器的模式，选择是否循环写入数据包：

- 循环模式：追踪数据包的大小超过系统存储器的容量（即 TRACE_MEM_CURRENT_ADDR_REG 的值达到 TRACE_MEM_END_ADDR_REG 的值）时，存储器循环写入数据，回到存储器的起始地址 TRACE_MEM_START_ADDR_REG，此前写入的数据会被新数据覆盖。
- 非循环模式：追踪数据包的大小超过系统存储器的容量时，存储器不循环，停在 TRACE_MEM_END_ADDR_REG，此前写入的数据保留。

1.5.7 自动重启

因 FIFO 溢出而丢包时，编码器将停止工作，需要软件手动恢复或硬件自动重启。如果 TRACE_TRIGGER_REG 寄存器的 TRACE_RESTART_ENA 位为 1，FIFO 清空后编码器会自动重启。

如果使能自动重启功能，编码器无论出现什么情况都会自动重启。因此，要关闭编码器，必须先清除 TRACE_TRIGGER_REG 寄存器的 TRACE_RESTART_ENA 位从而关闭自动重启。

1.6 编码器输出数据包

本节主要介绍 ESP32-S31 追踪编码器输出的数据包格式。

追踪数据包包括头部、索引和有效载荷，依次按照头部、索引和有效载荷的顺序以比特流形式传输。头部、索引和有效载荷中的字段按照下文表格的罗列顺序依次传输，多位字段先传输最低有效位。

1.6.1 头部

头部为 1 个字节，格式如表 1.6-1 所示。

表 1.6-1. 头部格式

字段	位	描述	值
length	5	数据包的总长度	4~13

字段	位	描述	值
flow	2	保留	0
timestamp	1	保留	0

1.6.2 索引

索引为 2 个字节，格式如表 1.6-2 所示。

表 1.6-2. 索引格式

字段	位	描述	值
index	16	每个数据包的索引	0~65535

1.6.3 有效载荷

有效载荷的宽度为 1 至 10 字节。

1.6.3.1 格式 3

格式 3 的数据包用于同步和发送支持信息。规范中定义了四种子格式，ESP32-S31 仅支持 3 种。

格式 3 子格式 0 - 同步

该格式的数据包包含解码器完全识别指令所需的全部信息。开始追踪第一条指令时（除非指令恰好是异常处理程序中的第一条指令），或同步定时器超时触发同步时，编码器会发送该格式的数据包。该格式数据包有效载荷的长度为 5 个字节。

表 1.6-3. 格式 3 子格式 0

字段名	位	描述
format	2	11 (sync)：同步
subformat	2	00 (start)：开始追踪，或开始同步
branch	1	0：地址指向分支指令，且分支跳转 1：指令非分支指令，或分支没有跳转
privilege	2	所追踪指令的特权等级
address	31	指令的完整地址。字段值需左移 1 位，以复原字节地址
sign_extend	2	保留

格式 3 子格式 1 - 异常

该格式的数据包同样包含解码器完全识别指令所需的全部信息。发生异常或触发中断时，编码器会发送该格式的数据包。该格式的数据包中包含了异常原因、异常指令的“陷阱值”，陷阱处理程序的地址和异常指令的地址。该格式数据包有效载荷的长度不定：如果 interrupt 字段为 1，则 tvalepc 字段省略，长度为 6 个字节；否则长度为 10 个字节。

表 1.6-4. 格式 3 子格式 1

字段名	位	描述
format	2	11 (sync)：同步
subformat	2	01 (exception)：异常原因和陷阱处理程序的地址
branch	1	0：地址指向分支指令，且分支跳转 1：指令非分支指令，或分支没有跳转
privilege	2	所追踪指令的特权等级
ecause	6	异常原因
interrupt	1	中断
theaddr	1	该字段为 1 时，address 字段指向陷阱处理程序的地址。该字段为 0 时，如果异常情况是无法推测出程序计数器的目标地址偏移量，则 address 字段指向异常程序计数器 (EPC) 的值，如果是其他异常中断，则 address 字段不定义。
address	31	指令的完整地址。地址必须向左偏移 1 位，以复原字节地址
tvalepc	32	utval、stval 或 mtval 控制/状态寄存器的值。如果 interrupt 字段为 1，该字段省略。
sign_extend	2	保留

格式 3 子格式 3 - 支持

该格式的数据包包含解码器需要的辅助信息，在追踪结束时、过滤器匹配失败或丢包时发出。该格式数据包有效载荷的长度为 2 个字节。

表 1.6-5. 格式 3 子格式 3

字段名	位	描述
format	2	11 (sync)：同步
subformat	2	11 (support)：解码器需要的辅助信息
enable	1	表示编码器已开启
encoder_mode	1	永远为 0。仅分支追踪支持
qual_status	2	表示指令的筛选状态： • 00 (no_change)：指令筛选状态没有变化 • 01 (ended_rep)：指令筛选已结束，上一个指令为最后一个通过筛选的指令 • 10 (trace lost)：丢失了一个或多个包 • 11 (ended_upd)：指令筛选已结束，即便上一个 te_inst 不是最后一个通过筛选的指令，也会因指令地址不连续、无法推测而发送

字段名	位	描述
ioptions	6	表示以下可选模式： • 根据顺序推测跳转（位 5）：该位为 1 时，不会报告根据顺序推测出的跳转目标地址。该功能未实现，因此此位始终为 0 • 分支预测（位 4）：该位为 1 时，使能分支预测。该功能未实现，因此此位始终为 0 • 缓存跳转目标（位 3）：该位为 1 时，使能跳转目标缓存。该功能未实现，因此此位始终为 0 • 完整地址（位 2）： – 0：增量地址模式 – 1：完整地址模式 • 隐式异常（位 1）：如果可以通过 ecause 字段（异常原因）确定陷阱向量，则格式 3 子格式 1 数据包不包含地址。该功能未实现，因此此位始终为 0 • 隐式返回（位 0）：该位为 1 时，不报告函数的返回地址。该功能未实现，因此此位始终为 0
sign_extend	2	保留

1.6.3.2 格式 2

该格式的数据包仅包含指令地址，在必须报告指令地址时使用（比如指令目标地址为无法推测的不连续地址，或指令为发生异常之前的最后一个指令），包中没有分支信息。

增量地址模式使能时，该格式数据包有效载荷的长度不定，在 5 到 8 个字节之间。地址字段宽度可为 8、16、24 或 32 位，取决于数据包长度。例如，如果数据包头部为 n 字节，则地址字段宽度为 $n-4$ 字节。地址字段需要通过有符号扩展到 32 位。

表 1.6-6. 格式 2

字段名	位	描述
format	2	10 (addr-only)：无分支信息
notify	1	若该位的值与地址字段的最高有效位不同，则表示 trigger[2] 或过滤器匹配触发了通知请求，数据包报告指令的目标地址不是无法推测的不连续地址
updiscon	1	若该位的值与 notify 的值不同，则表示指令的目标地址为无法推测的不连续地址，且该地址为异常、特权变更或同步之前的最后一条指令
zero_extend	4	保留
address	不定	指令的增量地址，位宽为 8、16、24 或 32 位

1.6.3.3 格式 1

该格式的数据包包含分支信息，用于以下任一场景：

- 需要报告分支信息（比如分支映射已满时）
- 上一个数据包后已出现至少一个分支，且需要报告指令地址

增量地址模式使能时，该格式数据包有效载荷的长度不定，在 5 到 12 个字节之间。地址字段宽度可为 8、16、24 或 32 位，取决于数据包长度。例如：

- 数据包头部为 n 字节
- format 字段的值为 1
- branches 字段的值为：
  - 0：数据包为不包含地址的格式 1 数据包
  - 1：branch_map 宽度为 1 位，zero_ext 宽度为 6 位，address 宽度为 $n-5$ 位
  - 2 ~ 3：branch_map 宽度为 3 位，zero_ext 宽度为 4 位，address 宽度为 $n-5$ 位
  - 4 ~ 7：branch_map 宽度为 7 位，zero_ext 宽度为 0 位，address 宽度为 $n-5$ 位
  - 8 ~ 15：branch_map 宽度为 15 位，zero_ext 宽度为 0 位，address 宽度为 $n-6$ 位
  - 16 ~ 31：branch_map 宽度为 31 位，zero_ext 宽度为 0 位，address 宽度为 $n-8$ 位

格式 1，有地址时 branch_map 字段

有效载荷长度不定。

表 1.6-7. 格式 1，有地址

字段名	位	描述
format	2	01：包含分支信息
branches	5	该字段的值决定了 branch_map 的位宽（branch_map 的有效位数量），具体如下： • 0：（该格式中此值无效） • 1：1 位 • 2 ~ 3：3 位 • 4 ~ 7：7 位 • 8 ~ 15：15 位 • 16 ~ 31：31 位 比如，若 branches = 12，则 branch_map 为 15 位，低 12 位有效。
branch_map	不定	表示分支是否跳转的位数组。位 0 对应最早执行的分支指令。每个位的含义如下： • 0：分支已跳转 • 1：分支未跳转 该字段的位宽不定，取决于 branches 字段的值
notify	1	若该位的值与地址字段的最高有效位不同，则表示 trigger[2] 或过滤器匹配触发了通知请求，数据包报告指令的目标地址不是无法推测的不连续地址
updiscon	1	若该位的值与 notify 的值不同，则表示指令的目标地址为无法推测的不连续地址，且该地址为异常、特权变更或同步之前的最后一条指令

字段名	位	描述
zero_ext	不定	该字段的宽度取决于 branches 字段，具体如下： • 1：6 位 • 2 ~ 3：4 位 • 4 ~ 31：0 位
address	不定	指令的增量地址，位宽为 8、16、24 或 32 位

格式 1，无地址时 branch_map 字段

该格式数据包有效载荷的长度为 5 个字节。

表 1.6-8. 格式 1，无地址

字段名	位	描述
format	2	01：包含分支信息
branches	5	该字段的值决定了 branch_map 的位宽，仅 0 有效，对应 branch_map 的位宽为 31 位
branch_map	31	表示分支是否跳转的位数组。位 0 对应最早执行的分支指令。每个位的含义如下： • 0：分支已跳转 • 1：分支未跳转
sign_extend	2	保留

1.7 中断

ESP32-S31 的 CORE*n*_TRACE（即 HP CPU 核 0 或 HP CPU 核 1 的追踪编码器）可以生成 CORE*n*_TRACE_INTR 中断信号并发送给中断矩阵。

CORE*n*_TRACE_INTR 中断信号由 CORE*n*_TRACE 的内部中断源生成。表 1.7-1 列出了 CORE*n*_TRACE 的内部中断源、中断触发条件、以及生成的中断信号。

表 1.7-1. CORE*n*_TRACE 的内部中断源

内部中断源	触发条件	中断信号
TRACE_MEM_FULL_INTR	数据包大小超过系统存储器容量（即 TRACE_MEM_CURRENT_ADDR_REG 的值达到 TRACE_MEM_END_ADDR_REG 的值）时触发。如有必要，可使能中断通知 HP CPU 处理，比如可再申请新的存储器空间	CORE*n*_TRACE_INTR
TRACE_FIFO_OVERFLOW_INTR	内部 FIFO 溢出时触发，表示一个或多个数据包丢失	CORE*n*_TRACE_INTR

说明：

中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 1.9 寄存器列表。

1.8 编程流程

1.8.1 配置编码器选项

编码器具有一些可选的配置选项。

- 完整地址模式
  - 编码器默认使用增量地址模式。可以置位 TRACE_FULL_ADDRESS 以启用完整地址模式用来调试。
- 丢包后重启
  - 因 FIFO 溢出而丢失追踪数据包时，编码器将自动停止。如果 TRACE_RESTART_ENA 为 1，编码器会在 FIFO 为空时重启。
- FIFO 将满时暂停 hart
  - 如果 TRACE_STALL_ENA 为 1，FIFO 将满时会向 hart 发出暂停请求，hart 会一直暂停，直到请求失效。
- hart 暂停时停止追踪

  如果 TRACE_HALT_ENA 为 1，编码器会在 hart 暂停时先输出一个数据包，报告暂停前最后一条指令的地址，然后再输出一个支持数据包指示追踪已停止。halted 信号失效时，编码器重新开始追踪，从同步数据包开始。
- hart 复位时停止追踪

  如果 TRACE_RESET_ENA 为 1，编码器会单独复位。编码器会在 hart 复位时先输出一个数据包，报告复位前最后一条指令的地址，然后再输出一个支持数据包指示追踪已停止。hart 退出复位状态时，编码器重新开始追踪，从同步数据包开始。
- 使用调试模块的触发输出

  RISC-V hart 的调试模块有一个触发单元，其匹配控制寄存器（mcontrol）有一个 4 位的 action 字段，该字段值为 2 ~ 4 时用于追踪。ESP32-S31 已实现此功能。action 字段的值详见表 1.8-1。

表 1.8-1. 调试模块的触发支持（mcontrol 寄存器的 action 字段）

值	描述
2	开始追踪
3	结束追踪
4	通知编码器报告地址

有关调试模块触发单元的详细信息，请参考章节 2 高性能处理器 *[to be added later]*。

1.8.2 配置过滤器

章节 1.5.4 中描述的过滤单元可以按以下方式配置：

1. 选择以下任一匹配模式：

- 置位 TRACE_MATCH_PRIVILEGE 启用特权匹配模式：
  - 通过 TRACE_MATCH_CHOICE_PRIVILEGE 配置特权级别
- 置位 TRACE_MATCH_ECAUSE 启用 ecause 匹配模式：
  - 通过 TRACE_MATCH_CHOICE_ECAUSE 配置 ecause 值
- 置位 TRACE_MATCH_INTERRUPT 启用中断匹配模式：
  - 通过 TRACE_MATCH_VALUE_INTERRUPT 配置中断级别
- 置位 TRACE_MATCH_COMP 启用比较器匹配模式：
  - 配置主比较器：
    * 通过 TRACE_P_INPUT 选择输入
    * 通过 TRACE_P_FUNCTION 选择比较器运算符
    * 通过 TRACE_FILTER_P_COMPARATOR_MATCH_REG 配置匹配值
  - 必要时配置次比较器：
    * 通过 TRACE_S_INPUT 选择输入
    * 通过 TRACE_S_FUNCTION 选择比较器运算符
    * 通过 TRACE_FILTER_S_COMPARATOR_MATCH_REG 配置匹配值
  - 通过 TRACE_MATCH_MODE 选择用于匹配的比较器：
    * 0：仅主比较器用于匹配，次比较器无效
    * 1：主比较器和次比较器都用于匹配 ($P\&\&S$)
    * 2：主比较器和次比较器不同时匹配 $!(P\&\&S)$
    * 3：主比较器匹配时开始过滤，次比较器匹配时结束过滤

  如果置位 TRACE_P_NOTIFY 或 TRACE_S_NOTIFY，比较器会以通知的形式报告匹配地址，指令不会过滤。例如：
  - 如果 TRACE_MATCH_MODE 为 0，并且 TRACE_P_NOTIFY 为 1，则所有指令都会追踪，指令与主比较器匹配时会发出通知。
  - 如果 TRACE_MATCH_MODE 为 0，并且 TRACE_S_NOTIFY 为 1，则指令将通过主比较器过滤，指令与次比较器匹配时会发出通知。

2. 置位 TRACE_FILTER_EN 以启用过滤单元。

1.8.3 使能编码器

- 通过 TRACE_MEM_START_ADDR_REG 和 TRACE_MEM_END_ADDR_REG 配置追踪数据包的存储空间。编码器可以访问系统存储器 TCM 和 PSRAM：
  - TCM: 0x2F000000 ~ 0x2F07FFFF
  - PSRAM: 0x50000000 ~ 0x53FFFFFF
- 置位 TRACE_MEM_CURRENT_ADDR_UPDATE，将 TRACE_MEM_CURRENT_ADDR_REG 的值更新为 TRACE_MEM_START_ADDR_REG 的值

- （可选）通过 TRACE_TRIGGER_REG 寄存器的 TRACE_MEM_LOOP 位配置写存储器的模式
  - 0：非循环模式
  - 1：循环模式（默认）
- 通过 TRACE_RESYNC_PROLONGED_REG 寄存器的 TRACE_RESYNC_MODE 位配置同步模式
  - 0：关闭同步计数器（默认）
  - 1：无效值，没有作用
  - 2：同步计数器按数据包计数
  - 3：同步计数器按周期计数
- （可选）通过 TRACE_RESYNC_PROLONGED_REG 配置同步计数器的阈值，默认为 128
- （可选）使能中断
  - 向 TRACE_INTR_ENA_REG 寄存器的相应位写 1 开启对应中断
  - 向TRACE_INTR_CLR_REG 寄存器的相应位写 1 清除对应中断
  - 读 TRACE_INTR_RAW_REG 寄存器的值，获取中断状态
- （可选）向 TRACE_TRIGGER_REG 寄存器的 TRACE_RESTART_ENA 位写 1，使能自动重启。该功能默认开启
- 使能追踪编码器，有以下两种方式：
  - 向 TRACE_TRIGGER_REG 寄存器的 TRACE_TRIGGER_ON 位写 1
  - 通过调试模块的 action 字段使能编码器。触发器地址匹配且 action 字段为 2 时，编码器使能。触发器的配置，请参考 RISC-V Debug Specification。

编码器使能后，会一直追踪 HP CPU 的指令接口，向追踪存储器写入数据包。

1.8.4 关闭编码器

停止输出追踪数据包，有以下两种方式：

1. 向 TRACE_TRIGGER_REG 寄存器的 TRACE_TRIGGER_OFF 位写 1 关闭编码器
2. 通过调试模块的 action 字段关闭编码器。触发器地址匹配且 action 字段为 3 时，编码器关闭。触发器的配置，请参考 RISC-V Debug Specification。

由于内部 FIFO 的存在，数据包在追踪结束后不会立即写入存储器。建议查询TRACE_FIFO_STATUS_REG 确认数据是否全部写入存储器。

表 1.8-2. 追踪状态

字段	描述
TRACE_FIFO_EMPTY	该字段如果为 1，则表示 FIFO 为空，所有数据已写入内存

字段	描述
TRACE_WORK_STATUS	编码器的工作状态： • 0：空闲状态，编码器未启动 • 1：编码器正常工作，输出数据包 • 2：编码器未输出数据包，hart 已暂停或复位 • 3：编码器未输出数据包，发生数据包丢失，等待 FIFO 为空时重新启动

1.8.5 通知

如果一个目标地址很特殊，即使不是无法推测的不连续地址，用户也可能想要报告。

通知和报告特殊地址有以下两种方式：

1. 调试模块的触发器地址匹配，并且 action 为 4
2. 过滤器比较器匹配

收到通知请求时，编码器会发出格式 2 或格式 1 的数据包报告地址，具体取决于 branch_map 字段。如果 branch_map 为 0，则发送格式 2 数据包，否则发送格式 1 数据包。notify 字段表示数据包是否报告的是通知地址。如果地址是无法推测的不连续地址，即使是通知用的数据包，notify 字段也不会变为高电平，否则解码器无法正确重建指令流。

1.8.6 解码数据包

- 找到解码的第一个地址
  - 读 TRACE_INTR_RAW_REG 寄存器的 TRACE_MEM_FULL_INTR_RAW 位，查看系统存储器是否已满
    * 如果值为 0，从 TRACE_MEM_START_ADDR_REG 寄存器读取追踪数据包
    * 如果值为 1，且开启了存储器循环模式，则之前的数据包已被覆盖。此时，读取 TRACE_MEM_CURRENT_ADDR_REG 的值，获取最后一个写入的地址，该地址即为解码器处理的第一个地址
- 用解码器解码数据包
  - 解码器从第一个地址开始读取所有数据包，结合二进制文件重建数据流
  - 如章节 1.6 所述，编码器向存储器分区边界写 14 个值为 0 的字节。因此出现 14 个值为 0 的字节时，则表示下一个非 0 字节为新数据包的头部

1.8.7 AHB 配置

ESP32-S31 编码器支持 AHB 总线，可选配置项可以控制编码器的传输带宽。

如果有多个 DMA 主机，可以通过 TRACE_AHB_CONFIG_REG 配置 AHB 突发传输，增加编码器的带宽。

- TRACE_HBURST 支持以下配置：
  - 0x0：SINGLE
  - 0x1：INCR（长度未定义）

  - 0x2：INCR4
  - 0x4：INCR8
  - 其他值：保留

  突发传输意味着仲裁成功后，可以连续传输多个字。SINGLE 传输为 1 拍，INCR 长度未定义，INCR4 为 4 拍，INCR8 为 8 拍。突发长度越长，带宽越大。

- 传输配置为 INCR 时，由于传输长度不确定，为了避免编码器长时间占用总线，传输长度达到 TRACE_MAX_INCR 后 INCR 传输结束，重新开始总线仲裁。

该配置用于调整带宽。带宽足够时，不需要配置突发传输，可使用默认配置。

1.8.8 软件保留

TRACE_WORK_STATUS 表示编码器的状态。编码器不处于空闲状态时，必须备份配置，并在芯片进入睡眠状态时还原配置。

1.9 寄存器列表

本小节的所有地址均为相对于 RISC-V 追踪编码器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

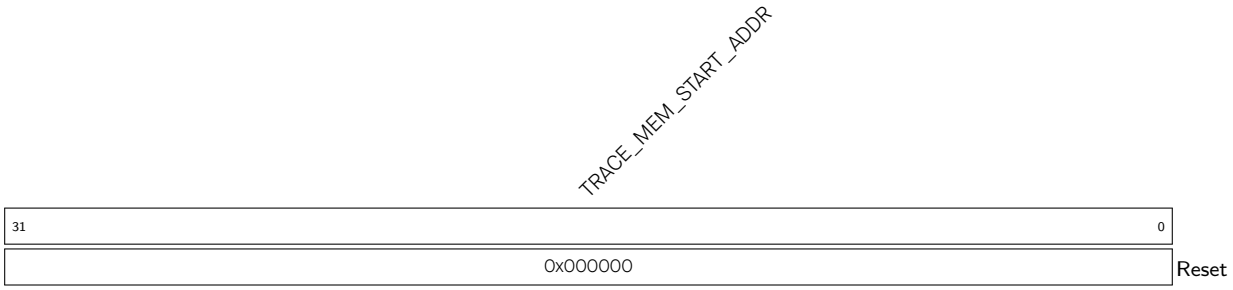
名称	描述	地址	访问
追踪存储器配置寄存器			
TRACE_MEM_START_ADDR_REG	存储器起始地址寄存器	0x0000	R/W
TRACE_MEM_END_ADDR_REG	存储器结束地址寄存器	0x0004	R/W
TRACE_MEM_CURRENT_ADDR_REG	存储器当前地址寄存器	0x0008	RO
TRACE_MEM_ADDR_UPDATE_REG	存储器地址更新寄存器	0x000C	WT
追踪 FIFO 状态寄存器			
TRACE_FIFO_STATUS_REG	FIFO 状态寄存器	0x0010	RO
追踪中断配置寄存器			
TRACE_INTR_ENA_REG	中断使能寄存器	0x0014	R/W
TRACE_INTR_RAW_REG	中断原始状态寄存器	0x0018	RO
TRACE_INTR_CLR_REG	中断清除寄存器	0x001C	WT
追踪配置寄存器			
TRACE_TRIGGER_REG	追踪触发寄存器	0x0020	varies
TRACE_CONFIG_REG	追踪配置寄存器	0x0024	R/W
TRACE_FILTER_CONTROL_REG	过滤器控制寄存器	0x0028	R/W
TRACE_FILTER_MATCH_CONTROL_REG	过滤器匹配控制寄存器	0x002C	R/W
TRACE_FILTER_COMPARATOR_CONTROL_REG	过滤器比较器控制寄存器	0x0030	R/W
TRACE_FILTER_P_COMPARATOR_MATCH_REG	主比较器匹配值寄存器	0x0034	R/W
TRACE_FILTER_S_COMPARATOR_MATCH_REG	次比较器匹配值寄存器	0x0038	R/W
TRACE_RESYNC_PROLONGED_REG	再同步配置寄存器	0x003C	R/W
TRACE_AHB_CONFIG_REG	AHB 配置寄存器	0x0040	R/W
时钟门控配置和控制寄存器			
TRACE_CLOCK_GATE_REG	时钟门控控制寄存器	0x0044	R/W
版本寄存器			
TRACE_DATE_REG	版本控制寄存器	0x03FC	R/W

1.10 寄存器

本小节的所有地址均为相对于 RISC-V 追踪编码器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

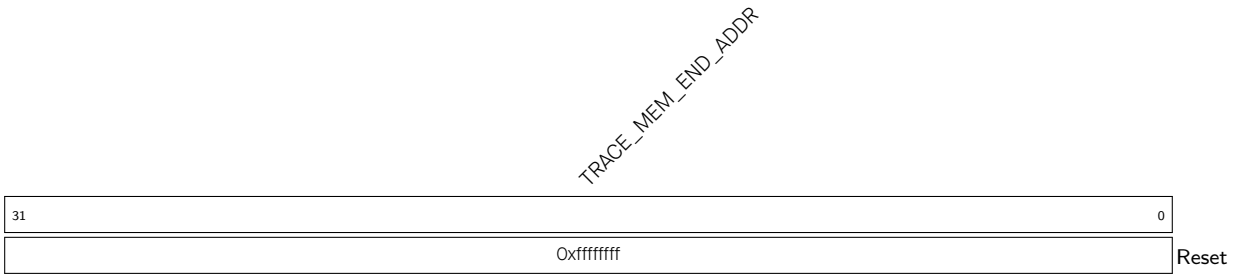
关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 1.1. TRACE_MEM_START_ADDR_REG (0x0000)



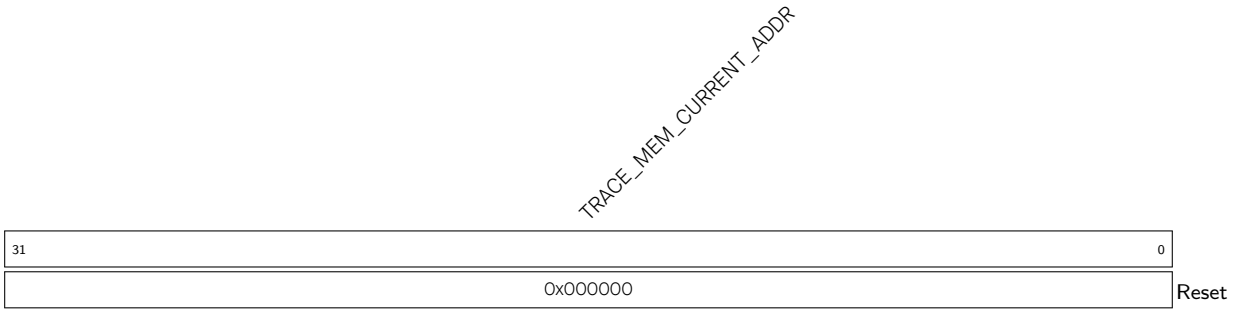
TRACE_MEM_START_ADDR 配置追踪存储器的起始地址。(R/W)

Register 1.2. TRACE_MEM_END_ADDR_REG (0x0004)



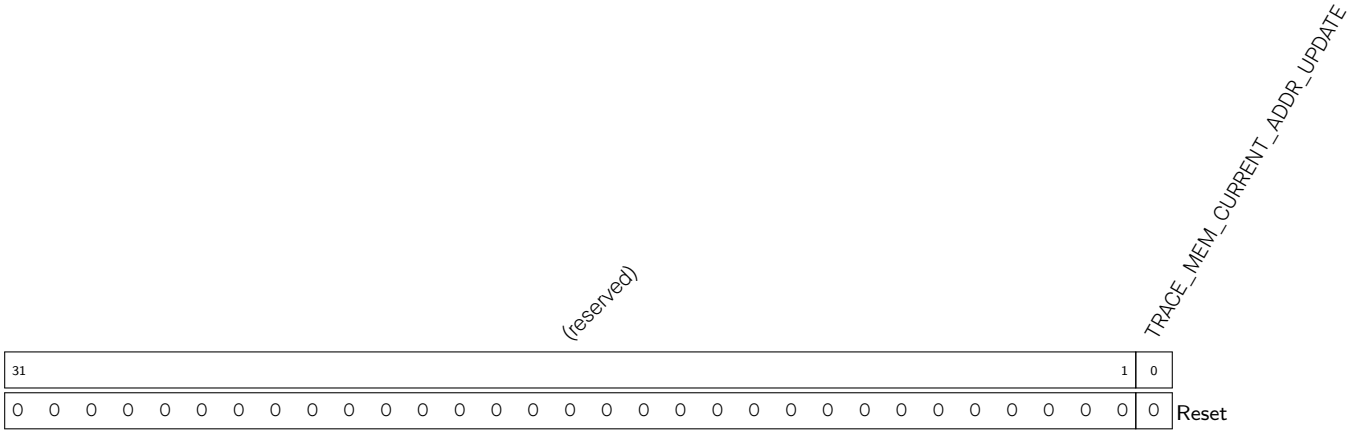
TRACE_MEM_END_ADDR 配置追踪存储器的结束地址。(R/W)

Register 1.3. TRACE_MEM_CURRENT_ADDR_REG (0x0008)



TRACE_MEM_CURRENT_ADDR 表示当前要写入的存储器地址。(RO)

Register 1.4. TRACE_MEM_ADDR_UPDATE_REG (0x000C)

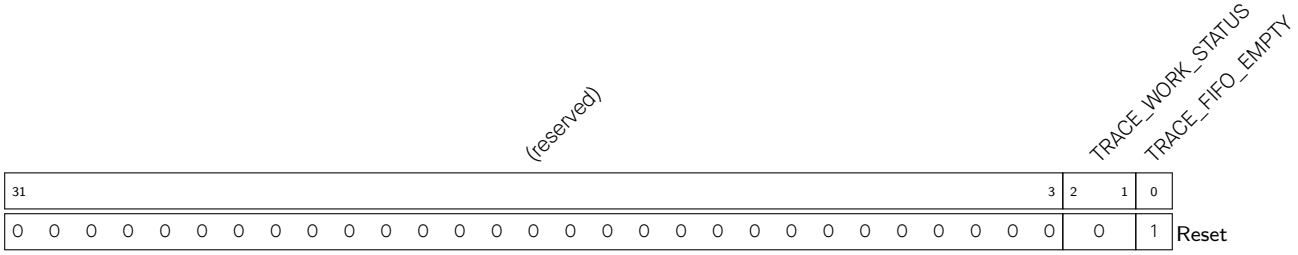


TRACE_MEM_CURRENT_ADDR_UPDATE 配置是否将 TRACE_MEM_CURRENT_ADDR 的值更新为 TRACE_MEM_START_ADDR 的值。

0：不更新

1：更新 (WT)

Register 1.5. TRACE_FIFO_STATUS_REG (0x0010)



TRACE_FIFO_EMPTY 表示 FIFO 的状态。

0：不为空

1：为空 (RO)

TRACE_WORK_STATUS 表示编码器的状态。

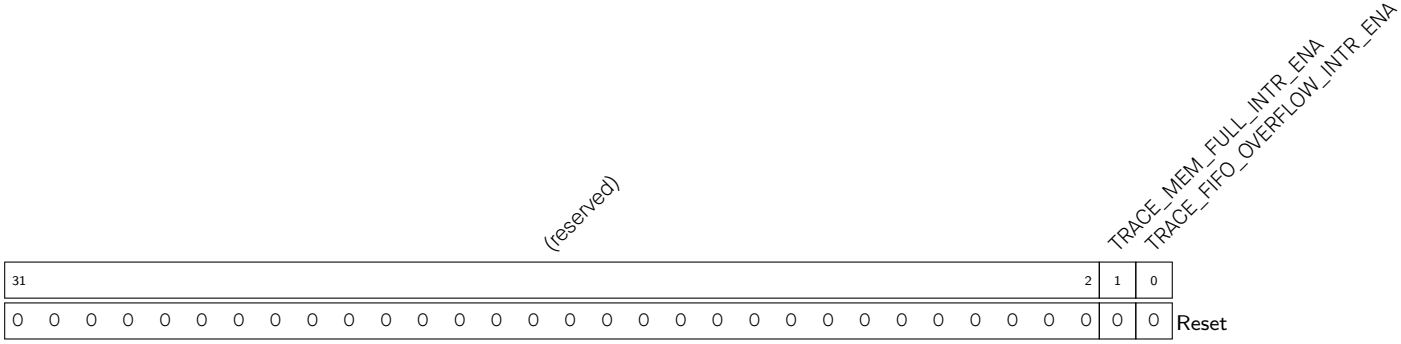
0：空闲状态

1：工作状态

2：因 hart 暂停或复位处于待机状态

3：丢失状态 (RO)

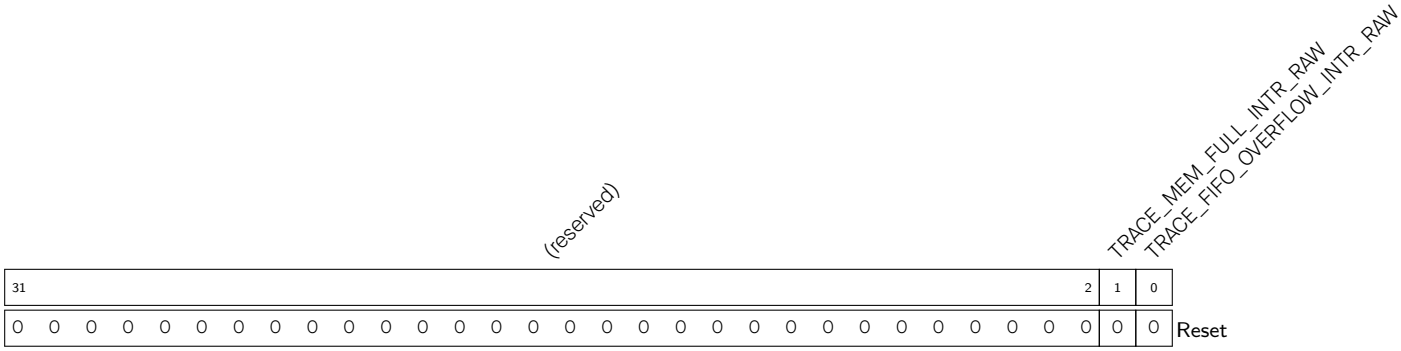
Register 1.6. TRACE_INTR_ENA_REG (0x0014)



TRACE_FIFO_OVERFLOW_INTR_ENA 写 1 使能 TRACE_FIFO_OVERFLOW_INTR。(R/W)

TRACE_MEM_FULL_INTR_ENA 写 1 使能 TRACE_MEM_FULL_INTR。(R/W)

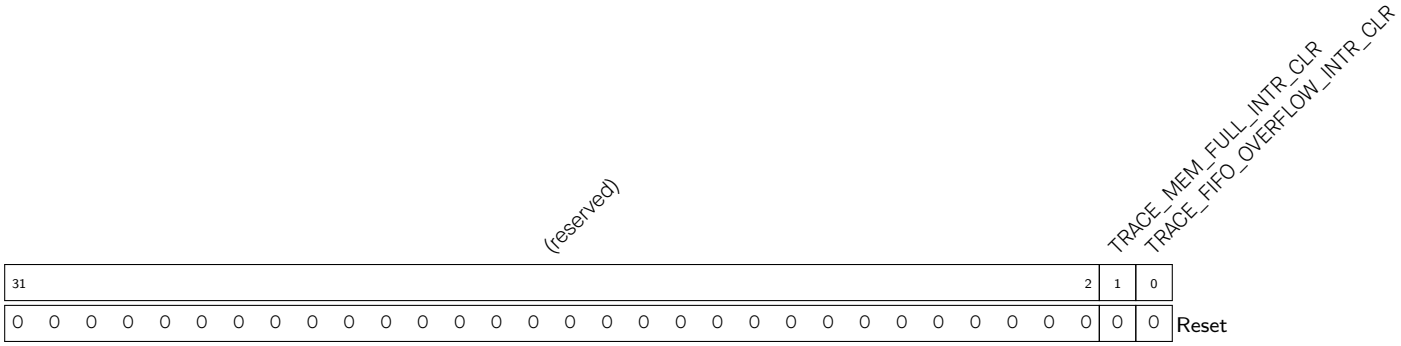
Register 1.7. TRACE_INTR_RAW_REG (0x0018)



TRACE_FIFO_OVERFLOW_INTR_RAW TRACE_FIFO_OVERFLOW_INTR 的原始中断状态。(RO)

TRACE_MEM_FULL_INTR_RAW TRACE_MEM_FULL_INTR 的原始中断状态。(RO)

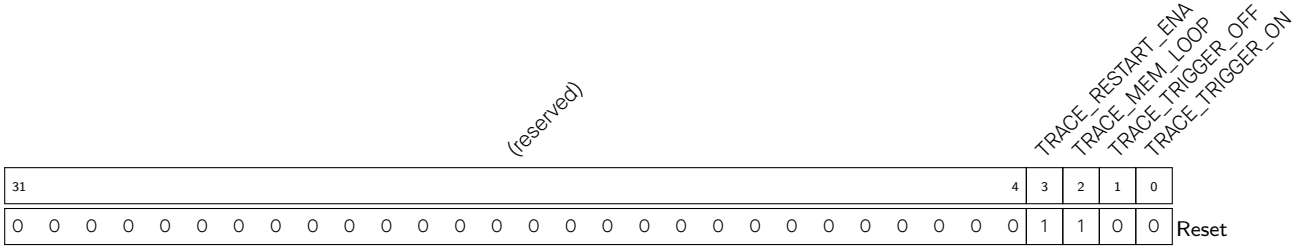
Register 1.8. TRACE_INTR_CLR_REG (0x001C)



TRACE_FIFO_OVERFLOW_INTR_CLR 写 1 清除 TRACE_FIFO_OVERFLOW_INTR。(WT)

TRACE_MEM_FULL_INTR_CLR 写 1 清除 TRACE_MEM_FULL_INTR。(WT)

Register 1.9. TRACE_TRIGGER_REG (0x0020)



TRACE_TRIGGER_ON 配置开启追踪编码器。

0：无效值，没有作用

1：开启 (WT)

TRACE_TRIGGER_OFF 配置关闭追踪编码器。

0：无效值，没有作用

1：关闭 (WT)

TRACE_MEM_LOOP 配置存储器写入模式。

0：非循环模式

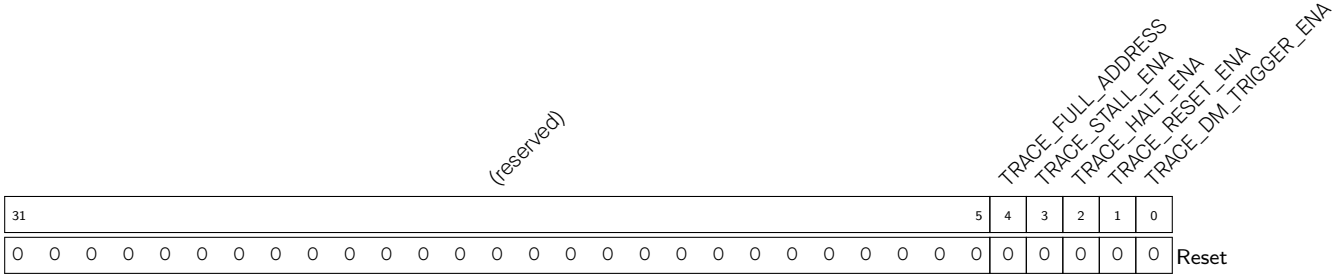
1：循环模式 (R/W)

TRACE_RESTART_ENA 配置是否开启编码器自动重启功能。

0：关闭

1：开启 (R/W)

Register 1.10. TRACE_CONFIG_REG (0x0024)



TRACE_DM_TRIGGER_ENA 配置是否使能 trigger 信号。

0：禁用

1：使能 (R/W)

TRACE_RESET_ENA 配置是否使能 reset 信号。

0：禁用

1：使能 (R/W)

TRACE_HALT_ENA 配置是否使能 halted 信号。

0：禁用

1：使能 (R/W)

TRACE_STALL_ENA 配置是否使能 stall 信号。

0：禁用

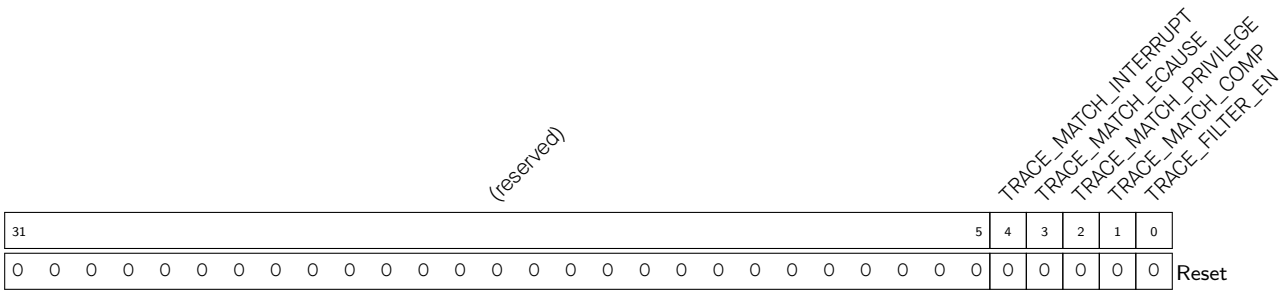
1：使能 (R/W)

TRACE_FULL_ADDRESS 配置地址模式。

0：增量地址模式

1：完整地址模式 (R/W)

Register 1.11. TRACE_FILTER_CONTROL_REG (0x0028)



TRACE_FILTER_EN 配置是否使能过滤功能。

0：禁用

1：使能 (R/W)

TRACE_MATCH_COMP 配置是否使能比较器匹配模式。

0：禁用

1：使能 (R/W)

TRACE_MATCH_PRIVILEGE 配置是否使能特权匹配模式。

0：禁用

1：使能 (R/W)

TRACE_MATCH_ECAUSE 配置是否使能 ecause 匹配模式。

0：禁用

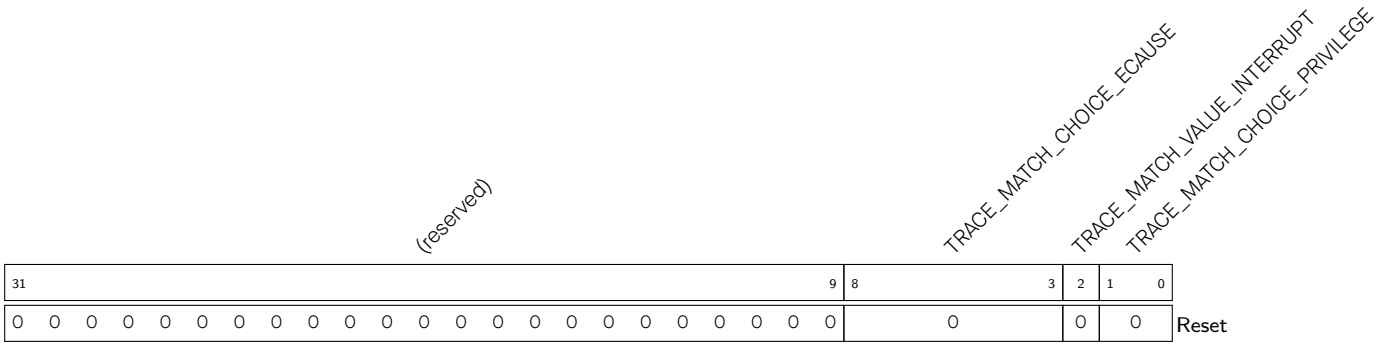
1：使能 (R/W)

TRACE_MATCH_INTERRUPT 配置是否使能中断匹配模式。

0：禁用

1：使能 (R/W)

Register 1.12. TRACE_FILTER_MATCH_CONTROL_REG (0x002C)



TRACE_MATCH_CHOICE_PRIVILEGE 配置匹配的特权等级。

0：用户模式

1：超级用户模式

2: 保留

3: 机器模式 (R/W)

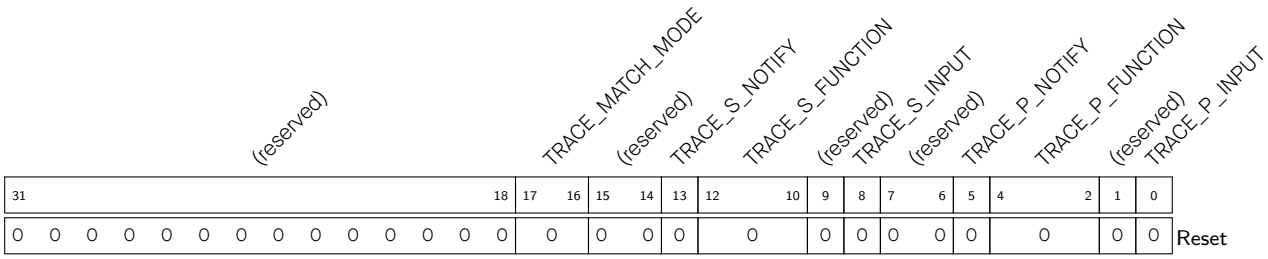
TRACE_MATCH_VALUE_INTERRUPT 配置匹配的中断。仅 TRACE_MATCH_INTERRUPT 为 1 时有效。

0：itype=1

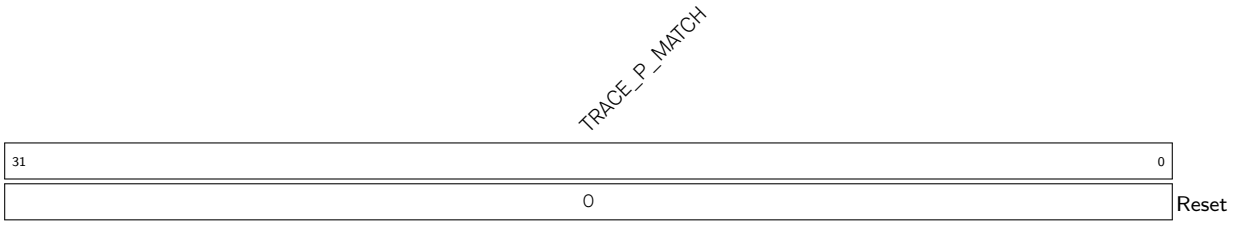
1：itype=2 (R/W)

TRACE_MATCH_CHOICE_ECAUSE 配置匹配的 ecause 代码。(R/W)

Register 1.13. TRACE_FILTER_COMPARATOR_CONTROL_REG (0x0030)

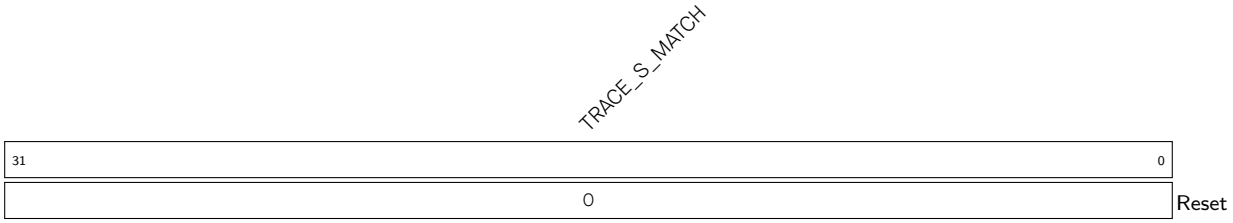

(reserved)
TRACE_MATCH_MODE
(reserved)
TRACE_S_NOTIFY
TRACE_S_FUNCTION
(reserved)
TRACE_S_INPUT
(reserved)
TRACE_P_NOTIFY
TRACE_P_FUNCTION
(reserved)
TRACE_P_INPUT
31 18 17 16 15 14 13 12 10 9 8 7 6 5 4 2 1 0
0 0 0

Register 1.14. TRACE_FILTER_P_COMPARATOR_MATCH_REG (0x0034)



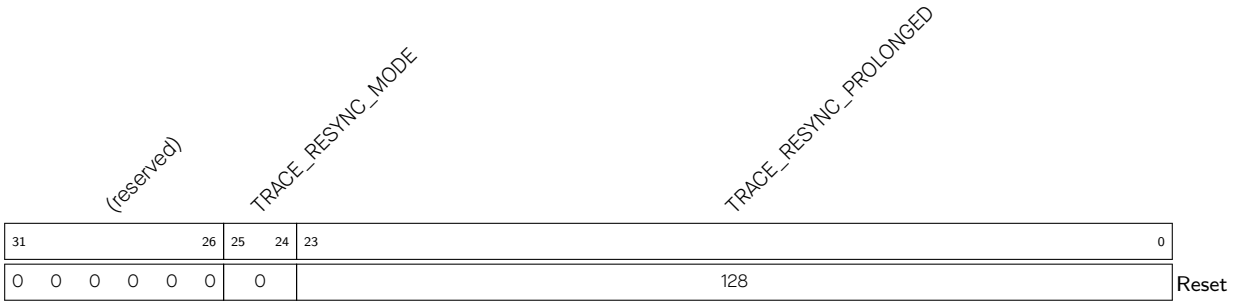
TRACE_P_MATCH 配置主比较器的匹配值。(R/W)

Register 1.15. TRACE_FILTER_S_COMPARATOR_MATCH_REG (0x0038)



TRACE_S_MATCH 配置次比较器的匹配值。(R/W)

Register 1.16. TRACE_RESYNC_PROLONGED_REG (0x003C)

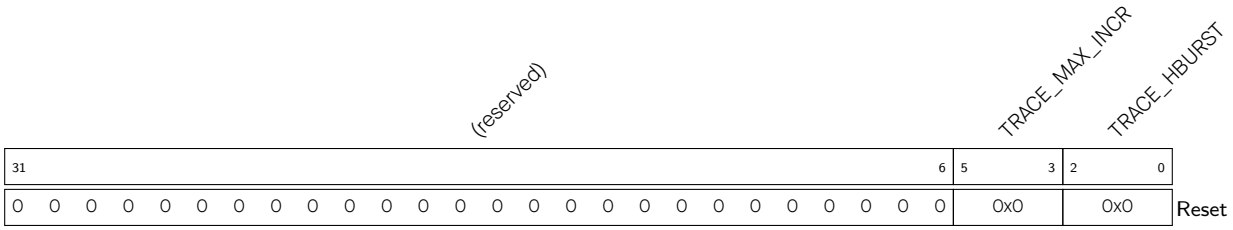


TRACE_RESYNC_PROLONGED 配置同步计数器的阈值。(R/W)

TRACE_RESYNC_MODE 配置同步模式。

0：关闭计数器
1：无效值，没有作用
2：按数据包计数同步
3：按周期计数同步 (R/W)

Register 1.17. TRACE_AHB_CONFIG_REG (0x0040)



TRACE_HBURST 配置 AHB 突发传输模式。

0：SINGLE

1：INCR（不定义长度）

2：INCR4

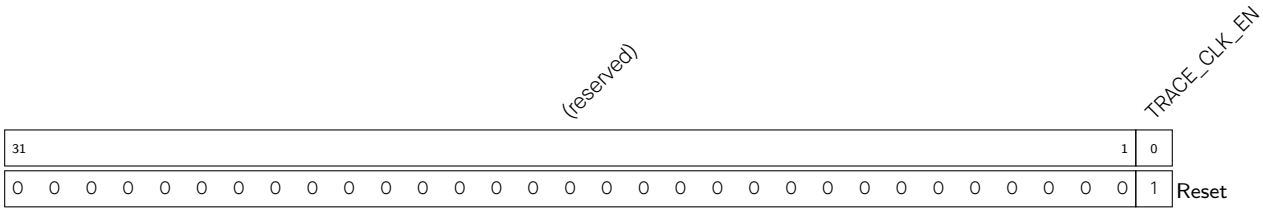
3：无效值，没有作用

4：INCR8

其他值：无效值，没有作用 (R/W)

TRACE_MAX_INCR 配置 INCR 模式下突发传输最大的长度。(R/W)

Register 1.18. TRACE_CLOCK_GATE_REG (0x0044)



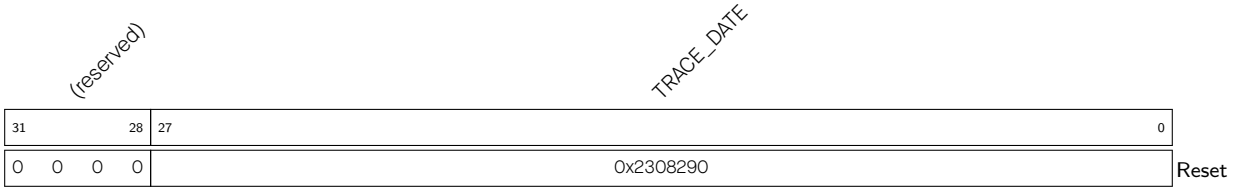
TRACE_CLK_EN 配置寄存器时钟门控。

0：仅在应用写寄存器时开启时钟，从而降低功耗

1：一直强制为寄存器开启时钟

该字段不影响寄存器访问。(R/W)

Register 1.19. TRACE_DATE_REG (0x03FC)



TRACE_DATE 版本控制寄存器。(R/W)

第 2 章

低功耗处理器

2.1 概述

ESP32-S31 低功耗处理器 (Low-Power CPU, LP CPU) 是基于 RISC-V ISA 的 32 位处理器，包括整数 (I)、乘法/除法 (M)、原子 (A) 和压缩 (C) 标准扩展。LP CPU 内核具有 2 级有序标量流水线，具有极低的功耗。CPU 内核架构包含中断控制器 (INTC)、调试模块 (DM) 和用于访问存储器和外设的系统总线 (SYS BUS) 接口。

LP CPU 默认处于睡眠状态（详见章节 2.10），在芯片进入 Deep-sleep 时可保持上电状态（详见章节 11 低功耗管理 *[to be added later]*），能够访问绝大部分外设和存储器，详情请参考 3 系统和存储器 *[to be added later]* 章节。LP CPU 有两种应用场景：

- 功耗不敏感场景：高性能处理器 (HP CPU) 处于工作状态，LP CPU 可协助 HP CPU 完成一些对速度和效率不敏感的控制或计算
- 功耗敏感场景：HP CPU 处于掉电状态来节省功耗时，可以唤醒 LP CPU 处理一些外部唤醒事件

HP CPU 和 LP CPU 可访问的资源如图 2.1-1 所示：

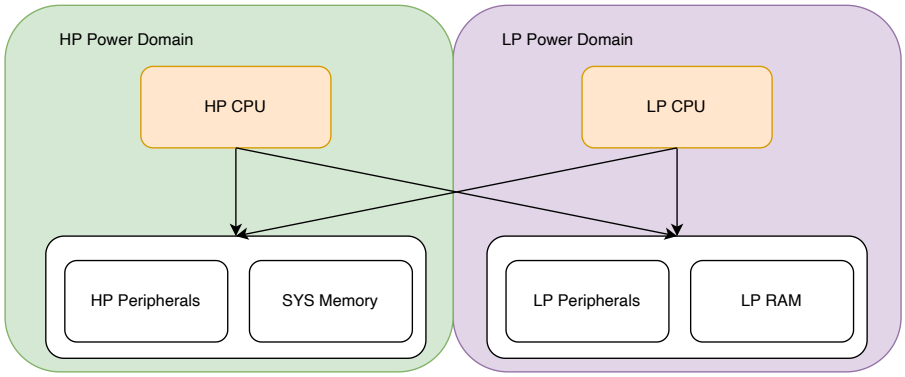


图 2.1-1. LP CPU 概览

在上图中，HP 电源域 (HP Power Domain) 和 LP 电源域 (LP Power Domain) 可分别上下电，当其中一个电源域处于掉电状态时，另一个电源域的 CPU 无法访问该域的资源，只有当该域处于上电状态时才可访问其资源，更多关于电源的信息，请参考章节 11 低功耗管理 *[to be added later]*。

2.2 特性

LP CPU 具有以下特性：

- 时钟工作频率高达 40 MHz
- 支持 18 个向量中断
- 调试模块 (DM) 符合 RISC-V 调试规范 v0.13，支持通过行业标准的 JTAG/USB 端口连接外部调试器

- 硬件触发器符合 RISC-V 调试规范 v0.13，具有 2 个断点/观察点
- 支持核心性能指标事件
- 可唤醒 HP CPU 或向 HP CPU 发送中断
- 可访问 HP 存储器和 LP 存储器
- 可访问所有外设空间
- 标准物理存储器保护 (PMP)，符合 RISC-V 规范

2.3 配置与状态寄存器 (CSR)

2.3.1 寄存器列表

下表为 CPU 可访问的 CSR 列表。除了自定义的性能计数器 CSR 外，所有已实现的 CSR 都遵循 RISC-V 指令集手册 V1.10 第二卷“特权架构”(RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10) 中所述的位域标准映射。必须注意的是，受 CPU 中实现的功能子集的限制，即使在标准 CSR 中也并非实现了所有位域。有关详细的 CSR 寄存器描述，请参阅章节 2.3.2。

请查看章节 寄存器的访问类型，了解“访问”列缩写的含义。

名称	描述	地址	访问
机器模式信息 CSR			
mhartid	机器模式 hart ID 寄存器	0xF14	RO
机器模式陷阱设置 CSR			
mstatus	机器模式状态寄存器	0x300	R/W
misa [1]	机器模式 ISA 寄存器	0x301	R/W
mie	机器模式中断使能寄存器	0x304	R/W
mtvec [2]	机器模式陷阱向量寄存器	0x305	R/W
机器模式陷阱处理 CSR			
mscratch	机器模式暂存寄存器	0x340	R/W
mepc	机器模式陷阱程序计数器	0x341	R/W
mcause [3]	机器模式陷阱原因寄存器	0x342	R/W
mtval	机器模式陷阱值寄存器	0x343	R/W
mip	机器模中断等待寄存器	0x344	R/W
触发器模块 CSR（与调试模块共用）			
tselect	触发器选择寄存器	0x7A0	R/W
tdata1	触发器抽象数据寄存器 1	0x7A1	R/W
tdata2	触发器抽象数据寄存器 2	0x7A2	R/W
调试模式 CSR			
dcsr	调试模式控制与状态寄存器	0x7B0	R/W
dpc	调试模式 PC 寄存器	0x7B1	R/W
dscratch0	调试模式暂存寄存器 0	0x7B2	R/W
dscratch1	调试模式暂存寄存器 1	0x7B3	R/W

[1]尽管 misa 具有 R/W 属性，但由于它的域是硬连线的，所以写操作无效。在 RISC-V 术语中称为 WARL（写入任意数值读取合法数值）。
[2]mtvec 仅支持在向量模式下对陷阱处理进行配置，基地址为 256 字节对齐。
[3]mcause 中反映的外部中断 ID 也包括 RISC-V 标准为处理器内部中断源预留的 ID。

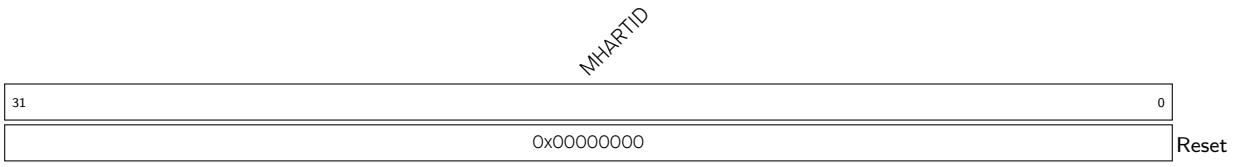
名称	描述	地址	访问
机器模式计数器/定时器 CSR			
mcycle	机器模式时钟周期计数器寄存器	0xB00	R/W
minstret	机器模式退役指令计数器寄存器	0xB02	R/W
mhpmcounter*n*(*n*:3-12)	机器模式性能监测计数器寄存器	0xB00+*n*	R/W
mcycleh	mcycle 的高 32 位	0xB80	R/W
minstreth	minstret 的高 32 位	0xB82	R/W
mhpmcounter*n*h(*n*:3-12)	mhpmcounter*n*(*n*:3-12) 的高 32 位	0xB80+*n*	R/W
机器模式计数器设置 CSR			
mcountinhibit	机器模式计数器控制寄存器	0x320	R/W
物理存储器保护 (PMP) CSR			
pmpcfg0	物理存储器保护配置	0x3A0	R/W
pmpcfg1	物理存储器保护配置	0x3A1	R/W
pmpcfg2	物理存储器保护配置	0x3A2	R/W
pmpcfg3	物理存储器保护配置	0x3A3	R/W
pmpaddr*n* (*n*: 0-15)	物理存储器保护地址寄存器	0x3B0+0x1**n*	R/W

请注意，如果对上表中只读属性的任何 CSR 尝试执行写入/置位/清除操作，CPU 将生成非法指令异常。

2.3.2 寄存器

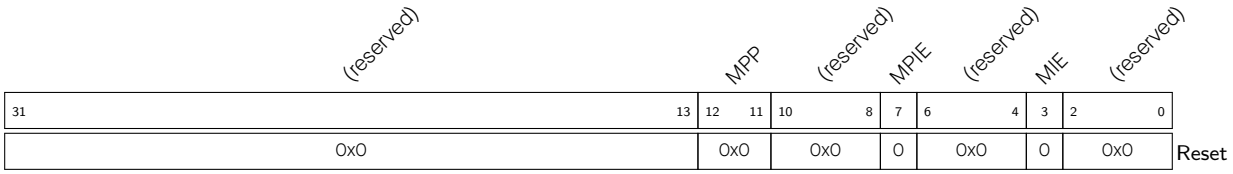
关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 2.1. mhartid (0xF14)



MHARTID 表示硬件线程编号。LP CPU 硬件线程编号为 0。(RO)

Register 2.2. mstatus (0x300)



MIE 写 1 使能全局机器模式中断。(R/W)

MPIE 保存陷阱发生之前的中断使能位的值。(R/W)

MPP 配置（陷阱）之前的机器特权模式。

0x1：用户模式

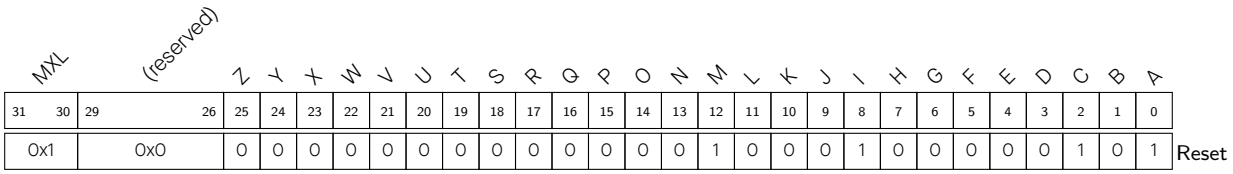
0x3：机器模式

其他值：无效

注意：仅低位可写。由于高位直接绑定低位，写入高位将被忽略。

(R/W)

Register 2.3. misa (0x301)



MXL	(reserved)	Z	Y	X	W	V	U	T	S	R	Q	P	O	N	M	L	K	J	I	H	G	F	E	D	C	B	A	
31 30	29 26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x1	0x0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	1	0	1	Reset

MXL 机器 XLEN = 1（32 位）。(RO)

Z 保留 = 0。(RO)

Y 保留 = 0。(RO)

X 非标准扩展 = 0。(RO)

W 保留 = 0。(RO)

V 保留 = 0。(RO)

U 实现用户模式 = 1。(RO)

T 保留 = 0。(RO)

S 实现监督模式 = 0。(RO)

R 保留 = 0。(RO)

Q 四精度浮点扩展 = 0。(RO)

P 保留 = 0。(RO)

O 保留 = 0。(RO)

N 支持用户级别中断 = 0。(RO)

M 整数乘除法标准扩展 = 1。(RO)

L 保留 = 0。(RO)

K 保留 = 0。(RO)

J 保留 = 0。(RO)

I RV32I 基本 ISA = 1。(RO)

H 虚拟机管理程序扩展 = 0。(RO)

G 其他标准扩展 = 0。(RO)

F 单精度浮点扩展 = 0。(RO)

E RV32E 基本 ISA = 0。(RO)

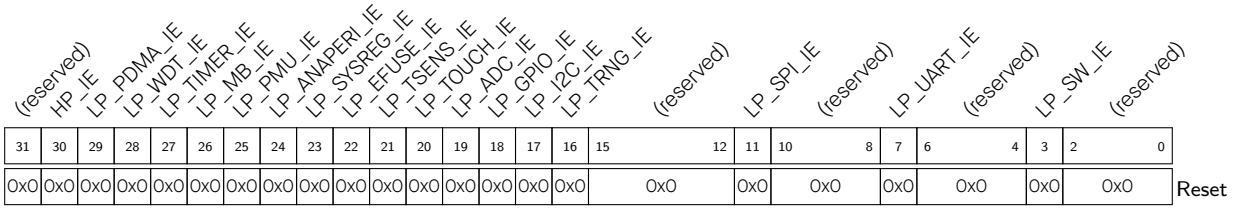
D 双精度浮点扩展 = 0。(RO)

C 压缩标准扩展 = 1。(RO)

B 保留 = 0。(RO)

A 原子标准扩展 = 1。(RO)

Register 2.4. mie (0x304)



LP_SW_IE 写 1 使能 LP 软件中断。(R/W)

LP_UART_IE 写 1 使能 LP UART 中断。(R/W)

LP_SPI_IE 写 1 使能 LP SPI 中断。(R/W)

LP_I2C_IE 写 1 使能 I2C 中断。(R/W)

LP_TRNG_IE 写 1 使能 TRNG 中断。(R/W)

LP_GPIO_IE 写 1 使能 GPIO 中断。(R/W)

LP_ADC_IE 写 1 使能 ADC 中断。(R/W)

LP_TOUCH_IE 写 1 使能 TOUCH 中断。(R/W)

LP_TSENS_IE 写 1 使能 TSENS 中断。(R/W)

LP_EFUSE_IE 写 1 使能 EFUSE 中断。(R/W)

LP_SYSREG_IE 写 1 使能 LP SYSREG 中断。(R/W)

LP_ANAPERI_IE 写 1 使能 ANAPERI 中断。(R/W)

LP_PMU_IE 写 1 使能 PMU 中断。(R/W)

LP_MB_IE 写 1 使能 Mailbox 中断。(R/W)

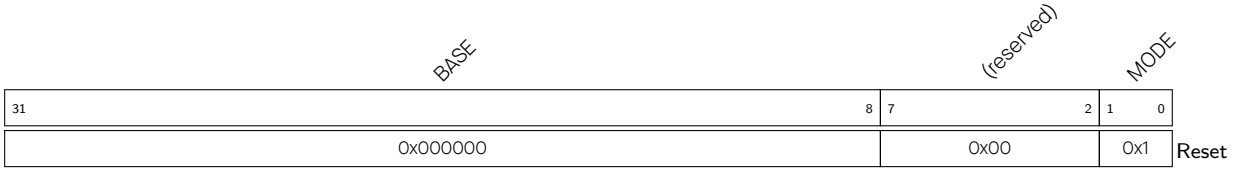
LP_TIMER_IE 写 1 使能 LP TIMER 中断。(R/W)

LP_WDT_IE 写 1 使能 WDT 中断。(R/W)

LP_PDMA_IE 写 1 使能 PDMA 中断。(R/W)

HP_IE 写 1 使能 HP 中断。(R/W)

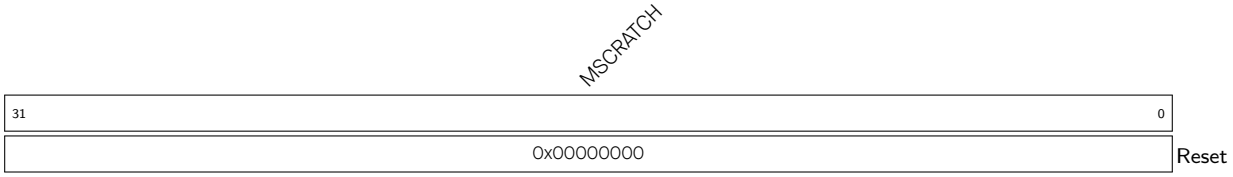
Register 2.5. mtvec (0x305)



MODE 表示机器模式中断是否为向量模式。仅支持向量模式 **0x1**。(RO)

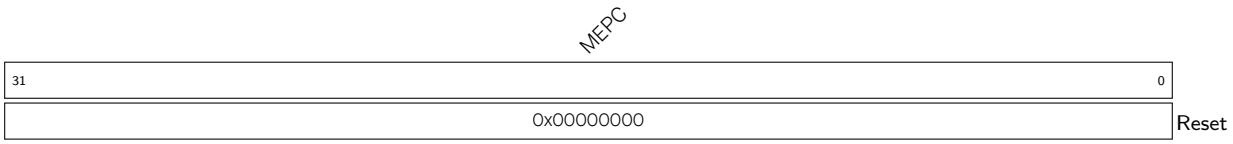
BASE 配置陷阱向量基址的高 24 位，地址为 256 字节对齐。(R/W)

Register 2.6. mscratch (0x340)



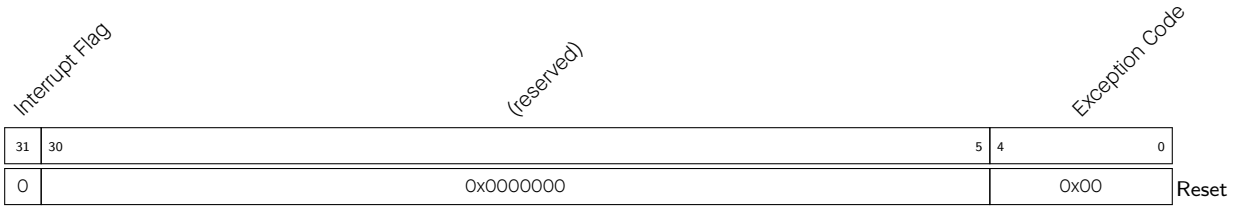
MSCRATCH 保存用户自定义的机器暂存信息。(R/W)

Register 2.7. mepc (0x341)



MEPC 配置机器陷阱/异常程序计数器。当 CPU 遇到陷阱时，此域将自动更新为 CPU 将要执行的指令的地址。(R/W)

Register 2.8. mcause (0x342)

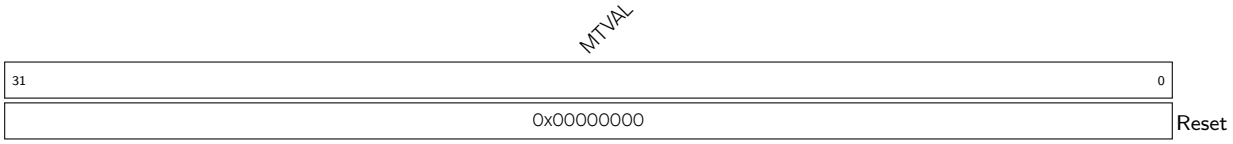


Exception Code CPU 进入异常时，此域将自动更新为最近的陷阱或中断的唯一 ID。可能的异常 ID：

0x1：指令访问错误
0x2：非法指令
0x3：硬件断点/观察点或 EBREAK
0x6：原子指令非对齐
0x5：加载访问异常
0x7：存储访问异常
注意：异常 ID 0x0（指令地址非对齐）不存在，因为 CPU 在取指时始终屏蔽地址的最低位。
(R/W)

Interrupt Flag CPU 进入陷阱时，此标志位将自动更新。如果被置位，则表示最近的陷阱是由中断引起。在异常情况下保持为 0。(R/W)

Register 2.9. mtval (0x343)



MTVAL 配置机器模式陷阱值。将自动更新为与异常有关的数据，该数据可能有助于处理该异常。根据异常编号有以下解读：

0x1：指令地址错误

0x2：指令 opcode 错误

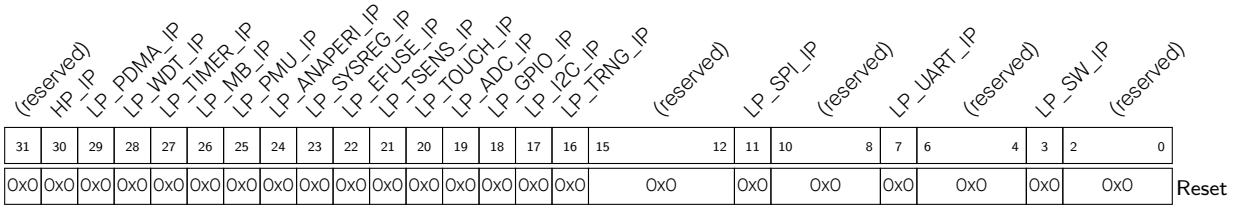
0x5：存储器读操作的数据地址错误

0x7：存储器写操作的数据地址错误

注意：该寄存器不支持其他异常 ID 和中断

(R/W)

Register 2.10. mip (0x344)



LP_SW_IP 配置 LP 软件中断的等待状态。

0：不等待

1：等待

(R/W)

LP_UART_IP 配置 LP UART 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_SPI_IP 配置 LP SPI 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_TRNG_IP 配置 LP TRNG 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_I2C_IP 配置 LP I2C 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_GPIO_IP 配置 LP GPIO 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_ADC_IP 配置 LP ADC 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_TOUCH_IP 配置 LP TOUCH 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_TSENS_IP 配置 LP TSENS 中断的等待状态。

0：不等待

1：等待

(R/W)

见下页……

Register 2.10. mip (0x344)

接上页……

LP_EFUSE_IP 配置 LP EFUSE 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_SYSREG_IP 配置 LP SYSREG 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_ANAPERI_IP 配置 LP ANAPERI 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_PMU_IP 配置 LP PMU 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_MB_IP 配置 mailbox 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_TIMER_IP 配置 LP TIMER 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_WDT_IP 配置 LP WDT 中断的等待状态。

0：不等待

1：等待

(R/W)

LP_PDMA_IP 配置 LP PDMA 中断的等待状态。

0：不等待

1：等待

(R/W)

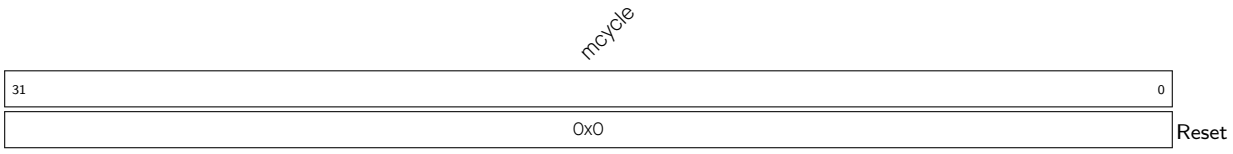
HP_IP 配置 HP 中断的等待状态。

0：不等待

1：等待

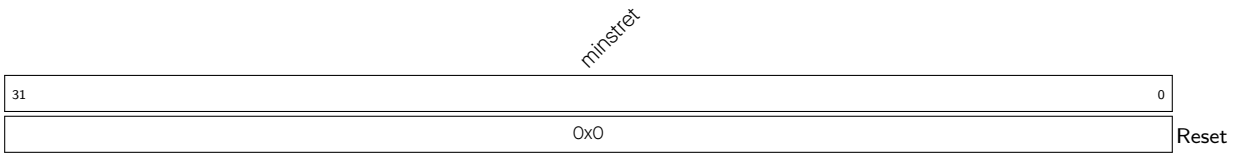
(R/W)

Register 2.11. mcycle (0xB00)



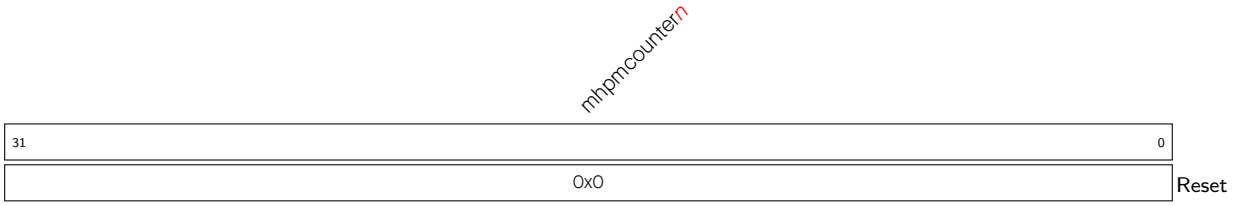
MCYCLE 配置时钟周期计数器的低 32 位。(R/W)

Register 2.12. minstret (0xB02)



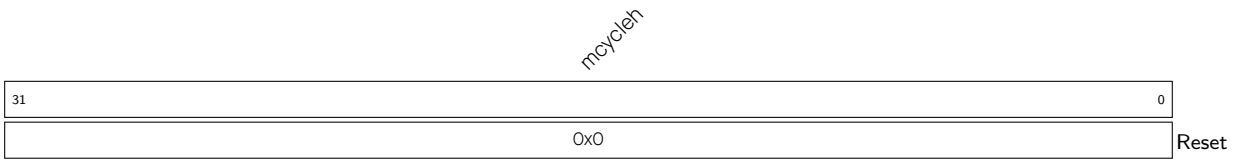
MINSTRET 配置指令计数器的低 32 位。(R/W)

Register 2.13. mhpmcounter*n*(*n*: 3-12) (0xB00+*n*)



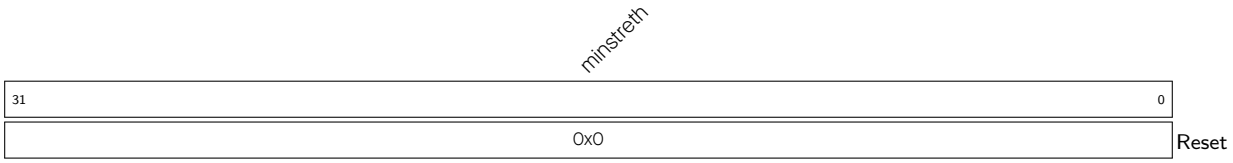
MHPMCOUNTER*n* 配置性能计数器 *n* 的低 32 位。(R/W)

Register 2.14. mcycleh (0xB80)



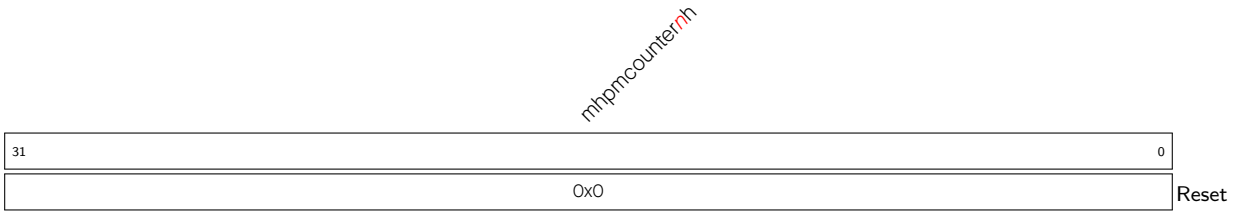
MCYCLEH 配置时钟周期计数器的高 32 位。(R/W)

Register 2.15. minstreth (0xB82)



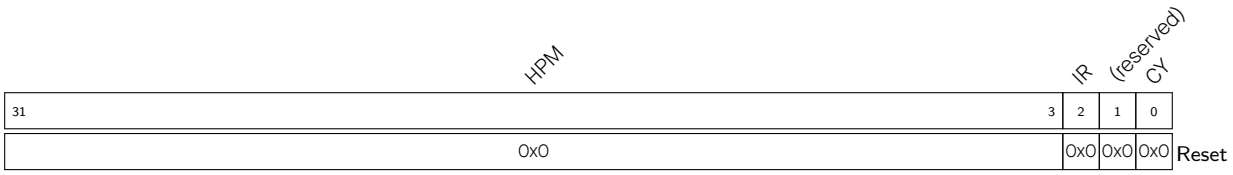
MINSTRETH 配置指令计数器的高 32 位。(R/W)

Register 2.16. mhpmcounter*n*(*n*: 3-12)h (0xB80+*n*)



MHPMCOUNTER*n*h 配置性能计数器 *n* 的高 32 位。(R/W)

Register 2.17. mcountinhibit (0x320)



HPM 配置性能计数器 *n*(*n*:3-12) 是否递增计数。

0：不计数

1：递增计数

(R/W)

IR 配置指令计数器是否递增计数。

0：不计数

1：递增计数

(R/W)

CY 配置时钟周期计数器是否递增计数。

0：不计数

1：递增计数

(R/W)

2.4 中断和异常

LP CPU 根据 RISC-V 指令集手册 V1.10 第二卷 “特权架构” 实现了中断和异常的处理。当 CPU 进入中断/异常处理程序时，内核将执行以下操作：

- 将当前 PC 保存到 mepc CSR 中
- 将 mstatus 的 MIE 保存到 mstatus 的 MPIE
- 将当前特权模式保存到 mstatus 的 MPP
- 将 mstatus 的 MIE 清 0
- 特权模式切换到机器模式
- 跳转到处理程序地址
  - 对于所有异常原因，跳转到 mtvec CSR 中向量表的基地址
  - 对于中断，跳转到 $mtvec + 4 \times ID$ (ID 为中断的编号，参见章节 2.4.1)
- 执行 mret 指令后，内核跳转到之前保存在 mepc CSR 中的程序计数器，并将 mstatus 的 MIE 恢复为 mstatus 的 MPIE，将当前特权模式恢复为 mstatus 的 MPP

当内核启动时，向量表的基地址被初始化为启动地址 0x2E000000。启动后可通过写入 mtvec CSR 更改基地址。有关配置和状态寄存器的详细信息，请参阅章节 2.3.1。

内核在复位后从 LPSYSREG_LP_CORE_BOOT_ADDR_REG 寄存器指定的地址偏移 0x80 开始取指。

2.4.1 中断

ESP32-S31 LP CPU 支持 18 个中断入口，每个中断对应一个 ID，假定为 n (n = 3, 7, 11, 16 ~ 30)，每个中断都有特定的入口地址 $mtvec + 4 \times n$，每个中断都连接特定的外设源，图 2.4-1 列出了外设源与中断号的映射关系：

读取寄存器 LPINTR_STATUS_REG（只读）中特定位的值可以获取 LP CPU 外部中断源当前的状态。寄存器 LPINTR_STATUS_REG 与中断源的关系请参考寄存器说明。中断源的详细信息可参考章节 9。

表 2.4-1. LP CPU 中断映射

中断 ID	外设源
3	LP_SW_INTR、LP_HUK_INTR
7	LP_UART_INTR
11	LP_SPI_INTR
16	LP_TRNG_INTR
17	LP_I2C_INTR
18	LP_GPIO_INTR
19	LP_ADC_INTR、LP_DAC_INTR
20	LP_TOUCH_INTR
21	LP_TSENS_INTR
22	LP_EFUSE_INTR
23	LP_SYSREG_INTR、LP_APM_INTR、 LP_PERI_PMS_INTR、LP_PERI_TIMEOUT_INTR

中断 ID	外设源
24	LP_ANAPERI_INTR
25	PMU_REG_0_INTR、PMU_REG_1_INTR
26	MB_HP_INTR、MB_LP_INTR
27	LP_TIMER_REG_0_INTR、LP_TIMER_REG_1_INTR
28	LP_WDT_INTR
29	LP_PDMA_INTR
30	HP_INTR

2.4.2 中断处理

默认情况下，全局禁用中断，因为 mstatus 的 MIE 的复位值为 0。软件必须设置该位以启用全局中断。

假定使能的中断 ID 为 n：

1. 中断使能
   - 置位 mstatus 的 MIE 域，启用全局中断使能
   - 置位 mie CSR 的第 n 比特，使能中断 n
2. 中断使能后，LP CPU 即可响应中断，同时去配置相应外设的中断，使其能够触发中断信号给 LP CPU。
3. 触发中断后 LP CPU 会跳转到地址 $mtvec + 4 \times n$。
4. 中断清除，只需要清除各外设的中断信号即可。

2.4.3 异常

LP CPU 支持 RISC-V 标准异常，能够触发以下异常：

表 2.4-2. LP CPU 异常原因

异常编号	描述
1	指令访问错误
2	非法指令
3	断点 (EBREAK)
5	加载访问错误
6	原子指令非对齐
7	存储访问错误

2.5 调试

本节介绍如何调试和测试 LP CPU。调试功能由标准 JTAG 管脚提供，并符合 RISC-V 外部调试支持规范 v0.13 (RISC-V External Debug Support, Version 0.13)。

ESP32-S31 系统调试结构请参考章节 *2 高性能处理器 [to be added later]* 中的图 *34 DAC 控制器 (DAC) [to be added later]*。

用户与运行调试器（Debugger，例如 GDB）的调试主机（DEBUG HOST，例如笔记本电脑）进行交互。调试器通过调试转换器（Debug Translator，可能包含硬件驱动，例如 OpenOCD）与调试传输硬件（DEBUG

TRANSPORT，例如 ESP-Prog 适配器）进行通信。调试传输硬件通过标准 JTAG 接口将调试主机连接到处理器内核的调试传输模块 (JTAG DTM)。JTAG DTM 使用调试模块接口 (DMI) 提供对调试模块 (DM) 的访问。

LP CPU 实现了四个内核调试寄存器：dcsr、dpc、dscratch0 和 dscratch1。所有这些寄存器只能从调试模式访问，如果软件试图在内核未处于调试模式的情况下进行访问，则会触发非法指令异常。

2.5.1 特性

LP CPU 调试模块支持以下特性：

- 访问有关实现的必要信息
- 支持暂停和恢复 CPU 内核
- 读写 CPU 内核寄存器（包括 CSR）
- 复位 CPU 内核
- 在软件断点（植入的断点指令）上暂停 CPU
- 硬件单步调试
- 支持两个硬件触发器（可用作断点/观察点），具体见章节 2.6

2.5.2 功能描述

调试机制遵守 RISC-V 外部调试支持规范 v0.13。有关调试功能的详细介绍，请参考 RISC-V 外部调试支持规范 v0.13 (RISC-V External Debug Support, Version 0.13)。

2.5.3 寄存器列表

下表列出了 LP CPU 支持的调试 CSR。

请查看章节 寄存器的访问类型，了解“访问”列缩写的含义。

名称	描述	地址	访问
dcsr	调试控制和状态寄存器	0x7B0	R/W
dpc	调试 PC 寄存器	0x7B1	R/W
dscratch0	调试暂存寄存器 0	0x7B2	R/W
dscratch1	调试暂存寄存器 1	0x7B3	R/W

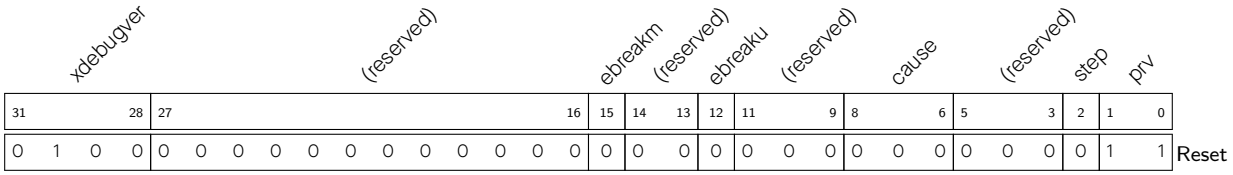
所有调试模块寄存器的实现均符合 RISC-V 外部调试支持规范 v0.13。请参考 RISC-V 外部调试支持规范 v0.13 获取详细信息。

2.5.4 寄存器

以下是 LP CPU 支持的调试 CSR 的详细描述。

关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 2.18. dcsr (0x7B0)



xdebugver 表示调试版本。

4：存在外部调试支持

(RO)

ebreakm 配置机器模式下执行 EBREAK 指令的行为。

0：触发异常，mcause 是 3。

1：进入调试模式

(R/W)

ebreaku 配置用户模式下执行 EBREAK 指令的行为。

0：如特权架构规范所述，触发异常，mcause 是 3。

1：进入调试模式

(R/W)

cause 表示进入调试模式的原因。当单个周期中有多个原因导致进入调试模式，会反映出具有最高优先级数值的那个原因。

1：执行了一条 EBREAK 指令（优先级 3）

2：触发模块引起暂停（优先级 4）

3：haltreq 被置位（优先级 2）

4：step 被置位导致 CPU 单步执行（优先级 1）

其他值：保留，供日后使用

(RO)

step 当被置位且不处于调试模式时，内核将仅执行单个指令，然后进入调试模式。

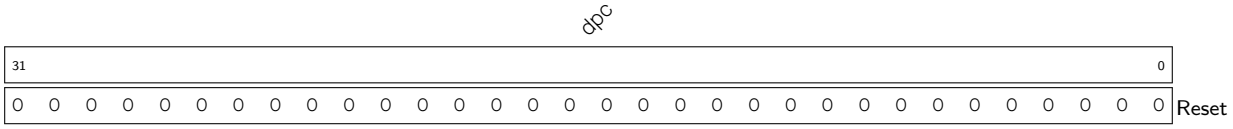
如果指令由于异常而未能完成，则内核将在执行异常处理程序之前立即进入调试模式，并置位相应的异常寄存器。

设置该位不会屏蔽中断，这与 RISC-V 外部调试支持规范 v0.13 中的规定不同。

(R/W)

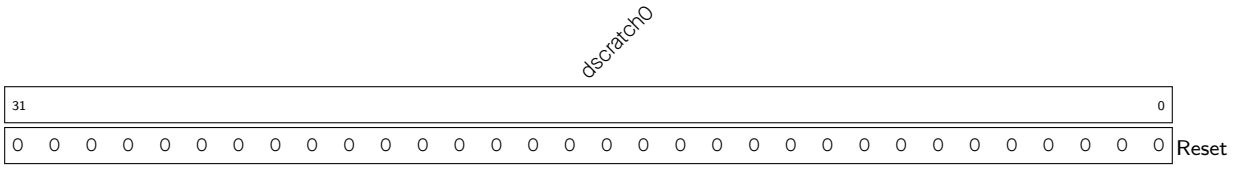
prv 保存 CPU 进入调试模式时的特权级别。退出调试模式时，调试器可以更改此值以改变内核的特权级别。仅支持 **0x3**（机器模式）和 **0x0**（用户模式）。(R/W)

Register 2.19. dpc (0x7B1)



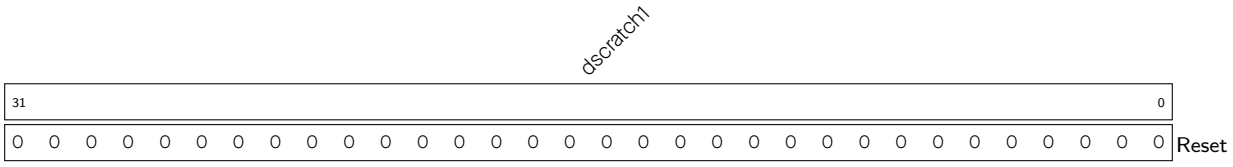
dpc 进入调试模式时，dpc 会更新为将要执行的下一条指令的地址。恢复时，PC 设置为存储在 dpc 中的地址。调试模式可能会修改 dpc。可在调试模式下访问。(R/W)

Register 2.20. dscratch0 (0x7B2)



dscratch0 供调试模块内部使用。(R/W)

Register 2.21. dscratch1 (0x7B3)



dscratch1 供调试模块内部使用。(R/W)

2.6 硬件触发器

2.6.1 特性

硬件触发器模块提供了断点和观察点功能，供调试使用。硬件触发器具有以下特性：

- 2 个独立触发单元
- 每个单元都可以配置为匹配程序计数器的地址
- 可以暂停执行并将控制权转移给调试器

2.6.2 功能描述

硬件触发器模块提供了 5 个 CSR，见章节 2.6.4。其中，tdata1 和 tdata2 是抽象 CSR，也就是说它们是用于访问某个触发单元中的内部寄存器的影子寄存器，一次访问一个触发单元。

要选择特定的触发单元，需要将相应的编号 (0-1) 写入 tselect CSR。当写入有效数值时，抽象 CSR tdata1 和 tdata2 将自动匹配该触发单元的内部寄存器。每个触发单元都有两个内部寄存器，即 mcontrol 和 maddress，它们分别与 tdata1 和 tdata2 匹配。

向 tselect 写入超过 1 的数值时会导致 tselect 写入 1。

由于软件或调试器可能需要知道所选触发器的类型以便正确解读 tdata1 和 tdata2，因此 tdata1 的 4 个位 (31-28) 对所选触发器的类型进行了编码。此域为只读访问属性，并且值始终为 0x2，代表触发器仅支持地址/数据匹配。因此，可以推断 tdata1 和 tdata2 会通过 mcontrol 和 maddress 被解读。RISC-V 外部调试支持规范 v0.13 提供了其他可能值的信息，但是该触发模块仅支持 0x2 类型。

一旦选定了触发单元，就可以通过置位 mcontrol CSR (tdata1) 中相应的域并将目标地址写入 maddress CSR (tdata2) 来对该触发单元进行配置。

2.6.3 触发执行流程

当触发器触发引起硬件线程暂停并进入调试模式时 (action = 1)：

- dpc 被设置为当前 PC（在解码阶段）
- dcsr 的 cause 域被设置为 2，表示暂停是由于触发器触发引起

2.6.4 寄存器列表

下表列出了 CPU 可访问的触发模块 CSR，只有在机器模式下才可以对它们进行读写。

请查看章节 寄存器的访问类型，了解“访问”列缩写的含义。

名称	描述	地址	访问
tselect	触发器选择寄存器	0x7A0	R/W
tdata1	触发器抽象数据寄存器 1	0x7A1	R/W
mcontrol	tdata1 影子寄存器	0x7A1	R/W
tdata2	触发器抽象数据寄存器 2	0x7A2	R/W
maddress	tdata2 影子寄存器	0x7A2	R/W

2.6.5 寄存器

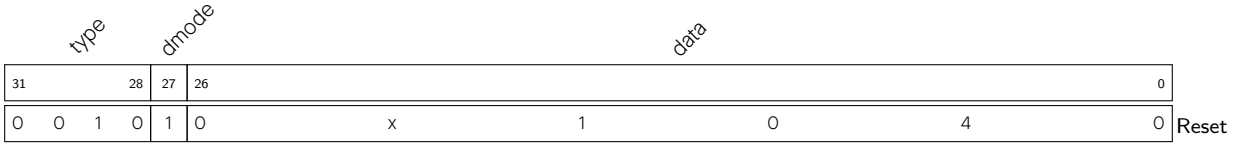
关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 2.22. tselect (0x7A0)



tselect 配置选择触发器单元。(R/W)

Register 2.23. tdata1 (0x7A1)



type 表示触发器类型。仅支持匹配类型 (0x2)，此域保留。(RO)

dmode 如果某触发器正在被调试器使用，则此域置为 1。仅支持调试模式使用，此域保留。(RO)

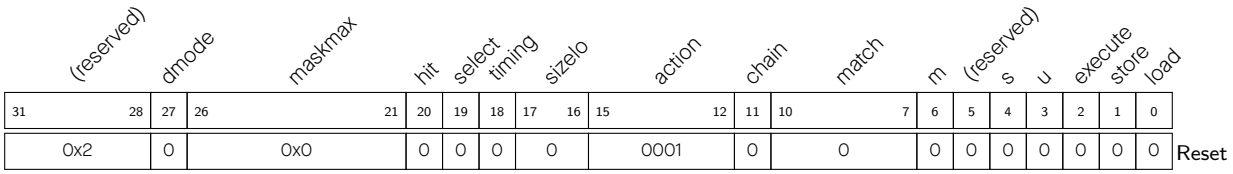
data 配置抽象 tdata1 的内容。由于仅支持匹配类型 (0x2) 触发器，此域将始终被解读为 mcontrol 的域。(R/W)

Register 2.24. tdata2 (0x7A2)



tdata2 配置抽象 tdata2 的内容。由于仅支持匹配类型 (0x2) 触发器，此域将始终被解读为 maddress。(R/W)

Register 2.25. mcontrol (0x7A1)



31–28	27	26–21	20	19	18	17–16	15–12	11	10–7	6	5	4	3	2	1	0
(reserved)	dmode	maskmax	hit	select	timing	sizelo	action	chain	match	m	(reserved)	s	u	execute	store	load
0x2	0	0x0	0	0	0	0	0001	0	0	0	0	0	0	0	0	0
																Reset

dmode 与 tdata1 的 dmode 一致。(RO)

maskmax 表示硬件支持的最大自然对齐的二次幂范围。

0：一个字节的范围，仅支持精确匹配

其他值：不支持

(RO)

hit 硬件未实现，始终为 0。(RO)

select 选择地址匹配或数据匹配。

0：匹配虚拟地址

1：匹配读取或写入的数据值或执行的指令

注意：只实现了地址匹配，此域始终为 0。

(RO)

timing 表示触发器何时被触发。

0：当匹配后，触发器会在指令执行前触发

1：当匹配后，触发器会在指令执行后触发

注意：此域始终为 0。

(RO)

sizelo 仅支持任意大小匹配，此域始终为 0。(RO)

action 配置选定的触发器在触发时进行以下操作。

0x0：引起断点异常

0x1：进入调试模式（仅当 dmode = 1 时有效）

注意：仅支持进入调试模式，此域总为 1。

(RO)

chain 硬件未实现，始终为 0。(RO)

match 配置触发器进行数据/指令地址的匹配操作。

0x0：精确字节匹配，即与访问中某个字节对应的地址必须精确匹配 maddress 的值。

0x1：NAPOT 匹配，即访问中至少有一个字节处于 maddress 中规定的 NAPOT 区域。

注意：仅支持精确字节匹配，此域始终为 0。

(R/W)

m 置位使选定的触发器在机器模式下操作。(RO)

s 置位使选定的触发器在监督模式下操作。不支持在监督模式下操作，此域始终为 0。(RO)

u 置位使选定的触发器在用户模式下操作。不支持在用户模式下操作，此域始终为 0。(RO)

见下页……

Register 2.25. mcontrol (0x7A1)

接上页……

execute 置位使选定的触发器匹配指令的虚拟地址。

0：不使能

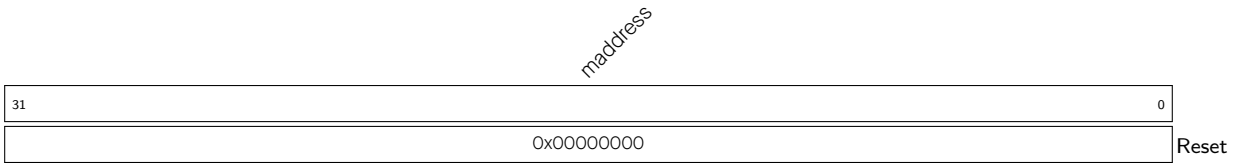
1：使能

(R/W)

store 置位使选定的触发器匹配存储器写操作的虚拟地址。硬件不支持，此域始终为 0。(RO)

load 置位使选定的触发器匹配存储器读操作的虚拟地址。硬件不支持，此域始终为 0。(RO)

Register 2.26. maddress (0x7A2)



maddress 配置选定的触发器执行匹配操作时使用的地址。(R/W)

2.7 性能计数器

LP CPU 实现了时钟周期计数器 mcycle(h)、指令计数器 minstret(h) 以及 10 个事件计数器 mhpmcounter*n*(*n*:3-12) 。时钟周期计数器和指令计数器始终可用且为 64 位，其余性能计数器为 40 位。

默认情况下，所有计数器在复位后启用。通过 mcountinhibit CSR 中的相应位来单独启用/禁用对应的计数器。

每个计数器的计数事件都是固定的，参考表 2.7-1。

表 2.7-1. 性能计数器

计数器	计数事件
mcycle	统计时钟周期数
minstret	统计指令数量
mhpmcounter3	统计内存访问的等待周期
mhpmcounter4	统计取指等待周期
mhpmcounter5	统计内存读操作的数量，非对齐读视作两次。
mhpmcounter6	统计内存写操作的数量，非对齐写视作两次。
mhpmcounter7	统计无条件跳转指令的数量 (jal, jr, jalr)
mhpmcounter8	统计分支指令的数量
mhpmcounter9	统计分支指令跳转的数量
mhpmcounter10	统计压缩指令的数量
mhpmcounter11	统计乘法指令的等待周期

计数器	计数事件
mhpmcounter12	统计除法指令的等待周期

2.8 系统访问

LP CPU 可访问存储器和外设，当访问非法地址（不属于存储器或者外设地址空间），将根据访问类型触发对应的访问错误异常。

2.8.1 存储器访问

ESP32-S31 LP CPU 可以访问的存储器资源有 LP SRAM 和 HP SRAM，详情参考章节 3 *系统和存储器 [to be added later]*。其中：

- LP SRAM：32 KB, 地址范围为 0x2E00_0000 ~ 0x2E00_7FFF，可进行取指、读数据、写数据等操作
- HP SRAM：512 KB, 地址范围为 0x2F00_0000 ~ 0x2F07_FFFF，可进行取指、读数据、写数据等操作

LP CPU 支持原子指令集，LP CPU 和 HP CPU 可通过原子指令访问存储器，从而实现内存访问的原子性。关于原子指令集的细节请参考 RISC-V 指令集手册 v2.2 第一卷“非特权架构”(RISC-V Instruction Set Manual Volume I: Unprivileged ISA, Version 2.2)。

注意：只有 LP SRAM 可以支持 HP CPU 和 LP CPU 的原子访问。

2.8.2 外设访问

章节 3 *系统和存储器 [to be added later]* 表 3 详细列出了 LP CPU 可访问的外设及基地址。

2.9 事件任务矩阵功能

在 ESP32-S31 中，LP CPU 支持事件任务矩阵 (ETM) 功能，即可以通过任意外设的 ETM 事件触发 LP CPU 的 ETM 任务，或者通过 LP CPU 的 ETM 事件触发任意外设的 ETM 任务。关于 ETM 更多详细信息，请参考章节 10 事件任务矩阵 (ETM) [to be added later]。这里仅介绍与 LP CPU 相关的 ETM 任务和 ETM 事件。

LP CPU 可接收的 ETM 任务有：

- ULP_TASK_WAKEUP_CPU：唤醒 LP CPU
- ULP_TASK_INT_CPU：触发 LP CPU 中断

LP CPU 可产生的 ETM 事件有：

- ULP_EVT_ERR_INTR：表示 LP CPU 发生异常
- ULP_EVT_START_INTR：表示 LP CPU 时钟打开
- ULP_EVT_HALT：表示 LP CPU 进入休眠状态

2.10 睡眠和唤醒流程

2.10.1 特性

以下是 LP CPU 支持的睡眠和唤醒特性：

- LP CPU 工作在低功耗系统中，可以在高性能系统进入睡眠的时候，独立进入睡眠、从睡眠中醒来以及进行工作
- LP CPU 可以根据软件运行状态，主动配置寄存器进入睡眠状态
- LP CPU 支持多个唤醒源
- 外设可拒绝 LP CPU 进入睡眠

2.10.2 流程

LP CPU 默认处于睡眠状态，LP CPU 的睡眠唤醒模块将按照如下流程唤醒开始工作以及进入睡眠。

LP CPU 的唤醒源配置如表 2.10-1 所示。

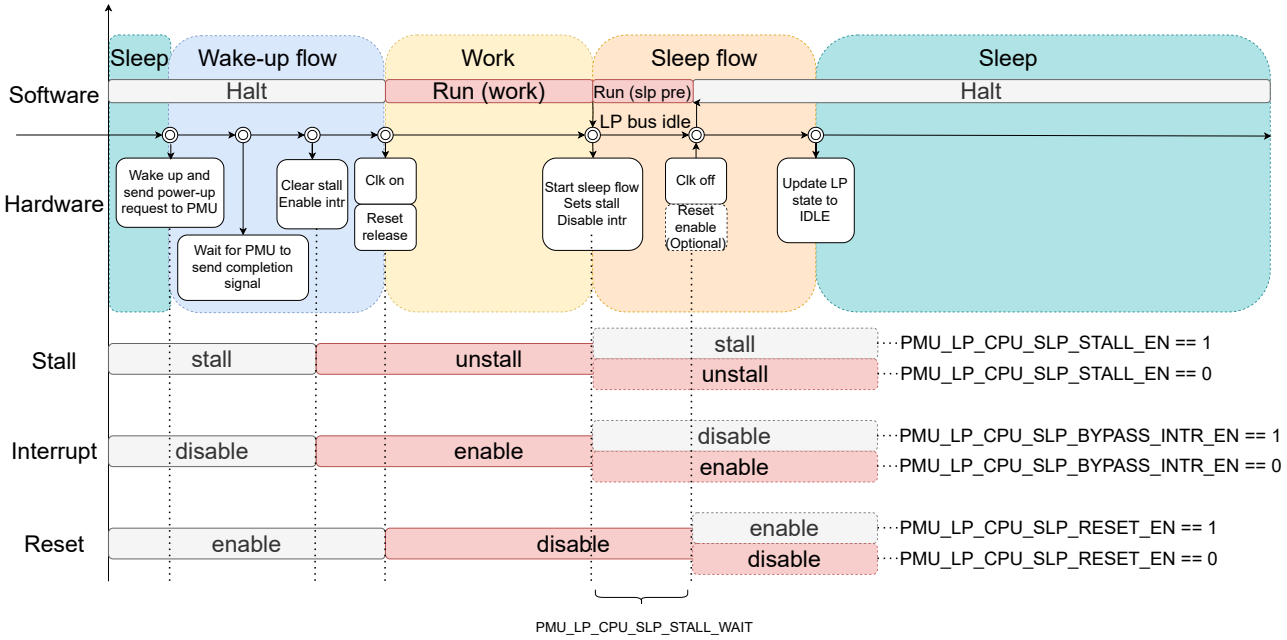


图 2.10-1. LP CPU 唤醒和睡眠流程

LP CPU 上电后的首次启动取决于 HP CPU 对能唤醒使能信号和唤醒源的配置。

- LP CPU 的初始化
  - 初始化 LP 存储器。
  - 启动 LP CPU。由于 LP CPU 的启动依赖于唤醒过程，所以推荐使用 PMU_HP_TRIGGER_LP 寄存器来开始 LP CPU 的初始化，方法如下：
    * 将 PMU_LP_CPU_WAKEUP_EN 设置为 0x00000002
    * 将 PMU_HP_TRIGGER_LP 设置为 0x1
    * LP CPU 被唤醒后开始运行
- 唤醒启动流程 (Wake-up flow)：
  - 睡眠唤醒模块接收到唤醒信号向电池管理单元 (PMU) 发送上电请求。

- 如果当前功耗状态（时钟、电源等）已经满足 LP CPU 需求，则 PMU 立刻回复完成信号，否则将调整功耗状态再回复完成信号。
- 睡眠唤醒模块关闭 LP CPU 的 STALL 状态，并打开中断接收使能。
- 睡眠唤醒模块开启时钟并释放复位（如果睡眠没有使能复位则忽略该步骤），然后开始工作。

- 睡眠流程 (Sleep flow)：
  - LP CPU 配置 PMU_LP_CPU_SLEEP_REQ 寄存器，睡眠唤醒模块启动睡眠流程。
  - 如果 PMU_LP_CPU_SLP_STALL_EN 为 1，睡眠唤醒模块则使能 LP CPU 的 STALL 状态，如果为 0，则不使能 LP CPU 的 STALL 状态。如果 PMU_LP_CPU_SLP_BYPASS_INTR_EN 为 1，睡眠唤醒模块则屏蔽 LP CPU 的所有中断，如果为 0，则不屏蔽 CPU 的中断。
  - 睡眠唤醒模块等待 PMU_LP_CPU_SLP_STALL_WAIT 个 LP CPU 时钟周期后，关闭 LP CPU 的时钟，如果 PMU_LP_CPU_SLP_RESET_EN 为 1，则使能 LP CPU 的复位。
  - 睡眠唤醒模块更新 LP CPU 的状态为 IDLE 状态，PMU 可以进入更深层次的睡眠。

2.10.3 唤醒源

表 2.10-1. 唤醒源

寄存器值[1]	唤醒源	描述
BIT(1)	寄存器 PMU_HP_TRIGGER_LP	通过主核置位寄存器 PMU_HP_TRIGGER_LP 可唤醒 LP CPU，置位 REG_HP_SW_TRIGGER_INT_CLR 可清除唤醒源
BIT(9)	LP IO	该唤醒源使用 LP IO 的中断状态寄存器信号，详情请见章节 *6 IO MUX 和 GPIO 交换矩阵 [to be added later]*
BIT(10)	LP UART	LP UART 接收一定数量的 RX 脉冲，需要使能 REG_UART_WAKEUP_EN，详情请见章节 *21 UART 控制器 (UART)*
BIT(13)	RTC 定时器	使用 RTC 定时器的 target 0 超时中断控制，详情请见章节 *11 低功耗管理 [to be added later]*
BIT(14)	欠压检测	发生欠压时，可唤醒 LP CPU，详情请见章节 *9 电源检测器 (PSDET)*
BIT(17)	ETM	可接收来自 ETM 的唤醒源，详情请见章节 *10 事件任务矩阵 (ETM) [to be added later]*
BIT(18)	RTC 定时器	使用 RTC 定时器的 target 1 超时中断控制，详情请见章节 *11 低功耗管理 [to be added later]*

[1] 寄存器 PMU_LP_CPU_WAKEUP_EN 的值

2.10.4 拒绝睡眠

当 LP CPU 准备进入睡眠状态时，可以被外设事件拒绝，即外设事件可以阻止 LP CPU 进入睡眠。以下是外设事件阻止 LP CPU 进入睡眠状态的配置流程：

1. LP CPU 配置 PMU_SLP_REJECT_EN 使能外设阻止 LP CPU 进入睡眠状态的功能
2. LP CPU 配置 PMU_SLEEP_REJECT_ENA 使能对应外设阻止 LP CPU 进入睡眠状态

阻止 LP CPU 进入睡眠的外设源头与唤醒源一致。

2.11 物理存储器保护

物理存储器保护单元（PMP）提供物理地址空间的访问权限控制，在机器模式可对 PMP 进行配置。LP CPU 核心包含 PMP 单元，完全符合 RISC-V 指令集手册 v1.10 第二卷 “特权架构”，在本章节未定义的信息，可以参照 RISC-V 协议。

2.11.1 特性

除了特权架构规范中所支持的特性，在实现层面，LP CPU 具有以下特性：

- 16 个 PMP 表项
- 各个表项最小可配置区域粒度：128 字节
- 支持三种地址匹配模式：OFF、TOR（范围顶部）和 NAPOT（自然对齐的 2 的幂）

2.11.2 功能描述

PMP 机制遵守 RISC-V 规范，PMP 的配置请参照 RISC-V 规范。在配置完成后，对于 PMP 检查失败的请求，将上报指令访问错误、加载访问错误、或存储访问错误异常。

2.11.3 寄存器列表

下表列出了 PMP 相关的 CSR，只有在机器模式下才可以对它们进行读写。

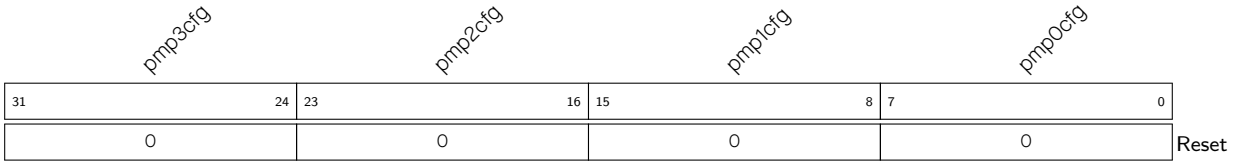
请查看章节 寄存器的访问类型，了解 “访问” 列缩写的含义。

名称	描述	地址	访问
pmpcfg0	物理存储器保护配置寄存器	0x3A0	R/W
pmpcfg1	物理存储器保护配置寄存器	0x3A1	R/W
pmpcfg2	物理存储器保护配置寄存器	0x3A2	R/W
pmpcfg3	物理存储器保护配置寄存器	0x3A3	R/W
pmpaddr*n* (*n*: 0-15)	物理存储器保护地址寄存器	0x3B0+0x1**n*	R/W

2.11.4 寄存器

关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 2.27. pmpcfg0 (0x3A0)



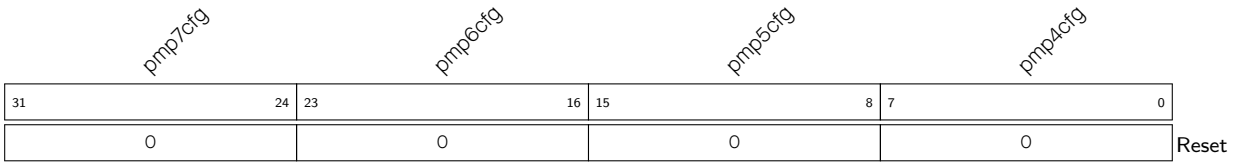
pmp3cfg PMP 表项 3 的配置寄存器。(R/W)

pmp2cfg PMP 表项 2 的配置寄存器。(R/W)

pmp1cfg PMP 表项 1 的配置寄存器。(R/W)

pmp0cfg PMP 表项 0 的配置寄存器。(R/W)

Register 2.28. pmpcfg1 (0x3A1)



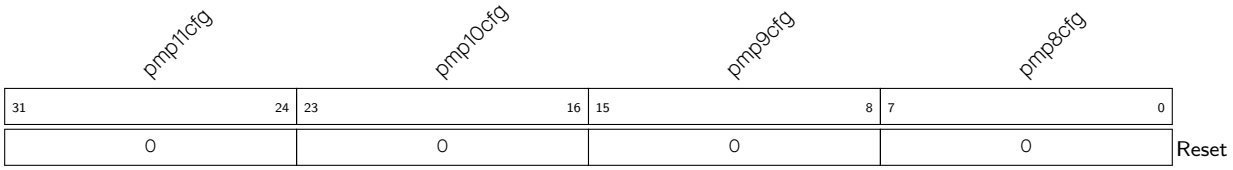
pmp7cfg PMP 表项 7 的配置寄存器。(R/W)

pmp6cfg PMP 表项 6 的配置寄存器。(R/W)

pmp5cfg PMP 表项 5 的配置寄存器。(R/W)

pmp4cfg PMP 表项 4 的配置寄存器。(R/W)

Register 2.29. pmpcfg2 (0x3A2)



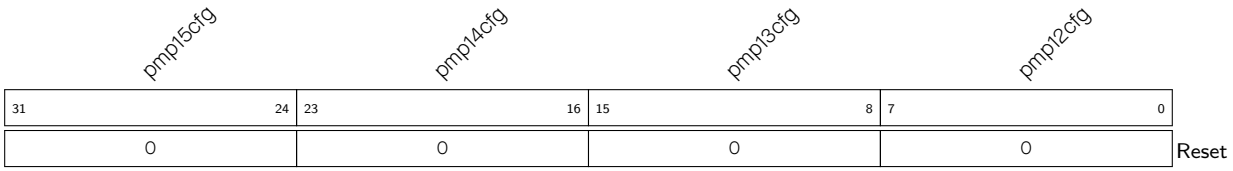
pmp11cfg PMP 表项 11 的配置寄存器。(R/W)

pmp10cfg PMP 表项 10 的配置寄存器。(R/W)

pmp9cfg PMP 表项 9 的配置寄存器。(R/W)

pmp8cfg PMP 表项 8 的配置寄存器。(R/W)

Register 2.30. pmpcfg3 (0x3A3)



pmp15cfg PMP 表项 15 的配置寄存器。(R/W)

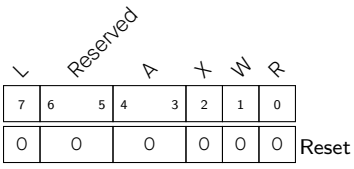
pmp14cfg PMP 表项 14 的配置寄存器。(R/W)

pmp13cfg PMP 表项 13 的配置寄存器。(R/W)

pmp12cfg PMP 表项 12 的配置寄存器。(R/W)

以下为 PMP 表项配置寄存器的格式。

Register 2.31. pmpXcfg



L 配置锁定位。一旦设置，只能通过核心复位来清除。(R/W)

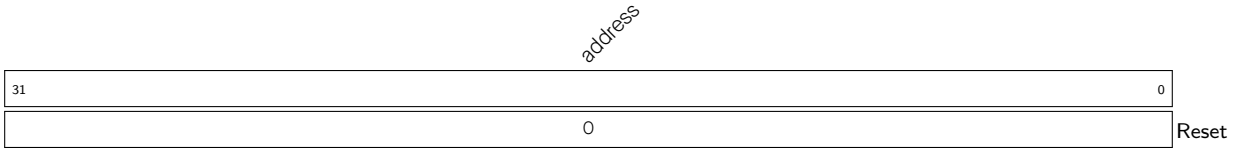
A 配置地址匹配模式。
0：关闭
1：TOR（范围顶部）
2：不支持
3：NAPOT（自然对齐的 2 的幂次方区域，大小 >= 128 字节）
(R/W)

X 配置执行权限。设置后，允许从地址范围执行。(R/W)

W 配置写入权限。设置后，允许对地址范围进行存储器写入。(R/W)

R 配置读取权限。设置后，允许从地址范围进行存储器读取。(R/W)

Register 2.32. pmpaddr*n* (*n*: 0-15) (0x3B0+0x1**n*)



address 配置 pmpaddr*n* 的编码地址。(R/W)

第 3 章

通用 DMA 控制器 (GDMA-AHB, GDMA-AXI)

3.1 概述

通用直接存储访问 (General Direct Memory Access, GDMA) 用于在外设与存储器之间以及存储器与存储器之间提供高速数据传输。借助 GDMA，软件可在无需 CPU 干预的情况下快速传输数据，从而降低 CPU 负载并提高系统效率。

ESP32-S31 有两种 GDMA 控制器，分别可直接访问 AHB 总线和 AXI 总线，下文分别简称为 GDMA-AHB 和 GDMA-AXI。其中，GDMA-AHB 根据高性能系统和低功耗系统的区别，又分为 GDMA-AHB-HP 和 GDMA-AHB-LP。GDMA 通道由支持 GDMA 功能的外设共享，用户可将通道分配给任一支持 GDMA 功能的外设。

- GDMA-AHB-HP：10 个独立通道，包含 5 个发送通道和 5 个接收通道。支持的外设包括 UHCI、I2S、RMT、音频采样率转换器 (ASRC) 和 CORDIC 加速器。
- GDMA-AHB-LP：4 个独立通道，包含 2 个发送通道和 2 个接收通道。支持的外设包括 ADC 和 DAC。
- GDMA-AXI：6 个独立通道，包含 3 个发送通道和 3 个接收通道。支持的外设包括 LCD、CAM、两个通用 SPI、PARLIO、AES 和 SHA。

GDMA-AHB-HP 和 GDMA-AHB-LP 除通道数和支持的外设不同外，功能完全一致。下文以 GDMA-AHB-HP 为例进行说明，相关描述同样适用于 GDMA-AHB-LP，对应寄存器将 **AHB_DMA_** 前缀替换为 **LP_AHB_DMA_** 即可。下文如无特殊说明，GDMA-AHB 指代 GDMA-AHB-HP 和 GDMA-AHB-LP，GDMA 指代 GDMA-AHB 和 GDMA-AXI。

在 ESP32-S31 上，GDMA-AHB 和 GDMA-AXI 均可访问内部和外部存储器。GDMA-AHB 和 GDMA-AXI 支持可配置的通道间优先级和权重仲裁，以满足不同外设的带宽需求。

3.2 特性

GDMA-AHB 和 GDMA-AXI 具有以下共同特性：

- 数据传输以字节为单位，传输数据量可软件编程
- 支持任意地址和大小 (size) 访问
- 支持链表
- 任一通道支持可配置的外设选择
- 支持通道间优先级和权重仲裁配置
- 支持存储器到存储器传输功能

GDMA-AHB 特性：

- 架构：AHB 总线架构

- 通道：GDMA-AHB-HP 有五个发送通道和五个接收通道，GDMA-AHB-LP 有两个发送通道和两个接收通道
- 对齐要求：
  - 描述符存储地址：字对齐
  - 数据地址和长度：
    * 内部存储器和外部存储器的非加密空间：无要求
    * 外部存储器的加密空间（仅 GDMA-AHB-HP 支持访问，GDMA-AHB-LP 不支持）：16 字节对齐
- 访问存储器时支持 INCR4、INCR8 或 INCR16 突发传输

GDMA-AXI 特性：

- 架构：AXI 总线架构，支持深度为 8 的乱序传输 (out-of-order) 和深度为 8 的挂起传输 (outstanding)
- 通道：各有三个发送通道和三个接收通道
- 对齐要求：
  - 描述符存储地址：双字对齐
  - 数据地址和长度：
    * 内部存储器和外部存储器的非加密空间：无要求
    * 外部存储器的加密空间：16 字节对齐
- 访问存储器时支持突发传输，突发长度可配置为 1、2、4、8、16 拍
- 支持软件切换链表中断响应机制

3.3 架构

GDMA 控制器与 CPU 的数据总线使用相同的地址空间访问存储器。ESP32-S31 中所有需要高速数据传输的模块均支持 GDMA 功能。图 3.3-1 为 GDMA 控制器基本架构图。

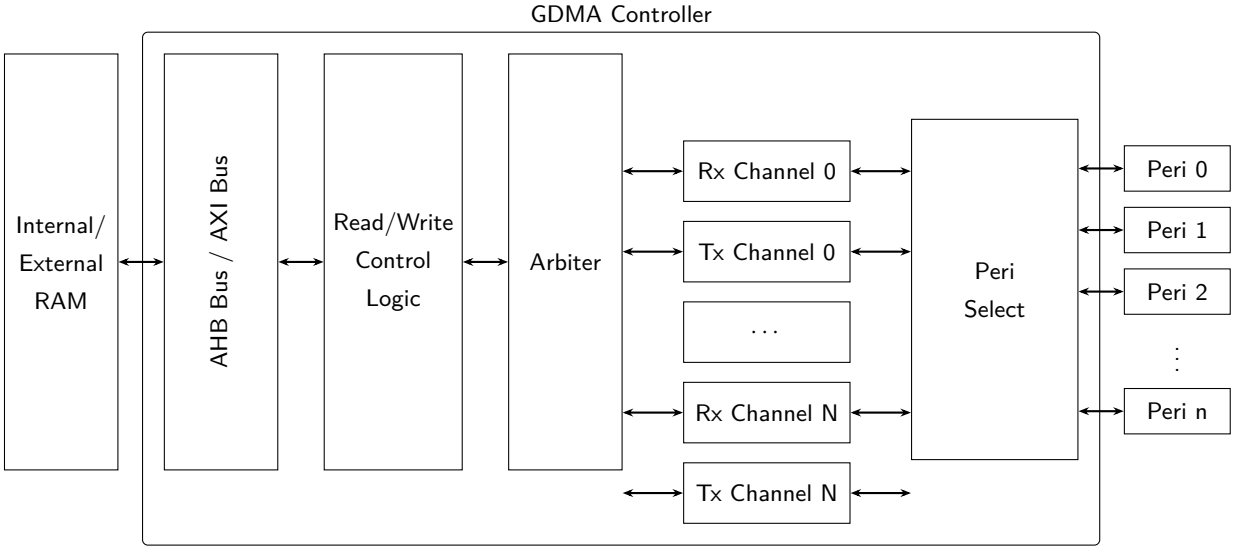


图 3.3-1. GDMA 控制器架构

GDMA-AHB 和 GDMA-AXI 都有固定数量的传输通道，其发送通道和接收通道的数量一致。每个通道可选择与不同的外设相连，从而实现通道资源由外设共享。

GDMA-AHB 和 GDMA-AXI 通过 AHB 总线或 AXI 总线将数据存入内部或外部存储器，或者从内部或外部存储器取出数据。在通过总线传输数据之前，GDMA 采用可配置的仲裁机制对每个通道的读写请求进行仲裁。存储器的具体使用范围详见章节 3 *系统和存储器 [to be added later]*。

软件可以通过挂载链表的方式来使用 GDMA 控制器。GDMA-AHB 链表存储在内部存储器中，GDMA-AXI 链表存储在内部或外部存储器中。这些链表包括 outlink*n* 与 inlink*n*，本文以 *n* 来表示通道号，*n* 为实际物理通道编号。

GDMA 从存储器中取得链表，然后根据 outlink*n* 中的内容将相应存储器中的数据发送出去，也可根据 inlink*n* 中的内容将接收的数据存入指定的存储器地址空间。

3.4 功能描述

3.4.1 链表

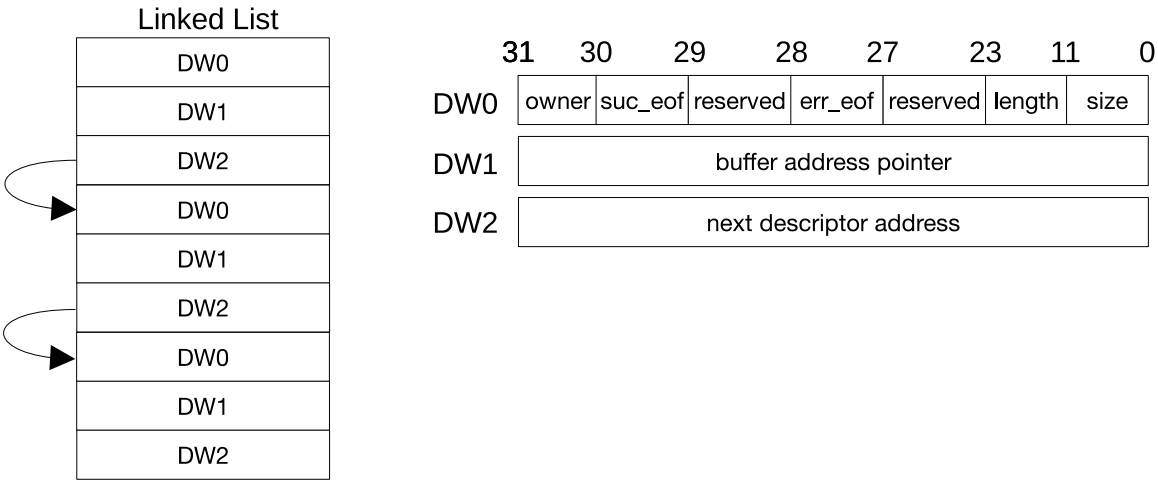


图 3.4-1. 链表结构图

图 3.4-1 所示为链表的结构图。发送链表与接收链表结构相同。每个链表由一个或多个描述符构成，每个描述符由三个字组成。链表应存放在存储器中，供 GDMA 控制器使用。描述符各字段的含义如下：

- owner (DW0) [31]：表示当前描述符对应的 buffer 允许的操作者。
  0：允许的操作者为 CPU。
  1：允许的操作者为 GDMA 控制器。
  在 GDMA 使用完该描述符对应的 buffer 后，对于接收链表描述符，硬件默认会自动将该位清零；对于发送链表描述符，需要将 AHB/AXI_DMA_OUT_AUTO_WRBACK_CH*n* 置 1，硬件才会自动将该位清零。软件也可通过置位 AHB/AXI_DMA_OUT_LOOP_TEST_CH*n* 或 AHB/AXI_DMA_IN_LOOP_TEST_CH*n* 来关闭硬件自动清零功能。软件在挂载链表时需要将该位置 1。

  注意： 本文以 AHB/AXI_DMA_OUT 开头的寄存器对应 TX 通道寄存器，以 AHB/AXI_DMA_IN 开头的寄存器对应 RX 通道寄存器。

- suc_eof (DW0) [30]：表示一个描述符对应的数据成功传输后是否触发 AHB/AXI_DMA_IN_SUC_EOF_CH*n*_INT 或 AHB/AXI_DMA_OUT_EOF_CH*n*_INT 中断。

0：当前描述符成功传输后不会触发中断；
1：当前描述符成功传输后触发中断。
对于接收链表描述符，需要软件将该位写 0，硬件会在接收到包含 EOF 标志的数据后将该位置 1。
对于发送链表描述符，需要软件按需要将描述符中的该位置 1。如果软件在某个描述符中将该位配置为 1，则 GDMA 在处理完该描述符时，会在发送给外设的数据中加入 EOF 标志，告知外设该段数据已阶段性结束。

- Reserved (DW0) [29]：保留。此位为无关项。
- err_eof (DW0) [28]：表示接收结束错误标志。
  0：接收数据没有错误。
  1：接收数据有错误。
  该位只用于 UHCI 或 PARLIO 利用 GDMA 接收数据。对于接收链表描述符，硬件在收完描述符对应的数据段并检测到接收数据错误时，会将该位置 1。
- Reserved (DW0) [27:24]：保留。
- length (DW0) [23:12]：表示当前描述符对应的 buffer 中的有效字节数。对于发送链表描述符，该段由软件填写，表示从 buffer 中读取数据时需要读取的字节数。对于接收链表描述符，该段由硬件在使用完该 buffer 后或接收到最后一个数据时自动填写，表示 buffer 中存储的有效字节数。
- size (DW0) [11:0]：表示当前描述符对应的 buffer 容量的字节数。
- buffer address pointer (DW1)：buffer 的地址。
- next descriptor address (DW2)：下一个描述符的地址。如果当前描述符为链表中最后一个描述符，该值为 0。对于 GDMA-AHB，该地址必须指向内部存储器的地址空间；对于 GDMA-AXI，该地址必须指向内部或外部存储器的地址空间。

> **说明：**
> GDMA-AHB 的链表描述符存储地址须 4 字节对齐，GDMA-AXI 须 8 字节对齐。

用 GDMA 接收数据时，如果接收数据的长度小于当前描述符指定的 buffer size，则下一个描述符对应的接收数据不会占用该 buffer 的剩余空间。

3.4.2 外设到存储器及存储器到外设的数据传输

GDMA 支持存储器到外设及外设到存储器的数据传输，分别对应 TX 及 RX 功能。TX 通道通过 outlink*n* 将指定存储区域中的数据传输到外设的发送端；RX 通道通过 inlink*n* 将外设接收到的数据传输到指定的存储区域。

每个 TX/RX 通道均可配置连接到任一支持 GDMA 功能的外设。表 3.4-1、3.4-2 和表 3.4-3 列出了通过寄存器配置通道与外设对应关系的方法。其中“Dummy-*n*”是 GDMA 实现存储器到存储器传输功能时可选配的寄存器值。当某一通道已与某一外设连接时，其他通道不能再配置为连接该外设。

表 3.4-1. GDMA-AHB-HP 配置寄存器与外设选择关系表

AHB_DMA_PERI_IN_SEL_CH*n* AHB_DMA_PERI_OUT_SEL_CH*n*	外设
0	UHCI0
1	I2S0-0
2	I2S0-1

3	I2S0-2
4	I2S0-3
5	I2S1-0
6	I2S1-1
7	I2S1-2
8	I2S1-3
9	Dummy-9
10	RMT
11	音频采样率转换器-0
12	音频采样率转换器-1
13	CORDIC 加速器
14 ~ 63	无效值

表 3.4-2. GDMA-AHB-LP 配置寄存器与外设选择关系表

LP_AHB_DMA_PERI_IN_SEL_CH*n* LP_AHB_DMA_PERI_OUT_SEL_CH*n*	外设
0	ADC
1	DAC
2 ~ 63	Dummy

表 3.4-3. GDMA-AXI 配置寄存器与外设选择关系表

AXI_DMA_PERI_IN_SEL_CH*n* AXI_DMA_PERI_OUT_SEL_CH*n*	外设
0	LCD 或 CAM
1	SPI2
2	SPI3
3	PARLIO
4	AES
5	SHA
6 ~ 15	Dummy-6 ~ 15
16 ~ 63	无效值

3.4.3 存储器到存储器数据传输

GDMA 支持存储器到存储器的数据传输。置位 AHB/AXI_DMA_MEM_TRANS_EN_CH*n* 后，TX 通道*n* 的输出将与 RX 通道*n* 的输入相连，从而启用存储器到存储器的数据传输功能。

一个 TX 通道仅与其编号对应的 RX 通道配对，实现存储器到存储器传输。同时，需要将 AHB/AXI_DMA_PERI_IN_SEL_CH*n* 和 AHB/AXI_DMA_PERI_OUT_SEL_CH*n* 配置为相同且对应 Dummy 的值。

GDMA-AHB-HP 的 5 个通道和 GDMA-AHB-LP 的 2 个通道均支持存储器到存储器的数据传输，

AHB_DMA_AHBINF_RESP_ERR_ID 记录到的 ID 为固定值：ID 为 16 时，指 GDMA-AHB 通道 0 的存储器到存储器传输；ID 为 17 时，指 GDMA-AHB 通道 1 的存储器到存储器传输，以此类推。

3.4.4 启动 GDMA

软件通过挂载链表的方式来使用 GDMA。对于接收数据，软件挂载好接收链表并准备好接收数据，配置 AHB/AXI_DMA_INLINK_ADDR_CH*n* 字段指向第一个接收链表描述符，置位 AHB/AXI_DMA_INLINK_START_CH*n* 位启动 GDMA。

对于发送数据，软件挂载好发送链表并准备好发送数据，配置 AHB/AXI_DMA_OUTLINK_ADDR_CH*n* 字段指向第一个发送链表描述符，置位 AHB/AXI_DMA_OUTLINK_START_CH*n* 位启动 GDMA。AHB/AXI_DMA_INLINK_START_CH*n* 与 AHB/AXI_DMA_OUTLINK_START_CH*n* 位由硬件自动清零。

对于 GDMA-AXI RX 通道，置位 AXI_DMA_IN_CMD_DISABLE_CH*n* 可暂停对应通道的数据传输。

有时需要在 DMA 数据传输已经开始后追加更多描述符。要追加描述符，看似只需清空已挂载链表最后一个描述符的 EOF 位，并将该描述符的 next descriptor address (DW2) 字段配置为新链表第一个描述符的地址。但如果 DMA 数据传输已经或即将结束，该方法便无法奏效。GDMA 控制器有专门的逻辑来确保数据传输继续或重启：如果数据传输仍在进行，GDMA 控制器会处理新追加的描述符；如果数据传输已经结束，GDMA 控制器会重启数据传输，传输新追加的描述符。该逻辑由 Restart 功能实现。

软件使用 Restart 功能时，需要重写已挂载链表的最后一个描述符，使其第三个字（即 DW2）指向新链表的首地址；然后置位 AHB/AXI_DMA_INLINK_RESTART_CH*n* 或 AHB/AXI_DMA_OUTLINK_RESTART_CH*n*（这两个位由硬件自动清零）。如图 3.4-2 所示，硬件在读取已挂载链表的最后一个描述符时，会获取新挂载链表的地址，从而继续处理新挂载的链表。

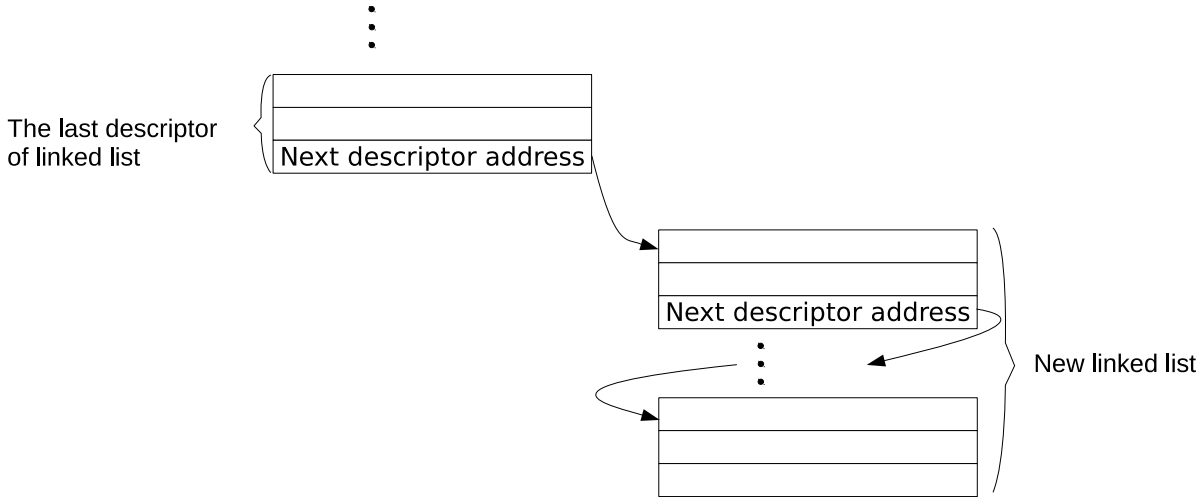


图 3.4-2. 链表关系图

3.4.5 读链表

软件在配置并启动 GDMA 后，GDMA 会从存储器读取链表，并检查读入的链表描述符是否正确。只有当链表描述符通过所有检查时，GDMA 对应的通道才会开始传输数据。链表描述符未通过检查时，硬件将触发描述符错误中断（AHB/AXI_DMA_IN_DSCR_ERR_CH*n*_INT 或 AHB/AXI_DMA_OUT_DSCR_ERR_CH*n*_INT），同时该通道将处于阻塞状态并停止工作。

描述符检查项包括：

- AHB/AXI_DMA_IN_CHECK_OWNER_CH*n* 或者 AHB/AXI_DMA_OUT_CHECK_OWNER_CH*n* 置 1 时，检查描述符的 owner 位。如果该位为 0，表示当前操作者为 CPU，则不通过检查。AHB/AXI_DMA_IN_CHECK_OWNER_CH*n* 或者 AHB/AXI_DMA_OUT_CHECK_OWNER_CH*n* 置 0 时，不检查描述符的 owner 位。
- 检查描述符本身的存储地址是否在配置的存储器地址空间内。
  - GDMA-AHB：描述符地址须在 AHB_DMA_ACCESS_INTR_MEM_START_ADDR ~ AHB_DMA_ACCESS_INTR_MEM_END_ADDR 值的范围内。
  - GDMA-AXI：描述符地址须在 AXI_DMA_ACCESS_INTR_MEM_START_ADDR ~ AXI_DMA_ACCESS_INTR_MEM_END_ADDR，或 AXI_DMA_ACCESS_EXTR_MEM_START_ADDR ~ AXI_DMA_ACCESS_EXTR_MEM_END_ADDR 值的范围内。

  请参见章节 3.4.8。

软件在检测到通道描述符错误中断后，需要复位对应的通道，并置位 AHB/AXI_DMA_OUTLINK_START_CH*n* 或 AHB/AXI_DMA_INLINK_START_CH*n* 位以重新启动 GDMA。

3.4.6 GDMA-AXI 切换链表的中断响应机制

GDMA-AXI 支持在链表切换完成时触发中断。软件可将 AXI_DMA_LINK_SWITCH_STATE_CH*n* 配置为 1，使能 GDMA-AXI 切换链表。链表切换成功后，如果 AXI_DMA_OUT_LINK_SWITCH_CH*n*_INT 已使能，GDMA-AXI 会产生该中断，并将 AXI_DMA_LINK_SWITCH_STATE_CH*n* 清零，表示链表已完成切换。

该功能仅用于状态指示，不影响 GDMA-AXI 的正常传输。

3.4.7 数据传输结束标志

注意：本章中发送链表描述符的 EOF 指描述符第一个字中的 bit 30 所指示的 suc_eof；接收链表描述符的 EOF 可以是 bit 30 指示的 suc_eof，或 bit 28 指示的 err_eof。

GDMA 通过 EOF 指示对应描述符的数据段传输已完成。

发送数据时，产生 EOF 和 TOTAL EOF 两种中断：

- AHB/AXI_DMA_OUT_EOF_CH*n*_INT：链表中的任意一个描述符配置了 suc_eof 标志位时，该描述符对应的数据传输完成后，GDMA 都会产生 EOF 中断。置位 AHB/AXI_DMA_OUT_EOF_CH*n*_INT_ENA 位使能该中断。
- AHB/AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT：如果该链表中的最后一个描述符配置了 suc_eof 标志位，则当该链表的最后一个描述符对应的数据传输完成后，GDMA 会产生 TOTAL EOF 中断。置位 AHB/AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT_ENA 位使能该中断。

接收数据时，可产生以下中断：

- AHB/AXI_DMA_IN_SUC_EOF_CH*n*_INT：表示带有 EOF 标志的数据段接收完成。置位 AHB/AXI_DMA_IN_SUC_EOF_CH*n*_INT_ENA 位使能该中断。
- （仅 UHCI 和 PARLIO）AHB/AXI_DMA_IN_ERR_EOF_CH*n*_INT：表示对应描述符所需传输的数据段接收完成但接收数据有错误。置位 AHB/AXI_DMA_IN_ERR_EOF_CH*n*_INT_ENA 使能该中断。该中断仅在通道连接的外设为 UHCI 或 PARLIO 时有效。

软件在检测到 AHB/AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT 或 AHB/AXI_DMA_IN_SUC_EOF_CH*n*_INT 中断时，可通过读取 AHB/AXI_DMA_OUT_EOF_DES_ADDR_CH*n* 或 AHB/AXI_DMA_IN_SUC_EOF_DES_ADDR_CH*n* 字

段的值，获取当前已完成数据传输的描述符存储地址。这样，软件可以知道哪些描述符已被使用，并根据需要回收描述符。

RX 方向还支持记录 AHB/AXI_DMA_IN_ERR_EOF_CH*n*_INT 中断触发时描述符的存储地址，可通过 AHB/AXI_DMA_IN_ERR_EOF_DES_ADDR_CH*n* 字段获取。

置位 AHB/AXI_DMA_INLINK_AUTO_RET_CH*n* 后，当外设向 GDMA 推送带有 err_eof 标志的数据时，GDMA 会返回到当前帧的起始描述符地址，并重新执行该帧的接收传输。

3.4.8 访问存储器

GDMA-AHB 的 RX/TX 通道可访问内部和外部存储器。可访问的地址空间由寄存器 AHB_DMA_ACCESS_INTR_MEM_START_ADDR ~ AHB_DMA_ACCESS_INTR_MEM_END_ADDR 的配置值决定。

GDMA-AXI 的 RX/TX 通道可访问：

- AXI_DMA_ACCESS_INTR_MEM_START_ADDR 和 AXI_DMA_ACCESS_INTR_MEM_END_ADDR 配置的内部存储器区间；
- AXI_DMA_ACCESS_EXTR_MEM_START_ADDR 和 AXI_DMA_ACCESS_EXTR_MEM_END_ADDR 配置的外部存储器区间。

为加速数据传输，GDMA 支持突发传输模式。GDMA-AHB 默认不使能突发传输模式，可通过置位 AHB_DMA_INDSCR_BURST_EN_CH*n* 使能 RX 通道描述符突发传输模式，置位 AHB_DMA_OUTDSCR_BURST_EN_CH*n* 使能 TX 通道描述符突发传输模式。配置AHB_DMA_IN_DATA_BURST_MODE_SEL_CH*n*选择 RX 通道数据突发传输模式，配置 AHB_DMA_OUT_DATA_BURST_MODE_SEL_CH*n*选择 TX 通道数据突发传输模式。突发传输模式可选如下：

- 0：SINGLE 传输（4 字节）
- 1：INCR4 传输（16 字节）
- 2：INCR8 传输（32 字节）
- 3：INCR16 传输（64 字节）

GDMA-AXI 突发传输模式默认打开且无法关闭。用户可通过 AXI_DMA_IN_BURST_SIZE_SEL_CH*n* 和 AXI_DMA_OUT_BURST_SIZE_SEL_CH*n* 控制 AXI 总线突发传输的数据量，将突发传输长度配置为 8、16、32、64 或 128 字节。

GDMA-AHB 和 GDMA-AXI 访问配置的内部存储器和外部存储器的非加密空间时，链表描述符参数无对齐要求。

访问外部存储器的加密空间（详见章节 *19 片外存储器加密与解密 (XTS_AES) [to be added later]*）时，buffer address pointer 和对应数据需按 16 字节对齐，其他参数无要求。

GDMA-AHB-LP 无法访问外部存储器的加密空间。

3.4.9 仲裁

为了确保及时响应 SPI 等高速低延迟外设的请求，GDMA 引入通道优先级仲裁和通道权重仲裁两种仲裁机制。

- 优先级仲裁方式：GDMA 支持 16（0 ~ 15）个等级的优先级，数字越大优先级越高。若多个通道在同一时刻存在数据传输需求，GDMA 会按照通道优先级从高到低顺序响应各通道的数据传输请求。
- 权重仲裁方式：
  - 每个通道的权重（即令牌数）可配置为 0 ~ 15 之间的任意值。GDMA 按一定的总线 (AHB/AXI) 时钟周期长度划分时间片。在每个时间片内，各通道的数据传输请求次数按所配置的令牌数分配：通道每发起一次数据传输请求即消耗一个令牌。

    若通道在某个时间片内耗尽令牌，则该通道的请求将不再被响应，直到仲裁时间达到时间片规定长度，或所有通道的令牌均已耗尽。此时将刷新令牌数，进入下一个时间片。
  - 若通道令牌数配置为非零值，即使无数据传输请求，也会在每个时间片内等待所有通道的令牌耗尽，不会提前退出当前仲裁时间片。通道无数据传输请求且令牌值非 0 时，会造成一定的带宽浪费。

    为此，GDMA 提供权重仲裁优化开关。启用权重仲裁优化后，若通道无数据传输需求，则该通道的令牌数视为 0，不再参与仲裁。当所有通道均无数据传输请求时，仲裁器提前退出当前时间片，从而提高仲裁响应速度和传输速率。

无论通道优先级怎么配置，描述符传输的优先级一定高于数据传输的优先级。但描述符传输仍然会受到权重配置的限制，当配置的令牌耗尽，只有进入下一个时间片才能继续申请仲裁。

说明：

- 若同时配置优先级仲裁和通道权重仲裁，且优先级不同、权重相同，则在各通道传输相同且较少的数据量时，优先级高的通道优先完成传输。
- 若优先级和权重均不同，且各通道传输的数据量相同且较大，则权重较大的通道能优先分配到更大带宽，从而优先完成传输。

3.4.10 总线错误响应信息记录

在 GDMA-AHB 传输数据期间，如果总线上从机侧回复了错误响应 (Error Response)，数据传输不会中断，GDMA-AHB 仍会按已有链表继续传输数据。这可能导致数据传输完成后，实际得到的数据与预期不符。为便于定位问题，GDMA-AHB 在传输数据过程中若发生错误响应，可通过以下内部寄存器记录相关传输信息：

- AHB_DMA_AHBINF_RESP_ERR_ADDR：记录本次总线传输对应的 32 位地址。
- AHB_DMA_AHBINF_RESP_ERR_CH_ID：记录本次总线传输使用的 GDMA-AHB 通道。
- AHB_DMA_AHBINF_RESP_ERR_ID：记录本次总线上外设与存储器传输或存储器与存储器传输的传输 ID。
- AHB_DMA_AHBINF_RESP_ERR_WR：记录本次总线传输对应的传输类型。

GDMA-AHB 作为主机发起 AHB 总线传输且收到错误响应时，还会产生错误响应中断以提醒 CPU。中断产生后，GDMA-AHB 仍不会中断数据传输。

此外，GDMA-AHB 不会锁存第一笔发生错误响应的 AHB 总线传输相关寄存器信息。若连续发生多笔错误响应传输，上述错误响应记录寄存器的内容会实时更新，最终寄存器中保留的是查询寄存器之前最后一笔发生错误响应的 AHB 总线传输信息。

3.4.11 自动时钟门控

GDMA-AHB 在工作时，某些场景只需要用到部分通道，此时可关闭未使用通道的时钟来降低功耗。同理，内部仲裁和总线接口模块仅在有数据传输时才需要工作，在空闲时也可以通过关闭时钟来降低功耗。目前 GDMA-AHB 内部有以下模块：

- 总线接口处理模块
- 仲裁模块
- RX 数据处理模块
- RX 描述符处理模块
- TX 数据处理模块
- TX 描述符处理模块
- 寄存器配置同步模块

这些模块可在空闲时自动关闭模块时钟，以最大限度降低功耗，也支持通过寄存器配置强制保持时钟开启。例如，置位 AHB_DMA_AHBINF_CLK_EN 可使总线接口处理模块时钟保持常开，其余模块时钟常开配置与其类似。

3.5 事件任务矩阵功能

在 ESP32-S31 中，GDMA 支持 ETM 功能，即可通过任意外设的 ETM 事件触发 GDMA 的 ETM 任务，或通过 GDMA 的 ETM 事件触发任意外设的 ETM 任务。有关 ETM 的更多详细信息，请参考章节 10 事件任务矩阵 (ETM) [to be added later]。本节仅介绍与 GDMA 相关的 ETM 任务和 ETM 事件。

GDMA 可接收的 ETM 任务有：

- GDMA_AHB/AXI_TASK_IN_START_CH*n*：触发时开启 GDMA 的接收通道 *n* 进行数据传输。
- GDMA_AHB/AXI_TASK_OUT_START_CH*n*：触发时开启 GDMA 的发送通道 *n* 进行数据传输。

说明：

以上两个 ETM 任务与 CPU 配置 AHB/AXI_DMA_INLINK_START_CH*n*、AHB/AXI_DMA_OUTLINK_START_CH*n* 实现的功能相同。

AHB/AXI_DMA_IN_ETM_EN_CH*n* 或 AHB/AXI_DMA_OUT_ETM_EN_CH*n* 为 1 时，只有 ETM 可以启动对应方向及对应通道的 GDMA。AHB/AXI_DMA_IN_ETM_EN_CH*n* 或 AHB/AXI_DMA_OUT_ETM_EN_CH*n* 为 0 时，仅可通过 CPU 使能对应的 GDMA。

GDMA 可产生的 ETM 事件有：

- GDMA_AHB/AXI_EVT_IN_DONE_CH*n*：表示通道 *n* 接收方向链表描述符指向的数据接收完成。
- GDMA_AHB/AXI_EVT_IN_SUC_EOF_CH*n*：通道 *n* 接收方向 EOF 位为 1 的链表描述符指向的数据接收完成指示。
- GDMA_AHB/AXI_EVT_IN_FIFO_EMPTY_CH*n*：通道 *n* 接收方向 FIFO 非空到空状态改变指示。
- GDMA_AHB/AXI_EVT_IN_FIFO_FULL_CH*n*：通道 *n* 接收方向 FIFO 非满到满状态改变指示。
- GDMA_AHB/AXI_EVT_OUT_DONE_CH*n*：通道 *n* 发送方向链表描述符指向的数据发送完成。

- GDMA_AHB/AXI_EVT_OUT_SUC_EOF_CH*n*：通道 *n* 发送方向 EOF 位为 1 的链表描述符指向的数据发送完成指示。
- GDMA_AHB/AXI_EVT_OUT_TOTAL_EOF_CH*n*：通道 *n* 发送方向最后一个且 EOF 位为 1 的链表描述符指向的数据发送完成指示。
- GDMA_AHB/AXI_EVT_OUT_FIFO_EMPTY_CH*n*：通道 *n* 发送方向 FIFO 非空到空状态改变指示。
- GDMA_AHB/AXI_EVT_OUT_FIFO_FULL_CH*n*：通道 *n* 发送方向 FIFO 非满到满状态改变指示。

在具体应用中，GDMA 的 ETM 事件可用来触发 GDMA 的 ETM 任务。例如，GDMA_AHB/AXI_EVT_OUT_TOTAL_EOF_CH0 事件可触发 GDMA_AHB/AXI_TASK_IN_START_CH1 任务，从而启动新一轮的 GDMA 操作。

3.6 中断

ESP32-S31 的 GDMA 可以生成以下中断信号并发送给中断矩阵。

- GDMA-AHB-HP:
  - AHB_PDMA_IN/OUT_CH0_INTR
  - AHB_PDMA_IN/OUT_CH1_INTR
  - AHB_PDMA_IN/OUT_CH2_INTR
  - AHB_PDMA_IN/OUT_CH3_INTR
  - AHB_PDMA_IN/OUT_CH4_INTR
- GDMA-AHB-LP:
  - AHB_PDMA_IN/OUT_CH0_INTR
  - AHB_PDMA_IN/OUT_CH1_INTR
- GDMA-AXI:
  - AXI_PDMA_IN/OUT_CH0_INTR
  - AXI_PDMA_IN/OUT_CH1_INTR
  - AXI_PDMA_IN/OUT_CH2_INTR

这些中断信号由 GDMA 的内部中断源生成。

以下中断源生成 AHB/AXI_PDMA_IN_CH*n*_INTR 中断信号：

- AHB/AXI_DMA_IN_DONE_CH*n*_INT：对于接收通道*n*，当一个接收链表描述符对应的数据接收完成时触发此中断。
- AHB/AXI_DMA_IN_SUC_EOF_CH*n*_INT：对于接收通道*n*，当一个接收链表描述符对应的数据接收完成，并且描述符的 suc_eof 为 1 时触发此中断。
- AHB/AXI_DMA_IN_ERR_EOF_CH*n*_INT：对于接收通道*n*，当接收的描述符对应数据段中有错误发生时触发此中断。（该中断只用于外设选择 UHCI 或 PARLIO 时）。
- AHB/AXI_DMA_IN_DSCR_EMPTY_CH*n*_INT：对于接收通道*n*，当接收链表描述符指向的 buffer size 小于待接收数据长度时触发此中断。

- AHB/AXI_DMA_IN_DSCR_ERR_CH*n*_INT：对于接收通道*n*，当接收链表描述符未通过任一描述符检查时触发此中断。
- AHB_DMA_INFIFO_OVF_CH*n*_INT 或 AXI_DMA_INFIFO_L1/L2/L3_OVF_CH*n*_INT：GDMA-AHB 接收数据方向缓存或 GDMA-AXI 接收数据方向 L1、L2、L3 缓存溢出时触发此中断。
- AHB/AXI_DMA_INFIFO_UDF_CH*n*_INT 或 AXI_DMA_INFIFO_L1/L2/L3_UDF_CH*n*_INT：GDMA-AHB 接收数据方向缓存或 GDMA-AXI 接收数据方向 L1、L2、L3 缓存无数据时触发此中断。
- AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT：GDMA-AHB 接收通道*n* 进行数据传输时，若 AHB 总线上发生错误响应，触发此中断。

以下中断源生成 AHB/AXI_PDMA_OUT_CH*n*_INTR 中断信号：

- AHB/AXI_DMA_OUT_DONE_CH*n*_INT：对于发送通道*n*，当一个发送链表描述符对应的数据发送完成时触发此中断。
- AHB/AXI_DMA_OUT_EOF_CH*n*_INT：对于发送通道*n*，当发送链表描述符的 EOF 位为 1 时触发此中断。触发时机取决于 AHB/AXI_DMA_OUT_EOF_MODE_CH*n* 的配置：
  - 为 0 时：该描述符对应的最后一个数据由总线传入 GDMA TX 通道时触发；
  - 为 1 时：该描述符对应的最后一个数据从 GDMA TX 通道发出时触发。
- AHB/AXI_DMA_OUT_DSCR_ERR_CH*n*_INT：对于发送通道*n*，当发送链表描述符未通过任一描述符检查时会触发此中断。
- AHB/AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT：对于发送通道*n*，当链表中最后一个描述符配置了 suc_eof 且该描述符对应的数据传输完成后触发此中断。
- AHB_DMA_OUTFIFO_OVF_CH*n*_INT 或 AXI_DMA_OUTFIFO_L1/L2/L3_OVF_CH*n*_INT：GDMA-AHB 发送数据方向缓存或 GDMA-AXI 发送数据方向 L1、L2、L3 缓存溢出时触发此中断。
- AHB_DMA_OUTFIFO_UDF_CH*n*_INT 或 AXI_DMA_OUTFIFO_L1/L2/L3_UDF_CH*n*_INT：GDMA-AHB 发送数据方向缓存或 GDMA-AXI 发送数据方向 L1、L2、L3 缓存无数据时触发此中断。
- AXI_DMA_OUT_LINK_SWITCH_CH*n*_INT：GDMA-AXI 发送通道 *n* 从一个链表切换到下一个链表时触发此中断。
- AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT：GDMA-AHB 发送通道*n* 进行数据传输时，若 AHB 总线上发生错误响应，触发此中断。

3.7 编程流程

可通过配置 HP_SYS_CLKRST_AHB/AXI_PDMA_SYS_CLK_EN 寄存器管理 GDMA 时钟门控，默认时钟门控打开。可通过配置 HP_SYS_CLKRST_RST_EN_AHB/AXI_PDMA 对 GDMA 模块全局复位。

3.7.1 GDMA TX 通道配置流程

利用 GDMA 发送数据时，GDMA TX 通道的软件配置流程如下：

1. 对寄存器 AHB/AXI_DMA_OUT_RST_CH*n* 置 1 然后置 0，复位 GDMA TX 通道状态机和 FIFO 指针；
2. 挂载好发送链表，配置寄存器 AHB/AXI_DMA_OUTLINK_ADDR_CH*n* 指向第一个发送链表描述符；
3. 配置 AHB/AXI_DMA_PERI_OUT_SEL_CH*n* 为对应的外设号，见表 3.4-1、3.4-2 和 3.4-3；

4. 置位 AHB/AXI_DMA_OUTLINK_START_CH*n* 启动 GDMA TX 通道发送数据；
5. 配置对应的外设，并启动该外设，具体配置请参考对应的外设章节；
6. 等待 AHB/AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT 中断，即数据传输完成。

3.7.2 GDMA RX 通道配置流程

利用 GDMA 接收数据时，GDMA RX 通道的软件配置流程如下：

1. 对寄存器 AHB/AXI_DMA_IN_RST_CH*n* 置 1 然后置 0，复位 GDMA RX 通道状态机和 FIFO 指针；
2. 挂载好接收链表，配置寄存器 AHB/AXI_DMA_INLINK_ADDR_CH*n* 指向第一个接收链表描述符；
3. 配置 AHB/AXI_DMA_PERI_IN_SEL_CH*n* 为对应的外设号，见表 3.4-1、3.4-2 和 3.4-3；
4. 置位 AHB/AXI_DMA_INLINK_START_CH*n* 启动 GDMA RX 通道接收数据；
5. 配置对应的外设，并启动该外设，具体配置请参考对应的外设章节。

3.7.3 GDMA 存储器到存储器配置流程

利用 GDMA 进行存储器到存储器数据传输时，配置流程如下：

1. 对寄存器 AHB/AXI_DMA_OUT_RST_CH*n* 置 1 然后置 0，复位 GDMA TX 通道状态机和 FIFO 指针；
2. 对寄存器 AHB/AXI_DMA_IN_RST_CH*n* 置 1 然后置 0，复位 GDMA RX 通道状态机和 FIFO 指针；
3. 挂载好发送链表，配置寄存器 AHB/AXI_DMA_OUTLINK_ADDR_CH*n* 指向第一个发送链表描述符；
4. 挂载好接收链表，配置寄存器 AHB/AXI_DMA_INLINK_ADDR_CH*n* 指向第一个接收链表描述符；
5. 置位 AHB/AXI_DMA_MEM_TRANS_EN_CH*n* 使能存储器到存储器传输功能；
6. 置位 AHB/AXI_DMA_OUTLINK_START_CH*n* 启动 GDMA TX 通道发送数据；
7. 置位 AHB/AXI_DMA_INLINK_START_CH*n* 启动 GDMA RX 通道接收数据；
8. 如果某个发送链表描述符的 suc_eof 位配置为 1，则该描述符对应的数据段传输结束后会触发 AHB/AXI_DMA_IN_SUC_EOF_CH*n*_INT 中断。

3.7.4 通道优先级和权重配置流程

优先级仲裁配置方法如下：

1. 配置 AHB/AXI_DMA_TX_PRI_CH*n* 和 AHB/AXI_DMA_RX_PRI_CH*n* 为 TX 和 RX 通道的优先级。

权重仲裁配置方法如下：

1. 配置权重仲裁的时间片长度：
   - GDMA-AHB：配置 AHB_DMA_ARB_TIMEOUT，同时适用于 TX 和 RX 方向；
   - GDMA-AXI：TX 和 RX 方向通过 AXI_DMA_ARB_TIMEOUT_TX 和 AXI_DMA_ARB_TIMEOUT_RX 独立配置；
2. 配置 AHB/AXI_DMA_TX_CH_ARB_WEIGH_CH*n* 和 AHB/AXI_DMA_RX_CH_ARB_WEIGH_CH*n* 为各个通道的令牌值；

3. 清除 AHB/AXI_DMA_TX_ARB_WEIGH_OPT_DIR_CH*n* 和 AHB/AXI_DMA_RX_ARB_WEIGH_OPT_DIR_CH*n*，使能 TX 和 RX 方向的权重仲裁优化；
4. 使能权重仲裁：
   - GDMA-AHB：置位同一字段 AHB_DMA_WEIGHT_EN，同时使能 TX 和 RX 方向；
   - GDMA-AXI：分别置位 AXI_DMA_WEIGHT_EN_TX 和 AXI_DMA_WEIGHT_EN_RX，使能 TX 和 RX 方向；

3.8 寄存器列表

3.8.1 GDMA-AHB 寄存器列表

3.8.1.1 GDMA-AHB-HP 寄存器列表

本小节的所有地址均为相对于 GDMA-AHB-HP 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
中断寄存器			
AHB_DMA_IN_INT_RAW_CH0_REG	RX 通道 0 原始中断状态寄存器	0x0000	R/WTC/SS
AHB_DMA_IN_INT_ST_CH0_REG	RX 通道 0 屏蔽中断状态寄存器	0x0004	RO
AHB_DMA_IN_INT_ENA_CH0_REG	RX 通道 0 中断使能寄存器	0x0008	R/W
AHB_DMA_IN_INT_CLR_CH0_REG	RX 通道 0 中断清除寄存器	0x000C	WT
AHB_DMA_IN_INT_RAW_CH1_REG	RX 通道 1 原始中断状态寄存器	0x0010	R/WTC/SS
AHB_DMA_IN_INT_ST_CH1_REG	RX 通道 1 屏蔽中断状态寄存器	0x0014	RO
AHB_DMA_IN_INT_ENA_CH1_REG	RX 通道 1 中断使能寄存器	0x0018	R/W
AHB_DMA_IN_INT_CLR_CH1_REG	RX 通道 1 中断清除寄存器	0x001C	WT
AHB_DMA_IN_INT_RAW_CH2_REG	RX 通道 2 原始中断状态寄存器	0x0020	R/WTC/SS
AHB_DMA_IN_INT_ST_CH2_REG	RX 通道 2 屏蔽中断状态寄存器	0x0024	RO
AHB_DMA_IN_INT_ENA_CH2_REG	RX 通道 2 中断使能寄存器	0x0028	R/W
AHB_DMA_IN_INT_CLR_CH2_REG	RX 通道 2 中断清除寄存器	0x002C	WT
AHB_DMA_IN_INT_RAW_CH3_REG	RX 通道 3 原始中断状态寄存器	0x0030	R/WTC/SS
AHB_DMA_IN_INT_ST_CH3_REG	RX 通道 3 屏蔽中断状态寄存器	0x0034	RO
AHB_DMA_IN_INT_ENA_CH3_REG	RX 通道 3 中断使能寄存器	0x0038	R/W
AHB_DMA_IN_INT_CLR_CH3_REG	RX 通道 3 中断清除寄存器	0x003C	WT
AHB_DMA_IN_INT_RAW_CH4_REG	RX 通道 4 原始中断状态寄存器	0x0040	R/WTC/SS
AHB_DMA_IN_INT_ST_CH4_REG	RX 通道 4 屏蔽中断状态寄存器	0x0044	RO
AHB_DMA_IN_INT_ENA_CH4_REG	RX 通道 4 中断使能寄存器	0x0048	R/W

GoBack

名称	描述	地址	访问
AHB_DMA_IN_INT_CLR_CH4_REG	RX 通道 4 中断清除寄存器	0x004C	WT
AHB_DMA_OUT_INT_RAW_CH0_REG	TX 通道 0 原始中断状态寄存器	0x0050	R/WTC/SS
AHB_DMA_OUT_INT_ST_CH0_REG	TX 通道 0 屏蔽中断状态寄存器	0x0054	RO
AHB_DMA_OUT_INT_ENA_CH0_REG	TX 通道 0 中断使能寄存器	0x0058	R/W
AHB_DMA_OUT_INT_CLR_CH0_REG	TX 通道 0 中断清除寄存器	0x005C	WT
AHB_DMA_OUT_INT_RAW_CH1_REG	TX 通道 1 原始中断状态寄存器	0x0060	R/WTC/SS
AHB_DMA_OUT_INT_ST_CH1_REG	TX 通道 1 屏蔽中断状态寄存器	0x0064	RO
AHB_DMA_OUT_INT_ENA_CH1_REG	TX 通道 1 中断使能寄存器	0x0068	R/W
AHB_DMA_OUT_INT_CLR_CH1_REG	TX 通道 1 中断清除寄存器	0x006C	WT
AHB_DMA_OUT_INT_RAW_CH2_REG	TX 通道 2 原始中断状态寄存器	0x0070	R/WTC/SS
AHB_DMA_OUT_INT_ST_CH2_REG	TX 通道 2 屏蔽中断状态寄存器	0x0074	RO
AHB_DMA_OUT_INT_ENA_CH2_REG	TX 通道 2 中断使能寄存器	0x0078	R/W
AHB_DMA_OUT_INT_CLR_CH2_REG	TX 通道 2 中断清除寄存器	0x007C	WT
AHB_DMA_OUT_INT_RAW_CH3_REG	TX 通道 3 原始中断状态寄存器	0x0080	R/WTC/SS
AHB_DMA_OUT_INT_ST_CH3_REG	TX 通道 3 屏蔽中断状态寄存器	0x0084	RO
AHB_DMA_OUT_INT_ENA_CH3_REG	TX 通道 3 中断使能寄存器	0x0088	R/W
AHB_DMA_OUT_INT_CLR_CH3_REG	TX 通道 3 中断清除寄存器	0x008C	WT
AHB_DMA_OUT_INT_RAW_CH4_REG	TX 通道 4 原始中断状态寄存器	0x0090	R/WTC/SS
AHB_DMA_OUT_INT_ST_CH4_REG	TX 通道 4 屏蔽中断状态寄存器	0x0094	RO
AHB_DMA_OUT_INT_ENA_CH4_REG	TX 通道 4 中断使能寄存器	0x0098	R/W
AHB_DMA_OUT_INT_CLR_CH4_REG	TX 通道 4 中断清除寄存器	0x009C	WT
调试寄存器			
AHB_DMA_AHB_TEST_REG	保留	0x00A0	R/W
配置寄存器			
AHB_DMA_MISC_CONF_REG	杂项配置寄存器	0x00A4	R/W
AHB_DMA_IN_CONF0_CH0_REG	RX 通道 0 配置 0 寄存器	0x0100	R/W
AHB_DMA_IN_CONF1_CH0_REG	RX 通道 0 配置 1 寄存器	0x0104	R/W
AHB_DMA_IN_POP_CH0_REG	RX 通道 0 FIFO 弹出寄存器	0x010C	varies
AHB_DMA_IN_LINK_CH0_REG	RX 通道 0 描述符控制寄存器	0x0110	varies

名称	描述	地址	访问
AHB_DMA_IN_LINK_ADDR_CH0_REG	RX 通道 0 描述符地址寄存器	0x0114	R/W
AHB_DMA_RX_CH_ARB_WEIGH_CH0_REG	RX 通道 0 仲裁权重寄存器	0x013C	R/W
AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH0_REG	RX 通道 0 权重优化寄存器	0x0140	R/W
AHB_DMA_OUT_CONF0_CH0_REG	TX 通道 0 配置 0 寄存器	0x0180	R/W
AHB_DMA_OUT_CONF1_CH0_REG	TX 通道 0 配置 1 寄存器	0x0184	R/W
AHB_DMA_OUT_PUSH_CH0_REG	TX 通道 0 FIFO 推入寄存器	0x018C	varies
AHB_DMA_OUT_LINK_CH0_REG	TX 通道 0 描述符控制寄存器	0x0190	varies
AHB_DMA_OUT_LINK_ADDR_CH0_REG	TX 通道 0 描述符地址寄存器	0x0194	R/W
AHB_DMA_TX_CH_ARB_WEIGH_CH0_REG	TX 通道 0 仲裁权重寄存器	0x01BC	R/W
AHB_DMA_TX_ARB_WEIGH_OPT_DIR_CH0_REG	TX 通道 0 权重优化寄存器	0x01C0	R/W
AHB_DMA_IN_CONF0_CH1_REG	RX 通道 1 配置 0 寄存器	0x0200	R/W
AHB_DMA_IN_CONF1_CH1_REG	RX 通道 1 配置 1 寄存器	0x0204	R/W
AHB_DMA_IN_POP_CH1_REG	RX 通道 1 FIFO 弹出寄存器	0x020C	varies
AHB_DMA_IN_LINK_CH1_REG	RX 通道 1 描述符控制寄存器	0x0210	varies
AHB_DMA_IN_LINK_ADDR_CH1_REG	RX 通道 1 描述符地址寄存器	0x0214	R/W
AHB_DMA_RX_CH_ARB_WEIGH_CH1_REG	RX 通道 1 仲裁权重寄存器	0x023C	R/W
AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH1_REG	RX 通道 1 权重优化寄存器	0x0240	R/W
AHB_DMA_OUT_CONF0_CH1_REG	TX 通道 1 配置 0 寄存器	0x0280	R/W
AHB_DMA_OUT_CONF1_CH1_REG	TX 通道 1 配置 1 寄存器	0x0284	R/W
AHB_DMA_OUT_PUSH_CH1_REG	TX 通道 1 FIFO 推入寄存器	0x028C	varies
AHB_DMA_OUT_LINK_CH1_REG	TX 通道 1 描述符控制寄存器	0x0290	varies
AHB_DMA_OUT_LINK_ADDR_CH1_REG	TX 通道 1 描述符地址寄存器	0x0294	R/W
AHB_DMA_TX_CH_ARB_WEIGH_CH1_REG	TX 通道 1 仲裁权重寄存器	0x02BC	R/W
AHB_DMA_TX_ARB_WEIGH_OPT_DIR_CH1_REG	TX 通道 1 权重优化寄存器	0x02C0	R/W
AHB_DMA_IN_CONF0_CH2_REG	RX 通道 2 配置 0 寄存器	0x0300	R/W
AHB_DMA_IN_CONF1_CH2_REG	RX 通道 2 配置 1 寄存器	0x0304	R/W
AHB_DMA_IN_POP_CH2_REG	RX 通道 2 FIFO 弹出寄存器	0x030C	varies
AHB_DMA_IN_LINK_CH2_REG	RX 通道 2 描述符控制寄存器	0x0310	varies
AHB_DMA_IN_LINK_ADDR_CH2_REG	RX 通道 2 描述符地址寄存器	0x0314	R/W

名称	描述	地址	访问
AHB_DMA_RX_CH_ARB_WEIGH_CH2_REG	RX 通道 2 仲裁权重寄存器	0x033C	R/W
AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH2_REG	RX 通道 2 权重优化寄存器	0x0340	R/W
AHB_DMA_OUT_CONF0_CH2_REG	TX 通道 2 配置 0 寄存器	0x0380	R/W
AHB_DMA_OUT_CONF1_CH2_REG	TX 通道 2 配置 1 寄存器	0x0384	R/W
AHB_DMA_OUT_PUSH_CH2_REG	TX 通道 2 FIFO 推入寄存器	0x038C	varies
AHB_DMA_OUT_LINK_CH2_REG	TX 通道 2 描述符控制寄存器	0x0390	varies
AHB_DMA_OUT_LINK_ADDR_CH2_REG	TX 通道 2 描述符地址寄存器	0x0394	R/W
AHB_DMA_TX_CH_ARB_WEIGH_CH2_REG	TX 通道 2 仲裁权重寄存器	0x03BC	R/W
AHB_DMA_TX_ARB_WEIGH_OPT_DIR_CH2_REG	TX 通道 2 权重优化寄存器	0x03C0	R/W
AHB_DMA_IN_CONF0_CH3_REG	RX 通道 3 配置 0 寄存器	0x0400	R/W
AHB_DMA_IN_CONF1_CH3_REG	RX 通道 3 配置 1 寄存器	0x0404	R/W
AHB_DMA_IN_POP_CH3_REG	RX 通道 3 FIFO 弹出寄存器	0x040C	varies
AHB_DMA_IN_LINK_CH3_REG	RX 通道 3 描述符控制寄存器	0x0410	varies
AHB_DMA_IN_LINK_ADDR_CH3_REG	RX 通道 3 描述符地址寄存器	0x0414	R/W
AHB_DMA_RX_CH_ARB_WEIGH_CH3_REG	RX 通道 3 仲裁权重寄存器	0x043C	R/W
AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH3_REG	RX 通道 3 权重优化寄存器	0x0440	R/W
AHB_DMA_OUT_CONF0_CH3_REG	TX 通道 3 配置 0 寄存器	0x0480	R/W
AHB_DMA_OUT_CONF1_CH3_REG	TX 通道 3 配置 1 寄存器	0x0484	R/W
AHB_DMA_OUT_PUSH_CH3_REG	TX 通道 3 FIFO 推入寄存器	0x048C	varies
AHB_DMA_OUT_LINK_CH3_REG	TX 通道 3 描述符控制寄存器	0x0490	varies
AHB_DMA_OUT_LINK_ADDR_CH3_REG	TX 通道 3 描述符地址寄存器	0x0494	R/W
AHB_DMA_TX_CH_ARB_WEIGH_CH3_REG	TX 通道 3 仲裁权重寄存器	0x04BC	R/W
AHB_DMA_TX_ARB_WEIGH_OPT_DIR_CH3_REG	TX 通道 3 权重优化寄存器	0x04C0	R/W
AHB_DMA_IN_CONF0_CH4_REG	RX 通道 4 配置 0 寄存器	0x0500	R/W
AHB_DMA_IN_CONF1_CH4_REG	RX 通道 4 配置 1 寄存器	0x0504	R/W
AHB_DMA_IN_POP_CH4_REG	RX 通道 4 FIFO 弹出寄存器	0x050C	varies
AHB_DMA_IN_LINK_CH4_REG	RX 通道 4 描述符控制寄存器	0x0510	varies
AHB_DMA_IN_LINK_ADDR_CH4_REG	RX 通道 4 描述符地址寄存器	0x0514	R/W
AHB_DMA_RX_CH_ARB_WEIGH_CH4_REG	RX 通道 4 仲裁权重寄存器	0x053C	R/W

名称	描述	地址	访问
AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH4_REG	RX 通道 4 权重优化寄存器	0x0540	R/W
AHB_DMA_OUT_CONF0_CH4_REG	TX 通道 4 配置 0 寄存器	0x0580	R/W
AHB_DMA_OUT_CONF1_CH4_REG	TX 通道 4 配置 1 寄存器	0x0584	R/W
AHB_DMA_OUT_PUSH_CH4_REG	TX 通道 4 FIFO 推入寄存器	0x058C	varies
AHB_DMA_OUT_LINK_CH4_REG	TX 通道 4 描述符控制寄存器	0x0590	varies
AHB_DMA_OUT_LINK_ADDR_CH4_REG	TX 通道 4 描述符地址寄存器	0x0594	R/W
AHB_DMA_TX_CH_ARB_WEIGH_CH4_REG	TX 通道 4 仲裁权重寄存器	0x05BC	R/W
AHB_DMA_TX_ARB_WEIGH_OPT_DIR_CH4_REG	TX 通道 4 权重优化寄存器	0x05C0	R/W
AHB_DMA_INTR_MEM_START_ADDR_REG	可访问起始地址寄存器	0x0600	R/W
AHB_DMA_INTR_MEM_END_ADDR_REG	可访问结束地址寄存器	0x0604	R/W
AHB_DMA_ARB_TIMEOUT_REG	仲裁超时寄存器	0x0608	R/W
AHB_DMA_WEIGHT_EN_REG	权重仲裁使能寄存器	0x0610	R/W
AHB_DMA_MODULE_CLK_EN_REG	模块时钟强制开启寄存器	0x0618	R/W
版本寄存器			
AHB_DMA_DATE_REG	版本控制寄存器	0x00A8	R/W
状态寄存器			
AHB_DMA_INFIFO_STATUS_CH0_REG	RX 通道 0 FIFO 状态寄存器	0x0108	RO
AHB_DMA_IN_STATE_CH0_REG	RX 通道 0 状态寄存器	0x0118	RO
AHB_DMA_IN_SUC_EOF_DES_ADDR_CH0_REG	RX 通道 0 成功 EOF 地址寄存器	0x011C	RO
AHB_DMA_IN_ERR_EOF_DES_ADDR_CH0_REG	RX 通道 0 错误 EOF 地址寄存器	0x0120	RO
AHB_DMA_IN_DONE_DES_ADDR_CH0_REG	RX 通道 0 已完成描述符地址寄存器	0x0124	RO
AHB_DMA_IN_DSCR_CH0_REG	RX 通道 0 下一个描述符地址寄存器	0x0128	RO
AHB_DMA_IN_DSCR_BF0_CH0_REG	RX 通道 0 当前描述符地址寄存器	0x012C	RO
AHB_DMA_IN_DSCR_BF1_CH0_REG	RX 通道 0 上一个描述符地址寄存器	0x0130	RO
AHB_DMA_OUTFIFO_STATUS_CH0_REG	TX 通道 0 FIFO 状态寄存器	0x0188	RO
AHB_DMA_OUT_STATE_CH0_REG	TX 通道 0 状态寄存器	0x0198	RO
AHB_DMA_OUT_EOF_DES_ADDR_CH0_REG	TX 通道 0 EOF 地址寄存器	0x019C	RO
AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH0_REG	TX 通道 0 上一个 EOF 地址寄存器	0x01A0	RO
AHB_DMA_OUT_DONE_DES_ADDR_CH0_REG	TX 通道 0 已完成描述符地址寄存器	0x01A4	RO

名称	描述	地址	访问
AHB_DMA_OUT_DSCR_CH0_REG	TX 通道 0 下一个描述符地址寄存器	0x01A8	RO
AHB_DMA_OUT_DSCR_BF0_CH0_REG	TX 通道 0 当前描述符地址寄存器	0x01AC	RO
AHB_DMA_OUT_DSCR_BF1_CH0_REG	TX 通道 0 上一个描述符地址寄存器	0x01B0	RO
AHB_DMA_INFIFO_STATUS_CH1_REG	RX 通道 1 FIFO 状态寄存器	0x0208	RO
AHB_DMA_IN_STATE_CH1_REG	RX 通道 1 状态寄存器	0x0218	RO
AHB_DMA_IN_SUC_EOF_DES_ADDR_CH1_REG	RX 通道 1 成功 EOF 地址寄存器	0x021C	RO
AHB_DMA_IN_ERR_EOF_DES_ADDR_CH1_REG	RX 通道 1 错误 EOF 地址寄存器	0x0220	RO
AHB_DMA_IN_DONE_DES_ADDR_CH1_REG	RX 通道 1 已完成描述符地址寄存器	0x0224	RO
AHB_DMA_IN_DSCR_CH1_REG	RX 通道 1 下一个描述符地址寄存器	0x0228	RO
AHB_DMA_IN_DSCR_BF0_CH1_REG	RX 通道 1 当前描述符地址寄存器	0x022C	RO
AHB_DMA_IN_DSCR_BF1_CH1_REG	RX 通道 1 上一个描述符地址寄存器	0x0230	RO
AHB_DMA_OUTFIFO_STATUS_CH1_REG	TX 通道 1 FIFO 状态寄存器	0x0288	RO
AHB_DMA_OUT_STATE_CH1_REG	TX 通道 1 状态寄存器	0x0298	RO
AHB_DMA_OUT_EOF_DES_ADDR_CH1_REG	TX 通道 1 EOF 地址寄存器	0x029C	RO
AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH1_REG	TX 通道 1 上一个 EOF 地址寄存器	0x02A0	RO
AHB_DMA_OUT_DONE_DES_ADDR_CH1_REG	TX 通道 1 已完成描述符地址寄存器	0x02A4	RO
AHB_DMA_OUT_DSCR_CH1_REG	TX 通道 1 下一个描述符地址寄存器	0x02A8	RO
AHB_DMA_OUT_DSCR_BF0_CH1_REG	TX 通道 1 当前描述符地址寄存器	0x02AC	RO
AHB_DMA_OUT_DSCR_BF1_CH1_REG	TX 通道 1 上一个描述符地址寄存器	0x02B0	RO
AHB_DMA_INFIFO_STATUS_CH2_REG	RX 通道 2 FIFO 状态寄存器	0x0308	RO
AHB_DMA_IN_STATE_CH2_REG	RX 通道 2 状态寄存器	0x0318	RO
AHB_DMA_IN_SUC_EOF_DES_ADDR_CH2_REG	RX 通道 2 成功 EOF 地址寄存器	0x031C	RO
AHB_DMA_IN_ERR_EOF_DES_ADDR_CH2_REG	RX 通道 2 错误 EOF 地址寄存器	0x0320	RO
AHB_DMA_IN_DONE_DES_ADDR_CH2_REG	RX 通道 2 已完成描述符地址寄存器	0x0324	RO
AHB_DMA_IN_DSCR_CH2_REG	RX 通道 2 下一个描述符地址寄存器	0x0328	RO
AHB_DMA_IN_DSCR_BF0_CH2_REG	RX 通道 2 当前描述符地址寄存器	0x032C	RO
AHB_DMA_IN_DSCR_BF1_CH2_REG	RX 通道 2 上一个描述符地址寄存器	0x0330	RO
AHB_DMA_OUTFIFO_STATUS_CH2_REG	TX 通道 2 FIFO 状态寄存器	0x0388	RO
AHB_DMA_OUT_STATE_CH2_REG	TX 通道 2 状态寄存器	0x0398	RO

名称	描述	地址	访问
AHB_DMA_OUT_EOF_DES_ADDR_CH2_REG	TX 通道 2 EOF 地址寄存器	0x039C	RO
AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH2_REG	TX 通道 2 上一个 EOF 地址寄存器	0x03A0	RO
AHB_DMA_OUT_DONE_DES_ADDR_CH2_REG	TX 通道 2 已完成描述符地址寄存器	0x03A4	RO
AHB_DMA_OUT_DSCR_CH2_REG	TX 通道 2 下一个描述符地址寄存器	0x03A8	RO
AHB_DMA_OUT_DSCR_BF0_CH2_REG	TX 通道 2 当前描述符地址寄存器	0x03AC	RO
AHB_DMA_OUT_DSCR_BF1_CH2_REG	TX 通道 2 上一个描述符地址寄存器	0x03B0	RO
AHB_DMA_INFIFO_STATUS_CH3_REG	RX 通道 3 FIFO 状态寄存器	0x0408	RO
AHB_DMA_IN_STATE_CH3_REG	RX 通道 3 状态寄存器	0x0418	RO
AHB_DMA_IN_SUC_EOF_DES_ADDR_CH3_REG	RX 通道 3 成功 EOF 地址寄存器	0x041C	RO
AHB_DMA_IN_ERR_EOF_DES_ADDR_CH3_REG	RX 通道 3 错误 EOF 地址寄存器	0x0420	RO
AHB_DMA_IN_DONE_DES_ADDR_CH3_REG	RX 通道 3 已完成描述符地址寄存器	0x0424	RO
AHB_DMA_IN_DSCR_CH3_REG	RX 通道 3 下一个描述符地址寄存器	0x0428	RO
AHB_DMA_IN_DSCR_BF0_CH3_REG	RX 通道 3 当前描述符地址寄存器	0x042C	RO
AHB_DMA_IN_DSCR_BF1_CH3_REG	RX 通道 3 上一个描述符地址寄存器	0x0430	RO
AHB_DMA_OUTFIFO_STATUS_CH3_REG	TX 通道 3 FIFO 状态寄存器	0x0488	RO
AHB_DMA_OUT_STATE_CH3_REG	TX 通道 3 状态寄存器	0x0498	RO
AHB_DMA_OUT_EOF_DES_ADDR_CH3_REG	TX 通道 3 EOF 地址寄存器	0x049C	RO
AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH3_REG	TX 通道 3 上一个 EOF 地址寄存器	0x04A0	RO
AHB_DMA_OUT_DONE_DES_ADDR_CH3_REG	TX 通道 3 已完成描述符地址寄存器	0x04A4	RO
AHB_DMA_OUT_DSCR_CH3_REG	TX 通道 3 下一个描述符地址寄存器	0x04A8	RO
AHB_DMA_OUT_DSCR_BF0_CH3_REG	TX 通道 3 当前描述符地址寄存器	0x04AC	RO
AHB_DMA_OUT_DSCR_BF1_CH3_REG	TX 通道 3 上一个描述符地址寄存器	0x04B0	RO
AHB_DMA_INFIFO_STATUS_CH4_REG	RX 通道 4 FIFO 状态寄存器	0x0508	RO
AHB_DMA_IN_STATE_CH4_REG	RX 通道 4 状态寄存器	0x0518	RO
AHB_DMA_IN_SUC_EOF_DES_ADDR_CH4_REG	RX 通道 4 成功 EOF 地址寄存器	0x051C	RO
AHB_DMA_IN_ERR_EOF_DES_ADDR_CH4_REG	RX 通道 4 错误 EOF 地址寄存器	0x0520	RO
AHB_DMA_IN_DONE_DES_ADDR_CH4_REG	RX 通道 4 已完成描述符地址寄存器	0x0524	RO
AHB_DMA_IN_DSCR_CH4_REG	RX 通道 4 下一个描述符地址寄存器	0x0528	RO
AHB_DMA_IN_DSCR_BF0_CH4_REG	RX 通道 4 当前描述符地址寄存器	0x052C	RO

名称	描述	地址	访问
AHB_DMA_IN_DSCR_BF1_CH4_REG	RX 通道 4 上一个描述符地址寄存器	0x0530	RO
AHB_DMA_OUTFIFO_STATUS_CH4_REG	TX 通道 4 FIFO 状态寄存器	0x0588	RO
AHB_DMA_OUT_STATE_CH4_REG	TX 通道 4 状态寄存器	0x0598	RO
AHB_DMA_OUT_EOF_DES_ADDR_CH4_REG	TX 通道 4 EOF 地址寄存器	0x059C	RO
AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH4_REG	TX 通道 4 上一个 EOF 地址寄存器	0x05A0	RO
AHB_DMA_OUT_DONE_DES_ADDR_CH4_REG	TX 通道 4 已完成描述符地址寄存器	0x05A4	RO
AHB_DMA_OUT_DSCR_CH4_REG	TX 通道 4 下一个描述符地址寄存器	0x05A8	RO
AHB_DMA_OUT_DSCR_BF0_CH4_REG	TX 通道 4 当前描述符地址寄存器	0x05AC	RO
AHB_DMA_OUT_DSCR_BF1_CH4_REG	TX 通道 4 上一个描述符地址寄存器	0x05B0	RO
AHB_DMA_AHBINF_RESP_ERR_STATUS0_REG	AHB 响应错误状态寄存器 0	0x0620	RO
AHB_DMA_AHBINF_RESP_ERR_STATUS1_REG	AHB 响应错误状态寄存器 1	0x0624	RO
优先级寄存器			
AHB_DMA_IN_PRI_CH0_REG	RX 通道 0 优先级寄存器	0x0134	R/W
AHB_DMA_OUT_PRI_CH0_REG	TX 通道 0 优先级寄存器	0x01B4	R/W
AHB_DMA_IN_PRI_CH1_REG	RX 通道 1 优先级寄存器	0x0234	R/W
AHB_DMA_OUT_PRI_CH1_REG	TX 通道 1 优先级寄存器	0x02B4	R/W
AHB_DMA_IN_PRI_CH2_REG	RX 通道 2 优先级寄存器	0x0334	R/W
AHB_DMA_OUT_PRI_CH2_REG	TX 通道 2 优先级寄存器	0x03B4	R/W
AHB_DMA_IN_PRI_CH3_REG	RX 通道 3 优先级寄存器	0x0434	R/W
AHB_DMA_OUT_PRI_CH3_REG	TX 通道 3 优先级寄存器	0x04B4	R/W
AHB_DMA_IN_PRI_CH4_REG	RX 通道 4 优先级寄存器	0x0534	R/W
AHB_DMA_OUT_PRI_CH4_REG	TX 通道 4 优先级寄存器	0x05B4	R/W
外设选择寄存器			
AHB_DMA_IN_PERI_SEL_CH0_REG	RX 通道 0 外设选择寄存器	0x0138	R/W
AHB_DMA_OUT_PERI_SEL_CH0_REG	TX 通道 0 外设选择寄存器	0x01B8	R/W
AHB_DMA_IN_PERI_SEL_CH1_REG	RX 通道 1 外设选择寄存器	0x0238	R/W
AHB_DMA_OUT_PERI_SEL_CH1_REG	TX 通道 1 外设选择寄存器	0x02B8	R/W
AHB_DMA_IN_PERI_SEL_CH2_REG	RX 通道 2 外设选择寄存器	0x0338	R/W
AHB_DMA_OUT_PERI_SEL_CH2_REG	TX 通道 2 外设选择寄存器	0x03B8	R/W

名称	描述	地址	访问
AHB_DMA_IN_PERI_SEL_CH3_REG	RX 通道 3 外设选择寄存器	0x0438	R/W
AHB_DMA_OUT_PERI_SEL_CH3_REG	TX 通道 3 外设选择寄存器	0x04B8	R/W
AHB_DMA_IN_PERI_SEL_CH4_REG	RX 通道 4 外设选择寄存器	0x0538	R/W
AHB_DMA_OUT_PERI_SEL_CH4_REG	TX 通道 4 外设选择寄存器	0x05B8	R/W

3.8.1.2 GDMA-AHB-LP 寄存器列表

本小节的所有地址均为相对于 GDMA-AHB-LP 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
中断寄存器			
LP_AHB_DMA_IN_INT_RAW_CH0_REG	RX 通道 0 原始中断状态寄存器	0x0000	R/WTC/SS
LP_AHB_DMA_IN_INT_ST_CH0_REG	RX 通道 0 屏蔽中断状态寄存器	0x0004	RO
LP_AHB_DMA_IN_INT_ENA_CH0_REG	RX 通道 0 中断使能寄存器	0x0008	R/W
LP_AHB_DMA_IN_INT_CLR_CH0_REG	RX 通道 0 中断清除寄存器	0x000C	WT
LP_AHB_DMA_IN_INT_RAW_CH1_REG	RX 通道 1 原始中断状态寄存器	0x0010	R/WTC/SS
LP_AHB_DMA_IN_INT_ST_CH1_REG	RX 通道 1 屏蔽中断状态寄存器	0x0014	RO
LP_AHB_DMA_IN_INT_ENA_CH1_REG	RX 通道 1 中断使能寄存器	0x0018	R/W
LP_AHB_DMA_IN_INT_CLR_CH1_REG	RX 通道 1 中断清除寄存器	0x001C	WT
LP_AHB_DMA_OUT_INT_RAW_CH0_REG	TX 通道 0 原始中断状态寄存器	0x0050	R/WTC/SS
LP_AHB_DMA_OUT_INT_ST_CH0_REG	TX 通道 0 屏蔽中断状态寄存器	0x0054	RO
LP_AHB_DMA_OUT_INT_ENA_CH0_REG	TX 通道 0 中断使能寄存器	0x0058	R/W
LP_AHB_DMA_OUT_INT_CLR_CH0_REG	TX 通道 0 中断清除寄存器	0x005C	WT
LP_AHB_DMA_OUT_INT_RAW_CH1_REG	TX 通道 1 原始中断状态寄存器	0x0060	R/WTC/SS
LP_AHB_DMA_OUT_INT_ST_CH1_REG	TX 通道 1 屏蔽中断状态寄存器	0x0064	RO
LP_AHB_DMA_OUT_INT_ENA_CH1_REG	TX 通道 1 中断使能寄存器	0x0068	R/W
LP_AHB_DMA_OUT_INT_CLR_CH1_REG	TX 通道 1 中断清除寄存器	0x006C	WT
调试寄存器			

GoBack

名称	描述	地址	访问
LP_AHB_DMA_AHB_TEST_REG	保留	0x00A0	R/W
配置寄存器			
LP_AHB_DMA_MISC_CONF_REG	杂项配置寄存器	0x00A4	R/W
LP_AHB_DMA_IN_CONF0_CH0_REG	RX 通道 0 配置 0 寄存器	0x0100	R/W
LP_AHB_DMA_IN_CONF1_CH0_REG	RX 通道 0 配置 1 寄存器	0x0104	R/W
LP_AHB_DMA_IN_POP_CH0_REG	RX 通道 0 FIFO 弹出寄存器	0x010C	varies
LP_AHB_DMA_IN_LINK_CH0_REG	RX 通道 0 描述符控制寄存器	0x0110	varies
LP_AHB_DMA_IN_LINK_ADDR_CH0_REG	RX 通道 0 描述符地址寄存器	0x0114	R/W
LP_AHB_DMA_RX_CH_ARB_WEIGH_CH0_REG	RX 通道 0 仲裁权重寄存器	0x013C	R/W
LP_AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH0_REG	RX 通道 0 权重优化寄存器	0x0140	R/W
LP_AHB_DMA_OUT_CONF0_CH0_REG	TX 通道 0 配置 0 寄存器	0x0180	R/W
LP_AHB_DMA_OUT_CONF1_CH0_REG	TX 通道 0 配置 1 寄存器	0x0184	R/W
LP_AHB_DMA_OUT_PUSH_CH0_REG	TX 通道 0 FIFO 推入寄存器	0x018C	varies
LP_AHB_DMA_OUT_LINK_CH0_REG	TX 通道 0 描述符控制寄存器	0x0190	varies
LP_AHB_DMA_OUT_LINK_ADDR_CH0_REG	TX 通道 0 描述符地址寄存器	0x0194	R/W
LP_AHB_DMA_TX_CH_ARB_WEIGH_CH0_REG	TX 通道 0 仲裁权重寄存器	0x01BC	R/W
LP_AHB_DMA_TX_ARB_WEIGH_OPT_DIR_CH0_REG	TX 通道 0 权重优化寄存器	0x01C0	R/W
LP_AHB_DMA_IN_CONF0_CH1_REG	RX 通道 1 配置 0 寄存器	0x0200	R/W
LP_AHB_DMA_IN_CONF1_CH1_REG	RX 通道 1 配置 1 寄存器	0x0204	R/W
LP_AHB_DMA_IN_POP_CH1_REG	RX 通道 1 FIFO 弹出寄存器	0x020C	varies
LP_AHB_DMA_IN_LINK_CH1_REG	RX 通道 1 描述符控制寄存器	0x0210	varies
LP_AHB_DMA_IN_LINK_ADDR_CH1_REG	RX 通道 1 描述符地址寄存器	0x0214	R/W
LP_AHB_DMA_RX_CH_ARB_WEIGH_CH1_REG	RX 通道 1 仲裁权重寄存器	0x023C	R/W
LP_AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH1_REG	RX 通道 1 权重优化寄存器	0x0240	R/W
LP_AHB_DMA_OUT_CONF0_CH1_REG	TX 通道 1 配置 0 寄存器	0x0280	R/W
LP_AHB_DMA_OUT_CONF1_CH1_REG	TX 通道 1 配置 1 寄存器	0x0284	R/W
LP_AHB_DMA_OUT_PUSH_CH1_REG	TX 通道 1 FIFO 推入寄存器	0x028C	varies
LP_AHB_DMA_OUT_LINK_CH1_REG	TX 通道 1 描述符控制寄存器	0x0290	varies
LP_AHB_DMA_OUT_LINK_ADDR_CH1_REG	TX 通道 1 描述符地址寄存器	0x0294	R/W

名称	描述	地址	访问
LP_AHB_DMA_TX_CH_ARB_WEIGH_CH1_REG	TX 通道 1 仲裁权重寄存器	0x02BC	R/W
LP_AHB_DMA_TX_ARB_WEIGH_OPT_DIR_CH1_REG	TX 通道 1 权重优化寄存器	0x02C0	R/W
LP_AHB_DMA_INTR_MEM_START_ADDR_REG	可访问起始地址寄存器	0x0600	R/W
LP_AHB_DMA_INTR_MEM_END_ADDR_REG	可访问结束地址寄存器	0x0604	R/W
LP_AHB_DMA_ARB_TIMEOUT_REG	仲裁超时寄存器	0x0608	R/W
LP_AHB_DMA_WEIGHT_EN_REG	权重仲裁使能寄存器	0x0610	R/W
LP_AHB_DMA_MODULE_CLK_EN_REG	模块时钟强制开启寄存器	0x0618	R/W
版本寄存器			
LP_AHB_DMA_DATE_REG	版本控制寄存器	0x00A8	R/W
状态寄存器			
LP_AHB_DMA_INFIFO_STATUS_CH0_REG	RX 通道 0 FIFO 状态寄存器	0x0108	RO
LP_AHB_DMA_IN_STATE_CH0_REG	RX 通道 0 状态寄存器	0x0118	RO
LP_AHB_DMA_IN_SUC_EOF_DES_ADDR_CH0_REG	RX 通道 0 成功 EOF 地址寄存器	0x011C	RO
LP_AHB_DMA_IN_ERR_EOF_DES_ADDR_CH0_REG	RX 通道 0 错误 EOF 地址寄存器	0x0120	RO
LP_AHB_DMA_IN_DONE_DES_ADDR_CH0_REG	RX 通道 0 已完成描述符地址寄存器	0x0124	RO
LP_AHB_DMA_IN_DSCR_CH0_REG	RX 通道 0 下一个描述符地址寄存器	0x0128	RO
LP_AHB_DMA_IN_DSCR_BF0_CH0_REG	RX 通道 0 当前描述符地址寄存器	0x012C	RO
LP_AHB_DMA_IN_DSCR_BF1_CH0_REG	RX 通道 0 上一个描述符地址寄存器	0x0130	RO
LP_AHB_DMA_OUTFIFO_STATUS_CH0_REG	TX 通道 0 FIFO 状态寄存器	0x0188	RO
LP_AHB_DMA_OUT_STATE_CH0_REG	TX 通道 0 状态寄存器	0x0198	RO
LP_AHB_DMA_OUT_EOF_DES_ADDR_CH0_REG	TX 通道 0 EOF 地址寄存器	0x019C	RO
LP_AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH0_REG	TX 通道 0 上一个 EOF 地址寄存器	0x01A0	RO
LP_AHB_DMA_OUT_DONE_DES_ADDR_CH0_REG	TX 通道 0 已完成描述符地址寄存器	0x01A4	RO
LP_AHB_DMA_OUT_DSCR_CH0_REG	TX 通道 0 下一个描述符地址寄存器	0x01A8	RO
LP_AHB_DMA_OUT_DSCR_BF0_CH0_REG	TX 通道 0 当前描述符地址寄存器	0x01AC	RO
LP_AHB_DMA_OUT_DSCR_BF1_CH0_REG	TX 通道 0 上一个描述符地址寄存器	0x01B0	RO
LP_AHB_DMA_INFIFO_STATUS_CH1_REG	RX 通道 1 FIFO 状态寄存器	0x0208	RO
LP_AHB_DMA_IN_STATE_CH1_REG	RX 通道 1 状态寄存器	0x0218	RO
LP_AHB_DMA_IN_SUC_EOF_DES_ADDR_CH1_REG	RX 通道 1 成功 EOF 地址寄存器	0x021C	RO

名称	描述	地址	访问
LP_AHB_DMA_IN_ERR_EOF_DES_ADDR_CH1_REG	RX 通道 1 错误 EOF 地址寄存器	0x0220	RO
LP_AHB_DMA_IN_DONE_DES_ADDR_CH1_REG	RX 通道 1 已完成描述符地址寄存器	0x0224	RO
LP_AHB_DMA_IN_DSCR_CH1_REG	RX 通道 1 下一个描述符地址寄存器	0x0228	RO
LP_AHB_DMA_IN_DSCR_BF0_CH1_REG	RX 通道 1 当前描述符地址寄存器	0x022C	RO
LP_AHB_DMA_IN_DSCR_BF1_CH1_REG	RX 通道 1 上一个描述符地址寄存器	0x0230	RO
LP_AHB_DMA_OUTFIFO_STATUS_CH1_REG	TX 通道 1 FIFO 状态寄存器	0x0288	RO
LP_AHB_DMA_OUT_STATE_CH1_REG	TX 通道 1 状态寄存器	0x0298	RO
LP_AHB_DMA_OUT_EOF_DES_ADDR_CH1_REG	TX 通道 1 EOF 地址寄存器	0x029C	RO
LP_AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH1_REG	TX 通道 1 上一个 EOF 地址寄存器	0x02A0	RO
LP_AHB_DMA_OUT_DONE_DES_ADDR_CH1_REG	TX 通道 1 已完成描述符地址寄存器	0x02A4	RO
LP_AHB_DMA_OUT_DSCR_CH1_REG	TX 通道 1 下一个描述符地址寄存器	0x02A8	RO
LP_AHB_DMA_OUT_DSCR_BF0_CH1_REG	TX 通道 1 当前描述符地址寄存器	0x02AC	RO
LP_AHB_DMA_OUT_DSCR_BF1_CH1_REG	TX 通道 1 上一个描述符地址寄存器	0x02B0	RO
LP_AHB_DMA_AHBINF_RESP_ERR_STATUS0_REG	AHB 响应错误状态寄存器 0	0x0620	RO
LP_AHB_DMA_AHBINF_RESP_ERR_STATUS1_REG	AHB 响应错误状态寄存器 1	0x0624	RO
优先级寄存器			
LP_AHB_DMA_IN_PRI_CH0_REG	RX 通道 0 优先级寄存器	0x0134	R/W
LP_AHB_DMA_OUT_PRI_CH0_REG	TX 通道 0 优先级寄存器	0x01B4	R/W
LP_AHB_DMA_IN_PRI_CH1_REG	RX 通道 1 优先级寄存器	0x0234	R/W
LP_AHB_DMA_OUT_PRI_CH1_REG	TX 通道 1 优先级寄存器	0x02B4	R/W
外设选择寄存器			
LP_AHB_DMA_IN_PERI_SEL_CH0_REG	RX 通道 0 外设选择寄存器	0x0138	R/W
LP_AHB_DMA_OUT_PERI_SEL_CH0_REG	TX 通道 0 外设选择寄存器	0x01B8	R/W
LP_AHB_DMA_IN_PERI_SEL_CH1_REG	RX 通道 1 外设选择寄存器	0x0238	R/W
LP_AHB_DMA_OUT_PERI_SEL_CH1_REG	TX 通道 1 外设选择寄存器	0x02B8	R/W

3.8.2 GDMA-AXI 寄存器列表

本小节的所有地址均为相对于 GDMA-AXI 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 寄存器的访问类型，了解“访问”列缩写的含义。

名称	描述	地址	访问
中断寄存器			
AXI_DMA_IN_INT_RAW_CH0_REG	RX 通道 0 原始中断状态寄存器	0x0000	R/WTC/SS
AXI_DMA_IN_INT_ST_CH0_REG	RX 通道 0 屏蔽中断状态寄存器	0x0004	RO
AXI_DMA_IN_INT_ENA_CH0_REG	RX 通道 0 中断使能寄存器	0x0008	R/W
AXI_DMA_IN_INT_CLR_CH0_REG	RX 通道 0 中断清除寄存器	0x000C	WT
AXI_DMA_IN_INT_RAW_CH1_REG	RX 通道 1 原始中断状态寄存器	0x0068	R/WTC/SS
AXI_DMA_IN_INT_ST_CH1_REG	RX 通道 1 屏蔽中断状态寄存器	0x006C	RO
AXI_DMA_IN_INT_ENA_CH1_REG	RX 通道 1 中断使能寄存器	0x0070	R/W
AXI_DMA_IN_INT_CLR_CH1_REG	RX 通道 1 中断清除寄存器	0x0074	WT
AXI_DMA_IN_INT_RAW_CH2_REG	RX 通道 2 原始中断状态寄存器	0x00D0	R/WTC/SS
AXI_DMA_IN_INT_ST_CH2_REG	RX 通道 2 屏蔽中断状态寄存器	0x00D4	RO
AXI_DMA_IN_INT_ENA_CH2_REG	RX 通道 2 中断使能寄存器	0x00D8	R/W
AXI_DMA_IN_INT_CLR_CH2_REG	RX 通道 2 中断清除寄存器	0x00DC	WT
AXI_DMA_OUT_INT_RAW_CH0_REG	TX 通道 0 原始中断状态寄存器	0x0138	R/WTC/SS
AXI_DMA_OUT_INT_ST_CH0_REG	TX 通道 0 屏蔽中断状态寄存器	0x013C	RO
AXI_DMA_OUT_INT_ENA_CH0_REG	TX 通道 0 中断使能寄存器	0x0140	R/W
AXI_DMA_OUT_INT_CLR_CH0_REG	TX 通道 0 中断清除寄存器	0x0144	WT
AXI_DMA_OUT_INT_RAW_CH1_REG	TX 通道 1 原始中断状态寄存器	0x01A0	R/WTC/SS
AXI_DMA_OUT_INT_ST_CH1_REG	TX 通道 1 屏蔽中断状态寄存器	0x01A4	RO
AXI_DMA_OUT_INT_ENA_CH1_REG	TX 通道 1 中断使能寄存器	0x01A8	R/W
AXI_DMA_OUT_INT_CLR_CH1_REG	TX 通道 1 中断清除寄存器	0x01AC	WT
AXI_DMA_OUT_INT_RAW_CH2_REG	TX 通道 2 原始中断状态寄存器	0x0208	R/WTC/SS
AXI_DMA_OUT_INT_ST_CH2_REG	TX 通道 2 屏蔽中断状态寄存器	0x020C	RO
AXI_DMA_OUT_INT_ENA_CH2_REG	TX 通道 2 中断使能寄存器	0x0210	R/W
AXI_DMA_OUT_INT_CLR_CH2_REG	TX 通道 2 中断清除寄存器	0x0214	WT
配置寄存器			
AXI_DMA_IN_CONF0_CH0_REG	RX 通道 0 配置 0 寄存器	0x0010	R/W

名称	描述	地址	访问
AXI_DMA_IN_CONF1_CH0_REG	RX 通道 0 配置 1 寄存器	0x0014	R/W
AXI_DMA_IN_POP_CH0_REG	RX 通道 0 FIFO 弹出寄存器	0x001C	varies
AXI_DMA_IN_LINK1_CH0_REG	RX 通道 0 描述符控制寄存器	0x0020	varies
AXI_DMA_IN_LINK2_CH0_REG	RX 通道 0 描述符地址寄存器	0x0024	R/W
AXI_DMA_IN_CONF0_CH1_REG	RX 通道 1 配置 0 寄存器	0x0078	R/W
AXI_DMA_IN_CONF1_CH1_REG	RX 通道 1 配置 1 寄存器	0x007C	R/W
AXI_DMA_IN_POP_CH1_REG	RX 通道 1 FIFO 弹出寄存器	0x0084	varies
AXI_DMA_IN_LINK1_CH1_REG	RX 通道 1 描述符控制寄存器	0x0088	varies
AXI_DMA_IN_LINK2_CH1_REG	RX 通道 1 描述符地址寄存器	0x008C	R/W
AXI_DMA_IN_CONF0_CH2_REG	RX 通道 2 配置 0 寄存器	0x00E0	R/W
AXI_DMA_IN_CONF1_CH2_REG	RX 通道 2 配置 1 寄存器	0x00E4	R/W
AXI_DMA_IN_POP_CH2_REG	RX 通道 2 FIFO 弹出寄存器	0x00EC	varies
AXI_DMA_IN_LINK1_CH2_REG	RX 通道 2 描述符控制寄存器	0x00F0	varies
AXI_DMA_IN_LINK2_CH2_REG	RX 通道 2 描述符地址寄存器	0x00F4	R/W
AXI_DMA_OUT_CONF0_CH0_REG	TX 通道 0 配置 0 寄存器	0x0148	R/W
AXI_DMA_OUT_CONF1_CH0_REG	TX 通道 0 配置 1 寄存器	0x014C	R/W
AXI_DMA_OUT_PUSH_CH0_REG	TX 通道 0 FIFO 推入寄存器	0x0154	varies
AXI_DMA_OUT_LINK1_CH0_REG	TX 通道 0 描述符控制寄存器	0x0158	varies
AXI_DMA_OUT_LINK2_CH0_REG	TX 通道 0 描述符地址寄存器	0x015C	R/W
AXI_DMA_OUT_CONF0_CH1_REG	TX 通道 1 配置 0 寄存器	0x01B0	R/W
AXI_DMA_OUT_CONF1_CH1_REG	TX 通道 1 配置 1 寄存器	0x01B4	R/W
AXI_DMA_OUT_PUSH_CH1_REG	TX 通道 1 FIFO 推入寄存器	0x01BC	varies
AXI_DMA_OUT_LINK1_CH1_REG	TX 通道 1 描述符控制寄存器	0x01C0	varies
AXI_DMA_OUT_LINK2_CH1_REG	TX 通道 1 描述符地址寄存器	0x01C4	R/W
AXI_DMA_OUT_CONF0_CH2_REG	TX 通道 2 配置 0 寄存器	0x0218	R/W
AXI_DMA_OUT_CONF1_CH2_REG	TX 通道 2 配置 1 寄存器	0x021C	R/W
AXI_DMA_OUT_PUSH_CH2_REG	TX 通道 2 FIFO 推入寄存器	0x0224	varies
AXI_DMA_OUT_LINK1_CH2_REG	TX 通道 2 描述符控制寄存器	0x0228	varies
AXI_DMA_OUT_LINK2_CH2_REG	TX 通道 2 描述符地址寄存器	0x022C	R/W

名称	描述	地址	访问
AXI_DMA_ARB_TIMEOUT_REG	仲裁超时寄存器	0x0270	R/W
AXI_DMA_WEIGHT_EN_REG	权重仲裁使能寄存器	0x0274	R/W
AXI_DMA_IN_MEM_CONF_REG	存储器配置寄存器	0x0278	R/W
AXI_DMA_INTR_MEM_START_ADDR_REG	可访问起始地址寄存器	0x027C	R/W
AXI_DMA_INTR_MEM_END_ADDR_REG	可访问结束地址寄存器	0x0280	R/W
AXI_DMA_EXTR_MEM_START_ADDR_REG	可访问起始地址寄存器	0x0284	R/W
AXI_DMA_EXTR_MEM_END_ADDR_REG	可访问结束地址寄存器	0x0288	R/W
AXI_DMA_IN_RESET_AVAIL_CH0_REG	RX 通道 0 复位状态寄存器	0x028C	RO
AXI_DMA_IN_RESET_AVAIL_CH1_REG	RX 通道 1 复位状态寄存器	0x0290	RO
AXI_DMA_IN_RESET_AVAIL_CH2_REG	RX 通道 2 复位状态寄存器	0x0294	RO
AXI_DMA_OUT_RESET_AVAIL_CH0_REG	TX 通道 0 复位状态寄存器	0x0298	RO
AXI_DMA_OUT_RESET_AVAIL_CH1_REG	TX 通道 1 复位状态寄存器	0x029C	RO
AXI_DMA_OUT_RESET_AVAIL_CH2_REG	TX 通道 2 复位状态寄存器	0x02A0	RO
AXI_DMA_MISC_CONF_REG	杂项配置寄存器	0x02A8	R/W
状态寄存器			
AXI_DMA_INFIFO_STATUS_CH0_REG	RX 通道 0 FIFO 状态寄存器	0x0018	RO
AXI_DMA_IN_STATE_CH0_REG	RX 通道 0 状态寄存器	0x0028	RO
AXI_DMA_IN_SUC_EOF_DES_ADDR_CH0_REG	RX 通道 0 成功 EOF 地址寄存器	0x002C	RO
AXI_DMA_IN_ERR_EOF_DES_ADDR_CH0_REG	RX 通道 0 错误 EOF 地址寄存器	0x0030	RO
AXI_DMA_IN_DSCR_CH0_REG	RX 通道 0 下一个描述符地址寄存器	0x0034	RO
AXI_DMA_IN_DSCR_BF0_CH0_REG	RX 通道 0 当前描述符地址寄存器	0x0038	RO
AXI_DMA_IN_DSCR_BF1_CH0_REG	RX 通道 0 上一个描述符地址寄存器	0x003C	RO
AXI_DMA_INFIFO_STATUS_CH1_REG	RX 通道 1 FIFO 状态寄存器	0x0080	RO
AXI_DMA_IN_STATE_CH1_REG	RX 通道 1 状态寄存器	0x0090	RO
AXI_DMA_IN_SUC_EOF_DES_ADDR_CH1_REG	RX 通道 1 成功 EOF 地址寄存器	0x0094	RO
AXI_DMA_IN_ERR_EOF_DES_ADDR_CH1_REG	RX 通道 1 错误 EOF 地址寄存器	0x0098	RO
AXI_DMA_IN_DSCR_CH1_REG	RX 通道 1 下一个描述符地址寄存器	0x009C	RO
AXI_DMA_IN_DSCR_BF0_CH1_REG	RX 通道 1 当前描述符地址寄存器	0x00A0	RO
AXI_DMA_IN_DSCR_BF1_CH1_REG	RX 通道 1 上一个描述符地址寄存器	0x00A4	RO

GoBack

名称	描述	地址	访问
AXI_DMA_INFIFO_STATUS_CH2_REG	RX 通道 2 FIFO 状态寄存器	0x00E8	RO
AXI_DMA_IN_STATE_CH2_REG	RX 通道 2 状态寄存器	0x00F8	RO
AXI_DMA_IN_SUC_EOF_DES_ADDR_CH2_REG	RX 通道 2 成功 EOF 地址寄存器	0x00FC	RO
AXI_DMA_IN_ERR_EOF_DES_ADDR_CH2_REG	RX 通道 2 错误 EOF 地址寄存器	0x0100	RO
AXI_DMA_IN_DSCR_CH2_REG	RX 通道 2 下一个描述符地址寄存器	0x0104	RO
AXI_DMA_IN_DSCR_BF0_CH2_REG	RX 通道 2 当前描述符地址寄存器	0x0108	RO
AXI_DMA_IN_DSCR_BF1_CH2_REG	RX 通道 2 上一个描述符地址寄存器	0x010C	RO
AXI_DMA_OUTFIFO_STATUS_CH0_REG	TX 通道 0 FIFO 状态寄存器	0x0150	RO
AXI_DMA_OUT_STATE_CH0_REG	TX 通道 0 状态寄存器	0x0160	RO
AXI_DMA_OUT_EOF_DES_ADDR_CH0_REG	TX 通道 0 EOF 地址寄存器	0x0164	RO
AXI_DMA_OUT_EOF_BFR_DES_ADDR_CH0_REG	TX 通道 0 上一个 EOF 地址寄存器	0x0168	RO
AXI_DMA_OUT_DSCR_CH0_REG	TX 通道 0 下一个描述符地址寄存器	0x016C	RO
AXI_DMA_OUT_DSCR_BF0_CH0_REG	TX 通道 0 当前描述符地址寄存器	0x0170	RO
AXI_DMA_OUT_DSCR_BF1_CH0_REG	TX 通道 0 上一个描述符地址寄存器	0x0174	RO
AXI_DMA_OUTFIFO_STATUS_CH1_REG	TX 通道 1 FIFO 状态寄存器	0x01B8	RO
AXI_DMA_OUT_STATE_CH1_REG	TX 通道 1 状态寄存器	0x01C8	RO
AXI_DMA_OUT_EOF_DES_ADDR_CH1_REG	TX 通道 1 EOF 地址寄存器	0x01CC	RO
AXI_DMA_OUT_EOF_BFR_DES_ADDR_CH1_REG	TX 通道 1 上一个 EOF 地址寄存器	0x01D0	RO
AXI_DMA_OUT_DSCR_CH1_REG	TX 通道 1 下一个描述符地址寄存器	0x01D4	RO
AXI_DMA_OUT_DSCR_BF0_CH1_REG	TX 通道 1 当前描述符地址寄存器	0x01D8	RO
AXI_DMA_OUT_DSCR_BF1_CH1_REG	TX 通道 1 上一个描述符地址寄存器	0x01DC	RO
AXI_DMA_OUTFIFO_STATUS_CH2_REG	TX 通道 2 FIFO 状态寄存器	0x0220	RO
AXI_DMA_OUT_STATE_CH2_REG	TX 通道 2 状态寄存器	0x0230	RO
AXI_DMA_OUT_EOF_DES_ADDR_CH2_REG	TX 通道 2 EOF 地址寄存器	0x0234	RO
AXI_DMA_OUT_EOF_BFR_DES_ADDR_CH2_REG	TX 通道 2 上一个 EOF 地址寄存器	0x0238	RO
AXI_DMA_OUT_DSCR_CH2_REG	TX 通道 2 下一个描述符地址寄存器	0x023C	RO
AXI_DMA_OUT_DSCR_BF0_CH2_REG	TX 通道 2 当前描述符地址寄存器	0x0240	RO
AXI_DMA_OUT_DSCR_BF1_CH2_REG	TX 通道 2 上一个描述符地址寄存器	0x0244	RO
AXI_DMA_WRESP_CNT_REG	AXI 写响应计数寄存器	0x02B8	RO

名称	描述	地址	访问
AXI_DMA_RRESP_CNT_REG	AXI 读响应计数寄存器	0x02BC	RO
AXI_DMA_INFIFO_STATUS1_CH0_REG	RX 通道 0 FIFO 状态 1 寄存器	0x02C0	RO
AXI_DMA_INFIFO_STATUS1_CH1_REG	RX 通道 1 FIFO 状态 1 寄存器	0x02C4	RO
AXI_DMA_INFIFO_STATUS1_CH2_REG	RX 通道 2 FIFO 状态 1 寄存器	0x02C8	RO
AXI_DMA_OUTFIFO_STATUS1_CH0_REG	TX 通道 0 FIFO 状态 1 寄存器	0x02CC	RO
AXI_DMA_OUTFIFO_STATUS1_CH1_REG	TX 通道 1 FIFO 状态 1 寄存器	0x02D0	RO
AXI_DMA_OUTFIFO_STATUS1_CH2_REG	TX 通道 2 FIFO 状态 1 寄存器	0x02D4	RO
AXI_DMA_LINK_SWITCH_STATE_REG	链表切换寄存器	0x02DC	R/W
优先级寄存器			
AXI_DMA_IN_PRI_CH0_REG	RX 通道 0 优先级寄存器	0x0040	R/W
AXI_DMA_IN_PRI_CH1_REG	RX 通道 1 优先级寄存器	0x00A8	R/W
AXI_DMA_IN_PRI_CH2_REG	RX 通道 2 优先级寄存器	0x0110	R/W
AXI_DMA_OUT_PRI_CH0_REG	TX 通道 0 优先级寄存器	0x0178	R/W
AXI_DMA_OUT_PRI_CH1_REG	TX 通道 1 优先级寄存器	0x01E0	R/W
AXI_DMA_OUT_PRI_CH2_REG	TX 通道 2 优先级寄存器	0x0248	R/W
外设选择寄存器			
AXI_DMA_IN_PERI_SEL_CH0_REG	RX 通道 0 外设选择寄存器	0x0044	R/W
AXI_DMA_IN_PERI_SEL_CH1_REG	RX 通道 1 外设选择寄存器	0x00AC	R/W
AXI_DMA_IN_PERI_SEL_CH2_REG	RX 通道 2 外设选择寄存器	0x0114	R/W
AXI_DMA_OUT_PERI_SEL_CH0_REG	TX 通道 0 外设选择寄存器	0x017C	R/W
AXI_DMA_OUT_PERI_SEL_CH1_REG	TX 通道 1 外设选择寄存器	0x01E4	R/W
AXI_DMA_OUT_PERI_SEL_CH2_REG	TX 通道 2 外设选择寄存器	0x024C	R/W
调试寄存器			
AXI_DMA_RDN_RESULT_REG	保留	0x02AC	varies
AXI_DMA_RDN_ECO_HIGH_REG	保留	0x02B0	R/W
AXI_DMA_RDN_ECO_LOW_REG	保留	0x02B4	R/W
版本寄存器			
AXI_DMA_DATE_REG	版本控制寄存器	0x02D8	R/W

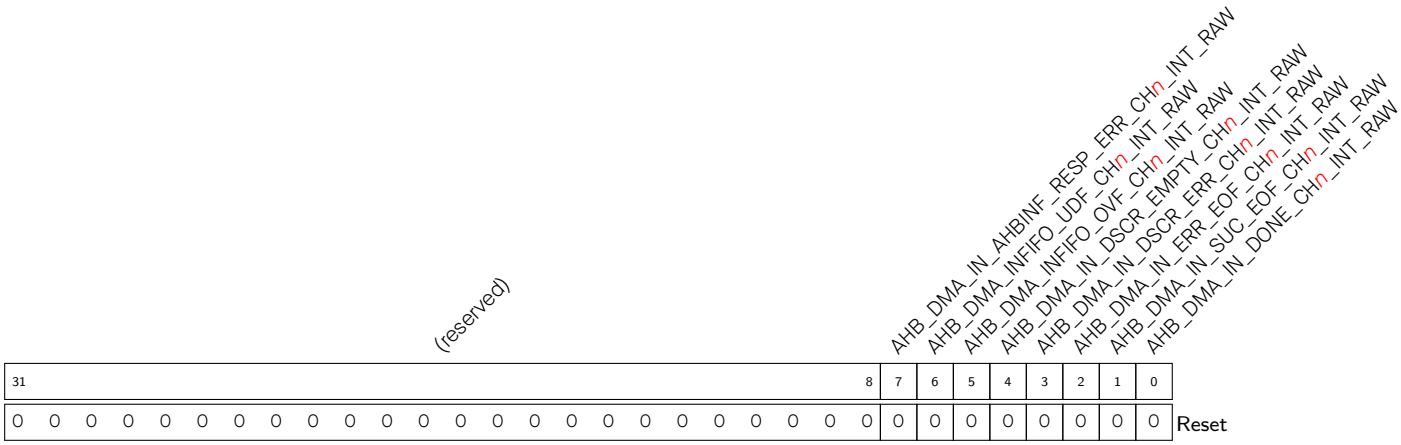
3.9 寄存器

3.9.1 GDMA-AHB-HP 寄存器

本小节的所有地址均为相对于 GDMA-AHB-HP 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 3.1. AHB_DMA_IN_INT_RAW_CH*n*_REG (*n*: 0-4) (0x0000+0x10n*)**



AHB_DMA_IN_DONE_CH*n*_INT_RAW AHB_DMA_IN_DONE_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AHB_DMA_IN_SUC_EOF_CH*n*_INT_RAW AHB_DMA_IN_SUC_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AHB_DMA_IN_ERR_EOF_CH*n*_INT_RAW AHB_DMA_IN_ERR_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AHB_DMA_IN_DSCR_ERR_CH*n*_INT_RAW AHB_DMA_IN_DSCR_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

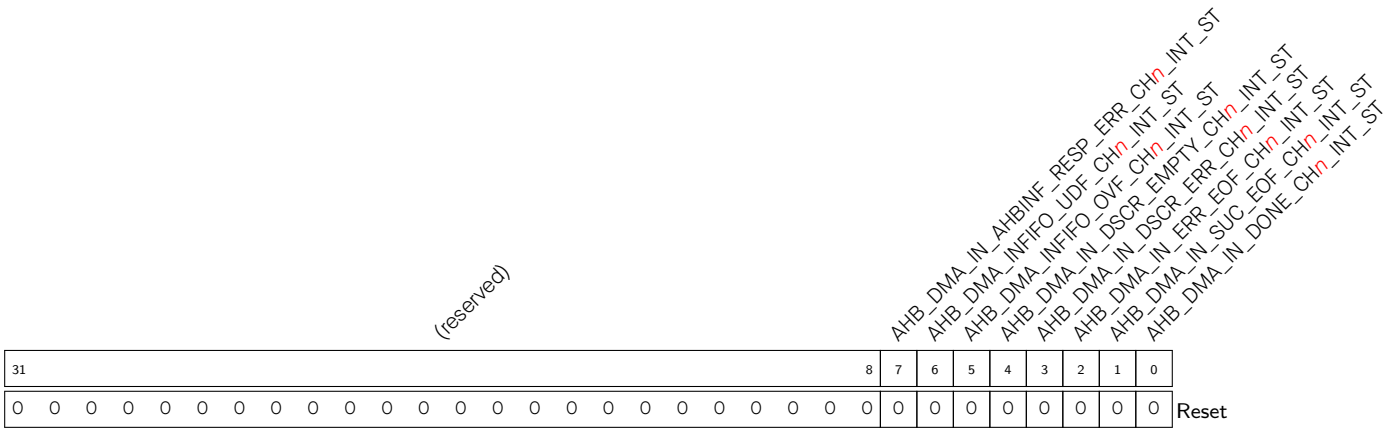
AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT_RAW AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AHB_DMA_INFIFO_OVF_CH*n*_INT_RAW AHB_DMA_INFIFO_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AHB_DMA_INFIFO_UDF_CH*n*_INT_RAW AHB_DMA_INFIFO_UDF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT_RAW AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

Register 3.2. AHB_DMA_IN_INT_ST_CH*n*_REG (*n*: 0-4) (0x0004+0x10***n*)



AHB_DMA_IN_DONE_CH*n*_INT_ST AHB_DMA_IN_DONE_CH*n*_INT 的屏蔽中断状态。(RO)

AHB_DMA_IN_SUC_EOF_CH*n*_INT_ST AHB_DMA_IN_SUC_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

AHB_DMA_IN_ERR_EOF_CH*n*_INT_ST AHB_DMA_IN_ERR_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

AHB_DMA_IN_DSCR_ERR_CH*n*_INT_ST AHB_DMA_IN_DSCR_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

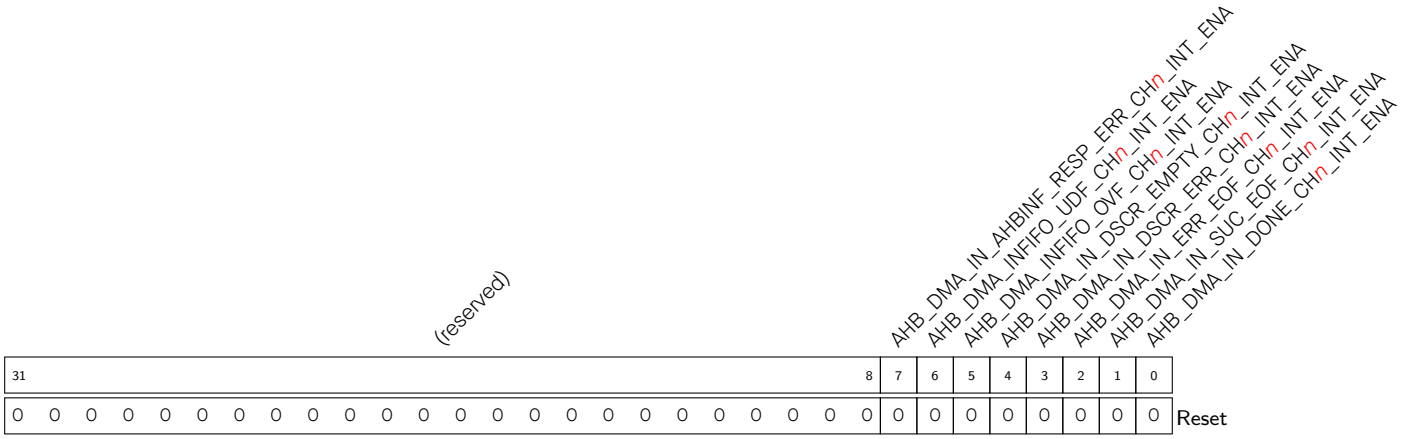
AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT_ST AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT 的屏蔽中断状态。(RO)

AHB_DMA_INFIFO_OVF_CH*n*_INT_ST AHB_DMA_INFIFO_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

AHB_DMA_INFIFO_UDF_CH*n*_INT_ST AHB_DMA_INFIFO_UDF_CH*n*_INT 的屏蔽中断状态。(RO)

AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT_ST AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

Register 3.3. AHB_DMA_IN_INT_ENA_CH*n*_REG (*n*: 0-4) (0x0008+0x10***n*)



AHB_DMA_IN_DONE_CH*n*_INT_ENA 写 1 使能 AHB_DMA_IN_DONE_CH*n*_INT。(R/W)

AHB_DMA_IN_SUC_EOF_CH*n*_INT_ENA 写 1 使能 AHB_DMA_IN_SUC_EOF_CH*n*_INT。(R/W)

AHB_DMA_IN_ERR_EOF_CH*n*_INT_ENA 写 1 使能 AHB_DMA_IN_ERR_EOF_CH*n*_INT。(R/W)

AHB_DMA_IN_DSCR_ERR_CH*n*_INT_ENA 写 1 使能 AHB_DMA_IN_DSCR_ERR_CH*n*_INT。(R/W)

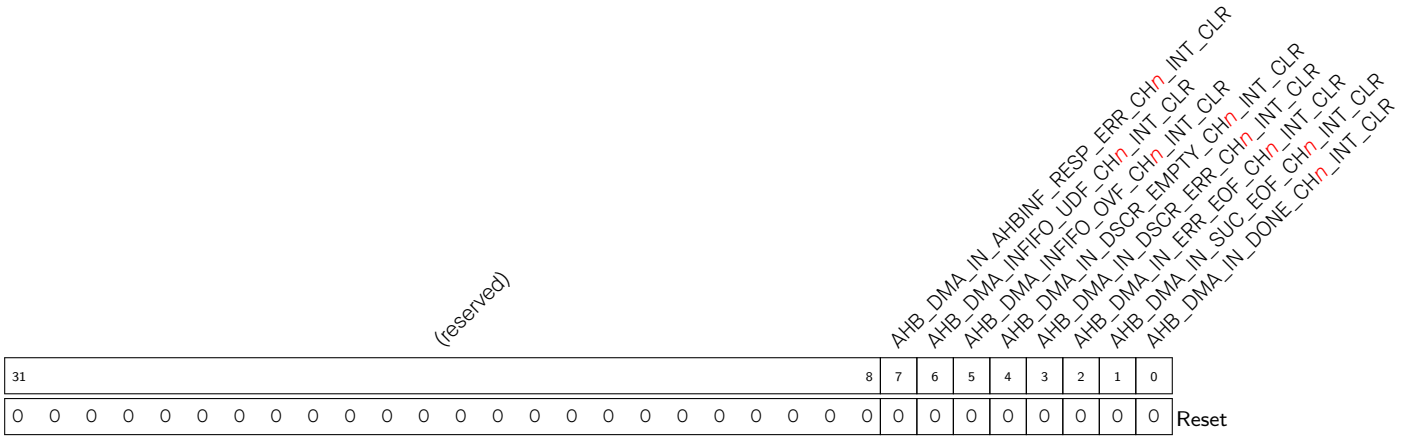
AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT_ENA 写 1 使能 AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT。(R/W)

AHB_DMA_INFIFO_OVF_CH*n*_INT_ENA 写 1 使能 AHB_DMA_INFIFO_OVF_CH*n*_INT。(R/W)

AHB_DMA_INFIFO_UDF_CH*n*_INT_ENA 写 1 使能 AHB_DMA_INFIFO_UDF_CH*n*_INT。(R/W)

AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT_ENA 写 1 使能 AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT。(R/W)

Register 3.4. AHB_DMA_IN_INT_CLR_CH*n*_REG (*n*: 0-4) (0x000C+0x10**n*)



AHB_DMA_IN_DONE_CH*n*_INT_CLR 写 1 清除 AHB_DMA_IN_DONE_CH*n*_INT。(WT)

AHB_DMA_IN_SUC_EOF_CH*n*_INT_CLR 写 1 清除 AHB_DMA_IN_SUC_EOF_CH*n*_INT。(WT)

AHB_DMA_IN_ERR_EOF_CH*n*_INT_CLR 写 1 清除 AHB_DMA_IN_ERR_EOF_CH*n*_INT。(WT)

AHB_DMA_IN_DSCR_ERR_CH*n*_INT_CLR 写 1 清除 AHB_DMA_IN_DSCR_ERR_CH*n*_INT。(WT)

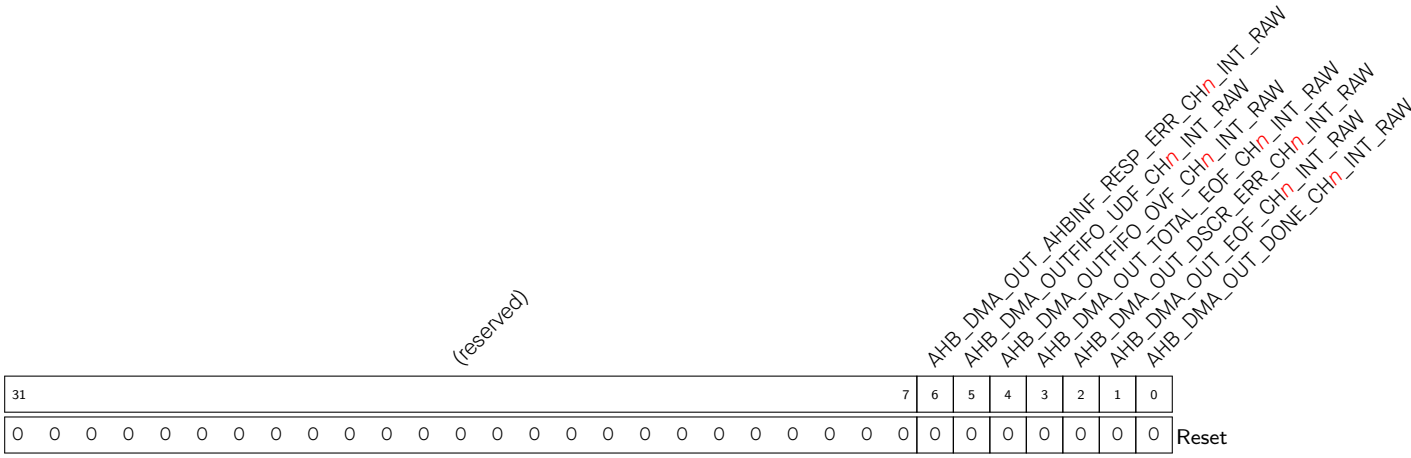
AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT_CLR 写 1 清除 AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT。(WT)

AHB_DMA_INFIFO_OVF_CH*n*_INT_CLR 写 1 清除 AHB_DMA_INFIFO_OVF_CH*n*_INT。(WT)

AHB_DMA_INFIFO_UDF_CH*n*_INT_CLR 写 1 清除 AHB_DMA_INFIFO_UDF_CH*n*_INT。(WT)

AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT_CLR 写 1 清除 AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT。(WT)

Register 3.5. AHB_DMA_OUT_INT_RAW_CH*n*_REG (*n*: 0-4) (0x0050+0x10**n*)



AHB_DMA_OUT_DONE_CH*n*_INT_RAW AHB_DMA_OUT_DONE_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AHB_DMA_OUT_EOF_CH*n*_INT_RAW AHB_DMA_OUT_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AHB_DMA_OUT_DSCR_ERR_CH*n*_INT_RAW AHB_DMA_OUT_DSCR_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

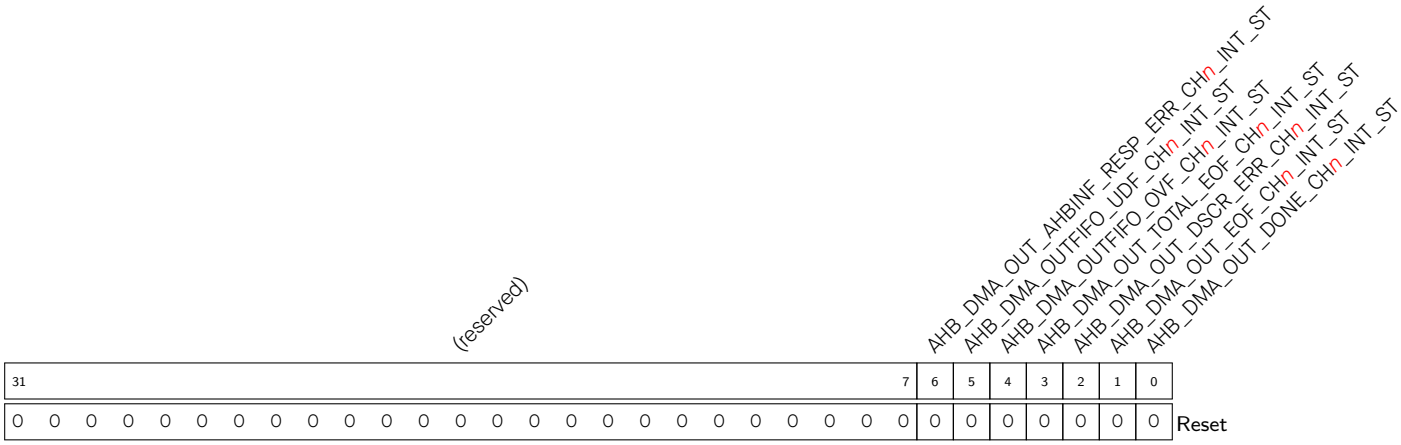
AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT_RAW AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AHB_DMA_OUTFIFO_OVF_CH*n*_INT_RAW AHB_DMA_OUTFIFO_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AHB_DMA_OUTFIFO_UDF_CH*n*_INT_RAW AHB_DMA_OUTFIFO_UDF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT_RAW AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

Register 3.6. AHB_DMA_OUT_INT_ST_CH*n*_REG (*n*: 0-4) (0x0054+0x10**n*)



AHB_DMA_OUT_DONE_CH*n*_INT_ST AHB_DMA_OUT_DONE_CH*n*_INT 的屏蔽中断状态。(RO)

AHB_DMA_OUT_EOF_CH*n*_INT_ST AHB_DMA_OUT_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

AHB_DMA_OUT_DSCR_ERR_CH*n*_INT_ST AHB_DMA_OUT_DSCR_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

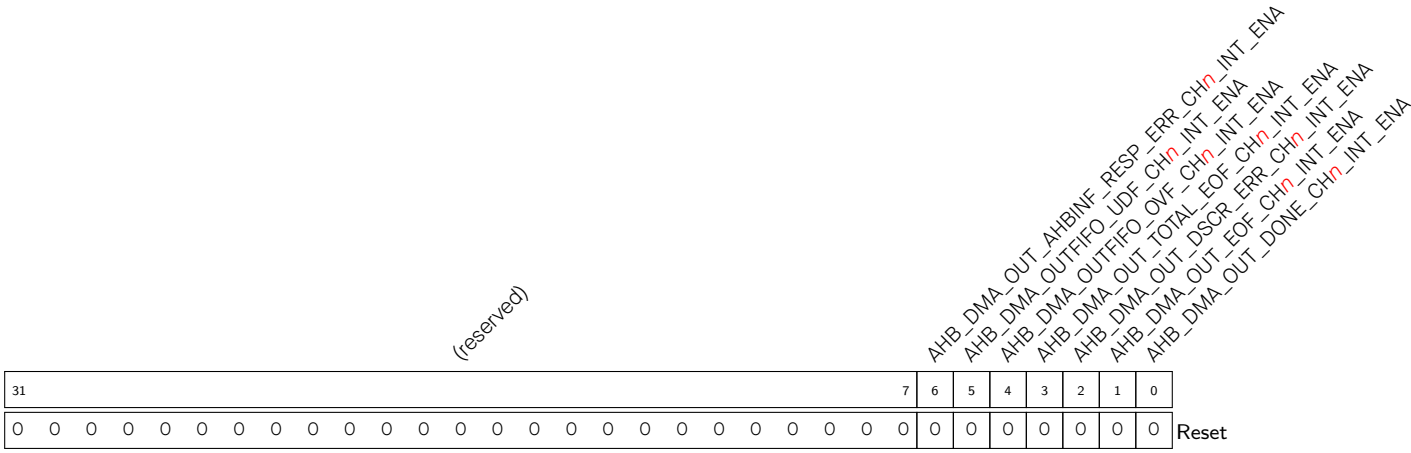
AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT_ST AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

AHB_DMA_OUTFIFO_OVF_CH*n*_INT_ST AHB_DMA_OUTFIFO_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

AHB_DMA_OUTFIFO_UDF_CH*n*_INT_ST AHB_DMA_OUTFIFO_UDF_CH*n*_INT 的屏蔽中断状态。(RO)

AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT_ST AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

Register 3.7. AHB_DMA_OUT_INT_ENA_CH*n*_REG (*n*: 0-4) (0x0058+0x10**n*)



AHB_DMA_OUT_DONE_CH*n*_INT_ENA 写 1 使能 AHB_DMA_OUT_DONE_CH*n*_INT。(R/W)

AHB_DMA_OUT_EOF_CH*n*_INT_ENA 写 1 使能 AHB_DMA_OUT_EOF_CH*n*_INT。(R/W)

AHB_DMA_OUT_DSCR_ERR_CH*n*_INT_ENA 写 1 使能 AHB_DMA_OUT_DSCR_ERR_CH*n*_INT。(R/W)

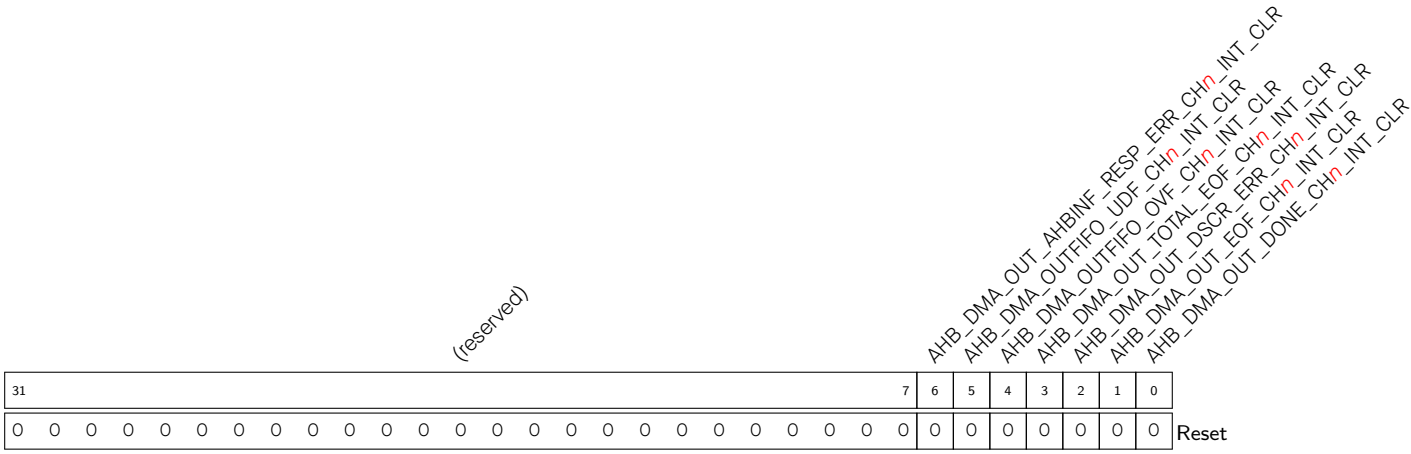
AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT_ENA 写 1 使能 AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT。(R/W)

AHB_DMA_OUTFIFO_OVF_CH*n*_INT_ENA 写 1 使能 AHB_DMA_OUTFIFO_OVF_CH*n*_INT。(R/W)

AHB_DMA_OUTFIFO_UDF_CH*n*_INT_ENA 写 1 使能 AHB_DMA_OUTFIFO_UDF_CH*n*_INT。(R/W)

AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT_ENA 写 1 使能 AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT。(R/W)

Register 3.8. AHB_DMA_OUT_INT_CLR_CH*n*_REG (*n*: 0-4) (0x005C+0x10*n*)



AHB_DMA_OUT_DONE_CH*n*_INT_CLR 写 1 清除 AHB_DMA_OUT_DONE_CH*n*_INT。(WT)

AHB_DMA_OUT_EOF_CH*n*_INT_CLR 写 1 清除 AHB_DMA_OUT_EOF_CH*n*_INT。(WT)

AHB_DMA_OUT_DSCR_ERR_CH*n*_INT_CLR 写 1 清除 AHB_DMA_OUT_DSCR_ERR_CH*n*_INT。(WT)

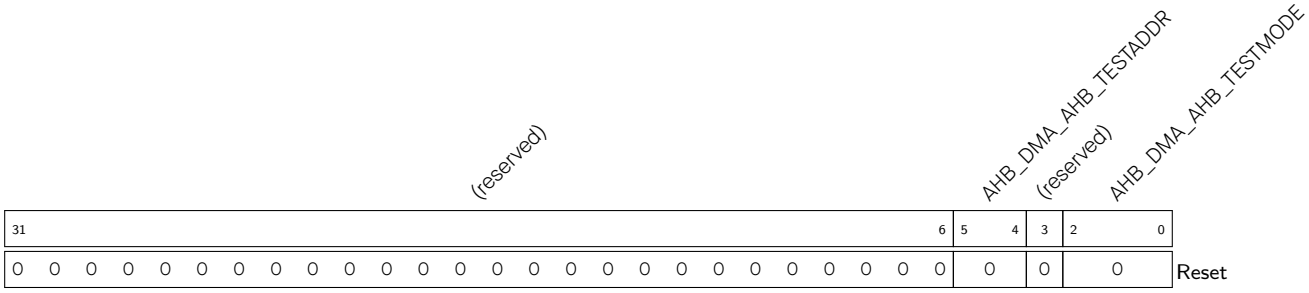
AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT_CLR 写 1 清除 AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT。(WT)

AHB_DMA_OUTFIFO_OVF_CH*n*_INT_CLR 写 1 清除 AHB_DMA_OUTFIFO_OVF_CH*n*_INT。(WT)

AHB_DMA_OUTFIFO_UDF_CH*n*_INT_CLR 写 1 清除 AHB_DMA_OUTFIFO_UDF_CH*n*_INT。(WT)

AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT_CLR 写 1 清除 AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT。(WT)

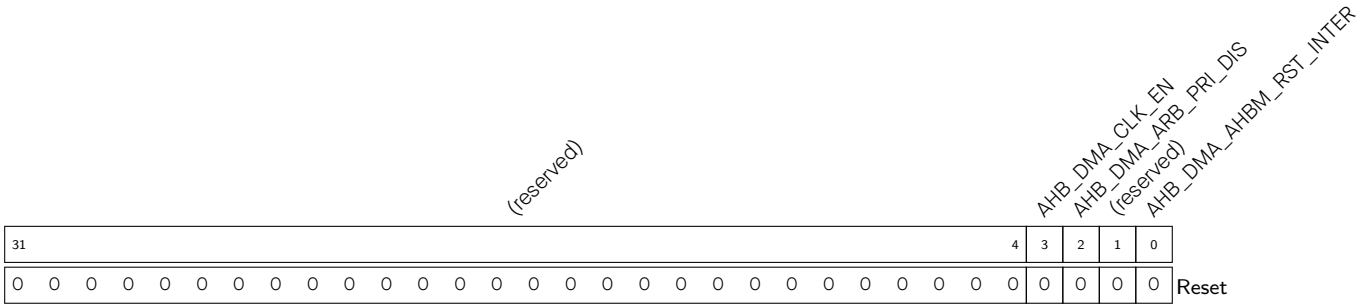
Register 3.9. AHB_DMA_AHB_TEST_REG (0x00A0)



AHB_DMA_AHB_TESTMODE 保留。(R/W)

AHB_DMA_AHB_TESTADDR 保留。(R/W)

Register 3.10. AHB_DMA_MISC_CONF_REG (0x00A4)



AHB_DMA_AHBM_RST_INTER 配置内部 AHB FSM 的复位状态。

0：解除复位

1：复位

(R/W)

AHB_DMA_ARB_PRI_DIS 配置是否禁用固定优先级通道仲裁。

0：使能

1：禁用

(R/W)

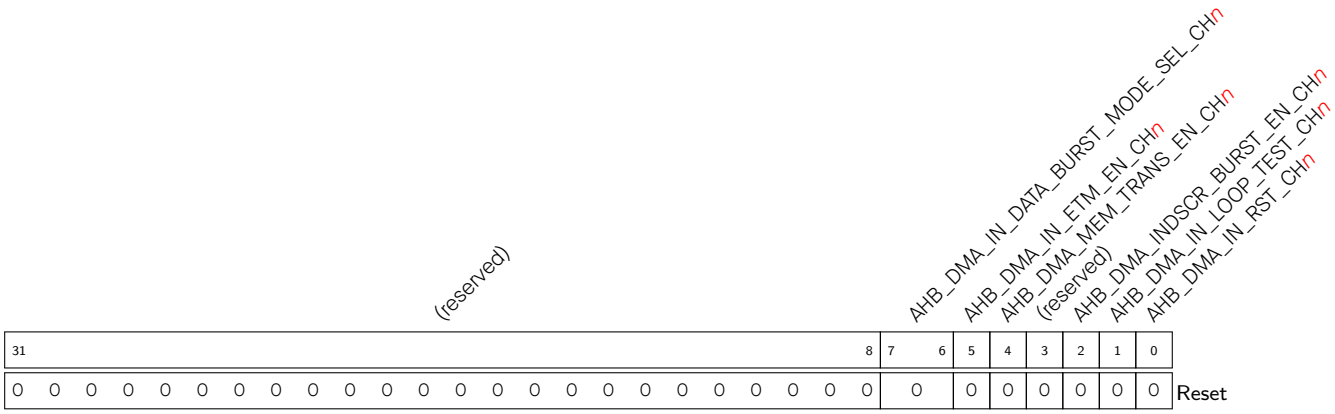
AHB_DMA_CLK_EN 配置 AHB DMA 时钟门控。

0：仅在应用写寄存器时开启时钟

1：一直强制为寄存器开启时钟

(R/W)

Register 3.11. AHB_DMA_IN_CONFO_CH*n*_REG (*n*: 0-4) (0x0100+0x100**n*)



AHB_DMA_IN_RST_CH*n* 配置 AHB DMA RX 通道 *n* FSM 和 FIFO 指针的复位状态。

0：解除复位

1：复位

(R/W)

AHB_DMA_IN_LOOP_TEST_CH*n* 配置是否关闭 RX 通道 *n* 的 owner 位自动清零。

0：无效值。没有作用

1：关闭

(R/W)

AHB_DMA_INDSCR_BURST_EN_CH*n* 配置是否使能 RX 通道 *n* 读取接收链表描述符的 INCR 突发传输。

0：禁用

1：使能

(R/W)

AHB_DMA_MEM_TRANS_EN_CH*n* 配置是否使能存储器到存储器数据传输。

0：禁用

1：使能

(R/W)

AHB_DMA_IN_ETM_EN_CH*n* 配置是否使能 RX 通道 *n* 的 ETM 控制。

0：禁用

1：使能

(R/W)

AHB_DMA_IN_DATA_BURST_MODE_SEL_CH*n* 配置 RX 通道 *n* 的突发传输模式。

0：单次

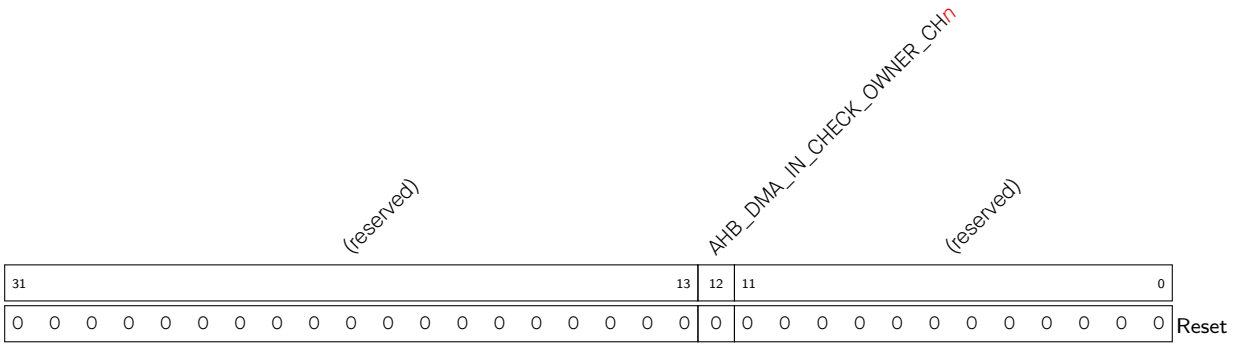
1：INCR4

2：INCR8

3：INCR16

(R/W)

Register 3.12. AHB_DMA_IN_CONF1_CH*n*_REG (*n*: 0-4) (0x0104+0x100*n*)



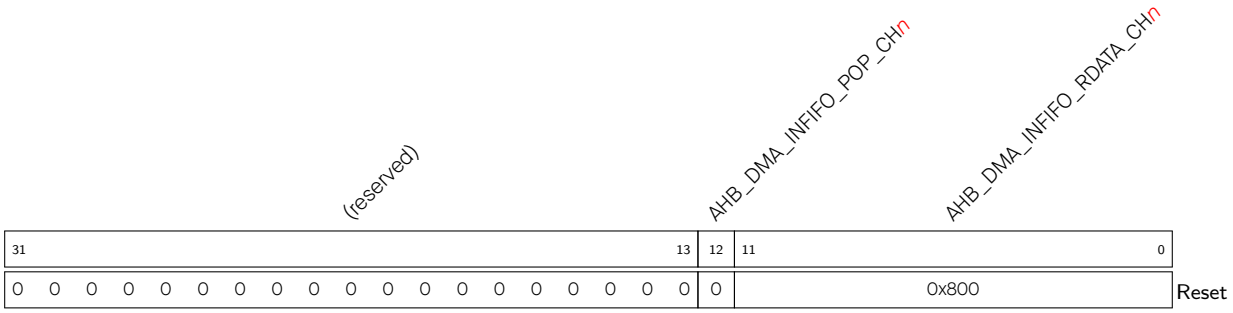
AHB_DMA_IN_CHECK_OWNER_CH*n* 配置是否使能 RX 通道 *n* 的 owner 位检查。

0：禁用

1：使能

(R/W)

Register 3.13. AHB_DMA_IN_POP_CH*n*_REG (*n*: 0-4) (0x010C+0x100*n*)



AHB_DMA_INFIFO_RDATA_CH*n* 表示从 AHB DMA FIFO 接收的数据。(RO)

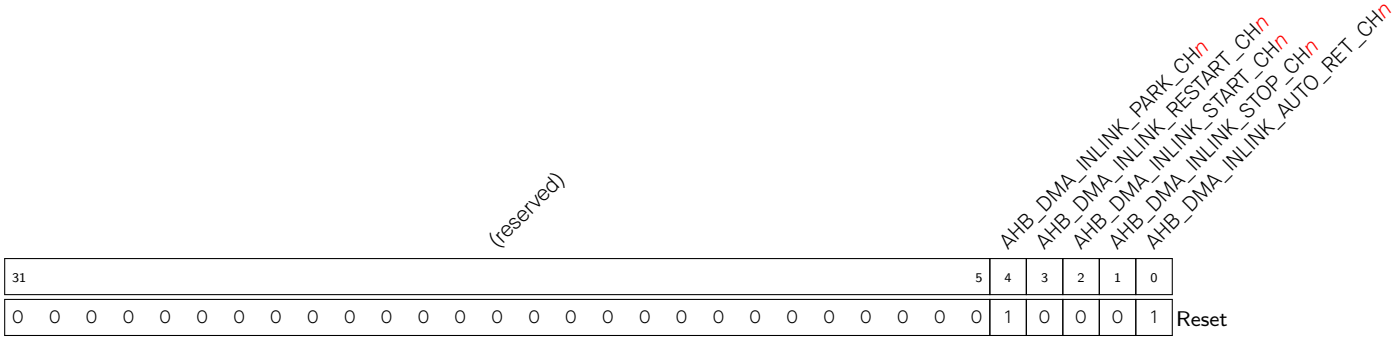
AHB_DMA_INFIFO_POP_CH*n* 配置是否从 AHB DMA FIFO 为 RX 通道 *n* 弹出数据。

0：无效值。没有作用

1：弹出

(WT)

Register 3.14. AHB_DMA_IN_LINK_CH*n*_REG (*n*: 0-4) (0x0110+0x100**n*)



AHB_DMA_INLINK_AUTO_RET_CH*n* 配置当 RX 通道 *n* 当前接收数据发生错误时，是否返回到当前接收链表描述符地址。

0：不返回

1：返回

(R/W)

AHB_DMA_INLINK_STOP_CH*n* 配置是否停止处理 RX 通道 *n* 的描述符。

0：无效值。没有作用

1：停止

(WT)

AHB_DMA_INLINK_START_CH*n* 配置是否开始处理 RX 通道 *n* 的描述符。

0：无效值。没有作用

1：开始

(WT)

AHB_DMA_INLINK_RESTART_CH*n* 配置是否重新开始处理 RX 通道 *n* 的描述符。

0：无效值。没有作用

1：重新开始

(WT)

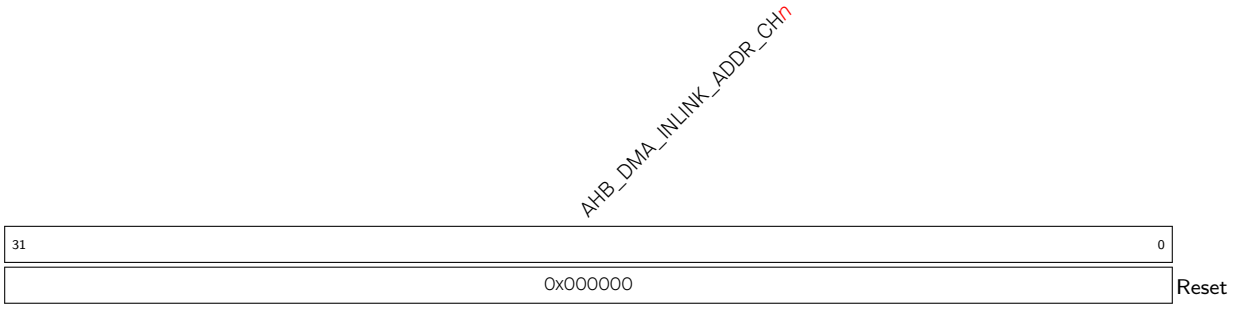
AHB_DMA_INLINK_PARK_CH*n* 表示 RX 通道 *n* 接收链表描述符 FSM 的状态。

0：运行

1：空闲

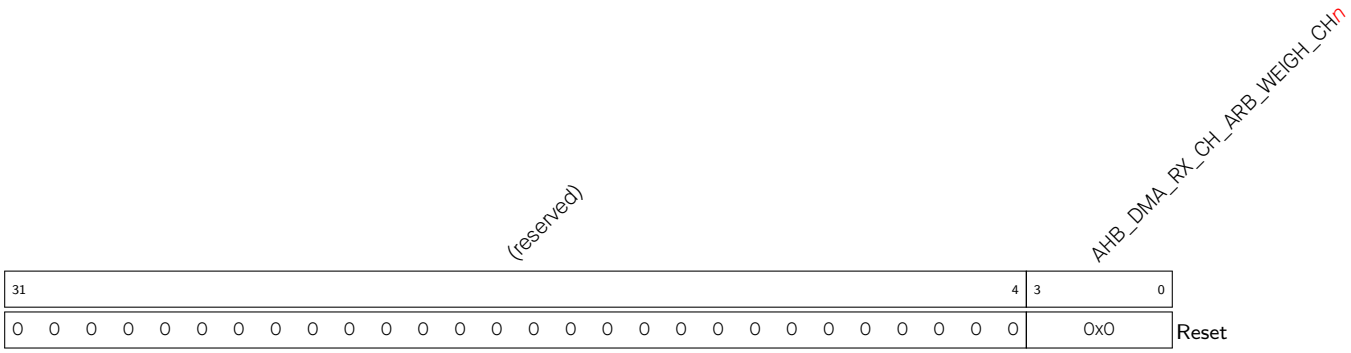
(RO)

Register 3.15. AHB_DMA_IN_LINK_ADDR_CH*n*_REG (*n*: 0-4) (0x0114+0x100*n*)



AHB_DMA_INLINK_ADDR_CH*n* 配置 RX 通道 *n* 的第一个接收链表描述符地址。(R/W)

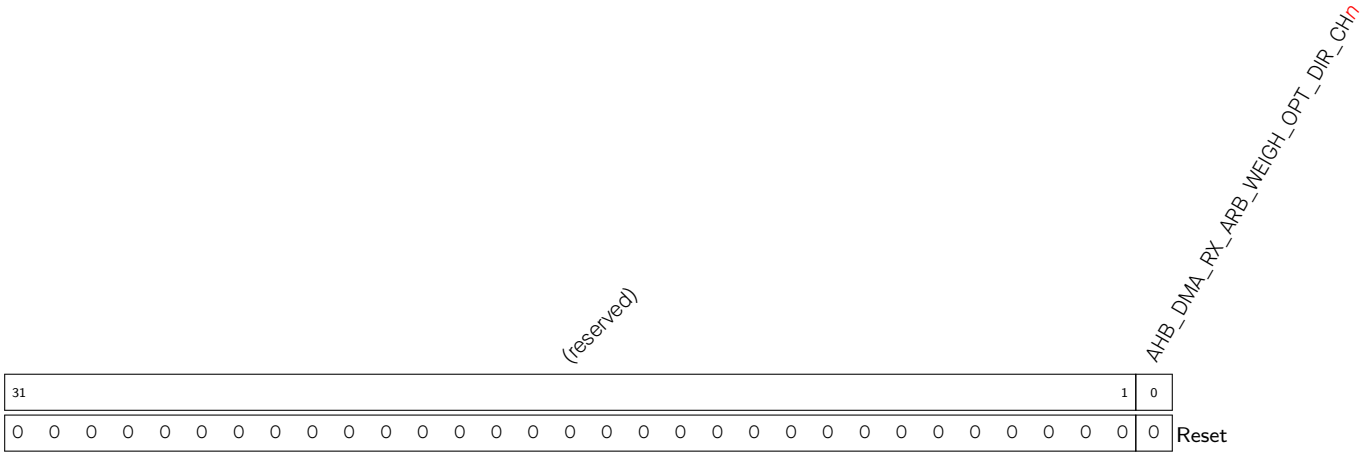
Register 3.16. AHB_DMA_RX_CH_ARB_WEIGH_CH*n*_REG (*n*: 0-4) (0x013C+0x100*n*)



AHB_DMA_RX_CH_ARB_WEIGH_CH*n* 配置 RX 通道 *n* 的仲裁权重。

单位：令牌。(R/W)

Register 3.17. AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH*n*_REG (*n*: 0-4) (0x0140+0x100**n*)



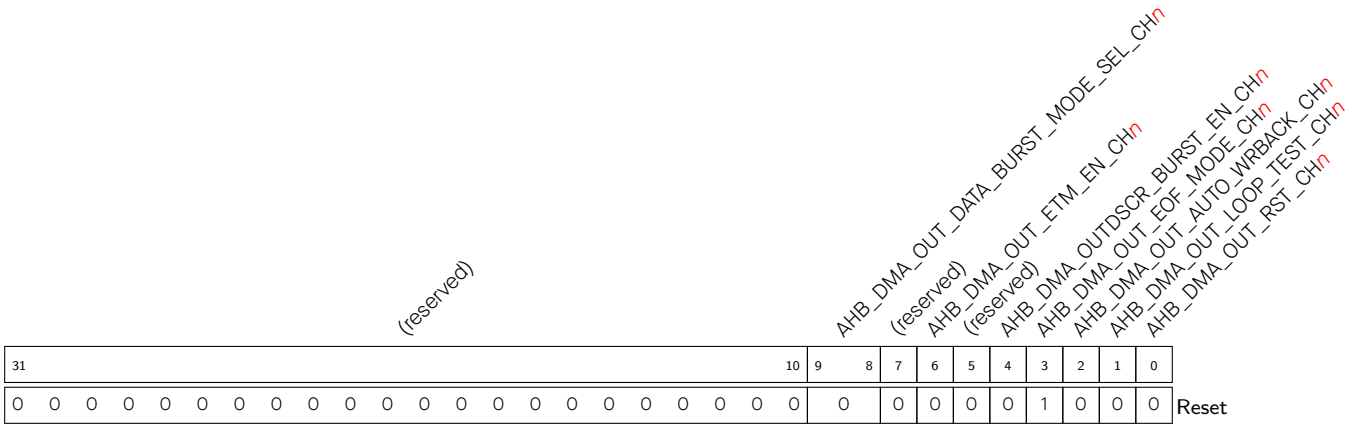
AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH*n* 配置接收通道 *n* 是否开启权重仲裁优化。

0：开启

1：关闭

(R/W)

Register 3.18. AHB_DMA_OUT_CONFO_CH*n*_REG (*n*: 0-4) (0x0180+0x100***n*)



AHB_DMA_OUT_RST_CH*n* 配置 AHB DMA TX 通道 *n* FSM 和 FIFO 指针的复位状态。

0：解除复位

1：复位

(R/W)

AHB_DMA_OUT_LOOP_TEST_CH*n* 配置是否关闭 TX 通道 *n* 的 owner 位自动清零。

0：无效值。没有作用

1：关闭

(R/W)

AHB_DMA_OUT_AUTO_WRBACK_CH*n* 配置当 TX 通道 *n* 的 TX FIFO 中所有数据发送完成时，是否使能自动 outlink 回写。

0：禁用

1：使能

(R/W)

AHB_DMA_OUT_EOF_MODE_CH*n* 配置 TX 通道 *n* EOF 标志的生成时机。

0：待发送数据推入 FIFO 时生成

1：待发送数据从 FIFO 弹出时生成

(R/W)

AHB_DMA_OUTDSCR_BURST_EN_CH*n* 配置是否使能 TX 通道 *n* 读取发送链表描述符的 INCR 突发传输。

0：禁用

1：使能

(R/W)

AHB_DMA_OUT_ETM_EN_CH*n* 配置是否使能 TX 通道 *n* 的 ETM 控制。

0：禁用

1：使能

(R/W)

AHB_DMA_OUT_DATA_BURST_MODE_SEL_CH*n* 配置 TX 通道 *n* 的突发传输模式。

0：单次

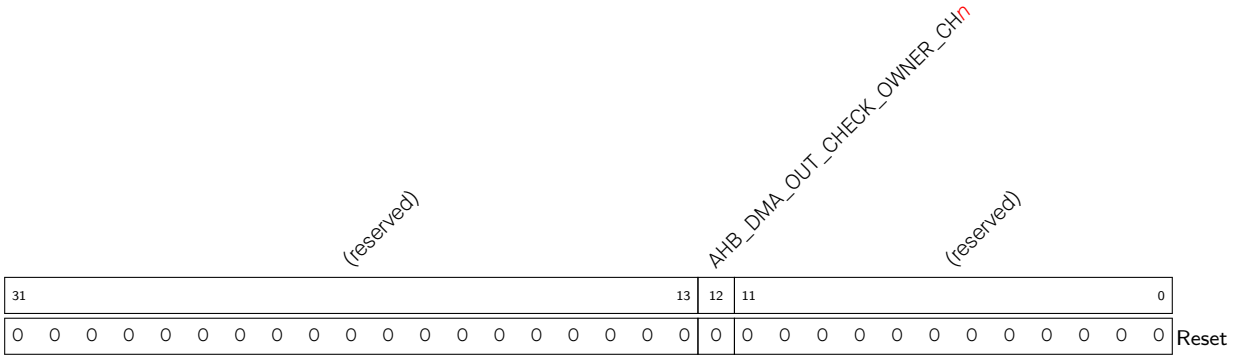
1：INCR4

2：INCR8

3：INCR16

(R/W)

Register 3.19. AHB_DMA_OUT_CONF1_CH*n*_REG (*n*: 0-4) (0x0184+0x100***n*)



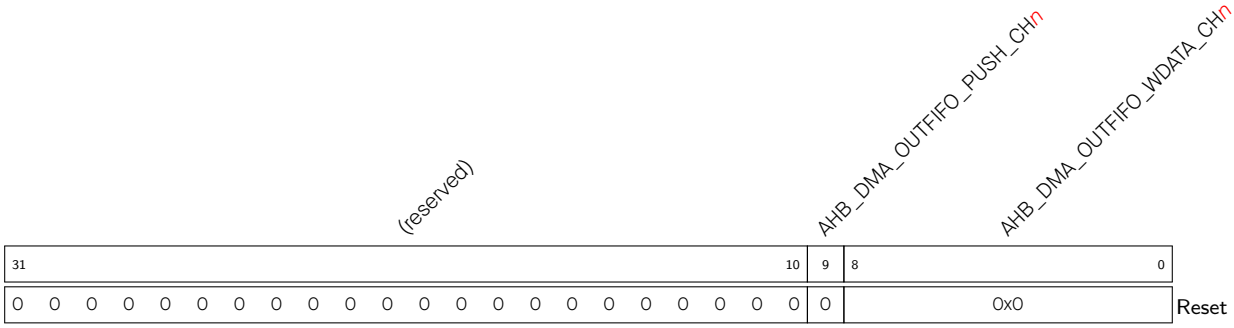
AHB_DMA_OUT_CHECK_OWNER_CH*n* 配置是否使能 TX 通道 *n* 的 owner 位检查。

0：禁用

1：使能

(R/W)

Register 3.20. AHB_DMA_OUT_PUSH_CH*n*_REG (*n*: 0-4) (0x018C+0x100***n*)



AHB_DMA_OUTFIFO_WDATA_CH*n* 待传输至 AHB DMA FIFO 的数据。(R/W)

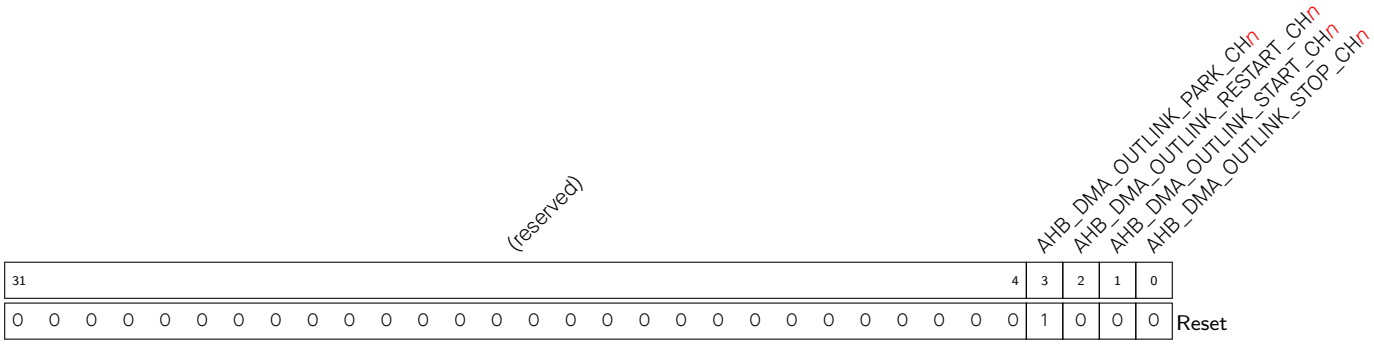
AHB_DMA_OUTFIFO_PUSH_CH*n* 配置是否将数据推入 AHB DMA FIFO（TX 通道 *n*）。

0：无效值。没有作用

1：推入

(WT)

Register 3.21. AHB_DMA_OUT_LINK_CH*n*_REG (*n*: 0-4) (0x0190+0x100**n*)



AHB_DMA_OUTLINK_STOP_CH*n* 配置是否停止处理 TX 通道 *n* 的描述符。

0：无效值。没有作用

1：停止

(WT)

AHB_DMA_OUTLINK_START_CH*n* 配置是否开始处理 TX 通道 *n* 的描述符。

0：无效值。没有作用

1：开始

(WT)

AHB_DMA_OUTLINK_RESTART_CH*n* 配置是否重新开始处理 TX 通道 *n* 的描述符。

0：无效值。没有作用

1：重新开始

(WT)

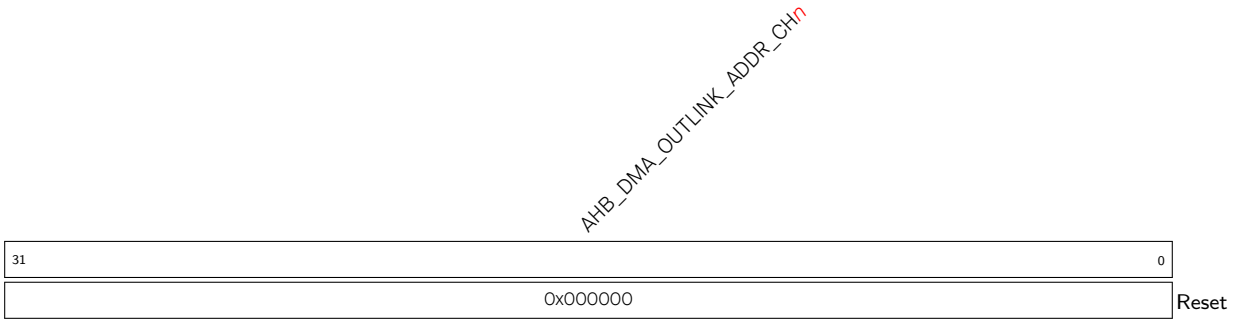
AHB_DMA_OUTLINK_PARK_CH*n* 表示 TX 通道 *n* 发送链表描述符 FSM 的状态。

0：运行

1：空闲

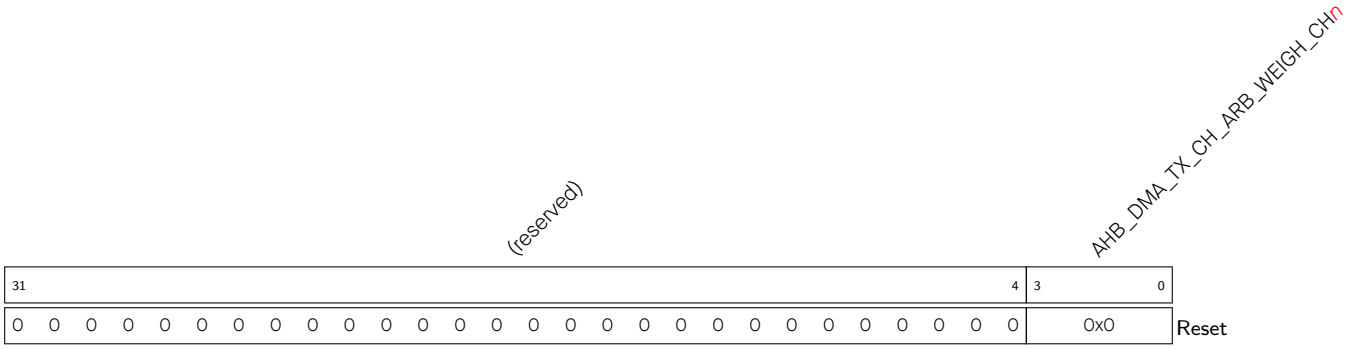
(RO)

Register 3.22. AHB_DMA_OUT_LINK_ADDR_CH*n*_REG (*n*: 0-4) (0x0194+0x100**n*)



AHB_DMA_OUTLINK_ADDR_CH*n* 配置 TX 通道 *n* 的第一个发送链表描述符地址。(R/W)

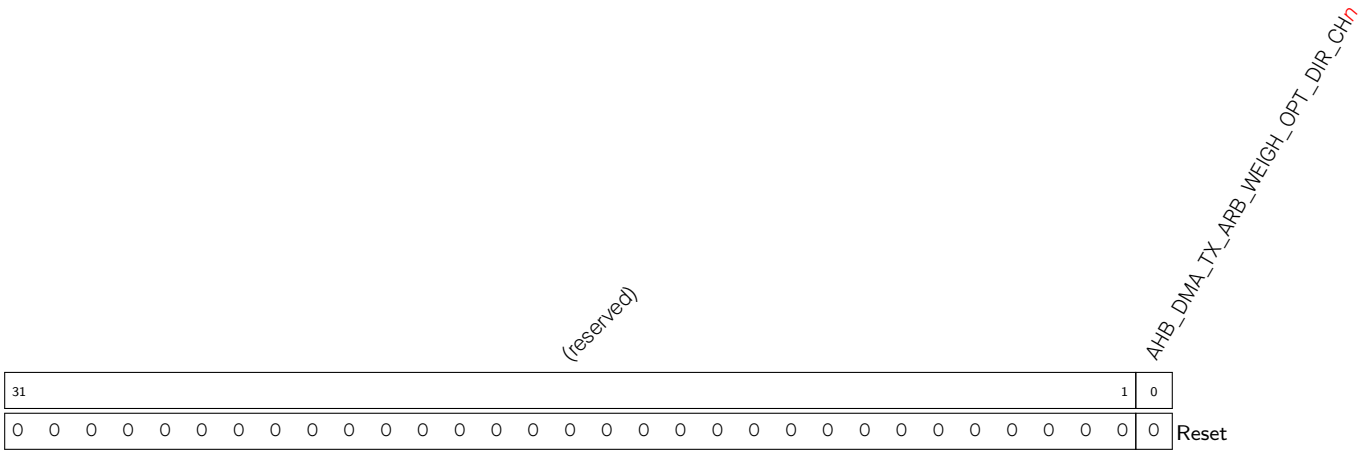
Register 3.23. AHB_DMA_TX_CH_ARB_WEIGH_CH*n*_REG (*n*: 0-4) (0x01BC+0x100*n*)



AHB_DMA_TX_CH_ARB_WEIGH_CH*n* 配置 TX 通道 *n* 的仲裁权重。

单位：令牌。(R/W)

Register 3.24. AHB_DMA_TX_ARB_WEIGH_OPT_DIR_CH*n*_REG (*n*: 0-4) (0x01C0+0x100*n*)



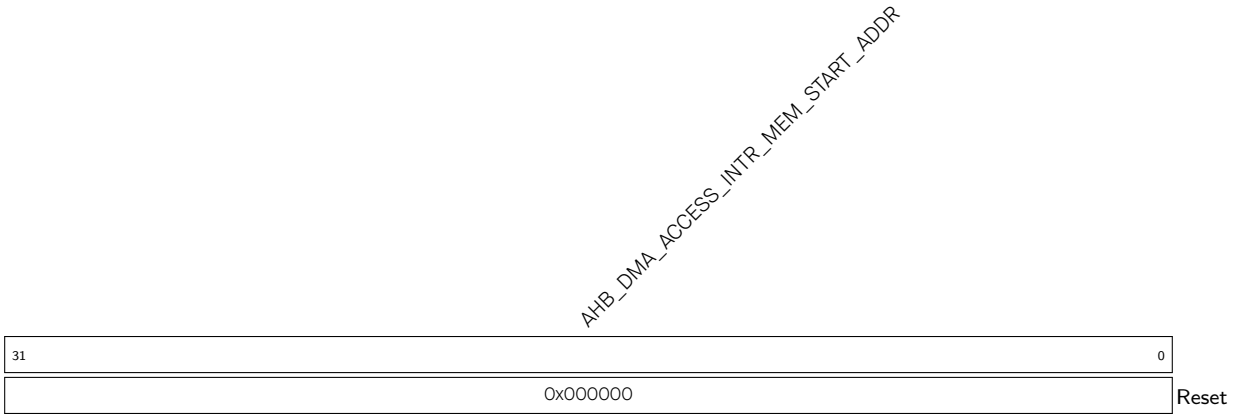
AHB_DMA_TX_ARB_WEIGH_OPT_DIR_CH*n* 配置发送通道 *n* 是否开启权重仲裁优化。

0：开启

1：关闭

(R/W)

Register 3.25. AHB_DMA_INTR_MEM_START_ADDR_REG (0x0600)



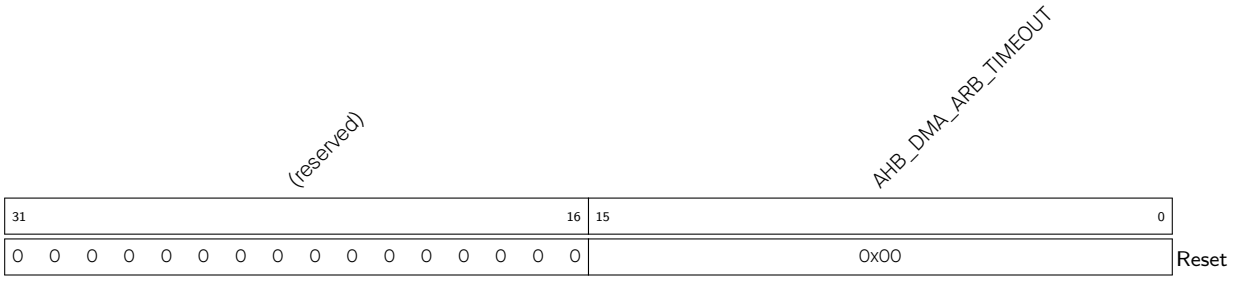
AHB_DMA_ACCESS_INTR_MEM_START_ADDR 配置可访问地址空间的起始地址。(R/W)

Register 3.26. AHB_DMA_INTR_MEM_END_ADDR_REG (0x0604)



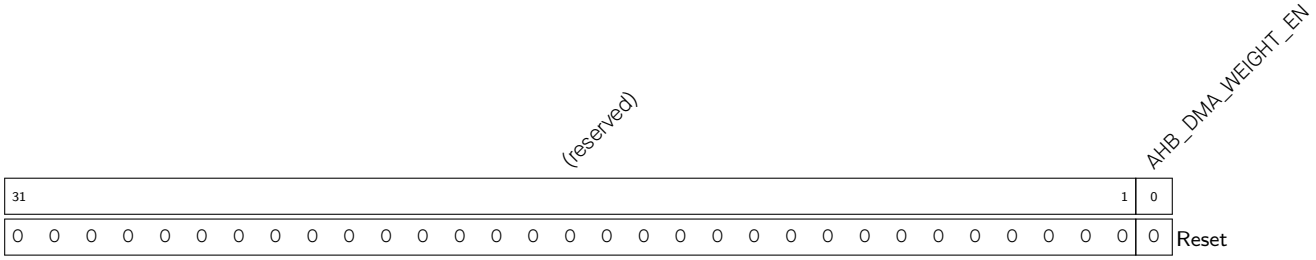
AHB_DMA_ACCESS_INTR_MEM_END_ADDR 配置可访问地址空间的结束地址。(R/W)

Register 3.27. AHB_DMA_ARB_TIMEOUT_REG (0x0608)



AHB_DMA_ARB_TIMEOUT 配置权重仲裁时间片。单位：AHB 总线时钟周期。(R/W)

Register 3.28. AHB_DMA_WEIGHT_EN_REG (0x0610)



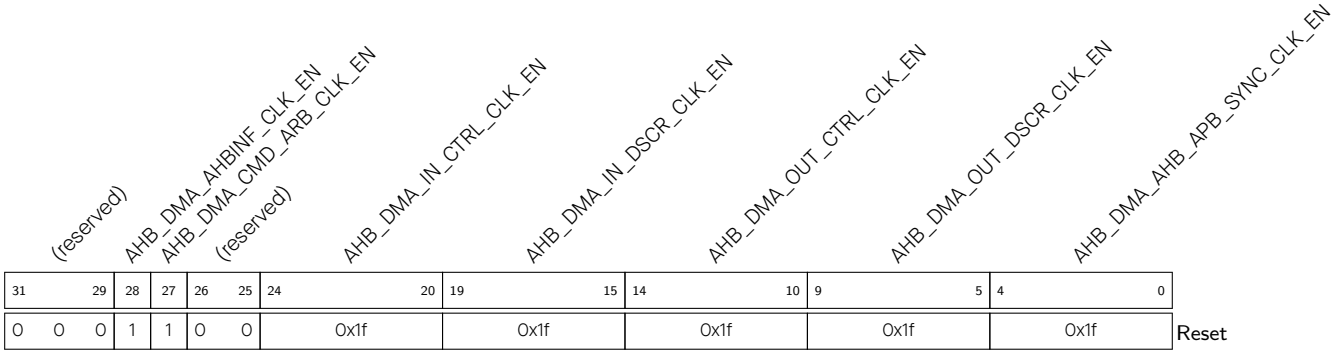
AHB_DMA_WEIGHT_EN 配置是否使能权重仲裁。

0：禁用

1：使能

(R/W)

Register 3.29. AHB_DMA_MODULE_CLK_EN_REG (0x0618)



AHB_DMA_AHB_APB_SYNC_CLK_EN 配置是否强制开启 AHB DMA 寄存器配置同步模块的时钟。

位 0 对应 TX 和 RX 通道 0，位 1 对应 TX 和 RX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

AHB_DMA_OUT_DSCR_CLK_EN 配置是否强制开启 AHB DMA TX 描述符处理模块的时钟。

位 0 对应 TX 通道 0，位 1 对应 TX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

AHB_DMA_OUT_CTRL_CLK_EN 配置是否强制开启 AHB DMA TX 数据处理模块的时钟。

位 0 对应 TX 通道 0，位 1 对应 TX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

AHB_DMA_IN_DSCR_CLK_EN 配置是否强制开启 AHB DMA RX 描述符处理模块的时钟。

位 0 对应 RX 通道 0，位 1 对应 RX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

AHB_DMA_IN_CTRL_CLK_EN 配置是否强制开启 AHB DMA RX 数据处理模块的时钟。

位 0 对应 RX 通道 0，位 1 对应 RX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

AHB_DMA_CMD_ARB_CLK_EN 配置是否强制开启 AHB DMA 仲裁模块的时钟。

位 0 对应 TX 和 RX 通道 0，位 1 对应 TX 和 RX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

见下页...

Register 3.29. AHB_DMA_MODULE_CLK_EN_REG (0x0618)

接上页...

AHB_DMA_AHBINF_CLK_EN 配置是否强制开启 AHB DMA 总线接口处理模块的时钟。

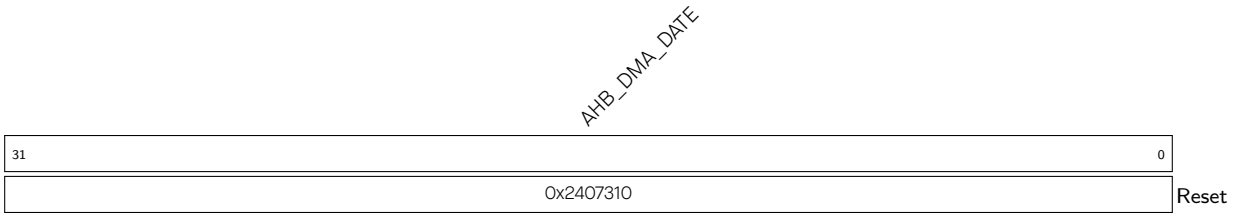
位 0 对应 TX 和 RX 通道 0，位 1 对应 TX 和 RX 通道 1，以此类推。

0：不强制开启

1：强制开启

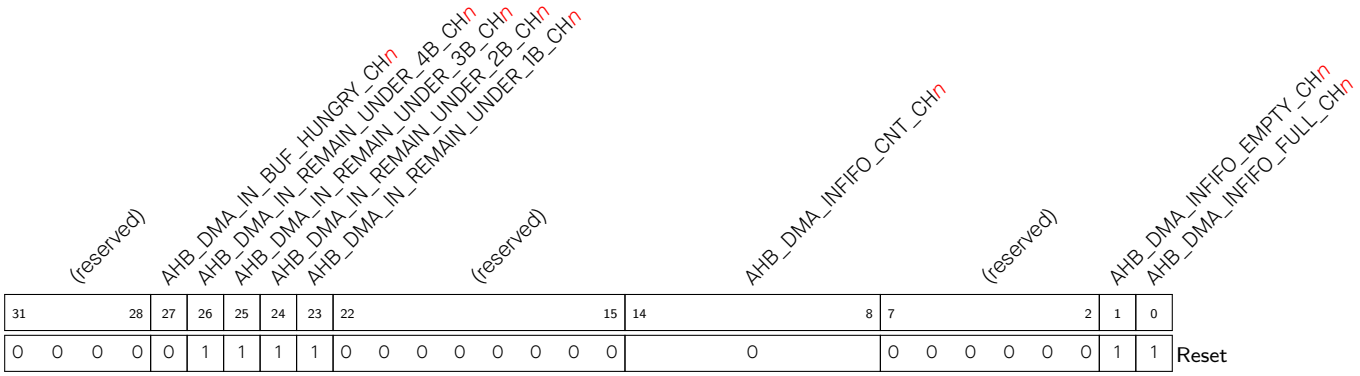
(R/W)

Register 3.30. AHB_DMA_DATE_REG (0x00A8)



AHB_DMA_DATE 版本控制寄存器。(R/W)

Register 3.31. AHB_DMA_INFIFO_STATUS_CH*n*_REG (*n*: 0-4) (0x0108+0x100**n*)



AHB_DMA_INFIFO_FULL_CH*n* 表示 RX 通道 *n* 的 RX FIFO 是否已满。

0：未满

1：已满

(RO)

AHB_DMA_INFIFO_EMPTY_CH*n* 表示 RX 通道 *n* 的 RX FIFO 是否为空。

0：非空

1：空

(RO)

AHB_DMA_INFIFO_CNT_CH*n* 表示 RX 通道 *n* RX FIFO 中的数据字节数。

单位：字节。(RO)

AHB_DMA_IN_REMAIN_UNDER_1B_CH*n* 保留。(RO)

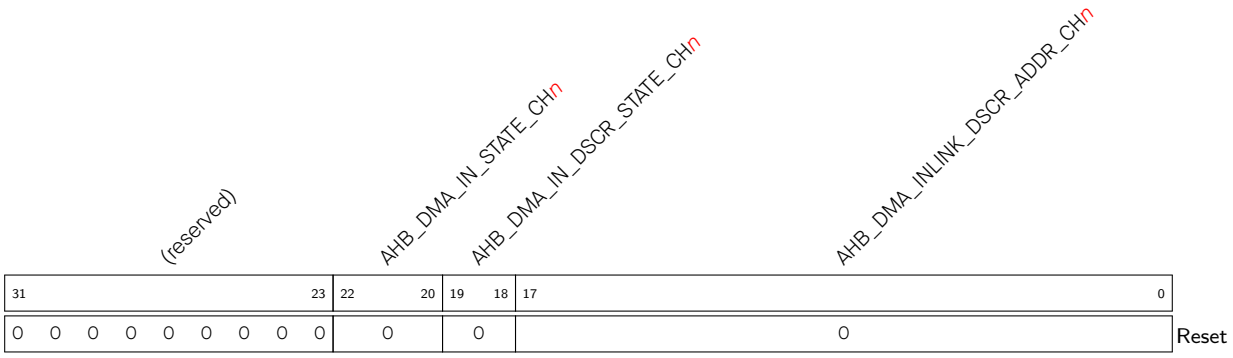
AHB_DMA_IN_REMAIN_UNDER_2B_CH*n* 保留。(RO)

AHB_DMA_IN_REMAIN_UNDER_3B_CH*n* 保留。(RO)

AHB_DMA_IN_REMAIN_UNDER_4B_CH*n* 保留。(RO)

AHB_DMA_IN_BUF_HUNGRY_CH*n* 保留。(RO)

Register 3.32. AHB_DMA_IN_STATE_CH*n*_REG (*n*: 0-4) (0x0118+0x100*n*)

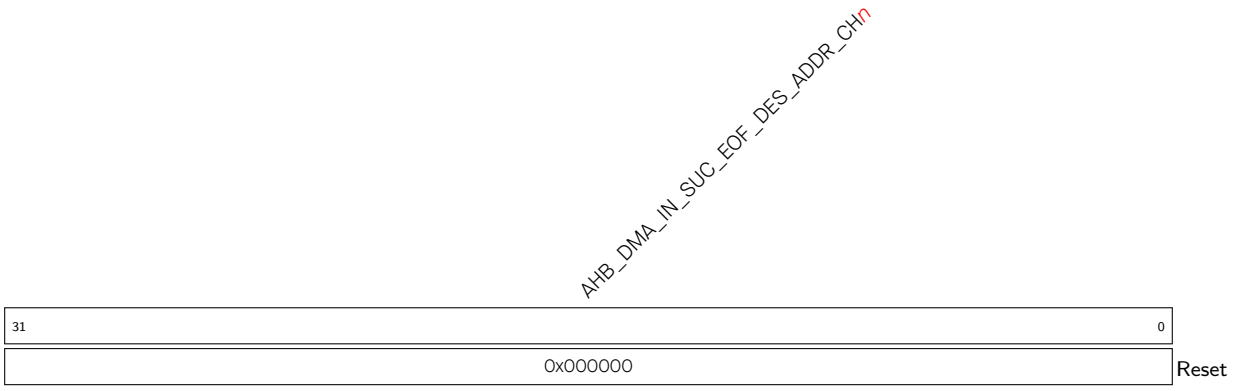


AHB_DMA_INLINK_DSCR_ADDR_CH*n* 表示下一个预读取（但未处理）的接收链表描述符所在地址的低 18 位。如果当前处理的接收链表描述符是最后一个描述符，则该字段表示当前处理的接收链表描述符地址。(RO)

AHB_DMA_IN_DSCR_STATE_CH*n* 保留。(RO)

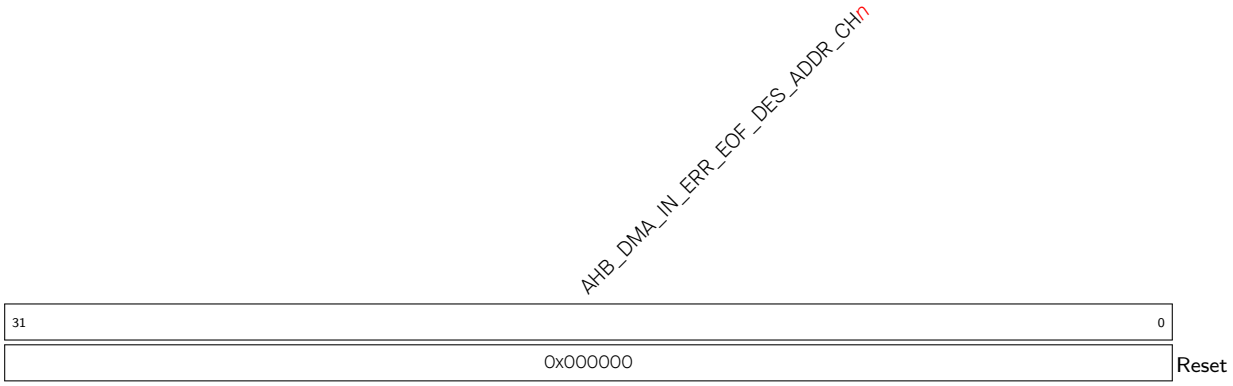
AHB_DMA_IN_STATE_CH*n* 保留。(RO)

Register 3.33. AHB_DMA_IN_SUC_EOF_DES_ADDR_CH*n*_REG (*n*: 0-4) (0x011C+0x100*n*)



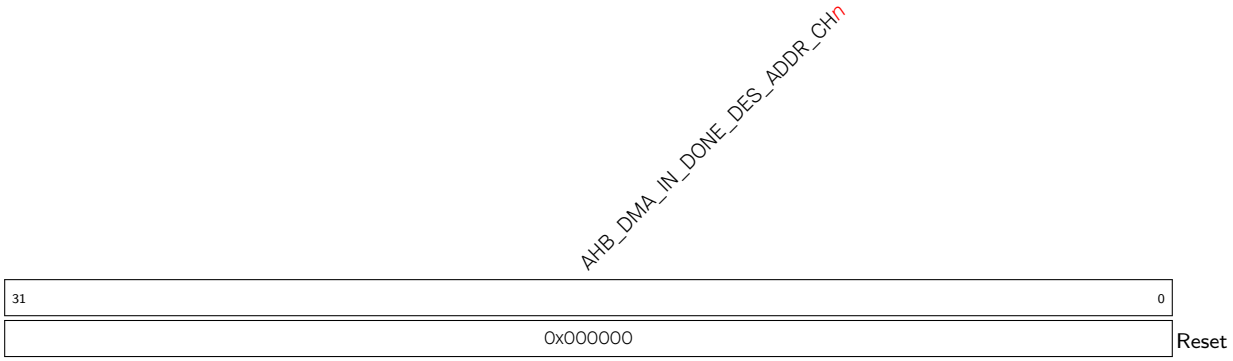
AHB_DMA_IN_SUC_EOF_DES_ADDR_CH*n* 表示 EOF 位为 1 的接收链表描述符的地址。(RO)

Register 3.34. AHB_DMA_IN_ERR_EOF_DES_ADDR_CH*n*_REG (*n*: 0-4) (0x0120+0x100*n*)



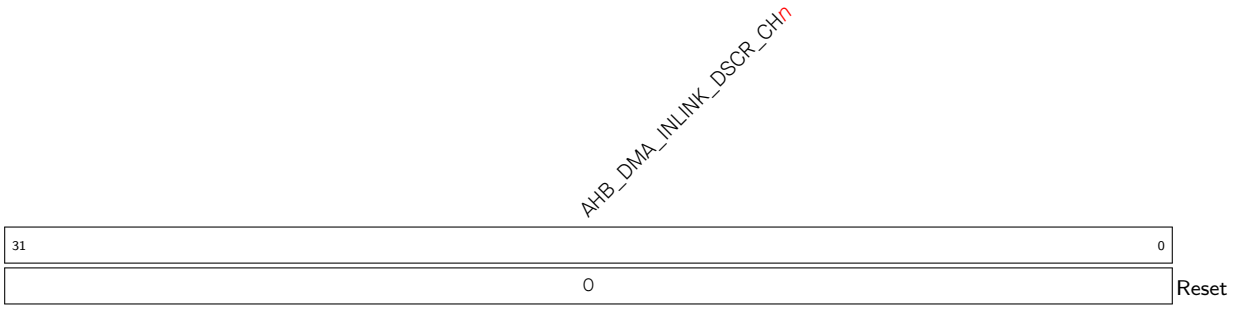
AHB_DMA_IN_ERR_EOF_DES_ADDR_CH*n* 表示当前接收数据有错时，相应接收链表描述符的位置。(RO)

Register 3.35. AHB_DMA_IN_DONE_DES_ADDR_CH*n*_REG (*n*: 0-4) (0x0124+0x100*n*)



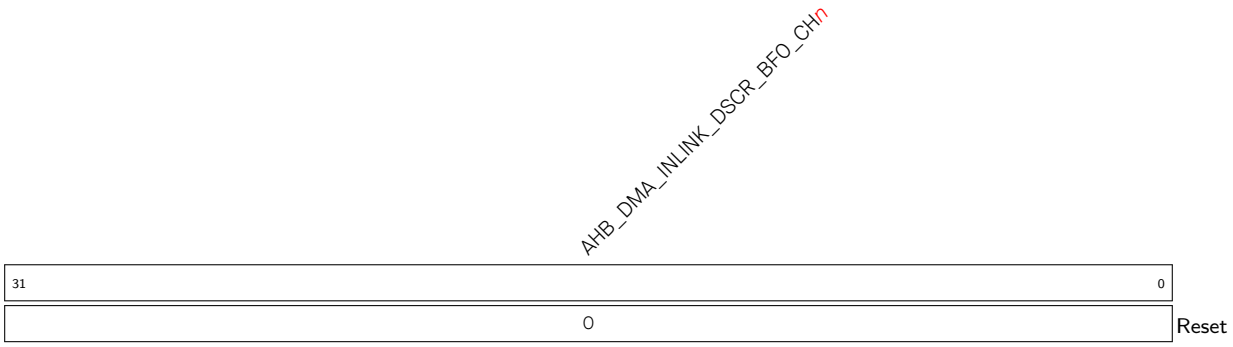
AHB_DMA_IN_DONE_DES_ADDR_CH*n* 表示 RX 方向传输完成时对应描述符的地址。(RO)

Register 3.36. AHB_DMA_IN_DSCR_CH*n*_REG (*n*: 0-4) (0x0128+0x100*n*)



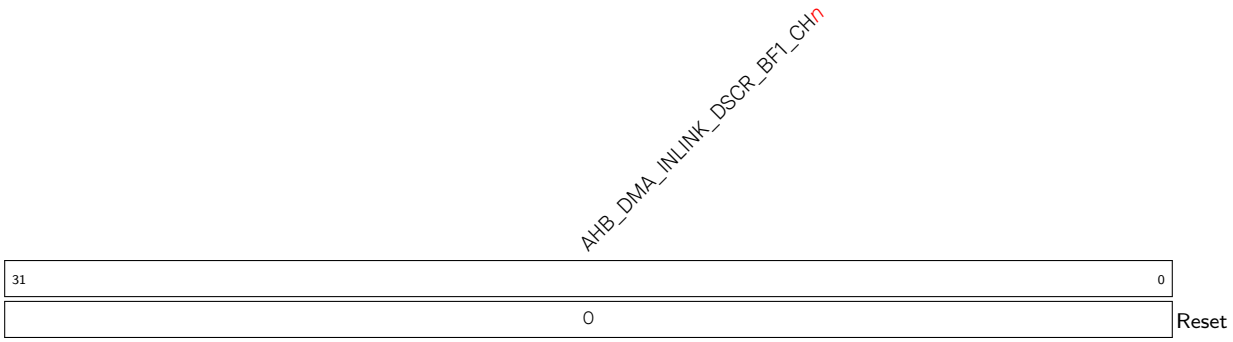
AHB_DMA_INLINK_DSCR_CH*n* 表示当前已预读取的接收链表描述符指向的下一个接收链表描述符地址 x+1。(RO)

Register 3.37. AHB_DMA_IN_DSCR_BF0_CH*n*_REG (*n*: 0-4) (0x012C+0x100**n*)



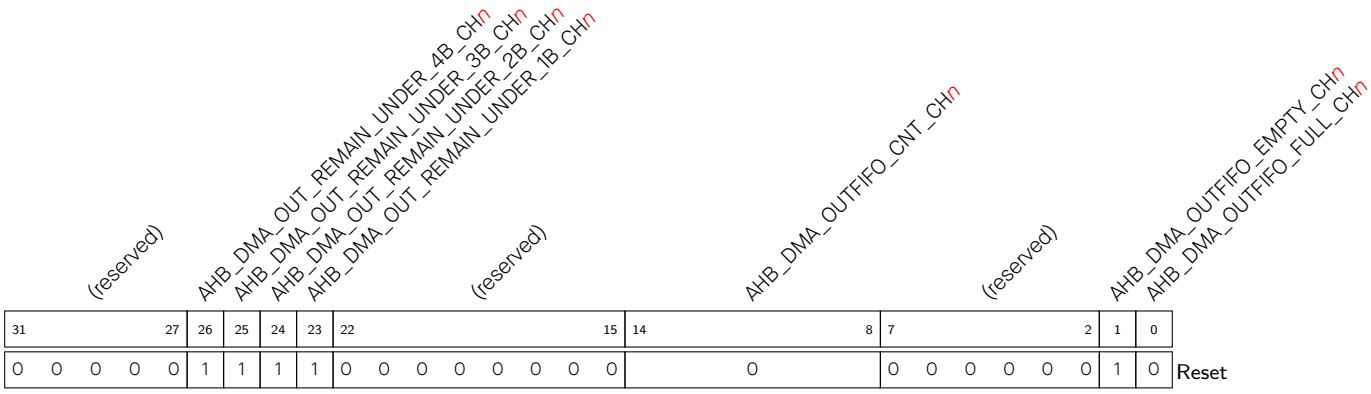
AHB_DMA_INLINK_DSCR_BF0_CH*n* 表示当前已预读取的接收链表描述符所在地址 x。(RO)

Register 3.38. AHB_DMA_IN_DSCR_BF1_CH*n*_REG (*n*: 0-4) (0x0130+0x100**n*)



AHB_DMA_INLINK_DSCR_BF1_CH*n* 表示前一个已预读取的接收链表描述符所在地址 x-1。(RO)

Register 3.39. AHB_DMA_OUTFIFO_STATUS_CH*n*_REG (*n*: 0-4) (0x0188+0x100**n*)



AHB_DMA_OUTFIFO_FULL_CH*n* 表示 TX 通道 *n* 的 TX FIFO 是否已满。

0：未满

1：已满

(RO)

AHB_DMA_OUTFIFO_EMPTY_CH*n* 表示 TX 通道 *n* 的 TX FIFO 是否为空。

0：非空

1：空

(RO)

AHB_DMA_OUTFIFO_CNT_CH*n* 表示 TX 通道 *n* TX FIFO 中的数据字节数。

单位：字节。(RO)

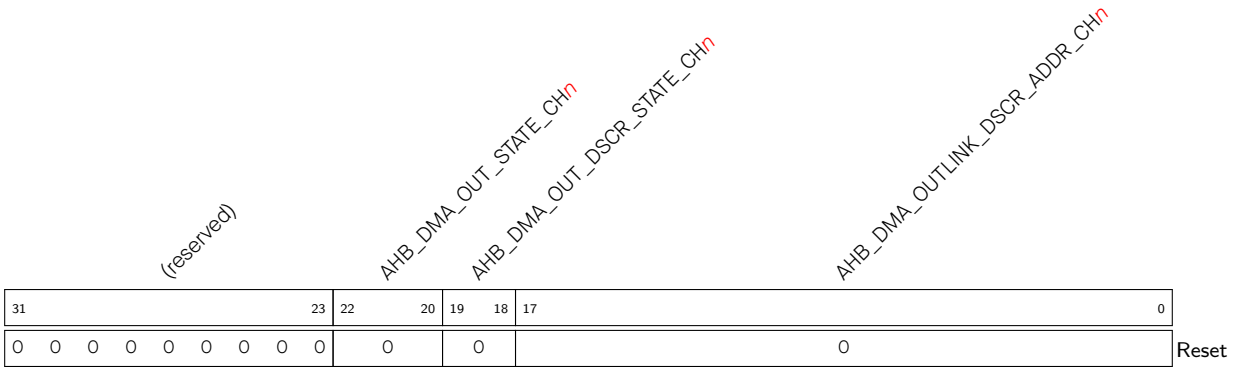
AHB_DMA_OUT_REMAIN_UNDER_1B_CH*n* 保留。(RO)

AHB_DMA_OUT_REMAIN_UNDER_2B_CH*n* 保留。(RO)

AHB_DMA_OUT_REMAIN_UNDER_3B_CH*n* 保留。(RO)

AHB_DMA_OUT_REMAIN_UNDER_4B_CH*n* 保留。(RO)

Register 3.40. AHB_DMA_OUT_STATE_CH*n*_REG (*n*: 0-4) (0x0198+0x100**n*)

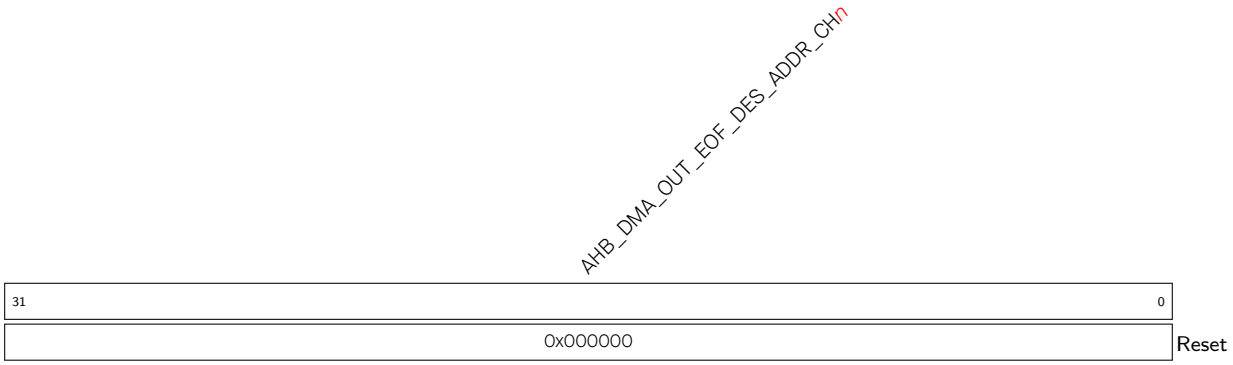


AHB_DMA_OUTLINK_DSCR_ADDR_CH*n* 表示下一个预读取（但未处理）的发送接收链表描述符所在地址的低 18 位。如果当前处理的发送链表描述符是最后一个描述符，则该字段表示当前处理的发送链表描述符地址。(RO)

AHB_DMA_OUT_DSCR_STATE_CH*n* 保留。(RO)

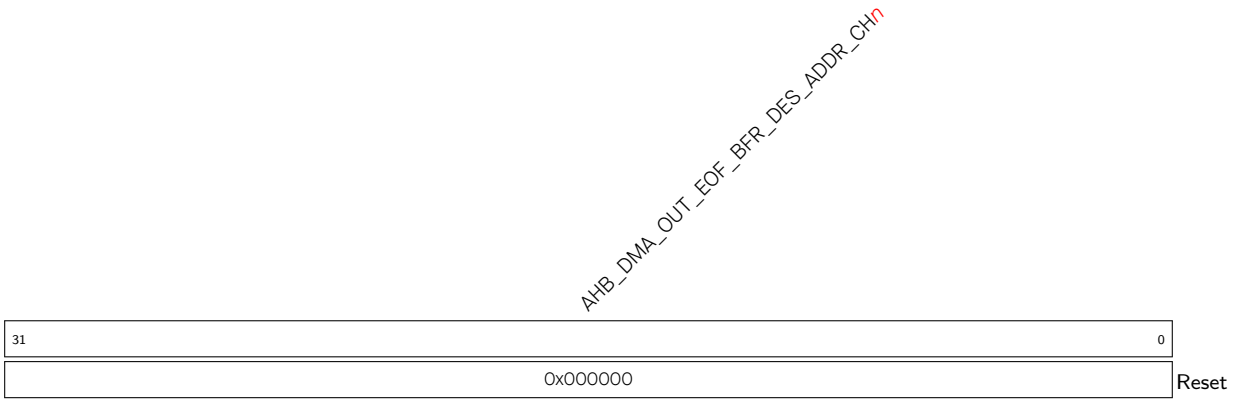
AHB_DMA_OUT_STATE_CH*n* 保留。(RO)

Register 3.41. AHB_DMA_OUT_EOF_DES_ADDR_CH*n*_REG (*n*: 0-4) (0x019C+0x100**n*)



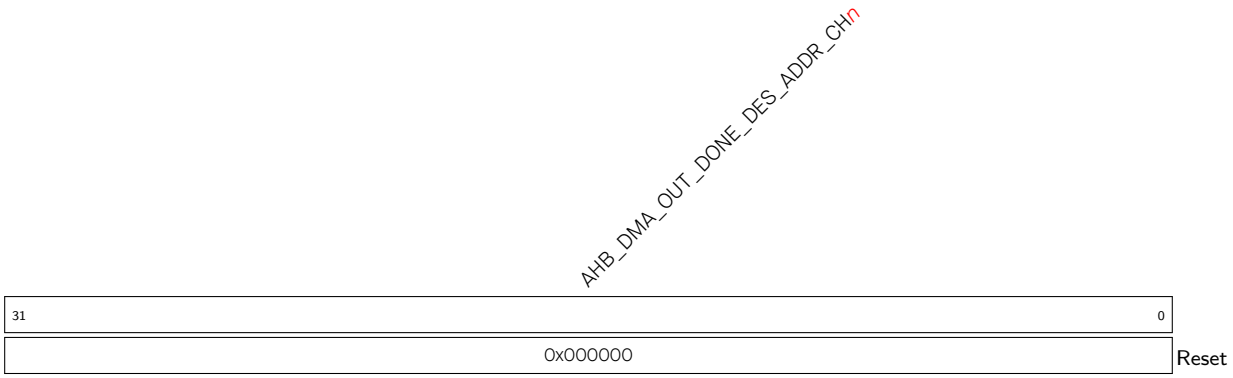
AHB_DMA_OUT_EOF_DES_ADDR_CH*n* 表示 EOF 位为 1 的发送链表描述符的地址。(RO)

Register 3.42. AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH*n*_REG (*n*: 0-4) (0x01A0+0x100*n*)



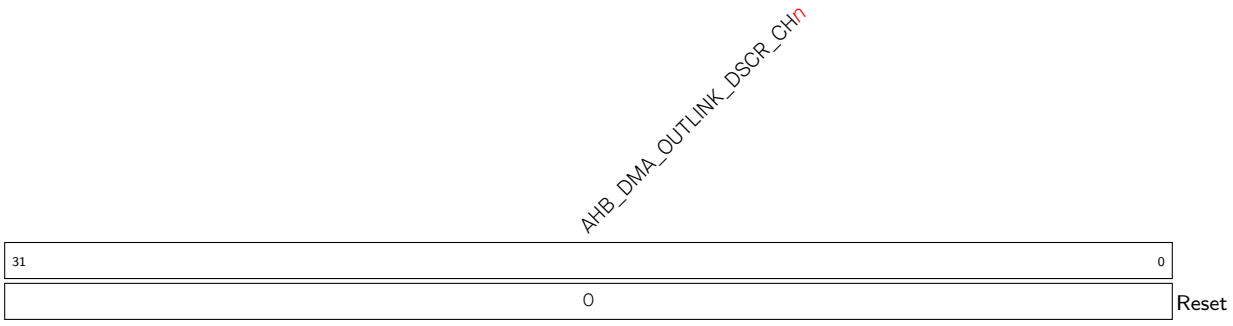
AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH*n* 表示倒数第二个发送链表描述符的地址。(RO)

Register 3.43. AHB_DMA_OUT_DONE_DES_ADDR_CH*n*_REG (*n*: 0-4) (0x01A4+0x100*n*)



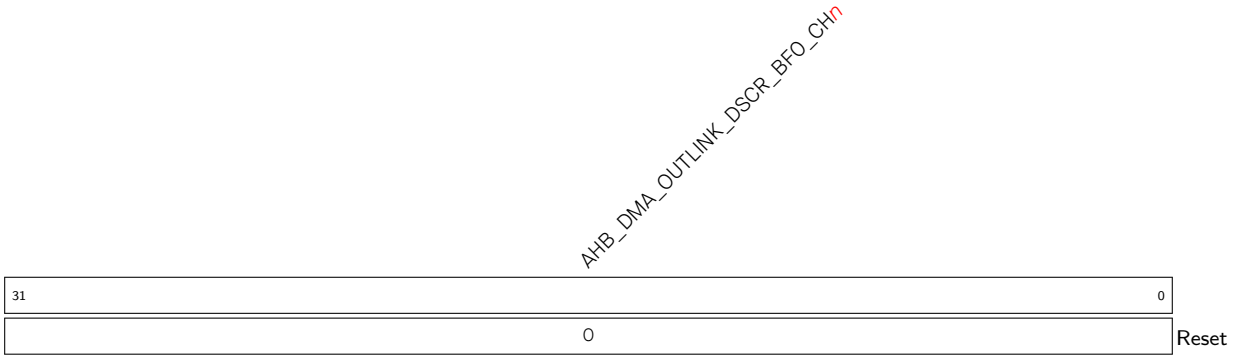
AHB_DMA_OUT_DONE_DES_ADDR_CH*n* 表示 TX 方向传输完成时对应描述符的地址。(RO)

Register 3.44. AHB_DMA_OUT_DSCR_CH*n*_REG (*n*: 0-4) (0x01A8+0x100*n*)



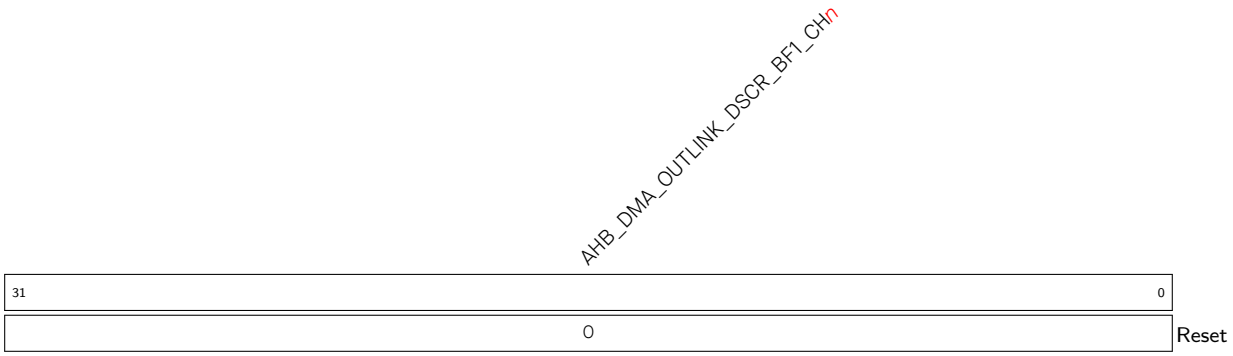
AHB_DMA_OUTLINK_DSCR_CH*n* 表示当前已预读取的发送链表描述符指向的下一个发送链表描述符地址 y+1。(RO)

Register 3.45. AHB_DMA_OUT_DSCR_BFO_CH*n*_REG (*n*: 0-4) (0x01AC+0x100*n*)



AHB_DMA_OUTLINK_DSCR_BFO_CH*n* 表示当前已预读取的发送链表描述符所在地址 y。(RO)

Register 3.46. AHB_DMA_OUT_DSCR_BF1_CH*n*_REG (*n*: 0-4) (0x01B0+0x100*n*)



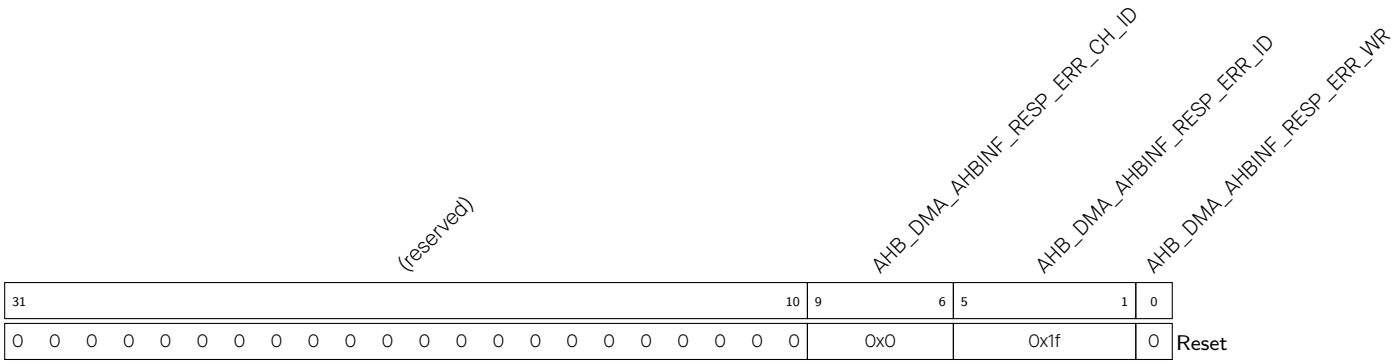
AHB_DMA_OUTLINK_DSCR_BF1_CH*n* 表示前一个已预读取的发送链表描述符所在地址 y-1。(RO)

Register 3.47. AHB_DMA_AHBINF_RESP_ERR_STATUS0_REG (0x0620)



AHB_DMA_AHBINF_RESP_ERR_ADDR 表示 AHB DMA 收到 AHB 总线错误响应对应传输的地址。(RO)

Register 3.48. AHB_DMA_AHBINF_RESP_ERR_STATUS1_REG (0x0624)



AHB_DMA_AHBINF_RESP_ERR_WR 表示 AHB 响应错误是否由写请求引起。

0：读请求

1：写请求

(RO)

AHB_DMA_AHBINF_RESP_ERR_ID 表示 AHB 响应错误请求 ID。

0：UHCI0

1：I2S0-0

2：I2S0-1

3：I2S0-2

4：I2S0-3

5：I2S1-0

6：I2S1-1

7：I2S1-2

8：I2S1-3

10：RMT

11：ASRC-0

12：ASRC-1

13：CORDIC

16：GDMA-AHB-HP 通道 0 的存储器到存储器传输

17：GDMA-AHB-HP 通道 1 的存储器到存储器传输

18：GDMA-AHB-HP 通道 2 的存储器到存储器传输

19：GDMA-AHB-HP 通道 3 的存储器到存储器传输

20：GDMA-AHB-HP 通道 4 的存储器到存储器传输

其余值无效

(RO)

AHB_DMA_AHBINF_RESP_ERR_CH_ID 表示 AHB 响应错误请求通道 ID。

位 [3]：表示通道方向

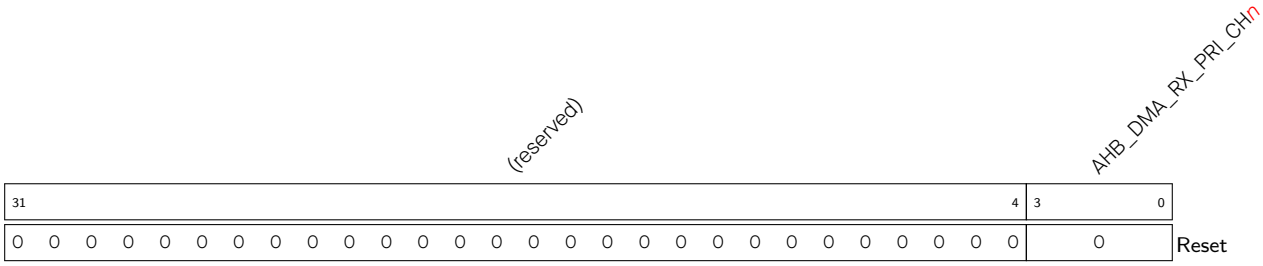
0：RX 通道

1：TX 通道

位 [0:2]：表示通道号。取值范围：0 ~4

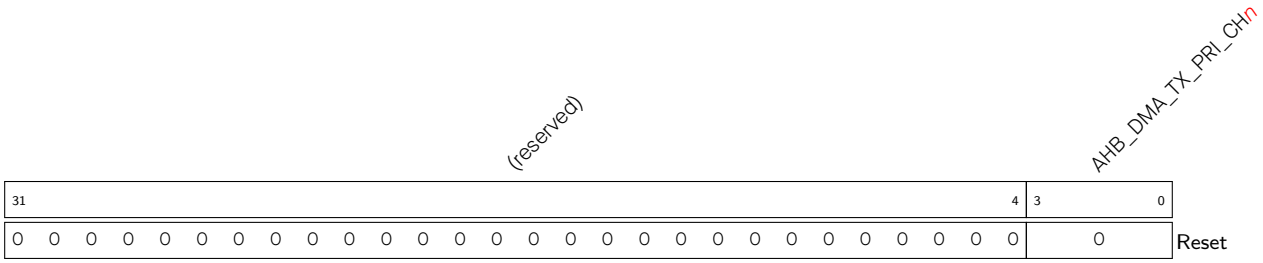
(RO)

Register 3.49. AHB_DMA_IN_PRI_CH*n*_REG (*n*: 0-4) (0x0134+0x100**n*)



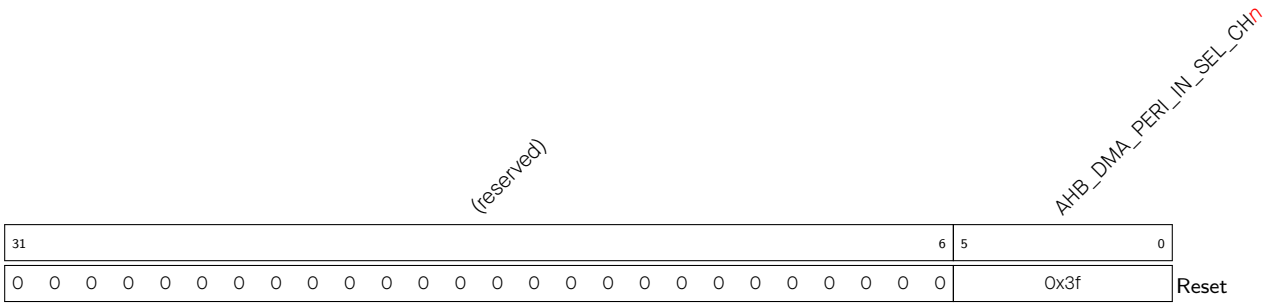
AHB_DMA_RX_PRI_CH*n* 配置 RX 通道 *n* 的优先级。数值越大，优先级越高。取值范围：0 ~ 15。(R/W)

Register 3.50. AHB_DMA_OUT_PRI_CH*n*_REG (*n*: 0-4) (0x01B4+0x100**n*)



AHB_DMA_TX_PRI_CH*n* 配置 TX 通道 *n* 的优先级。数值越大，优先级越高。取值范围：0 ~ 15。(R/W)

Register 3.51. AHB_DMA_IN_PERI_SEL_CH*n*_REG (*n*: 0-4) (0x0138+0x100**n*)

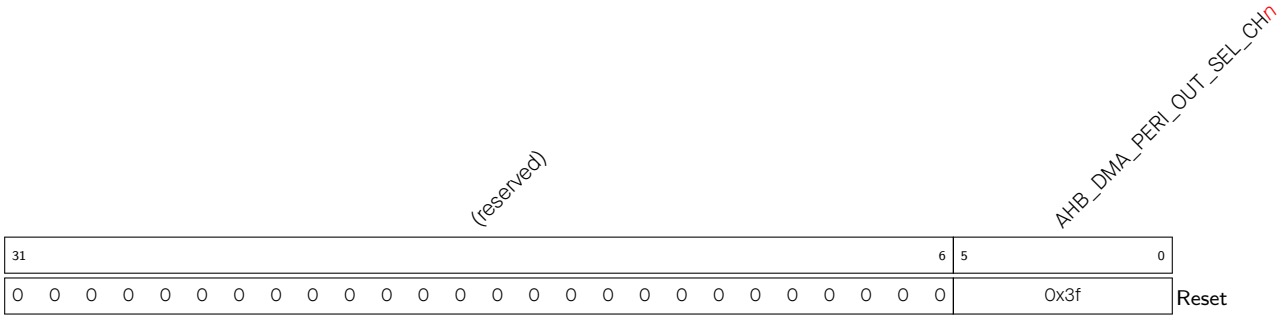


AHB_DMA_PERI_IN_SEL_CH*n* 配置连接至 RX 通道 *n* 的外设。

0：UHCI0
1：I2S0-0
2：I2S0-1
3：I2S0-2
4：I2S0-3
5：I2S1-0
6：I2S1-1
7：I2S1-2
8：I2S1-3
9：Dummy-9
10：RMT
11：ASRC-0
12：ASRC-1
13：CORDIC
14 ~ 63：无效值

(R/W)

Register 3.52. AHB_DMA_OUT_PERI_SEL_CH*n*_REG (*n*: 0-4) (0x01B8+0x100**n*)



AHB_DMA_PERI_OUT_SEL_CH*n* 配置连接至 TX 通道 *n* 的外设。

0：UHCI0
1：I2S0-0
2：I2S0-1
3：I2S0-2
4：I2S0-3
5：I2S1-0
6：I2S1-1
7：I2S1-2
8：I2S1-3
9：Dummy-9
10：RMT
11：ASRC-0
12：ASRC-1
13：CORDIC
14 ~ 63：无效值

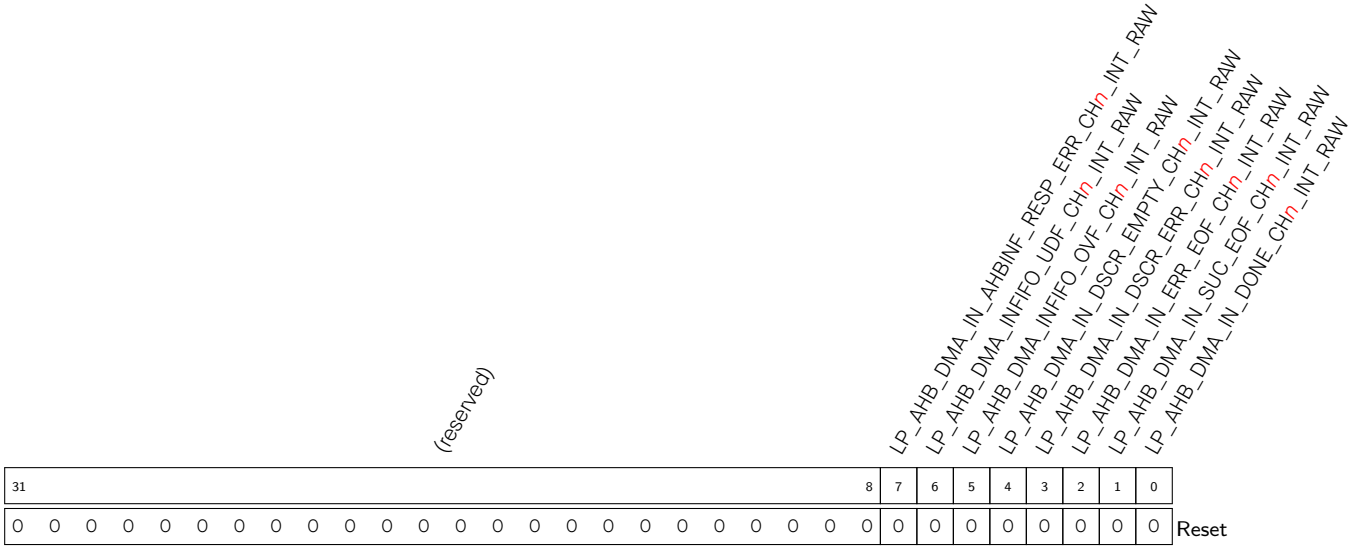
(R/W)

3.9.2 GDMA-AHB-LP 寄存器

本小节的所有地址均为相对于 GDMA-AHB-LP 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 3.53. LP_AHB_DMA_IN_INT_RAW_CH*n*_REG (*n*: 0-1) (0x0000+0x10**n*)



LP_AHB_DMA_IN_DONE_CH*n*_INT_RAW LP_AHB_DMA_IN_DONE_CH*n*_INT 的原始中断状态。(R/WTC/SS)

LP_AHB_DMA_IN_SUC_EOF_CH*n*_INT_RAW LP_AHB_DMA_IN_SUC_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

LP_AHB_DMA_IN_ERR_EOF_CH*n*_INT_RAW LP_AHB_DMA_IN_ERR_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

LP_AHB_DMA_IN_DSCR_ERR_CH*n*_INT_RAW LP_AHB_DMA_IN_DSCR_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

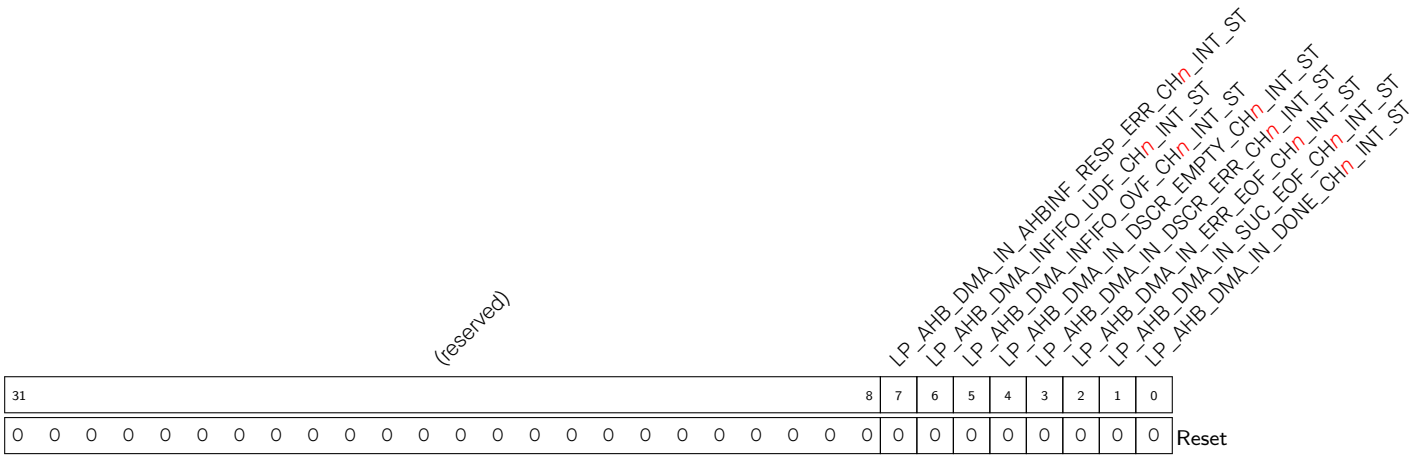
LP_AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT_RAW LP_AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT 的原始中断状态。(R/WTC/SS)

LP_AHB_DMA_INFIFO_OVF_CH*n*_INT_RAW LP_AHB_DMA_INFIFO_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

LP_AHB_DMA_INFIFO_UDF_CH*n*_INT_RAW LP_AHB_DMA_INFIFO_UDF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

LP_AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT_RAW LP_AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

Register 3.54. LP_AHB_DMA_IN_INT_ST_CH*n*_REG (*n*: 0-1) (0x0004+0x10***n*)



LP_AHB_DMA_IN_DONE_CH*n*_INT_ST LP_AHB_DMA_IN_DONE_CH*n*_INT 的屏蔽中断状态。(RO)

LP_AHB_DMA_IN_SUC_EOF_CH*n*_INT_ST LP_AHB_DMA_IN_SUC_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

LP_AHB_DMA_IN_ERR_EOF_CH*n*_INT_ST LP_AHB_DMA_IN_ERR_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

LP_AHB_DMA_IN_DSCR_ERR_CH*n*_INT_ST LP_AHB_DMA_IN_DSCR_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

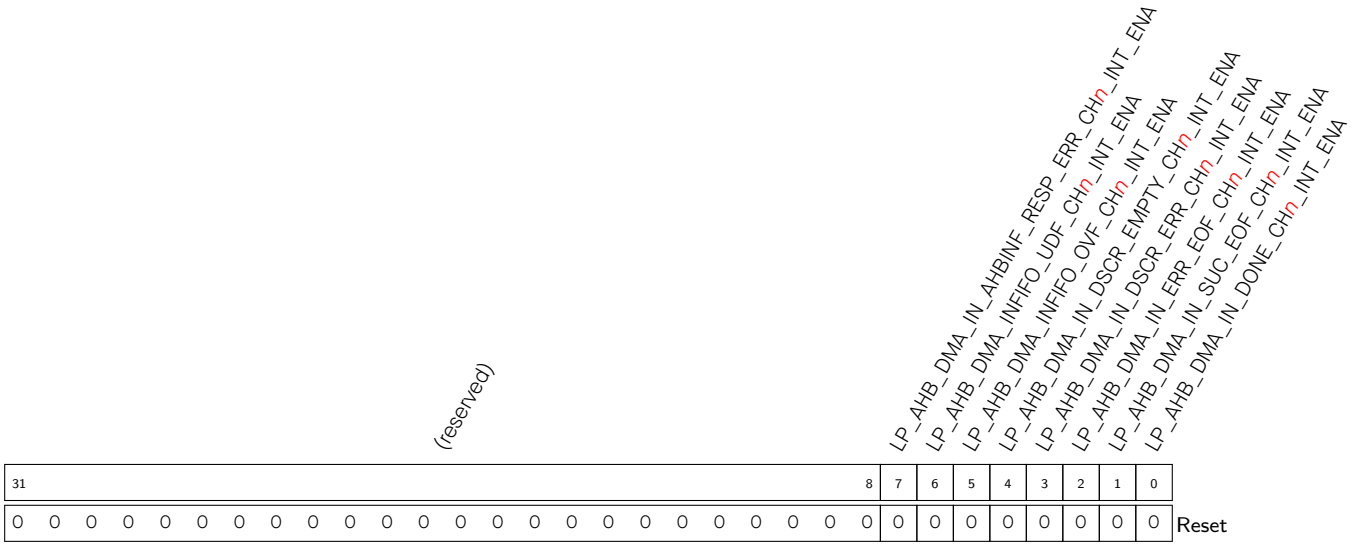
LP_AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT_ST LP_AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT 的屏蔽中断状态。(RO)

LP_AHB_DMA_INFIFO_OVF_CH*n*_INT_ST LP_AHB_DMA_INFIFO_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

LP_AHB_DMA_INFIFO_UDF_CH*n*_INT_ST LP_AHB_DMA_INFIFO_UDF_CH*n*_INT 的屏蔽中断状态。(RO)

LP_AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT_ST LP_AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

Register 3.55. LP_AHB_DMA_IN_INT_ENA_CH*n*_REG (*n*: 0-1) (0x0008+0x10**n*)



LP_AHB_DMA_IN_DONE_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_IN_DONE_CH*n*_INT。(R/W)

LP_AHB_DMA_IN_SUC_EOF_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_IN_SUC_EOF_CH*n*_INT。(R/W)

LP_AHB_DMA_IN_ERR_EOF_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_IN_ERR_EOF_CH*n*_INT。(R/W)

LP_AHB_DMA_IN_DSCR_ERR_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_IN_DSCR_ERR_CH*n*_INT。(R/W)

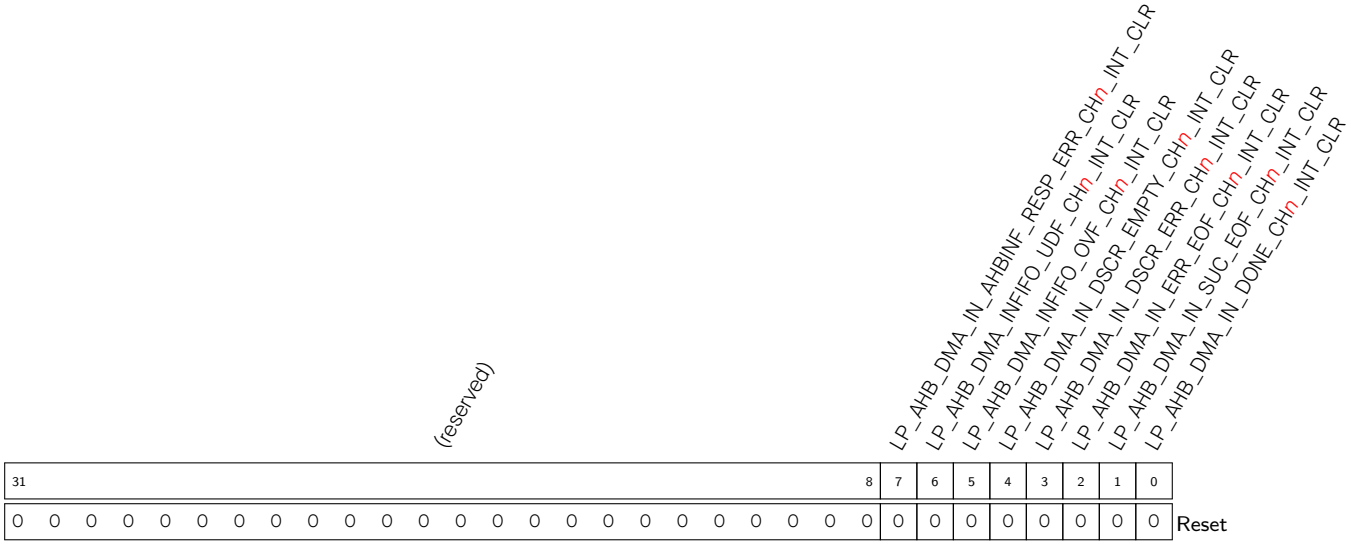
LP_AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT。(R/W)

LP_AHB_DMA_INFIFO_OVF_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_INFIFO_OVF_CH*n*_INT。(R/W)

LP_AHB_DMA_INFIFO_UDF_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_INFIFO_UDF_CH*n*_INT。(R/W)

LP_AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT。(R/W)

Register 3.56. LP_AHB_DMA_IN_INT_CLR_CH*n*_REG (*n*: 0-1) (0x000C+0x10***n*)



LP_AHB_DMA_IN_DONE_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_IN_DONE_CH*n*_INT。(WT)

LP_AHB_DMA_IN_SUC_EOF_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_IN_SUC_EOF_CH*n*_INT。(WT)

LP_AHB_DMA_IN_ERR_EOF_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_IN_ERR_EOF_CH*n*_INT。(WT)

LP_AHB_DMA_IN_DSCR_ERR_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_IN_DSCR_ERR_CH*n*_INT。(WT)

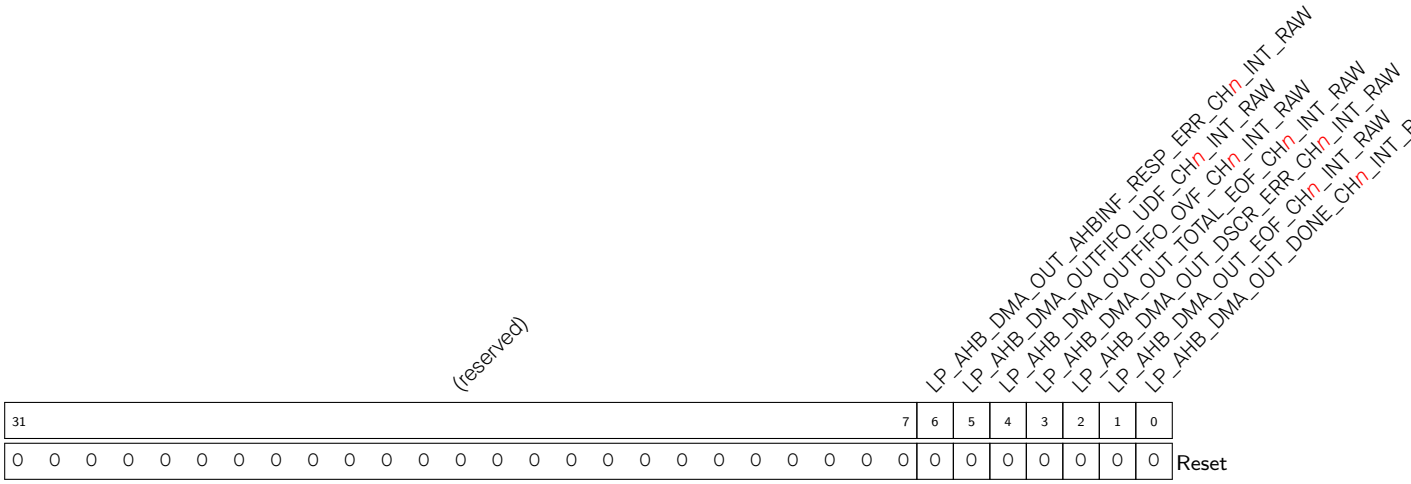
LP_AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_IN_DSCR_EMPTY_CH*n*_INT。(WT)

LP_AHB_DMA_INFIFO_OVF_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_INFIFO_OVF_CH*n*_INT。(WT)

LP_AHB_DMA_INFIFO_UDF_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_INFIFO_UDF_CH*n*_INT。(WT)

LP_AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_IN_AHBINF_RESP_ERR_CH*n*_INT。(WT)

Register 3.57. LP_AHB_DMA_OUT_INT_RAW_CH*n*_REG (*n*: 0-1) (0x0050+0x10*n*)



LP_AHB_DMA_OUT_DONE_CH*n*_INT_RAW LP_AHB_DMA_OUT_DONE_CH*n*_INT 的原始中断状态。(R/WTC/SS)

LP_AHB_DMA_OUT_EOF_CH*n*_INT_RAW LP_AHB_DMA_OUT_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

LP_AHB_DMA_OUT_DSCR_ERR_CH*n*_INT_RAW LP_AHB_DMA_OUT_DSCR_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

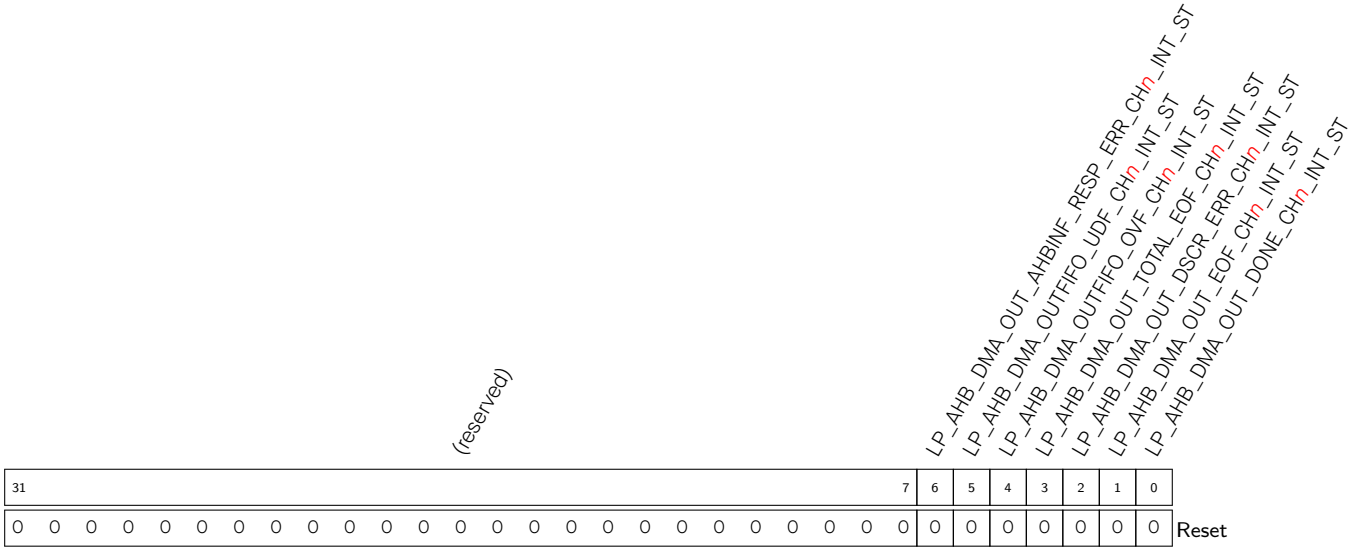
LP_AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT_RAW LP_AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

LP_AHB_DMA_OUTFIFO_OVF_CH*n*_INT_RAW LP_AHB_DMA_OUTFIFO_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

LP_AHB_DMA_OUTFIFO_UDF_CH*n*_INT_RAW LP_AHB_DMA_OUTFIFO_UDF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

LP_AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT_RAW LP_AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

Register 3.58. LP_AHB_DMA_OUT_INT_ST_CH*n*_REG (*n*: 0-1) (0x0054+0x10**n*)



LP_AHB_DMA_OUT_DONE_CH*n*_INT_ST LP_AHB_DMA_OUT_DONE_CH*n*_INT 的屏蔽中断状态。(RO)

LP_AHB_DMA_OUT_EOF_CH*n*_INT_ST LP_AHB_DMA_OUT_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

LP_AHB_DMA_OUT_DSCR_ERR_CH*n*_INT_ST LP_AHB_DMA_OUT_DSCR_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

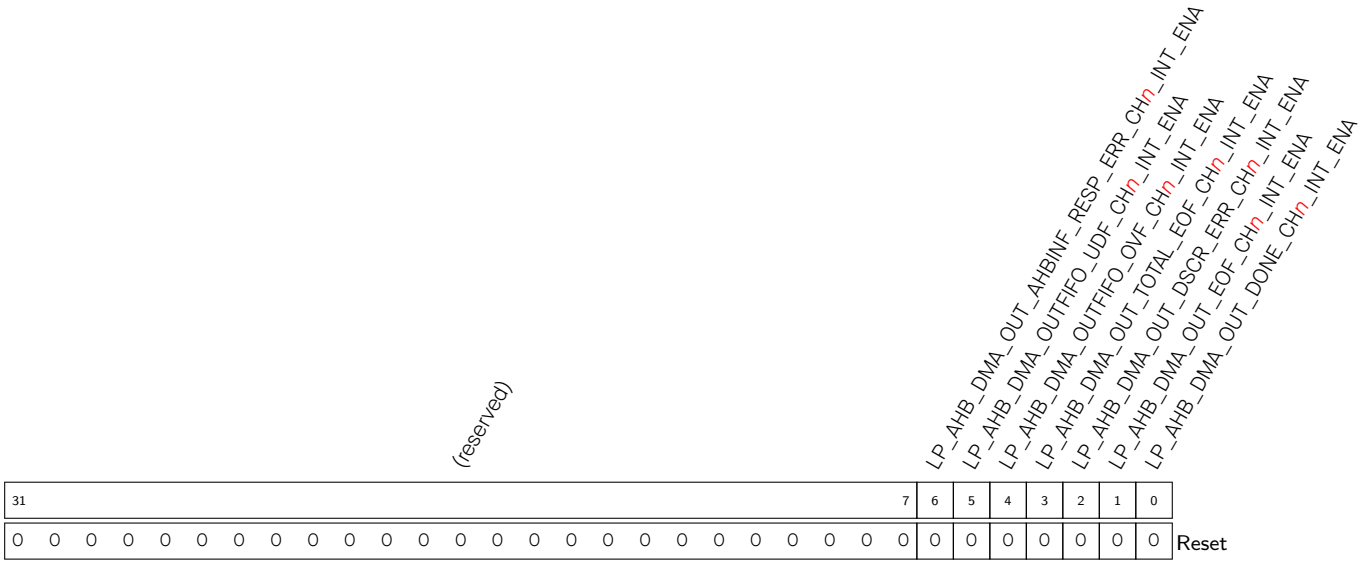
LP_AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT_ST LP_AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

LP_AHB_DMA_OUTFIFO_OVF_CH*n*_INT_ST LP_AHB_DMA_OUTFIFO_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

LP_AHB_DMA_OUTFIFO_UDF_CH*n*_INT_ST LP_AHB_DMA_OUTFIFO_UDF_CH*n*_INT 的屏蔽中断状态。(RO)

LP_AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT_ST LP_AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

Register 3.59. LP_AHB_DMA_OUT_INT_ENA_CH*n*_REG (*n*: 0-1) (0x0058+0x10**n*)



LP_AHB_DMA_OUT_DONE_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_OUT_DONE_CH*n*_INT。(R/W)

LP_AHB_DMA_OUT_EOF_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_OUT_EOF_CH*n*_INT。(R/W)

LP_AHB_DMA_OUT_DSCR_ERR_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_OUT_DSCR_ERR_CH*n*_INT。(R/W)

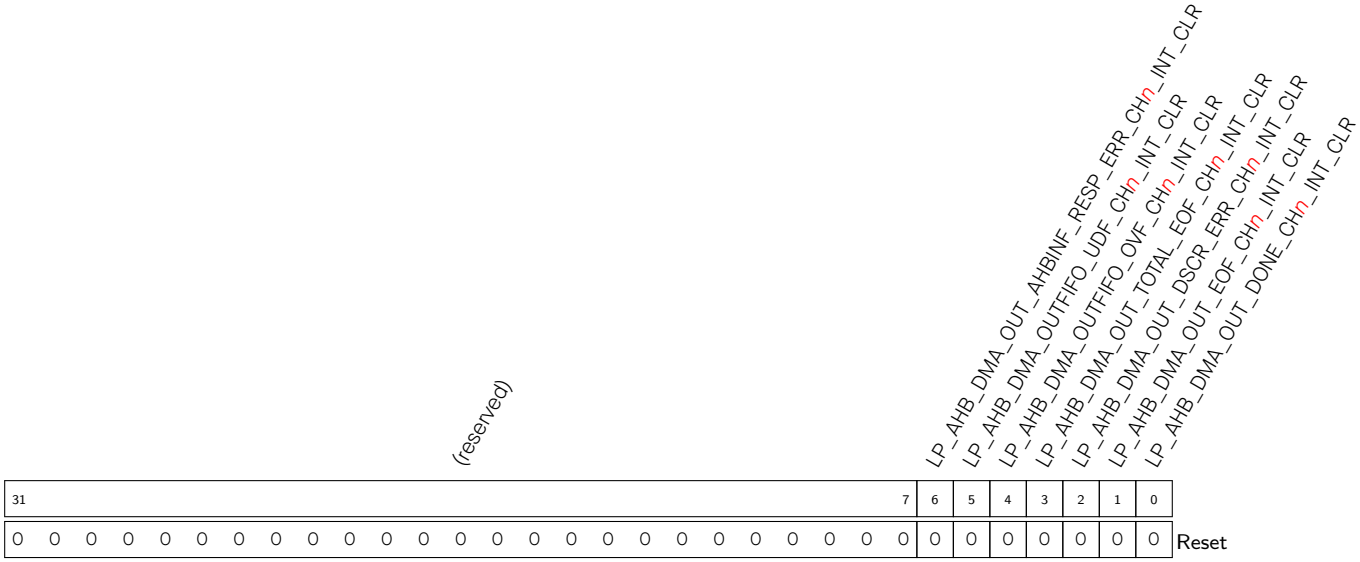
LP_AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT。(R/W)

LP_AHB_DMA_OUTFIFO_OVF_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_OUTFIFO_OVF_CH*n*_INT。(R/W)

LP_AHB_DMA_OUTFIFO_UDF_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_OUTFIFO_UDF_CH*n*_INT。(R/W)

LP_AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT_ENA 写 1 使能 LP_AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT。(R/W)

Register 3.60. LP_AHB_DMA_OUT_INT_CLR_CH*n*_REG (*n*: 0-1) (0x005C+0x10*n*)



LP_AHB_DMA_OUT_DONE_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_OUT_DONE_CH*n*_INT。(WT)

LP_AHB_DMA_OUT_EOF_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_OUT_EOF_CH*n*_INT。(WT)

LP_AHB_DMA_OUT_DSCR_ERR_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_OUT_DSCR_ERR_CH*n*_INT。(WT)

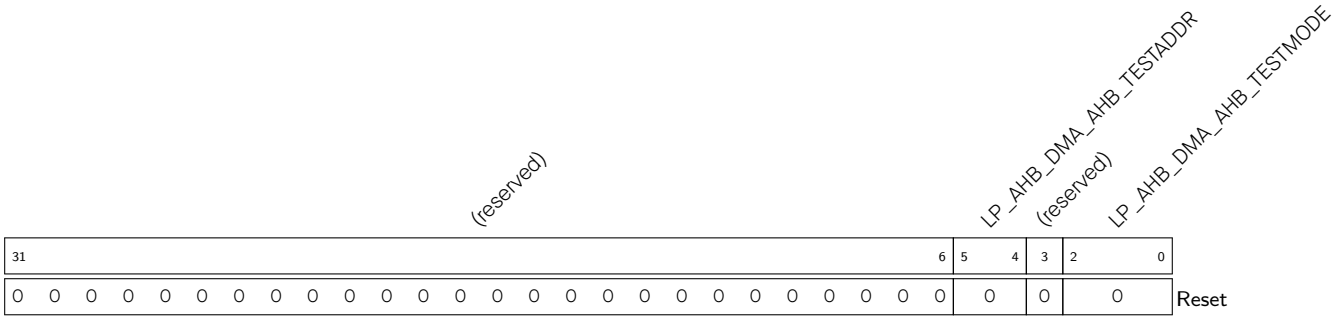
LP_AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_OUT_TOTAL_EOF_CH*n*_INT。(WT)

LP_AHB_DMA_OUTFIFO_OVF_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_OUTFIFO_OVF_CH*n*_INT。(WT)

LP_AHB_DMA_OUTFIFO_UDF_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_OUTFIFO_UDF_CH*n*_INT。(WT)

LP_AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT_CLR 写 1 清除 LP_AHB_DMA_OUT_AHBINF_RESP_ERR_CH*n*_INT。(WT)

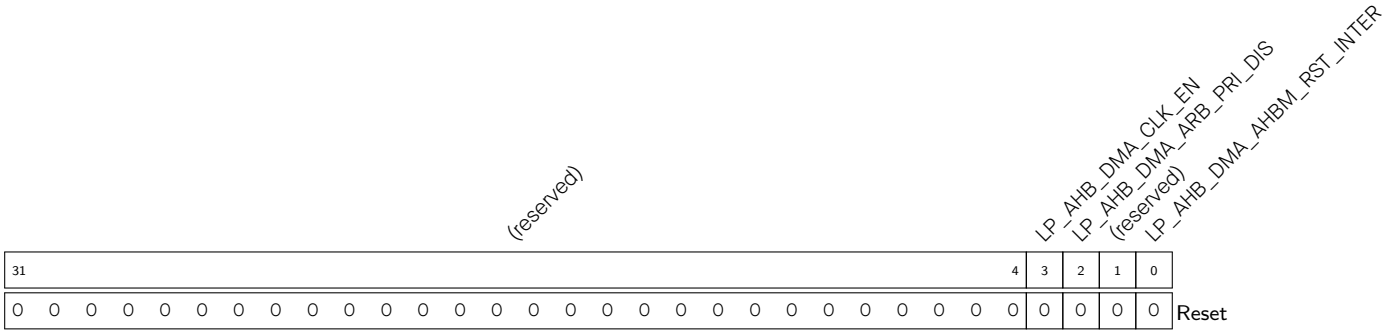
Register 3.61. LP_AHB_DMA_AHB_TEST_REG (0x00A0)



LP_AHB_DMA_AHB_TESTMODE 保留。(R/W)

LP_AHB_DMA_AHB_TESTADDR 保留。(R/W)

Register 3.62. LP_AHB_DMA_MISC_CONF_REG (0x00A4)



LP_AHB_DMA_AHBM_RST_INTER 配置内部 LP AHB FSM 的复位状态。

0：解除复位

1：复位

(R/W)

LP_AHB_DMA_ARB_PRI_DIS 配置是否禁用固定优先级通道仲裁。

0：使能

1：禁用

(R/W)

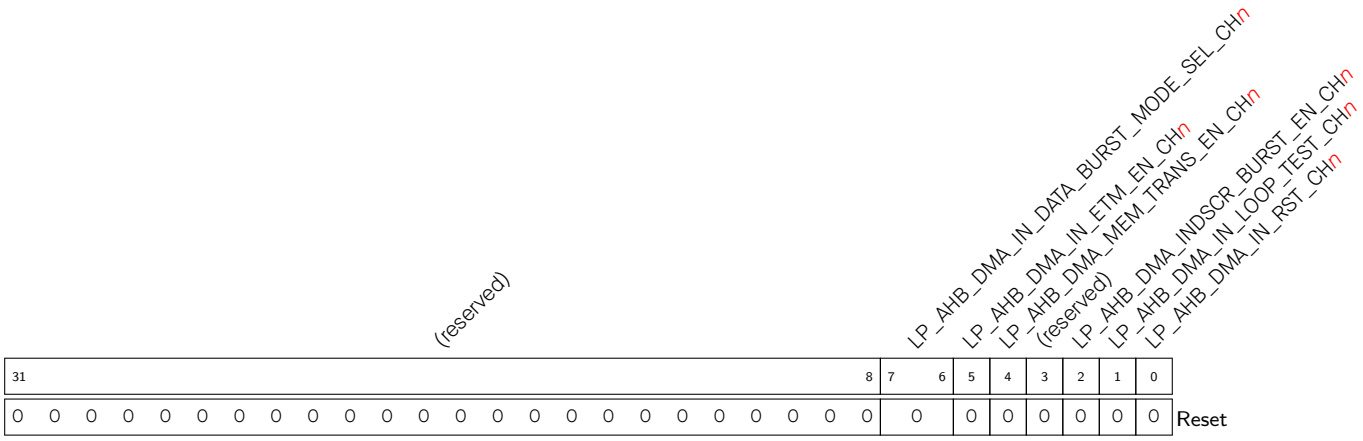
LP_AHB_DMA_CLK_EN 配置时钟门控。

0：仅在应用写寄存器时提供开启时钟

1：始终强制为寄存器开启时钟

(R/W)

Register 3.63. LP_AHB_DMA_IN_CONF0_CH*n*_REG (*n*: 0-1) (0x0100+0x100**n*)



LP_AHB_DMA_IN_RST_CH*n* 配置 LP AHB DMA RX 通道 *n* FSM 和 FIFO 指针的复位状态。

0：解除复位

1：复位

(R/W)

LP_AHB_DMA_IN_LOOP_TEST_CH*n* 配置是否关闭 RX 通道 *n* 的 owner 位自动清零。

0：无效值。没有作用

1：关闭

(R/W)

LP_AHB_DMA_INDSCR_BURST_EN_CH*n* 配置是否使能 RX 通道 *n* 读取接收链表描述符的 INCR 突发传输。

0：禁用

1：使能

(R/W)

LP_AHB_DMA_MEM_TRANS_EN_CH*n* 配置是否使能存储器到存储器数据传输。

0：禁用

1：使能

(R/W)

LP_AHB_DMA_IN_ETM_EN_CH*n* 配置是否使能 RX 通道 *n* 的 ETM 控制。

0：禁用

1：使能

(R/W)

LP_AHB_DMA_IN_DATA_BURST_MODE_SEL_CH*n* 配置 RX 通道 *n* 的突发传输模式。

0：单次

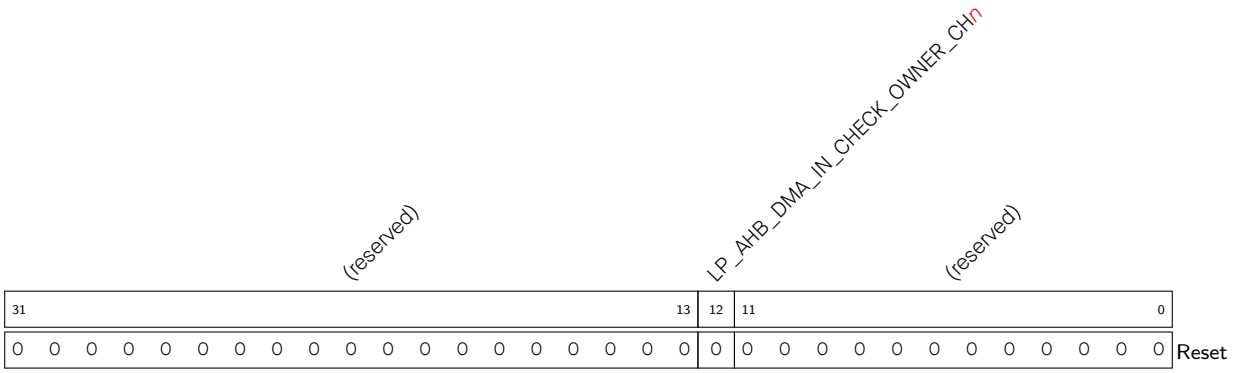
1：INCR4

2：INCR8

3：INCR16

(R/W)

Register 3.64. LP_AHB_DMA_IN_CONF1_CH*n*_REG (*n*: 0-1) (0x0104+0x100*n*)



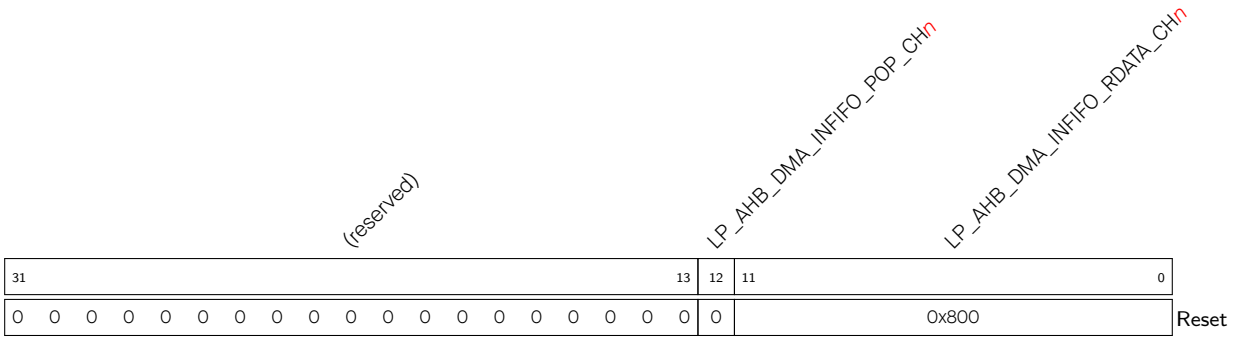
LP_AHB_DMA_IN_CHECK_OWNER_CH*n* 配置是否使能 RX 通道 *n* 的 owner 位检查。

0：禁用

1：使能

(R/W)

Register 3.65. LP_AHB_DMA_IN_POP_CH*n*_REG (*n*: 0-1) (0x010C+0x100*n*)



LP_AHB_DMA_INFIFO_RDATA_CH*n* 从 LP AHB DMA FIFO 接收的数据。(RO)

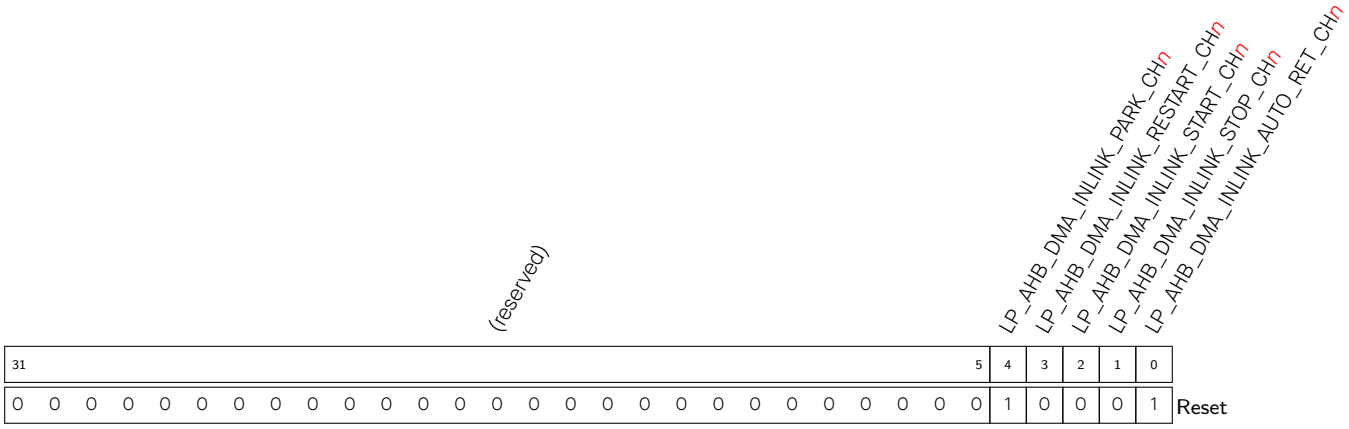
LP_AHB_DMA_INFIFO_POP_CH*n* 配置是否从 LP AHB DMA FIFO 为 RX 通道 *n* 弹出数据。

0：无效值。没有作用

1：弹出

(WT)

Register 3.66. LP_AHB_DMA_IN_LINK_CH*n*_REG (*n*: 0-1) (0x0110+0x100**n*)



LP_AHB_DMA_INLINK_AUTO_RET_CH*n* 配置当 RX 通道 *n* 当前接收数据发生错误时，是否返回到当前接收链表描述符地址。

0：不返回

1：返回

(R/W)

LP_AHB_DMA_INLINK_STOP_CH*n* 配置是否停止处理 RX 通道 *n* 的描述符。

0：无效值。没有作用

1：停止

(WT)

LP_AHB_DMA_INLINK_START_CH*n* 配置是否开始处理 RX 通道 *n* 的描述符。

0：无效值。没有作用

1：开始

(WT)

LP_AHB_DMA_INLINK_RESTART_CH*n* 配置是否重新开始处理 RX 通道 *n* 的描述符。

0：无效值。没有作用

1：重新开始

(WT)

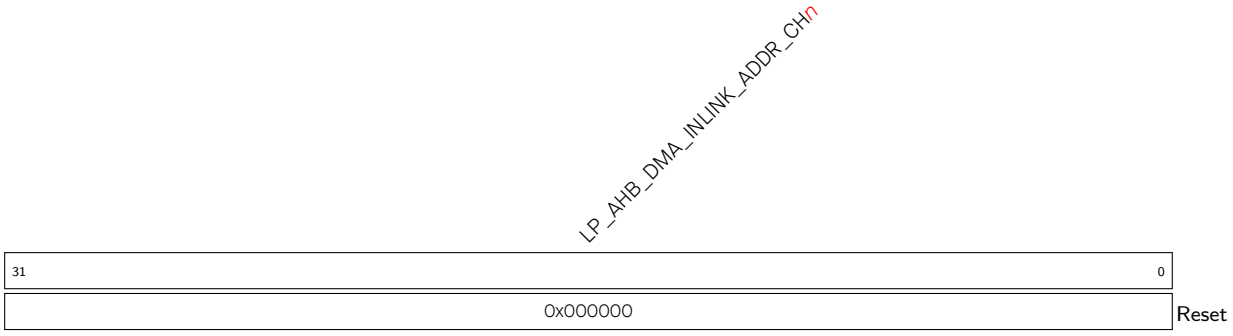
LP_AHB_DMA_INLINK_PARK_CH*n* 表示 RX 通道 *n* 接收链表描述符 FSM 的状态。

0：运行

1：空闲

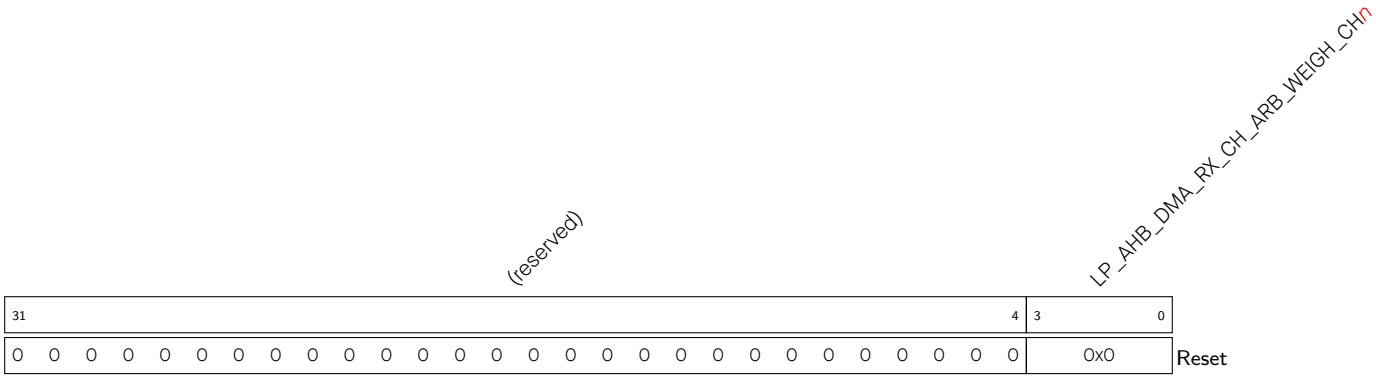
(RO)

Register 3.67. LP_AHB_DMA_IN_LINK_ADDR_CH*n*_REG (*n*: 0-1) (0x0114+0x100*n*)



LP_AHB_DMA_INLINK_ADDR_CH*n* 配置 RX 通道 *n* 的第一个接收链表描述符地址。(R/W)

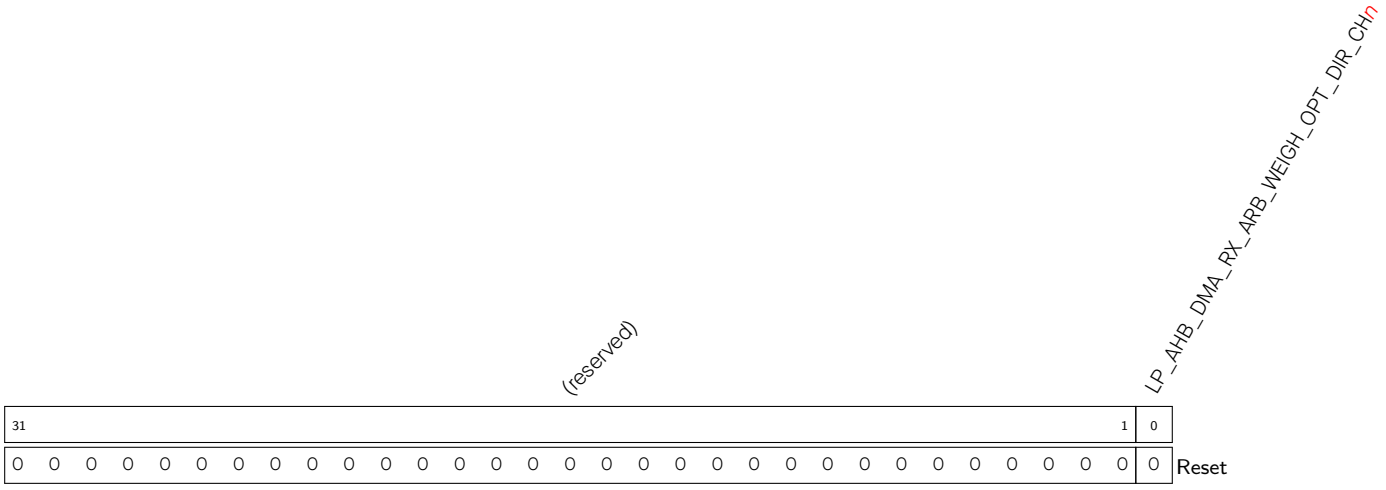
Register 3.68. LP_AHB_DMA_RX_CH_ARB_WEIGH_CH*n*_REG (*n*: 0-1) (0x013C+0x100*n*)



LP_AHB_DMA_RX_CH_ARB_WEIGH_CH*n* 配置 RX 通道 *n* 的仲裁权重。

单位：令牌。(R/W)

Register 3.69. LP_AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH*n*_REG (*n*: 0-1) (0x0140+0x100**n*)



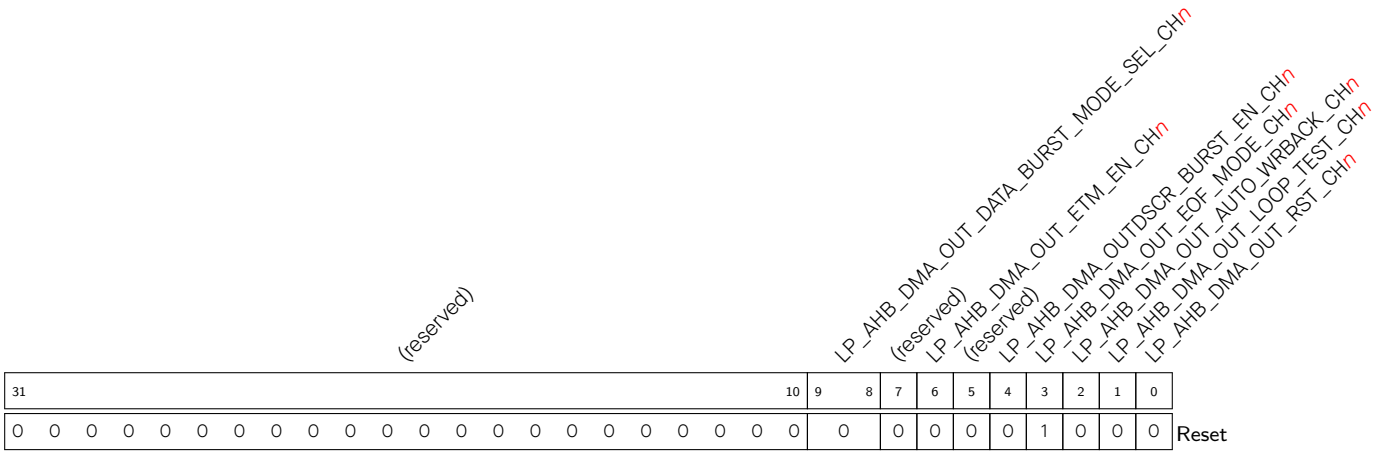
LP_AHB_DMA_RX_ARB_WEIGH_OPT_DIR_CH*n* 配置接收通道 *n* 是否开启权重仲裁优化。

0：开启

1：关闭

(R/W)

Register 3.70. LP_AHB_DMA_OUT_CONF0_CH*n*_REG (*n*: 0-1) (0x0180+0x100***n*)



LP_AHB_DMA_OUT_RST_CH*n* 配置 LP AHB DMA TX 通道 *n* FSM 和 FIFO 指针的复位状态。

0：解除复位

1：复位

(R/W)

LP_AHB_DMA_OUT_LOOP_TEST_CH*n* 配置是否关闭 TX 通道 *n* 的 owner 位自动清零。

0：无效值。没有作用

1：关闭

(R/W)

LP_AHB_DMA_OUT_AUTO_WRBACK_CH*n* 配置当 TX 通道 *n* 的 TX FIFO 中所有数据发送完成时，是否使能自动 outlink 回写。

0：禁用

1：使能

(R/W)

LP_AHB_DMA_OUT_EOF_MODE_CH*n* 配置 TX 通道 *n* EOF 标志的生成时机。

0：待发送数据推入 FIFO 时生成

1：待发送数据从 FIFO 弹出时生成

(R/W)

LP_AHB_DMA_OUTDSCR_BURST_EN_CH*n* 配置是否使能 TX 通道 *n* 读取发送链表描述符的 INCR 突发传输。

0：禁用

1：使能

(R/W)

LP_AHB_DMA_OUT_ETM_EN_CH*n* 配置是否使能 TX 通道 *n* 的 ETM 控制。

0：禁用

1：使能

(R/W)

LP_AHB_DMA_OUT_DATA_BURST_MODE_SEL_CH*n* 配置 TX 通道 *n* 的突发传输模式。

0：单次

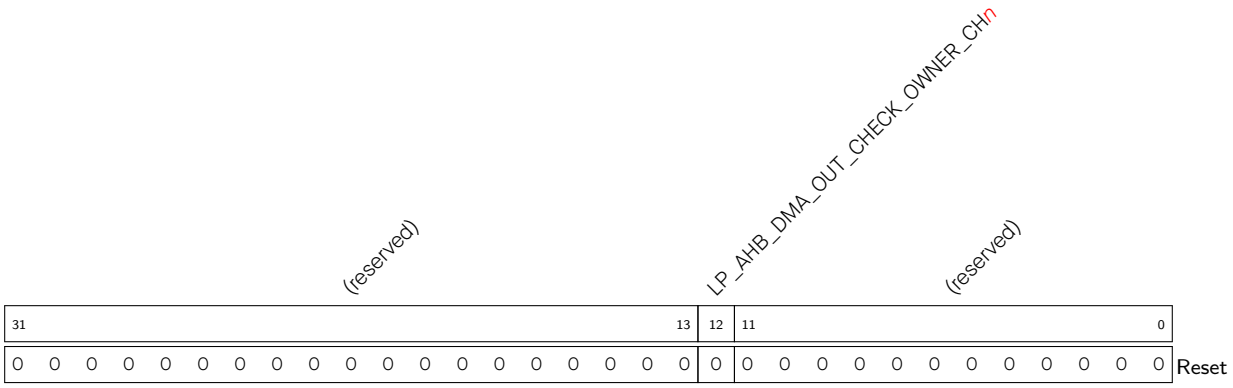
1：INCR4

2：INCR8

3：INCR16

(R/W)

Register 3.71. LP_AHB_DMA_OUT_CONF1_CH*n*_REG (*n*: 0-1) (0x0184+0x100**n*)



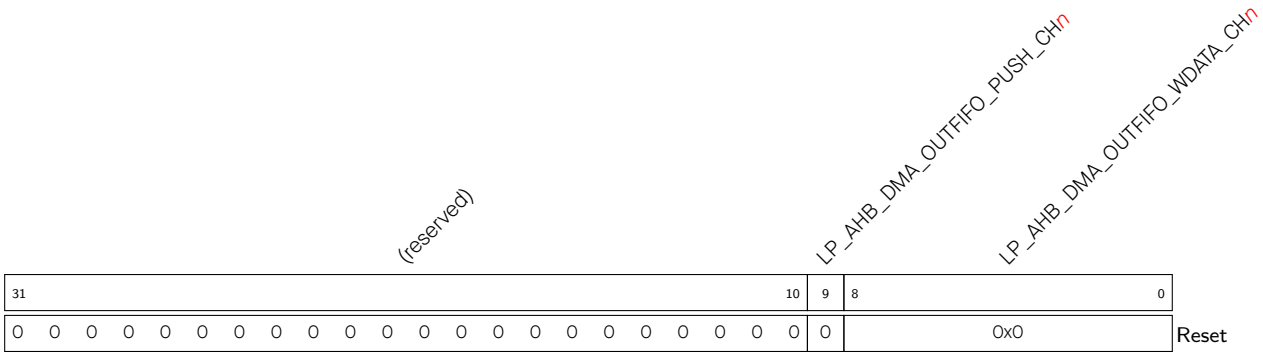
LP_AHB_DMA_OUT_CHECK_OWNER_CH*n* 配置是否使能 TX 通道 *n* 的 owner 位检查。

0：禁用

1：使能

(R/W)

Register 3.72. LP_AHB_DMA_OUT_PUSH_CH*n*_REG (*n*: 0-1) (0x018C+0x100**n*)



LP_AHB_DMA_OUTFIFO_WDATA_CH*n* 待传输至 LP AHB DMA FIFO 的数据。(R/W)

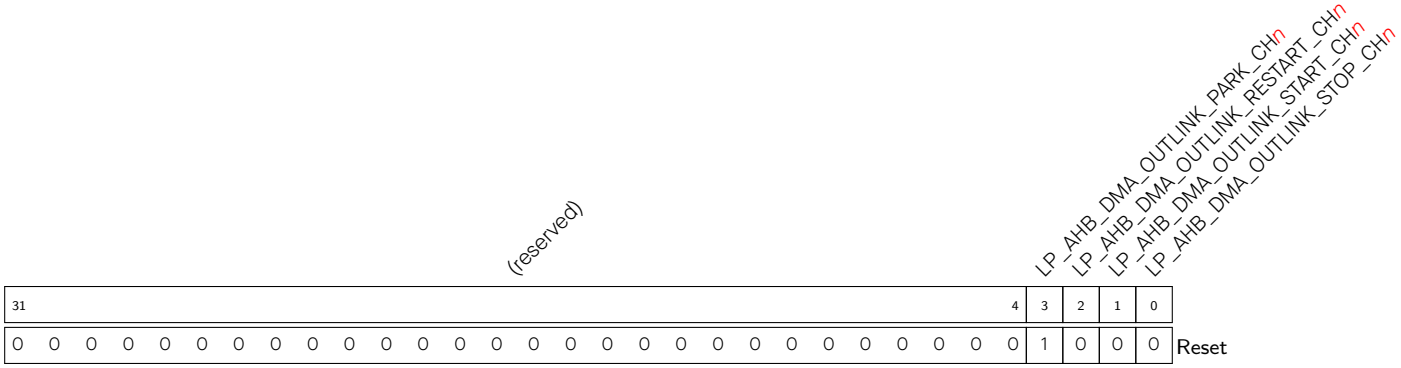
LP_AHB_DMA_OUTFIFO_PUSH_CH*n* 配置是否将数据推入 LP AHB DMA FIFO（TX 通道 *n*）。

0：无效值。没有作用

1：推入

(WT)

Register 3.73. LP_AHB_DMA_OUT_LINK_CH*n*_REG (*n*: 0-1) (0x0190+0x100*n*)



LP_AHB_DMA_OUTLINK_STOP_CH*n* 配置是否停止处理 TX 通道 *n* 的描述符。

0：无效值。没有作用

1：停止

(WT)

LP_AHB_DMA_OUTLINK_START_CH*n* 配置是否开始处理 TX 通道 *n* 的描述符。

0：无效值。没有作用

1：开始

(WT)

LP_AHB_DMA_OUTLINK_RESTART_CH*n* 配置是否重新开始处理 TX 通道 *n* 的描述符。

0：无效值。没有作用

1：重新开始

(WT)

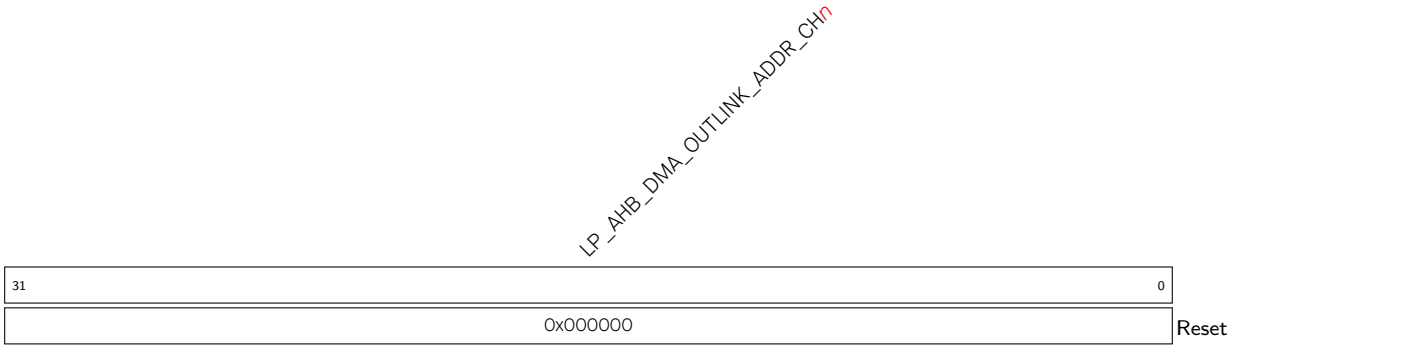
LP_AHB_DMA_OUTLINK_PARK_CH*n* 表示 TX 通道 *n* 发送链表描述符 FSM 的状态。

0：运行

1：空闲

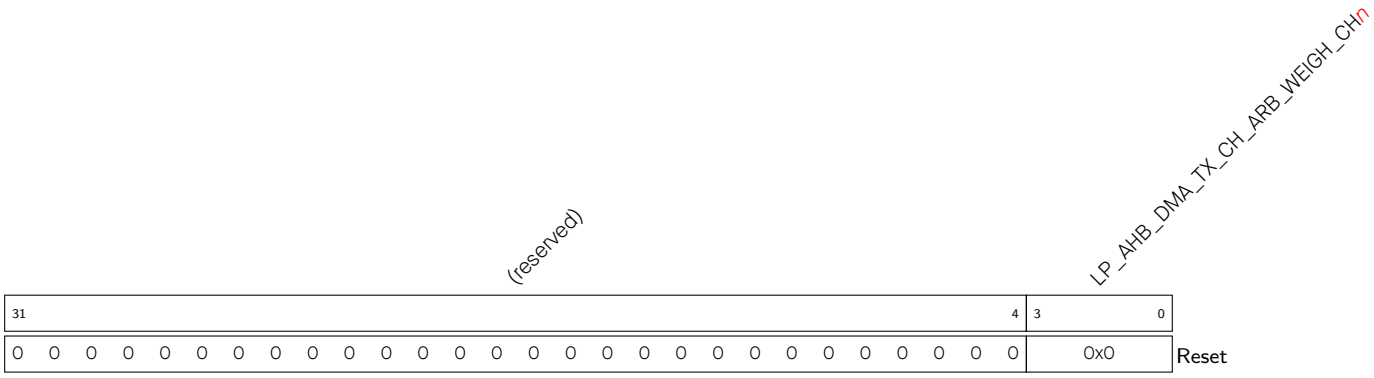
(RO)

Register 3.74. LP_AHB_DMA_OUT_LINK_ADDR_CH*n*_REG (*n*: 0-1) (0x0194+0x100*n*)



LP_AHB_DMA_OUTLINK_ADDR_CH*n* 配置 TX 通道 *n* 的第一个发送链表描述符地址。(R/W)

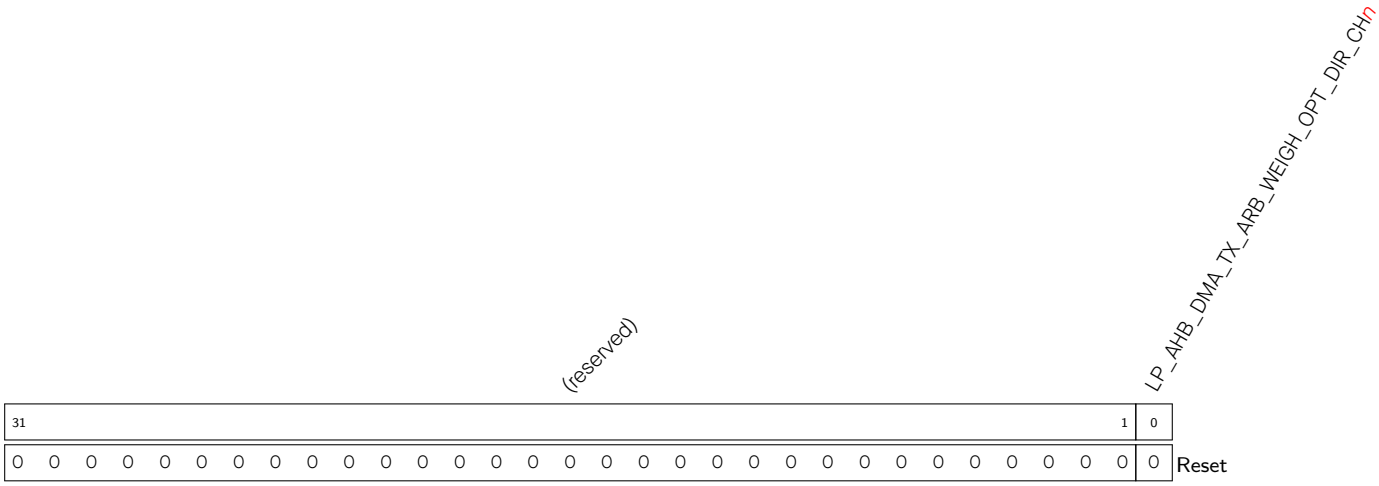
Register 3.75. LP_AHB_DMA_TX_CH_ARB_WEIGH_CH*n*_REG (*n*: 0-1) (0x01BC+0x100**n*)



LP_AHB_DMA_TX_CH_ARB_WEIGH_CH*n* 配置 TX 通道 *n* 的仲裁权重。

单位：令牌。(R/W)

Register 3.76. LP_AHB_DMA_TX_ARB_WEIGH_OPT_DIR_CH*n*_REG (*n*: 0-1) (0x01C0+0x100**n*)



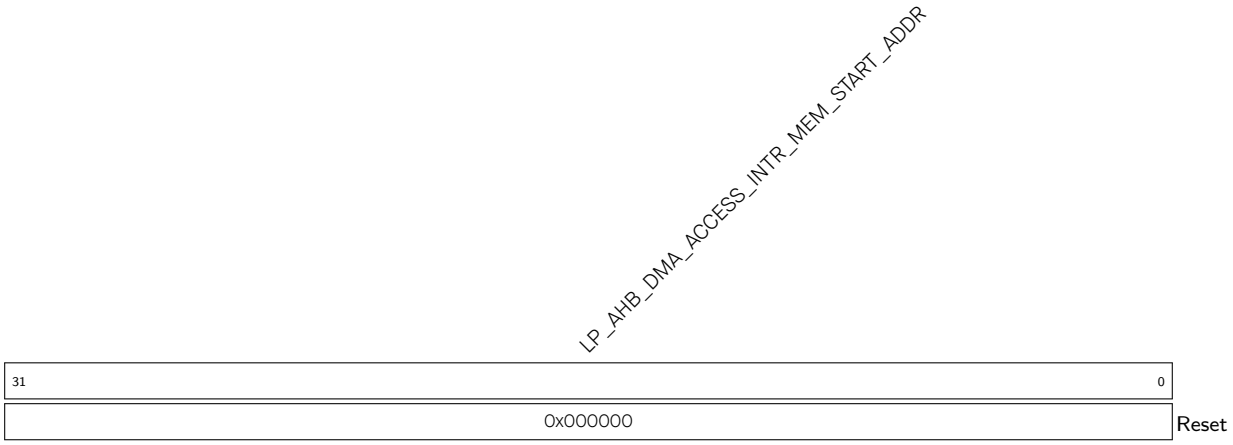
LP_AHB_DMA_TX_ARB_WEIGH_OPT_DIR_CH*n* 配置发送通道 *n* 是否开启权重仲裁优化。

0：开启

1：关闭

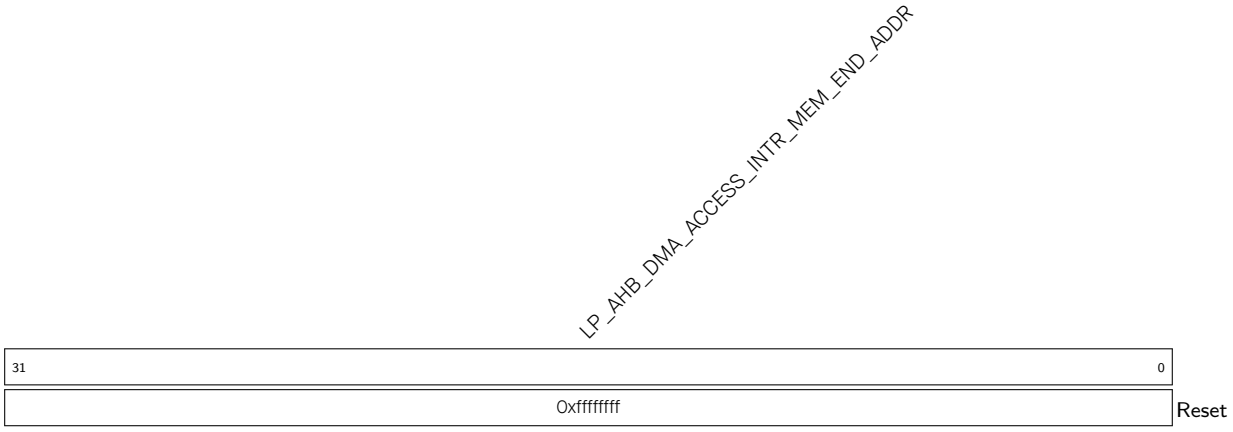
(R/W)

Register 3.77. LP_AHB_DMA_INTR_MEM_START_ADDR_REG (0x0600)



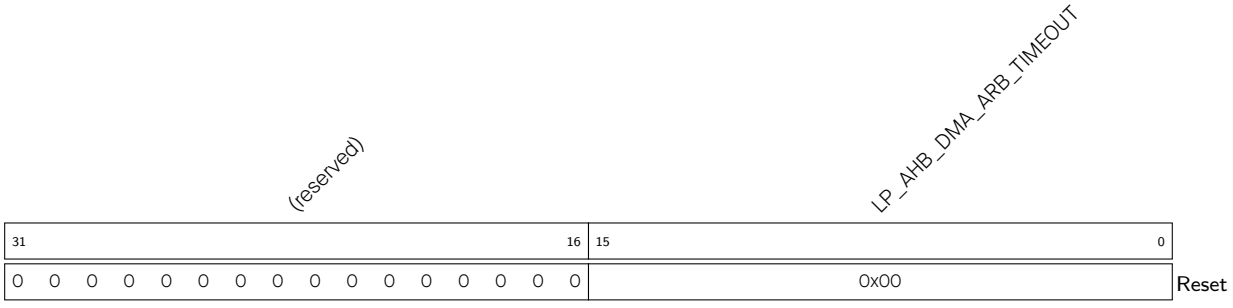
LP_AHB_DMA_ACCESS_INTR_MEM_START_ADDR 配置可访问地址空间的起始地址。(R/W)

Register 3.78. LP_AHB_DMA_INTR_MEM_END_ADDR_REG (0x0604)



LP_AHB_DMA_ACCESS_INTR_MEM_END_ADDR 配置可访问地址空间的结束地址。(R/W)

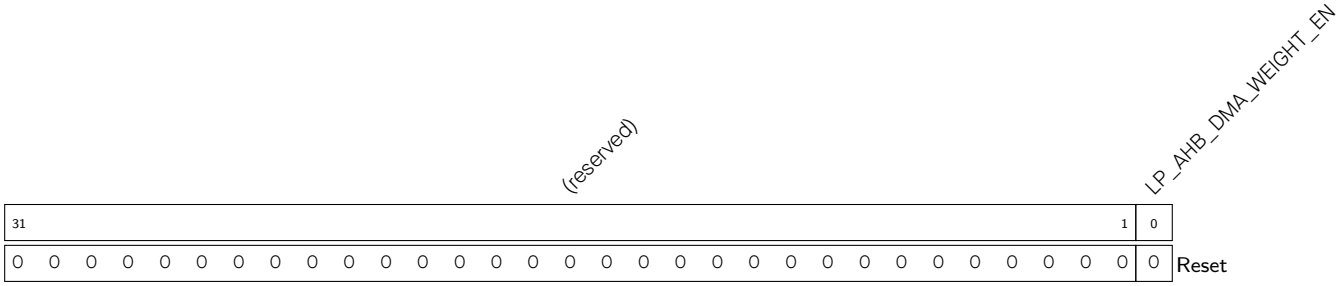
Register 3.79. LP_AHB_DMA_ARB_TIMEOUT_REG (0x0608)



LP_AHB_DMA_ARB_TIMEOUT 配置权重仲裁时间片。

单位：总线时钟周期。(R/W)

Register 3.80. LP_AHB_DMA_WEIGHT_EN_REG (0x0610)



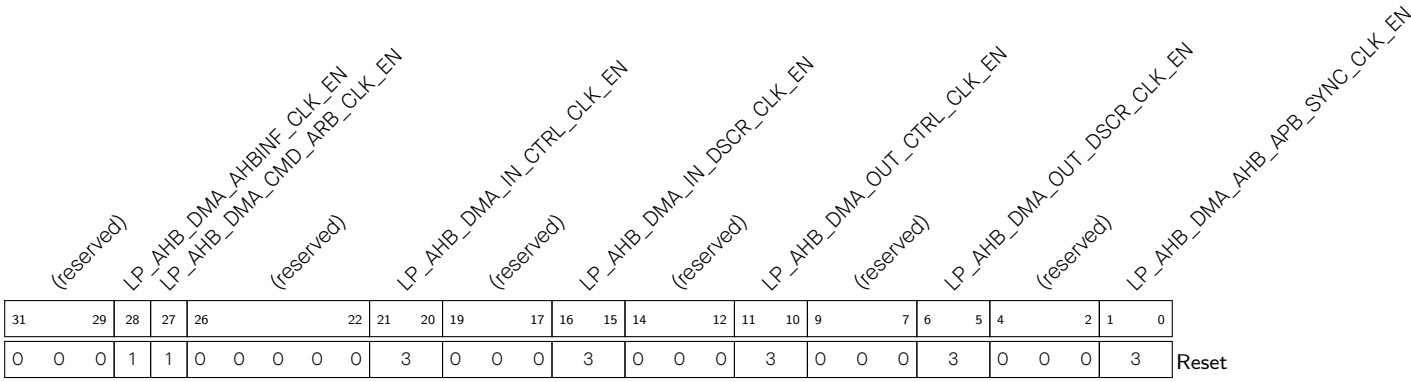
LP_AHB_DMA_WEIGHT_EN 配置是否使能权重仲裁。

0：禁用

1：使能

(R/W)

Register 3.81. LP_AHB_DMA_MODULE_CLK_EN_REG (0x0618)



LP_AHB_DMA_AHB_APB_SYNC_CLK_EN 配置是否强制开启 LP AHB DMA 寄存器配置同步模块的时钟。

位 0 对应 TX 和 RX 通道 0，位 1 对应 TX 和 RX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

LP_AHB_DMA_OUT_DSCR_CLK_EN 配置是否强制开启 LP AHB DMA TX 描述符处理模块的时钟。

位 0 对应 TX 通道 0，位 1 对应 TX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

LP_AHB_DMA_OUT_CTRL_CLK_EN 配置是否强制开启 LP AHB DMA TX 数据处理模块的时钟。

位 0 对应 TX 通道 0，位 1 对应 TX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

LP_AHB_DMA_IN_DSCR_CLK_EN 配置是否强制开启 LP AHB DMA RX 描述符处理模块的时钟。

位 0 对应 RX 通道 0，位 1 对应 RX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

LP_AHB_DMA_IN_CTRL_CLK_EN 配置是否强制开启 LP AHB DMA RX 数据处理模块的时钟。

位 0 对应 RX 通道 0，位 1 对应 RX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

LP_AHB_DMA_CMD_ARB_CLK_EN 配置是否强制开启 LP AHB DMA 仲裁模块的时钟。

位 0 对应 TX 和 RX 通道 0，位 1 对应 TX 和 RX 通道 1，以此类推。

0：不强制开启

1：强制开启

(R/W)

见下页...

Register 3.81. LP_AHB_DMA_MODULE_CLK_EN_REG (0x0618)

接上页...

LP_AHB_DMA_AHBINF_CLK_EN 配置是否强制开启 LP AHB DMA 总线接口处理模块的时钟。

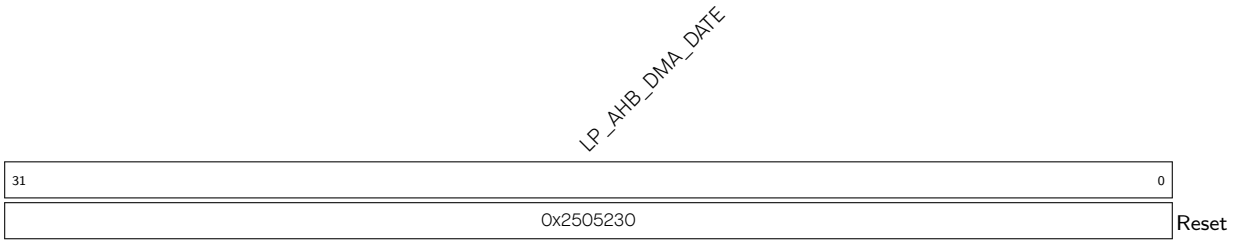
位 0 对应 TX 和 RX 通道 0，位 1 对应 TX 和 RX 通道 1，以此类推。

0：不强制开启

1：强制开启

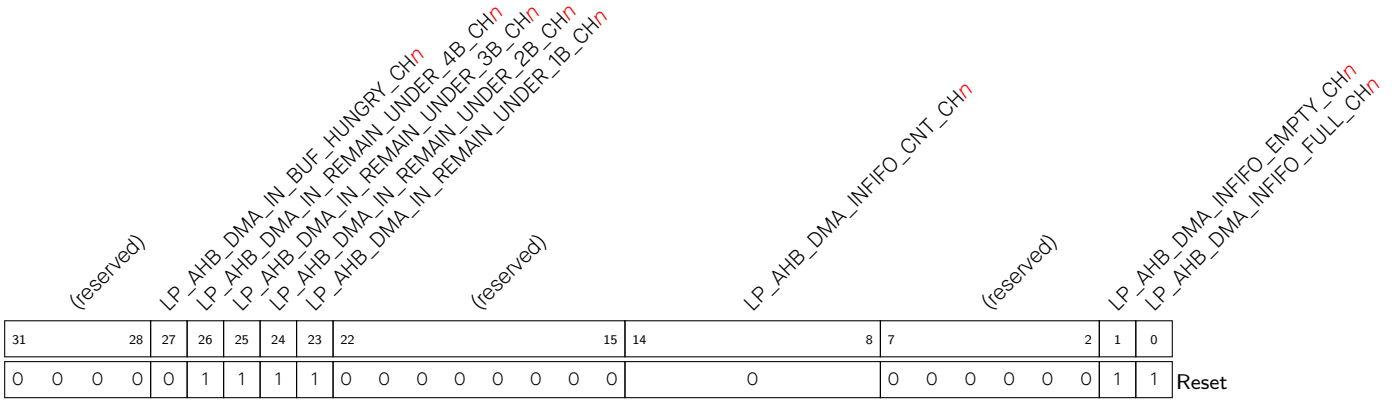
(R/W)

Register 3.82. LP_AHB_DMA_DATE_REG (0x00A8)



LP_AHB_DMA_DATE 版本控制寄存器。(R/W)

Register 3.83. LP_AHB_DMA_INFIFO_STATUS_CH*n*_REG (*n*: 0-1) (0x0108+0x100**n*)



LP_AHB_DMA_INFIFO_FULL_CH*n* 表示 RX 通道 *n* 的 RX FIFO 是否已满。

0：未满

1：已满

(RO)

LP_AHB_DMA_INFIFO_EMPTY_CH*n* 表示 RX 通道 *n* 的 RX FIFO 是否为空。

0：非空

1：空

(RO)

LP_AHB_DMA_INFIFO_CNT_CH*n* 表示 RX 通道 *n* RX FIFO 中的数据字节数。

单位：字节。(RO)

LP_AHB_DMA_IN_REMAIN_UNDER_1B_CH*n* 保留。(RO)

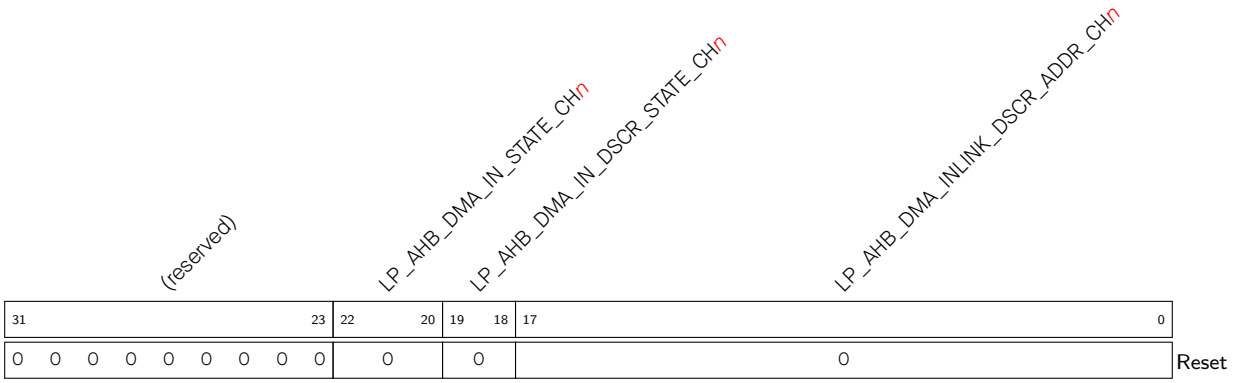
LP_AHB_DMA_IN_REMAIN_UNDER_2B_CH*n* 保留。(RO)

LP_AHB_DMA_IN_REMAIN_UNDER_3B_CH*n* 保留。(RO)

LP_AHB_DMA_IN_REMAIN_UNDER_4B_CH*n* 保留。(RO)

LP_AHB_DMA_IN_BUF_HUNGRY_CH*n* 保留。(RO)

Register 3.84. LP_AHB_DMA_IN_STATE_CH*n*_REG (*n*: 0-1) (0x0118+0x100**n*)

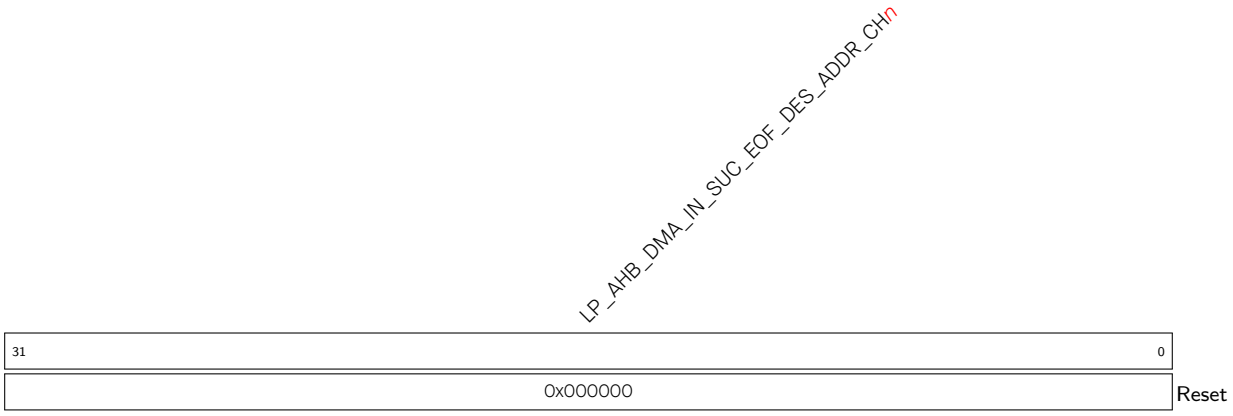


LP_AHB_DMA_INLINK_DSCR_ADDR_CH*n* 表示下一个待处理接收链表描述符地址的低 18 位。(RO)

LP_AHB_DMA_IN_DSCR_STATE_CH*n* 保留。(RO)

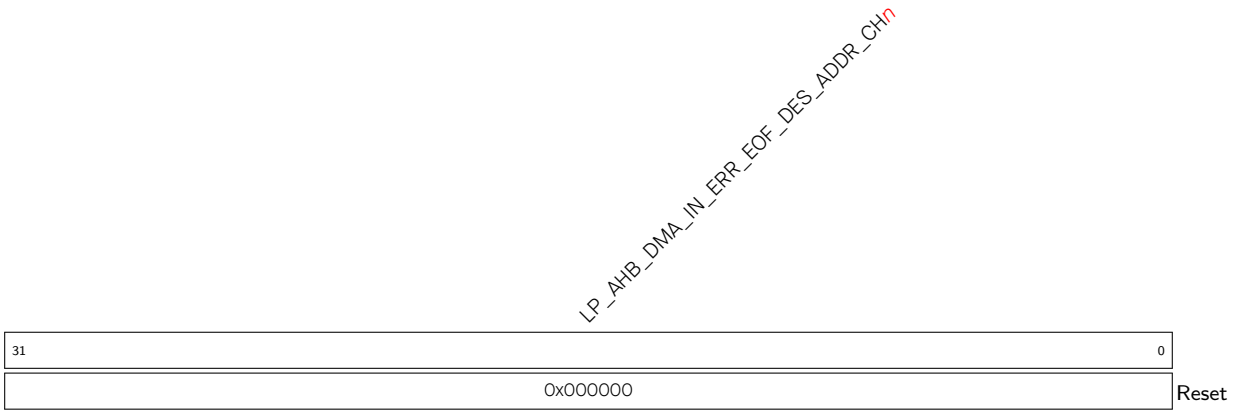
LP_AHB_DMA_IN_STATE_CH*n* 保留。(RO)

Register 3.85. LP_AHB_DMA_IN_SUC_EOF_DES_ADDR_CH*n*_REG (*n*: 0-1) (0x011C+0x100**n*)



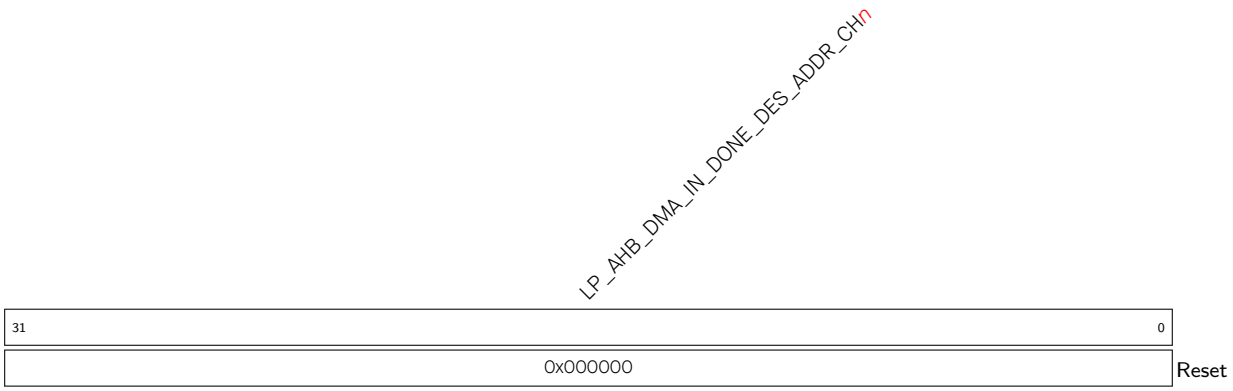
LP_AHB_DMA_IN_SUC_EOF_DES_ADDR_CH*n* 表示 RX 通道 *n* 中 EOF 标志为 1 的接收链表描述符地址。(RO)

Register 3.86. LP_AHB_DMA_IN_ERR_EOF_DES_ADDR_CH*n*_REG (*n*: 0-1) (0x0120+0x100**n*)



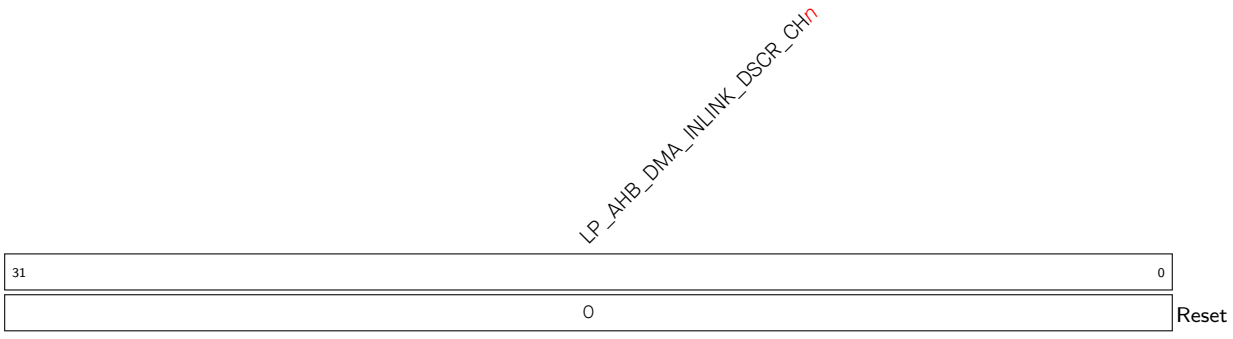
LP_AHB_DMA_IN_ERR_EOF_DES_ADDR_CH*n* 表示 RX 通道 *n* 当前接收数据发生错误时的接收链表描述符地址。(RO)

Register 3.87. LP_AHB_DMA_IN_DONE_DES_ADDR_CH*n*_REG (*n*: 0-1) (0x0124+0x100**n*)



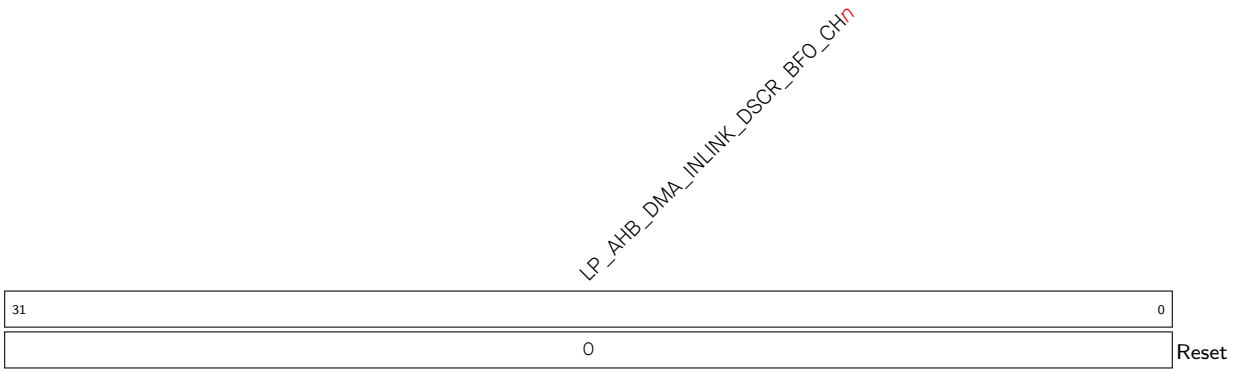
LP_AHB_DMA_IN_DONE_DES_ADDR_CH*n* 表示 RX 通道 *n* 中该描述符完成时的描述符地址。(RO)

Register 3.88. LP_AHB_DMA_IN_DSCR_CH*n*_REG (*n*: 0-1) (0x0128+0x100**n*)



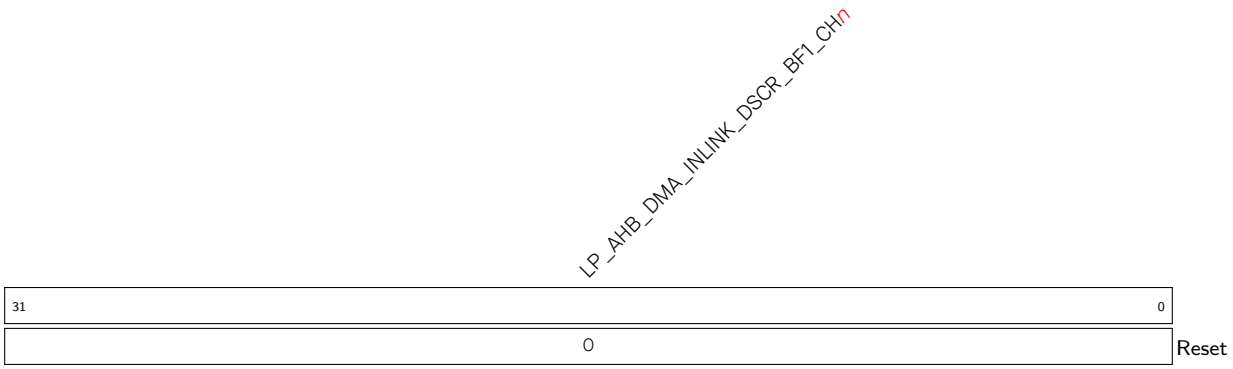
LP_AHB_DMA_INLINK_DSCR_CH*n* 表示当前已预读取的接收链表描述符所指向的下一个接收链表描述符地址。(RO)

Register 3.89. LP_AHB_DMA_IN_DSCR_BF0_CH*n*_REG (*n*: 0-1) (0x012C+0x100*n*)



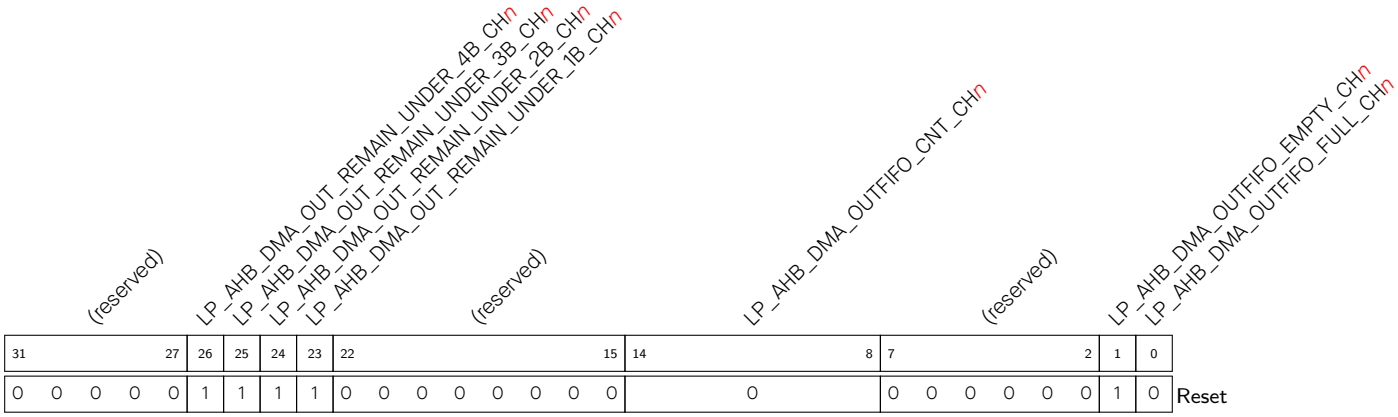
LP_AHB_DMA_INLINK_DSCR_BF0_CH*n* 表示 RX 通道 *n* 当前已预读取的接收链表描述符地址。(RO)

Register 3.90. LP_AHB_DMA_IN_DSCR_BF1_CH*n*_REG (*n*: 0-1) (0x0130+0x100*n*)



LP_AHB_DMA_INLINK_DSCR_BF1_CH*n* 表示 RX 通道 *n* 上一个已预读取的接收链表描述符地址。(RO)

Register 3.91. LP_AHB_DMA_OUTFIFO_STATUS_CH*n*_REG (*n*: 0-1) (0x0188+0x100*n*)



LP_AHB_DMA_OUTFIFO_FULL_CH*n* 表示 TX 通道 *n* 的 TX FIFO 是否已满。

0：未满

1：已满

(RO)

LP_AHB_DMA_OUTFIFO_EMPTY_CH*n* 表示 TX 通道 *n* 的 TX FIFO 是否为空。

0：非空

1：空

(RO)

LP_AHB_DMA_OUTFIFO_CNT_CH*n* 表示 TX 通道 *n* TX FIFO 中的数据字节数。

单位：字节。(RO)

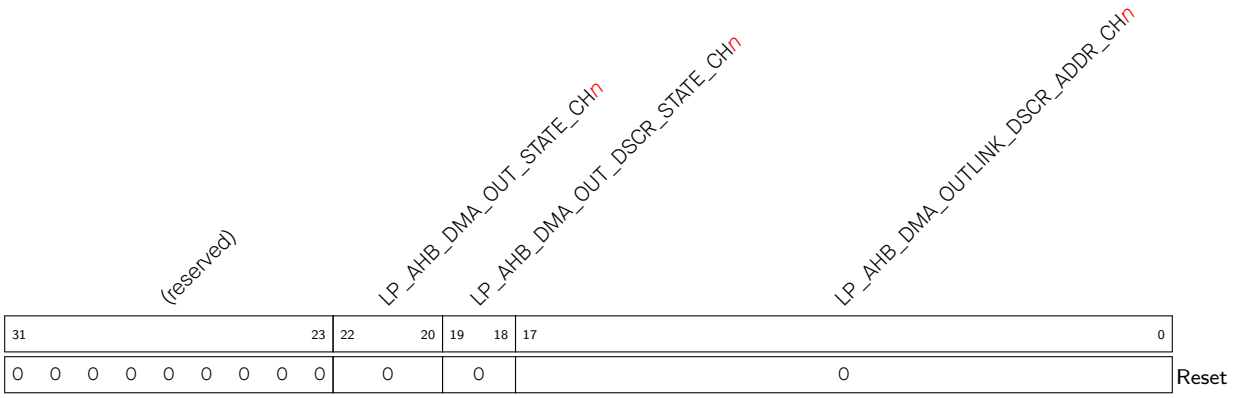
LP_AHB_DMA_OUT_REMAIN_UNDER_1B_CH*n* 保留。(RO)

LP_AHB_DMA_OUT_REMAIN_UNDER_2B_CH*n* 保留。(RO)

LP_AHB_DMA_OUT_REMAIN_UNDER_3B_CH*n* 保留。(RO)

LP_AHB_DMA_OUT_REMAIN_UNDER_4B_CH*n* 保留。(RO)

Register 3.92. LP_AHB_DMA_OUT_STATE_CH*n*_REG (*n*: 0-1) (0x0198+0x100**n*)

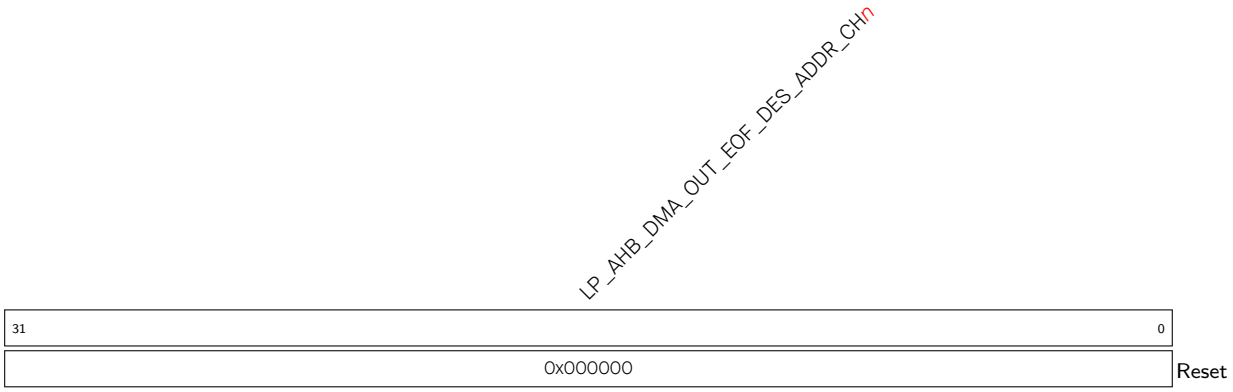


LP_AHB_DMA_OUTLINK_DSCR_ADDR_CH*n* 表示下一个待处理发送链表描述符地址的低 18 位。(RO)

LP_AHB_DMA_OUT_DSCR_STATE_CH*n* 保留。(RO)

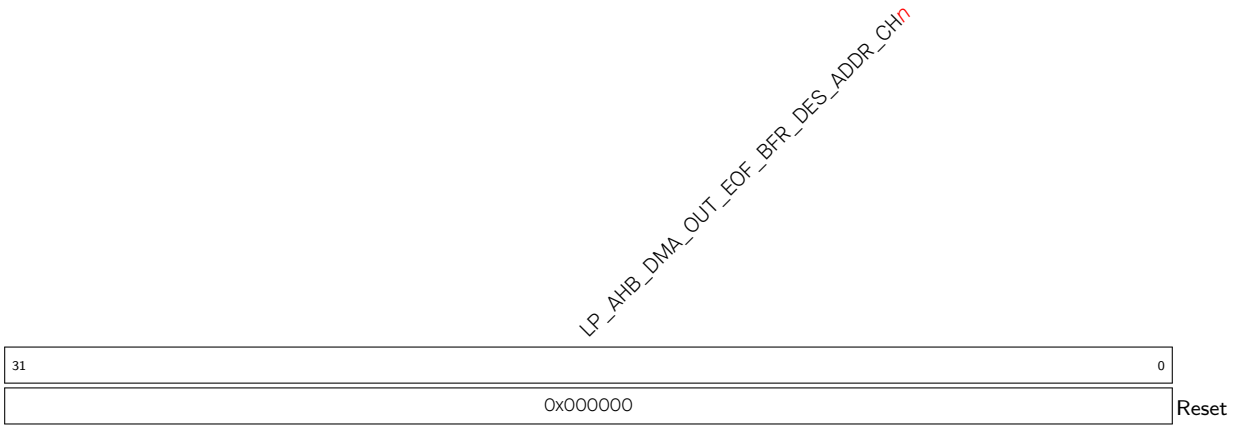
LP_AHB_DMA_OUT_STATE_CH*n* 保留。(RO)

Register 3.93. LP_AHB_DMA_OUT_EOF_DES_ADDR_CH*n*_REG (*n*: 0-1) (0x019C+0x100**n*)



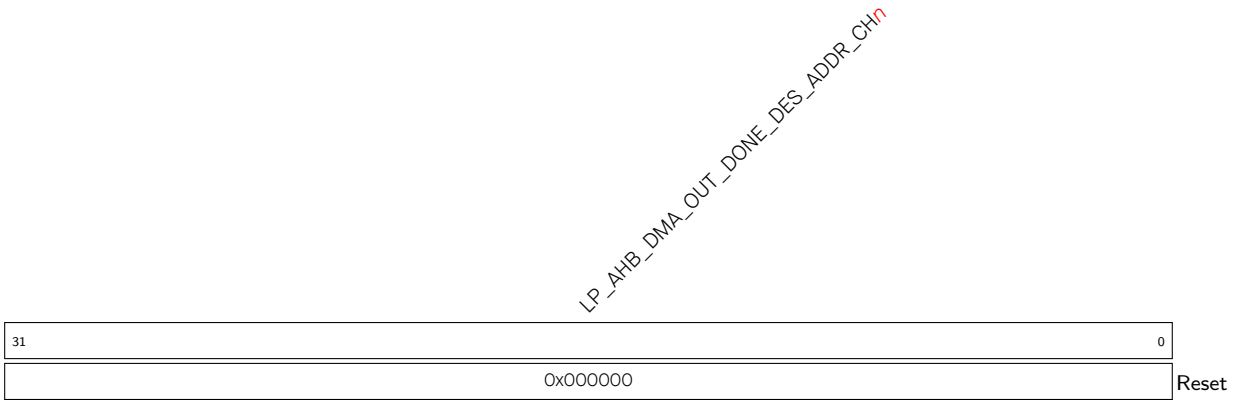
LP_AHB_DMA_OUT_EOF_DES_ADDR_CH*n* 表示 TX 通道 *n* 中 EOF 标志为 1 的发送链表描述符地址。(RO)

Register 3.94. LP_AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH*n*_REG (*n*: 0-1) (0x01A0+0x100*n*)



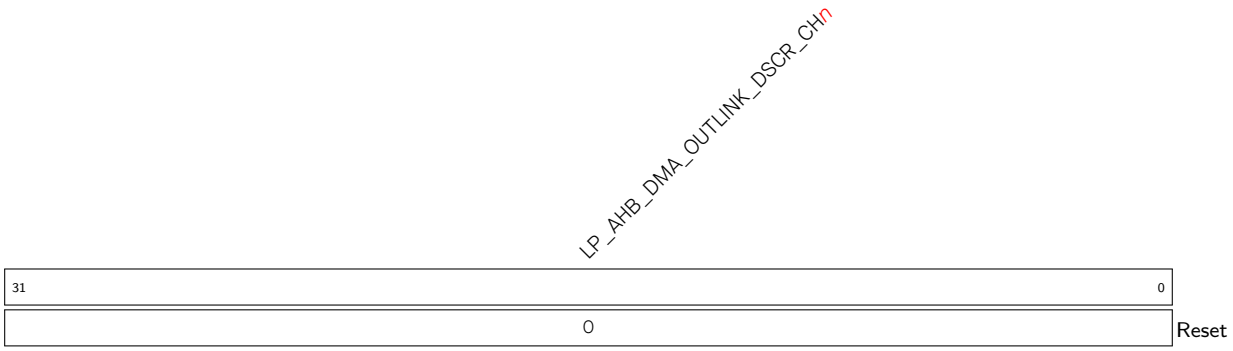
LP_AHB_DMA_OUT_EOF_BFR_DES_ADDR_CH*n* 表示 TX 通道 *n* 最后一个发送链表描述符之前的发送链表描述符地址。(RO)

Register 3.95. LP_AHB_DMA_OUT_DONE_DES_ADDR_CH*n*_REG (*n*: 0-1) (0x01A4+0x100*n*)



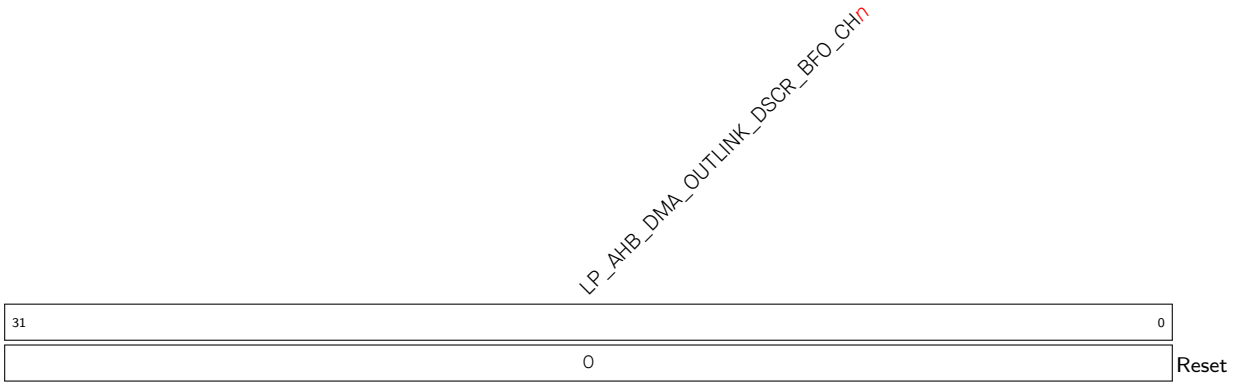
LP_AHB_DMA_OUT_DONE_DES_ADDR_CH*n* 表示 TX 通道 *n* 中该描述符完成时的描述符地址。(RO)

Register 3.96. LP_AHB_DMA_OUT_DSCR_CH*n*_REG (*n*: 0-1) (0x01A8+0x100**n*)



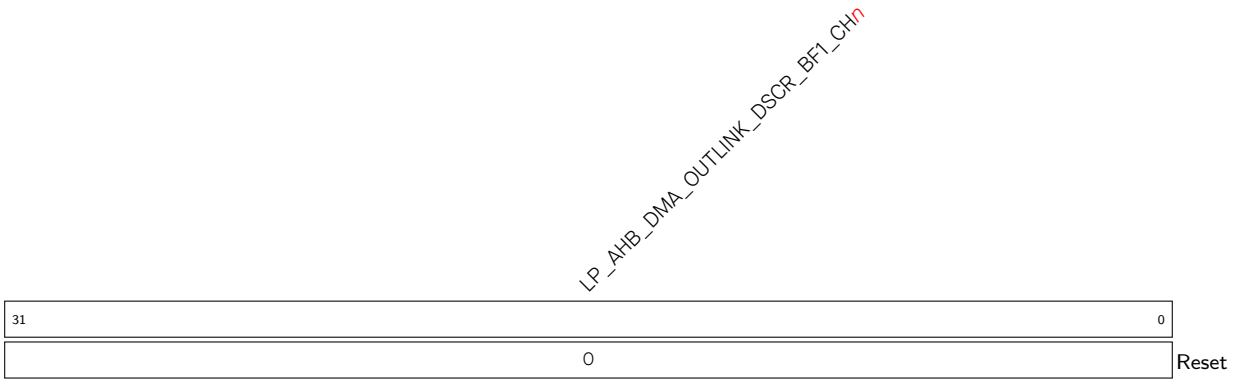
LP_AHB_DMA_OUTLINK_DSCR_CH*n* 表示当前已预读取的发送链表描述符所指向的下一个发送链表描述符地址。(RO)

Register 3.97. LP_AHB_DMA_OUT_DSCR_BF0_CH*n*_REG (*n*: 0-1) (0x01AC+0x100**n*)



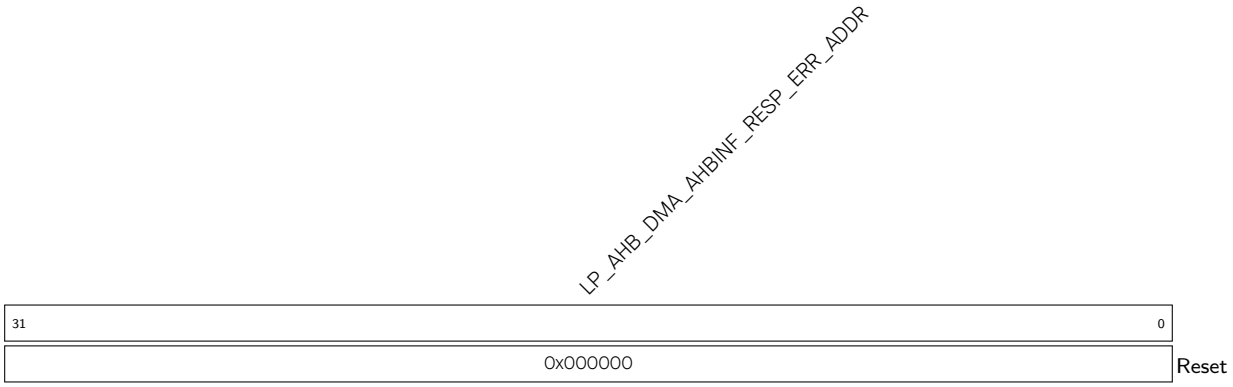
LP_AHB_DMA_OUTLINK_DSCR_BF0_CH*n* 表示 TX 通道 *n* 当前已预读取的发送链表描述符地址。(RO)

Register 3.98. LP_AHB_DMA_OUT_DSCR_BF1_CH*n*_REG (*n*: 0-1) (0x01B0+0x100**n*)



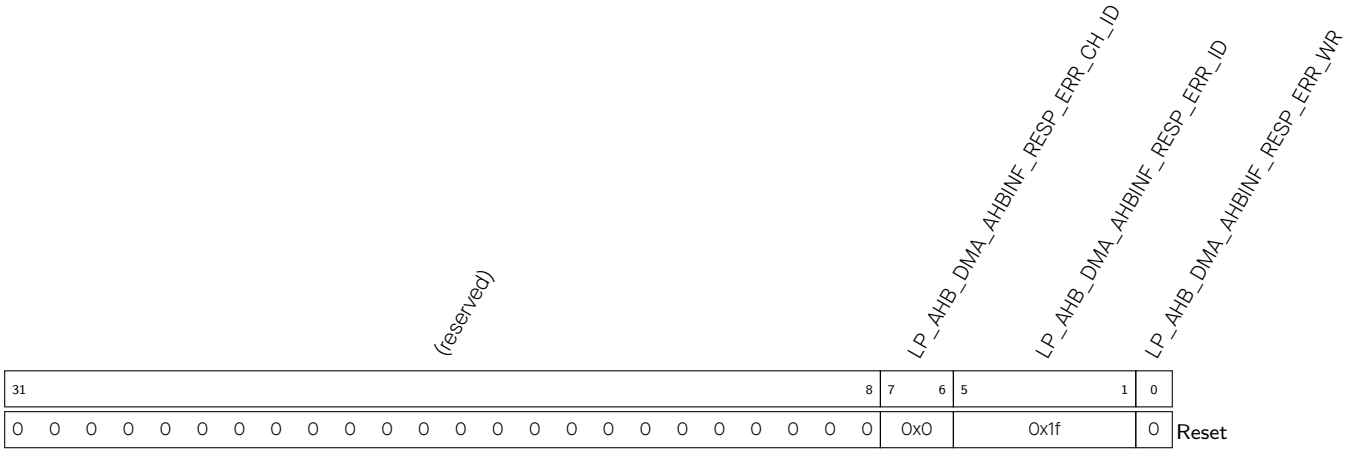
LP_AHB_DMA_OUTLINK_DSCR_BF1_CH*n* 表示 TX 通道 *n* 上一个已预读取的发送链表描述符地址。(RO)

Register 3.99. LP_AHB_DMA_AHBINF_RESP_ERR_STATUS0_REG (0x0620)



LP_AHB_DMA_AHBINF_RESP_ERR_ADDR 表示触发错误响应的当前 LP AHB 总线传输地址。(RO)

Register 3.100. LP_AHB_DMA_AHBINF_RESP_ERR_STATUS1_REG (0x0624)



LP_AHB_DMA_AHBINF_RESP_ERR_WR 表示 LP AHB 响应错误是否由写请求引起。

0：读请求

1：写请求

(RO)

LP_AHB_DMA_AHBINF_RESP_ERR_ID 表示 LP AHB 响应错误请求 ID。

0：ADC

1：DAC

16：GDMA-AHB-LP 通道 0 的存储器到存储器传输

17：GDMA-AHB-LP 通道 1 的存储器到存储器传输

其余值无效

(RO)

LP_AHB_DMA_AHBINF_RESP_ERR_CH_ID 表示 LP AHB 响应错误请求通道 ID。

位 [1]：表示通道方向

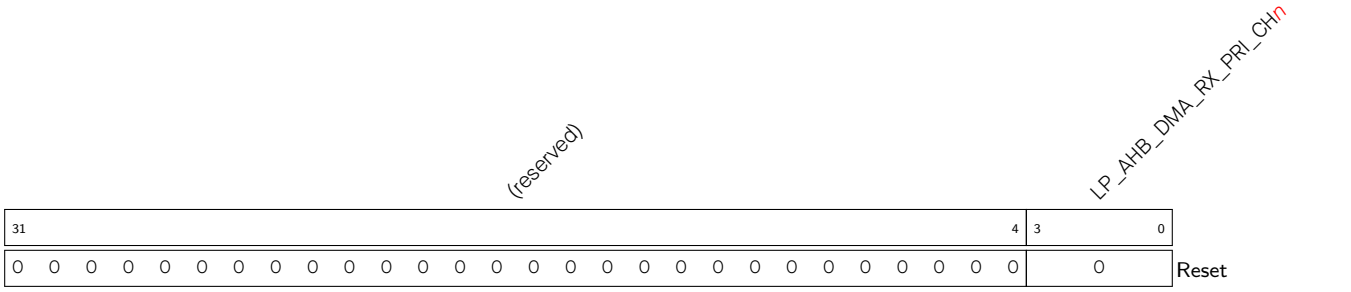
0：RX 通道

1：TX 通道

位 [0]：表示通道号。取值范围：0 ~1

(RO)

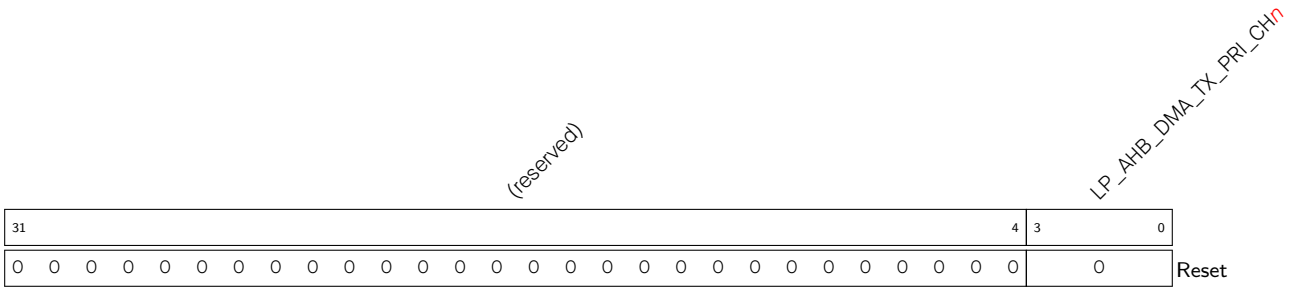
Register 3.101. LP_AHB_DMA_IN_PRI_CH*n*_REG (*n*: 0-1) (0x0134+0x100**n*)



LP_AHB_DMA_RX_PRI_CH*n* 配置 RX 通道 *n* 的优先级。数值越大，优先级越高。取值范围：0 ~ 15。

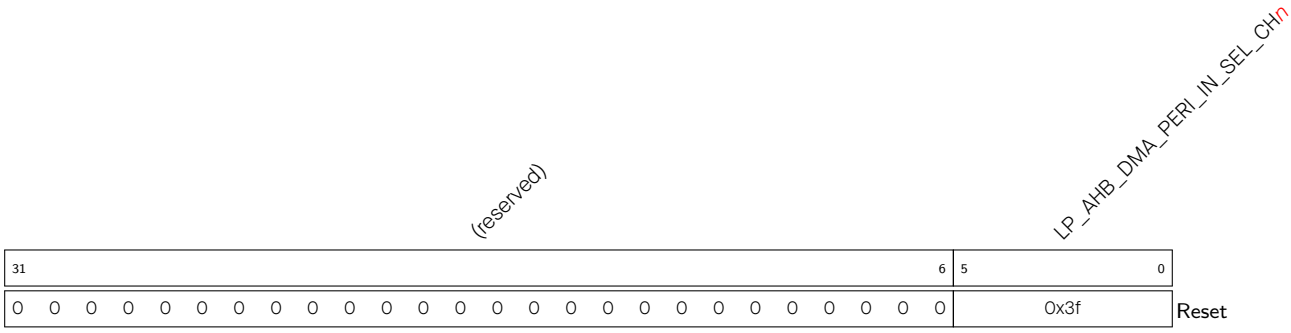
(R/W)

Register 3.102. LP_AHB_DMA_OUT_PRI_CH*n*_REG (*n*: 0-1) (0x01B4+0x100**n*)



LP_AHB_DMA_TX_PRI_CH*n* 配置 TX 通道 *n* 的优先级。数值越大，优先级越高。取值范围：0 ~ 15。(R/W)

Register 3.103. LP_AHB_DMA_IN_PERI_SEL_CH*n*_REG (*n*: 0-1) (0x0138+0x100**n*)



LP_AHB_DMA_PERI_IN_SEL_CH*n* 配置连接至 RX 通道 *n* 的外设。

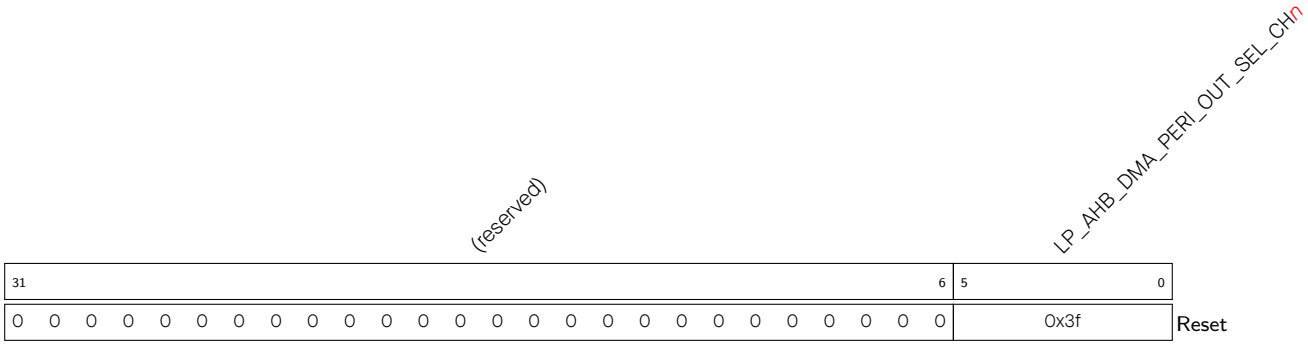
0：ADC

1：DAC

2 ~ 63：Dummy

(R/W)

Register 3.104. LP_AHB_DMA_OUT_PERI_SEL_CH*n*_REG (*n*: 0-1) (0x01B8+0x100**n*)



LP_AHB_DMA_PERI_OUT_SEL_CH*n* 配置连接至 TX 通道 *n* 的外设。

0：ADC

1：DAC

2 ~ 63：Dummy

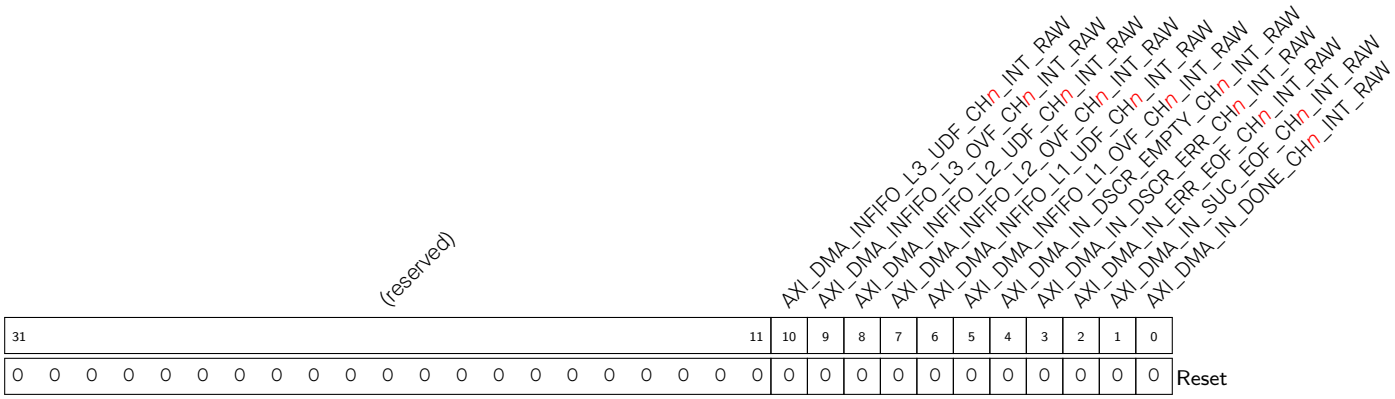
(R/W)

3.9.3 GDMA-AXI 寄存器

本小节的所有地址均为相对于 GDMA-AXI 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 3.105. AXI_DMA_IN_INT_RAW_CH*n*_REG (*n*: 0-2) (0x0000+0x68**n*)



AXI_DMA_IN_DONE_CH*n*_INT_RAW AXI_DMA_IN_DONE_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_IN_SUC_EOF_CH*n*_INT_RAW AXI_DMA_IN_SUC_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_IN_ERR_EOF_CH*n*_INT_RAW AXI_DMA_IN_ERR_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_IN_DSCR_ERR_CH*n*_INT_RAW AXI_DMA_IN_DSCR_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_IN_DSCR_EMPTY_CH*n*_INT_RAW AXI_DMA_IN_DSCR_EMPTY_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_INFIFO_L1_OVF_CH*n*_INT_RAW AXI_DMA_INFIFO_L1_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_INFIFO_L1_UDF_CH*n*_INT_RAW AXI_DMA_INFIFO_L1_UDF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

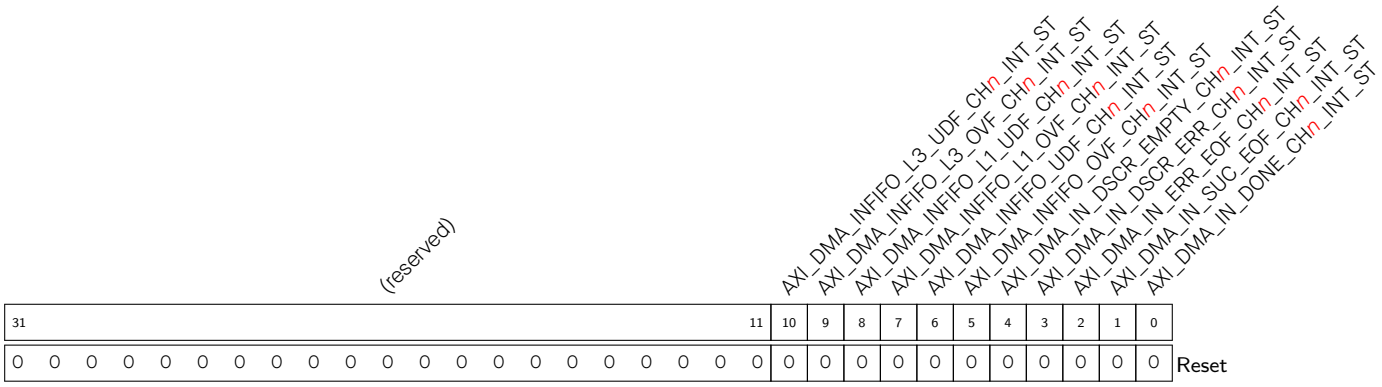
AXI_DMA_INFIFO_L2_OVF_CH*n*_INT_RAW AXI_DMA_INFIFO_L2_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_INFIFO_L2_UDF_CH*n*_INT_RAW AXI_DMA_INFIFO_L2_UDF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_INFIFO_L3_OVF_CH*n*_INT_RAW AXI_DMA_INFIFO_L3_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_INFIFO_L3_UDF_CH*n*_INT_RAW AXI_DMA_INFIFO_L3_UDF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

Register 3.106. AXI_DMA_IN_INT_ST_CH*n*_REG (*n*: 0-2) (0x0004+0x68**n*)



AXI_DMA_IN_DONE_CH*n*_INT_ST AXI_DMA_IN_DONE_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_IN_SUC_EOF_CH*n*_INT_ST AXI_DMA_IN_SUC_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_IN_ERR_EOF_CH*n*_INT_ST AXI_DMA_IN_ERR_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_IN_DSCR_ERR_CH*n*_INT_ST AXI_DMA_IN_DSCR_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_IN_DSCR_EMPTY_CH*n*_INT_ST AXI_DMA_IN_DSCR_EMPTY_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_INFIFO_OVF_CH*n*_INT_ST AXI_DMA_INFIFO_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_INFIFO_UDF_CH*n*_INT_ST AXI_DMA_INFIFO_UDF_CH*n*_INT 的屏蔽中断状态。(RO)

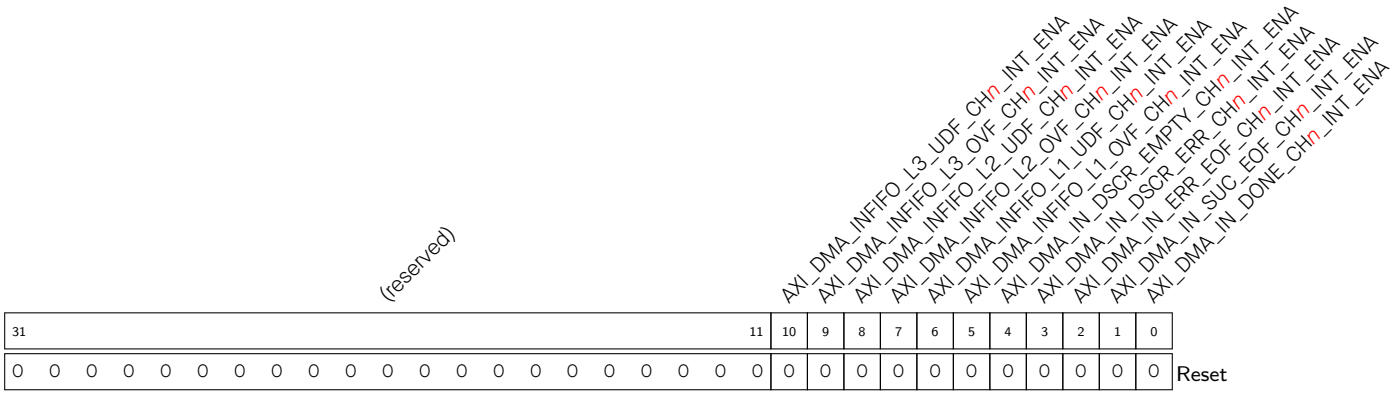
AXI_DMA_INFIFO_L1_OVF_CH*n*_INT_ST AXI_DMA_INFIFO_L1_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_INFIFO_L1_UDF_CH*n*_INT_ST AXI_DMA_INFIFO_L1_UDF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_INFIFO_L3_OVF_CH*n*_INT_ST AXI_DMA_INFIFO_L3_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_INFIFO_L3_UDF_CH*n*_INT_ST AXI_DMA_INFIFO_L3_UDF_CH*n*_INT 的屏蔽中断状态。(RO)

Register 3.107. AXI_DMA_IN_INT_ENA_CH*n*_REG (*n*: 0-2) (0x0008+0x68**n*)



AXI_DMA_IN_DONE_CH*n*_INT_ENA 写 1 使能 AXI_DMA_IN_DONE_CH*n*_INT。(R/W)

AXI_DMA_IN_SUC_EOF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_IN_SUC_EOF_CH*n*_INT。(R/W)

AXI_DMA_IN_ERR_EOF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_IN_ERR_EOF_CH*n*_INT。(R/W)

AXI_DMA_IN_DSCR_ERR_CH*n*_INT_ENA 写 1 使能 AXI_DMA_IN_DSCR_ERR_CH*n*_INT。(R/W)

AXI_DMA_IN_DSCR_EMPTY_CH*n*_INT_ENA 写 1 使能 AXI_DMA_IN_DSCR_EMPTY_CH*n*_INT。(R/W)

AXI_DMA_INFIFO_L1_OVF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_INFIFO_L1_OVF_CH*n*_INT。(R/W)

AXI_DMA_INFIFO_L1_UDF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_INFIFO_L1_UDF_CH*n*_INT。(R/W)

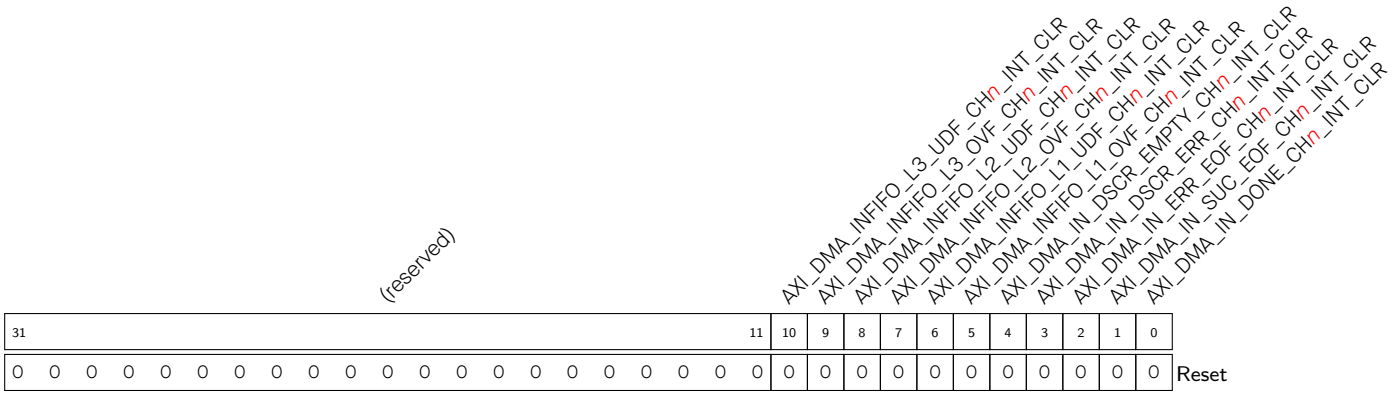
AXI_DMA_INFIFO_L2_OVF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_INFIFO_L2_OVF_CH*n*_INT。(R/W)

AXI_DMA_INFIFO_L2_UDF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_INFIFO_L2_UDF_CH*n*_INT。(R/W)

AXI_DMA_INFIFO_L3_OVF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_INFIFO_L3_OVF_CH*n*_INT。(R/W)

AXI_DMA_INFIFO_L3_UDF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_INFIFO_L3_UDF_CH*n*_INT。(R/W)

Register 3.108. AXI_DMA_IN_INT_CLR_CH*n*_REG (*n*: 0-2) (0x000C+0x68**n*)



AXI_DMA_IN_DONE_CH*n*_INT_CLR 写 1 清除 AXI_DMA_IN_DONE_CH*n*_INT。(WT)

AXI_DMA_IN_SUC_EOF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_IN_SUC_EOF_CH*n*_INT。(WT)

AXI_DMA_IN_ERR_EOF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_IN_ERR_EOF_CH*n*_INT。(WT)

AXI_DMA_IN_DSCR_ERR_CH*n*_INT_CLR 写 1 清除 AXI_DMA_IN_DSCR_ERR_CH*n*_INT。(WT)

AXI_DMA_IN_DSCR_EMPTY_CH*n*_INT_CLR 写 1 清除 AXI_DMA_IN_DSCR_EMPTY_CH*n*_INT。(WT)

AXI_DMA_INFIFO_L1_OVF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_INFIFO_L1_OVF_CH*n*_INT。(WT)

AXI_DMA_INFIFO_L1_UDF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_INFIFO_L1_UDF_CH*n*_INT。(WT)

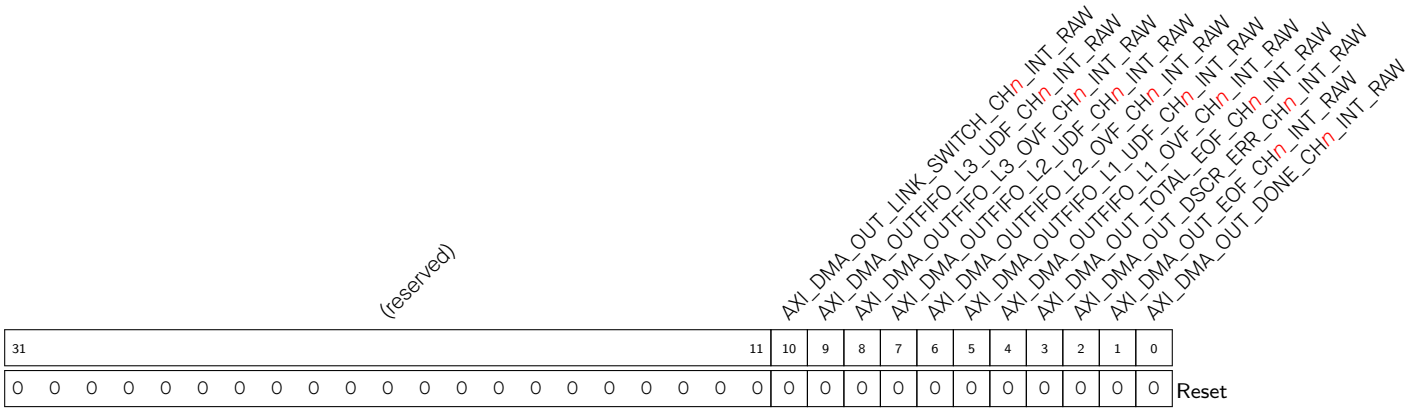
AXI_DMA_INFIFO_L2_OVF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_INFIFO_L2_OVF_CH*n*_INT。(WT)

AXI_DMA_INFIFO_L2_UDF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_INFIFO_L2_UDF_CH*n*_INT。(WT)

AXI_DMA_INFIFO_L3_OVF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_INFIFO_L3_OVF_CH*n*_INT。(WT)

AXI_DMA_INFIFO_L3_UDF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_INFIFO_L3_UDF_CH*n*_INT。(WT)

Register 3.109. AXI_DMA_OUT_INT_RAW_CH*n*_REG (*n*: 0-2) (0x0138+0x68**n*)



AXI_DMA_OUT_DONE_CH*n*_INT_RAW AXI_DMA_OUT_DONE_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_OUT_EOF_CH*n*_INT_RAW AXI_DMA_OUT_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_OUT_DSCR_ERR_CH*n*_INT_RAW AXI_DMA_OUT_DSCR_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT_RAW AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_OUTFIFO_L1_OVF_CH*n*_INT_RAW AXI_DMA_OUTFIFO_L1_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_OUTFIFO_L1_UDF_CH*n*_INT_RAW AXI_DMA_OUTFIFO_L1_UDF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_OUTFIFO_L2_OVF_CH*n*_INT_RAW AXI_DMA_OUTFIFO_L2_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

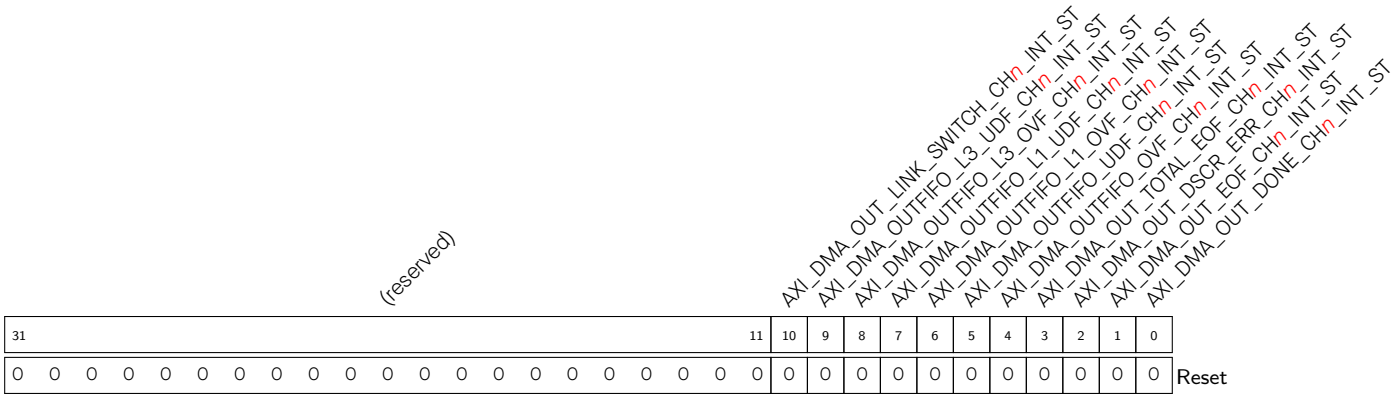
AXI_DMA_OUTFIFO_L2_UDF_CH*n*_INT_RAW AXI_DMA_OUTFIFO_L2_UDF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_OUTFIFO_L3_OVF_CH*n*_INT_RAW AXI_DMA_OUTFIFO_L3_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_OUTFIFO_L3_UDF_CH*n*_INT_RAW AXI_DMA_OUTFIFO_L3_UDF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

AXI_DMA_OUT_LINK_SWITCH_CH*n*_INT_RAW AXI_DMA_OUT_LINK_SWITCH_CH*n*_INT 的原始中断状态。(R/WTC/SS)

Register 3.110. AXI_DMA_OUT_INT_ST_CH*n*_REG (*n*: 0-2) (0x013C+0x68***n*)



AXI_DMA_OUT_DONE_CH*n*_INT_ST AXI_DMA_OUT_DONE_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_OUT_EOF_CH*n*_INT_ST AXI_DMA_OUT_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_OUT_DSCR_ERR_CH*n*_INT_ST AXI_DMA_OUT_DSCR_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT_ST AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_OUTFIFO_OVF_CH*n*_INT_ST AXI_DMA_OUTFIFO_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_OUTFIFO_UDF_CH*n*_INT_ST AXI_DMA_OUTFIFO_UDF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_OUTFIFO_L1_OVF_CH*n*_INT_ST AXI_DMA_OUTFIFO_L1_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

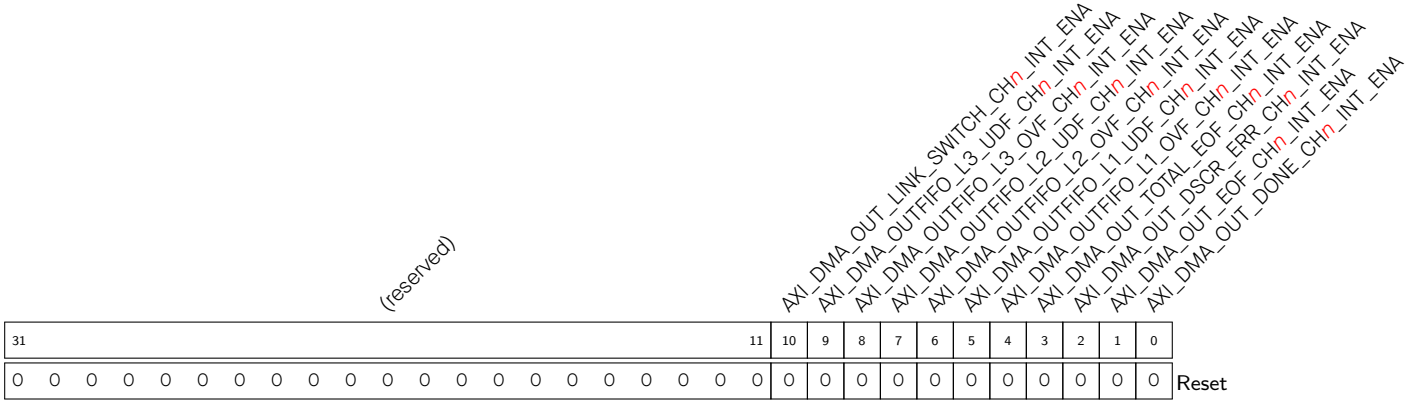
AXI_DMA_OUTFIFO_L1_UDF_CH*n*_INT_ST AXI_DMA_OUTFIFO_L1_UDF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_OUTFIFO_L3_OVF_CH*n*_INT_ST AXI_DMA_OUTFIFO_L3_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_OUTFIFO_L3_UDF_CH*n*_INT_ST AXI_DMA_OUTFIFO_L3_UDF_CH*n*_INT 的屏蔽中断状态。(RO)

AXI_DMA_OUT_LINK_SWITCH_CH*n*_INT_ST AXI_DMA_OUT_LINK_SWITCH_CH*n*_INT 的屏蔽中断状态。(RO)

Register 3.111. AXI_DMA_OUT_INT_ENA_CH*n*_REG (*n*: 0-2) (0x0140+0x68*n*)



AXI_DMA_OUT_DONE_CH*n*_INT_ENA 写 1 使能 AXI_DMA_OUT_DONE_CH*n*_INT。(R/W)

AXI_DMA_OUT_EOF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_OUT_EOF_CH*n*_INT。(R/W)

AXI_DMA_OUT_DSCR_ERR_CH*n*_INT_ENA 写 1 使能 AXI_DMA_OUT_DSCR_ERR_CH*n*_INT。(R/W)

AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT。(R/W)

AXI_DMA_OUTFIFO_L1_OVF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_OUTFIFO_L1_OVF_CH*n*_INT。(R/W)

AXI_DMA_OUTFIFO_L1_UDF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_OUTFIFO_L1_UDF_CH*n*_INT。(R/W)

AXI_DMA_OUTFIFO_L2_OVF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_OUTFIFO_L2_OVF_CH*n*_INT。(R/W)

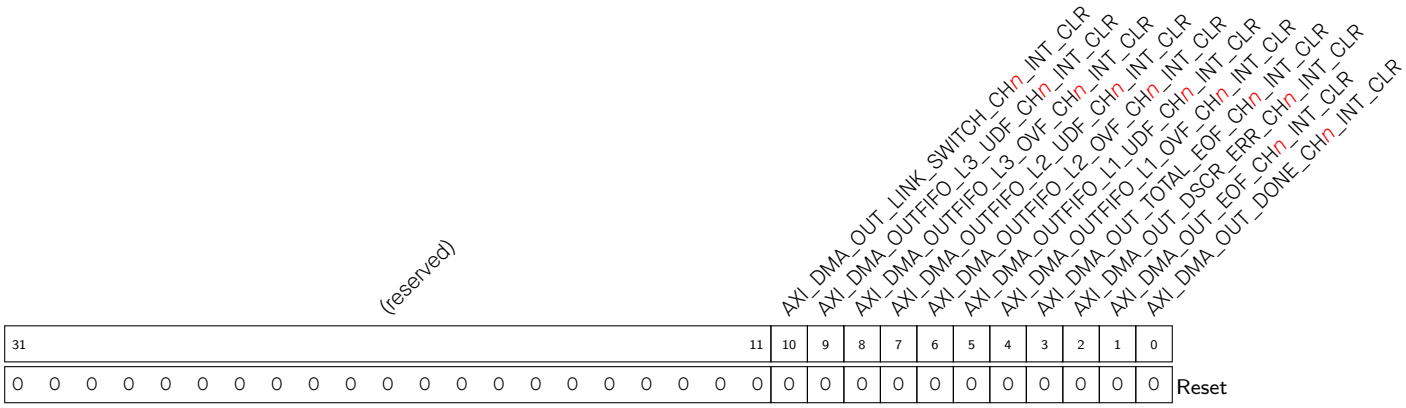
AXI_DMA_OUTFIFO_L2_UDF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_OUTFIFO_L2_UDF_CH*n*_INT。(R/W)

AXI_DMA_OUTFIFO_L3_OVF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_OUTFIFO_L3_OVF_CH*n*_INT。(R/W)

AXI_DMA_OUTFIFO_L3_UDF_CH*n*_INT_ENA 写 1 使能 AXI_DMA_OUTFIFO_L3_UDF_CH*n*_INT。(R/W)

AXI_DMA_OUT_LINK_SWITCH_CH*n*_INT_ENA 写 1 使能 AXI_DMA_OUT_LINK_SWITCH_CH*n*_INT。(R/W)

Register 3.112. AXI_DMA_OUT_INT_CLR_CH*n*_REG (*n*: 0-2) (0x0144+0x68*n*)



AXI_DMA_OUT_DONE_CH*n*_INT_CLR 写 1 清除 AXI_DMA_OUT_DONE_CH*n*_INT。(WT)

AXI_DMA_OUT_EOF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_OUT_EOF_CH*n*_INT。(WT)

AXI_DMA_OUT_DSCR_ERR_CH*n*_INT_CLR 写 1 清除 AXI_DMA_OUT_DSCR_ERR_CH*n*_INT。(WT)

AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_OUT_TOTAL_EOF_CH*n*_INT。(WT)

AXI_DMA_OUTFIFO_L1_OVF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_OUTFIFO_L1_OVF_CH*n*_INT。(WT)

AXI_DMA_OUTFIFO_L1_UDF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_OUTFIFO_L1_UDF_CH*n*_INT。(WT)

AXI_DMA_OUTFIFO_L2_OVF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_OUTFIFO_L2_OVF_CH*n*_INT。(WT)

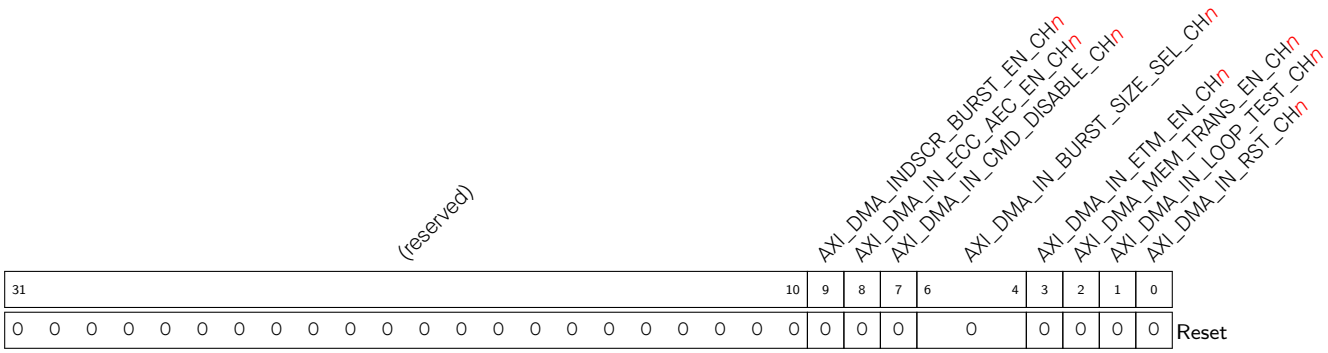
AXI_DMA_OUTFIFO_L2_UDF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_OUTFIFO_L2_UDF_CH*n*_INT。(WT)

AXI_DMA_OUTFIFO_L3_OVF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_OUTFIFO_L3_OVF_CH*n*_INT。(WT)

AXI_DMA_OUTFIFO_L3_UDF_CH*n*_INT_CLR 写 1 清除 AXI_DMA_OUTFIFO_L3_UDF_CH*n*_INT。(WT)

AXI_DMA_OUT_LINK_SWITCH_CH*n*_INT_CLR 写 1 清除 AXI_DMA_OUT_LINK_SWITCH_CH*n*_INT。(WT)

Register 3.113. AXI_DMA_IN_CONF0_CH*n*_REG (*n*: 0-2) (0x0010+0x68**n*)



AXI_DMA_IN_RST_CH*n* 配置 AXI DMA RX 通道 *n* FSM 和 FIFO 指针的复位状态。

0：解除复位

1：复位

(R/W)

AXI_DMA_IN_LOOP_TEST_CH*n* 配置是否关闭 RX 通道 *n* 的 owner 位自动清零。

0：无效值。没有作用

1：关闭

(R/W)

AXI_DMA_MEM_TRANS_EN_CH*n* 配置是否使能存储器到存储器数据传输。

0：禁用

1：使能

(R/W)

AXI_DMA_IN_ETM_EN_CH*n* 配置是否使能 RX 通道 *n* 的 ETM 控制。

0：禁用

1：使能

(R/W)

AXI_DMA_IN_BURST_SIZE_SEL_CH*n* 配置 RX 通道 *n* 的突发长度。

0：8 字节

1：16 字节

2：32 字节

3：64 字节

4：128 字节

5 ~ 7：无效值

(R/W)

AXI_DMA_IN_CMD_DISABLE_CH*n* 配置是否禁用 RX 通道 *n* 的命令。

0：使能命令

1：禁用命令

(R/W)

见下页...

Register 3.113. AXI_DMA_IN_CONF0_CH*n*_REG (*n*: 0-2) (0x0010+0x68**n*)

接上页...

AXI_DMA_IN_ECC_AEC_EN_CH*n* 配置是否访问 RX 通道 *n* 的 ECC 或 AES 域。

0：禁用

1：使能

(R/W)

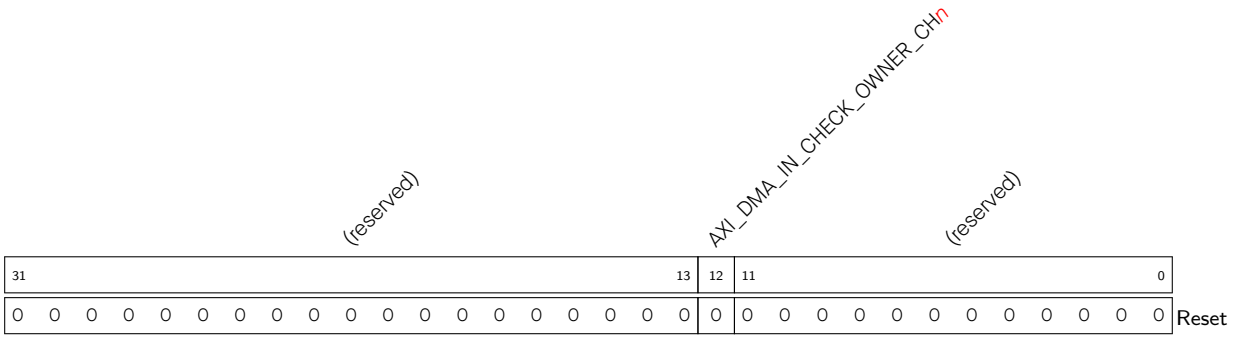
AXI_DMA_INDSCR_BURST_EN_CH*n* 配置是否使能 RX 通道 *n* 读取接收链表描述符的 INCR 突发传输。

0：禁用

1：使能

(R/W)

Register 3.114. AXI_DMA_IN_CONF1_CH*n*_REG (*n*: 0-2) (0x0014+0x68**n*)



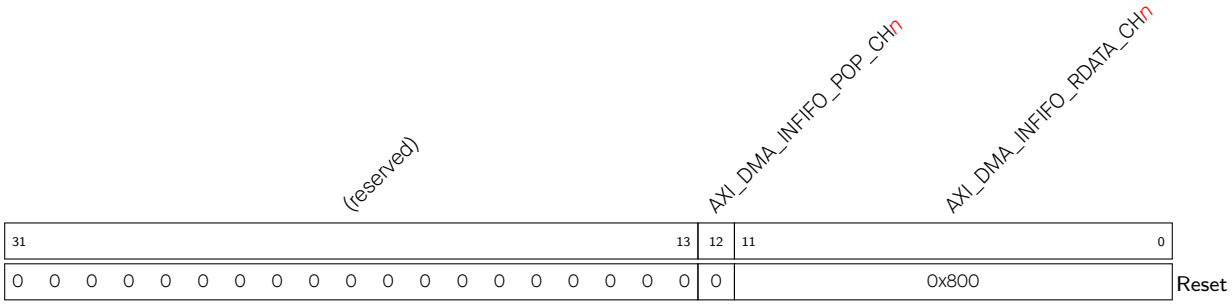
AXI_DMA_IN_CHECK_OWNER_CH*n* 配置是否使能 RX 通道 *n* 的 owner 位检查。

0：禁用

1：使能

(R/W)

Register 3.115. AXI_DMA_IN_POP_CH*n*_REG (*n*: 0-2) (0x001C+0x68**n*)



AXI_DMA_INFIFO_RDATA_CH*n* 从 AXI DMA FIFO 接收的数据。(RO)

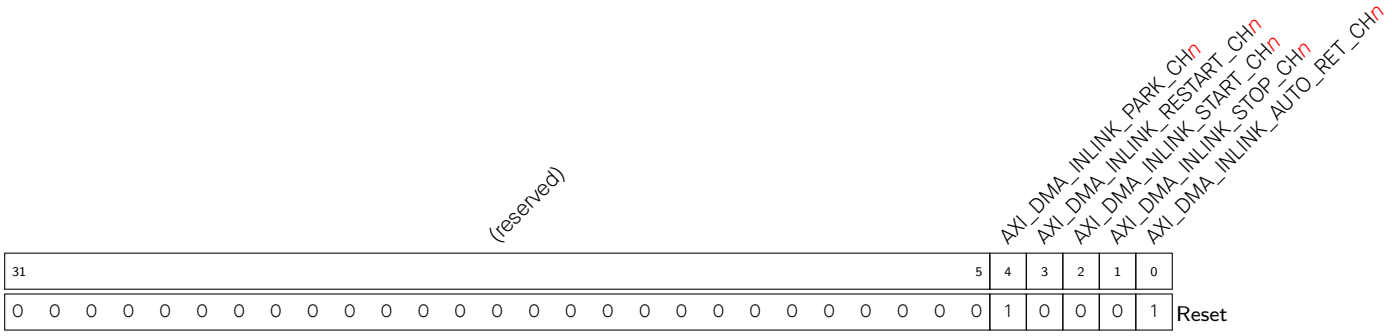
AXI_DMA_INFIFO_POP_CH*n* 配置是否从 AXI DMA FIFO 为 RX 通道 *n* 弹出数据。

0：无效值。没有作用

1：弹出

(WT)

Register 3.116. AXI_DMA_IN_LINK1_CH*n*_REG (*n*: 0-2) (0x0020+0x68**n*)



AXI_DMA_INLINK_AUTO_RET_CH*n* 配置当 RX 通道 *n* 当前接收数据发生错误时，是否返回到当前接收链表描述符地址。

0：不返回

1：返回

(R/W)

AXI_DMA_INLINK_STOP_CH*n* 配置是否停止处理 RX 通道 *n* 的描述符。

0：无效值。没有作用

1：停止

(WT)

AXI_DMA_INLINK_START_CH*n* 配置是否开始处理 RX 通道 *n* 的描述符。

0：无效值。没有作用

1：开始

(WT)

AXI_DMA_INLINK_RESTART_CH*n* 配置是否重新开始处理 RX 通道 *n* 的描述符。

0：无效值。没有作用

1：重新开始

(WT)

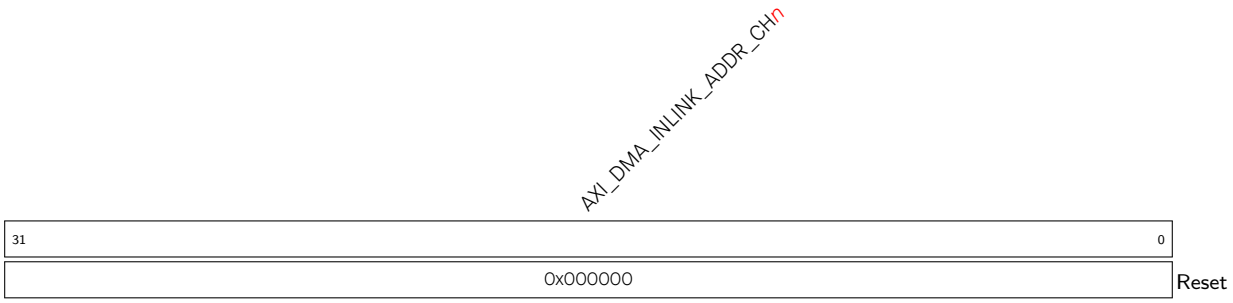
AXI_DMA_INLINK_PARK_CH*n* 表示 RX 通道 *n* 接收链表描述符 FSM 的状态。

0：运行

1：空闲

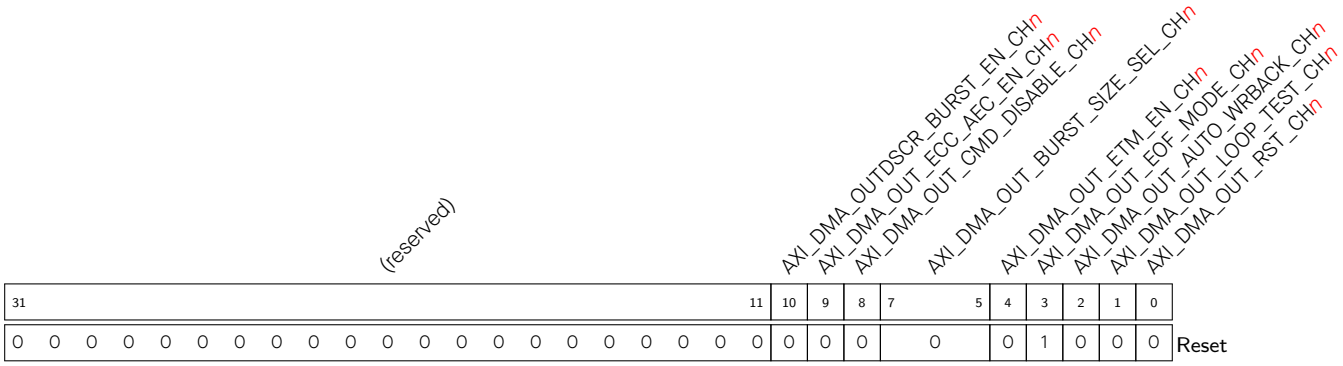
(RO)

Register 3.117. AXI_DMA_IN_LINK2_CH*n*_REG (*n*: 0-2) (0x0024+0x68**n*)



AXI_DMA_INLINK_ADDR_CH*n* 配置 RX 通道 *n* 的第一个接收链表描述符地址。(R/W)

Register 3.118. AXI_DMA_OUT_CONFO_CHn_REG (n: 0-2) (0x0148+0x68*n)


(reserved)
AXI_DMA_OUTDSCR_BURST_EN_CHn
AXI_DMA_OUT_ECC_AEC_EN_CHn
AXI_DMA_OUT_CMD_DISABLE_CHn
AXI_DMA_OUT_BURST_SIZE_SEL_CHn
AXI_DMA_OUT_ETM_EN_CHn
AXI_DMA_OUT_EOF_MODE_CHn
AXI_DMA_OUT_AUTO_WRBACK_CHn
AXI_DMA_OUT_LOOP_TEST_CHn
AXI_DMA_OUT_RST_CHn
31 11 10 9 8 7 5 4 3 2 1 0
0 0 0

Register 3.118. AXI_DMA_OUT_CONF0_CH*n*_REG (*n*: 0-2) (0x0148+0x68**n*)

接上页...

AXI_DMA_OUT_CMD_DISABLE_CH*n* 配置是否禁用 TX 通道 *n* 的命令。

0：使能命令

1：禁用命令

(R/W)

AXI_DMA_OUT_ECC_AEC_EN_CH*n* 配置是否访问 TX 通道 *n* 的 ECC 或 AES 域。

0：禁用

1：使能

(R/W)

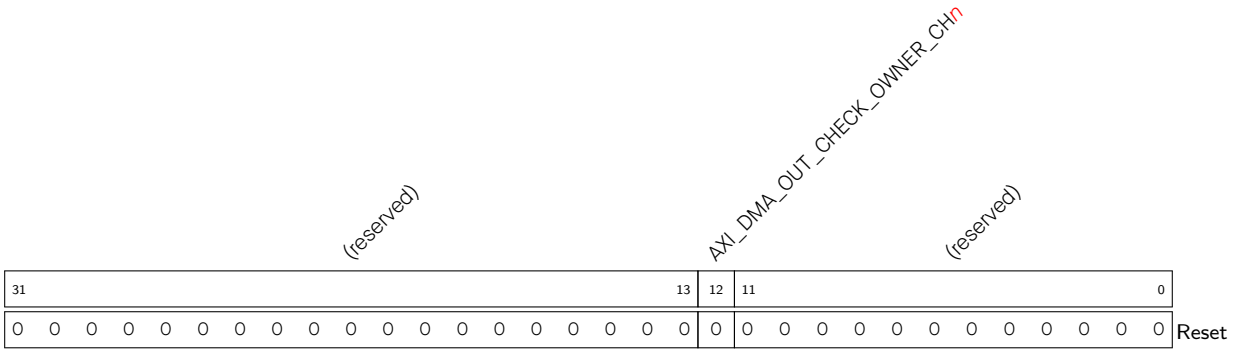
AXI_DMA_OUTDSCR_BURST_EN_CH*n* 配置是否使能 TX 通道 *n* 读取发送链表描述符的 INCR 突发传输。

0：禁用

1：使能

(R/W)

Register 3.119. AXI_DMA_OUT_CONF1_CH*n*_REG (*n*: 0-2) (0x014C+0x68**n*)



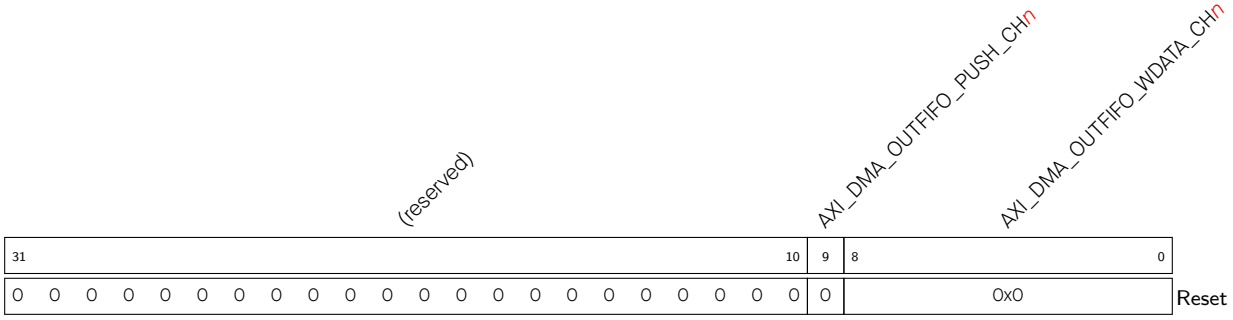
AXI_DMA_OUT_CHECK_OWNER_CH*n* 配置是否使能 TX 通道 *n* 的 owner 位检查。

0：禁用

1：使能

(R/W)

Register 3.120. AXI_DMA_OUT_PUSH_CH*n*_REG (*n*: 0-2) (0x0154+0x68**n*)



AXI_DMA_OUTFIFO_WDATA_CH*n* 待传输至 AXI DMA FIFO 的数据。(R/W)

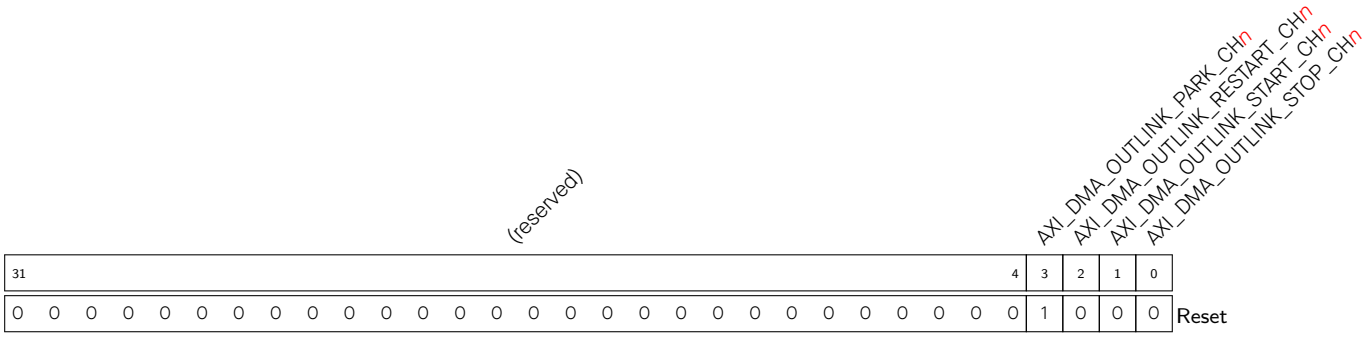
AXI_DMA_OUTFIFO_PUSH_CH*n* 配置是否将数据推入 AXI DMA FIFO（TX 通道 *n*）。

0：无效值。没有作用

1：推入

(WT)

Register 3.121. AXI_DMA_OUT_LINK1_CH*n*_REG (*n*: 0-2) (0x0158+0x68**n*)



AXI_DMA_OUTLINK_STOP_CH*n* 配置是否停止处理 TX 通道 *n* 的描述符。

0：无效值。没有作用

1：停止

(WT)

AXI_DMA_OUTLINK_START_CH*n* 配置是否开始处理 TX 通道 *n* 的描述符。

0：无效值。没有作用

1：开始

(WT)

AXI_DMA_OUTLINK_RESTART_CH*n* 配置是否重新开始处理 TX 通道 *n* 的描述符。

0：无效值。没有作用

1：重新开始

(WT)

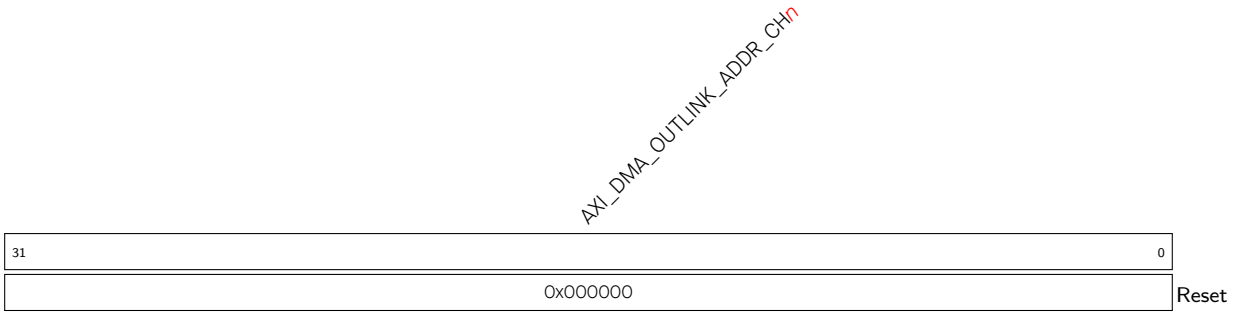
AXI_DMA_OUTLINK_PARK_CH*n* 表示 TX 通道 *n* 发送链表描述符 FSM 的状态。

0：运行

1：空闲

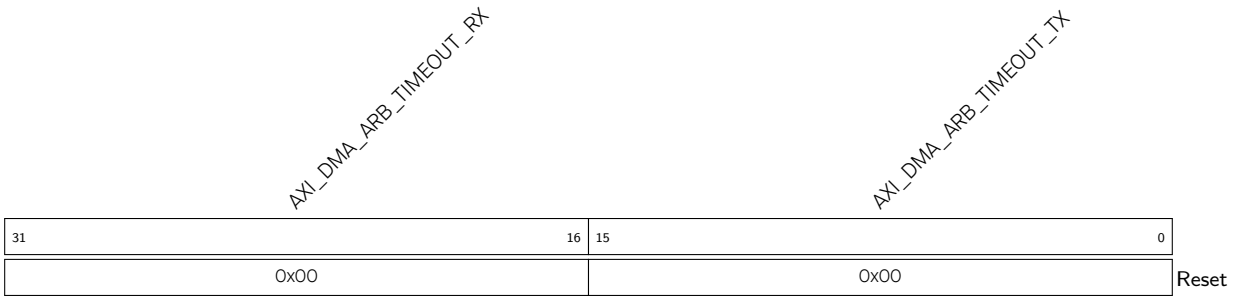
(RO)

Register 3.122. AXI_DMA_OUT_LINK2_CH*n*_REG (*n*: 0-2) (0x015C+0x68**n*)



AXI_DMA_OUTLINK_ADDR_CH*n* 配置 TX 通道 *n* 的第一个发送链表描述符地址。(R/W)

Register 3.123. AXI_DMA_ARB_TIMEOUT_REG (0x0270)



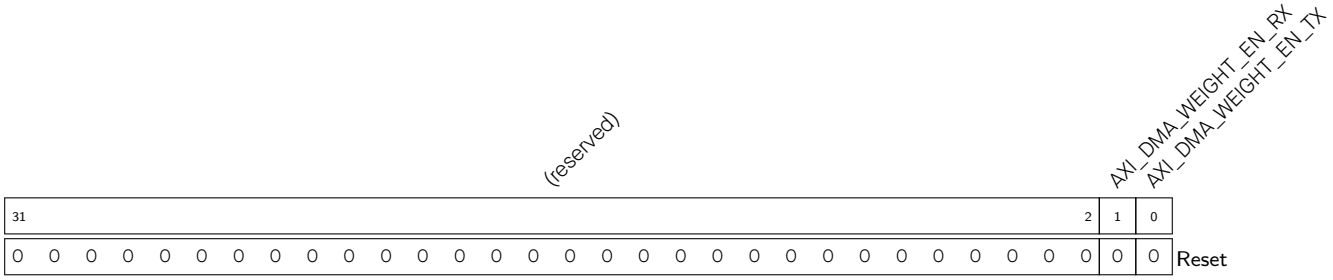
AXI_DMA_ARB_TIMEOUT_TX 配置 TX 仲裁时间片。

单位：总线时钟周期。(R/W)

AXI_DMA_ARB_TIMEOUT_RX 配置 RX 仲裁时间片。

单位：总线时钟周期。(R/W)

Register 3.124. AXI_DMA_WEIGHT_EN_REG (0x0274)



AXI_DMA_WEIGHT_EN_TX 配置是否使能 TX 通道的权重仲裁。

0：禁用

1：使能

(R/W)

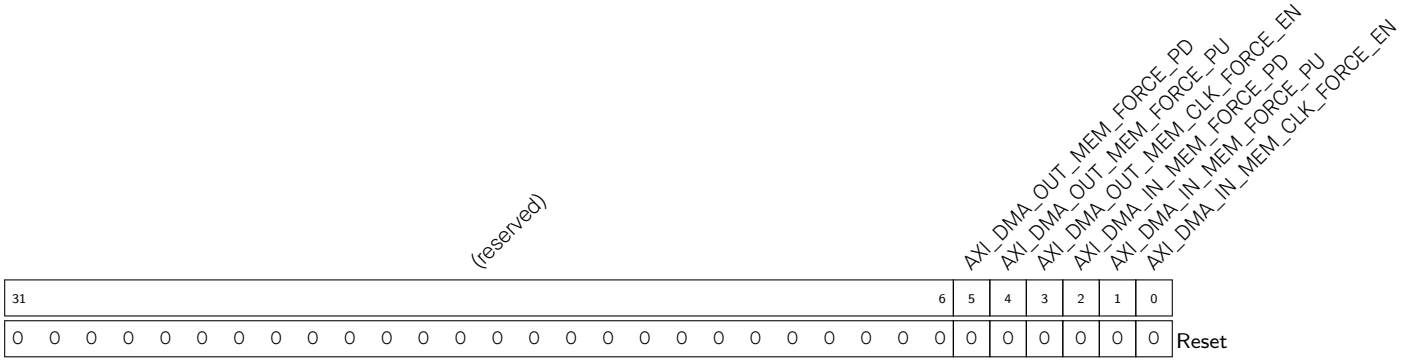
AXI_DMA_WEIGHT_EN_RX 配置是否使能 RX 通道的权重仲裁。

0：禁用

1：使能

(R/W)

Register 3.125. AXI_DMA_IN_MEM_CONF_REG (0x0278)



AXI_DMA_IN_MEM_CLK_FORCE_EN 配置是否强制开启 RX RAM 时钟。

0：不强制开启

1：强制开启

(R/W)

AXI_DMA_IN_MEM_FORCE_PU 配置是否强制 RX RAM 上电。

0：不强制上电

1：强制上电

(R/W)

AXI_DMA_IN_MEM_FORCE_PD 配置是否强制 RX RAM 断电。

0：不强制断电

1：强制断电

(R/W)

AXI_DMA_OUT_MEM_CLK_FORCE_EN 配置是否强制开启 TX RAM 时钟。

0：不强制开启

1：强制开启

(R/W)

AXI_DMA_OUT_MEM_FORCE_PU 配置是否强制 TX RAM 上电。

0：不强制上电

1：强制上电

(R/W)

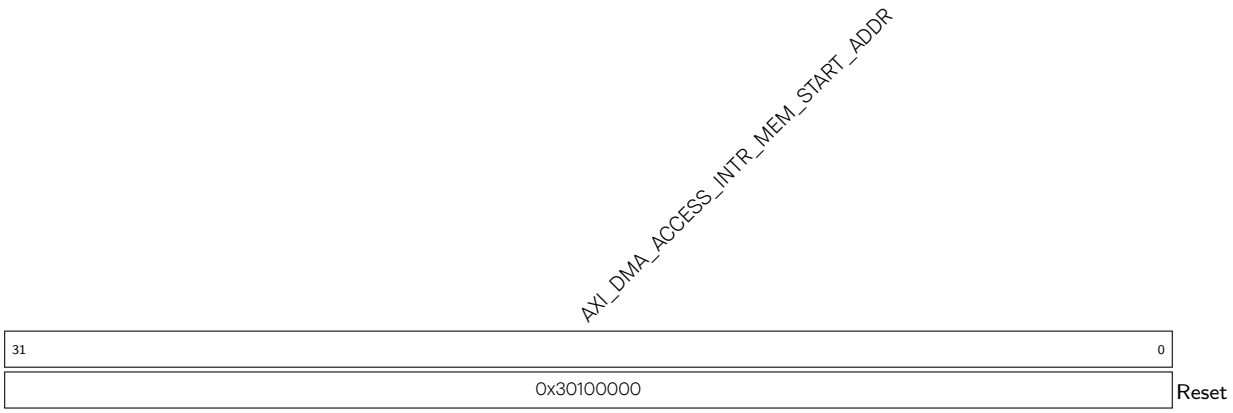
AXI_DMA_OUT_MEM_FORCE_PD 配置是否强制 TX RAM 断电。

0：不强制断电

1：强制断电

(R/W)

Register 3.126. AXI_DMA_INTR_MEM_START_ADDR_REG (0x027C)



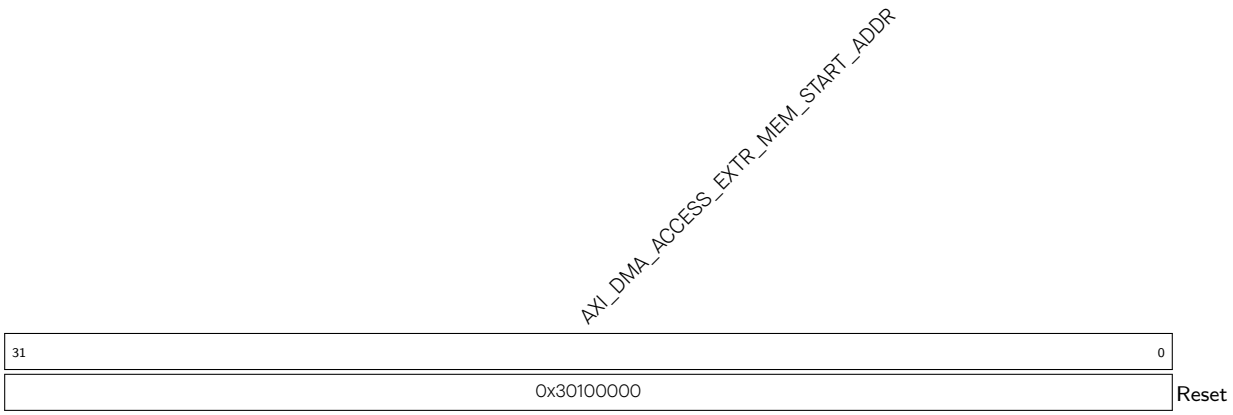
AXI_DMA_ACCESS_INTR_MEM_START_ADDR 配置内部存储器可访问地址空间的起始地址。(R/W)

Register 3.127. AXI_DMA_INTR_MEM_END_ADDR_REG (0x0280)



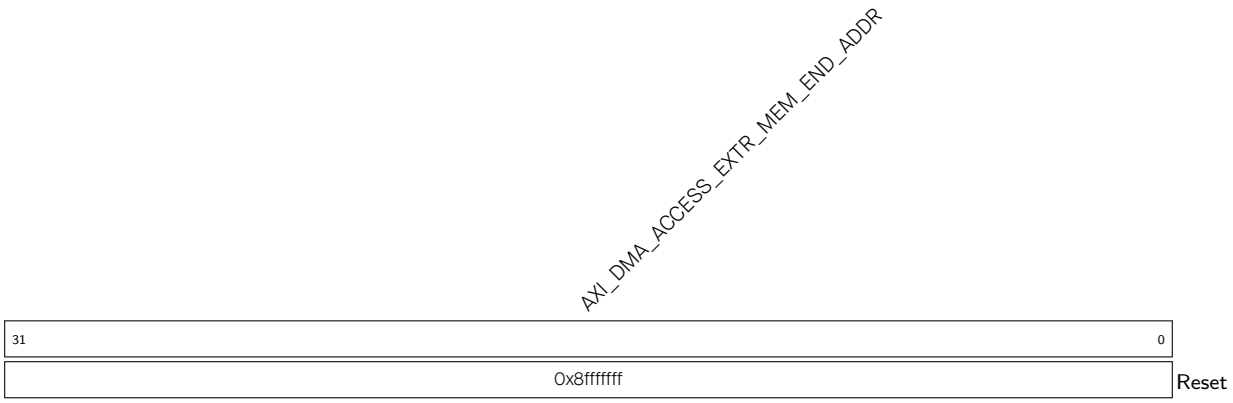
AXI_DMA_ACCESS_INTR_MEM_END_ADDR 配置内部存储器可访问地址空间的结束地址。(R/W)

Register 3.128. AXI_DMA_EXTR_MEM_START_ADDR_REG (0x0284)



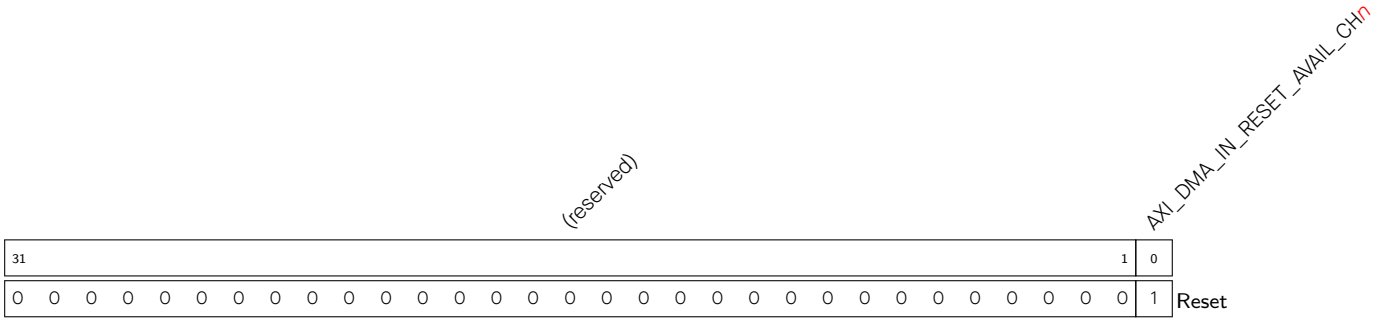
AXI_DMA_ACCESS_EXTR_MEM_START_ADDR 配置外部存储器可访问地址空间的起始地址。(R/W)

Register 3.129. AXI_DMA_EXTR_MEM_END_ADDR_REG (0x0288)



AXI_DMA_ACCESS_EXTR_MEM_END_ADDR 配置外部存储器可访问地址空间的结束地址。(R/W)

Register 3.130. AXI_DMA_IN_RESET_AVAIL_CH*n*_REG (*n*: 0-2) (0x028C+0x4**n*)



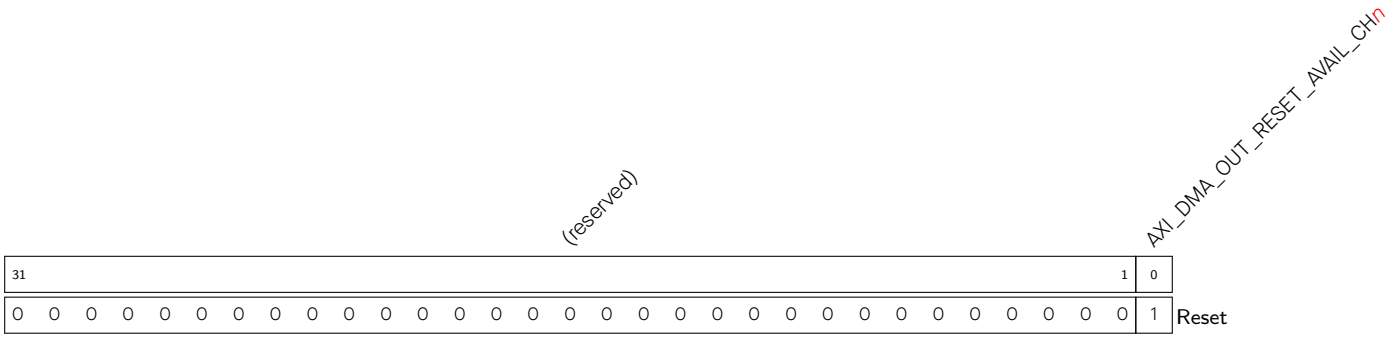
AXI_DMA_IN_RESET_AVAIL_CH*n* 表示 RX 通道 *n* 的复位是否有效。

0：无效

1：有效

(RO)

Register 3.131. AXI_DMA_OUT_RESET_AVAIL_CH*n*_REG (*n*: 0-2) (0x0298+0x4*n*)



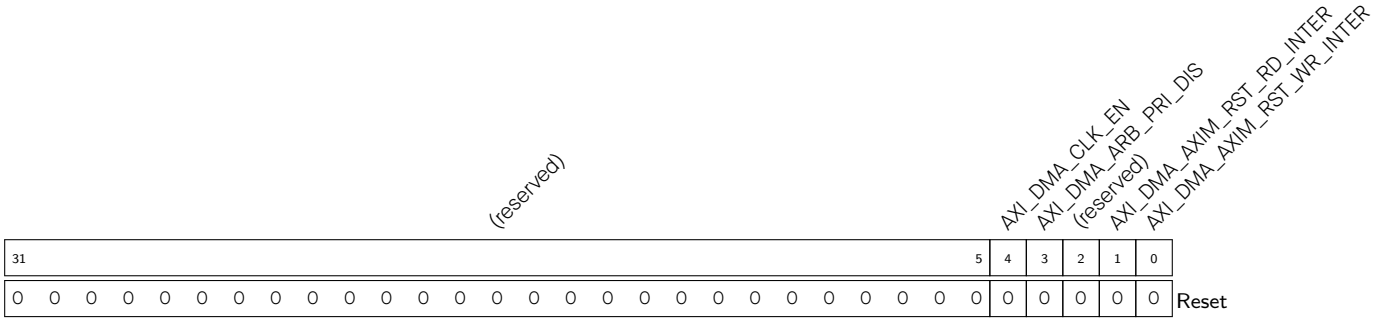
AXI_DMA_OUT_RESET_AVAIL_CH*n* 表示 TX 通道 *n* 的复位是否有效。

0：无效

1：有效

(RO)

Register 3.132. AXI_DMA_MISC_CONF_REG (0x02A8)



AXI_DMA_AXIM_RST_WR_INTER 配置内部 AXI 写 FSM 的复位状态。

0：解除复位

1：复位

(R/W)

AXI_DMA_AXIM_RST_RD_INTER 配置内部 AXI 读 FSM 的复位状态。

0：解除复位

1：复位

(R/W)

AXI_DMA_ARB_PRI_DIS 配置是否禁用固定优先级通道仲裁。

0：使能

1：禁用

(R/W)

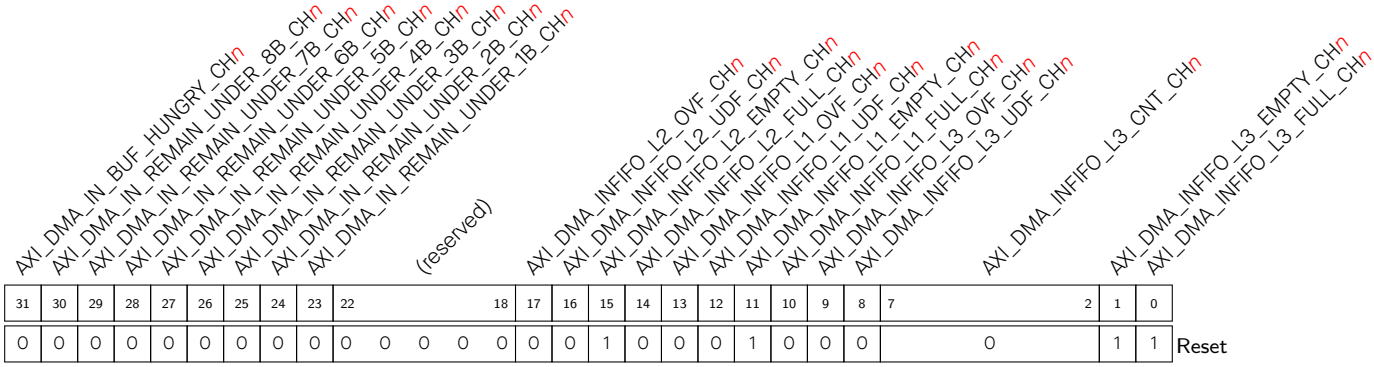
AXI_DMA_CLK_EN 配置时钟门控。

0：仅在应用写寄存器时开启时钟

1：始终强制为寄存器开启时钟

(R/W)

Register 3.133. AXI_DMA_INFIFO_STATUS_CH*n*_REG (*n*: 0-2) (0x0018+0x68**n*)



AXI_DMA_INFIFO_L3_FULL_CH*n* 表示 RX 通道 *n* 的 L3 RX FIFO 是否已满。

0：未满

1：已满

(RO)

AXI_DMA_INFIFO_L3_EMPTY_CH*n* 表示 RX 通道 *n* 的 L3 RX FIFO 是否为空。

0：非空

1：空

(RO)

AXI_DMA_INFIFO_L3_CNT_CH*n* 表示 RX 通道 *n* L3 RX FIFO 中的数据字节数。

单位：字节。(RO)

AXI_DMA_INFIFO_L3_UDF_CH*n* 表示 RX 通道 *n* 的 L3 RX FIFO 是否下溢。

0：未下溢

1：下溢

(RO)

AXI_DMA_INFIFO_L3_OVF_CH*n* 表示 RX 通道 *n* 的 L3 RX FIFO 是否溢出。

0：未溢出

1：溢出

(RO)

AXI_DMA_INFIFO_L1_FULL_CH*n* 表示 RX 通道 *n* 的 L1 RX FIFO 是否已满。

0：未满

1：已满

(RO)

AXI_DMA_INFIFO_L1_EMPTY_CH*n* 表示 RX 通道 *n* 的 L1 RX FIFO 是否为空。

0：非空

1：空

(RO)

AXI_DMA_INFIFO_L1_UDF_CH*n* 表示 RX 通道 *n* 的 L1 RX FIFO 是否下溢。

0：未下溢

1：下溢

(RO)

见下页...

Register 3.133. AXI_DMA_INFIFO_STATUS_CH*n*_REG (*n*: 0-2) (0x0018+0x68**n*)

接上页...

AXI_DMA_INFIFO_L1_OVF_CH*n* 表示 RX 通道 *n* 的 L1 RX FIFO 是否溢出。

0：未溢出

1：溢出

(RO)

AXI_DMA_INFIFO_L2_FULL_CH*n* 表示 RX 通道 *n* 的 L2 RX FIFO 是否已满。

0：未满

1：已满

(RO)

AXI_DMA_INFIFO_L2_EMPTY_CH*n* 表示 RX 通道 *n* 的 L2 RX FIFO 是否为空。

0：非空

1：空

(RO)

AXI_DMA_INFIFO_L2_UDF_CH*n* 表示 RX 通道 *n* 的 L2 RX FIFO 是否下溢。

0：未下溢

1：下溢

(RO)

AXI_DMA_INFIFO_L2_OVF_CH*n* 表示 RX 通道 *n* 的 L2 RX FIFO 是否溢出。

0：未溢出

1：溢出

(RO)

AXI_DMA_IN_REMAIN_UNDER_1B_CH*n* 保留。(RO)

AXI_DMA_IN_REMAIN_UNDER_2B_CH*n* 保留。(RO)

AXI_DMA_IN_REMAIN_UNDER_3B_CH*n* 保留。(RO)

AXI_DMA_IN_REMAIN_UNDER_4B_CH*n* 保留。(RO)

AXI_DMA_IN_REMAIN_UNDER_5B_CH*n* 保留。(RO)

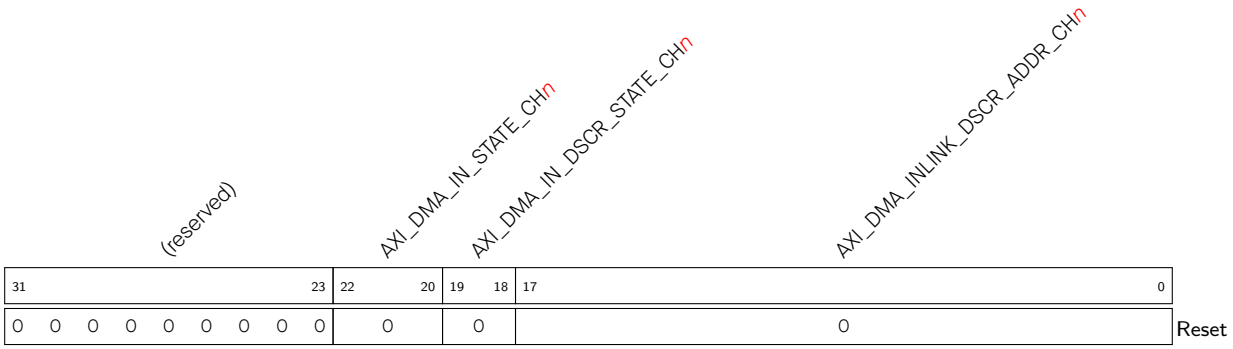
AXI_DMA_IN_REMAIN_UNDER_6B_CH*n* 保留。(RO)

AXI_DMA_IN_REMAIN_UNDER_7B_CH*n* 保留。(RO)

AXI_DMA_IN_REMAIN_UNDER_8B_CH*n* 保留。(RO)

AXI_DMA_IN_BUF_HUNGRY_CH*n* 保留。(RO)

Register 3.134. AXI_DMA_IN_STATE_CH*n*_REG (*n*: 0-2) (0x0028+0x68**n*)

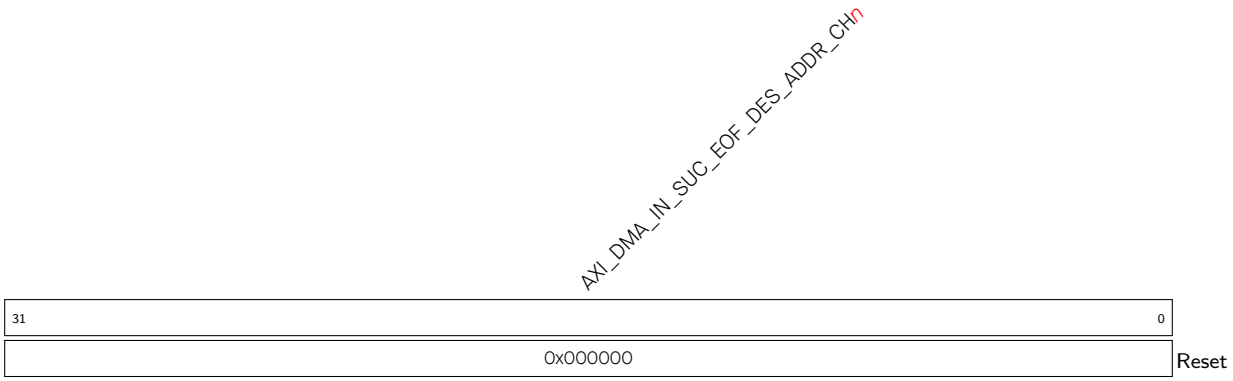


AXI_DMA_INLINK_DSCR_ADDR_CH*n* 表示下一个待处理接收链表描述符地址的低 18 位。(RO)

AXI_DMA_IN_DSCR_STATE_CH*n* 保留。(RO)

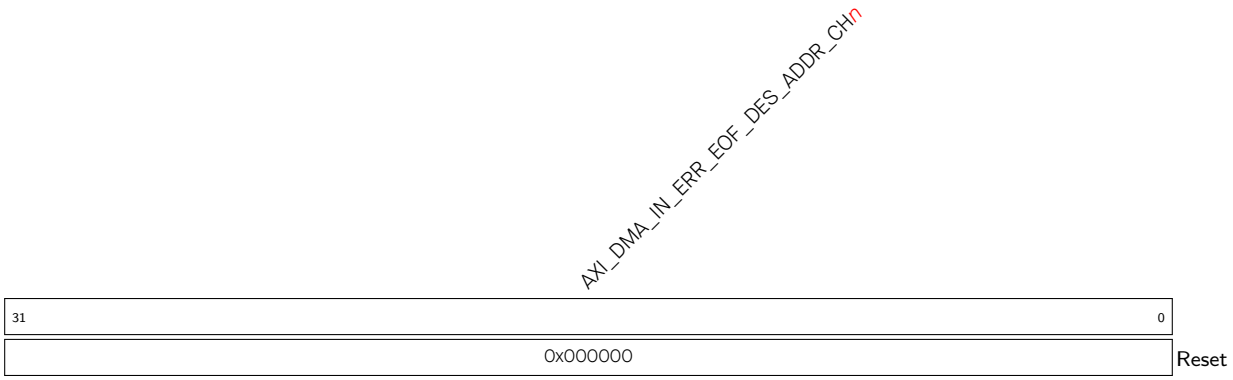
AXI_DMA_IN_STATE_CH*n* 保留。(RO)

Register 3.135. AXI_DMA_IN_SUC_EOF_DES_ADDR_CH*n*_REG (*n*: 0-2) (0x002C+0x68**n*)



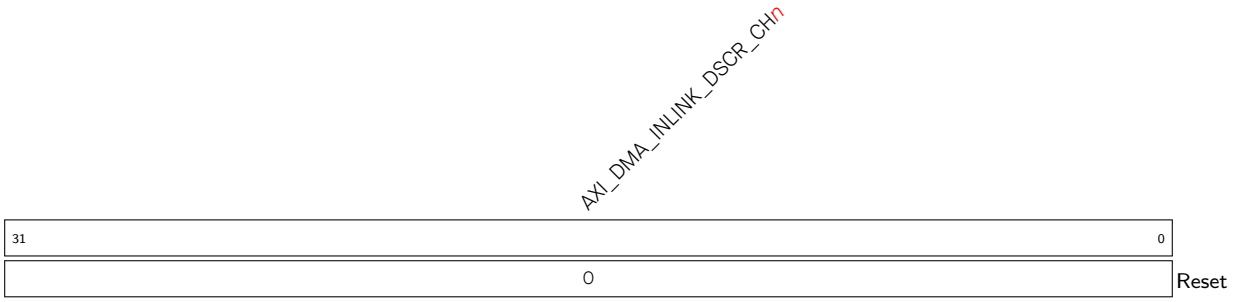
AXI_DMA_IN_SUC_EOF_DES_ADDR_CH*n* 表示 RX 通道 *n* 中 EOF 标志为 1 的接收链表描述符地址。(RO)

Register 3.136. AXI_DMA_IN_ERR_EOF_DES_ADDR_CH*n*_REG (*n*: 0-2) (0x0030+0x68**n*)



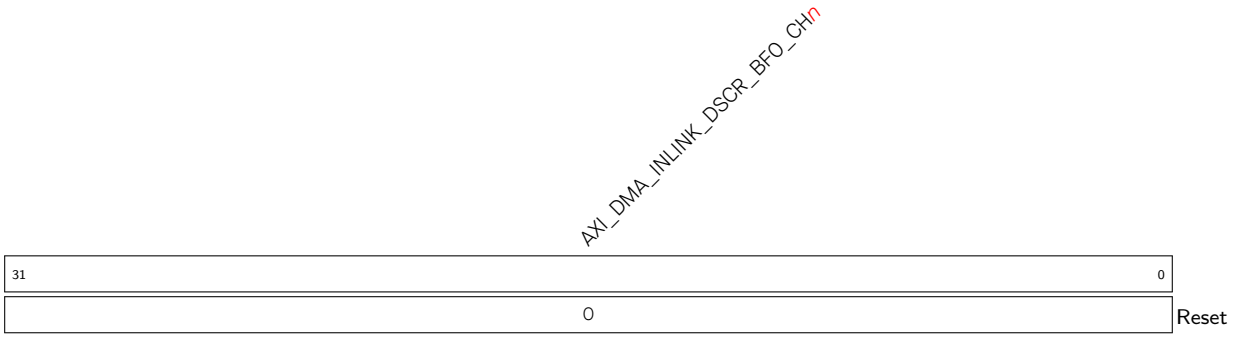
AXI_DMA_IN_ERR_EOF_DES_ADDR_CH*n* 表示 RX 通道 *n* 当前接收数据发生错误时的接收链表描述符地址。(RO)

Register 3.137. AXI_DMA_IN_DSCR_CH*n*_REG (*n*: 0-2) (0x0034+0x68**n*)



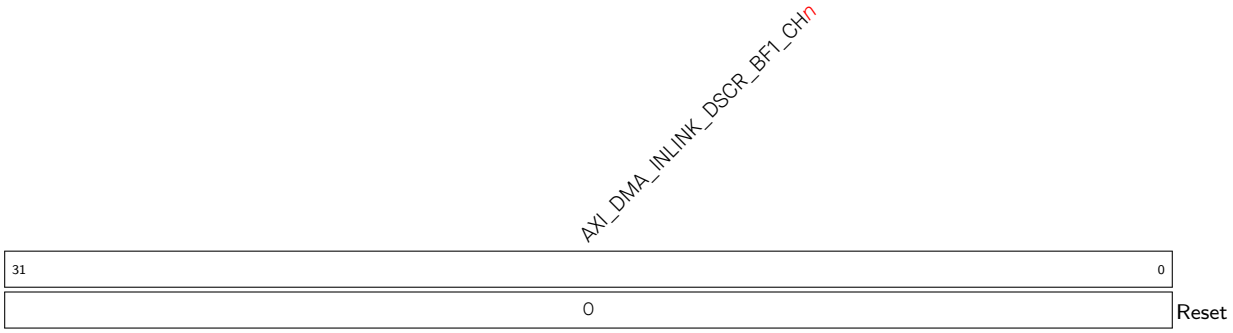
AXI_DMA_INLINK_DSCR_CH*n* 表示当前已预读取的接收链表描述符所指向的下一个接收链表描述符地址。(RO)

Register 3.138. AXI_DMA_IN_DSCR_BF0_CH*n*_REG (*n*: 0-2) (0x0038+0x68**n*)



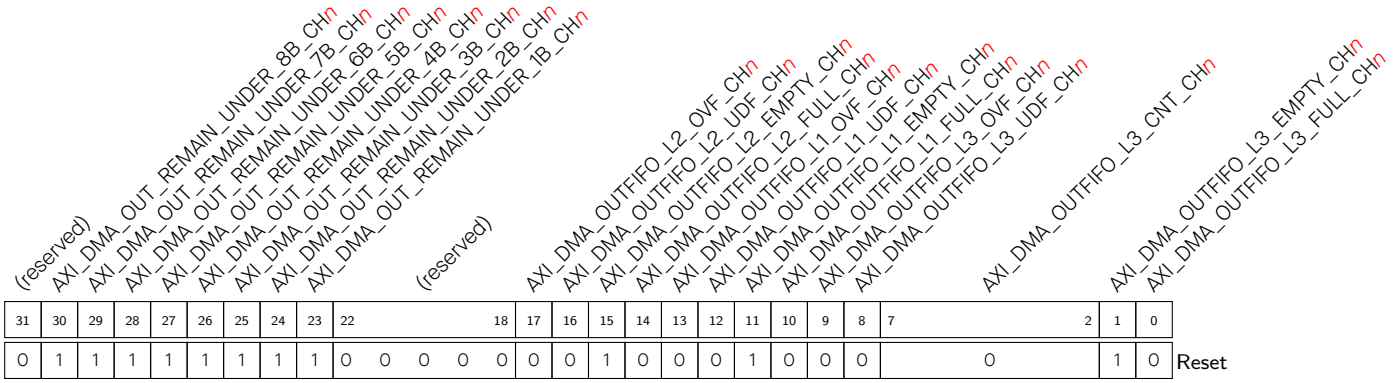
AXI_DMA_INLINK_DSCR_BF0_CH*n* 表示 RX 通道 *n* 当前已预读取的接收链表描述符地址。(RO)

Register 3.139. AXI_DMA_IN_DSCR_BF1_CH*n*_REG (*n*: 0-2) (0x003C+0x68**n*)



AXI_DMA_INLINK_DSCR_BF1_CH*n* 表示 RX 通道 *n* 上一个已预读取的接收链表描述符地址。(RO)

Register 3.140. AXI_DMA_OUTFIFO_STATUS_CH*n*_REG (*n*: 0-2) (0x0150+0x68*n)



AXI_DMA_OUTFIFO_L3_FULL_CH*n* 表示 TX 通道 *n* 的 L3 TX FIFO 是否已满。

0：未满

1：已满

(RO)

AXI_DMA_OUTFIFO_L3_EMPTY_CH*n* 表示 TX 通道 *n* 的 L3 TX FIFO 是否为空。

0：非空

1：空

(RO)

AXI_DMA_OUTFIFO_L3_CNT_CH*n* 表示 TX 通道 *n* L3 TX FIFO 中的数据字节数。

单位：字节。(RO)

AXI_DMA_OUTFIFO_L3_UDF_CH*n* 表示 TX 通道 *n* 的 L3 TX FIFO 是否下溢。

0：未下溢

1：下溢

(RO)

AXI_DMA_OUTFIFO_L3_OVF_CH*n* 表示 TX 通道 *n* 的 L3 TX FIFO 是否溢出。

0：未溢出

1：溢出

(RO)

AXI_DMA_OUTFIFO_L1_FULL_CH*n* 表示 TX 通道 *n* 的 L1 TX FIFO 是否已满。

0：未满

1：已满

(RO)

AXI_DMA_OUTFIFO_L1_EMPTY_CH*n* 表示 TX 通道 *n* 的 L1 TX FIFO 是否为空。

0：非空

1：空

(RO)

AXI_DMA_OUTFIFO_L1_UDF_CH*n* 表示 TX 通道 *n* 的 L1 TX FIFO 是否下溢。

0：未下溢

1：下溢

(RO)

见下页...

Register 3.140. AXI_DMA_OUTFIFO_STATUS_CH*n*_REG (*n*: 0-2) (0x0150+0x68***n*)

接上页...

AXI_DMA_OUTFIFO_L1_OVF_CH*n* 表示 TX 通道 *n* 的 L1 TX FIFO 是否溢出。

0：未溢出

1：溢出

(RO)

AXI_DMA_OUTFIFO_L2_FULL_CH*n* 表示 TX 通道 *n* 的 L2 TX FIFO 是否已满。

0：未满

1：已满

(RO)

AXI_DMA_OUTFIFO_L2_EMPTY_CH*n* 表示 TX 通道 *n* 的 L2 TX FIFO 是否为空。

0：非空

1：空

(RO)

AXI_DMA_OUTFIFO_L2_UDF_CH*n* 表示 TX 通道 *n* 的 L2 TX FIFO 是否下溢。

0：未下溢

1：下溢

(RO)

AXI_DMA_OUTFIFO_L2_OVF_CH*n* 表示 TX 通道 *n* 的 L2 TX FIFO 是否溢出。

0：未溢出

1：溢出

(RO)

AXI_DMA_OUT_REMAIN_UNDER_1B_CH*n* 保留。(RO)

AXI_DMA_OUT_REMAIN_UNDER_2B_CH*n* 保留。(RO)

AXI_DMA_OUT_REMAIN_UNDER_3B_CH*n* 保留。(RO)

AXI_DMA_OUT_REMAIN_UNDER_4B_CH*n* 保留。(RO)

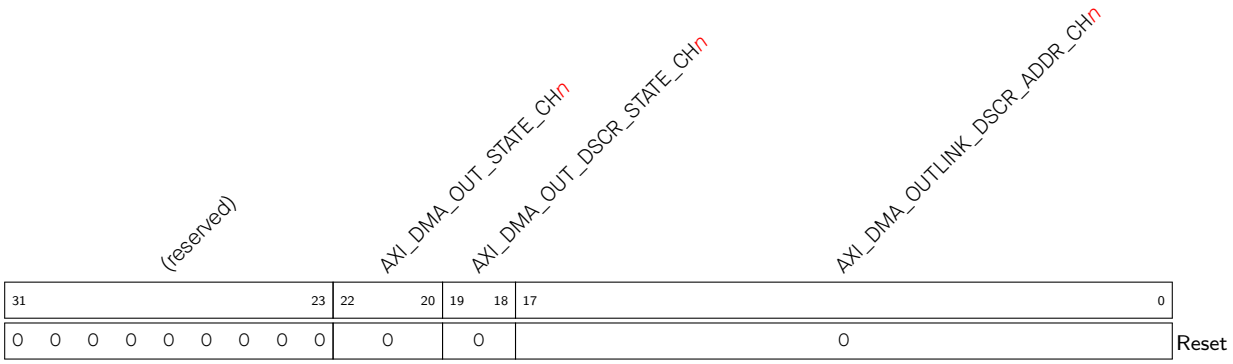
AXI_DMA_OUT_REMAIN_UNDER_5B_CH*n* 保留。(RO)

AXI_DMA_OUT_REMAIN_UNDER_6B_CH*n* 保留。(RO)

AXI_DMA_OUT_REMAIN_UNDER_7B_CH*n* 保留。(RO)

AXI_DMA_OUT_REMAIN_UNDER_8B_CH*n* 保留。(RO)

Register 3.141. AXI_DMA_OUT_STATE_CH*n*_REG (*n*: 0-2) (0x0160+0x68**n*)

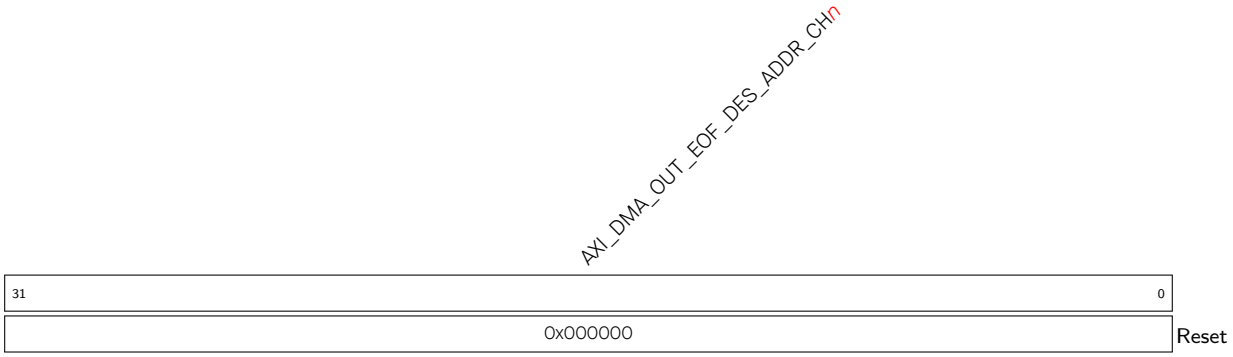


AXI_DMA_OUTLINK_DSCR_ADDR_CH*n* 表示下一个待处理发送链表描述符地址的低 18 位。(RO)

AXI_DMA_OUT_DSCR_STATE_CH*n* 保留。(RO)

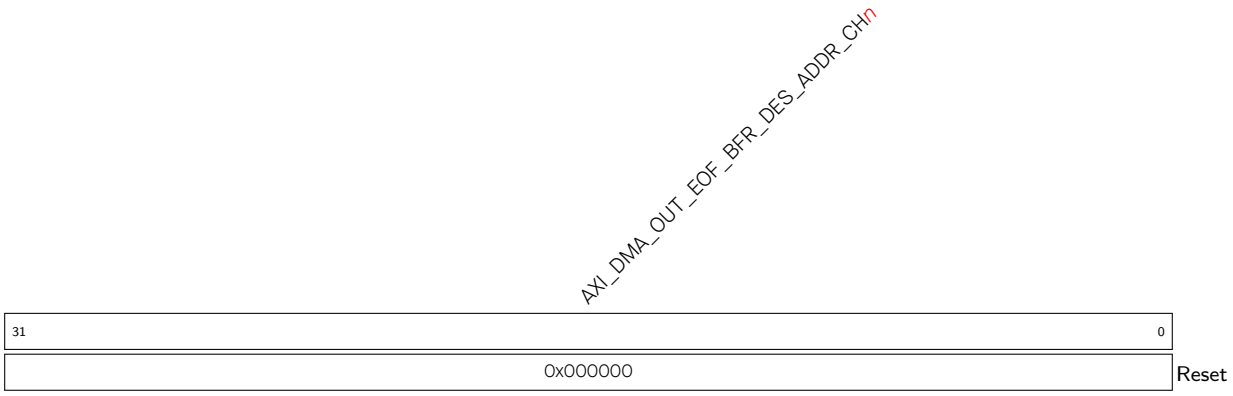
AXI_DMA_OUT_STATE_CH*n* 保留。(RO)

Register 3.142. AXI_DMA_OUT_EOF_DES_ADDR_CH*n*_REG (*n*: 0-2) (0x0164+0x68**n*)



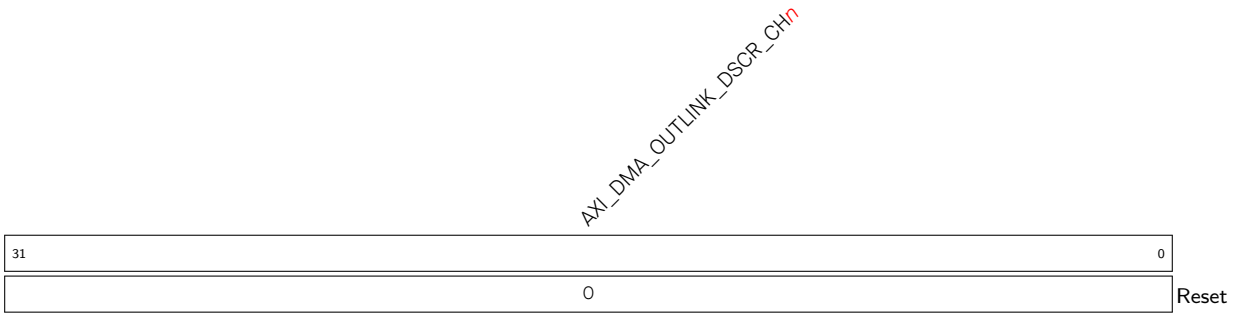
AXI_DMA_OUT_EOF_DES_ADDR_CH*n* 表示 TX 通道 *n* 中 EOF 标志为 1 的发送链表描述符地址。(RO)

Register 3.143. AXI_DMA_OUT_EOF_BFR_DES_ADDR_CH*n*_REG (*n*: 0-2) (0x0168+0x68**n*)



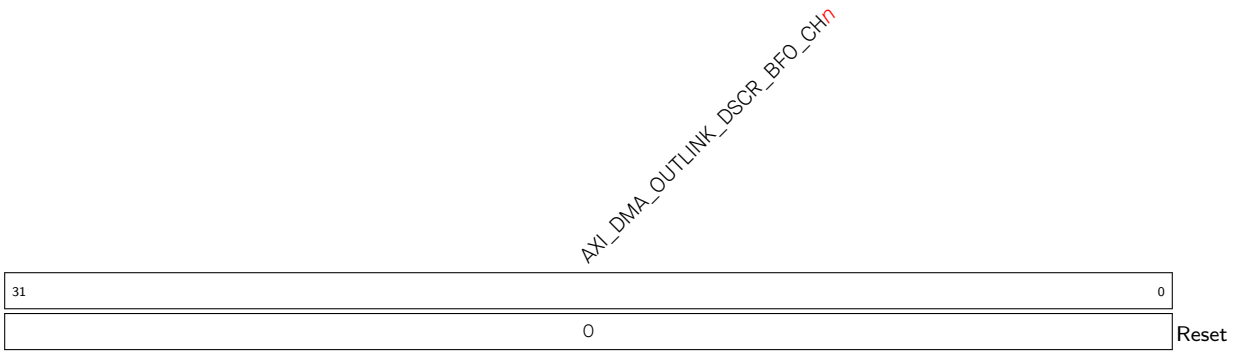
AXI_DMA_OUT_EOF_BFR_DES_ADDR_CH*n* 表示 TX 通道 *n* 最后一个发送链表描述符之前的发送链表描述符地址。(RO)

Register 3.144. AXI_DMA_OUT_DSCR_CH*n*_REG (*n*: 0-2) (0x016C+0x68**n*)



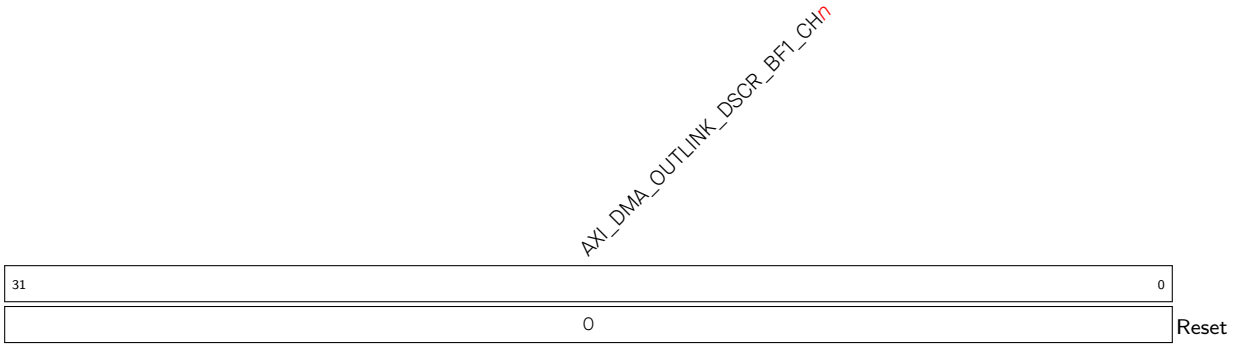
AXI_DMA_OUTLINK_DSCR_CH*n* 表示当前已预读取的发送链表描述符所指向的下一个发送链表描述符地址。(RO)

Register 3.145. AXI_DMA_OUT_DSCR_BF0_CH*n*_REG (*n*: 0-2) (0x0170+0x68**n*)



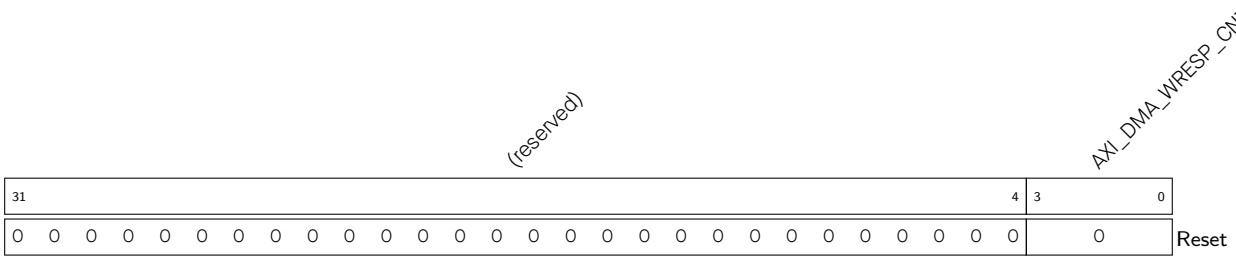
AXI_DMA_OUTLINK_DSCR_BF0_CH*n* 表示 TX 通道 *n* 当前已预读取的发送链表描述符地址。(RO)

Register 3.146. AXI_DMA_OUT_DSCR_BF1_CH*n*_REG (*n*: 0-2) (0x0174+0x68**n*)



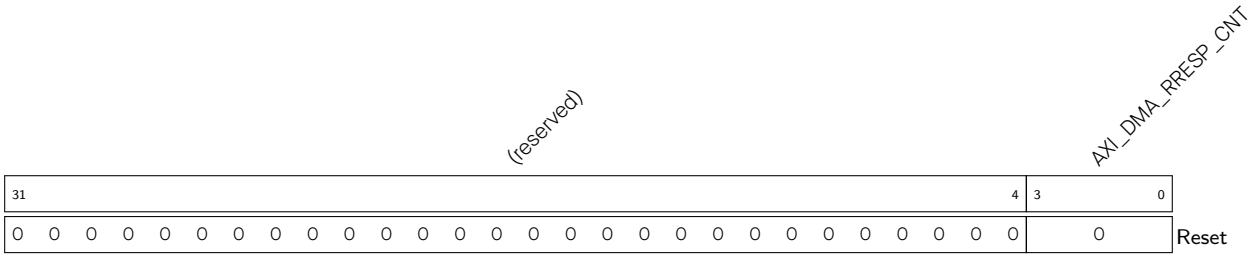
AXI_DMA_OUTLINK_DSCR_BF1_CH*n* 表示 TX 通道 *n* 上一个已预读取的发送链表描述符地址。(RO)

Register 3.147. AXI_DMA_WRESP_CNT_REG (0x02B8)



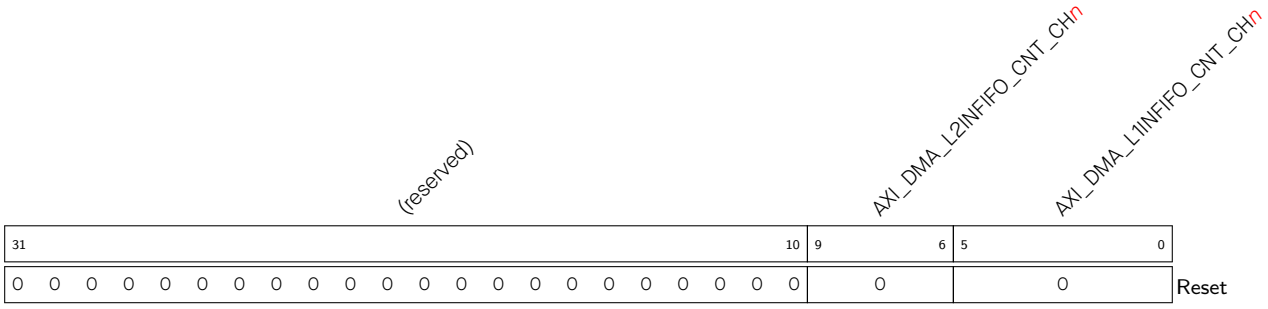
AXI_DMA_WRESP_CNT 表示 AXI 写响应计数。(RO)

Register 3.148. AXI_DMA_RRESP_CNT_REG (0x02BC)



AXI_DMA_RRESP_CNT 表示 AXI 读响应计数。(RO)

Register 3.149. AXI_DMA_INFIFO_STATUS1_CH*n*_REG (*n*: 0-2) (0x02C0+0x4*n*)



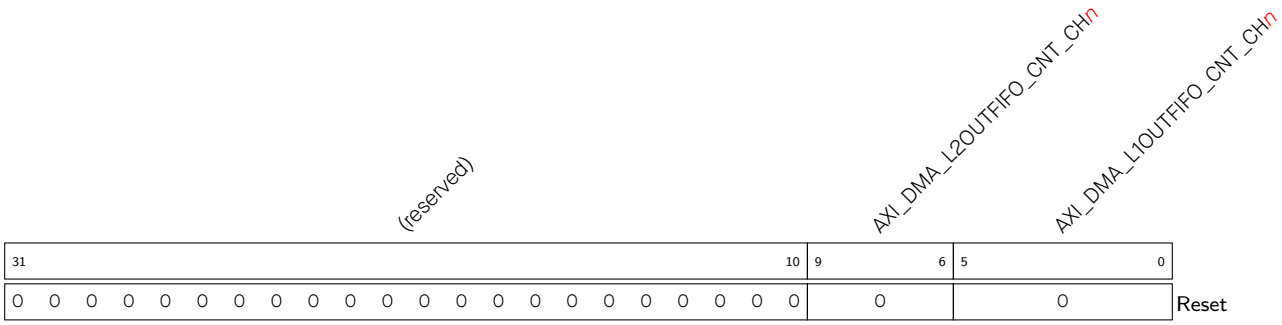
AXI_DMA_L1INFIFO_CNT_CH*n* 表示 RX 通道 *n* RX FIFO 中的数据字节数。

单位：字节。(RO)

AXI_DMA_L2INFIFO_CNT_CH*n* 表示 RX 通道 *n* RX FIFO 中的数据字节数。

单位：字节。(RO)

Register 3.150. AXI_DMA_OUTFIFO_STATUS1_CH*n*_REG (*n*: 0-2) (0x02CC+0x4*n*)



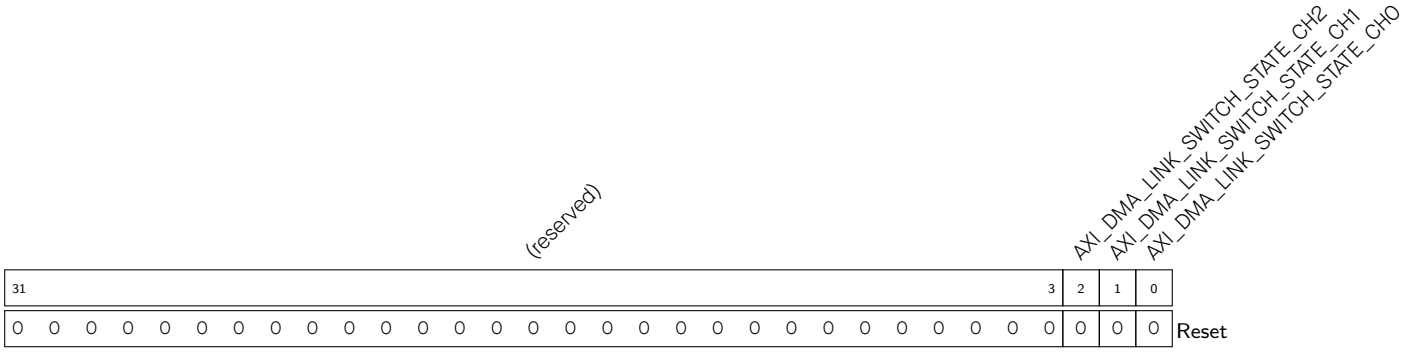
AXI_DMA_L1OUTFIFO_CNT_CH*n* 表示 TX 通道 *n* TX FIFO 中的数据字节数。

单位：字节。(RO)

AXI_DMA_L2OUTFIFO_CNT_CH*n* 表示 TX 通道 *n* TX FIFO 中的数据字节数。

单位：字节。(RO)

Register 3.151. AXI_DMA_LINK_SWITCH_STATE_REG (0x02DC)



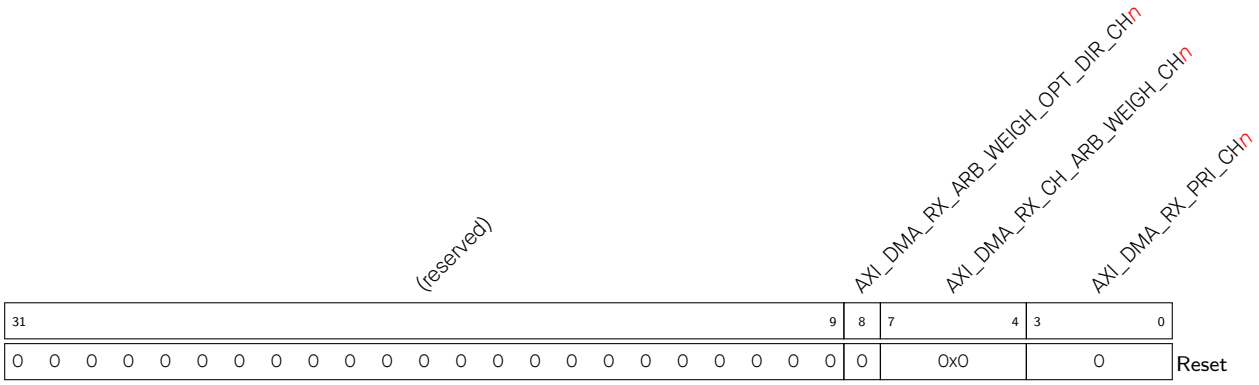
AXI_DMA_LINK_SWITCH_STATE_CH*n* (*n*: 0-2) 配置是否使能通道 *n* 的链表切换。

0：禁用

1：使能

(R/W)

Register 3.152. AXI_DMA_IN_PRI_CH*n*_REG (*n*: 0-2) (0x0040+0x68**n*)



AXI_DMA_RX_PRI_CH*n* 配置 RX 通道 *n* 的优先级。数值越大，优先级越高。取值范围：0 ~ 15。(R/W)

AXI_DMA_RX_CH_ARB_WEIGH_CH*n* 配置 RX 通道 *n* 的仲裁权重。

单位：令牌。(R/W)

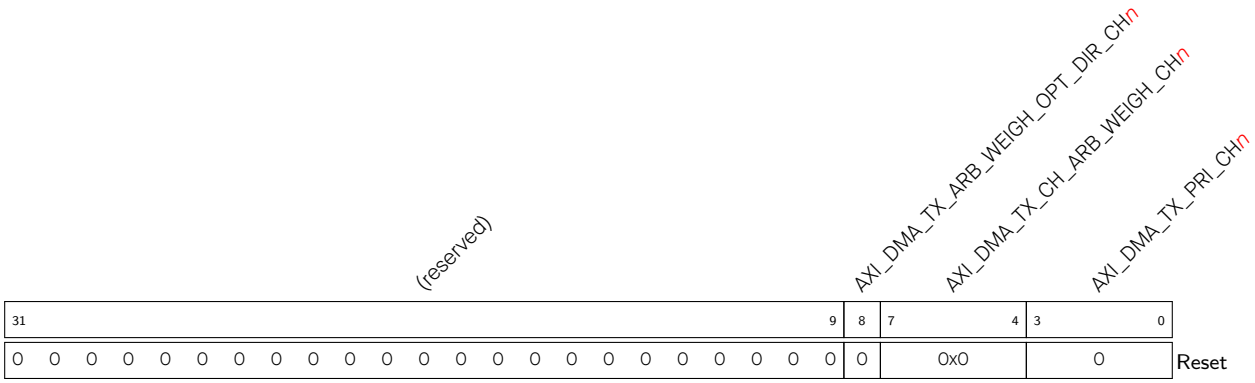
AXI_DMA_RX_ARB_WEIGH_OPT_DIR_CH*n* 配置是否使能 RX 通道 *n* 的权重仲裁优化。

0：使能

1：禁用

(R/W)

Register 3.153. AXI_DMA_OUT_PRI_CH*n*_REG (*n*: 0-2) (0x0178+0x68**n*)



AXI_DMA_TX_PRI_CH*n* 配置 TX 通道 *n* 的优先级。数值越大，优先级越高。取值范围：0 ~ 15。(R/W)

AXI_DMA_TX_CH_ARB_WEIGH_CH*n* 配置 TX 通道 *n* 的仲裁权重。

单位：令牌。(R/W)

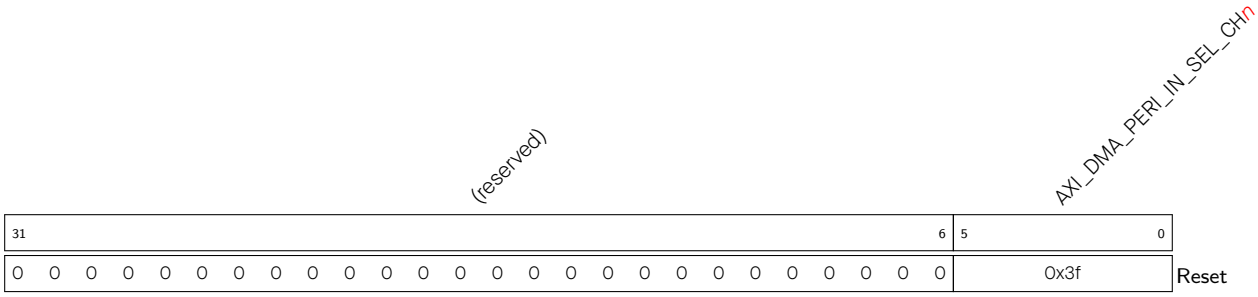
AXI_DMA_TX_ARB_WEIGH_OPT_DIR_CH*n* 配置是否使能 TX 通道 *n* 的权重仲裁优化。

0：使能

1：禁用

(R/W)

Register 3.154. AXI_DMA_IN_PERI_SEL_CH*n*_REG (*n*: 0-2) (0x0044+0x68**n*)



AXI_DMA_PERI_IN_SEL_CH*n* 配置连接至 RX 通道 *n* 的外设。

0：LCD 或 CAM

1：SPI2

2：SPI3

3：PARLIO

4：AES

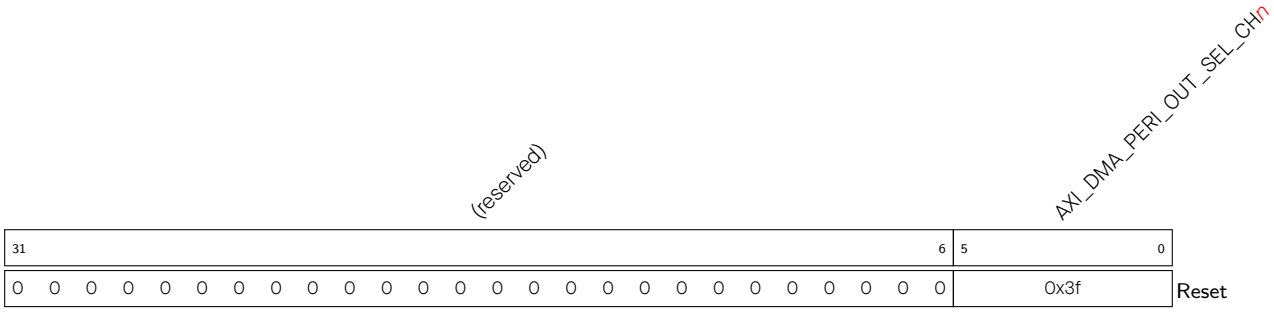
5：SHA

6 ~ 15：Dummy-6 ~ Dummy-15

16 ~ 63：无效值

(R/W)

Register 3.155. AXI_DMA_OUT_PERI_SEL_CH*n*_REG (*n*: 0-2) (0x017C+0x68**n*)



AXI_DMA_PERI_OUT_SEL_CH*n* 配置连接至 TX 通道 *n* 的外设。

0：LCD 或 CAM

1：SPI2

2：SPI3

3：PARLIO

4：AES

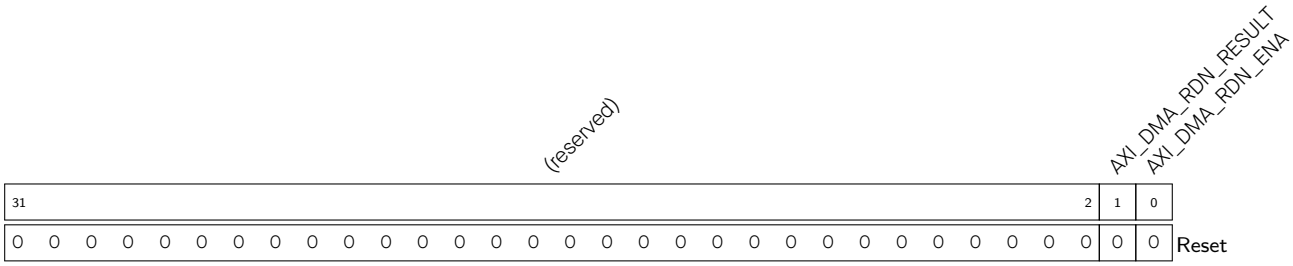
5：SHA

6 ~ 15：Dummy-6 ~ Dummy-15

16 ~ 63：无效值

(R/W)

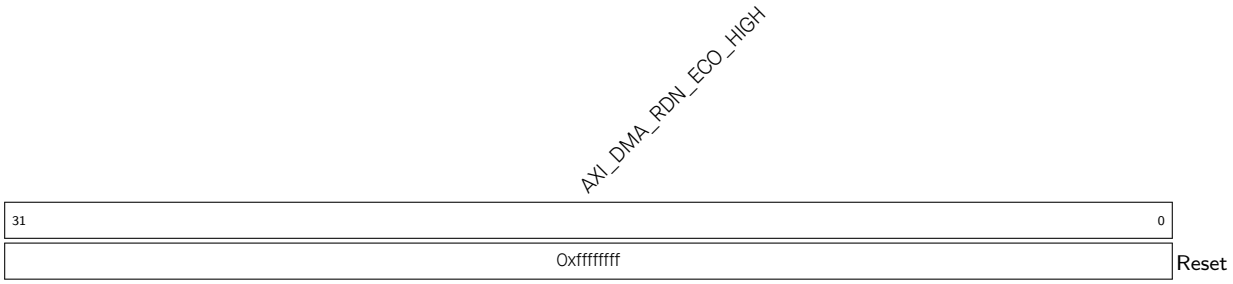
Register 3.156. AXI_DMA_RDN_RESULT_REG (0x02AC)



AXI_DMA_RDN_ENA 保留。(R/W)

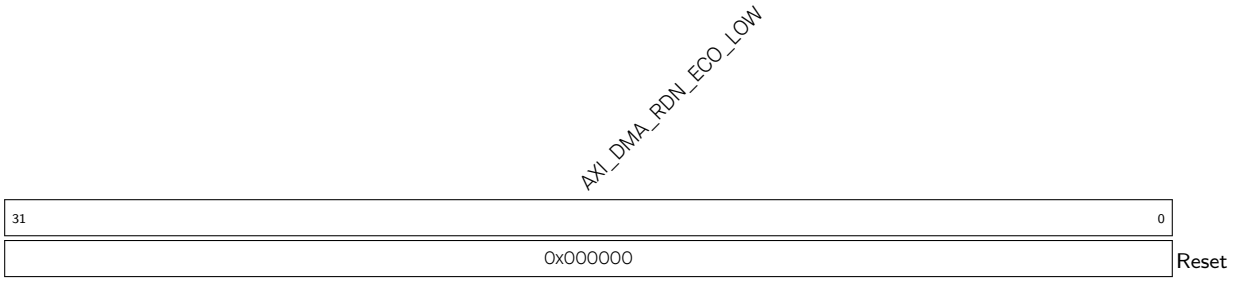
AXI_DMA_RDN_RESULT 保留。(RO)

Register 3.157. AXI_DMA_RDN_ECO_HIGH_REG (0x02B0)



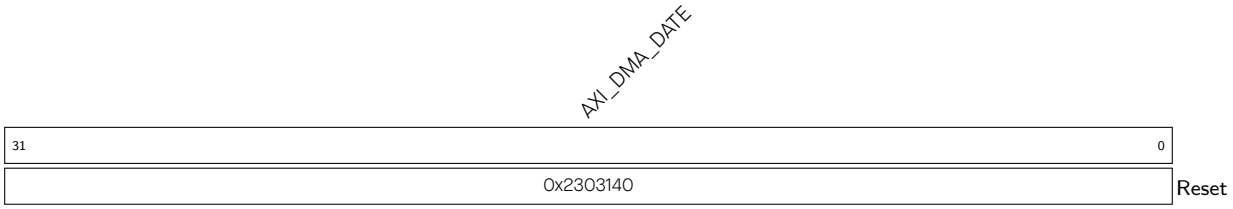
AXI_DMA_RDN_ECO_HIGH 保留。(R/W)

Register 3.158. AXI_DMA_RDN_ECO_LOW_REG (0x02B4)



AXI_DMA_RDN_ECO_LOW 保留。(R/W)

Register 3.159. AXI_DMA_DATE_REG (0x02D8)



AXI_DMA_DATE 版本控制寄存器。(R/W)

第 4 章

2D-DMA 控制器 (2D-DMA)

4.1 概述

2D-DMA 控制器是专用于二维图像处理的 DMA，在支持 GDMA-AXI 的全部功能基础上，增加了宏块重排 (Reorder) 和颜色空间转换（Color Space Convert: CSC）功能，能够更好地支持 JPEG 和 PPA 外设的数据传输需求。2D-DMA 支持存储器到存储器的传输，可以将宏块从存储器的一段地址空间搬运到另一段地址空间，并完成颜色空间转换。

4.2 特性

- 1 个 AXI 主机接口
- 支持首地址非对齐的数据传输
- 支持存储到存储、外设到存储 (RX)、以及存储到外设 (TX) 的数据传输
- 包含 4 个存储到外设通道，3 个外设到存储通道
- 支持 PPA 和 JPEG 图像编解码器外设
- 支持宏块重排序功能
- 支持颜色空间转换功能
- 支持通道优先级、权重配置

4.3 架构

ESP32-S31 中只有 JPEG 和 PPA 两个外设支持 2D-DMA 功能。2D-DMA 控制器与 CPU 的数据总线使用相同的地址空间访问存储器。图 4.3-1 为 2D-DMA 控制器基本架构图。

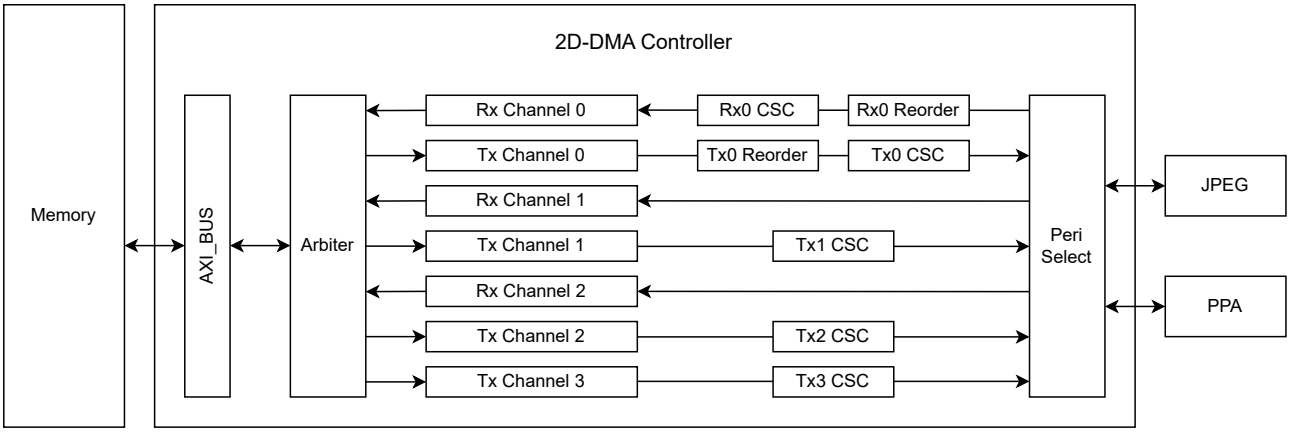


图 4.3-1. 2D-DMA 基本架构图

2D-DMA 控制器有 7 个独立的通道，其中包括 4 个发送通道和 3 个接收通道。每个通道可选择与不同的外设相连，从而实现通道资源由外设共享。

2D-DMA 控制器支持宏块重排 (Reorder) 和颜色空间转换 (CSC) 功能，仅发送通道 0 和接收通道 0 支持宏块重排，仅发送通道 0、1、2、3 和接收通道 0 支持颜色空间转换功能。

2D-DMA 通过 AXI_BUS 将数据写入存储器，或者从存储器读取数据，其中存储器包含片内存储器和片外存储器。在通过总线传输数据之前，2D-DMA 采用可配置的仲裁机制对每个通道的读写请求进行仲裁。内外部存储器的具体使用范围详见章节 3 *系统和存储器 [to be added later]*。

软件可以通过挂载链表的方式来使用 2D-DMA 控制器。链表可以存储在片内或片外存储器中。这些链表包括 outlink*n* 与 inlink*n*，本文以 *n* 来表示通道号，outlink*n* 中，n 的范围是 0 ~ 3，但是 inlink*n* 中，*n* 的范围是 0 ~ 2。2D-DMA 从存储器中取得链表，然后根据 outlink*n* 中的内容将相应存储器中的数据发送出去，也可根据 inlink*n* 中的内容将接收的数据存入指定存储器地址空间。

4.4 功能描述

4.4.1 传输模式

2D-DMA 共支持 4 种传输模式：

- 1D 模式：将链表描述符 DW0 的 2DEN 置 0，DW2 的 mod 置 0 进入该模式，该模式下 2D-DMA 将根据链表配置的数据长度、存储器空间大小进行存储器数据的读写操作，操作地址是连续的，与 GDMA-AXI 功能相同。
- 2D-MOD0 模式：将链表描述符 DW0 的 2DEN 置 1，DW2 的 mod 置 0 进入该模式，该模式下，2D-DMA 从 HA*VA 图像的 (X, Y) 坐标开始，读/写一个 hb*vb 宏块，地址在每行最后一个读/写数据处理完成后都会跳变到下一行需要被读/写的第一个数据地址。
- 2D-MOD1 模式：将链表描述符 DW0 的 2DEN 置 1，DW2 的 mod 置 1 进入该模式，该模式下，2D-DMA 从 HA*VA 图像的首地址开始，连续读/写多个单位大小的宏块，直到 HA*VA 图像读/写完成。对于 JPEG 外设，在该模式下需要配置寄存器 DMA2D_IN_MACRO_BLOCK_SIZE_CH*n* 或 DMA2D_OUT_MACRO_BLOCK_SIZE_CH*n*，选择读取宏块的单位大小，有三种尺寸可选（水平 × 垂直）：8 像素 × 8 像素、8 像素 × 16 像素、16 像素 × 16 像素，如何选择可参照章节 17 *JPEG 图像编解码器*。宏块单位大小与链表描述符 hb、vb 的选择可参照表 4.4-4 和表 4.4-5。

- DSCR-PORT 模式：将链表描述符 DW0 的 2DEN 置 1，DW2 的 mod 置 0，并配置寄存器 DMA2D_OUT_DSCR_PORT_EN_CH*n* 或 DMA2D_IN_DSCR_PORT_EN_CH*n* 为 1 进入该模式，该模式仅为 PPA 应用，具体操作可参照章节 18 像素处理加速器 (PPA)。

1D 模式、2D-MOD0 模式、2D-MOD1 模式和 DSCR-PORT 模式均支持外设到存储及存储到外设的数据传输。仅 1D 模式和 2D-MOD0 模式支持存储到存储数据传输。

4.4.2 链表

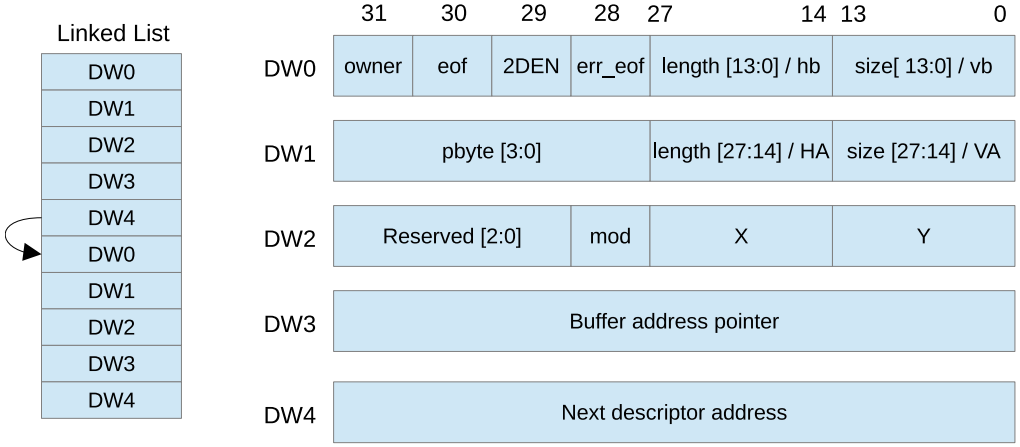


图 4.4-1. 链表结构图

图 4.4-1 所示为链表的结构图。发送链表与接收链表结构相同。每个链表由一个或者若干个描述符构成，一个描述符由 5 个字组成。链表应存放在存储器中，供 2D-DMA 控制器使用。描述符每一字段的意义如下：

注意：本文以 DMA2D_OUT 开头的寄存器对应 TX 通道寄存器，以 DMA2D_IN 开头的寄存器对应 RX 通道寄存器。

表 4.4-1. 描述符字段说明

位	名称	说明
DW0[31]	owner	表示当前描述符对应的 buffer 允许的操作者。 • 0：允许的操作者为 CPU • 1: 1：允许的操作者为 2D-DMA 控制器 在 2D-DMA 使用完该描述符对应的 buffer 后，对于接收描述符，硬件默认会自动将该位清零；对于发送描述符，需要将 DMA2D_OUT_AUTO_WRBACK_CH*n* 置 1，硬件才会自动将该位清零。软件在挂载链表时需要将该位置 1。
DW0[30]	eof	表示结束标志。 • 0：当前描述符不是一张图片的最后一个描述符 • 1：当前描述符为一张图片的最后一个描述符 对于接收描述符，需要软件将该位写 0，硬件会在收完一张图片后将该位置 1。对于发送描述符，需要软件在一张图片最后一个描述符中的该位置 1。

位	名称	说明
DW0[29]	2DEN	2D 功能使能标志。 • 0：2D-DMA 启用 1D 工作模式 • 1：2D-DMA 启用 2D 工作模式
DW0[28]	err_eof	表示接收结束错误标志（仅 JPEG 支持）。 • 0：接收数据没有错误 • 1：接收数据有错误 对于接收描述符，硬件在收完一张图片并检测到接收数据错误会将该位置 1。
DW0[27:14]	length[13:0]/hb	启用 2D 工作模式时为 **hb** – 表示想要搬运的宏块的水平宽度，单位为像素。 启用 1D 工作模式时为 **length[13:0]** – 即 length 的低 14 位。length 表示当前描述符对应的 buffer 中的有效字节数。对于发送描述符，该段由软件填写，表示从 buffer 中读取数据时需要读取的字节数；对于接收描述符，该段由硬件使用完该 buffer 后或者接收到最后一个数据时自动填写，表示 buffer 中存储的有效字节数。
DW0[13:0]	size[13:0]/vb	启用 2D 工作模式时为 **vb** – 表示想要搬运的宏块的垂直高度，单位为像素。 启用 1D 工作模式时为 **size[13:0]** – 即 size 的低 14 位。size 表示当前描述符对应的 buffer 容量的字节数。
DW1[31:28]	pbyte	启用 2D 工作模式时，表示从存储器搬运 / 搬运到存储器的图像单个像素占用的字节数。（如应用了颜色空间转换功能，该字段表示的是存储器中存放的图像每个像素所占用的字节数。） • 0: 0.5 byte/pixel • 1: 1 byte/pixel • 2: 1.5 bytes/pixel • 3: 2 bytes/pixel • 4: 3 bytes/pixel • 5: 4 bytes/pixel 启用 1D 工作模式时，该字段没有使用，可为任意值。
DW1[27:14]	length[27:14]/HA	启用 2D 工作模式时为 **HA** – 表示想要搬运的宏块对应的原始图像的水平宽度，单位为像素。 启用 1D 工作模式时为 **length[27:14]** – 即 length 的高 14 位。
DW1[13:0]	size[27:14]/VA	启用 2D 工作模式时为 **VA** – 表示想要搬运的宏块对应的原始图像的垂直高度，单位为像素。 启用 1D 工作模式时为 **size[27:14]** – 即 size 的高 14 位。
DW2[31:29]	Reserved	保留。此位为无关项。

位	名称	说明
DW2[28]	mod	宏块读取模式。 • 0：2D-DMA 只读/写一次水平宽度为 hb、垂直高度为 vb 的宏块 • 1：2D-DMA 读/写多次水平宽度为 hb、垂直高度为 vb 的宏块，直到将水平宽度为 HA，垂直高度为 VA 的图像数据读/写完成 1D 模式下，该位必须为 0。 **说明：** hb/vb/HA/VA 的单位为像素。
DW2[27:14]	X	启用 2D 工作模式时，表示 hb*vb 宏块在 HA*VA 图像中的 X 坐标，单位为像素。 启用 1D 工作模式时，该域没有使用，可为任意值。
DW2[13:0]	Y	启用 2D 工作模式时，表示 hb*vb 宏块在 HA*VA 图像中的 Y 坐标，单位为像素。 启用 1D 工作模式时，该域没有使用，可为任意值。
DW3	Buffer address pointer	启用 2D 工作模式时，表示 HA*VA 宏块原点坐标在 buffer 中的地址（原始图像在存储器中第一个字节的地址）。 启用 1D 工作模式时，表示 buffer 的起始地址。
DW4	Next descriptor address	下一个描述符的地址。如果当前描述符为链表中最后一个描述符时，该值可以为 0。该地址可以指向片内或片外存储器的地址空间。 **说明：** 需要注意，2D-DMA 链表描述符的存放首地址要求 8 字节对齐。

4.4.3 2D-MOD1 模式和 DSCR-PORT 模式的补数方式

本小节仅应用于 2D-DMA 的发送方向且选择 2D-MOD1 或 DSCR-PORT 模式。

JPEG 解码方向输入的图像数据的基本单元为 8 × 8 或 8 × 16 或 16 × 16（像素）的块，在 2D-MOD1 模式下当图像的长宽不满足基本单元的整数倍时，需要根据上一个宏块或者当前宏块作补齐操作，完成整数个基本单元的传输。

PPA 在发送通道 DSCR-PORT 模式的补数基本单元为 36 × 36（YUV420）、36 × 34（YUV422）和 34 × 34 三种（水平尺寸 × 垂直尺寸），可通过配置寄存器 DMA2D_OUT_DSCR_PORT_BLK_H_CH*n* 和 DMA2D_OUT_DSCR_PORT_BLK_V_CH*n* 来配置不同发送通道基本单元的宽/高。当图像的长宽不满足基本单元的整数倍时，需要做数据补齐操作。

补数规则如下：

水平方向不满一行时，补齐的数据与描述符中 pbyte 字段相关：

- 当 pbyte 为 0 (0.5 byte/piexl)，要求剩余像素为偶数个，补齐数据为最后 0.5 byte

- 当 pbyte 为 1 (1 byte/piexl)，补齐数据为最后 1 byte
- 当 pbyte 为 2 (1.5 byte/piexl)，要求剩余像素为偶数个，补齐数据为最后 3 byte
- 当 pbyte 为 3 (2 byte/piexl)，补齐数据为最后 2 byte
- 当 pbyte 为 4 (3 byte/piexl)，补齐数据为最后 3 byte
- 当 pbyte 为 5 (4 byte/piexl)，补齐数据为最后 4 byte

垂直方向不满一个 block 时，以最后一行补齐剩下的行。

补齐方式如图 4.4-2 和图 4.4-3 所示。

图 4.4-2 中，一个蓝色方框表示一个基本单元 (8 × 8 或 8 × 16 或 16 × 16（像素）的块)，右侧和下侧一个黄色方框加上一个绿色方框表示一个基本单元，右下角一个黄色方框和两个绿色方框表示一个基本单元。

图 4.4-3 中一个方框表示一个像素，方框内的不同数字表示不同的像素值。

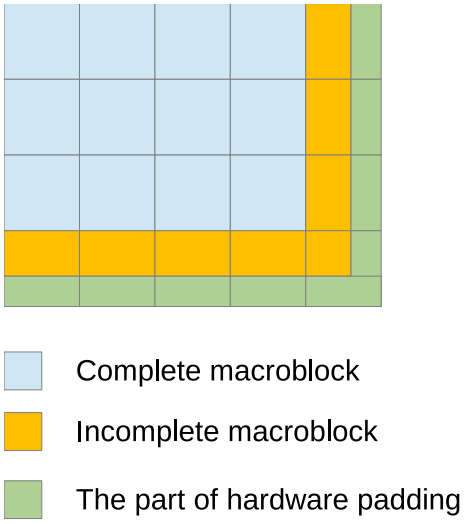


图 4.4-2. 补齐示意图（整体）

1	2	3	3	3	3	3	3
4	5	6	6	6	6	6	6
7	8	9	9	9	9	9	9
7	8	9	9	9	9	9	9
7	8	9	9	9	9	9	9
7	8	9	9	9	9	9	9
7	8	9	9	9	9	9	9
7	8	9	9	9	9	9	9

图 4.4-3. 补齐示意图（细节）

4.4.4 外设到存储及存储到外设的数据传输

当 DMA2D_MEM_TRANS_EN_CH*n* 配置为 0 时，2D-DMA 进入外设模式。发送通道从存储器中读取数据后传输给外设，接收通道接收来自外设的数据后写入存储器。

2D-DMA 支持存储到外设及外设到存储的数据传输，分别对应 TX 及 RX 功能。TX 通道通过 outlink*n* 实现将指定存储区域中的数据搬运到外设的发送端；RX 通道通过 inlink*n* 实现将外设接收到的数据搬运到指定的存储区域。

每个 RX/TX 通道均可以被配置连接到任意一个支持 2D-DMA 功能的外设，表 4.4-2 和表 4.4-3 所示为通过寄存器配置通道与其对应外设的关系。其中"Dummy-*n*" 是 2D-DMA 在实现存储到存储数据搬运功能时可选配的寄存器值。当其中一个通道已经与某一个外设连接时，其他通道将不能配置为与该外设连接。

表 4.4-2. 2D-DMA TX 方向配置寄存器与外设选择关系表

DMA2D_PERI_OUT_SEL_CH*n*	外设
0	JPEG
1	PPA 的 SRM 模块
2	PPA 的 BLENDO 模块
3	PPA 的 BLEND1 模块
4 ~ 7	Dummy-4 ~ 7

表 4.4-3. 2D-DMA RX 方向配置寄存器与外设选择关系表

DMA2D_PERI_IN_SEL_CH*n*	外设
0	JPEG
1	PPA 的 SRM 模块
2	PPA 的 BLEND 通道
3 ~ 7	Dummy-3 ~ 7

4.4.5 存储到存储数据传输

2D-DMA 只有 1D 模式和 2D-MODO 模式支持存储到存储的数据传输。置位 DMA2D_IN_MEM_TRANS_EN_CH*n*，TX 通道*n* 的输出将与 RX 通道*n* 的输入相连，从而使能存储到存储的数据传输功能。需要注意的是，一个 TX 通道只与其编号对应的 RX 通道相连而实现存储到存储的数据传输，并且需要将 DMA2D_IN_PERI_SEL_CH*n* 和 DMA2D_OUT_PERI_SEL_CH*n* 配置成 Dummy 的值。

可以搭配颜色空间转换功能，完成将一块宏块从一段存储器地址空间转移到另一段地址空间，并转换颜色空间。

4.4.6 宏块重排功能

4.4.6.1 发送方向宏块重排功能

当 2D-DMA 通过 SPI 访问外部存储器时，为了提高 SPI 的带宽利用率，需要以较大的 burst length 发起 AXI 传输。对于 2D-MOD1，2D-DMA 可以通过一次取若干个宏块来提高 AXI 的 burst length。例如，一次读取 5 个 8x8

的宏块，此时，描述符中的 hb 设为 40，vb 设为 8。而对于 JPEG，2D-DMA 需要按照宏块顺序给出数据。也就是需要重新排列取出的若干个宏块以满足 JPEG 的需求。

推荐宏块重排的配置见表 4.4-4。

表 4.4-4. 2D-DMA 发送方向宏块重排推荐配置表

byte/pixel	宏块大小（像素）	最佳宏块数	hb	vb
3	8 × 8	5	40	8
3	16 × 8	2	32	8
3	16 × 16	2	32	16
2	8 × 8	8	64	8
2	16 × 8	4	64	8
2	16 × 16	3	48	16
1	8 × 8	16	128	8

发送方向宏块重排需要配置宏块大小寄存器 DMA2D_OUT_MACRO_BLOCK_SIZE_CH*n*。

4.4.6.2 接收方向宏块重排功能

目前接收宏块重排仅支持 JPEG 解码（码流为 1 个 scan，详见章节 17 *JPEG 图像编解码器*）数据重排，将来自 JPEG 解码的颜色分量数据包拆解为像素格式并组装成较大的宏块通过 2D-DMA 写入存储器。

JPEG 解码的数据格式包括：GRAY、YUV444、YUV422、YUV420。推荐宏块重排的配置见表 4.4-5。

表 4.4-5. 2D-DMA 接收方向宏块重排推荐配置表（不做颜色格式转换）

颜色格式	宏块大小（像素）	最佳宏块数	hb	vb
GRAY	8 × 8	12	96	8
YUV444	8 × 8	5	40	8
YUV422	16 × 8	4	64	8
YUV420	16 × 16	3	48	16

接收方向宏块重排需要配置宏块大小寄存器 DMA2D_IN_MACRO_BLOCK_SIZE_CH*n*。

JPEG 解码输出的宏块排布如图 4.4-4 所示，其中每一个蓝色正方形表示连续的 8x8 byte 数据。图中每一个颜色格式均展示了 4 个 8 × 8 像素的宏块。

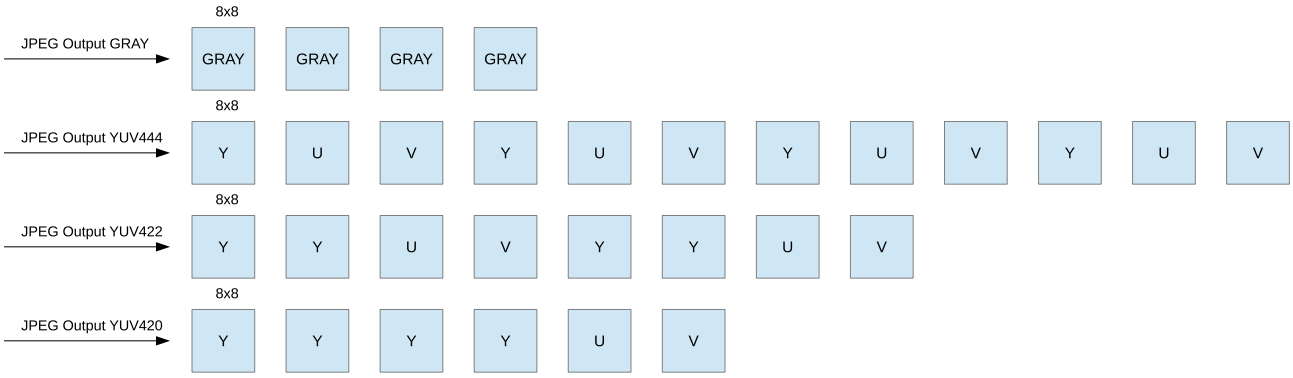


图 4.4-4. JPEG 解码输出数据宏块排布

4.4.7 颜色空间转换功能

发送侧将从存储器读取到的数据按像素组装后，像素数据可进行颜色空间转换。暂不支持 YUV420 格式转换。可支持：

- RGB888 转 RGB565
- RGB565 转 RGB888
- RGB888 转 YUV444
- RGB888 转 YUV422（RGB 一行像素个数需要为偶数个）
- YUV444 转 RGB888
- YUV422 转 RGB888

接收侧只有 JPEG 需要颜色空间转换，支持的格式如下：

- YUV444 转 RGB888/RGB565/YUV422/YUV420
- YUV422 转 YUV444 转 RGB888/RGB565/YUV420
- YUV420 转 YUV444 转 RGB888/RGB565

说明：

- 对于 JPEG 解码，即 2D-DMA 的接收侧，硬件会强制将 YUV422 和 YUV420 转换成 YUV444 格式，软件可选是否利用颜色空间转换功能，将其转换为 RGB888/RGB565。

颜色空间转换模块的结构示意图如 4.4-5 所示。

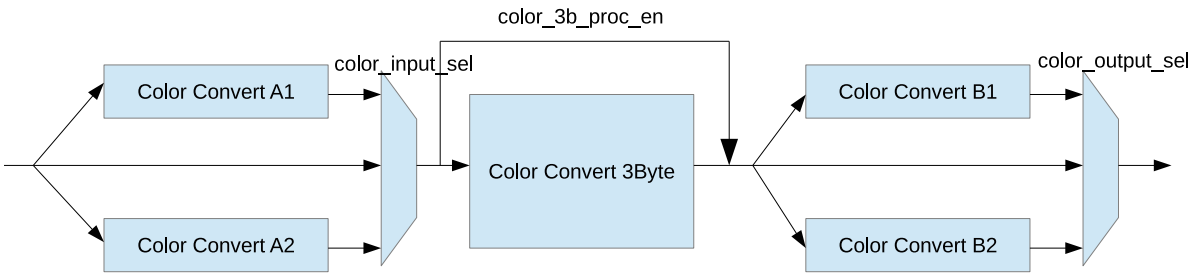


图 4.4-5. 颜色空间转换示意图

颜色空间转换分为 3 级进行，第一级进行输入颜色格式转换，可将特定的 2 Byte/pixel 的像素格式转换为 3 Byte/pixel，通过 DMA2D_OUT_COLOR_INPUT_SEL_CH*n* 或 DMA2D_IN_COLOR_INPUT_SEL_CH0 进行源数据选择。（可点击寄存器链接查看第一级支持的颜色格式转换）

第二级通过公式对选择后的源数据计算处理。计算公式如下：

$$256 * Q = A[9:0] * x + B[10:0] * y + C[9:0] * z + D[17:0]$$

其中 Q 为输出数据，x、y、z 为输入数据的高、中、低字节，A、B、C、D 为对应乘法/加法系数，为有符号数。同一时刻可处理最多 3 Byte 数据，因此需要通过以下字段配置 3*4=12 个参数：

- DMA2D_OUT_COLOR_PARAM_H0_CH*n*/DMA2D_IN_COLOR_PARAM_H0_CH0

- DMA2D_OUT_COLOR_PARAM_H1_CH*n*/DMA2D_IN_COLOR_PARAM_H1_CH0
- DMA2D_OUT_COLOR_PARAM_M0_CH*n*/DMA2D_IN_COLOR_PARAM_M0_CH0
- DMA2D_OUT_COLOR_PARAM_M1_CH*n*/DMA2D_IN_COLOR_PARAM_M1_CH0
- DMA2D_OUT_COLOR_PARAM_L0_CH*n*/DMA2D_IN_COLOR_PARAM_L0_CH0
- DMA2D_OUT_COLOR_PARAM_L1_CH*n*/DMA2D_IN_COLOR_PARAM_L1_CH0

字段名称中的 PARAM_H*x*、PARAM_M*x* 和 PARAM_L*x*（*x* 为 0 或 1）分别表示高、中、低三个字节各自的参数。以 PARAM_H*x* 为例，PARAM_H0 的低 10 位为系数 A，PARAM_H0 的高 11 位为系数 B，PARAM_H1 的低 10 位为系数 C，PARAM_H1 的高 18 位为系数 D。

第三级与第一级类似，通过 DMA2D_OUT_COLOR_OUTPUT_SEL_CH*n* 或 DMA2D_IN_COLOR_OUTPUT_SEL_CH0 选择直接输出或者进行第三级颜色空间转换模块后输出。

下表列出常用的颜色格式转换需要的参数配置：

表 4.4-6. TX 方向颜色格式转换参数配置

转换类型	input_sel	proc_en	output_sel	param_h0	param_h1	param_m0	param_m1	param_l0	param_l1
不转换	7	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
仅乱序	2/3	0	2	N/A	N/A	N/A	N/A	N/A	N/A
RGB888 → RGB565	3	0	0	N/A	N/A	N/A	N/A	N/A	N/A
RGB565 → RGB888	0	0	2	N/A	N/A	N/A	N/A	N/A	N/A
RGB888 → (601) YUV444	3	1	2	20442	400019	1EDBDA	2000070	1E8870	20003EE
RGB888 → (709) YUV444	3	1	2	2742F	400010	1EABE6	2000070	1E6870	20003F6
RGB888 → (601) YUV422-MIPI	3	1	1	20442	400019	1EDBDA	2000070	1E8870	20003EE
RGB888 → (709) YUV422-MIPI	3	1	1	2742F	400010	1EABE6	2000070	1E6870	20003F6
YUV444 → (601) RGB888	3	1	2	12A	C86D999	1E712A	21E4F30	8112A	BAD3000
YUV444 → (709) RGB888	3	1	2	12A	C21E5CB	1F252A	1338778	8752A	B7D0800
YUV422-MIPI → (601) RGB888	1	1	2	12A	C86D999	1E712A	21E4F30	8112A	BAD3000
YUV422-MIPI → (709) RGB888	1	1	2	12A	C21E5CB	1F252A	1338778	8752A	B7D0800

表 4.4-7. RX 方向颜色格式转换参数配置

转换类型	input_sel	proc_en	output_sel	param_h0	param_h1	param_m0	param_m1	param_l0	param_l1
不转换	7	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A
仅乱序	1/2	0	1	N/A	N/A	N/A	N/A	N/A	N/A
YUV422 → YUV444	0	0	1	N/A	N/A	N/A	N/A	N/A	N/A

见下页

表 4.4-7 – 接上页

转换类型	input_sel	proc_en	output_sel	param_h0	param_h1	param_m0	param_m1	param_l0	param_l1
YUV420 → YUV444	0	0	1	N/A	N/A	N/A	N/A	N/A	N/A
YUV444 → YUV422	0	0	2	N/A	N/A	N/A	N/A	N/A	N/A
YUV444 → YUV420	0	0	3	N/A	N/A	N/A	N/A	N/A	N/A
YUV422 → YUV420	0	0	3	N/A	N/A	N/A	N/A	N/A	N/A
YUV420 → (601) RGB888/RGB565	0	1	1/0	12A	C86D999	1E712A	21E4F30	8112A	BAD3000
YUV422 → (601) RGB888/RGB565	0	1	1/0	12A	C86D999	1E712A	21E4F30	8112A	BAD3000
YUV444 → (601) RGB888/RGB565	2	1	1/0	12A	C86D999	1E712A	21E4F30	8112A	BAD3000
YUV420 → (709) RGB888/RGB565	0	1	1/0	12A	C21E5CB	1F252A	1338778	8752A	B7D0800
YUV422 → (709) RGB888/RGB565	0	1	1/0	12A	C21E5CB	1F252A	1338778	8752A	B7D0800
YUV444 → (709) RGB888/RGB565	2	1	1/0	12A	C21E5CB	1F252A	1338778	8752A	B7D0800

另外，颜色格式转换模块支持单个像素（最大 3 byte）的字节乱序处理，发送侧在格式转换之前支持，接收侧在格式转换之前及之后均支持。颜色格式转换模块需要的像素字节顺序如表 4.4-8、表 4.4-9、表 4.4-10 所示。对于 JPEG 解码方向，接收侧宏块重排后在不配置乱序处理的情况下，像素字节顺序符合颜色格式转换所需字节顺序。假设输入数据的字节序 MSB 到 LSB 用 2、1、0 表示，字节乱序支持输出 210、201、102、120、021、012 六种顺序，默认输出为 2、1、0，可根据需要进行配置。可配置 DMA2D_IN_SCRAMBLE_SEL_PRE_CH0 / DMA2D_IN_SCRAMBLE_SEL_POST_CH0 在颜色格式转换前 / 后对接收侧数据进行乱序处理。可配置 DMA2D_OUT_SCRAMBLE_SEL_PRE_CH*n* 在颜色格式转换前对发送侧数据进行乱序处理。

表 4.4-8. RGBB888 的存储格式

	LSB								MSB
LINE	B1	G1	R1	B2	G2	R2	B3	G3	R3

表 4.4-9. YUV444 的存储格式

	LSB								MSB
LINE	V1	U1	Y1	V2	U2	Y2	V3	U3	Y3

表 4.4-10. YUV422 的存储格式

	LSB							MSB
LINE	U1	Y1	V1	Y2	U3	Y3	V3	Y4

4.4.8 启动 2D-DMA

软件通过挂载链表的方式来使用 2D-DMA。对于接收通道，软件挂载好接收链表并准备好接收数据，配置 DMA2D_INLINK_ADDR_CH*n* 字段指向第一个接收链表描述符，置位 DMA2D_INLINK_START_CH*n* 位启动 2D-DMA。对于发送通道，软件挂载好发送链表并准备好发送数据，配置 DMA2D_OUTLINK_ADDR_CH*n* 字段指向第一个发送链表描述符，置位 DMA2D_OUTLINK_START_CH*n* 位启动 2D-DMA。
DMA2D_INLINK_START_CH*n* 与 DMA2D_OUTLINK_START_CH*n* 位由硬件自动清零。

有时您可能想要在 2D-DMA 数据传输已经开始后追加更多描述符。要挂载更多描述符，原本看似只需清空已挂载链表最后一个描述符的 eof 位，并将该描述符的 next descriptor address (DW4) 字段配置为新链表第一个描述符的地址。但如果 DMA 数据传输已经或即将结束，这个方法便无法奏效。2D-DMA 控制器有专门的逻辑来确保数据传输继续或重启：如果数据传输仍在进行，2D-DMA 控制器会确保顾及到新追加的描述符；如果数据传输已经结束，2D-DMA 控制器会重启数据传输，传输新追加的描述符。这个逻辑由 Restart 功能实现。

软件使用 Restart 功能时，需要先置位 DMA2D_INLINK_STOP_CH*n* 或者 DMA2D_OUTLINK_STOP_CH*n* 寄存器，让 2D-DMA 处理完当前描述符后暂停处理下一个描述符，然后重写已挂载链表的最后一个描述符，使其第五个字中的内容（即 DW4）指向新链表的首地址，最后置位 DMA2D_INLINK_RESTART_CH*n* 或者 DMA2D_OUTLINK_RESTART_CH*n*（这两个位由硬件自动清零），硬件会在读取已挂载链表的最后一个描述符时，获取新挂载链表的地址，从而继续处理新挂载的链表。

4.4.9 读链表

软件在配置并启动 2D-DMA 后，2D-DMA 会从存储器读取链表。2D-DMA 会检查读入的链表描述符是否正确。只有当链表描述符通过检查时，2D-DMA 对应的通道才会开始搬运数据。当链表描述符没有通过检查，硬件将触发描述符错误中断（DMA2D_IN_DSCR_ERR_CH*n*_INT 或者 DMA2D_OUT_DSCR_ERR_CH*n*_INT），同时该通道将会处于阻塞状态，停止工作，可重新置位 DMA2D_OUTLINK_START_CH*n* 或者 DMA2D_INLINK_START_CH*n* 再次开启该通道。

描述符检查项包括：

- DMA2D_IN_CHECK_OWNER_CH*n* 或者 DMA2D_OUT_CHECK_OWNER_CH*n* 置 1 时，检查描述符的 owner 位。如果该位为 0，表示当前操作者为 CPU，则不通过检查。DMA2D_IN_CHECK_OWNER_CH*n* 或者 DMA2D_OUT_CHECK_OWNER_CH*n* 置 0 时，不检查描述符的 owner 位。
- 检查描述符本身的存储地址是否在配置的存储器地址空间内。如果 2D-DMA 的描述符地址在 DMA2D_ACCESS_INTR_MEM_START_ADDR ~ DMA2D_ACCESS_INTR_MEM_END_ADDR（配置值应为片内存储器地址）或 DMA2D_ACCESS_EXTR_MEM_START_ADDR ~ DMA2D_ACCESS_EXTR_MEM_END_ADDR（配置值应为片外存储器地址）值的范围内，则通过检查。请参见章节 4.4.11）。
- 检查描述符的 HA/VA/hb/vb/SIZE 其中任意一个字段是否为 0，如果为 0，则不通过检查。其中 HA/VA/hb/vb 在 1D 模式下不检查，SIZE 仅在 1D 模式下检查。

软件在检查到通道描述符错误中断后，需要复位对应的通道，并置位 DMA2D_OUTLINK_START_CH*n* 或者 DMA2D_INLINK_START_CH*n* 位重新启动 2D-DMA。
可以将 DMA2D_INDSCR_BURST_EN_CH*n* 或 DMA2D_OUTDSCR_BURST_EN_CH*n* 置 1，使能 2D-DMA 以 AXI burst 传输快速从存储器获取接收/发送侧链表描述符。

4.4.10 数据传输结束标志

注意： 本章中提到发送链表描述符的 EOF 为描述符中的第一个字中的 bit 30 指示的 eof，接收链表描述符的

EOF 可以为 bit 30 指示的 eof 和 bit 28 指示的 err_eof。

2D-DMA 通过 EOF 来指示一张图片传输结束。

发送数据时，产生 EOF 和 TOTAL EOF 两种中断：

- DMA2D_OUT_EOF_CH*n*_INT：链表中的任意一个描述符有配置 eof 标志位，则当该描述符对应的数据传输完成后，2D-DMA 都会产生 EOF 中断。通常在一张图片的最后一个描述符配置 eof。置位 DMA2D_OUT_EOF_CH*n*_INT_ENA 位使能该中断。
- DMA2D_OUT_TOTAL_EOF_CH*n*_INT：如果该链表中的最后一个描述符配置了 eof 标志位，则当该链表的最后一个描述符对应的数据传输完成后，2D-DMA 会产生 TOTAL EOF 中断。置位 DMA2D_OUT_TOTAL_EOF_CH*n*_INT_ENA 位使能该中断。

接收数据时，可以产生以下中断：

- DMA2D_IN_SUC_EOF_CH*n*_INT：表示一张图片数据接收完成。置位 DMA2D_IN_SUC_EOF_CH*n*_INT_ENA 位使能该中断。
- （仅 JPEG）DMA2D_IN_ERR_CH*n*_EOF_INT：表示一张图片接收完成但接收数据有错误。置位 DMA2D_IN_ERR_EOF_CH*n*_INT_ENA 使能该中断。需要注意的是，只有当通道连接的外设为 JPEG 时，才支持该中断。

软件在检测到 DMA2D_OUT_TOTAL_EOF_CH*n*_INT 或 DMA2D_IN_SUC_EOF_CH*n*_INT 中断时，可以通过获取 DMA2D_OUT_EOF_DES_ADDR_CH*n* 或 DMA2D_IN_SUC_EOF_DES_ADDR_CH*n* 字段中的的值，获取当前已经完成数据传输的描述符的存储地址。这样，软件可以知道哪些描述符已经被使用并根据需要回收描述符。RX 方向还支持记录 DMA2D_IN_ERR_EOF_CH*n*_INT 中断触发时描述符的存储地址，可通过 DMA2D_IN_ERR_EOF_DES_ADDR_CH*n* 字段获取。

4.4.11 访问存储

2D-DMA 任意 RX/TX 通道均可以访问片内和片外存储器，其可访问的片内地址空间为寄存器 DMA2D_ACCESS_INTR_MEM_START_ADDR ~ DMA2D_ACCESS_INTR_MEM_END_ADDR 的配置值区间，可访问的片外地址空间为 DMA2D_ACCESS_EXTR_MEM_START_ADDR ~ DMA2D_ACCESS_EXTR_MEM_END_ADDR 的配置值区间。

2D-DMA 利用 AXI 总线在片内进行数据传输，AXI 总线的突发传输模式默认打开且无法关闭，但是用户可以通过DMA2D_IN_MEM_BURST_LENGTH_CH*n* 和 DMA2D_OUT_MEM_BURST_LENGTH_CH*n* 来控制 AXI 总线突发传输传输的数据量，将突发传输长度配置为 8/16/32/64/128 字节。

2D-DMA 访问外部存储器的加密空间（详见章节 19 *片外存储器加密与解密 (XTS_AES) [to be added later]*）时，buffer address pointer 和对应数据量需按 16 字节对齐，其他参数无要求；访问配置的片内存储器空间和外部存储器的非加密空间时，链表描述符参数无对齐要求。

4.4.12 仲裁

为了确保及时响应高速低延迟外设的请求，2D-DMA 引入通道优先级仲裁和通道权重仲裁两种仲裁机制。

- 优先级仲裁方式：2D-DMA 支持 16（0 ~15）个等级的优先级，数字越大优先级越高。若多个通道在同一时刻存在数据传输需求，2D-DMA 会按照通道优先级从高到低顺序响应每个通道的数据传输请求；优先级越高，请求响应越及时。
- 权重仲裁方式：

- 每个通道的的权重（即令牌数）可以配置为 0 ~ 15 之间的任意值。2D-DMA 将时间按照一定的总线 (AXI) 时钟周期长度划分成片，在每个时间片内，各个通道的数据传输请求次数应该将按照通道所配置的令牌数进行分配，在任意一个时间片内，某通道每请求一次数据传输消耗一次令牌，若某个通道在某个时间片内消耗尽令牌数，则该通道请求将不再被响应，直到仲裁时间达到时间片规定长度或者所有通道的令牌数均已经消耗尽，此时将刷新令牌数，进入下一个时间片周期内。
- 若通道令牌数配置为非零值，即使无数据传输请求，也会在每个时间片内等待所有通道的令牌数耗尽，不会提前退出当前仲裁时间片。通道无数据传输请求且令牌值非 0 时，会导致一些带宽浪费，基于此，2D-DMA 提供额外的权重仲裁优化开关。权重仲裁优化打开时，如果通道无数据传输需求，则该通道令牌数会被认为是 0，不再参与仲裁，当所有通道均无数据传输请求时，仲裁器提前退出当前时间片，以此提高仲裁响应时间，提高传输速率。

说明：

- 若同时配置优先级仲裁和通道权重仲裁，且优先级不同，权重相同，则在各个通道传输相同的数据量，且数据量较少的前提下，优先级高的通道优先传输完成。
- 若优先级不同且权重也不同，各个通道传输数据量相同且较大，则权重较大的通道是能优先分配到大带宽的，会优先完成传输。

4.5 事件任务矩阵功能

在 ESP32-S31 中，2D-DMA 支持 ETM 功能，即可以通过任意外设的 ETM 事件触发 2D-DMA 的 ETM 任务，或者通过 2D-DMA 的 ETM 事件触发任意外设的 ETM 任务。关于 ETM 更多详细信息，请参考章节 10 事件任务矩阵 (ETM) [to be added later]。这里仅介绍与 2D-DMA 相关的 ETM 任务和 ETM 事件。

2D-DMA 可接收的 ETM 任务有：

- DMA2D_TASK_IN_START_CH*n*：触发时开启 2D-DMA 的接收通道 *n* 进行数据传输。
- DMA2D_TASK_OUT_START_CH*n*：触发时开启 2D-DMA 的发送通道 *n* 进行数据传输。

说明：

以上两个 ETM 任务与 CPU 配置 DMA2D_INLINK_START_CH*n*、DMA2D_OUTLINK_START_CH*n* 实现的功能相同。当 DMA2D_IN_ETM_EN_CH*n* 或 DMA2D_OUT_ETM_EN_CH*n* 为 1 时，只有 ETM 可以配置对应方向及对应通道 2D-DMA 的启动，当 DMA2D_IN_ETM_EN_CH*n* 或 DMA2D_OUT_ETM_EN_CH*n* 为 0 时，仅可通过 CPU 使能对应的 2D-DMA。

- DMA2D_TASK_IN_DSCR_READY_CH*n*：触发时允许 2D-DMA 的接收通道 *n* 处理下一个链表描述符。
- DMA2D_TASK_OUT_DSCR_READY_CH*n*：触发时允许 2D-DMA 的发送通道 *n* 处理下一个链表描述符。

说明：

正常模式下 2D-DMA 开始工作后，每处理完一个链表描述符，如果还存在下一个链表描述符（即 next descriptor address 字段非 0），则会继续处理下一个链表描述符并按照描述符的描述信息读/写存储器，直到所有描述符都被处理完成。当配置 DMA2D_IN_ETM_LOOP_EN_CH*n* 或 DMA2D_OUT_ETM_LOOP_EN_CH*n* 为 1 时，2D-DMA 处理完当前描述符后会等待 DMA2D_TASK_IN_DSCR_READY_CH*n* 或 DMA2D_TASK_OUT_DSCR_READY_CH*n* 任务，才可以开始处理下一个链表描述符。

DMA2D_TASK_IN_DSCR_READY_CH*n* 或 DMA2D_TASK_OUT_DSCR_READY_CH*n* 任务可以存储到任务缓存中，

最多可以缓存 4 个任务。当 2D-DMA 接收到此任务，但是当前链表描述符还未处理完成时，可以缓存任务。任务缓存的深度由 DMA2D_IN_DSCR_TASK_MAX_CH*n* 或 DMA2D_OUT_DSCR_TASK_MAX_CH*n* 决定。
当任务缓存超过最大值时，会上报 DMA2D_IN_DSCR_TASK_OVF_CH*n*_INT 或 DMA2D_OUT_DSCR_TASK_OVF_CH*n*_INT 中断。

2D-DMA 可产生的 ETM 事件有：

- DMA2D_EVT_IN_DONE_CH*n*：表示通道 *n* 接收方向链表描述符指向的数据接收完成。
- DMA2D_EVT_IN_SUC_EOF_CH*n*：通道 *n* 接收方向 eof 位为 1 的链表描述符指向的数据接收完成指示。
- DMA2D_EVT_OUT_DONE_CH*n*：通道 *n* 表示通道 *n* 发送方向链表描述符指向的数据发送完成。
- DMA2D_EVT_OUT_SUC_EOF_CH*n*：通道 *n* 发送方向 eof 位为 1 的链表描述符指向的数据发送完成指示。
- DMA2D_EVT_OUT_TOTAL_EOF_CH*n*：通道 *n* 发送方向最后一个并且 eof 位为 1 的链表描述指向的数据发送符完成指示。

在具体应用中，2D-DMA 的 ETM 事件可以用来触发 2D-DMA 的 ETM 任务，例如，DMA2D_EVT_OUT_TOTAL_EOF_CH0 事件可以触发 DMA2D_TASK_IN_START_CH1 任务，从而触发新一轮的 2D-DMA 操作。

4.6 中断

ESP32-S31 的 2D-DMA 可以生成以下中断信号并发送给中断矩阵。

- DMA2D_IN_CH0_INTR
- DMA2D_IN_CH1_INTR
- DMA2D_IN_CH2_INTR
- DMA2D_OUT_CH0_INTR
- DMA2D_OUT_CH1_INTR
- DMA2D_OUT_CH2_INTR
- DMA2D_OUT_CH3_INTR

这些中断信号由 2D-DMA 的内部中断源生成。

以下中断源生成 DMA2D_IN_CH*n*_INTR (*n* 为 0 ~ 2) 中断信号：

- DMA2D_IN_DONE_CH*n*_INT：对于接收通道*n*，当一个接收链表描述符对应的数据接收完成时触发此中断。
- DMA2D_IN_SUC_EOF_CH*n*_INT：对于接收通道*n*，当一个接收链表描述符对应的数据接收完成，并且描述符的 eof 为 1 时（即当一个接收链表对应的一张图片接收完成时）触发此中断。
- DMA2D_IN_ERR_EOF_CH*n*_INT：对于接收通道*n*，当接收的一张图片中有错误发生时触发此中断。(该中断只用于外设选择 JPEG 时)。
- DMA2D_IN_DSCR_EMPTY_CH*n*_INT：对于接收通道*n*，当接收链表描述符指向的 buffer size 小于待接收数据长度时触发此中断。
- DMA2D_IN_DSCR_ERR_CH*n*_INT：对于接收通道*n*，当接收链表描述符未通过任一描述符检查时触发此中断。

- DMA2D_INFIFO_L1/L2/L3_OVF_CH*n*_INT：当 2D-DMA 接收数据方向缓存写溢出时触发此中断。
- DMA2D_INFIFO_L1/L2/L3_UDF_CH*n*_INT：当 2D-DMA 接收数据方向缓存读溢出时触发此中断。
- DMA2D_IN_DSCR_TASK_OVF_CH*n*_INT：当 2D-DMA 的 DMA2D_TASK_IN_DSCR_READY_CH*n* 任务缓存超过最大值时触发此中断。
- DMA2D_INFIFO_RO_OVF_CH0_INT（仅通道 0 支持）：当 2D-DMA 接收数据方向宏块重排功能的缓存写溢出时触发此中断。
- DMA2D_INFIFO_RO_UDF_CH0_INT（仅通道 0 支持）：当 2D-DMA 接收数据方向宏块重排功能的缓存读溢出时触发此中断。

以下中断源生成 DMA2D_OUT_CH*n*_INTR (*n* 为 0 ~ 3) 中断信号：

- DMA2D_OUT_DONE_CH*n*_INT：对于发送通道*n*，当一个发送链表描述符对应的数据发送完成时触发此中断。
- DMA2D_OUT_EOF_CH*n*_INT：对于发送通道*n*，当发送描述符的 eof 位为 1，且 DMA2D_OUT_EOF_MODE_CH*n* 为 0 时，该描述符对应的最后一个数据由总线传入 2D-DMA TX 通道时，该中断触发；当发送描述符的 eof 位为 1，且 DMA2D_OUT_EOF_MODE_CH*n* 为 1 时，该描述符对应的最后一个数据从 2D-DMA TX 通道发出时，该中断触发。
- DMA2D_OUT_DSCR_ERR_CH*n*_INT：对于发送通道*n*，当发送链表描述符未通过任一描述符检查时会触发此中断。
- DMA2D_OUT_TOTAL_EOF_CH*n*_INT：对于发送通道*n*，当一个链表（可包含多个链表描述符）对应的所有数据都已发送完成时触发此中断。
- DMA2D_OUTFIFO_L1/L2/L3_OVF_CH*n*_INT：当 2D-DMA 发送数据方向缓存写溢出时触发此中断。
- DMA2D_OUTFIFO_L1/L2/L3_UDF_CH*n*_INT：当 2D-DMA 发送数据方向缓存读溢出时触发此中断。
- DMA2D_OUT_DSCR_TASK_OVF_CH*n*_INT：当 2D-DMA 的 DMA2D_TASK_OUT_DSCR_READY_CH*n* 任务缓存超过最大值时时触发此中断。
- DMA2D_OUTFIFO_RO_OVF_CH0_INT（仅通道 0 支持）：当 2D-DMA 发送数据方向宏块重排/颜色空间转换功能的缓存写溢出时触发此中断。
- DMA2D_OUTFIFO_RO_UDF_CH0_INT（仅通道 0 支持）：当 2D-DMA 发送数据方向宏块重排/颜色空间转换功能的缓存读溢出时触发此中断。

4.7 编程流程

可通过配置 HP_SYS_CLKRST_DMA2D_SYS_CLK_EN 寄存器管理 2D-DMA 时钟门控，默认时钟门控打开。可通过配置 HP_SYS_CLKRST_RST_EN_DMA2D 对 2D-DMA 模块全局复位。

4.7.1 2D-DMA 通用配置

配置片内、片外存储器访问空间及突发传输的数据量，详见章节 4.4.11。

4.7.2 2D-DMA 模式专属配置

- 1D 模式：无专属配置

- 2D-MODO 模式：无专属配置
- 2D-MOD1 模式：对于 JPEG 外设，需要配置寄存器 DMA2D_IN_MACRO_BLOCK_SIZE_CH*n* 或 DMA2D_OUT_MACRO_BLOCK_SIZE_CH*n*，选择读取宏块的基本单元，有三种尺寸可选（水平 × 垂直）：8 像素 × 8 像素、8 像素 × 16 像素、16 像素 × 16 像素，如何选择可参照 JPEG 相关章节。宏块单位大小与链表描述符 hb、vb 的选择可参照表 4.4-4、4.4-5。
- DSCR-PORT 模式：配置寄存器 DMA2D_OUT_DSCR_PORT_EN_CH*n* 为 1 进入该模式，该模式仅为 PPA 应用，具体操作可参照 PPA 相关章节。

4.7.3 2D-DMA TX 通道配置流程

利用 2D-DMA 发送数据时，2D-DMA TX 通道的软件配置流程如下：

1. 对寄存器 DMA2D_OUT_RST_CH*n* 置 1 然后置 0，复位 2D-DMA TX 通道状态机和 FIFO 指针；
2. 挂载好发送链表，配置寄存器 DMA2D_OUTLINK_ADDR_CH*n* 指向第一个发送链表描述符；
3. 配置 DMA2D_PERI_OUT_SEL_CH*n* 为对应的外设号，见表 4.4-2；
4. 置位 DMA2D_OUTLINK_START_CH*n* 启动 2D-DMA TX 通道发送数据；
5. 配置对应的外设，并启动该外设，具体配置请参考对应的外设章节；
6. 等待 DMA2D_OUT_EOF_CH*n*_INT 中断，即数据传输完成。

4.7.4 2D-DMA RX 通道配置流程

利用 2D-DMA 接收数据时，2D-DMA RX 通道的软件配置流程如下：

1. 对寄存器 DMA2D_IN_RST_CH*n* 置 1 然后置 0，复位 2D-DMA RX 通道状态机和 FIFO 指针；
2. 挂载好接收链表，配置寄存器 DMA2D_INLINK_ADDR_CH*n* 指向第一个接收链表描述符；
3. 配置 DMA2D_PERI_IN_SEL_CH*n* 为对应的外设号，见表 4.4-3；
4. 置位 DMA2D_INLINK_START_CH*n* 启动 2D-DMA RX 通道发送数据；
5. 配置对应的外设，并启动该外设，具体配置请参考对应的外设章节；
6. 等待 DMA2D_IN_SUC_EOF_CH*n*_INT 中断，即一张图片接收完成。

4.7.5 2D-DMA 存储器到存储器配置流程

利用 2D-DMA 从存储到存储搬运数据时配置流程如下：

1. 对寄存器 DMA2D_OUT_RST_CH*n* 置 1 然后置 0，复位 2D-DMA TX 通道状态机和 FIFO 指针；
2. 对寄存器 DMA2D_IN_RST_CH*n* 置 1 然后置 0，复位 2D-DMA RX 通道状态机和 FIFO 指针；
3. 挂载好发送链表，配置寄存器 DMA2D_OUTLINK_ADDR_CH*n* 指向第一个发送链表描述符；
4. 挂载好接收链表，配置寄存器 DMA2D_INLINK_ADDR_CH*n* 指向第一个接收链表描述符；
5. 配置 DMA2D_PERI_OUT_SEL_CH*n* 为某一个 Dummy 号，见表 4.4-2；
6. 配置 DMA2D_PERI_IN_SEL_CH*n* 为某一个 Dummy 号，见表 4.4-3；
7. 置位 DMA2D_MEM_TRANS_EN_CH*n* 使能存储器到存储器传输功能；

8. 置位 DMA2D_OUTLINK_START_CH*n* 启动 2D-DMA TX 通道发送数据；
9. 置位 DMA2D_INLINK_START_CH*n* 启动 2D-DMA RX 通道发送数据；
10. 等待 DMA2D_IN_SUC_EOF_CH*n*_INT 中断，即一次数据搬运完成。

4.7.6 通道优先级和权重配置流程

优先级仲裁配置方法如下：

1. 配置 DMA2D_OUT_ARB_PRIORITY_CH*n* 和 DMA2D_IN_ARB_PRIORITY_CH*n* 为 TX 和 RX 通道的优先级。

权重仲裁配置方法如下：

1. 配置 DMA2D_OUT_ARB_TIMEOUT_NUM 和 DMA2D_IN_ARB_TIMEOUT_NUM 为 TX 和 RX 权重仲裁的时间片长度；
2. 配置 DMA2D_OUT_ARB_TOKEN_NUM_CH*n* 和 DMA2D_IN_ARB_TOKEN_NUM_CH*n* 为各个通道的令牌值；
3. 清零 DMA2D_OUT_ARB_WEIGHT_OPT_DIS_CH_CH*n* 和DMA2D_IN_ARB_WEIGHT_OPT_DIS_CH*n*，使能 TX 和 RX 方向的权重仲裁优化；
4. 置位 DMA2D_OUT_WEIGHT_EN 和 DMA2D_IN_WEIGHT_EN，使能权重仲裁。

4.7.7 工作状态下复位流程

当 2D-DMA 处于工作状态，即链表描述符未全部处理完成时，如果想要复位 2D-DMA 的 AXI 功能（包括 2D-DMA 系统复位 HP_SYS_CLKRST_RST_EN_DMA2D 及 AXI 功能复位 DMA2D_AXIM_RD_RST 和DMA2D_AXIM_WR_RST），必须先暂停当前 2D-DMA 工作，流程如下：

1. 置位 DMA2D_IN_CMD_DISABLE_CH*n* 和 DMA2D_OUT_CMD_DISABLE_CH*n* 寄存器，暂停 2D-DMA；
2. 等待 DMA2D_OUT_RESET_AVAIL_CH*n* 和 DMA2D_IN_RESET_AVAIL_CH*n* 寄存器置 1 后可进行复位操作；
3. 复位完成后将 DMA2D_IN_CMD_DISABLE_CH*n* 和 DMA2D_OUT_CMD_DISABLE_CH*n* 寄存器置 0。

4.8 寄存器列表

本小节的所有地址均为相对于 2D-DMA 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
配置寄存器			
DMA2D_OUT_CONF0_CH0_REG	配置 TX 通道 0	0x0000	R/W
DMA2D_OUT_PUSH_CH0_REG	配置 TX 通道 0 的 FIFO	0x0018	varies
DMA2D_OUT_LINK_CONF_CH0_REG	配置 TX 通道 0 发送链表描述符的操作	0x001C	varies
DMA2D_OUT_LINK_ADDR_CH0_REG	配置 TX 通道 0 发送链表描述符的地址	0x0020	R/W
DMA2D_OUT_ARB_CH0_REG	配置 TX 通道 0 的仲裁	0x003C	R/W
DMA2D_OUT_RO_PD_CONF_CH0_REG	配置 TX 通道 0 宏块重排存储器	0x0044	R/W
DMA2D_OUT_COLOR_CONVERT_CH0_REG	配置 TX 通道 0 的颜色空间转换功能	0x0048	R/W
DMA2D_OUT_SCRAMBLE_CH0_REG	配置 TX 通道 0 的乱序处理	0x004C	R/W
DMA2D_OUT_COLOR_PARAM0_CH0_REG	配置 TX 通道 0 的颜色空间转换参数	0x0050	R/W
DMA2D_OUT_COLOR_PARAM1_CH0_REG	配置 TX 通道 0 的颜色空间转换参数	0x0054	R/W
DMA2D_OUT_COLOR_PARAM2_CH0_REG	配置 TX 通道 0 的颜色空间转换参数	0x0058	R/W
DMA2D_OUT_COLOR_PARAM3_CH0_REG	配置 TX 通道 0 的颜色空间转换参数	0x005C	R/W
DMA2D_OUT_COLOR_PARAM4_CH0_REG	配置 TX 通道 0 的颜色空间转换参数	0x0060	R/W
DMA2D_OUT_COLOR_PARAM5_CH0_REG	配置 TX 通道 0 的颜色空间转换参数	0x0064	R/W
DMA2D_OUT_ETM_CONF_CH0_REG	配置 TX 通道 0 的 ETM	0x0068	R/W
DMA2D_OUT_DSCR_PORT_BLK_CH0_REG	配置 DSCR-PORT 模式的 TX 通道 0 的块大小	0x006C	R/W
DMA2D_OUT_CONF0_CH1_REG	配置 TX 通道 1	0x0100	R/W
DMA2D_OUT_PUSH_CH1_REG	配置 TX 通道 1 的 FIFO	0x0118	varies
DMA2D_OUT_LINK_CONF_CH1_REG	配置 TX 通道 1 发送链表描述符的操作	0x011C	varies
DMA2D_OUT_LINK_ADDR_CH1_REG	配置 TX 通道 1 发送链表描述符的地址	0x0120	R/W
DMA2D_OUT_ARB_CH1_REG	配置 TX 通道 1 的仲裁	0x013C	R/W
DMA2D_OUT_COLOR_CONVERT_CH1_REG	配置 TX 通道 1 的颜色空间转换功能	0x0148	R/W
DMA2D_OUT_SCRAMBLE_CH1_REG	配置 TX 通道 1 的乱序处理	0x014C	R/W

GoBack

GoBack

名称	描述	地址	访问
DMA2D_OUT_COLOR_PARAM0_CH1_REG	配置 TX 通道 1 的颜色空间转换参数	0x0150	R/W
DMA2D_OUT_COLOR_PARAM1_CH1_REG	配置 TX 通道 1 的颜色空间转换参数	0x0154	R/W
DMA2D_OUT_COLOR_PARAM2_CH1_REG	配置 TX 通道 1 的颜色空间转换参数	0x0158	R/W
DMA2D_OUT_COLOR_PARAM3_CH1_REG	配置 TX 通道 1 的颜色空间转换参数	0x015C	R/W
DMA2D_OUT_COLOR_PARAM4_CH1_REG	配置 TX 通道 1 的颜色空间转换参数	0x0160	R/W
DMA2D_OUT_COLOR_PARAM5_CH1_REG	配置 TX 通道 1 的颜色空间转换参数	0x0164	R/W
DMA2D_OUT_ETM_CONF_CH1_REG	配置 TX 通道 1 的 ETM	0x0168	R/W
DMA2D_OUT_DSCR_PORT_BLK_CH1_REG	配置 DSCR-PORT 模式的 TX 通道 1 的块大小	0x016C	R/W
DMA2D_OUT_CONF0_CH2_REG	配置 TX 通道 2	0x0200	R/W
DMA2D_OUT_PUSH_CH2_REG	配置 TX 通道 2 的 FIFO	0x0218	varies
DMA2D_OUT_LINK_CONF_CH2_REG	配置 TX 通道 2 发送链表描述符的操作	0x021C	varies
DMA2D_OUT_LINK_ADDR_CH2_REG	配置 TX 通道 2 发送链表描述符的地址	0x0220	R/W
DMA2D_OUT_ARB_CH2_REG	配置 TX 通道 2 的仲裁	0x023C	R/W
DMA2D_OUT_COLOR_CONVERT_CH2_REG	配置 TX 通道 2 的颜色空间转换功能	0x0248	R/W
DMA2D_OUT_SCRAMBLE_CH2_REG	配置 TX 通道 2 的乱序处理	0x024C	R/W
DMA2D_OUT_COLOR_PARAM0_CH2_REG	配置 TX 通道 2 的颜色空间转换参数	0x0250	R/W
DMA2D_OUT_COLOR_PARAM1_CH2_REG	配置 TX 通道 2 的颜色空间转换参数	0x0254	R/W
DMA2D_OUT_COLOR_PARAM2_CH2_REG	配置 TX 通道 2 的颜色空间转换参数	0x0258	R/W
DMA2D_OUT_COLOR_PARAM3_CH2_REG	配置 TX 通道 2 的颜色空间转换参数	0x025C	R/W
DMA2D_OUT_COLOR_PARAM4_CH2_REG	配置 TX 通道 2 的颜色空间转换参数	0x0260	R/W
DMA2D_OUT_COLOR_PARAM5_CH2_REG	配置 TX 通道 2 的颜色空间转换参数	0x0264	R/W
DMA2D_OUT_ETM_CONF_CH2_REG	配置 TX 通道 2 的 ETM	0x0268	R/W
DMA2D_OUT_DSCR_PORT_BLK_CH2_REG	配置 DSCR-PORT 模式的 TX 通道 2 的块大小	0x026C	R/W
DMA2D_OUT_CONF0_CH3_REG	配置 TX 通道 3	0x0300	R/W
DMA2D_OUT_PUSH_CH3_REG	配置 TX 通道 3 的 FIFO	0x0318	varies
DMA2D_OUT_LINK_CONF_CH3_REG	配置 TX 通道 3 发送链表描述符的操作	0x031C	varies
DMA2D_OUT_LINK_ADDR_CH3_REG	配置 TX 通道 3 发送链表描述符的地址	0x0320	R/W
DMA2D_OUT_ARB_CH3_REG	配置 TX 通道 3 的仲裁	0x033C	R/W
DMA2D_OUT_COLOR_CONVERT_CH3_REG	配置 TX 通道 3 的颜色空间转换功能	0x0348	R/W

名称	描述	地址	访问
DMA2D_OUT_SCRAMBLE_CH3_REG	配置 TX 通道 3 的乱序处理	0x034C	R/W
DMA2D_OUT_COLOR_PARAM0_CH3_REG	配置 TX 通道 3 的颜色空间转换参数	0x0350	R/W
DMA2D_OUT_COLOR_PARAM1_CH3_REG	配置 TX 通道 3 的颜色空间转换参数	0x0354	R/W
DMA2D_OUT_COLOR_PARAM2_CH3_REG	配置 TX 通道 3 的颜色空间转换参数	0x0358	R/W
DMA2D_OUT_COLOR_PARAM3_CH3_REG	配置 TX 通道 3 的颜色空间转换参数	0x035C	R/W
DMA2D_OUT_COLOR_PARAM4_CH3_REG	配置 TX 通道 3 的颜色空间转换参数	0x0360	R/W
DMA2D_OUT_COLOR_PARAM5_CH3_REG	配置 TX 通道 3 的颜色空间转换参数	0x0364	R/W
DMA2D_OUT_ETM_CONF_CH3_REG	配置 TX 通道 3 的 ETM	0x0368	R/W
DMA2D_OUT_DSCR_PORT_BLK_CH3_REG	配置 DSCR-PORT 模式的 TX 通道 3 的块大小	0x036C	R/W
DMA2D_IN_CONF0_CH0_REG	配置 RX 通道 0	0x0500	R/W
DMA2D_IN_POP_CH0_REG	配置 RX 通道 0 的 FIFO	0x0518	varies
DMA2D_IN_LINK_CONF_CH0_REG	配置 RX 通道 0 接收链表描述符的操作	0x051C	varies
DMA2D_IN_LINK_ADDR_CH0_REG	配置 RX 通道 0 接收链表描述符的地址	0x0520	R/W
DMA2D_IN_ARB_CH0_REG	配置 RX 通道 0 的仲裁	0x0540	R/W
DMA2D_IN_RO_PD_CONF_CH0_REG	配置 RX 通道 0 的宏块重排存储器	0x0548	R/W
DMA2D_IN_COLOR_CONVERT_CH0_REG	配置 RX 通道 0 的颜色空间转换功能	0x054C	R/W
DMA2D_IN_SCRAMBLE_CH0_REG	配置 RX 通道 0 的乱序处理	0x0550	R/W
DMA2D_IN_COLOR_PARAM0_CH0_REG	配置 RX 通道 0 的颜色空间转换参数	0x0554	R/W
DMA2D_IN_COLOR_PARAM1_CH0_REG	配置 RX 通道 0 的颜色空间转换参数	0x0558	R/W
DMA2D_IN_COLOR_PARAM2_CH0_REG	配置 RX 通道 0 的颜色空间转换参数	0x055C	R/W
DMA2D_IN_COLOR_PARAM3_CH0_REG	配置 RX 通道 0 的颜色空间转换参数	0x0560	R/W
DMA2D_IN_COLOR_PARAM4_CH0_REG	配置 RX 通道 0 的颜色空间转换参数	0x0564	R/W
DMA2D_IN_COLOR_PARAM5_CH0_REG	配置 RX 通道 0 的颜色空间转换参数	0x0568	R/W
DMA2D_IN_ETM_CONF_CH0_REG	配置 RX 通道 0 的 ETM	0x056C	R/W
DMA2D_IN_CONF0_CH1_REG	配置 RX 通道 1	0x0600	R/W
DMA2D_IN_POP_CH1_REG	配置 RX 通道 1 的 RX FIFO	0x0618	varies
DMA2D_IN_LINK_CONF_CH1_REG	配置 RX 通道 1 接收链表描述符的操作	0x061C	varies
DMA2D_IN_LINK_ADDR_CH1_REG	配置 RX 通道 1 接收链表描述符的地址	0x0620	R/W
DMA2D_IN_ARB_CH1_REG	配置 RX 通道 1 的仲裁	0x0640	R/W

GoBack

名称	描述	地址	访问
DMA2D_IN_ETM_CONF_CH1_REG	配置 RX 通道 1 的 ETM	0x0648	R/W
DMA2D_IN_CONF0_CH2_REG	配置 RX 通道 2	0x0700	R/W
DMA2D_IN_POP_CH2_REG	配置 RX 通道 2 的 RX FIFO	0x0718	varies
DMA2D_IN_LINK_CONF_CH2_REG	配置 RX 通道 2 接收链表描述符的操作	0x071C	varies
DMA2D_IN_LINK_ADDR_CH2_REG	配置 RX 通道 2 接收链表描述符的地址	0x0720	R/W
DMA2D_IN_ARB_CH2_REG	配置 RX 通道 2 的仲裁	0x0740	R/W
DMA2D_IN_ETM_CONF_CH2_REG	配置 RX 通道 2 的 ETM	0x0748	R/W
DMA2D_RST_CONF_REG	配置 AXI 复位	0x0A04	R/W
DMA2D_INTR_MEM_START_ADDR_REG	内部存储器可访问地址空间的起始地址	0x0A08	R/W
DMA2D_INTR_MEM_END_ADDR_REG	内部存储器可访问地址空间的结束地址	0x0A0C	R/W
DMA2D_EXTR_MEM_START_ADDR_REG	外部存储器可访问地址空间的起始地址	0x0A10	R/W
DMA2D_EXTR_MEM_END_ADDR_REG	外部存储器可访问地址空间的结束地址	0x0A14	R/W
DMA2D_OUT_ARB_CONFIG_REG	配置 TX 方向的仲裁	0x0A18	R/W
DMA2D_IN_ARB_CONFIG_REG	配置 RX 方向的仲裁	0x0A1C	R/W
中断寄存器			
DMA2D_OUT_INT_RAW_CH0_REG	TX 通道 0 的原始中断状态	0x0004	R/WTC/SS
DMA2D_OUT_INT_ENA_CH0_REG	TX 通道 0 的中断使能位	0x0008	R/W
DMA2D_OUT_INT_ST_CH0_REG	TX 通道 0 的屏蔽中断状态	0x000C	RO
DMA2D_OUT_INT_CLR_CH0_REG	TX 通道 0 的中断清除位	0x0010	WT
DMA2D_OUT_INT_RAW_CH1_REG	TX 通道 1 的原始中断状态	0x0104	R/WTC/SS
DMA2D_OUT_INT_ENA_CH1_REG	TX 通道 1 的中断使能位	0x0108	R/W
DMA2D_OUT_INT_ST_CH1_REG	TX 通道 1 的屏蔽中断状态	0x010C	RO
DMA2D_OUT_INT_CLR_CH1_REG	TX 通道 1 的中断清除位	0x0110	WT
DMA2D_OUT_INT_RAW_CH2_REG	TX 通道 2 的原始中断状态	0x0204	R/WTC/SS
DMA2D_OUT_INT_ENA_CH2_REG	TX 通道 2 的中断使能位	0x0208	R/W
DMA2D_OUT_INT_ST_CH2_REG	TX 通道 2 的屏蔽中断状态	0x020C	RO
DMA2D_OUT_INT_CLR_CH2_REG	TX 通道 2 的中断清除位	0x0210	WT
DMA2D_OUT_INT_RAW_CH3_REG	TX 通道 3 的原始中断状态	0x0304	R/WTC/SS
DMA2D_OUT_INT_ENA_CH3_REG	TX 通道 3 的中断使能位	0x0308	R/W

名称	描述	地址	访问
DMA2D_OUT_INT_ST_CH3_REG	TX 通道 3 的屏蔽中断状态	0x030C	RO
DMA2D_OUT_INT_CLR_CH3_REG	TX 通道 3 的中断清除位	0x0310	WT
DMA2D_IN_INT_RAW_CH0_REG	RX 通道 0 的原始中断状态	0x0504	R/WTC/SS
DMA2D_IN_INT_ENA_CH0_REG	RX 通道 0 的中断使能位	0x0508	R/W
DMA2D_IN_INT_ST_CH0_REG	RX 通道 0 的屏蔽中断状态	0x050C	RO
DMA2D_IN_INT_CLR_CH0_REG	RX 通道 0 的中断清除位	0x0510	WT
DMA2D_IN_INT_RAW_CH1_REG	RX 通道 1 的原始中断状态	0x0604	R/WTC/SS
DMA2D_IN_INT_ENA_CH1_REG	RX 通道 1 的中断使能位	0x0608	R/W
DMA2D_IN_INT_ST_CH1_REG	RX 通道 1 的屏蔽中断状态	0x060C	RO
DMA2D_IN_INT_CLR_CH1_REG	RX 通道 1 的中断清除位	0x0610	WT
DMA2D_IN_INT_RAW_CH2_REG	RX 通道 2 的原始中断状态	0x0704	R/WTC/SS
DMA2D_IN_INT_ENA_CH2_REG	RX 通道 2 的中断使能位	0x0708	R/W
DMA2D_IN_INT_ST_CH2_REG	RX 通道 2 的屏蔽中断状态	0x070C	RO
DMA2D_IN_INT_CLR_CH2_REG	RX 通道 2 的中断清除位	0x0710	WT
状态寄存器			
DMA2D_OUTFIFO_STATUS_CH0_REG	表示 TX 通道 0 的 FIFO 状态	0x0014	RO
DMA2D_OUT_STATE_CH0_REG	表示 TX 通道 0 发送链表描述符的工作状态	0x0024	RO
DMA2D_OUT_EOF_DES_ADDR_CH0_REG	表示 TX 通道 0 传输结束 (EOF) 时的发送链表描述符地址	0x0028	RO
DMA2D_OUT_DSCR_CH0_REG	表示 TX 通道 0 已预读取的发送链表描述符指向的下一个发送链表描述符地址	0x002C	RO
DMA2D_OUT_DSCR_BF0_CH0_REG	表示 TX 通道 0 当前已预读取的发送链表描述符地址	0x0030	RO
DMA2D_OUT_DSCR_BF1_CH0_REG	表示 TX 通道 0 上一个已预读取的发送链表描述符地址	0x0034	RO
DMA2D_OUTFIFO_STATUS_CH1_REG	表示 TX 通道 1 的 FIFO 状态	0x0114	RO
DMA2D_OUT_STATE_CH1_REG	表示 TX 通道 0 发送链表描述符的工作状态	0x0124	RO
DMA2D_OUT_EOF_DES_ADDR_CH1_REG	表示 TX 通道 1 传输结束 (EOF) 时的发送链表描述符地址	0x0128	RO
DMA2D_OUT_DSCR_CH1_REG	表示 TX 通道 1 已预读取的发送链表描述符指向的下一个发送链表描述符地址	0x012C	RO
DMA2D_OUT_DSCR_BF0_CH1_REG	表示 TX 通道 1 当前已预读取的发送链表描述符地址	0x0130	RO
DMA2D_OUT_DSCR_BF1_CH1_REG	表示 TX 通道 1 上一个已预读取的发送链表描述符地址	0x0134	RO

名称	描述	地址	访问
DMA2D_OUTFIFO_STATUS_CH2_REG	表示 TX 通道 2 的 FIFO 状态	0x0214	RO
DMA2D_OUT_STATE_CH2_REG	表示 TX 通道 0 发送链表描述符的工作状态	0x0224	RO
DMA2D_OUT_EOF_DES_ADDR_CH2_REG	表示 TX 通道 2 传输结束 (EOF) 时的发送链表描述符地址	0x0228	RO
DMA2D_OUT_DSCR_CH2_REG	表示 TX 通道 2 已预读取的发送链表描述符指向的下一个发送链表描述符地址	0x022C	RO
DMA2D_OUT_DSCR_BF0_CH2_REG	表示 TX 通道 2 当前已预读取的发送链表描述符地址	0x0230	RO
DMA2D_OUT_DSCR_BF1_CH2_REG	表示 TX 通道 2 上一个已预读取的发送链表描述符地址	0x0234	RO
DMA2D_INFIFO_STATUS_CH0_REG	表示 RX 通道 0 的 FIFO 状态	0x0514	RO
DMA2D_IN_STATE_CH0_REG	表示 RX 通道 0 接收链表描述符的工作状态	0x0524	RO
DMA2D_IN_SUC_EOF_DES_ADDR_CH0_REG	表示 RX 通道 0 传输结束 (EOF) 时的接收链表描述符地址	0x0528	RO
DMA2D_IN_ERR_EOF_DES_ADDR_CH0_REG	表示 RX 通道 0 上发生错误时的接收链表描述符地址	0x052C	RO
DMA2D_IN_DSCR_CH0_REG	表示 RX 通道 0 已预读取的接收链表描述符指向的下一个接收链表描述符地址	0x0530	RO
DMA2D_IN_DSCR_BF0_CH0_REG	表示 RX 通道 0 当前已预读取的接收链表描述符地址	0x0534	RO
DMA2D_IN_DSCR_BF1_CH0_REG	表示 RX 通道 0 上一个已预读取的接收链表描述符地址	0x0538	RO
DMA2D_INFIFO_STATUS_CH1_REG	表示 RX 通道 1 的 FIFO 状态	0x0614	RO
DMA2D_IN_STATE_CH1_REG	表示 RX 通道 1 接收链表描述符的工作状态	0x0624	RO
DMA2D_IN_SUC_EOF_DES_ADDR_CH1_REG	表示 RX 通道 1 传输结束 (EOF) 时的接收链表描述符地址	0x0628	RO
DMA2D_IN_ERR_EOF_DES_ADDR_CH1_REG	表示 RX 通道 1 上发生错误时的接收链表描述符地址	0x062C	RO
DMA2D_IN_DSCR_CH1_REG	表示 RX 通道 1 已预读取的接收链表描述符指向的下一个接收链表描述符地址	0x0630	RO
DMA2D_IN_DSCR_BF0_CH1_REG	表示 RX 通道 1 当前已预读取的接收链表描述符地址	0x0634	RO
DMA2D_IN_DSCR_BF1_CH1_REG	表示 RX 通道 1 上一个已预读取的接收链表描述符地址	0x0638	RO
DMA2D_AXI_ERR_REG	表示 AXI 总线的状态	0x0A00	RO
DMA2D_DATE_REG	版本寄存器	0x0A2C	R/W
外设选择寄存器			
DMA2D_OUT_PERI_SEL_CH0_REG	配置 TX 通道 0 连接的外设	0x0038	R/W
DMA2D_OUT_PERI_SEL_CH1_REG	配置 TX 通道 1 连接的外设	0x0138	R/W
DMA2D_OUT_PERI_SEL_CH2_REG	配置 TX 通道 2 连接的外设	0x0238	R/W

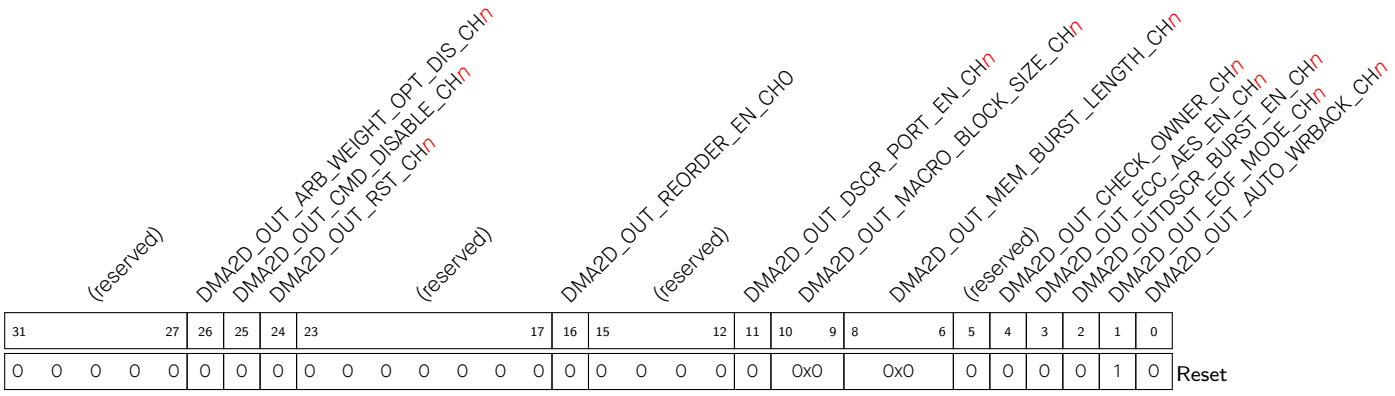
名称	描述	地址	访问
DMA2D_OUT_PERI_SEL_CH3_REG	配置 TX 通道 3 连接的外设	0x0338	R/W
DMA2D_IN_PERI_SEL_CH0_REG	配置 RX 通道 0 连接的外设	0x053C	R/W
DMA2D_IN_PERI_SEL_CH1_REG	配置 RX 通道 1 连接的外设	0x063C	R/W
DMA2D_IN_PERI_SEL_CH2_REG	配置 RX 通道 2 连接的外设	0x073C	R/W

4.9 寄存器

本小节的所有地址均为相对于 2D-DMA 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 4.1. DMA2D_OUT_CONFO_CH*n*_REG (*n*: 0-3) (0x0000+0x100n*)**


(reserved)
DMA2D_OUT_ARB_WEIGHT_OPT_DIS_CHn
DMA2D_OUT_CMD_DISABLE_CHn
DMA2D_OUT_RST_CHn
(reserved)
DMA2D_OUT_REORDER_EN_CHn
(reserved)
DMA2D_OUT_DSCR_PORT_EN_CHn
DMA2D_OUT_MACRO_BLOCK_SIZE_CHn
DMA2D_OUT_MEM_BURST_LENGTH_CHn
(reserved)
DMA2D_OUT_CHECK_OWNER_CHn
DMA2D_OUT_ECC_AES_EN_CHn
DMA2D_OUTDSCR_BURST_EN_CHn
DMA2D_OUT_EOF_MODE_CHn
DMA2D_OUT_AUTO_WRBACK_CHn
31 27 26 25 24 23 17 16 15 12 11 10 9 8 6 5 4 3 2 1 0
0 0 0

Register 4.1. DMA2D_OUT_CONF0_CH*n*_REG (*n*: 0-3) (0x0000+0x100**n*)

接上页...

DMA2D_OUT_CHECK_OWNER_CH*n* 配置发送通道 *n* 是否开启 owner 位检查。

0：关闭

1：开启

(R/W)

DMA2D_OUT_MEM_BURST_LENGTH_CH*n* 配置发送通道 *n* 的突发传输长度。

0：8 字节

1：16 字节

2：32 字节

3：64 字节

4：128 字节

其他值：无效

(R/W)

DMA2D_OUT_MACRO_BLOCK_SIZE_CH*n* 配置发送通道 *n* 的宏块大小。仅用于 2D-MOD1 模式。

0：8 * 8 像素

1：8 * 16 像素

2：16 *16 像素

3：无宏块

(R/W)

DMA2D_OUT_DSCR_PORT_EN_CH*n* 配置发送通道 *n* 是否开启 DSCR-PORT 模式。

0：关闭

1：开启

(R/W)

DMA2D_OUT_REORDER_EN_CH0 配置发送通道 0 是否开启宏块重排。

0：关闭

1：开启

(R/W)

DMA2D_OUT_RST_CH*n* 配置是否复位发送通道 *n*。

0：复位释放

1：复位

(R/W)

见下页...

Register 4.1. DMA2D_OUT_CONF0_CH*n*_REG (*n*: 0-3) (0x0000+0x100**n*)

接上页...

DMA2D_OUT_CMD_DISABLE_CH*n* 配置发送通道 *n* 的复位指令。

0：复位释放时清零

1：复位前暂停传输

(R/W)

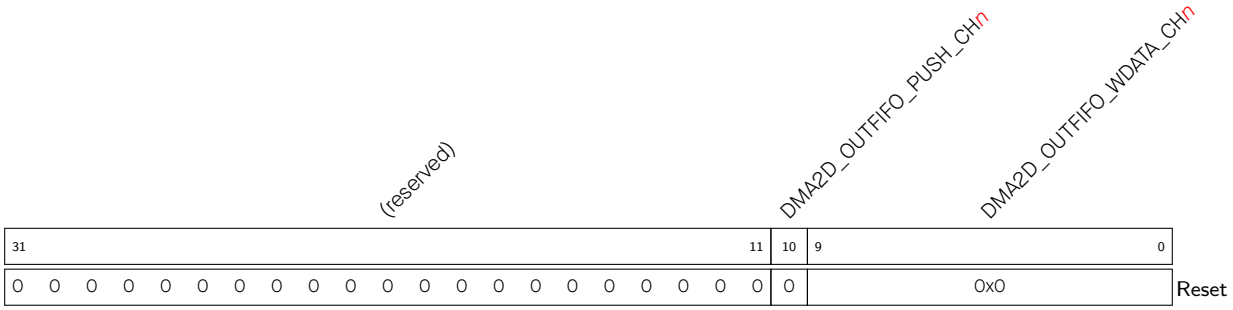
DMA2D_OUT_ARB_WEIGHT_OPT_DIS_CH*n* 配置发送通道 *n* 是否开启权重仲裁优化。

0：开启

1：关闭

(R/W)

Register 4.2. DMA2D_OUT_PUSH_CH*n*_REG (*n*: 0-3) (0x0018+0x100**n*)



DMA2D_OUTFIFO_WDATA_CH*n* 配置需要推入 2D-DMA TX FIFO 的数据。(R/W)

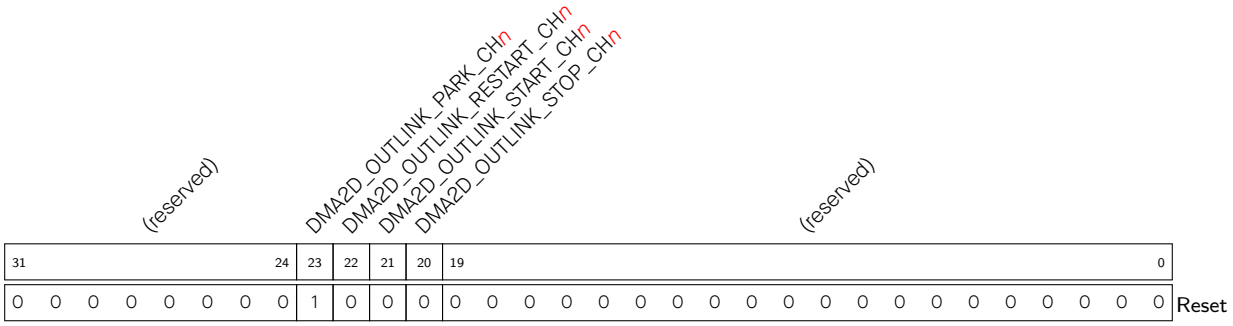
DMA2D_OUTFIFO_PUSH_CH*n* 配置是否将数据推入 2D-DMA TX FIFO。

0：不推送

1：推送

(R/W/SC)

Register 4.3. DMA2D_OUT_LINK_CONF_CH*n*_REG (*n*: 0-3) (0x001C+0x100**n*)



DMA2D_OUTLINK_STOP_CH*n* 配置是否停止发送通道 *n* 的数据传输。

0：无效值。没有作用

1：停止

(R/W/SC)

DMA2D_OUTLINK_START_CH*n* 配置是否开启发送通道 *n* 进行数据传输。

0：关闭

1：开启

(R/W/SC)

DMA2D_OUTLINK_RESTART_CH*n* 配置是否重启发送通道 *n* 进行 2D-DMA 传输。

0：无效值。没有作用

1：重启

(R/W/SC)

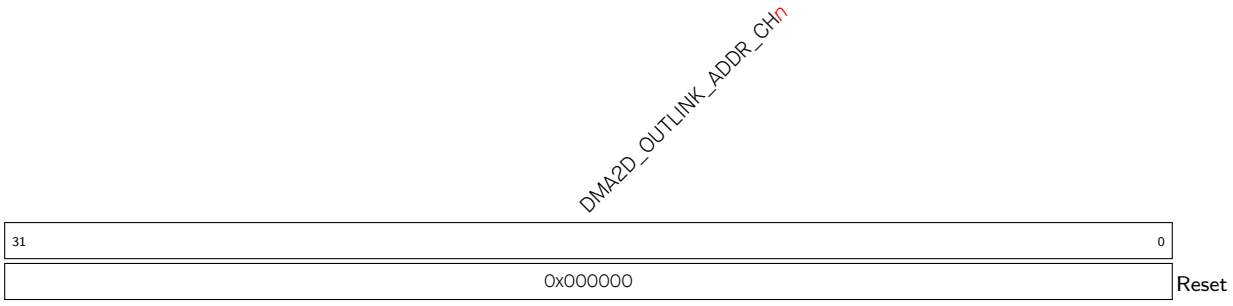
DMA2D_OUTLINK_PARK_CH*n* 表示发送链表描述符的 FSM 的状态。

0：运行

1：空闲

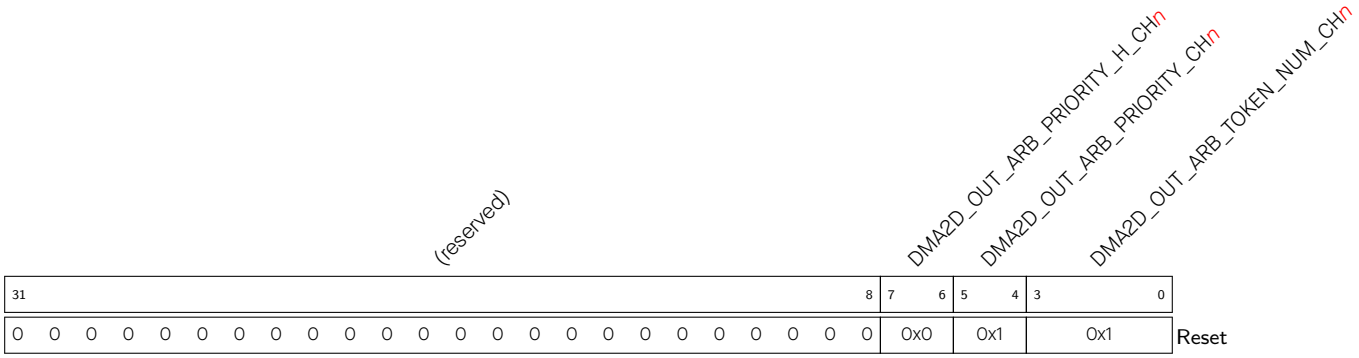
(RO)

Register 4.4. DMA2D_OUT_LINK_ADDR_CH*n*_REG (*n*: 0-3) (0x0020+0x100**n*)



DMA2D_OUTLINK_ADDR_CH*n* 表示第一个发送链表描述符的地址。(R/W)

Register 4.5. DMA2D_OUT_ARB_CH*n*_REG (*n*: 0-3) (0x003C+0x100**n*)



DMA2D_OUT_ARB_TOKEN_NUM_CH*n* 配置发送通道 *n* 的权重（即令牌数）。

取值范围：0 ~ 15。(R/W)

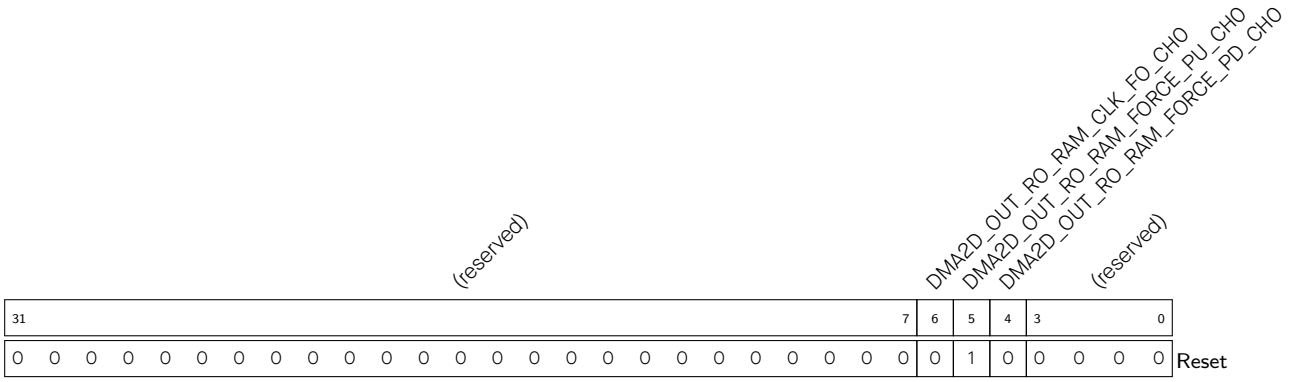
DMA2D_OUT_ARB_PRIORITY_CH*n* 配置发送通道 *n* 优先级的低两位。

该字段和 DMA2D_OUT_ARB_PRIORITY_H_CH*n* 共同配置发送通道 *n* 的四位优先级数值，最低值为 0，最高值为 15。(R/W)

DMA2D_OUT_ARB_PRIORITY_H_CH*n* 配置发送通道 *n* 优先级的高两位。

该字段和 DMA2D_OUT_ARB_PRIORITY_CH*n* 共同配置发送通道 *n* 的四位优先级数值，最低值为 0，最高值为 15。(R/W)

Register 4.6. DMA2D_OUT_RO_PD_CONF_CH0_REG (0x0044)



DMA2D_OUT_RO_RAM_FORCE_PD_CH0 配置是否强制关闭发送通道 0 的宏块重排 RAM 的电源。

0：不强制关闭电源

1：强制关闭电源

(R/W)

DMA2D_OUT_RO_RAM_FORCE_PU_CH0 配置是否强制开启发送通道 0 的宏块重排 RAM 的电源。

0：不强制开启电源

1：强制开启电源

(R/W)

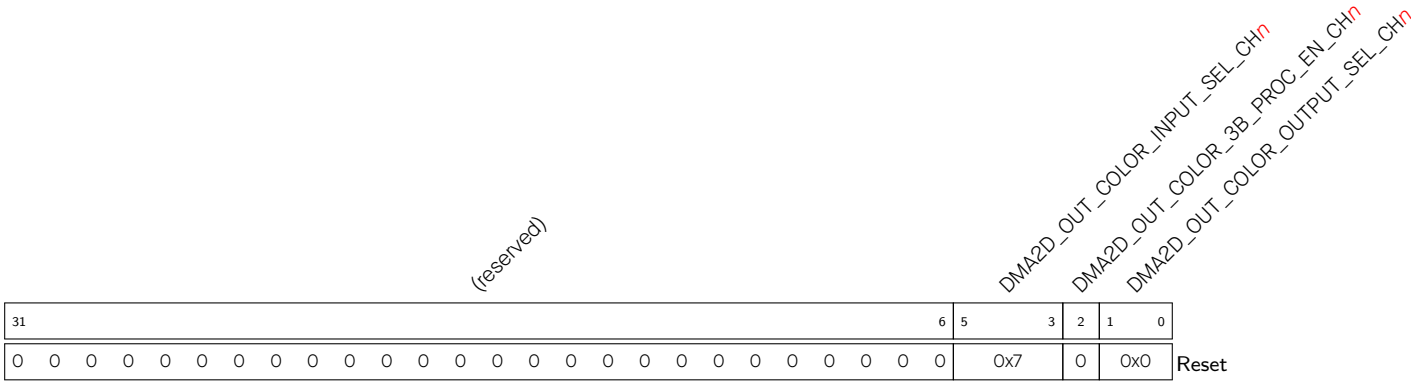
DMA2D_OUT_RO_RAM_CLK_FO_CH0 配置发送通道 0 访问 2D-DMA RAM 的时钟门控。

0：使用时钟门控

1：强制开启时钟，绕过时钟门控

(R/W)

Register 4.7. DMA2D_OUT_COLOR_CONVERT_CH*n*_REG (*n*: 0-3) (0x0048+0x100**n*)



DMA2D_OUT_COLOR_OUTPUT_SEL_CH*n* 配置发送通道 *n* 颜色空间转换第三级的输出。

0：RGB888 转 RGB565

1：YUV444 转 YUV422

2：直接输出

其他值：无效

(R/W)

DMA2D_OUT_COLOR_3B_PROC_EN_CH*n* 配置是否开启发送通道 *n* 颜色空间转换的第二级。

0：关闭

1：开启

(R/W)

DMA2D_OUT_COLOR_INPUT_SEL_CH*n* 配置发送通道 *n* 颜色空间转换第一级的输出。

0：RGB565 转 RGB888

1：YUV422 转 YUV444

2：其他 2 字节/像素的颜色格式

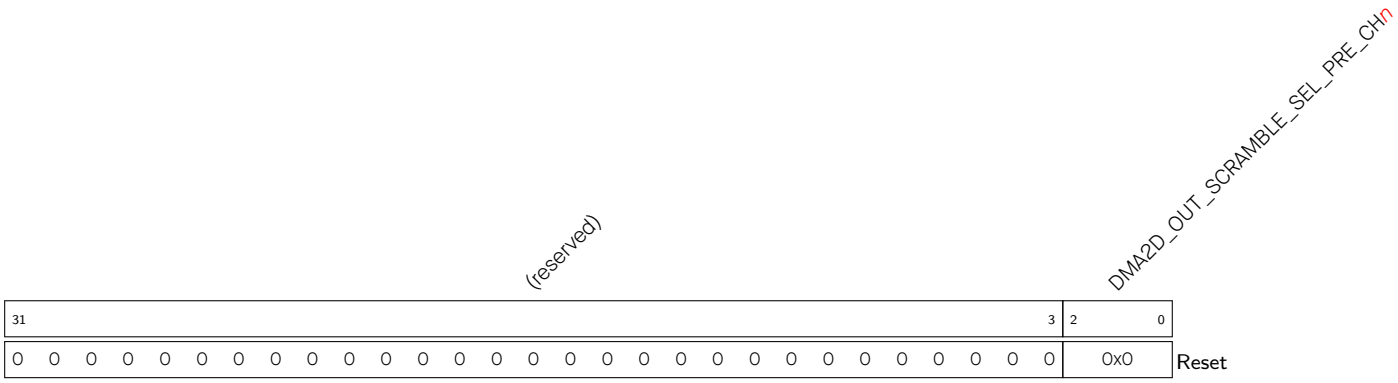
3：其他 3 字节/像素的颜色格式

7：关闭颜色空间转换

其他值：无效

(R/W)

Register 4.8. DMA2D_OUT_SCRAMBLE_CH*n*_REG (*n*: 0-3) (0x004C+0x100*n*)



DMA2D_OUT_SCRAMBLE_SEL_PRE_CH*n* 配置发送通道 *n* 颜色空间转换前的字节乱序处理。

0：BYTE2-1-0

1：BYTE2-0-1

2：BYTE1-0-2

3：BYTE1-2-0

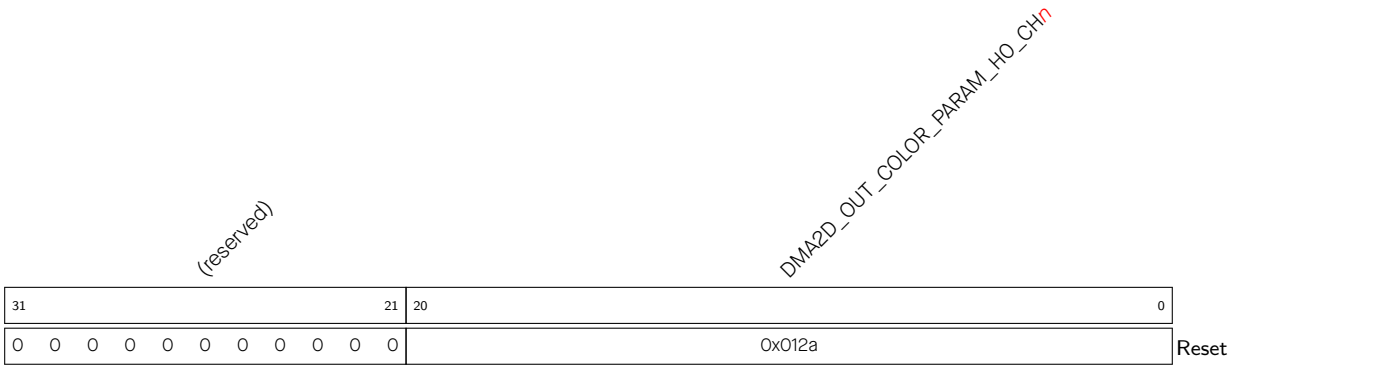
4：BYTE0-2-1

5：BYTE0-1-2

其他值：无效

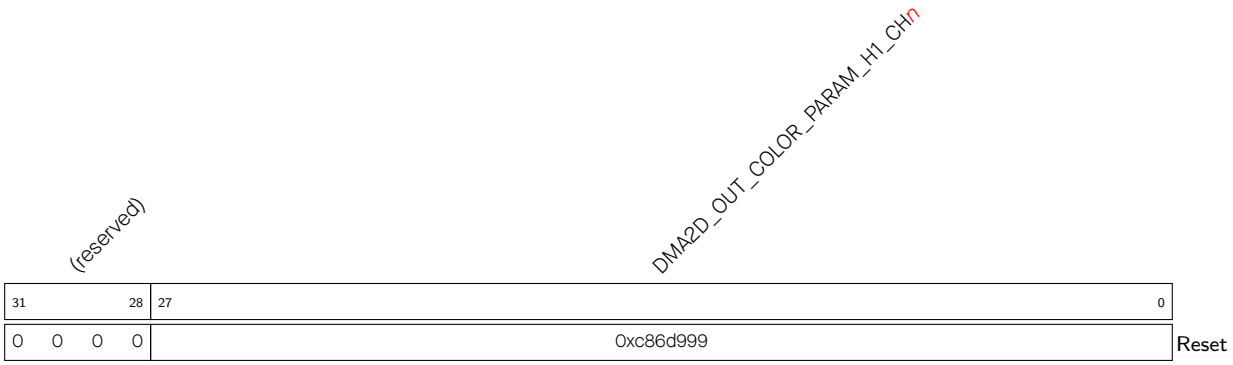
(R/W)

Register 4.9. DMA2D_OUT_COLOR_PARAM0_CH*n*_REG (*n*: 0-3) (0x0050+0x100*n*)



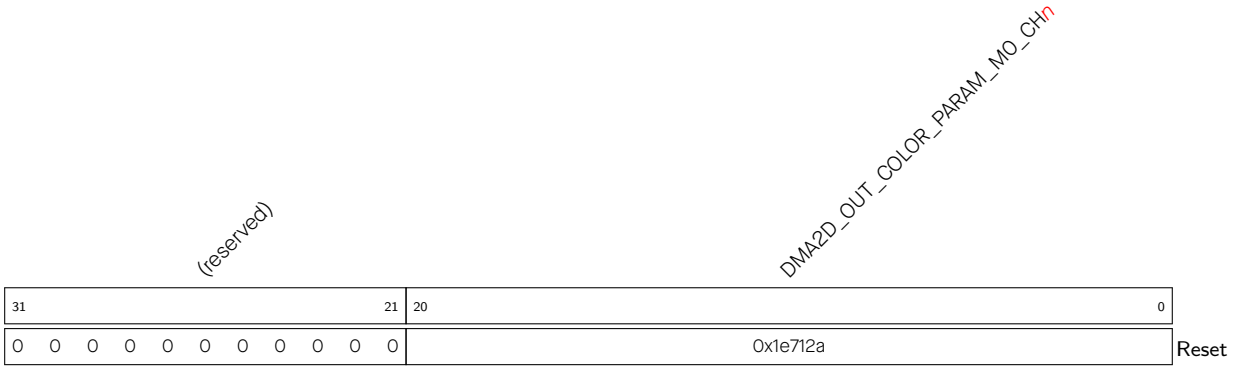
DMA2D_OUT_COLOR_PARAM_H0_CH*n* 配置发送通道 *n* 颜色空间转换第二级输入的最高有效字节的系数 A 和 B。(R/W)

Register 4.10. DMA2D_OUT_COLOR_PARAM1_CH*n*_REG (*n*: 0-3) (0x0054+0x100*n*)



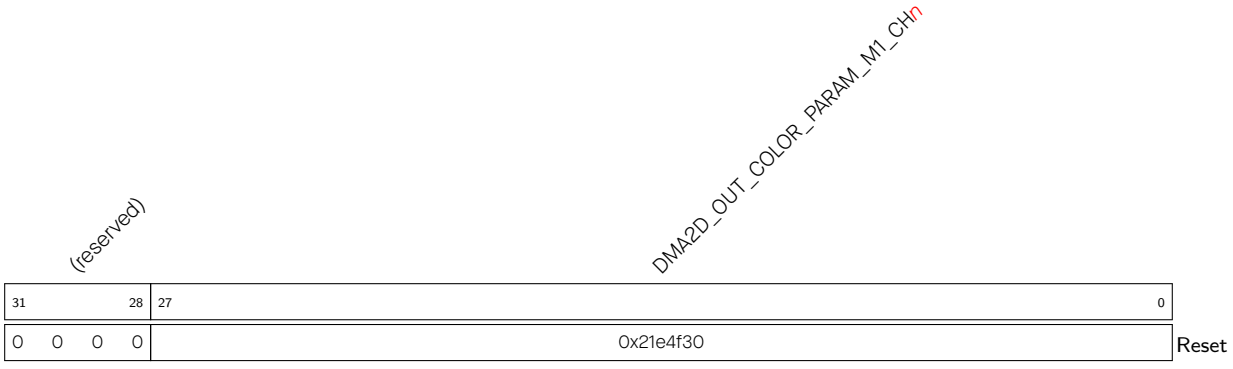
DMA2D_OUT_COLOR_PARAM_H1_CH*n* 配置发送通道 *n* 颜色空间转换第二级输入的最高有效字节的系数 C 和 D。(R/W)

Register 4.11. DMA2D_OUT_COLOR_PARAM2_CH*n*_REG (*n*: 0-3) (0x0058+0x100*n*)



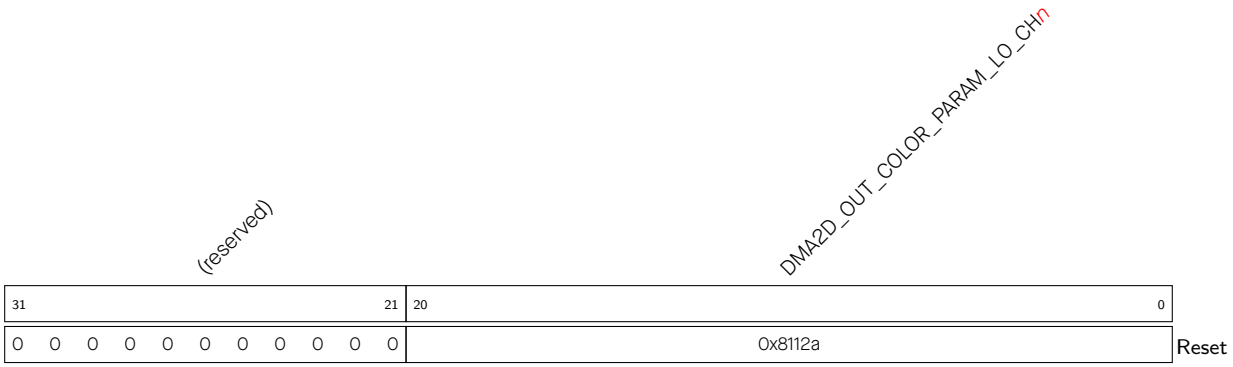
DMA2D_OUT_COLOR_PARAM_M0_CH*n* 配置发送通道 *n* 颜色空间转换第二级输入的中间字节的系数 A 和 B。(R/W)

Register 4.12. DMA2D_OUT_COLOR_PARAM3_CH*n*_REG (*n*: 0-3) (0x005C+0x100*n*)



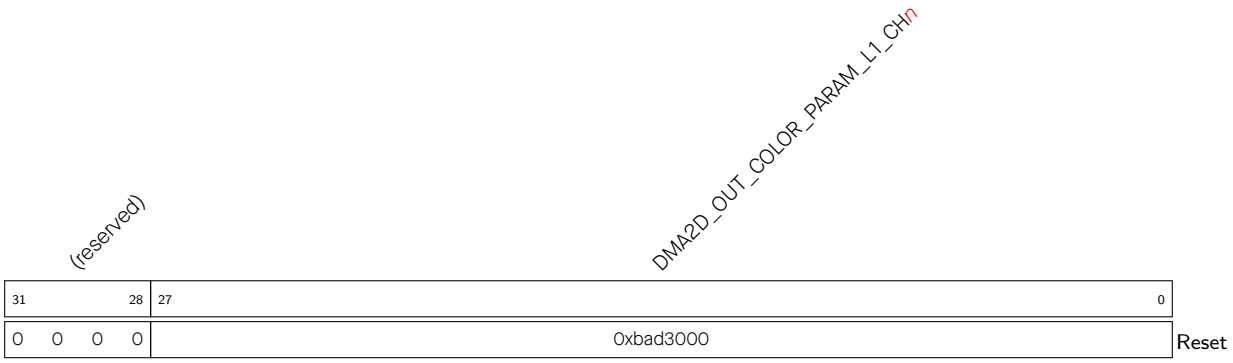
DMA2D_OUT_COLOR_PARAM_M1_CH*n* 配置发送通道 *n* 颜色空间转换第二级输入的中间字节的系数 C 和 D。(R/W)

Register 4.13. DMA2D_OUT_COLOR_PARAM4_CH*n*_REG (*n*: 0-3) (0x0060+0x100**n*)



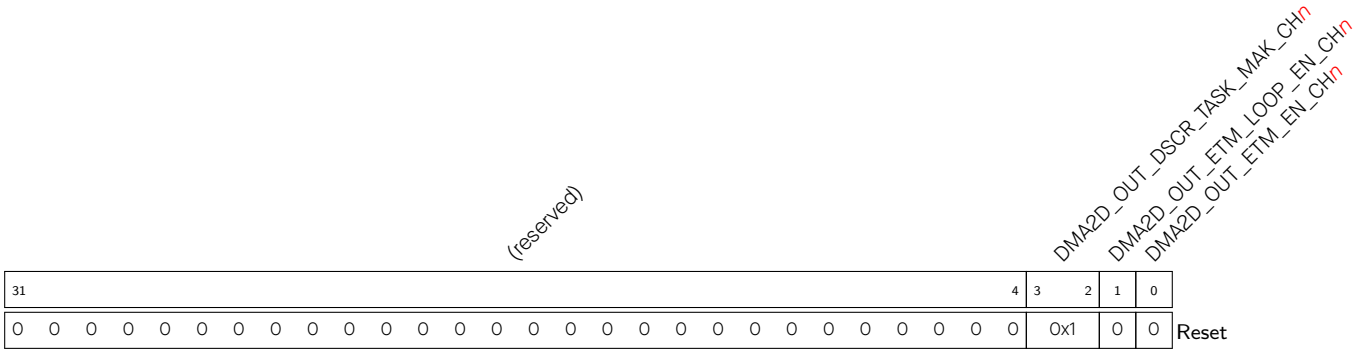
DMA2D_OUT_COLOR_PARAM_L0_CH*n* 配置发送通道 *n* 颜色空间转换第二级输入的最低有效字节的系数 A 和 B。(R/W)

Register 4.14. DMA2D_OUT_COLOR_PARAM5_CH*n*_REG (*n*: 0-3) (0x0064+0x100**n*)



DMA2D_OUT_COLOR_PARAM_L1_CH*n* 配置发送通道 *n* 颜色空间转换第二级输入的最低有效字节的系数 C 和 D。(R/W)

Register 4.15. DMA2D_OUT_ETM_CONF_CH*n*_REG (*n*: 0-3) (0x0068+0x100**n*)



DMA2D_OUT_ETM_EN_CH*n* 配置发送通道 *n* 是否开启 ETM 功能。

0：关闭

1：开启

((R/W)

DMA2D_OUT_ETM_LOOP_EN_CH*n* 配置发送通道 *n* 是否使用 ETM 任务指示下一个描述符的处理。

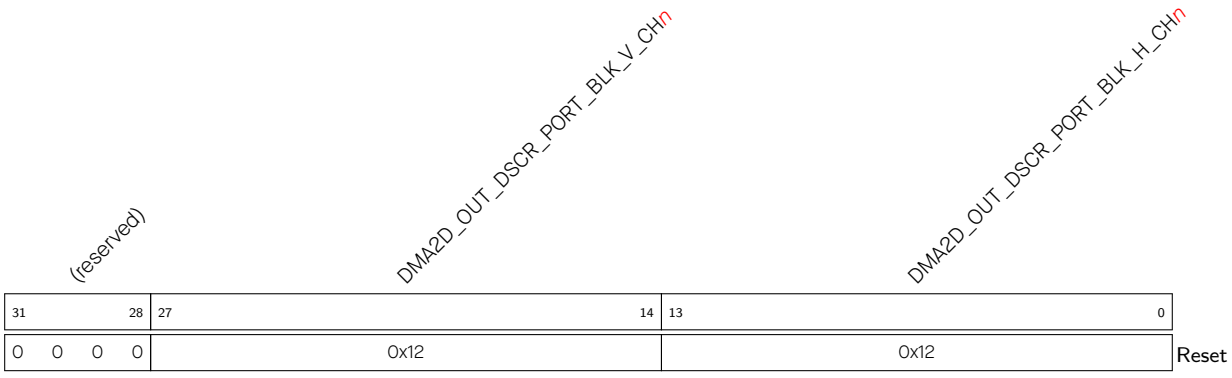
0：不使用 ETM 任务

1：使用 ETM 任务

(R/W)

DMA2D_OUT_DSCR_TASK_MAK_CH*n* 配置发送通道 *n* 可缓存任务的最大值。(R/W)

Register 4.16. DMA2D_OUT_DSCR_PORT_BLK_CH*n*_REG (*n*: 0-3) (0x006C+0x100**n*)



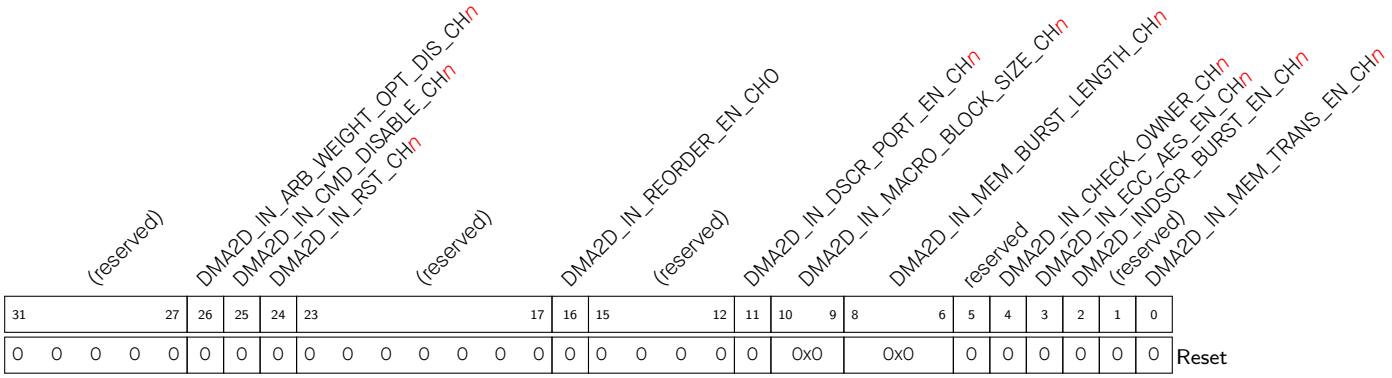
DMA2D_OUT_DSCR_PORT_BLK_H_CH*n* 配置 DSCR-PORT 模式下发送通道 *n* 基本单元的水平宽度。

单位：像素。(R/W)

DMA2D_OUT_DSCR_PORT_BLK_V_CH*n* 配置 DSCR-PORT 模式下发送通道 *n* 基本单元的水平高度。

单位：像素。(R/W)

Register 4.17. DMA2D_IN_CONF0_CH*n*_REG (*n*: 0-2) (0x0500+0x100*n)


(reserved)
DMA2D_IN_ARB_WEIGHT_OPT_DIS_CHn
DMA2D_IN_CMD_DISABLE_CHn
DMA2D_IN_RST_CHn
(reserved)
DMA2D_IN_REORDER_EN_CH0
(reserved)
DMA2D_IN_DSCR_PORT_EN_CHn
DMA2D_IN_MACRO_BLOCK_SIZE_CHn
DMA2D_IN_MEM_BURST_LENGTH_CHn
reserved
DMA2D_IN_CHECK_OWNER_CHn
DMA2D_IN_ECC_AES_EN_CHn
DMA2D_INDSCR_BURST_EN_CHn
(reserved)
DMA2D_IN_MEM_TRANS_EN_CHn
31 27 26 25 24 23 17 16 15 12 11 10 9 8 6 5 4 3 2 1 0
0 0 0

Register 4.17. DMA2D_IN_CONF0_CH*n*_REG (*n*: 0-2) (0x0500+0x100**n*)

接上页...

DMA2D_IN_MACRO_BLOCK_SIZE_CH*n* 配置接收通道 *n* 的宏块大小。仅用于 2D-MOD1 模式。

0：8 * 8 像素

1：8 * 16 像素

2：16 *16 像素

3：无宏块

(R/W)

DMA2D_IN_DSCR_PORT_EN_CH*n* 配置接收通道 *n* 是否开启 DSCR-PORT 模式。

0：关闭

1：开启

(R/W)

DMA2D_IN_REORDER_EN_CHO 配置接收通道 0 是否开启宏块重排。

0：关闭

1：开启

(R/W)

DMA2D_IN_RST_CH*n* 配置是否复位接收通道 *n*。

0：复位释放

1：复位

(R/W)

DMA2D_IN_CMD_DISABLE_CH*n* 配置接收通道 *n* 的复位指令。

0：复位释放时清零

1：复位前暂停传输

(R/W)

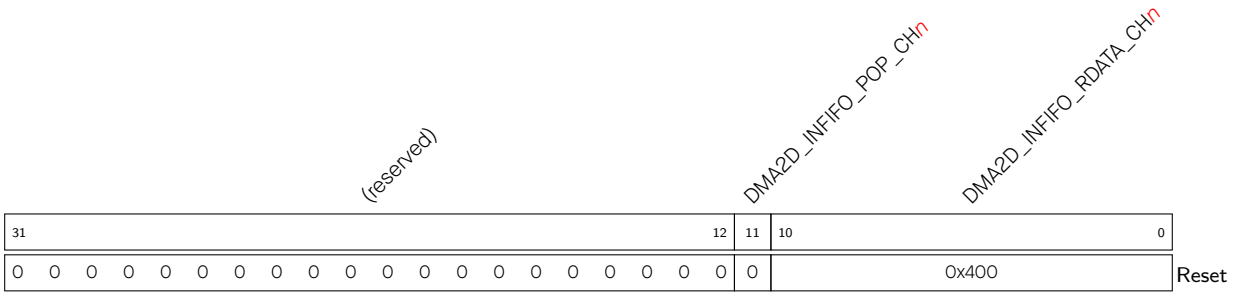
DMA2D_IN_ARB_WEIGHT_OPT_DIS_CH*n* 配置接收通道 *n* 是否开启权重仲裁优化。

0：开启

1：关闭

(R/W)

Register 4.18. DMA2D_IN_POP_CH*n*_REG (*n*: 0-2) (0x0518+0x100*n*)



DMA2D_INFIFO_RDATA_CH*n* 表示从 2D-DMA RX FIFO 弹出的数据。(RO)

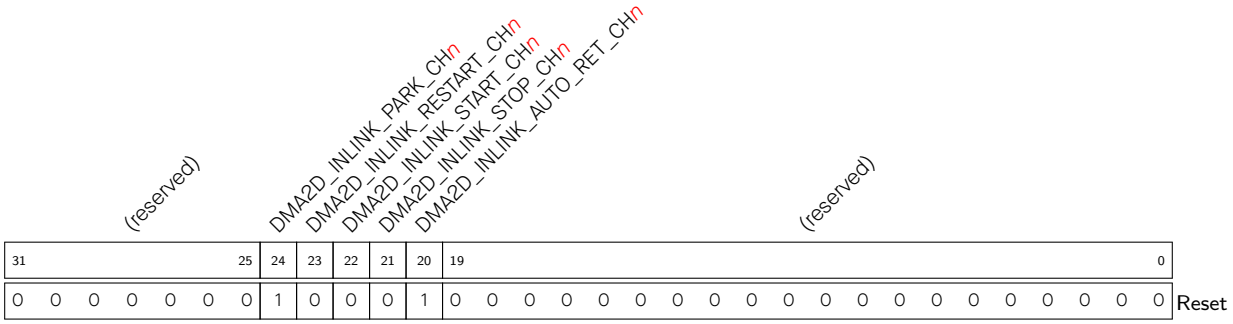
DMA2D_INFIFO_POP_CH*n* 配置是否从 2D-DMA RX FIFO 弹出数据。

0：不弹出

1：弹出

(R/W/SC)

Register 4.19. DMA2D_IN_LINK_CONF_CH*n*_REG (*n*: 0-2) (0x051C+0x100**n*)



DMA2D_INLINK_AUTO_RET_CH*n* 配置回写到接收链表描述符 owner 位的值。

0：回写 0

1：回写 1

(R/W)

DMA2D_INLINK_STOP_CH*n* 配置接收通道 *n* 是否停止接收数据。

0：无效值。没有作用

1：停止

(R/W/SC)

DMA2D_INLINK_START_CH*n* 配置是否开启接收通道 *n* 进行数据传输。

0：关闭

1：开启

(R/W/SC)

DMA2D_INLINK_RESTART_CH*n* 配置是否重启接收通道 *n* 进行 2D-DMA 传输。

0：无效值。没有作用

1：重启

(R/W/SC)

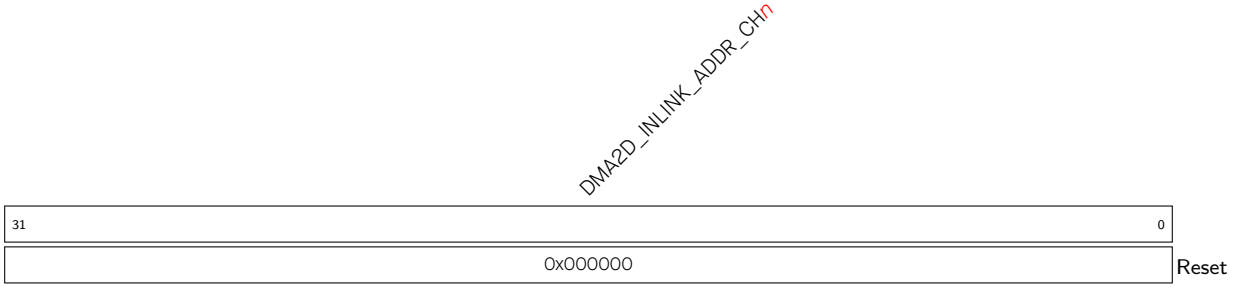
DMA2D_INLINK_PARK_CH*n* 表示接收链表描述符 FSM 的状态。

0：运行

1：空闲

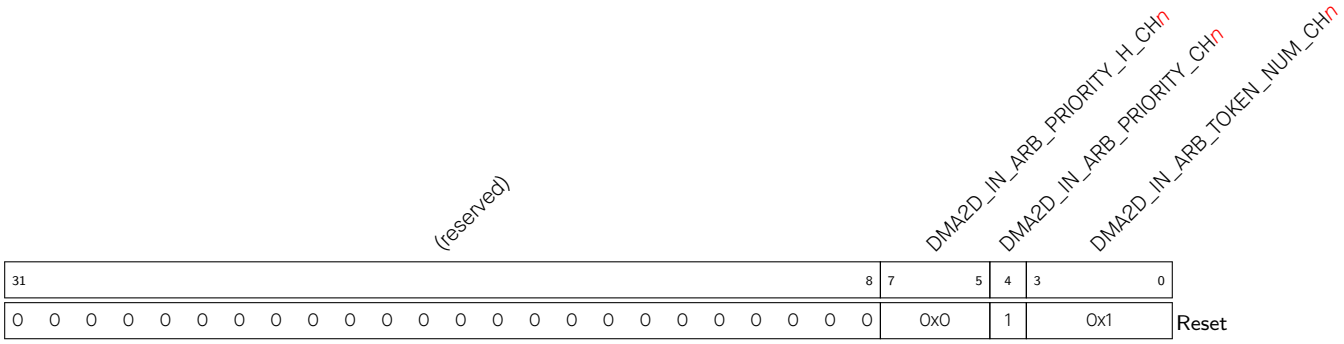
(RO)

Register 4.20. DMA2D_IN_LINK_ADDR_CH*n*_REG (*n*: 0-2) (0x0520+0x100**n*)



DMA2D_INLINK_ADDR_CH*n* 表示第一个接收链表描述符的地址。(R/W)

Register 4.21. DMA2D_IN_ARB_CH*n*_REG (*n*: 0-2) (0x0540+0x100**n*)



DMA2D_IN_ARB_TOKEN_NUM_CH*n* 配置接收通道 *n* 的权重（即令牌数）。

取值范围：0 ~ 15。(R/W)

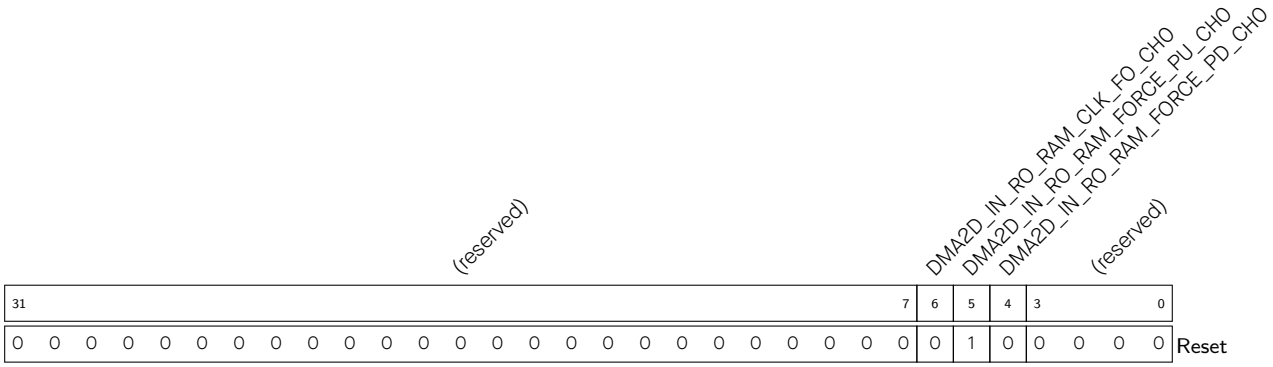
DMA2D_IN_ARB_PRIORITY_CH*n* 配置接收通道 *n* 优先级的低两位。

该字段和 [DMA2D_IN_ARB_PRIORITY_H_CH*n* 共同配置接收通道 *n* 的四位优先级数值，最低值为 0，最高值为 15。(R/W)

DMA2D_IN_ARB_PRIORITY_H_CH*n* 配置接收通道 *n* 优先级的高两位。

该字段和 DMA2D_IN_ARB_PRIORITY_CH*n* 共同配置接收通道 *n* 的四位优先级数值，最低值为 0，最高值为 15。(R/W)

Register 4.22. DMA2D_IN_RO_PD_CONF_CH0_REG (0x0548)



DMA2D_IN_RO_RAM_FORCE_PD_CH0 配置是否强制关闭接收通道 0 的宏块重排 RAM 的电源。

0：不强制关闭电源

1：强制关闭电源

(R/W)

DMA2D_IN_RO_RAM_FORCE_PU_CH0 配置是否强制开启接收通道 0 的宏块重排 RAM 的电源。

0：不强制开启电源

1：强制开启电源

(R/W)

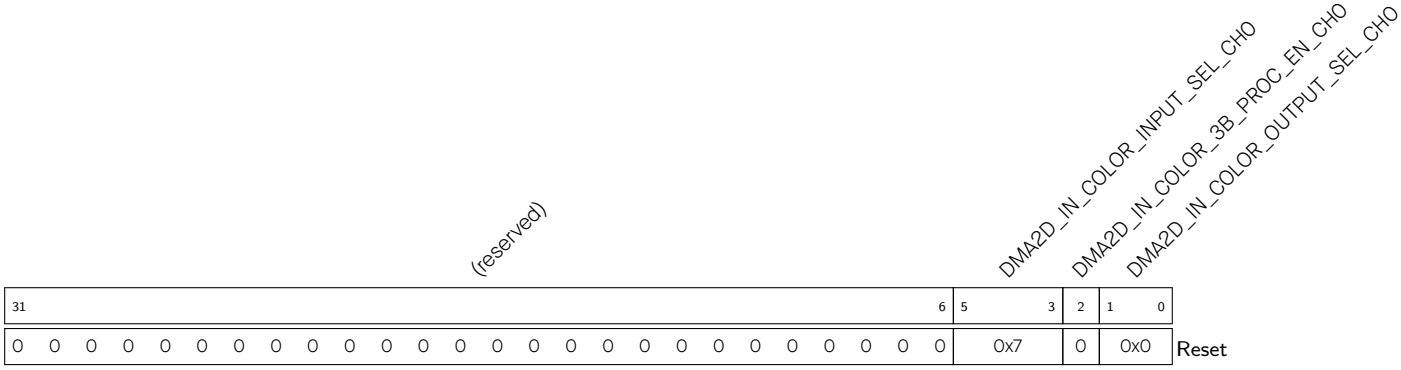
DMA2D_IN_RO_RAM_CLK_FO_CH0 配置接收通道 0 访问 2D-DMA RAM 的时钟门控。

0：使用时钟门控

1：强制开启时钟，绕过时钟门控

(R/W)

Register 4.23. DMA2D_IN_COLOR_CONVERT_CH0_REG (0x054C)



DMA2D_IN_COLOR_OUTPUT_SEL_CH0 配置接收通道 0 颜色空间转换第三级的输出。

0：RGB888 转 RGB565

1：直接输出

(R/W)

DMA2D_IN_COLOR_3B_PROC_EN_CH0 配置是否开启接收通道 0 颜色空间转换的第二级。

0：关闭

1：开启

(R/W)

DMA2D_IN_COLOR_INPUT_SEL_CH0 配置接收通道 0 颜色空间转换第一级的输出。

0：YUV422/420 转 YUV444

1：YUV422

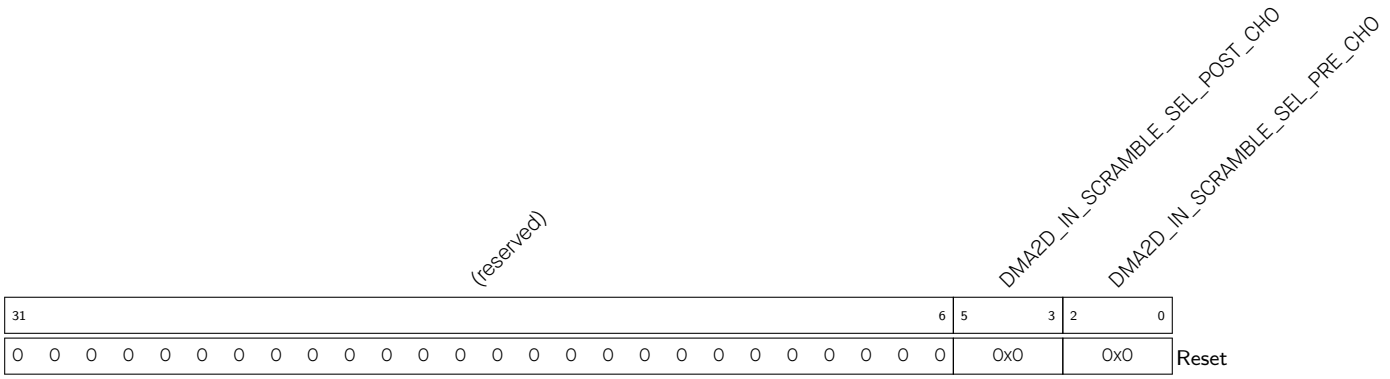
2：YUV444/420

7：关闭颜色空间转换

其他值：无效

(R/W)

Register 4.24. DMA2D_IN_SCRAMBLE_CH0_REG (0x0550)



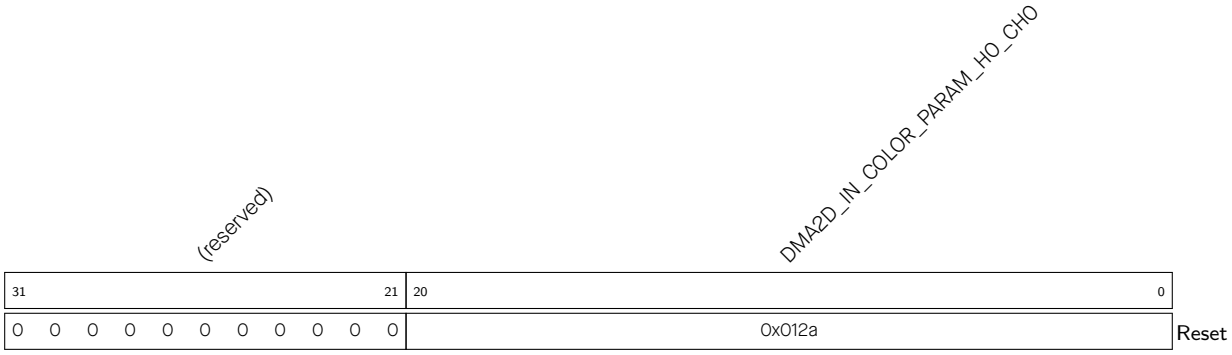
DMA2D_IN_SCRAMBLE_SEL_PRE_CH0 配置接收通道 0 颜色空间转换前的字节乱序处理。

0：BYTE2-1-0
1：BYTE2-0-1
2：BYTE1-0-2
3：BYTE1-2-0
4：BYTE0-2-1
5：BYTE0-1-2
其他值：无效
(R/W)

DMA2D_IN_SCRAMBLE_SEL_POST_CH0 配置接收通道 0 颜色空间转换后的字节乱序处理。

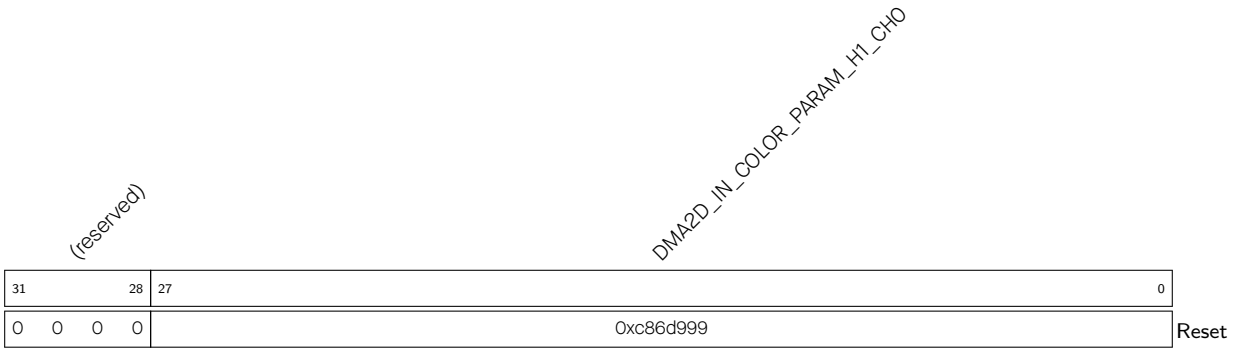
0：BYTE2-1-0
1：BYTE2-0-1
2：BYTE1-0-2
3：BYTE1-2-0
4：BYTE0-2-1
5：BYTE0-1-2
其他值：无效
(R/W)

Register 4.25. DMA2D_IN_COLOR_PARAM0_CH0_REG (0x0554)



DMA2D_IN_COLOR_PARAM_H0_CH0 配置接收通道 0 颜色空间转换第二级输入的最高有效字节的系数 A 和 B。(R/W)

Register 4.26. DMA2D_IN_COLOR_PARAM1_CH0_REG (0x0558)



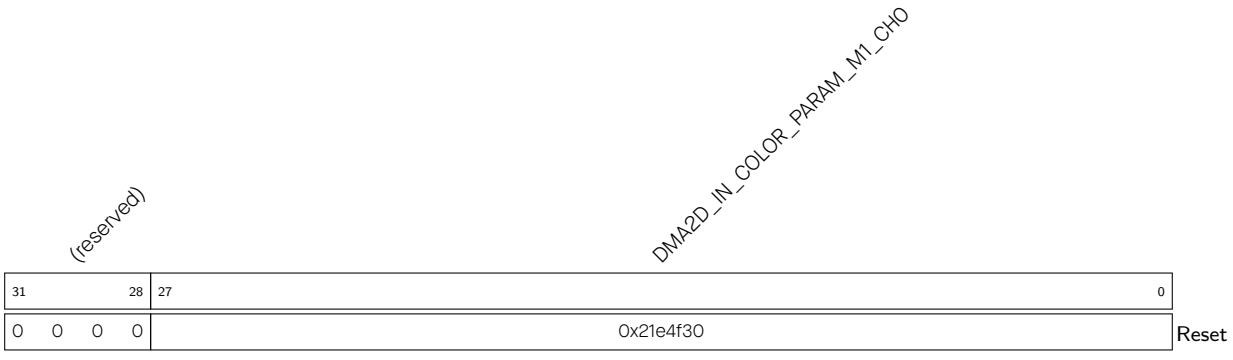
DMA2D_IN_COLOR_PARAM_H1_CH0 配置接收通道 0 颜色空间转换第二级输入的最高有效字节的系数 C 和 D。(R/W)

Register 4.27. DMA2D_IN_COLOR_PARAM2_CH0_REG (0x055C)



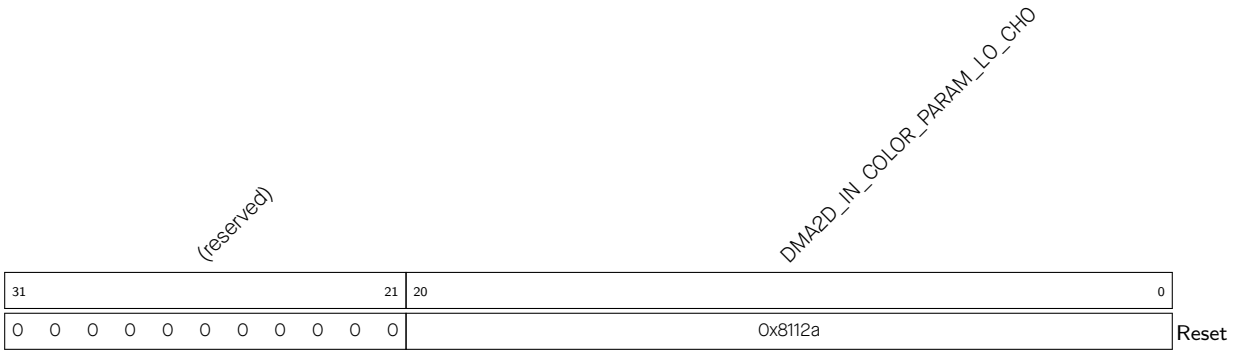
DMA2D_IN_COLOR_PARAM_M0_CH0 配置接收通道 0 颜色空间转换第二级输入的中间字节的系数 A 和 B。(R/W)

Register 4.28. DMA2D_IN_COLOR_PARAM3_CH0_REG (0x0560)



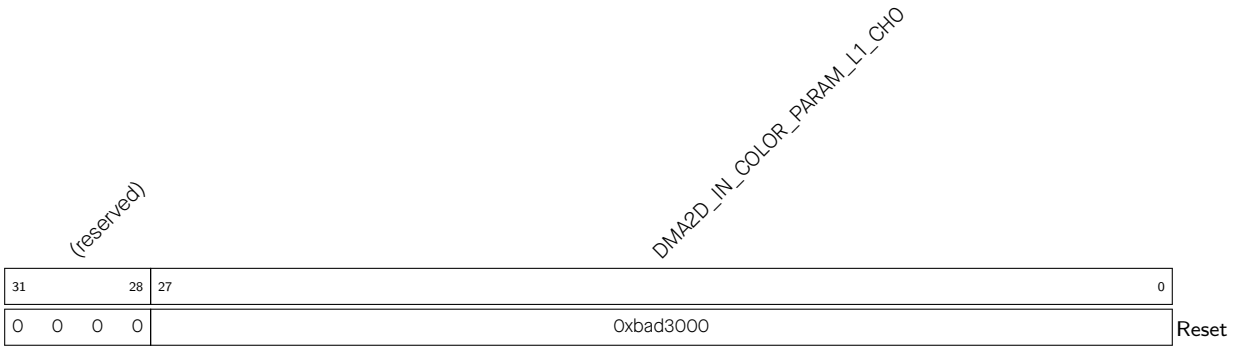
DMA2D_IN_COLOR_PARAM_M1_CH0 配置接收通道 0 颜色空间转换第二级输入的中间字节的系数 C 和 D。(R/W)

Register 4.29. DMA2D_IN_COLOR_PARAM4_CH0_REG (0x0564)



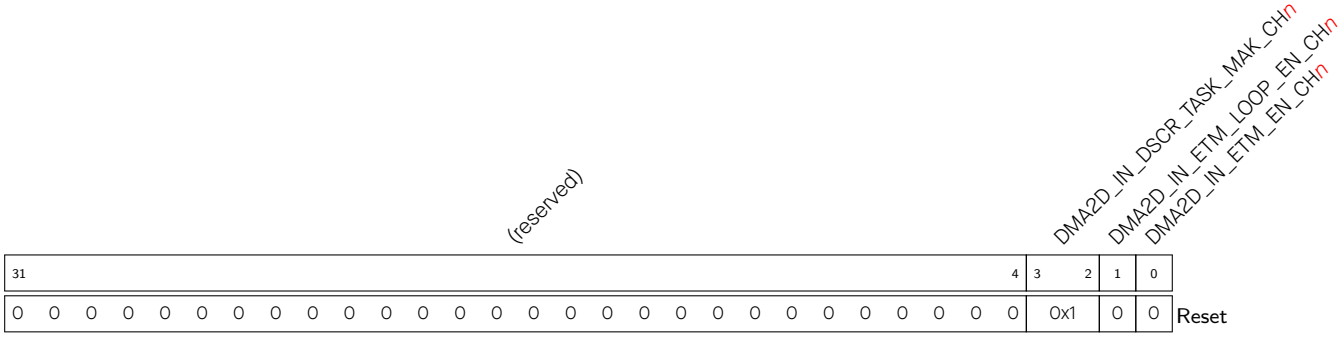
DMA2D_IN_COLOR_PARAM_L0_CH0 配置接收通道 0 颜色空间转换第二级输入的最低有效字节的系数 A 和 B。(R/W)

Register 4.30. DMA2D_IN_COLOR_PARAM5_CH0_REG (0x0568)



DMA2D_IN_COLOR_PARAM_L1_CH0 配置接收通道 0 颜色空间转换第二级输入的最低有效字节的系数 C 和 D。(R/W)

Register 4.31. DMA2D_IN_ETM_CONF_CH*n*_REG (*n*: 0-2) (0x056C+0x100**n*)



DMA2D_IN_ETM_EN_CH*n* 配置接收通道 *n* 是否开启 ETM 功能。

0：关闭

1：开启

(R/W)

DMA2D_IN_ETM_LOOP_EN_CH*n* 配置接收通道 *n* 是否使用 ETM 任务指示下一个描述符的处理。

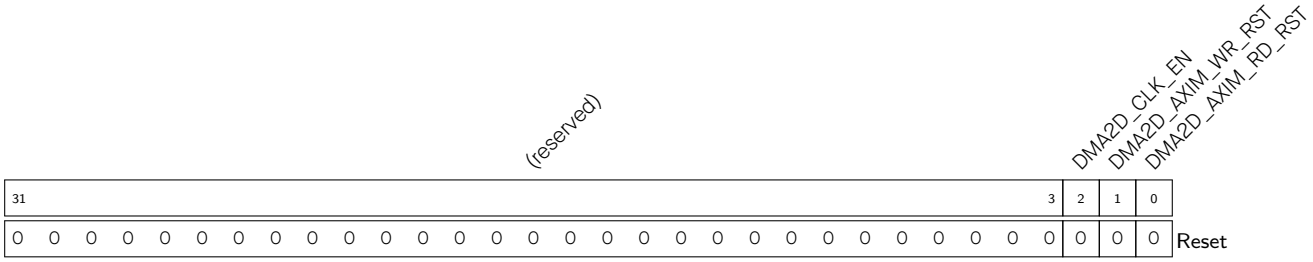
0：不使用 ETM 任务

1：使用 ETM 任务

(R/W)

DMA2D_IN_DSCR_TASK_MAK_CH*n* 配置接收通道 *n* 可缓存任务的最大值。(R/W)

Register 4.32. DMA2D_RST_CONF_REG (0x0A04)



DMA2D_AXIM_RD_RST 配置是否复位 AXI 主机读数据 FIFO。

0：无效值

1：复位

(R/W)

DMA2D_AXIM_WR_RST 配置是否复位 AXI 主机写数据 FIFO。

0：无效值

1：复位

(R/W)

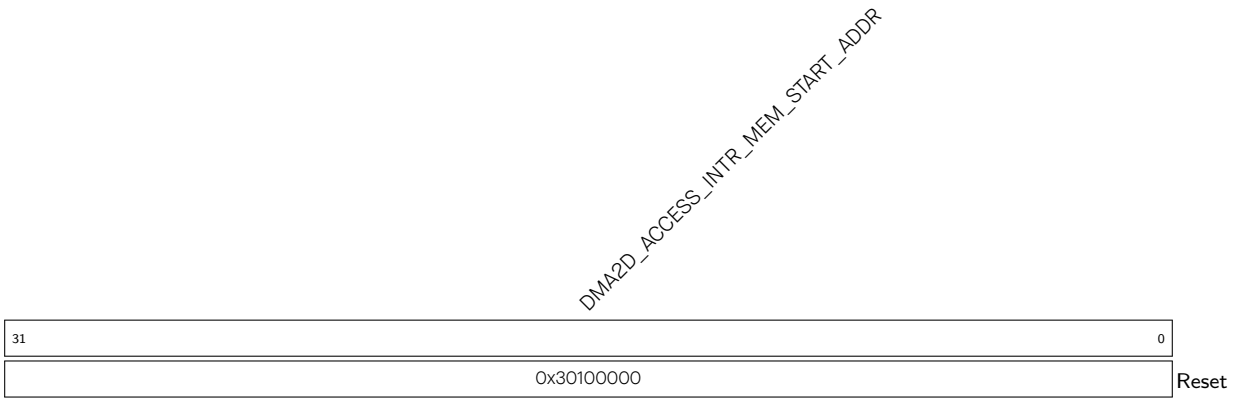
DMA2D_CLK_EN 配置寄存器时钟门控。

0：仅在应用程序写入寄存器时支持时钟。

1：始终强制打开寄存器的时钟。

(R/W)

Register 4.33. DMA2D_INTR_MEM_START_ADDR_REG (0x0A08)



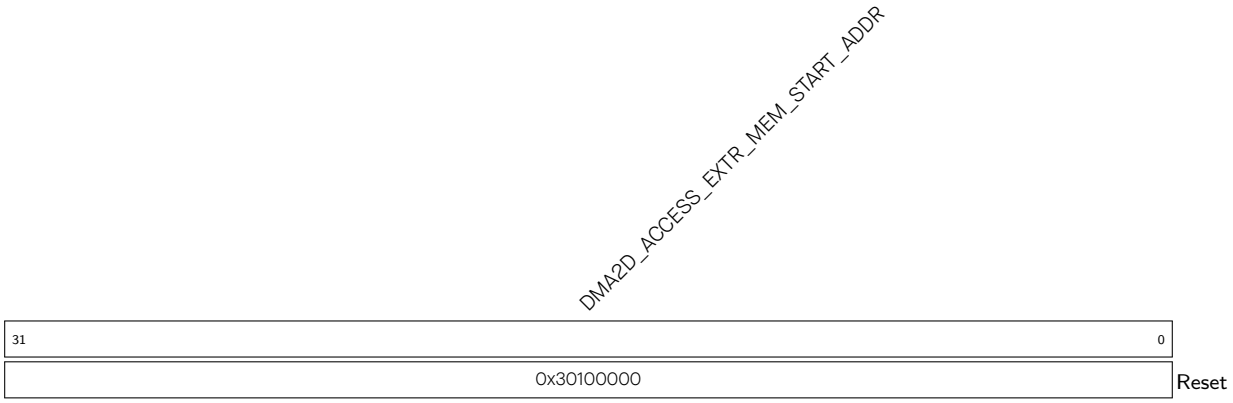
DMA2D_ACCESS_INTR_MEM_START_ADDR 配置可访问内部存储器地址空间的起始地址。(R/W)

Register 4.34. DMA2D_INTR_MEM_END_ADDR_REG (0x0A0C)



DMA2D_ACCESS_INTR_MEM_END_ADDR 配置可访问内部存储器地址空间的结束地址。访问超过区间的地址会造成描述符错误。(R/W)

Register 4.35. DMA2D_EXTR_MEM_START_ADDR_REG (0x0A10)



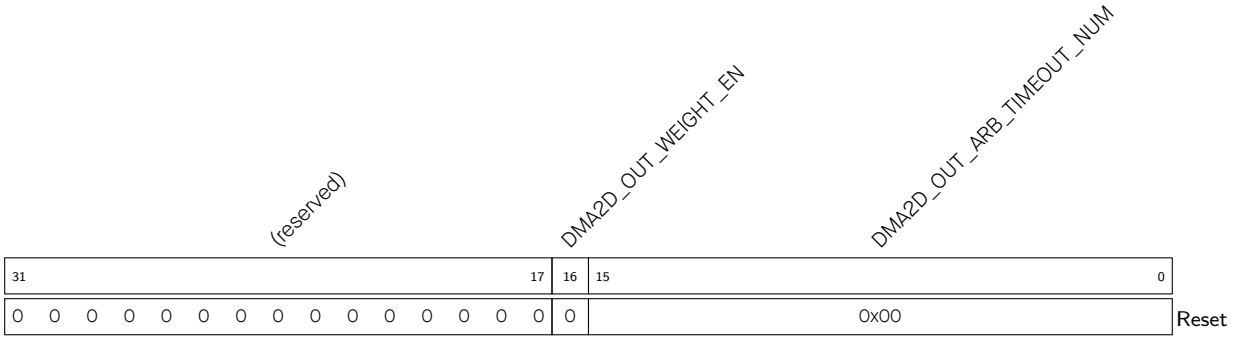
DMA2D_ACCESS_EXTR_MEM_START_ADDR 配置可访问外部存储器地址空间的起始地址。(R/W)

Register 4.36. DMA2D_EXTR_MEM_END_ADDR_REG (0x0A14)



DMA2D_ACCESS_EXTR_MEM_END_ADDR 配置可访问外部存储器地址空间的结束地址。访问超过区间的地址会造成描述符错误。(R/W)

Register 4.37. DMA2D_OUT_ARB_CONFIG_REG (0x0A18)



DMA2D_OUT_ARB_TIMEOUT_NUM 配置 TX 方向时间片。单位：AXI 总线时钟周期。(R/W)

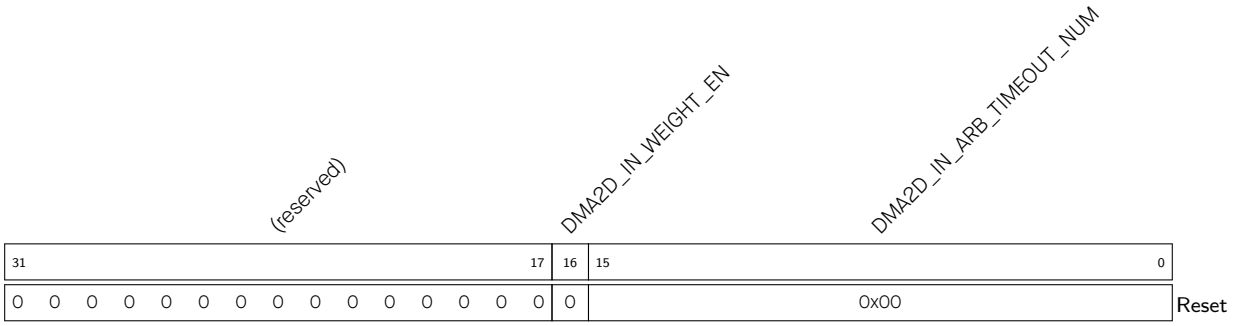
DMA2D_OUT_WEIGHT_EN 配置开启 TX 方向权重仲裁。

0：关闭

1：开启

(R/W)

Register 4.38. DMA2D_IN_ARB_CONFIG_REG (0x0A1C)



DMA2D_IN_ARB_TIMEOUT_NUM 配置 RX 方向时间片。单位：AXI 总线时钟周期。(R/W)

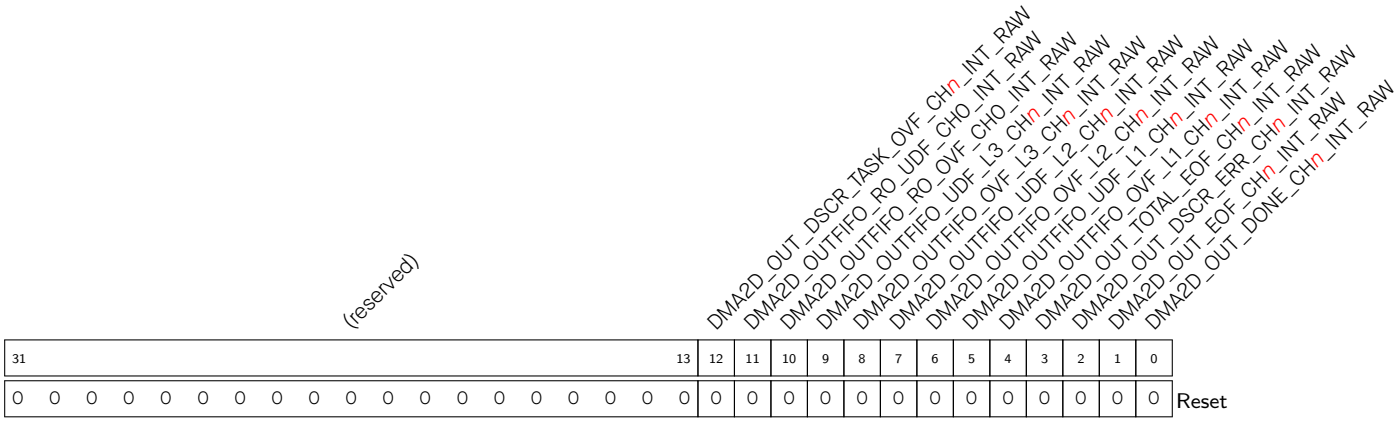
DMA2D_IN_WEIGHT_EN 配置开启 RX 方向权重仲裁。

0：关闭

1：开启

(R/W)

Register 4.39. DMA2D_OUT_INT_RAW_CH*n*_REG (*n*: 0-3) (0x0004+0x100*n*)



DMA2D_OUT_DONE_CH*n*_INT_RAW DMA2D_OUT_DONE_CH*n*_INT 的原始中断状态。(R/ WTC/SS)

DMA2D_OUT_EOF_CH*n*_INT_RAW DMA2D_OUT_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_OUT_DSCR_ERR_CH*n*_INT_RAW DMA2D_OUT_DSCR_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_OUT_TOTAL_EOF_CH*n*_INT_RAW DMA2D_OUT_TOTAL_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_OUTFIFO_OVF_L1_CH*n*_INT_RAW DMA2D_OUTFIFO_OVF_L1_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_OUTFIFO_UDF_L1_CH*n*_INT_RAW DMA2D_OUTFIFO_UDF_L1_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_OUTFIFO_OVF_L2_CH*n*_INT_RAW DMA2D_OUTFIFO_OVF_L2_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_OUTFIFO_UDF_L2_CH*n*_INT_RAW DMA2D_OUTFIFO_UDF_L2_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_OUTFIFO_OVF_L3_CH*n*_INT_RAW DMA2D_OUTFIFO_OVF_L3_CH*n*_INT 的原始中断状态。(R/WTC/SS)

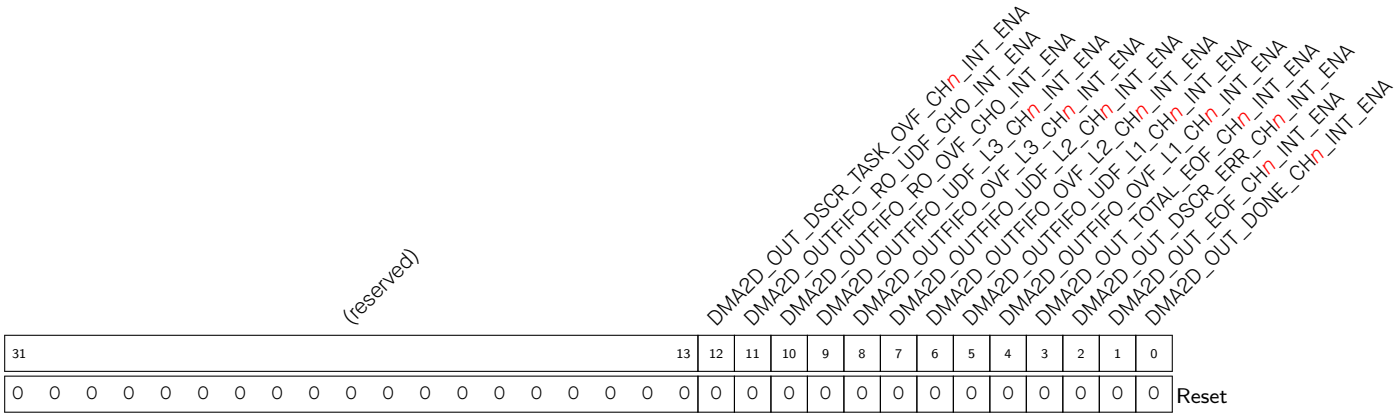
DMA2D_OUTFIFO_UDF_L3_CH*n*_INT_RAW DMA2D_OUTFIFO_UDF_L3_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_OUTFIFO_RO_OVF_CHO_INT_RAW DMA2D_OUTFIFO_RO_OVF_CHO_INT 的原始中断状态。(R/WTC/SS)

DMA2D_OUTFIFO_RO_UDF_CHO_INT_RAW DMA2D_OUTFIFO_RO_UDF_CHO_INT 的原始中断状态。(R/WTC/SS)

DMA2D_OUT_DSCR_TASK_OVF_CH*n*_INT_RAW DMA2D_OUT_DSCR_TASK_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

Register 4.40. DMA2D_OUT_INT_ENA_CH*n*_REG (*n*: 0-3) (0x0008+0x100*n)



DMA2D_OUT_DONE_CH*n*_INT_ENA 写 1 使能 DMA2D_OUT_DONE_CH*n*_INT。(R/W)

DMA2D_OUT_EOF_CH*n*_INT_ENA 写 1 使能 DMA2D_OUT_EOF_CH*n*_INT。(R/W)

DMA2D_OUT_DSCR_ERR_CH*n*_INT_ENA 写 1 使能 DMA2D_OUT_DSCR_ERR_CH*n*_INT。(R/W)

DMA2D_OUT_TOTAL_EOF_CH*n*_INT_ENA 写 1 使能 DMA2D_OUT_TOTAL_EOF_CH*n*_INT。(R/W)

DMA2D_OUTFIFO_OVF_L1_CH*n*_INT_ENA 写 1 使能 DMA2D_OUTFIFO_OVF_L1_CH*n*_INT。(R/W)

DMA2D_OUTFIFO_UDF_L1_CH*n*_INT_ENA 写 1 使能 DMA2D_OUTFIFO_UDF_L1_CH*n*_INT。(R/W)

DMA2D_OUTFIFO_OVF_L2_CH*n*_INT_ENA 写 1 使能 DMA2D_OUTFIFO_OVF_L2_CH*n*_INT。(R/W)

DMA2D_OUTFIFO_UDF_L2_CH*n*_INT_ENA 写 1 使能 DMA2D_OUTFIFO_UDF_L2_CH*n*_INT。(R/W)

DMA2D_OUTFIFO_OVF_L3_CH*n*_INT_ENA 写 1 使能 DMA2D_OUTFIFO_OVF_L3_CH*n*_INT。(R/W)

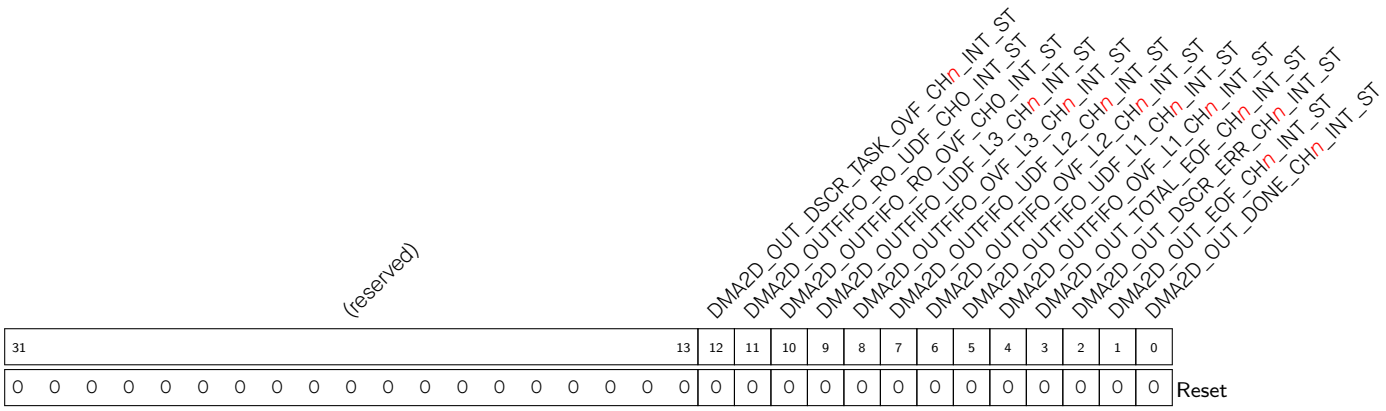
DMA2D_OUTFIFO_UDF_L3_CH*n*_INT_ENA 写 1 使能 DMA2D_OUTFIFO_UDF_L3_CH*n*_INT。(R/W)

DMA2D_OUTFIFO_RO_OVF_CH0_INT_ENA 写 1 使能 DMA2D_OUTFIFO_RO_OVF_CH0_INT。(R/W)

DMA2D_OUTFIFO_RO_UDF_CH0_INT_ENA 写 1 使能 DMA2D_OUTFIFO_RO_UDF_CH0_INT。(R/W)

DMA2D_OUT_DSCR_TASK_OVF_CH*n*_INT_ENA 写 1 使能 DMA2D_OUT_DSCR_TASK_OVF_CH*n*_INT。(R/W)

Register 4.41. DMA2D_OUT_INT_ST_CH*n*_REG (*n*: 0-3) (0x000C+0x100**n*)



DMA2D_OUT_DONE_CH*n*_INT_ST DMA2D_OUT_DONE_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_OUT_EOF_CH*n*_INT_ST DMA2D_OUT_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_OUT_DSCR_ERR_CH*n*_INT_ST DMA2D_OUT_DSCR_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_OUT_TOTAL_EOF_CH*n*_INT_ST DMA2D_OUT_TOTAL_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_OUTFIFO_OVF_L1_CH*n*_INT_ST DMA2D_OUTFIFO_OVF_L1_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_OUTFIFO_UDF_L1_CH*n*_INT_ST DMA2D_OUTFIFO_UDF_L1_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_OUTFIFO_OVF_L2_CH*n*_INT_ST DMA2D_OUTFIFO_OVF_L2_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_OUTFIFO_UDF_L2_CH*n*_INT_ST DMA2D_OUTFIFO_UDF_L2_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_OUTFIFO_OVF_L3_CH*n*_INT_ST DMA2D_OUTFIFO_OVF_L3_CH*n*_INT 的屏蔽中断状态。(RO)

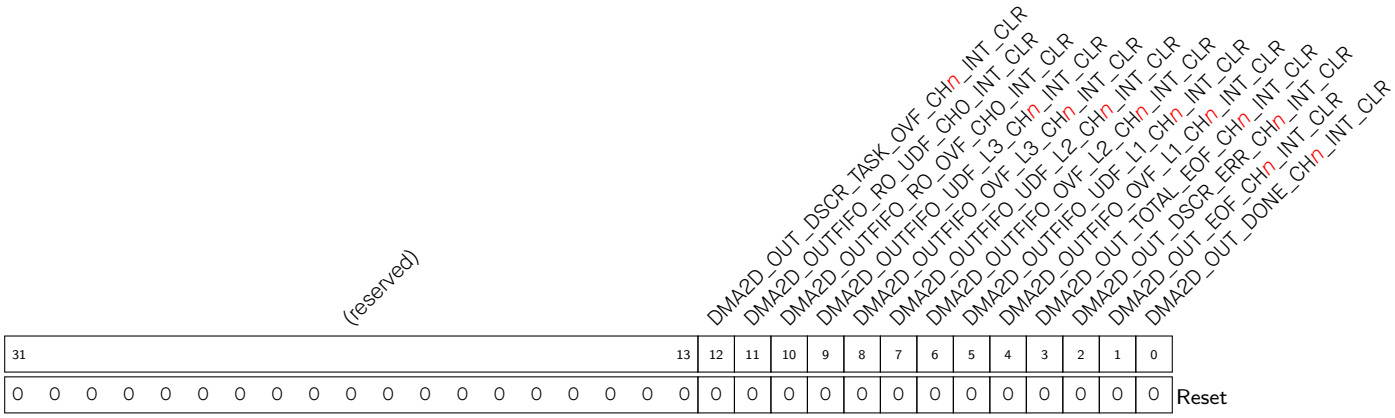
DMA2D_OUTFIFO_UDF_L3_CH*n*_INT_ST DMA2D_OUTFIFO_UDF_L3_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_OUTFIFO_RO_OVF_CH0_INT_ST DMA2D_OUTFIFO_RO_OVF_CH0_INT 的屏蔽中断状态。(RO)

DMA2D_OUTFIFO_RO_UDF_CH0_INT_ST DMA2D_OUTFIFO_RO_UDF_CH0_INT 的屏蔽中断状态。(RO)

DMA2D_OUT_DSCR_TASK_OVF_CH*n*_INT_ST DMA2D_OUT_DSCR_TASK_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

Register 4.42. DMA2D_OUT_INT_CLR_CH*n*_REG (*n*: 0-3) (0x0010+0x100***n*)



DMA2D_OUT_DONE_CH*n*_INT_CLR 写 1 清除 DMA2D_OUT_DONE_CH*n*_INT。(WT)

DMA2D_OUT_EOF_CH*n*_INT_CLR 写 1 清除 DMA2D_OUT_EOF_CH*n*_INT。(WT)

DMA2D_OUT_DSCR_ERR_CH*n*_INT_CLR 写 1 清除 DMA2D_OUT_DSCR_ERR_CH*n*_INT。(WT)

DMA2D_OUT_TOTAL_EOF_CH*n*_INT_CLR 写 1 清除 DMA2D_OUT_TOTAL_EOF_CH*n*_INT。(WT)

DMA2D_OUTFIFO_OVF_L1_CH*n*_INT_CLR 写 1 清除 DMA2D_OUTFIFO_OVF_L1_CH*n*_INT。(WT)

DMA2D_OUTFIFO_UDF_L1_CH*n*_INT_CLR 写 1 清除 DMA2D_OUTFIFO_UDF_L1_CH*n*_INT。(WT)

DMA2D_OUTFIFO_OVF_L2_CH*n*_INT_CLR 写 1 清除 DMA2D_OUTFIFO_OVF_L2_CH*n*_INT。(WT)

DMA2D_OUTFIFO_UDF_L2_CH*n*_INT_CLR 写 1 清除 DMA2D_OUTFIFO_UDF_L2_CH*n*_INT。(WT)

DMA2D_OUTFIFO_OVF_L3_CH*n*_INT_CLR 写 1 清除 DMA2D_OUTFIFO_OVF_L3_CH*n*_INT。(WT)

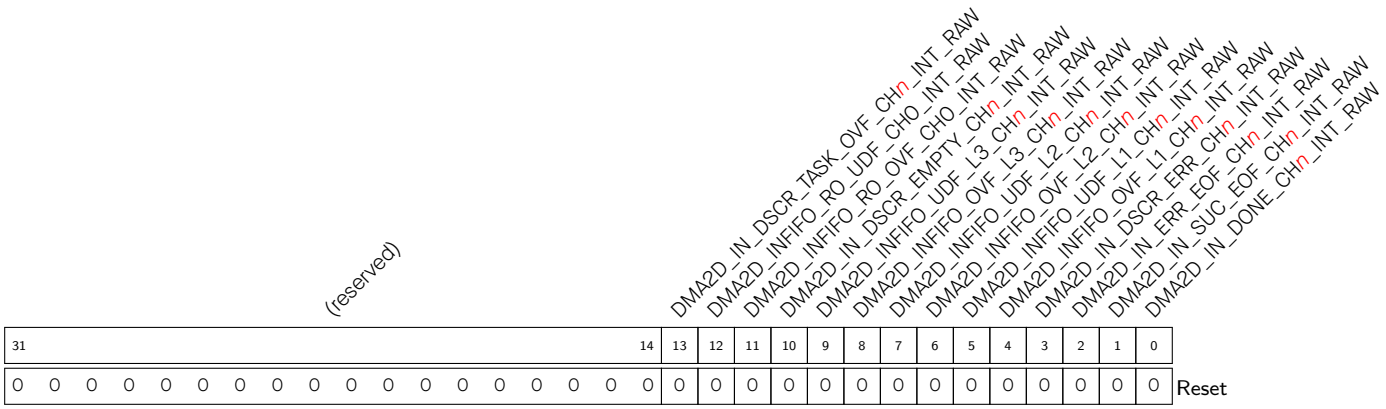
DMA2D_OUTFIFO_UDF_L3_CH*n*_INT_CLR 写 1 清除 DMA2D_OUTFIFO_UDF_L3_CH*n*_INT。(WT)

DMA2D_OUTFIFO_RO_OVF_CH0_INT_CLR 写 1 清除 DMA2D_OUTFIFO_RO_OVF_CH0_INT。(WT)

DMA2D_OUTFIFO_RO_UDF_CH0_INT_CLR 写 1 清除 DMA2D_OUTFIFO_RO_UDF_CH0_INT。(WT)

DMA2D_OUT_DSCR_TASK_OVF_CH*n*_INT_CLR 写 1 清除 DMA2D_OUT_DSCR_TASK_OVF_CH*n*_INT。(WT)

Register 4.43. DMA2D_IN_INT_RAW_CH*n*_REG (*n*: 0-2) (0x0504+0x100**n*)



DMA2D_IN_DONE_CH*n*_INT_RAW DMA2D_IN_DONE_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_IN_SUC_EOF_CH*n*_INT_RAW DMA2D_IN_SUC_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_IN_ERR_EOF_CH*n*_INT_RAW DMA2D_IN_ERR_EOF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_IN_DSCR_ERR_CH*n*_INT_RAW DMA2D_IN_DSCR_ERR_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_INFIFO_OVF_L1_CH*n*_INT_RAW DMA2D_INFIFO_OVF_L1_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_INFIFO_UDF_L1_CH*n*_INT_RAW DMA2D_INFIFO_UDF_L1_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_INFIFO_OVF_L2_CH*n*_INT_RAW DMA2D_INFIFO_OVF_L2_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_INFIFO_UDF_L2_CH*n*_INT_RAW DMA2D_INFIFO_UDF_L2_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_INFIFO_OVF_L3_CH*n*_INT_RAW DMA2D_INFIFO_OVF_L3_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_INFIFO_UDF_L3_CH*n*_INT_RAW DMA2D_INFIFO_UDF_L3_CH*n*_INT 的原始中断状态。(R/WTC/SS)

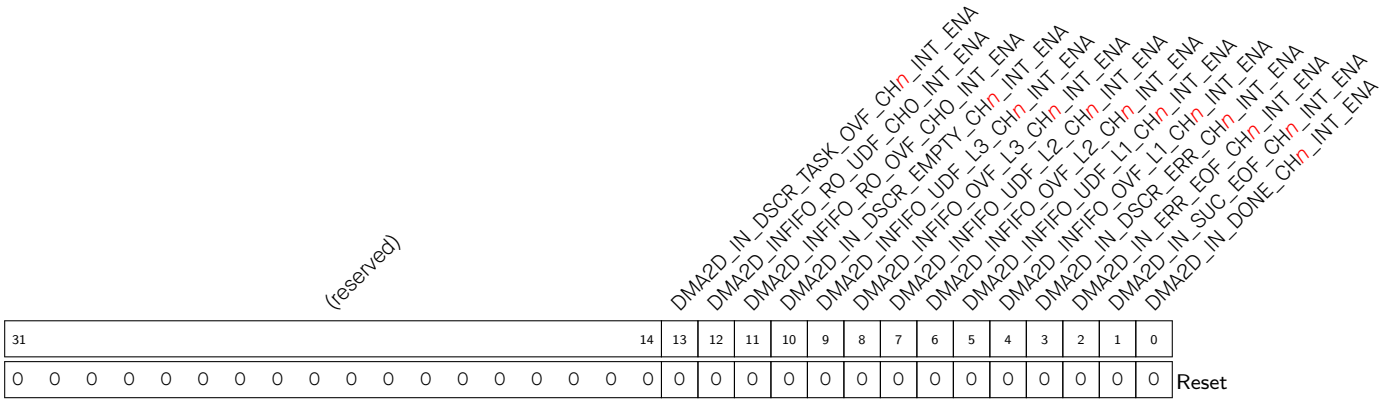
DMA2D_IN_DSCR_EMPTY_CH*n*_INT_RAW DMA2D_IN_DSCR_EMPTY_CH*n*_INT 的原始中断状态。(R/WTC/SS)

DMA2D_INFIFO_RO_OVF_CHO_INT_RAW DMA2D_INFIFO_RO_OVF_CHO_INT 的原始中断状态。(R/WTC/SS)

DMA2D_INFIFO_RO_UDF_CHO_INT_RAW DMA2D_INFIFO_RO_UDF_CHO_INT 的原始中断状态。(R/WTC/SS)

DMA2D_IN_DSCR_TASK_OVF_CH*n*_INT_RAW DMA2D_IN_DSCR_TASK_OVF_CH*n*_INT 的原始中断状态。(R/WTC/SS)

Register 4.44. DMA2D_IN_INT_ENA_CH*n*_REG (*n*: 0-2) (0x0508+0x100**n*)



DMA2D_IN_DONE_CH*n*_INT_ENA 写 1 使能 DMA2D_IN_DONE_CH*n*_INT。(R/W)

DMA2D_IN_SUC_EOF_CH*n*_INT_ENA 写 1 使能 DMA2D_IN_SUC_EOF_CH*n*_INT。(R/W)

DMA2D_IN_ERR_EOF_CH*n*_INT_ENA 写 1 使能 DMA2D_IN_ERR_EOF_CH*n*_INT。(R/W)

DMA2D_IN_DSCR_ERR_CH*n*_INT_ENA 写 1 使能 DMA2D_IN_DSCR_ERR_CH*n*_INT。(R/W)

DMA2D_INFIFO_OVF_L1_CH*n*_INT_ENA 写 1 使能 DMA2D_INFIFO_OVF_L1_CH*n*_INT。(R/W)

DMA2D_INFIFO_UDF_L1_CH*n*_INT_ENA 写 1 使能 DMA2D_INFIFO_UDF_L1_CH*n*_INT。(R/W)

DMA2D_INFIFO_OVF_L2_CH*n*_INT_ENA 写 1 使能 DMA2D_INFIFO_OVF_L2_CH*n*_INT。(R/W)

DMA2D_INFIFO_UDF_L2_CH*n*_INT_ENA 写 1 使能 DMA2D_INFIFO_UDF_L2_CH*n*_INT。(R/W)

DMA2D_INFIFO_OVF_L3_CH*n*_INT_ENA 写 1 使能 DMA2D_INFIFO_OVF_L3_CH*n*_INT。(R/W)

DMA2D_INFIFO_UDF_L3_CH*n*_INT_ENA 写 1 使能 DMA2D_INFIFO_UDF_L3_CH*n*_INT。(R/W)

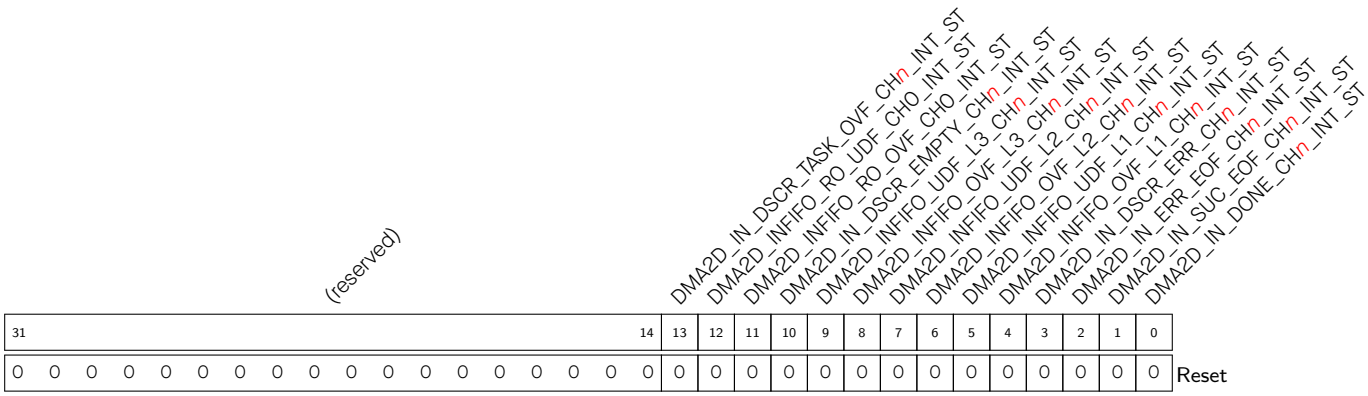
DMA2D_IN_DSCR_EMPTY_CH*n*_INT_ENA 写 1 使能 DMA2D_IN_DSCR_EMPTY_CH*n*_INT。(R/W)

DMA2D_INFIFO_RO_OVF_CHO_INT_ENA 写 1 使能 DMA2D_INFIFO_RO_OVF_CHO_INT。(R/W)

DMA2D_INFIFO_RO_UDF_CHO_INT_ENA 写 1 使能 DMA2D_INFIFO_RO_UDF_CHO_INT。(R/W)

DMA2D_IN_DSCR_TASK_OVF_CH*n*_INT_ENA 写 1 使能 DMA2D_IN_DSCR_TASK_OVF_CH*n*_INT。(R/W)

Register 4.45. DMA2D_IN_INT_ST_CH*n*_REG (*n*: 0-2) (0x050C+0x100**n*)



DMA2D_IN_DONE_CH*n*_INT_ST DMA2D_IN_DONE_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_IN_SUC_EOF_CH*n*_INT_ST DMA2D_IN_SUC_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_IN_ERR_EOF_CH*n*_INT_ST DMA2D_IN_ERR_EOF_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_IN_DSCR_ERR_CH*n*_INT_ST DMA2D_IN_DSCR_ERR_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_INFIFO_OVF_L1_CH*n*_INT_ST the DMA2D_INFIFO_OVF_L1_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_INFIFO_UDF_L1_CH*n*_INT_ST the DMA2D_INFIFO_UDF_L1_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_INFIFO_OVF_L2_CH*n*_INT_ST the DMA2D_INFIFO_OVF_L2_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_INFIFO_UDF_L2_CH*n*_INT_ST the DMA2D_INFIFO_UDF_L2_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_INFIFO_OVF_L3_CH*n*_INT_ST the DMA2D_INFIFO_OVF_L3_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_INFIFO_UDF_L3_CH*n*_INT_ST the DMA2D_INFIFO_UDF_L3_CH*n*_INT 的屏蔽中断状态。(RO)

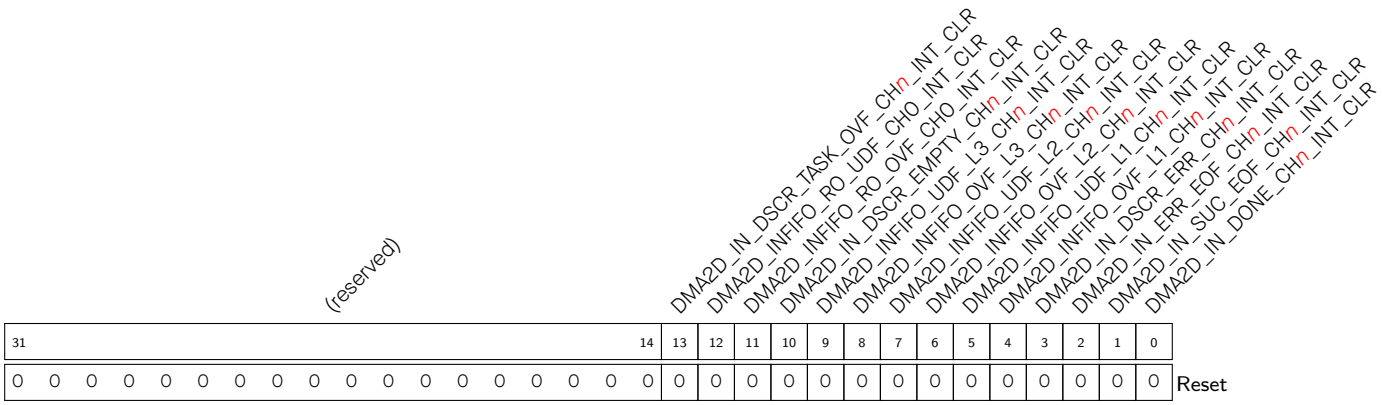
DMA2D_IN_DSCR_EMPTY_CH*n*_INT_ST DMA2D_IN_DSCR_EMPTY_CH*n*_INT 的屏蔽中断状态。(RO)

DMA2D_INFIFO_RO_OVF_CH0_INT_ST the DMA2D_INFIFO_RO_OVF_CH0_INT 的屏蔽中断状态。(RO)

DMA2D_INFIFO_RO_UDF_CH0_INT_ST the DMA2D_INFIFO_RO_UDF_CH0_INT 的屏蔽中断状态。(RO)

DMA2D_IN_DSCR_TASK_OVF_CH*n*_INT_ST the DMA2D_IN_DSCR_TASK_OVF_CH*n*_INT 的屏蔽中断状态。(RO)

Register 4.46. DMA2D_IN_INT_CLR_CH*n*_REG (*n*: 0-2) (0x0510+0x100**n*)



DMA2D_IN_DONE_CH*n*_INT_CLR 写 1 清除 DMA2D_IN_DONE_CH*n*_INT。(WT)

DMA2D_IN_SUC_EOF_CH*n*_INT_CLR 写 1 清除 DMA2D_IN_SUC_EOF_CH*n*_INT。(WT)

DMA2D_IN_ERR_EOF_CH*n*_INT_CLR 写 1 清除 DMA2D_IN_ERR_EOF_CH*n*_INT。(WT)

DMA2D_IN_DSCR_ERR_CH*n*_INT_CLR 写 1 清除 DMA2D_IN_DSCR_ERR_CH*n*_INT。(WT)

DMA2D_INFIFO_OVF_L1_CH*n*_INT_CLR 写 1 清除 DMA2D_INFIFO_OVF_L1_CH*n*_INT。(WT)

DMA2D_INFIFO_UDF_L1_CH*n*_INT_CLR 写 1 清除 DMA2D_INFIFO_UDF_L1_CH*n*_INT。(WT)

DMA2D_INFIFO_OVF_L2_CH*n*_INT_CLR 写 1 清除 DMA2D_INFIFO_OVF_L2_CH*n*_INT。(WT)

DMA2D_INFIFO_UDF_L2_CH*n*_INT_CLR 写 1 清除 DMA2D_INFIFO_UDF_L2_CH*n*_INT。(WT)

DMA2D_INFIFO_OVF_L3_CH*n*_INT_CLR 写 1 清除 DMA2D_INFIFO_OVF_L3_CH*n*_INT。(WT)

DMA2D_INFIFO_UDF_L3_CH*n*_INT_CLR 写 1 清除 DMA2D_INFIFO_UDF_L3_CH*n*_INT。(WT)

DMA2D_IN_DSCR_EMPTY_CH*n*_INT_CLR 写 1 清除 DMA2D_IN_DSCR_EMPTY_CH*n*_INT。(WT)

DMA2D_INFIFO_RO_OVF_CHO_INT_CLR 写 1 清除 DMA2D_INFIFO_RO_OVF_CHO_INT。(WT)

DMA2D_INFIFO_RO_UDF_CHO_INT_CLR 写 1 清除 DMA2D_INFIFO_RO_UDF_CHO_INT。(WT)

DMA2D_IN_DSCR_TASK_OVF_CH*n*_INT_CLR 写 1 清除 DMA2D_IN_DSCR_TASK_OVF_CH*n*_INT。(WT)

Register 4.47. DMA2D_OUTFIFO_STATUS_CH*n*_REG (*n*: 0-3) (0x0014+0x100**n*)



DMA2D_OUTFIFO_FULL_L2_CH*n* 表示发送通道 *n* 的 L2 TX FIFO 是否为满。Represents whether the L2 TX FIFO for TX channel *n* 是否为满。

0：未满

1：已满

(RO)

DMA2D_OUTFIFO_EMPTY_L2_CH*n* 表示发送通道 *n* 的 L2 TX FIFO 是否为空。

0：未空

1：已空

(RO)

DMA2D_OUTFIFO_CNT_L2_CH*n* 表示发送通道 *n* L2 TX FIFO 中的字节数。(RO)

DMA2D_OUT_REMAIN_UNDER_1B_CH*n* 保留。(RO)

DMA2D_OUT_REMAIN_UNDER_2B_CH*n* 保留。(RO)

DMA2D_OUT_REMAIN_UNDER_3B_CH*n* 保留。(RO)

DMA2D_OUT_REMAIN_UNDER_4B_CH*n* 保留。(RO)

DMA2D_OUT_REMAIN_UNDER_5B_CH*n* 保留。(RO)

DMA2D_OUT_REMAIN_UNDER_6B_CH*n* 保留。(RO)

DMA2D_OUT_REMAIN_UNDER_7B_CH*n* 保留。(RO)

DMA2D_OUT_REMAIN_UNDER_8B_CH*n* 保留。(RO)

DMA2D_OUTFIFO_FULL_L1_CH*n* 表示发送通道 *n* 的 L1 TX FIFO 是否为满。

0：未满

1：已满

(RO)

DMA2D_OUTFIFO_EMPTY_L1_CH*n* 表示发送通道 *n* 的 L1 TX FIFO 是否为空。

0：未空

1：已空

(RO)

DMA2D_OUTFIFO_CNT_L1_CH*n* 表示发送通道 *n* L1 TX FIFO 中的字节数。(RO)

见下页...

Register 4.47. DMA2D_OUTFIFO_STATUS_CH*n*_REG (*n*: 0-3) (0x0014+0x100**n*)

接上页...

DMA2D_OUTFIFO_FULL_L3_CH*n* 表示发送通道 *n* 的 L3 TX FIFO 是否为满。

0：未满

1：已满

(RO)

DMA2D_OUTFIFO_EMPTY_L3_CH*n* 表示发送通道 *n* 的 L3 TX FIFO 是否为空。

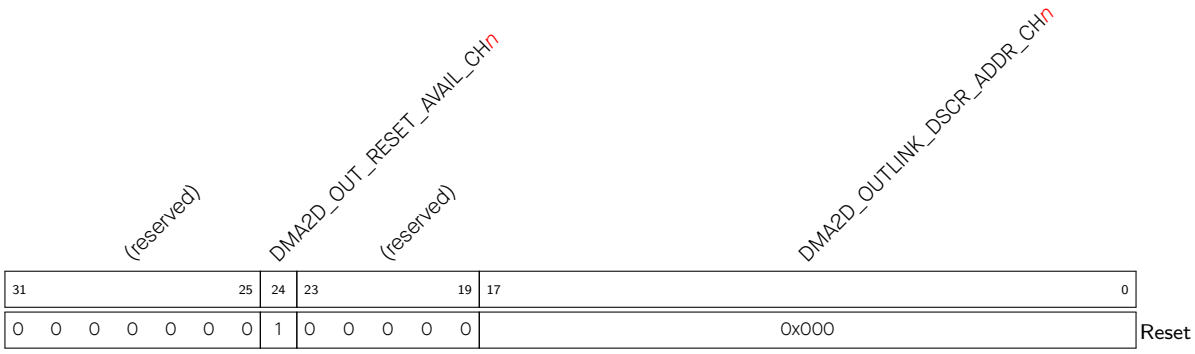
0：未空

1：已空

(RO)

DMA2D_OUTFIFO_CNT_L3_CH*n* 表示发送通道 *n* L3 TX FIFO 中的字节数。(RO)

Register 4.48. DMA2D_OUT_STATE_CH*n*_REG (*n*: 0-3) (0x0024+0x100**n*)



DMA2D_OUTLINK_DSCR_ADDR_CH*n* 表示下一个预读取（但未处理）的发送接收链表描述符所在地址的低 18 位。如果当前处理的发送链表描述符是最后一个描述符，则该字段表示当前处理的发送链表描述符地址。(RO)

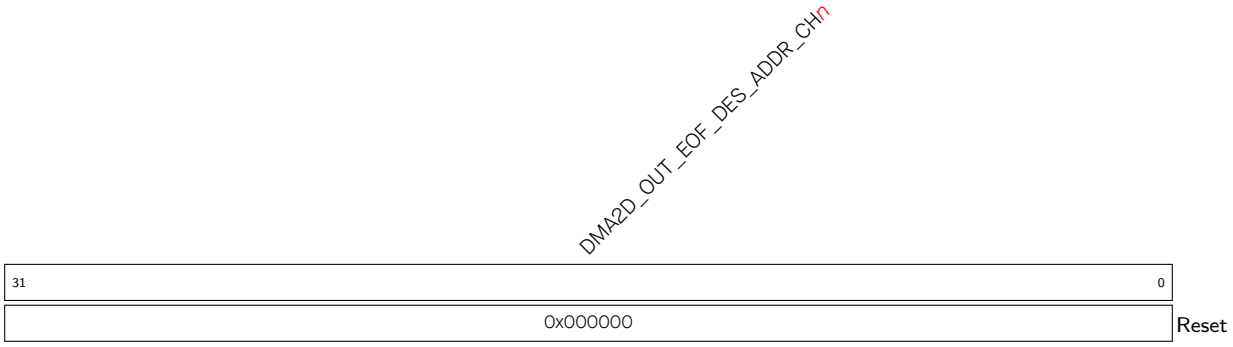
DMA2D_OUT_RESET_AVAIL_CH*n* 表示复位发送通道 *n* 是否安全。

0：不安全

1：安全

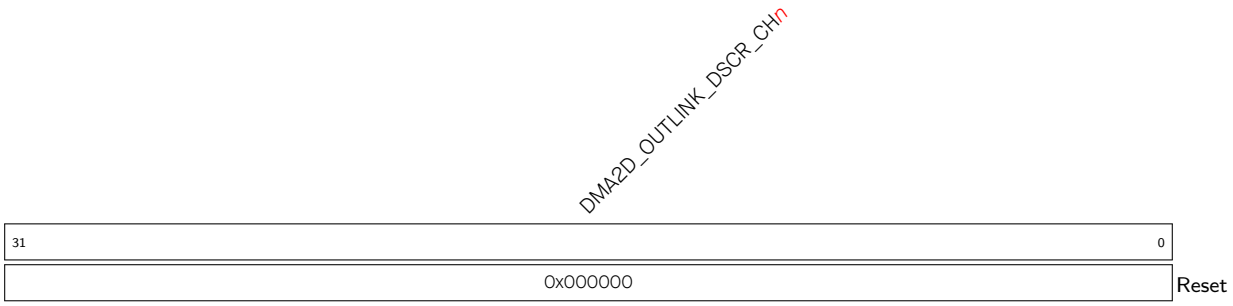
(RO)

Register 4.49. DMA2D_OUT_EOF_DES_ADDR_CH*n*_REG (*n*: 0-3) (0x0028+0x100**n*)



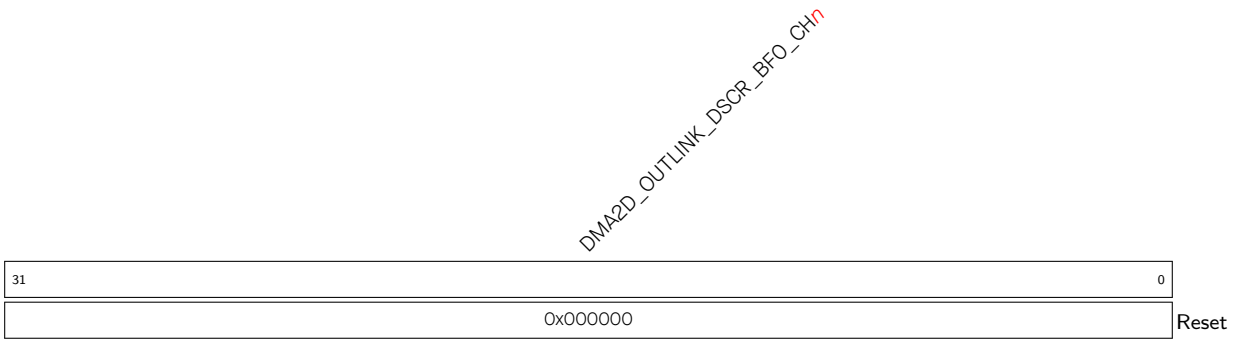
DMA2D_OUT_EOF_DES_ADDR_CH*n* 表示 eof 位为 1 的发送链表描述符的地址。(RO)

Register 4.50. DMA2D_OUT_DSCR_CH*n*_REG (*n*: 0-3) (0x002C+0x100**n*)



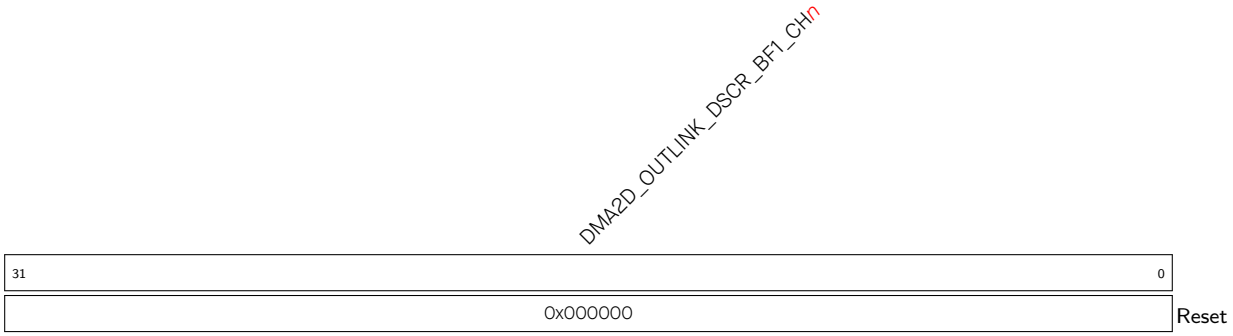
DMA2D_OUTLINK_DSCR_CH*n* 表示当前已预读取的发送链表描述符指向的下一个发送链表描述符地址。(RO)

Register 4.51. DMA2D_OUT_DSCR_BF0_CH*n*_REG (*n*: 0-3) (0x0030+0x100**n*)



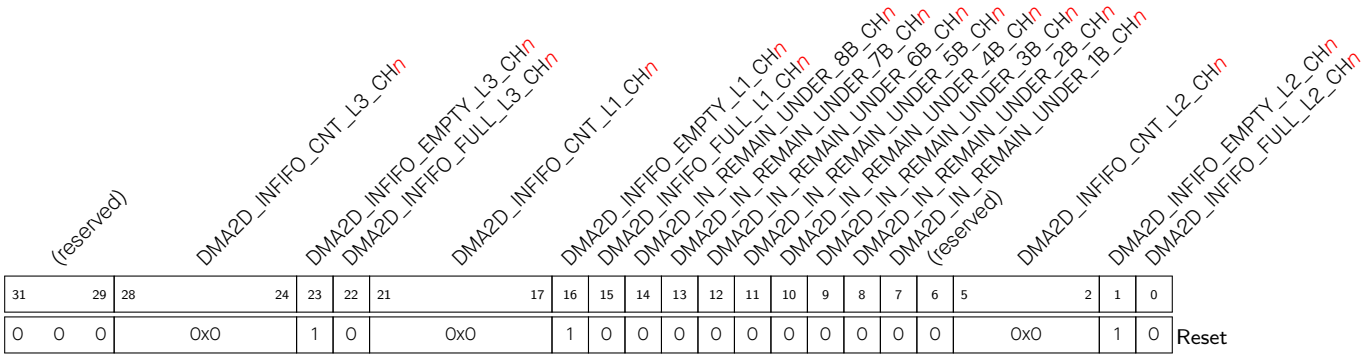
DMA2D_OUTLINK_DSCR_BF0_CH*n* 表示当前已预读取的发送链表描述符所在地址。(RO)

Register 4.52. DMA2D_OUT_DSCR_BF1_CH*n*_REG (*n*: 0-3) (0x0034+0x100**n*)



DMA2D_OUTLINK_DSCR_BF1_CH*n* 表示前一个已预读取的发送链表描述符所在地址。(RO)

Register 4.53. DMA2D_INFIFO_STATUS_CH*n*_REG (*n*: 0-2) (0x0514+0x100**n*)



DMA2D_INFIFO_FULL_L2_CH*n* 表示接收通道 *n* 的 L2 RX FIFO 是否为满。

0：未满

1：已满

(RO)

DMA2D_INFIFO_EMPTY_L2_CH*n* 表示接收通道 *n* 的 L2 RX FIFO 是否为空。

0：未空

1：已空

(RO)

DMA2D_INFIFO_CNT_L2_CH*n* 表示接收通道 *n* L2 RX FIFO 中的字节数。(RO)

DMA2D_IN_REMAIN_UNDER_1B_CH*n* 保留。(RO)

DMA2D_IN_REMAIN_UNDER_2B_CH*n* 保留。(RO)

DMA2D_IN_REMAIN_UNDER_3B_CH*n* 保留。(RO)

DMA2D_IN_REMAIN_UNDER_4B_CH*n* 保留。(RO)

DMA2D_IN_REMAIN_UNDER_5B_CH*n* 保留。(RO)

DMA2D_IN_REMAIN_UNDER_6B_CH*n* 保留。(RO)

DMA2D_IN_REMAIN_UNDER_7B_CH*n* 保留。(RO)

DMA2D_IN_REMAIN_UNDER_8B_CH*n* 保留。(RO)

DMA2D_INFIFO_FULL_L1_CH*n* 表示接收通道 *n* 的 L1 RX FIFO 是否为满。

0：未满

1：已满

(RO)

DMA2D_INFIFO_EMPTY_L1_CH*n* 表示接收通道 *n* 的 L1 RX FIFO 是否为空。

0：未空

1：已空

(RO)

DMA2D_INFIFO_CNT_L1_CH*n* 表示接收通道 *n* L1 RX FIFO 中的字节数。(RO)

见下页...

Register 4.53. DMA2D_INFIFO_STATUS_CH*n*_REG (*n*: 0-2) (0x0514+0x100**n*)

接上页...

DMA2D_INFIFO_FULL_L3_CH*n* 表示接收通道 *n* 的 L3 RX FIFO 是否为满。

0：未满

1：已满

(RO)

DMA2D_INFIFO_EMPTY_L3_CH*n* 表示接收通道 *n* 的 L3 RX FIFO 是否为空。

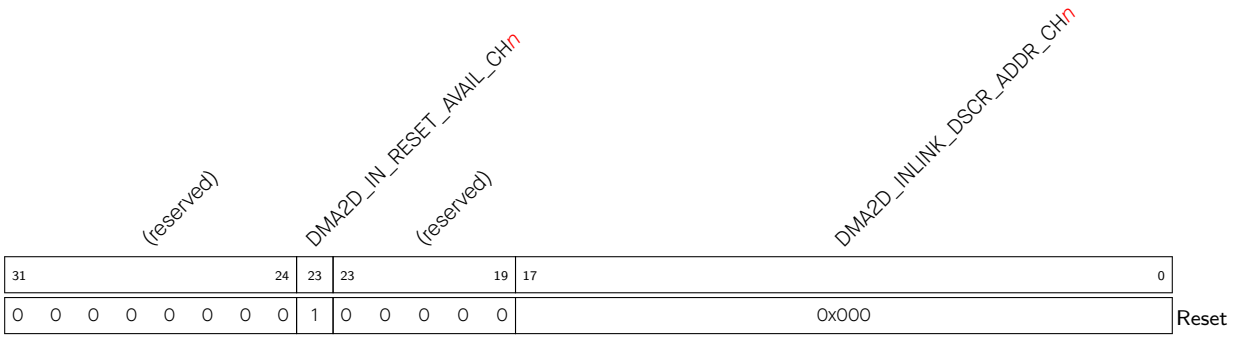
0：未空

1：已空

(RO)

DMA2D_INFIFO_CNT_L3_CH*n* 表示接收通道 *n* L3 RX FIFO 中的字节数。(RO)

Register 4.54. DMA2D_IN_STATE_CH*n*_REG (*n*: 0-2) (0x0524+0x100**n*)



DMA2D_INLINK_DSCR_ADDR_CH*n* 表示下一个预读取（但未处理）的接收链表描述符所在地址的低 18 位。如果当前处理的接收链表描述符是最后一个描述符，则该字段表示当前处理的接收链表描述符地址。(RO)

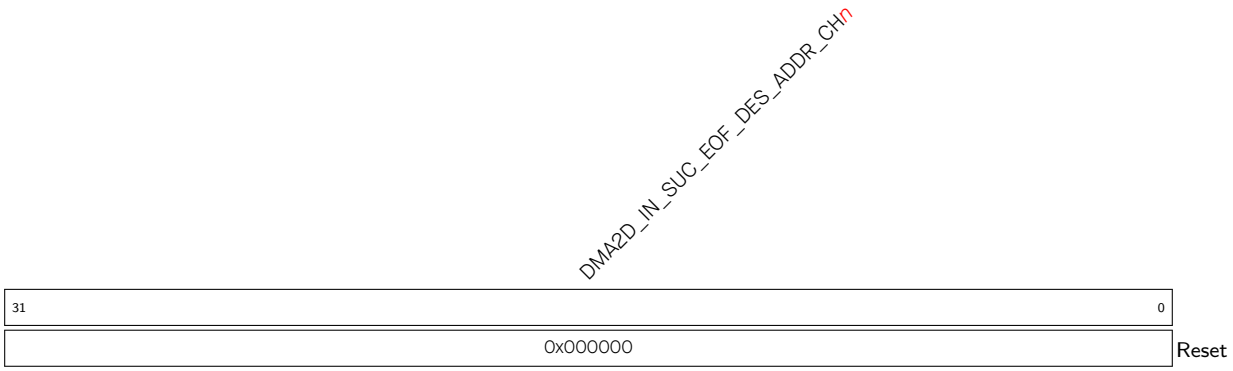
DMA2D_IN_RESET_AVAIL_CH*n* 表示复位接收通道 *n* 是否安全。

0：不安全

1：安全

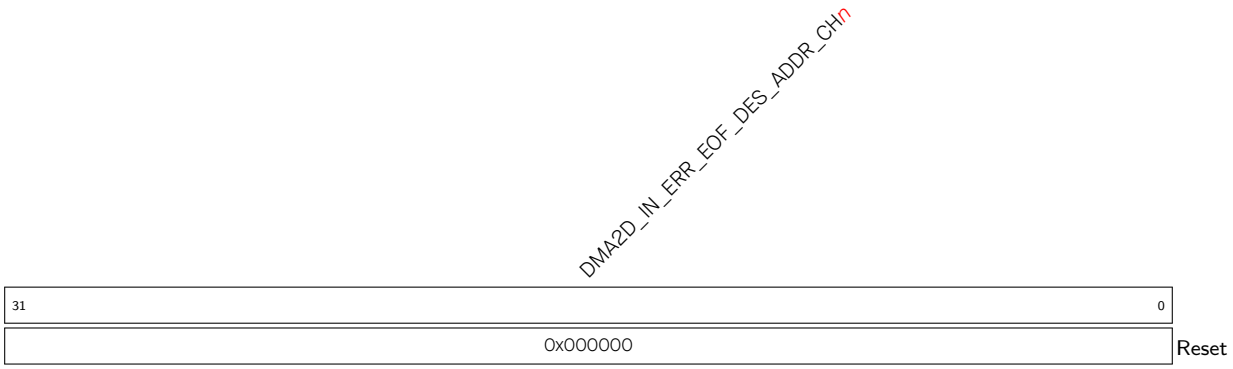
(RO)

Register 4.55. DMA2D_IN_SUC_EOF_DES_ADDR_CH*n*_REG (*n*: 0-2) (0x0528+0x100**n*)



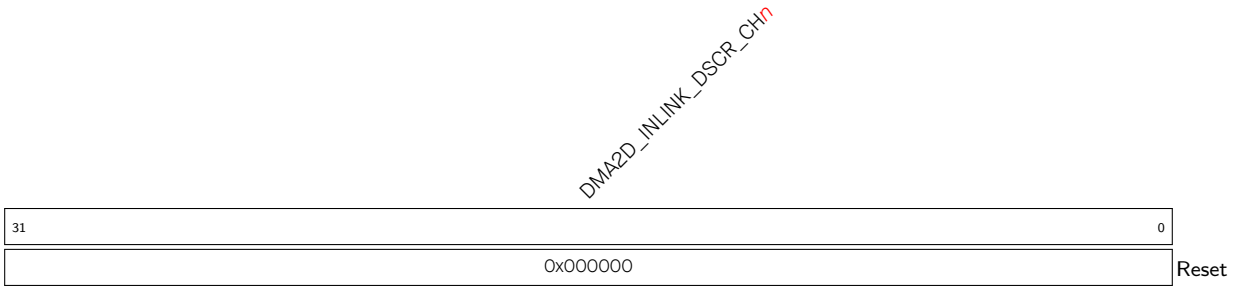
DMA2D_IN_SUC_EOF_DES_ADDR_CH*n* 表示 eof 位为 1 的接收链表描述符的地址。(RO)

Register 4.56. DMA2D_IN_ERR_EOF_DES_ADDR_CH*n*_REG (*n*: 0-2) (0x052C+0x100**n*)



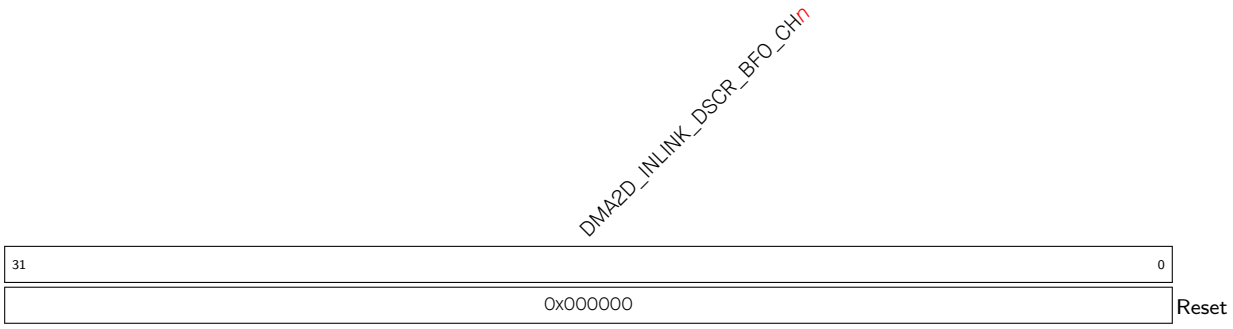
DMA2D_IN_ERR_EOF_DES_ADDR_CH*n* 表示当前接收数据有错时，相应接收链表描述符的位置。仅对 JPEG 有效。(RO)

Register 4.57. DMA2D_IN_DSCR_CH*n*_REG (*n*: 0-2) (0x0530+0x100**n*)



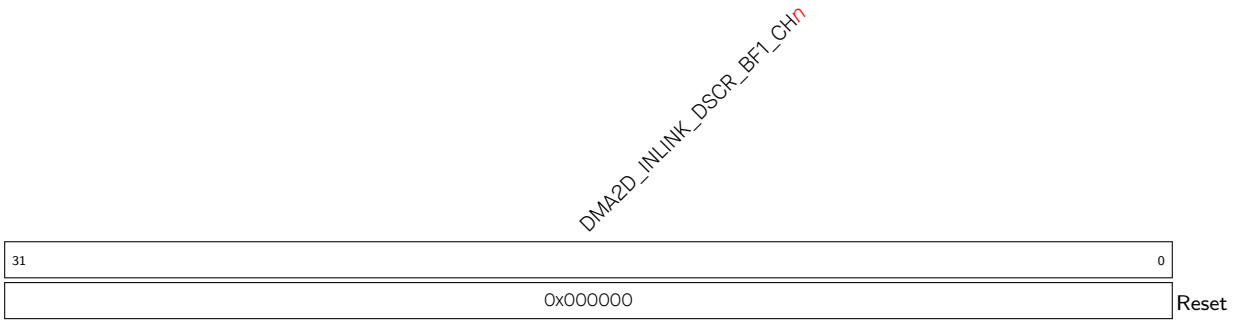
DMA2D_INLINK_DSCR_CH*n* 表示当前已预读取的接收链表描述符指向的下一个接收链表描述符地址。(RO)

Register 4.58. DMA2D_IN_DSCR_BF0_CH*n*_REG (*n*: 0-2) (0x0534+0x100**n*)



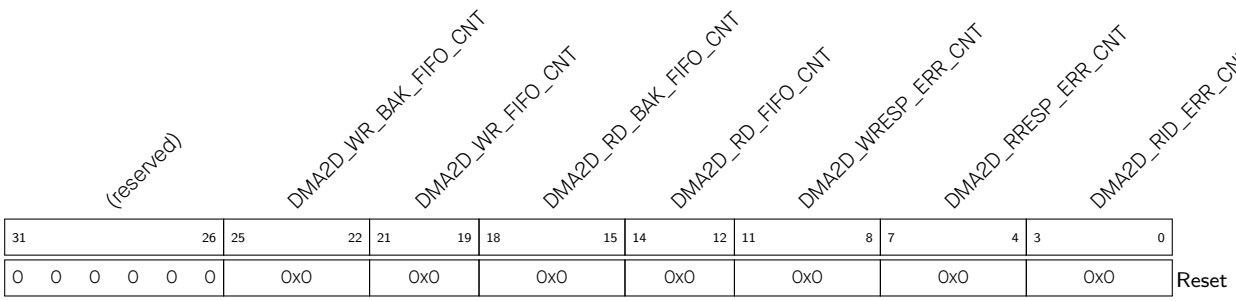
DMA2D_INLINK_DSCR_BF0_CH*n* 表示当前已预读取的接收链表描述符所在地址。(RO)

Register 4.59. DMA2D_IN_DSCR_BF1_CH*n*_REG (*n*: 0-2) (0x0538+0x100**n*)



DMA2D_INLINK_DSCR_BF1_CH*n* 表示前一个已预读取的接收链表描述符所在地址。(RO)

Register 4.60. DMA2D_AXI_ERR_REG (0x0A00)



DMA2D_RID_ERR_CNT 表示 AXI 读取 ID 错误的数量。(RO)

DMA2D_RRESP_ERR_CNT 表示 AXI 读取响应错误的数量。(RO)

DMA2D_WRESP_ERR_CNT 表示 AXI 写入响应错误的数量。(RO)

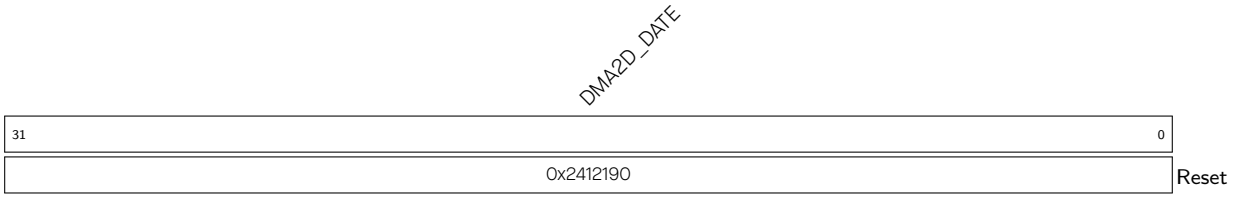
DMA2D_RD_FIFO_CNT 表示 AXI 读取命令 FIFO 中剩余的命令数量。(RO)

DMA2D_RD_BAK_FIFO_CNT 表示 AXI 读取备用命令 FIFO 中剩余的命令数量。(RO)

DMA2D_WR_FIFO_CNT 表示 AXI 写入命令 FIFO 中剩余的命令数量。(RO)

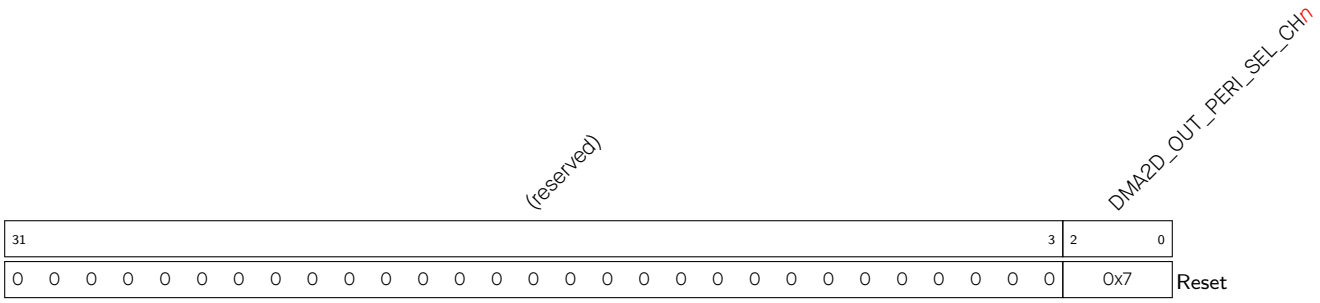
DMA2D_WR_BAK_FIFO_CNT 表示 AXI 写入备用命令 FIFO 中剩余的命令数量。(RO)

Register 4.61. DMA2D_DATE_REG (0x0A2C)



DMA2D_DATE 版本控制寄存器。(R/W)

Register 4.62. DMA2D_OUT_PERI_SEL_CH*n*_REG (*n*: 0-3) (0x0038+0x100**n*)



DMA2D_OUT_PERI_SEL_CH*n* 配置发送通道 *n* 连接的外设。

0：JPEG

1：PPA 的 SRM 模块

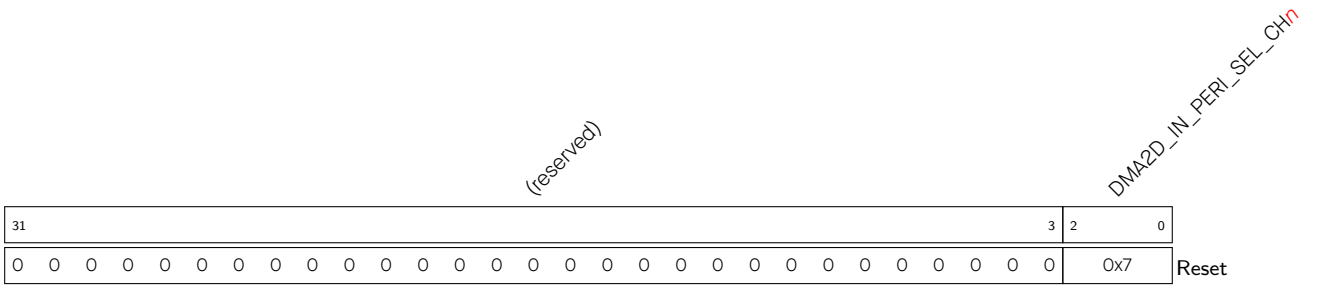
2：PPA 的 BLEND0 通道

3：PPA 的 BLEND1 通道

4 ~ 7：Dummy 4 ~ 7

(R/W)

Register 4.63. DMA2D_IN_PERI_SEL_CH*n*_REG (*n*: 0-2) (0x053C+0x100**n*)



DMA2D_IN_PERI_SEL_CH*n* 配置接收通道 *n* 连接的外设。

0：JPEG

1：PPA 的 SRM 模块

2：PPA 的 BLEND 模块

3 ~ 7：Dummy 3 ~ 7

(R/W)

第 II 卷

系统组件

该部分介绍芯片的系统级功能，包括系统启动、时钟、GPIO、定时器、看门狗、调试辅助、事件和中断处理、低功耗管理以及各种系统寄存器。

第 5 章

看门狗定时器 (WDT)

5.1 概述

看门狗定时器是一种硬件定时器，用于监视系统运行状态。软件必须定期喂狗（复位看门狗定时器）。若系统出现故障、死锁、程序卡住或其他异常情况导致软件未及时喂狗，看门狗定时器将超时并触发中断或复位，将系统导向至已知状态进行异常处理或重新启动。因此，看门狗定时器有助于检测、处理系统或软件的错误行为。

如图 5.1-1 所示，ESP32-S31 中有三个数字看门狗定时器：章节 13 定时器组 *(TIMG) [to be added later]* 描述的两个定时器组中各有一个主系统看门狗定时器，缩写为 MWDT，LP 系统中有一个 RTC 看门狗定时器，缩写为 RWDT。数字看门狗在运行期间会经历四个阶段（除非看门狗按时喂狗或者处于关闭状态），每个阶段均可配置单独的超时时间和超时动作，其中 MWDT 支持中断、HP CPU 复位和 HP 内核复位三种超时动作，RWDT 支持中断、HP CPU 复位、HP 内核复位和系统复位四种超时动作（详见章节 5.2.2.2 阶段与超时动作）。每个阶段的超时时间都可单独设置。

在 SPI Boot 模式下，RWDT 和定时器组 0 的 MWDT 会默认使能，以检测引导过程中发生的错误，并恢复运行。

ESP32-S31 中还有一个模拟看门狗定时器——超级看门狗 (SWD)。超级看门狗是模拟域的超低功耗电路，可以防止系统在数字电路异常状态下运行，并在必要时复位系统（即超时动作中的系统复位）。

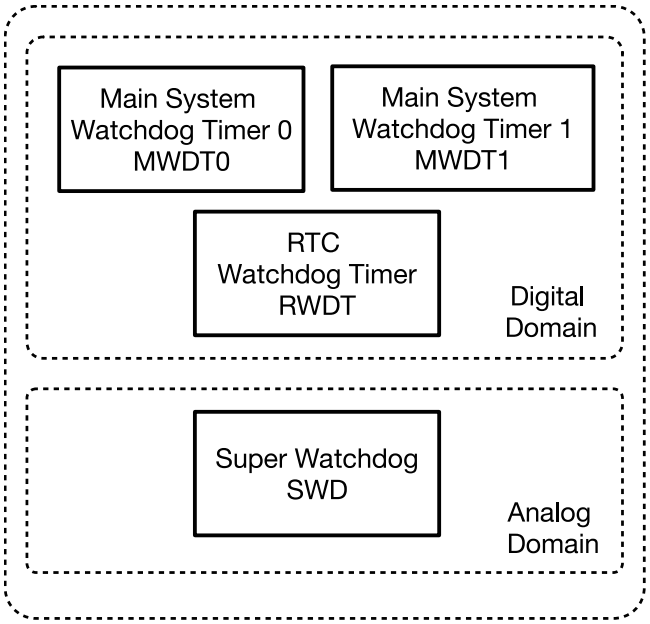


图 5.1-1. 看门狗定时器概览

请注意，MWDT 寄存器部分详见章节 13 定时器组 *(TIMG) [to be added later]*，RWDT 和 SWD 寄存器部分详见 5.5 寄存器列表。

5.2 数字看门狗定时器

5.2.1 主要特性

看门狗定时器具有如下特性：

- 四个阶段，每个阶段都可配置超时时间和超时动作
- 超时动作
  - MWDT：中断、HP CPU 复位、HP 内核复位
  - RWDT：中断、HP CPU 复位、HP 内核复位、系统复位
- 阶段 0 flash 启动保护（SPI Boot 模式）：
  - MWDT0：超时触发 HP 内核复位
  - RWDT：超时触发系统复位
- 写保护，使能时寄存器仅可读取
- 32 位超时计数器
- 时钟源：
  - MWDT：PLL_F80M_CLK、RC_FAST_CLK 或 XTAL_CLK
  - RWDT：LP_DYN_SLOW_CLK

5.2.2 功能描述

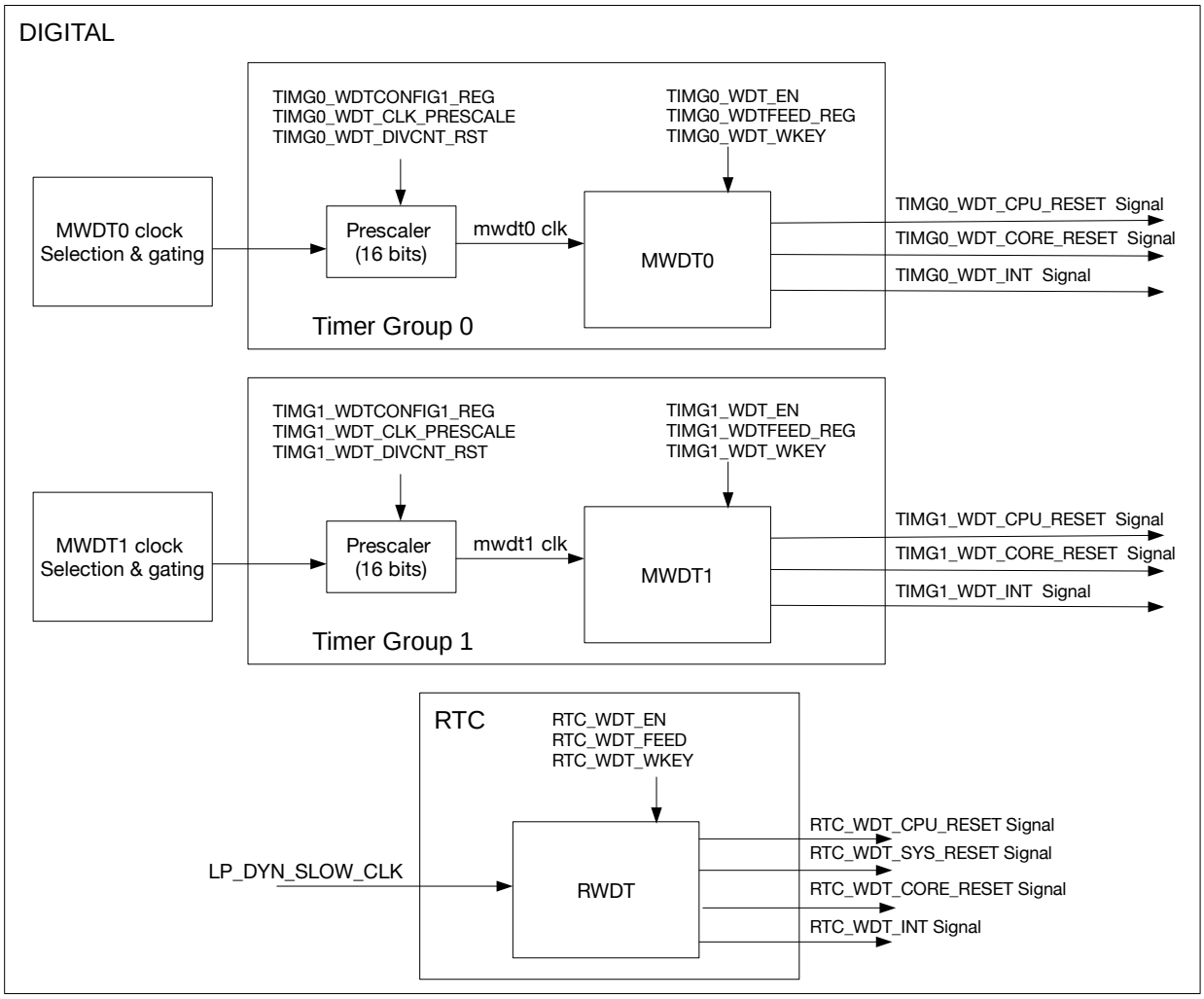


图 5.2-1. ESP32-S31 的数字看门狗定时器

图 5.2-1 为 ESP32-S31 数字系统中的三个看门狗定时器。

5.2.2.1 时钟源与 32 位计数器

看门狗定时器的核心是一个 32 位计数器。

以 MWDT0 为例：

- MWDT0 可通过设置 HP_SYS_CLKRST_PERI_CLK_CTRL20_REG 寄存器的 HP_SYS_TIMERGRP0_WDT_SRC_SEL 字段选择 PLL_F80M_CLK 时钟、RC_FAST_CLK 或 XTAL_CLK（外部时钟）作为时钟源。
- 将 HP_SYS_CLKRST_PERI_CLK_CTRL20_REG 寄存器的 HP_SYS_TIMERGRP0_WDT_CLK_EN 字段置 1 开启时钟，清零关闭时钟。时钟经由可配置的 16 位预分频器分频。详见章节 *7 复位和时钟 [to be added later]* 的表 **??**。
- MWDT0 的 16 位预分频器可通过 TIMG_WDTCONFIG1_REG 寄存器的 TIMG_WDT_CLK_PRESCALE 字段配置。TIMG_WDT_DIVCNT_RST 字段置位时，预分频器复位，可立即重新配置。

RWDT 直接将 RTC 慢速时钟 LP_DYN_SLOW_CLK（详见章节 7 *复位和时钟 [to be added later]*）用作时钟源。

MWDT 和 RWDT 看门狗可分别通过设置 TIMG_WDT_EN 和 RTC_WDT_EN 字段使能。看门狗使能后，其内部 32 位计数器的值会在每个时钟源周期内累加 1，直到达到该阶段的超时时间（即在该阶段发生超时）。如发生超时，计数器的值会重置为 0，同时看门狗进入下一阶段。如果软件成功喂狗，看门狗定时器会回到阶段 0，并将计数器的值重置为 0。软件向 TIMG_WDTFEED_REG 寄存器内写入任意值，便可为 MDWT 喂狗；向 RTC_WDT_FEED 写 1，便可为 RWDT 喂狗。

5.2.2.2 阶段与超时动作

定时器在各阶段可以配置不同的超时时间和对应的超时动作。某一阶段超时会触发对应的超时动作，同时计数器的值被重置为 0，看门狗进入下一阶段。

MWDT 和 RWDT 有四个阶段（称为阶段 0 至阶段 3）。看门狗定时器会循环工作（即从阶段 0 至阶段 3，再回到阶段 0）。

MWDT 每个阶段的超时时间可用 TIMG_WDTCONFIG*i*_REG（i 的范围是 2 到 5）寄存器配置，RWDT 的超时时间可用 RTC_WDT_STG*j*_HOLD（j 的范围是 0 到 3）字段配置。

值得注意的是，RWDT 在阶段 0 的超时时间 ($Thold_0$) 受 eFuse 寄存器 EFUSE_RD_REPEAT_DATA0_REG 的 EFUSE_WDT_DELAY_SEL 字段和 RTC_WDT_STG0_HOLD 字段共同影响，关系如下：

$$T_{hold0} = RTC_WDT_STG0_HOLD << (EFUSE_WDT_DELAY_SEL + 1)$$

其中，<< 为左移运算符。例如，RTC_WDT_STG0_HOLD 配置为 100，EFUSE_WDT_DELAY_SEL 为 1，则 $Thold_0$ 为 400 个时钟周期。

如某个阶段超时，下列超时动作之一将会执行：

表 5.2-1. 超时动作

超时动作	描述
中断	触发中断
HP CPU 复位	复位 HP CPU0 和 HP CPU1
HP 内核复位	HP 内核复位包括 HP CPU0、HP CPU1、HP 外设、HP GPIO 等
系统复位	复位包括低功耗系统在内的整个数字系统
关闭	对系统不产生影响

MWDT 所有阶段的超时动作均在 TIMG_WDTCONFIG0_REG 寄存器中配置。RWDT 的超时动作可在 RTC_WDT_CONFIG0_REG 寄存器配置。

5.2.2.3 写保护

看门狗定时器对于检测和处理系统或软件错误而言至关重要，不应轻易关闭（例如，因写寄存器位置错误而误将看门狗关闭）。因此，MWDT 和 RWDT 引入写保护机制，防止看门狗因无意的写操作而被关闭或篡改。

写保护机制通过每个看门狗定时器的写密钥字段运行（MWDT 看门狗使用 TIMG_WDT_WKEY，RWDT 看门狗使用 RTC_WDT_WKEY）。必须向看门狗定时器的写密钥字段写入 0x50D83AA1，才能修改其它看门狗寄存器。如果写密钥字段的值不是 0x50D83AA1，任何试图向看门狗定时器寄存器（除了向写密钥字段本身）写值的操作都会被忽略。推荐按以下步骤访问看门狗定时器：

1. 将 0x50D83AA1 写入看门狗定时器的写密钥字段，关闭写保护。
2. 根据需要修改看门狗，如喂狗或改变配置。
3. 向看门狗定时器的写密钥字段上写入除 0x50D83AA1 以外的任意值，重新使能写保护。

5.2.2.4 flash 引导保护

在 SPI boot 模式下，MWDT0 和 RWDT 会默认使能。MWDT0 的阶段 0 的默认超时动作为 HP 内核复位（复位主系统）。RWDT 的阶段 0 超时动作为系统复位（复位主系统和 RTC）。完成 ROM 阶段的 SPI boot 引导后，应将 TIMG_WDT_FLASHBOOT_MOD_EN 和 RTC_WDT_FLASHBOOT_MOD_EN 位清零，分别关闭 MWDT0 和 RWDT 的 flash 引导保护。然后，软件可以配置 MWDT0 和 RWDT。

5.3 超级看门狗定时器

超级看门狗 (SWD) 是模拟域的超低功耗电路，可以防止系统在数字电路异常状态下运行，并在必要时复位系统（即超时动作中的系统复位）。SWD 包含一个看门狗电路，需在每个超时阶段（约不足一秒）至少喂狗一次。该电路会在看门狗超时时间约 100 ms 之前发送 WD_INTR 信号提醒系统喂狗。

如果系统不回应 SWD 的喂狗请求，看门狗超时，SWD 会产生系统电平信号 SWD_RSTB，复位芯片上的整个数字电路（即超时动作中的系统复位）。

SWD 的时钟源固定，不可选择。

5.3.1 主要特性

SWD 具有如下特性：

- 超低功耗
- 用中断提醒 SWD 即将超时
- 软件有多种专用的方法喂 SWD，让 SWD 监控整个操作系统的工作状态

5.3.2 SWD 控制器

5.3.2.1 结构

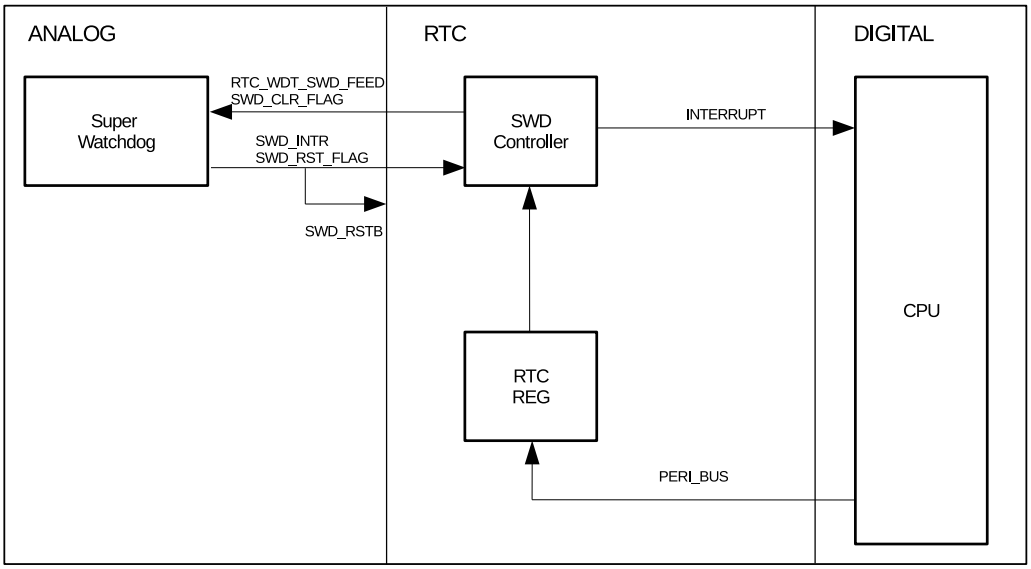


图 5.3-1. SWD 控制器结构

5.3.2.2 工作流程

正常状态下：

- SWD 控制器收到 SWD 的喂狗请求。
- SWD 控制器向主 CPU 发送中断。
- 主 CPU 通过置位 RTC_WDT_SWD_FEED 直接喂狗。
- CPU 喂狗时，需要先向 RTC_WDT_SWD_WKEY 写 0x50D83AA1 关闭 SWD 控制器的写保护。这样做可以防止系统在数字电路异常状态下运行时误喂 SWD。
- 如将 RTC_WDT_SWD_AUTO_FEED_EN 置 1，SWD 控制器也可配置为在不需要 CPU 干预的情况下喂 SWD。

复位后：

- 可查看 LP_AONCLKRST_LPCORE_RESET_CAUSE[4:0] 获知 HP CPU 复位原因。
  如 LP_AONCLKRST_LPCORE_RESET_CAUSE[4:0] == 0x12，则表示上一次复位的原因是 SWD 复位。
- 置位 RTC_WDT_SWD_RST_FLAG_CLR 清除 SWD 复位标志。

5.4 中断

看门狗定时器中断，请前往章节 13 定时器组 *(TIMG) [to be added later]* 的第 **??** 节 **??** 查看。

5.5 寄存器列表

MWDT 寄存器是定时器组模块的一部分，在章节 13 定时器组 *(TIMG) [to be added later]* 的第 **??** 节 **??** 中有详细描述。

本小节包含 RWDT 和 SWD 相关寄存器介绍，其地址均为相对于 RTC_WDT 基地址的地址偏移量（相对地址），具体基地址请见章节 3 系统和存储器 *[to be added later]* 中的表 3。

请查看章节 寄存器的访问类型，了解“访问”列缩写的含义。

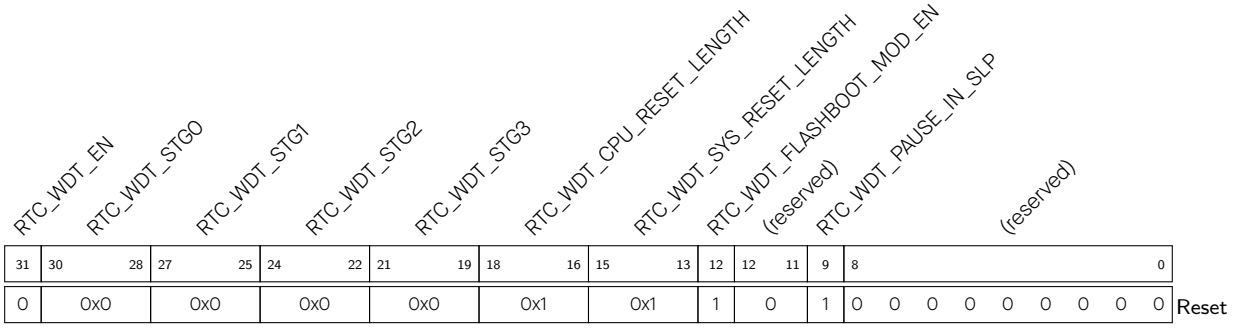
名称	描述	地址	访问
配置寄存器			
RTC_WDT_CONFIG0_REG	RWDT 配置寄存器	0x0000	R/W
RTC_WDT_CONFIG1_REG	配置 RWDT 阶段 0 的超时时间	0x0004	R/W
RTC_WDT_CONFIG2_REG	配置 RWDT 阶段 1 的超时时间	0x0008	R/W
RTC_WDT_CONFIG3_REG	配置 RWDT 阶段 2 的超时时间	0x000C	R/W
RTC_WDT_CONFIG4_REG	配置 RWDT 阶段 2 的超时时间	0x0010	R/W
RTC_WDT_FEED_REG	配置喂狗 RWDT	0x0014	WT
RTC_WDT_WPROTECT_REG	配置锁住 RWDT 的配置寄存器	0x0018	R/W
RTC_WDT_SWD_CONFIG_REG	SWD 配置寄存器	0x001C	varies
RTC_WDT_SWD_WPROTECT_REG	配置锁住 SWD 的配置寄存器	0x0020	R/W
RTC_WDT_INT_RAW_REG	配置是否产生 WDT 超时中断	0x0024	R/WTC/SS
RTC_WDT_INT_ST_REG	WDT 中断状态寄存器	0x0028	RO
RTC_WDT_INT_ENA_REG	WDT 中断使能寄存器	0x002C	R/W
RTC_WDT_INT_CLR_REG	WDT 中断清除寄存器	0x0030	WT
RTC_WDT_DATE_REG	版本控制寄存器	0x03FC	R/W

5.6 寄存器

MWDT 寄存器是定时器组模块的一部分，在章节 13 定时器组 *(TIMG) [to be added later]* 的第 ?? 节 ?? 中有详细描述。

本小节包含 RWDT 和 SWD 相关寄存器介绍，其地址均为相对于 RTC_WDT 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

Register 5.1. RTC_WDT_CONFIG0_REG (0x0000)



RTC_WDT_PAUSE_IN_SLP 配置芯片处于睡眠模式（CPU 不可工作）时是否禁用 RWDT。

0：使能

1：禁用

(R/W)

RTC_WDT_FLASHBOOT_MOD_EN 配置在 SPI boot 模式下是否自动使能 RWDT。

0：禁用

1：使能

(R/W)

RTC_WDT_SYS_RESET_LENGTH 配置 HP 内核复位时间。

单位：LP_DYN_FAST_CLK

(R/W)

RTC_WDT_CPU_RESET_LENGTH 配置 HP CPU 复位时间。

单位：LP_DYN_FAST_CLK

(R/W)

RTC_WDT_STG3 配置阶段 3 的超时复位动作。

0：无效

1：中断

2：HP CPU 复位

3：HP 内核复位

4：系统复位

(R/W)

见下页...

Register 5.1. RTC_WDT_CONFIG0_REG (0x0000)

接上页...

RTC_WDT_STG2 配置阶段 2 的超时复位动作。

0：无效
1：中断
2：HP CPU 复位
3：HP 内核复位
4：系统复位
(R/W)

RTC_WDT_STG1 配置阶段 1 的超时复位动作。

0：无效
1：中断
2：HP CPU 复位
3：HP 内核复位
4：系统复位
(R/W)

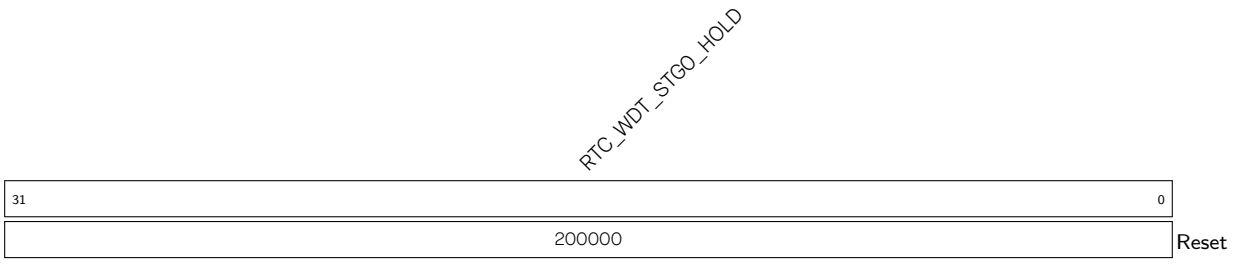
RTC_WDT_STG0 配置阶段 0 的超时复位动作。

0：无效
1：中断
2：HP CPU 复位
3：HP 内核复位
4：系统复位
(R/W)

RTC_WDT_EN 配置是否使能 RWDT。

0：禁用 RWDT
1：使能 RWDT
(R/W)

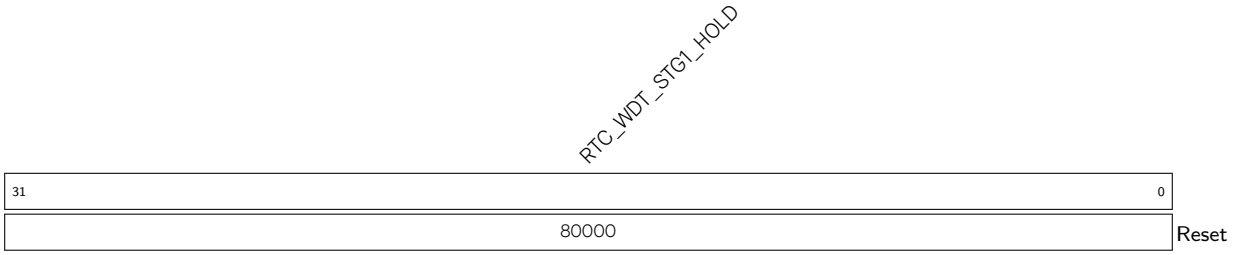
Register 5.2. RTC_WDT_CONFIG1_REG (0x0004)



RTC_WDT_STG0_HOLD 配置阶段 0 的超时时间。

单位：LP_DYN_SLOW_CLK
(R/W)

Register 5.3. RTC_WDT_CONFIG2_REG (0x0008)

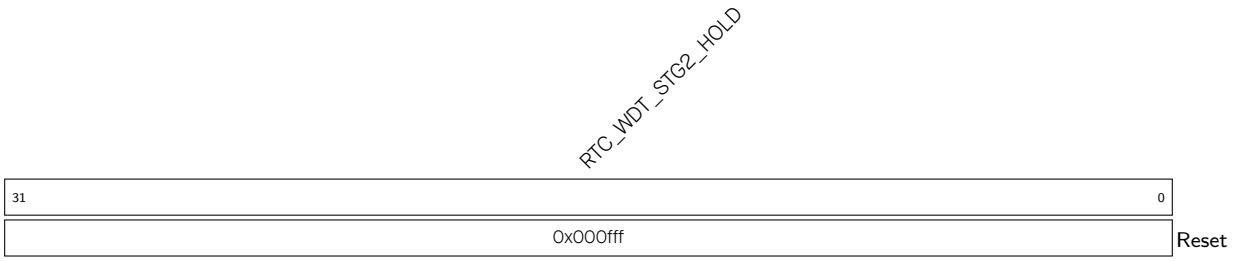


RTC_WDT_STG1_HOLD 配置阶段 1 的超时时间。

单位：LP_DYN_SLOW_CLK

(R/W)

Register 5.4. RTC_WDT_CONFIG3_REG (0x000C)

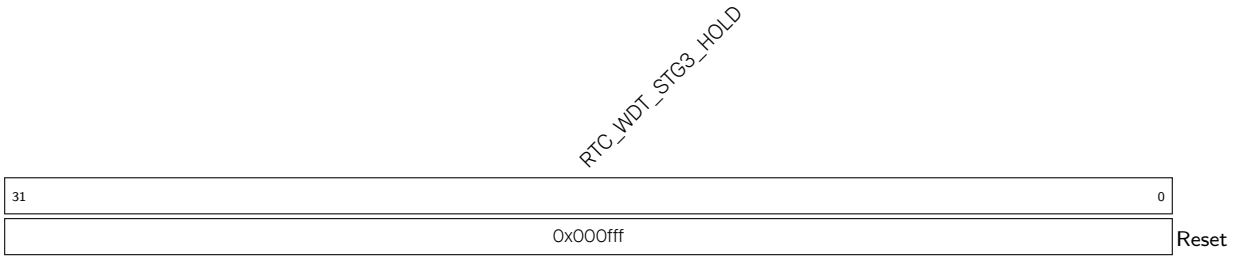


RTC_WDT_STG2_HOLD 配置阶段 2 的超时时间。

单位：LP_DYN_SLOW_CLK

(R/W)

Register 5.5. RTC_WDT_CONFIG4_REG (0x0010)

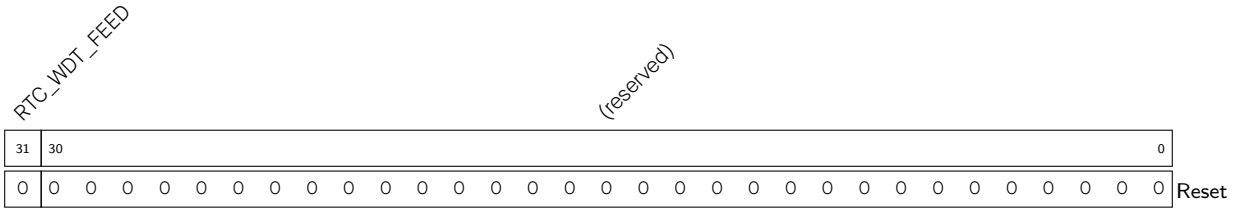


RTC_WDT_STG3_HOLD 配置阶段 3 的超时时间。

单位：LP_DYN_SLOW_CLK

(R/W)

Register 5.6. RTC_WDT_FEED_REG (0x0014)



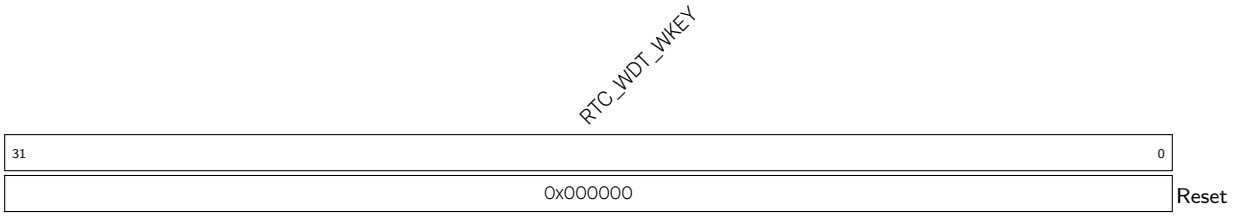
RTC_WDT_FEED 配置此位喂狗 RWDT。

0：无效

1：喂狗

(WT)

Register 5.7. RTC_WDT_WPROTECT_REG (0x0018)



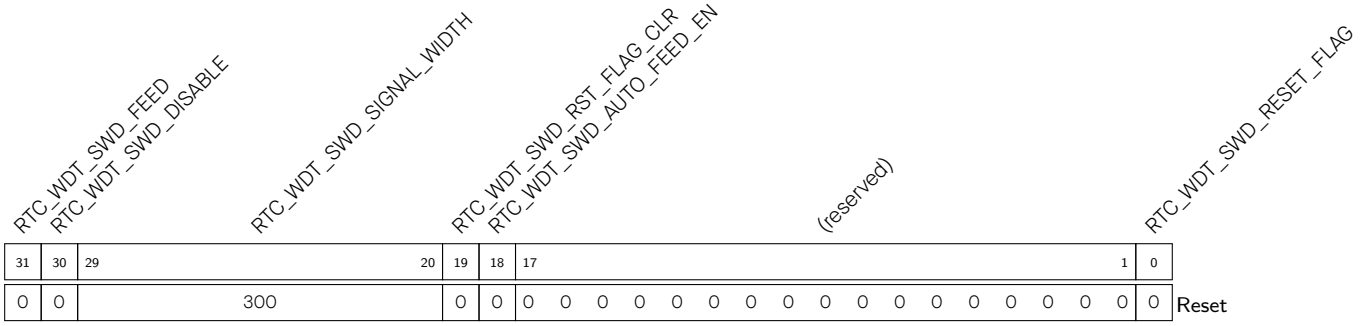
RTC_WDT_WKEY 配置该寄存器来锁住或解锁 RWDT 的配置寄存器。

0x50D83AA1: 解锁 RWDT 的配置寄存器

其他值: 锁住 RWDT 的配置寄存器，软件不能修改 RWDT 的配置

(R/W)

Register 5.8. RTC_WDT_SWD_CONFIG_REG (0x001C)



RTC_WDT_SWD_RESET_FLAG 表示 SWD 是否产生了复位信号。

0：否

1：是

(RO)

RTC_WDT_SWD_AUTO_FEED_EN 配置此位使能硬件自动喂狗。

0：禁止硬件自动喂狗

1：使能硬件自动喂狗

(R/W)

RTC_WDT_SWD_RST_FLAG_CLR 配置此位清除 SWD 复位标志。

0：无效

1：清除复位标志

(WT)

RTC_WDT_SWD_SIGNAL_WIDTH 配置输出到模拟电路的信号长度。

单位：LP_DYN_FAST_CLK

(R/W)

RTC_WDT_SWD_DISABLE 配置此位禁用 SWD。

0：使能

1：禁用

(R/W)

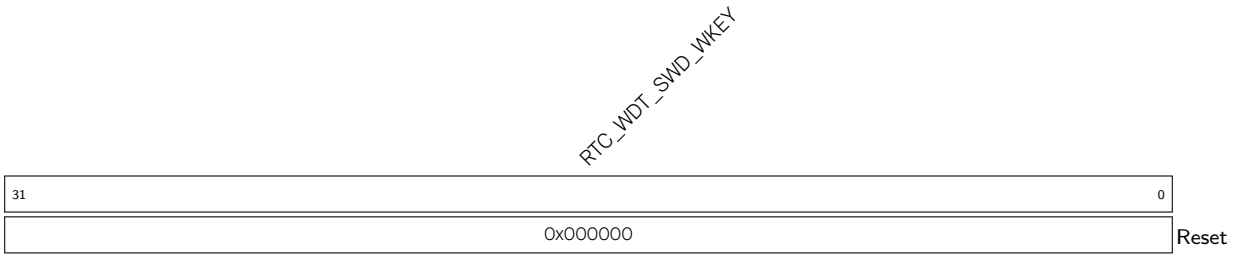
RTC_WDT_SWD_FEED 配置此位喂狗 SWD。

0：无效

1: 喂狗 SWD

(WT)

Register 5.9. RTC_WDT_SWD_WPROTECT_REG (0x0020)



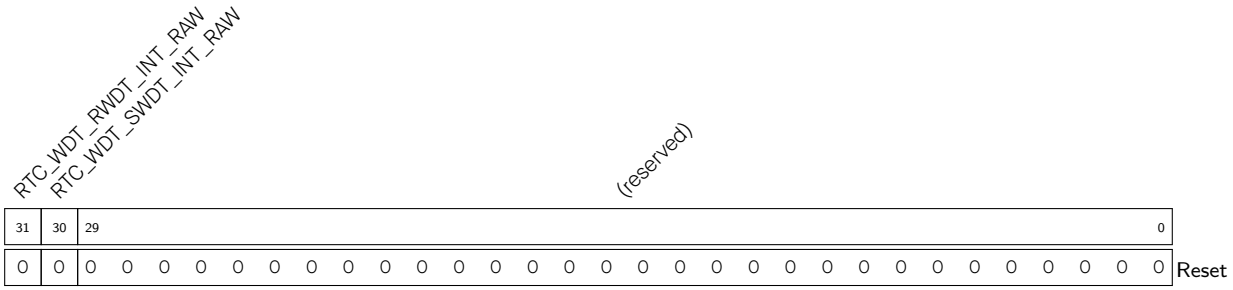
RTC_WDT_SWD_WKEY 配置该寄存器来锁住或解锁 SWDT 的配置寄存器。

0x50D83AA1: 解锁 SWD 的配置寄存器

其他值: 锁住 SWD 的配置寄存器，软件不能修改 SWD 的配置

(R/W)

Register 5.10. RTC_WDT_INT_RAW_REG (0x0024)



RTC_WDT_SWDT_INT_RAW 表示 SWD 是否产生超时中断。

0：否

1：是

(R/WTC/SS)

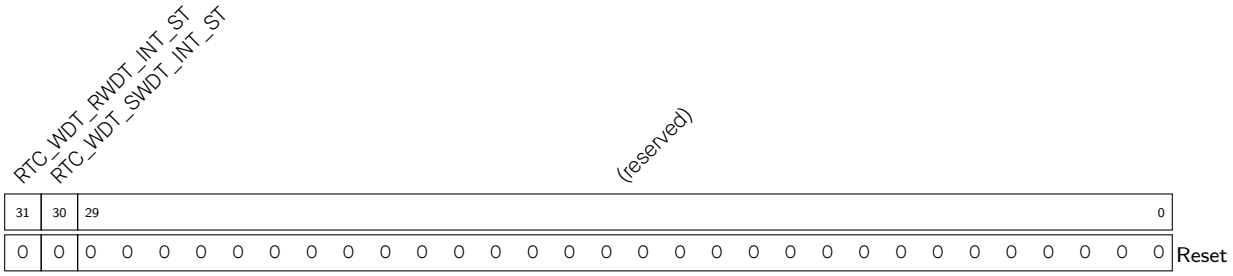
RTC_WDT_RWDT_INT_RAW 表示 RWDT 是否产生超时中断。

0：否

1：是

(R/WTC/SS)

Register 5.11. RTC_WDT_INT_ST_REG (0x0028)



RTC_WDT_SWDT_INT_ST 表示 SWD 是否产生并发送超时中断给 CPU。

0：否

1：是

(RO)

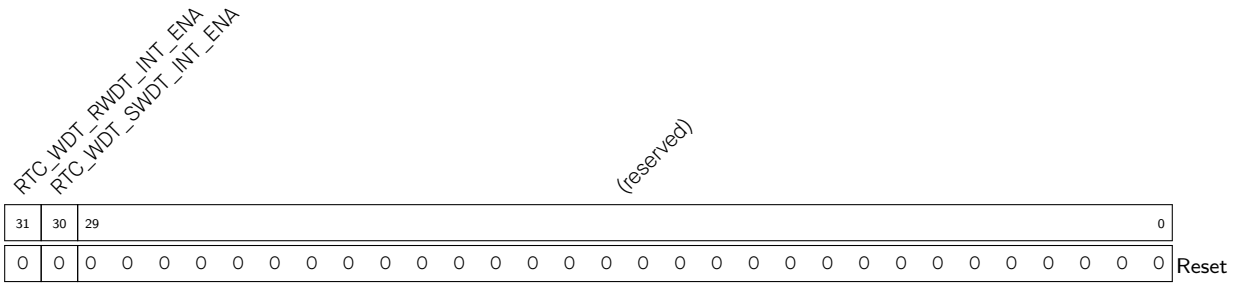
RTC_WDT_RWDT_INT_ST 表示 RWDT 是否产生并发送超时中断给 CPU。

0：否

1：是

(RO)

Register 5.12. RTC_WDT_INT_ENA_REG (0x002C)



RTC_WDT_SWDT_INT_ENA 配置是否使能 SWD 发送超时中断。

0：否

1：是

(R/W)

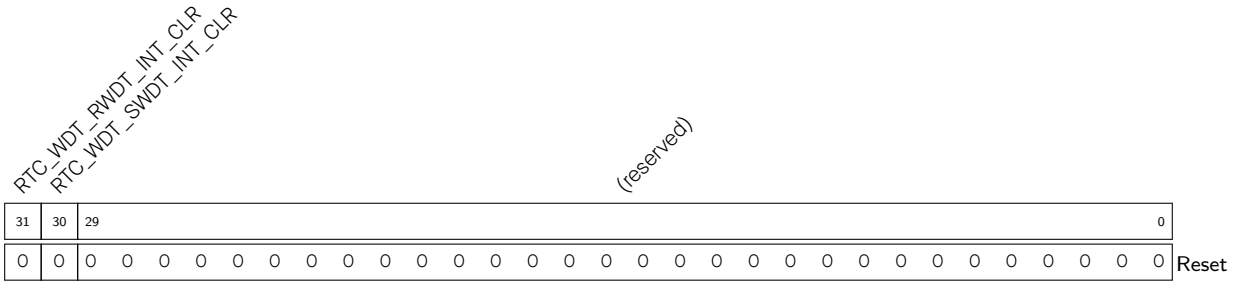
RTC_WDT_RWDT_INT_ENA 配置是否使能 RWDT 发送超时中断。

0：否

1：是

(R/W)

Register 5.13. RTC_WDT_INT_CLR_REG (0x0030)



RTC_WDT_SWDT_INT_CLR 配置是否清除 SWD 发送到 CPU 的超时中断信号。

0：否

1：是

(WT)

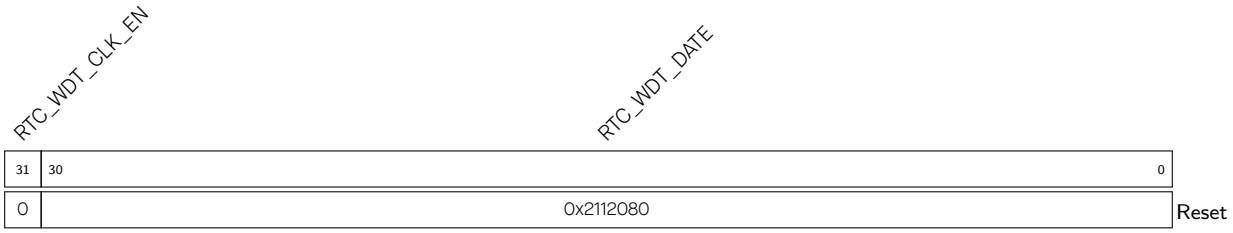
RTC_WDT_RWDT_INT_CLR 配置是否清除 RWDT 发送到 CPU 的超时中断信号。

0：否

1：是

(WT)

Register 5.14. RTC_WDT_DATE_REG (0x03FC)



RTC_WDT_DATE 版本控制寄存器。(R/W)

RTC_WDT_CLK_EN 保留。(R/W)

第 6 章

实时时钟定时器

6.1 概述

RTC Timer 是实现 ESP32-S31 低功耗管理的一个重要模块。RTC Timer 是一个 48 位的可读计时器，主要作用是在低功耗模式下，当 HP 系统中的定时器外设不可用时，继续为系统提供定时器服务。同时还支持配置定时器中断、记录系统中特定事件发生的时刻。

6.2 主要特性

- 48 位的计时器
- 触发特定事件时可记录事件发生的时刻，支持的特定事件有：
  - HP 系统复位
  - CPU 进入 stall 状态
  - CPU 退出 stall 状态
  - 晶振时钟开启
  - 晶振时钟关闭
- 通过配置寄存器触发 RTC Timer 记录当前时间
- 支持缓存最近两次特定事件的发生的时间
- 支持在目标时刻产生中断，目标时刻可配置，可同时配置两个目标时刻
- 除 LP 系统上电复位外的其余任何复位/睡眠均不会使 RTC Timer 停止或复位

6.3 功能描述

- 48 位计时器
  - RTC Timer 的实现基于一个 48 位的计时器，其驱动时钟是由位于 AlwaysOn 电源域的 RTC_SLOW_CLK 提供，采用不间断的循环计时（LP 系统复位除外），当 48 位计时器溢出时会产生溢出中断。
- 记录特定事件的发生时刻
  - RTC Timer 支持记录三类特定事件的发生时刻：
    * HP 系统复位
    * CPU 进入或退出 stall 状态

* 晶振时钟开启或关闭

- 三类事件都有各自独立的配置寄存器，当配置寄存器使能后，若发生对应的事件，产生的硬件脉冲就会触发 RTC Timer 记录当前时间。对应的配置寄存器见下表：

表 6.3-1. 配置 RTC Timer 记录特定事件的发生时刻

配置寄存器	描述
RTC_TIMER_MAIN_TIMER_SYS_RST	使能 RTC Timer 记录系统复位时刻。
RTC_TIMER_MAIN_TIMER_SYS_STALL	使能 RTC Timer 记录 CPU 进入或退出 stall 状态时刻。
RTC_TIMER_MAIN_TIMER_XTAL_OFF	使能 RTC Timer 记录晶振时钟开启或关闭时刻。

- 通过配置寄存器触发 RTC Timer 记录当前时间
  - 除了上述三类特定事件发生时可以触发 RTC Timer 记录当前时间，还可以通过配置寄存器 RTC_TIMER_MAIN_TIMER_UPDATE 触发 RTC Timer 记录当前时间。
  - 配置 RTC_TIMER_MAIN_TIMER_UPDATE 寄存器会产生一个脉冲信号，触发 RTC Timer 记录当前时间。
- 目标时刻产生计数中断
  - RTC Timer 支持同时配置两个目标时刻，分别为目标时刻 0 和目标时刻 1，当计时器到达目标时刻 0 时会触发 RTC_TIMER_CMP0_INT 中断，当计时器到达目标时刻 1 时会触发 RTC_TIMER_CMP1_INT 中断。
  - 目标时刻的配置寄存器为：

表 6.3-2. RTC Timer 目标时刻配置

配置寄存器	描述
RTC_TIMER_MAIN_TIMER_TAR_EN*n*	*n* = 0 or 1，配置目标时刻使能。
RTC_TIMER_MAIN_TIMER_TAR_LOW*n*	*n* = 0 or 1，配置目标时刻的低 32 位。
RTC_TIMER_MAIN_TIMER_TAR_HIGH*n*	*n* = 0 or 1，配置目标时刻的高 16 位。

- 读取缓存时刻
  - RTC Timer 有两组寄存器用于缓存特定事件的发生时刻，分别为寄存器组 0 和寄存器组 1；
  - 寄存器组 0 包括寄存器RTC_TIMER_MAIN_BUF0_LOW_REG 和 RTC_TIMER_MAIN_BUF0_HIGH_REG，用于记录 RTC Timer 定时器在当前触发下的计数值；
  - 寄存器组 1 包括寄存器 RTC_TIMER_MAIN_BUF1_LOW_REG 和 RTC_TIMER_MAIN_BUF1_HIGH_REG，用于记录 RTC Timer 定时器在上一次触发下的计数值；
  - 每次有新的触发，上一次触发时的记录将从寄存器组 0 移至寄存器组 1（寄存器组 1 中之前的记录将被覆盖），而本次触发的记录将存储在寄存器组 0。因此，RTC Timer 最多可同时记录两次触发的值。

6.4 事件任务矩阵功能

在 ESP32-S31 中，RTC Timer 支持 ETM 功能，即可以通过任意外设的 ETM 事件触发 RTC Timer 的 ETM 任务，或者通过 RTC Timer 的 ETM 事件触发任意外设的 ETM 任务。关于 ETM 更多详细信息，请参考章节 10 事件任务矩阵 (ETM) [to be added later]。这里仅介绍与 RTC Timer 相关的 ETM 任务和 ETM 事件。

RTC Timer 不支持 ETM 任务。

RTC Timer 可产生的 ETM 事件有：

- RTC_EVT_CMP：表示 RTC Timer 计时到达寄存器 RTC_TIMER_MAIN_TIMER_TAR_LOW0 和 RTC_TIMER_MAIN_TIMER_TAR_HIGH0 所配置的目标时刻 0。
- RTC_EVT_OVF：表示 48 位计时器计数溢出。
- RTC_EVT_TICK：表示 RTC 计时器递增 1 时触发的事件。

6.5 中断

ESP32-S31 的 RTC Timer 可以生成以下中断信号并发送给中断矩阵。

- LP_TIMER_REG_0_INTR
- LP_TIMER_REG_1_INTR

这些中断信号由 RTC Timer 的内部中断源生成。表 6.5-1 列出了 RTC Timer 的内部中断源、中断触发条件、以及生成的中断信号。

表 6.5-1. RTC Timer 的内部中断源

内部中断源	触发条件	中断信号
RTC_TIMER_CMP0_INT	RTC Timer 计时到达目标时刻 0	LP_TIMER_REG_0_INTR
RTC_TIMER_CMP1_INT	RTC Timer 计时到达目标时刻 1	LP_TIMER_REG_1_INTR
RTC_TIMER_OVERFLOW_INT	48 位计时器计数溢出	LP_TIMER_REG_0_INTR LP_TIMER_REG_1_INTR

说明：
中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 6.6 寄存器列表。

6.6 寄存器列表

本小节的所有地址均为相对于 RTC Timer 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

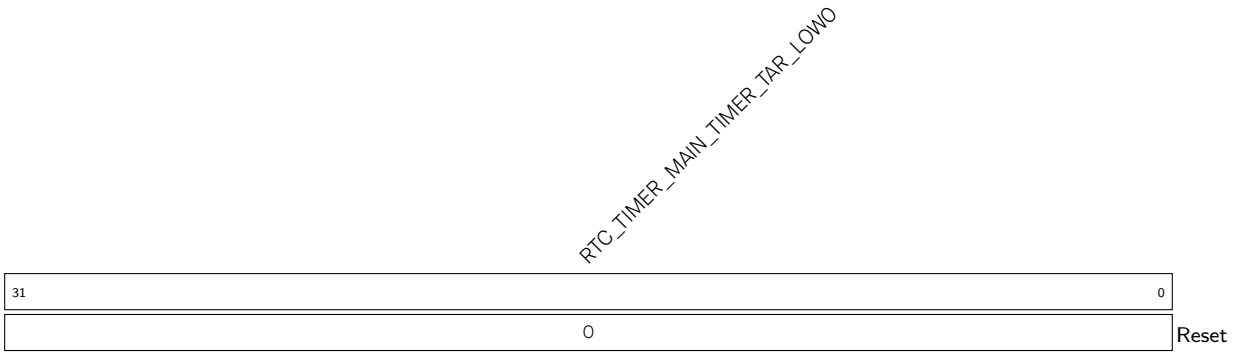
名称	描述	地址	访问
配置寄存器			
RTC_TIMER_TAR0_LOW_REG	目标时刻 0，低 32 位	0x0000	R/W
RTC_TIMER_TAR0_HIGH_REG	目标时刻 0，高 16 位	0x0004	varies
RTC_TIMER_TAR1_LOW_REG	目标时刻 1，低 32 位	0x0008	R/W
RTC_TIMER_TAR1_HIGH_REG	目标时刻 1，高 16 位	0x000C	varies
RTC_TIMER_UPDATE_REG	锁存计时器当前值使能	0x0010	varies
RTC_TIMER_MAIN_BUF0_LOW_REG	计时器缓存值 0，低 32 位	0x0014	RO
RTC_TIMER_MAIN_BUF0_HIGH_REG	计时器缓存值 0，高 16 位	0x0018	RO
RTC_TIMER_MAIN_BUF1_LOW_REG	计时器缓存值 1，低 32 位	0x001C	RO
RTC_TIMER_MAIN_BUF1_HIGH_REG	计时器缓存值 1，高 16 位	0x0020	RO
中断寄存器			
RTC_TIMER_INT_RAW_REG	计时器到达目标时刻 0 的原始中断状态寄存器	0x0028	RO
RTC_TIMER_INT_ST_REG	计时器到达目标时刻 0 的中断状态寄存器	0x002C	RO
RTC_TIMER_INT_ENA_REG	计时器到达目标时刻 0 的中断使能寄存器	0x0030	R/W
RTC_TIMER_INT_CLR_REG	计时器到达目标时刻 0 的中断清除寄存器	0x0034	WO
RTC_TIMER_LP_INT_RAW_REG	计时器到达目标时刻 1 的原始中断状态寄存器	0x0038	RO
RTC_TIMER_LP_INT_ST_REG	计时器到达目标时刻 1 的中断状态寄存器	0x003C	RO
RTC_TIMER_LP_INT_ENA_REG	计时器到达目标时刻 1 的中断使能寄存器	0x0040	R/W
RTC_TIMER_LP_INT_CLR_REG	计时器到达目标时刻 1 的中断清除寄存器	0x0044	WO
版本寄存器			
RTC_TIMER_DATE_REG	版本控制寄存器	0x03FC	R/W

6.7 寄存器

本小节的所有地址均为相对于 RTC Timer 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

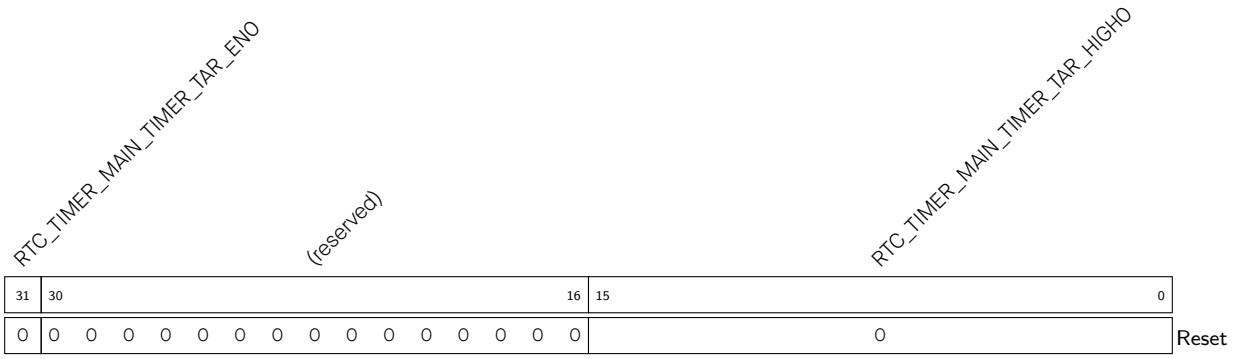
关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 6.1. RTC_TIMER_TAR0_LOW_REG (0x0000)



RTC_TIMER_MAIN_TIMER_TAR_LOW0 计时器的目标时刻 0，低 32 位。(R/W)

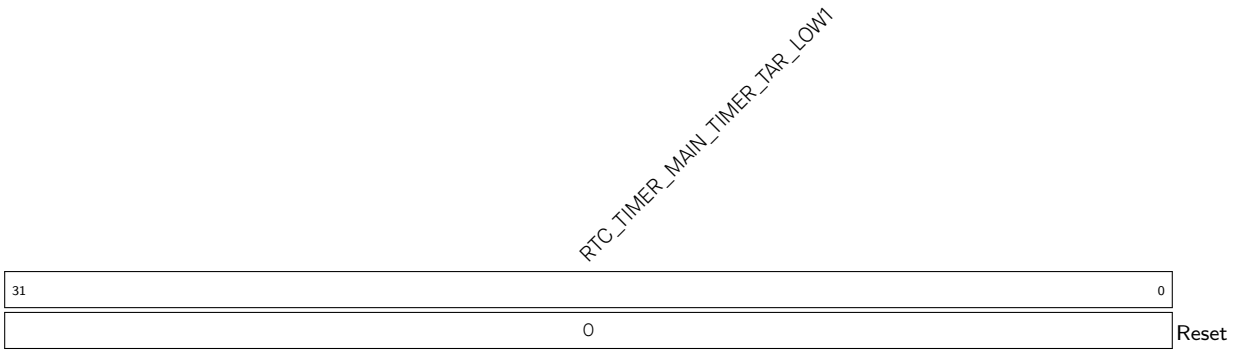
Register 6.2. RTC_TIMER_TAR0_HIGH_REG (0x0004)



RTC_TIMER_MAIN_TIMER_TAR_HIGH0 计时器的目标时刻 0，高 16 位。(R/W)

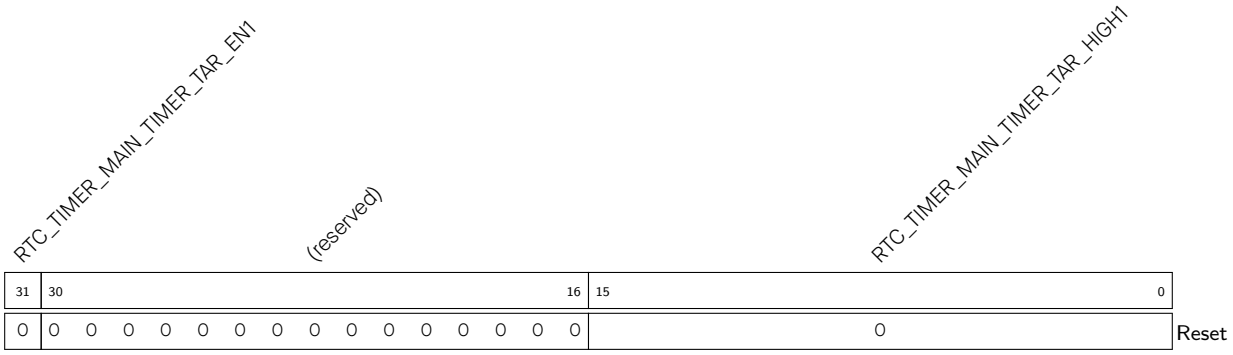
RTC_TIMER_MAIN_TIMER_TAR_EN0 计时器的目标时刻 0 使能。(WO)

Register 6.3. RTC_TIMER_TAR1_LOW_REG (0x0008)



RTC_TIMER_MAIN_TIMER_TAR_LOW1 计时器的目标时刻 1，低 32 位。(R/W)

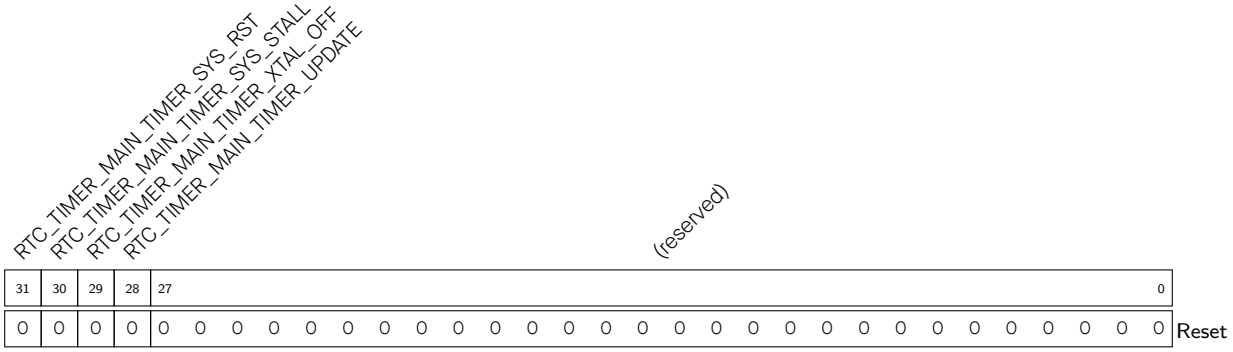
Register 6.4. RTC_TIMER_TAR1_HIGH_REG (0x000C)



RTC_TIMER_MAIN_TIMER_TAR_HIGH1 计时器的目标时刻 1，高 16 位。(R/W)

RTC_TIMER_MAIN_TIMER_TAR_EN1 计时器的目标时刻 1 使能。(WO)

Register 6.5. RTC_TIMER_UPDATE_REG (0x0010)



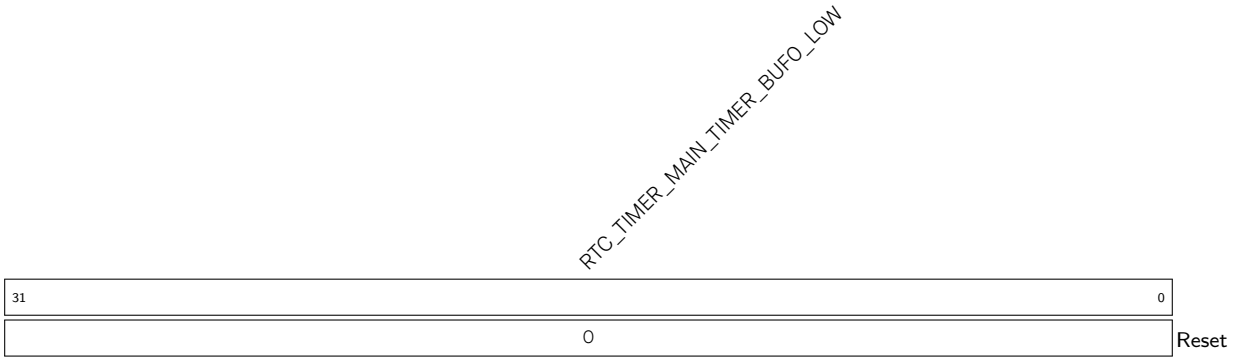
RTC_TIMER_MAIN_TIMER_UPDATE 软件配置记录当前时刻。(WO)

RTC_TIMER_MAIN_TIMER_XTAL_OFF 记录晶振时钟开启或关闭时刻使能。(R/W)

RTC_TIMER_MAIN_TIMER_SYS_STALL 记录 CPU 进入或退出 stall 状态时刻使能。(R/W)

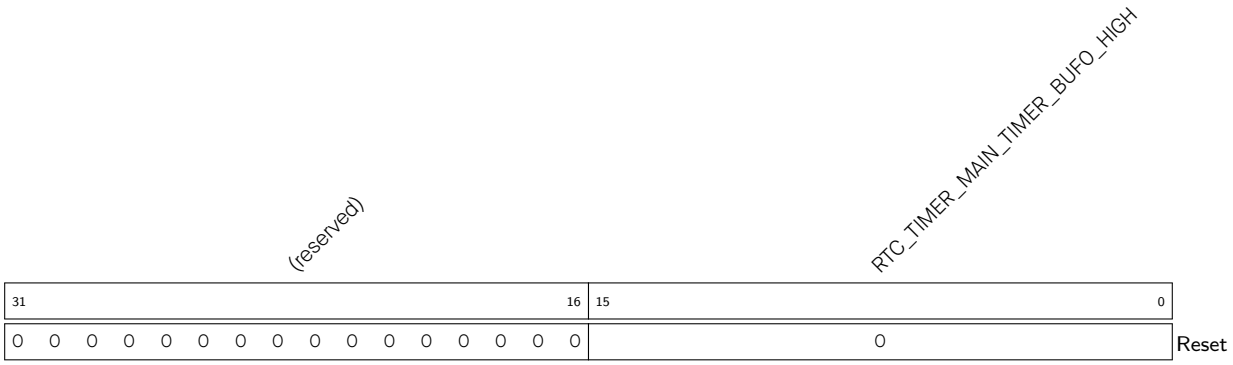
RTC_TIMER_MAIN_TIMER_SYS_RST 记录系统复位时刻使能。(R/W)

Register 6.6. RTC_TIMER_MAIN_BUF0_LOW_REG (0x0014)



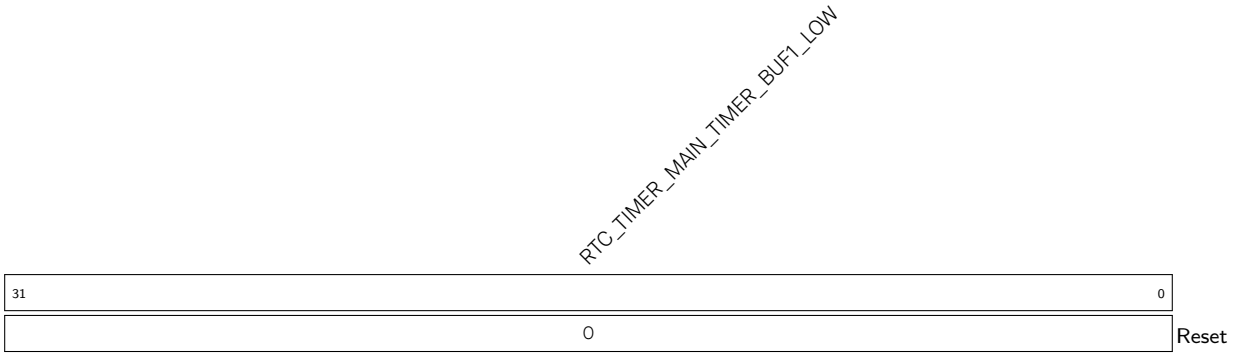
RTC_TIMER_MAIN_TIMER_BUF0_LOW 计时器缓存值 0，低 32 位。(RO)

Register 6.7. RTC_TIMER_MAIN_BUF0_HIGH_REG (0x0018)



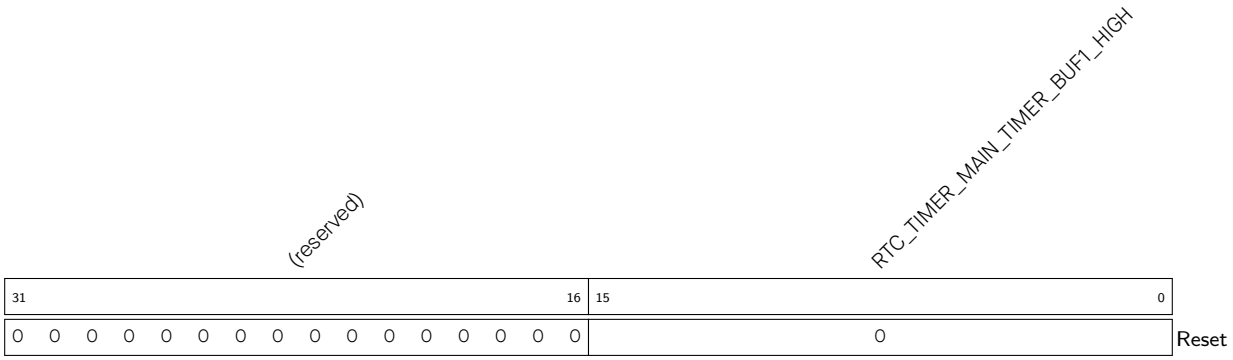
RTC_TIMER_MAIN_TIMER_BUF0_HIGH 计时器缓存值 0，高 16 位。(RO)

Register 6.8. RTC_TIMER_MAIN_BUF1_LOW_REG (0x001C)



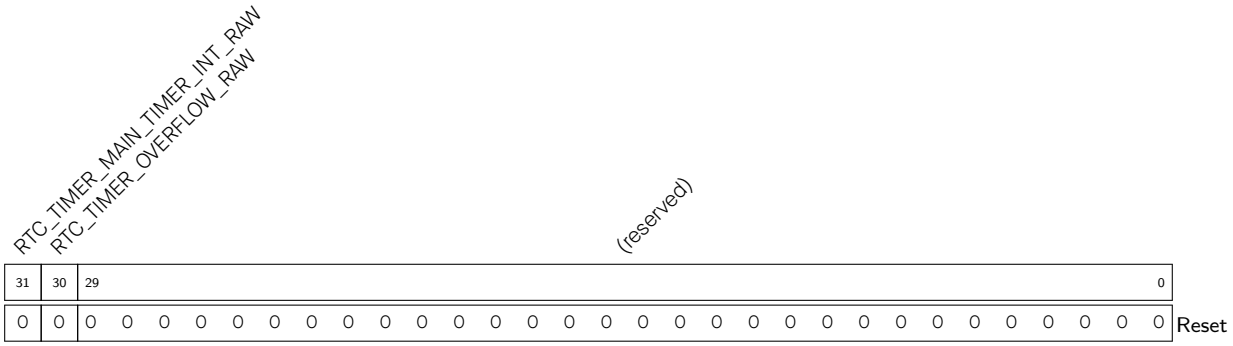
RTC_TIMER_MAIN_TIMER_BUF1_LOW 计时器缓存值 1，低 32 位。(RO)

Register 6.9. RTC_TIMER_MAIN_BUF1_HIGH_REG (0x0020)



RTC_TIMER_MAIN_TIMER_BUF1_HIGH 计时器缓存值 1，高 16 位。(RO)

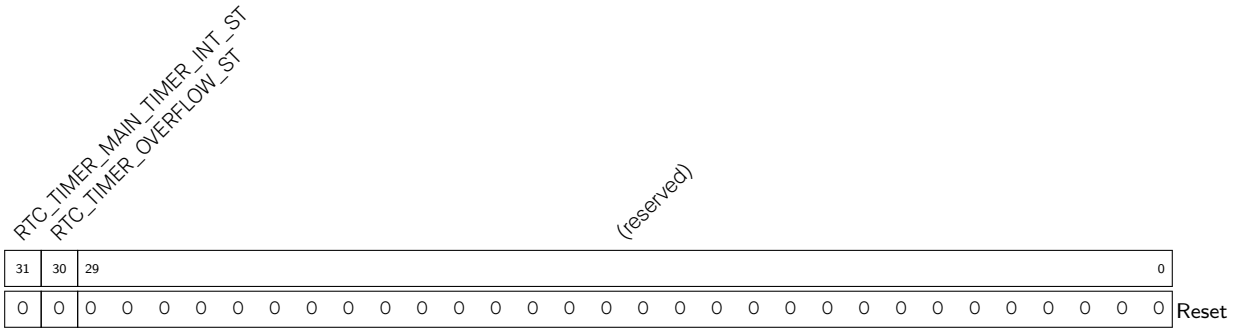
Register 6.10. RTC_TIMER_INT_RAW_REG (0x0028)



RTC_TIMER_OVERFLOW_RAW 48 位计时器溢出的原始中断。(RO)

RTC_TIMER_MAIN_TIMER_INT_RAW 计时器到达目标时刻 0 的原始中断。(RO)

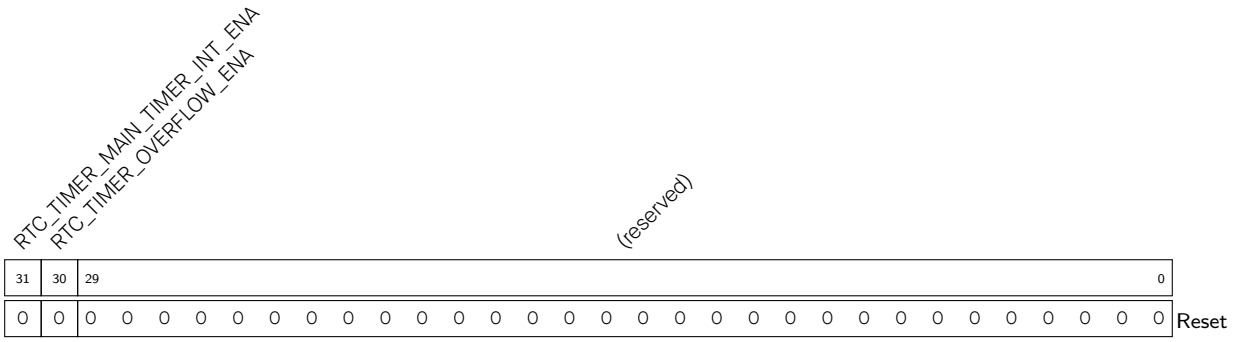
Register 6.11. RTC_TIMER_INT_ST_REG (0x002C)



RTC_TIMER_OVERFLOW_ST 48 位计时器溢出的状态中断。(RO)

RTC_TIMER_MAIN_TIMER_INT_ST 计时器到达目标时刻 0 的状态中断。(RO)

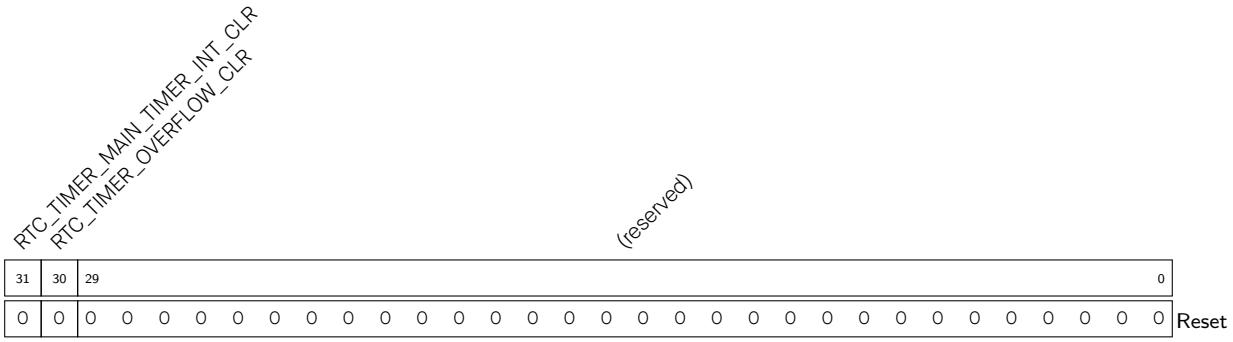
Register 6.12. RTC_TIMER_INT_ENA_REG (0x0030)



RTC_TIMER_OVERFLOW_ENA 48 位计时器溢出的中断使能。(R/W)

RTC_TIMER_MAIN_TIMER_INT_ENA 计时器到达目标时刻 0 的中断使能。(R/W)

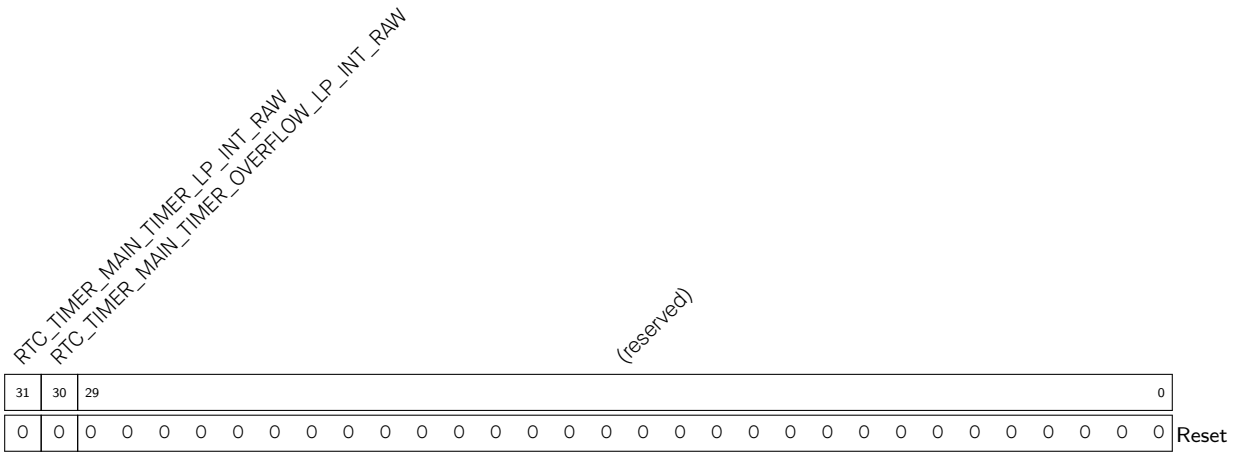
Register 6.13. RTC_TIMER_INT_CLR_REG (0x0034)



RTC_TIMER_OVERFLOW_CLR 48 位计时器溢出中断清除。(WO)

RTC_TIMER_MAIN_TIMER_INT_CLR 计时器到达目标时刻 0 的中断清除。(WO)

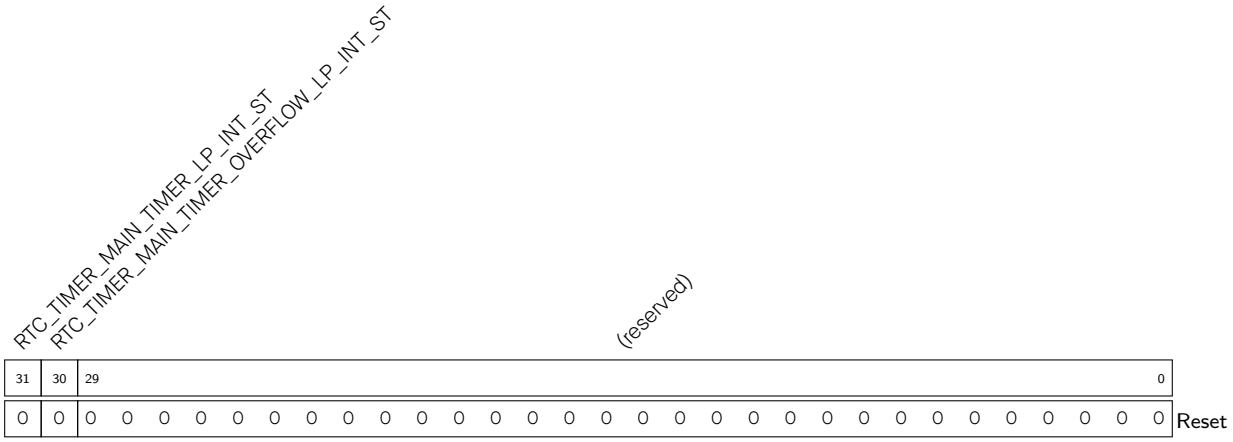
Register 6.14. RTC_TIMER_LP_INT_RAW_REG (0x0038)



RTC_TIMER_MAIN_TIMER_OVERFLOW_LP_INT_RAW 48 位计时器溢出的原始中断状态。(RO)

RTC_TIMER_MAIN_TIMER_LP_INT_RAW 计时器到达目标时刻 1 的原始中断。(RO)

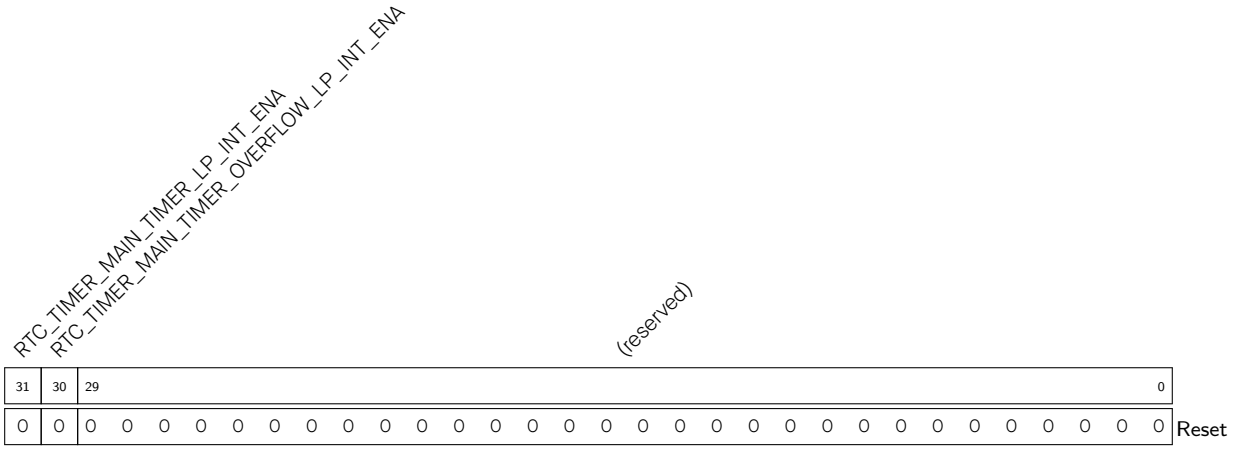
Register 6.15. RTC_TIMER_LP_INT_ST_REG (0x003C)



RTC_TIMER_MAIN_TIMER_OVERFLOW_LP_INT_ST 48 位计时器溢出的状态中断。(RO)

RTC_TIMER_MAIN_TIMER_LP_INT_ST 计时器到达目标时刻 1 的状态中断。(RO)

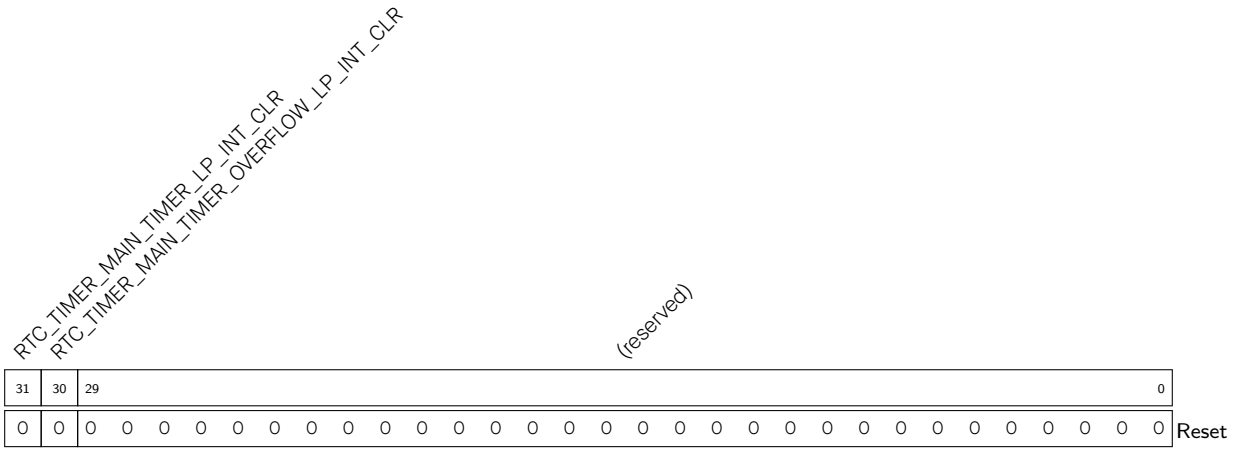
Register 6.16. RTC_TIMER_LP_INT_ENA_REG (0x0040)



RTC_TIMER_MAIN_TIMER_OVERFLOW_LP_INT_ENA 48 位计时器溢出的中断使能。(R/W)

RTC_TIMER_MAIN_TIMER_LP_INT_ENA 计时器到达目标时刻 1 的中断使能。(R/W)

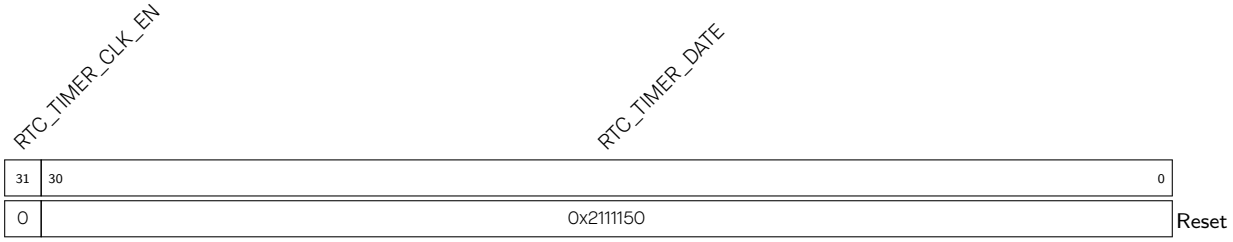
Register 6.17. RTC_TIMER_LP_INT_CLR_REG (0x0044)



RTC_TIMER_MAIN_TIMER_OVERFLOW_LP_INT_CLR 48 位计时器溢出中断清除。(WO)

RTC_TIMER_MAIN_TIMER_LP_INT_CLR 计时器到达目标时刻 1 的中断清除。(WO)

Register 6.18. RTC_TIMER_DATE_REG (0x03FC)



RTC_TIMER_DATE 版本控制寄存器。(R/W)

RTC_TIMER_CLK_EN 寄存器读写时钟使能。(R/W)

第 7 章

辅助调试

7.1 概述

ESP32-S31 辅助调试模块提供一套调试功能，可在软件开发时用于调试，定位问题所在。

7.2 主要特性

辅助调试模块具有以下特性：

- **区域读写监测**：监测高性能双核处理器（High-Performance CPU，HP CPU0 和 HP CPU1）总线在限定存储器地址范围内的读写操作，若发生读写操作则触发中断。
- **栈指针 (SP) 监测**：监测栈指针是否超出限定范围，若超出则产生中断。
- **程序计数器 (PC) 和栈指针 (SP) 记录**：记录 PC 值和 SP 值，以获取上一次 HP CPU*n*（以下，*n*=0，1）复位时的 PC 值和 SP 值。
- **核心锁定 (lockup) 监测**：监测 HP CPU*n* 是否发生核心锁定 (lockup)，并记录第一次和第二次异常发生时的异常原因 (cause)、异常陷阱值 (tval)、指令 PC (iaddr) 和当前特权级别 (priv)。
- **总线访问记录**：记录 HP CPU*n* 数据总线和 DMA 总线的写操作信息。支持写监测，以及按字、半字、字节监测特定写入值；记录总线类型、地址，以及 HP CPU*n* 写操作时的 PC，并按格式存入 HP SRAM。该功能通过总线访问记录 1（TCM_MEM_MONITOR，HP SRAM 或外部 RAM）和总线访问记录 2（PSRAM_MEM_MONITOR，外部 RAM）实现。

7.3 功能描述

7.3.1 区域读写监测

辅助调试模块监测 HP CPU*n* 是否在某段地址范围（即区域）进行读写操作。当总线在监测地址范围内进行读写操作时，触发中断。详情请见章节 2 高性能处理器 *[to be added later]* 中的表 **??**。

当 HP CPU*n* 访问数据空间时，其总线称为**数据总线**；当访问 AHB 外设空间时，其总线称为**外设总线**。辅助调试模块可在数据总线上同时监测两个地址范围的读写操作，分别称为 HP CPU*n* 数据总线区域 0 和 HP CPU*n* 数据总线区域 1；也可在外设总线上监测两个地址范围，称为 HP CPU*n* 外设总线区域 0 和 HP CPU*n* 外设总线区域 1。区域 0 和区域 1 根据需要进行配置，各区域的读、写操作可分别使能。

7.3.2 栈指针监测

辅助调试模块监测栈指针，以防止 HP CPU*n* 栈溢出或错误的压栈出栈。

当 HP CPU*n* 栈指针超出监测区域上边界或低于下边界时，模块记录 PC 指针并产生中断，边界由软件配置。

7.3.3 PC 和 SP 记录

辅助调试模块记录 HP CPU*n* 复位时的 PC 和 SP，供软件读取以进行调试。

7.3.4 核心锁定监测

当 CPU 发生异常后，会跳转到异常处理程序，在第一次异常处理过程中，如果再次触发异常，CPU 就会发生核心锁定。详情请见章节 2 高性能处理器 *[to be added later]* 中的小节 **?? ??**。当 HP CPU*n* 发生核心锁定时，第二次的异常上下文会覆盖第一次异常时的上下文，给软件开发者的调试带来困难。辅助调试模块记录 HP CPU*n* 第一次和第二次异常时的异常原因、异常陷阱值、指令 PC 和当前特权级别，方便开发者调试。

7.3.5 CPU/DMA 总线访问记录

辅助调试模块实时记录 HP CPU*n* 数据总线和 DMA 总线的写操作。当总线在某个范围内发生写操作或写入特定值时，记录总线类型、地址和 PC（仅记录 HP CPU*n* 写操作的 PC）等信息，并按照一定格式存储到 HP SRAM 中。此处提到的地址范围只能位于 HP SRAM 或外部 RAM 的范围内。

7.4 中断

ESP32-S31 的辅助调试模块可以生成 BUS_MONITOR_INTR 中断信号并发送给中断矩阵。这一中断信号由以下辅助调试模块的内部中断源生成（以下，*n*=0，1）：

- BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_RD_INT：当 HP CPU*n* 在 HP CPU*n* 数据总线区域 0 进行读操作时触发。
- BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_WR_INT：当 HP CPU*n* 在 HP CPU*n* 数据总线区域 0 进行写操作时触发。
- BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_RD_INT：当 HP CPU*n* 在 HP CPU*n* 数据总线区域 1 进行读操作时触发。
- BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_WR_INT：当 HP CPU*n* 在 HP CPU*n* 数据总线区域 1 进行写操作时触发。
- BUS_MONITOR_CORE_*n*_AREA_PIF_0_RD_INT：当 HP CPU*n* 在 HP CPU*n* 外设总线区域 0 进行读操作时触发。
- BUS_MONITOR_CORE_*n*_AREA_PIF_0_WR_INT：当 HP CPU*n* 在 HP CPU*n* 外设总线区域 0 进行写操作时触发。
- BUS_MONITOR_CORE_*n*_AREA_PIF_1_RD_INT：当 HP CPU*n* 在 HP CPU*n* 外设总线区域 1 进行读操作时触发。
- BUS_MONITOR_CORE_*n*_AREA_PIF_1_WR_INT：当 HP CPU*n* 在 HP CPU*n* 外设总线区域 1 进行写操作时触发。
- BUS_MONITOR_CORE_*n*_SP_SPILL_MIN_INT：当 HP CPU*n* 栈指针小于 HP CPU*n* 栈指针监测区域的下边界时触发。

- BUS_MONITOR_CORE_*n*_SP_SPILL_MAX_INT：当 HP CPU*n* 栈指针大于 HP CPU*n* 栈指针监测区域的上边界时触发。

说明：
对于中断、中断信号、中断源及其关系的详细描述，可以查看章节 9 中断矩阵 *[to be added later]* > 小节 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 7.6 寄存器列表。

7.5 配置流程

7.5.1 区域监测和栈指针监测配置

区域监测和栈指针监测的配置流程如下（以下，*n*=0，1）。

1. 配置监测区域和栈指针。
   - HP CPU*n* 数据总线区域 0 由 BUS_MONITOR_CORE_*n*_AREA_DRAMO_0_MIN_REG 和 BUS_MONITOR_CORE_*n*_AREA_DRAMO_0_MAX_REG 配置。
   - HP CPU*n* 数据总线区域 1 由 BUS_MONITOR_CORE_*n*_AREA_DRAMO_1_MIN_REG 和 BUS_MONITOR_CORE_*n*_AREA_DRAMO_1_MAX_REG 配置。
   - HP CPU*n* 外设总线区域 0 由 BUS_MONITOR_CORE_*n*_AREA_PIF_0_MIN_REG 和 BUS_MONITOR_CORE_*n*_AREA_PIF_0_MAX_REG 配置。
   - HP CPU*n* 外设总线区域 1 由 BUS_MONITOR_CORE_*n*_AREA_PIF_1_MIN_REG 和 BUS_MONITOR_CORE_*n*_AREA_PIF_1_MAX_REG 配置。
   - HP CPU*n* 栈指针监测区域由 BUS_MONITOR_CORE_*n*_SP_MIN_REG 和 BUS_MONITOR_CORE_*n*_SP_MAX_REG 配置。
2. 配置 BUS_MONITOR_CORE_*n*_INTR_ENA_REG 使能不同模式的中断。
3. 配置 BUS_MONITOR_CORE_*n*_MONTR_ENA_REG 使能不同的监测模式，可同时使能。
4. 读取 BUS_MONITOR_CORE_*n*_INTR_RAW_REG 查询不同模式的原始中断状态。
5. 配置 BUS_MONITOR_CORE_*n*_INTR_CLR_REG 清除不同模式的中断。

若要监测 HP CPU0 数据总线地址 A 到地址 B 范围内的写操作，可通过 HP CPU0 数据总线区域 0 或区域 1 进行监测。以 HP CPU0 数据总线区域 0 为例：

1. 配置 BUS_MONITOR_CORE_0_RCD_PDEBUGEN 为 1，使能 HP CPU0 更新 PC 信号给辅助调试模块。
2. 配置 BUS_MONITOR_CORE_0_AREA_DRAMO_0_MIN_REG 为 A 地址。
3. 配置 BUS_MONITOR_CORE_0_AREA_DRAMO_0_MAX_REG 为 B 地址。
4. 配置 BUS_MONITOR_CORE_0_INTR_ENA_REG bit[1] 使能 HP CPU0 数据总线区域 0 写监测中断。
5. 配置 BUS_MONITOR_CORE_0_MONTR_ENA_REG bit[1] 使能 HP CPU0 数据总线区域 0 写监测。
6. 配置中断矩阵将 BUS_MONITOR_INTR 映射到 HP CPU0 中断（参考章节 9 中断矩阵 *[to be added later]*）。
7. 触发中断后：

- 读取 BUS_MONITOR_CORE_0_INTR_RAW_REG 确认中断源。
- 如果中断源是 HP CPU0 区域监测，读取 BUS_MONITOR_CORE_0_AREA_PC 获取触发中断时的 HP CPU0 PC 值，读取 BUS_MONITOR_CORE_0_AREA_SP 获取触发中断时的 HP CPU0 栈指针。
- 如果中断源是栈指针监测，读取 BUS_MONITOR_CORE_0_SP_PC 获取触发中断时的 HP CPU0 PC 值。
- 配置 BUS_MONITOR_CORE_0_INTR_CLR_REG 对应位为 1 清除中断。

7.5.2 PC 和 SP 记录配置

将 BUS_MONITOR_CORE_0_RCD_PDEBUGEN 配置为 1，以使能 HP CPU0 更新 PC 信号给辅助调试模块。如果 BUS_MONITOR_CORE_0_RCD_RECORDEN 也配置为 1，BUS_MONITOR_CORE_0_RCD_PDEBUGPC_REG 将记录 HP CPU0 的 PC 信号，同时 BUS_MONITOR_CORE_0_RCD_PDEBUGSP_REG 将记录 HP CPU0 的栈指针，否则保留原值。

当 HP CPU0 复位时，BUS_MONITOR_CORE_0_RCD_EN_REG 将被复位，但 BUS_MONITOR_CORE_0_RCD_PDEBUGPC_REG 和 BUS_MONITOR_CORE_0_RCD_PDEBUGSP_REG 不会被复位，因此这两个寄存器将保留 HP CPU0 复位时的 PC 值和栈指针。

HP CPU1 同理。

7.5.3 核心锁定监测配置

配置 BUS_MONITOR_CORE_0_MONTR_ENA_REG 中的 BUS_MONITOR_CORE_0_TRACE_LOCKUP_ENA，使能 HP CPU0 核心锁定监测模式。

当 HP CPU0 发生核心锁定后：

- 读取 BUS_MONITOR_CORE_0_TRACE_LOCKUP_CAUSE_1_REG 可获取 HP CPU0 第一次发生核心锁定时的异常原因。
- 读取 BUS_MONITOR_CORE_0_TRACE_LOCKUP_TVAL_1_REG 可获取 HP CPU0 第一次发生核心锁定时的异常陷阱值。
- 读取 BUS_MONITOR_CORE_0_TRACE_LOCKUP_IADDR_1_REG 可获取 HP CPU0 第一次发生核心锁定时的指令 PC。
- 读取 BUS_MONITOR_CORE_0_TRACE_LOCKUP_PRIV_1_REG 可获取 HP CPU0 第一次发生核心锁定时的特权级别。
- 读取 BUS_MONITOR_CORE_0_TRACE_LOCKUP_CAUSE_0_REG 可获取 HP CPU0 第二次发生核心锁定时的异常原因。
- 读取 BUS_MONITOR_CORE_0_TRACE_LOCKUP_TVAL_0_REG 可获取 HP CPU0 第二次发生核心锁定时的异常陷阱值。
- 读取 BUS_MONITOR_CORE_0_TRACE_LOCKUP_IADDR_0_REG 可获取 HP CPU0 第二次发生核心锁定时的指令 PC。
- 读取 BUS_MONITOR_CORE_0_TRACE_LOCKUP_PRIV_0_REG 可获取 HP CPU0 第二次发生核心锁定时的特权级别。

记录核心锁定信息的寄存器不会随 HP CPU0 复位而复位，记录的信息可以保留供开发者读取进行调试。HP CPU1 同理。

7.5.4 CPU/DMA 总线访问记录配置

7.5.4.1 总线访问记录 1 配置

总线访问记录 1 包括 HP CPU*n* 总线访问记录（HP SRAM 和外部 RAM 监控功能）和 DMA 总线访问记录（HP SRAM 监控功能）。

其配置寄存器的基地址为存储器访问监控 1 (TCM_MEM_MONITOR) 的基地址，具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。由于总线访问记录 1 只用到了 DMA_0、DMA_1 和 DMA_2 通道，因此在寄存器列表中的 DMA_3 相关的寄存器可忽略。配置流程如下：

1. 配置监测地址范围：
   - 配置 MEM_MONITOR_LOG_MIN_REG 和 MEM_MONITOR_LOG_MAX_REG 确定地址范围。对于 HP CPU*n*，监测地址范围需在 HP SRAM (0x2F00_0000 ~ 0x2F07_FFFF) 或外部 RAM (0x5000_0000 ~ 0x53FF_FFFF) 的地址区间内；对于 DMA，监测地址范围需在 HP SRAM (0x2F00_0000 ~ 0x2F07_FFFF) 的地址区间内。
   - 可通过配置 MEM_MONITOR_LOG_MON_ADDR_UPDATE_0_REG 和 MEM_MONITOR_LOG_MON_ADDR_UPDATE_1_REG 的对应寄存器域将当前 MEM_MONITOR_LOG_MIN_REG 和 MEM_MONITOR_LOG_MAX_REG 内配置的地址范围同步为对应的 HP CPU*n* 或 DMA 总线的监测范围。
2. 配置监测模式 (MEM_MONITOR_LOG_MODE)：
   - 1：写监测（监测总线写操作）
   - 2：字 (word) 监测（监测总线写特定 word）
   - 4：半字 (halfword) 监测（监测总线写特定 halfword）
   - 8：字节 (byte) 监测（监测总线写特定 byte）
3. 配置需要监测的特殊值：
   - 在 word 监测模式下，MEM_MONITOR_LOG_CHECK_DATA_REG 为监测的特定 word。
   - 在 halfword 监测模式下，MEM_MONITOR_LOG_CHECK_DATA_REG[15:0] 为监测的特定 halfword。
   - 在 byte 监测模式下，MEM_MONITOR_LOG_CHECK_DATA_REG[7:0] 为监测的特定 byte。
   - MEM_MONITOR_LOG_DATA_MASK_REG 用于屏蔽 MEM_MONITOR_LOG_CHECK_DATA_REG 的相应 byte。当某一 byte 被屏蔽，表示该 byte 可为任意值。例如，在 word 监测模式下，若 MEM_MONITOR_LOG_CHECK_DATA_REG 配置为 0x01020304，MEM_MONITOR_LOG_DATA_MASK_REG 配置为 0x1，此时只要总线写 0x010203XX，都会被记录。
4. 配置记录信息的存储地址范围：
   - MEM_MONITOR_LOG_MEM_START_REG 和 MEM_MONITOR_LOG_MEM_END_REG 为存储地址范围配置寄存器。记录信息的存储地址范围为 0x2F00_0000 ~ 0x2F07_FFFF。
   - 置位 MEM_MONITOR_LOG_MEM_ADDR_UPDATE_REG，使 MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG 更新为 MEM_MONITOR_LOG_MEM_START_REG。
   - 配置辅助调试模块对 HP SRAM 的使用权限。只有开启辅助调试模块对 HP SRAM 的使用权限，才能访问 HP SRAM。关于如何配置，详情见章节 14 *权限控制 (PMS) [to be added later]*。

5. 配置记录信息的存储模式，分为 loop 模式和非 loop 模式。

   - 在 loop 模式下，记录信息循环写入配置地址范围。当写到结束地址时，回到起始地址开始写，覆盖之前记录的信息。置位 MEM_MONITOR_LOG_MEM_LOOP_ENABLE 使能 loop 模式。
     例如，存储记录信息的地址范围为 0 ~ 4 ，在总线访问过程中进行 10 次写操作。第 5 次写操作写到地址 4 后，第 6 次写操作会回到地址 0 开始写，第 6 ~ 10 次写操作会覆盖之前的第 1 ~ 5 次写操作的数据。
   - 在非 loop 模式下，当记录信息写到结束地址后，会停留在结束地址，不再写入记录信息，不会覆盖最初的记录信息，并丢弃后续的记录信息。清零 MEM_MONITOR_LOG_MEM_LOOP_ENABLE 使能非 loop 模式。
     例如，存储记录信息的地址范围为 0 ~ 4 ，在总线访问过程中进行 10 次写操作。第 5 次写操作写到地址 4 后，第 6 ~ 10 次写操作的地址会停留在地址 4，但不再进行写操作，因此地址范围 0 ~ 4 保留前 5 次写操作的信息，并丢弃第 6 ~ 10 次写操作的信息。

6. 配置总线使能

   - 配置 MEM_MONITOR_LOG_CORE_ENA、MEM_MONITOR_LOG_DMA_0_ENA、MEM_MONITOR_LOG_DMA_1_ENA 和 MEM_MONITOR_LOG_DMA_2_ENA 使能 HP CPU*n*、DMA_0、DMA_1 和 DMA_2 总线访问记录，可同时使能。

辅助调试模块将记录信息缓存到内部 buffer A 中，然后从 buffer 中取出记录信息存储到配置的存储区域。当监测行为连续触发时，产生的记录数据包可能会填满 buffer，无法缓存新的记录数据包，此时模块会舍弃无法及时缓存到 buffer 的记录数据包，并在 buffer 不满时缓存一个 LOST 数据包作为替代。此时仅能获取 LOST 数据包生成时所舍弃的记录数据包的总线类型，但无法得知之前舍弃了多少个记录数据包。

当总线访问记录结束后，需要从内存中读取记录数据并进行解码。记录数据包含七种包格式：HP CPU0 数据包、HP CPU1 数据包、DMA_0 通道 0 数据包、DMA_0 通道 1 数据包、DMA_1 通道 0 数据包、DMA_2 通道 0 数据包和 LOST 数据包，七种包格式如表 7.5-1、7.5-2 和 7.5-3 所示：

表 7.5-1. HP CPU*n* 包格式

Bit[63:33]	Bit[32]	Bit[31:4]	Bit[3:1]	Bit[0]
pc_offset	anchored(1)	addr_offset	format	anchored(0)

表 7.5-2. DMA 包格式

Bit[31:5]	Bit[4]	Bit[3:1]	Bit[0]
addr_offset	保留	format	anchored(0)

表 7.5-3. LOST 包格式

Bit[31:12]	Bit[11:4]	Bit[3:1]	Bit[0]
保留	lost_source	format	anchored(0)

从包格式可以看出，HP CPU*n* 数据包共 64 位，其他数据包均为 32 位。以下为各个域的含义：

- **format** 表示数据包的类型：
  - 0：HP CPU0 数据包
  - 1：HP CPU1 数据包

  - 2：DMA_0 通道 0 数据包
  - 3：DMA_0 通道 1 数据包
  - 4：DMA_1 通道 0 数据包
  - 5：DMA_2 通道 0 数据包
  - 6：LOST 数据包
- **pc_offset** 记录 HP CPU 的 PC 指针偏移量，实际 PC = pc_offset + 0x0000_0000。
- **addr_offset** 记录此次写操作的地址偏移，实际地址 = addr_offset + {MEM_MONITOR_LOG_MIN_REG[31:2], 2'b0}。
- **lost_source** 记录 LOST 数据包生成时舍弃的数据包：
  - Bit[5:4]：
    * 0：未舍弃 HP CPU0 数据包
    * 3：舍弃 HP CPU0 数据包
    * 其他值：保留
  - Bit[7:6]：
    * 0：未舍弃 HP CPU1 数据包
    * 3：舍弃 HP CPU1 数据包
    * 其他值：保留
  - Bit[8]：
    * 0：未舍弃 DMA_0 通道 0 数据包
    * 1：舍弃 DMA_0 通道 0 数据包
  - Bit[9]：
    * 0：未舍弃 DMA_0 通道 1 数据包
    * 1：舍弃 DMA_0 通道 1 数据包
  - Bit[10]：
    * 0：未舍弃 DMA_1 通道 0 数据包
    * 1：舍弃 DMA_1 通道 0 数据包
  - Bit[11]：
    * 0：未舍弃 DMA_2 通道 0 数据包
    * 1：舍弃 DMA_2 通道 0 数据包
- **anchored** 表示该 32 位数据在数据包中的位置：
  - 0：该 32 位数据是数据包的低 32 位
  - 1：该 32 位数据是数据包的高 32 位

模块内部 buffer 的数据宽度是 32 位。若同时使能 HP CPU0、HP CPU1、DMA_0 通道 0、DMA_0 通道 1、DMA_1 通道 0 和 DMA_2 通道 0 总线访问记录，并且在某一时刻同时生成记录数据，则先将 HP CPU0 数据包缓存到 buffer 中，再缓存 HP CPU1 数据包，接着是 DMA_0 通道 0 数据包、DMA_0 通道 1 数据包、DMA_1 通道 0 数据包，最后缓存 DMA_2 通道 0 数据包。辅助调试模块会自动将缓存数据从 buffer 取出，并将取出的数据以 32 位数据宽度存入指定范围内存中。

当启用 loop 模式时，若在配置的存储地址范围内循环若干次，可能存在残留数据干扰解析，即 HP CPU*n* 数据包的低 32 位数据被覆盖，使得高 32 位数据成为残留数据。因此，确定第一个有效数据包的位置时，必须过滤可能存在的残留数据。读取 MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG 可确定数据包的起始位置，检查数据包 anchored 位的数值，0 则保留，1 则舍弃。

数据包解析流程如下：

- 读取 MEM_MONITOR_LOG_MEM_FULL_FLAG 确定数据是否溢出配置范围。
  - 如果未溢出，则数据读取的地址范围为 MEM_MONITOR_LOG_MEM_START_REG ~ MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG – 4；
  - 若溢出且开启了 loop 模式，则数据读取的地址范围为 MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG ~ MEM_MONITOR_LOG_MEM_END_REG 和 MEM_MONITOR_LOG_MEM_START_REG ~ MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG – 4；
  - 若溢出且未开启 loop 模式，则数据读取的地址范围为 MEM_MONITOR_LOG_MEM_START_REG ~ MEM_MONITOR_LOG_MEM_END_REG。
- 从起始地址处开始读取数据并解析，每次读取 32 位。

数据解析完成之后需要通过配置 MEM_MONITOR_CLR_LOG_MEM_FULL_FLAG 为 1 清除 MEM_MONITOR_LOG_MEM_FULL_FLAG 标志位。

7.5.4.2 总线访问记录 2 配置

总线访问记录 2 包括 DMA 总线访问记录（外部 RAM 监控功能）。

其配置寄存器的基地址为存储器访问监控 2 (PSRAM_MEM_MONITOR) 的基地址，具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。由于总线访问记录 2 只用到了 DMA_0 通道，因此在寄存器列表中的 DMA_1、DMA_2、DMA_3 相关的寄存器可忽略。配置流程如下：

1. 配置监测地址范围：
   - 配置 MEM_MONITOR_LOG_MIN_REG 和 MEM_MONITOR_LOG_MAX_REG 确定地址范围。监测地址范围需在 0x5000_0000 ~ 0x53FF_FFFF 之间。
   - 可通过配置 MEM_MONITOR_LOG_MON_ADDR_UPDATE_0_REG 和 MEM_MONITOR_LOG_MON_ADDR_UPDATE_1_REG 的对应寄存器域将当前 MEM_MONITOR_LOG_MIN_REG 和 MEM_MONITOR_LOG_MAX_REG 内配置的地址范围同步为对应的 DMA_0 总线的监测范围。
2. 配置 MEM_MONITOR_LOG_MODE 选择监测模式：
   - 1：写监测（监测总线写操作）
   - 2：字 (word) 监测（监测总线写特定 word）
   - 4：半字 (halfword) 监测（监测总线写特定 halfword）

- 8：字节 (byte) 监测（监测总线写特定 byte）

3. 配置需要监测的特殊值：
   - 在 word 监测模式下，MEM_MONITOR_LOG_CHECK_DATA_REG 为监测的特定 word。
   - 在 halfword 监测模式下，MEM_MONITOR_LOG_CHECK_DATA_REG[15:0] 为监测的特定 halfword。
   - 在 byte 监测模式下，MEM_MONITOR_LOG_CHECK_DATA_REG[7:0] 为监测的特定 byte。
   - MEM_MONITOR_LOG_DATA_MASK_REG 用于屏蔽 MEM_MONITOR_LOG_CHECK_DATA_REG 的相应 byte。当某一 byte 被屏蔽时，该 byte 可为任意值。例如，在 word 监测中，若 MEM_MONITOR_LOG_CHECK_DATA_REG 配置为 0x01020304，MEM_MONITOR_LOG_DATA_MASK_REG 配置为 0x1，则只要总线写 0x010203XX 都会被记录。
4. 配置记录信息的存储地址范围：
   - MEM_MONITOR_LOG_MEM_START_REG 和 MEM_MONITOR_LOG_MEM_END_REG 为存储地址范围配置寄存器。记录信息的存储地址范围为 0x2F00_0000 ~ 0x2F07_FFFF。
   - 置位 MEM_MONITOR_LOG_MEM_ADDR_UPDATE_REG，使 MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG 更新为 MEM_MONITOR_LOG_MEM_START_REG。
   - 配置辅助调试模块对 HP SRAM 的使用权限。只有开启辅助调试模块对 HP SRAM 的使用权限才能访问 HP SRAM。关于如何配置，详情见章节 14 权限控制 *(PMS) [to be added later]*。
5. 配置记录信息的存储模式，分为 loop 模式和非 loop 模式。
   - 在 loop 模式下，记录信息循环写入配置地址范围。当写到结束地址时，回到起始地址开始写，覆盖之前记录的信息。置位 MEM_MONITOR_LOG_MEM_LOOP_ENABLE 启用 loop 模式。
   - 在非 loop 模式下，当记录信息写到结束地址后，停留在结束地址，不再写入记录信息，不会覆盖最开始的记录信息，并丢弃后续记录信息。清零 MEM_MONITOR_LOG_MEM_LOOP_ENABLE 启用非 loop 模式。
   - 示例参考小节 7.5.4.1 的步骤 5。
6. 配置 MEM_MONITOR_LOG_DMA_0_ENA 使能 DMA_0 通道组总线访问记录。

辅助调试模块将 DMA 记录信息缓存到内部 buffer B，然后从 buffer 中取出记录信息并存储到配置的存储区域。当监测行为连续触发时，产生的记录数据包可能使 buffer 满，无法缓存新的记录数据包，此时模块会舍弃无法及时缓存的记录数据包，并在 buffer 不满时缓存一个 DMA LOST 数据包作为替代。此时仅能知道 DMA LOST 数据包生成时所舍弃的记录数据包的总线类型，但无法得知之前舍弃了多少个记录数据包。

当总线访问记录结束后，需要从内存中读取记录数据并进行解码。记录数据仅有两种包格式：DMA_0 数据包和 DMA LOST 数据包。DMA_0 数据包对应 DMA_0 通道组数据总线记录信息，两种包格式如表 7.5-4 和 7.5-5 所示：

表 7.5-4. DMA_0 包格式

Bit[31:6]	Bit[5:2]	Bit[1]	Bit[0]
addr_offset	保留	format	anchored(0)

从包格式可以看出，DMA_0 和 DMA LOST 数据包均为 32 位。以下为各个域的含义：

- **format** 表示数据包的类型：

表 7.5-5. DMA LOST 包格式

Bit[31:3]	Bit[2]	Bit[1]	Bit[0]
保留	lost_source	format	anchored(0)

  - 0：DMA_0 数据包
  - 1：DMA LOST 数据包
- **addr_offset** 记录此次写操作的地址偏移，实际地址 = addr_offset + {MEM_MONITOR_LOG_MIN_REG[31:2]，2'b0}。
- **lost_source** 记录 DMA LOST 数据包生成时舍弃的数据包：
  - 0：未舍弃 DMA_0 数据包
  - 1：舍弃了 DMA_0 数据包
- **anchored** 表示该 32 位数据在数据包中的位置：
  - 0：该 32 位数据是数据包的低 32 位
  - 1：该 32 位数据是数据包的高 32 位

模块内部 buffer 的数据宽度为 32 位。若使能 DMA_0 通道组总线访问记录，并在某一时刻生成记录数据，DMA_0 的数据包会缓存到 buffer 中。辅助调试模块会自动将缓存数据从 buffer 取出，并将取出的数据以 32 位数据宽度存入指定范围内存。

数据包解析流程如下：

- 读取 MEM_MONITOR_LOG_MEM_FULL_FLAG 确定数据是否溢出配置范围。
  - 如果未溢出，则数据读取的地址范围为 MEM_MONITOR_LOG_MEM_START_REG ~ MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG – 4；
  - 若溢出并且开启了 loop 模式，则数据读取的地址范围为 MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG ~ MEM_MONITOR_LOG_MEM_END_REG 和 MEM_MONITOR_LOG_MEM_START_REG ~ MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG – 4；
  - 若溢出并且未开启 loop 模式，则数据读取的地址范围为 MEM_MONITOR_LOG_MEM_START_REG ~ MEM_MONITOR_LOG_MEM_END_REG。
- 从起始地址处开始读取数据并解析，每次读取 32 位。

数据解析完成之后需要通过配置 MEM_MONITOR_CLR_LOG_MEM_FULL_FLAG 为 1 清除 MEM_MONITOR_LOG_MEM_FULL_FLAG 标志位。

7.6 寄存器列表

本小节中**总线记录配置寄存器**的地址为相对于存储器访问监控 1 (TCM_MEM_MONITOR) 基地址和存储器访问监控 2 (PSRAM_MEM_MONITOR) 基地址的地址偏移量（相对地址），详见小节 7.6.1，其它寄存器的所有地址均为相对于总线访问监控 (BUS_MONITOR) 基地址的地址偏移量（相对地址），详见小节 7.6.2，具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

7.6.1 总线记录配置寄存器列表

名称	描述	地址	访问
总线记录配置寄存器			
MEM_MONITOR_LOG_SETTING_REG	总线访问记录配置寄存器	0x0000	R/W
MEM_MONITOR_LOG_SETTING1_REG	总线访问记录配置寄存器	0x0004	R/W
MEM_MONITOR_LOG_CHECK_DATA_REG	总线访问监测数据配置寄存器	0x0008	R/W
MEM_MONITOR_LOG_DATA_MASK_REG	总线访问监测数据屏蔽配置寄存器	0x000C	R/W
MEM_MONITOR_LOG_MIN_REG	总线访问监测地址下边界配置寄存器	0x0010	R/W
MEM_MONITOR_LOG_MAX_REG	总线访问监测地址上边界配置寄存器	0x0014	R/W
MEM_MONITOR_LOG_MON_ADDR_UPDATE_0_REG	HP CPU 总线访问监测范围更新寄存器	0x0018	WT
MEM_MONITOR_LOG_MON_ADDR_UPDATE_1_REG	DMA 总线访问监测范围更新寄存器	0x001C	WT
MEM_MONITOR_LOG_MEM_START_REG	记录数据写入内存的起始地址	0x0020	R/W
MEM_MONITOR_LOG_MEM_END_REG	记录数据写入内存的结束地址	0x0024	R/W
MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG	表示下一次写内存的写地址	0x0028	RO
MEM_MONITOR_LOG_MEM_ADDR_UPDATE_REG	更新下一次写内存的写地址为记录数据写入内存的起始地址	0x002C	WT
MEM_MONITOR_LOG_MEM_FULL_FLAG_REG	记录溢出状态寄存器	0x0030	varies
时钟门控控制寄存器			
MEM_MONITOR_CLOCK_GATE_REG	寄存器时钟门控控制	0x0034	R/W
版本寄存器			
MEM_MONITOR_DATE_REG	版本寄存器	0x03FC	R/W

GoBack

7.6.2 其他寄存器列表

名称	描述	地址	访问
监测配置寄存器（以下，*n*=0，1）			
BUS_MONITOR_CORE_*n*_MONTR_ENA_REG	HP CPU*n* 监测使能配置寄存器	0x0000+0x88×n	R/W
BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_MIN_REG	HP CPU*n* 数据总线区域 0 监测地址下边界配置寄存器	0x0010+0x88×n	R/W
BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_MAX_REG	HP CPU*n* 数据总线区域 0 监测地址上边界配置寄存器	0x0014+0x88×n	R/W
BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_MIN_REG	HP CPU*n* 数据总线区域 1 监测地址下边界配置寄存器	0x0018+0x88×n	R/W
BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_MAX_REG	HP CPU*n* 数据总线区域 1 监测地址上边界配置寄存器	0x001C+0x88×n	R/W
BUS_MONITOR_CORE_*n*_AREA_PIF_0_MIN_REG	HP CPU*n* 外设总线区域 0 监测地址下边界配置寄存器	0x0020+0x88×n	R/W
BUS_MONITOR_CORE_*n*_AREA_PIF_0_MAX_REG	HP CPU*n* 外设总线区域 0 监测地址上边界配置寄存器	0x0024+0x88×n	R/W
BUS_MONITOR_CORE_*n*_AREA_PIF_1_MIN_REG	HP CPU*n* 外设总线区域 1 监测地址下边界配置寄存器	0x0028+0x88×n	R/W
BUS_MONITOR_CORE_*n*_AREA_PIF_1_MAX_REG	HP CPU*n* 外设总线区域 1 监测地址上边界配置寄存器	0x002C+0x88×n	R/W
BUS_MONITOR_CORE_*n*_AREA_PC_REG	HP CPU*n* 区域监测 PC 状态寄存器	0x0030+0x88×n	RO
BUS_MONITOR_CORE_*n*_AREA_SP_REG	HP CPU*n* 区域监测栈指针状态寄存器	0x0034+0x88×n	RO
BUS_MONITOR_CORE_*n*_SP_MIN_REG	HP CPU*n* 栈指针监测地址下边界配置寄存器	0x0038+0x88×n	R/W
BUS_MONITOR_CORE_*n*_SP_MAX_REG	HP CPU*n* 栈指针监测地址上边界配置寄存器	0x003C+0x88×n	R/W
BUS_MONITOR_CORE_*n*_SP_PC_REG	HP CPU*n* 栈指针监测 PC 状态寄存器	0x0040+0x88×n	RO
中断配置寄存器（以下，*n*=0，1）			
BUS_MONITOR_CORE_*n*_INTR_RAW_REG	HP CPU*n* 监测原始中断状态寄存器	0x0004+0x88×n	RO
BUS_MONITOR_CORE_*n*_INTR_ENA_REG	HP CPU*n* 监测中断使能配置寄存	0x0008+0x88×n	R/W
BUS_MONITOR_CORE_*n*_INTR_CLR_REG	HP CPU*n* 监测清除中断配置寄存器	0x000C+0x88×n	WT
PC 和 SP 记录配置寄存器			
BUS_MONITOR_CORE_0_RCD_EN_REG	HP CPU0 PC 和 SP 记录使能配置寄存器	0x0044	R/W
BUS_MONITOR_CORE_1_RCD_EN_REG	HP CPU1 PC 和 SP 记录使能配置寄存器	0x00CC	R/W
PC 和 SP 记录状态寄存器			
BUS_MONITOR_CORE_0_RCD_PDEBUGPC_REG	HP CPU0 PC 记录寄存器	0x0048	RO
BUS_MONITOR_CORE_0_RCD_PDEBUGSP_REG	HP CPU0 栈指针记录寄存器	0x004C	RO
BUS_MONITOR_CORE_1_RCD_PDEBUGPC_REG	HP CPU1 PC 记录寄存器	0x00D0	RO

名称	描述	地址	访问
BUS_MONITOR_CORE_1_RCD_PDEBUGSP_REG	HP CPU1 栈指针记录寄存器	0x00D4	RO
CPU 状态寄存器			
BUS_MONITOR_CORE_0_LASTPC_BEFORE_EXCEPTION_REG	HP CPU0 异常前的最后一条指令的 PC	0x0060	RO
BUS_MONITOR_CORE_0_DEBUG_MODE_REG	HP CPU0 处于调试模式的标志寄存器	0x0064	RO
BUS_MONITOR_CORE_1_LASTPC_BEFORE_EXCEPTION_REG	HP CPU1 异常前的最后一条指令的 PC	0x00E8	RO
BUS_MONITOR_CORE_1_DEBUG_MODE_REG	HP CPU1 处于调试模式的标志寄存器	0x00EC	RO
核心锁定记录状态寄存器（以下，*n*=0，1）			
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_CAUSE_0_REG	HP CPU*n* TRACE 第二次核心锁定异常原因记录寄存器	0x0068+0x88×n	RO
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_TVAL_0_REG	HP CPU*n* TRACE 第二次核心锁定异常陷阱值记录寄存器	0x006C+0x88×n	RO
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_IADDR_0_REG	HP CPU*n* TRACE 第二次核心锁定指令 PC 记录寄存器	0x0070+0x88×n	RO
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_PRIV_0_REG	HP CPU*n* TRACE 第二次核心锁定当前特权级别记录寄存器	0x0074+0x88×n	RO
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_CAUSE_1_REG	HP CPU*n* TRACE 首次核心锁定异常原因记录寄存器	0x0078+0x88×n	RO
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_TVAL_1_REG	HP CPU*n* TRACE 首次核心锁定异常陷阱值记录寄存器	0x007C+0x88×n	RO
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_IADDR_1_REG	HP CPU*n* TRACE 首次核心锁定指令 PC 记录寄存器	0x0080+0x88×n	RO
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_PRIV_1_REG	HP CPU*n* TRACE 首次核心锁定当前特权级别记录寄存器	0x0084+0x88×n	RO
时钟控制寄存器			
BUS_MONITOR_CLOCK_GATE_REG	寄存器时钟控制	0x0114	R/W
版本寄存器			
BUS_MONITOR_DATE_REG	版本寄存器	0x03FC	R/W

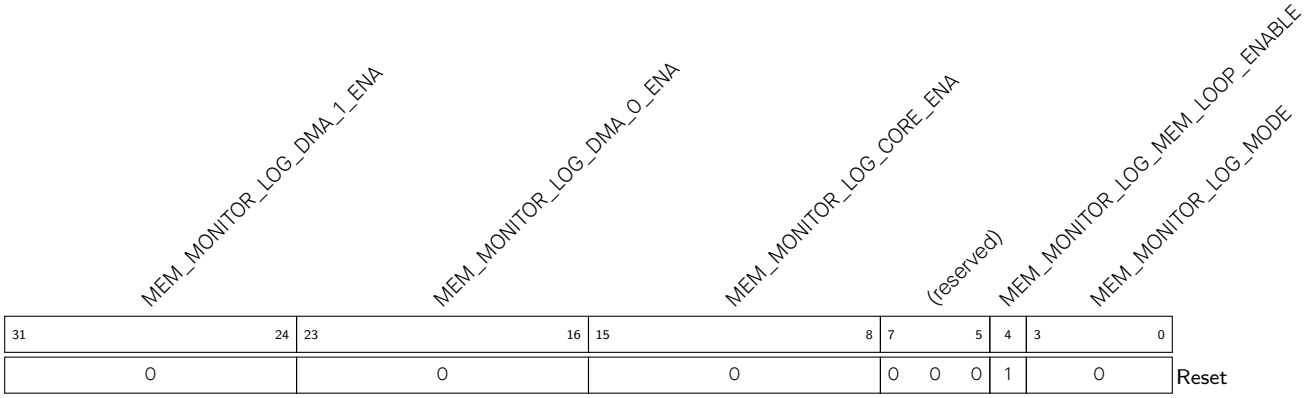
7.7 寄存器

本小节中**总线记录配置寄存器**的地址为相对于存储器访问监控 1 (TCM_MEM_MONITOR) 基地址和存储器访问监控 2 (PSRAM_MEM_MONITOR) 基地址的地址偏移量（相对地址），详见小节 7.7.1，其它寄存器的所有地址均为相对于总线访问监控 (BUS_MONITOR) 基地址的地址偏移量（相对地址），详见小节 7.7.2，具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

7.7.1 总线记录配置寄存器

Register 7.1. MEM_MONITOR_LOG_SETTING_REG (0x0000)



MEM_MONITOR_LOG_MODE 配置监测模式。

1：使能写监测
2：使能 word 监测
4：使能 halfword 监测
8：使能 byte 监测
其他值：无效
(R/W)

MEM_MONITOR_LOG_MEM_LOOP_ENABLE 配置写内存的存储模式。

0：非 loop 模式
1：loop 模式
(R/W)

MEM_MONITOR_LOG_CORE_ENA 配置是否使能 HP CPU 总线访问记录。

0：不使能
1：使能
其他值：无效
(R/W)

见下页...

Register 7.1. MEM_MONITOR_LOG_SETTING_REG (0x0000)

接上页...

MEM_MONITOR_LOG_DMA_0_ENA 配置是否使能 DMA_0 总线访问记录。

0：不使能

1：使能

其他值：无效

(R/W)

MEM_MONITOR_LOG_DMA_1_ENA 配置是否使能 DMA_1 总线访问记录。

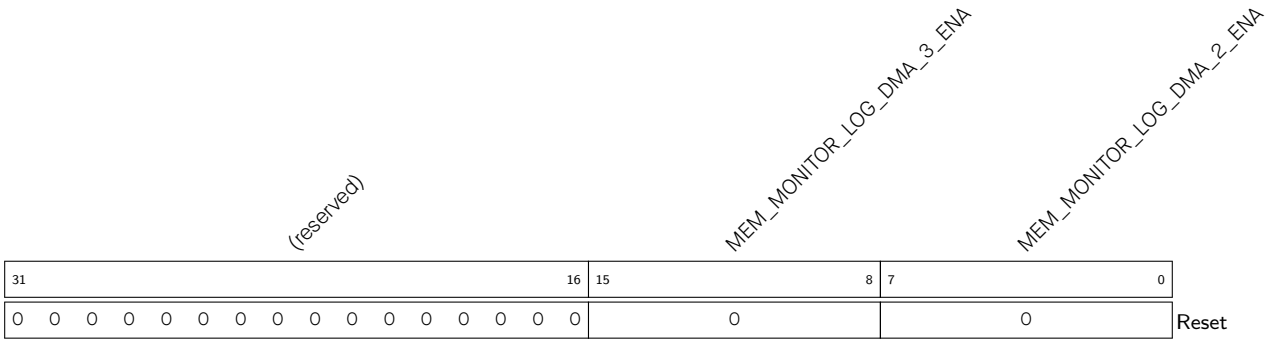
0：不使能

1：使能

其他值：无效

(R/W)

Register 7.2. MEM_MONITOR_LOG_SETTING1_REG (0x0004)



MEM_MONITOR_LOG_DMA_2_ENA 配置是否使能 DMA_2 总线访问记录。

0：不使能

1：使能

其他值：无效

(R/W)

MEM_MONITOR_LOG_DMA_3_ENA 配置是否使能 DMA_3 总线访问记录。

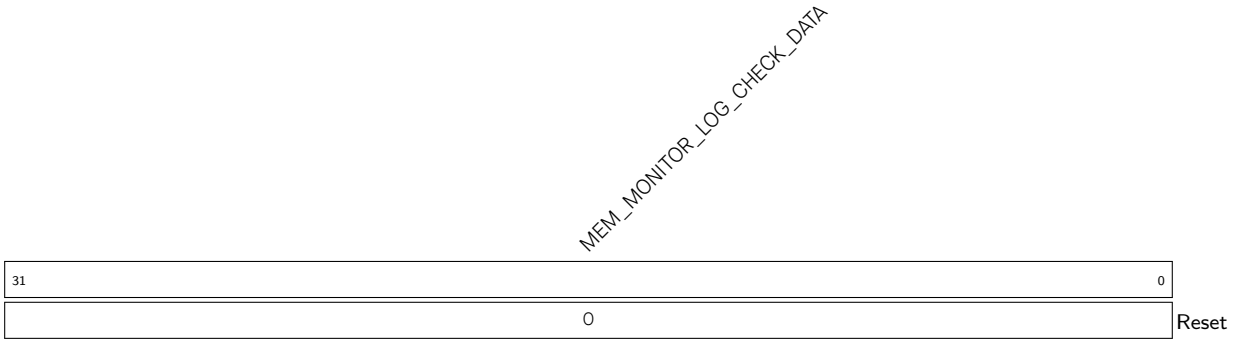
0：不使能

1：使能

其他值：无效

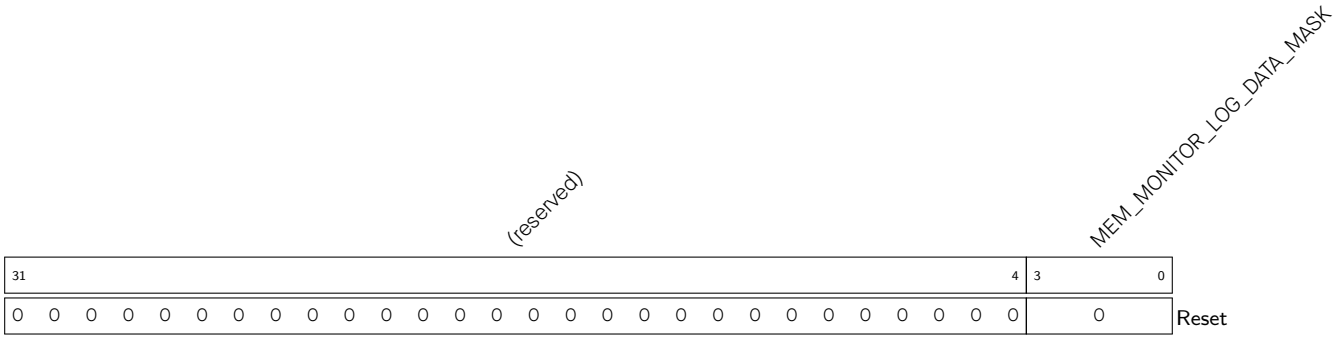
(R/W)

Register 7.3. MEM_MONITOR_LOG_CHECK_DATA_REG (0x0008)



MEM_MONITOR_LOG_CHECK_DATA 配置总线访问监测的特殊值。(R/W)

Register 7.4. MEM_MONITOR_LOG_DATA_MASK_REG (0x000C)



MEM_MONITOR_LOG_DATA_MASK 配置是否屏蔽 MEM_MONITOR_LOG_CHECK_DATA_REG 的相应字节。

1：屏蔽最低位字节。

2：屏蔽第二个低位字节。

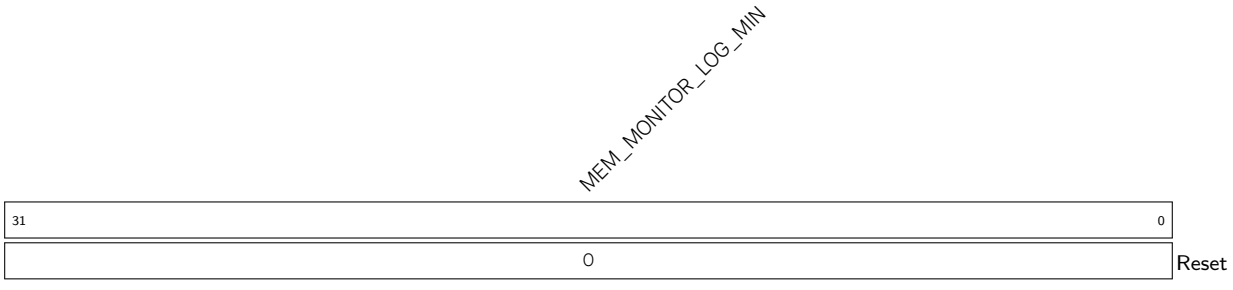
4：屏蔽第二个高位字节。

8：屏蔽最高位字节。

其他值：无效

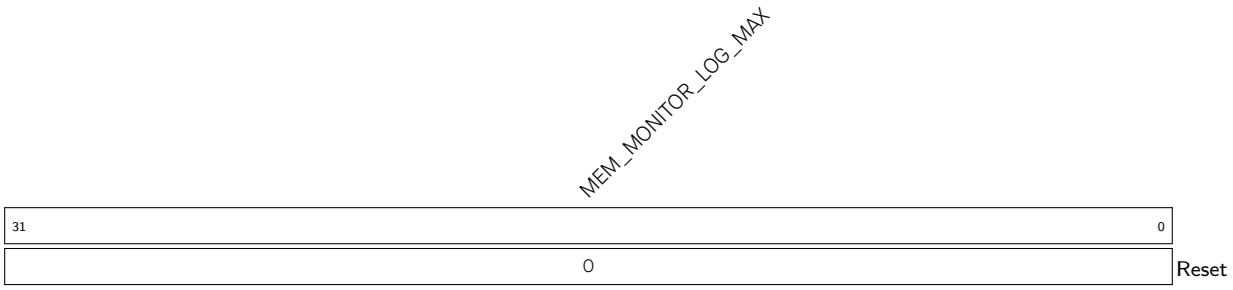
(R/W)

Register 7.5. MEM_MONITOR_LOG_MIN_REG (0x0010)



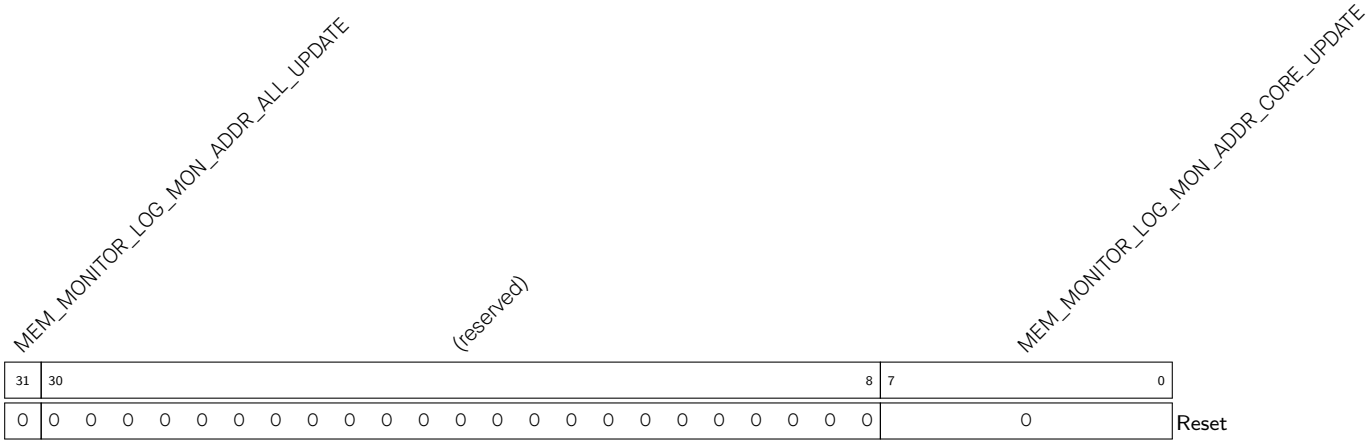
MEM_MONITOR_LOG_MIN 配置监测地址的下边界。(R/W)

Register 7.6. MEM_MONITOR_LOG_MAX_REG (0x0014)



MEM_MONITOR_LOG_MAX 配置监测地址的上边界。(R/W)

Register 7.7. MEM_MONITOR_LOG_MON_ADDR_UPDATE_0_REG (0x0018)



MEM_MONITOR_LOG_MON_ADDR_CORE_UPDATE 配置是否将 MEM_MONITOR_LOG_MIN_REG 到 MEM_MONITOR_LOG_MAX_REG 的地址更新为 HP CPU 总线的监测地址空间。

0：不更新

1：更新

其他值：无效 (WT)

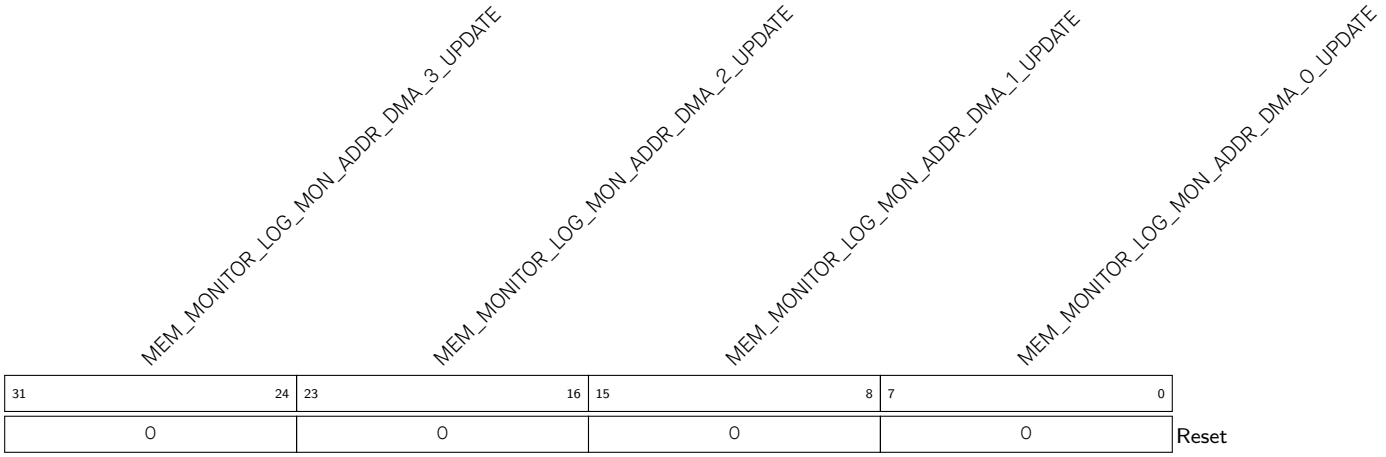
MEM_MONITOR_LOG_MON_ADDR_ALL_UPDATE 配置是否将 MEM_MONITOR_LOG_MIN_REG 到 MEM_MONITOR_LOG_MAX_REG 的地址更新为所有主机的监测地址空间。

0：不更新

1：更新

(WT)

Register 7.8. MEM_MONITOR_LOG_MON_ADDR_UPDATE_1_REG (0x001C)



MEM_MONITOR_LOG_MON_ADDR_DMA_0_UPDATE 配置是否将 MEM_MONITOR_LOG_MIN_REG 到 MEM_MONITOR_LOG_MAX_REG 的地址更新为 DMA_0 总线的监测地址。

0：不更新

1：更新

其他值：无效

(WT)

MEM_MONITOR_LOG_MON_ADDR_DMA_1_UPDATE 配置是否将 MEM_MONITOR_LOG_MIN_REG 到 MEM_MONITOR_LOG_MAX_REG 的地址更新为 DMA_1 总线的监测地址空间。

0：不更新

1：更新

其他值：无效

(WT)

MEM_MONITOR_LOG_MON_ADDR_DMA_2_UPDATE 配置是否将 MEM_MONITOR_LOG_MIN_REG 到 MEM_MONITOR_LOG_MAX_REG 的地址更新为 DMA_2 总线的监测地址空间。

0：不更新

1：更新

其他值：无效

(WT)

MEM_MONITOR_LOG_MON_ADDR_DMA_3_UPDATE 配置是否将 MEM_MONITOR_LOG_MIN_REG 到 MEM_MONITOR_LOG_MAX_REG 的地址更新为 DMA_3 总线的监测地址空间。

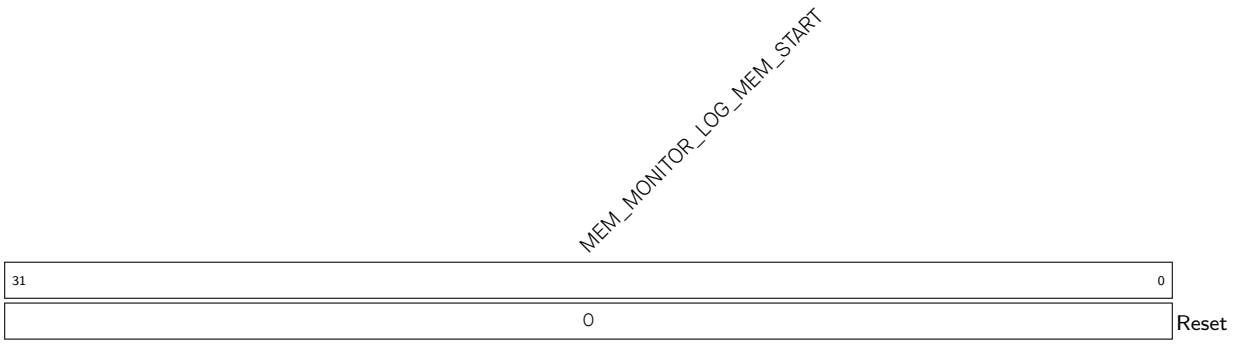
0：不更新

1：更新

其他值：无效

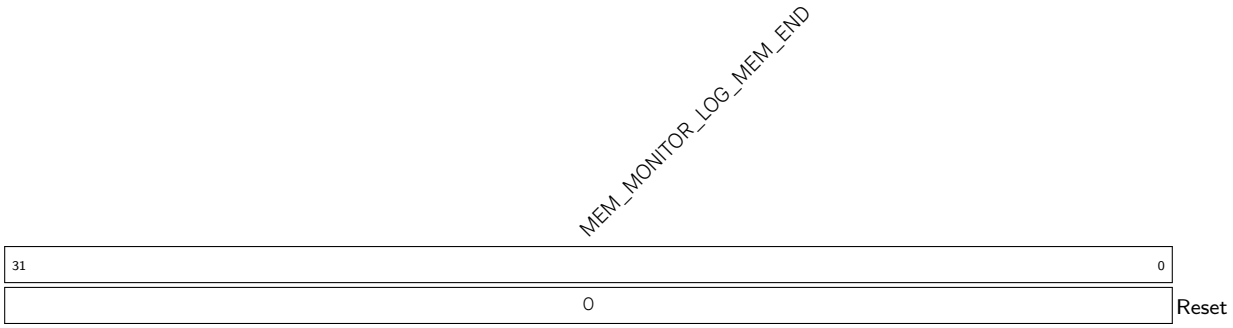
(WT)

Register 7.9. MEM_MONITOR_LOG_MEM_START_REG (0x0020)



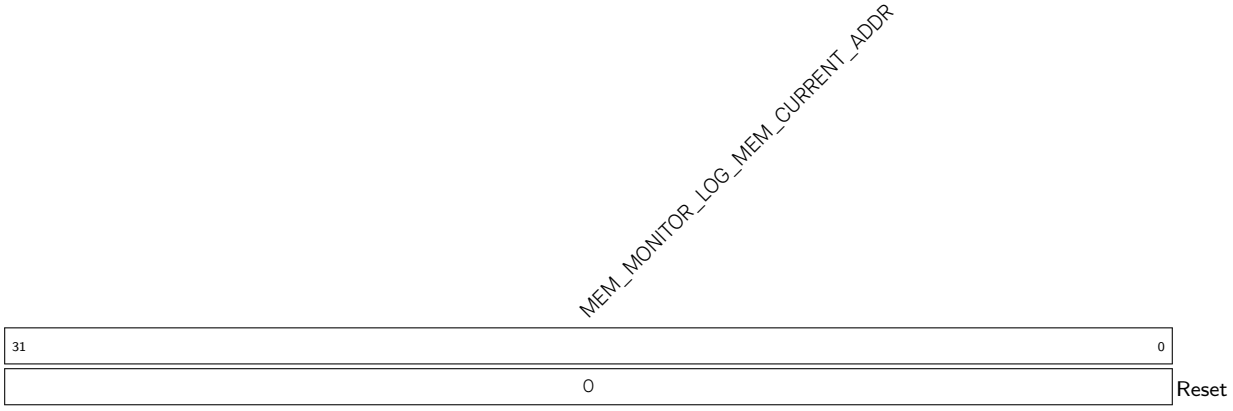
MEM_MONITOR_LOG_MEM_START 配置记录数据写入内存的起始地址。(R/W)

Register 7.10. MEM_MONITOR_LOG_MEM_END_REG (0x0024)



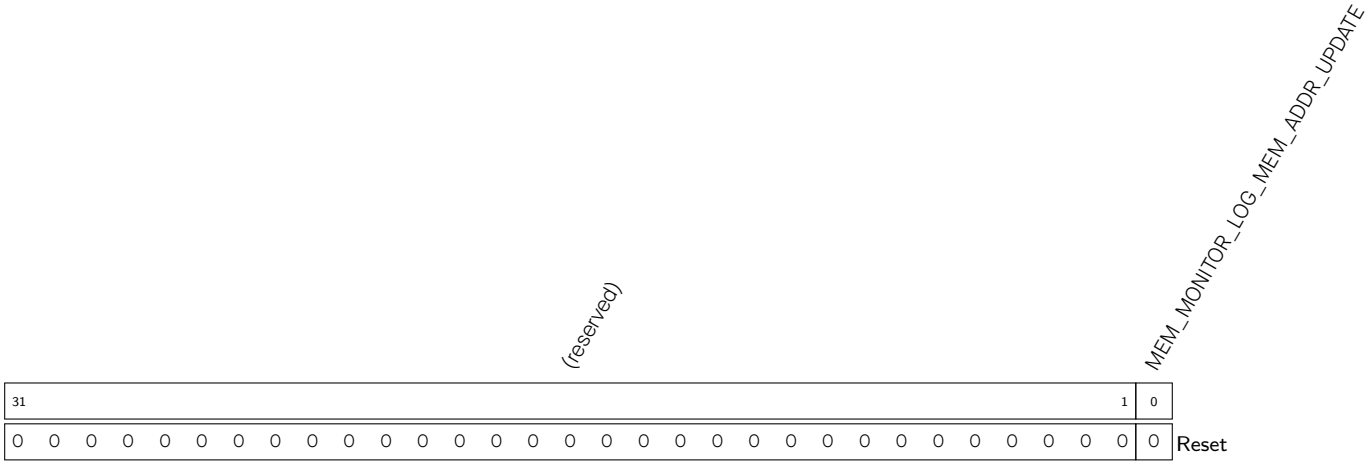
MEM_MONITOR_LOG_MEM_END 配置记录数据写入内存的结束地址。(R/W)

Register 7.11. MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG (0x0028)



MEM_MONITOR_LOG_MEM_CURRENT_ADDR 表示下一次写内存的写地址。(RO)

Register 7.12. MEM_MONITOR_LOG_MEM_ADDR_UPDATE_REG (0x002C)



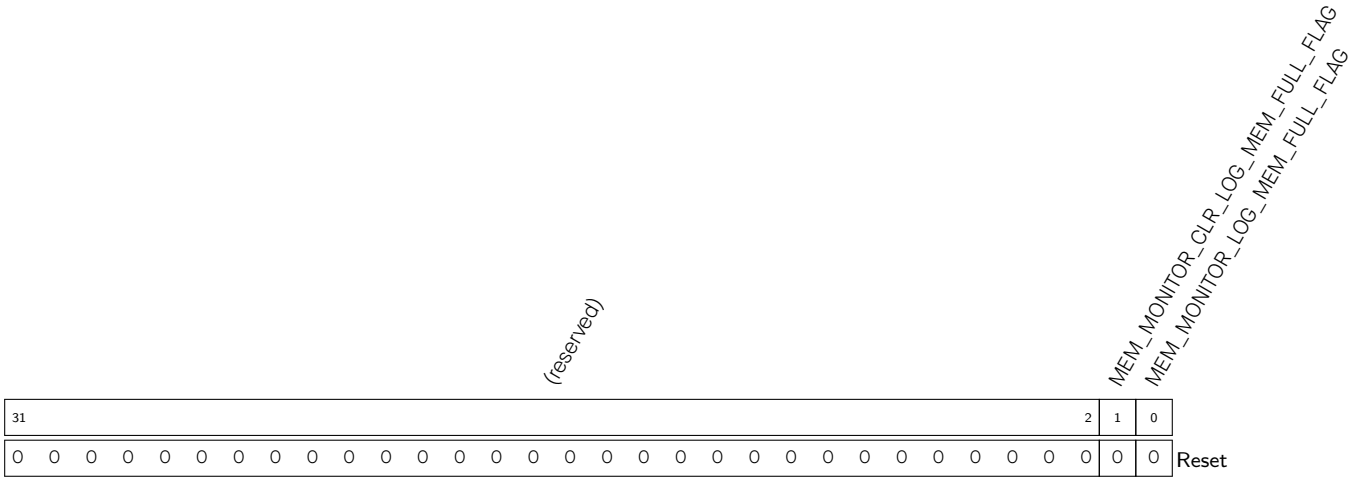
MEM_MONITOR_LOG_MEM_ADDR_UPDATE 配置是否将 MEM_MONITOR_LOG_MEM_CURRENT_ADDR_REG 中的值更新为 MEM_MONITOR_LOG_MEM_START_REG 中的值。

0：不更新

1：更新

(WT)

Register 7.13. MEM_MONITOR_LOG_MEM_FULL_FLAG_REG (0x0030)



MEM_MONITOR_LOG_MEM_FULL_FLAG 表示数据是否溢出存储地址范围。

0：未溢出

1：溢出

(RO)

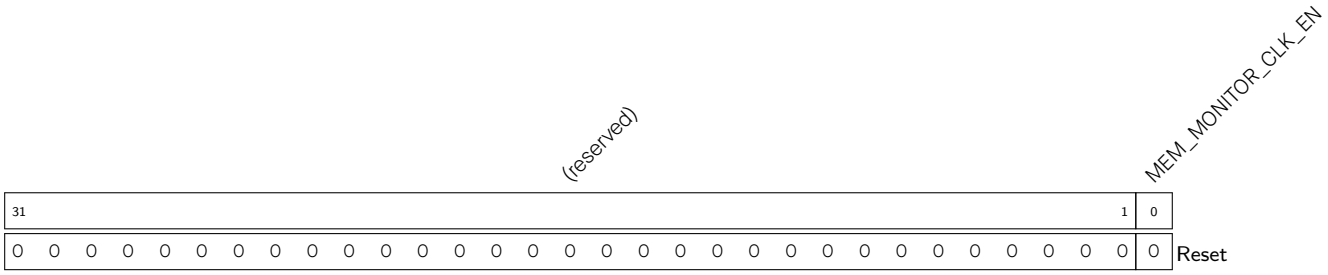
MEM_MONITOR_CLR_LOG_MEM_FULL_FLAG 配置是否清除 MEM_MONITOR_LOG_MEM_FULL_FLAG 标志位。

0：不清除

1：清除

(WT)

Register 7.14. MEM_MONITOR_CLOCK_GATE_REG (0x0034)



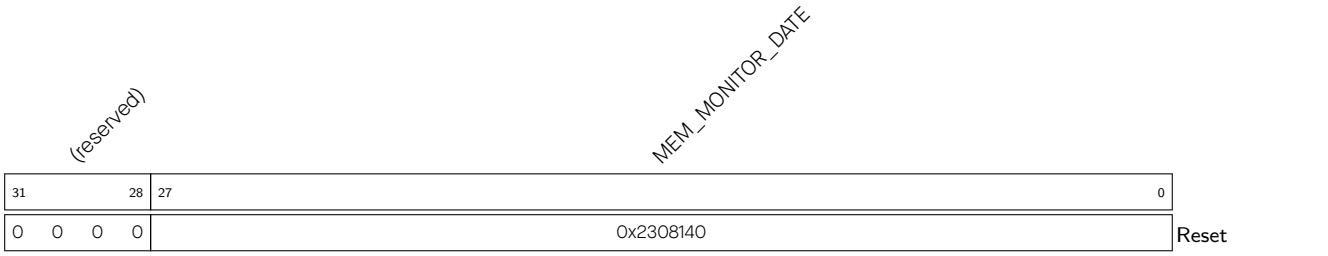
MEM_MONITOR_CLK_EN 配置是否使能寄存器时钟门控。

0：不使能

1：使能

(R/W)

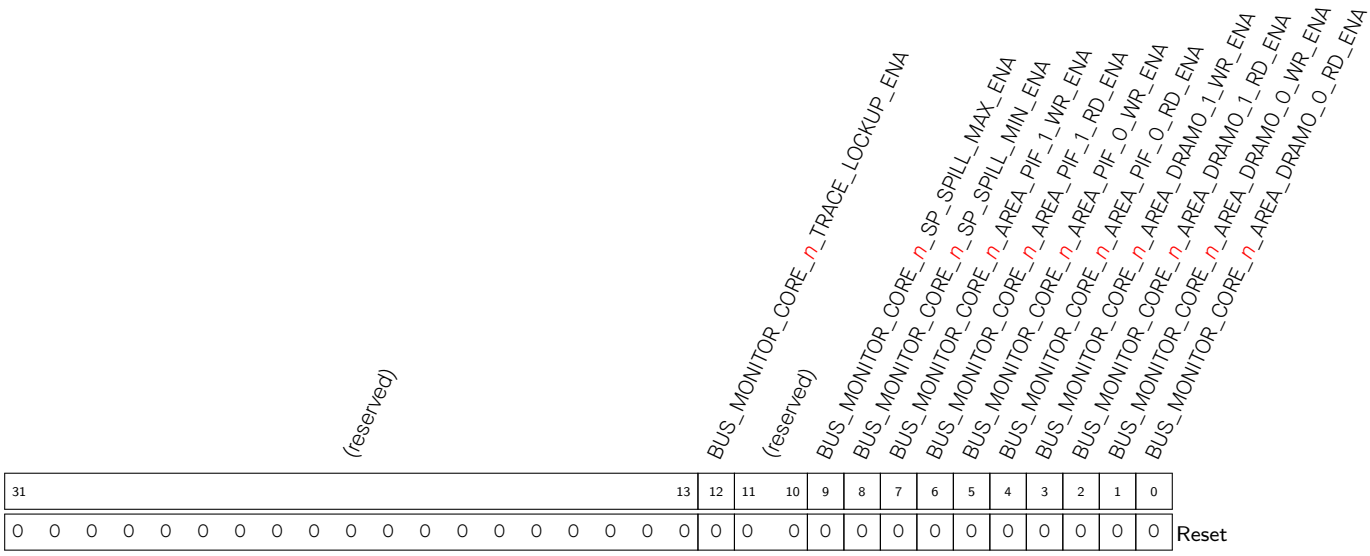
Register 7.15. MEM_MONITOR_DATE_REG (0x03FC)



MEM_MONITOR_DATE 版本控制寄存器。(R/W)

7.7.2 其他寄存器

Register 7.16. BUS_MONITOR_CORE_*n*_MONTR_ENA_REG (*n*=0,1) (0x0000+0x88×n)



BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_RD_ENA 配置是否监测数据总线在区域 0 内读操作。

0：不监测

1：监测

(R/W)

BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_WR_ENA 配置是否监测数据总线在区域 0 内写操作。

0：不监测

1：监测

(R/W)

BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_RD_ENA 配置是否监测数据总线在区域 1 内读操作。

0：不监测

1：监测

(R/W)

BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_WR_ENA 配置是否监测数据总线在区域 1 内写操作。

0：不监测

1：监测

(R/W)

BUS_MONITOR_CORE_*n*_AREA_PIF_0_RD_ENA 配置是否监测外设总线在区域 0 内读操作。

0：不监测

1：监测

(R/W)

见下页...

Register 7.16. BUS_MONITOR_CORE_*n*_MONTR_ENA_REG (*n*=0,1) (0x0000+0x88×n)

接上页...

BUS_MONITOR_CORE_*n*_AREA_PIF_0_WR_ENA 配置是否监测外设总线在区域 0 内写操作。

0：不监测

1：监测

(R/W)

BUS_MONITOR_CORE_*n*_AREA_PIF_1_RD_ENA 配置是否监测外设总线在区域 1 内读操作。

0：不监测

1：监测

(R/W)

BUS_MONITOR_CORE_*n*_AREA_PIF_1_WR_ENA 配置是否监测外设总线在区域 1 内写操作。

0：不监测

1：监测

(R/W)

BUS_MONITOR_CORE_*n*_SP_SPILL_MIN_ENA 配置是否监测栈指针小于栈监测区域的下边界。

0：不监测

1：监测

(R/W)

BUS_MONITOR_CORE_*n*_SP_SPILL_MAX_ENA 配置是否监测栈指针大于栈监测区域的上边界。

0：不监测

1：监测

(R/W)

BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_ENA 配置是否监测核心锁定。

0：不监测

1：监测

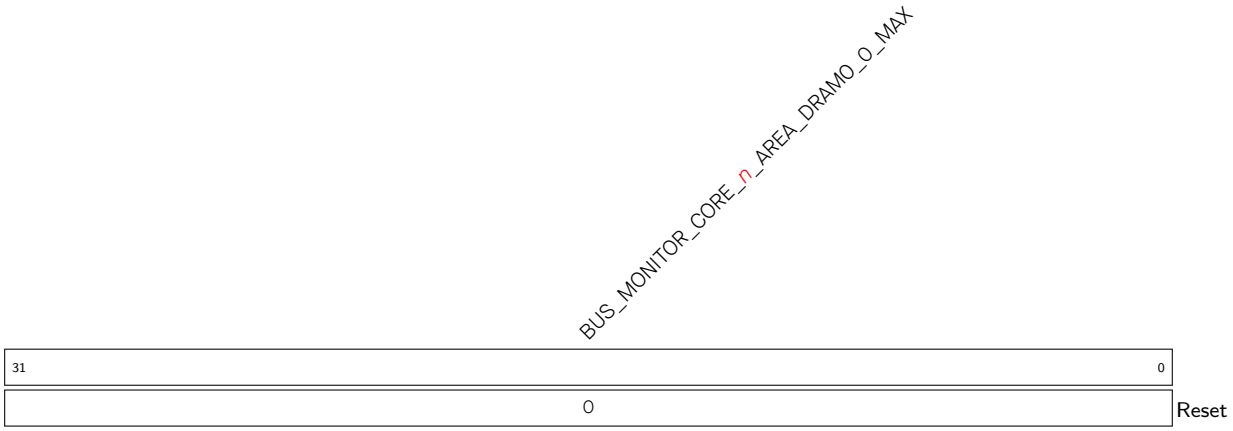
(R/W)

Register 7.17. BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_MIN_REG (*n*=0,1) (0x0010+0x88×n)



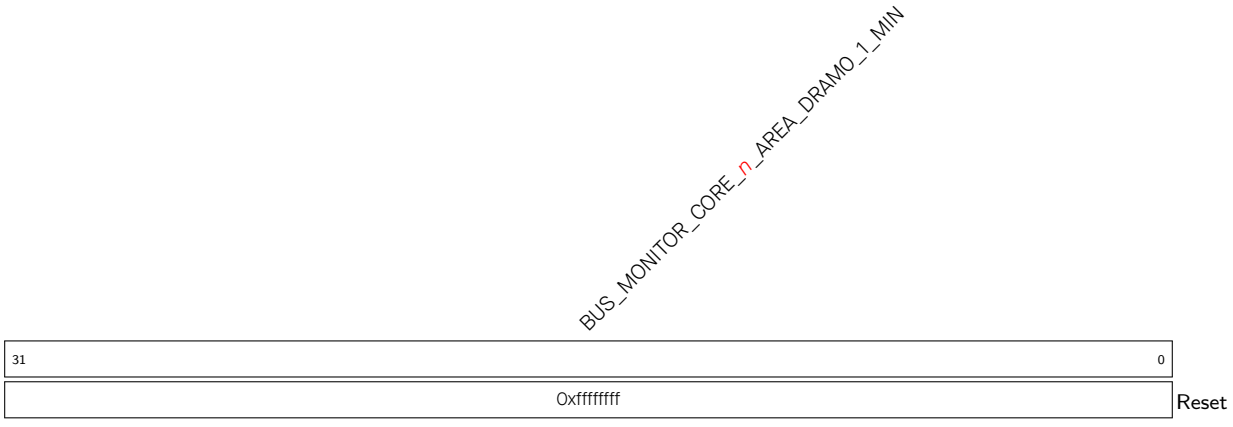
BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_MIN 配置数据总线区域 0 的下边界。(R/W)

Register 7.18. BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_MAX_REG (*n*=0,1) (0x0014+0x88×n)



BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_MAX 配置数据总线区域 0 的上边界。(R/W)

Register 7.19. BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_MIN_REG (*n*=0,1) (0x0018+0x88×n)



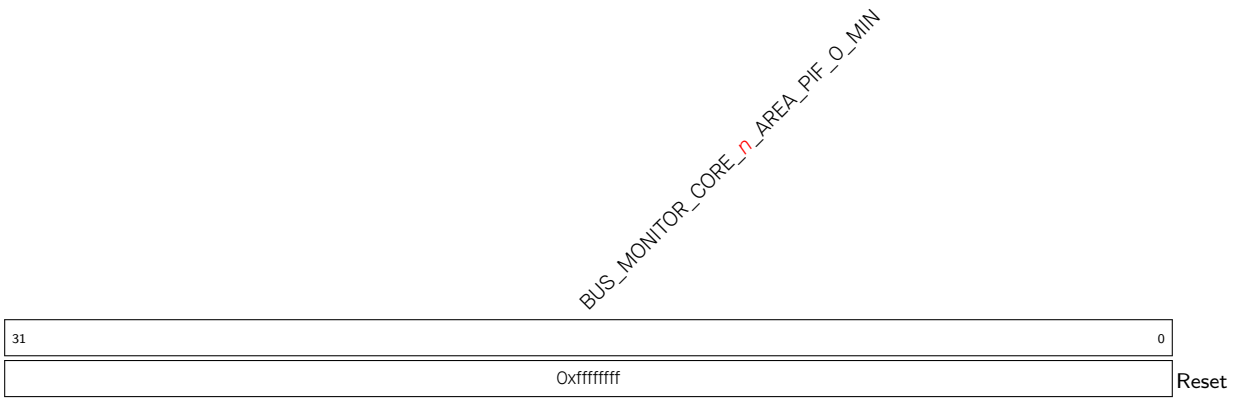
BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_MIN 配置数据总线区域 1 的下边界。(R/W)

Register 7.20. BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_MAX_REG (*n*=0,1) (0x001C+0x88×n)



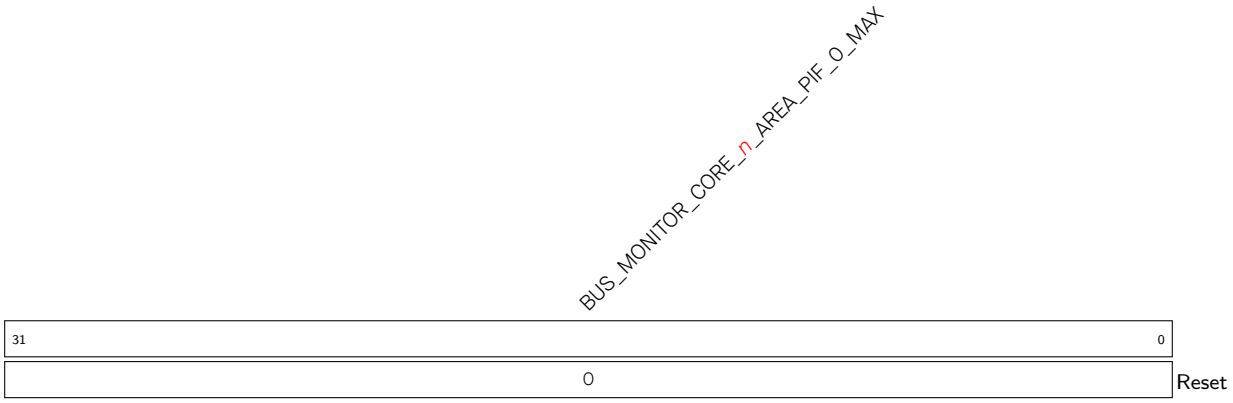
BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_MAX 配置数据总线区域 1 的上边界。(R/W)

Register 7.21. BUS_MONITOR_CORE_*n*_AREA_PIF_0_MIN_REG (*n*=0,1) (0x0020+0x88×n)



BUS_MONITOR_CORE_*n*_AREA_PIF_0_MIN 配置外设总线区域 0 的下边界。(R/W)

Register 7.22. BUS_MONITOR_CORE_*n*_AREA_PIF_0_MAX_REG (*n*=0,1) (0x0024+0x88×n)



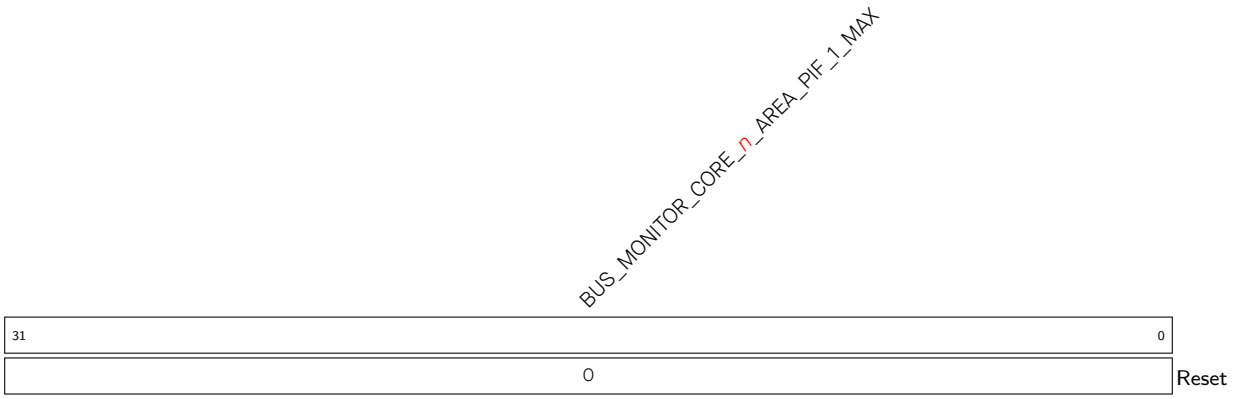
BUS_MONITOR_CORE_*n*_AREA_PIF_0_MAX 配置外设总线区域 0 的上边界。(R/W)

Register 7.23. BUS_MONITOR_CORE_*n*_AREA_PIF_1_MIN_REG (*n*=0,1) (0x0028+0x88×n)



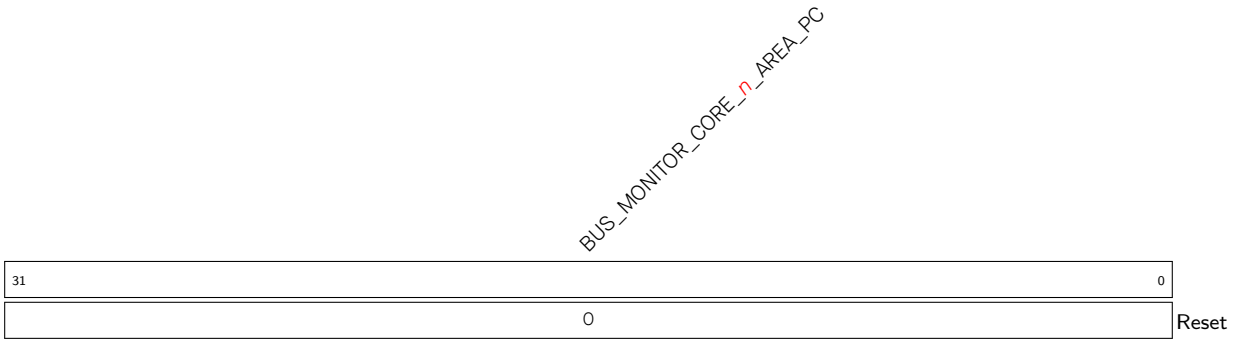
BUS_MONITOR_CORE_*n*_AREA_PIF_1_MIN 配置外设总线区域 1 的下边界。(R/W)

Register 7.24. BUS_MONITOR_CORE_*n*_AREA_PIF_1_MAX_REG (*n*=0,1) (0x002C+0x88×n)



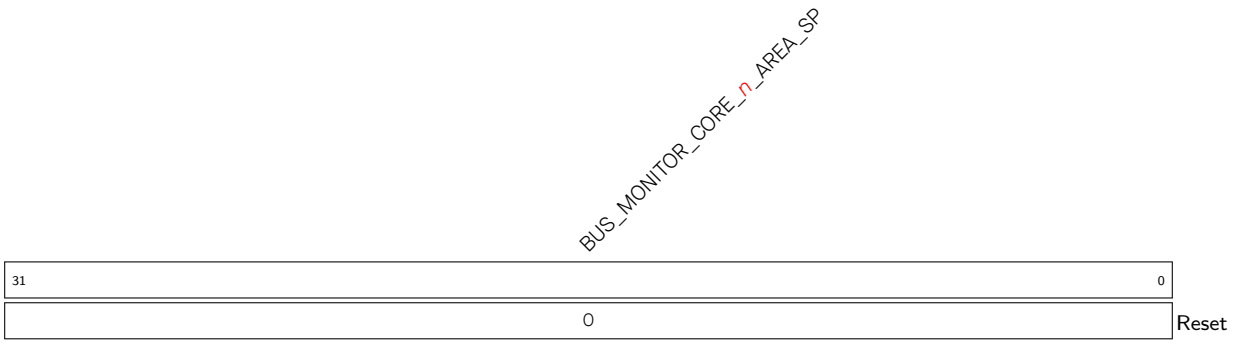
BUS_MONITOR_CORE_*n*_AREA_PIF_1_MAX 配置外设总线区域 1 的上边界。(R/W)

Register 7.25. BUS_MONITOR_CORE_*n*_AREA_PC_REG (*n*=0,1) (0x0030+0x88×n)



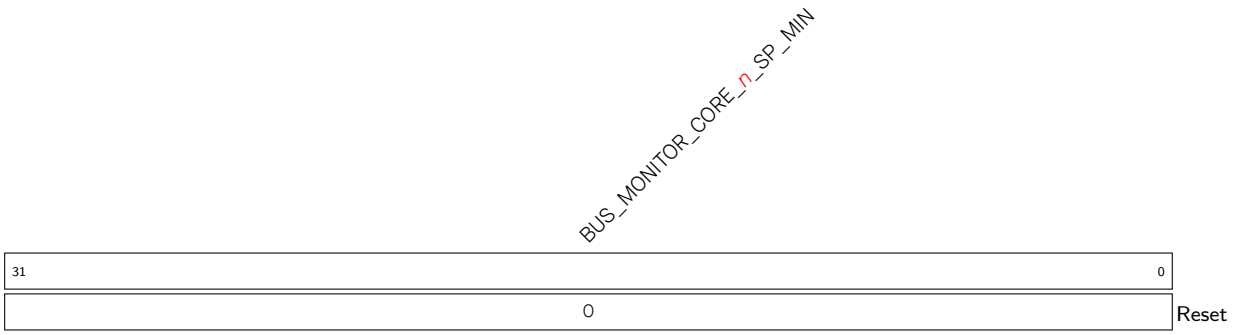
BUS_MONITOR_CORE_*n*_AREA_PC 表示区域监测下触发中断时刻的 HP CPU*n* PC 值。(RO)

Register 7.26. BUS_MONITOR_CORE_*n*_AREA_SP_REG (*n*=0,1) (0x0034+0x88×n)



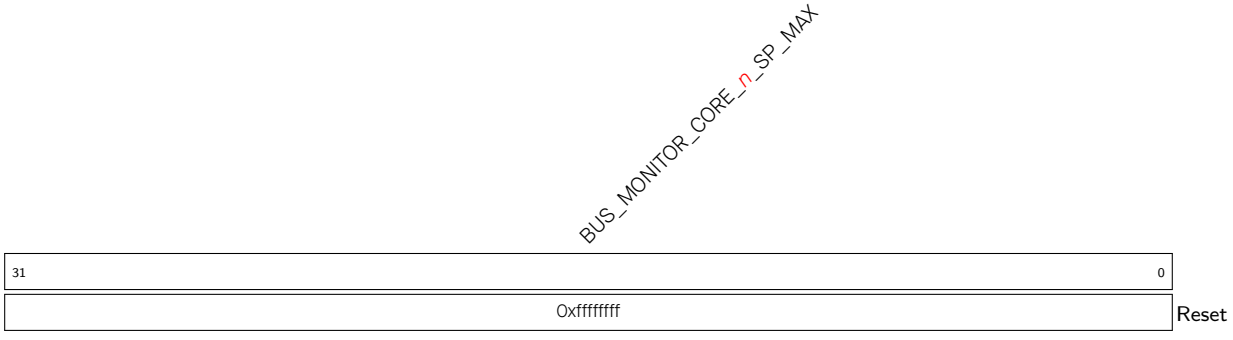
BUS_MONITOR_CORE_*n*_AREA_SP 表示区域监测下触发中断时刻的 HP CPU*n* SP 值。(RO)

Register 7.27. BUS_MONITOR_CORE_*n*_SP_MIN_REG (*n*=0,1) (0x0038+0x88×n)



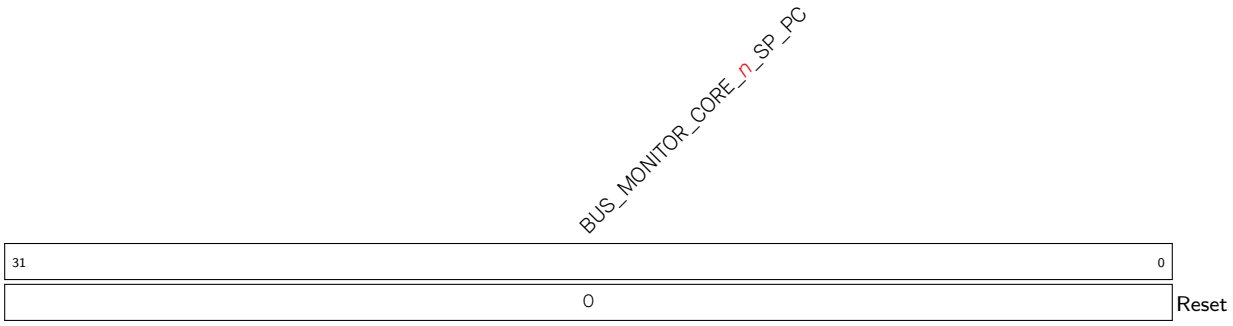
BUS_MONITOR_CORE_*n*_SP_MIN 配置栈指针的下边界。(R/W)

Register 7.28. BUS_MONITOR_CORE_*n*_SP_MAX_REG (*n*=0,1) (0x003C+0x88×n)



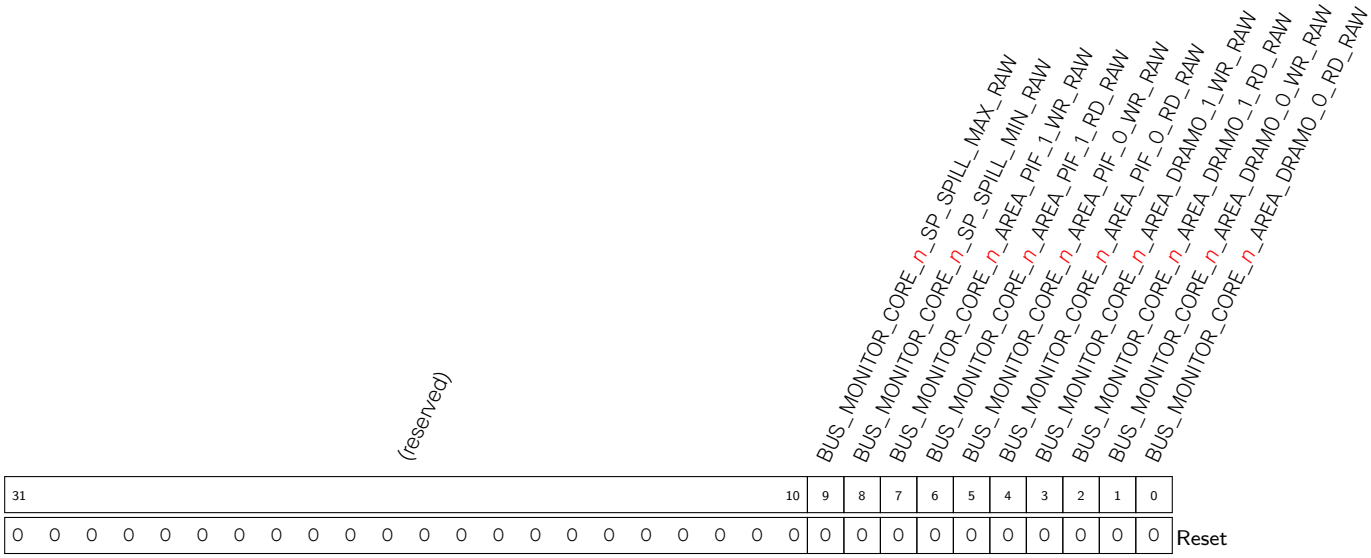
BUS_MONITOR_CORE_*n*_SP_MAX 配置栈指针的上边界。(R/W)

Register 7.29. BUS_MONITOR_CORE_*n*_SP_PC_REG (*n*=0,1) (0x0040+0x88×n)



BUS_MONITOR_CORE_*n*_SP_PC 表示栈指针监测的 HP CPU*n* PC 值。(RO)

Register 7.30. BUS_MONITOR_CORE_*n*_INTR_RAW_REG (*n*=0,1) (0x0004+0x88×n)



BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_RD_RAW BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_RD_INT 的原始中断状态。(RO)

BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_WR_RAW BUS_MONITOR_CORE_*n*_AREA_DRAM0_0_WR_INT 的原始中断状态。(RO)

BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_RD_RAW BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_RD_INT 的原始中断状态。(RO)

BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_WR_RAW BUS_MONITOR_CORE_*n*_AREA_DRAM0_1_WR_INT 的原始中断状态。(RO)

BUS_MONITOR_CORE_*n*_AREA_PIF_0_RD_RAW BUS_MONITOR_CORE_*n*_AREA_PIF_0_RD_INT 的原始中断状态。(RO)

BUS_MONITOR_CORE_*n*_AREA_PIF_0_WR_RAW BUS_MONITOR_CORE_*n*_AREA_PIF_0_WR_INT 的原始中断状态。(RO)

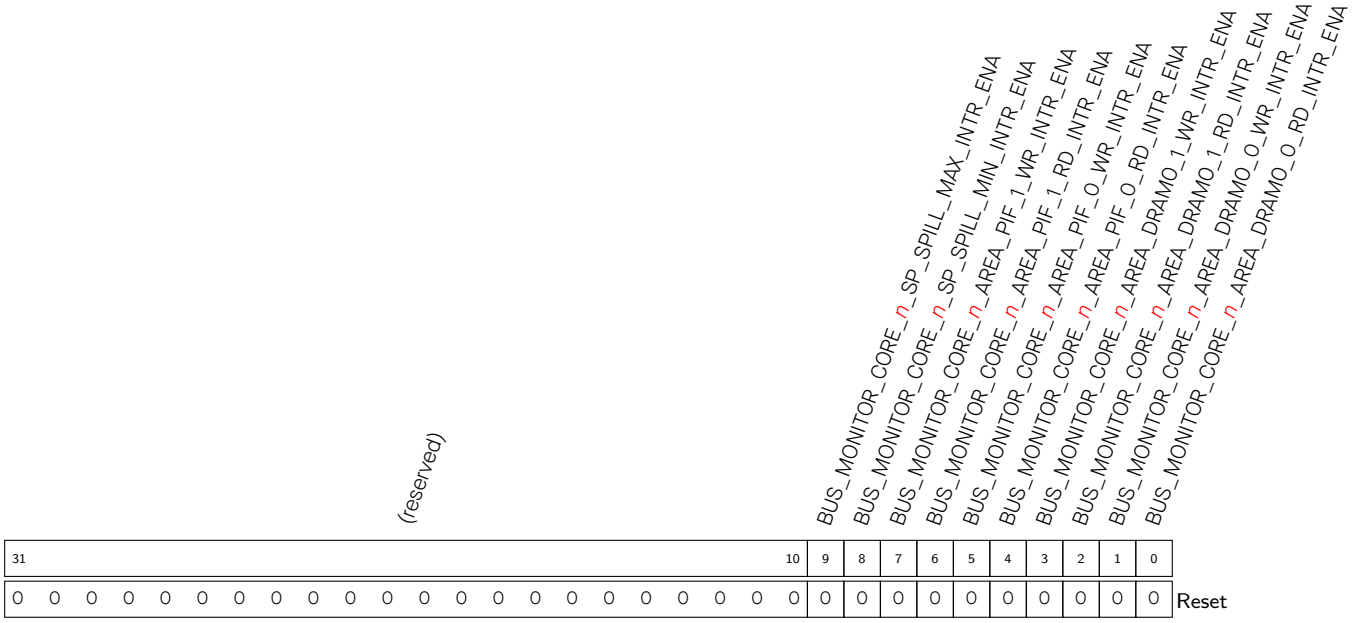
BUS_MONITOR_CORE_*n*_AREA_PIF_1_RD_RAW BUS_MONITOR_CORE_*n*_AREA_PIF_1_RD_INT 的原始中断状态。(RO)

BUS_MONITOR_CORE_*n*_AREA_PIF_1_WR_RAW BUS_MONITOR_CORE_*n*_AREA_PIF_1_WR_INT 的原始中断状态。(RO)

BUS_MONITOR_CORE_*n*_SP_SPILL_MIN_RAW BUS_MONITOR_CORE_*n*_SP_SPILL_MIN_INT 的原始中断状态。(RO)

BUS_MONITOR_CORE_*n*_SP_SPILL_MAX_RAW BUS_MONITOR_CORE_*n*_SP_SPILL_MAX_INT 的原始中断状态。(RO)

Register 7.31. BUS_MONITOR_CORE_*n*_INTR_ENA_REG (*n*=0,1) (0x0008+0x88×n)



BUS_MONITOR_CORE_*n*_AREA_DRAMO_0_RD_INTR_ENA 写 1 使能 BUS_MONITOR_CORE_*n*_AREA_DRAMO_0_RD_INT。(R/W)

BUS_MONITOR_CORE_*n*_AREA_DRAMO_0_WR_INTR_ENA 写 1 使能 BUS_MONITOR_CORE_*n*_AREA_DRAMO_0_WR_INT。(R/W)

BUS_MONITOR_CORE_*n*_AREA_DRAMO_1_RD_INTR_ENA 写 1 使能 BUS_MONITOR_CORE_*n*_AREA_DRAMO_1_RD_INT。(R/W)

BUS_MONITOR_CORE_*n*_AREA_DRAMO_1_WR_INTR_ENA 写 1 使能 BUS_MONITOR_CORE_*n*_AREA_DRAMO_1_WR_INT。(R/W)

BUS_MONITOR_CORE_*n*_AREA_PIF_0_RD_INTR_ENA 写 1 使能 BUS_MONITOR_CORE_*n*_AREA_PIF_0_RD_INT。(R/W)

BUS_MONITOR_CORE_*n*_AREA_PIF_0_WR_INTR_ENA 写 1 使能 BUS_MONITOR_CORE_*n*_AREA_PIF_0_WR_INT。(R/W)

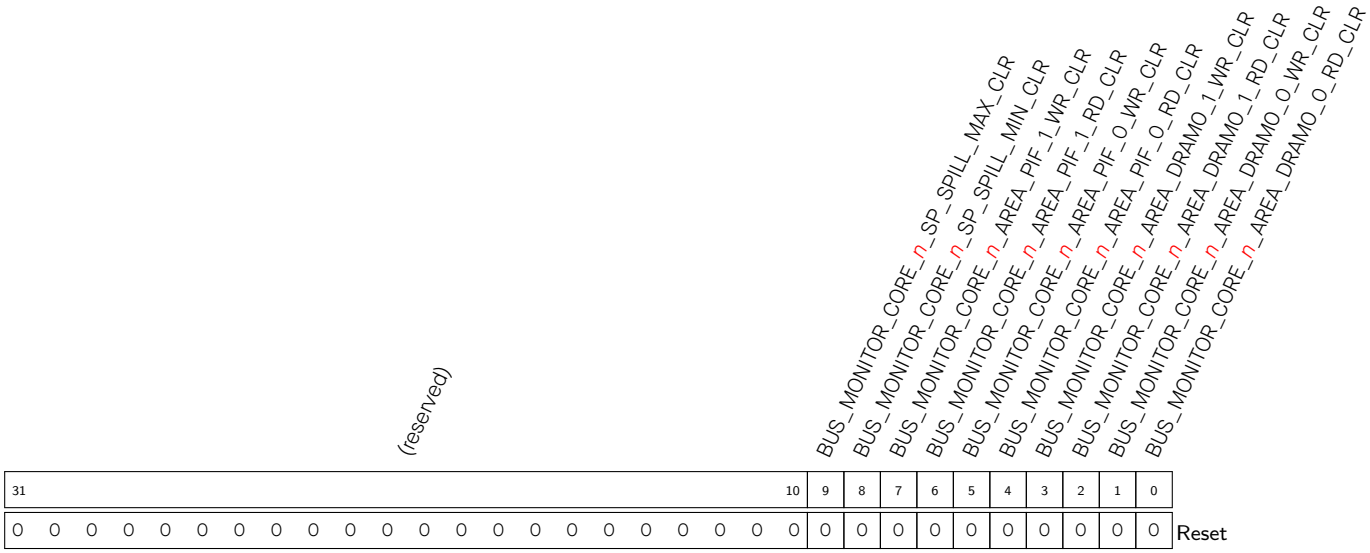
BUS_MONITOR_CORE_*n*_AREA_PIF_1_RD_INTR_ENA 写 1 使能 BUS_MONITOR_CORE_*n*_AREA_PIF_1_RD_INT。(R/W)

BUS_MONITOR_CORE_*n*_AREA_PIF_1_WR_INTR_ENA 写 1 使能 BUS_MONITOR_CORE_*n*_AREA_PIF_1_WR_INT。(R/W)

BUS_MONITOR_CORE_*n*_SP_SPILL_MIN_INTR_ENA 写 1 使能 BUS_MONITOR_CORE_*n*_SP_SPILL_MIN_INT。(R/W)

BUS_MONITOR_CORE_*n*_SP_SPILL_MAX_INTR_ENA 写 1 使能 BUS_MONITOR_CORE_*n*_SP_SPILL_MAX_INT。(R/W)

Register 7.32. BUS_MONITOR_CORE_*n*_INTR_CLR_REG (*n*=0,1) (0x000C+0x88×n)



BUS_MONITOR_CORE_*n*_AREA_DRAMO_0_RD_CLR 写 1 清除 BUS_MONITOR_CORE_*n*_AREA_DRAMO_0_RD_INT。(WT)

BUS_MONITOR_CORE_*n*_AREA_DRAMO_0_WR_CLR 写 1 清除 BUS_MONITOR_CORE_*n*_AREA_DRAMO_0_WR_INT。(WT)

BUS_MONITOR_CORE_*n*_AREA_DRAMO_1_RD_CLR 写 1 清除 BUS_MONITOR_CORE_*n*_AREA_DRAMO_1_RD_INT。(WT)

BUS_MONITOR_CORE_*n*_AREA_DRAMO_1_WR_CLR 写 1 清除 BUS_MONITOR_CORE_*n*_AREA_DRAMO_1_WR_INT。(WT)

BUS_MONITOR_CORE_*n*_AREA_PIF_0_RD_CLR 写 1 清除 BUS_MONITOR_CORE_*n*_AREA_PIF_0_RD_INT。(WT)

BUS_MONITOR_CORE_*n*_AREA_PIF_0_WR_CLR 写 1 清除 BUS_MONITOR_CORE_*n*_AREA_PIF_0_WR_INT。(WT)

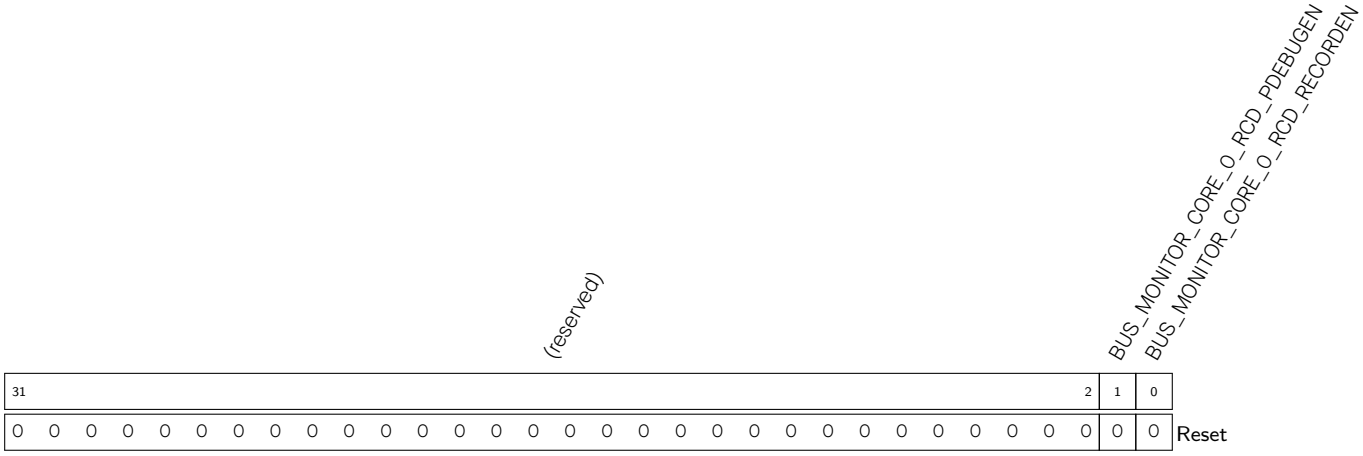
BUS_MONITOR_CORE_*n*_AREA_PIF_1_RD_CLR 写 1 清除 BUS_MONITOR_CORE_*n*_AREA_PIF_1_RD_INT。(WT)

BUS_MONITOR_CORE_*n*_AREA_PIF_1_WR_CLR 写 1 清除 BUS_MONITOR_CORE_*n*_AREA_PIF_1_WR_INT。(WT)

BUS_MONITOR_CORE_*n*_SP_SPILL_MIN_CLR 写 1 清除 BUS_MONITOR_CORE_*n*_SP_SPILL_MIN_INT。(WT)

BUS_MONITOR_CORE_*n*_SP_SPILL_MAX_CLR 写 1 清除 BUS_MONITOR_CORE_*n*_SP_SPILL_MAX_INT。(WT)

Register 7.33. BUS_MONITOR_CORE_0_RCD_EN_REG (0x0044)



BUS_MONITOR_CORE_0_RCD_RECORDEN 配置是否使能 PC 和 SP 记录。

0：不使能

1：BUS_MONITOR_CORE_0_RCD_PDEBUGPC_REG 开始实时记录 HP CPU0 PC，同时 BUS_MONITOR_CORE_0_RCD_PDEBUGSP_REG 开始实时记录 HP CPU0 栈指针

(R/W)

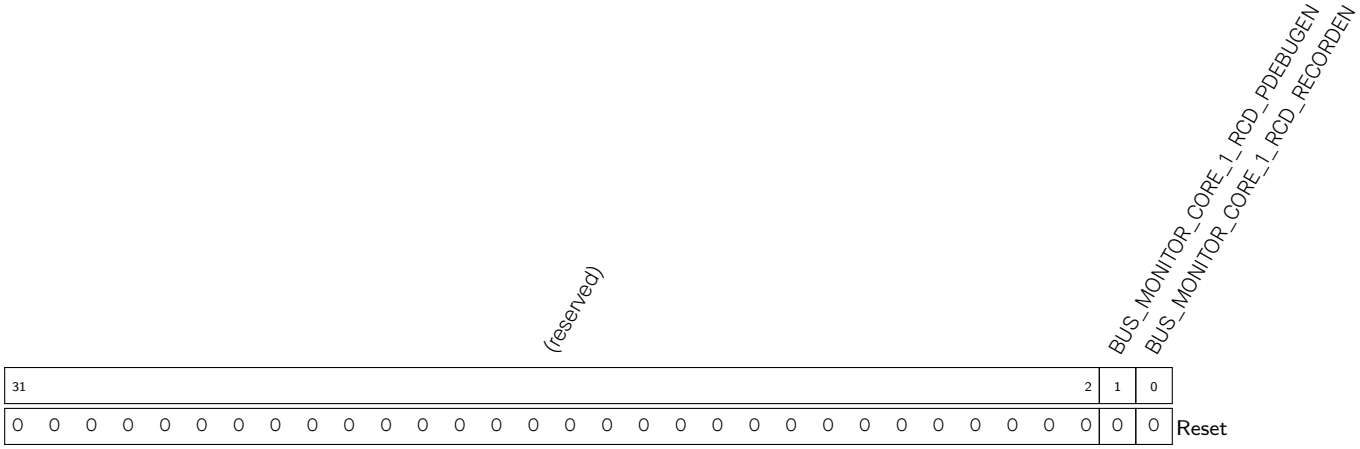
BUS_MONITOR_CORE_0_RCD_PDEBUGEN 配置是否使能 HP CPU0 调试。

0：不使能

1：HP CPU0 输出 HP CPU0 PC

(R/W)

Register 7.34. BUS_MONITOR_CORE_1_RCD_EN_REG (0x00CC)



BUS_MONITOR_CORE_1_RCD_RECORDEN 配置是否使能 PC 和 SP 记录。

0：不使能

1：BUS_MONITOR_CORE_1_RCD_PDEBUGPC_REG 开始实时记录 HP CPU1 PC，同时 BUS_MONITOR_CORE_1_RCD_PDEBUGSP_REG 开始实时记录 HP CPU1 栈指针

(R/W)

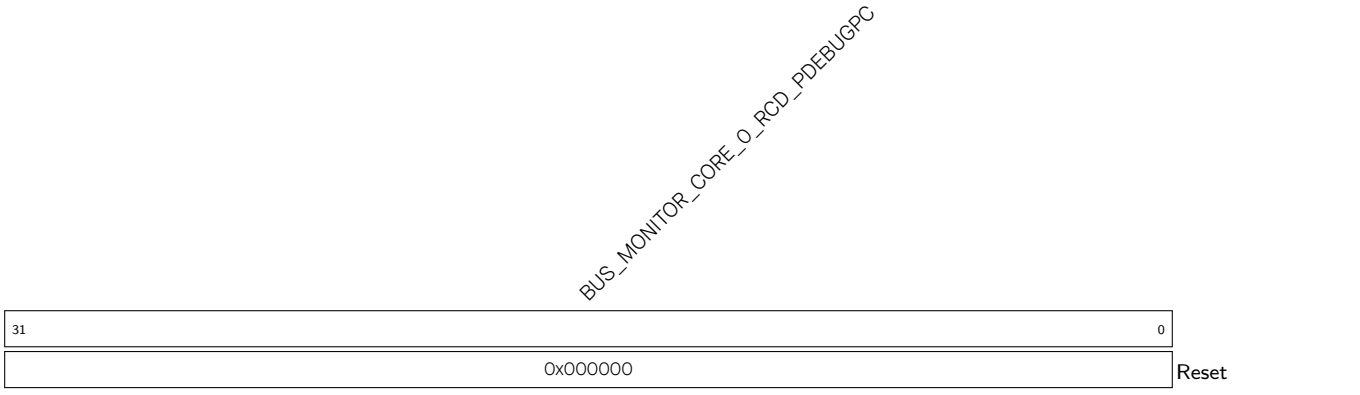
BUS_MONITOR_CORE_1_RCD_PDEBUGEN 配置是否使能 HP CPU1 调试。

0：不使能

1：HP CPU1 输出 HP CPU1 PC

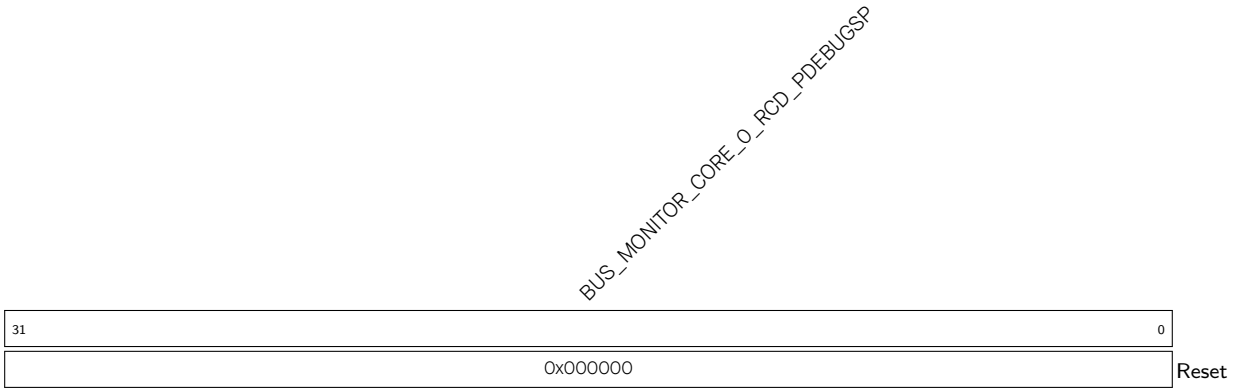
(R/W)

Register 7.35. BUS_MONITOR_CORE_0_RCD_PDEBUGPC_REG (0x0048)



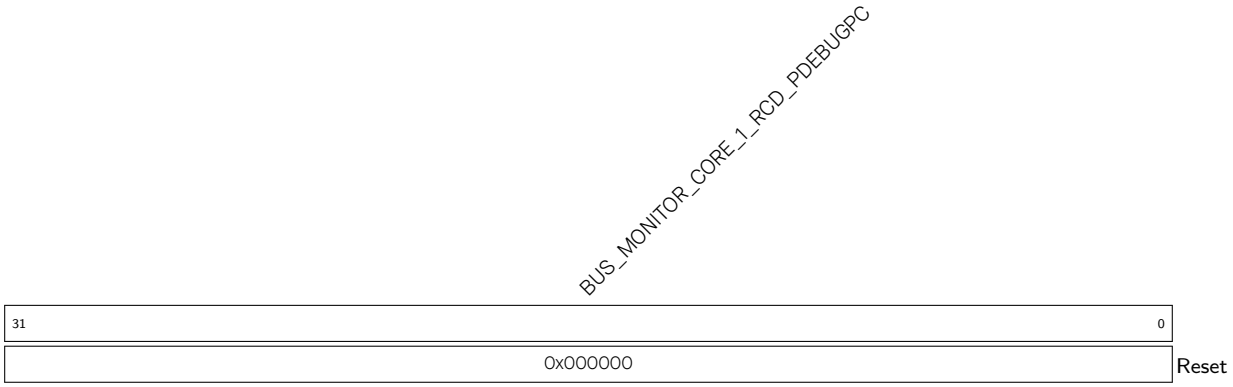
BUS_MONITOR_CORE_0_RCD_PDEBUGPC 表示 HP CPU0 复位时刻的 PC 值。(RO)

Register 7.36. BUS_MONITOR_CORE_0_RCD_PDEBUGSP_REG (0x004C)



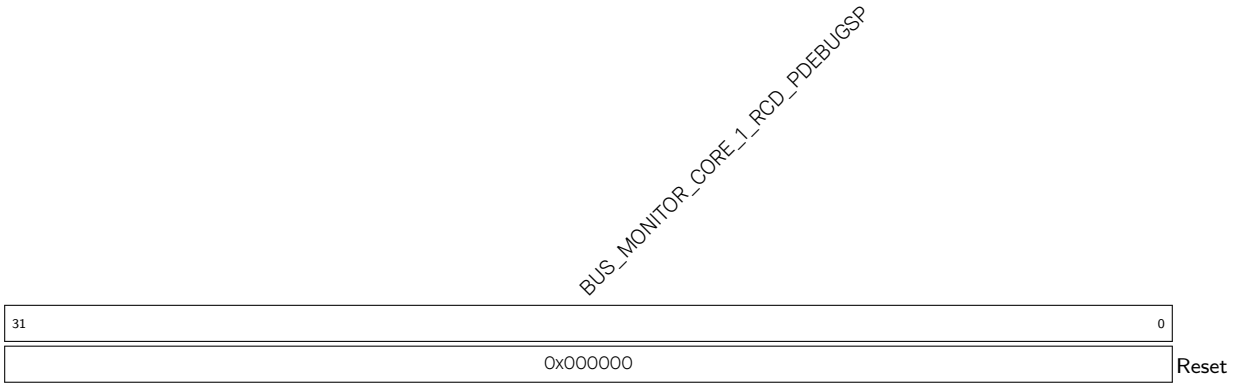
BUS_MONITOR_CORE_0_RCD_PDEBUGSP 表示 HP CPU0 SP。(RO)

Register 7.37. BUS_MONITOR_CORE_1_RCD_PDEBUGPC_REG (0x00D0)



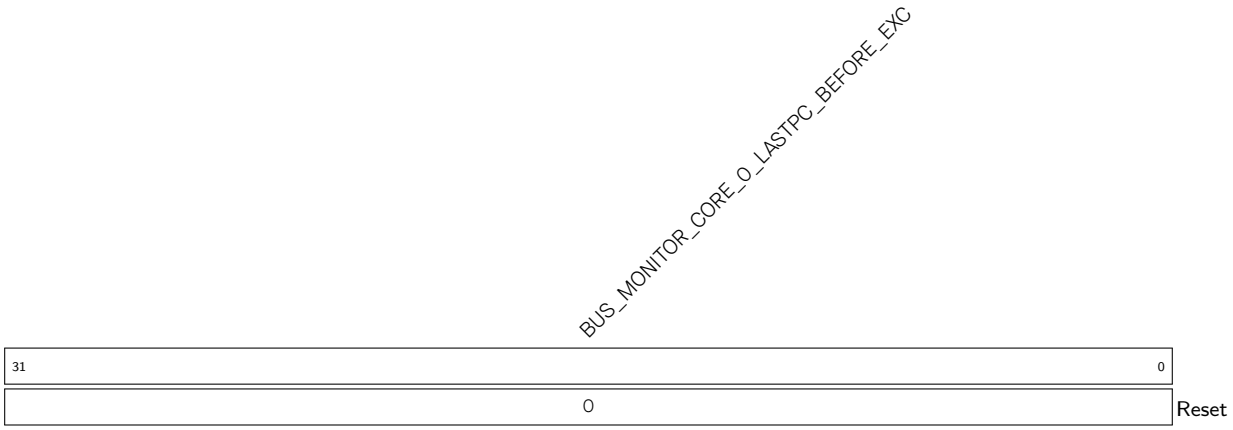
BUS_MONITOR_CORE_1_RCD_PDEBUGPC 表示 HP CPU1 复位时刻的 PC 值。(RO)

Register 7.38. BUS_MONITOR_CORE_1_RCD_PDEBUGSP_REG (0x00D4)



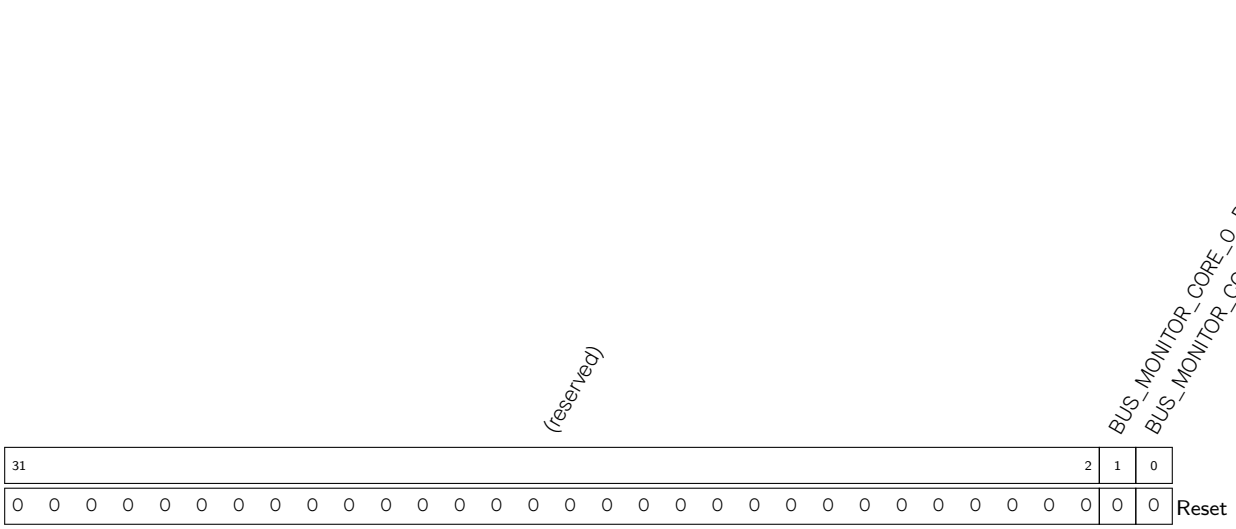
BUS_MONITOR_CORE_1_RCD_PDEBUGSP 表示 HP CPU1 SP。(RO)

Register 7.39. BUS_MONITOR_CORE_0_LASTPC_BEFORE_EXCEPTION_REG (0x0060)



BUS_MONITOR_CORE_0_LASTPC_BEFORE_EXC 表示 HP CPU0 异常前的最后一条指令的 HP CPU0 PC。(RO)

Register 7.40. BUS_MONITOR_CORE_0_DEBUG_MODE_REG (0x0064)



BUS_MONITOR_CORE_0_DEBUG_MODE 表示 RISC-V CPU0 (HP CPU0) 是否处于调试模式。

0：未处于调试模式

1：处于调试模式

(RO)

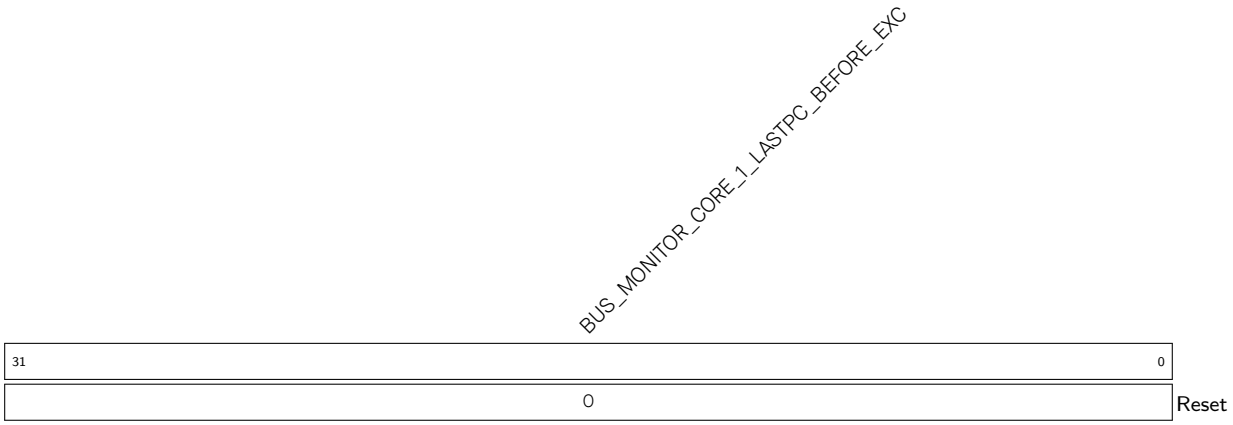
BUS_MONITOR_CORE_0_DEBUG_MODULE_ACTIVE 表示 RISC-V CPU0 (HP CPU0) 调试模块的工作状态。

1：处于工作状态

其它：未处于工作状态

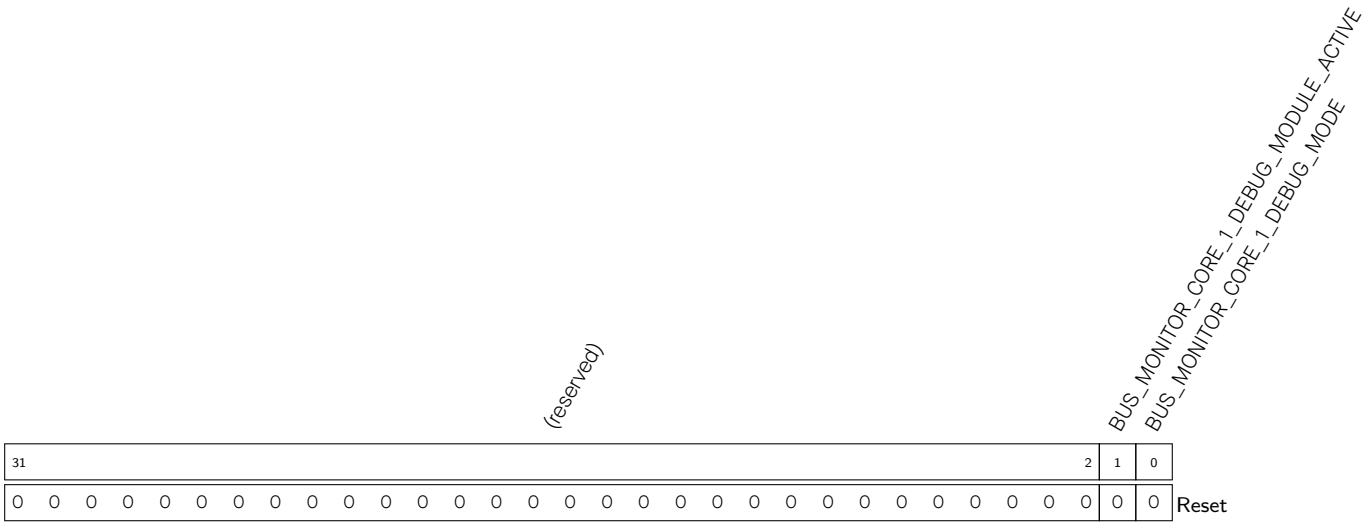
(RO)

Register 7.41. BUS_MONITOR_CORE_1_LASTPC_BEFORE_EXCEPTION_REG (0x00E8)



BUS_MONITOR_CORE_1_LASTPC_BEFORE_EXC 表示 HP CPU1 异常前的最后一条指令的 HP CPU1 PC。(RO)

Register 7.42. BUS_MONITOR_CORE_1_DEBUG_MODE_REG (0x00EC)



BUS_MONITOR_CORE_1_DEBUG_MODE 表示 RISC-V CPU1 (HP CPU1) 是否处于调试模式。

0：未处于调试模式

1：处于调试模式

(RO)

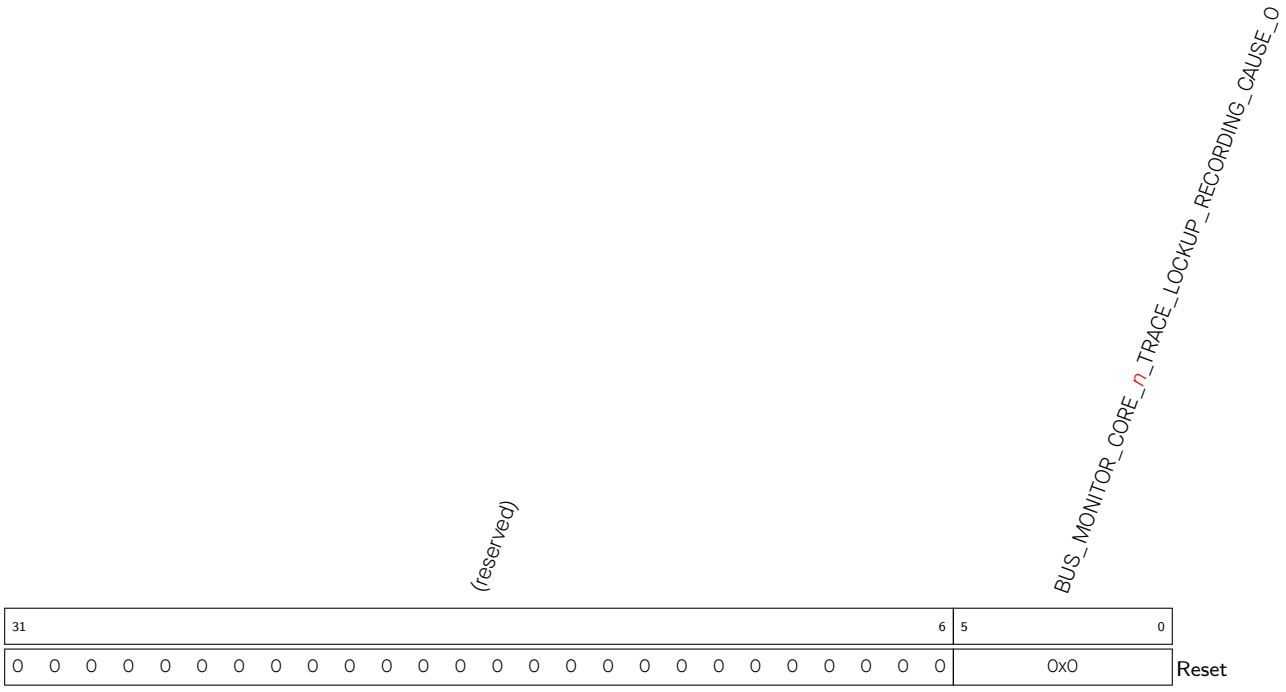
BUS_MONITOR_CORE_1_DEBUG_MODULE_ACTIVE 表示 RISC-V CPU1 (HP CPU1) 调试模块的工作状态。

1：处于工作状态

其它：未处于工作状态

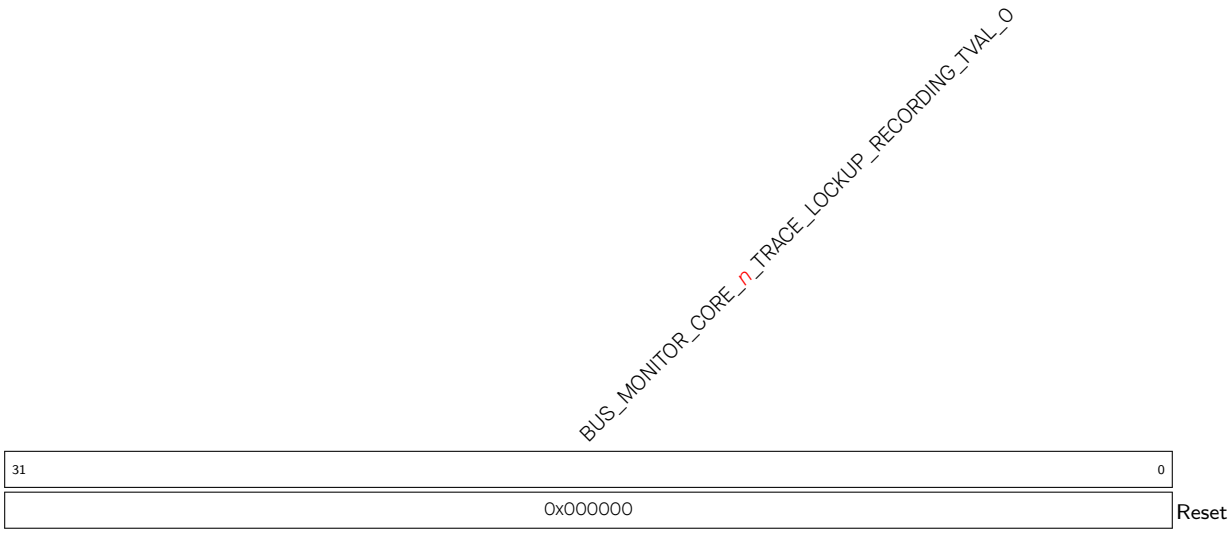
(RO)

Register 7.43. BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_CAUSE_0_REG (*n*=0,1) (0x0068+0x88×n)



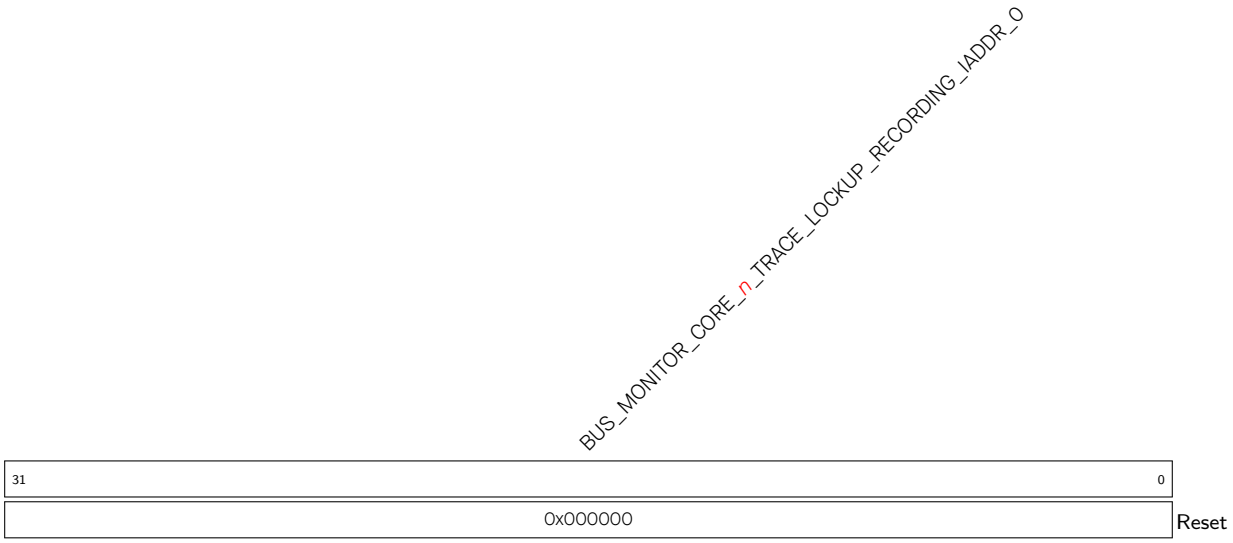
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_RECORDING_CAUSE_0 表示第二次核心锁定的异常原因。(RO)

Register 7.44. BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_TVAL_0_REG (*n*=0,1) (0x006C+0x88×n)



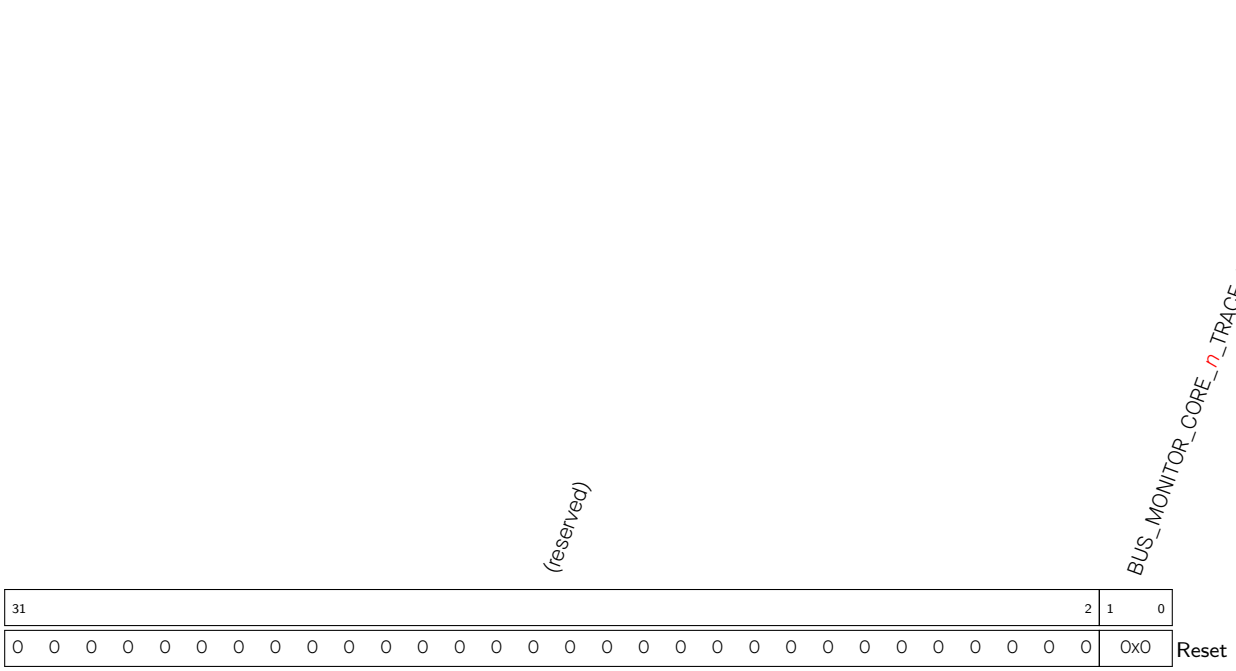
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_RECORDING_TVAL_0 表示第二次核心锁定时的异常陷阱值。(RO)

Register 7.45. BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_IADDR_0_REG (*n*=0,1) (0x0070+0x88×n)



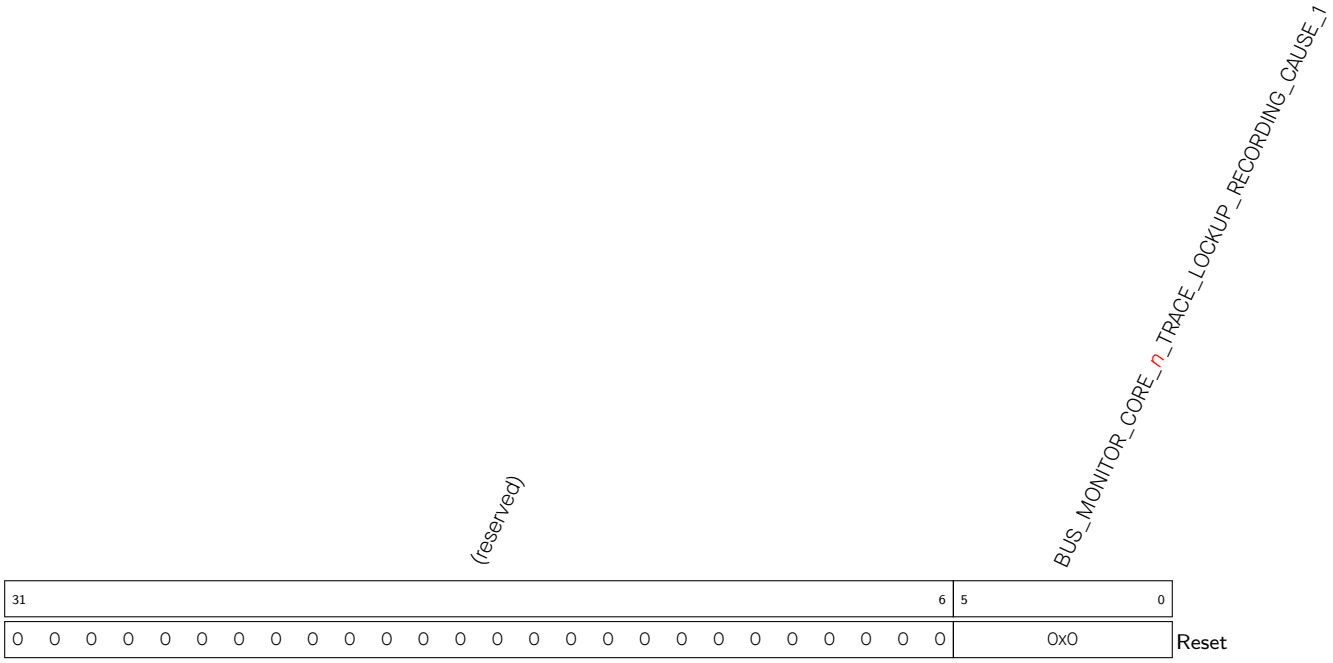
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_RECORDING_IADDR_0 表示第二次核心锁定时的指令 PC。(RO)

Register 7.46. BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_PRIV_0_REG (*n*=0,1) (0x0074+0x88×n)



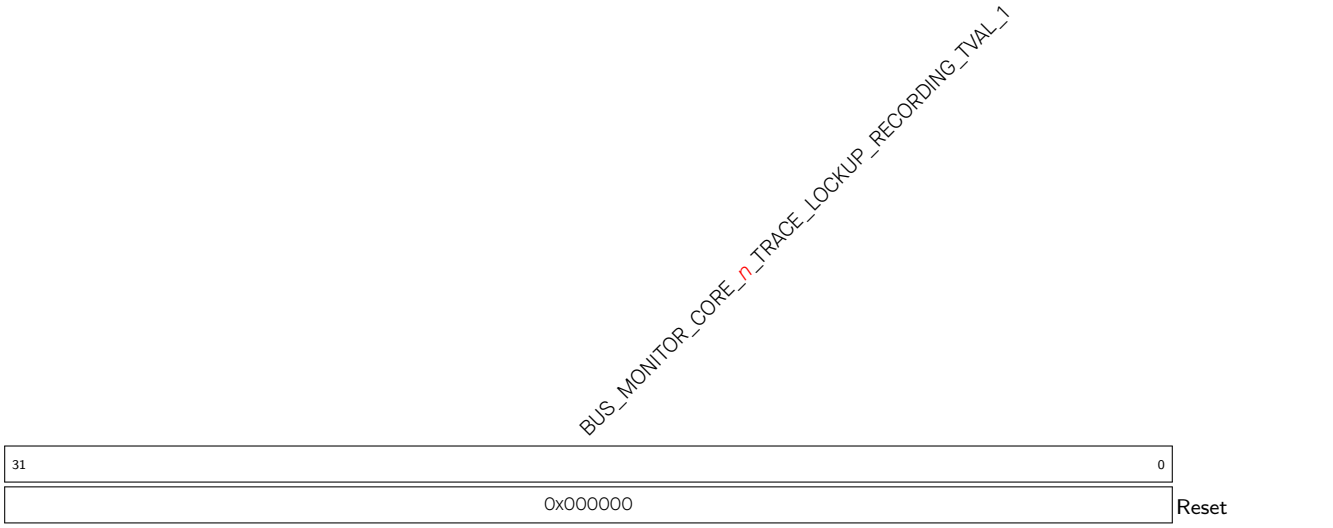
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_RECORDING_PRIV_0 表示第二次核心锁定时的当前特权级别。(RO)

Register 7.47. BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_CAUSE_1_REG (*n*=0,1) (0x0078+0x88×n)



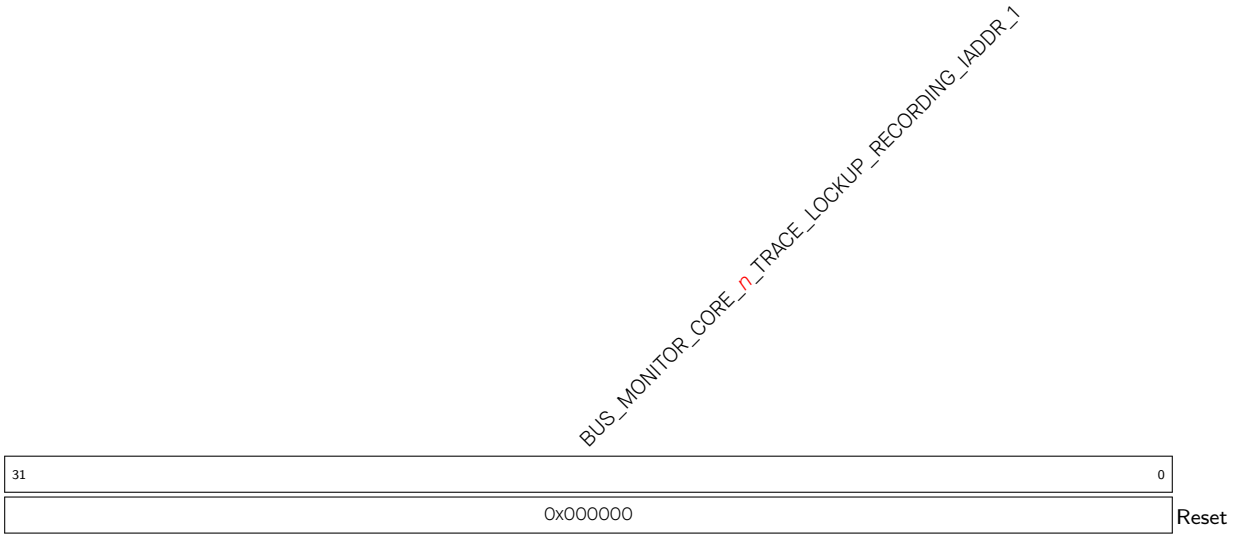
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_RECORDING_CAUSE_1 表示首次核心锁定的异常原因。(RO)

Register 7.48. BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_TVAL_1_REG (*n*=0,1) (0x007C+0x88×n)



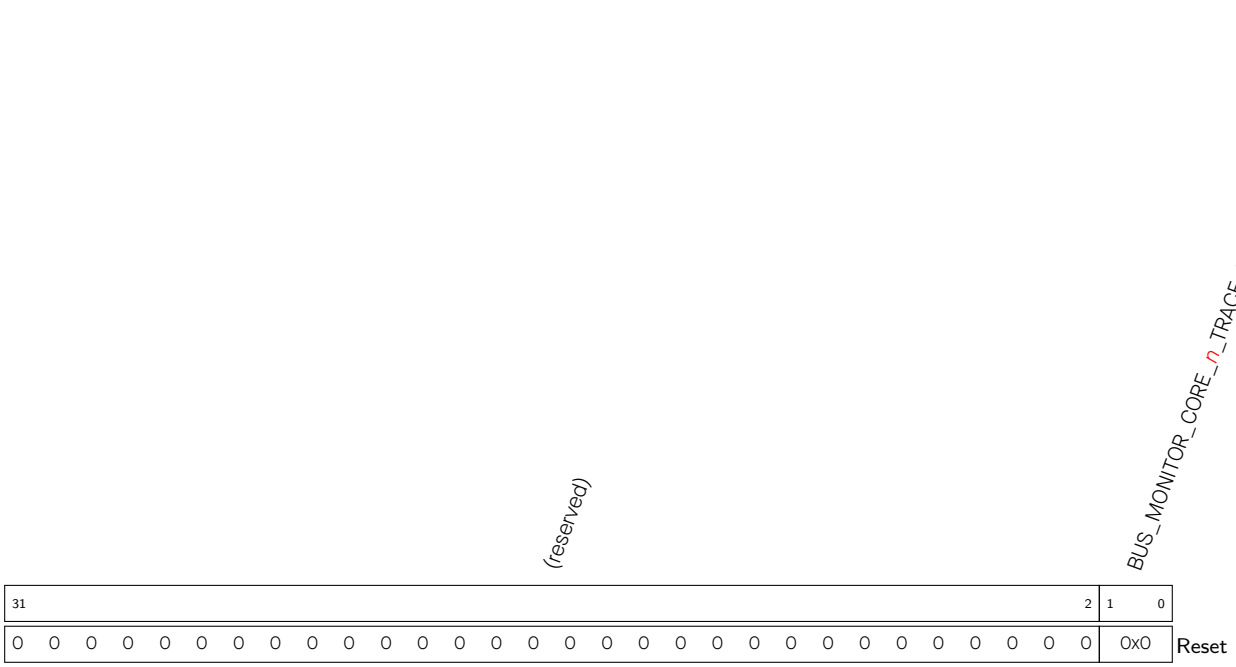
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_RECORDING_TVAL_1 表示首次核心锁定时的异常陷阱值。(RO)

Register 7.49. BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_IADDR_1_REG (*n*=0,1) (0x0080+0x88×n)



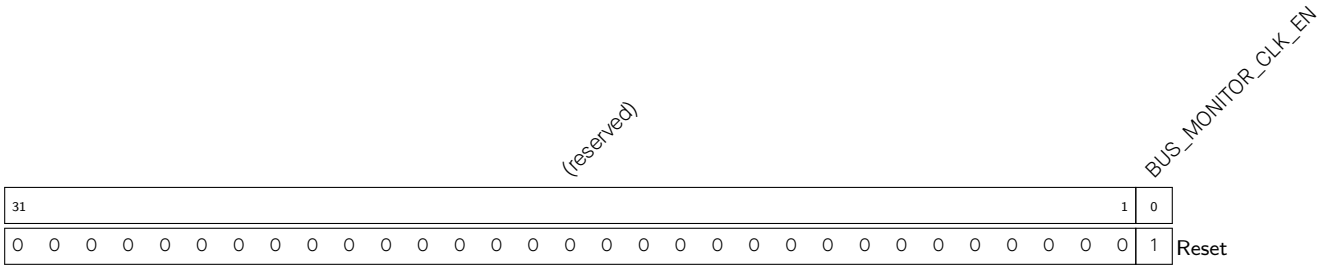
BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_RECORDING_IADDR_1 表示首次核心锁定时的指令 PC。(RO)

Register 7.50. BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_PRIV_1_REG (*n*=0,1) (0x0084+0x88×n)



BUS_MONITOR_CORE_*n*_TRACE_LOCKUP_RECORDING_PRIV_1 表示首次核心锁定时的当前特权级别。(RO)

Register 7.51. BUS_MONITOR_CLOCK_GATE_REG (0x0114)



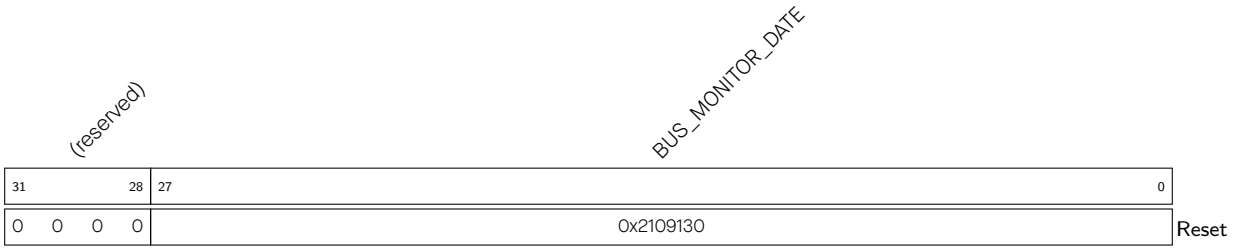
BUS_MONITOR_CLK_EN 配置是否使能寄存器时钟门控。

0：不使能

1：使能

(R/W)

Register 7.52. BUS_MONITOR_DATE_REG (0x03FC)



BUS_MONITOR_DATE 版本控制寄存器。(R/W)

第 8 章

LP 信箱控制器

8.1 概述

ESP32-S31 中包含一个 LP 信箱控制器模块，通过硬件机制实现 LP CPU 和 HP CPU0/1 之间高效的核间通信。LP 信箱控制器模块中包含 16 个 32 位信息寄存器可供 LP CPU 和 HP CPU0/1 存储并传递信息，并通过中断机制实现 LP CPU 和 HP CPU0/1 之间的核间通信。

8.2 特性

ESP32-S31 的 LP 信箱控制器模块具有以下特性：

- 支持多达 16 个 32 位信息寄存器用于核间通信
- 支持 LP CPU 外部中断信号 MB_LP_INTR
- 支持 HP CPU*x* (*x*: 0~1) 外部中断信号 MB_HP_INTR

8.3 功能描述

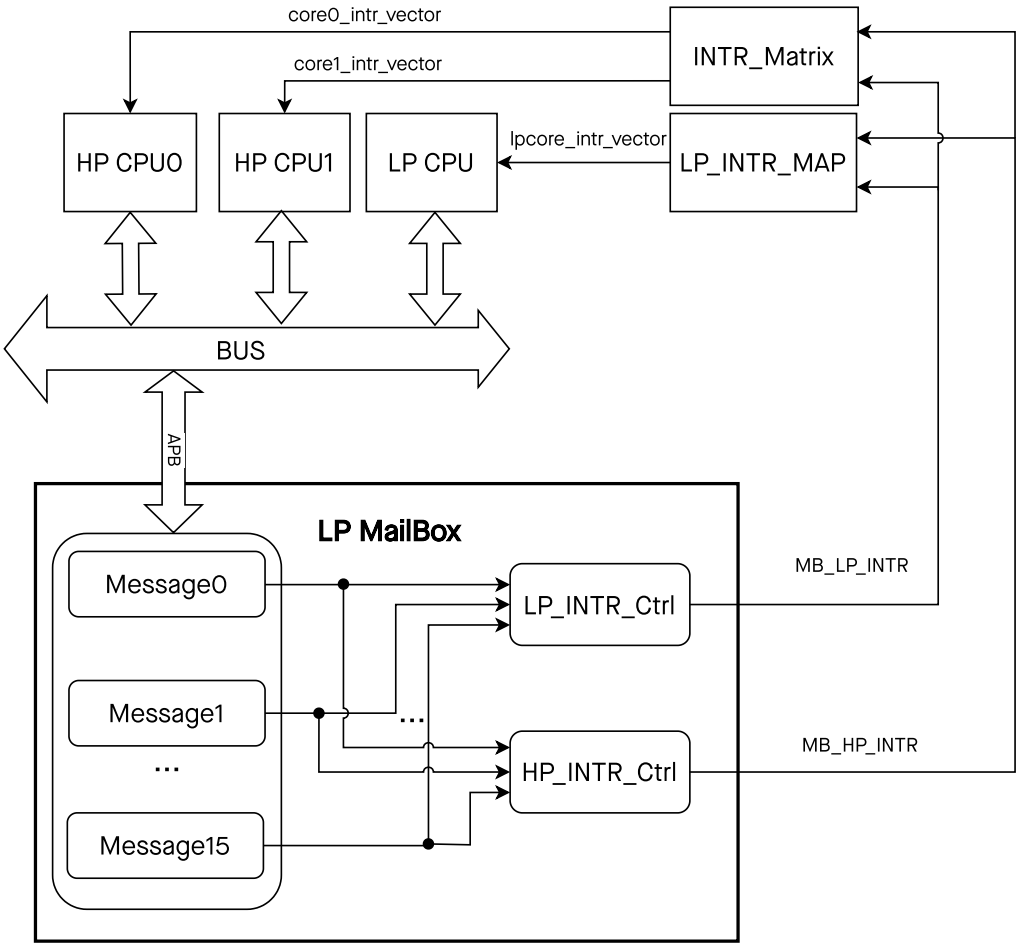


图 8.3-1. LP 信箱控制器系统框图

8.3.1 信息寄存器

LP 信箱控制器中包含了 MB_MESSAGE_0_REG~MB_MESSAGE_15_REG 共计 16 个 32 位信息寄存器，用于缓存核间通信信息。LP CPU 和 HP CPU0/1 均可读写这些信息寄存器以传递或获取信息。

读写信息寄存器通过 APB 总线来完成，使用 LP_PERI_CLK 时钟。该时钟为 LP 信箱控制器使用的唯一时钟，更多时钟信息，见章节 7 复位和时钟 *[to be added later]*。

8.3.2 中断

ESP32-S31 的 LP 信箱控制器可以生成如下外部中断信号：

- MB_LP_INTR
- MB_HP_INTR

MB_LP_INTR 直接映射至 LP CPU（见章节 2 低功耗处理器），MB_HP_INTR 可通过中断矩阵映射至 HP CPU0/1。这些中断信号由 LP 信箱控制器模块的内部中断源生成。表 8.3-1 列出了 LP 信箱控制器的内部中断源、中断使能、中断触发条件、中断清除、以及生成的中断信号。当满足中断使能条件的时候，中断触发条件满足后，内部中断源才会产生。当中断清除条件满足的时候，内部中断源会被清除。

表 8.3-1. LP 信箱控制器的内部中断源

内部中断源	中断使能条件	中断触发条件	中断清除条件	中断信号
MB_HP_*i*_INT	MB_HP_*i*_INT_ENA 为 1	写 MB_MESSAGE_*i*_REG	向 MB_HP_*i*_INT_CLR 写 1	MB_HP_INTR
MB_LP_*i*_INT	MB_LP_*i*_INT_ENA 为 1	写 MB_MESSAGE_*i*_REG	向 MB_LP_*i*_INT_CLR 写 1	MB_LP_INTR

说明：

- *i* 取值范围是 0~15。
- LP 信箱控制器并不识别是哪一个 CPU 在访问它，只要满足表 8.3-1 的触发条件，中断就会产生或被清除。
- MB_HP_INTR 同样可作为 LP CPU 的外部中断源，通过 LP 中断映射模块映射（LP_INTR_MAP，见图 8.3-1）至 LP CPU。有关 LP_INTR_MAP 的更多信息，见章节 2 低功耗处理器。
- MB_LP_INTR 同样可以作为 HP CPU*x* 的外部中断源，可通过中断矩阵将其映射至 HP CPU*x*。
- 当 LP 信箱控制器需要为 HP CPU0 和 HP CPU1 分配不同的中断源时，可通过中断矩阵将 MB_HP_INTR 和 MB_LP_INTR 分别映射至 HP CPU0 和 HP CPU1。

8.3.3 核间通信

当 LP CPU 与 HP CPU*x* 之间需要通信时，可以根据需求利用 LP 信箱控制器中部分或者全部信息寄存器来分享信息，并通过相应的中断来同步信息。在通信期间，需要软件保证仅发送方可写 LP 信箱控制器中的信息寄存器，防止信息寄存器中的数据被篡改。

- 以 HP CPU0 通过 MB_MESSAGE_*i*_REG 向 LP CPU 发起通信为例：
  - 配置 LP 中断映射模块（LP_INTR_MAP，见图 8.3-1），将 MB_LP_*i*_INT 中断映射至 LP CPU。
  - 置位 MB_LP_*i*_INT_ENA 使能 MB_LP_*i*_INT 中断。
  - HP CPU0 写 MB_MESSAGE_*i*_REG，缓存共享信息。
  - LP CPU 感知到中断发生，进入中断处理程序。在中断处理程序中检测 MB_LP_INT_ST_REG 中断状态寄存器发现 MB_LP_*i*_INT_ST 状态位置位，判定发生了 MB_LP_*i*_INT 中断。
  - LP CPU 读取 MB_MESSAGE_*i*_REG 获取共享信息。
  - LP CPU 写 1 到 MB_LP_INT_CLR_REG 寄存器的 MB_LP_*i*_INT_CLR 比特位，会触发中断清除动作。
  - LP CPU 读取 MB_LP_INT_ST_REG 中断状态寄存器，直到其 MB_LP_*i*_ST 状态位清零。然后 LP CPU 会退出中断处理程序。
- 以 LP CPU 向 HP CPU0 发起通信为例：
  - 配置中断矩阵，将 MB_HP_INTR 映射至 HP CPU0（见章节 9 中断矩阵 *[to be added later]*）。
  - 置位 MB_HP_*i*_INT_ENA 使能 MB_HP_*i*_INT 中断。
  - LP CPU 写 MB_MESSAGE_*i*_REG，缓存共享信息。
  - HP CPU0 感知到中断发生，进入中断处理程序。在中断处理程序中检测 MB_HP_INT_ST_REG 中断状态寄存器，发现 MB_HP_*i*_INT_ST 状态位置位，判定发生了 MB_HP_*i*_INT 中断。
  - HP CPU0 读取 MB_MESSAGE_*i*_REG 获取共享信息。

- HP CPU0 写 1 到 MB_HP_INT_CLR_REG 寄存器的 MB_HP_*i*_INT_CLR 比特位，会触发中断清除动作。
- HP CPU0 读取 MB_HP_INT_ST_REG 中断状态寄存器，直到其 MB_HP_*i*_ST 状态位清零。然后 HP CPU0 会退出中断处理程序。

8.4 寄存器列表

本小节的所有地址均为相对于 LP 信箱控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

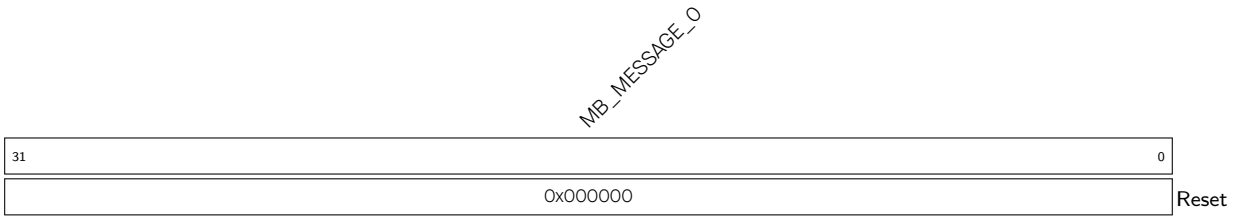
名称	描述	地址	访问
配置寄存器			
MB_MESSAGE_0_REG	信息寄存器 0	0x0000	R/W
MB_MESSAGE_1_REG	信息寄存器 1	0x0004	R/W
MB_MESSAGE_2_REG	信息寄存器 2	0x0008	R/W
MB_MESSAGE_3_REG	信息寄存器 3	0x000C	R/W
MB_MESSAGE_4_REG	信息寄存器 4	0x0010	R/W
MB_MESSAGE_5_REG	信息寄存器 5	0x0014	R/W
MB_MESSAGE_6_REG	信息寄存器 6	0x0018	R/W
MB_MESSAGE_7_REG	信息寄存器 7	0x001C	R/W
MB_MESSAGE_8_REG	信息寄存器 8	0x0020	R/W
MB_MESSAGE_9_REG	信息寄存器 9	0x0024	R/W
MB_MESSAGE_10_REG	信息寄存器 10	0x0028	R/W
MB_MESSAGE_11_REG	信息寄存器 11	0x002C	R/W
MB_MESSAGE_12_REG	信息寄存器 12	0x0030	R/W
MB_MESSAGE_13_REG	信息寄存器 13	0x0034	R/W
MB_MESSAGE_14_REG	信息寄存器 14	0x0038	R/W
MB_MESSAGE_15_REG	信息寄存器 15	0x003C	R/W
MB_REG_CLK_EN_REG	时钟门控寄存器	0x0060	R/W
中断寄存器			
MB_LP_INT_RAW_REG	MB_LP_INTR 原始中断位	0x0040	RO
MB_LP_INT_ST_REG	MB_LP_INTR 中断状态位	0x0044	RO
MB_LP_INT_ENA_REG	MB_LP_INTR 中断使能位	0x0048	R/W
MB_LP_INT_CLR_REG	MB_LP_INTR 中断清除位	0x004C	WO
MB_HP_INT_RAW_REG	MB_HP_INTR 原始中断位	0x0050	RO
MB_HP_INT_ST_REG	MB_HP_INTR 中断状态位	0x0054	RO
MB_HP_INT_ENA_REG	MB_HP_INTR 中断使能位	0x0058	R/W
MB_HP_INT_CLR_REG	MB_HP_INTR 中断清除位	0x005C	WO

8.5 寄存器

本小节的所有地址均为相对于 LP 信箱控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

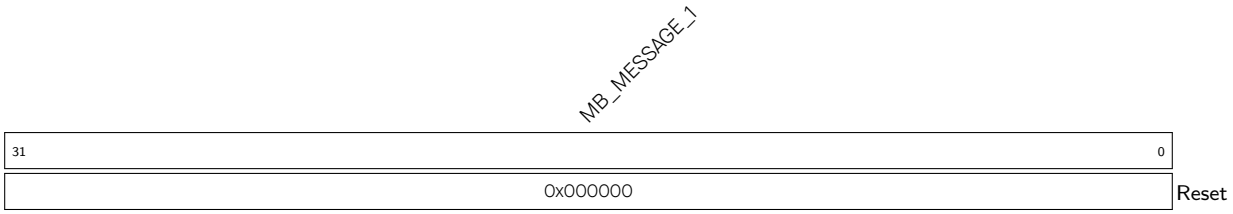
关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 8.1. MB_MESSAGE_0_REG (0x0000)



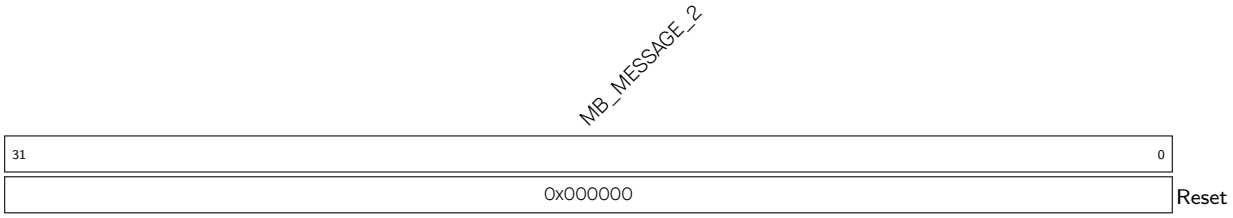
MB_MESSAGE_0 信息寄存器 0。(R/W)

Register 8.2. MB_MESSAGE_1_REG (0x0004)



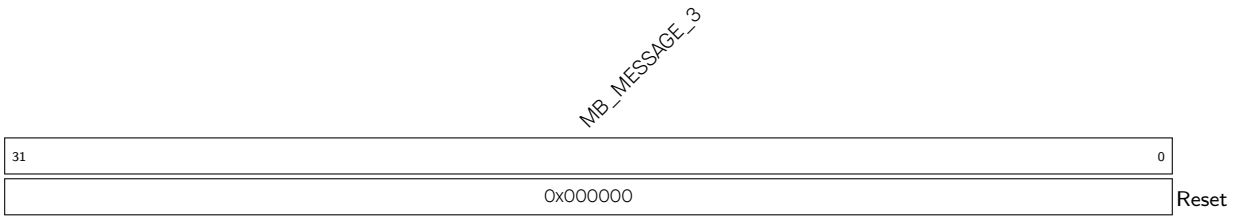
MB_MESSAGE_1 信息寄存器 1。(R/W)

Register 8.3. MB_MESSAGE_2_REG (0x0008)



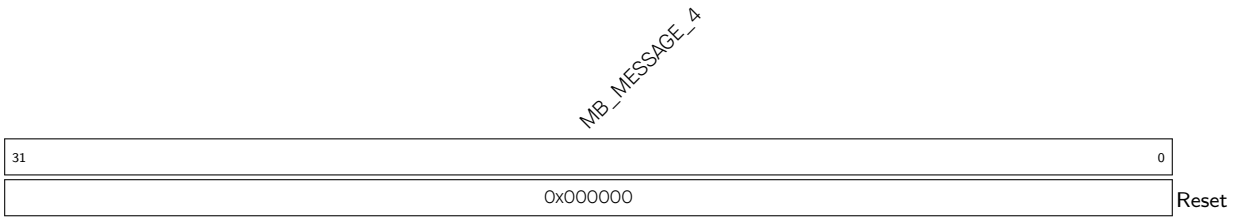
MB_MESSAGE_2 信息寄存器 2。(R/W)

Register 8.4. MB_MESSAGE_3_REG (0x000C)



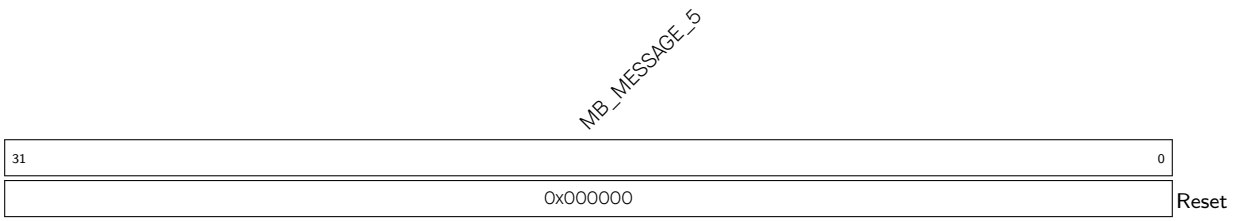
MB_MESSAGE_3 信息寄存器 3。(R/W)

Register 8.5. MB_MESSAGE_4_REG (0x0010)



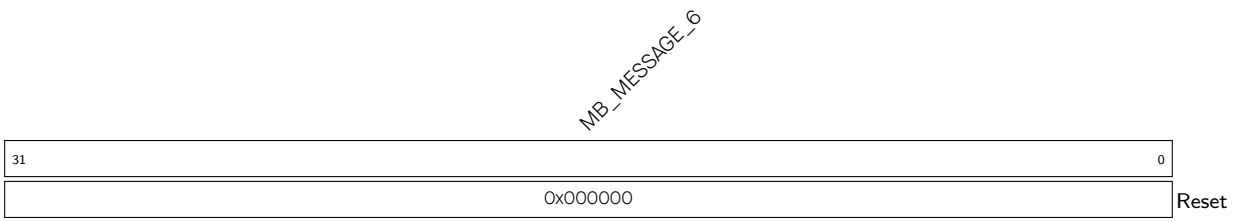
MB_MESSAGE_4 信息寄存器 4。(R/W)

Register 8.6. MB_MESSAGE_5_REG (0x0014)



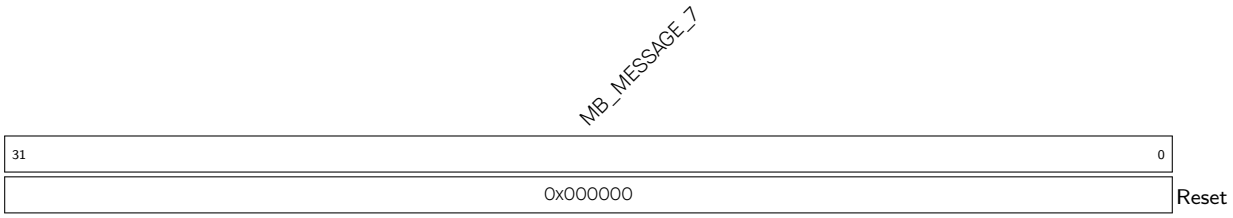
MB_MESSAGE_5 信息寄存器 5。(R/W)

Register 8.7. MB_MESSAGE_6_REG (0x0018)



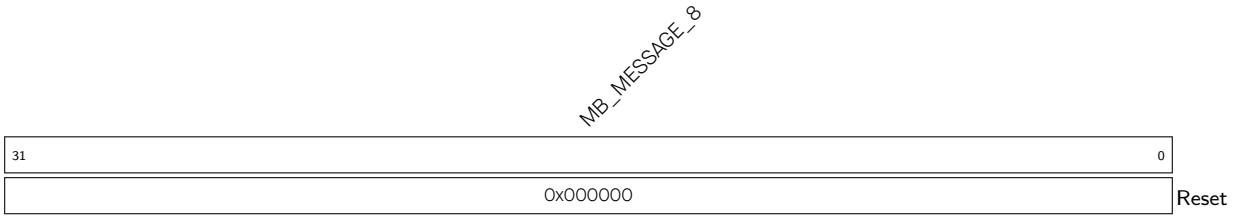
MB_MESSAGE_6 信息寄存器 6。(R/W)

Register 8.8. MB_MESSAGE_7_REG (0x001C)



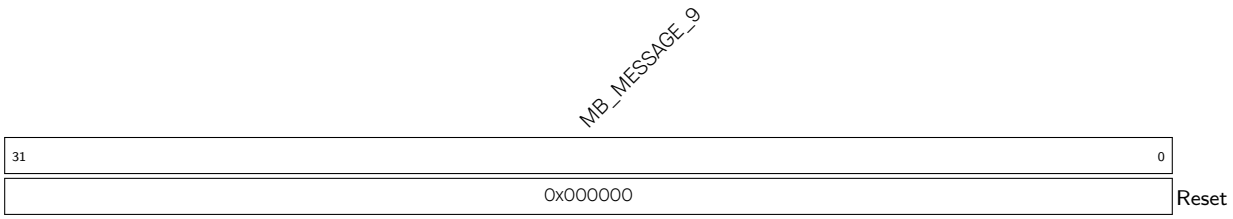
MB_MESSAGE_7 信息寄存器 7。(R/W)

Register 8.9. MB_MESSAGE_8_REG (0x0020)



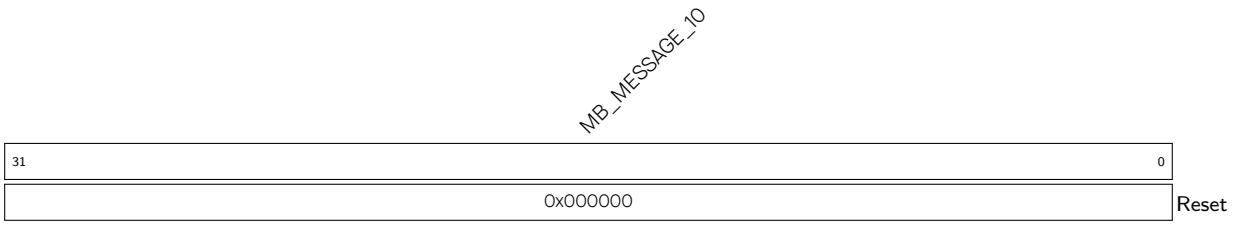
MB_MESSAGE_8 信息寄存器 8。(R/W)

Register 8.10. MB_MESSAGE_9_REG (0x0024)



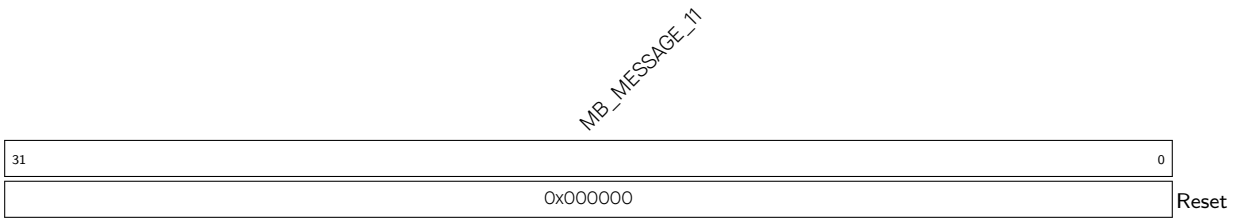
MB_MESSAGE_9 信息寄存器 9。(R/W)

Register 8.11. MB_MESSAGE_10_REG (0x0028)



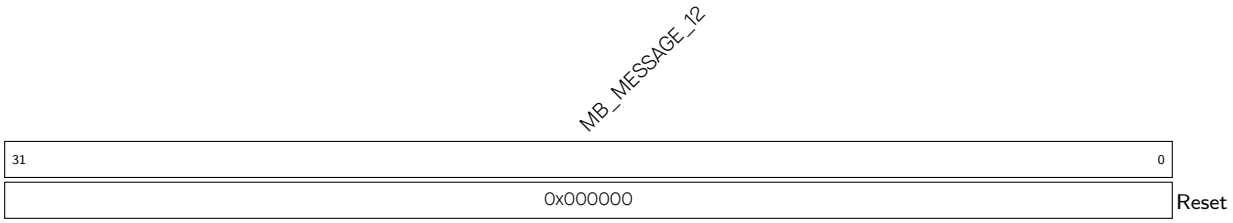
MB_MESSAGE_10 信息寄存器 10。(R/W)

Register 8.12. MB_MESSAGE_11_REG (0x002C)



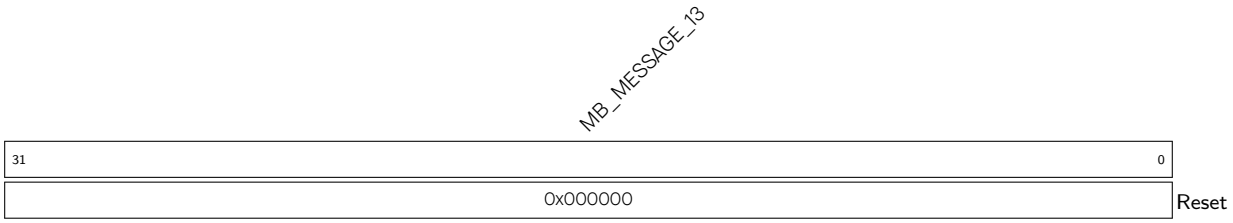
MB_MESSAGE_11 信息寄存器 11。(R/W)

Register 8.13. MB_MESSAGE_12_REG (0x0030)



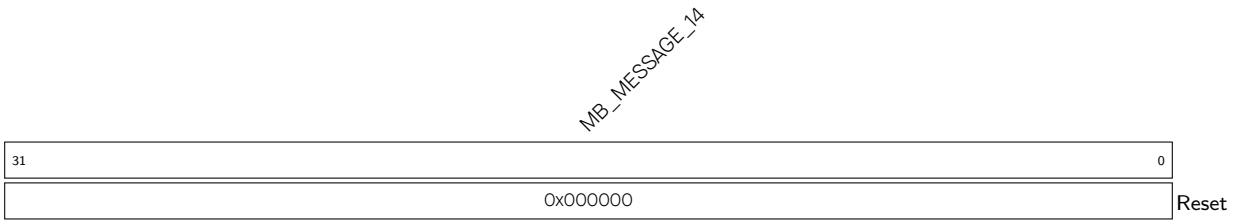
MB_MESSAGE_12 信息寄存器 12。(R/W)

Register 8.14. MB_MESSAGE_13_REG (0x0034)



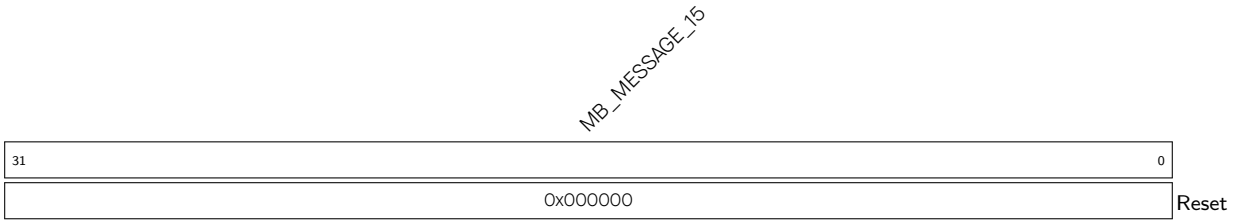
MB_MESSAGE_13 信息寄存器 13。(R/W)

Register 8.15. MB_MESSAGE_14_REG (0x0038)



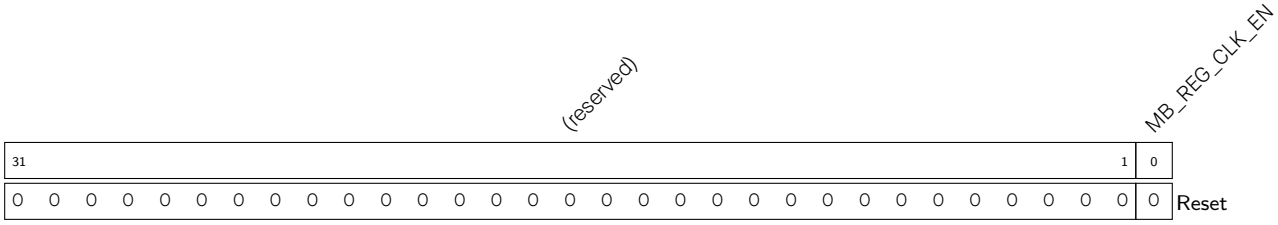
MB_MESSAGE_14 信息寄存器 14。(R/W)

Register 8.16. MB_MESSAGE_15_REG (0x003C)



MB_MESSAGE_15 信息寄存器 15。(R/W)

Register 8.17. MB_REG_CLK_EN_REG (0x0060)



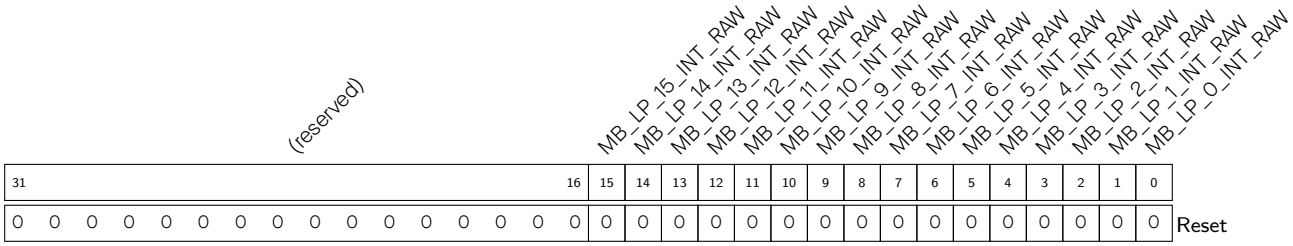
MB_REG_CLK_EN 配置寄存器时钟门控。

1：读写寄存器时，始终开启时钟。

0：仅在读写寄存器操作时，使能时钟。

(R/W)

Register 8.18. MB_LP_INT_RAW_REG (0x0040)



MB_LP_0_INT_RAW MB_LP_0_INT 的原始中断状态。(RO)

MB_LP_1_INT_RAW MB_LP_1_INT 的原始中断状态。(RO)

MB_LP_2_INT_RAW MB_LP_2_INT 的原始中断状态。(RO)

MB_LP_3_INT_RAW MB_LP_3_INT 的原始中断状态。(RO)

MB_LP_4_INT_RAW MB_LP_4_INT 的原始中断状态。(RO)

MB_LP_5_INT_RAW MB_LP_5_INT 的原始中断状态。(RO)

MB_LP_6_INT_RAW MB_LP_6_INT 的原始中断状态。(RO)

MB_LP_7_INT_RAW MB_LP_7_INT 的原始中断状态。(RO)

MB_LP_8_INT_RAW MB_LP_8_INT 的原始中断状态。(RO)

MB_LP_9_INT_RAW MB_LP_9_INT 的原始中断状态。(RO)

MB_LP_10_INT_RAW MB_LP_10_INT 的原始中断状态。(RO)

MB_LP_11_INT_RAW MB_LP_11_INT 的原始中断状态。(RO)

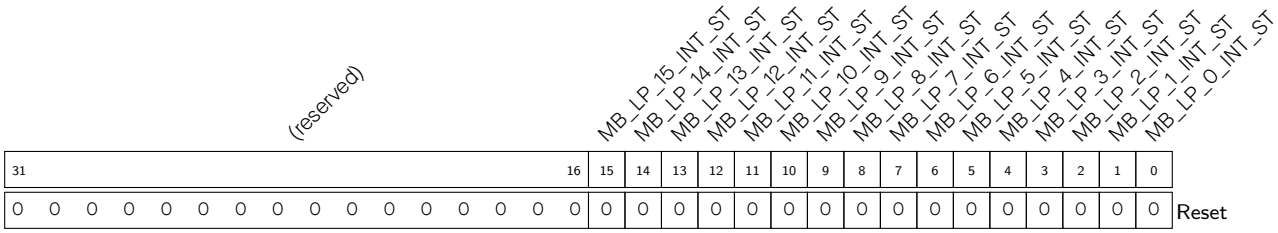
MB_LP_12_INT_RAW MB_LP_12_INT 的原始中断状态。(RO)

MB_LP_13_INT_RAW MB_LP_13_INT 的原始中断状态。(RO)

MB_LP_14_INT_RAW MB_LP_14_INT 的原始中断状态。(RO)

MB_LP_15_INT_RAW MB_LP_15_INT 的原始中断状态。(RO)

Register 8.19. MB_LP_INT_ST_REG (0x0044)



MB_LP_0_INT_ST MB_LP_0_INT 的屏蔽中断状态。(RO)

MB_LP_1_INT_ST MB_LP_1_INT 的屏蔽中断状态。(RO)

MB_LP_2_INT_ST MB_LP_2_INT 的屏蔽中断状态。(RO)

MB_LP_3_INT_ST MB_LP_3_INT 的屏蔽中断状态。(RO)

MB_LP_4_INT_ST MB_LP_4_INT 的屏蔽中断状态。(RO)

MB_LP_5_INT_ST MB_LP_5_INT 的屏蔽中断状态。(RO)

MB_LP_6_INT_ST MB_LP_6_INT 的屏蔽中断状态。(RO)

MB_LP_7_INT_ST MB_LP_7_INT 的屏蔽中断状态。(RO)

MB_LP_8_INT_ST MB_LP_8_INT 的屏蔽中断状态。(RO)

MB_LP_9_INT_ST MB_LP_9_INT 的屏蔽中断状态。(RO)

MB_LP_10_INT_ST MB_LP_10_INT 的屏蔽中断状态。(RO)

MB_LP_11_INT_ST MB_LP_11_INT 的屏蔽中断状态。(RO)

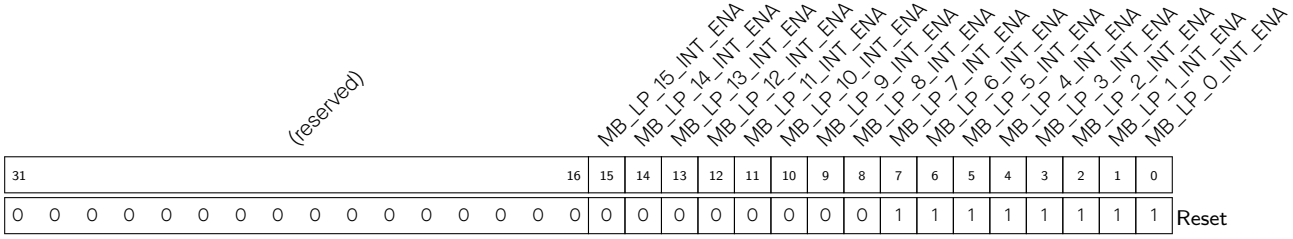
MB_LP_12_INT_ST MB_LP_12_INT 的屏蔽中断状态。(RO)

MB_LP_13_INT_ST MB_LP_13_INT 的屏蔽中断状态。(RO)

MB_LP_14_INT_ST MB_LP_14_INT 的屏蔽中断状态。(RO)

MB_LP_15_INT_ST MB_LP_15_INT 的屏蔽中断状态。(RO)

Register 8.20. MB_LP_INT_ENA_REG (0x0048)



MB_LP_0_INT_ENA 写 1 使能 MB_LP_0_INT 中断。(R/W)

MB_LP_1_INT_ENA 写 1 使能 MB_LP_1_INT 中断。(R/W)

MB_LP_2_INT_ENA 写 1 使能 MB_LP_2_INT 中断。(R/W)

MB_LP_3_INT_ENA 写 1 使能 MB_LP_3_INT 中断。(R/W)

MB_LP_4_INT_ENA 写 1 使能 MB_LP_4_INT 中断。(R/W)

MB_LP_5_INT_ENA 写 1 使能 MB_LP_5_INT 中断。(R/W)

MB_LP_6_INT_ENA 写 1 使能 MB_LP_6_INT 中断。(R/W)

MB_LP_7_INT_ENA 写 1 使能 MB_LP_7_INT 中断。(R/W)

MB_LP_8_INT_ENA 写 1 使能 MB_LP_8_INT 中断。(R/W)

MB_LP_9_INT_ENA 写 1 使能 MB_LP_9_INT 中断。(R/W)

MB_LP_10_INT_ENA 写 1 使能 MB_LP_10_INT 中断。(R/W)

MB_LP_11_INT_ENA 写 1 使能 MB_LP_11_INT 中断。(R/W)

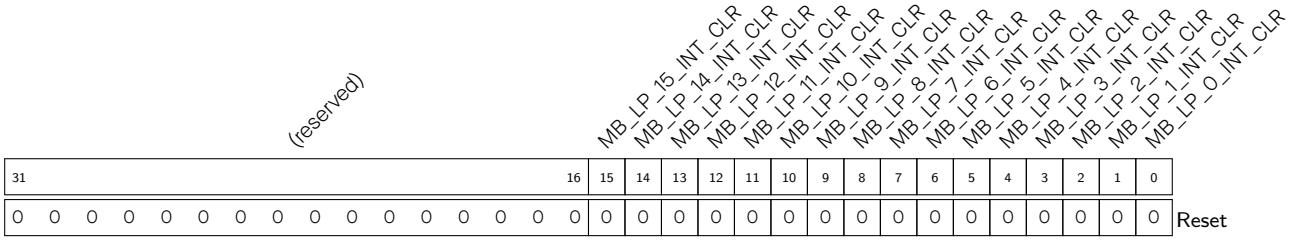
MB_LP_12_INT_ENA 写 1 使能 MB_LP_12_INT 中断。(R/W)

MB_LP_13_INT_ENA 写 1 使能 MB_LP_13_INT 中断。(R/W)

MB_LP_14_INT_ENA 写 1 使能 MB_LP_14_INT 中断。(R/W)

MB_LP_15_INT_ENA 写 1 使能 MB_LP_15_INT 中断。(R/W)

Register 8.21. MB_LP_INT_CLR_REG (0x004C)



MB_LP_0_INT_CLR 写 1 清除 MB_LP_0_INT 中断。(WO)

MB_LP_1_INT_CLR 写 1 清除 MB_LP_1_INT 中断。(WO)

MB_LP_2_INT_CLR 写 1 清除 MB_LP_2_INT 中断。(WO)

MB_LP_3_INT_CLR 写 1 清除 MB_LP_3_INT 中断。(WO)

MB_LP_4_INT_CLR 写 1 清除 MB_LP_4_INT 中断。(WO)

MB_LP_5_INT_CLR 写 1 清除 MB_LP_5_INT 中断。(WO)

MB_LP_6_INT_CLR 写 1 清除 MB_LP_6_INT 中断。(WO)

MB_LP_7_INT_CLR 写 1 清除 MB_LP_7_INT 中断。(WO)

MB_LP_8_INT_CLR 写 1 清除 MB_LP_8_INT 中断。(WO)

MB_LP_9_INT_CLR 写 1 清除 MB_LP_9_INT 中断。(WO)

MB_LP_10_INT_CLR 写 1 清除 MB_LP_10_INT 中断。(WO)

MB_LP_11_INT_CLR 写 1 清除 MB_LP_11_INT 中断。(WO)

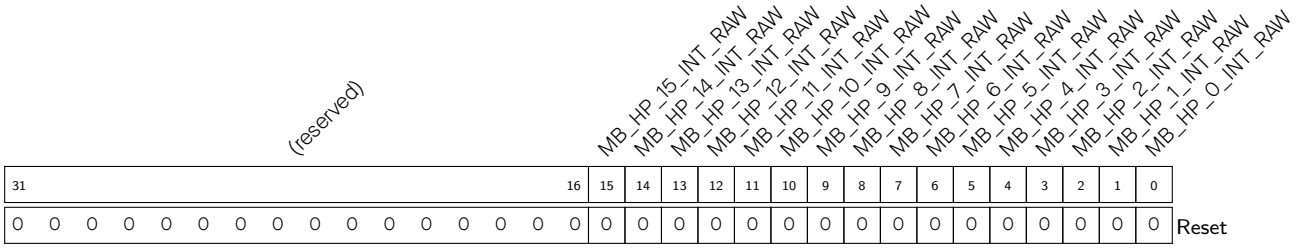
MB_LP_12_INT_CLR 写 1 清除 MB_LP_12_INT 中断。(WO)

MB_LP_13_INT_CLR 写 1 清除 MB_LP_13_INT 中断。(WO)

MB_LP_14_INT_CLR 写 1 清除 MB_LP_14_INT 中断。(WO)

MB_LP_15_INT_CLR 写 1 清除 MB_LP_15_INT 中断。(WO)

Register 8.22. MB_HP_INT_RAW_REG (0x0050)



MB_HP_0_INT_RAW MB_HP_0_INT 的原始中断状态。(RO)

MB_HP_1_INT_RAW MB_HP_1_INT 的原始中断状态。(RO)

MB_HP_2_INT_RAW MB_HP_2_INT 的原始中断状态。(RO)

MB_HP_3_INT_RAW MB_HP_3_INT 的原始中断状态。(RO)

MB_HP_4_INT_RAW MB_HP_4_INT 的原始中断状态。(RO)

MB_HP_5_INT_RAW MB_HP_5_INT 的原始中断状态。(RO)

MB_HP_6_INT_RAW MB_HP_6_INT 的原始中断状态。(RO)

MB_HP_7_INT_RAW MB_HP_7_INT 的原始中断状态。(RO)

MB_HP_8_INT_RAW MB_HP_8_INT 的原始中断状态。(RO)

MB_HP_9_INT_RAW MB_HP_9_INT 的原始中断状态。(RO)

MB_HP_10_INT_RAW MB_HP_10_INT 的原始中断状态。(RO)

MB_HP_11_INT_RAW MB_HP_11_INT 的原始中断状态。(RO)

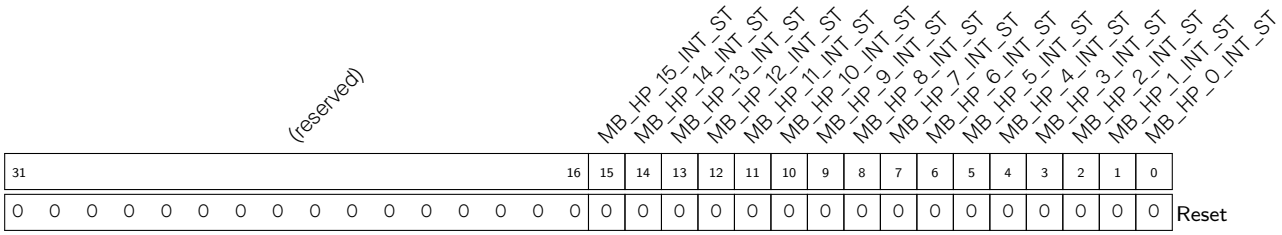
MB_HP_12_INT_RAW MB_HP_12_INT 的原始中断状态。(RO)

MB_HP_13_INT_RAW MB_HP_13_INT 的原始中断状态。(RO)

MB_HP_14_INT_RAW MB_HP_14_INT 的原始中断状态。(RO)

MB_HP_15_INT_RAW MB_HP_15_INT 的原始中断状态。(RO)

Register 8.23. MB_HP_INT_ST_REG (0x0054)



MB_HP_0_INT_ST MB_HP_0_INT 的屏蔽中断状态。(RO)

MB_HP_1_INT_ST MB_HP_1_INT 的屏蔽中断状态。(RO)

MB_HP_2_INT_ST MB_HP_2_INT 的屏蔽中断状态。(RO)

MB_HP_3_INT_ST MB_HP_3_INT 的屏蔽中断状态。(RO)

MB_HP_4_INT_ST MB_HP_4_INT 的屏蔽中断状态。(RO)

MB_HP_5_INT_ST MB_HP_5_INT 的屏蔽中断状态。(RO)

MB_HP_6_INT_ST MB_HP_6_INT 的屏蔽中断状态。(RO)

MB_HP_7_INT_ST MB_HP_7_INT 的屏蔽中断状态。(RO)

MB_HP_8_INT_ST MB_HP_8_INT 的屏蔽中断状态。(RO)

MB_HP_9_INT_ST MB_HP_9_INT 的屏蔽中断状态。(RO)

MB_HP_10_INT_ST MB_HP_10_INT 的屏蔽中断状态。(RO)

MB_HP_11_INT_ST MB_HP_11_INT 的屏蔽中断状态。(RO)

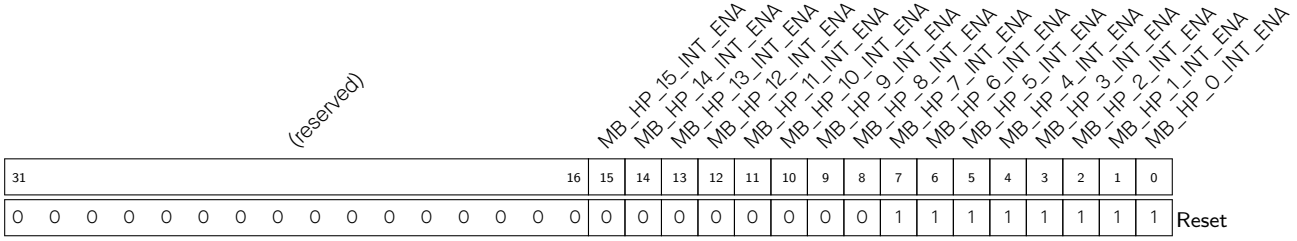
MB_HP_12_INT_ST MB_HP_12_INT 的屏蔽中断状态。(RO)

MB_HP_13_INT_ST MB_HP_13_INT 的屏蔽中断状态。(RO)

MB_HP_14_INT_ST MB_HP_14_INT 的屏蔽中断状态。(RO)

MB_HP_15_INT_ST MB_HP_15_INT 的屏蔽中断状态。(RO)

Register 8.24. MB_HP_INT_ENA_REG (0x0058)



MB_HP_0_INT_ENA 写 1 使能 MB_HP_0_INT 中断。(R/W)

MB_HP_1_INT_ENA 写 1 使能 MB_HP_1_INT 中断。(R/W)

MB_HP_2_INT_ENA 写 1 使能 MB_HP_2_INT 中断。(R/W)

MB_HP_3_INT_ENA 写 1 使能 MB_HP_3_INT 中断。(R/W)

MB_HP_4_INT_ENA 写 1 使能 MB_HP_4_INT 中断。(R/W)

MB_HP_5_INT_ENA 写 1 使能 MB_HP_5_INT 中断。(R/W)

MB_HP_6_INT_ENA 写 1 使能 MB_HP_6_INT 中断。(R/W)

MB_HP_7_INT_ENA 写 1 使能 MB_HP_7_INT 中断。(R/W)

MB_HP_8_INT_ENA 写 1 使能 MB_HP_8_INT 中断。(R/W)

MB_HP_9_INT_ENA 写 1 使能 MB_HP_9_INT 中断。(R/W)

MB_HP_10_INT_ENA 写 1 使能 MB_HP_10_INT 中断。(R/W)

MB_HP_11_INT_ENA 写 1 使能 MB_HP_11_INT 中断。(R/W)

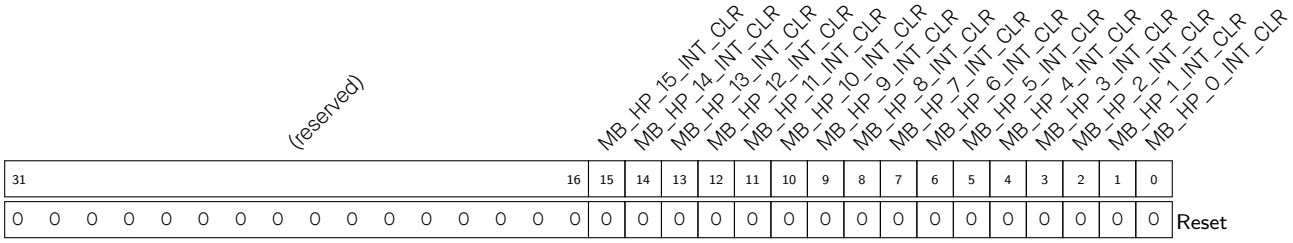
MB_HP_12_INT_ENA 写 1 使能 MB_HP_12_INT 中断。(R/W)

MB_HP_13_INT_ENA 写 1 使能 MB_HP_13_INT 中断。(R/W)

MB_HP_14_INT_ENA 写 1 使能 MB_HP_14_INT 中断。(R/W)

MB_HP_15_INT_ENA 写 1 使能 MB_HP_15_INT 中断。(R/W)

Register 8.25. MB_HP_INT_CLR_REG (0x005C)



MB_HP_0_INT_CLR 写 1 清除 MB_HP_0_INT 中断。(WO)

MB_HP_1_INT_CLR 写 1 清除 MB_HP_1_INT 中断。(WO)

MB_HP_2_INT_CLR 写 1 清除 MB_HP_2_INT 中断。(WO)

MB_HP_3_INT_CLR 写 1 清除 MB_HP_3_INT 中断。(WO)

MB_HP_4_INT_CLR 写 1 清除 MB_HP_4_INT 中断。(WO)

MB_HP_5_INT_CLR 写 1 清除 MB_HP_5_INT 中断。(WO)

MB_HP_6_INT_CLR 写 1 清除 MB_HP_6_INT 中断。(WO)

MB_HP_7_INT_CLR 写 1 清除 MB_HP_7_INT 中断。(WO)

MB_HP_8_INT_CLR 写 1 清除 MB_HP_8_INT 中断。(WO)

MB_HP_9_INT_CLR 写 1 清除 MB_HP_9_INT 中断。(WO)

MB_HP_10_INT_CLR 写 1 清除 MB_HP_10_INT 中断。(WO)

MB_HP_11_INT_CLR 写 1 清除 MB_HP_11_INT 中断。(WO)

MB_HP_12_INT_CLR 写 1 清除 MB_HP_12_INT 中断。(WO)

MB_HP_13_INT_CLR 写 1 清除 MB_HP_13_INT 中断。(WO)

MB_HP_14_INT_CLR 写 1 清除 MB_HP_14_INT 中断。(WO)

MB_HP_15_INT_CLR 写 1 清除 MB_HP_15_INT 中断。(WO)

第 9 章

电源检测器 (PSDET)

9.1 概述

ESP32-S31 芯片上的电源检测器能够监测片上时钟和数字电路相关的电源管脚上的电压。它包含一个欠压检测器和四个电压毛刺检测器。欠压检测器通过触发欠压复位，防止 SoC 在低电压条件下工作；电压毛刺检测器用于防止电压毛刺攻击。电源检测器属于 always-on（常开）电源区域，可以在任何时刻监控电压。

9.2 主要特性

- 欠压检测器支持两种欠压检测模式（模式 0 和模式 1）
  - 模式 0 支持如下动作：
    * 发出中断
    * 关闭射频电路
    * 请求 flash 进入 suspend 状态
    * 发出系统复位
  - 模式 1 支持如下动作：
    * 发出系统复位直到电压恢复正常
- 电压毛刺检测器能够检测持续 50 ns 及以上的电压毛刺并触发系统复位。

9.3 功能描述

9.3.1 架构图

图 9.3-1 为电源检测器的架构图。由图可知，电源检测器包含一个欠压检测器 (Brown-out Detector) 和四个电压毛刺检测器 (Voltage Glitch Detector)。其中，VDDPST_* 为 VDDPST2/3/4。

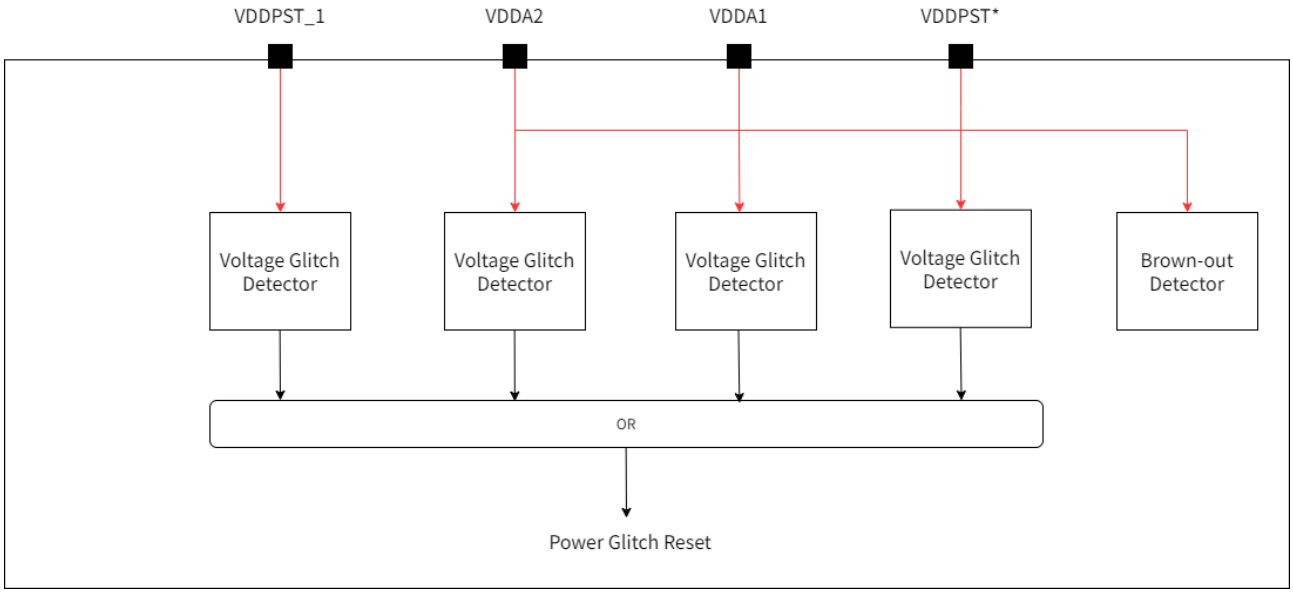


图 9.3-1. 电源检测器架构图

9.3.2 欠压检测器

ESP32-S31 的欠压检测器可以周期性轮流检查管脚 VDDPST_1、VDDA2、VDDPST_* 的电压，周期约 280 μs。一旦被检测电压下落至预设阈值（默认为 2.7 V）以下时，欠压检测器发出触发信号，并进行相应处理，关闭功耗较高的射频电路，为数字模块争取更多时间，用以保存、转移重要数据。欠压检测器的功耗非常低，在芯片开启时将永远保持开启。LP_ANA_BOD_MODE0_INT_RAW 用于指示欠压检测器的输出结果，默认为 0，可在检测管脚电压下降至阈值以下时置位。

欠压检测器检测到欠压信号后，有两种处理模式（默认为模式 1）：

- 模式 0：
  - 此模式需通过 `bod_mode0_en` (LP_ANA_BOD_MODE0_INTR_ENA) 使能。
  - 当欠压计数器达到 Int Comparer (LP_ANA_BOD_MODE0_INTR_WAIT) 中设定的阈值后触发中断，如果使能了 LP_ANA_BOD_MODE0_CLOSE_FLASH_ENA，将会使 flash 进入 SUSPEND 状态；如果使能了 LP_ANA_BOD_MODE0_PD_RF_ENA，将会关闭射频模块。
  - 当欠压计数器达到 Rst Comparer (LP_ANA_BOD_MODE0_RESET_WAIT) 设定的阈值后根据 `bod_mode0_rst_sel` (LP_ANA_BOD_MODE0_RESET_SEL) 的配置选择复位方式。复位需通过 `bod_mode0_rst_en` (LP_ANA_BOD_MODE0_RESET_ENA) 使能。
- 模式 1：直接触发系统复位。

详情如下图所示。

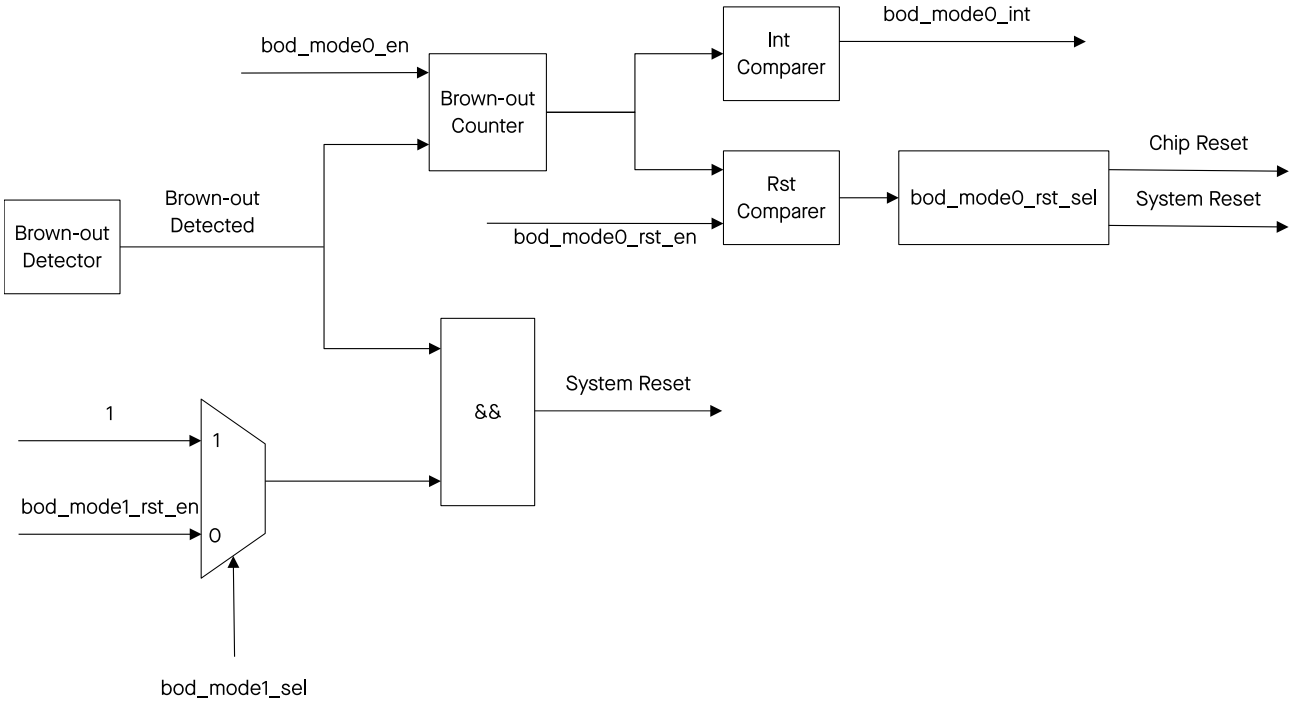


图 9.3-2. 欠压检测器处理方式

图中的选择信号均由寄存器控制，对应关系如下：

- `bod_mode0_en`：LP_ANA_BOD_MODE0_INTR_ENA
- `bod_mode0_rst_en`：LP_ANA_BOD_MODE0_RESET_ENA
- `bod_mode0_rst_sel`：LP_ANA_BOD_MODE0_RESET_SEL 用于选择模式 0 欠压检测器被触发后的复位方式：
  - 0：芯片复位
  - 1：系统复位

  更多有关复位的信息请参考章节 7 复位和时钟 *[to be added later]*。
- `bod_mode1_sel`：LP_ANA_ANA_FIB_ENA 的 bit1
- `bod_mode1_rst_en`：LP_ANA_BOD_MODE1_RESET_ENA

9.3.3 电压毛刺检测器

ESP32-S31 的电压毛刺检测器的结构如下图所示：

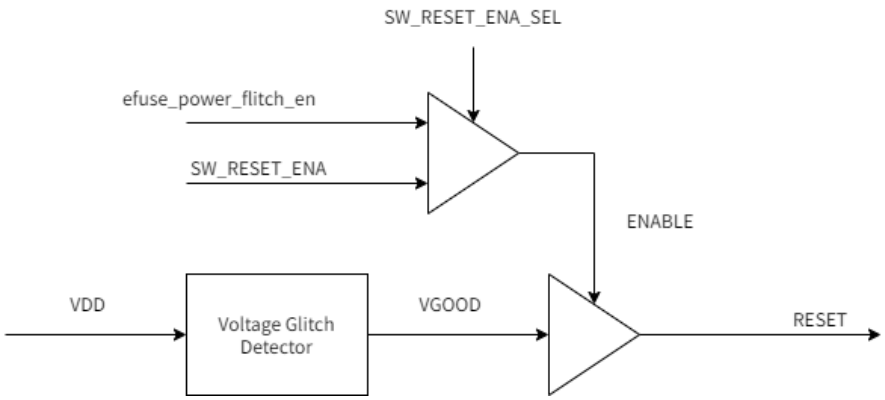


图 9.3-3. 电压毛刺检测器

ESP32-S31 有 4 个电压毛刺检测器，每个检测器可以检测对应电源上持续 50 ns 及以上的毛刺。各检测器对应的电源管脚为 VDDPST2/3/4、VDDA1、VDDA2、VDDPST_1，其中 VDDPST2 和 VDDPST3 互连，因此可以共用一个检测器。所有检测器产生的复位合并为一个电压毛刺复位。电压毛刺检测器的使能控制由 SW_RESET_ENA_SEL 控制，选择为 1 时，由硬件强制开启，选 0 时，由软件寄存器 SW_RESET_ENA 控制。对于不同的电源管脚检测的配置如下表所示：

表 9.3-1. 电源管脚检测的配置

VDD 管脚	SW_RESET_ENA	SW_RESET_ENA_SEL
VDDPST2/3/4	LP_ANA_POWER_GLITCH_RESET_ENA[0]	LP_ANA_ANA_FIB_ENA[2]
VDDA1	LP_ANA_POWER_GLITCH_RESET_ENA[1]	LP_ANA_ANA_FIB_ENA[3]
VDDA2	LP_ANA_POWER_GLITCH_RESET_ENA[2]	LP_ANA_ANA_FIB_ENA[4]
VDDPST_1	LP_ANA_POWER_GLITCH_RESET_ENA[3]	LP_ANA_ANA_FIB_ENA[5]

9.4 中断

ESP32-S31 的电源检测器可以生成以下中断信号并发送给 中断矩阵。

- LP_ANA_INTR（该中断供 HP CPU 使用）
- LP_ANA_LP_INTR（该中断供 LP CPU 使用）

这些中断信号由电源检测器的内部中断源生成。表 9.4-1 列出了电源检测器的内部中断源、中断触发条件、以及生成的中断信号。

表 9.4-1. 电源检测器的内部中断源

内部中断源	触发条件	中断信号
LP_ANA_BOD_MODE0_INT	欠压检测器检测到电压低于阈值时触发	LP_ANA_INTR
LP_ANA_BOD_MODE0_LP_INT	欠压检测器检测到电压低于阈值时触发	LP_ANA_LP_INTR

说明：
中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 9.5 寄存器列表。

9.5 寄存器列表

本小节的所有地址均为相对于电源检测器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

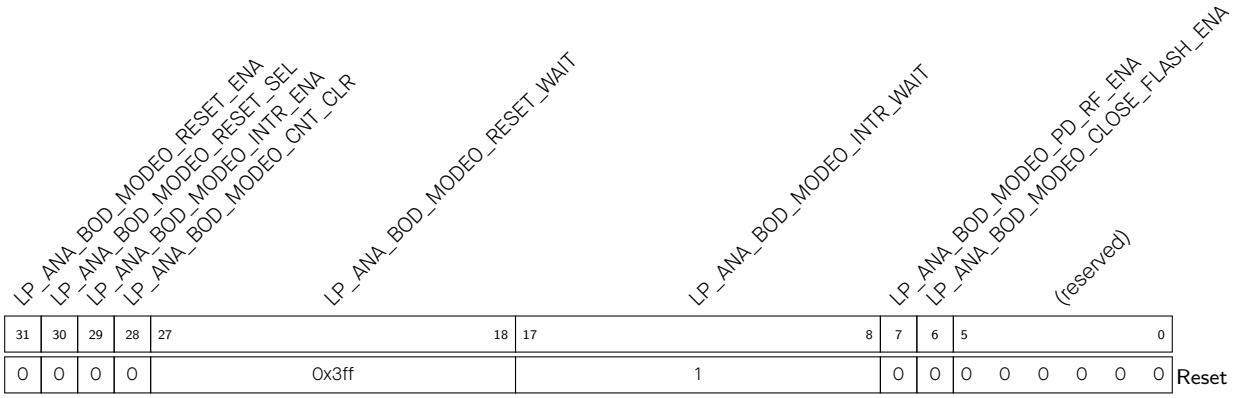
名称	描述	地址	访问
配置寄存器			
LP_ANA_BOD_MODE0_CNTL_REG	配置欠压检测器模式 0	0x0000	R/W
LP_ANA_BOD_MODE1_CNTL_REG	配置欠压检测器模式 1	0x0004	R/W
LP_ANA_PG_GLITCH_CNTL_REG	配置毛刺检测器的复位使能	0x0018	R/W
LP_ANA_FIB_ENABLE_REG	配置电压毛刺检测器的使能控制	0x001C	R/W
LP_ANA_INT_RAW_REG	LP_ANA_BOD_MODE0_INT 原始中断状态	0x0020	R/WTC/SS
LP_ANA_INT_ST_REG	LP_ANA_BOD_MODE0_INT 屏蔽中断状态	0x0024	RO
LP_ANA_INT_ENA_REG	LP_ANA_BOD_MODE0_INT 使能中断状态	0x0028	R/W
LP_ANA_INT_CLR_REG	LP_ANA_BOD_MODE0_INT 中断清除	0x002C	WT
LP_ANA_LP_INT_RAW_REG	LP_ANA_BOD_MODE0_LP_INT 原始中断状态	0x0030	R/WTC/SS
LP_ANA_LP_INT_ST_REG	LP_ANA_BOD_MODE0_LP_INT 屏蔽中断状态	0x0034	RO
LP_ANA_LP_INT_ENA_REG	LP_ANA_BOD_MODE0_LP_INT 使能中断状态	0x0038	R/W
LP_ANA_LP_INT_CLR_REG	LP_ANA_BOD_MODE0_LP_INT 中断清除	0x003C	WT
版本控制寄存器			
LP_ANA_DATE_REG	版本控制寄存器	0x03FC	R/W

9.6 寄存器

本小节的所有地址均为相对于电源检测器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 9.1. LP_ANA_BOD_MODE0_CNTL_REG (0x0000)



LP_ANA_BOD_MODE0_CLOSE_FLASH_ENA 配置是否使能欠压检测器触发 flash 挂起。

0：禁用

1：使能

(R/W)

LP_ANA_BOD_MODE0_PD_RF_ENA 配置是否使能欠压检测器关闭射频模块。

0：禁用

1：使能

(R/W)

LP_ANA_BOD_MODE0_INTR_WAIT 配置欠压信号有效后，产生中断的时间，单位是 LP_FAST_CLK 周期。(R/W)

LP_ANA_BOD_MODE0_RESET_WAIT 配置欠压信号有效后，产生复位的时间，单位是 LP_FAST_CLK 周期。(R/W)

LP_ANA_BOD_MODE0_CNT_CLR 配置是否清空欠压检测器计数器。

0：不清空

1：清空

(R/W)

LP_ANA_BOD_MODE0_INTR_ENA 配置是否使能欠压检测器模式 0 的中断。此域为 1 时 LP_ANA_BOD_MODE0_INT_RAW 和 LP_ANA_BOD_MODE0_LP_INT_RAW 才有效。

0：禁用

1：使能

(R/W)

LP_ANA_BOD_MODE0_RESET_SEL 配置欠压检测器被触发后的复位方式。

0：芯片复位

1：系统复位

(R/W)

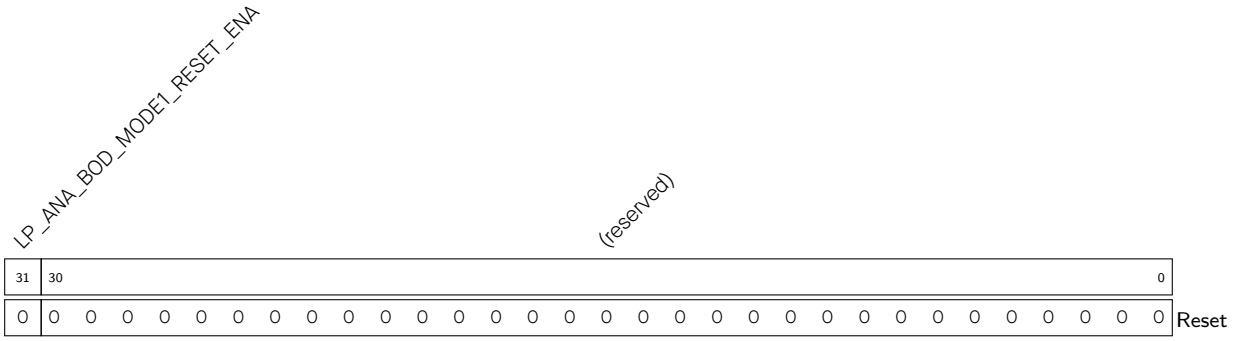
LP_ANA_BOD_MODE0_RESET_ENA 配置是否使能欠压检测器复位。

0：禁用

1：使能

(R/W)

Register 9.2. LP_ANA_BOD_MODE1_CNTL_REG (0x0004)



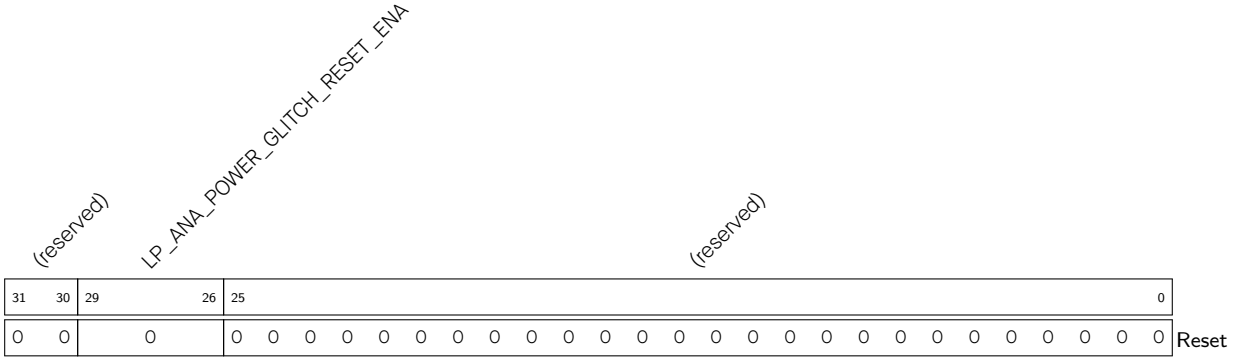
LP_ANA_BOD_MODE1_RESET_ENA 配置是否使能欠压检测器模式 1。

0：禁用

1：使能

(R/W)

Register 9.3. LP_ANA_PG_GLITCH_CNTL_REG (0x0018)



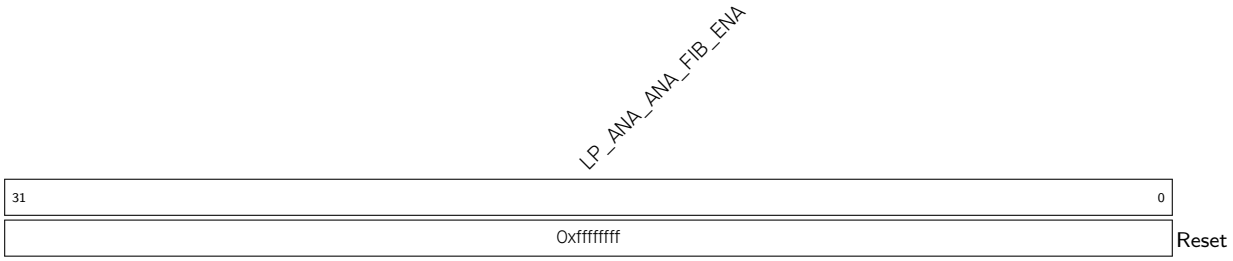
LP_ANA_POWER_GLITCH_RESET_ENA 控制电压毛刺的复位使能位图。

0：禁止复位

1：使能复位

(R/W)

Register 9.4. LP_ANA_FIB_ENABLE_REG (0x001C)



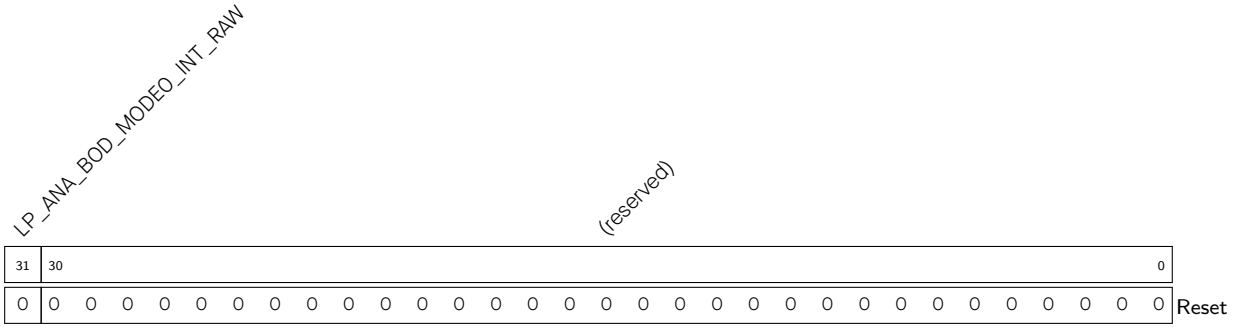
LP_ANA_ANA_FIB_ENA 配置电压毛刺检测器和欠压检测器的使能控制方式。其中 bit2，bit3，bit4，bit5 分别对应 VDDPST_2/3/4，VDDA1，VDDA2，VDDPST_1；bit1 对应欠压检测器。

0：bit2 ~bit5 由 LP_ANA_POWER_GLITCH_RESET_ENA 控制; bit1 由 LP_ANA_BOD_MODE1_RESET_ENA 控制

1：由硬件强制开启

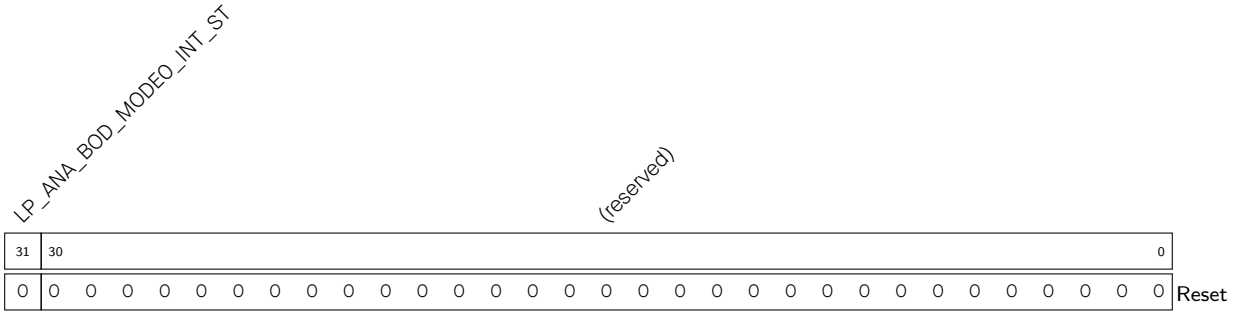
(R/W)

Register 9.5. LP_ANA_INT_RAW_REG (0x0020)



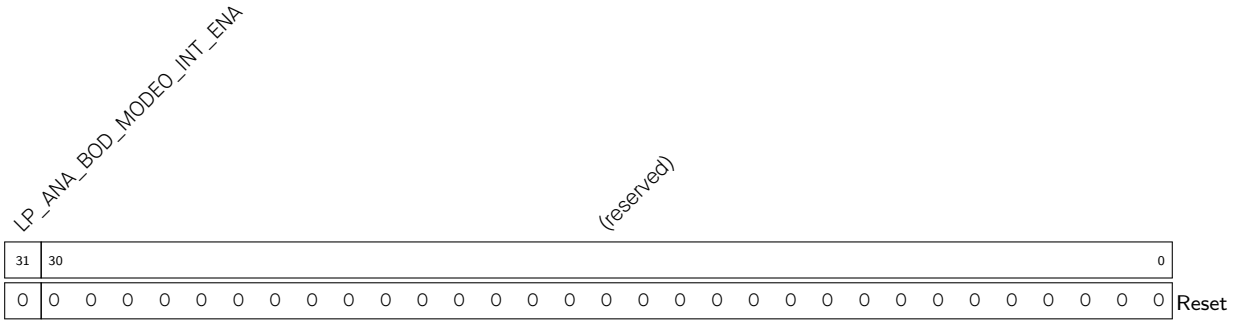
LP_ANA_BOD_MODE0_INT_RAW LP_ANA_BOD_MODE0_INT 的原始中断状态。(R/WTC/SS)

Register 9.6. LP_ANA_INT_ST_REG (0x0024)



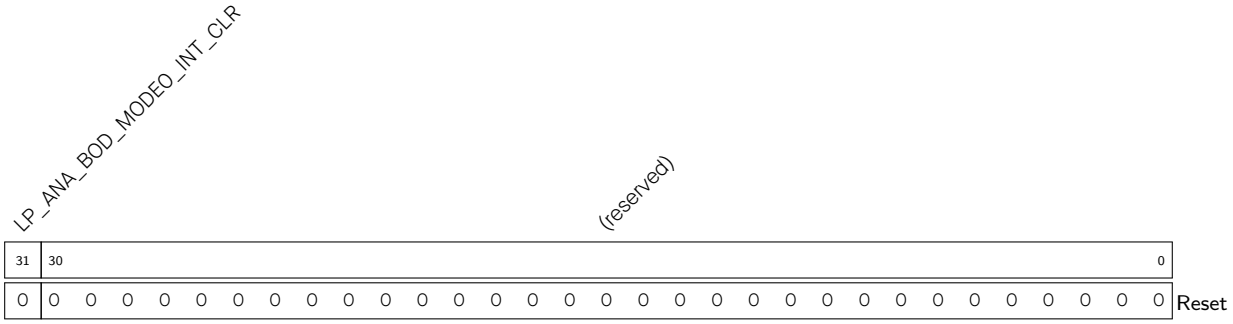
LP_ANA_BOD_MODE0_INT_ST LP_ANA_BOD_MODE0_INT 的屏蔽中断状态。(RO)

Register 9.7. LP_ANA_INT_ENA_REG (0x0028)



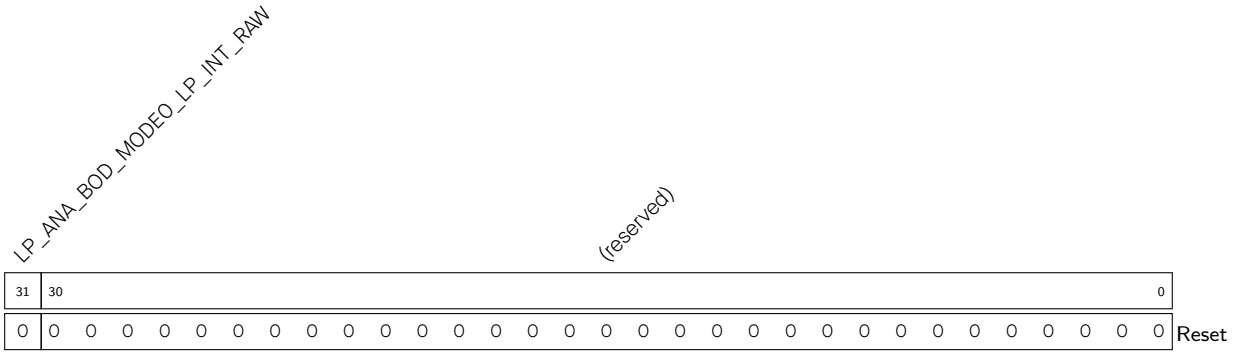
LP_ANA_BOD_MODE0_INT_ENA 写 1 使能 LP_ANA_BOD_MODE0_INT。(R/W)

Register 9.8. LP_ANA_INT_CLR_REG (0x002C)



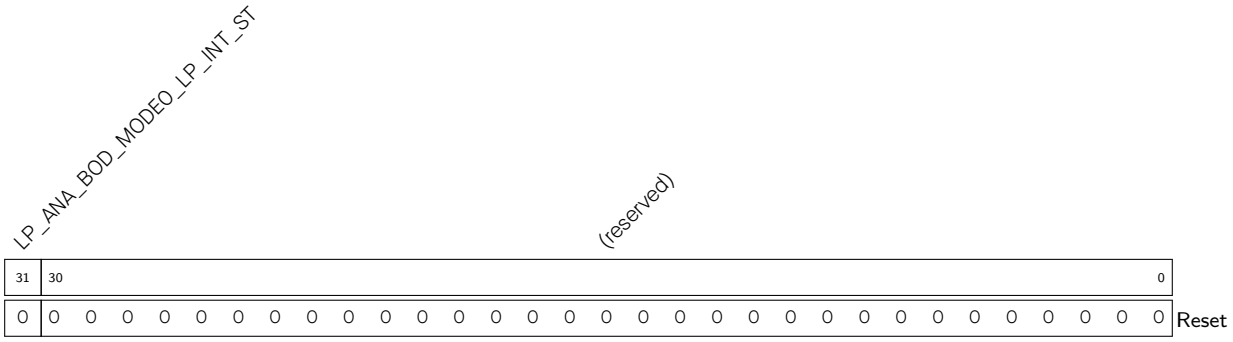
LP_ANA_BOD_MODE0_INT_CLR 写 1 清除 LP_ANA_BOD_MODE0_INT。(WT)

Register 9.9. LP_ANA_LP_INT_RAW_REG (0x0030)



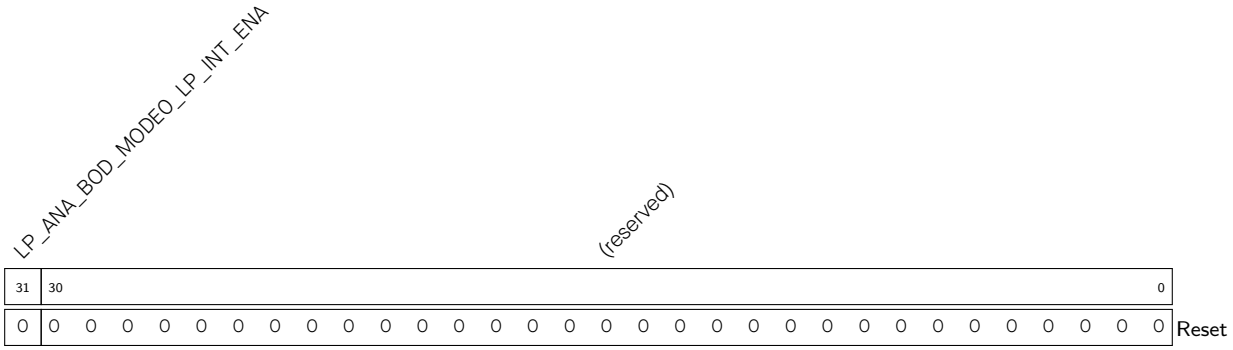
LP_ANA_BOD_MODE0_LP_INT_RAW LP_ANA_BOD_MODE0_LP_INT 的原始中断状态。(R/WTC/SS)

Register 9.10. LP_ANA_LP_INT_ST_REG (0x0034)



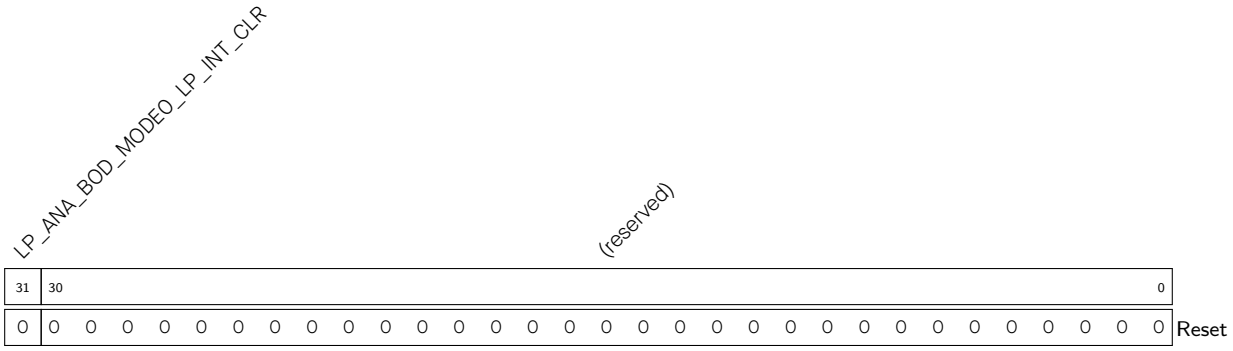
LP_ANA_BOD_MODE0_LP_INT_ST LP_ANA_BOD_MODE0_LP_INT 的屏蔽中断状态。(RO)

Register 9.11. LP_ANA_LP_INT_ENA_REG (0x0038)



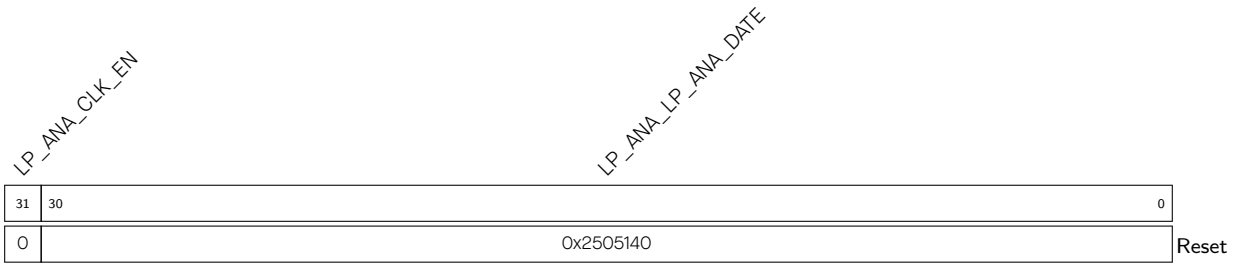
LP_ANA_BOD_MODE0_LP_INT_ENA 写 1 使能 LP_ANA_BOD_MODE0_LP_INT。(R/W)

Register 9.12. LP_ANA_LP_INT_CLR_REG (0x003C)



LP_ANA_BOD_MODE0_LP_INT_CLR 写 1 清除 LP_ANA_BOD_MODE0_LP_INT。(WT)

Register 9.13. LP_ANA_DATE_REG (0x03FC)



LP_ANA_LP_ANA_DATE 版本控制寄存器。(R/W)

LP_ANA_CLK_EN 使能强制打开配置寄存器时钟。

0：自动时钟门控

1：强制打开寄存器时钟

此域的配置不影响寄存器的读写。

(R/W)

第 III 卷

加密和安全组件

该部分专门介绍芯片的安全功能，包括 AES、ECC 等加密加速器，随机数生成、加密/解密算法等安全特性，展示了芯片在加密和安全数据处理方面的性能。

第 10 章

AES 加速器 (AES)

10.1 概述

ESP32-S31 内置 AES（高级加密标准）硬件加速器可使用 AES 算法，完成数据的加解密运算，具有 Typical AES 和 DMA-AES 两种工作模式。整体而言，相比基于纯软件的 AES 运算，AES 硬件加速器能够极大地提高运算速度。

10.2 主要特性

ESP32-S31 支持以下特性：

- Typical AES 工作模式
  - AES-128/AES-256 加解密运算
- DMA-AES 工作模式
  - AES-128/AES-256 加解密运算
  - 块（加密）模式
    - ECB (Electronic Codebook)
    - CBC (Cipher Block Chaining)
    - OFB (Output Feedback)
    - CTR (Counter)
    - CFB8 (8-bit Cipher Feedback)
    - CFB128 (128-bit Cipher Feedback)
  - 伽罗瓦/计数器模式 (Galois/Counter Mode, GCM)
  - 中断发生

10.3 时钟和复位

AES 加速器的激活需使能 HP_SYS_CLKRST_CRYPTO_CTRL0_REG 寄存器中的 HP_SYS_CLKRST_REG_CRYPTO_AES_CLK_EN 位，并同时清零 HP_SYS_CLKRST_REG_CRYPTO_AES_RST_EN 位。

10.4 工作模式简介

ESP32-S31 内置的 AES 加速器支持 Typical AES 和 DMA-AES 两种工作模式。

- Typical AES 工作模式：
  - 支持使用 128 位或 256 位密钥进行加密与解密运算，即 NIST FIPS 197 标准中的 AES-128 和 AES-256 加解密运算。

  这种情况下，明文和密文的读写操作统一通过 CPU 访问完成。
- DMA-AES 工作模式支持：
  - 使用 128 位或 256 位密钥进行加密与解密运算，即 NIST FIPS 197 标准中的 AES-128 和 AES-256 加解密运算；
  - NIST SP 800-38A 标准中的 ECB、CBC、OFB、CTR、CFB8、CFB128 块加密模式运算。
  - NIST SP 800-38D 标准中的 GCM。

  在这种情况下，明文和密文的传输通过硬件上的 DMA 完成，运算完成时会有中断发生。

用户可通过配置 AES_DMA_ENABLE_REG 选择 AES 加速器的工作模式，具体参考表 10.4-1。

表 10.4-1. 工作模式

AES_DMA_ENABLE_REG	工作模式
0	Typical AES
1	DMA-AES

用户可通过配置 AES_MODE_REG 寄存器选择密钥长度和加解密方向，具体可参考表 10.4-2。

表 10.4-2. 密钥长度和加解密方向

AES_MODE_REG[2:0]	密钥长度和加解密方向
0	AES-128 加密
1	保留
2	AES-256 加密
3	保留
4	AES-128 解密
5	保留
6	AES-256 解密
7	保留

有关 Typical AES 和 DMA-AES 两种工作模式的具体介绍，请见章节 10.5 和 10.6。

注意：

ESP32-S31 的 RSA 数字签名外设 (RSA_DS) 模块在工作时也会调用 AES 加速器。默认情况下，AES_FREE（值为 1）允许 RSA 数字签名外设 (RSA_DS) 模块立即重置当前正在进行的 AES 操作，并使用 AES 执行其自身操作。用户应将 AES_FREE 设为 0，以防止该情况发生。

10.5 Typical AES 工作模式

在 Typical AES 工作模式下，AES 加速器的状态值可查看寄存器 AES_STATE_REG，具体见表 10.5-1 所示：

表 10.5-1. 状态返回值

返回值	描述	状态说明
0	IDLE	加速器空闲或运算完成
1	WORK	加速器忙于运算

10.5.1 密钥、明文、密文

寄存器 AES_KEY_*n*_REG 用于存放密钥，由 8 个 32 位寄存器组成。

- 如果为 AES-128 加解密运算，则 128 位密钥在寄存器 AES_KEY_0_REG ~ AES_KEY_3_REG 中。
- 如果为 AES-256 加解密运算，则 256 位密钥在寄存器 AES_KEY_0_REG ~ AES_KEY_7_REG 中。
- 另外，也可以通过 密钥管理器 [to be added later] 设置 AES-128 或 AES-256 密钥。

寄存器 AES_TEXT_IN_*m*_REG 和 AES_TEXT_OUT_*m*_REG 用于存放明文和密文，各由 4 个 32 位寄存器组成。

- 如果为 AES-128 或 AES-256 加密运算，则运算开始之前用明文初始化寄存器 AES_TEXT_IN_*m*_REG。运算完成之后，AES 加速器将把密文更新入寄存器 AES_TEXT_OUT_*m*_REG。
- 如果为 AES-128 或 AES-256 解密运算，则运算开始之前用密文初始化寄存器 AES_TEXT_IN_*m*_REG。运算完成之后，AES 加速器将把明文更新入寄存器 AES_TEXT_OUT_*m*_REG。

10.5.2 字节序

文本字节序

在 Typical AES 工作模式下，AES 加速器可以使用密钥对 128 位的块进行加解密。在读写寄存器 AES_TEXT_IN_*m*_REG 和 AES_TEXT_OUT_*m*_REG 中的数据时，用户应遵循表 10.5-2 中定义的文本字节序。

表 10.5-2. Typical AES 文本字节序

明文/密文					
State[1]		c[2]			
		0	1	2	3
r	0	AES_TEXT_*x*_0_REG[7:0]	AES_TEXT_*x*_1_REG[7:0]	AES_TEXT_*x*_2_REG[7:0]	AES_TEXT_*x*_3_REG[7:0]
	1	AES_TEXT_*x*_0_REG[15:8]	AES_TEXT_*x*_1_REG[15:8]	AES_TEXT_*x*_2_REG[15:8]	AES_TEXT_*x*_3_REG[15:8]
	2	AES_TEXT_*x*_0_REG[23:16]	AES_TEXT_*x*_1_REG[23:16]	AES_TEXT_*x*_2_REG[23:16]	AES_TEXT_*x*_3_REG[23:16]
	3	AES_TEXT_*x*_0_REG[31:24]	AES_TEXT_*x*_1_REG[31:24]	AES_TEXT_*x*_2_REG[31:24]	AES_TEXT_*x*_3_REG[31:24]

[1] 有关 “State（以及 c 和 r）” 的详细定义，请参考 NIST FIPS 197 中 3.4 The State 章节。
[2] 其中，*x* = IN 或 OUT。

密钥字节序

在 Typical AES 工作模式下，在向寄存器 AES_KEY_*n*_REG 中填入数据时，用户应遵循表 10.5-3 和表 10.5-4 中定义的文本字节序。

表 10.5-3. AES-128 密钥字节序

Bit[1]	w[0]	w[1]	w[2]	w[3][2]
[31:24]	AES_KEY_0_REG[7:0]	AES_KEY_1_REG[7:0]	AES_KEY_2_REG[7:0]	AES_KEY_3_REG[7:0]
[23:16]	AES_KEY_0_REG[15:8]	AES_KEY_1_REG[15:8]	AES_KEY_2_REG[15:8]	AES_KEY_3_REG[15:8]
[15:8]	AES_KEY_0_REG[23:16]	AES_KEY_1_REG[23:16]	AES_KEY_2_REG[23:16]	AES_KEY_3_REG[23:16]
[7:0]	AES_KEY_0_REG[31:24]	AES_KEY_1_REG[31:24]	AES_KEY_2_REG[31:24]	AES_KEY_3_REG[31:24]

[1] Bit 列代表 w[0] ~ w[3] 每个 word 中的各个字节。

[2] w[0] ~ w[3] 符合标准 NIST FIPS 197 中 5.2 Key Expansion 章节中对 “the first Nk words of the expanded key” 的描述。

表 10.5-4. AES-256 密钥字节序

Bit[1]	w[0]	w[1]	w[2]	w[3]	w[4]	w[5]	w[6]	w[7][2]
[31:24]	AES_KEY_0_REG[7:0]	AES_KEY_1_REG[7:0]	AES_KEY_2_REG[7:0]	AES_KEY_3_REG[7:0]	AES_KEY_4_REG[7:0]	AES_KEY_5_REG[7:0]	AES_KEY_6_REG[7:0]	AES_KEY_7_REG[7:0]
[23:16]	AES_KEY_0_REG[15:8]	AES_KEY_1_REG[15:8]	AES_KEY_2_REG[15:8]	AES_KEY_3_REG[15:8]	AES_KEY_4_REG[15:8]	AES_KEY_5_REG[15:8]	AES_KEY_6_REG[15:8]	AES_KEY_7_REG[15:8]
[15:8]	AES_KEY_0_REG[23:16]	AES_KEY_1_REG[23:16]	AES_KEY_2_REG[23:16]	AES_KEY_3_REG[23:16]	AES_KEY_4_REG[23:16]	AES_KEY_5_REG[23:16]	AES_KEY_6_REG[23:16]	AES_KEY_7_REG[23:16]
[7:0]	AES_KEY_0_REG[31:24]	AES_KEY_1_REG[31:24]	AES_KEY_2_REG[31:24]	AES_KEY_3_REG[31:24]	AES_KEY_4_REG[31:24]	AES_KEY_5_REG[31:24]	AES_KEY_6_REG[31:24]	AES_KEY_7_REG[31:24]

[1] Bit 列代表 w[0] ~ w[7] 每个 word 中的各个字节。

[2] w[0] ~ w[7] 符合标准 NIST FIPS 197 中 5.2 Key Expansion 章节中对 “the first Nk words of the expanded key” 的描述。

GoBack

10.5.3 Typical AES 工作流程

单次运算

1. 对寄存器 AES_DMA_ENABLE_REG 写入 0。
2. 初始化寄存器 AES_MODE_REG、AES_KEY_*n*_REG、AES_TEXT_IN_*m*_REG。
3. 如果使用来自 密钥管理器 [to be added later] 的密钥，将 AES_USE_HARDWARE_KEY 寄存器设置为 1。
4. 启动运算。对寄存器 AES_TRIGGER_REG 写入 1。
5. 等待运算完成。轮询寄存器 AES_STATE_REG，直到读到 0。
6. 从寄存器 AES_TEXT_OUT_*m*_REG 读取结果。

连续运算

在连续运算过程中，每次运算完成之后，只有寄存器 AES_TEXT_IN_*m*_REG 和 AES_TEXT_OUT_*m*_REG (*m*: 0-3) 会被 AES 加速器更新，而 AES_DMA_ENABLE_REG、AES_MODE_REG、AES_KEY_*n*_REG 等寄存器中的内容不会变化。所以进行连续运算时可以简化初始化操作。

1. 第一次运算之前对寄存器 AES_DMA_ENABLE_REG 写入 0。
2. 第一次运算之前初始化寄存器 AES_MODE_REG 和 AES_KEY_*n*_REG。
3. 更新寄存器 AES_TEXT_IN_*m*_REG。
4. 启动运算。对寄存器 AES_TRIGGER_REG 写入 1。
5. 等待运算完成。轮询寄存器 AES_STATE_REG，直到读到 0。
6. 从寄存器 AES_TEXT_OUT_*m*_REG 读取结果 。返回步骤 3，进行下一轮运算。

10.6 DMA-AES 工作模式

在 DMA-AES 工作模式下，AES 加速器可支持 ECB、CBC、OFB、CTR、CFB8、CFB128 六种块模式运算。用户可通过配置 AES_BLOCK_MODE_REG 寄存器选择具体运算类型，具体可参考表 10.6-1。

表 10.6-1. 块模式选择

AES_BLOCK_MODE_REG[2:0]	块模式
0	ECB (Electronic Code Book)
1	CBC (Cipher Block Chaining)
2	OFB (Output FeedBack)
3	CTR (Counter)
4	CFB8 (8-bit Cipher FeedBack)
5	CFB128 (128-bit Cipher FeedBack)
6	GCM (Galois/Counter Mode)
7	保留

AES 加速器的状态值可查看寄存器 AES_STATE_REG，具体见表 10.6-2 所示：

表 10.6-2. 状态返回值

返回值	描述	状态说明
0	IDLE	加速器空闲
1	WORK	加速器忙于运算
2	DONE	加速器运算完成

AES 加速器在 DMA-AES 工作模式下允许中断发生，软件清零。中断功能默认关闭，用户可通过将 AES_INT_ENA_REG 寄存器配置为 1 开启中断。如开启中断功能，AES 加速器在完成运算时，中断发生。

10.6.1 密钥、明文、密文

块运算模式

在块运算模式下，AES 加速器的源数据来自 DMA，结果数据也将被写入 DMA。

- 如果为加密运算，则 DMA 从存储器中读取明文数据流并将其传给 AES。AES 运算出密文后将密文写入 DMA。DMA 再将密文写入存储器。
- 如果为解密运算，则 DMA 从存储器中读取密文数据流并将其传给 AES。AES 运算出明文后将明文写入 DMA。DMA 再将明文写入存储器。

AES 加速器在进行块运算时，结果数据与源数据的大小保持一致。此时，DMA 的数据搬运过程和 AES 的运算过程有所交叠，因此总工作时间有所减少。

值得注意的是，AES 加速器在 DMA-AES 工作模式下要求源数据的大小必须是 128 位的整数倍，否则需要将原始明文封装为 128 位的整数倍，即在源数据末尾应尽可能少补 0，具体过程见表 10.6-3 所示。

表 10.6-3. TEXT-PADDING

函数： TEXT-PADDING( )
输入： X，比特串。 **输出：** Y = TEXT-PADDING(X)，长度为最接近 128 比特整数倍的值。
步骤： 假设 X 是一个数据流，可以按如下方式分成 n 个部分： $X = X_1 \| \| X_2 \| \| \cdots \| \| X_{n-1} \| \| X_n$ 此处，X_1、X_2、$\cdots$、X_{n-1} 的长度均为 128 比特，而 X_n 的长度为 t (0<=t<=127)。 如果 $t = 0$，则 TEXT-PADDING(X) = X; 如果 $0 < t <= 127$，定义 X_n^* 块的长度为 128 比特，且 $X_n^* = X_n \| \| 0^{128-t}$，则 TEXT-PADDING($X$) = $X_1 \| \| X_2 \| \| \cdots \| \| X_{n-1} \| \| X_n^*$ = $X \| \| 0^{128-t}$

10.6.2 字节序

在 DMA-AES 工作模式下，源数据和结果数据的传输完全由 DMA 完成，因此不支持字节序的控制调节，但要求它们在存储器中以一定的方式来存放，且要求数据量必须是块的整数倍。

举例说明，假设 DMA 需要搬运源数据：

- 十六进制：0102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F20
- 数据大小：相当于 2 个块

假设起始地址为 0x0280，则源数据在存储器中的存放位置如表 10.6-4 所示。结果数据也遵从相同的存放规则，在此不多做介绍。

表 10.6-4. DMA-AES 存储字节序

地址	字节	地址	字节	地址	字节	地址	字节
0x0280	0x01	0x0281	0x02	0x0282	0x03	0x0283	0x04
0x0284	0x05	0x0285	0x06	0x0286	0x07	0x0287	0x08
0x0288	0x09	0x0289	0x0A	0x028A	0x0B	0x028B	0x0C
0x028C	0x0D	0x028D	0x0E	0x028E	0x0F	0x028F	0x10
0x0290	0x11	0x0291	0x12	0x0292	0x13	0x0293	0x14
0x0294	0x15	0x0295	0x16	0x0296	0x17	0x0297	0x18
0x0298	0x19	0x0299	0x1A	0x029A	0x1B	0x029B	0x1C
0x029C	0x1D	0x029D	0x1E	0x029E	0x1F	0x029F	0x20

10.6.3 标准增量函数

AES 加速器在进行 CTR 块运算时，还可提供两种标准增量函数供用户选择：INC_{32} 和 INC_{128}。用户可通过将寄存器 AES_INC_SEL_REG 置为 0 或 1 选择 INC_{32} 或 INC_{128} 标准增量函数。更多有关标准增量函数的内容，请见 NIST SP 800-38A 标准中的 B.1 The Standard Incrementing Function 章节。

10.6.4 块个数

寄存器 AES_BLOCK_NUM_REG 存放明文或密文的块个数 (Block Number)，其值等于 length(**TEXT-PADDING**(P))/128，也等于 length(**TEXT-PADDING**(C))/128。这里的 P 指明文 (plaintext)，C 指密文 (ciphertext)。该寄存器仅在 DMA-AES 工作模式下有意义。

10.6.5 初始向量

存储器 AES_IV_MEM 的空间大小为 16 字节，仅在块运算模式下有效。对于 CBC/OFB/CFB8/CFB128 操作，AES_IV_MEM 用于存放初始向量 (Initialization Vector, IV) 的值。对于 CTR 操作，AES_IV_MEM 存放初始计数器 (Initial Counter Block, ICB) 的值。

IV 和 ICB 都是 128 位的比特串，从左向右被分割成 16 个字节 (Byte0, Byte1, Byte2, ⋯, Byte15)，构成一个字节序列，在 AES_IV_MEM 中存放时需要遵循表 10.6-4 中的字节序规则，即 Byte0 存放在 AES_IV_MEM 中的最低地址中，Byte15 存放在 AES_IV_MEM 中的最高地址中。

更多有关 IV 和 ICB 的信息，请参考 NIST SP 800-38A 标准。

10.6.6 块工作流程

1. 选择一条 DMA 通道与 AES 加速器连接，配置 DMA 链表，而后启动 DMA。详情请见章节 *3 通用 DMA 控制器 (GDMA-AHB, GDMA-AXI)*。
2. 初始化与 AES 加速器相关的寄存器：
    - 对寄存器 AES_DMA_ENABLE_REG 写入 1。

- 配置寄存器 AES_INT_ENA_REG 以开启或禁用中断。
- 初始化寄存器 AES_MODE_REG 和 AES_KEY_*n*_REG，或者将 AES_USE_HARDWARE_KEY 配置为允许使用 密钥管理器 [to be added later] 中部署的 AES 密钥。
- 配置寄存器 AES_BLOCK_MODE_REG，选择具体块加密模式。详见表 10.6-1。
- 初始化寄存器 AES_BLOCK_NUM_REG，请参照章节 10.6.4。
- 初始化寄存器 AES_INC_SEL_REG（仅在 CTR 块模式下使用）。
- 初始化存储器 AES_IV_MEM（在 ECB 块模式下不使用）。

3. 对寄存器 AES_TRIGGER_REG 写入 1，启动运算。
4. 等待运算完成。轮询寄存器 AES_STATE_REG，直到读到 2。如果开启了中断功能，也可以等待 AES_INT 中断产生。
5. 确认 DMA 完成从 AES 到内存的数据传输。此时，结果数据已经被 DMA 写入内存，可以直接读取。详情请参考章节 3 通用 *DMA 控制器 (GDMA-AHB, GDMA-AXI)*。
6. 如果开启了中断，当处理中断程序完成后，请及时对寄存器 AES_INT_CLEAR_REG 写 1 以清除中断。
7. 对寄存器 AES_DMA_EXIT_REG 写入 1 释放 AES 加速器，再读取寄存器 AES_STATE_REG 将读到 0。该步操作可以提前完成，但必须在步骤 4 之后完成。

10.6.7 GCM 工作流程

1. 配置 DMA 链并启动 DMA。有关 DMA 的详细信息，请见章节 3 通用 *DMA 控制器 (GDMA-AHB, GDMA-AXI)*。
2. 初始化与 AES 加速器相关的寄存器：
   - 对寄存器 AES_DMA_ENABLE_REG 写入 1。
   - 配置寄存器 AES_INT_ENA_REG 以开启或禁用中断。
   - 初始化寄存器 AES_MODE_REG 和 AES_KEY_*n*_REG，或者将 AES_USE_HARDWARE_KEY 配置为允许使用 密钥管理器 [to be added later] 中部署的 AES 密钥。
   - 对寄存器 AES_BLOCK_MODE_REG 写入 6。
   - 初始化寄存器 AES_BLOCK_NUM_REG，详情请见章节 10.6.4 块个数。
   - 初始化寄存器 AES_AAD_BLOCK_NUM_REG，详情请见章节 10.7.4 *AAD* 块个数。
   - 初始化寄存器 AES_REMAINDER_BIT_NUM_REG ，详情请见章节 10.7.5 不完整块的有效位数。
3. 对寄存器 AES_TRIGGER_REG 写入 1，启动运算。
4. 等待运算完成。轮询寄存器 AES_STATE_REG，直到读到 2。有关 AES 加速器工作状态的详细信息，请参考表 10.6-2 状态返回值。此步骤中不会产生中断。
5. 从存储器 AES_H_MEM 中读取 H。
6. 生成 J_0 并将其写入存储器 AES_J0_MEM。
7. 对寄存器 AES_CONTINUE_REG 写入 1 以继续操作。
8. 等待运算完成。轮询寄存器 AES_STATE_REG，直到读到 2。如果开启了中断功能，也可以等待 AES_INT 中断产生。有关 AES 加速器工作状态的详细信息，请参考表 10.6-2 状态返回值。

9. 从存储器 AES_TO_MEM 中读取 T_0。
10. 确认 DMA 完成从 AES 到内存的数据传输。此时，结果数据已经被 DMA 写入内存，可以直接读取。详情请参考章节 3 通用 DMA 控制器 (GDMA-AHB, GDMA-AXI)。
11. 如果开启了中断，当处理中断程序完成后，请及时对寄存器 AES_INT_CLEAR_REG 写 1 以清除中断。
12. 对寄存器 AES_DMA_EXIT_REG 写入 1 退出 DMA，再读取寄存器 AES_STATE_REG 将读到 0。该步操作可以提前完成，但必须在步骤 8 之后完成。

10.7 GCM 算法

ESP32-S31 的 AES 加速器支持 GCM 算法。通常情况下，很少会使用超过 $2^{32}-1$ 位长度的 AAD、C 和 P。因此，本章节规定 AAD、C 和 P 的长度不应超过 $2^{32}-1$。下图 10.7-1 为 ESP32-S31 AES 加速器中的 GCM 加密过程。

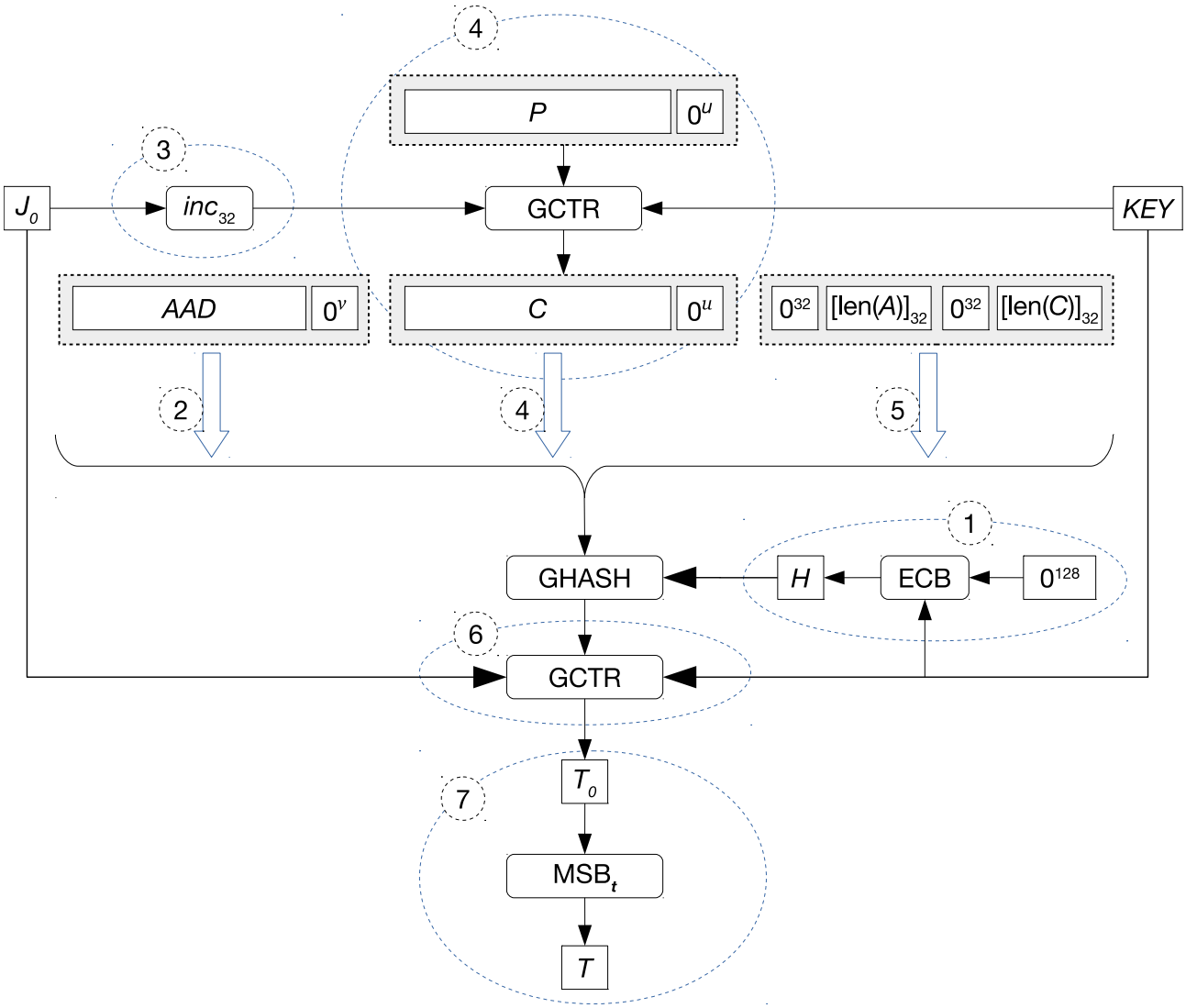


图 10.7-1. GCM 加密过程

GCM 加密过程如下：

1. 硬件执行 ECB 算法以获取哈希子密钥 H，该子密钥将用于哈希运算。

2. 硬件执行 GHASH 算法，使用 已填充的 AAD 进行哈希运算。
3. 硬件以 J_0 作为输入执行标准递增函数 INC_{32}，该结果将作为 CTR 加密的初始计数器值。
4. 硬件执行 GCTR 算法加密已填充的明文 P，然后执行 GHASH 算法使用已填充的密文 C 进行哈希运算。
5. 硬件执行 GHASH 算法对 AAD 块进行哈希运算，得到 128 位的哈希结果。
6. 硬件执行 GCTR 算法加密 J_0，得到 T_0。
7. 软件从硬件获取结果 T_0，执行 MSB_t 算法，得到最终的认证标签 T。

GCM 解密与 GCM 加密唯一的区别在于图 10.7-1 中的步骤 4。具体而言，在 GCM 解密中，AES 加速器不是执行 GCTR 算法加密已填充的明文，而是执行这一算法解密已填充的密文，详见 NIST SP 800-38D。

10.7.1 哈希子密钥

在 GCM 操作中，哈希子密钥 H 是硬件运算出的 128 位值，详见图 10.7-1 中的步骤 1。更多信息请参考 NIST SP 800-38D 的 7 GCM Specification 章节中关于 "Step 1. Let H = $CIPH_K(0^{128})$" 的描述。

哈希子密钥 H 存放在存储器 AES_H_MEM 中。与其他字节序相同，其最高有效字节（即最左侧字节）Byte0 存放在存储器的最低地址中，而最低有效字节（即最右侧字节）Byte15 存放在最高地址中，详见表 10.5-2。

10.7.2 J_0

J_0 是硬件运算出的 128 位值，用于图 10.7-1 的步骤 3 和步骤 6。有关生成 J_0 的详细信息，请参考 NIST SP 800-38D 的 7 GCM Specification 章节。

J_0 存放在存储器 AES_J0_MEM 中。与其他字节序相同，其最高有效字节（即最左侧字节）Byte0 存放在存储器的最低地址中，而最低有效字节（即最右侧字节）Byte15 存放在最高地址中，详见表 10.5-2。

10.7.3 认证标签

认证标签（下文简称标签）是 GCM 的关键输出之一，详见图 10.7-1 的步骤 7。标签值由其长度 t $(1 <= t <= 128)$ 决定：

- 当 $t = 128$ 时，标签值等于 128 位的字符串 T_0，存放在 AES_T0_MEM 中。与其他字节序相同，其最高有效字节（即最左侧字节）Byte0 存放在存储器的最低地址中，而最低有效字节（即最右侧字节）Byte15 存放在最高地址中，详见表 10.5-2。
- 当 $1 <= t < 128$ 时，标签值等于 T_0 的 t 位最高有效位（即最左侧位）。在这种情况下的标签为 $MSB_t(T_0)$，代表 T_0 的 t 位最高有效位。例如，$MSB_4(11011010)$ = 1110，$MSB_5(11010011010)$ = 11010。有关 $MSB_t()$ 函数的详细信息，请参考 NIST SP 800-38D 中的 6 Mathematical Components of GCM 章节。

10.7.4 AAD 块个数

寄存器 AES_AAD_BLOCK_NUM_REG 存储附加认证数据 (AAD) 的块个数。该寄存器的长度等于 **TEXT-PADDING**(AAD) 长度)/128。AES 加速器在 DMA-AES 模式下工作时才会使用此寄存器。

10.7.5 不完整块的有效位数

寄存器 AES_REMAINDER_BIT_NUM_REG 表示明文或密文中不完整块的有效位数。该寄存器中存储的值等于 P 长度/128 或 C 长度/128。AES 加速器在 DMA-AES 模式下工作时才会使用此寄存器。

AES_REMAINDER_BIT_NUM_REG 不影响明文或密文的相关运算结果，但会影响 T_0 的值，因此也会影响认证标签的值。

GCM 算法可以看作是 GCTR 操作和 GHASH 操作的结合，其中 GCTR 执行加密和解密，而 GHASH 运算认证标签的值。

请注意，AES_REMAINDER_BIT_NUM_REG 寄存器仅对 GCM 加密有效。具体而言：

- 对于 GCM 加密来说，硬件首先运算 C，然后将其以 **TEXT-PADDING**(C) 的形式传递给 GHASH 操作。此时，硬件根据 AES_REMAINDER_BIT_NUM_REG 的值确定在数据末尾应添加多少 0。
- GCM 解密使用 **TEXT-PADDING**(C) 函数完成填充。此时，AES_REMAINDER_BIT_NUM_REG 寄存器无效。

10.8 伪轮次抗 DPA 功能

ESP32-S31 AES 模块的伪轮次抗 DPA 功能可以增加芯片的抵御攻击的性能。当开启伪轮次功能时，AES 模块将在原始操作轮次之前和之后随机添加伪轮次。在一个伪轮次中，AES 将使用由硬件自动生成的伪密钥执行伪计算。伪轮次中的计算不会影响原始 AES 操作的结果。

伪轮次抗 DPA 功能的主要配置参数包括：

伪轮次功能的开关

用户可通过配置 AES_PSEUDO_EN 寄存器，选择是否开启伪轮次功能。

- 0：计算过程中不增加伪轮次；
- 1：计算过程中增加伪轮次。

伪轮次的总数

在每次 AES 计算中随机插入伪轮次的具体数量可通过以下寄存器配置：

- pseudo_base：等于 AES_PSEUDO_BASE 中配置的值，该值配置伪轮次功能的基本轮数。
- pseudo_inc：等于 AES_PSEUDO_INC 中配置的值，该值配置伪轮次功能的随机递增轮数。

伪轮次的总数将在 $[pseudo_base, pseudo_base + (2^{pseudo_inc} - 1)]$ 范围内随机生成。

随机数更新频率

用户可通过配置 AES_PSEUDO_RNG_CNT 的值来确定伪轮次功能中随机密钥的更新频率。配置的值越大，代表更新频率越高。通常建议将此值配置为最大值 7。

10.9 中断

ESP32-S31 的 AES 可以生成以下中断信号并发送给 中断矩阵。

- AES_INTR

以上中断信号由 AES 的内部中断源生成。表 10.9-1 列出了 AES 的内部中断源、中断触发条件、以及生成的中断信号。

表 10.9-1. AES 的内部中断源

内部中断源	触发条件	中断信号

AES_DMA_DONE_INT	AES DMA 运算完成	AES_INTR

说明：
中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

10.10 存储器列表

本小节的所有地址均为相对于 AES 加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 系统和存储器 *[to be added later]* 中的表 3。

请查看章节 寄存器的访问类型，了解“访问”列缩写的含义。

名称	描述	大小	起始地址	结束地址	访问
AES_IV_MEM	存储器 IV	16 字节	0x0050	0x005F	R/W
AES_H_MEM	存储器 H	16 字节	0x0060	0x006F	RO
AES_J0_MEM	存储器 J0	16 字节	0x0070	0x007F	R/W
AES_T0_MEM	存储器 T0	16 字节	0x0080	0x008F	RO

10.11 寄存器列表

本小节的所有地址均为相对于 AES 加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 系统和存储器 *[to be added later]* 中的表 3。

请查看章节 寄存器的访问类型，了解“访问”列缩写的含义。

名称	描述	地址	访问
密钥寄存器			
AES_KEY_0_REG	AES 密钥资料寄存器 0	0x0000	R/W
AES_KEY_1_REG	AES 密钥资料寄存器 1	0x0004	R/W
AES_KEY_2_REG	AES 密钥资料寄存器 2	0x0008	R/W
AES_KEY_3_REG	AES 密钥资料寄存器 3	0x000C	R/W
AES_KEY_4_REG	AES 密钥资料寄存器 4	0x0010	R/W
AES_KEY_5_REG	AES 密钥资料寄存器 5	0x0014	R/W
AES_KEY_6_REG	AES 密钥资料寄存器 6	0x0018	R/W
AES_KEY_7_REG	AES 密钥资料寄存器 7	0x001C	R/W
TEXT_IN 寄存器			
AES_TEXT_IN_0_REG	源数据资料寄存器 0	0x0020	R/W
AES_TEXT_IN_1_REG	源数据资料寄存器 1	0x0024	R/W
AES_TEXT_IN_2_REG	源数据资料寄存器 2	0x0028	R/W
AES_TEXT_IN_3_REG	源数据资料寄存器 3	0x002C	R/W
TEXT_OUT 寄存器			
AES_TEXT_OUT_0_REG	结果数据资料寄存器 0	0x0030	RO
AES_TEXT_OUT_1_REG	结果数据资料寄存器 1	0x0034	RO

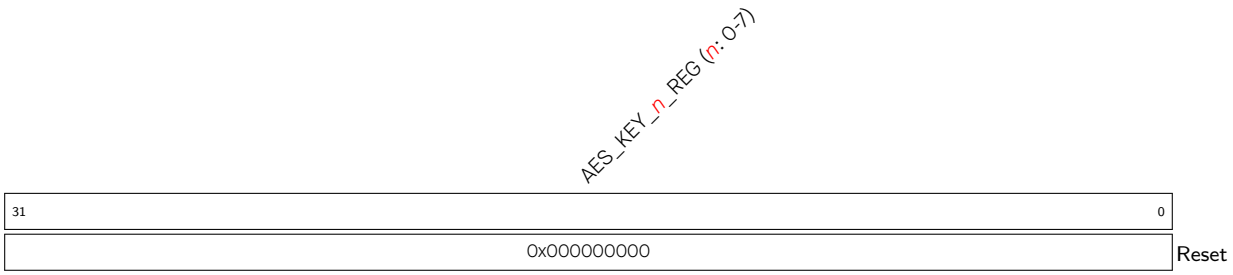
AES_TEXT_OUT_2_REG	结果数据资料寄存器 2	0x0038	RO
AES_TEXT_OUT_3_REG	结果数据资料寄存器 3	0x003C	RO
控制或配置寄存器			
AES_MODE_REG	配置密钥长度和加解密方向	0x0040	R/W
AES_TRIGGER_REG	开始运算	0x0048	WT
AES_DMA_ENABLE_REG	配置 AES 加速器工作模式	0x0090	R/W
AES_BLOCK_MODE_REG	配置 DMA-AES 下的块运算模式	0x0094	R/W
AES_BLOCK_NUM_REG	配置块数量	0x0098	R/W
AES_INC_SEL_REG	配置标准增量函数	0x009C	R/W
AES_AAD_BLOCK_NUM_REG	配置 ADD 块数量	0x00A0	R/W
AES_REMAINDER_BIT_NUM_REG	不完整块的有效比特位	0x00A4	R/W
AES_CONTINUE_REG	继续运算	0x00A8	WT
AES_DMA_EXIT_REG	退出运算	0x00B8	WT
AES_RX_RESET_REG	重置 RX FIFO	0x00C0	WT
AES_TX_RESET_REG	重置 TX FIFO	0x00C4	WT
AES_PSEUDO_REG	配置伪轮次抗攻击功能	0x00D0	R/W
状态寄存器			
AES_STATE_REG	运算状态	0x004C	RO
AES_FREE_REG	释放状态	0x00D4	R/W
中断寄存器			
AES_INT_CLEAR_REG	DMA-AES 中断清除	0x00AC	WT
AES_INT_ENA_REG	DMA-AES 中断使能	0x00B0	R/W
版本控制寄存器			
AES_DATE_REG	版本控制寄存器	0x00B4	R/W

10.12 寄存器

本小节的所有地址均为相对于 AES 加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

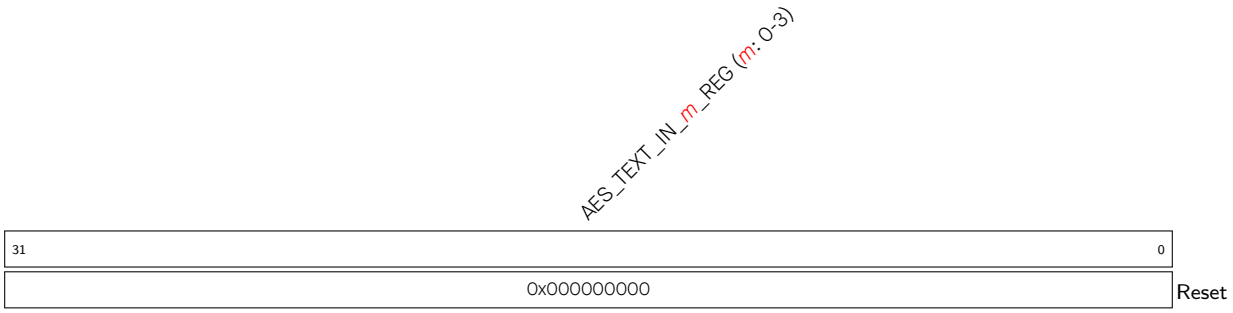
关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 10.1. AES_KEY_*n*_REG (*n*: 0-7) (0x0000+4n*)**



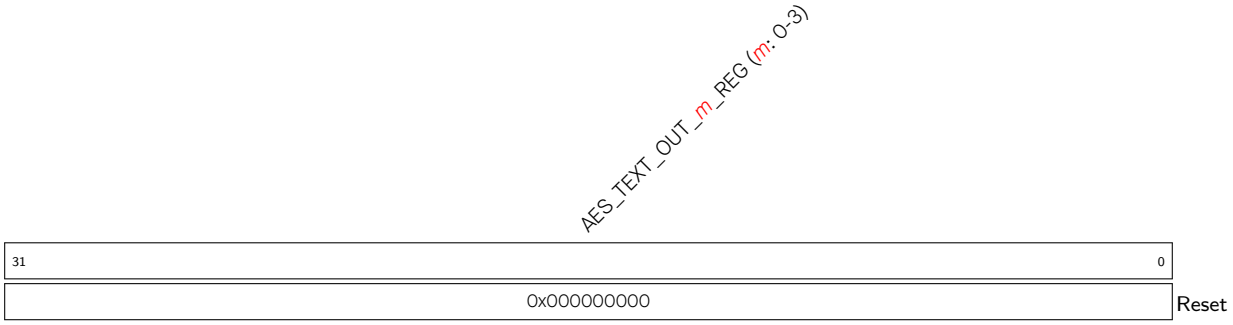
AES_KEY_*n*_REG (*n*: 0-7) 表示 AES 密钥资料。(R/W)

Register 10.2. AES_TEXT_IN_*m*_REG (*m*: 0-3) (0x0020+4m*)**



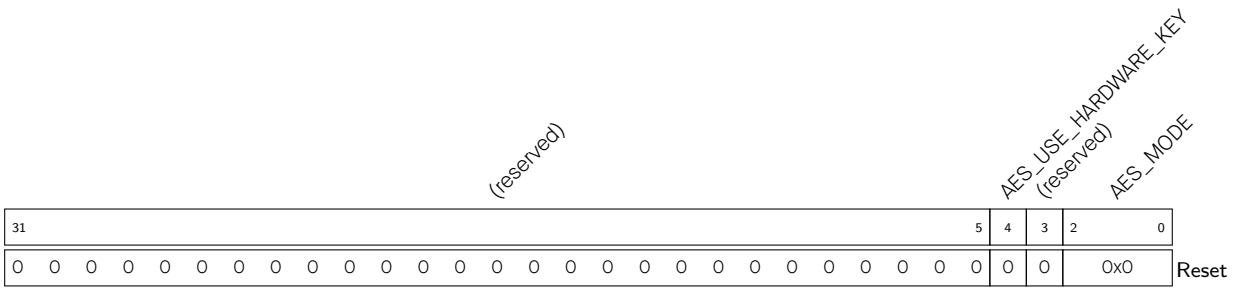
AES_TEXT_IN_*m*_REG (*m*: 0-3) 表示 Typical AES 工作模式下的输入文本。(R/W)

Register 10.3. AES_TEXT_OUT_*m*_REG (*m*: 0-3) (0x0030+4m*)**



AES_TEXT_OUT_*m*_REG (*m*: 0-3) 表示 Typical AES 工作模式下的输出文本。(RO)

Register 10.4. AES_MODE_REG (0x0040)



AES_MODE 配置 AES 加速器的密钥长度和加解密方向。

0：AES-128 加密

1：保留

2：AES-256 加密

3：保留

4：AES-128 解密

5：保留

6：AES-256 解密

7：保留

(R/W)

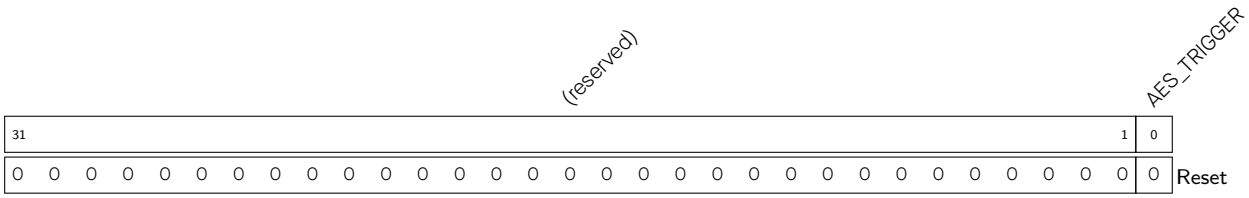
AES_USE_HARDWARE_KEY 配置 AES 加速器使用软件密钥或硬件密钥。

0：软件密钥

1：硬件密钥

(R/W)

Register 10.5. AES_TRIGGER_REG (0x0048)



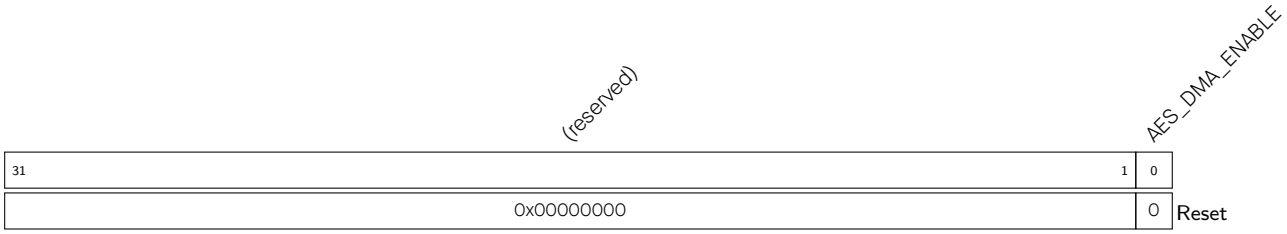
AES_TRIGGER 配置是否启动 AES 运算。

0：无效果

1：启动

(WT)

Register 10.6. AES_DMA_ENABLE_REG (0x0090)



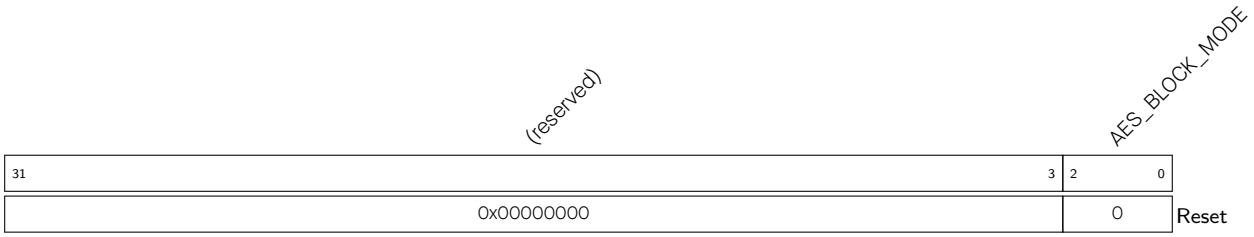
AES_DMA_ENABLE 配置 AES 加速器的工作模式。

0：Typical AES

1：DMA-AES

(R/W)

Register 10.7. AES_BLOCK_MODE_REG (0x0094)



AES_BLOCK_MODE 配置 AES 加速器在 DMA-AES 工作模式下的块模式。

0：ECB (Electronic Code Block)

1：CBC (Cipher Block Chaining)

2：OFB (Output FeedBack)

3：CTR (Counter)

4：CFB8 (8-bit Cipher FeedBack)

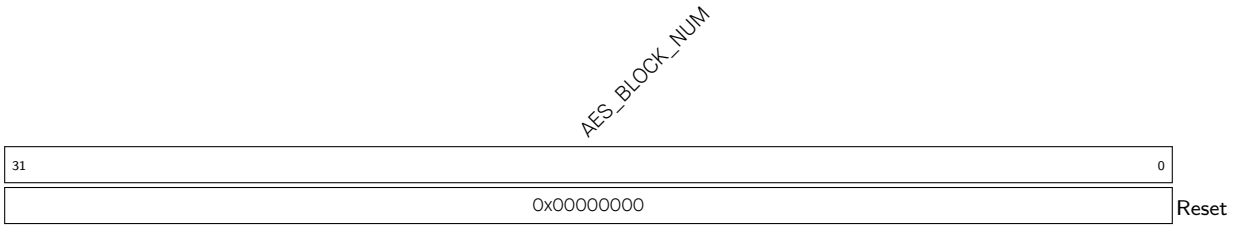
5：CFB128 (128-bit Cipher FeedBack)

6：GCM (Galois/Counter Mode)

7：保留

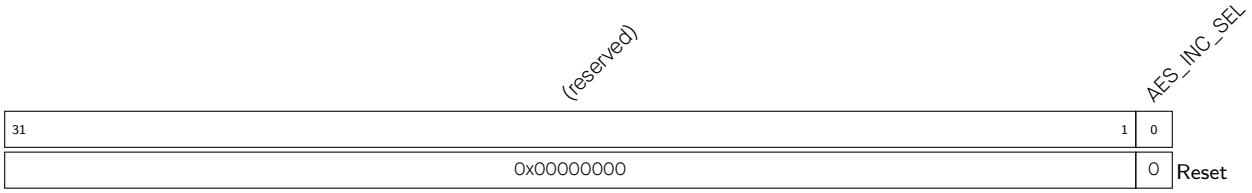
(R/W)

Register 10.8. AES_BLOCK_NUM_REG (0x0098)



AES_BLOCK_NUM 表示 DMA-AES 工作模式下待加解密的文本块数，详见章节 10.6.4。(R/W)

Register 10.9. AES_INC_SEL_REG (0x009C)



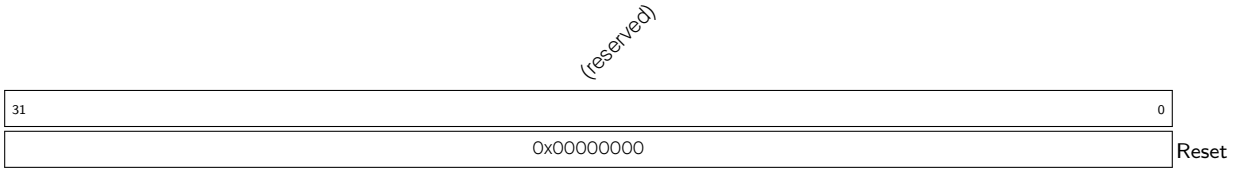
AES_INC_SEL 配置 CTR 块模式使用的标准增量函数。

0：INC_{32} 标准增量函数

1：INC_{128} 标准增量函数

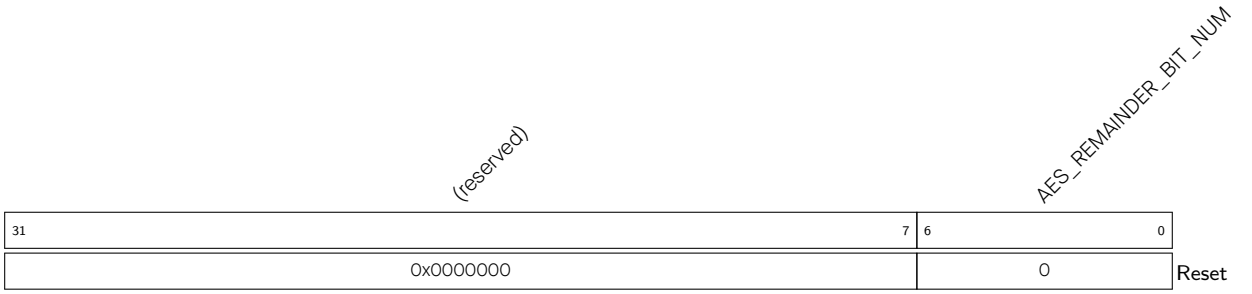
(R/W)

Register 10.10. AES_AAD_BLOCK_NUM_REG (0x00A0)



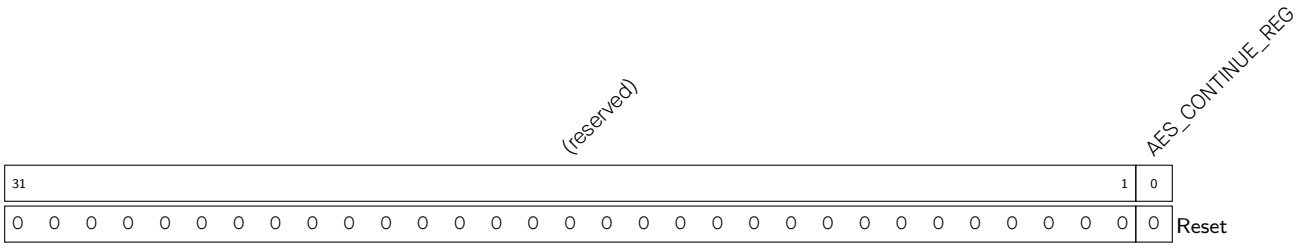
AES_AAD_BLOCK_NUM 表示 GCM 运算附加认证数据 (AAD) 的块个数，详见章节 10.7.4。(R/W)

Register 10.11. AES_REMAINDER_BIT_NUM_REG (0x00A4)



AES_REMAINDER_BIT_NUM 表示 GCM 运算不完整块的有效位数，详见章节 10.7.5。(R/W)

Register 10.12. AES_CONTINUE_REG (0x00A8)



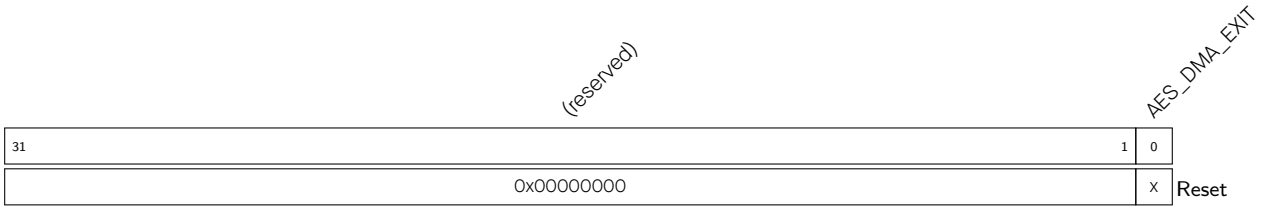
AES_CONTINUE_REG 配置是否继续 AES 运算。

0：无效果

1：继续

(WO)

Register 10.13. AES_DMA_EXIT_REG (0x00B8)



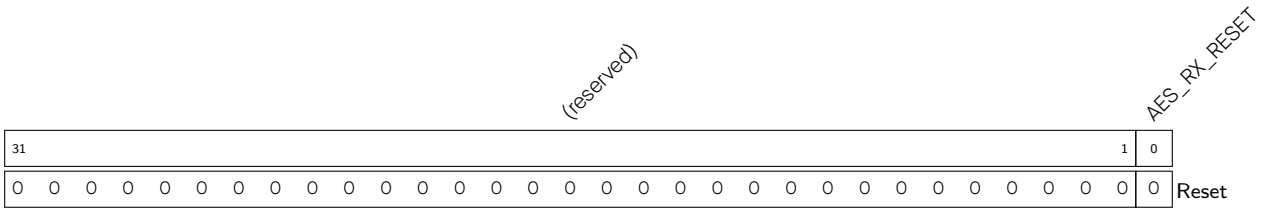
AES_DMA_EXIT 配置是否退出 AES 运算。

0：无效果

1：退出

仅在 DMA-AES 模式下有效。(WO)

Register 10.14. AES_RX_RESET_REG (0x00C0)



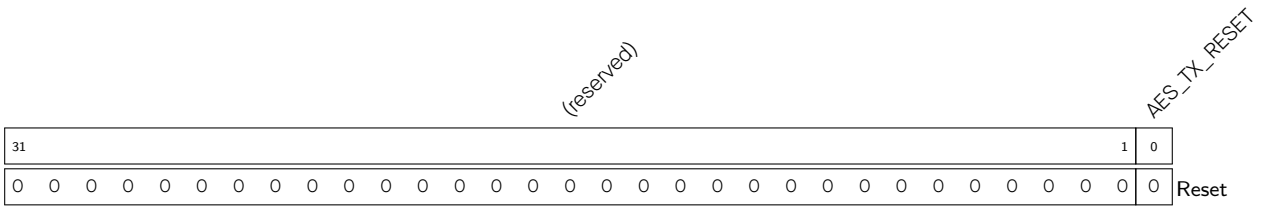
AES_RX_RESET 配置在 DMA-AES 工作模式下是否复位 RX_FIFO。

0：无效过

1：复位

(WT)

Register 10.15. AES_TX_RESET_REG (0x00C4)



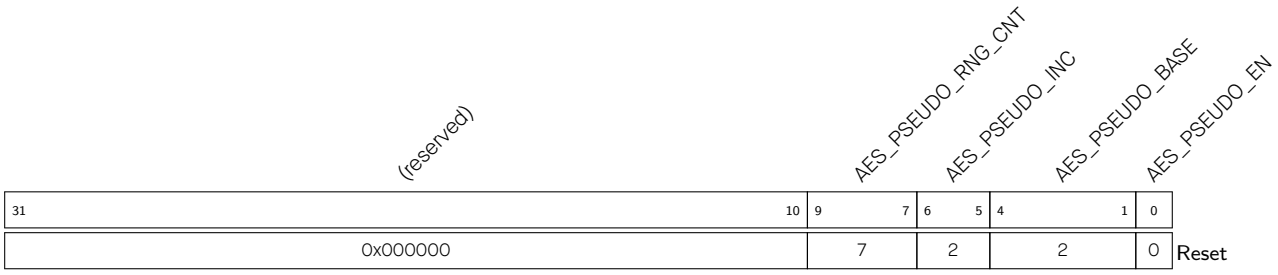
AES_TX_RESET 配置在 DMA-AES 工作模式下是否复位 TX_FIFO。

0：无效过

1：复位

(WT)

Register 10.16. AES_PSEUDO_REG (0x00D0)



AES_PSEUDO_EN 配置是否启用伪轮函数。

0：不启用

1：启用

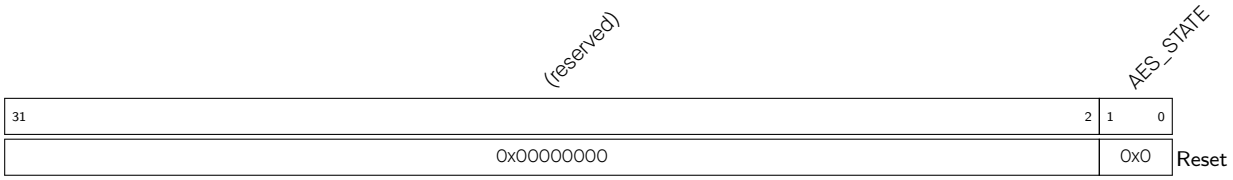
(R/W)

AES_PSEUDO_BASE 配置基本轮数。(R/W)

AES_PSEUDO_INC 配置递增轮数。(R/W)

AES_PSEUDO_RNG_CNT 配置伪密钥的更新频率。(R/W)

Register 10.17. AES_STATE_REG (0x004C)



AES_STATE 表示 AES 加速器的状态。

在 Typical AES 工作模式下，

0：IDLE

1：WORK

2：无效果

3：无效果

在 DMA-AES 工作模式下，

0：IDLE

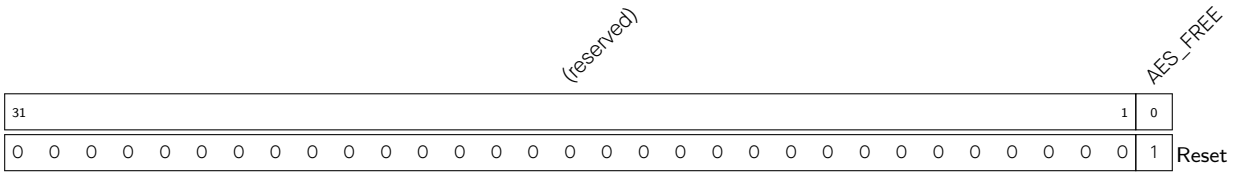
1：WORK

2：DONE

3：无效果

(RO)

Register 10.18. AES_FREE_REG (0x00D4)



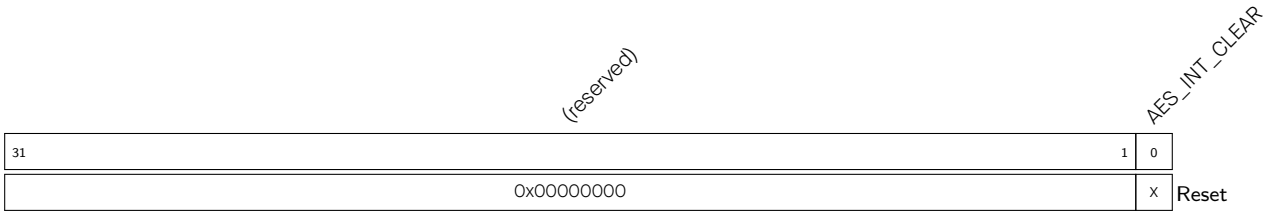
AES_FREE 配置是否允许释放 AES（即使 AES 正在运行中），以便其他需要使用 AES 的模块可以占用 AES 资源。

0：不可释放 AES

1：释放 AES

(R/W)

Register 10.19. AES_INT_CLEAR_REG (0x00AC)



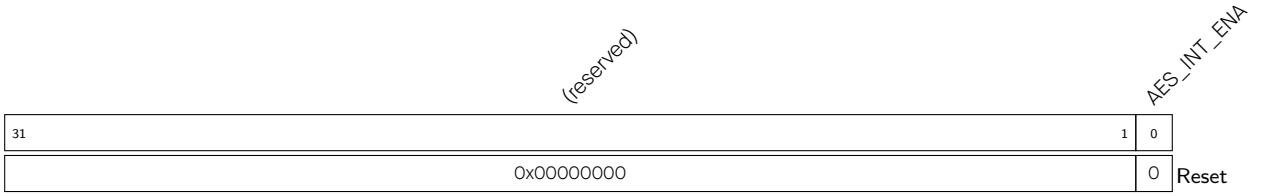
AES_INT_CLEAR 配置是否清除 AES 中断。

0：无效果

1：清除

(WT)

Register 10.20. AES_INT_ENA_REG (0x00B0)



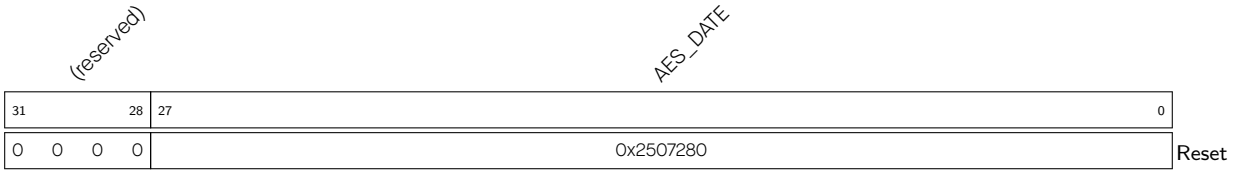
AES_INT_ENA 配置是否使能 AES 中断功能。

0：禁用

1：使能

(R/W)

Register 10.21. AES_DATE_REG (0x00B4)



AES_DATE 版本控制寄存器。(R/W)

第 11 章

ECC 加速器 (ECC)

11.1 概述

椭圆曲线密码学 (Elliptic Curve Cryptography, ECC) 是一种基于椭圆曲线数学的公开密钥加密演算法，其优势在于相对于 RSA 算法，使用较小长度的密钥就能够提供相当等级的加密安全性。

ESP32-S31 ECC 硬件加速器支持基于可选曲线的多种基础运算，用以实现对 ECC 基本运算及其衍生算法（如 ECDSA 等算法）的加速。

11.2 主要特性

ESP32-S31 ECC 硬件加速器具备以下特性：

- 支持四种可选 ECC 曲线，即 FIPS 186-5 中定义的 P-192、P-256 和 P-384，以及 GM/T 0003.2-2012 中定义的 SM2
- 提供 11 种可选工作模式
- 提升抗攻击性能
- 提供计算完成的中断和中断控制

11.3 ECC 背景知识

本节介绍了与 ECC 硬件加速器相关的背景知识以及章节中会应用到的专业名词。

11.3.1 椭圆曲线与曲线上的点

ECC 是一种基于大素数的有限域椭圆曲线的算法，这一椭圆曲线的数学表达式为：

$$y^2 = x^3 + ax + b \bmod p$$

其中，

- p 是素数，
- a 和 b 为两个小于 p 的非负整数，
- (x, y) 为满足该椭圆曲线方程的点。

11.3.2 仿射坐标系与 Jacobian 坐标系

一条椭圆曲线：

- 在仿射坐标系下的表达式为：

$$y^2 = x^3 + ax + b \bmod p$$

- 在 Jacobian 坐标系下的表达式为：

$$Y^2 = X^3 + aXZ^4 + bZ^6 \bmod p$$

一个曲线上的点在仿射坐标系下的表示 (x, y) 与在 Jacobian 坐标系下的表示 (X, Y, Z) 有如下转换关系：

- 从 Jacobian 坐标系到仿射坐标系的转换：

$$x = X/Z^2 \bmod p$$

$$y = Y/Z^3 \bmod p$$

- 从仿射坐标系到 Jacobian 坐标系的转换：

$$X = x$$

$$Y = y$$

$$Z = 1$$

11.3.3 内存块

ECC 硬件加速器的内存块用于存储 ECC 运算中所用到的输入数据和输出数据。

表 11.3-1. ECC 硬件加速器内存块

内存块[1]	大小（字节）	起始地址[2]	结束地址[2]	访问权限
ECC_MULT_Mem_k	48	0x100	0x12F	R/W
ECC_MULT_Mem_Px	48	0x130	0x15F	R/W
ECC_MULT_Mem_Py	48	0x160	0x18F	R/W
ECC_MULT_Mem_Qx	48	0x190	0x1BF	R/W
ECC_MULT_Mem_Qy	48	0x1C0	0x1EF	R/W
ECC_MULT_Mem_Qz	48	0x1F0	0x21F	R/W

[1] 内存块在各个工作模式中储存的数据内容不同，详见章节 11.4.2。

[2] 采用相对于 ECC 加速器基地址的偏移量。详见章节 3 *系统和存储器 [to be added later]* 中的表 3。

11.3.4 数据与数据块

在 ECC 硬件加速器模块中会用到的数据位宽可能为 192、256 或 384 位，这取决于进行运算的曲线。

例如，一个数据为 $D[255:0]$，则其可以被划分为八个 32 位的数据块 $D[n][31:0](n = 0, 1, \cdots, 7)$，序号数低的数据块对应二进制低位，即：

$$D[255:0] = D[7][31:0], D[6][31:0], D[5][31:0], D[4][31:0], D[3][31:0], D[2][31:0], D[1][31:0], D[0][31:0]$$

11.3.5 数据存储

数据存储即将数据存储进一个内存块的操作，该数据为 ECC 算法的输入数据。具体而言，将数据写入一个 ECC 内存块相当于将该数据 $D[n][31:0](n = 0, 1, \cdots, N-1)$ 依次写入“该内存块起始地址 + $4 \times n$”，其中 N 为所选曲线所需的 32 位数据块数量（192 位对应 6，256 位对应 8，384 位对应 12）：

- 写入 $D[0]$ 至“起始地址”
- 写入 $D[1]$ 至“起始地址 + 4”
- $\cdots$
- 写入 $D[N-1]$ 至“起始地址 + $4 \times (N-1)$”

说明：
进行数据存储操作时，只需要写入数据长度的数据块即可，不需要在高位补 0。例如，在写入 192 位长度的数据时，只需要存储 192 位的数据，不需要在数据的高位补 0。

11.3.6 数据读取

数据读取即将一个数据从一个内存块读出的操作，该数据为 ECC 算法的输出数据。具体来说，从一个 ECC 内存块读数据相当于从“该内存块起始地址 + $4 \times n$”依次读出 $D[n][31:0](n = 0, 1, \cdots, N-1)$，其中 N 为所选曲线所需的 32 位数据块数量（192 位对应 6，256 位对应 8，384 位对应 12）：

- 从“起始地址”读出 $D[0]$
- 从“起始地址 + 4”读出 $D[1]$
- $\cdots$
- 从“起始地址 + $4 \times (N-1)$”读出 $D[N-1]$

说明：
进行数据读取操作时，只需要读取数据长度的数据块即可。例如，在读取 192 位长度的数据时，只需要读取低 192 位（即 6 个数据块）的数据。

11.3.7 标准运算与 Jacobian 运算

ESP32-S31 ECC 硬件加速器中，所有标准运算（包括标准点验证、标准点加和标准点乘）的输入数据以及输出数据中的点均在仿射坐标系中；相对应地，所有 Jacobian 运算（包括 Jacobian 点验证、Jacobian 点加和 Jacobian 点乘）的输入数据以及输出数据中的点均在 Jacobian 坐标系中。

11.4 功能描述

11.4.1 曲线模式

ESP32-S31 ECC 硬件加速器共支持四种曲线模式，每种曲线模式唯一对应一条椭圆曲线。用户通过配置 ECC_MULT_CURVE_MODE 可选定曲线模式，其与椭圆曲线的对应关系如表 11.4-1 所示。对国密标准 SM2 的支持可通过 EFUSE_DIS_SM_CRYPT 禁用。

表 11.4-1. ECC 加速器曲线模式控制

ECC_MULT_CURVE_MODE	曲线模式*
0	FIPS P-192
1	FIPS P-256
2	FIPS P-384
3	GM/T SM2

* FIPS P-192/P-256/P-384 的曲线定义，请参考 FIPS 186-5；SM2 的曲线定义，请参考 GM/T 0003.2-2012。

11.4.2 工作模式

ESP32-S31 ECC 硬件加速器共有 11 种工作模式，且这些工作模式均适用于上一小节描述的四条椭圆曲线。用户通过配置 ECC_MULT_WORK_MODE 可选定工作模式，其值与工作模式的对应关系如表 11.4-2 所示。

表 11.4-2. ECC 硬件加速器工作模式控制

ECC_MULT_WORK_MODE	对应模式
0	标准点乘
1	**保留项，不可用**
2	标准点验证
3	标准点验证 + 标准点乘
4	Jacobian 点乘
5	点加
6	Jacobian 点验证
7	标准点验证 + Jacobian 点乘
8	模加
9	模减
10	模乘
11	模除

说明：
请注意，Jacobian 点乘模式的运算速度比标准点乘模式约快 10%。

每种工作模式的具体计算和输入/输出数据，请参照下述子章节。

11.4.2.1 标准点乘模式

标准点乘模式计算公式如下：

$$Q = (Q_x, Q_y) = (J_x, J_y, J_z) = k \cdot (P_x, P_y)$$

其中，

- (Q_x, Q_y) 为仿射表示的曲线上的点。
- (J_x, J_y, J_z) 为 Jacobian 表示的曲线上的点。

- 输入数据：P_x、P_y 和 k 对应的内存块为 ECC_MULT_Mem_Px、ECC_MULT_Mem_Py 和 ECC_MULT_Mem_k。
- 输出数据：
  - Q_x 和 Q_y 对应的内存块为 ECC_MULT_Mem_Px 和 ECC_MULT_Mem_Py。
  - J_x、J_y 和 J_z 对应的内存块为 ECC_MULT_Mem_Qx、ECC_MULT_Mem_Qy 和 ECC_MULT_Mem_Qz。

11.4.2.2 标准点验证模式

标准点验证模式用于计算点 (P_x, P_y) 是否在选定的椭圆曲线上。

- 输入数据：P_x 和 P_y 对应的内存块为 ECC_MULT_Mem_Px 和 ECC_MULT_Mem_Py。
- 输出数据：点验证的结果存储在 ECC_MULT_VERIFICATION_RESULT 中。

11.4.2.3 标准点验证 + 标准点乘模式

标准点验证 + 标准点乘模式先计算点 (P_x, P_y) 是否在选定的椭圆曲线上，如果其在选定的椭圆曲线上，则继续计算如下公式：

$$Q = (Q_x, Q_y) = (J_x, J_y, J_z) = k \cdot (P_x, P_y)$$

其中，

- (Q_x, Q_y) 为仿射表示的曲线上的点。
- (J_x, J_y, J_z) 为 Jacobian 表示的曲线上的点。
- 输入数据：P_x、P_y 和 k 对应的内存块为 ECC_MULT_Mem_Px、ECC_MULT_Mem_Py 和 ECC_MULT_Mem_k。
- 输出数据：
  - 点验证的结果存储在 ECC_MULT_VERIFICATION_RESULT 中。
  - Q_x 和 Q_y 对应的内存块为 ECC_MULT_Mem_Px 和 ECC_MULT_Mem_Py。
  - J_x、J_y 和 J_z 对应的内存块为 ECC_MULT_Mem_Qx、ECC_MULT_Mem_Qy 和 ECC_MULT_Mem_Qz。

11.4.2.4 Jacobian 点乘模式

Jacobian 点乘模式计算公式如下：

$$Q = (Q_x, Q_y, Q_z) = k \cdot (P_x, P_y, 1)$$

其中，

- (Q_x, Q_y, Q_z) 为 Jacobian 表示的曲线上的点。
- 输入点 P 的 Jacobian 表示中添加的 1 为硬件默认补全，不需要输入。
- 输入数据：P_x、P_y 和 k 对应的内存块为 ECC_MULT_Mem_Px、ECC_MULT_Mem_Py 和 ECC_MULT_Mem_k。
- 输出数据：Q_x、Q_y 和 Q_z 对应的内存块为 ECC_MULT_Mem_Qx、ECC_MULT_Mem_Qy 和 ECC_MULT_Mem_Qz。

11.4.2.5 点加模式

点加模式计算公式如下：

$$R = (R_x, R_y) = (J_x, J_y, J_z) = (P_x, P_y, 1) + (Q_x, Q_y, Q_z)$$

其中，

- (R_x, R_y) 为仿射表示的曲线上的点。
- (J_x, J_y, J_z) 为 Jacobian 表示的曲线上的点。
- 输入点 P 的 Jacobian 表示中添加的 1 为硬件默认补全，不需要输入。
- 输入数据：
  - P_x 和 P_y 对应的内存块为 ECC_MULT_Mem_Px 和 ECC_MULT_Mem_Py。
  - Q_x、Q_y 和 Q_z 对应的内存块为 ECC_MULT_Mem_Qx、ECC_MULT_Mem_Qy 和 ECC_MULT_Mem_Qz。
- 输出数据：
  - R_x 和 R_y 对应的内存块为 ECC_MULT_Mem_Px 和 ECC_MULT_Mem_Py。
  - J_x、J_y 和 J_z 对应的内存块为 ECC_MULT_Mem_Qx、ECC_MULT_Mem_Qy 和 ECC_MULT_Mem_Qz。

11.4.2.6 Jacobian 点验证模式

Jacobian 点验证模式用于计算点 (Q_x, Q_y, Q_z) 是否在选定的椭圆曲线上。

- (Q_x, Q_y, Q_z) 为 Jacobian 表示的曲线上的点。
- 输入数据：Q_x、Q_y 和 Q_z 对应的内存块为 ECC_MULT_Mem_Qx、ECC_MULT_Mem_Qy 和 ECC_MULT_Mem_Qz。
- 输出数据：点验证的结果存储在 ECC_MULT_VERIFICATION_RESULT 中。

11.4.2.7 标准点验证 + Jacobian 点乘模式

标准点验证 + Jacobian 点乘模式先计算点 (P_x, P_y) 是否在选定的椭圆曲线上，如果其在选定的椭圆曲线上，则继续计算如下公式：

$$Q = (Q_x, Q_y, Q_z) = k \cdot (P_x, P_y, 1)$$

其中，

- (Q_x, Q_y, Q_z) 为 Jacobian 表示的曲线上的点。
- 输入点 P 的 Jacobian 表示中添加的 1 为硬件默认补全，不需要输入。
- 输入数据：P_x、P_y 和 k 对应的内存块为 ECC_MULT_Mem_Px, ECC_MULT_Mem_Py 和 ECC_MULT_Mem_k。
- 输出数据：
  - 点验证的结果存储在 ECC_MULT_VERIFICATION_RESULT 中。
  - Q_x、Q_y 和 Q_z 对应的内存块为 ECC_MULT_Mem_Qx、ECC_MULT_Mem_Qy 和 ECC_MULT_Mem_Qz。

11.4.2.8 模加模式

模加模式计算公式如下：

$$R = A + B \bmod N$$

其中，

- 输入数据：
  - A 和 B 对应的内存块为 ECC_MULT_Mem_Px 和 ECC_MULT_Mem_Py。
  - N 的值可通过以下寄存器字段配置：
    * 配置 ECC_MULT_CURVE_MODE 来选择曲线。
    * 配置 ECC_MULT_MOD_BASE 来选择使用曲线模数或基点阶数。
- 输出数据：R 对应的内存块为 ECC_MULT_Mem_Px。

11.4.2.9 模减模式

模减模式计算公式如下：

$$R = A - B \bmod N$$

其中，

- 输入数据：
  - A 和 B 对应的内存块为 ECC_MULT_Mem_Px 和 ECC_MULT_Mem_Py。
  - N 的值可通过以下寄存器字段配置：
    * 配置 ECC_MULT_CURVE_MODE 来选择曲线。
    * 配置 ECC_MULT_MOD_BASE 来选择使用曲线模数或基点阶数。
- 输出数据：R 对应的内存块为 ECC_MULT_Mem_Px。

11.4.2.10 模乘模式

模乘模式计算公式如下：

$$R = A \cdot B \bmod N$$

其中，

- 输入数据：
  - A 和 B 对应的内存块为 ECC_MULT_Mem_Px 和 ECC_MULT_Mem_Py。
  - N 的值可通过以下寄存器字段配置：
    * 配置 ECC_MULT_CURVE_MODE 来选择曲线。
    * 配置 ECC_MULT_MOD_BASE 来选择使用曲线模数或基点阶数。
- 输出数据：R 对应的内存块为 ECC_MULT_Mem_Py。

11.4.2.11 模除模式

模除模式计算公式如下：

$$R = A \cdot B^{-1} \bmod N$$

其中，

- 输入数据：
  - A 和 B 对应的内存块为 ECC_MULT_Mem_Px 和 ECC_MULT_Mem_Py。
  - N 的值可通过以下寄存器字段配置：
    * 配置 ECC_MULT_CURVE_MODE 来选择曲线。
    * 配置 ECC_MULT_MOD_BASE 来选择使用曲线模数或基点阶数。
- 输出数据：R 对应的内存块为 ECC_MULT_Mem_Py。

11.4.3 提升抗攻击性能

在执行不同类型的点乘计算（即模式 0、3、4 和 7）时，ESP32-S31 的 ECC 加速器提供了一个选项，可保证所有点乘运算都维持相同的最长运行时间，从而进一步提升其抗攻击性能。

要启用强抗攻击性能，请将 ECC_MULT_SECURITY_MODE 置 1。启用此设置后，ECC 加速器在每次进行点乘计算时：

- 对于每次操作，执行延迟保持恒定。
- 功耗波动最小化。

11.4.4 时钟

ESP32-S31 ECC 模块有两种时钟：

- CRYPTO_ECC_CLK：ECC 模块运行所使用的功能时钟
- CRYPTO_ECC_APB_CLK：用于配置 ECC 模块寄存器的总线时钟

两路时钟的使能条件如下：

- CRYPTO_ECC_CLK：由时钟门控寄存器 HP_SYS_CLKRST_REG_CRYPTO_SYS_CLK_EN 和 HP_SYS_CLKRST_REG_CRYPTO_ECC_CLK_EN 共同控制。当这两个寄存器值同时为 1 时，CRYPTO_ECC_CLK 使能。该功能时钟的分频还受 HP_SYS_CLKRST_DPA_CTRLO_REG 和 eFuse 字段 EFUSE_SEC_DPA_LEVEL 控制（更多信息请参考章节 *7 复位和时钟 [to be added later]*）。
- CRYPTO_ECC_APB_CLK：由时钟门控寄存器 HP_SYS_CLKRST_REG_CRYPTO_SEC_CLK_EN 和 HP_SYS_CLKRST_REG_CRYPTO_ECC_CLK_EN 共同控制。当这两个寄存器值同时为 1 时，CRYPTO_ECC_APB_CLK 使能。

在配置或启动 ECC 加速器之前，用户应先确认 CRYPTO_ECC_CLK 和 CRYPTO_ECC_APB_CLK 均已使能。

注意： ESP32-S31 的 ECDSA 数字签名外设 (ECDSA_DS) 模块在工作时也会调用 ECC 加速器。默认情况下，ECC_MULT_FREE 置 1 时允许其他模块（如 ECDSA 数字签名外设 (ECDSA_DS)）抢占 ECC 加速器资源，即使 ECC 正在执行运算也可被中断。如需保证当前 ECC 运算不被打断，用户应将 ECC_MULT_FREE 设为 0。

11.4.5 复位

ESP32-S31 ECC 有一个复位模块 CRYPTO_ECC_RST，用于复位 ECC 功能与配置寄存器。将 HP_SYS_CLKRST_REG_CRYPTO_ECC_RST_EN 配置为 0，即可释放 CRYPTO_ECC_RST 复位。

建议在两路时钟均使能后再释放 CRYPTO_ECC_RST，并在复位释放后进行寄存器配置和运算启动。

11.5 中断

ESP32-S31 ECC 硬件加速器可产生一个中断信号 ECC_INTR，并将其发送给 中断矩阵。

中断信号 ECC_INTR 由 ECC 硬件加速器的内部中断源 ECC_MULT_CALC_DONE_INT 生成。ECC 硬件加速器运算完成即触发该中断源。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。中断源 ECC_MULT_CALC_DONE_INT 由以下寄存器位控制（更多信息请参考 11.7 寄存器列表）：

- ECC_MULT_CALC_DONE_INT_RAW：ECC 硬件加速器运算完成时置 1。
- ECC_MULT_CALC_DONE_INT_ST：表示 ECC 硬件加速器运算完成中断的状态，用 ECC_MULT_CALC_DONE_INT_ENA 使能或屏蔽 ECC_MULT_CALC_DONE_INT_RAW 来生成该位。
- ECC_MULT_CALC_DONE_INT_ENA：用于使能或屏蔽 ECC 硬件加速器运算完成中断状态位。
- ECC_MULT_CALC_DONE_INT_CLR：置 1 此位清除 ECC 硬件加速器运算完成中断，对应的 ECC_MULT_CALC_DONE_INT_RAW 和 ECC_MULT_CALC_DONE_INT_ST 位会清零。

说明：
中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

11.6 配置流程

软件配置 ECC 硬件加速器的流程如下：

1. 配置 ECC 硬件加速器的时钟与复位。配置细节请参考小节 11.4.4。
2. 根据 11.4 小节的描述，按照需求配置 ECC 加速器曲线模式和工作模式。
3. 根据 11.5 小节的描述，使能 ECC_MULT_CALC_DONE_INT 中断。
4. 置位 ECC_MULT_START 以启动 ECC 运算。
5. 等待 ECC_MULT_CALC_DONE_INT 中断的产生，即 ECC 运算结束。
6. 根据 11.4.2 小节的描述，查看运算结果。

11.7 寄存器列表

本小节的所有地址均为相对于 ECC 加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

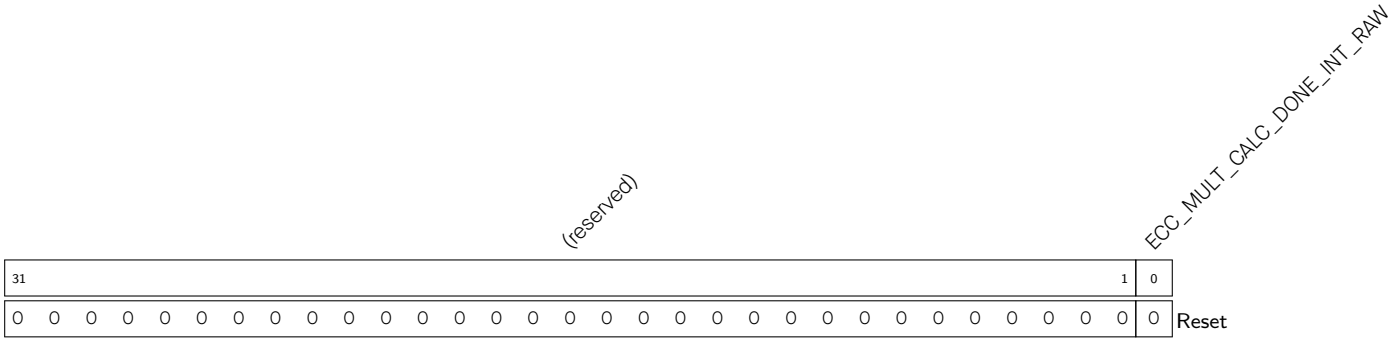
名称	描述	地址	访问权限
中断寄存器			
ECC_MULT_INT_RAW_REG	原始中断状态寄存器	0x000C	R/SS/WTC
ECC_MULT_INT_ST_REG	中断屏蔽状态寄存器	0x0010	RO
ECC_MULT_INT_ENA_REG	中断使能寄存器	0x0014	R/W
ECC_MULT_INT_CLR_REG	中断清除寄存器	0x0018	WT
配置寄存器			
ECC_MULT_CONF_REG	ECC 加速器配置寄存器	0x001C	varies
ECC_MULT_FREE_REG	ECC 抢占配置寄存器	0x0020	R/W
版本寄存器			
ECC_MULT_DATE_REG	版本控制寄存器	0x00FC	R/W

11.8 寄存器

本小节的所有地址均为相对于 ECC 加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

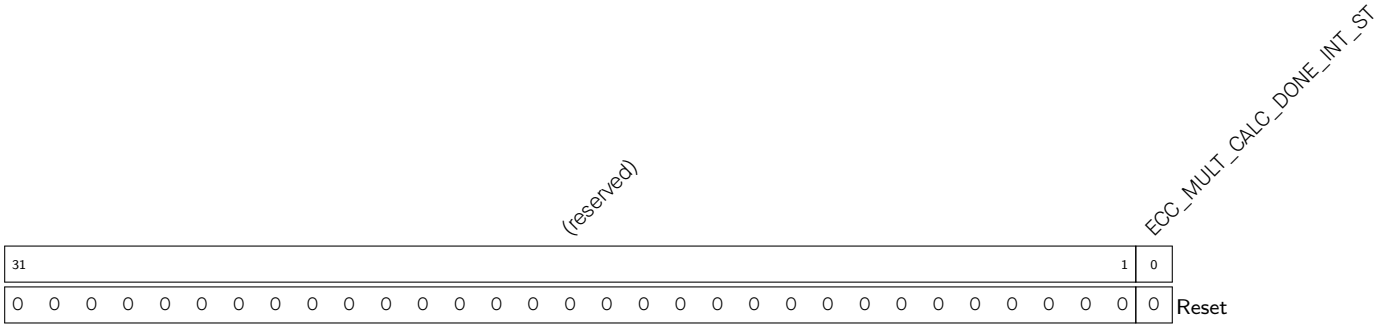
关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 11.1. ECC_MULT_INT_RAW_REG (0x000C)



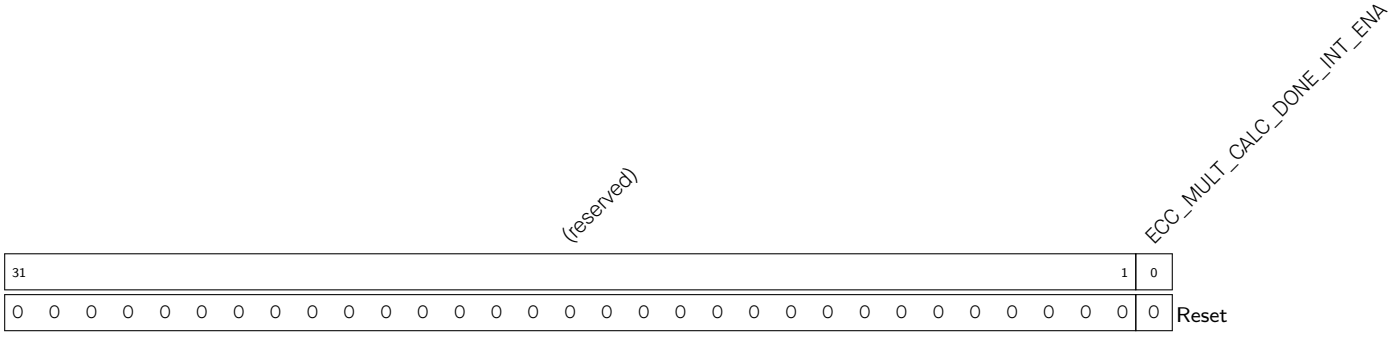
ECC_MULT_CALC_DONE_INT_RAW ECC_MULT_CALC_DONE_INT 的原始中断状态。(R/SS/WTC)

Register 11.2. ECC_MULT_INT_ST_REG (0x0010)



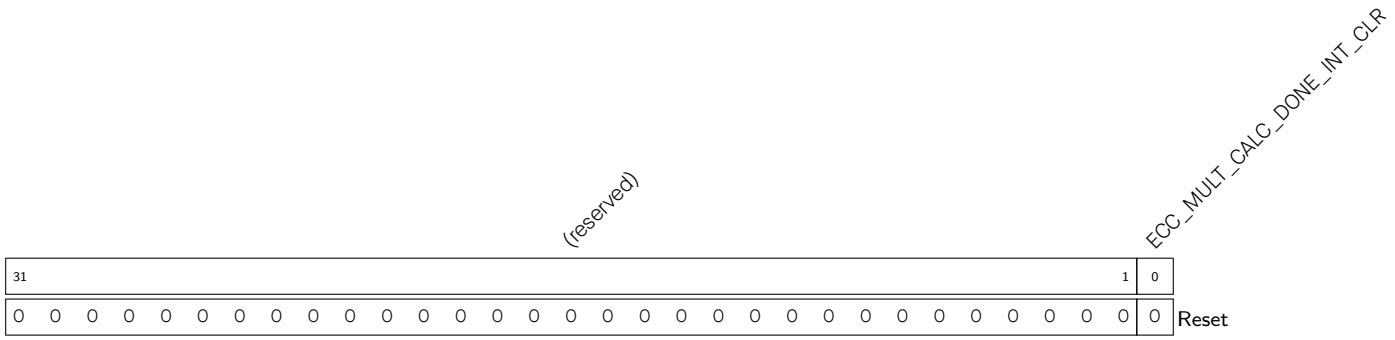
ECC_MULT_CALC_DONE_INT_ST ECC_MULT_CALC_DONE_INT 的屏蔽中断状态。(RO)

Register 11.3. ECC_MULT_INT_ENA_REG (0x0014)



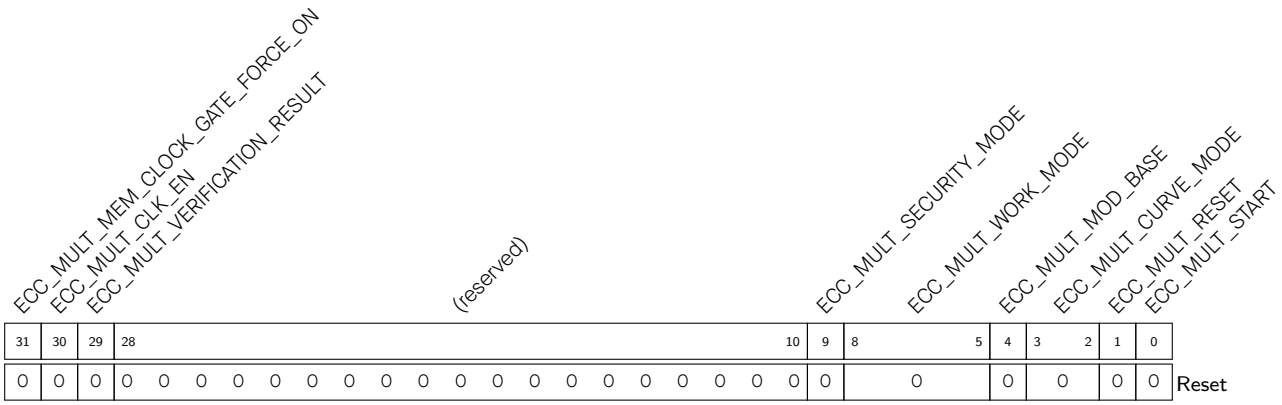
ECC_MULT_CALC_DONE_INT_ENA 写 1 使能 ECC_MULT_CALC_DONE_INT 中断。(R/W)

Register 11.4. ECC_MULT_INT_CLR_REG (0x0018)



ECC_MULT_CALC_DONE_INT_CLR 写 1 清除 ECC_MULT_CALC_DONE_INT 中断。(WT)

Register 11.5. ECC_MULT_CONF_REG (0x001C)



ECC_MULT_START 配置是否启动 ECC 加速器。此位运算结束后自动清 0。

0：无效

1：启动 ECC 加速器

(R/W/SC)

ECC_MULT_RESET 配置是否复位 ECC 加速器。

0：无效

1：复位

(WT)

ECC_MULT_CURVE_MODE 配置 ECC 加速器的曲线模式。

0：P-192

1：P-256

2：P-384

3：SM2

(R/W)

ECC_MULT_MOD_BASE 配置在模运算操作中选择使用曲线模数或基点阶数。仅在工作模式 8-11 中有效。

0: n（基点阶数）

1: p（曲线模数）

(R/W)

见下页...

Register 11.5. ECC_MULT_CONF_REG (0x001C)

接上页...

ECC_MULT_WORK_MODE 配置 ECC 加速器的工作模式。

0：标准点乘
1：保留项，不可用
2：标准点验证
3：标准点验证 + 标准点乘
4：Jacobian 点乘
5：点加
6：Jacobian 点验证
7：标准点验证 + Jacobian 点乘
8：模加
9：模减
10：模乘
11：模除
(R/W)

ECC_MULT_SECURITY_MODE 配置是否针对点乘计算提升抗攻击性能。仅在工作模式 0、3、4 和 7 中有效。

0：禁用
1：启用
(R/W)

ECC_MULT_VERIFICATION_RESULT 表示 ECC 加速器的验证结果，仅在运算完成时有效。

0：验证失败
1：验证成功
(R/SS)

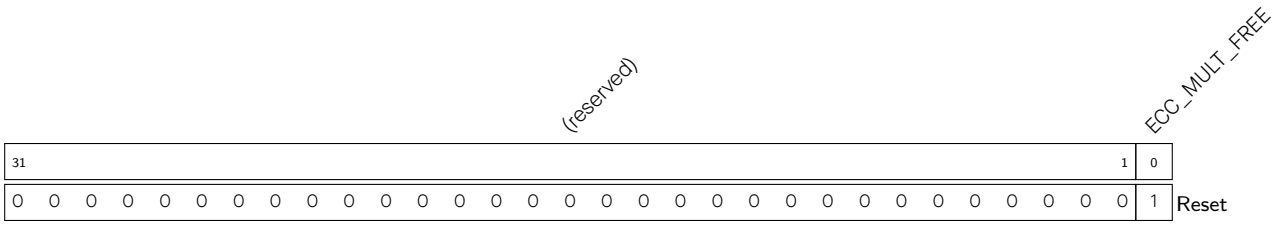
ECC_MULT_CLK_EN 配置是否强制打开寄存器门控。

0：无效
1：强制打开
(R/W)

ECC_MULT_MEM_CLOCK_GATE_FORCE_ON 配置是否强制打开 ECC 内存时钟门控。

0：无效
1：强制打开
(R/W)

Register 11.6. ECC_MULT_FREE_REG (0x0020)



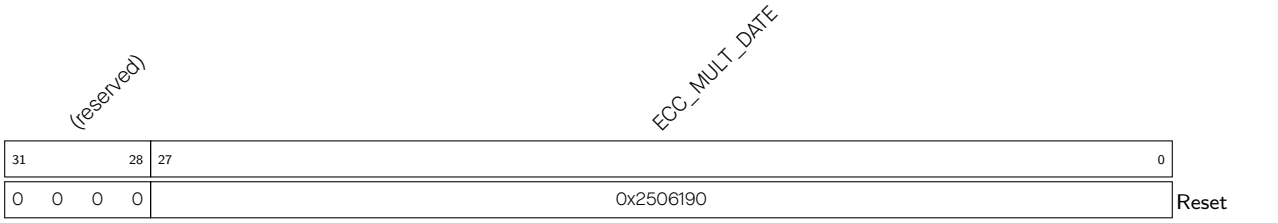
ECC_MULT_FREE 配置 ECC 加速器是否可被其他模块抢占。当置 1 时，即使 ECC 正在执行运算，其他需要 ECC 的模块也可以中断当前操作并占用 ECC 资源。

0：ECC 不可被抢占

1：ECC 可被抢占

(R/W)

Register 11.7. ECC_MULT_DATE_REG (0x00FC)



ECC_MULT_DATE 版本控制寄存器。(R/W)

第 12 章

HMAC 加速器 (HMAC)

12.1 概述

HMAC 模块通过 hash 算法 SHA-256 和密钥计算得到数据信息的信息认证码 (MAC)，采用 RFC 2104 标准。长度为 256 位的 HMAC 密钥存储在 eFuse 的密钥块中，可配置为读保护，即用户无法直接从 HMAC 加速器外部访问密钥。

12.2 主要特性

HMAC 模块的主要特性包括：

- 使用标准 HMAC-SHA-256 算法
- 基于 eFuse 密钥的 HMAC-SHA-256 算法计算，
  - 为提高安全性，软件在下行模式下无法访问其计算结果
  - 软件在上行模式下可访问其计算结果
- 下行模式下，支持生成 RSA 数字签名外设 (RSA_DS) 所需的密钥
- 下行模式下，支持重启软禁用的 JTAG

12.3 功能描述

HMAC 模块有两种工作模式，即上行模式和下行模式。上行模式中，由用户提供 HMAC 信息并回读计算结果。

下行模式中，HMAC 模块为其他两个可能的内部硬件模块提供输入：

- **JTAG 重新使能：** 如 JTAG 被暂时禁用，HMAC 可使能 JTAG。
- **RSA_DS 密钥派生：** HMAC 模块可生成用于解密存储在 flash 中的数字签名参数的密钥，供 RSA_DS 外设使用。

此外，这些计算在下行模式下自动进行，只要配置正确，就可以确保密钥及派生密钥材料的保密性。

说明：

芯片释放复位信号后，HMAC 模块将检查 eFuse 中是否烧写 RSA_DS_KEY 密钥。如果存在该密钥，HMAC 模块将自动进入下行 RSA_DS_KEY 派生模式，完成相应密钥计算。该过程由硬件自动完成，无需软件参与。当复位后的下行操作完成后，HMAC 将自动返回空闲状态。

12.3.1 上行模式

在上行模式下计算 HMAC 值，用户应执行以下步骤：

1. 初始化 HMAC 模块，进入上行模式。
2. 将正确填充的信息写入 HMAC 中，一次写入一个块。
3. 从 HMAC 回读计算结果。

有关此过程的详细步骤，可参见章节 12.5。

说明：
使用上行模式的较多为支持 HMAC-SHA-256 算法的挑战-应答协议。假设挑战-应答协议中的通讯双方分别称为 A 和 B，想要交换的数据信息为 M，则协议的一般验证流程为：

- A 计算出一个特殊的随机数 M。
- A 将 M 发送给 B。
- B（通过 M 和密钥）计算 HMAC 结果，并将结果发送给 A。
- A（通过 M 和密钥）内部计算 HMAC 结果。
- A 比较两次计算结果。如结果相同，则验证通过 B 的身份。

12.3.2 下行模式 - JTAG 启动功能

JTAG 可被 eFuse 暂时关闭，关闭后，用户可使用 HMAC 或 *安全调试控制器 (SDC) [to be added later]* 模块重启 JTAG 功能。（详细步骤可参见章节 *4 eFuse 控制器 (eFuse) [to be added later]*。）默认情况下，该功能由安全调试控制器模块控制。但是，设置 EFUSE_RE_ENABLE_JTAG_SOURCE 后，JTAG 重新启用功能的控制权将转移至 HMAC 模块。

该模式下，用户应提供某 eFuse 密钥的 HMAC 计算结果，HMAC 模块将检查用户提供的数值与计算的数值是否匹配。如果二者相同，JTAG 将被启动，直到用户再次调用 HMAC 模块清除计算结果并关闭 JTAG。

用户可执行以下步骤重启 JTAG：

1. 通过初始化时钟和 HMAC 复位信号启动 HMAC 模块，并通过配置 HMAC_SET_PARA_PURPOSE_REG 位进入下行 JTAG 启动模式，等待计算完成。详细步骤可参见章节 12.5。
2. 将 1 写入 HMAC_SOFT_JTAG_CTRL_REG 寄存器，进入 JTAG 重启模式。
3. 将 256 位的 HMAC 值写入寄存器 HMAC_WR_JTAG_REG。该值是预先在本地使用 SHA-256 和已被写入 eFuse 的密钥对 32 字节的 0x00 进行 HMAC 计算得到的。需以大端字序分 8 次写入，每次 32 位。
4. 如果由 eFuse 中密钥计算的 HMAC 结果与用户在步骤 3 中写入的数值一致，则 JTAG 重启。否则，JTAG 保持关闭状态。
5. 在用户将 1 写入寄存器 HMAC_SET_INVALIDATE_JTAG_REG 或复位重启之后，JTAG 将保持关闭状态。如需再次重启 JTAG，请重复上述步骤。

12.3.3 下行模式 - RSA 数字签名外设密钥派生功能

RSA 数字签名外设 (RSA_DS) 模块使用 AES-CBC 算法加密其参数。HMAC 模块作为密钥导出函数 (KDF) 导出解密上述参数的 AES 密钥。

在启用 RSA_DS 外设之前，用户必须先通过 HMAC 模块计算得到 RSA_DS 外设工作时所需的密钥。详细信息可参见章节 15 *RSA 数字签名外设 (RSA_DS)*。芯片上电后，HMAC 模块将检查 eFuse 中是否烧写有 RSA_DS 外设所需要的功能密钥。如果存在该密钥，HMAC 模块将自动进入下行 RSA_DS_KEY 派生模式，完成相应密钥计算。

12.4 HMAC eFuse 配置

每个烧写入 eFuse 密钥块的 HMAC 密钥都有其功能，指定该密钥用于实现哪一功能。HMAC 中，未配置匹配的功能数值的密钥无法执行计算。当前 HMAC 模块共支持三种功能：下行模式下的 JTAG 重启功能和 RSA_DS 密钥导出功能，以及上行模式下的 HMAC 计算功能。表 12.4-1 列出了上述各项功能，以及一种可指定密钥同时用于 JTAG 重启和 RSA_DS 密钥导出的功能。

启动 HMAC 模块进行计算之前，用户应先读取章节 4 *eFuse 控制器 (eFuse) [to be added later]* 中 EFUSE_KEY_PURPOSE_x 的值（基于 eFuse 中共存在 5 个密钥，故 x 的值为 0 ~ 4，EFUSE_KEY_PURPOSE_0 ~ EFUSE_KEY_PURPOSE_4 属于寄存器 EFUSE_RD_REPEAT_DATA2_REG），检查 eFuse 中是否已烧写密钥。例如，在上行模式下，如果 EFUSE_KEY_PURPOSE_0~4 中没有 EFUSE_KEY_PURPOSE_HMAC_UP，则说明 eFuse 中没有上行模式可用的密钥。此时，可按照下述步骤将密钥烧写入 eFuse 中：

1. 准备一个 256 位 HMAC 密钥，将其烧写到空的 eFuse 密钥块 y 中。eFuse 中共有 5 个密钥块，分别为 4 ~ 8，因此 y 的值为 4 ~ 8。也就是说，若密钥为 key0，则对应的密钥块为 block4。接着，将功能对应数值烧写至 EFUSE_KEY_PURPOSE_$(y-4)$。例如在上行模式下，烧写密钥后，应将 EFUSE_KEY_PURPOSE_HMAC_UP（对应值为 8）写入 EFUSE_KEY_PURPOSE_$(y-4)$。更多烧写 eFuse 密钥的详细信息，可参见章节 4 *eFuse 控制器 (eFuse) [to be added later]*。
2. 如有需要，配置 eFuse 密钥块读保护功能，使用户无法读取密钥值。用户应保留步骤中生成的随机密钥副本以便验证。

请注意，功能为 EFUSE_KEY_PURPOSE_HMAC_DOWN_ALL 的密钥既可用于 JTAG 重启，也可用于 RSA_DS 密钥导出。

表 12.4-1. HMAC 功能及配置数值

功能 (m)	模式	数值	描述
JTAG 重启	下行模式	6	EFUSE_KEY_PURPOSE_HMAC_DOWN_JTAG
RSA_DS 密钥导出	下行模式	7	EFUSE_KEY_PURPOSE_HMAC_DOWN_DIGITAL_SIGNATURE
HMAC 计算	上行模式	8	EFUSE_KEY_PURPOSE_HMAC_UP
JTAG 重启和 RSA_DS 密钥导出	下行模式	5	EFUSE_KEY_PURPOSE_HMAC_DOWN_ALL

选取 eFuse 的密钥块与 HMAC 功能

eFuse 控制器共有 5 个密钥块，即 KEY0 ~ 4 。用户将编号 n 写入寄存器 HMAC_SET_PARA_KEY_REG，表示选择 KEYn 作为本次 HMAC 模块运行时使用的密钥。

将所使用功能对应的数值写入寄存器 HMAC_SET_PARA_PURPOSE_REG 完成配置（可参见章节 12.5）。需要注意的是，eFuse 存储器中的密钥在烧写时都定义了功能用途，只有当 HMAC 的配置功能与 KEYn 定义的功能用

途相匹配时，HMAC 模块才会执行配置好的计算任务。否则，HMAC 模块将返回匹配错误结果并结束当前计算任务。

比如，如果用户选择了 KEY3 作为本次计算的密钥，且烧写入 EFUSE_KEY_PURPOSE_3 中的数值为 6，即功能 EFUSE_KEY_PURPOSE_HMAC_DOWN_JTAG。参照表 12.4-1 可知，KEY3 是用于 JTAG 重启的密钥。如果此时寄存器 HMAC_SET_PARA_PURPOSE_REG 中配置的数值也是 6，HMAC 模块才会启动 JTAG 重启功能的计算。

从密钥管理器中选取密钥

用户可以在密钥管理器中部署 HMAC 的密钥。要选择密钥管理器中的该密钥，请将密钥编号 7 写入寄存器 HMAC_SET_PARA_KEY_REG。用户从密钥管理器中选取密钥时，寄存器中 HMAC 操作的预定目标会自动设置为"HMAC Calculation"，其他设置都不会生效。

12.5 配置流程

用户调用 ESP32-S31 中 HMAC 流程如下：

12.5.1 启动 HMAC 模块

1. 置位寄存器 HP_SYS_CLKRST_REG_CRYPTO_HMAC_CLK_EN 中 HMAC 和 SHA 的外设时钟位，清除寄存器 HP_SYS_CLKRST_REG_CRYPTO_HMAC_RST_EN 中相应的外设复位位。关于如何配置时钟和重启信号，请参见章节 7 *复位和时钟 [to be added later]*。
2. 将 1 写入寄存器 HMAC_SET_START_REG。

12.5.2 配置 HMAC 密钥和密钥功能

1. 将密钥功能 m 写入寄存器 HMAC_SET_PARA_PURPOSE_REG。表 12.4-1 列出了 m 对应的数值，可参见章节 12.4。
2. 将数值 n 写入寄存器 HMAC_SET_PARA_KEY_REG，选择 eFuse 存储器中的 KEYn 作为本次计算的密钥（n 的取值范围为 0 ~ 4，7），可参见章节 12.4。
3. 将 1 写入寄存器 HMAC_SET_PARA_FINISH_REG，完成配置。
4. 读取寄存器 HMAC_QUERY_ERROR_REG。如果返回值为 1，表明选取的密钥块与配置的密钥功能不匹配，本次计算任务中止。如果返回值为 0，表明选取的密钥块与配置的密钥功能匹配，可以执行计算流程。
5. 如果 HMAC_SET_PARA_PURPOSE_REG 的数值不为 8，表明 HMAC 模块当前处于下行模式，请参考章节 12.5.3 进行下一步配置。如果数值为 8，表明 HMAC 模块当前处于上行模式，请参考章节 12.5.4 进行下一步配置。

12.5.3 下行模式流程

1. 轮询状态寄存器 HMAC_QUERY_BUSY_REG，当读取到该寄存器的值为 0 时，继续下一步骤。
2. 如需清除计算结果，并确保后续不再使用相关硬件（JTAG 或 RSA_DS），用户可将 1 写入寄存器 HMAC_SET_INVALIDATE_JTAG_REG，清除 JTAG 密钥生成的结果；也可将 1 写入寄存器 HMAC_SET_INVALIDATE_DS_REG，清除 RSA_DS_KEY 生成的结果。此后，如需重启 JTAG 或 RSA_DS 外设，需重新进行 HMAC 调用流程。

12.5.4 上行模式流程

1. 上行模式下传输数据：
   (a) 轮询状态寄存器 HMAC_QUERY_BUSY_REG，当读取到该寄存器的值为 0 时，继续下一步骤。
   (b) 用户根据章节 12.6.1 的描述对数据进行 SHA 附加填充并获取数据块 Block_n ($n >= 0$)
   (c) 根据数据块 n 的数量，选择其中一种操作：
   - 如果只有唯一一个数据块，同时该数据块已经包含了所有的填充位：
     i. 将 512 位的数据块 Block_n 写入寄存器 HMAC_WR_MESSAGE_*n*_REG (*n*: 0-15) 中。随后，将 1 写入寄存器 HMAC_SET_MESSAGE_ONE_REG 中，HMAC 模块将计算该数据块。
     ii. 轮询状态寄存器 HMAC_QUERY_BUSY_REG，当读取到该寄存器的值为 0 时，继续下一步骤。
     iii. 写 1 到寄存器 HMAC_ONE_BLOCK_REG 中，完成数据传输。
   - 如果 Block_n 是最后一个填充数据块：
     i. 将 512 位的数据块 Block_0 写入寄存器 HMAC_WR_MESSAGE_*n*_REG (*n*: 0-15) 中。随后，将 1 写入寄存器 HMAC_SET_MESSAGE_ONE_REG 中，HMAC 模块将计算该数据块。
     ii. 轮询状态寄存器 HMAC_QUERY_BUSY_REG，当读取到该寄存器的值为 0 时，继续下一步骤。
     iii. 自动完成数据传输。
   - 如果 Block_n 是倒数第二个填充数据块：
     i. 将 512 位的数据块 Block_n 写入寄存器 HMAC_WR_MESSAGE_*n*_REG (*n*: 0-15) 中。随后，将 1 写入寄存器 HMAC_SET_MESSAGE_ONE_REG 中，HMAC 模块将计算该数据块。
     ii. 轮询状态寄存器 HMAC_QUERY_BUSY_REG，当读取到该寄存器的值为 0 时，继续下一步骤。
     iii. 写 1 到寄存器 HMAC_SET_MESSAGE_PAD_REG 中，随后跳转到步骤 c 继续数据传输。
   - 如果 Block_n 既不是最后一个也不是倒数第二个数据块：
     i. 将 512 位的数据块 Block_n 写入寄存器 HMAC_WR_MESSAGE_*n*_REG (*n*: 0-15) 中。随后，将 1 写入寄存器 HMAC_SET_MESSAGE_ONE_REG 中，HMAC 模块将计算该数据块。
     ii. 轮询状态寄存器 HMAC_QUERY_BUSY_REG，当读取到该寄存器的值为 0 时，继续下一步骤。
     iii. 写 1 到寄存器 HMAC_SET_MESSAGE_ING_REG 中，随后跳转到步骤 c 继续数据传输。
2. 读取上行模式下的结果 hash 值：
   (a) 轮询状态寄存器 HMAC_QUERY_BUSY_REG，当读取到该寄存器的值为 0 时，继续下一步骤。
   (b) 从寄存器 HMAC_RD_RESULT_*n*_REG (*n*: 0-7) 中读取 hash 结果数值。
   (c) 将 1 写入寄存器 HMAC_SET_RESULT_FINISH_REG，结束当次计算，并同时清除计算结果。
   (d) 上行模式下的操作完成。

12.6 HMAC 算法细节

12.6.1 附加填充位

HMAC 模块中采用 SHA-256 作为加密 hash 算法。该算法中，若待输入数据的总位数不是 512 的倍数，用户须在软件中应用 SHA-256 附加填充算法。SHA-256 附加填充算法与 FIPS PUB 180-4 中章节 *Padding the Message* 所描述相同。下行模式下，用户无需输入数据或进行数据填充，HMAC 模块默认使用 32 字节 0x00 方式重启 JTAG，以及 32 字节 0xff 方式生成 RSA_DS 外设的 AES 密钥。

为方便阅读，以下将简要介绍数据填充的步骤。如图 12.6-1 所示，假设待处理数据长度为 m 个位，填充步骤如下：

1. 在待处理数据末尾附加一个 “1”。
2. 附加 k 个 “0”。其中，k 为满足 $m+1+k \equiv 448 (mod 512)$ 的最小非负数。
3. 附加一个 64 位的整数值作为二进制块。该二进制块的内容为待填充数据作为一个大端二进制整数值 m 的长度。

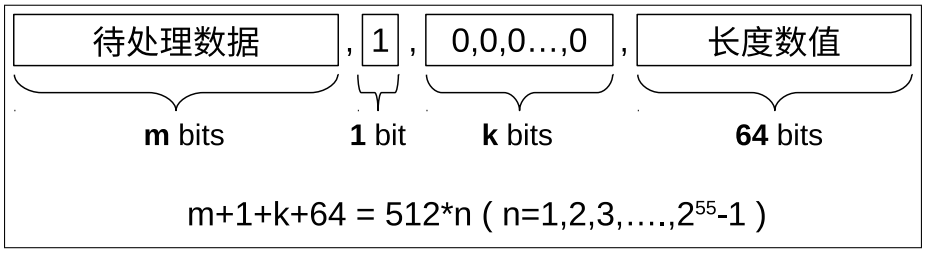


图 12.6-1. HMAC 附加填充位示意图

上行模式下，若待填充数据总位数是 512 的整数倍，则用户可通过将 1 写入 HMAC_SET_MESSAGE_END_REG 配置由硬件完成 SHA 附加填充操作，也可通过将 1 写入 HMAC_SET_MESSAGE_PAD_REG 自行完成填充操作。若待填充数据总位数不是 512 的整数倍，则用户只能自行完成 SHA 附加填充操作。数据填充操作完成后，用户应参照章节 12.5 完成后续配置。

12.6.2 HMAC 算法结构

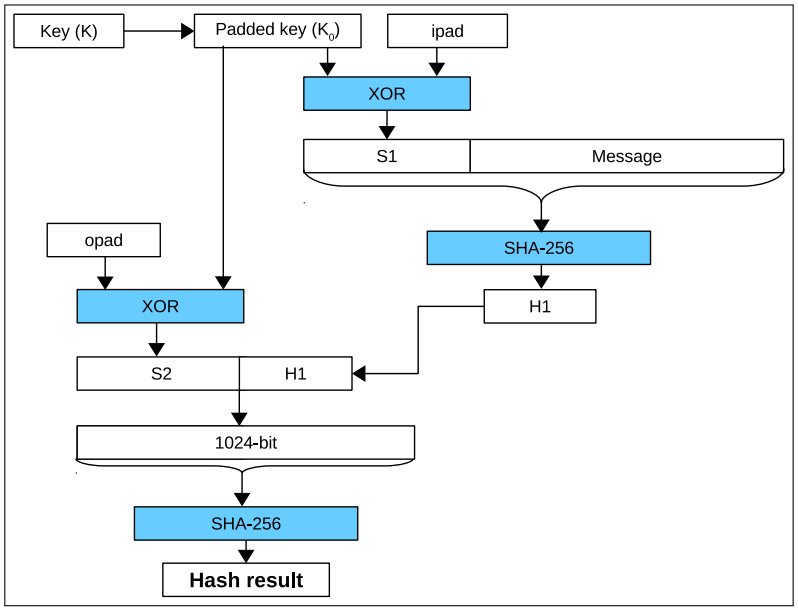


图 12.6-2. HMAC 结构示意图

HMAC 模块中应用的算法结构示意图如 12.6-2 所示。这是 RFC 2104 中描述的标准 HMAC 算法。

图 12.6-2 中，

1. ipad 是由 64 个 0x36 字节组成的 512 位数据块。
2. opad 是由 64 个 0x5c 字节组成的 512 位数据块。

首先，HMAC 模块在 256 位的密钥 K 的位序列后附加上 256 位的 0 序列，得到 512 位的 K_0。再对 K_0 和 ipad 进行异或运算，得到 512 位的 S1。将总位数为 512 倍数的待输入数据附加到 512 位的 S1 数值后，使用 SHA-256 加密算法计算得到 256 位的 H1。

HMAC 模块通过对 K_0 和 opad 进行异或运算得到 S2，将 256 位的 hash 计算结果附加到 512 位的 S2 数值后，得到 768 位长度的序列，使用章节 12.6.1 中描述的 SHA 附加填充算法将该序列填充成 1024 位的序列，最后使用 SHA-256 加密算法计算得到的最终 hash 结果（256 位）。

12.6.3 硬件占用

在运算过程中，ESP32-S31 的 HMAC 将占用 SHA 加速器，占用规则如下：

- SHA 加速器的寄存器字段 SHA_FREE 用于指示该加速器是否处于空闲状态：
  - 若该字段为 1，HMAC 将在开始运算时立即复位并占用该加速器。
  - 若该字段为 0，HMAC 将等待当前运算完成后再占用该加速器，或等待至 HMAC_TIMEOUT_LIMIT 设定的超时时间到期。
- HMAC 在运算期间将持续占用 SHA 加速器，直至运算完成后释放。
- 如果 HMAC 在占用 SHA 加速器期间接收到外部复位（通过 HP_SYS_CLKRST_REG_CRYPTO_HMAC_RST_EN 配置），该复位也会同步传播至 SHA 加速器，以清除其内部暂存的中间结果，从而防止其因外部复位攻击而发生泄漏。

说明：

硬件占用是一种保护复用模块和存储空间的机制。当一个模块被硬件占用时，用户将无法执行以下操作：

- 读取或写入模块的寄存器或存储器数据。
- 关闭模块时钟。
- 复位模块。

在硬件占用结束时，占用的模块将自动复位。此外，当用户对主模块执行软件复位时，所有占用的模块将同时复位。

12.7 寄存器列表

本小节的所有地址均为相对于 HMAC 加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

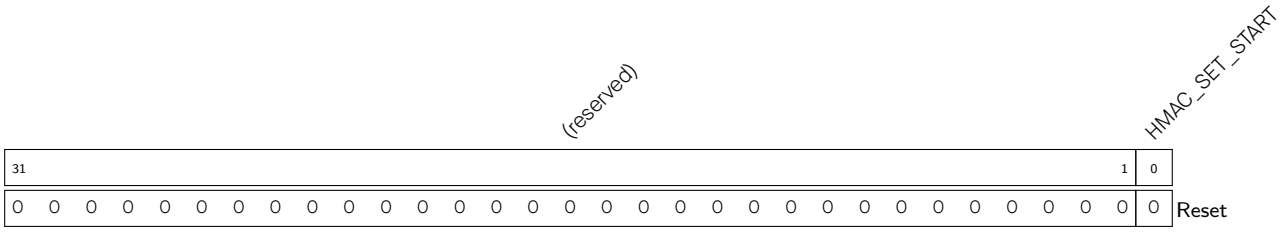
名称	描述	地址	访问
控制/状态寄存器			
HMAC_SET_START_REG	HMAC 开始控制寄存器	0x0040	WS
HMAC_SET_PARA_FINISH_REG	HMAC 配置完成寄存器	0x004C	WS
HMAC_SET_MESSAGE_ONE_REG	HMAC 信息控制寄存器	0x0050	WS
HMAC_SET_MESSAGE_ING_REG	HMAC 信息继续寄存器	0x0054	WS
HMAC_SET_MESSAGE_END_REG	HMAC 信息终止寄存器	0x0058	WS
HMAC_SET_RESULT_FINISH_REG	HMAC 读取结果完成寄存器	0x005C	WS
HMAC_SET_INVALIDATE_JTAG_REG	注销 JTAG 结果寄存器	0x0060	WS
HMAC_SET_INVALIDATE_DS_REG	注销数字签名结果寄存器	0x0064	WS
HMAC_QUERY_ERROR_REG	存储用户配置的密钥和功能的配对结果	0x0068	RO
HMAC_QUERY_BUSY_REG	存储 HMAC 模块的忙碌状态	0x006C	RO
HMAC_SET_MESSAGE_PAD_REG	软件填充寄存器	0x00F0	WO
HMAC_ONE_BLOCK_REG	One block 信息寄存器	0x00F4	WS
配置寄存器			
HMAC_SET_PARA_PURPOSE_REG	HMAC 参数配置寄存器	0x0044	WO
HMAC_SET_PARA_KEY_REG	HMAC 密钥配置寄存器	0x0048	WO
HMAC_TIMEOUT_LIMIT_REG	HMAC 超时限制配置寄存器	0x0070	R/W
HMAC_WR_JTAG_REG	重启 JTAG 寄存器 1	0x00FC	WO
HMAC_SOFT_JTAG_CTRL_REG	重启 JTAG 寄存器 0	0x00F8	WS
HMAC 信息块			
HMAC_WR_MESSAGE_*n*_REG	信息寄存器 *n* (*n*: 0-15)	0x0080+4×*n*	WO
HMAC 上行结果			
HMAC_RD_RESULT_*n*_REG	Hash 结果寄存器 *n* (*n*: 0-7)	0x00C0+4×*n*	RO
版本寄存器			
HMAC_DATE_REG	版本控制寄存器	0x01FC	R/W

12.8 寄存器

本小节的所有地址均为相对于 HMAC 加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 12.1. HMAC_SET_START_REG (0x0040)



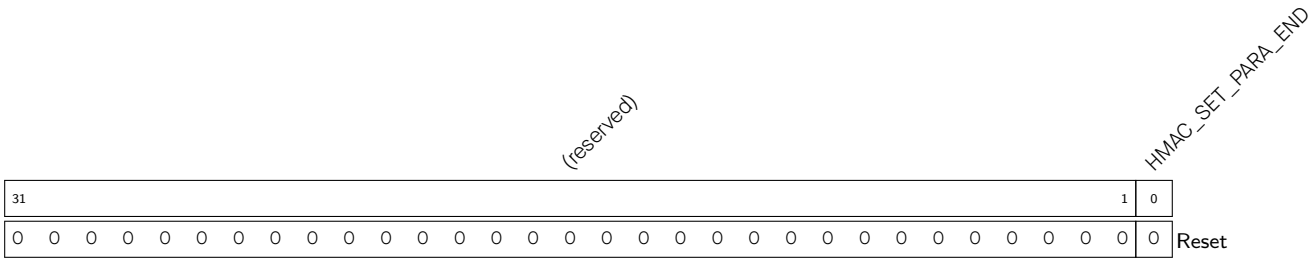
HMAC_SET_START 配置是否启动 HMAC。

0：关闭 HMAC

1：启动 HMAC

(WS)

Register 12.2. HMAC_SET_PARA_FINISH_REG (0x004C)



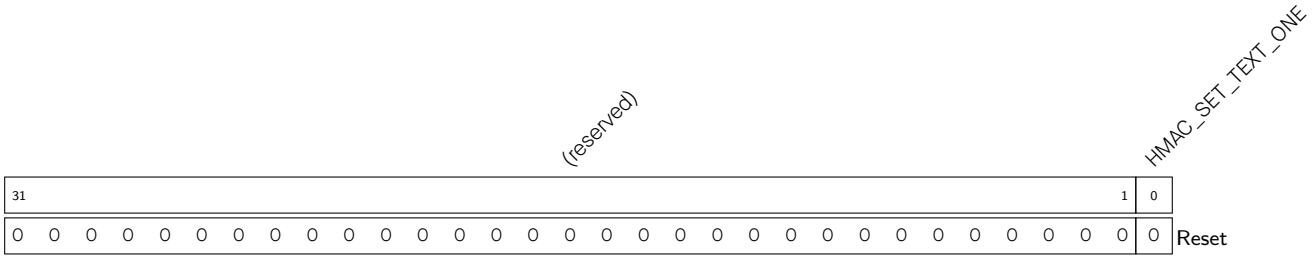
HMAC_SET_PARA_END 配置是否完成 HMAC 配置。

0：无效

1：完成配置

(WS)

Register 12.3. HMAC_SET_MESSAGE_ONE_REG (0x0050)



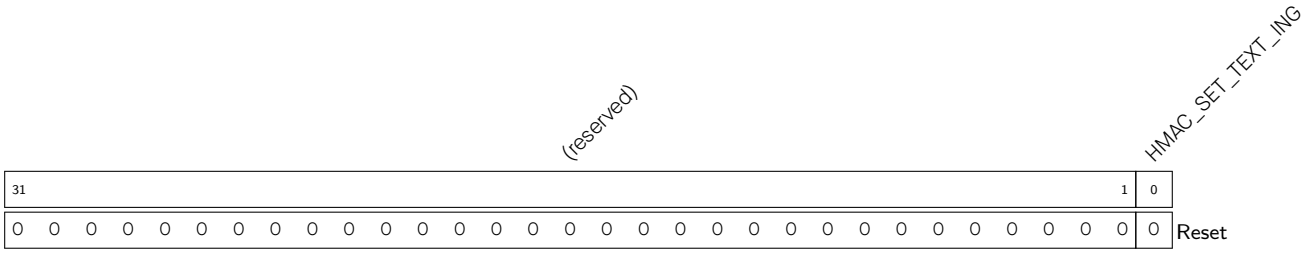
HMAC_SET_TEXT_ONE 调用 SHA 计算一个数据块。

0：无效

1：调用

(WS)

Register 12.4. HMAC_SET_MESSAGE_ING_REG (0x0054)



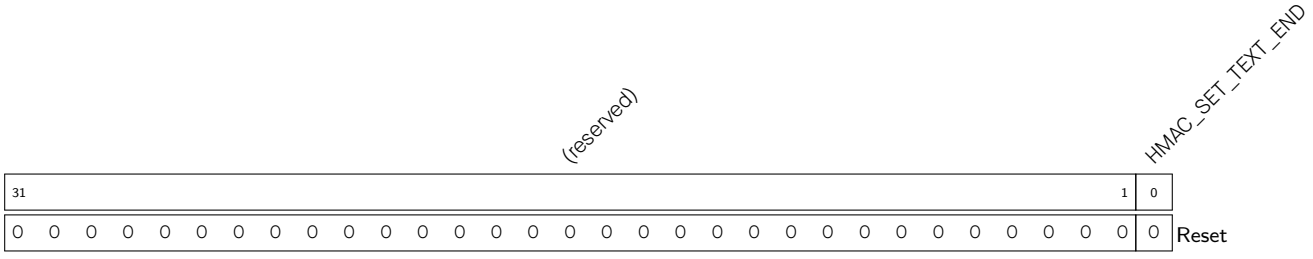
HMAC_SET_TEXT_ING 配置是否存在未处理的数据块。

0：不存在

1：存在

(WS)

Register 12.5. HMAC_SET_MESSAGE_END_REG (0x0058)



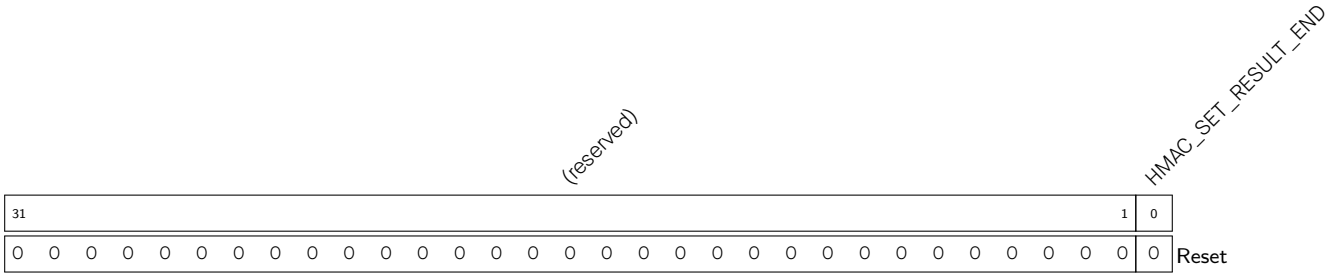
HMAC_SET_TEXT_END 配置是否开始硬件填充。

0：无效

1：开始硬件填充

(WS)

Register 12.6. HMAC_SET_RESULT_FINISH_REG (0x005C)



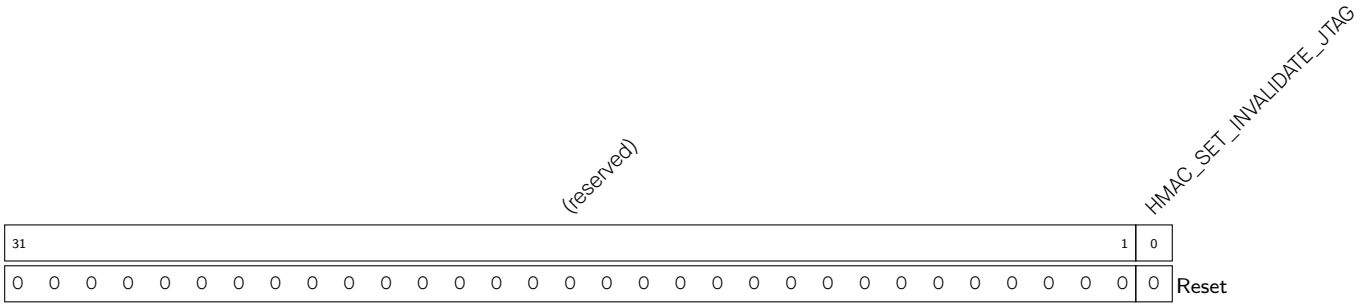
HMAC_SET_RESULT_END 配置是否退出上行模式并清空计算结果。

0：无效

1：退出上行模式并清空计算结果

(WS)

Register 12.7. HMAC_SET_INVALIDATE_JTAG_REG (0x0060)



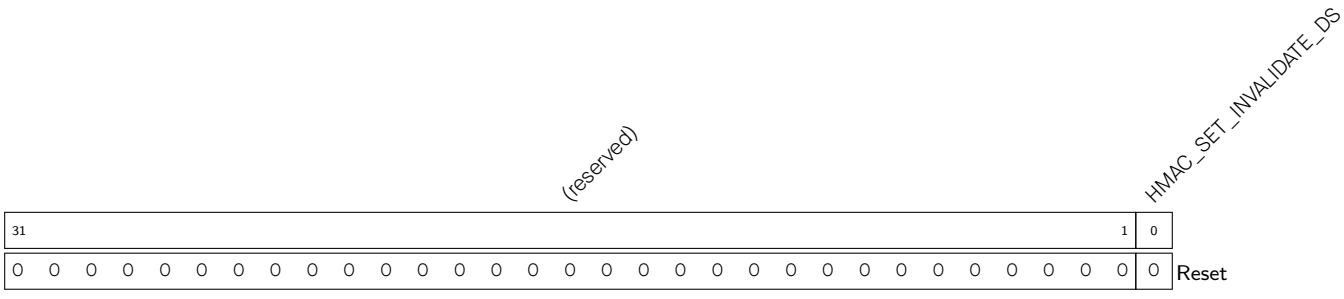
HMAC_SET_INVALIDATE_JTAG 配置是否清空下行模式下 JTAG 重启功能的计算结果。

0：无效

1：清空计算结果

(WS)

Register 12.8. HMAC_SET_INVALIDATE_DS_REG (0x0064)



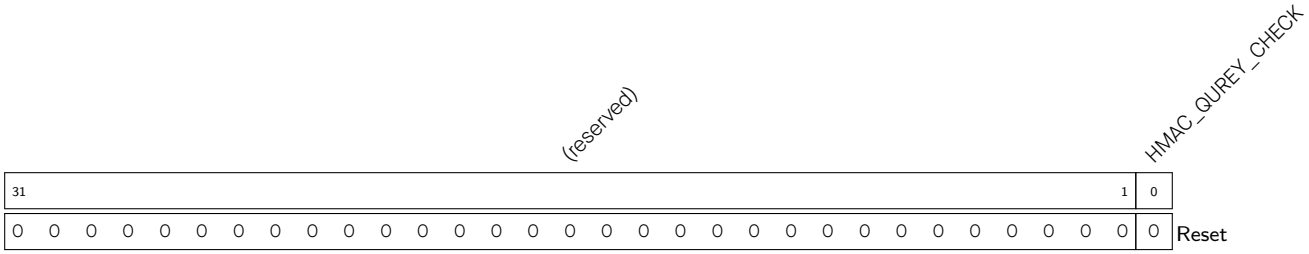
HMAC_SET_INVALIDATE_DS 配置是否清空下行模式下 RSA_DS 功能的计算结果。

0：无效

1：清空计算结果

(WS)

Register 12.9. HMAC_QUERY_ERROR_REG (0x0068)



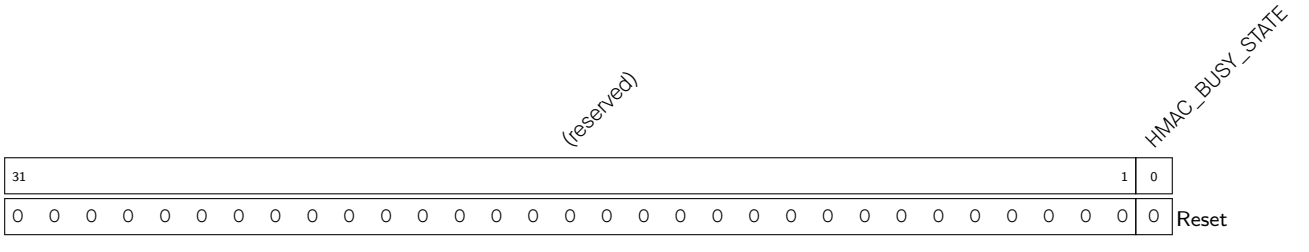
HMAC_QUREY_CHECK 表示 HMAC 密钥与功能是否匹配。

0：HMAC 密钥和功能匹配。

1：不匹配

(RO)

Register 12.10. HMAC_QUERY_BUSY_REG (0x006C)



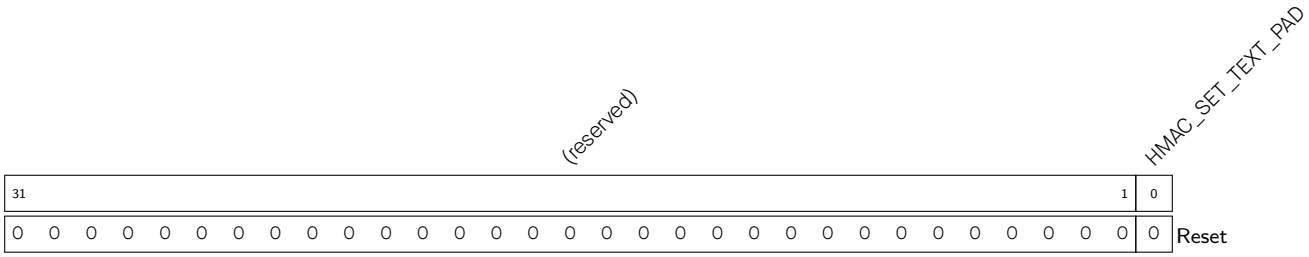
HMAC_BUSY_STATE 表示 HMAC 是否处于忙碌状态。执行计算任务之前，请确保 HMAC 已空闲。

0：空闲

1：HMAC 仍处于工作状态

(RO)

Register 12.11. HMAC_SET_MESSAGE_PAD_REG (0x00F0)



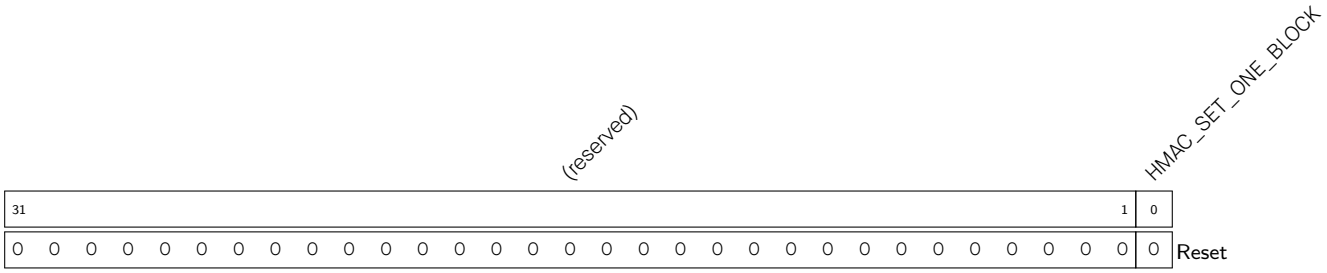
HMAC_SET_TEXT_PAD 表示是否由软件执行填充操作。

0：不由软件执行

1：由软件执行

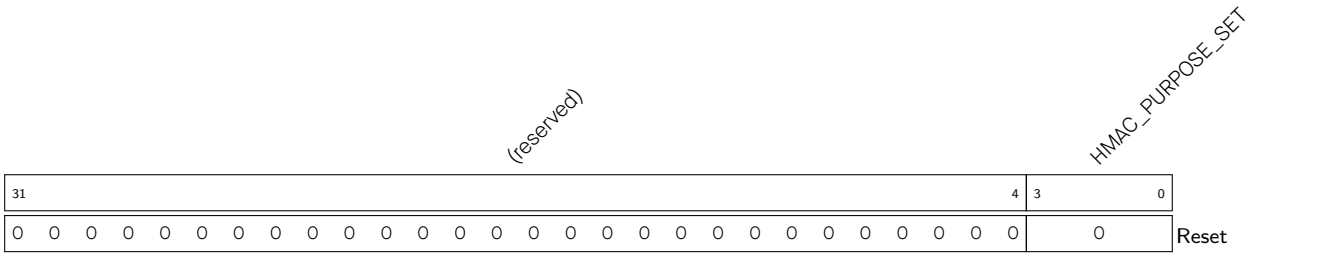
(WO)

Register 12.12. HMAC_ONE_BLOCK_REG (0x00F4)



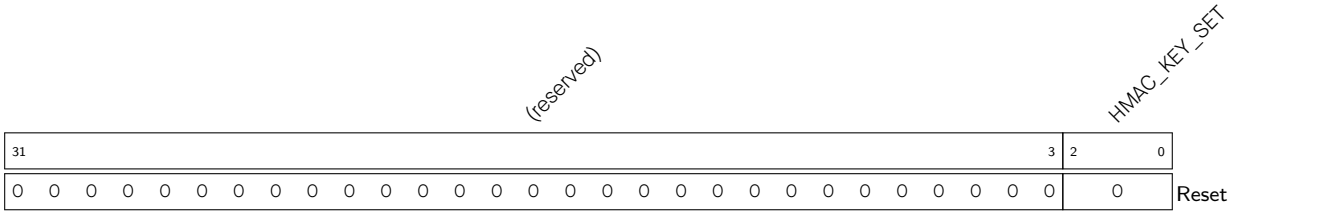
HMAC_SET_ONE_BLOCK 置 1 表示 Block_n 是唯一一个数据块，同时 Block_1 已经包含了所有的填充位，无需填充。(WS)

Register 12.13. HMAC_SET_PARA_PURPOSE_REG (0x0044)



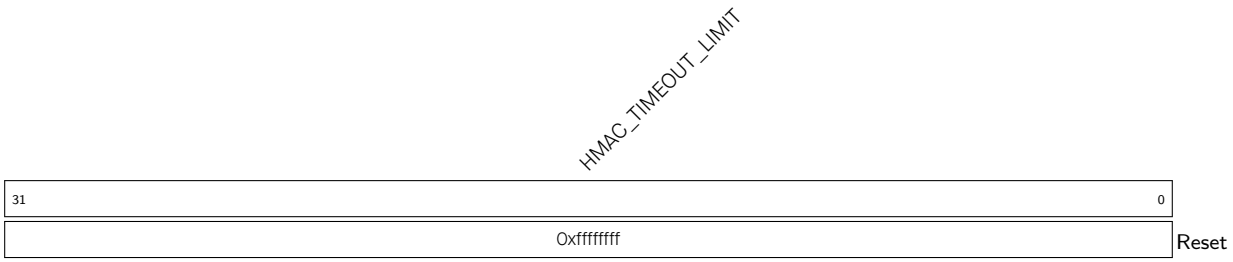
HMAC_PURPOSE_SET 设置 HMAC 功能，请参阅表 12.4-1。(WO)

Register 12.14. HMAC_SET_PARA_KEY_REG (0x0048)



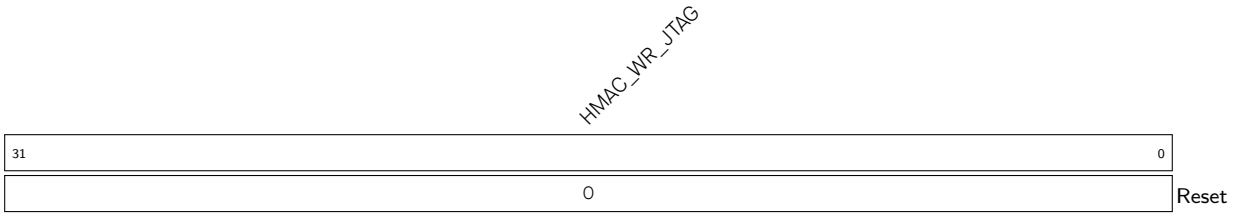
HMAC_KEY_SET 配置 HMAC 密钥。共有 5 个 eFuse 密钥和 1 个 key manager 密钥，编号 0 ~ 4 对应 5 个 eFuse 密钥，编号 7 对应 key manager 密钥，将选择的密钥编号写入该字段即可。(WO)

Register 12.15. HMAC_TIMEOUT_LIMIT_REG (0x0070)



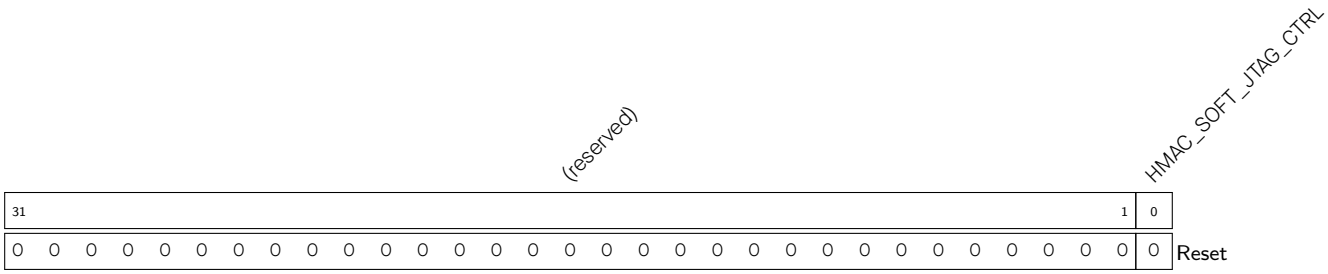
HMAC_TIMEOUT_LIMIT 配置等待 SHA IP 的超时限制。(R/W)

Register 12.16. HMAC_WR_JTAG_REG (0x00FC)



HMAC_WR_JTAG 写入用于重启 JTAG 的输入比较数值。(WO)

Register 12.17. HMAC_SOFT_JTAG_CTRL_REG (0x00F8)



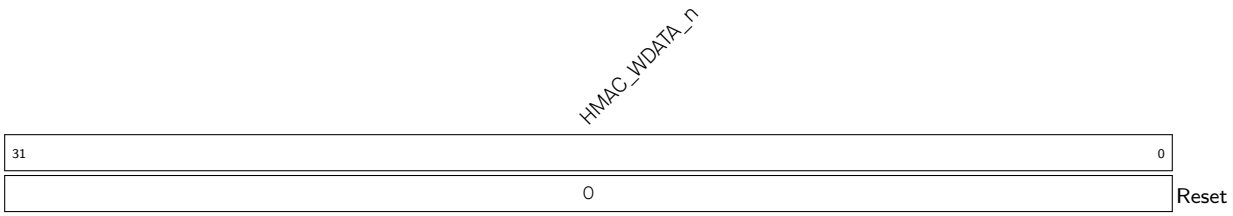
HMAC_SOFT_JTAG_CTRL 配置是否开启 JTAG 验证模式。

0：关闭

1：开启

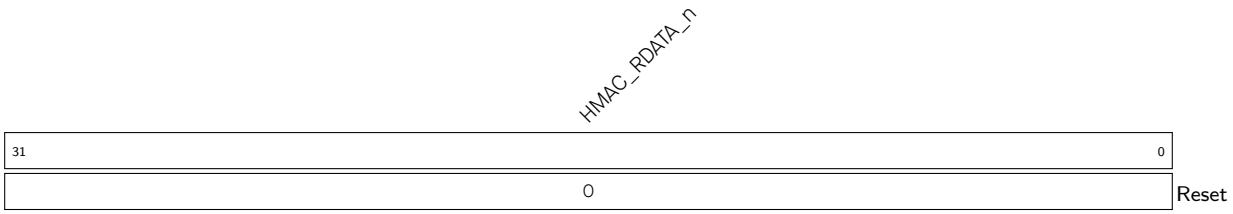
(WS)

Register 12.18. HMAC_WR_MESSAGE_*n*_REG (*n*: 0-15) (0x0080+4×*n*)



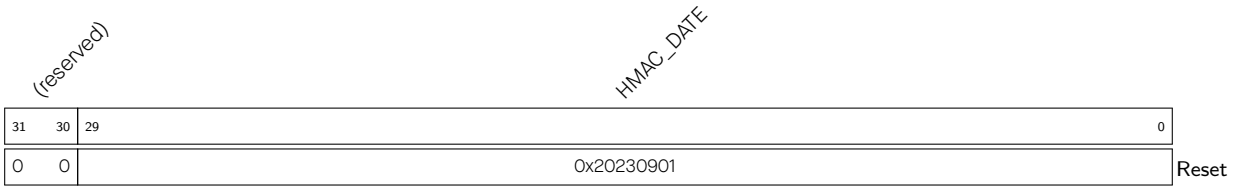
HMAC_WDATA_*n* 表示信息的第 *n* 个 32 位数据信息。(WO)

Register 12.19. HMAC_RD_RESULT_*n*_REG (*n*: 0-7) (0x00C0+4×*n*)



HMAC_RDATA_*n* 读取 hash 结果的第 *n* 个 32 位。(RO)

Register 12.20. HMAC_DATE_REG (0x01FC)



HMAC_DATE 版本控制寄存器。(R/W)

第 13 章

RSA 加速器 (RSA)

13.1 概述

RSA 加速器可为多种运用于“RSA 非对称式加密演算法”的高精度计算提供硬件支持，与纯软件 RSA 算法相比，能够极大地降低此类运算的运行时间和软件复杂度。RSA 加速器还支持多种“运算子长度”，具有很高的灵活性。

13.2 主要特性

RSA 加速器支持以下功能：

- 大数模幂运算（最大运算子长度为 4,096 位，支持两个加速选项，可实现进一步加速）
- 大数模乘运算（最大运算子长度为 4,096 位）
- 大数乘法运算（最大运算子长度为 2,048 位）
- 多种运算子长度
- 支持在运算完成后触发中断

13.3 功能描述

RSA 加速器在 HP_SYS_CLKRST_CRYPTO_CTRLO_REG 寄存器中完成使能：将 HP_SYS_CLKRST_REG_CRYPTO_RSA_CLK_EN 置 1 以打开 RSA 时钟，并将 HP_SYS_CLKRST_REG_CRYPTO_RSA_RST_EN 清零以释放 RSA 加速器复位。

不过，RSA 加速器激活后还须等待 RSA 相关存储器 初始化完成后才能开始工作。具体来说，RSA_QUERY_CLEAN_REG 读 0 时初始化开始，读 1 时初始化完成。因此，在复位后首次使用 RSA 加速器时，软件需要先查询 RSA_QUERY_CLEAN_REG 的值是否为 1，以确保 RSA 加速器可正常工作。

此外，RSA 加速器支持中断功能，可对 RSA_INT_ENA 字段写 1 或 0 以开启或关闭中断。RSA 加速器的中断功能默认关闭。

注意：

ESP32-S31 的 RSA 数字签名外设 (RSA_DS) 模块在工作期间也会调用 RSA 加速器。默认情况下，若 RSA_FREE_REG 为 1，则 RSA 数字签名外设 (RSA_DS) 模块可立即中止当前 RSA 运算并占用 RSA 完成其运算。用户应将 RSA_FREE_REG 置为 0，以避免上述情况。

13.3.1 定义与表达

ESP32-S31 的 RSA 加速器支持运算子长度为 $N = 32 \times n$ 的运算。

为采用基于 b 的位置表示法，设进制数

$$b = 2^{32}$$

则运算子可以由若干个 b 进制数来表示：

$$n = \frac{N}{32}$$
$$Z = (Z_{n-1}Z_{n-2}\cdots Z_0)_b$$
$$X = (X_{n-1}X_{n-2}\cdots X_0)_b$$
$$Y = (Y_{n-1}Y_{n-2}\cdots Y_0)_b$$
$$M = (M_{n-1}M_{n-2}\cdots M_0)_b$$
$$\overline{r} = (\overline{r}_{n-1}\overline{r}_{n-2}\cdots \overline{r}_0)_b$$

其中 $Z_{n-1}\cdots Z_0$、$X_{n-1}\cdots X_0$、$Y_{n-1}\cdots Y_0$、$M_{n-1}\cdots M_0$、$\overline{r}_{n-1}\cdots \overline{r}_0$ 分别表示一个 b 进制数，位宽皆为 32。且 Z_{n-1}、X_{n-1}、Y_{n-1}、M_{n-1}、$\overline{r}_{n-1}$ 分别为 Z、X、Y、M、$\overline{r}$ 最高位的 b 进制数，而 Z_0、X_0、Y_0、M_0、$\overline{r}_0$ 分别为 Z、X、Y、M、$\overline{r}$ 最低位的 b 进制数。

另设 $R = b^n$，则计算得参数 $\overline{r} = R^2 \bmod M$。

另外 M' 可使用下方公式计算：

$$M' = -M^{-1} \bmod b$$

其中，M^{-1} 为 M 的模乘逆元，可使用扩展二进制 GCD 算法计算。

13.3.2 大数模幂运算

大数模幂运算的算法是 $Z = X^Y \bmod M$，它是基于 Montgomery Multiplication（蒙哥马利乘法）实现的。因此，对于大数模幂运算，除了需要运算子 X、Y、M 外，还需要额外两个运算子，即参数 $\overline{r}$ 和 M'。这两个参数需要通过软件提前运算得到。

RSA 加速器支持运算子长度为 $N = 32 \times n$ ($n \in \{1, 2, 3, \ldots, 128\}$) 的大数模幂运算。$Z$、$X$、$Y$、$M$ 和 $\overline{r}$ 的位宽为这 128 种中的任意一种，要求它们的位宽必须相同，而 M' 的位宽始终是 32。

大数模幂运算的软件流程为：

1. 对 RSA_INT_ENA 字段写 1 或 0 以开启或关闭中断。
2. 配置相关寄存器。
   (a) 对寄存器 RSA_MODE_REG 写入 ($\frac{N}{32} - 1$)。
   (b) 对寄存器 RSA_M_PRIME_REG 写入 M'。
   (c) 根据需要配置加速选项相关寄存器。请参照章节 13.3.5 获取详细信息。
3. 将 X_i、Y_i、M_i、$\overline{r}_i (i \in \{0, 1, \ldots, n-1\})$ 分别写入存储器 RSA_X_MEM、RSA_Y_MEM、RSA_M_MEM、RSA_Z_MEM。每块存储器的容量都是 128 字 (word)。每块存储器的每一个字刚好存放一个 b 进制数。这些存储器使用小端序存储数据，低地址存放运算子的低位进制数，高地址存放运算子的高位进制数。

   只需按运算子长度向各存储器写入有效数据；超出该长度的数据将被忽略。

4. 对 RSA_SET_START_MODEXP 写入 1 启动计算。
5. 等待运算结束。轮询 RSA_QUERY_IDLE 直到读到 1，或者等待 RSA 中断产生（如使能）。
6. 从存储器 RSA_Z_MEM 读出运算结果 $Z_i(i \in \{0, 1, \ldots, n-1\})$。
7. 若中断功能已开启，对 RSA_CLEAR_INTERRUPT 写入 1 以清除中断。

运算结束后，寄存器 RSA_MODE_REG 中存储的运算子长度信息以及存储器 RSA_Y_MEM 中的 Y_i、存储器 RSA_M_MEM 中的 M_i、寄存器 RSA_M_PRIME_REG 中的 M' 都不会变化。但是，存储器 RSA_X_MEM 中的 X_i 与存储器 RSA_Z_MEM 中的 $\overline{r}_i$ 都已经被覆盖。所以当需要连续运算时，只需要更新被覆盖的存储器即可。

13.3.3 大数模乘运算

大数模乘运算 $Z = X \times Y \bmod M$ 也是基于 Montgomery Multiplication 实现的。因此，与大数模幂运算类似，还须预先通过软件计算额外的两个运算子 $\overline{r}$ 和 M'。

RSA 加速器也支持运算子长度为 $N = 32 \times n$ $(n \in \{1, 2, 3, \ldots, 128\})$ 的大数模乘运算。

大数模乘运算的软件流程为：

1. 对 RSA_INT_ENA 字段写 1 或 0 以开启或关闭中断。
2. 配置相关寄存器。
   (a) 对寄存器 RSA_MODE_REG 写入 $(\frac{N}{32} - 1)$。
   (b) 对寄存器 RSA_M_PRIME_REG 写入 M'。
3. 将 X_i、Y_i、M_i、$\overline{r}_i$ $(i \in \{0, 1, \ldots, n-1\})$ 分别写入存储器 RSA_X_MEM、RSA_Y_MEM、RSA_M_MEM、RSA_Z_MEM。每块存储器的容量都是 128 字 (word)。

   每块存储器的每一个字刚好存放一个 b 进制数。这些存储器使用小端序存储数据，低地址存放运算子的低位进制数，高地址存放运算子的高位进制数。

   只需按运算子长度向各存储器写入有效数据；超出该长度的数据将被忽略。
4. 对 RSA_SET_START_MODMULT 写入 1。
5. 等待运算结束。轮询 RSA_QUERY_IDLE 直到读到 1，或者等待 RSA 中断产生（如使能）。
6. 从存储器 RSA_Z_MEM 读出运算结果 Z_i $(i \in \{0, 1, \ldots, n-1\})$。
7. 若中断功能已开启，对寄存器 RSA_CLEAR_INTERRUPT 写入 1 以清除中断。

运算结束后，寄存器 RSA_MODE_REG 中存储的运算子长度信息以及存储器 RSA_X_MEM 中的 X_i、存储器 RSA_Y_MEM 中的 Y_i、存储器 RSA_M_MEM 中的 M_i、寄存器 RSA_M_PRIME_REG 中的 M' 都不会变化。但是，存储器 RSA_Z_MEM 中的 $\overline{r}_i$ 已经被覆盖。所以当需要连续运算时，只需要更新被覆盖的存储器即可。

13.3.4 大数乘法运算

大数乘法运算实现了 $Z = X \times Y$。其中 Z 的长度是运算子 X、Y 长度的两倍，所以 RSA 加速器只支持运算子 X、Y 长度为 $N = 32 \times n$ $(n \in \{1, 2, 3, \ldots, 64\})$ 的大数乘法运算。运算子 Z 的长度 $\hat{N}$ 为 $2 \times N$。

大数乘法运算的软件流程为：

1. 对 RSA_INT_ENA 字段写 1 或 0 以开启或关闭中断。

2. 配置相关寄存器。对寄存器 RSA_MODE_REG 写入 $(\frac{\hat{N}}{32}-1)$，即 $(\frac{N}{16}-1)$。
3. 将 X_i、Y_i $(i \in \{0,1,\ldots,n-1\})$ 分别写入存储器 RSA_X_MEM、RSA_Z_MEM。每块存储器的每一个字刚好存放一个 b 进制数。这些存储器使用小端序存储数据，低地址存放运算子的低位进制数，高地址存放运算子的高位进制数。n 为 $\frac{N}{32}$ 。

   X_i $(i \in \{0,1,\ldots,n-1\})$ 要填充到存储器 RSA_X_MEM 中的第 i 个字对应的地址中，但需要注意的是，Y_i $(i \in \{0,1,\ldots,n-1\})$ 并不是要填充到存储器 RSA_Z_MEM 中的第 i 个字对应的地址中，而是需要填充到存储器 RSA_Z_MEM 中的第 $n+i$ 个字对应的地址中，即存储器 RSA_Z_MEM 的基地址加上偏移量 $4\times(n+i)$。

   只需按运算子长度向各存储器写入有效数据；超出该长度的数据将被忽略。
4. 对 RSA_SET_START_MULT 写入 1。
5. 等待运算结束。轮询 RSA_QUERY_IDLE 直到读到 1，或者等待 RSA 中断产生（如使能）。
6. 从存储器 RSA_Z_MEM 读出运算结果 Z_i $(i \in \{0,1,\ldots,\hat{n}-1\})$。$\hat{n}$ 为 $2\times n$ 。
7. 若中断功能已开启，对 RSA_CLEAR_INTERRUPT 写入 1 以清除中断。

运算结束后，寄存器 RSA_MODE_REG 中存储的运算子长度信息以及存储器 RSA_X_MEM 中的 X_i 都不会变化。但是，存储器 RSA_Z_MEM 中的 Y_i 已经被覆盖。所以当需要连续运算时，只需要更新必需的寄存器与存储器即可。

13.3.5 控制加速

对于大数模幂运算，ESP32-S31 的 RSA 加速器还提供 SEARCH 和 CONSTANT_TIME 两个选项，可进一步加速大数模幂运算。默认情况下，这两个选项均配置为不额外加速。

用户可选用其中一个或两个选项以进一步加速运算。注意：即使这两个选项均未配置额外加速，使用硬件 RSA 加速器仍比纯软件实现 RSA 算法快得多。

具体来说，当这两个选项均未配置额外加速时，求解 $Z=X^Y \bmod M$ 的时间开销完全由运算子长度决定。当其中一个或两个选项配置为额外加速时，运算的时间开销还与 Y 的 0/1 位分布有关。

为了更清楚地说明问题，首先假设 Y 的二进制表示为：

$$Y=(\widetilde{Y}_{N-1}\widetilde{Y}_{N-2}\cdots\widetilde{Y}_{t+1}\widetilde{Y}_t\widetilde{Y}_{t-1}\cdots\widetilde{Y}_0)_2$$

其中，

- N 代表 Y 的长度，
- $\widetilde{Y}_t$ 的值为 1，
- $\widetilde{Y}_{N-1}$, $\widetilde{Y}_{N-2}$, ..., $\widetilde{Y}_{t+1}$ 的值均为 0，
- 且 $\widetilde{Y}_{t-1}$, $\widetilde{Y}_{t-2}$, ..., $\widetilde{Y}_0$ 中包括 m 个 0，其余 t-m 全部为 1，即 $\widetilde{Y}_{t-1}\widetilde{Y}_{t-2},\cdots,\widetilde{Y}_0$ 的汉明重量 (Hamming weight) 为 $t-m$。

此时，当启动任一额外加速选项时：

- SEARCH 选项（RSA_SEARCH_ENABLE 置 1 开启额外加速）

  - RSA 加速器将忽略所有 $\widetilde{Y}_i$（$i > \alpha$）位。其中，加速位置 α 可通过 RSA_SEARCH_POS_REG 寄存器配置。α 在配置时需小于 N-1，否则相当于没有额外加速；且不建议小于 t，否则无法正确求解 $Z = X^Y \bmod M$。当设置 α 为 t 时，加速效果最佳。此时，$\widetilde{Y}_{N-1}, \widetilde{Y}_{N-2}, ..., \widetilde{Y}_{t+1}$ 中的 0 位将在运算中全部被忽略。
  - 注意，这个选项会忽略一些位，进而导致密钥长度变短，使安全性降低。因此，不应使用在安全要求高的应用中。
- CONSTANT_TIME 选项（RSA_CONSTANT_TIME_REG 置 0 开启额外加速）
  - RSA 加速器在运算过程中将简化对 Y 中 0 位的处理。因此不难想象，Y 中的 0 越多，加速效果越明显。
  - 注意，这个选项的加速效果与密钥中的 0/1 分布密切相关，这可能会被侧信道攻击 (SCA) 等攻击加以利用，使安全性降低。因此，不应使用在安全要求高的应用中。

为了直观地展示这两个选项带来的额外加速效果，下面通过一个典型实例加以说明。本实例中：

- $Z = X^Y \bmod M$
- N 等于 3072
- Y 等于 65537
- X 和 M 取任意值
- 当 SEARCH 选项开启时，设定 α 为 16，即 RSA_SEARCH_POS_REG 配置为 16。

表 13.3-1 展示了在进行上述 $Z = X^Y \bmod M$ 运算时，开启不同加速选项组合分别对应的时间开销。

表 13.3-1. 加速效果

SEARCH 选项	CONSTANT_TIME 选项	时间开销（时钟周期）
不加速	不加速	174.7×10^6
加速	不加速	1.023×10^6
不加速	加速	0.546×10^6
加速	加速	0.540×10^6

如表 13.3-1 所示：

- 当两个选项均处于不额外加速状态时，时间开销最大。
- 当两个选项均处于额外加速状态时，时间开销最小。
- 相比于不额外加速状态，任一选项处于额外加速状态时的时间开销明显大幅度降低。

13.4 中断

ESP32-S31 的 RSA 加速器可以生成以下中断信号并发送给中断矩阵。

- RSA_INTR

这些中断信号由 RSA 加速器的内部中断源生成。表 13.4-1 列出了 RSA 加速器的内部中断源、中断触发条件、以及生成的中断信号。

表 13.4-1. RSA 加速器的内部中断源

内部中断源	触发条件	中断信号
RSA_CALC_DONE_INT	RSA 计算完成	RSA_INTR

说明：

中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

13.5 存储器列表

请注意，这里的地址都是相对于 RSA 加速器基地址的地址偏移量（相对地址），详见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	大小（字节）	起始地址	结束地址	访问
RSA_M_MEM	存储器 M	512	0x0000	0x01FF	R/W
RSA_Z_MEM	存储器 Z	512	0x0200	0x03FF	R/W
RSA_Y_MEM	存储器 Y	512	0x0400	0x05FF	R/W
RSA_X_MEM	存储器 X	512	0x0600	0x07FF	R/W

13.6 寄存器列表

本小节的所有地址均为相对于 RSA 加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

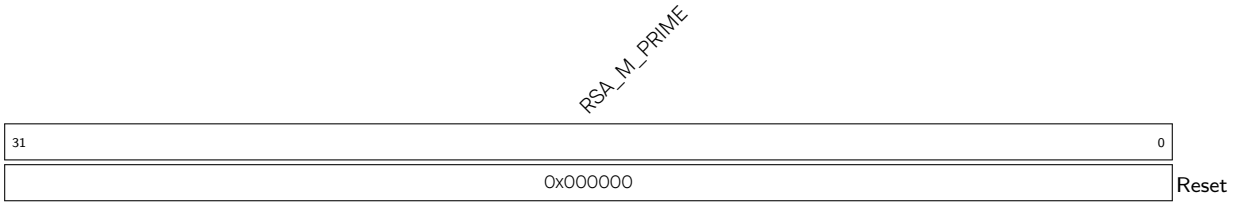
名称	描述	地址	访问
控制或配置寄存器			
RSA_M_PRIME_REG	代表 M’	0x0800	R/W
RSA_MODE_REG	配置 RSA 长度模式	0x0804	R/W
RSA_SET_START_MODEXP_REG	启动模幂运算	0x080C	WT
RSA_SET_START_MODMULT_REG	启动模乘运算	0x0810	WT
RSA_SET_START_MULT_REG	启动乘法运算	0x0814	WT
RSA_QUERY_IDLE_REG	代表 RSA 状态	0x0818	RO
RSA_CONSTANT_TIME_REG	配置 constant_time 选项	0x0820	R/W
RSA_SEARCH_ENABLE_REG	配置 search 选项	0x0824	R/W
RSA_SEARCH_POS_REG	配置 search 开始位置	0x0828	R/W
RSA_CLK_FORCE_ON_REG	配置 RSA 存储器时钟强制常开选项	0x0834	R/W
RSA_FREE_REG	配置 RSA 释放选项	0x0838	R/W
状态寄存器			
RSA_QUERY_CLEAN_REG	表示 RSA 内存初始化状态	0x0808	RO
中断寄存器			
RSA_INT_CLR_REG	清除 RSA 中断	0x081C	WT
RSA_INT_ENA_REG	使能 RSA 中断	0x082C	R/W
版本控制寄存器			
RSA_DATE_REG	版本控制寄存器	0x0830	R/W

13.7 寄存器

本小节的所有地址均为相对于 RSA 加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

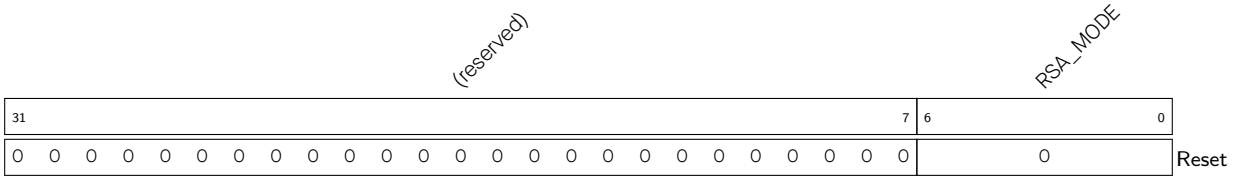
关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 13.1. RSA_M_PRIME_REG (0x0800)



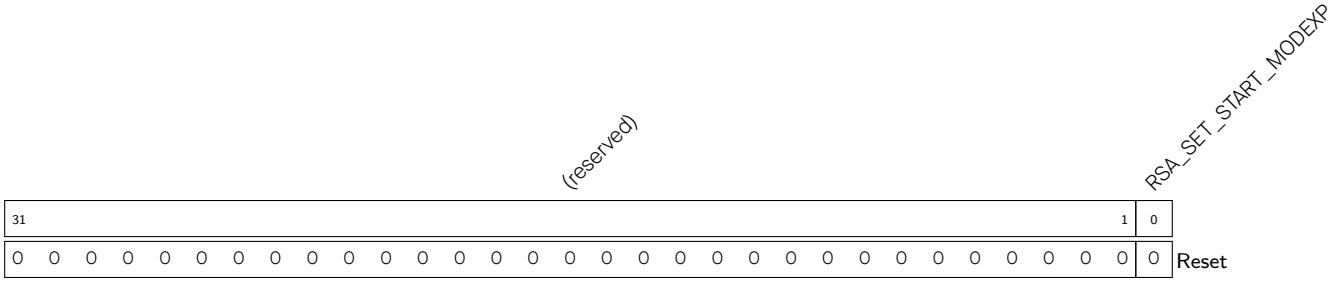
RSA_M_PRIME 代表 M'。(R/W)

Register 13.2. RSA_MODE_REG (0x0804)



RSA_MODE 配置 RSA 长度。(R/W)

Register 13.3. RSA_SET_START_MODEXP_REG (0x080C)



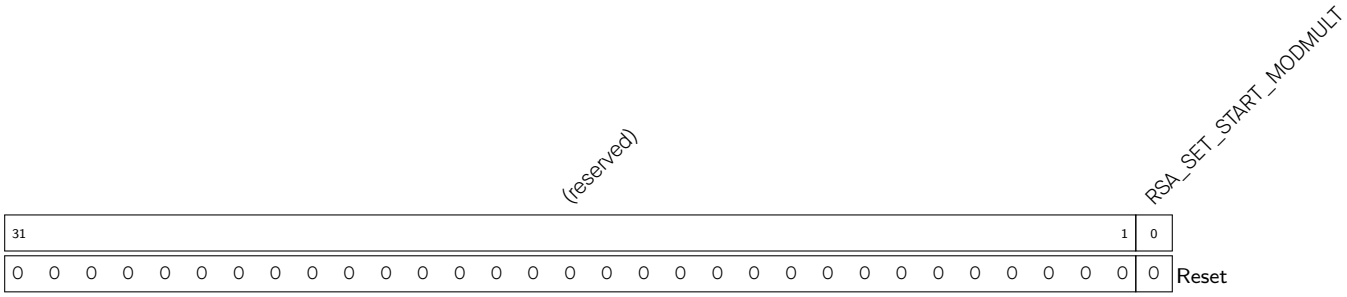
RSA_SET_START_MODEXP 配置是否启动模幂运算。

0：无效果

1：启动

(WT)

Register 13.4. RSA_SET_START_MODMULT_REG (0x0810)



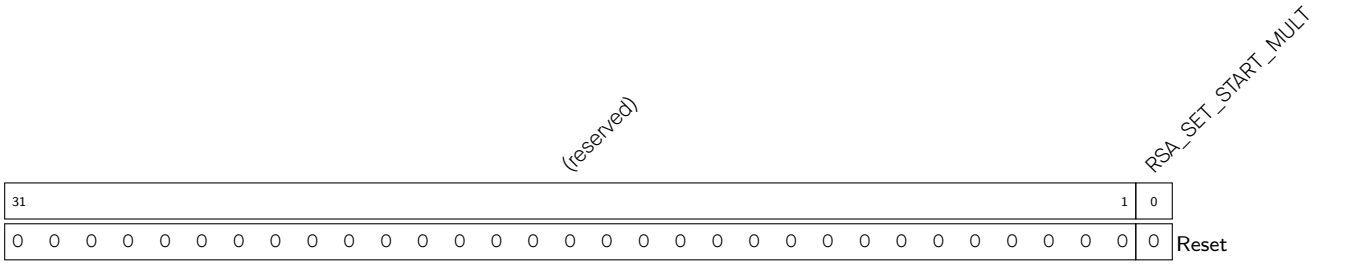
RSA_SET_START_MODMULT 配置是否启动模乘运算。

0：无效果

1：启动

(WT)

Register 13.5. RSA_SET_START_MULT_REG (0x0814)



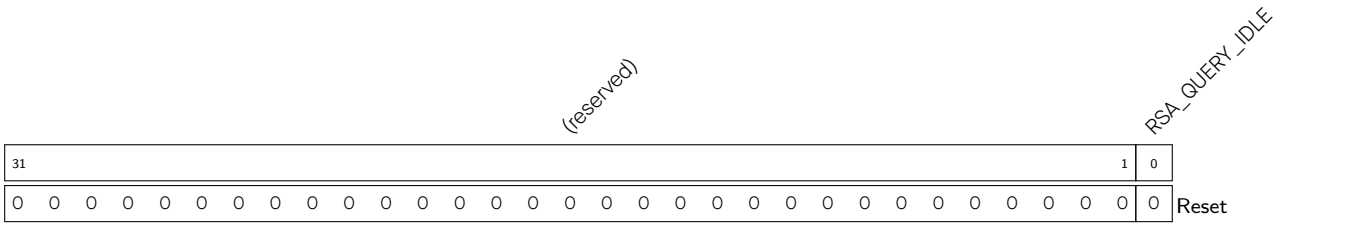
RSA_SET_START_MULT 配置是否启动乘法运算。

0：无效果

1：启动

(WT)

Register 13.6. RSA_QUERY_IDLE_REG (0x0818)



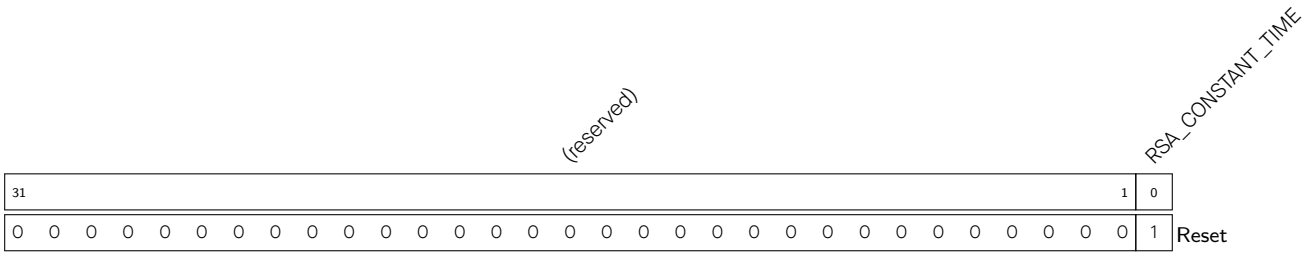
RSA_QUERY_IDLE 代表 RSA 状态。

0：忙碌

1：空闲

(RO)

Register 13.7. RSA_CONSTANT_TIME_REG (0x0820)



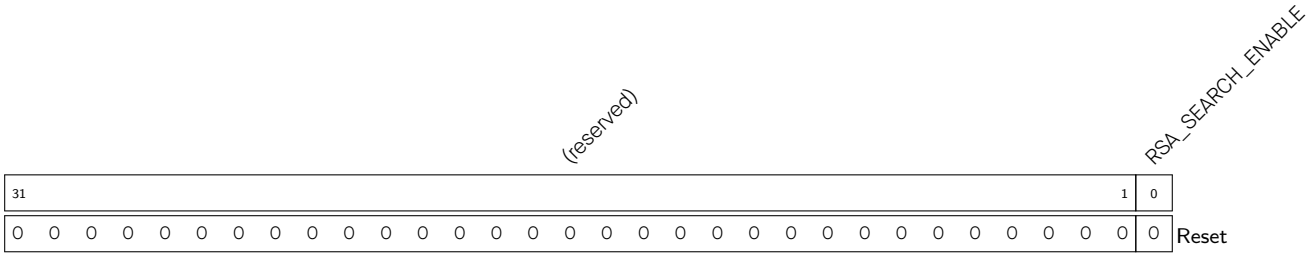
RSA_CONSTANT_TIME 配置 constant_time 选项。

0：加速

1：不加速（默认）

(R/W)

Register 13.8. RSA_SEARCH_ENABLE_REG (0x0824)



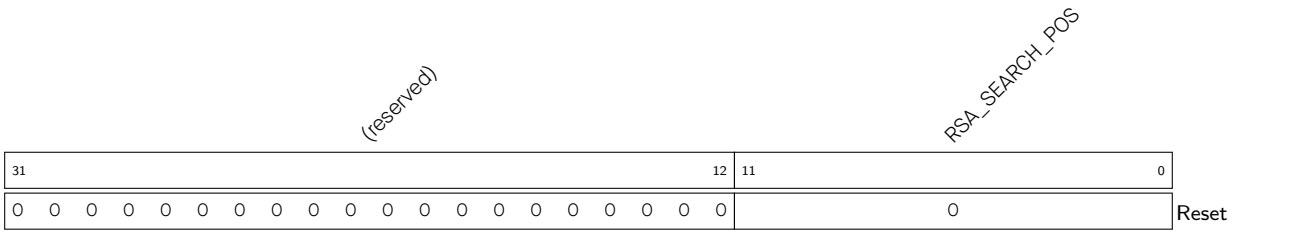
RSA_SEARCH_ENABLE 配置 search 选项。

0：不加速（默认）

1：加速

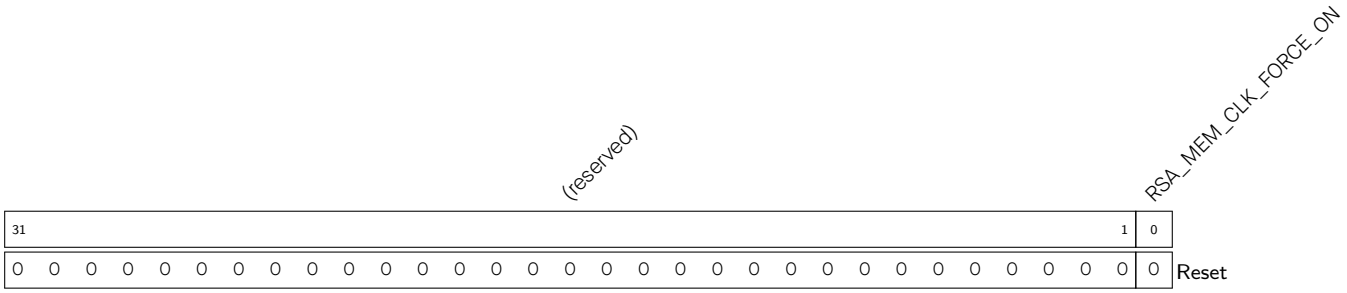
此选项需配合 RSA_SEARCH_POS_REG 使用。(R/W)

Register 13.9. RSA_SEARCH_POS_REG (0x0828)



RSA_SEARCH_POS 配置 search 的起始比特位。此选项需配合 RSA_SEARCH_ENABLE_REG 使用，仅在 RSA_SEARCH_ENABLE 为 1 时有效。(R/W)

Register 13.10. RSA_CLK_FORCE_ON_REG (0x0834)



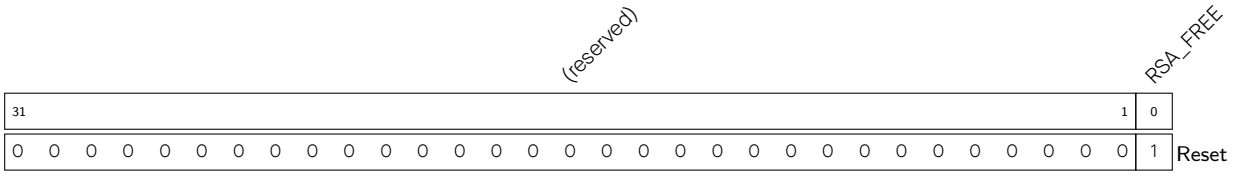
RSA_MEM_CLK_FORCE_ON 配置 RSA 存储器时钟强制常开选项。

0：不强制常开（默认）

1：强制常开

(R/W)

Register 13.11. RSA_FREE_REG (0x0838)



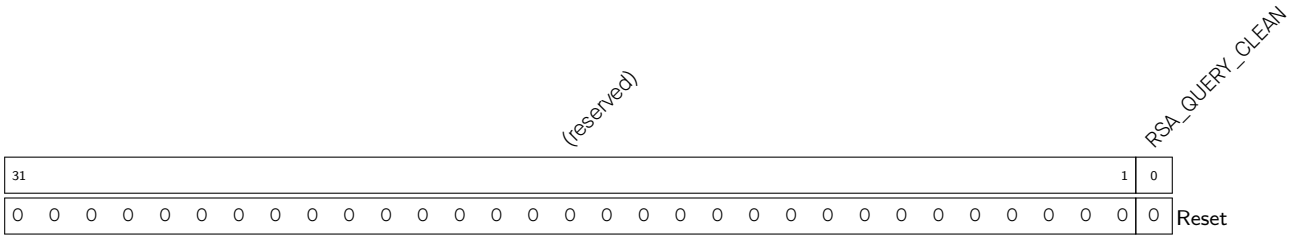
RSA_FREE 配置 RSA 释放选项。

0：不可释放

1：可释放（默认）

(R/W)

Register 13.12. RSA_QUERY_CLEAN_REG (0x0808)



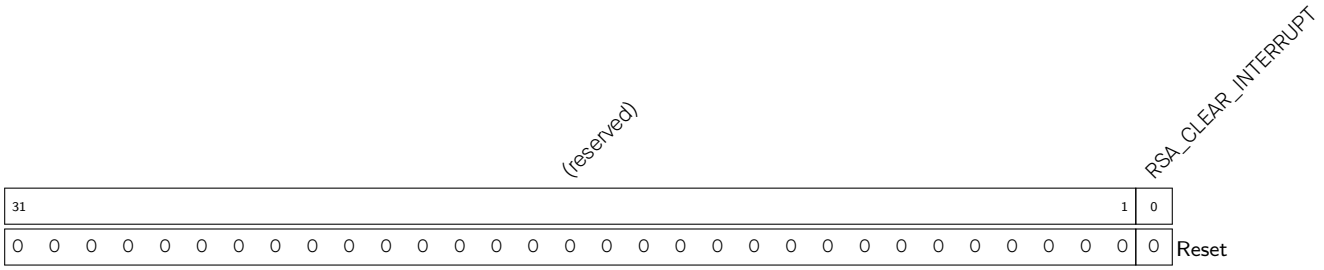
RSA_QUERY_CLEAN 代表 RSA 内存初始化的状态。

0：没有完成

1：已经完成

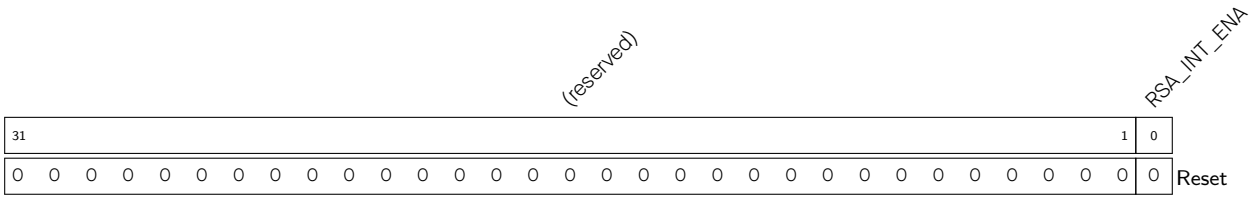
(RO)

Register 13.13. RSA_INT_CLR_REG (0x081C)



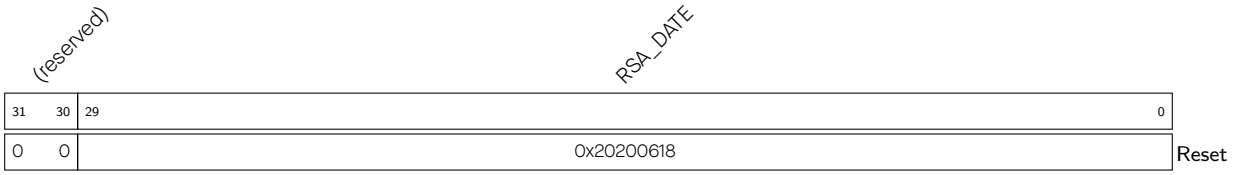
RSA_CLEAR_INTERRUPT 写 1 清除 RSA 中断。(WT)

Register 13.14. RSA_INT_ENA_REG (0x082C)



RSA_INT_ENA 写 1 使能 RSA 中断。(R/W)

Register 13.15. RSA_DATE_REG (0x0830)



RSA_DATE 版本控制寄存器。(R/W)

第 14 章

SHA 加速器 (SHA)

14.1 概述

ESP32-S31 内置的 SHA（安全哈希算法）硬件加速器可完成 SHA 运算，具有 Typical SHA 和 DMA-SHA 两种工作模式。整体而言，相比基于纯软件的 SHA 运算，SHA 硬件加速器能够极大地提高运算速度。

14.2 主要特性

ESP32-S31 的 SHA 硬件加速器：

- 支持 FIPS PUB 180-4 规范中的以下运算标准
  - SHA-1
  - SHA-224
  - SHA-256
  - SHA-384
  - SHA-512
  - SHA-512/224
  - SHA-512/256
  - SHA-512/*t*
- 支持 GM/T 0004-2012 规范中的以下哈希算法
  - SM3
- 提供两种工作模式
  - Typical SHA 工作模式
  - DMA-SHA 工作模式
- 允许插入 (interleaved) 功能（仅限 Typical SHA 工作模式）
- 允许中断功能（仅限 DMA-SHA 工作模式）

14.3 工作模式简介

ESP32-S31 内置的 SHA 加速器支持两种工作模式。

- Typical SHA 工作模式：所有数据读写统一通过 CPU 访问完成。

- DMA-SHA 工作模式：所有数据均通过 DMA 读取。也就是说，可配置 GDMA-AXI 控制器读取 SHA 哈希运算所需的全部数据，从而释放 CPU 以完成其他任务。

激活 SHA 加速器，需置位 HP_SYS_CLKRST_CRYPTO_SHA_CLK_EN 寄存器域，并清零 HP_SYS_CLKRST_SHA_RST_EN 寄存器域。用户可通过配置寄存器 SHA_START_REG 和 SHA_DMA_START_REG 以不同工作模式启动 SHA 加速器。先配置的工作模式生效，详情请见表 14.3-1。

表 14.3-1. SHA 加速器工作模式

工作模式	配置方式
Typical SHA	SHA_START_REG 置 1
DMA-SHA	SHA_DMA_START_REG 置 1

用户可通过配置 SHA_MODE_REG 寄存器选择 SHA 加速器的哈希算法。详情请见表 14.3-2。对国密标准 SM3 的支持可通过 EFUSE_DIS_SM_CRYPT 禁用。

表 14.3-2. SHA 哈希算法选择

哈希算法	SHA_MODE_REG 配置
SHA-1	0
SHA-224	1
SHA-256	2
SHA-384	3
SHA-512	4
SHA-512/224	5
SHA-512/256	6
SHA-512/*t*	7
SM3	14

注意：

ESP32-S31 的 RSA 数字签名外设 (RSA_DS) 和 HMAC 加速器 (HMAC) 模块在工作时也会调用 SHA 加速器。默认情况下，SHA_FREE（值为 1）将允许 RSA 数字签名外设 (RSA_DS) 和 HMAC 加速器 (HMAC) 模块立即复位当前任何 SHA 运算并使用 SHA 完成其运算。用户应将 SHA_FREE 置 0，以防止上述情况发生。

14.4 功能描述

SHA 加速器可以提取信息摘要 (message digest)，其主要工作流程分为两步：信息预处理和哈希运算。

14.4.1 信息预处理

信息预处理分为三个主要步骤：附加填充比特、信息解析和设置初始哈希值。

14.4.1.1 附加填充比特

根据选择的不同运算标准，SHA 加速器仅能处理长度为 512 位及其整倍数或 1024 位及其整倍数的信息。因此，在将信息送至 SHA 加速器进行运算前，应先通过软件操作将信息填充为符合要求的长度。

假设待处理信息 M 的长度为 m 位，则填充步骤如下：

- **SHA-1、SHA-224、SHA-256 和 SM3**
  1. 首先，在待处理信息后填充 1 个 “1”；
  2. 随后，再填充 k 个 “0”。其中，k 为满足 $m+1+k \equiv 448 \ mod \ 512$ 的最小非负数解；
  3. 最后，在末尾填充一个 64 位的信息块。该信息块的内容为用二进制表示的待处理信息的长度，即 m 的值。
- **SHA-384、SHA-512、SHA-512/224、SHA-512/256 和 SHA-512/t**
  1. 首先，在待处理信息后填充 1 个 “1”；
  2. 随后，再填充 k 个 “0”。其中，k 为满足 $m+1+k \equiv 896 \ mod \ 1024$ 的最小非负数解；
  3. 最后，在末尾填充一个 128 位的信息块。该信息块的内容为用二进制表示的待处理信息的长度，即 m 的值。

更多详情，请参考 《FIPS PUB 180-4 规范》 > 章节 “Padding the Message”。

14.4.1.2 信息解析

在完成信息填充后，还需将待处理信息（及其填充）解析为 N 个 512 位或 1024 位的信息块。SHA 加速器在工作时，所有的信息块都会写入 SHA_M_*n*_REG 中。

- 对于 **SHA-1、SHA-224、SHA-256 和 SM3**：

  待处理信息（及其填充）应解析为 N 个 512 位的信息块，即 $M^{(1)}$、$M^{(2)}$、...、$M^{(N)}$。一个 512 位信息块包括 16 个 32 位的字 (word)，则第一个 32 位字表示为 $M_0^{(i)}$，第二个 32 位字表示为 $M_1^{(i)}$，...，直到第 16 个 32 位字表示为 $M_{15}^{(i)}$。

  SHA 加速器在工作时，$M_0^{(i)}$ 会写入 SHA_M_0_REG 中，$M_1^{(i)}$ 会写入 SHA_M_1_REG，...，$M_{15}^{(i)}$ 会写入 SHA_M_15_REG中。

- 对于 **SHA-384、SHA-512、SHA-512/224、SHA-512/256 和 SHA-512/t**：

  待处理信息（及其填充）应解析为 N 个 1024 位的信息块，即 $M^{(1)}$、$M^{(2)}$、...、$M^{(N)}$。一个 1024 位信息块包括 16 个 64 位的字 (word)，则第 i 个信息块的第一个 64 位字表示为 $M_0^{(i)}$，第二个 64 位字表示为 $M_1^{(i)}$，...，第 16 个 64 位字表示为 $M_{15}^{(i)}$。

  SHA 加速器在工作时，$M_0^{(i)}$ 的高 32 位和低 32 位会分别写入 SHA_M_0_REG 和 SHA_M_1_REG中，...，$M_{15}^{(i)}$ 的高 32 位和低 32 位会分别写入 SHA_M_30_REG 和 SHA_M_31_REG 中。

> **说明：**
> 有关 “信息块” 及相关概念的描述，请参考 《FIPS PUB 180-4 规范》 > 章节 “Glossary of Terms and Acronyms”。

14.4.1.3 设置哈希初始值

在进行哈希运算前，首先必须设置哈希初始值 $H^{(0)}$。不同哈希算法的哈希初始值设置要求不同，其中 SHA-1、SHA-224、SHA-256、SHA-384、SHA-512、SHA-512/224、SHA-512/256 和 SM3 算法使用存储在硬件中的初始哈希值（常量 C）。

然而，SHA-512/t 对于不同的 t 均需要一个不同的哈希初始值。简单来说，SHA-512/t 是一种基于 SHA-512 的 t 位运算标准，其运算结果将截断至 t 位。其中，运算标准对 t 值的要求为 “大于 0 小于 512 且不等于 384 的正

整数”。对于不同 *t* 值的 SHA-512/*t* 运算，其哈希初始值可通过对 “SHA-512/t” 字符串的十六进制表示进行 SHA-512 运算获得。简言之，对于 *t* 取值不同的 SHA-512/*t* 运算标准，其不同之处仅在于 *t* 值不同。

因此，为了简化 SHA-512/*t* 的哈希初始值计算，可使用以下方法：

1. **计算 t_string 和 t_length**：其中，t_string 为 *t* 的字符串信息，长度为 32-bit。t_length 指明字符串长度信息，长度为 7-bit。根据 t 的取值范围不同，t_string 和 t_length 的计算过程如下：

   - 如果 $1 <= t <= 9$，则 $t_length = 7'h48$，t_string 需要按照如下格式封装：

$8'h30 + 8'ht_0$	$1'b1$	$23'b0$

     其中，$t_0 = t$。

     举例，如果 *t* = 8，则 t_0 = 8，t_string = $32'h38800000$。

   - 如果 $10 <= t <= 99$，则 $t_length = 7'h50$，t_string 需要按照如下格式封装：

$8'h30 + 8'ht_1$	$8'h30 + 8'ht_0$	$1'b1$	$15'b0$

     其中，$t_0 = t\%10$，$t_1 = t/10$。

     举例，如果 *t* = 56，则 t_0 = 6，t_1 = 5，t_string = $32'h35368000$。

   - 如果 $100 <= t < 512$，则 $t_length = 7'h58$，t_string 需要按照如下格式封装：

$8'h30 + 8'ht_2$	$8'h30 + 8'ht_1$	$8'h30 + 8'ht_0$	$1'b1$	$7'b0$

     其中，$t_0 = t\%10$，$t_1 = (t/10)\%10$，$t_2 = t/100$。

     举例，如果 *t* = 231，则 t_0 = 1，t_1 = 3，t_2 = 2，t_string = $32'h32333180$。

2. **配置计算哈希初始值所需的寄存器**：用 t_string 和 t_length 初始化文本寄存器 SHA_T_STRING_REG 和 SHA_T_LENGTH_REG。

3. **计算得到哈希初始值**：对 SHA_MODE_REG 寄存器置 7 选择 SHA-512/*t* 运算，并对 SHA_START_REG 寄存器置 1，启动 SHA 加速器的运算即可。最后，轮询寄存器 SHA_BUSY_REG 结果为 0，则哈希初始值已计算完毕。

此外，也可以按照 FIPS PUB 180-4 Spec 中 “5.3.6 SHA-512/*t*” 章节的描述计算 SHA-512/*t* 的哈希初始值，也就是对 “SHA-512/*t*” 字符串的十六进制表示进行一次 “特殊” 的 SHA-512 运算，其运算得到的信息摘要即为所需的哈希初始值。这里的 “特殊” 指本次 SHA-512 运算的哈希初始值为 “SHA-512 运算标准的初始值常量 C 与 0xa5 每 8 位进行一次异或位运算后得到的结果”。

14.4.2 哈希运算流程

在完成信息预处理后，ESP32-S31 SHA 加速器将正式开始哈希运算，最终根据不同运算标准得到不同长度的信息摘要。正如上文所述，ESP32-S31 SHA 加速器支持 Typical SHA 和 DMA-SHA 两种工作模式，下面将对这两种工作模式的具体流程进行介绍。

14.4.2.1 Typical SHA 模式下的运算流程

通常情况下，ESP32-S31 的 SHA 会先处理完当前信息的所有信息块并生成该信息的信息摘要，之后再开始计算新的信息摘要。

不过，ESP32-S31 SHA 加速器还支持插入运算 (interleaved)（仅 Typical SHA 支持），即在完成当前信息的所有运算前，用户在每个信息块计算完成后均可插入新的运算。

具体来说，用户可以将存储在 SHA_H_*n*_REG 寄存器中的信息摘要暂时保存到其他地方，然后让 SHA 加速器来完成其他优先级更高的运算任务。当插入的运算结束后，用户再将之前暂存的信息摘要重新写入 SHA_H_*n*_REG 中，并继续完成之前中断的计算。

Typical SHA 的具体运算流程（SHA-512/*t* 除外）

1. 选择运算标准。
   - 根据表 14.3-2 配置 SHA_MODE_REG 寄存器，设置运算标准。
2. 处理当前信息块。
   - 将当前信息块写入 SHA_M_*n*_REG 寄存器。
3. 启动 SHA 加速器 [1]。
   - 如果为首次运算，则对 SHA_START_REG 寄存器置 1，启动 SHA 加速器的运算。此时，SHA 加速器按照步骤 1 中选定的运算标准，使用硬件中固定的哈希初始值进行运算；
   - 如果非首次运算[2]，则对 SHA_CONTINUE_REG 寄存器置 1，启动 SHA 加速器的运算。此时，SHA 加速器使用 SHA_H_*n*_REG 寄存器中的值作为哈希初始值进行运算。
4. 查询当前信息块的处理进度。
   - 轮询寄存器 SHA_BUSY_REG 一直到读回的值为 0，代表 SHA 硬件加速器已完成对当前信息块的计算，进入“空闲”状态[3]。
5. 选择是否有后续的待处理信息块。
   - 如果存在后续待处理信息块，则跳回执行步骤 2。
   - 否则，继续执行。
6. 获取信息摘要：
   - 从寄存器堆 SHA_H_*n*_REG 取出信息摘要。

Typical SHA 的具体运算流程（SHA-512/*t*）

1. 选择运算标准。
   - 配置 SHA_MODE_REG 寄存器为 7 选择 SHA-512/*t* 运算标准。
2. 计算哈希初始值。
   (a) 配置 t_string 和 t_length，并将其写入 SHA_T_STRING_REG 和 SHA_T_LENGTH_REG 寄存器。具体请见 14.4.1.3 章节。
   (b) 对 SHA_START_REG 寄存器置 1，启动 SHA 加速器的运算。
   (c) 轮询寄存器 SHA_BUSY_REG 一直到读回的值为 0，代表 SHA 硬件加速器已完成哈希初始值的计算。
3. 处理当前信息块[1]。
   - 将当前信息块写入 SHA_M_*n*_REG 寄存器。
4. 启动 SHA 加速器。

- 对 SHA_CONTINUE_REG 寄存器置 1，启动 SHA 加速器的运算。此时，SHA 加速器使用 SHA_H_*n*_REG 寄存器中的值作为哈希初始值进行运算。

5. 查询当前信息块的处理进度。
   - 轮询寄存器 SHA_BUSY_REG 一直到读回的值为 0，代表 SHA 硬件加速器已完成对当前信息块的计算，进入“空闲”状态[3]。
6. 选择是否有后续的待处理信息块。
   - 如果存在后续待处理信息块，则跳回执行步骤 3。
   - 否则，继续执行。
7. 获取信息摘要：
   - 从寄存器堆 SHA_H_*n*_REG 读取信息摘要。

说明：

1. 这里，在 SHA 加速器进行硬件运算时，如果存在后续待处理信息块，软件还可以同时将后续信息块写入 SHA_M_*n*_REG 寄存器，以节省时间。
2. 这里，用户可以选择是否需要插入其他任务。如需插入，可查看下文 插入任务工作流程。

如上文所述，ESP32-S31 SHA 加速器**支持在 Typical SHA 模式下插入任务**。插入任务工作流程具体如下：

1. 保存插入前任务的以下数据，准备将 SHA 加速器的使用权移交给插入的任务。
   - 读取并保存寄存器 SHA_MODE_REG 中的运算标准类型。
   - 读取并保存寄存器堆 SHA_H_*n*_REG 中的信息摘要。
2. 执行插入的任务。具体按照插入运算类型的不同，请见 Typical SHA 或 DMA-SHA 工作流程。
3. 恢复插入前任务的以下数据，准备将 SHA 加速器的使用权交还给插入前的任务。
   - 将获得使用权前保存的运算标准类型重新写入寄存器 SHA_MODE_REG;
   - 将获得使用权前保存的信息摘要写入寄存器堆 SHA_H_*n*_REG。
4. 将之前任务的下一个待处理信息块写入 SHA_M_*n*_REG 寄存器，并对 SHA_CONTINUE_REG 寄存器置 1，重新启动 SHA 加速器，完成之前暂停的任务。

14.4.2.2 DMA-SHA 模式下的运算流程

ESP32-S31 SHA 加速器在 DMA-SHA 工作模式下不支持插入 (interleaved) 运算，即用户必须在每次 DMA 运算（可能包括 1 个或多个信息块）全部结束后才能插入新的运算。这种情况下，用户如有插入运算需求，可将较大信息块进行拆分，并进行多次 DMA 运算。每次 DMA 运算之间允许插入其他运算标准的计算任务。

单次 DMA-SHA 运算最多可处理 63 个信息块。

与 Typical SHA 工作模式不同，SHA 加速器在 DMA-SHA 模式下，所有数据读取均通过 GDMA-AXI 完成。因此，用户需按照章节 3 *通用 DMA 控制器 (GDMA-AHB, GDMA-AXI)* 中的说明配置 GDMA-AXI 控制器。

DMA-SHA 的具体工作流程（SHA-512/*t* 除外）

1. 选择运算标准。

- 根据表 14.3-2 配置 SHA_MODE_REG 寄存器，设置运算标准。

2. 选择是否启用中断。可配置 SHA_INT_ENA_REG 寄存器以启用或禁用中断（置 1 以启用）。
3. 配置块个数。
   - 将待加密数据的总块数 *M* 写入 SHA_DMA_BLOCK_NUM_REG 寄存器。
4. 开始 DMA-SHA 运算。
   - 如果当前 DMA-SHA 运算为接着另一次 DMA-SHA 的运算，需要提前将另一次计算得到的信息摘要写入寄存器堆 SHA_H_*n*_REG 中，随后将 1 写入寄存器 SHA_DMA_CONTINUE_REG；
   - 否则，只需要将 1 写入寄存器 SHA_DMA_START_REG。
5. 等待 DMA-SHA 运算结束。判断 DMA-SHA 运算结束有以下两种方法：
   - 轮询寄存器 SHA_BUSY_REG 结果为 0。
   - 等待中断信号产生。此时，应及时通过软件将 SHA_INT_CLEAR_REG 寄存器置为 1 以清除中断。
6. 获取信息摘要
   - 从寄存器堆 SHA_H_*n*_REG 取出信息摘要。

DMA-SHA 的具体工作流程（SHA-512/*t*）

1. 选择运算标准。
   - 配置 SHA_MODE_REG 寄存器为 7 选择 SHA-512/*t* 运算标准。
2. 选择是否启用中断。可配置 SHA_INT_ENA_REG 寄存器以启用或禁用中断（置 1 以启用）。
3. 计算哈希初始值。
   (a) 配置 t_string 和 t_length，并将其写入 SHA_T_STRING_REG 和 SHA_T_LENGTH_REG 寄存器。具体请见 14.4.1.3 章节。
   (b) 对 SHA_START_REG 寄存器置 1，启动 SHA 加速器的运算。
   (c) 轮询寄存器 SHA_BUSY_REG 一直到读回的值为 0，代表 SHA 硬件加速器已完成哈希初始值的计算。
4. 配置块个数。
   - 将待加密数据的总块数 M 写入 SHA_DMA_BLOCK_NUM_REG 寄存器。
5. 开始 DMA-SHA 运算。
   - 对 SHA_DMA_CONTINUE_REG 置 1 启动 SHA 加速器。
6. 等待 DMA-SHA 运算结束。判断 DMA-SHA 运算结束有以下两种方法：
   - 轮询寄存器 SHA_BUSY_REG 结果为 0。
   - 等待中断信号产生。此时，应及时通过软件将 SHA_INT_CLEAR_REG 寄存器置为 1 以清除中断。
7. 获取信息摘要
   - 从寄存器堆 SHA_H_*n*_REG 取出信息摘要。

14.4.3 信息摘要存储

哈希运算完成之后，计算得到的信息摘要被 SHA 加速器更新至对应的 SHA_H_*n*_REG (*n*: 0 ~ 15) 寄存器中。不同运算标准得到的信息摘要长度也不同，详情见表 14.4-4：

表 14.4-4. 不同运算标准信息摘要的寄存器占用情况

哈希运算标准	信息摘要长度（位）	寄存器占用情况[1]
SHA-1	160	SHA_H_0_REG ~ SHA_H_4_REG
SHA-224	224	SHA_H_0_REG ~ SHA_H_6_REG
SHA-256	256	SHA_H_0_REG ~ SHA_H_7_REG
SHA-384	384	SHA_H_0_REG ~ SHA_H_11_REG
SHA-512	512	SHA_H_0_REG ~ SHA_H_15_REG
SHA-512/224	224	SHA_H_0_REG ~ SHA_H_6_REG
SHA-512/256	256	SHA_H_0_REG ~ SHA_H_7_REG
SHA-512/*t*[2]	*t*	SHA_H_0_REG ~ SHA_H_*x*_REG
SM3	256	SHA_H_0_REG ~ SHA_H_7_REG

[1] 信息摘要从最高有效位至最低有效位存放，第一个 word 存放在寄存器 SHA_H_0_REG 中，第二个 word 存放在寄存器 SHA_H_1_REG 中，以此类推，具体可参考 14.4.1.2。

[2] SHA-512/*t* 运算标准使用的寄存器与 *t* 的取值有关。*x*+1 代表用于存储 *t* 位信息摘要的 32 位寄存器个数，因此 *x* = $roundup(t/32)$-1。举例：

- 当 *t* = 8 时，则 *x* = 0，代表最终的信息摘要长度为 8 位，存放在寄存器 SHA_H_0_REG 的高 8 位中；
- 当 *t* = 32 时，则 *x* = 0，代表最终的信息摘要长度为 32 位，存放在寄存器 SHA_H_0_REG 中；
- 当 *t* = 132 时，则 *x* = 4，代表最终的信息摘要长度为 132 位，存放在寄存器 SHA_H_0_REG、SHA_H_1_REG、SHA_H_2_REG、SHA_H_3_REG，及 SHA_H_4_REG 中。

14.5 中断

ESP32-S31 的 SHA 加速器可以生成以下中断信号并发送给中断矩阵。

- SHA_INTR

这些中断信号由 SHA 加速器的内部中断源生成。表 14.5-1 列出了 SHA 加速器的内部中断源、中断触发条件、以及生成的中断信号。

表 14.5-1. SHA 加速器的内部中断源

内部中断源	触发条件	中断信号
SHA_CALC_DONE_INT	DMA-SHA 模式下完成信息摘要的计算	SHA_INTR

说明：

- 中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。
- 与标准中断寄存器组不同，SHA 的中断寄存器组仅包含 INT_ENA（SHA_INT_ENA_REG）和 INT_CLR（SHA_INT_CLEAR_REG）字段，并不支持用户读取 INT_RAW 和 INT_ST 字段。

14.6 寄存器列表

本小节的所有地址均为相对于 SHA 加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	权限
控制与配置寄存器			
SHA_MODE_REG	配置 SHA 加速器的运算标准	0x0000	R/W
SHA_T_STRING_REG	t_string，用于计算 SHA-512/t 的哈希初始值	0x0004	R/W
SHA_T_LENGTH_REG	t_length，用于计算 SHA-512/t 的哈希初始值	0x0008	R/W
SHA_CONTINUE_REG	继续 SHA 运算（仅用于 Typical SHA 模式）	0x0014	WO
SHA_DMA_START_REG	启动 SHA 加速器的 DMA-SHA 模式	0x001C	WO
SHA_START_REG	启动 SHA 加速器的 Typical SHA 模式	0x0010	WO
SHA_DMA_CONTINUE_REG	继续 SHA 运算（仅用于 DMA-SHA 模式）	0x0020	WO
SHA_DMA_BLOCK_NUM_REG	信息块个数寄存器（仅用于 DMA-SHA 模式）	0x000C	R/W
SHA_DMA_RX_RESET_REG	DMA RX 复位信号	0x0030	WO
SHA_DMA_TX_RESET_REG	DMA TX 复位信号	0x0034	WO
状态寄存器			
SHA_BUSY_REG	表示 SHA 加速器是否处于“忙碌”状态	0x0018	RO
SHA_FREE_REG	SHA 释放寄存器	0x0038	R/W
中断寄存器			
SHA_INT_CLEAR_REG	DMA-SHA 中断清除寄存器	0x0024	WO
SHA_INT_ENA_REG	DMA-SHA 中断使能寄存器	0x0028	R/W
数据寄存器			
SHA_H_0_REG	哈希值	0x0040	R/W
SHA_H_1_REG	哈希值	0x0044	R/W
SHA_H_2_REG	哈希值	0x0048	R/W
SHA_H_3_REG	哈希值	0x004C	R/W
SHA_H_4_REG	哈希值	0x0050	R/W
SHA_H_5_REG	哈希值	0x0054	R/W
SHA_H_6_REG	哈希值	0x0058	R/W
SHA_H_7_REG	哈希值	0x005C	R/W
SHA_H_8_REG	哈希值	0x0060	R/W
SHA_H_9_REG	哈希值	0x0064	R/W
SHA_H_10_REG	哈希值	0x0068	R/W
SHA_H_11_REG	哈希值	0x006C	R/W
SHA_H_12_REG	哈希值	0x0070	R/W
SHA_H_13_REG	哈希值	0x0074	R/W
SHA_H_14_REG	哈希值	0x0078	R/W
SHA_H_15_REG	哈希值	0x007C	R/W
SHA_M_0_REG	输入信息	0x0080	R/W
SHA_M_1_REG	输入信息	0x0084	R/W
SHA_M_2_REG	输入信息	0x0088	R/W

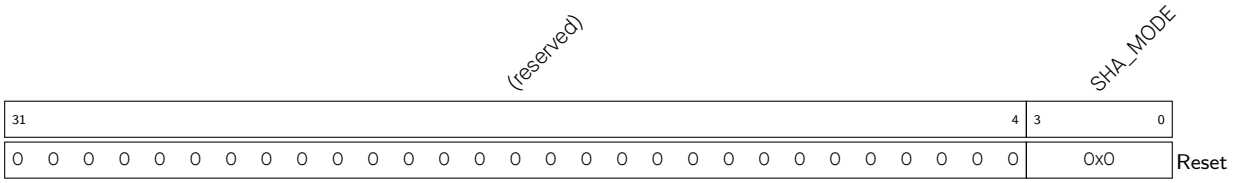
名称	描述	地址	权限
SHA_M_3_REG	输入信息	0x008C	R/W
SHA_M_4_REG	输入信息	0x0090	R/W
SHA_M_5_REG	输入信息	0x0094	R/W
SHA_M_6_REG	输入信息	0x0098	R/W
SHA_M_7_REG	输入信息	0x009C	R/W
SHA_M_8_REG	输入信息	0x00A0	R/W
SHA_M_9_REG	输入信息	0x00A4	R/W
SHA_M_10_REG	输入信息	0x00A8	R/W
SHA_M_11_REG	输入信息	0x00AC	R/W
SHA_M_12_REG	输入信息	0x00B0	R/W
SHA_M_13_REG	输入信息	0x00B4	R/W
SHA_M_14_REG	输入信息	0x00B8	R/W
SHA_M_15_REG	输入信息	0x00BC	R/W
SHA_M_16_REG	输入信息	0x00C0	R/W
SHA_M_17_REG	输入信息	0x00C4	R/W
SHA_M_18_REG	输入信息	0x00C8	R/W
SHA_M_19_REG	输入信息	0x00CC	R/W
SHA_M_20_REG	输入信息	0x00D0	R/W
SHA_M_21_REG	输入信息	0x00D4	R/W
SHA_M_22_REG	输入信息	0x00D8	R/W
SHA_M_23_REG	输入信息	0x00DC	R/W
SHA_M_24_REG	输入信息	0x00E0	R/W
SHA_M_25_REG	输入信息	0x00E4	R/W
SHA_M_26_REG	输入信息	0x00E8	R/W
SHA_M_27_REG	输入信息	0x00EC	R/W
SHA_M_28_REG	输入信息	0x00F0	R/W
SHA_M_29_REG	输入信息	0x00F4	R/W
SHA_M_30_REG	输入信息	0x00F8	R/W
SHA_M_31_REG	输入信息	0x00FC	R/W
版本寄存器			
SHA_DATE_REG	版本控制寄存器	0x002C	R/W

14.7 寄存器

本小节的所有地址均为相对于 SHA 加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

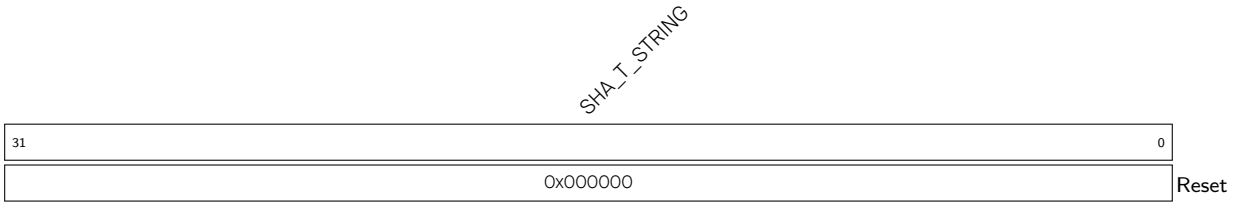
Register 14.1. SHA_MODE_REG (0x0000)



SHA_MODE 配置 SHA 加速器的运算标准。

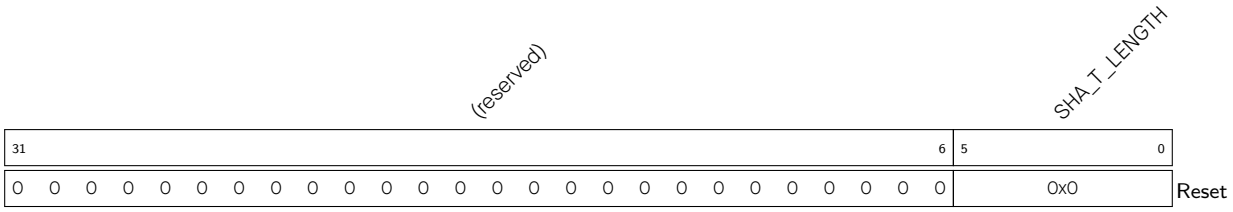
0：SHA-1
1：SHA-224
2：SHA-256
3：SHA-384
4：SHA-512
5：SHA-512/224
6：SHA-512/256
7：SHA-512/t
8-13：保留
14：SM3
15：保留
(R/W)

Register 14.2. SHA_T_STRING_REG (0x0004)



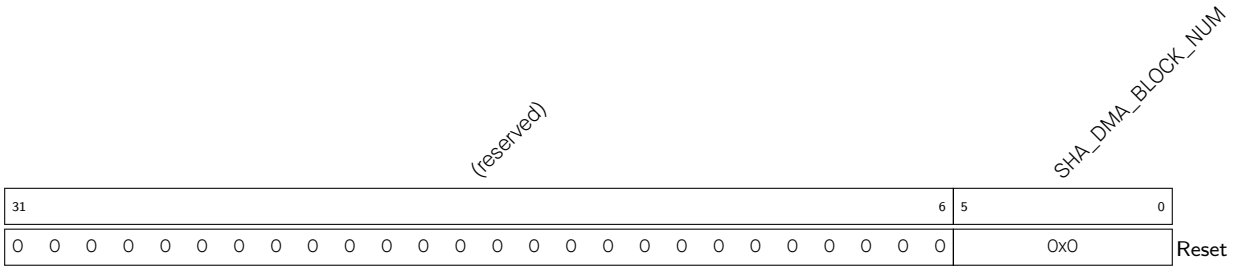
SHA_T_STRING 表示哈希字符串内容（仅用于计算 SHA-512/t 的哈希初始值）。(R/W)

Register 14.3. SHA_T_LENGTH_REG (0x0008)



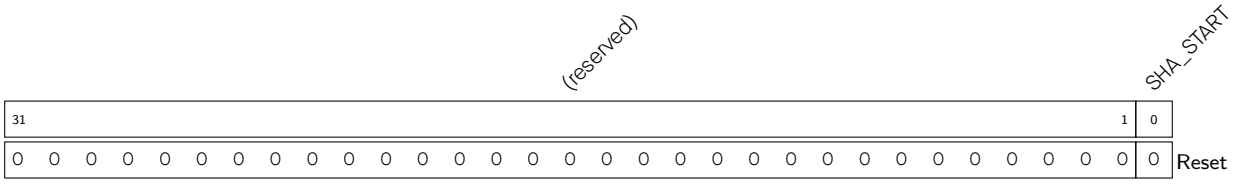
SHA_T_LENGTH 表示哈希字符串长度（仅用于计算 SHA-512/t 的哈希初始值）。(R/W)

Register 14.4. SHA_DMA_BLOCK_NUM_REG (0x000C)



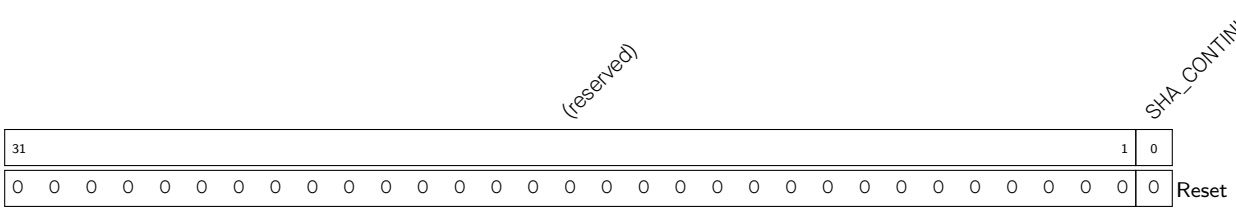
SHA_DMA_BLOCK_NUM 配置 DMA-SHA 工作模式下的信息块个数。(R/W)

Register 14.5. SHA_START_REG (0x0010)



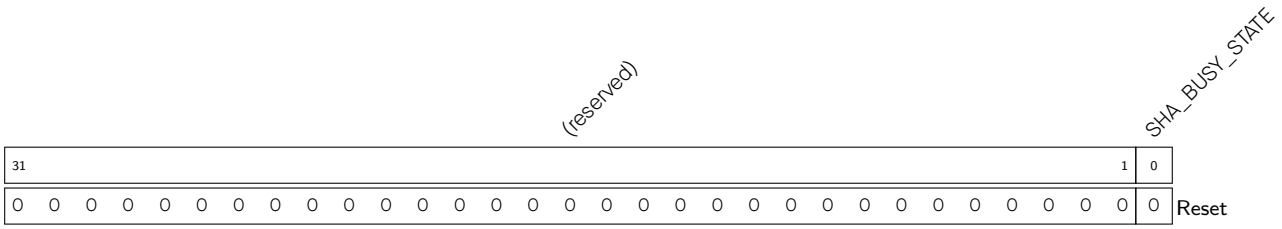
SHA_START 写 1 启动 SHA 加速器的 Typical SHA 模式。(WO)

Register 14.6. SHA_CONTINUE_REG (0x0014)



SHA_CONTINUE 写 1 继续 SHA 加速器的 Typical SHA 运算。(WO)

Register 14.7. SHA_BUSY_REG (0x0018)



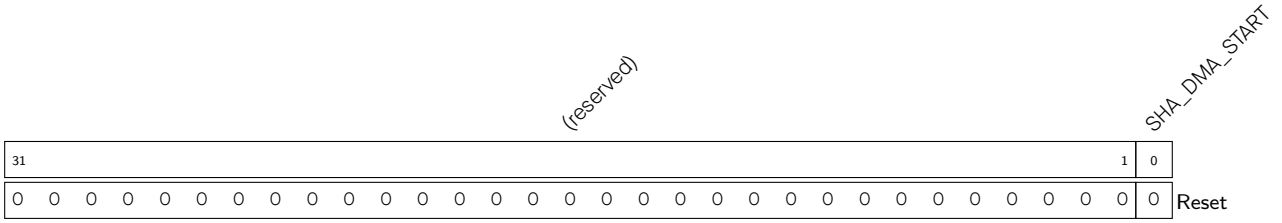
SHA_BUSY_STATE 表示 SHA 是否处于“忙碌”状态。

0：空闲

1：忙碌

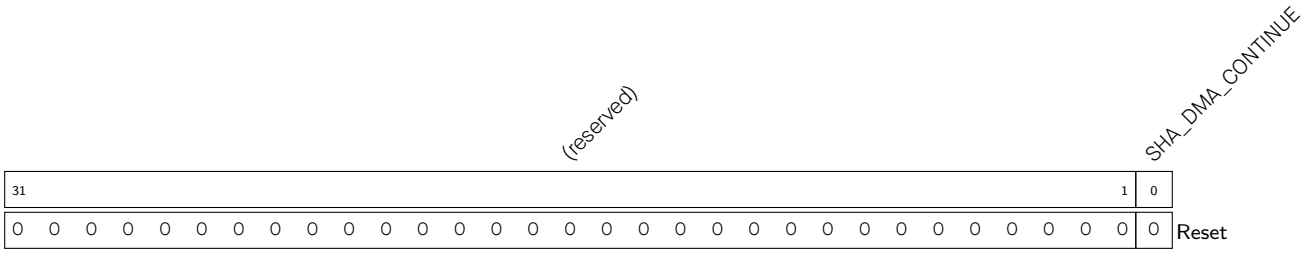
(RO)

Register 14.8. SHA_DMA_START_REG (0x001C)



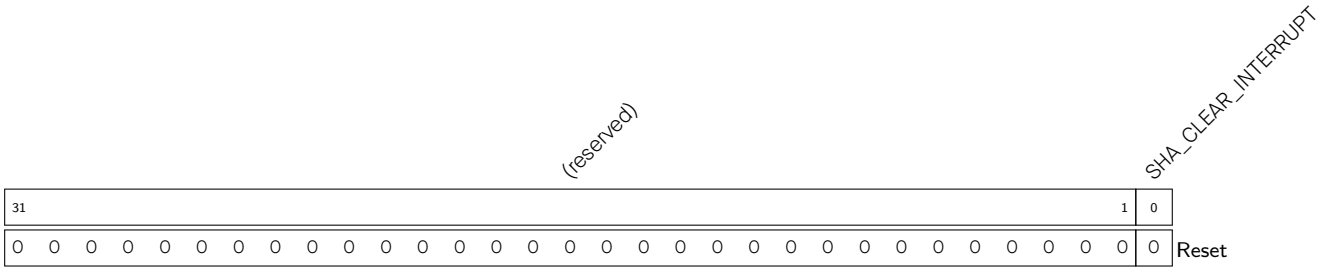
SHA_DMA_START 写 1 启动 SHA 加速器的 DMA-SHA 模式。(WO)

Register 14.9. SHA_DMA_CONTINUE_REG (0x0020)



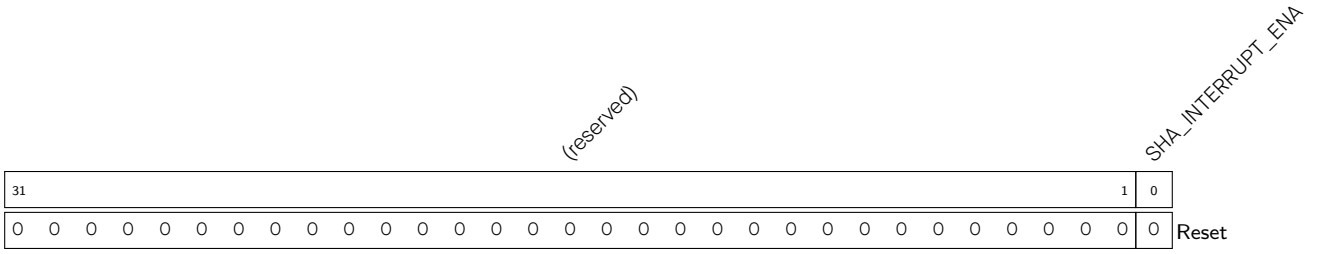
SHA_DMA_CONTINUE 写 1 继续 SHA 加速器的 DMA-SHA 运算。(WO)

Register 14.10. SHA_INT_CLEAR_REG (0x0024)



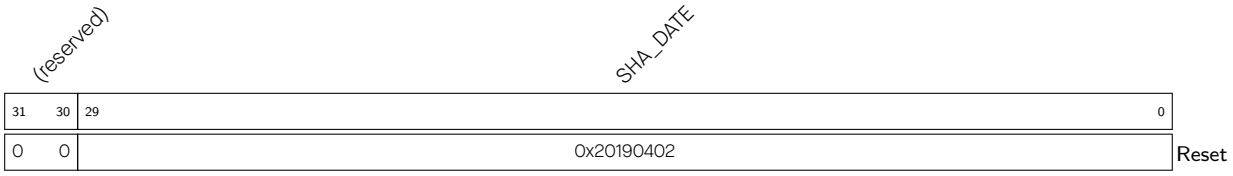
SHA_CLEAR_INTERRUPT 写 1 清除 DMA-SHA 中断。(WO)

Register 14.11. SHA_INT_ENA_REG (0x0028)



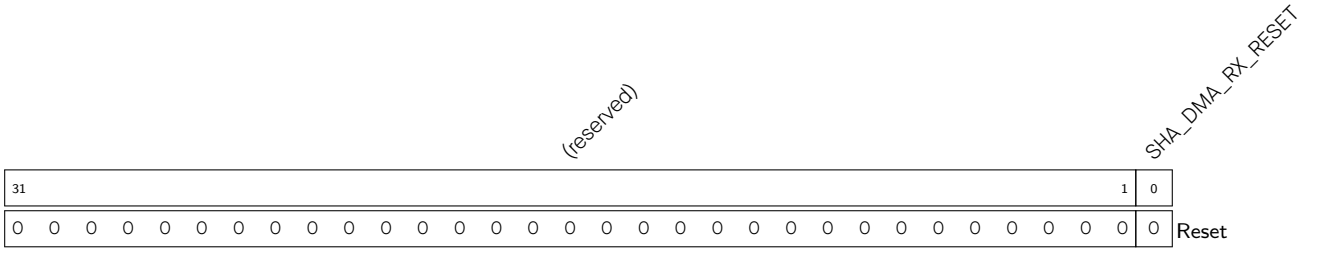
SHA_INTERRUPT_ENA 写 1 使能 DMA-SHA 中断。(R/W)

Register 14.12. SHA_DATE_REG (0x002C)



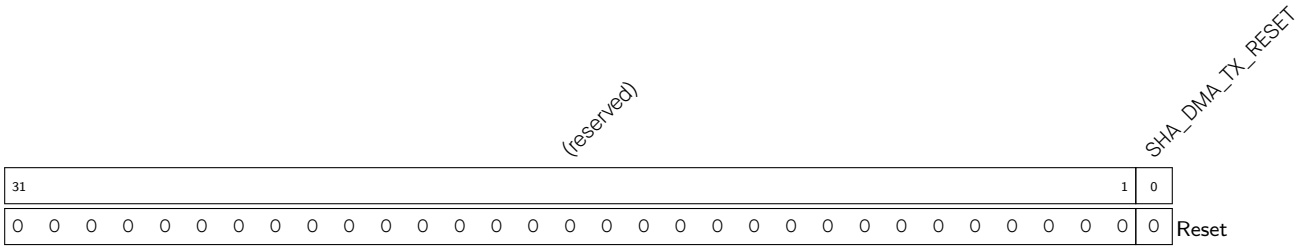
SHA_DATE 版本控制寄存器。(R/W)

Register 14.13. SHA_DMA_RX_RESET_REG (0x0030)



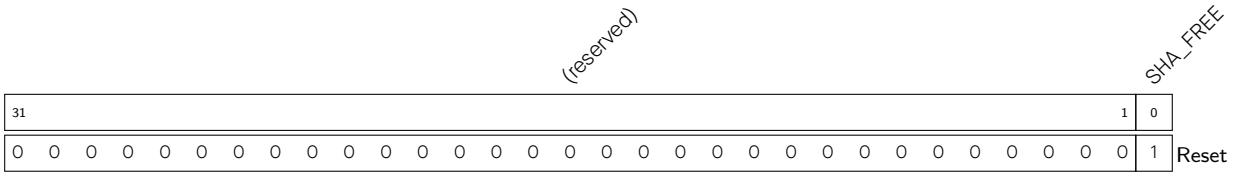
SHA_DMA_RX_RESET 写 1 复位 DMA-SHA RX FIFO。(WO)

Register 14.14. SHA_DMA_TX_RESET_REG (0x0034)



SHA_DMA_TX_RESET 写 1 复位 DMA-SHA TX FIFO。(WO)

Register 14.15. SHA_FREE_REG (0x0038)



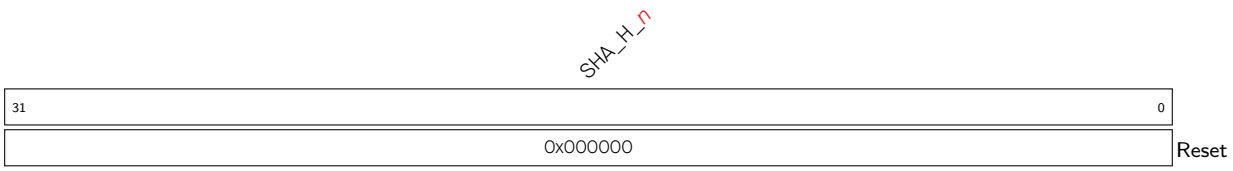
SHA_FREE 配置 SHA 是否可被释放（即使 SHA 正在运算），以供其他需要 SHA 的模块占用。

0：SHA 不可被释放

1：SHA 可被释放（默认值）

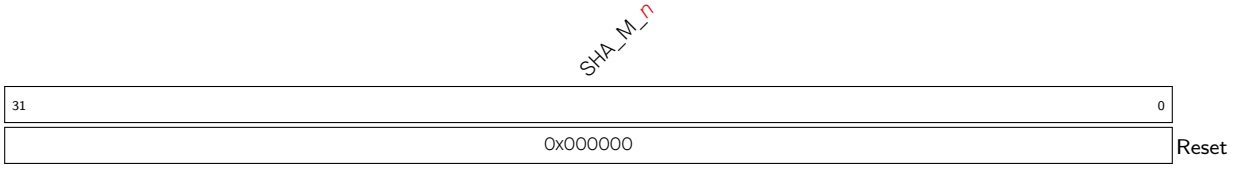
(R/W)

Register 14.16. SHA_H_*n*_REG (*n*: 0-15) (0x0040+4**n*)



SHA_H_*n* 表示第 *n* 个 32 位哈希值。(R/W)

Register 14.17. SHA_M_*n*_REG (*n*: 0-31) (0x0080+4**n*)



SHA_M_*n* 表示第 *n* 个 32 位输入信息。(R/W)

第 15 章

RSA 数字签名外设 (RSA_DS)

15.1 概述

RSA 数字签名外设 (RSA_DS) 使用密码学算法，用于验证消息的真实性和完整性。该算法也可用于比如向服务器验证设备身份。

ESP32-S31 包含一个 RSA 数字签名外设 (RSA_DS)，可提供硬件加速，完成基于 RSA 的数字签名操作。HMAC 外设作为密钥导出函数 (Key Derivation Function, KDF)，使用存储在 eFuse 或密钥管理器生成的密钥作为输入密钥，输出 RSA_DS_KEY。随后，RSA_DS 外设使用 RSA_DS_KEY 解密预先加密的参数，计算出签名。上述过程都发生在硬件层面，因此，在计算过程中，不论是解密 RSA 参数的密钥，还是用于 HMAC 密钥导出函数的输入密钥，都对用户不可见。

15.2 主要特性

ESP32-S31 的 RSA_DS 外设具有以下特性：

- 硬件实现 RSA 签名运算 $Z = X^Y \bmod M$，私钥相关运算子最大 4096 位
- 私钥参数以密文存放在 flash 中，在 RSA_DS 外设内以 RSA_DS_KEY 解密，软件不可见明文
- 对私钥数据及相关参数使用 SHA-256 计算摘要，用于防篡改检测
- RSA_DS_KEY 由硬件提供（对用户不可读），来源为基于 eFuse 中 $HMAC_KEY$ 的 HMAC-SHA256 导出，或由密钥管理器部署

15.3 功能描述

15.3.1 概述

RSA_DS 外设计算 RSA 签名的方式为 $Z = X^Y \bmod M$，其中 Z 是签名，X 是输入消息，Y 和 M 是 RSA 私钥参数。

私钥参数以密文形式存储在 flash 中。用于解密私钥参数的密钥 (RSA_DS_KEY) 可以来自于 HMAC 模块或密钥管理器模块。当该密钥：

- 由 RSA_DS 外设通过 HMAC 模块计算获取时，生成密钥所需的 HMAC 输入 $HMAC_KEY$ 存放在 eFuse 中，只允许被 HMAC 模块访问。也就是说，只有 RSA_DS 硬件才能解密私钥密文，软件无法获取私钥明文。关于 eFuse 和 HMAC 的相关细节请参照章节 *4 eFuse 控制器 (eFuse) [to be added later]* 和章节 *12 HMAC 加速器 (HMAC)*。
- 由 RSA_DS 外设通过密钥管理器模块获取时，该密钥必须以安全的方式由密钥管理器部署而来。

需要签名时，软件直接将输入消息 X 发送到 RSA_DS 外设。RSA 签名计算完成后，软件将读取签名结果 Z。

为方便描述，此处约定几个符号，它们的作用域仅限于本章。

- $\mathbf{1}^s$: 表示一个完全由 “1” 组成的长度为 s 位的位串。
- $[x]_s$: 一个长度为 s 位的位串，要求 s 为 8 的整数倍。如果 x 是一个数 ($x < 2^s$)，那么其在位串中遵循小端字节序。x 可以是一个变量，例如 $[Y]_{4096}$，或一个十六进制的常数，比如 $[0xOC]_8$。根据需要，$[x]_t$ 右边可以加上$(s-t)$ 个 0，使字串长度扩展成 s 位，得到 $[x]_s$。例如：$[0x05]_8 = 00000101$，$[0x05]_{16} = 0000010100000000$，$[0x0005]_{16} = 0000000000000101$，$[0x13]_8 = 00010011$，$[0x13]_{16} = 0001001100000000$，$[0x0013]_{16} = 0000000000010011$。
- ||: 表示位串粘接操作符，用于将两个位串前后粘成一个较长的位串。

15.3.2 私钥运算子

私钥运算子 Y（私钥指数）和 M（密钥模数）由用户生成。它们具有特定的 RSA 密钥长度（最大为 4096 位）。RSA 签名操作还额外需要两个运算子，即参数 $\overline{r}$ 和 M'，由 Y 和 M 运算得到。

用户加密运算子 Y、M、$\overline{r}$、M' 与验证摘要，并以密文 C 的形式存储。密文 C 输入到 RSA_DS 外设之后，先由硬件解密，然后参与 RSA 签名运算。有关生成 C 的方法，请参考章节 15.3.3。

RSA_DS 外设支持运算子长度为 $N = 32 \times x$ ($x \in \{1, 2, 3, \ldots, 128\}$) 的 RSA 签名运算 $Z = X^Y \bmod M$。其中，参数的位宽需要满足 RSA 计算的运算子长度要求，即 Z、X、Y、M 和 $\overline{r}$ 的位宽必须相同，且为 N 中的任意值，而 M' 的位宽则始终是 32。更多 RSA 计算相关信息，请参考章节 13 *RSA 加速器 (RSA)* 中的 13.3.2 大数模幂运算 部分。

15.3.3 软件准备工作

如果用户想使用 RSA_DS 外设进行数字签名，则需要提前做好软件准备工作，如图 15.3-1 所示。图中左半部分列出了在硬件开始 RSA 签名计算之前软件需要做的准备工作，右半部分则列出了硬件在整个签名计算过程中的计算流程。

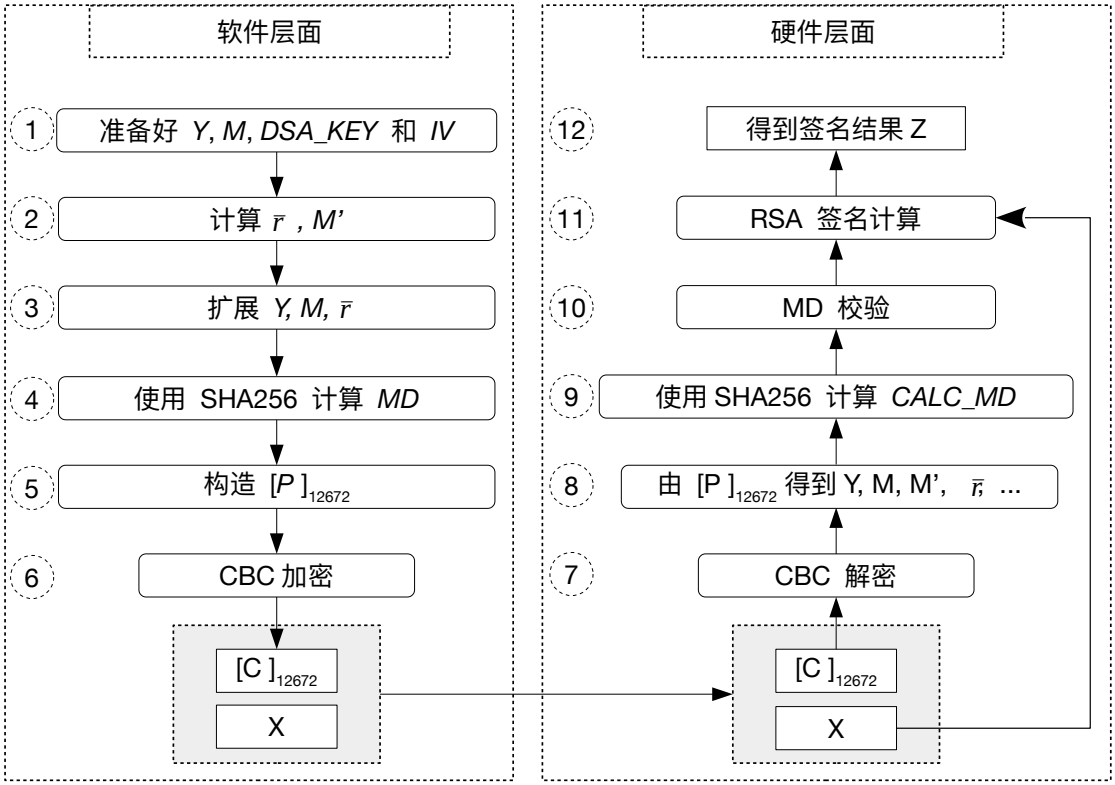


图 15.3-1. 软件准备工作与硬件工作流程

说明：

1. 即便有多次签名计算，也只需进行一次软件准备工作（图 15.3-1 左侧），但每一次签名计算都需要重新进行硬件运算（图 15.3-1 右侧）。
2. 软件准备需要配置时钟复位，相关信息请参考章节 7 复位和时钟 *[to be added later]*。

用户需要依照图 15.3-1 左侧中指定的步骤来计算 C。详细过程如下：

- **步骤 1**：按照章节 15.3.2 所述，准备好符合运算子长度要求的 RSA 私钥运算子 Y 和 M。定义 $[L]_{32} = \frac{N}{32} - 1$（比如，对于 RSA 4096，$[L]_{32}$ == [0x80-1]$_{32}$）。根据私钥的生成方法准备 $[RSA_DS_KEY]_{256}$：
  - 当使用 HMAC 作为密钥来源时，准备好 $[HMAC_KEY]_{256}$，并计算出 $[RSA_DS_KEY]_{256}$，即 RSA_DS_KEY = HMAC-SHA256($[HMAC_KEY]_{256}, 1^{256}$)。
  - 当使用密钥管理器作为密钥来源时，准备好 256 位的 RSA_DS_KEY。

  接着，生成一个符合 AES-CBC 块加密算法要求的随机 $[IV]_{128}$。有关 AES 的更多信息，请参考章节 10 *AES 加速器 (AES)*。
- **步骤 2**：根据 M 求解 $\bar{r}$ 和 M'。
- **步骤 3**：扩展 Y、M 和 $\bar{r}$，得到 $[Y]_{4096}$、$[M]_{4096}$ 和 $[\bar{r}]_{4096}$。此步骤是由于 Y、M 和 $\bar{r}$ 的最大位宽为 4096，故位宽小于 4096 的运算子需要扩展至 4096，而位宽等于 4096 的则不需要扩展。
- **步骤 4**：使用 SHA-256 计算 MD 校验码：$[MD]_{256}$ = SHA256 ($[Y]_{4096}||[M]_{4096}||[\bar{r}]_{4096}||[M']_{32}||[L]_{32}||[IV]_{128}$)

- **步骤 5**：构造 $[P]_{12672}$ = ($[Y]_{4096}||[M]_{4096}||[\overline{r}]_{4096}||[Box]_{384}$)。其中 $[Box]_{384}$ = ($[MD]_{256}||[M']_{32}||[L]_{32}||[\beta]_{64}$)，$[\beta]_{64}$ 为 PKCS#7 填充值，即由 8 个字节（0x08）组成的 $[0x0808080808080808]_{64}$ 字符串。$[\beta]_{64}$ 的作用是使 P 的位长度为 128 的整数倍。
- **步骤 6**：计算密文形式的私钥参数 C = $[C]_{12672}$ = AES-CBC-ENC ($[P]_{12672}$, $[RSA_DS_KEY]_{256}$, $[IV]_{128}$)，其中 C 为密文，长度为 12672 位。C 也可以表示为 C = $[C]_{12672}$ = ($[\widehat{Y}]_{4096}||[\widehat{M}]_{4096}||[\widehat{\overline{r}}]_{4096}||[\widehat{Box}]_{384}$)。其中，$[\widehat{Y}]_{4096}$、$[\widehat{M}]_{4096}$、$[\widehat{\overline{r}}]_{4096}$、$[\widehat{Box}]_{384}$ 是 C 的四个子参数，分别对应 $[Y]_{4096}$、$[M]_{4096}$、$[\overline{r}]_{4096}$、$[Box]_{384}$ 的密文形式。

15.3.4 硬件工作流程

每次需要计算 RSA 数字签名时，都会触发硬件操作。硬件需要三个输入信息：预先生成的私钥密文 C、唯一消息 X 以及 IV。

RSA_DS 外设的工作流程可以分为三个阶段：

1. **解析阶段，即图 15.3-1 中的步骤 7 和步骤 8**

   解析过程是图 15.3-1 中步骤 6 的逆过程。RSA_DS 外设将调用 AES 硬件加速器以 CBC 块模式对输入的密文信息 C 进行解密，获取明文信息。该过程可以表示为 P = AES-CBC-DEC (C, RSA_DS_KEY, IV)。其中，IV 就是 $[IV]_{128}$，由用户直接指定；$[RSA_DS_KEY]_{256}$ 由：

   - 硬件 HMAC 提供，通过存储在 eFuse 中的 $HMAC_KEY$ 得到，软件无法获取。
   - 硬件密钥管理器提供时，通过部署在密钥管理器中的 RSA_DS_KEY 得到，软件无法获取。

   通过 P，RSA_DS 外设能够解析出 $[Y]_{4096}$、$[M]_{4096}$、$[\overline{r}]_{4096}$、$[M']_{32}$、$[L]_{32}$、MD 校验码和追加码 $[\beta]_{64}$，这相当于步骤 5 的逆过程。

2. **校验阶段，即图 15.3-1 中的步骤 9 和步骤 10**

   RSA_DS 外设会执行两种校验操作：MD 校验和填充 (padding) 校验。由于填充校验和 MD 校验同步进行，因此填充校验未在图 15.3-1 中体现。

   - MD 校验：RSA_DS 外设调用 SHA-256 进行哈希计算获取哈希值 $[CALC_MD]_{256}$（计算方式与参数同 $[MD]_{256}$，参见步骤 4）。然后，将 $[CALC_MD]_{256}$ 与 MD 校验码 $[MD]_{256}$ 作比较，当且仅当二者相同时，MD 校验通过。
   - 填充校验：RSA_DS 外设检查解析阶段解析出的追加码 $[\beta]_{64}$ 是否符合 PKCS#7 标准，当且仅当符合标准时，填充校验通过。

   仅当 MD 校验通过后，RSA_DS 外设才会执行后续计算，否则 RSA_DS 外设将拒绝执行。如果填充校验失败，将生成警告信息，但不会影响 RSA_DS 外设的后续操作。

3. **计算阶段，即图 15.3-1 中的步骤 11 和步骤 12**

   RSA_DS 外设将把用户输入的 X，以及解析得到的 Y、M 和 $\overline{r}$ 都视为大数，结合解析得到的 M'，构成了大数模幂运算 X^Y mod M 的所有必要输入参数。大数模幂运算的运算长度由 L 的值唯一指定。RSA_DS 外设调用 RSA 完成大数模幂运算 $Z = X^Y$ mod M，Z 为签名结果。

15.3.4.1 硬件占用

在运算过程中，ESP32-S31 的 RSA_DS 将占用以下硬件模块：

- AES 加速器

- RSA 加速器
- SHA 加速器

每个加速器都有一个对应的 free 寄存器字段，用于指示 RSA_DS 是否可以立即占用该加速器。如果 free 寄存器字段设置为 1，RSA_DS 将在开始运算时立即复位该加速器。如果 free 寄存器字段设置为 0，RSA_DS 将等待该加速器完成当前运算，或等到 RSA_DS_TIMEOUT_LIMIT 设置的超时上限。

当 RSA_DS 开始运算并占用加速器后，各加速器的释放方式如下：

- CBC 解密完成后，AES 加速器将被释放。
- RSA 加速器在整个 RSA_DS 运算期间保持占用。
- MD 校验完成后，SHA 加速器将被释放。

如果 RSA_DS 在占用一个或多个硬件加速器期间接收到外部复位，该复位也会同步传播至其占用的加速器。该机制确保暂存在这些加速器内部的 RSA_DS 中间结果会被清除，从而防止其因外部复位攻击而发生泄漏。

说明：

硬件占用是一种保护复用模块和存储空间的机制。当一个模块被硬件占用时，用户将无法执行以下操作：

- 读取或写入模块的寄存器或存储器数据。
- 关闭模块时钟。
- 复位模块。

在硬件占用结束时，占用的模块将自动复位。此外，当用户对主模块执行软件复位时，所有占用的模块将同时复位。

15.3.5 软件工作流程

每次需要计算数字签名时，都应执行以下软件操作。输入内容为预先生成的私钥密文 C、唯一的消息 X、IV。本节中的软件步骤会触发章节 15.3.4 中描述的硬件工作流程。
下述流程基于一个假设：软件已经调用了 HMAC 模块，HMAC 已经根据 $HMAC_KEY$ 计算出了 RSA_DS_KEY。

1. **准备工作**：准备好 C、X、IV。具体方法请参考章节 15.3.3 。
   - 若需禁用 PKCS#7 填充校验，请将 RSA_DS_STATIC_REG 中的 RSA_DS_SKIP_PADDING_CHECK 置为 1。
2. **启动 RSA_DS**：对寄存器 RSA_DS_SET_START_REG 写 1。
3. **选择 RSA_DS_KEY 的来源**：配置 RSA_DS_STATIC_REG 中的 RSA_DS_KEY_SOURCE 域，并检查是否已经准备好：
   - 将 RSA_DS_KEY_SOURCE 置为 1，即选择密钥管理器中部署的 RSA_DS_KEY。
     检查 RSA_DS_KEY 已经成功部署，检查方式为：读取寄存器域 KEYMNG_KEY_DS_VLD，当其值为 1 时，代表 RSA_DS_KEY 已经成功部署并准备好。
   - 将 RSA_DS_KEY_SOURCE 置为 0，即选择从 HMAC 模块计算 RSA_DS_KEY。
     轮询寄存器 RSA_DS_QUERY_BUSY_REG 直到读到 0。

     如果 RSA_DS_QUERY_BUSY_REG 超过 1 ms 还没读到 0，则说明 HMAC 初始化出现了问题。此时，软件应当读寄存器 RSA_DS_QUERY_KEY_WRONG_REG，根据返回值判断具体是下述哪一种情况：

- 如果读到零值，说明 HMAC 未被调用。
- 如果读到非零值 (1 ~ 15)，则说明 HMAC 被调用过，但是 RSA_DS 外设没有成功从 HMAC 模块获取到 RSA_DS_KEY。此种情况可能是因为受到了其他程序的干扰。

4. **将 IV 写入存储器 RSA_DS_IV_MEM**：将 IV block 中的内容写入存储器 RSA_DS_IV_MEM（大小为 16 字节）。有关 IV block 的更多信息，请参考章节 10 *AES 加速器 (AES)*。
5. **将 X 写入存储器 RSA_DS_X_MEM**：将 X_i ($i \in \{0, 1, \ldots, n-1\}$) 写入存储器 RSA_DS_X_MEM，容量为 128 个字 (word)，其中 $n = \frac{N}{32}$ 。每一个字刚好存放一个 b 进制数。存储块以小端字节序进行存储，即存储器的低地址存放运算子的低位进制数，高地址存放运算子的高位进制数。当 X 的长度小于 128 个字时，存储器 RSA_DS_X_MEM 中有一部分空间未使用，该部分空间中的数据可以是任意值。
6. **将 C 写入存储器**：即将 C 中的四个子参数分别写入对应的存储器。
   - 将 $\widehat{Y_i}$ ($i \in \{0, 1, \ldots, 127\}$) 写入存储器 RSA_DS_Y_MEM。
   - 将 $\widehat{M_i}$ ($i \in \{0, 1, \ldots, 127\}$) 写入存储器 RSA_DS_M_MEM。
   - 将 $\widehat{r_i}$ ($i \in \{0, 1, \ldots, 127\}$) 写入存储器 RSA_DS_RB_MEM。
   - 将 $\widehat{Box_i}$ ($i \in \{0, 1, \ldots, 11\}$) 写入存储器 RSA_DS_BOX_MEM。

     其中存储器 RSA_DS_Y_MEM、RSA_DS_M_MEM、RSA_DS_RB_MEM 的容量均为 128 个字，而存储器 RSA_DS_BOX_MEM 容量只有 12 个字。每一个字刚好存放一个 b 进制数。存储块以小端字节序进行存储，即存储器的低地址存放运算子的低位进制数，高地址存放运算子的高位进制数。
7. **启动计算**：对寄存器 RSA_DS_SET_CONTINUE_REG 写入 1。
8. **等待运算结束**：轮询寄存器 RSA_DS_QUERY_BUSY_REG 直到读到 0。
9. **检查校验结果**：读寄存器 RSA_DS_QUERY_CHECK_REG，根据返回值决定后续操作。
   - 如果返回值为 0，则说明填充校验通过，MD 校验通过，可以继续读取 Z 结果值。
   - 如果返回值为 1，则说明填充校验通过，但 MD 校验失败。Z 结果值无效，跳至步骤 11。
   - 如果返回值为 2，则说明填充校验失败，但 MD 校验通过，用户可以继续读取 Z 结果值。但仍需注意的是，此时数据填充不符合 PKCS#7 封装方式。
   - 如果返回值为 3，则说明填充校验失败，且 MD 校验失败。这种情况说明有致命错误发生。Z 结果值无效。跳至步骤 11。
10. **读出运算结果**：从存储器 RSA_DS_Z_MEM 读出运算结果 Z_i ($i \in \{0, 1, \ldots, n-1\}$)，其中 $n = \frac{N}{32}$ 。Z 以小端字节序存储在存储器中。
11. **退出计算环境**：对寄存器 RSA_DS_SET_FINISH_REG 写入 1，然后轮询寄存器 RSA_DS_QUERY_BUSY_REG 直到读到 0。

运算结束后，所有输入/输出寄存器和存储器中的数据均已被清除。

15.4 存储器列表

请注意，这里的起始地址和结束地址都是相对于 RSA 数字签名外设 (RSA_DS) 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

名称	描述	大小（字节）	起始地址	结束地址	访问
RSA_DS_Y_MEM	存储器 Y	512	0x0000	0x01FF	R/W
RSA_DS_M_MEM	存储器 M	512	0x0200	0x03FF	R/W
RSA_DS_RB_MEM	存储器 r	512	0x0400	0x05FF	R/W
RSA_DS_BOX_MEM	存储器 Box	48	0x0600	0x062F	R/W
RSA_DS_IV_MEM	存储器 IV	16	0x0630	0x063F	R/W
RSA_DS_X_MEM	存储器 X	512	0x0800	0x09FF	R/W
RSA_DS_Z_MEM	存储器 Z	512	0x0A00	0x0BFF	R/W

15.5 寄存器列表

本小节的所有地址均为相对于 RSA 数字签名外设 (RSA_DS) 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

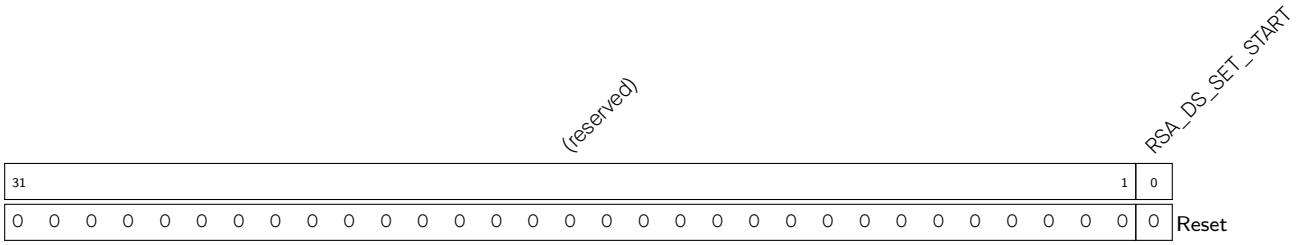
请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
状态/控制寄存器			
RSA_DS_SET_START_REG	启动 RSA_DS 外设	0x0E00	WT
RSA_DS_SET_CONTINUE_REG	继续 RSA_DS 计算	0x0E04	WT
RSA_DS_SET_FINISH_REG	结束 RSA_DS 计算	0x0E08	WT
RSA_DS_QUERY_BUSY_REG	表示 RSA_DS 外设状态	0x0E0C	RO
RSA_DS_QUERY_KEY_WRONG_REG	查询 RSA_DS_KEY 未准备好的原因	0x0E10	RO
RSA_DS_QUERY_CHECK_REG	查询 RSA_DS 校验结果	0x0E14	RO
配置寄存器			
RSA_DS_STATIC_REG	配置 RSA_DS_KEY 来源及填充校验旁路	0x0E18	R/W
RSA_DS_TIMEOUT_LIMIT_REG	配置等待子 IP 的超时上限	0x0E24	R/W
版本控制寄存器			
RSA_DS_DATE_REG	版本控制寄存器	0x0E20	R/W

15.6 寄存器

本小节的所有地址均为相对于 RSA 数字签名外设 (RSA_DS) 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

Register 15.1. RSA_DS_SET_START_REG (0x0E00)



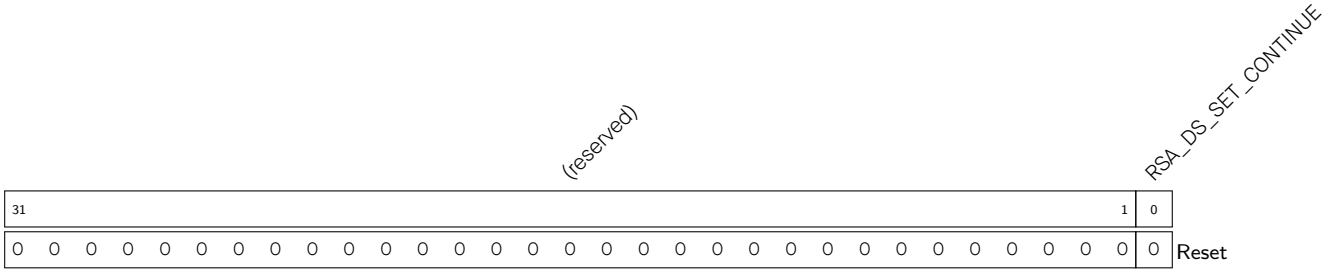
RSA_DS_SET_START 配置是否启动 RSA_DS 外设。

0：无效

1：启动 RSA_DS 外设

(WT)

Register 15.2. RSA_DS_SET_CONTINUE_REG (0x0E04)



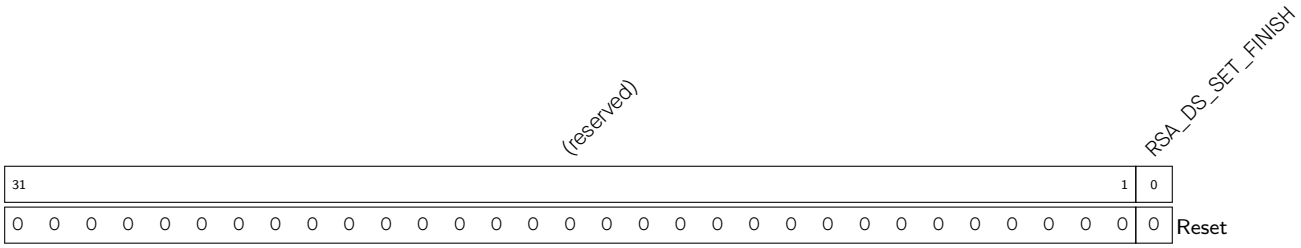
RSA_DS_SET_CONTINUE 配置是否继续 RSA_DS 计算。

0：无效

1：继续 RSA_DS 计算

(WT)

Register 15.3. RSA_DS_SET_FINISH_REG (0x0E08)



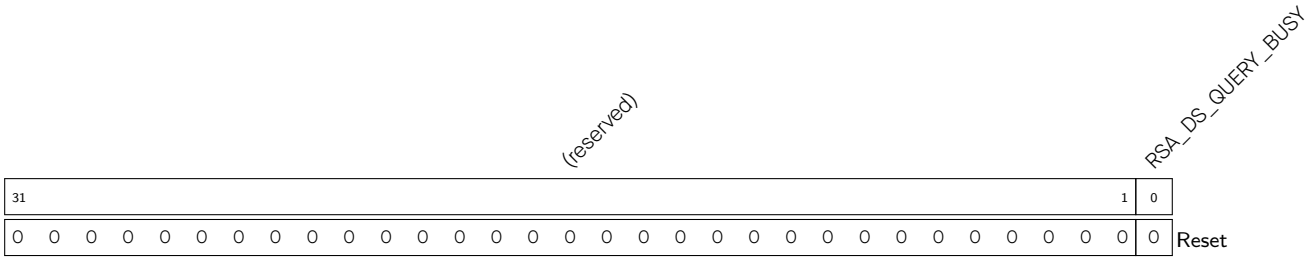
RSA_DS_SET_FINISH 配置是否结束 RSA_DS 计算。

0：无效

1：结束 RSA_DS 计算

(WT)

Register 15.4. RSA_DS_QUERY_BUSY_REG (0x0E0C)



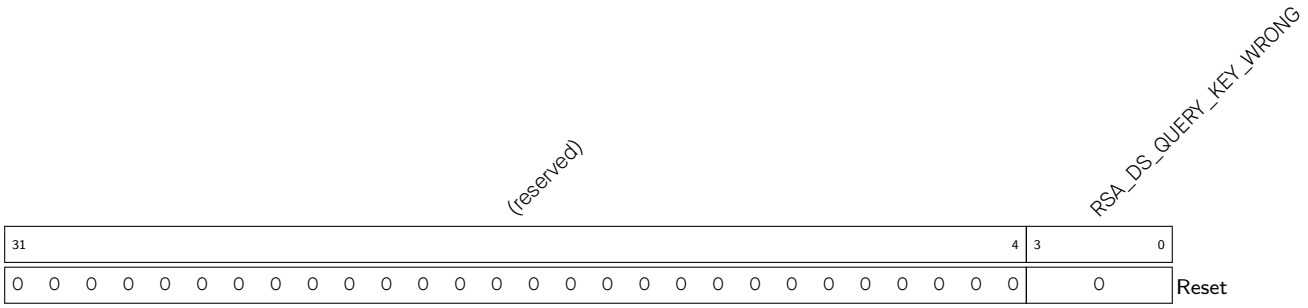
RSA_DS_QUERY_BUSY 表示 RSA_DS 外设是否空闲。

0：RSA_DS 外设空闲

1：RSA_DS 外设正忙

(RO)

Register 15.5. RSA_DS_QUERY_KEY_WRONG_REG (0x0E10)



RSA_DS_QUERY_KEY_WRONG 表示 HMAC 模块的错误情况。

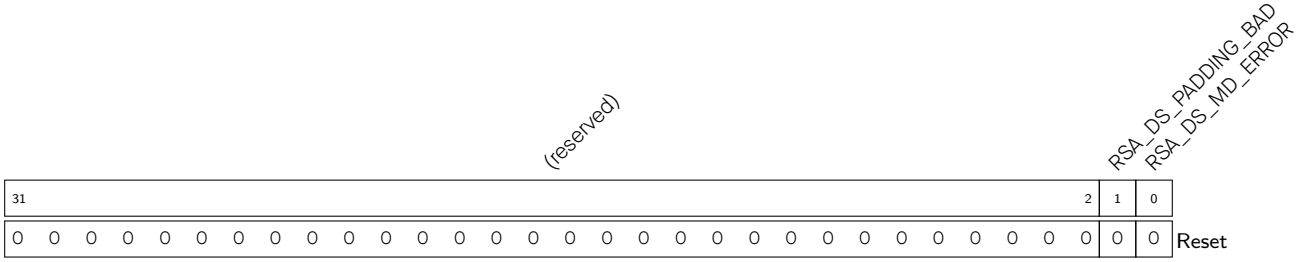
0：HMAC 未被调用

1-15：HMAC 曾被调用，RSA_DS 外设没有成功从 HMAC 模块获取到 RSA_DS_KEY

大于 15：无效

(RO)

Register 15.6. RSA_DS_QUERY_CHECK_REG (0x0E14)



RSA_DS_MD_ERROR 表示 MD 校验是否通过。

0：MD 校验通过

1：MD 校验失败

(RO)

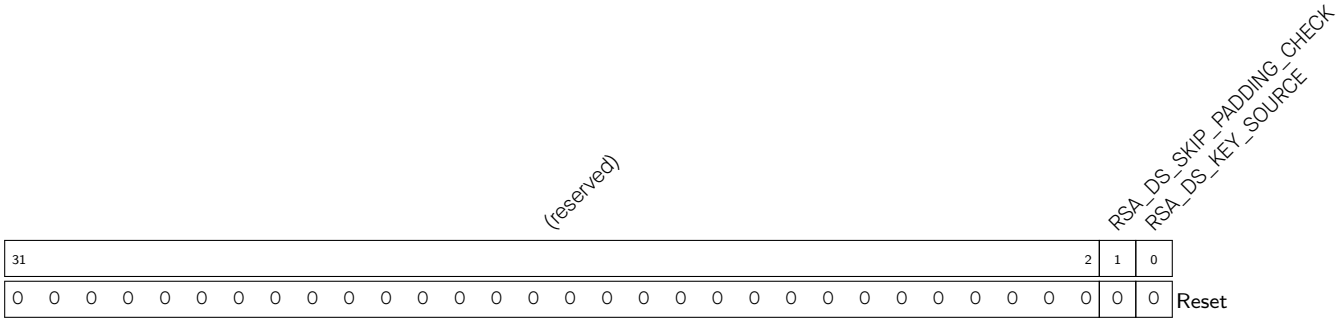
RSA_DS_PADDING_BAD 表示填充校验是否通过。

0：填充校验通过

1：填充校验失败

(RO)

Register 15.7. RSA_DS_STATIC_REG (0x0E18)



RSA_DS_KEY_SOURCE 表示 RSA_DS_KEY 的来源。

0：HMAC

1：密钥管理器

(R/W)

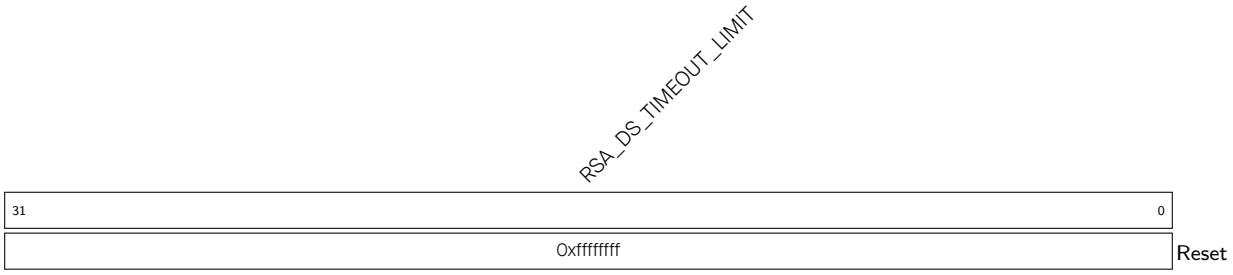
RSA_DS_SKIP_PADDING_CHECK 表示是否跳过填充校验。

0：启用填充校验

1：跳过填充校验

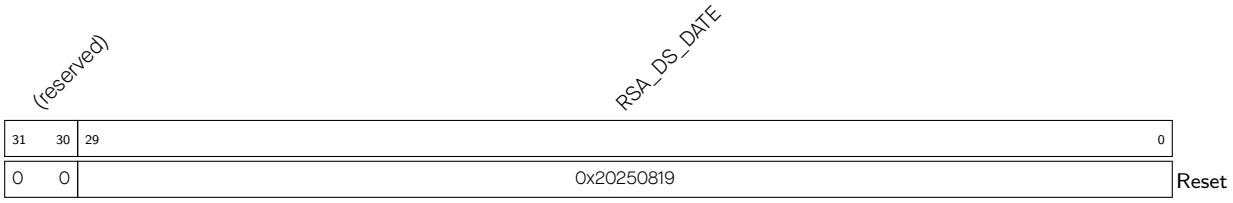
(R/W)

Register 15.8. RSA_DS_TIMEOUT_LIMIT_REG (0x0E24)



RSA_DS_TIMEOUT_LIMIT 配置等待 AES、RSA 和 SHA 加速器的超时上限。(R/W)

Register 15.9. RSA_DS_DATE_REG (0x0E20)



RSA_DS_DATE 版本控制寄存器。(R/W)

第 16 章

ECDSA 数字签名外设 (ECDSA_DS)

16.1 概述

在密码学中，椭圆曲线数字签名算法 (ECDSA) 是数字签名算法 (DSA) 基于椭圆曲线密码学的一种变体。

ESP32-S31 的 ECDSA 数字签名外设 (ECDSA_DS) 可在安全高效的环境中计算 ECDSA 签名，不仅可保证签名过程的机密性，还实现了快速计算，非常适合对加密安全性和加密速度有高要求的应用。通过 ECDSA_DS 模块，用户可以在不牺牲性能的情况下保证数据得到保护。

16.2 主要特性

ESP32-S31 的 ECDSA_DS 模块支持：

- 数字签名的生成和验证
- 四种椭圆曲线，分别是 P-192、P-256 和 P-384（具体定义见 FIPS 186-5 规范），以及 SM2（具体定义见 GM/T 0003.2-2012 规范）
- 用于 ECDSA_DS 运算的多种消息哈希算法，包括 SHA-224、SHA-256、SHA-384、SHA-512、SHA-512-224 和 SHA-512-256（具体定义见 FIPS PUB 180-4 规范），以及 SM3（具体定义见 GM/T 0004-2012 规范）
- 基于不同工作状态的寄存器访问控制，以确保信息安全

16.3 ECDSA 背景知识

16.3.1 域参数

ECDSA 使用的参数通常被称为域参数，可用于生成密钥、生成签名和验证签名等运算，主要包括有限域中定义的椭圆曲线参数、曲线上的基点和基点阶数等。

ECDSA 使用的域参数主要包括两大类：

- 椭圆曲线域参数：
  - 质数模数 p，指定定义椭圆曲线的有限域的大小。
  - 系数 a 和 b，用于指定椭圆曲线方程。
  - 曲线上的基点 G，用于生成公钥。
  - 基点的阶数 n，代表在椭圆曲线上对基点 G 进行重复点加的次数。
- 哈希函数参数：
  - 哈希算法，用于从待签名的消息生成固定长度的哈希值。

- 哈希值的长度，决定签名的大小。

这些参数共同定义了 ECDSA 域。

16.3.2 密钥生成

ECDSA 中的密钥生成过程如下所述：

1. 选择一个椭圆曲线域，定义如下参数：
   - 质数模数 p
   - 系数 a 和 b
   - 基点 G 和其阶数 n
2. 生成私钥：
   - 私钥 d 为在 1 到 n-1 之间随机选择的整数。注意，必须确保私钥是真正随机的，应当不可预测或不可重现，因此，使用安全的 随机数发生器 (RNG) [to be added later] 非常重要。
   - 私钥用于生成签名，需妥善保密保管。
   - 私钥选择完成后，应将其烧录到 ESP32-S31 的 eFuse 中（参考章节 *4 eFuse 控制器 (eFuse) [to be added later]*）。
3. 计算公钥：
   - 公钥 Q 的计算方式为 $Q = dG$。
4. 导出公钥：
   - 公钥 Q 通常以 (Qx, Qy) 表示，可公开给他人。

16.3.3 签名生成

ECDSA 签名生成过程如下所述：

1. 选择一条消息 m，用于签名。
2. 计算消息 m 的哈希值 e：e = HASH(m)，其中 HASH 是一个密码学哈希函数，例如 SHA-256。
3. 计算消息哈希值 e 的摘要 z：令 z 为 e 的左端 L_n 位，其中 L_n 是基点阶数 n 的位长度。
4. 选择一个随机数 k：范围在 1 到 n-1 之间，其中 n 是椭圆曲线上基点的阶数。
5. 计算签名：签名计算如下：
   (a) 计算点 (x, y) = kG
   (b) 计算 r = x mod n。如果 r 等于零，返回上一步并选择新的 k 值。
   (c) 计算 s = k^{-1} * $(z + d * r)$ mod n。如果 s 等于零，返回步骤 4 并选择新的 k 值。
   (d) 签名为 (r, s)。
6. 发送消息 m 和签名 (r, s) 给接收者。

16.3.4 签名验证

接收方可以使用密钥生成阶段导出的公钥，来验证对应私钥生成的签名。验证过程涉及检查签名是否由对应的私钥生成，以及签名是否对给定的消息有效。

ECDSA 签名验证过程如下所述：

1. 获得公钥 Q，接收消息 m 和签名 (r, s)。
2. 计算消息的哈希值：e 等于 HASH(m)，其中 HASH 是一个密码学哈希函数，例如 SHA-256。
3. 计算消息的摘要：令 z 为 e 的左端 L_n 位，其中 L_n 是基点阶数 n 的位长度。
4. 验证签名：签名验证如下所示：
   (a) 验证 r 和 s 是否是介于 1 到 n-1 之间的整数，其中 n 是椭圆曲线上基点的阶数。如果 r 或 s 超出此范围，则签名无效。
   (b) 计算 $u_1 = z * s^{-1} \bmod n$ 和 $u_2 = r * s^{-1} \bmod n$。
   (c) 计算点 $(x_1, y_1) = u_1 * G + u_2 * Q$，其中 G 是椭圆曲线上的基点，Q 是与用于签名生成的私钥相关联的公钥。
   (d) 验证 $r = x_1 \bmod n$。如果 r 不等于 $x_1 \bmod n$，则签名无效。
5. 接受或拒绝签名：如果签名有效，接收方可以确信消息没有被篡改，且消息来自预期的发送方，可以接收。否则，接收方将拒绝并判定为无效消息。

16.4 功能描述

本节详细介绍 ESP32-S31 的 ECDSA_DS 模块。

16.4.1 ECDSA_DS 工作模式

ESP32-S31 内置的 ECDSA_DS 具有三种工作模式，即签名生成、签名验证和公钥导出模式。

用户可以通过配置 ECDSA_DS_WORK_MODE，根据下表 16.4-1 选择 ECDSA_DS 的工作模式。

表 16.4-1. ECDSA_DS 工作模式

ECDSA_DS_WORK_MODE	工作模式
0	签名验证
1	签名生成
2	公钥导出

用户可以通过配置 ECDSA_DS_ECC_CURVE，根据下表 16.4-2 选择 ECDSA_DS 使用的椭圆曲线。

表 16.4-2. ECDSA_DS 椭圆曲线选择

ECDSA_DS_ECC_CURVE	椭圆曲线
0	P-192
1	P-256
2	P-384
3	SM2

用户可以通过配置 ECDSA_DS_SHA_MODE，根据下表 16.4-3 选择用于消息哈希的 SHA 算法。

表 16.4-3. ECDSA_DS SHA 算法

ECDSA_DS_SHA_MODE	SHA 算法
1	SHA-224
2	SHA-256
3	SHA-384
4	SHA-512
5	SHA-512-224
6	SHA-512-256
14	SM3
其他	无效

说明：
国密算法 (SM2&SM3) 的支持可通过 EFUSE_DIS_SM_CRYPT 禁用。

此外，用户可以通过查询 ECDSA_DS_STATE_REG 寄存器并将返回值与下表 16.4-4 进行比较，来确认 ECDSA_DS 的工作状态。

表 16.4-4. ECDSA_DS 工作状态

ECDSA_DS_STATE_REG	状态	描述
0	IDLE	空闲或已完成的运算，对应于 IDLE 阶段。
1	LOAD	等待用户将信息加载到 ECDSA_DS 中，对应于 LOAD 阶段。
2	GAIN	等待用户从 ECDSA_DS 获取信息，对应于 GAIN 阶段。
3	BUSY	硬件正在运行中，对应于 PREP、PROC 和 POST 阶段。

16.4.2 密钥来源

ESP32-S31 的 ECDSA_DS 模块可以使用硬件部署的密钥或软件编程的密钥。硬件部署的密钥可以来自 eFuse（参考章节 *4 eFuse 控制器 (eFuse) [to be added later]*）或密钥管理器（参考章节 *21 密钥管理器 [to be added later]*）。如果使用软件编程的密钥，需要将密钥分成 32 位的数据块，并写入 ECDSA_DS_KEY_*n*_REG（*n* = 0 ~ 11）。在配置时，应将 ECDSA_DS_USE_HARDWARE_KEY 设置为 0，以使 ECDSA_DS 使用软件编程的密

钥。

16.4.3 数据和数据块

ESP32-S31 的 ECDSA_DS 会用到的数据位宽为 192、256、384 或 512 位。

假设一个数据为 $D[255:0]$，则其可以被划分为八个 32 位的数据块 $D[n][31:0](n=0,1,\cdots,7)$，序号较小的数据块对应数据低位。以 256 位的数据为例：

$$D[255:0]=D[7][31:0],D[6][31:0],D[5][31:0],D[4][31:0],D[3][31:0],D[2][31:0],D[1][31:0],D[0][31:0]$$

16.4.3.1 数据写入

数据写入即将数据写入到 ECDSA_DS 内存块的过程，该数据为 ECDSA_DS 的输入数据。具体而言，将数据写入 ECDSA_DS 内存块相当于将该数据 $D[n][31:0]$ 依次写入“该内存块起始地址 + $4\times n$”。

以 256 位的数据为例：

- 将 $D[0]$ 写入“起始地址”
- 将 $D[1]$ 写入“起始地址 + 4”
- $\cdots$
- 将 $D[7]$ 写入“起始地址 + 28”

说明：
进行数据写入操作时，只需写入所需长度的数据块，无需在高位补 0。例如，写入 192 位数据时，只需存储 192 位即可。

16.4.3.2 数据读取

数据读取即从 ECDSA_DS 内存块中读出数据的过程，该数据为 ECDSA_DS 的输出数据。具体来说，从 ECDSA_DS 内存块中读取数据相当于从“该内存块起始地址 + $4\times n$”依次读出 $D[n][31:0]$。以 256 位的数据为例：

- 从“起始地址”读出 $D[0]$
- 从“起始地址 + 4”读出 $D[1]$
- $\cdots$
- 从“起始地址 + 28”读出 $D[7]$

说明：
进行数据读取操作时，只需读取所需长度的数据块。例如，读取 192 位数据时，只需读取低 192 位（即 6 个数据块）。

16.4.3.3 消息填充

SHA 加速器仅能处理 512 位的消息块。因此，所有消息在进行哈希运算前，应通过软件填充至 512 位的整数倍。

假设待处理消息 M 的长度为 L_M 位，则填充步骤如下：

1. 首先，在消息末尾填充 1 个 “1”；
2. 随后，再填充 L_A 个 “0”。其中，L_A 为满足 $L_M + 1 + L_A \equiv 448 \ mod \ 512$ 的最小非负数解；
3. 最后，在末尾填充一个 64 位的信息块。该信息块的内容为用二进制表示的消息的长度，即 L_M 的值。

更多详情，请参考 《FIPS PUB 180-4 规范》 > 章节 “Padding the Message”。

16.4.3.4 解析消息

完成消息填充后，还需将消息（及其填充）解析为 N 个 512 位的消息块，即 $M^{(1)}$、$M^{(2)}$、...、$M^{(N)}$。

说明：

1. 有关 “解析消息” 的详细信息，请参考 FIPS PUB 180-4 规范 > 章节 “Parsing the Message”。
2. 有关” 消息块” 及相关概念的描述，请参考 FIPS PUB 180-4 规范 > 章节 “Glossary of Terms and Acronyms”。

16.4.4 安全功能

为确保运算过程的安全性，ECDSA_DS 模块实现了多种安全功能。

16.4.4.1 抗攻击性能

ESP32-S31 的 ECDSA_DS 模块每次执行操作时，都会利用 ECC 加速器的强抗攻击性能（参考小节 11.4.3 提升抗攻击性能）。这意味着 ECDSA_DS 在每次生成和验证签名时：

- 对于给定操作，执行延迟保持恒定。
- 功耗波动最小化。

这为 ESP32-S31 的 ECDSA_DS 模块提供了强大的抗攻击性能。

16.4.4.2 基于状态的寄存器访问控制

ESP32-S31 的 ECDSA_DS 模块实施了基于状态的寄存器访问控制，可有效防止运算过程中可能发生的非法配置篡改或数据访问，从而保证密钥的安全性。

具体来说，ECDSA_DS 的寄存器在不同工作状态下的访问权限是不同的。例如，ECDSA_DS_CONF_REG 寄存器仅在 ECDSA_DS 处于 IDLE 状态时允许读写操作。因此，当 ECDSA_DS 处于其他状态（如 LOAD、GAIN 和 BUSY）时，该寄存器中的配置信息则不允许读取或写入，从而得到很好的保护。有关 ECDSA_DS 所有工作状态的详细信息，请见表 16.4-4。

有关基于状态的 ECDSA_DS 寄存器访问控制的详细信息，请参阅寄存器列表。

16.4.4.3 硬件占用

在运算过程中，ECDSA_DS 将占用以下硬件模块：

- SHA 加速器
- ECC 加速器

每个加速器都有一个对应的 free 字段（SHA_FREE 和 ECC_MULT_FREE），用于指示 ECDSA_DS 是否可以立即占用该加速器。如果 free 字段设置为 1，ECDSA_DS 在开始运算时会立即复位该加速器。如果 free 字段设置为 0，ECDSA_DS 将等待该加速器完成当前运算，或等到 ECDSA_DS_TIMEOUT_LIMIT 设置的超时上限。

当 ECDSA_DS 开始运算并占用加速器后，各加速器的释放方式如下：当 ECDSA_DS_SHA_RELEASE_INT 触发时，SHA 加速器将被释放；而 ECC 加速器在整个 ECDSA_DS 运算期间保持占用。

如果 ECDSA_DS 在占用一个或多个硬件加速器期间接收到外部复位，该复位也会同步传播至这些加速器。该机制确保暂存在这些加速器内部的中间结果会被清除，从而防止其因外部复位攻击而发生泄漏。

说明：

硬件占用是一种保护复用模块和存储空间的机制。当一个模块被硬件占用时，用户将无法执行以下操作：

- 读取或写入模块的寄存器或存储器数据。
- 关闭模块时钟。
- 复位模块。

在硬件占用结束时，占用的模块将自动复位。此外，当用户对主模块执行软件复位时，所有占用的模块将同时复位。

注意： ESP32-S31 的 安全调试控制器 (SDC) [to be added later] 模块在工作时也会使用 ECDSA_DS。默认情况下，ECDSA_DS_FREE（值为 1）允许 安全调试控制器 (SDC) [to be added later] 立即复位任何正在进行的 ECDSA_DS 运算并占用 ECDSA_DS。为防止这种行为，请将 ECDSA_DS_FREE 设置为 0。

16.5 配置流程

16.5.1 ECDSA_DS 流程

ECDSA_DS 的完整工作流程包括以下六个阶段。

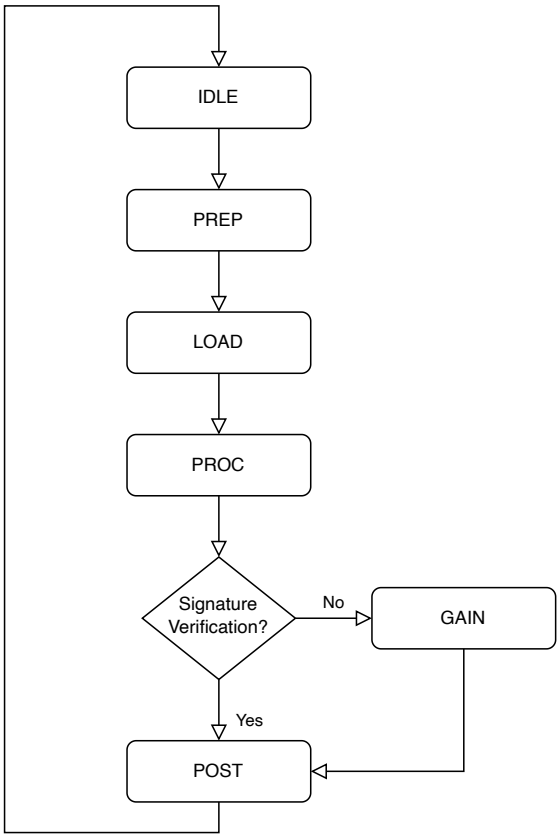


图 16.5-1. ECDSA_DS 流程

每个阶段的详细配置流程在下文中描述。

16.5.1.1 IDLE 阶段

在 IDLE 阶段：

1. 配置静态参数，包括 eFuse 位：
   (a) ECDSA_KEY：ECDSA_DS 中的私钥 d 的值。请选择以下密钥来源之一：
      i. eFuse：将密钥值写入 KEY*n*（*n* = 0 ~ 5），并将相应的 EFUSE_KEY_PURPOSE_*n* 设置为 ECDSA_KEY。有关更详细的配置步骤，请参阅章节 *4 eFuse 控制器 (eFuse) [to be added later]*。
      ii. 密钥管理器：使用密钥管理器的其中一种模式为 ECDSA 部署密钥。详细的配置步骤，请参阅章节 *21 密钥管理器 [to be added later]*。
      iii. ECDSA 软件寄存器：在 LOAD 阶段，将密钥值写入 ECDSA_DS_KEY_*n*_REG（*n* = 0 ~ 11）。
2. 配置 ECDSA_DS_CONF_REG，包括以下字段：
   (a) ECDSA_DS_WORK_MODE：选择 ECDSA_DS 的工作模式。
   (b) ECDSA_DS_ECC_CURVE：选择 ECDSA_DS 的椭圆曲线。
   (c) ECDSA_DS_SOFTWARE_SET_Z：选择是否使用直接输入的 z。
   (d) ECDSA_DS_DETERMINISTIC_K：选择是否使用确定性的 k。

(e) ECDSA_DS_USE_HARDWARE_KEY：选择 ECDSA 密钥来源是软件编程寄存器还是硬件来源（eFuse 或密钥管理器）。

3. 配置寄存器字段 ECDSA_DS_START 以进入 PREP 阶段。

说明：
更多有关“确定性 ECDSA”(Deterministic ECDSA) 的信息，请参考 RFC6979。

16.5.1.2 PREP 阶段

在 PREP 阶段，ECDSA_DS_STATE_REG 为 BUSY，ECDSA_DS 执行准备工作。

轮询 ECDSA_DS_BUSY 直到其不为 BUSY，等待 PREP 阶段结束。然后 ECDSA_DS 将自动进入 LOAD 阶段。

16.5.1.3 LOAD 阶段

在 LOAD 阶段：

1. 使用以下选项之一将输入 z 提供给 ECDSA_DS：
   - 直接输入 z：将 z 写入 ECDSA_DS_MEM_Z。
   - ECDSA_DS SHA 接口：从消息生成 z。有关详细信息，请参阅第 16.5.1.7 节。
2. 根据所选的 ECDSA_DS_WORK_MODE，进行以下配置：
   - 签名验证：
     (a) 将签名 (r，s) 写入 ECDSA_DS_MEM_R 和 ECDSA_DS_MEM_S。
     (b) 将公钥 (Qx，Qy) 写入 ECDSA_DS_MEM_Qx 和 ECDSA_DS_MEM_Qy。
   - 签名生成：
     (a) 不需要其他配置。
   - 公钥导出：
     (a) 不需要其他配置。
3. 如果使用 ECDSA 软件寄存器中的密钥，需以大端格式配置 ECDSA_DS_KEY_*n*_REG（*n* = 0 ~ 11）。192 位密钥使用 *n* = 0 ~ 5，256 位密钥使用 *n* = 0 ~ 7，384 位密钥使用 *n* = 0 ~ 11。例如，$Key[255:0]$ 中，$Key[255:224]$ 写入 *n* = 7，$Key[31:0]$ 写入 *n* = 0。
4. 将 1 写入 ECDSA_DS_LOAD_DONE，表示配置完成。然后 ECDSA_DS 将自动进入 PROC 阶段。

16.5.1.4 PROC 阶段

在 PROC 阶段，ECDSA_DS_STATE_REG 为 BUSY，ECDSA_DS 根据所选的工作模式执行运算。

通过轮询 ECDSA_DS_STATE_REG 直到其中的 ECDSA_DS_BUSY 字段指示不为 BUSY，表示 PROC 阶段已结束。然后 ECDSA_DS 将根据所选的工作模式自动进入 GAIN 阶段或 POST 阶段：

- 签名生成：GAIN 阶段
- 签名验证：POST 阶段

- 公钥导出：GAIN 阶段

16.5.1.5 GAIN 阶段

当选择了签名生成或公钥导出模式时，ECDSA_DS 在 PROC 阶段之后进入 GAIN 阶段：

1. 从 ECDSA_DS 内存中读取数据：
   - 从 ECDSA_DS_MEM_R 和 ECDSA_DS_MEM_S 读取签名 (r, s)，仅在签名生成模式下。
   - 从 ECDSA_DS_MEM_Qx 和 ECDSA_DS_MEM_Qy 读取公钥 (Qx, Qy)，在签名生成或公钥导出模式下。
2. 向 ECDSA_DS_GET_DONE 写入 1，表示 GAIN 阶段已完成。然后 ECDSA_DS 将自动进入 POST 阶段。

16.5.1.6 POST 阶段

在 POST 阶段，ECDSA_DS_STATE_REG 为 BUSY，ECDSA_DS 执行运算的收尾工作。

通过轮询 ECDSA_DS_BUSY 直到不再为 BUSY，表示 POST 阶段已结束。然后 ECDSA_DS 将自动返回 IDLE 阶段。

16.5.1.7 ECDSA_DS SHA 接口

ESP32-S31 的 ECDSA_DS 模块可以自动执行哈希运算并基于直接输入的消息生成 z。

对于消息哈希，ECDSA_DS 支持 SHA 算法 SHA-224（仅当选择 P-192 作为椭圆曲线时有效）和 SHA-256。

要使用 ECDSA_DS SHA 接口，请完成以下步骤：

1. 根据第 16.4.3.3 节中的描述，对消息进行填充。
2. 将消息及其填充解析为消息块。详细信息请参见第 16.4.3.4 节。
3. 处理当前消息块。
   - 将当前消息块写入 ECDSA_DS_MEM_M。
4. 启动 ECDSA_DS SHA 接口 [1]。
   - 首次执行此步骤时，请将 1 写入 ECDSA_DS_SHA_START 以启动 ECDSA_DS SHA 接口；
   - 非首次执行此步骤 [2]，请将 1 写入 ECDSA_DS_SHA_CONTINUE 以继续运算。
5. 查询当前消息块的处理进度。
   - 轮询寄存器 ECDSA_DS_SHA_BUSY，直到其为 IDLE，表示接口已完成当前消息块的运算。
6. 处理下一个消息块：
   - 如有下一个消息块，则返回到步骤 3。
   - 如没有更多的消息块，退出。

说明：

1. 在此步骤中，软件还可以在接口启动 SHA 运算时将下一个要处理的消息块（如有）写入寄存器 ECDSA_DS_MEM_M，以节省时间。
2. 即正在继续之前暂停的 ECDSA_DS SHA 接口运算。

16.5.2 时钟

ECDSA_DS 有两种时钟：

- clk_ecdsa：ECDSA_DS 运行所使用的功能时钟
- clk_apb_ecdsa：用于配置 ECDSA_DS 寄存器的总线时钟

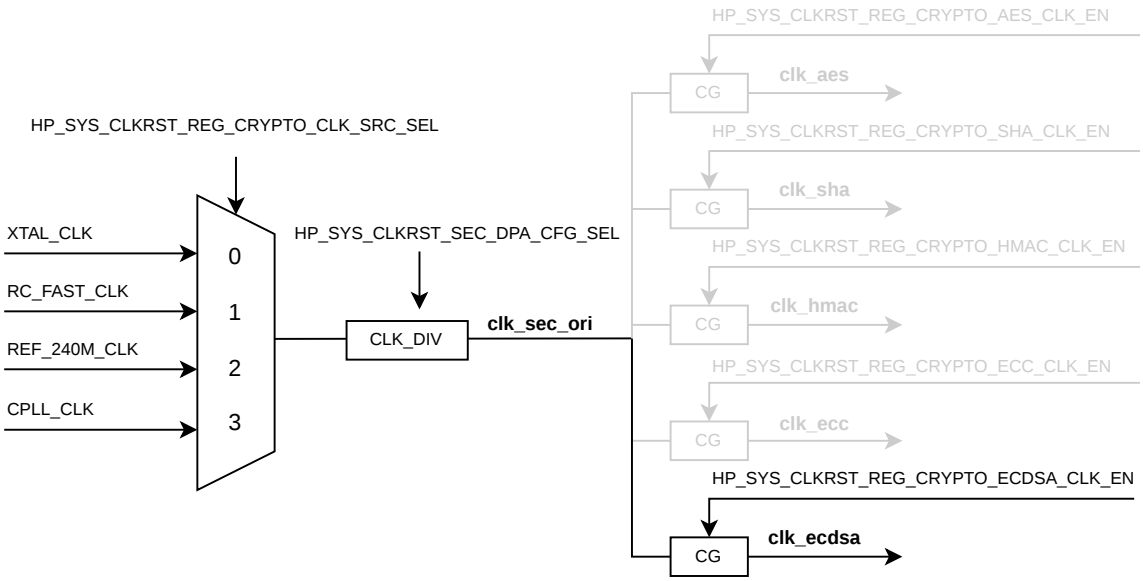


图 16.5-2. clk_ecdsa 时钟图

如图 16.5-2 *clk_ecdsa 时钟图* 所示，安全功能源时钟 clk_sec_ori 有以下时钟源：

- XTAL_CLK
- RC_FAST_CLK
- REF_240M_CLK
- CPLL_CLK

ECDSA_DS 功能时钟 clk_ecdsa 由两个时钟门控寄存器 HP_SYS_CLKRST_REG_CRYPTO_SYS_CLK_EN 和 HP_SYS_CLKRST_REG_CRYPTO_ECDSA_CLK_EN 控制，当这两个寄存器值同时为 1 时，时钟门控开启，使能 clk_ecdsa。此功能时钟的时钟分频也受 HP_SYS_CLKRST_DPA_CTRL0_REG 和 eFuse 字段 EFUSE_SEC_DPA_LEVEL 控制（更多信息，请参考章节 7 *复位和时钟 [to be added later]*）。

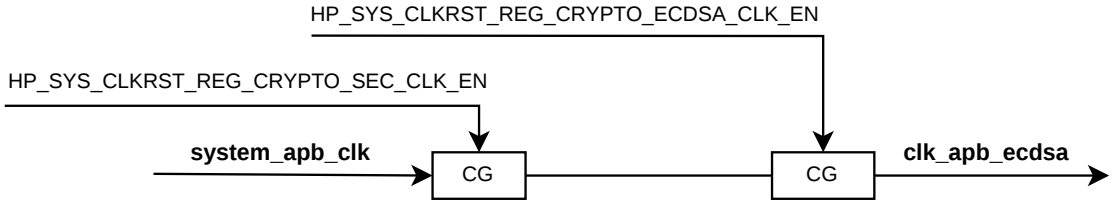


图 16.5-3. clk_apb_ecdsa 时钟图

如图 16.5-3 *clk_apb_ecdsa 时钟图* 所示，ECDSA_DS 配置时钟 clk_apb_ecdsa 的时钟源为

system_apb_clk。

ECDSA_DS 配置时钟 clk_apb_ecdsa 由两个时钟门控寄存器 HP_SYS_CLKRST_REG_CRYPTO_SEC_CLK_EN 和 HP_SYS_CLKRST_REG_CRYPTO_ECDSA_CLK_EN 控制，当这两个寄存器值同时为 1 时，时钟门控开启，使能 clk_apb_ecdsa。

有关时钟的更多信息，请查看章节 *7 复位和时钟 [to be added later]*。

16.5.3 复位

ECDSA_DS 支持一个复位域，向 HP_SYS_CLKRST_REG_CRYPTO_ECDSA_RST_EN 先写 1 再写 0，即可复位整个 ECDSA_DS 模块。

16.6 中断

ESP32-S31 的 ECDSA_DS 模块可以生成一个中断信号 ECDSA_DS_INTR 并发送给 中断矩阵。

ECDSA_DS 有四个中断源可产生中断信号 ECDSA_DS_INTR，如下所示。每个中断源都有一组通用的配置寄存器，详见章节 *中断配置寄存器*。

- ECDSA_DS_PREP_DONE_INT：在 PREP 阶段结束时触发。可以通过以下寄存器配置此中断源：
  - ECDSA_DS_PREP_DONE_INT_RAW：存储 ECDSA_DS_PREP_DONE_INT 的原始中断状态。
  - ECDSA_DS_PREP_DONE_INT_ST：代表 ECDSA_DS_PREP_DONE_INT 中断的状态。通过 ECDSA_DS_PREP_DONE_INT_RAW 字段启用/禁用此字段生成。
  - ECDSA_DS_PREP_DONE_INT_ENA：启用/禁用 ECDSA_DS_PREP_DONE_INT 中断。
  - ECDSA_DS_PREP_DONE_INT_CLR：将此位设置为 1 以清除 ECDSA_DS_PREP_DONE_INT 中断状态。通过将此位设置为 1，字段 ECDSA_DS_PREP_DONE_INT_RAW 和 ECDSA_DS_PREP_DONE_INT_ST 将被清除。
- ECDSA_DS_PROC_DONE_INT：在 PROC 阶段结束时触发。可以通过以下寄存器配置此中断源：
  - ECDSA_DS_PROC_DONE_INT_RAW：存储 ECDSA_DS_PROC_DONE_INT 的原始中断状态。
  - ECDSA_DS_PROC_DONE_INT_ST：代表 ECDSA_DS_PROC_DONE_INT 中断的状态。通过 ECDSA_DS_PROC_DONE_INT_RAW 字段启用/禁用此字段生成。
  - ECDSA_DS_PROC_DONE_INT_ENA：启用/禁用 ECDSA_DS_PROC_DONE_INT 中断。
  - ECDSA_DS_PROC_DONE_INT_CLR：将此位设置为 1 以清除 ECDSA_DS_PROC_DONE_INT 中断状态。通过将此位设置为 1，字段 ECDSA_DS_PROC_DONE_INT_RAW 和 ECDSA_DS_PROC_DONE_INT_ST 将被清除。
- ECDSA_DS_POST_DONE_INT：在 POST 阶段结束时触发。可以通过以下寄存器配置此中断源：
  - ECDSA_DS_POST_DONE_INT_RAW：存储 ECDSA_DS_POST_DONE_INT 的原始中断状态。
  - ECDSA_DS_POST_DONE_INT_ST：代表 ECDSA_DS_POST_DONE_INT 中断的状态。通过 ECDSA_DS_POST_DONE_INT_RAW 字段启用/禁用此字段生成。
  - ECDSA_DS_POST_DONE_INT_ENA：启用/禁用 ECDSA_DS_POST_DONE_INT 中断。

  - ECDSA_DS_POST_DONE_INT_CLR：将此位设置为 1 以清除 ECDSA_DS_POST_DONE_INT 中断状态。通过将此位设置为 1，字段 ECDSA_DS_POST_DONE_INT_RAW 和 ECDSA_DS_POST_DONE_INT_ST 将被清除。

- ECDSA_DS_SHA_RELEASE_INT：在 SHA 释放时触发。可以通过以下寄存器配置此中断源：
  - ECDSA_DS_SHA_RELEASE_INT_RAW：存储 ECDSA_DS_SHA_RELEASE_INT 的原始中断状态。
  - ECDSA_DS_SHA_RELEASE_INT_ST：代表 ECDSA_DS_SHA_RELEASE_INT 中断的状态。通过 ECDSA_DS_SHA_RELEASE_INT_RAW 字段启用/禁用此字段生成。
  - ECDSA_DS_SHA_RELEASE_INT_ENA：启用/禁用 ECDSA_DS_SHA_RELEASE_INT 中断。
  - ECDSA_DS_SHA_RELEASE_INT_CLR：将此位设置为 1 以清除 ECDSA_DS_SHA_RELEASE_INT 中断状态。通过将此位设置为 1，字段 ECDSA_DS_SHA_RELEASE_INT_RAW 和 ECDSA_DS_SHA_RELEASE_INT_ST 将被清除。

说明：

中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

16.7 存储器块

ECDSA_DS 的存储器块用于存储运算的输入数据和输出数据。

表 16.7-1. ECDSA_DS 存储器块

存储器块	大小（字节）	起始地址*	结束地址*	访问权限
ECDSA_DS_MEM_M	128	0x280	0x2FF	R/W
ECDSA_DS_MEM_R	48	0x3E0	0x40F	R/W
ECDSA_DS_MEM_S	48	0x410	0x43F	R/W
ECDSA_DS_MEM_Z	48	0x440	0x46F	R/W
ECDSA_DS_MEM_Qx	48	0x470	0x49F	R/W
ECDSA_DS_MEM_Qy	48	0x4A0	0x4CF	R/W

* 相对于 ECDSA_DS 基地址的地址偏移在第 3 章*系统和存储器 [to be added later]* 中的表 3 中提供。

16.8 寄存器列表

本小节的所有地址均为相对于 ECDSA_DS 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

为了提高安全性，在不同工作状态下，ECDSA_DS 寄存器的读写权限不同。以下是每个工作状态的缩写及其与阶段的对应关系：

- PI：IDLE 状态，对应 IDLE 阶段。
- PL：LOAD 状态，对应 LOAD 阶段。
- PG：GAIN 状态，对应 GAIN 阶段。
- PB：BUSY 状态，对应 PREP、PROC 和 POST 阶段。

请查看章节 寄存器的访问类型，了解“访问”列缩写的含义。

名称	描述	地址	访问权限			
			PI	PL	PG	PB
数据存储器	见表 16.7-1。					
配置寄存器						
ECDSA_DS_CONF_REG	ECDSA_DS 配置寄存器	0x0004	R/W	N/A		
ECDSA_DS_START_REG	ECDSA_DS 启动寄存器	0x001C	WT			N/A
ECDSA_DS_TIMEOUT_LIMIT_REG	ECDSA_DS 超时上限配置寄存器	0x0028	R/W	RO		
ECDSA_DS_FREE_REG	ECDSA_DS 空闲状态寄存器	0x002C	R/W			
时钟和复位寄存器						
ECDSA_DS_CLK_REG	ECDSA_DS 时钟门控寄存器	0x0008	R/W	N/A		
中断寄存器						
ECDSA_DS_INT_RAW_REG	ECDSA_DS 中断原始状态寄存器	0x000C	RO/WTC/SS			
ECDSA_DS_INT_ST_REG	ECDSA_DS 中断状态寄存器	0x0010	RO			
ECDSA_DS_INT_ENA_REG	ECDSA_DS 中断使能寄存器	0x0014	R/W			
ECDSA_DS_INT_CLR_REG	ECDSA_DS 中断清除寄存器	0x0018	WT			
状态寄存器						
ECDSA_DS_STATE_REG	ECDSA_DS 状态寄存器	0x0020	RO			
结果寄存器						
ECDSA_DS_RESULT_REG	ECDSA_DS 结果寄存器	0x0024	RO/SS			N/A
密钥寄存器						
ECDSA_DS_KEY_0_REG	ECDSA_DS 密钥数据寄存器 0	0x0040	RO	RW	RO	
ECDSA_DS_KEY_1_REG	ECDSA_DS 密钥数据寄存器 1	0x0044	RO	RW	RO	
ECDSA_DS_KEY_2_REG	ECDSA_DS 密钥数据寄存器 2	0x0048	RO	RW	RO	
ECDSA_DS_KEY_3_REG	ECDSA_DS 密钥数据寄存器 3	0x004C	RO	RW	RO	
ECDSA_DS_KEY_4_REG	ECDSA_DS 密钥数据寄存器 4	0x0050	RO	RW	RO	
ECDSA_DS_KEY_5_REG	ECDSA_DS 密钥数据寄存器 5	0x0054	RO	RW	RO	
ECDSA_DS_KEY_6_REG	ECDSA_DS 密钥数据寄存器 6	0x0058	RO	RW	RO	
ECDSA_DS_KEY_7_REG	ECDSA_DS 密钥数据寄存器 7	0x005C	RO	RW	RO	
ECDSA_DS_KEY_8_REG	ECDSA_DS 密钥数据寄存器 8	0x0060	RO	RW	RO	
ECDSA_DS_KEY_9_REG	ECDSA_DS 密钥数据寄存器 9	0x0064	RO	RW	RO	
ECDSA_DS_KEY_10_REG	ECDSA_DS 密钥数据寄存器 10	0x0068	RO	RW	RO	
ECDSA_DS_KEY_11_REG	ECDSA_DS 密钥数据寄存器 11	0x006C	RO	RW	RO	
SHA 寄存器						

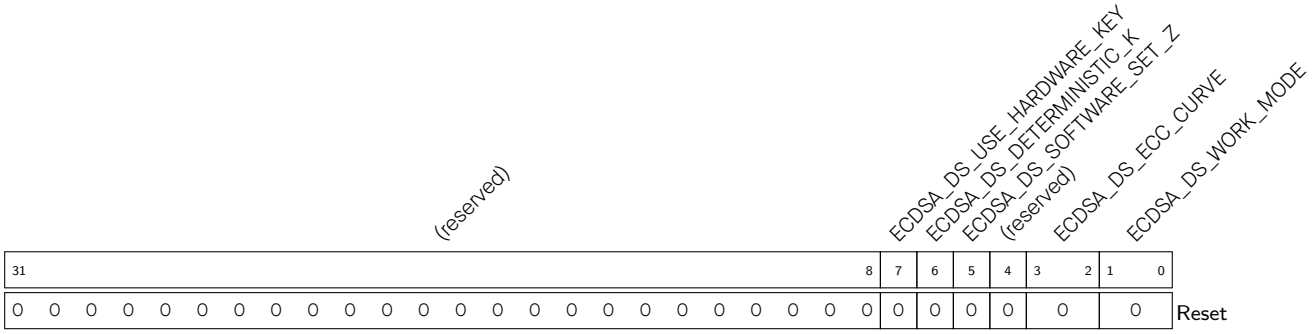
名称	描述	地址	访问权限			
			PI	PL	PG	PB
ECDSA_DS_SHA_MODE_REG	ECDSA_DS 控制 SHA 寄存器（哈希算法）	0x0200	N/A	R/W	N/A	
ECDSA_DS_SHA_START_REG	ECDSA_DS 控制 SHA 寄存器（运算）	0x0210	N/A	WT	N/A	
ECDSA_DS_SHA_CONTINUE_REG	ECDSA_DS 控制 SHA 寄存器（运算）	0x0214	N/A	WT	N/A	
ECDSA_DS_SHA_BUSY_REG	ECDSA_DS 控制 SHA 状态寄存器	0x0218	N/A	RO	N/A	
版本寄存器						
ECDSA_DS_DATE_REG	版本控制寄存器	0x00FC	R/W	N/A		

16.9 寄存器

本小节的所有地址均为相对于 ECDSA_DS 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 16.1. ECDSA_DS_CONF_REG (0x0004)



ECDSA_DS_WORK_MODE 配置 ECDSA_DS 的工作模式。

0：签名验证模式
1：签名生成模式
2：公钥导出模式
3：无效
(R/W)

ECDSA_DS_ECC_CURVE 配置使用的椭圆曲线。

0：P-192
1：P-256
2：P-384
3：SM2
(R/W)

ECDSA_DS_SOFTWARE_SET_Z 配置参数 z 的配置方式。

0：SHA 结果生成
1：软件写入
(R/W)

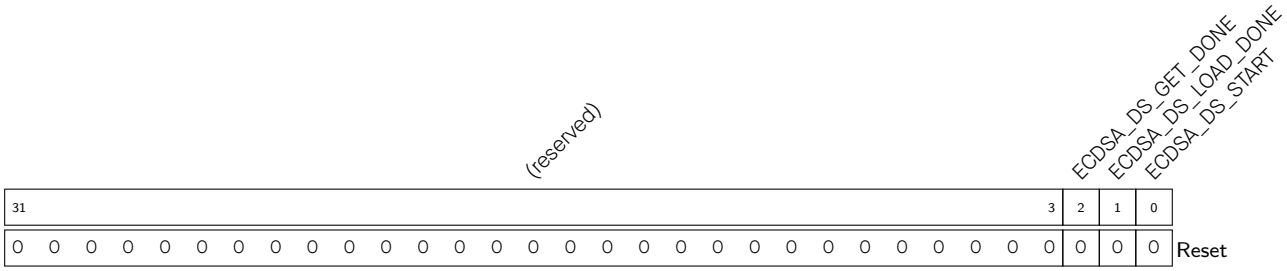
ECDSA_DS_DETERMINISTIC_K 配置参数 k 的配置方式。

0：TRNG 自动生成
1：由确定性推导算法生成
(R/W)

ECDSA_DS_USE_HARDWARE_KEY 配置使用软件密钥还是硬件密钥。

0：软件密钥
1：硬件密钥
(R/W)

Register 16.2. ECDSA_DS_START_REG (0x001C)



ECDSA_DS_START 配置是否启动 ECDSA_DS 运算。此位配置后自动清 0。

0：无效

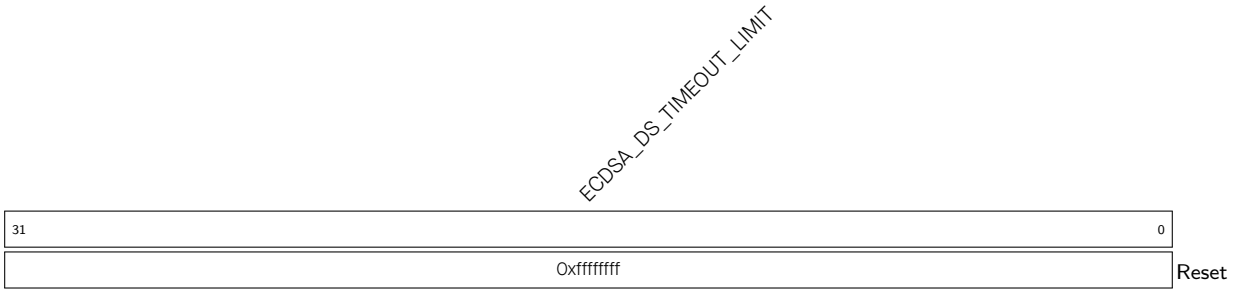
1：启动 ECDSA_DS 运算

(WT)

ECDSA_DS_LOAD_DONE 写 1 生成中断信号，表示 ECDSA_DS 的 LOAD 阶段已完成。此位配置后自动清 0。(WT)

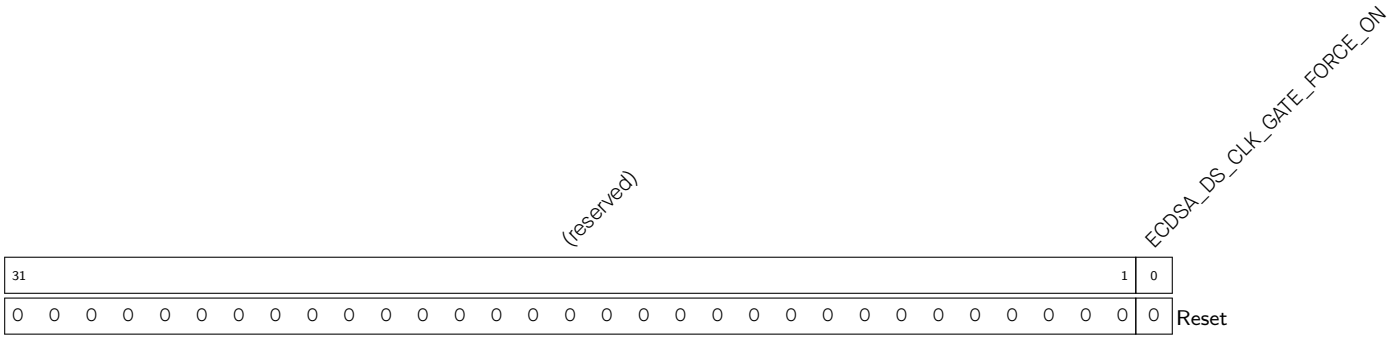
ECDSA_DS_GET_DONE 写 1 生成中断信号，表示 ECDSA_DS 的 GAIN 阶段已完成。此位配置后自动清 0。(WT)

Register 16.3. ECDSA_DS_TIMEOUT_LIMIT_REG (0x0028)



ECDSA_DS_TIMEOUT_LIMIT 配置等待共享加密加速器完成当前运算后再复用的超时上限。(R/W)

Register 16.4. ECDSA_DS_CLK_REG (0x0008)



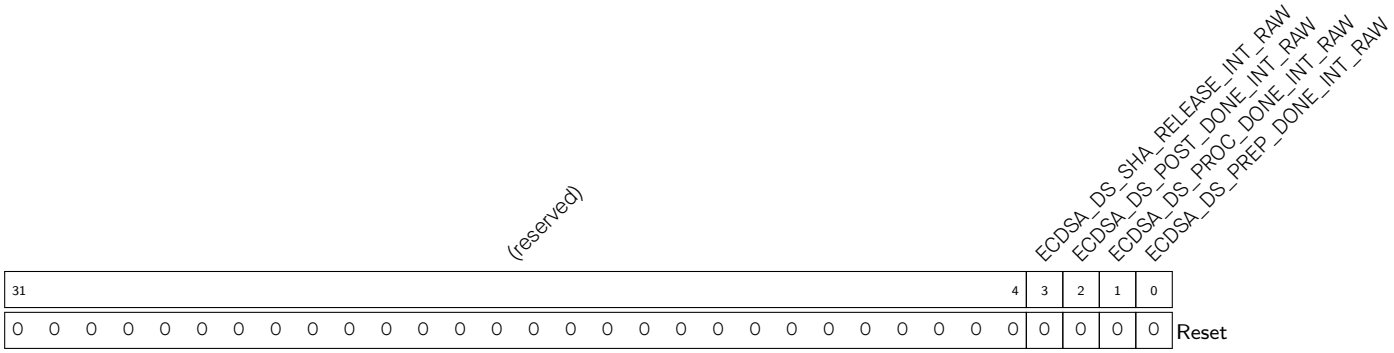
ECDSA_DS_CLK_GATE_FORCE_ON 配置是否强制启动 ECDSA_DS 内存的时钟门控。

0：无效

1：强制启动

(R/W)

Register 16.5. ECDSA_DS_INT_RAW_REG (0x000C)



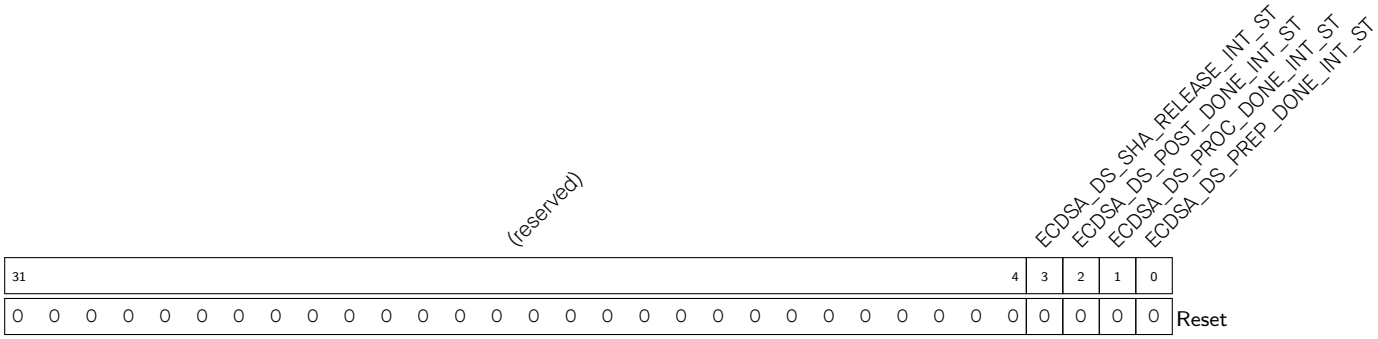
ECDSA_DS_PREP_DONE_INT_RAW ECDSA_DS_PREP_DONE_INT 的原始中断状态。(R/SS/WTC)

ECDSA_DS_PROC_DONE_INT_RAW ECDSA_DS_PROC_DONE_INT 的原始中断状态。(R/SS/WTC)

ECDSA_DS_POST_DONE_INT_RAW ECDSA_DS_POST_DONE_INT 的原始中断状态。(R/SS/WTC)

ECDSA_DS_SHA_RELEASE_INT_RAW ECDSA_DS_SHA_RELEASE_INT 的原始中断状态。(R/ SS/ WTC)

Register 16.6. ECDSA_DS_INT_ST_REG (0x0010)



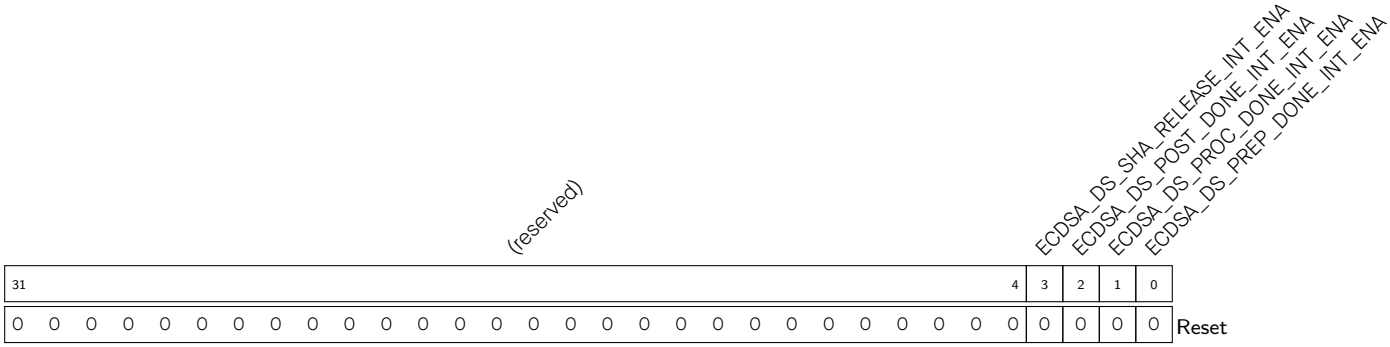
ECDSA_DS_PREP_DONE_INT_ST ECDSA_DS_PREP_DONE_INT 的屏蔽中断状态。(RO)

ECDSA_DS_PROC_DONE_INT_ST ECDSA_DS_PROC_DONE_INT 的屏蔽中断状态。(RO)

ECDSA_DS_POST_DONE_INT_ST ECDSA_DS_POST_DONE_INT 的屏蔽中断状态。(RO)

ECDSA_DS_SHA_RELEASE_INT_ST ECDSA_DS_SHA_RELEASE_INT 的屏蔽中断状态。(RO)

Register 16.7. ECDSA_DS_INT_ENA_REG (0x0014)



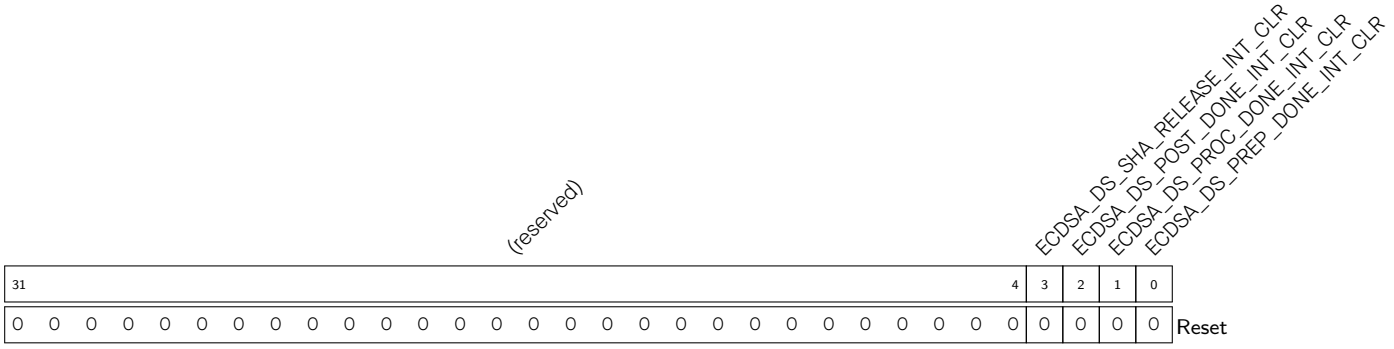
ECDSA_DS_PREP_DONE_INT_ENA 写 1 使能 ECDSA_DS_PREP_DONE_INT 中断。(R/W)

ECDSA_DS_PROC_DONE_INT_ENA 写 1 使能 ECDSA_DS_PROC_DONE_INT 中断。(R/W)

ECDSA_DS_POST_DONE_INT_ENA 写 1 使能 ECDSA_DS_POST_DONE_INT 中断。(R/W)

ECDSA_DS_SHA_RELEASE_INT_ENA 写 1 使能 ECDSA_DS_SHA_RELEASE_INT 中断。(R/W)

Register 16.8. ECDSA_DS_INT_CLR_REG (0x0018)



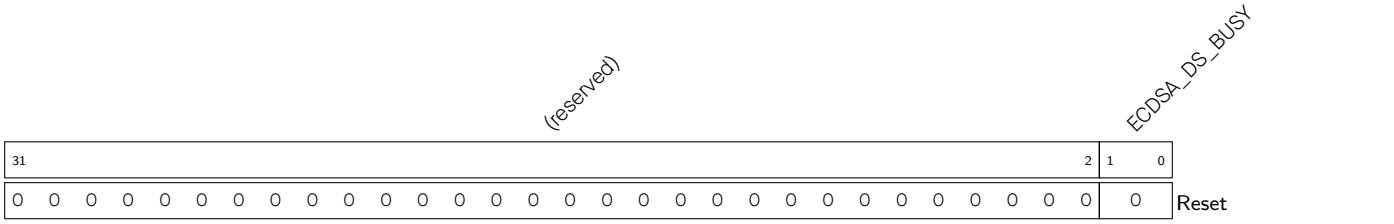
ECDSA_DS_PREP_DONE_INT_CLR 写 1 清除 ECDSA_DS_PREP_DONE_INT 中断。(WT)

ECDSA_DS_PROC_DONE_INT_CLR 写 1 清除 ECDSA_DS_PROC_DONE_INT 中断。(WT)

ECDSA_DS_POST_DONE_INT_CLR 写 1 清除 ECDSA_DS_POST_DONE_INT 中断。(WT)

ECDSA_DS_SHA_RELEASE_INT_CLR 写 1 清除 ECDSA_DS_SHA_RELEASE_INT 中断。(WT)

Register 16.9. ECDSA_DS_STATE_REG (0x0020)



ECDSA_DS_BUSY 表示 ECDSA_DS 的工作状态。

0：IDLE

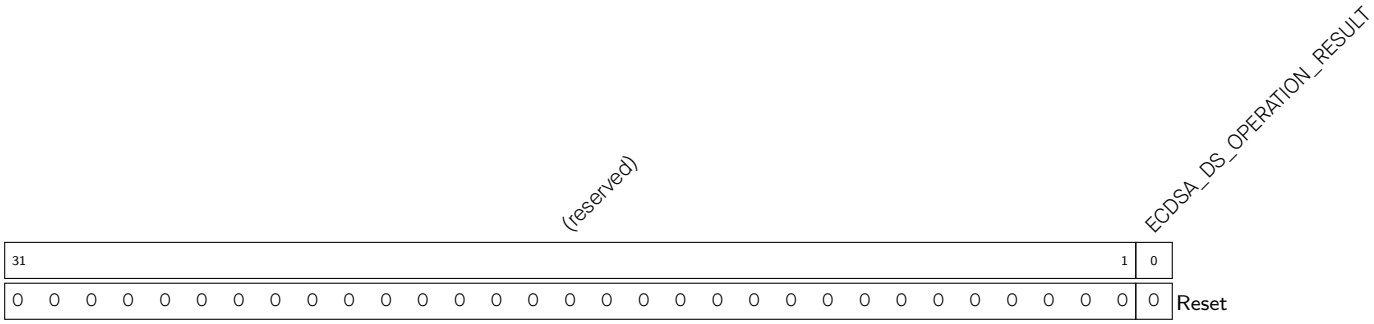
1：LOAD

2：GAIN

3：BUSY

(RO)

Register 16.10. ECDSA_DS_RESULT_REG (0x0024)



ECDSA_DS_OPERATION_RESULT 表示 ECDSA_DS 运算是否成功。

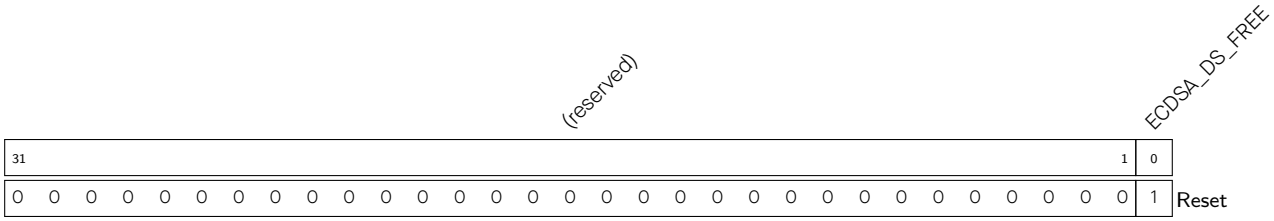
0：失败

1：成功

仅在 ECDSA_DS 运算完成后有效。

(RO/SS)

Register 16.11. ECDSA_DS_FREE_REG (0x002C)



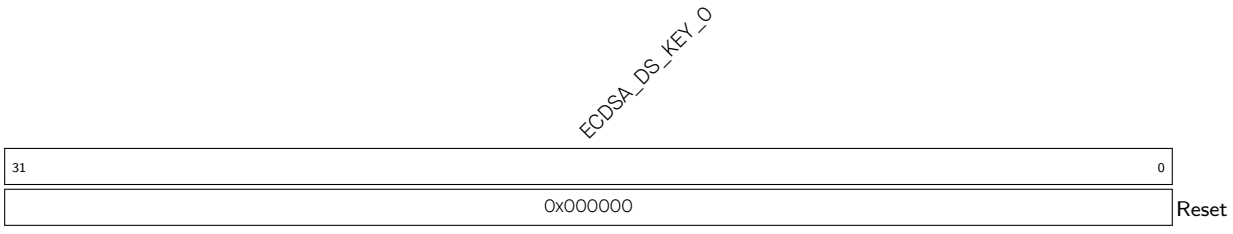
ECDSA_DS_FREE 配置 ECDSA_DS 在运算过程中是否可以被释放给其他模块。

0：不可释放

1：可释放

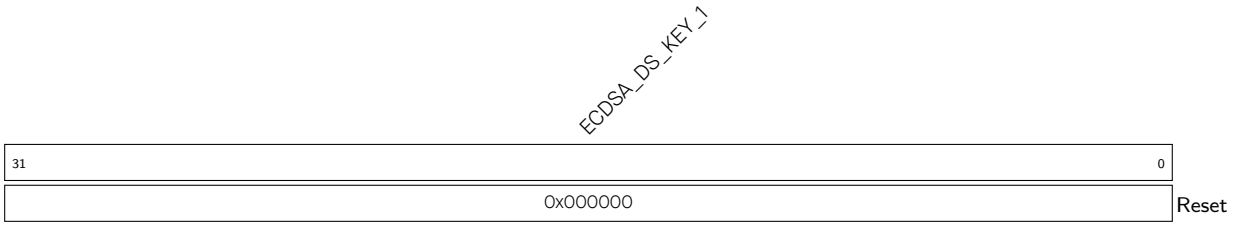
(R/W)

Register 16.12. ECDSA_DS_KEY_0_REG (0x0040)



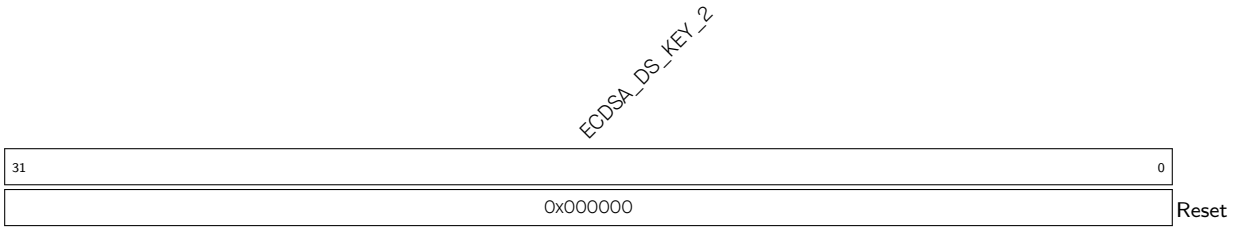
ECDSA_DS_KEY_0 存储密钥材料中的 key_0。(R/W)

Register 16.13. ECDSA_DS_KEY_1_REG (0x0044)



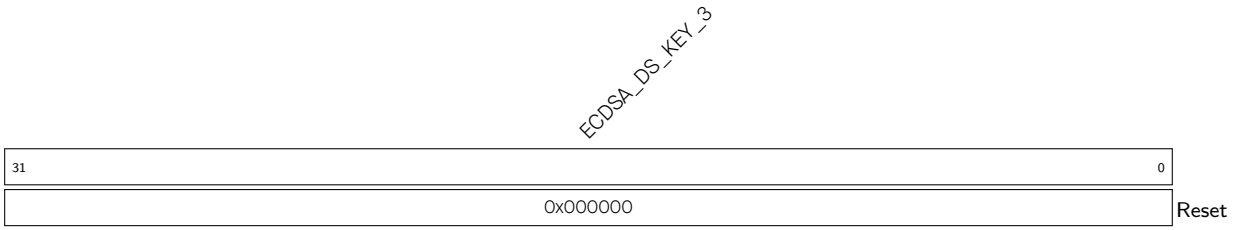
ECDSA_DS_KEY_1 存储密钥材料中的 key_1。(R/W)

Register 16.14. ECDSA_DS_KEY_2_REG (0x0048)



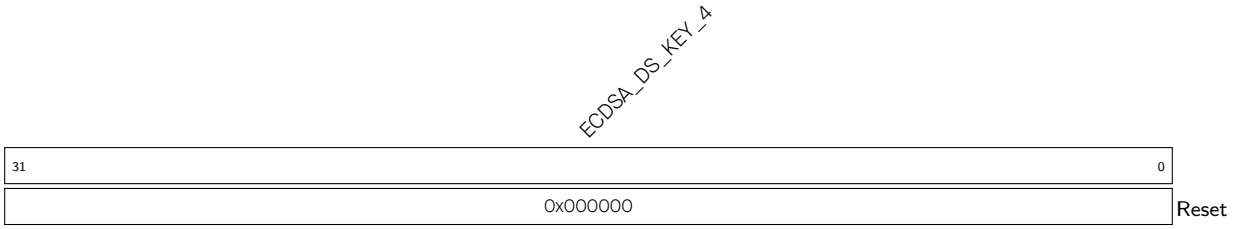
ECDSA_DS_KEY_2 存储密钥材料中的 key_2。(R/W)

Register 16.15. ECDSA_DS_KEY_3_REG (0x004C)



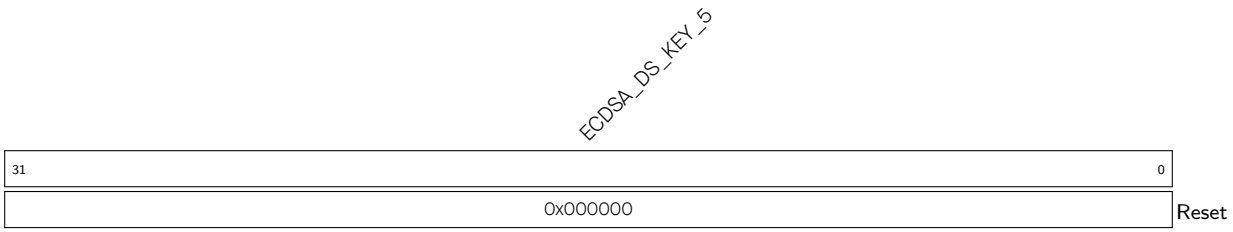
ECDSA_DS_KEY_3 存储密钥材料中的 key_3。(R/W)

Register 16.16. ECDSA_DS_KEY_4_REG (0x0050)



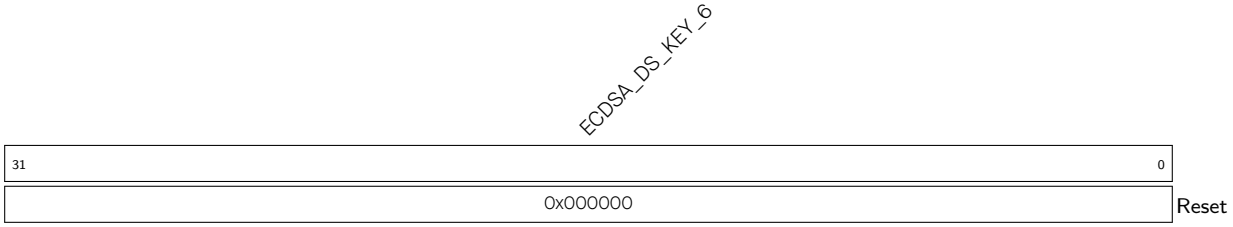
ECDSA_DS_KEY_4 存储密钥材料中的 key_4。(R/W)

Register 16.17. ECDSA_DS_KEY_5_REG (0x0054)



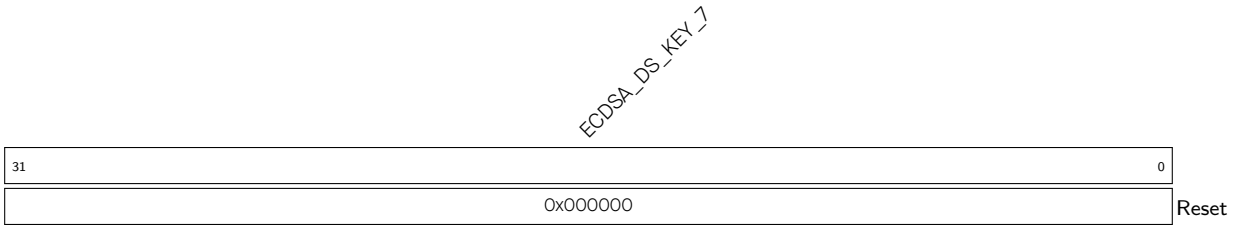
ECDSA_DS_KEY_5 存储密钥材料中的 key_5。(R/W)

Register 16.18. ECDSA_DS_KEY_6_REG (0x0058)



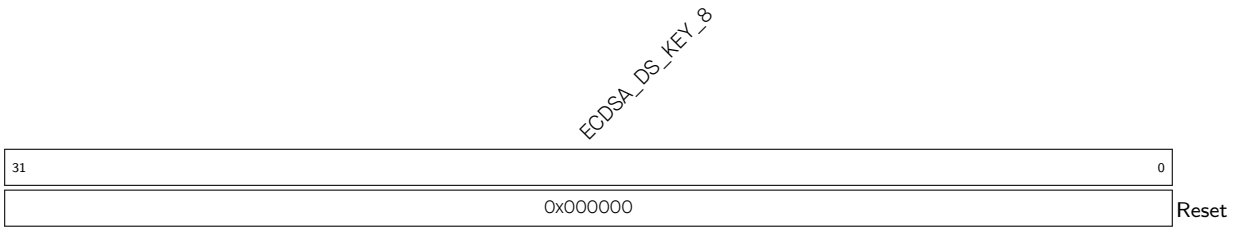
ECDSA_DS_KEY_6 存储密钥材料中的 key_6。(R/W)

Register 16.19. ECDSA_DS_KEY_7_REG (0x005C)



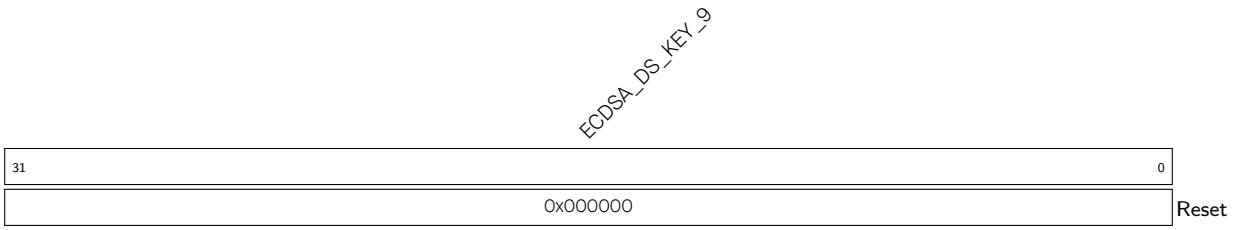
ECDSA_DS_KEY_7 存储密钥材料中的 key_7。(R/W)

Register 16.20. ECDSA_DS_KEY_8_REG (0x0060)



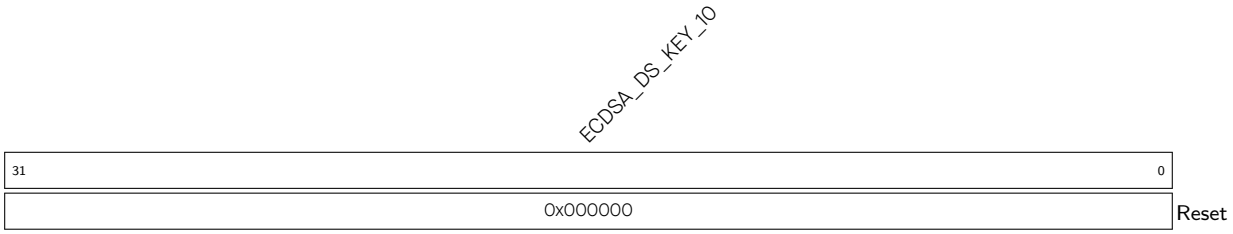
ECDSA_DS_KEY_8 存储密钥材料中的 key_8。(R/W)

Register 16.21. ECDSA_DS_KEY_9_REG (0x0064)



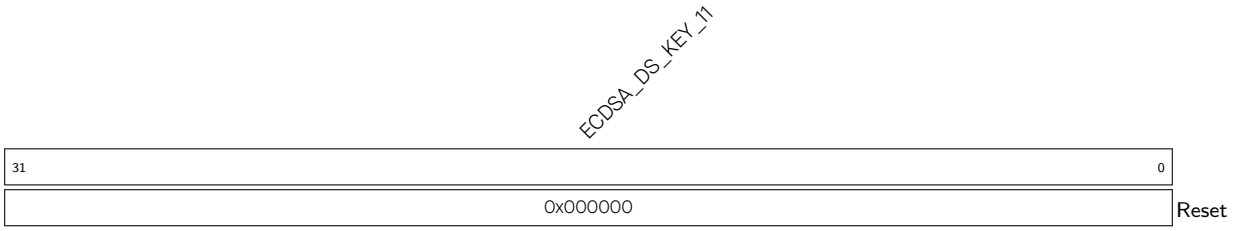
ECDSA_DS_KEY_9 存储密钥材料中的 key_9。(R/W)

Register 16.22. ECDSA_DS_KEY_10_REG (0x0068)



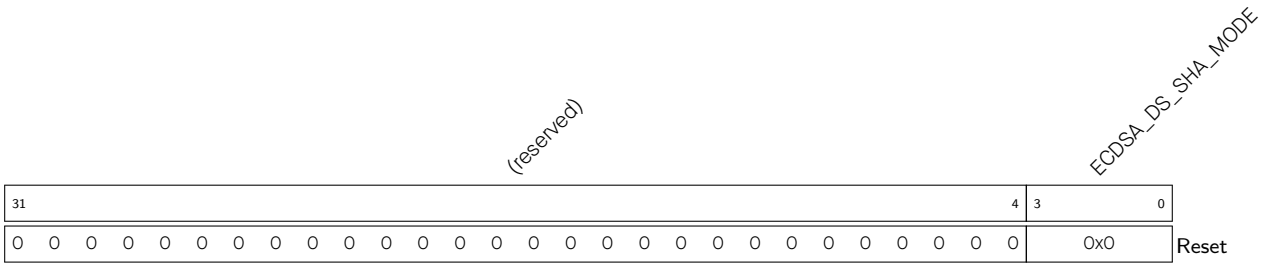
ECDSA_DS_KEY_10 存储密钥材料中的 key_10。(R/W)

Register 16.23. ECDSA_DS_KEY_11_REG (0x006C)



ECDSA_DS_KEY_11 存储密钥材料中的 key_11。(R/W)

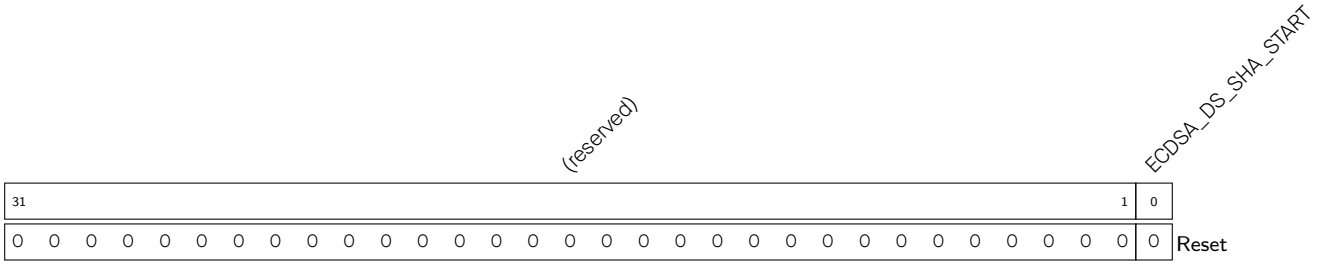
Register 16.24. ECDSA_DS_SHA_MODE_REG (0x0200)



ECDSA_DS_SHA_MODE 配置哈希运算的 SHA 算法。

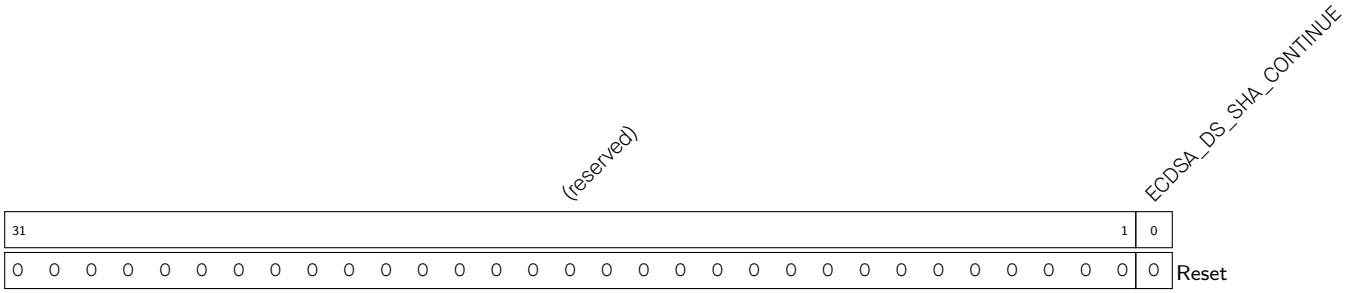
1：SHA-224
2：SHA-256
3：SHA-384
4：SHA-512
5：SHA-512-224
6：SHA-512-256
14：SM3
其他：无效
(R/W)

Register 16.25. ECDSA_DS_SHA_START_REG (0x0210)



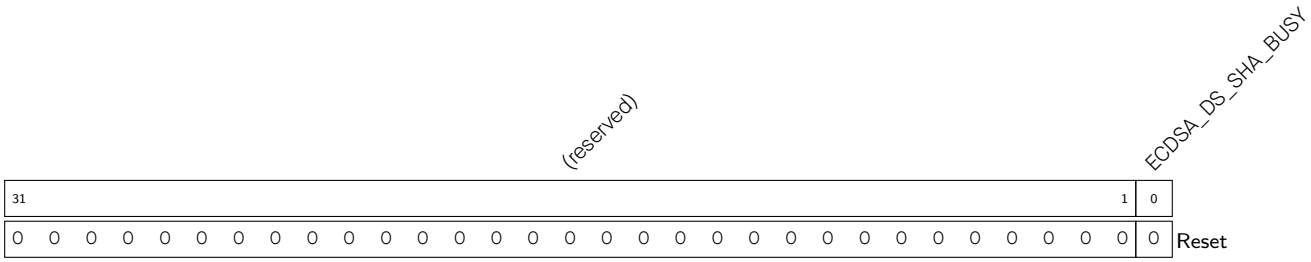
ECDSA_DS_SHA_START 写 1 启动 ECDSA_DS 运算中的首个 SHA 运算。此位配置后自动清 0。(WT)

Register 16.26. ECDSA_DS_SHA_CONTINUE_REG (0x0214)



ECDSA_DS_SHA_CONTINUE 写 1 启动 ECDSA_DS 运算中的后续 SHA 运算。此位配置后自动清 0。(WT)

Register 16.27. ECDSA_DS_SHA_BUSY_REG (0x0218)



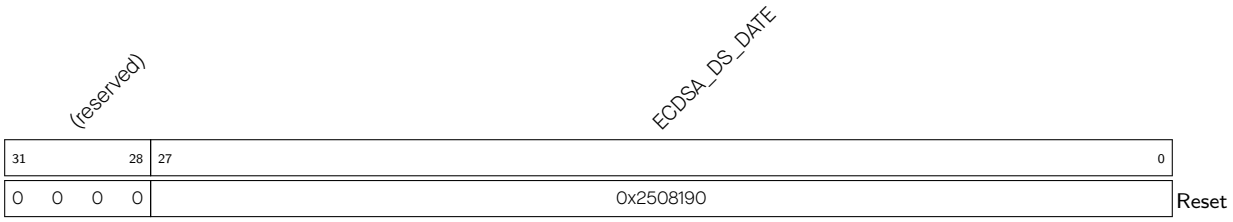
ECDSA_DS_SHA_BUSY 表示 ECDSA_DS 运算过程中 SHA 加速器的工作状态。

0：IDLE

1：BUSY

(RO)

Register 16.28. ECDSA_DS_DATE_REG (0x00FC)



ECDSA_DS_DATE ECDSA_DS 版本控制寄存器。(R/W)

第 IV 卷

图像和语音处理

该部分涉及芯片的多媒体处理功能，介绍了与图像和语音处理相关的组件，包括编解码器、像素处理加速器、音频采样率转换器以及摄像头控制器。

第 17 章

JPEG 图像编解码器

ESP32-S31 的 JPEG 图像编解码器遵循 JPEG 基线标准，支持对图像进行压缩（编码）和解压缩（解码）。该模块可降低传输带宽与存储空间占用，处理高分辨率图像。

编码流程主要包括离散余弦变换 (DCT)、Z 形 (Zigzag) 扫描、量化和哈夫曼编码。

解码流程为编码流程的逆操作，主要包括哈夫曼解码、反量化、反 Z 形扫描和反离散余弦变换。

17.1 术语

MCU	最小编码单元，即编码和解码的基础单位
基线 (baseline)	JPEG 标准规定的一种基于 DCT 的顺序编解码方式，在所有基于 DCT 的解码处理中都需用到
图像 (image)	整数二维数组集
分量 (component)	组成图像的二维数组之一
数据单元 (data unit)	一个分量中的 8 × 8 像素块
标记 (marker)	两字节码，第一个字节为十六进制的 0xFF，第二个字节为 1 至 0xFE 区间内的值，用于识别压缩数据格式的各个组成部分
恢复标记 (restart marker)	区分两个编码后数据段的标记，用于解码器再同步、错误恢复等特定用途
恢复间隔 (restart interval)	两个恢复标记间隔的整数个 MCU
扫描 (scan)	对图像中一个或多个分量数据的一次遍历处理

图 17.1-1 *JPEG 码流格式* 为 JPEG 码流的组成结构图。图中虚线框表示对应码流段的组成结构，橙色部分表示标记码。

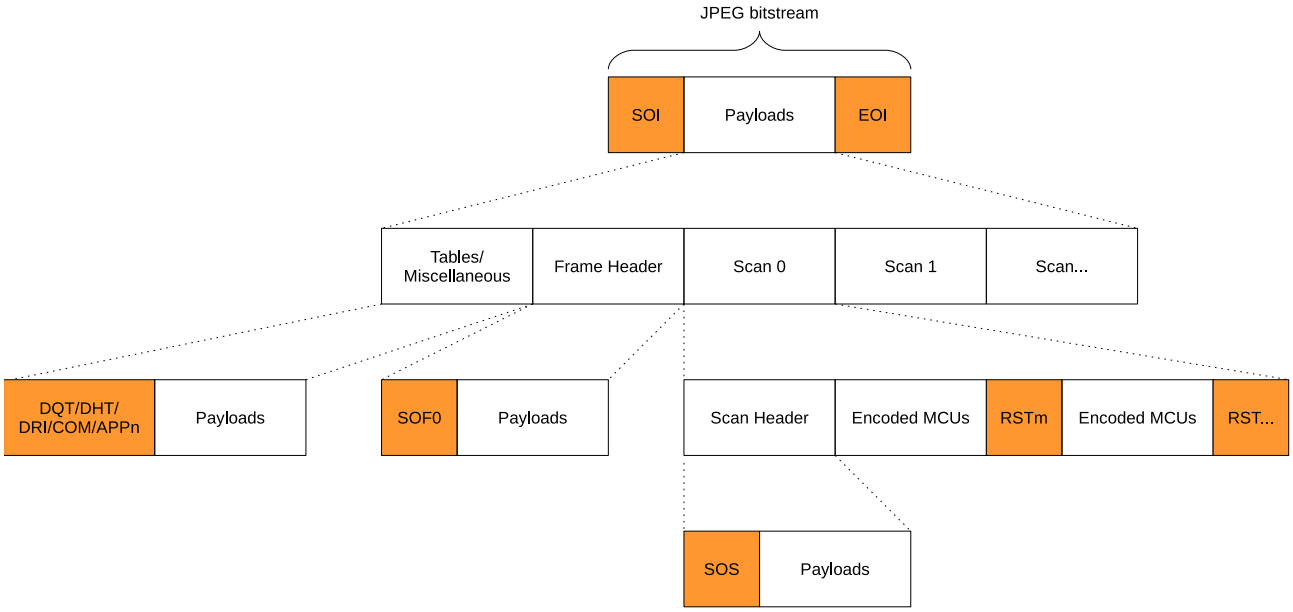


图 17.1-1. JPEG 码流格式

表 17.1-2 标记码描述 为码流中包含的各个标记码及其描述。

表 17.1-2. 标记码描述

代码	标记	描述
0xFFC0	SOF0	帧开始标记（基线 DCT）
0xFFC4	DHT	定义的哈夫曼表标记
0xFFDm	RSTm	恢复标记，模数为 8
0xFFD8	SOI	图像开始标记
0xFFD9	EOI	图像结束标记
0xFFDA	SOS	开始扫描标记
0xFFDB	DQT	定义的量化表标记
0xFFDD	DRI	定义的恢复间隔标记
0xFFEn	APPn	用于应用段的标记，“n” 为 0 到 F 区间内的值
0xFFFE	COM	注释标记

17.2 概述

ESP32-S31 的 JPEG 图像编解码器基于 JPEG 基线标准。该标准规定编码处理通过一次或多次扫描完成，每次扫描可处理一个或多个最小编码单元（MCU）。MCU 由数据单元序列构成；数据单元序列取决于各图像分量的采样因子。不同图像格式的 MCU 可能包含不同的数据单元序列，如表 17.2-1 不同图像格式 MCU 包含的数据单元 所示。

表 17.2-1. 不同图像格式 MCU 包含的数据单元

图像格式	MCU[1]		
	亮度分量 Y	色度分量 U	色度分量 V
YUV444	1 × 1	1 × 1	1 × 1

YUV422	2 × 1	1 × 1	1 × 1
YUV420	2 × 2	1 × 1	1 × 1
GRAY	1 × 1	无	无

[1] 每个图像格式的 MCU 可表示为：各分量水平方向数据单元数 × 垂直方向数据单元数。

同一时刻，JPEG 图像编解码器只能配置为编码器或解码器，不能同时作为编码器和解码器工作：

- 作为编码器使用时，通过 2D DMA 获取待压缩图像，将编码后的码流发送给 2D DMA；
- 作为解码器使用时，通过 2D DMA 获取编码后的码流，将解码后的图像发送给 2D DMA。

JPEG 图像编解码器的工作流程如图 17.2-1 *JPEG 图像编解码器的系统连接框图* 所示。

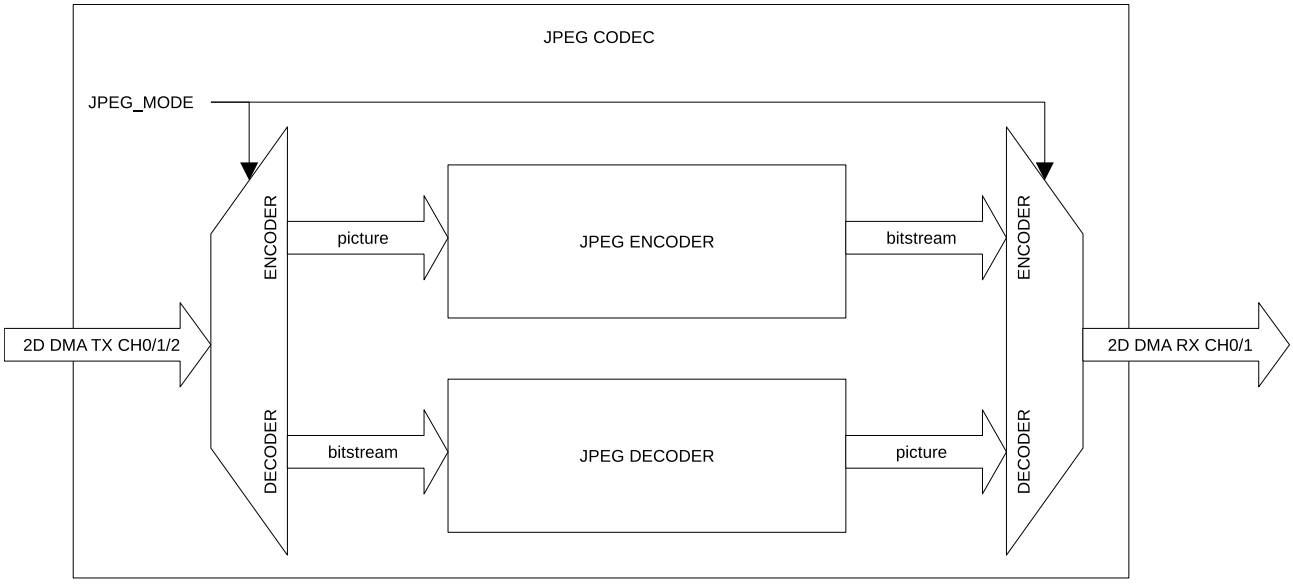


图 17.2-1. JPEG 图像编解码器的系统连接框图

JPEG 图像编解码器的功能时钟是 SYS_CLK。

17.3 主要特性

JPEG 图像编解码器作为编码器使用时，具有以下特性：

- 编码过程中暂停
- 使用离散余弦变换算法
- 使用范式哈夫曼编码
- 支持以下输入图像格式：
  - YUV444、YUV422、YUV420 和 GRAY
  - RGB888 和 RGB565（转换为 YUV444、YUV422 或 YUV420 后）
- 支持 4 个 8 位或 12 位精度的可配置量化系数表

- 性能：
  - 静态图像压缩最大支持 4K 分辨率
  - 动态图像压缩最大支持 720P@30 fps（不包括包头编码时间）
- 自动填充零字节
- 自动添加 EOI 标记
- 超时检测

JPEG 图像编解码器作为解码器使用时，具有以下特性：

- 使用反离散余弦变换算法
- 使用哈夫曼解码
- 支持 YUV444、YUV422、YUV420、GRAY 图像格式的压缩码流解码
- 支持解析 RST 标记
- 支持 4 个 8 位或 12 位精度的可配置量化系数表
- 支持 2 个 DC 和 2 个 AC 哈夫曼表
- 支持任意分辨率的图像解码。输出分辨率会按输入图像格式向上对齐，与码流中的原始分辨率可能不同：
  - YUV444、GRAY：水平和垂直分辨率均对齐至 8 的倍数。例如，150 × 150 的图像输出分辨率为 152 × 152
  - YUV422：水平分辨率对齐至 16 的倍数，垂直分辨率对齐至 8 的倍数。例如，150 × 150 的图像输出分辨率为 160 × 152
  - YUV420：水平和垂直分辨率均对齐至 16 的倍数。例如，150 × 150 的图像输出分辨率为 160 × 160
- 超时检测
- 性能：
  - 静态图像解码最大支持 4K 分辨率
  - 动态图像解码最大支持 720P@30 fps（不包括包头解析时间）

17.4 结构概览

JPEG 图像编解码器配置为编码器时，其架构如图 17.4-1 *JPEG 编码器架构* 所示。

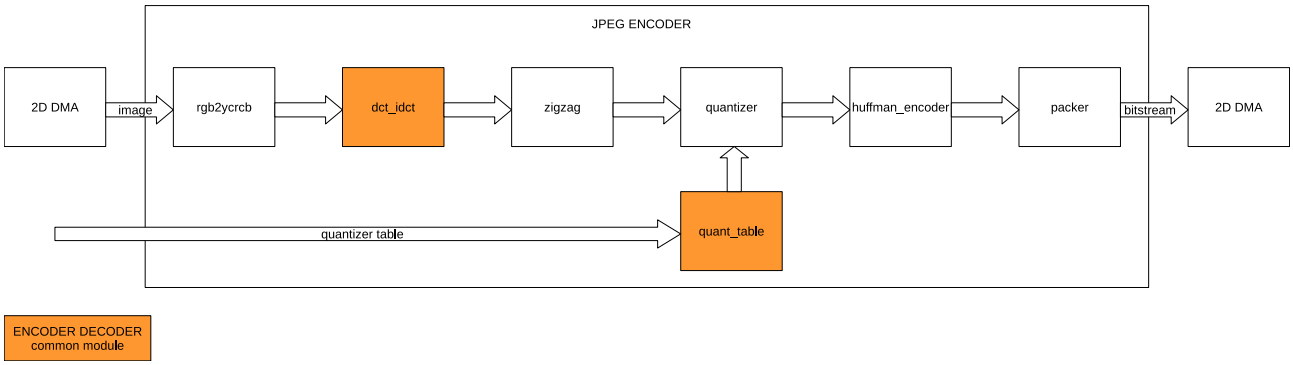


图 17.4-1. JPEG 编码器架构

- **rgb2ycrcb** 模块实现颜色空间转换，将原始图像转换为编解码器支持的待压缩图像格式
- **dct_idct** 模块对数据单元执行二维 DCT 变换
- **zigzag** 模块对 DCT 变换后的数据单元进行 Z 形扫描排序
- **quant_table** 模块存储软件下发的量化系数表
- **quantizer** 模块根据 **quant_table** 模块存储的量化系数表，对 Z 形排序后的数据进行量化
- **huffman_encoder** 模块对量化后的数据单元进行哈夫曼编码
- **packer** 模块将哈夫曼编码后的数据按 32 位为一组打包，输出编码码流

JPEG 图像编解码器配置为解码器时，其架构如图 17.4-2 *JPEG 解码器架构* 所示。

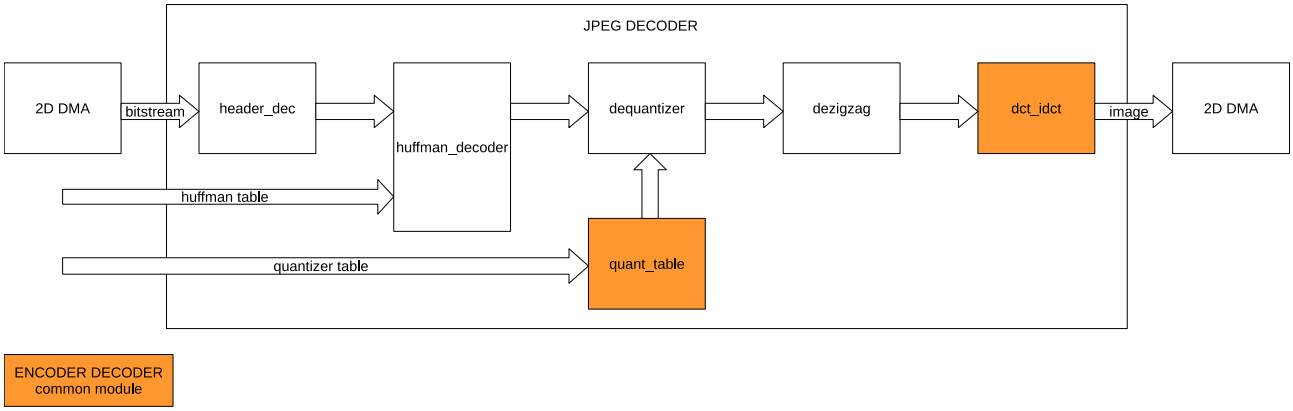


图 17.4-2. JPEG 解码器架构

- **header_dec** 模块提取码流并解析 SOS 和 EOI 标记，控制解码流程
- **huffman_decoder** 模块存储软件下发的哈夫曼表，并对码流进行哈夫曼解码
- **quant_table** 模块存储软件下发的量化系数表
- **dequantizer** 模块根据 **quant_table** 模块存储的量化系数表，对哈夫曼解码后的数据进行反量化
- **dezigzag** 模块对反量化后的数据进行反 Z 形扫描排序，还原为数据单元
- **dct_idct** 模块对反 Z 形扫描排序后的数据单元执行二维反 DCT 变换，输出解码后的数据单元

由于 JPEG 图像编解码器不能同时作为编码器和解码器使用，编码器和解码器可共用 **quant_table** 模块和 **dct_idct** 模块，以节省硬件资源。

17.5 功能描述

JPEG 图像编解码器可配置为编码器或解码器。

17.5.1 JPEG 编码器

17.5.1.1 暂停

在执行更高优先级任务时，JPEG 编码器可暂停。该功能通过 JPEG_PAUSE_EN 位控制。

- JPEG_PAUSE_EN 置 1 时，JPEG 编码器不再输出码流到 2D DMA 的接收通道

- JPEG_PAUSE_EN 清 0 时，JPEG 编码器恢复运行，正常输出码流

17.5.1.2 颜色空间转换

根据 JPEG 标准，JPEG 编码器支持的图像格式为 YUV444、YUV422、YUV420 和 GRAY。ESP32-S31 的 JPEG 编码器集成了颜色空间转换功能，可在编码前将 RGB888 和 RGB565 格式转换为编码器支持的上述图像格式，从而支持更多输入格式。

RGB 转换为 YUV 颜色空间的公式为：

$$Y = (0.299 \times R) + (0.587 \times G) + (0.114 \times B)$$
$$U = (-0.1687 \times R) + (-0.3313 \times G) + (0.5 \times B) + 128$$
$$V = (0.5 \times R) + (-0.4187 \times G) + (-0.0813 \times B) + 128$$

转换前的图像格式由格式和像素顺序表示。表 17.5-1 颜色空间转换支持的图像格式 列出了颜色空间转换支持的图像格式及其表示方式：

表 17.5-1. 颜色空间转换支持的图像格式

JPEG_COLOR_SPACE	格式[1]	JPEG_PIXEL_REV	像素顺序
0	RGB888	0	RGB
		1	BGR
1	YUV422[2]	0	YUYV
		1	UYVY
2	RGB565	0	RGB
		1	BGR
3	GRAY[2]	N/A	YYY
4	YUV444	0	YUV
		1	VUY
5	YUV420[2]	N/A	YYU/ YYV[3]

[1] 可通过配置寄存器字段 JPEG_SAMPLE_SEL，将图像格式转换为编码器支持的其他格式：
- 0：YUV444
- 1：YUV422
- 2：YUV420

[2] 图像格式转换存在以下限制：
- YUV422：仅可保持不变或下采样为 YUV420。将 JPEG_SAMPLE_SEL 配置为 0 无效。
- YUV420 或 GRAY：无论 JPEG_SAMPLE_SEL 取何值，均不进行转换。

[3] 奇数行图像为 YYU，偶数行图像为 YYV。

17.5.1.3 可配置量化系数表

ESP32-S31 的 JPEG 编码器硬件支持四个 8 × 8 量化系数表，每个表可以通过软件配置 64 个系数值。

每个量化表的精度可通过 JPEG_QNR_PRECISION 配置：

- 0：低 8 位有效
- 1：低 12 位有效

量化表系数值的配置模式可通过 JPEG_QNR_FIFO_EN 选择：

- 0：非 FIFO 模式，系数写入特定地址
- 1：FIFO 模式，所有系数写入同一地址

表 17.5-2. 量化系数表中系数的位置编号（示例）

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
...							
56	57	58	59	60	61	62	63

不同模式下，量化系数表的配置方式会有所不同。注意，*n* (*n* = 0, 1, 2, 3) 表示每个量化系数表的编号。

- 非 FIFO 模式：量化系数表 *n* 中的所有量化系数必须按位置写入到特定地址，即（JPEG 图像编解码器基地址 + *n* * 0x100 + 量化系数位置编号 * 0x4），其中 量化系数位置编号 可以是 0、1、2、...、63（详见表 17.5-2 量化系数表中系数的位置编号（示例））。在该模式下，由于每个系数都写入到特定地址，因此不需要按顺序配置这 64 个量化系数。
- FIFO 模式：量化系数表 *n* 中的 64 个量化系数全部写入相同的地址，即 JPEG_T*n*QNR_REG。在该模式下，由于所有系数都写入相同的地址，因此需要按顺序配置这 64 个量化系数。然后，硬件将自动按照表 17.5-2 量化系数表中系数的位置编号（示例） 中的顺序存储这 64 个量化系数。

若实际使用的量化表少于四个，未使用的量化表无需配置。

要将特定量化系数表 *n* 用于亮度分量或色度分量，只需将相应的寄存器字段设置为 *n*：

- 亮度分量：JPEG_LQNR_TBL_SEL
- 色度分量：JPEG_CQNR_TBL_SEL

表 17.5-3 推荐用于亮度分量的 8 位精度量化系数表 和表 17.5-4 推荐用于色度分量的 8 位精度量化系数表 分别为 JPEG 基线标准推荐用于亮度分量和色度分量的 8 位精度量化系数表。

表 17.5-3. 推荐用于亮度分量的 8 位精度量化系数表

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

表 17.5-4. 推荐用于色度分量的 8 位精度量化系数表

17	18	24	47	99	99	99	99

18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

详见 ITU T.81 Recommendation。调整量化系数表会改变压缩比（未压缩图像大小与压缩图像大小之比）和编码图像质量。量化系数越大，压缩比越高，图像质量越低。

17.5.1.4 填充零字节

JPEG 基线标准要求在哈夫曼编码段内避免使用任何标记。哈夫曼编码生成的每个 0xFF 字节后须填充一个 0x00 字节。

JPEG 编码器实现了该功能，可在码流中哈夫曼编码段的 0xFF 字节后自动填充 0x00 字节。

是否在每个 0xFF 字节后添加 0x00 字节可通过 JPEG_FF_CHECK_EN 配置：

- 0：不添加
- 1：添加（默认）

17.5.1.5 EOI 标记

图像结束（EOI）标记用于标记压缩图像码流的结束。EOI 标记代码为 0xFFD9。

JPEG 编码器可以在码流末尾自动添加 EOI 标记。

是否添加 EOI 标记可通过 JPEG_TAILER_EN 配置：

- 0：不添加
- 1：添加

17.5.1.6 超时检测

JPEG 图像编解码器用作编码器时支持超时检测。出现以下情况时，硬件会触发超时中断 JPEG_ENCODE_TIMEOUT_INT：

- 编码器无法将编码后的码流写入 2D DMA，持续时间超过 $(2^{JPEG_ENCODE_TIMEOUT_THRES} - 1)$ 个时钟周期

17.5.2 JPEG 解码器

码流中的信息须先由软件解析，再通过 APB 配置总线发送给硬件。注意，硬件不支持解析 SOS、RST 和 EOI 标记段以外的任何标记段。

17.5.2.1 多个分量

帧头包含图像分量数量、各分量的 ID，以及各分量水平和垂直方向的采样因子等信息。

JPEG 解码器的软件须解析上述信息，并写入相应寄存器 JPEG_COMPONENT_NUM、JPEG_C*n*_ID、JPEG_C*n*_X_FACTOR 和 JPEG_C*n*_Y_FACTOR。此处 *n* 可为 0、1、2，表示硬件最多支持 3 个分量（如 YUV 的 Y、U、V）。若实际使用的分量少于 3 个，未使用的分量对应的寄存器无需配置。

硬件根据各个分量寄存器 JPEG_C*n*_ID 的值，检查其是否与扫描头信息中的 ID 一致。采样因子表示一个 MCU 在水平和垂直方向上包含该分量的数据单元数。根据 JPEG_COMPONENT_NUM、JPEG_C*n*_X_FACTOR 和 JPEG_C*n*_Y_FACTOR 字段的值，硬件可确定各分量的比例关系，即待解码图像的格式。详见表 17.5-5 采样因子和图像格式。

表 17.5-5. 采样因子和图像格式

JPEG_COMPONENT_NUM	JPEG_C*n*_X/Y_FACTOR	待解码图像的格式
0	-	无效
1	C0_X_FACTOR 为 1 C0_Y_FACTOR 为 1	GRAY
2	-	无效
3	C0_X_FACTOR 为 1 C0_Y_FACTOR 为 1 C1_X_FACTOR 为 1 C1_Y_FACTOR 为 1 C2_X_FACTOR 为 1 C2_Y_FACTOR 为 1	YUV444
	C0_X_FACTOR 为 2 C0_Y_FACTOR 为 1 C1_X_FACTOR 为 1 C1_Y_FACTOR 为 1 C2_X_FACTOR 为 1 C2_Y_FACTOR 为 1	YUV422
	C0_X_FACTOR 为 2 C0_Y_FACTOR 为 2 C1_X_FACTOR 为 1 C1_Y_FACTOR 为 1 C2_X_FACTOR 为 1 C2_Y_FACTOR 为 1	YUV420

无论待解码图像为何种格式，解码器仅支持一次扫描所有分量。解码完成后，图像将发送至 2D DMA。各解码图像格式的布局请参阅章节 4 *2D-DMA 控制器 (2D-DMA)*。

17.5.2.2 解析 RST 标记

扫描段比特流中可能包含 RST 标记。RST 标记的代码为 0xFFD*x* (*x* = 0, 1, ..., 7)。

JPEG 解码器的硬件可以解析 RST 标记，检查两个 RST 标记之间实际的 MCU 个数是否与 Tables/Miscellaneous 段中 DRI 定义的恢复间隔相同。

首先，软件会解析 DRI 段，并将定义的恢复间隔传递给 JPEG_RESTART_INTERVAL 字段。之后，硬件每解码 JPEG_RESTART_INTERVAL 个 MCU，就会检查后续码流是否包含 RST 标记。

如果 JPEG_RESTART_INTERVAL 配置为 0，则码流中不包含 RST 标记，此时硬件将不检查 RST 标记。

17.5.2.3 可配置量化系数表

ESP32-S31 JPEG 解码器同样支持 4 个可配置量化系数表。由于 JPEG 图像编解码器不能同时用作编码器和解码器，解码器可以和编码器共用相同的 4 个可配置系数表，从而节省硬件资源。

解码器量化系数表的配置与编码器的（详见章节 17.5.1.3 可配置量化系数表）类似，但有以下不同：

- 软件需要解析 Tables/Miscellaneous 段中的 DQT 段，获取解码器的量化系数表。推荐的解码器量化系数表与编码器相同，如表 17.5-3 推荐用于亮度分量的 8 位精度量化系数表 和表 17.5-4 推荐用于色度分量的 8 位精度量化系数表 所示。
- 量化系数表 *n* 的精度可以通过 JPEG_T*n*_DQT_INFO 字段的位 [4] 配置：
  - 0：低 8 位有效
  - 1：高 12 位有效

  精度也是从 Tables/Miscellaneous 段中的 DQT 段获取。
- 每个分量可选择不同的量化系数表。JPEG_T*n*_DQT_INFO 字段的低 4 位配置量化系数表的 ID，JPEG_C*n*_DQT_TBL_SEL 字段配置每个分量选择的量化系数表 ID。每个量化系数表的 ID 也从 Tables/Miscellaneous 段中的 DQT 段获取。

17.5.2.4 可配置哈夫曼表

ESP32-S31 支持最多 2 个 DC 哈夫曼表和 2 个 AC 哈夫曼表，可在硬件支持范围内按需配置。

每个哈夫曼表可以包含三类信息，这三类信息可从 Tables/Miscellaneous 段中的 DHT 段获取：

- 长度为 1 到 16 位的码字数量
- 解码后码字对应的符号
  - DC 表：最多 16 个符号
  - AC 表：最多 256 个符号
- 长度为 1 到 16 位的最小码字

DC 和 AC 哈夫曼表都可以通过 JPEG_DHT_FIFO_EN 选择配置模式：

- 0：非 FIFO 模式
- 1：FIFO 模式

不同模式下，哈夫曼表的配置方式会有所不同。以下是 DC 表*n* (*n* = 0, 1) 的配置方式：

- 非 FIFO 模式：DC*n* 哈夫曼表的不同信息必须按信息类型写入到特定地址。
  - 长度为 1 到 16 位的码字数量：JPEG 图像编解码器基地址 + 0x500 + *n* * 0x80 + （码字长度 - 1）* 0x4，其中 码字长度 = 1、2、...、16。
  - 解码后码字对应的符号：JPEG 图像编解码器基地址 + 0x600 + *n* * 0x40 + 符号位置 * 0x4，其中 符号位置 = 0、1、...、15。
  - 长度为 1 到 16 位的最小码字：JPEG 图像编解码器基地址 + 0xF00 + *n* * 0x80 + （码字长度 - 1）* 0x4，其中 码字长度 = 1、2、...、16。

  在该模式下，由于信息按类型都写入到特定地址，因此不需要按顺序配置。
- FIFO 模式：DC*n* 哈夫曼表同一类型的信息全部写入到同个地址。

  - 长度为 1 到 16 位的码字数量：JPEG_DHT_TOTLEN_DC*n*_REG。
  - 解码后码字对应的所有符号（DC 表最多 16 个符号）：JPEG_DHT_VAL_DC*n*_REG。
  - 长度为 1 到 16 位的最小码字：JPEG_DHT_CODEMIN_DC*n*_REG。

  在该模式下，由于同一类型的信息全部写入到同个地址，因此需要按顺序配置同一类型的信息。之后，硬件将自动按顺序存储这些信息。

AC*n* (*n* = 0, 1) 表的配置方式与 DC*n* (*n* = 0, 1) 表类似。以下是 AC*n* (*n* = 0, 1) 的配置方式：

- 非 FIFO 模式：AC*n* 哈夫曼表的不同信息必须按信息类型写入到特定地址。
  - 长度为 1 到 16 位的码字数量：JPEG 图像编解码器基地址 + 0x540 + *n* * 0x80 + （码字长度 - 1）* 0x4，其中 码字长度 = 1、2、...、16。
  - 解码后码字对应的符号：JPEG 图像编解码器基地址 + 0x680 + *n* * 0x400 + 符号位置 * 0x4，其中 符号位置 = 0、1、...、255。
  - 长度为 1 到 16 位的最小码字：JPEG 图像编解码器基地址 + 0xF40 + *n* * 0x80 + （码字长度 - 1）* 0x4，其中 码字长度 = 1、2、...、16。

  在该模式下，由于信息按类型都写入到特定地址，因此不需要按顺序配置。
- FIFO 模式：AC*n* 哈夫曼表同一类型的信息全部写入到同个地址。
  - 长度为 1 到 16 位的码字数量：JPEG_DHT_TOTLEN_AC*n*_REG。
  - 解码后码字对应的所有符号（AC 表最多 256 个符号）：JPEG_DHT_VAL_AC*n*_REG。
  - 长度为 1 到 16 位的最小码字：JPEG_DHT_CODEMIN_AC*n*_REG。

  在该模式下，由于同一类型的信息全部写入到同个地址，因此需要按顺序配置同一类型的信息。之后，硬件将自动按顺序存储这些信息。

长度为 1 到 16 位的最小码字应该向左偏移到 16 位配置值的高位。如果没有码字长度，则配置值为 0xFFFF。例如，如果最小码字为 0x7C，则 16 位配置值应为 0x7C00。

JPEG 基线标准推荐的 DC 和 AC 哈夫曼表，可在 ITU T.81 Recommendation 中找到。

软件通过解析 Tables/Miscellaneous 段中的 DHT 段获取 DC*n* 和 AC*n* 哈夫曼表的 ID，并将 ID 写入相应字段 JPEG_DC*n*_DHT_ID 和 JPEG_AC*n*_DHT_ID。

硬件会自动使用与 JPEG_DC*n*_DHT_ID 和 JPEG_AC*n*_DHT_ID ID 值相同的 DC*n* 和 AC*n* 哈夫曼表。

17.5.2.5 超时检测

JPEG 图像编解码器用作解码器时支持超时检测。出现以下情况时，硬件会触发超时中断 JPEG_DECODE_TIMEOUT_INT：

- 解码器停止读取码流超过 $(2^{JPEG_DECODE_TIMEOUT_THRES} - 1)$ 个时钟周期

超时中断触发后，解码器会根据 JPEG_DECODE_TIMEOUT_TASK_SEL 的配置复位：

- 0：硬件不会执行任何操作，需要软件配置 JPEG_SOFT_RST 或 JPEG_FSM_RST 来执行复位操作。
- 1：硬件将自动复位解码器，从而退出超时状态并进入初始状态。

17.6 中断

ESP32-S31 的 JPEG 图像编解码器可以生成 JPEG_INTR 中断信号并发送给中断矩阵。

JPEG_INTR 中断信号由下列中断源生成：

- JPEG_DONE_INT：当一帧图像编码或解码结束时触发。
- JPEG_RLE_PARALLEL_ERR_INT：当游程编码出错时触发。
- JPEG_CID_ERR_INT：当解码得到的各个分量 ID 和软件配置的各个分量 ID 不同时触发。
- JPEG_C_DHT_DC_ID_ERR_INT：当解码得到的各个分量 DC 哈夫曼表 ID 不是软件配置的 DC0 哈夫曼表 ID 或 DC1 哈夫曼表 ID 时触发。
- JPEG_C_DHT_AC_ID_ERR_INT：当解码得到的各个分量 AC 哈夫曼表 ID 不是软件配置的 AC0 哈夫曼表 ID 或 AC1 哈夫曼表 ID 时触发。
- JPEG_C_DQT_ID_ERR_INT：当解码得到的各个分量量化表 ID 和软件配置的各个分量量化表 ID 不同时触发。
- JPEG_RST_UXP_ERR_INT：当软件配置的 JPEG_RESTART_INTERVAL 为 0 但解码器解析到 RST 标记时触发。
- JPEG_RST_CHECK_NONE_ERR_INT：当软件配置的 JPEG_RESTART_INTERVAL 非 0 但解码器解析不到 RST 标记时触发。
- JPEG_RST_CHECK_POS_ERR_INT：当两个解析到的 RST 标记之间的 MCU 数不等于软件配置的 JPEG_RESTART_INTERVAL 时触发。
- JPEG_OUT_EOF_INT：当从 2D DMA 的 TX 通道读取到 EOF 标志时触发。
- JPEG_SR_COLOR_MODE_ERR_INT：该中断无效。
- JPEG_DCT_DONE_INT：当一个数据单元 DCT 或 IDCT 计算完成时触发。
- JPEG_BS_LAST_BLOCK_EOF_INT：当最后一个数据单元编码完成时触发。
- JPEG_SCAN_CHECK_NONE_ERR_INT：当一帧图像有多个待解码扫描并且在任意扫描头信息中（JPEG_SOS_CHECK_BYTE_NUM + 1）个字节都未解析到 SOS 标记时触发。
- JPEG_SCAN_CHECK_POS_ERR_INT：当解码器解析到的扫描头信息位置错误时触发。
- JPEG_UXP_DET_INT：当解码器解析到的标记不是硬件支持的标记时触发。
- JPEG_EN_FRAME_EOF_ERR_INT：当从 2D DMA 读取到的待编码数据单元数量小于根据软件配置的图像分辨率计算出的数据单元数量时触发。
- JPEG_EN_FRAME_EOF_LACK_INT：当从 2D DMA 读取完一帧待编码图像但未读取到 EOF 标志时触发。
- JPEG_DE_FRAME_EOF_ERR_INT：当解码完一帧图像得到的数据单元数量和根据软件配置的图像分辨率计算出的数据单元数量不一致时触发。
- JPEG_DE_FRAME_EOF_LACK_INT：当从 2D DMA 读取完一帧图像的码流但未读到 EOF 标志或 EOI 标记时触发。
- JPEG_SOS_UNMATCH_ERR_INT：当解码器解析到的扫描头信息出现以下任一情况时触发：
  - 分量数为 0

  - 头长度与实际不符

- JPEG_MARKER_ERR_FST_SCAN_INT：当解码器解析到的第一个扫描头信息中有错时触发。错误包括触发以下中断的错误：
  - JPEG_CID_ERR_INT
  - JPEG_C_DHT_DC_ID_ERR_INT
  - JPEG_C_DHT_AC_ID_ERR_INT
  - JPEG_C_DQT_ID_ERR_INT
  - JPEG_SOS_UNMATCH_ERR_INT
- JPEG_MARKER_ERR_OTHER_SCAN_INT：当解码器解析到的非第一个扫描头信息中有错时触发，错误类型与触发 JPEG_MARKER_ERR_FST_SCAN_INT 的错误相同。
- JPEG_UNDET_INT：当从 2D DMA 读取完一帧图像的码流但未读到过 SOS 标记时触发。
- JPEG_DECODE_TIMEOUT_INT：当解码器超时时触发。
- JPEG_ENCODE_TIMEOUT_INT：当编码器超时时触发。

上述中断源仅在对应中断使能寄存器 中断源名称_ENA 置 1 时才能触发。向 中断源名称_CLR 写 1 可清除中断。

> **说明：**
> 中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 17.8 寄存器列表。

17.7 编程流程

17.7.1 JPEG 编码器

JPEG 图像编解码器作为编码器使用时，配置流程为：

1. 激活 JPEG 图像编解码器
   - 将 HP_SYS_CLKRST_JPEG_SYS_CLK_EN 置 1，使能 JPEG 图像编解码器时钟。
   - 将 HP_SYS_CLKRST_RST_EN_JPEG 置 0，释放 JPEG 图像编解码器系统级复位。
2. 配置 JPEG 图像编解码器
   - 将 JPEG_SOFT_RST 置 1，复位 JPEG 图像编解码器；再将 JPEG_SOFT_RST 置 0，释放复位。
   - 将 JPEG_MODE 置 0，将 JPEG 图像编解码器配置为编码器。
   - 配置以下字段，选择原始图像格式：
     - 格式：JPEG_COLOR_SPACE
     - 像素顺序：JPEG_PIXEL_REV

     详见表 17.5-1 颜色空间转换支持的图像格式。

- 配置 JPEG_TAILER_EN，选择是否在码流末尾添加 EOI 标记（0xFFD9）：
  - 0：禁用
  - 1：使能
- 配置 JPEG_SAMPLE_SEL 选择待压缩图像格式：
  - 0：YUV444
  - 1：YUV422
  - 2：YUV420

  注意：该字段仅在原始图像格式为 RGB 时有效。详见表 17.5-1 颜色空间转换支持的图像格式。
- 将 JPEG_HA 配置为原始图像宽度，JPEG_VA 配置为原始图像高度。
- 参考章节 17.5.1.3 可配置量化系数表 的说明配置量化系数表。
- 参考章节 17.5.1.6 超时检测 的说明配置超时检测功能。

3. 配置 2D DMA（具体配置方法和相关概念介绍，请参考章节 *4 2D-DMA 控制器 (2D-DMA)*。）

(a) 配置 2D DMA TX 通道的链表：

- 将 2DEN 置 1，使能 2D 功能。
- 将 Buffer address pointer 配置为存储原始图像的起始地址。
- 将 pbyte 配置为原始图像一个像素的字节数。
- 将 HA 配置为原始图像宽度，VA 配置为原始图像高度。
- 将 hb 配置为一个图像块水平方向的像素个数，vb 配置为一个图像块垂直方向的像素个数。
- 将 mod 配置为 1，表示连续读取 hb × vb 大小图像块。
- 将 (X，Y) 配置为起始图像块的第一个像素坐标。根据原始图像格式和待压缩图像格式，hb 和 vb 须满足表 17.7-1 *JPEG 编码器 hb 和 vb 配置* 中的限制：

表 17.7-1. JPEG 编码器 hb 和 vb 配置

原始图像格式	待压缩图像格式	hb	vb
RGB	YUV444	8 的倍数	8 的倍数
RGB	YUV422	16 的倍数	8 的倍数
RGB	YUV420	16 的倍数	16 的倍数
YUV422	YUV422	16 的倍数	8 的倍数
GRAY	GRAY	8 的倍数	8 的倍数

(b) 配置 2D DMA RX 通道的链表：

- 将 2DEN 置 0，使能 1D 功能。
- 将 Buffer address pointer 配置为存储编码码流的起始地址。
- 将 size 配置为存储编码码流空间的字节数。
- 将 mod 配置为 0。2DEN 置 0 时，mod 配置无效。

(c) 连接 JPEG 图像编解码器和 2D DMA 通道：

- 将 DMA2D_OUT_PERI_SEL_CH*n* 配置为 0，连接 JPEG 图像编解码器和 TX*n*；
- 将 DMA2D_IN_PERI_SEL_CH*n* 配置为 0，连接 JPEG 图像编解码器和 RX*n*。

(d) 复位 2D DMA 通道：

- 2D DMA TX 通道*n*：将 DMA2D_OUT_RST_CH*n* 置 1，复位 2D DMA TX 通道*n*，再将 DMA2D_OUT_RST_CH*n* 置 0，释放 2D DMA TX 通道*n* 复位。
- 2D DMA RX 通道*n*：将 DMA2D_IN_RST_CH*n* 置 1，复位 2D DMA RX 通道*n*，再将 DMA2D_IN_RST_CH*n* 置 0，释放 2D DMA RX 通道*n* 复位。

(e) 选择 2D DMA 通道的突发长度：

- 配置 DMA2D_OUT_MEM_BURST_LENGTH_CH*n*，选择 2D DMA TX 通道*n* 的突发长度。
- 配置 DMA2D_IN_MEM_BURST_LENGTH_CH*n*，选择 2D DMA RX 通道*n* 的突发长度。

(f) 根据原始图像的格式和待压缩图像的格式，配置 DMA2D_OUT_MACRO_BLOCK_SIZE_CH*n* 选择 2D DMA 输出图像块大小。详见表 17.7-2 *DMA2D_OUT_MACRO_BLOCK_SIZE_CHn 配置*：

- 0：8 × 8
- 1：16 × 8
- 2：16 × 16

表 17.7-2. DMA2D_OUT_MACRO_BLOCK_SIZE_CH*n* 配置

原始图像格式	待压缩图像格式	DMA2D_OUT_MACRO_BLOCK_SIZE_CH*n*
RGB	YUV444	0
RGB	YUV422	1
RGB	YUV420	2
YUV422	YUV422	1
GRAY	GRAY	0

(g) 将 DMA2D_OUT_REORDER_EN_CH0 置 1 使能重排 (Reorder) 功能，提高带宽利用率。此功能仅适用于 2D DMA TX 通道 0，不适用于任何其他 TX 通道。

(h) 配置 2D DMA 通道链表地址：

- 2D DMA TX 通道：DMA2D_OUTLINK_ADDR_CH*n*
- 2D DMA RX 通道：DMA2D_INLINK_ADDR_CH*n*

4. 使能中断源：
   - DMA2D_OUT_INT_ENA_CH*n*_REG
   - DMA2D_IN_INT_ENA_CH*n*_REG
   - JPEG_INT_ENA_REG
5. 开启 2D DMA 通道传输：
   - 2D DMA TX 通道：将 DMA2D_OUTLINK_START_CH*n* 先置 1 再置 0
   - 2D DMA RX 通道：将 DMA2D_INLINK_START_CH*n* 先置 1 再置 0
6. 将 JPEG_JPEG_START 置 1，启动 JPEG 图像编解码器编码。

7. 等待 DMA2D_IN_SUC_EOF_CH*n*_INT 变为 1，表示已将编码后的码流完全写入存储器。

17.7.2 JPEG 解码器

JPEG 图像编解码器作为解码器使用时，配置流程为：

1. 激活 JPEG 图像编解码器
   - 将 HP_SYS_CLKRST_JPEG_SYS_CLK_EN 置 1，使能 JPEG 图像编解码器时钟。
   - 将 HP_SYS_CLKRST_RST_EN_JPEG 置 0，释放 JPEG 图像编解码器系统级复位。
2. 配置 JPEG 图像编解码器
   - 将 JPEG_SOFT_RST 置 1，复位 JPEG 图像编解码器；再将 JPEG_SOFT_RST 置 0，释放复位。
   - 将 JPEG_MODE 置 1，将 JPEG 图像编解码器配置为解码器。
   - 参考章节 17.5.2.2 *解析 RST 标记* 的说明，根据需要使能 RST 标记检测。
   - 参考章节 17.5.2.3 *可配置量化系数表* 的说明配置量化系数表。
   - 参考章节 17.5.2.4 *可配置哈夫曼表* 的说明配置哈夫曼表。
   - 参考章节 17.5.2.5 *超时检测* 的说明配置超时检测功能。
   - 配置解码图像的宽度和高度：
     - 宽度：将解码图像的宽度写入 JPEG_HA。
     - 高度：将解码图像的高度写入 JPEG_VA。
     - 根据不同的解码图像格式，JPEG_HA 和 JPEG_VA 必须满足表 17.7-3 *JPEG 解码器 JPEG_HA、JPEG_VA 的配置* 的要求。

表 17.7-3. JPEG 解码器 JPEG_HA、JPEG_VA 的配置

解码图像格式	JPEG_HA	JPEG_VA
YUV444	8 的倍数	8 的倍数
YUV422	16 的倍数	8 的倍数
YUV420	16 的倍数	16 的倍数
GRAY	8 的倍数	8 的倍数

3. 配置 2D DMA（具体配置方法和相关概念介绍，请参考章节 *4 2D-DMA 控制器 (2D-DMA)*。）

   (a) 配置 2D DMA TX 通道的链表：
   - 将 2DEN 置 0，使能 1D 功能。
   - 将 Buffer address pointer 配置为存储待解码码流的起始地址。
   - 将 size 配置为存储待解码码流空间的字节数。
   - 将 mod 配置为 0。2DEN 置 0 时，mod 配置无效。

   (b) 配置 2D DMA RX 通道的链表：
   - 将 2DEN 置 1，使能 2D 功能。
   - 将 Buffer address pointer 配置为存储解码图像的起始地址。

- 将 pbyte 配置为解码并经过 2D DMA 颜色空间转换的图像一个像素的字节数。
- 将 HA 配置为输出解码图像宽度，VA 配置为输出解码图像高度，hb 配置为一个图像块水平方向的像素个数，vb 配置为一个图像块垂直方向的像素个数。
- 将 mod 配置为 1，表示连续写出 hb × vb 大小图像块。
- 将 (X，Y) 配置为起始图像块的第一个像素坐标。根据解码图像格式，hb 和 vb 须满足表 17.7-4 *JPEG 解码器 hb 和 vb 配置* 中的限制：

表 17.7-4. JPEG 解码器 hb 和 vb 配置

解码图像格式	hb	vb
YUV444	8 的倍数	8 的倍数
YUV422	16 的倍数	8 的倍数
YUV420	16 的倍数	16 的倍数
GRAY	8 的倍数	8 的倍数

(c) 连接 JPEG 图像编解码器和 2D DMA 通道：

- 将 DMA2D_OUT_PERI_SEL_CH*n* 配置为 0，连接 JPEG 图像编解码器和 TX 通道*n*；
- 将 DMA2D_IN_PERI_SEL_CH*n* 配置为 0，连接 JPEG 图像编解码器和 RX 通道*n*。

(d) 复位 2D DMA 通道：

- 2D DMA TX 通道*n*：将 DMA2D_OUT_RST_CH*n* 置 1，复位 2D DMA TX 通道*n*，再将 DMA2D_OUT_RST_CH*n* 置 0，释放 2D DMA TX 通道*n* 复位。
- 2D DMA RX 通道*n*：将 DMA2D_IN_RST_CH*n* 置 1，复位 2D DMA RX 通道*n*，再将 DMA2D_IN_RST_CH*n* 置 0，释放 2D DMA RX 通道*n* 复位。

(e) 选择 2D DMA 通道的突发长度：

- 配置 DMA2D_OUT_MEM_BURST_LENGTH_CH*n*，选择 2D DMA TX 通道*n* 的突发长度。
- 配置 DMA2D_IN_MEM_BURST_LENGTH_CH*n*，选择 2D DMA RX 通道*n* 的突发长度。

(f) 根据解码图像的格式，配置 DMA2D_IN_MACRO_BLOCK_SIZE_CH*n* 选择 2D DMA 接收图像块大小。详见表 17.7-5 *DMA2D_IN_MACRO_BLOCK_SIZE_CHn 配置*：

- 0：8 × 8
- 1：16 × 8
- 2：16 × 16

表 17.7-5. DMA2D_IN_MACRO_BLOCK_SIZE_CH*n* 配置

解码图像格式	DMA2D_IN_MACRO_BLOCK_SIZE_CH*n*
YUV444	0
YUV422	1
YUV420	2
GRAY	0

(g) 将 DMA2D_IN_REORDER_EN_CH0 置 1 使能重排 (Reorder) 功能，提高带宽利用率。此功能仅适用于 2D DMA RX 通道 0，不适用于任何其他 RX 通道。

(h) 配置 DMA2D_IN_SCRAMBLE_SEL_PRE_CH0，选择在 2D DMA 颜色空间转换前如何调换解码图像的像素顺序：

- 0：BYTE2-1-0
- 1：BYTE2-0-1
- 2：BYTE1-0-2
- 3：BYTE1-2-0
- 4：BYTE0-2-1
- 5：BYTE0-1-2

字段配置为 5 表示调换像素顺序，例如将 JPEG 解码器输出的 YUV444 调换为 VUY444。请注意，该功能仅适用于 2D DMA RX 通道 0，不适用于其他 RX 通道。

(i) 配置 DMA2D_IN_COLOR_INPUT_SEL_CH0、DMA2D_IN_COLOR_3B_PROC_EN_CH0 和 DMA2D_IN_COLOR_OUTPUT_SEL_CH0 选择颜色空间转换方式。详见表 17.7-6 *2D DMA 颜色空间转换配置*：

表 17.7-6. 2D DMA 颜色空间转换配置

颜色空间转换方式	INPUT_SEL_CH0	3B_PROC_EN_CH0	OUTPUT_SEL_CH0
YUV420 → RGB	0	1	1
YUV422 → RGB	0	1	1
YUV444 → RGB	2	1	1
YUV444（仅调换像素顺序）	2	0	1
不转换颜色空间	7	N/A	N/A

请注意，该功能仅适用于 2D DMA RX 通道 0，不适用于其他 RX 通道。

(j) 配置颜色空间转换公式所需的参数，包括 DMA2D_IN_COLOR_PARAM_H0_CH0、DMA2D_IN_COLOR_PARAM_H1_CH0、DMA2D_IN_COLOR_PARAM_M0_CH0、DMA2D_IN_COLOR_PARAM_M1_CH0、DMA2D_IN_COLOR_PARAM_L0_CH0 和 DMA2D_IN_COLOR_PARAM_L1_CH0。请注意，该功能仅适用于 2D DMA RX 通道 0，不适用于其他 RX 通道。

(k) 配置 DMA2D_IN_SCRAMBLE_SEL_POST_CH0，选择在 2D DMA 颜色空间转换后如何调换图像的像素顺序：

- 0：BYTE2-1-0
- 1：BYTE2-0-1
- 2：BYTE1-0-2
- 3：BYTE1-2-0
- 4：BYTE0-2-1

- 5：BYTE0-1-2

字段配置为 5 表示翻转像素顺序，例如将 2D DMA 颜色空间转换后的 RGB 翻转为 BGR。请注意，该功能仅适用于 2D DMA RX 通道 0，不适用于其他 RX 通道。

(l) 配置 2D DMA 通道链表地址：

- 2D DMA TX 通道：DMA2D_OUTLINK_ADDR_CH*n*
- 2D DMA RX 通道：DMA2D_INLINK_ADDR_CH*n*

4. 使能中断源：
   - DMA2D_OUT_INT_ENA_CH*n*_REG
   - DMA2D_IN_INT_ENA_CH*n*_REG
   - JPEG_INT_ENA_REG
5. 开启 2D DMA 通道传输：
   - 2D DMA TX 通道：将 DMA2D_OUTLINK_START_CH*n* 先置 1 再置 0
   - 2D DMA RX 通道：将 DMA2D_INLINK_START_CH*n* 先置 1 再置 0
6. 将 JPEG_JPEG_START 置 1，启动 JPEG 图像编解码器解码。
7. 等待 DMA2D_IN_SUC_EOF_CH*n*_INT 变为 1，表示解码图像已完全写入存储器。

17.7.3 复位

根据实际场景，用户可以在编码或解码过程中 **复位** JPEG 图像编解码器的全部模块或部分模块：

- 复位全部模块（包括所有 RAM、FIFO 和状态机）：将 JPEG_SOFT_RST 置 1。寄存器配置不会复位。
- 复位所有 RAM 和 FIFO（例如更改量化系数表或哈夫曼表时）：将 JPEG_FIFO_RST 置 1。
- 复位所有状态机（例如终止当前编码或解码流程时）：将 JPEG_FSM_RST 置 1。

17.8 寄存器列表

本小节的所有地址均为相对于 JPEG 图像编解码器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
控制和配置寄存器			
JPEG_CONFIG_REG	系统级控制和配置寄存器	0x0000	varies
JPEG_DQT_INFO_REG	量化系数表信息寄存器	0x0004	R/W
JPEG_PIC_SIZE_REG	图像大小寄存器	0x0008	R/W
JPEG_TOQNR_REG	量化系数表 0 寄存器	0x0010	HRO
JPEG_T1QNR_REG	量化系数表 1 寄存器	0x0014	HRO
JPEG_T2QNR_REG	量化系数表 2 寄存器	0x0018	HRO
JPEG_T3QNR_REG	量化系数表 3 寄存器	0x001C	HRO
JPEG_DECODE_CONF_REG	解码器配置寄存器	0x0020	varies
JPEG_C0_REG	分量 0 寄存器	0x0024	R/W
JPEG_C1_REG	分量 1 寄存器	0x0028	R/W
JPEG_C2_REG	分量 2 寄存器	0x002C	R/W
JPEG_DHT_INFO_REG	哈夫曼表信息寄存器	0x0034	R/W
中断寄存器			
JPEG_INT_RAW_REG	中断原始状态寄存器	0x0038	R/WTC/SS
JPEG_INT_ENA_REG	中断使能寄存器	0x003C	R/W
JPEG_INT_ST_REG	中断屏蔽状态寄存器	0x0040	RO
JPEG_INT_CLR_REG	中断清除寄存器	0x0044	WT
追踪和调试寄存器			
JPEG_STATUS0_REG	追踪和调试状态寄存器 0	0x0048	RO
JPEG_STATUS2_REG	追踪和调试状态寄存器 2	0x004C	RO
JPEG_STATUS3_REG	追踪和调试状态寄存器 3	0x0050	RO
JPEG_STATUS4_REG	追踪和调试状态寄存器 4	0x0054	RO
JPEG_DHT_TOTLEN_DC0_REG	DC0 哈夫曼表码字长度寄存器	0x0058	HRO
JPEG_DHT_VAL_DC0_REG	DC0 哈夫曼表符号寄存器	0x005C	HRO
JPEG_DHT_TOTLEN_AC0_REG	AC0 哈夫曼表码字长度寄存器	0x0060	HRO
JPEG_DHT_VAL_AC0_REG	AC0 哈夫曼表符号寄存器	0x0064	HRO
JPEG_DHT_TOTLEN_DC1_REG	DC1 哈夫曼表码字长度寄存器	0x0068	HRO
JPEG_DHT_VAL_DC1_REG	DC1 哈夫曼表符号寄存器	0x006C	HRO
JPEG_DHT_TOTLEN_AC1_REG	AC1 哈夫曼表码字长度寄存器	0x0070	HRO
JPEG_DHT_VAL_AC1_REG	AC1 哈夫曼表符号寄存器	0x0074	HRO
JPEG_DHT_CODEMIN_DC0_REG	DC0 哈夫曼表最小码字寄存器	0x0078	HRO
JPEG_DHT_CODEMIN_AC0_REG	AC0 哈夫曼表最小码字寄存器	0x007C	HRO
JPEG_DHT_CODEMIN_DC1_REG	DC1 哈夫曼表最小码字寄存器	0x0080	HRO
JPEG_DHT_CODEMIN_AC1_REG	AC1 哈夫曼表最小码字寄存器	0x0084	HRO

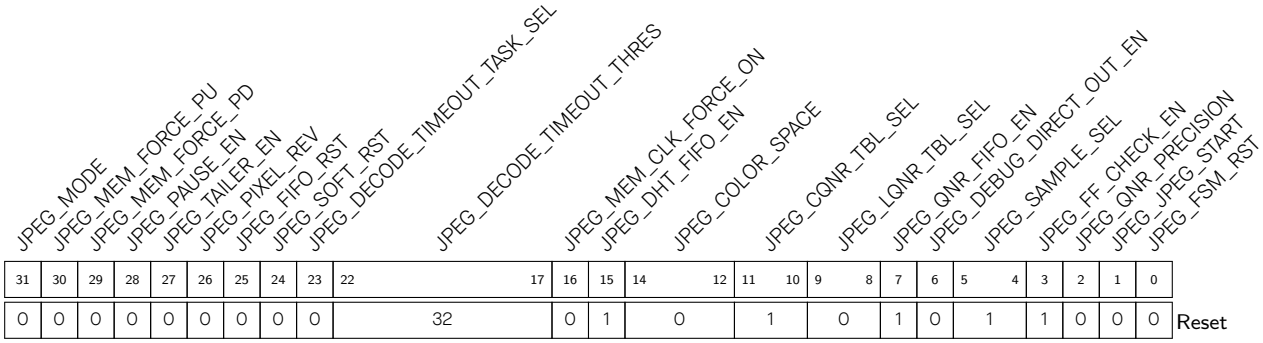
名称	描述	地址	访问
JPEG_DECODER_STATUS0_REG	解码器追踪和调试状态寄存器 0	0x0088	RO
JPEG_DECODER_STATUS1_REG	解码器追踪和调试状态寄存器 1	0x008C	RO
JPEG_DECODER_STATUS2_REG	解码器追踪和调试状态寄存器 2	0x0090	RO
JPEG_DECODER_STATUS3_REG	解码器追踪和调试状态寄存器 3	0x0094	RO
JPEG_DECODER_STATUS4_REG	解码器追踪和调试状态寄存器 4	0x0098	RO
JPEG_DECODER_STATUS5_REG	解码器追踪和调试状态寄存器 5	0x009C	varies
JPEG_STATUS5_REG	追踪和调试状态寄存器 5	0x00A0	RO
JPEG_ECO_LOW_REG	冗余寄存器（低）	0x00A4	R/W
JPEG_ECO_HIGH_REG	冗余寄存器（高）	0x00A8	R/W
JPEG_CONFIG1_REG	JPEG 图像编解码器配置寄存器 1	0x00AC	R/W
JPEG_SYS_REG	系统配置寄存器	0x00F8	R/W
JPEG_VERSION_REG	版本控制寄存器	0x00FC	R/W

17.9 寄存器

本小节的所有地址均为相对于 JPEG 图像编解码器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 17.1. JPEG_CONFIG_REG (0x0000)



JPEG_FSM_RST 配置是否复位编解码器状态机。

0：无效值。没有作用

1：复位状态机

(WT)

JPEG_JPEG_START 配置是否启动 JPEG 图像编解码器。

0：无效值。没有作用

1：开启 JPEG 图像编解码器

(WT)

JPEG_QNR_PRECISION 配置编码器的量化系数表精度。

0：8 位精度

1：12 位精度

(R/W)

JPEG_FF_CHECK_EN 配置是否在“0xFF”后添加“0x00”。

0：不添加

1：添加

(R/W)

JPEG_SAMPLE_SEL 配置待压缩图像的格式。

0：YUV444

1：YUV422

2：YUV420

3：无效值。没有作用

(R/W)

JPEG_DEBUG_DIRECT_OUT_EN 配置是否启用编码器的调试模式。

0：禁用调试模式，编码器将在正常模式下工作

1：启用调试模式，输入图像将直接由编码器输出

(R/W)

JPEG_QNR_FIFO_EN 配置是否启用 FIFO 模式配置量化系数表。

0：禁用。使用非 FIFO 模式

1：启用

(R/W)

见下页...

Register 17.1. JPEG_CONFIG_REG (0x0000)

接上页...

JPEG_LQNR_TBL_SEL 配置编码器的亮度量化系数表 ID。(R/W)

JPEG_CQNR_TBL_SEL 配置编码器的色度量化系数表 ID。(R/W)

JPEG_COLOR_SPACE 配置原始图像的颜色空间。

0：RGB888
1：YUV422
2：RGB565
3：GRAY
4：YUV444
5：YUV420
6：无效值。没有作用
7：无效值。没有作用
(R/W)

JPEG_DHT_FIFO_EN 配置是否启用 FIFO 模式配置哈夫曼表。

0：禁用。使用非 FIFO 模式
1：启用
注意，读取哈夫曼表仅支持非 FIFO 模式。
(R/W)

JPEG_MEM_CLK_FORCE_ON 配置是否强制打开内存的时钟门。

0：不强制打开
1：强制打开
(R/W)

JPEG_DECODE_TIMEOUT_THRES 配置触发 JPEG_DECODE_TIMEOUT_INT_RAW 的解码器超时时间。超时时间为 $2^{\text{JPEG_DECODE_TIMEOUT_THRES}} - 1$ 个时钟周期。(R/W)

JPEG_DECODE_TIMEOUT_TASK_SEL 配置解码器超时时的复位模式。

0：软件使用复位来中止解码过程
1：硬件自动中止解码过程
(R/W)

JPEG_SOFT_RST 配置是否对 JPEG 图像编解码器应用软复位。

0：软复位释放
1：对 JPEG 图像编解码器应用软复位（寄存器配置值不会被复位）
(R/W)

JPEG_FIFO_RST 配置是否复位 JPEG 图像编解码器的 FIFO。

0：复位释放
1：复位 JPEG 图像编解码器的 FIFO
(R/W)

见下页...

Register 17.1. JPEG_CONFIG_REG (0x0000)

接上页...

JPEG_PIXEL_REV 配置是否调换原始图像的像素顺序。

0：不调换

1：调换

(R/W)

JPEG_TAILER_EN 配置是否在码流的末尾添加 EOI 标记 “0xFFD9”。

0：不添加

1：添加

(R/W)

JPEG_PAUSE_EN 配置是否暂停 JPEG 图像编解码器。

0：不暂停

1：暂停

(R/W)

JPEG_MEM_FORCE_PD 配置是否强制 JPEG 存储器下电。

0：无操作

1：强制下电

(R/W)

JPEG_MEM_FORCE_PU 配置是否强制 JPEG 存储器上电。

0：无操作

1：强制上电

(R/W)

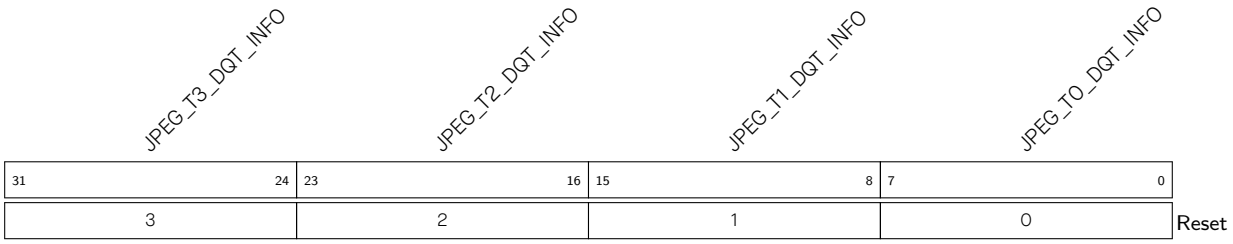
JPEG_MODE 配置 JPEG 图像编解码器的工作模式是编码器还是解码器。

0：编码器

1：解码器

(R/W)

Register 17.2. JPEG_DQT_INFO_REG (0x0004)



JPEG_T0_DQT_INFO 配置解码器量化系数表 0 的精度和 ID。

位 [4]：配置精度

0：低 8 位有效

1：高 12 位有效

位 [3:0]：配置 ID

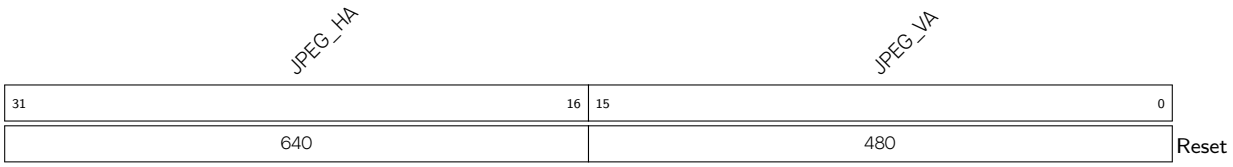
(R/W)

JPEG_T1_DQT_INFO 配置解码器量化系数表 1 的精度和 ID。详见 JPEG_T0_DQT_INFO。(R/W)

JPEG_T2_DQT_INFO 配置解码器量化系数表 2 的精度和 ID。详见 JPEG_T0_DQT_INFO。(R/W)

JPEG_T3_DQT_INFO 配置解码器量化系数表 3 的精度和 ID。详见 JPEG_T0_DQT_INFO。(R/W)

Register 17.3. JPEG_PIC_SIZE_REG (0x0008)



JPEG_VA 配置图像的高度。

当 JPEG 图像编解码器作为编码器工作时，最大可配置位数为 14。

当 JPEG 图像编解码器作为解码器工作时，最大可配置位数为 16。

(R/W)

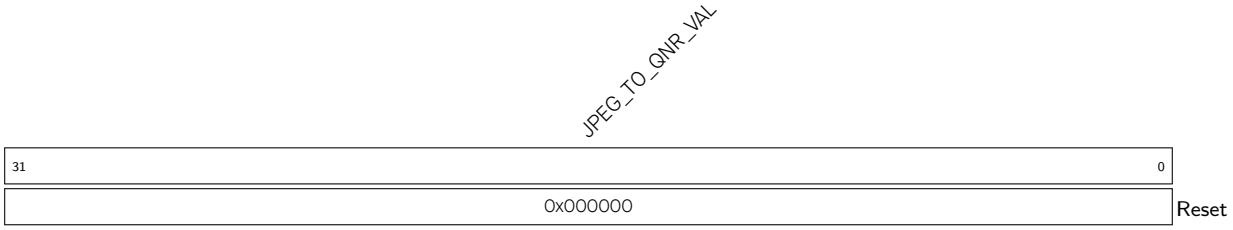
JPEG_HA 配置图像的宽度。

当 JPEG 图像编解码器作为编码器工作时，最大可配置位数为 14。

当 JPEG 图像编解码器作为解码器工作时，最大可配置位数为 16。

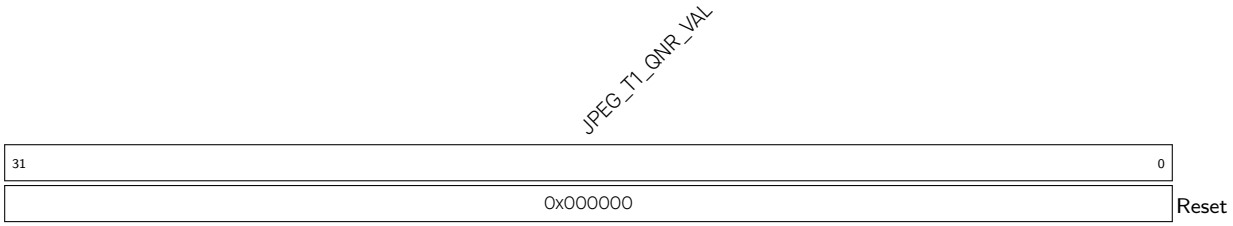
(R/W)

Register 17.4. JPEG_T0QNR_REG (0x0010)



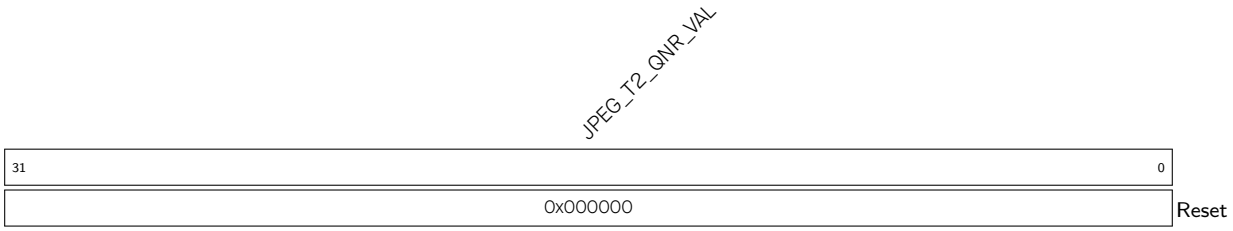
JPEG_T0_QNR_VAL 配置 FIFO 模式下量化系数表 0 的量化系数。(HRO)

Register 17.5. JPEG_T1QNR_REG (0x0014)



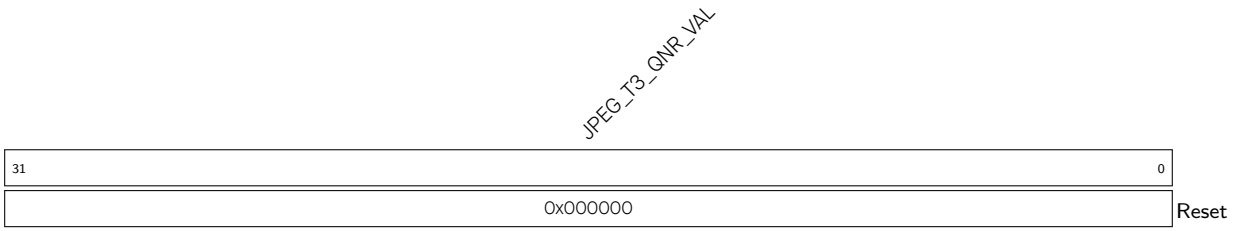
JPEG_T1_QNR_VAL 配置 FIFO 模式下量化系数表 1 的量化系数。(HRO)

Register 17.6. JPEG_T2QNR_REG (0x0018)



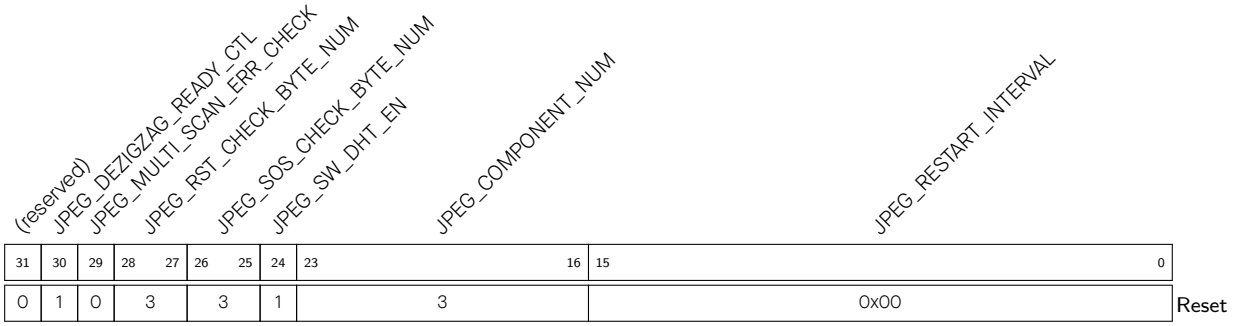
JPEG_T2_QNR_VAL 配置 FIFO 模式下量化系数表 2 的量化系数。(HRO)

Register 17.7. JPEG_T3QNR_REG (0x001C)



JPEG_T3_QNR_VAL 配置 FIFO 模式下量化系数表 3 的量化系数。(HRO)

Register 17.8. JPEG_DECODE_CONF_REG (0x0020)



JPEG_RESTART_INTERVAL 配置解码时 DRI 段的重启间隔。(R/W)

JPEG_COMPONENT_NUM 配置解码时图像中的分量数量。(R/W)

JPEG_SW_DHT_EN 表示由软件为解码器配置的哈夫曼表。该位的值始终为 1。(RO)

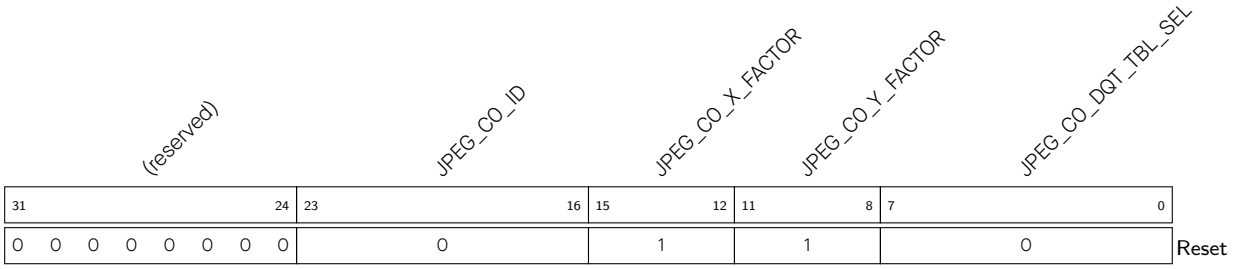
JPEG_SOS_CHECK_BYTE_NUM 配置解码需多次的图像时，一次扫描 SOS 标记检查的字节数。实际检查数为 JPEG_SOS_CHECK_BYTE_NUM + 1。(R/W)

JPEG_RST_CHECK_BYTE_NUM 配置解码一个 RST 间隔后，检查下一个 RST 标记的字节数。实际检查数为 JPEG_RST_CHECK_BYTE_NUM + 1。(R/W)

JPEG_MULTI_SCAN_ERR_CHECK 保留给解码器，请勿配置。(R/W)

JPEG_DEZIGZAG_READY_CTL 保留给解码器，请勿配置。(R/W)

Register 17.9. JPEG_C0_REG (0x0024)



JPEG_C0_DQT_TBL_SEL 配置解码器模式下分量 0 的选定量化系数表 ID。(R/W)

JPEG_C0_Y_FACTOR 配置解码器模式下分量 0 的垂直采样因子。(R/W)

JPEG_C0_X_FACTOR 配置解码器模式下分量 0 的水平采样因子。(R/W)

JPEG_C0_ID 配置解码器模式下分量 0 的 ID。(R/W)

Register 17.10. JPEG_C1_REG (0x0028)



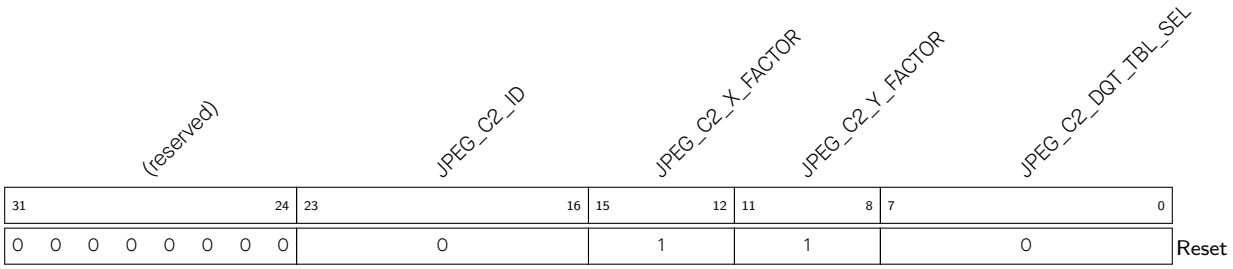
JPEG_C1_DQT_TBL_SEL 配置解码器模式下分量 1 的选定量化系数表 ID。(R/W)

JPEG_C1_Y_FACTOR 配置解码器模式下分量 1 的垂直采样因子。(R/W)

JPEG_C1_X_FACTOR 配置解码器模式下分量 1 的水平采样因子。(R/W)

JPEG_C1_ID 配置解码器模式下分量 1 的 ID。(R/W)

Register 17.11. JPEG_C2_REG (0x002C)



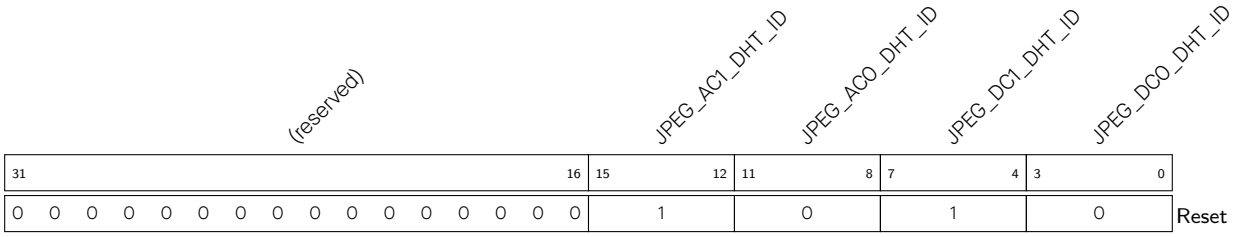
JPEG_C2_DQT_TBL_SEL 配置解码器模式下分量 2 的选定量化系数表 ID。(R/W)

JPEG_C2_Y_FACTOR 配置解码器模式下分量 2 的垂直采样因子。(R/W)

JPEG_C2_X_FACTOR 配置解码器模式下分量 2 的水平采样因子。(R/W)

JPEG_C2_ID 配置解码器模式下分量 2 的 ID。(R/W)

Register 17.12. JPEG_DHT_INFO_REG (0x0034)



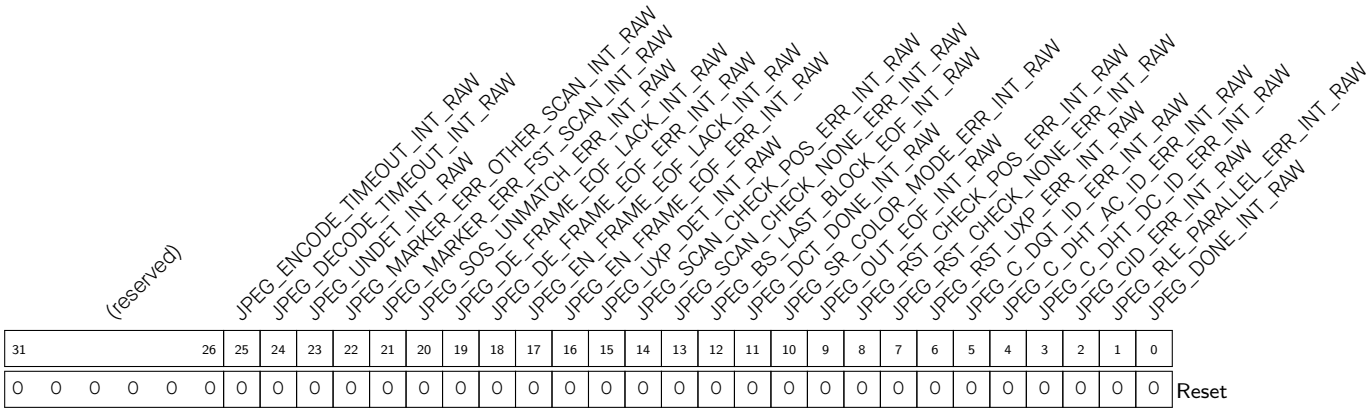
JPEG_DC0_DHT_ID 配置解码器模式下 DC0 哈夫曼表的 ID。(R/W)

JPEG_DC1_DHT_ID 配置解码器模式下 DC1 哈夫曼表的 ID。(R/W)

JPEG_AC0_DHT_ID 配置解码器模式下 AC0 哈夫曼表的 ID。(R/W)

JPEG_AC1_DHT_ID 配置解码器模式下 AC1 哈夫曼表的 ID。(R/W)

Register 17.13. JPEG_INT_RAW_REG (0x0038)



JPEG_DONE_INT_RAW JPEG_DONE_INT 的原始中断状态。(R/WTC/SS)

JPEG_RLE_PARALLEL_ERR_INT_RAW JPEG_RLE_PARALLEL_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_CID_ERR_INT_RAW JPEG_CID_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_C_DHT_DC_ID_ERR_INT_RAW JPEG_C_DHT_DC_ID_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_C_DHT_AC_ID_ERR_INT_RAW JPEG_C_DHT_AC_ID_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_C_DQT_ID_ERR_INT_RAW JPEG_C_DQT_ID_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_RST_UXP_ERR_INT_RAW JPEG_RST_UXP_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_RST_CHECK_NONE_ERR_INT_RAW JPEG_RST_CHECK_NONE_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_RST_CHECK_POS_ERR_INT_RAW JPEG_RST_CHECK_POS_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_OUT_EOF_INT_RAW JPEG_OUT_EOF_INT 的原始中断状态。(R/WTC/SS)

JPEG_SR_COLOR_MODE_ERR_INT_RAW JPEG_SR_COLOR_MODE_ERR_INT 的原始中断状态。该中断无效。(R/WTC/SS)

JPEG_DCT_DONE_INT_RAW JPEG_DCT_DONE_INT 的原始中断状态。(R/WTC/SS)

JPEG_BS_LAST_BLOCK_EOF_INT_RAW JPEG_BS_LAST_BLOCK_EOF_INT 的原始中断状态。(R/WTC/SS)

JPEG_SCAN_CHECK_NONE_ERR_INT_RAW JPEG_SCAN_CHECK_NONE_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_SCAN_CHECK_POS_ERR_INT_RAW JPEG_SCAN_CHECK_POS_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_UXP_DET_INT_RAW JPEG_UXP_DET_INT 的原始中断状态。(R/WTC/SS)

见下页...

Register 17.13. JPEG_INT_RAW_REG (0x0038)

接上页...

JPEG_EN_FRAME_EOF_ERR_INT_RAW JPEG_EN_FRAME_EOF_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_EN_FRAME_EOF_LACK_INT_RAW JPEG_EN_FRAME_EOF_LACK_INT 的原始中断状态。(R/WTC/SS)

JPEG_DE_FRAME_EOF_ERR_INT_RAW JPEG_DE_FRAME_EOF_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_DE_FRAME_EOF_LACK_INT_RAW JPEG_DE_FRAME_EOF_LACK_INT 的原始中断状态。(R/WTC/SS)

JPEG_SOS_UNMATCH_ERR_INT_RAW JPEG_SOS_UNMATCH_ERR_INT 的原始中断状态。(R/WTC/SS)

JPEG_MARKER_ERR_FST_SCAN_INT_RAW JPEG_MARKER_ERR_FST_SCAN_INT 的原始中断状态。(R/WTC/SS)

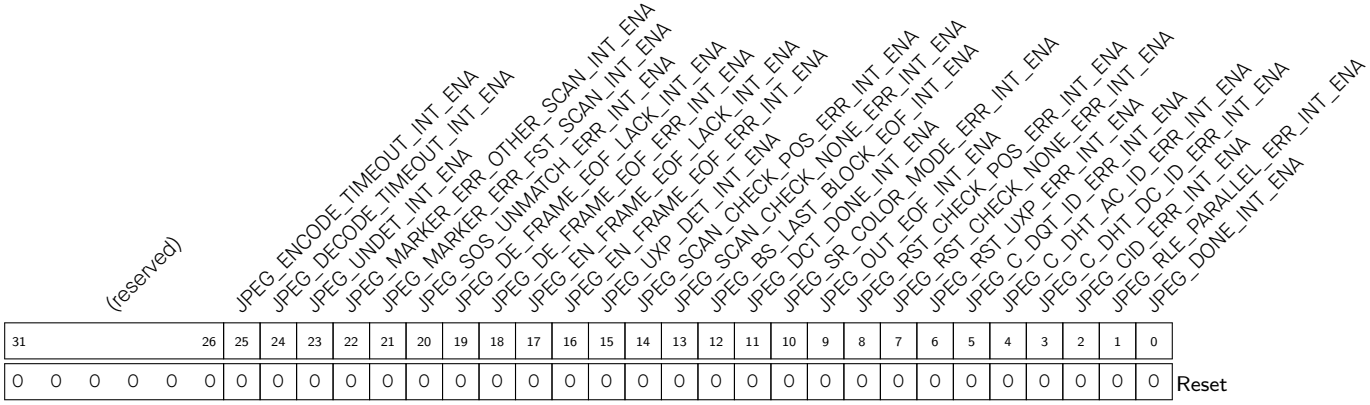
JPEG_MARKER_ERR_OTHER_SCAN_INT_RAW JPEG_MARKER_ERR_OTHER_SCAN_INT 的原始中断状态。(R/WTC/SS)

JPEG_UNDET_INT_RAW JPEG_UNDET_INT 的原始中断状态。(R/WTC/SS)

JPEG_DECODE_TIMEOUT_INT_RAW JPEG_DECODE_TIMEOUT_INT 的原始中断状态。(R/WTC/SS)

JPEG_ENCODE_TIMEOUT_INT_RAW JPEG_ENCODE_TIMEOUT_INT 的原始中断状态。(R/WTC/SS)

Register 17.14. JPEG_INT_ENA_REG (0x003C)



JPEG_DONE_INT_ENA 写 1 使能 JPEG_DONE_INT。(R/W)

JPEG_RLE_PARALLEL_ERR_INT_ENA 写 1 使能 JPEG_RLE_PARALLEL_ERR_INT。(R/W)

JPEG_CID_ERR_INT_ENA 写 1 使能 JPEG_CID_ERR_INT。(R/W)

JPEG_C_DHT_DC_ID_ERR_INT_ENA 写 1 使能 JPEG_C_DHT_DC_ID_ERR_INT。(R/W)

JPEG_C_DHT_AC_ID_ERR_INT_ENA 写 1 使能 JPEG_C_DHT_AC_ID_ERR_INT。(R/W)

JPEG_C_DQT_ID_ERR_INT_ENA 写 1 使能 JPEG_C_DQT_ID_ERR_INT。(R/W)

JPEG_RST_UXP_ERR_INT_ENA 写 1 使能 JPEG_RST_UXP_ERR_INT。(R/W)

JPEG_RST_CHECK_NONE_ERR_INT_ENA 写 1 使能 JPEG_RST_CHECK_NONE_ERR_INT。(R/W)

JPEG_RST_CHECK_POS_ERR_INT_ENA 写 1 使能 JPEG_RST_CHECK_POS_ERR_INT。(R/W)

JPEG_OUT_EOF_INT_ENA 写 1 使能 JPEG_OUT_EOF_INT。(R/W)

JPEG_SR_COLOR_MODE_ERR_INT_ENA 写 1 使能 JPEG_SR_COLOR_MODE_ERR_INT。(R/W)

JPEG_DCT_DONE_INT_ENA 写 1 使能 JPEG_DCT_DONE_INT。(R/W)

JPEG_BS_LAST_BLOCK_EOF_INT_ENA 写 1 使能 JPEG_BS_LAST_BLOCK_EOF_INT。(R/W)

JPEG_SCAN_CHECK_NONE_ERR_INT_ENA 写 1 使能 JPEG_SCAN_CHECK_NONE_ERR_INT。(R/W)

JPEG_SCAN_CHECK_POS_ERR_INT_ENA 写 1 使能 JPEG_SCAN_CHECK_POS_ERR_INT。(R/W)

JPEG_UXP_DET_INT_ENA 写 1 使能 JPEG_UXP_DET_INT。(R/W)

JPEG_EN_FRAME_EOF_ERR_INT_ENA 写 1 使能 JPEG_EN_FRAME_EOF_ERR_INT。(R/W)

JPEG_EN_FRAME_EOF_LACK_INT_ENA 写 1 使能 JPEG_EN_FRAME_EOF_LACK_INT。(R/W)

JPEG_DE_FRAME_EOF_ERR_INT_ENA 写 1 使能 JPEG_DE_FRAME_EOF_ERR_INT。(R/W)

JPEG_DE_FRAME_EOF_LACK_INT_ENA 写 1 使能 JPEG_DE_FRAME_EOF_LACK_INT。(R/W)

JPEG_SOS_UNMATCH_ERR_INT_ENA 写 1 使能 JPEG_SOS_UNMATCH_ERR_INT。(R/W)

见下页...

Register 17.14. JPEG_INT_ENA_REG (0x003C)

接上页...

JPEG_MARKER_ERR_FST_SCAN_INT_ENA 写 1 使能 JPEG_MARKER_ERR_FST_SCAN_INT。(R/W)

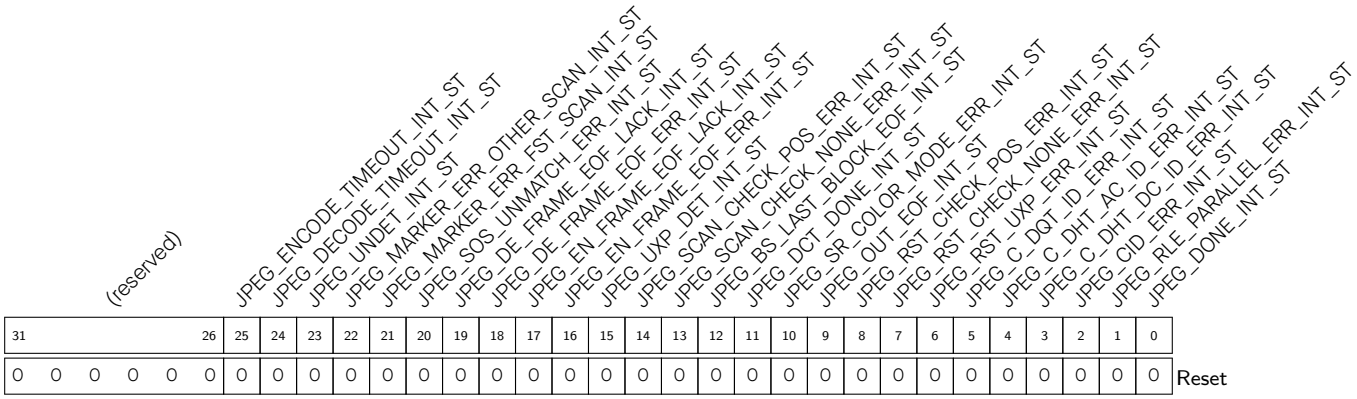
JPEG_MARKER_ERR_OTHER_SCAN_INT_ENA 写 1 使能 JPEG_MARKER_ERR_OTHER_SCAN_INT。(R/W)

JPEG_UNDET_INT_ENA 写 1 使能 JPEG_UNDET_INT。(R/W)

JPEG_DECODE_TIMEOUT_INT_ENA 写 1 使能 JPEG_DECODE_TIMEOUT_INT。(R/W)

JPEG_ENCODE_TIMEOUT_INT_ENA 写 1 使能 JPEG_ENCODE_TIMEOUT_INT。(R/W)

Register 17.15. JPEG_INT_ST_REG (0x0040)



JPEG_DONE_INT_ST JPEG_DONE_INT 的屏蔽中断状态。(RO)

JPEG_RLE_PARALLEL_ERR_INT_ST JPEG_RLE_PARALLEL_ERR_INT 的屏蔽中断状态。(RO)

JPEG_CID_ERR_INT_ST JPEG_CID_ERR_INT 的屏蔽中断状态。(RO)

JPEG_C_DHT_DC_ID_ERR_INT_ST JPEG_C_DHT_DC_ID_ERR_INT 的屏蔽中断状态。(RO)

JPEG_C_DHT_AC_ID_ERR_INT_ST JPEG_C_DHT_AC_ID_ERR_INT 的屏蔽中断状态。(RO)

JPEG_C_DQT_ID_ERR_INT_ST JPEG_C_DQT_ID_ERR_INT 的屏蔽中断状态。(RO)

JPEG_RST_UXP_ERR_INT_ST JPEG_RST_UXP_ERR_INT 的屏蔽中断状态。(RO)

JPEG_RST_CHECK_NONE_ERR_INT_ST JPEG_RST_CHECK_NONE_ERR_INT 的屏蔽中断状态。(RO)

JPEG_RST_CHECK_POS_ERR_INT_ST JPEG_RST_CHECK_POS_ERR_INT 的屏蔽中断状态。(RO)

JPEG_OUT_EOF_INT_ST JPEG_OUT_EOF_INT 的屏蔽中断状态。(RO)

JPEG_SR_COLOR_MODE_ERR_INT_ST JPEG_SR_COLOR_MODE_ERR_INT 的屏蔽中断状态。(RO)

JPEG_DCT_DONE_INT_ST JPEG_DCT_DONE_INT 的屏蔽中断状态。(RO)

JPEG_BS_LAST_BLOCK_EOF_INT_ST JPEG_BS_LAST_BLOCK_EOF_INT 的屏蔽中断状态。(RO)

JPEG_SCAN_CHECK_NONE_ERR_INT_ST JPEG_SCAN_CHECK_NONE_ERR_INT 的屏蔽中断状态。(RO)

JPEG_SCAN_CHECK_POS_ERR_INT_ST JPEG_SCAN_CHECK_POS_ERR_INT 的屏蔽中断状态。(RO)

JPEG_UXP_DET_INT_ST JPEG_UXP_DET_INT 的屏蔽中断状态。(RO)

JPEG_EN_FRAME_EOF_ERR_INT_ST JPEG_EN_FRAME_EOF_ERR_INT 的屏蔽中断状态。(RO)

JPEG_EN_FRAME_EOF_LACK_INT_ST JPEG_EN_FRAME_EOF_LACK_INT 的屏蔽中断状态。(RO)

见下页...

Register 17.15. JPEG_INT_ST_REG (0x0040)

接上页...

JPEG_DE_FRAME_EOF_ERR_INT_ST JPEG_DE_FRAME_EOF_ERR_INT 的屏蔽中断状态。(RO)

JPEG_DE_FRAME_EOF_LACK_INT_ST JPEG_DE_FRAME_EOF_LACK_INT 的屏蔽中断状态。(RO)

JPEG_SOS_UNMATCH_ERR_INT_ST JPEG_SOS_UNMATCH_ERR_INT 的屏蔽中断状态。(RO)

JPEG_MARKER_ERR_FST_SCAN_INT_ST JPEG_MARKER_ERR_FST_SCAN_INT 的屏蔽中断状态。(RO)

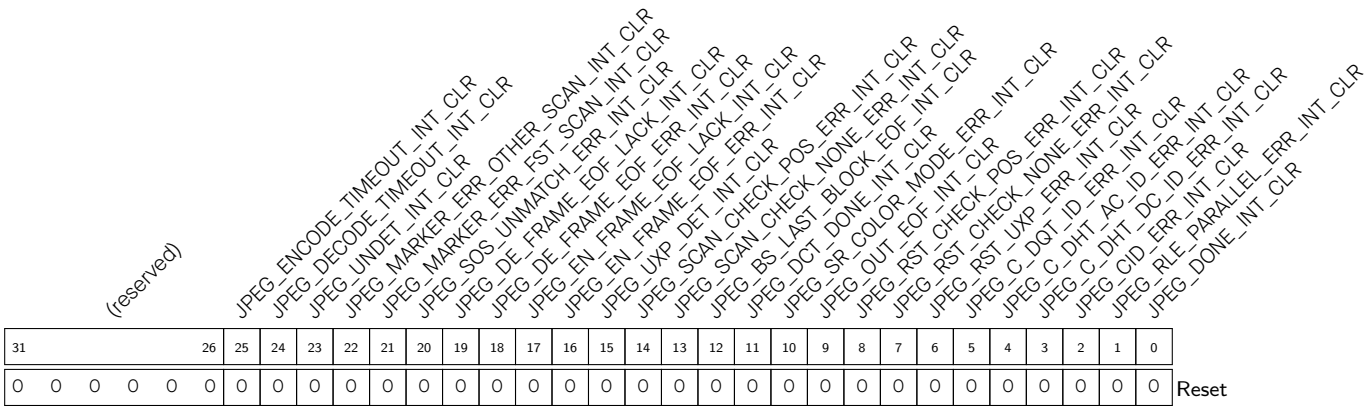
JPEG_MARKER_ERR_OTHER_SCAN_INT_ST JPEG_MARKER_ERR_OTHER_SCAN_INT 的屏蔽中断状态。(RO)

JPEG_UNDET_INT_ST JPEG_UNDET_INT 的屏蔽中断状态。(RO)

JPEG_DECODE_TIMEOUT_INT_ST JPEG_DECODE_TIMEOUT_INT 的屏蔽中断状态。(RO)

JPEG_ENCODE_TIMEOUT_INT_ST JPEG_ENCODE_TIMEOUT_INT 的屏蔽中断状态。(RO)

Register 17.16. JPEG_INT_CLR_REG (0x0044)



JPEG_DONE_INT_CLR 写 1 清除 JPEG_DONE_INT。(WT)

JPEG_RLE_PARALLEL_ERR_INT_CLR 写 1 清除 JPEG_RLE_PARALLEL_ERR_INT。(WT)

JPEG_CID_ERR_INT_CLR 写 1 清除 JPEG_CID_ERR_INT。(WT)

JPEG_C_DHT_DC_ID_ERR_INT_CLR 写 1 清除 JPEG_C_DHT_DC_ID_ERR_INT。(WT)

JPEG_C_DHT_AC_ID_ERR_INT_CLR 写 1 清除 JPEG_C_DHT_AC_ID_ERR_INT。(WT)

JPEG_C_DQT_ID_ERR_INT_CLR 写 1 清除 JPEG_C_DQT_ID_ERR_INT。(WT)

JPEG_RST_UXP_ERR_INT_CLR 写 1 清除 JPEG_RST_UXP_ERR_INT。(WT)

JPEG_RST_CHECK_NONE_ERR_INT_CLR 写 1 清除 JPEG_RST_CHECK_NONE_ERR_INT。(WT)

JPEG_RST_CHECK_POS_ERR_INT_CLR 写 1 清除 JPEG_RST_CHECK_POS_ERR_INT。(WT)

JPEG_OUT_EOF_INT_CLR 写 1 清除 JPEG_OUT_EOF_INT。(WT)

JPEG_SR_COLOR_MODE_ERR_INT_CLR 写 1 清除 JPEG_SR_COLOR_MODE_ERR_INT。(WT)

JPEG_DCT_DONE_INT_CLR 写 1 清除 JPEG_DCT_DONE_INT。(WT)

JPEG_BS_LAST_BLOCK_EOF_INT_CLR 写 1 清除 JPEG_BS_LAST_BLOCK_EOF_INT。(WT)

JPEG_SCAN_CHECK_NONE_ERR_INT_CLR 写 1 清除 JPEG_SCAN_CHECK_NONE_ERR_INT。(WT)

JPEG_SCAN_CHECK_POS_ERR_INT_CLR 写 1 清除 JPEG_SCAN_CHECK_POS_ERR_INT。(WT)

JPEG_UXP_DET_INT_CLR 写 1 清除 JPEG_UXP_DET_INT。(WT)

JPEG_EN_FRAME_EOF_ERR_INT_CLR 写 1 清除 JPEG_EN_FRAME_EOF_ERR_INT。(WT)

JPEG_EN_FRAME_EOF_LACK_INT_CLR 写 1 清除 JPEG_EN_FRAME_EOF_LACK_INT。(WT)

JPEG_DE_FRAME_EOF_ERR_INT_CLR 写 1 清除 JPEG_DE_FRAME_EOF_ERR_INT。(WT)

JPEG_DE_FRAME_EOF_LACK_INT_CLR 写 1 清除 JPEG_DE_FRAME_EOF_LACK_INT。(WT)

JPEG_SOS_UNMATCH_ERR_INT_CLR 写 1 清除 JPEG_SOS_UNMATCH_ERR_INT。(WT)

见下页...

Register 17.16. JPEG_INT_CLR_REG (0x0044)

接上页...

JPEG_MARKER_ERR_FST_SCAN_INT_CLR 写 1 清除 JPEG_MARKER_ERR_FST_SCAN_INT。(WT)

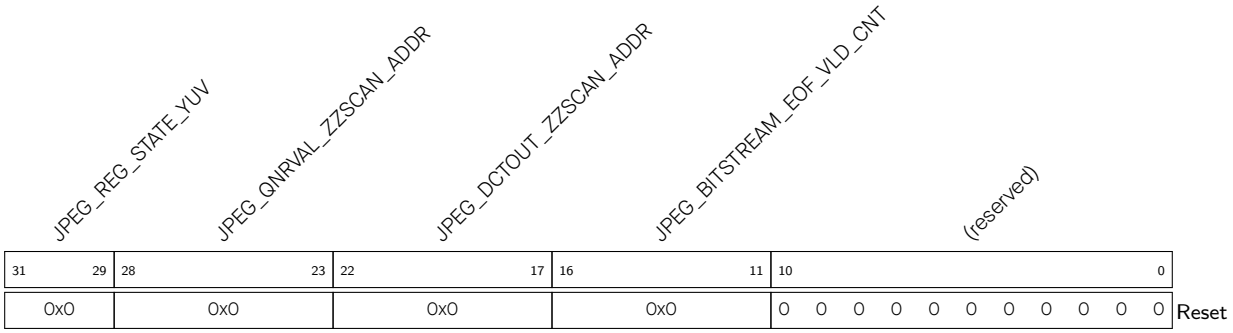
JPEG_MARKER_ERR_OTHER_SCAN_INT_CLR 写 1 清除 JPEG_MARKER_ERR_OTHER_SCAN_INT。(WT)

JPEG_UNDET_INT_CLR 写 1 清除 JPEG_UNDET_INT。(WT)

JPEG_DECODE_TIMEOUT_INT_CLR 写 1 清除 JPEG_DECODE_TIMEOUT_INT。(WT)

JPEG_ENCODE_TIMEOUT_INT_CLR 写 1 清除 JPEG_ENCODE_TIMEOUT_INT。(WT)

Register 17.17. JPEG_STATUS0_REG (0x0048)



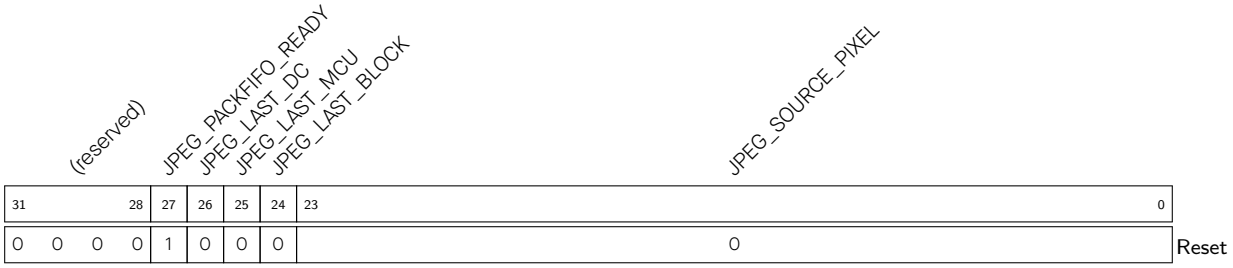
JPEG_BITSTREAM_EOF_VLD_CNT 表示最后一个码流的有效比特数。(RO)

JPEG_DCTOUT_ZZSCAN_ADDR 表示从 dctout_ram 读取的 zigzag 地址。(RO)

JPEG_QNRVAL_ZZSCAN_ADDR 表示从 qnrval_ram 读取的 zigzag 地址。(RO)

JPEG_REG_STATE_YUV 表示 JPEG 图像编解码器状态机的状态。(RO)

Register 17.18. JPEG_STATUS2_REG (0x004C)



JPEG_SOURCE_PIXEL 表示从 DMA 获取的源像素。(RO)

JPEG_LAST_BLOCK 表示编码过程是否处于图像的最后一个块。

0：不是最后一个图像块

1：最后一个图像块

(RO)

JPEG_LAST_MCU 表示编码过程是否处于图像的最后一个 MCU。0：不是最后一个 MCU

1：最后一个 MCU

(RO)

JPEG_LAST_DC 表示编码过程是否处于图像最后一个块的头部。(RO)

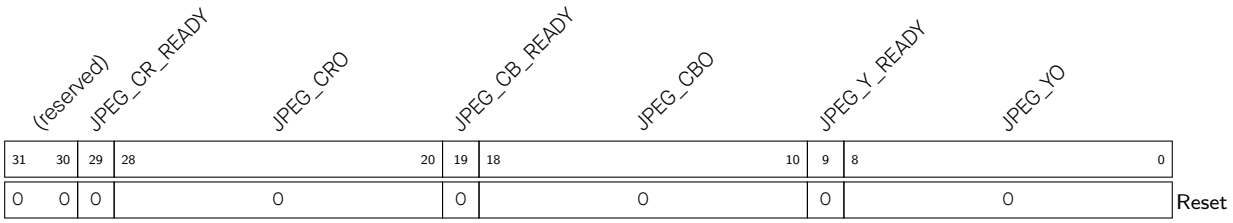
JPEG_PACKFIFO_READY 表示 JPEG pack_fifo 的就绪信号。

0：未就绪

1：就绪

(RO)

Register 17.19. JPEG_STATUS3_REG (0x0050)



JPEG_YO 表示从 RGB 输入转换的 Y 分量。(RO)

JPEG_Y_READY 表示 Y 分量有效信号。

0：无效

1：有效

(RO)

JPEG_CBO 表示从 RGB 输入转换的 Cb 分量。(RO)

JPEG_CB_READY 表示 Cb 分量有效信号。

0：无效

1：有效

(RO)

JPEG_CRO 表示从 RGB 输入转换的 Cr 分量。(RO)

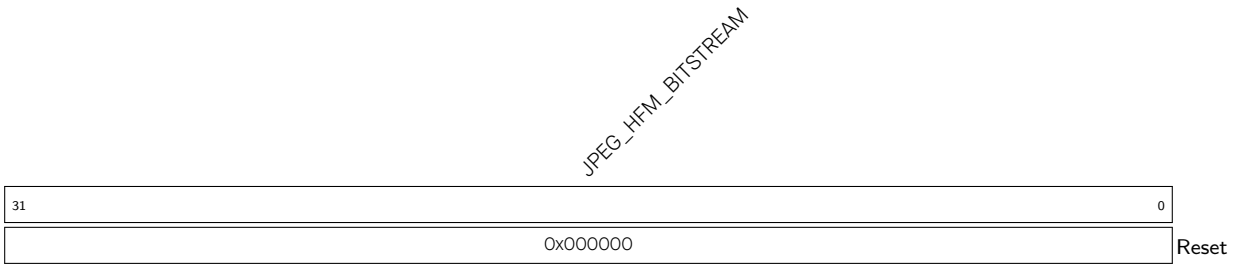
JPEG_CR_READY 表示 Cr 分量有效信号。

0：无效

1：有效

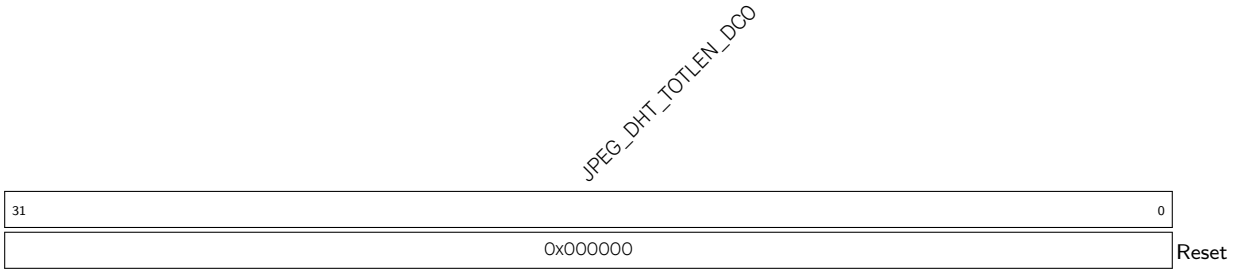
(RO)

Register 17.20. JPEG_STATUS4_REG (0x0054)



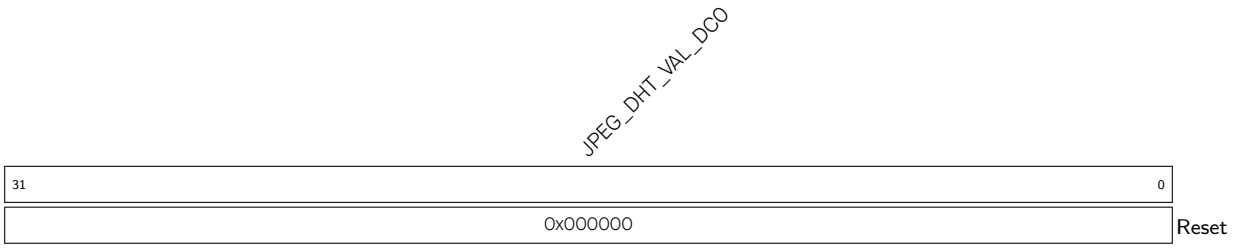
JPEG_HFM_BITSTREAM 表示编码过程中的 Huffman 码流。(RO)

Register 17.21. JPEG_DHT_TOTLEN_DCO_REG (0x0058)



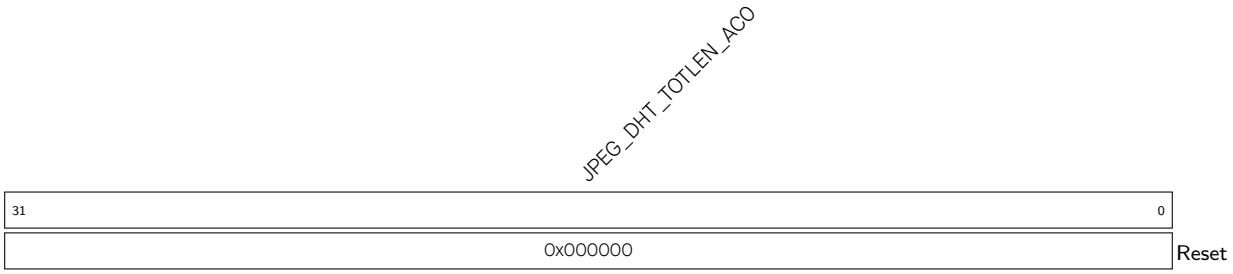
JPEG_DHT_TOTLEN_DCO 配置 FIFO 模式下 DCO 哈夫曼表长度为 1 到 16 位的码字数量。(HRO)

Register 17.22. JPEG_DHT_VAl_DCO_REG (0x005C)



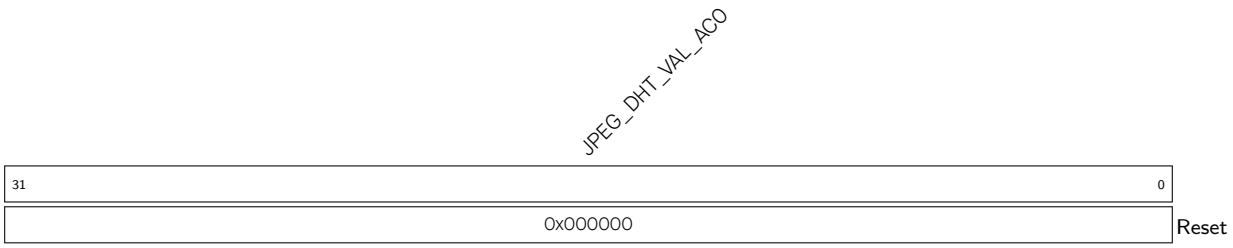
JPEG_DHT_VAL_DCO 配置 FIFO 模式下 DCO 哈夫曼表解码后码字对应的符号。(HRO)

Register 17.23. JPEG_DHT_TOTLEN_ACO_REG (0x0060)



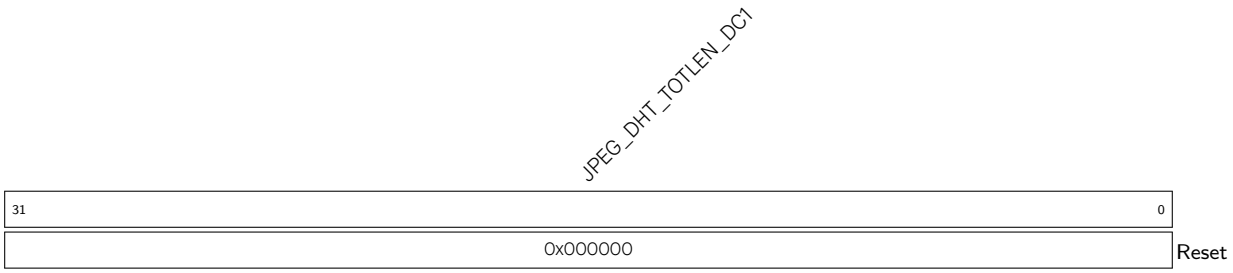
JPEG_DHT_TOTLEN_ACO 配置 FIFO 模式下 ACO 哈夫曼表长度为 1 到 16 位的码字数量。(HRO)

Register 17.24. JPEG_DHT_VAl_ACO_REG (0x0064)



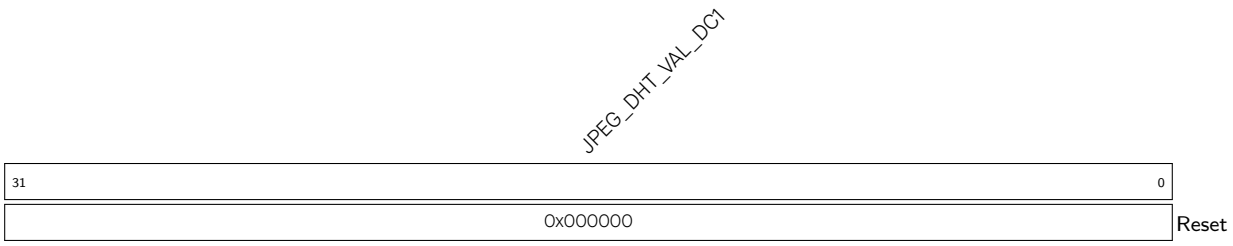
JPEG_DHT_VAL_ACO 配置 FIFO 模式下 ACO 哈夫曼表解码后码字对应的符号。(HRO)

Register 17.25. JPEG_DHT_TOTLEN_DC1_REG (0x0068)



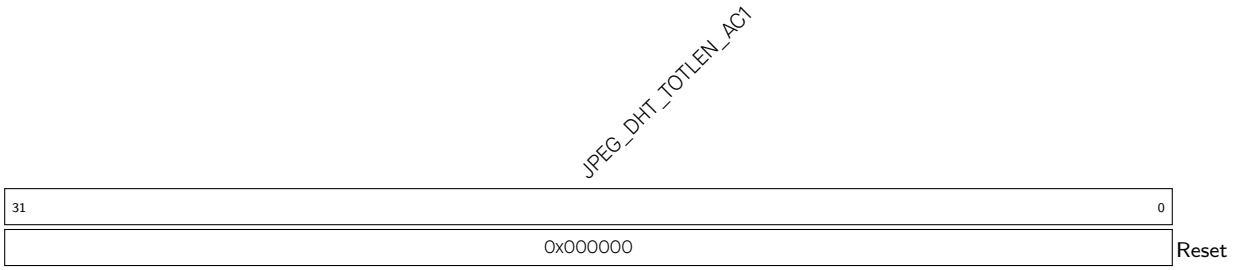
JPEG_DHT_TOTLEN_DC1 配置 FIFO 模式下 DC1 哈夫曼表长度为 1 到 16 位的码字数量。(HRO)

Register 17.26. JPEG_DHT_VAl_DC1_REG (0x006C)



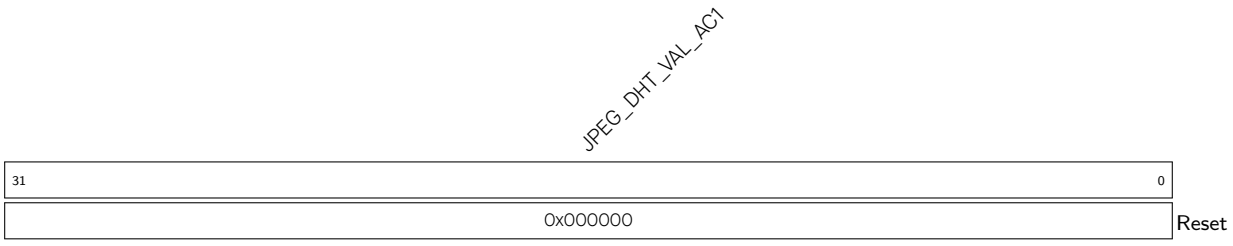
JPEG_DHT_VAL_DC1 配置 FIFO 模式下 DC1 哈夫曼表解码后码字对应的符号。(HRO)

Register 17.27. JPEG_DHT_TOTLEN_AC1_REG (0x0070)



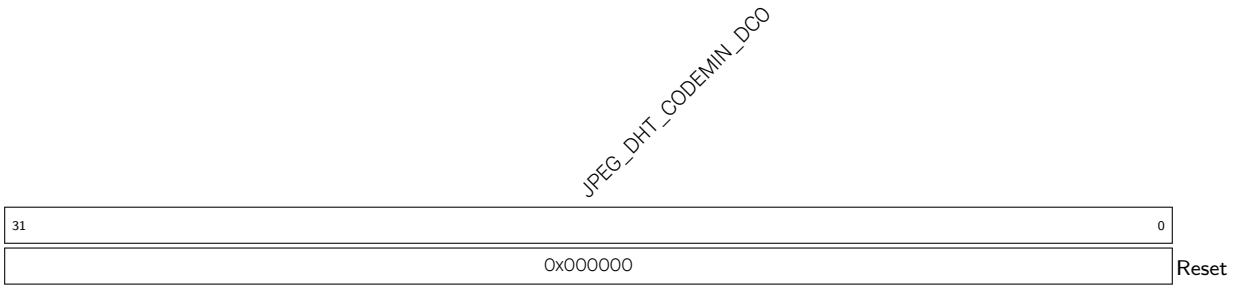
JPEG_DHT_TOTLEN_AC1 配置 FIFO 模式下 AC1 哈夫曼表长度为 1 到 16 位的码字数量。(HRO)

Register 17.28. JPEG_DHT_VAl_AC1_REG (0x0074)



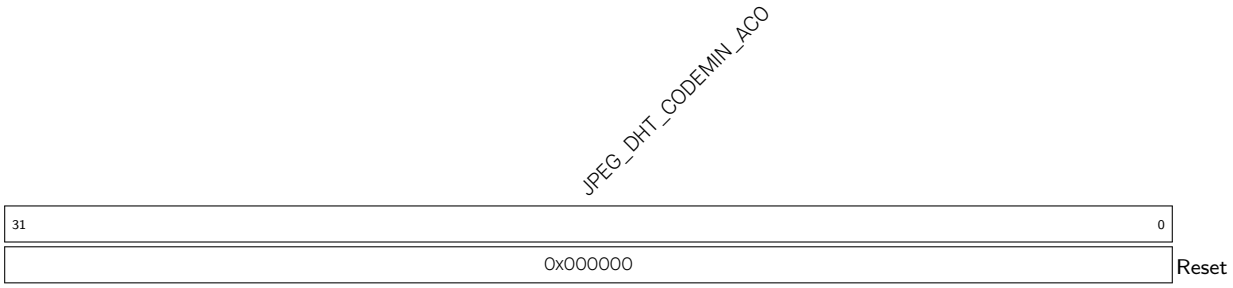
JPEG_DHT_VAL_AC1 配置 FIFO 模式下 AC1 哈夫曼表解码后码字对应的符号。(HRO)

Register 17.29. JPEG_DHT_CODEMIN_DC0_REG (0x0078)



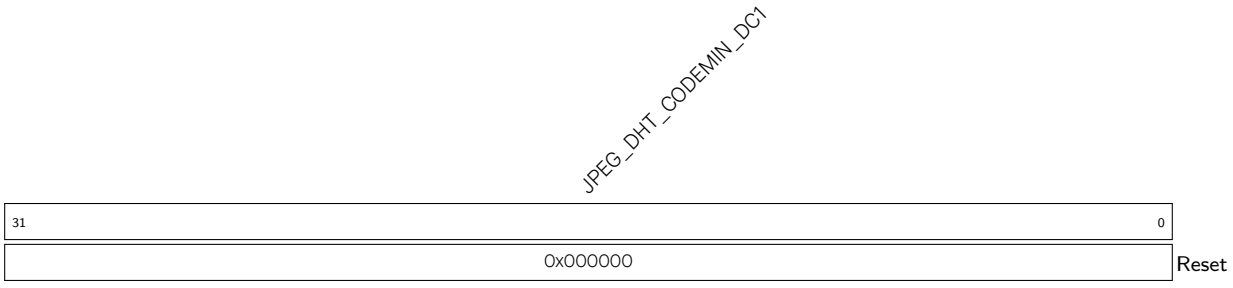
JPEG_DHT_CODEMIN_DC0 配置 FIFO 模式下 DC0 哈夫曼表长度为 1 到 16 位的最小码字。码字应向左偏移至 16 位的最高有效位。(HRO)

Register 17.30. JPEG_DHT_CODEMIN_AC0_REG (0x007C)



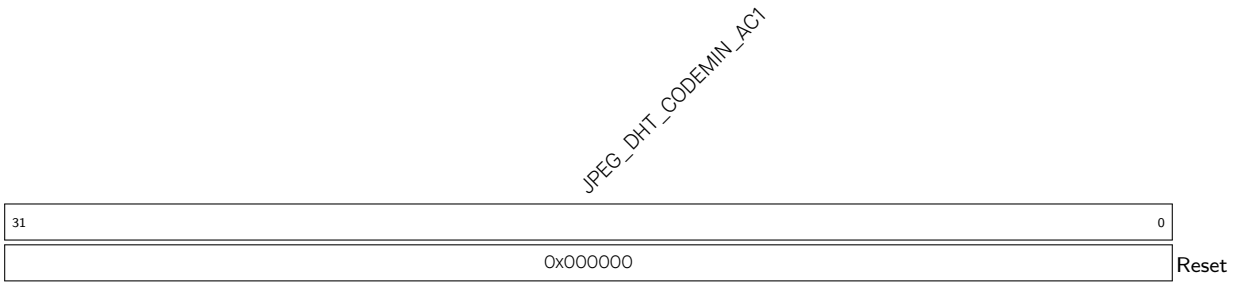
JPEG_DHT_CODEMIN_AC0 配置 FIFO 模式下 AC0 哈夫曼表长度为 1 到 16 位的最小码字。码字应向左偏移至 16 位的最高有效位。(HRO)

Register 17.31. JPEG_DHT_CODEMIN_DC1_REG (0x0080)



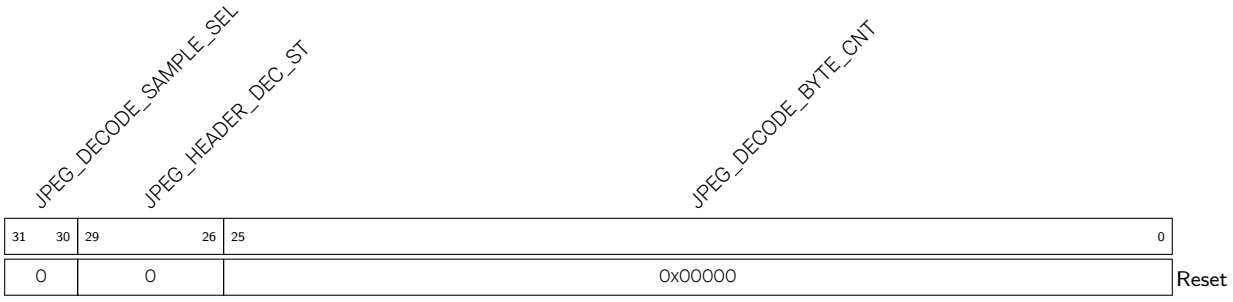
JPEG_DHT_CODEMIN_DC1 配置 FIFO 模式下 DC1 哈夫曼表长度为 1 到 16 位的最小码字。码字应向左偏移至 16 位的最高有效位。(HRO)

Register 17.32. JPEG_DHT_CODEMIN_AC1_REG (0x0084)



JPEG_DHT_CODEMIN_AC1 配置 FIFO 模式下 AC1 哈夫曼表长度为 1 到 16 位的最小码字。码字应向左偏移至 16 位的最高有效位。(HRO)

Register 17.33. JPEG_DECODER_STATUS0_REG (0x0088)

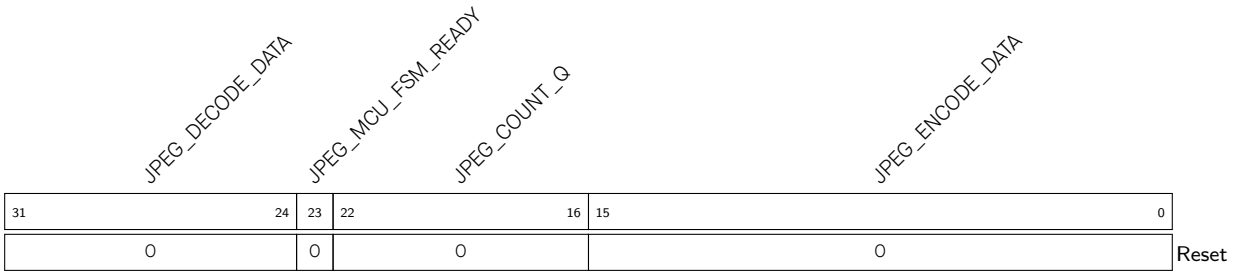


JPEG_DECODE_BYTE_CNT Reserved (RO)

JPEG_HEADER_DEC_ST 保留。(RO)

JPEG_DECODE_SAMPLE_SEL 保留。(RO)

Register 17.34. JPEG_DECODER_STATUS1_REG (0x008C)



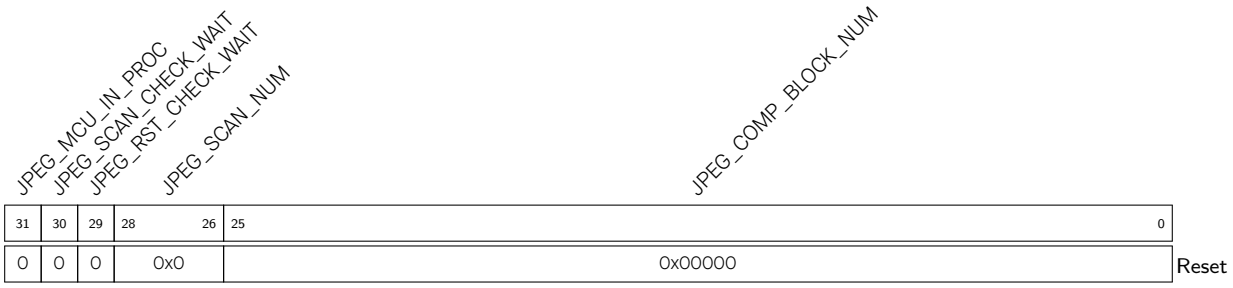
JPEG_ENCODE_DATA 保留。(RO)

JPEG_COUNT_Q 保留。(RO)

JPEG_MCU_FSM_READY 保留。(RO)

JPEG_DECODE_DATA 保留。(RO)

Register 17.35. JPEG_DECODER_STATUS2_REG (0x0090)



JPEG_COMP_BLOCK_NUM 保留。(RO)

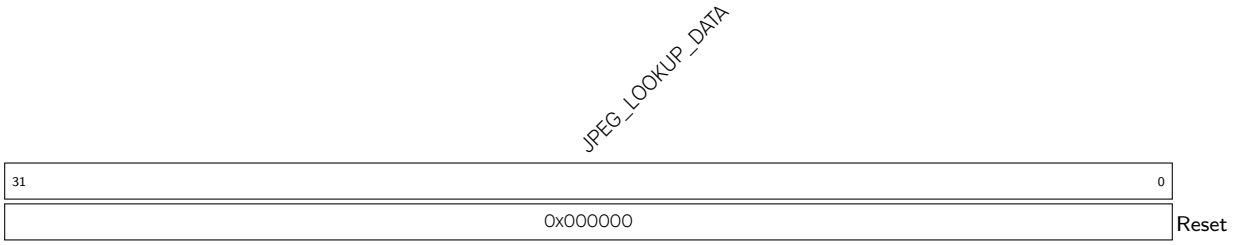
JPEG_SCAN_NUM 保留。(RO)

JPEG_RST_CHECK_WAIT 保留。(RO)

JPEG_SCAN_CHECK_WAIT 保留。(RO)

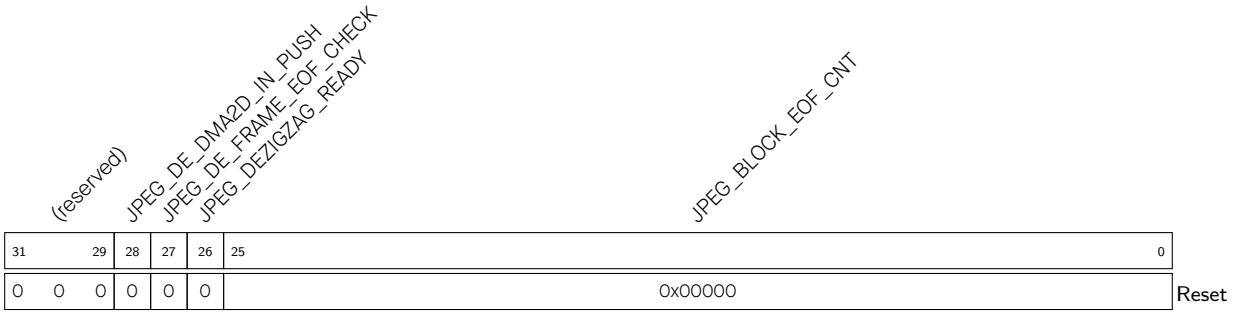
JPEG_MCU_IN_PROC 保留。(RO)

Register 17.36. JPEG_DECODER_STATUS3_REG (0x0094)



JPEG_LOOKUP_DATA 保留。(RO)

Register 17.37. JPEG_DECODER_STATUS4_REG (0x0098)



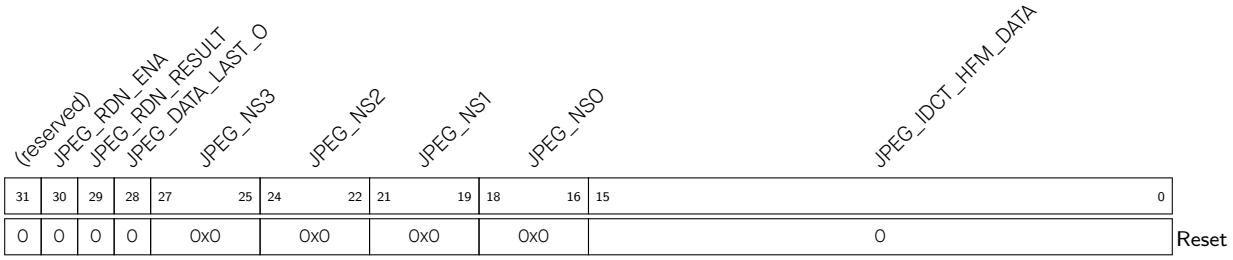
JPEG_BLOCK_EOF_CNT 保留。(RO)

JPEG_DEZIGZAG_READY 保留。(RO)

JPEG_DE_FRAME_EOF_CHECK 保留。(RO)

JPEG_DE_DMA2D_IN_PUSH 保留。(RO)

Register 17.38. JPEG_DECODER_STATUS5_REG (0x009C)



JPEG_IDCT_HFM_DATA 保留。(RO)

JPEG_NS0 保留。(RO)

JPEG_NS1 保留。(RO)

JPEG_NS2 保留。(RO)

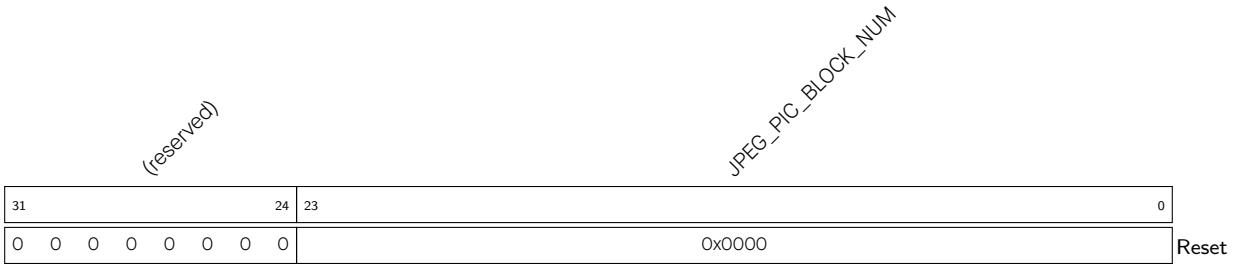
JPEG_NS3 保留。(RO)

JPEG_DATA_LAST_0 保留。(RO)

JPEG_RDN_RESULT 保留。(RO)

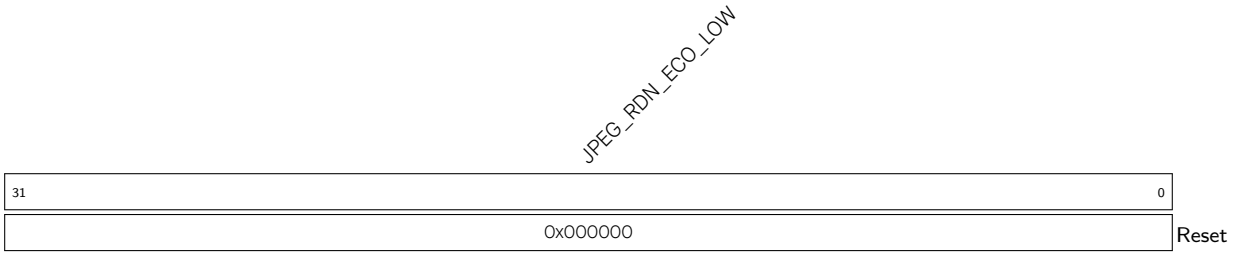
JPEG_RDN_ENA 保留。(R/W)

Register 17.39. JPEG_STATUS5_REG (0x00A0)



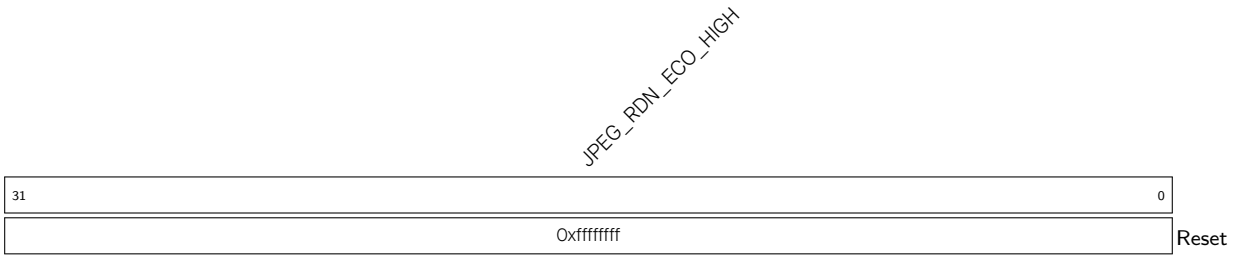
JPEG_PIC_BLOCK_NUM 保留。(RO)

Register 17.40. JPEG_ECO_LOW_REG (0x00A4)



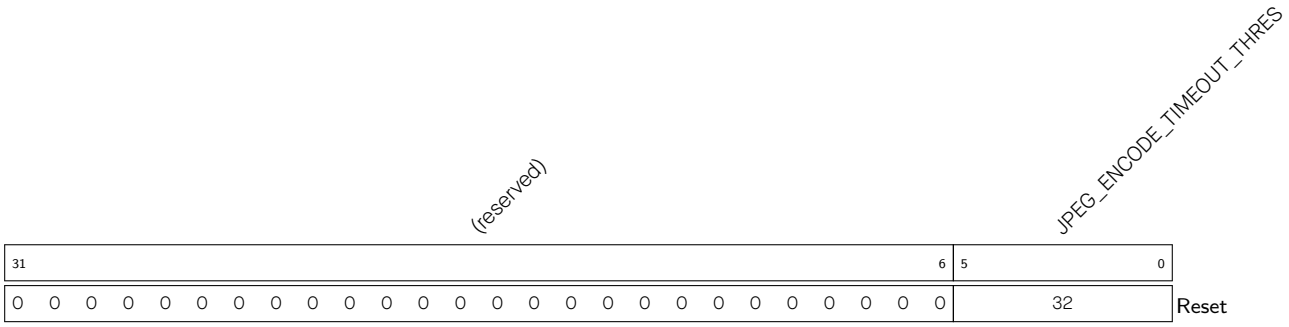
JPEG_RDN_ECO_LOW 保留。(R/W)

Register 17.41. JPEG_ECO_HIGH_REG (0x00A8)



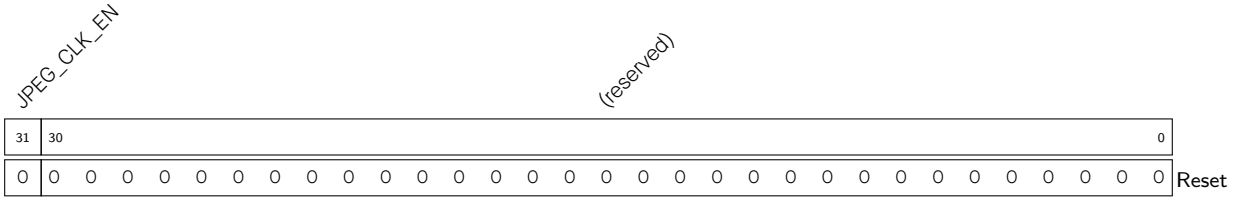
JPEG_RDN_ECO_HIGH 保留。(R/W)

Register 17.42. JPEG_CONFIG1_REG (0x00AC)



JPEG_ENCODE_TIMEOUT_THRES 配置触发 JPEG_ENCODE_TIMEOUT_INT_RAW 的编码器超时时间。超时时间为 $2^{\text{JPEG_ENCODE_TIMEOUT_THRES}} - 1$ 个时钟周期。(R/W)

Register 17.43. JPEG_SYS_REG (0x00F8)



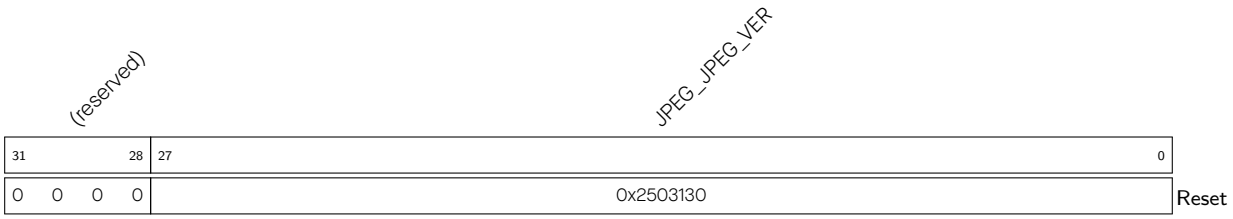
JPEG_CLK_EN 配置是否强制打开寄存器时钟门。

0：只有在应用程序写入寄存器时才打开时钟门

1：强制打开寄存器的时钟门

(R/W)

Register 17.44. JPEG_VERSION_REG (0x00FC)



JPEG_JPEG_VER 版本控制寄存器。(R/W)

第 18 章

像素处理加速器 (PPA)

18.1 概述

ESP32-S31 包括一个像素处理加速器 (PPA)。该模块为 2D 图像显示设计，用于对一些图像算法进行硬件加速，实现图像的旋转、缩放、镜像、混合功能。

PPA 主要包括三大功能模块：缩放 - 旋转 - 镜像 (SRM) 、图层混合 (BLEND)、填充（FILL）。SRM 模块实现图像旋转、缩放、XY 方向镜像功能；BLEND 模块实现根据 Alpha 通道（即透明度）对两个相同尺寸的图层进行混合的功能；FILL 模块根据配置的分辨率和像素数据生成一帧完整图像，并填充至系统内存。

PPA 按照像素块进行图像处理。像素块需要通过 2D-DMA 从内存里获取，PPA 与 2D-DMA 的连接关系如图 18.1-1 所示。

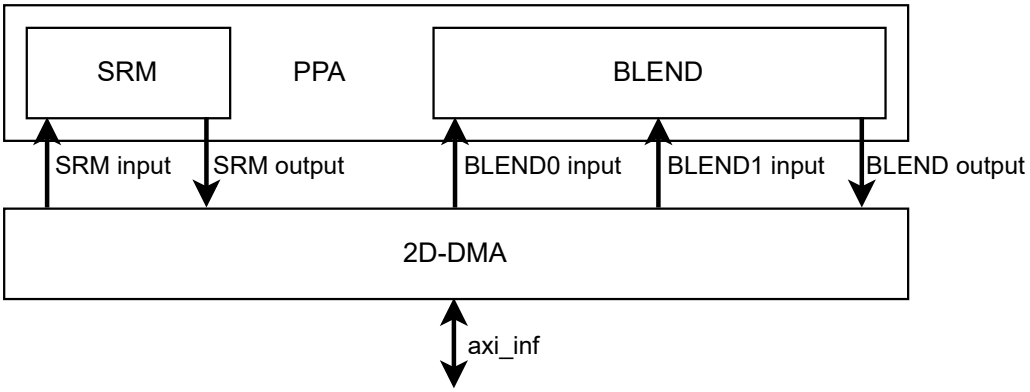


图 18.1-1. PPA 2D-DMA 连接关系

18.2 术语

为更好描述 PPA 功能，本章定义了以下术语：

图像 系统内存中存储的一幅完整图像
图像块 特指按一定尺寸从图像中截取的一块，最大尺寸等同于图像。即 2D-DMA 2D-MODO 下的 hb、vb 大小
像素块 特指从图像块中截取的一块，SRM 以像素块为单位读写数据

18.3 主要特性

- SRM 支持图像块旋转、缩放、镜像：
  - 输入格式支持 ARGB8888、RGB888、RGB565、YUV422、YUV420、GRAY

  - 输出格式支持 ARGB8888、RGB888、RGB565、YUV422、YUV420、GRAY
  - 逆时针旋转角度支持 0°、90°、180°、270°
  - 水平、垂直方向独立缩放支持 8 位整数及 4 位小数
  - 水平、垂直方向镜像
- BLEND 支持两个相同尺寸的图层混合以及输出特定像素的填充图像：
  - 前景输入格式支持 ARGB8888、RGB888、RGB565、L4、L8、A4、A8
  - 背景输入格式支持 ARGB8888、RGB888、RGB565、YUV422、YUV420、GRAY、L4、L8
  - 输出格式支持 ARGB8888、RGB888、RGB565、YUV422、YUV420、GRAY
  - 基于 Alpha 通道的图层混合，若图层没有 Alpha 通道，可通过寄存器配置提供
  - 前景和背景支持通过设置 color-key 范围实现特殊颜色抠图
  - 支持图像填充

18.4 结构概览

PPA 结构如图 18.4-1 所示，包括两个独立的功能模块：SRM 和 BLEND。

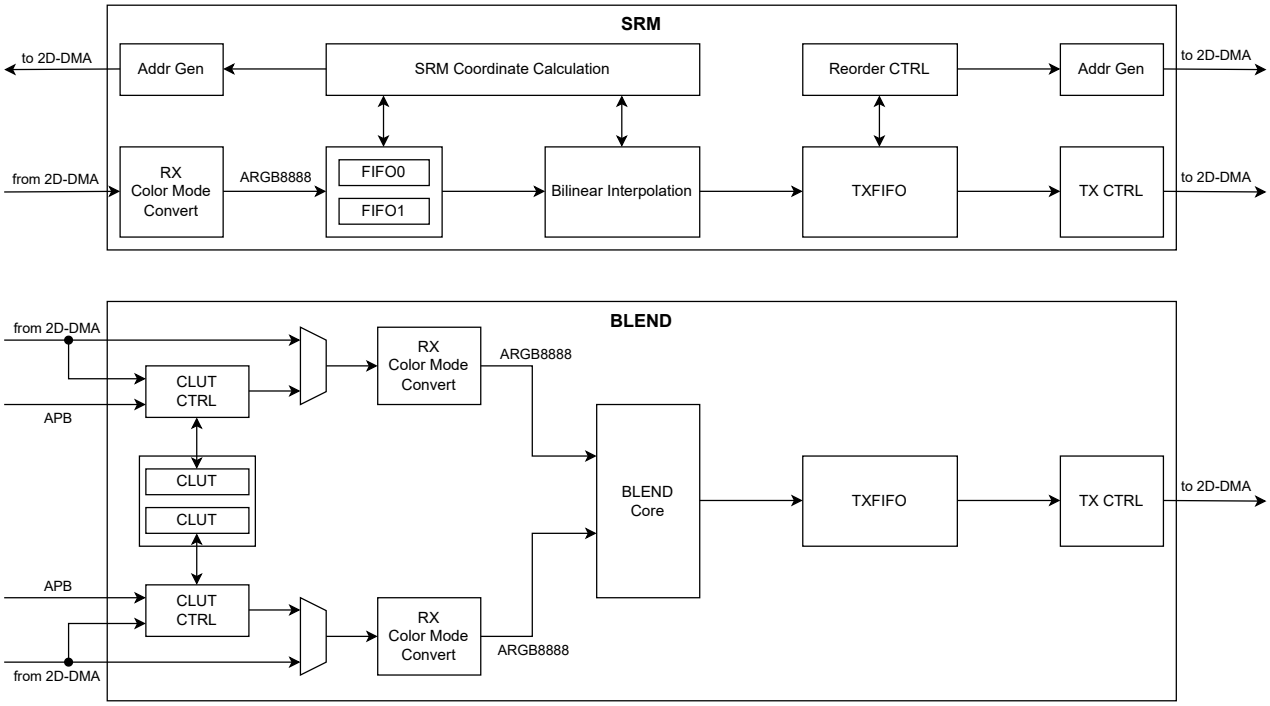


图 18.4-1. PPA 系统架构

对于 SRM，SRM Coordinate Calculation 对输出像素块坐标进行扫描，计算每个输出像素块所对应的输入像素块坐标。Addr Gen 再通过输入像素块坐标计算地址，发送给 2D-DMA。之后，PPA 从 2D-DMA 接受像素块数据，根据寄存器配置转成 ARGB8888 格式，并存入 RX FIFO（由两块 SRAM 组成的乒乓 FIFO，每一块都能完整存下整个像素块）。当一个完整像素块缓存完成后，Bilinear Interpolation 从 RX FIFO 中读取输入像素，经过插值运算得到输出像素并输出到 TX FIFO。Reorder CTRL 再根据寄存器配置，计算输出像素块坐标以及地址发送给

2D-DMA，同时按照输出像素块坐标从 TX FIFO 取出对应像素发送给 2D-DMA，由 2D-DMA 存到对应地址。

对于 BLEND，PPA 也会从 2D-DMA 接受数据，并根据输入数据的类型再转换为 ARGB8888 格式。转换后的像素会直接进入算法模块 BLEND Core，完成后再进入 TX FIFO 缓存并通过 2D-DMA 输出到系统内存。与 SRM 不同的是，BLEND 不需要自己计算地址，完全由 2D-DMA 通过链表控制。

18.5 功能描述

18.5.1 PPA 颜色空间

PPA 支持真彩色与伪彩色。真彩色是指在组成一幅彩色图像的每个像素中，存在 R、G、B 三个基色分量，每个分量直接决定显示设备的基色强度。PPA 支持的真彩色格式包括 ARGB8888、RGB888、RGB565。

伪彩色是指一幅彩色图像中每个像素的颜色不是由基色分量数值直接决定，而是把像素值当作彩色查找表 (Color Look-Up Table, CLUT) 的入口，去查找一个像素对应的 A、R、G、B 值。PPA 支持入口地址 8 位或 4 位位宽的 CLUT。

除此之外，PPA 的 SRM 操作以及 BLEND 背景图层，还支持 YUV422、YUV420、GRAY 数据格式。表 18.5-1 所示为支持颜色格式在内存中的存储顺序。其中，输入的 YUV422 存储格式可通过 SRM 或 BLEND 的 YUV422_BYTE_ORDER 寄存器配置，输出固定按照 YUV422_BYTE_ORDER = 1 存储。

表 18.5-1. 像素存储顺序

颜色格式	起始地址 + 3	起始地址 + 2	起始地址 + 1	起始地址 + 0
ARGB8888	A0[7:0]	R0[7:0]	G0[7:0]	B0[7:0]
RGB888	B1[7:0]	R0[7:0]	G0[7:0]	B0[7:0]
	G2[7:0]	B2[7:0]	R1[7:0]	G1[7:0]
	R3[7:0]	G3[7:0]	B3[7:0]	R2[7:0]
RGB565	R1[4:0]G1[5:3]	G1[2:0]B1[4:0]	R0[4:0]G0[5:3]	G0[2:0]B0[4:0]
L8	L3[7:0]	L2[7:0]	L1[7:0]	L0[7:0]
L4	L7[3:0]L6[3:0]	L5[3:0]L4[3:0]	L3[3:0]L2[3:0]	L1[3:0]L0[3:0]
A8	A3[7:0]	A2[7:0]	A1[7:0]	A0[7:0]
A4	A7[3:0]A6[3:0]	A5[3:0]A4[3:0]	A3[3:0]A2[3:0]	A1[3:0]A0[3:0]
YUV422 YUV422_BYTE_ORDER=0	Y1[7:0]	V1[7:0]	Y0[7:0]	U0[7:0]
YUV422 YUV422_BYTE_ORDER=1	Y1[7:0]	U1[7:0]	Y0[7:0]	V0[7:0]
YUV422 YUV422_BYTE_ORDER=2	V1[7:0]	Y1[7:0]	U0[7:0]	Y0[7:0]
YUV422 YUV422_BYTE_ORDER=3	U1[7:0]	Y1[7:0]	V0[7:0]	Y0[7:0]
YUV420 (1、3、5... 等奇数行)	U2[7:0]	Y1[7:0]	Y0[7:0]	U0[7:0]

见下页

表 18.5-1 – 接上页

颜色格式	起始地址 + 3	起始地址 + 2	起始地址 + 1	起始地址 + 0
	Y4[7:0]	U4[7:0]	Y3[7:0]	Y2[7:0]
	Y7[7:0]	Y6[7:0]	U6[7:0]	Y5[7:0]
YUV420	V2[7:0]	Y1[7:0]	Y0[7:0]	V0[7:0]
(2、4、6... 等	Y4[7:0]	V4[7:0]	Y3[7:0]	Y2[7:0]
偶数行)	Y7[7:0]	Y6[7:0]	V6[7:0]	Y5[7:0]
GRAY	P3[7:0]	P2[7:0]	P1[7:0]	P0[7:0]

18.5.1.1 SRM 颜色空间

SRM 的输入输出均支持 ARGB8888、RGB888、RGB565、YUV422、YUV420、GRAY 格式，输入格式由 PPA_SRM_RX_CM 配置，输出格式由 PPA_SRM_TX_CM 配置。其中 SRM 输入还有一系列像素处理流程，如图 18.5-1 所示。

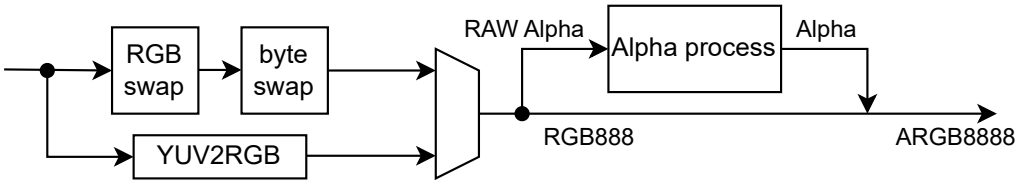


图 18.5-1. PPA SRM 输入像素处理

输入类型为 ARGB8888、RGB888、RGB565 时，支持进行 RGB 以及字节顺序翻转。具体配置分别如下：

- 向 PPA_SRM_RX_RGB_SWAP_EN 写 1 将 RGB 翻转为 BGR，将 ARGB 翻转为 BGRA。
- 向 PPA_SRM_RX_BYTE_SWAP_EN 写 1 分别将 0、1 字节和 2、3 字节翻转（即 {0，1，2，3} 翻转为 {1，0，3，2}）。由于 RGB888 一个像素只有 3 字节，因此不会受字节翻转寄存器配置影响。

SRM 内部均使用 ARGB8888 进行处理。当输入不带 Alpha 通道时，图像的 Alpha 通道值默认为 255。Alpha 通道还可以通过寄存器 PPA_SRM_FIX_ALPHA_REG 进一步配置，如图 18.5-2 所示。具体的配置效果可参考表 18.5-2。

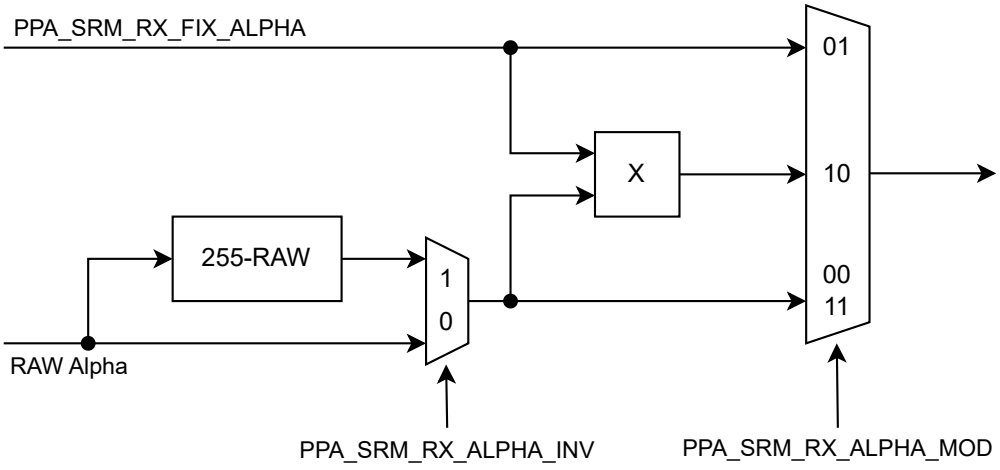


图 18.5-2. PPA SRM Alpha 配置逻辑

表 18.5-2. PPA SRM Alpha 配置效果

PPA_SRM_RX_ALPHA_INV	PPA_SRM_RX_ALPHA_MOD	输出结果
0	00/11	输出原始 Alpha 值
	01	输出 PPA_SRM_RX_FIX_ALPHA 配置的值
	10	输出原始 Alpha 值与 PPA_SRM_RX_FIX_ALPHA 相乘后的高 8 位
1	00/11	输出 255 减去原始 Alpha 值后得到的值
	01	输出 PPA_SRM_RX_FIX_ALPHA 配置值
	10	输出 255 减去原始 Alpha 值后再与 PPA_SRM_RX_FIX_ALPHA 相乘后的高 8 位

需注意，当输入或输出类型为 YUV 时，还需进一步通过寄存器配置 YUV、RGB 颜色转换。

当输入格式为 YUV422 或 YUV420 时，需通过寄存器域 PPA_YUV_RX_RANGE 将输入 YUV 数据范围配置为 full-range 或 limit-range，并通过寄存器域 PPA_YUV2RGB_PROTOCOL 将 YUV 转为 RGB 使用的协议配置为 BT601 或 BT709。转换公式如下：

- YUV_{full} 转为 YUV_{limit}
  - $Y_{limit} = \frac{220}{256} Y_{full} + 16$
  - $U_{limit} = \frac{225}{256} U_{full} + 16$
  - $V_{limit} = \frac{225}{256} V_{full} + 16$
- YUV_{limit} 按照 BT601 转为 RGB
  - $R = \frac{298}{256} Y_{limit} + \frac{409}{256} V_{limit} - \frac{56906}{256}$
  - $G = \frac{298}{256} Y_{limit} - \frac{100}{256} U_{limit} - \frac{208}{256} V_{limit} + \frac{34707}{256}$
  - $B = \frac{298}{256} Y_{limit} + \frac{516}{256} U_{limit} - \frac{70836}{256}$
- YUV_{limit} 按照 BT709 转为 RGB
  - $R = \frac{298}{256} Y_{limit} + \frac{459}{256} V_{limit} - \frac{63367}{256}$
  - $G = \frac{298}{256} Y_{limit} - \frac{55}{256} U_{limit} - \frac{136}{256} V_{limit} + \frac{19681}{256}$
  - $B = \frac{298}{256} Y_{limit} + \frac{541}{256} U_{limit} - \frac{73918}{256}$

当输出格式选为 YUV422 或 YUV420 时，需通过寄存器域 PPA_YUV_TX_RANGE 将输出 YUV 数据范围配置为 full-range 或 limit-range，通过寄存器域 PPA_RGB2YUV_PROTOCOL 将 RGB 转为 YUV 使用的协议配置为 BT601 或 BT709。转换公式如下：

- YUV_{limit} 转为 YUV_{full}
  - $Y_{full} = \frac{298}{256} Y_{limit} + \frac{4768}{256}$
  - $U_{full} = \frac{291}{256} U_{limit} + \frac{4550}{256}$
  - $V_{full} = \frac{291}{256} V_{limit} + \frac{4550}{256}$
- RGB 按照 BT601 转为 YUV_{limit}
  - $Y_{limit} = \frac{4096}{256} + \frac{66}{256} R + \frac{129}{256} G + \frac{25}{256} B$

  - $U_{limit} = \frac{32768}{256} - \frac{38}{256}R - \frac{74}{256}G + \frac{112}{256}B$
  - $V_{limit} = \frac{32768}{256} + \frac{112}{256}R - \frac{94}{256}G - \frac{18}{256}B$
- RGB 按照 BT709 转为 YUV_{limit}
  - $Y_{limit} = \frac{4096}{256} + \frac{47}{256}R + \frac{157}{256}G + \frac{16}{256}B$
  - $U_{limit} = \frac{32768}{256} - \frac{26}{256}R - \frac{86}{256}G + \frac{112}{256}B$
  - $V_{limit} = \frac{32768}{256} + \frac{112}{256}R - \frac{102}{256}G - \frac{10}{256}B$

当输入或输出数据类型为 GRAY 时，需要进一步配置相关寄存器。

当输入数据类型配置为 GRAY 时，SRM 会将 GRAY 像素值复制到 RGB 的三个通道，从而完成 GRAY 到 RGB 的转换。

当输出数据类型配置为 GRAY 时，RGB 数据会通过寄存器 PPA_RGB2GRAY_REG 转换为 GRAY。转换公式如下：

$$GRAY = \frac{R*PPA_RGB2GRAY_R+G*PPA_RGB2GRAY_G+B*PPA_RGB2GRAY_B}{256}$$

18.5.1.2 BLEND 颜色空间

BLEND 前景输入支持 ARGB8888、RGB888、RGB565、L4、L8、A4、A8，背景输入支持 ARGB8888、RGB888、RGB565、YUV422、YUV420、GRAY、L4、L8，可通过寄存器域 PPA_BLEND0/1_RX_CM 分别配置前景、背景图层的输入颜色格式。其中，A4、A8 只支持前景图层，且当输入类型为 L4、A4 时，图像块的尺寸 hb 以及在图像中的偏移 x 必须为偶数。BLEND 输出支持 ARGB8888、RGB888、RGB565、YUV422、YUV420、GRAY，可通过寄存器域 PPA_BLEND_TX_CM 进行配置。

BLEND 的 RGB 翻转、字节翻转、Alpha 通道配置和 SRM 输入一致。另外，当前景图层数据类型为 A4、A8 时，输入数据为 Alpha 通道数值。此时若 PPA_BLEND_BYPASS 配置为 1，则前景图层输入的 Alpha 值会替换掉背景图层的 Alpha 通道并输出；配置为 0，则为正常的 BLEND 操作，前景图层的 RGB 值由 PPA_BLEND_RGB_REG 指定。

当输入数据类型为 L4、L8 时，需要先对 CLUT 进行初始化。BLEND 有两个深度为 256、宽度为 32 的 CLUT，初始化有两种方式，分别为 FIFO 模式、MEM 模式。

- 向 PPA_APB_FIFO_MASK 写 0 进入 FIFO 模式。
  FIFO 模式下，通过读写 PPA_RDWR_WORD_BLENDx_CLUT 实现访问 CLUT。其中 0 对应背景通道 CLUT，1 对应前景通道 CLUT。
  每次读/写该寄存器，会自动将对应的读/写地址加 1，可通过向 PPA_BLENDx_CLUT_MEM_RST 写 1 再写 0 复位读写地址，可通过向PPA_BLENDx_CLUT_MEM_RDADDR_RST 写 1 再写 0 复位读地址。
- 向 PPA_APB_FIFO_MASK 写 1 进入 MEM 模式。
  MEM 模式下，可直接通过地址访问 CLUT。在 PPA 的总线地址范围内，CLUT 地址如表 18.5-3 所示。

表 18.5-3. BLEND CLUT 地址

addr[11:10]	addr[9:2]	addr[1:0]
01：选择 BLEND0 CLUT 10：选择 BLEND1 CLUT	CLUT 实际地址，0 ~ 255	00

18.5.2 PPA 2D-DMA 链表配置

PPA 通过 2D-DMA 与系统内存进行数据交互。对于 SRM 或 BLEND，都需要一次性配置好所需操作图像块的 2D-DMA 链表信息。2D-DMA 链表如图 18.5-3，详细可参考 4 *2D-DMA 控制器 (2D-DMA)*：

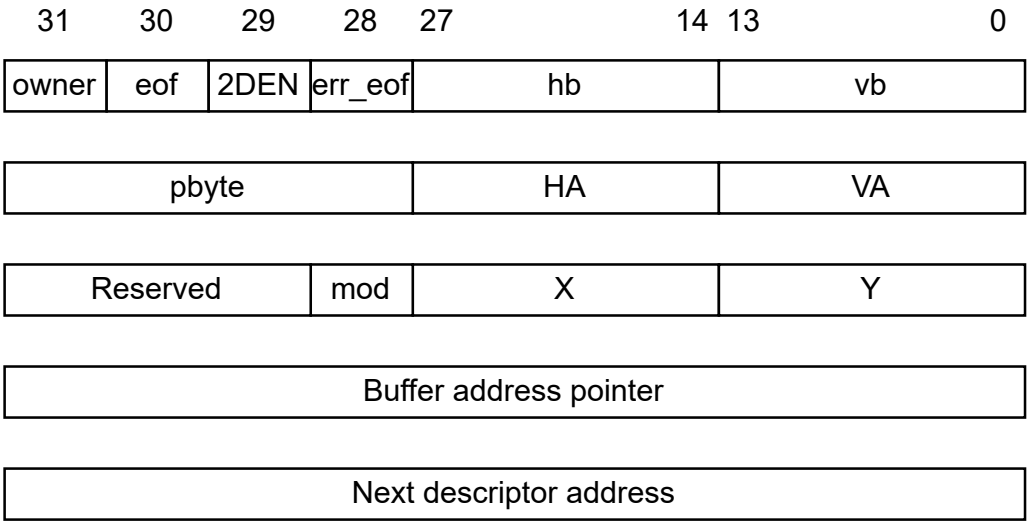


图 18.5-3. 2D-DMA 链表

对于 PPA，对应的链表参数如表 18.5-4 所示。

表 18.5-4. PPA 链表配置

域	配置
owner	1
eof	1
2DEN	1
err_eof	0
hb	图像块水平方向尺寸 若输入或输出数据为 YUV422 或 YUV420， 则必须为偶数
vb	图像块垂直方向尺寸 若输入或输出数据为 YUV420，则必须为偶数
pbyte	ARGB8888：5（4 字节/像素） RGB888：4（3 字节/像素） RGB565/YUV422：3（2 字节/像素） YUV420：2（1.5 字节/像素） GRAY/A8/L8：1（1 字节/像素） A4/L4：0（0.5 字节/像素）
HA	整幅图像水平方向尺寸 若输入或输出数据为 YUV422 或 YUV420， 则必须为偶数
VA	整幅图像垂直方向尺寸 若输入或输出数据为 YUV420，则必须为偶数

见下页

表 18.5-4 – 接上页

域	配置
mod	0
X	图像块在图像中的水平方向偏移（偏移数从 0 开始） 若输入或输出数据为 YUV422 或 YUV420， 则必须为偶数
Y	图像块在图像中的垂直方向偏移（偏移数从 0 开始） 若输入或输出数据为 YUV420，则必须为偶数
Buffer address pointer	图像起始地址
Next descriptor address	下一描述符地址

18.5.2.1 SRM 特殊配置

当配置好输入输出图像块的 2D-DMA 链表后，SRM 需要以像素块为单位，从输入图像块存储空间中选择出所需像素块，处理完成后再将其存储到输出图像块对应存储空间。这个过程中，每一个像素块的起始地址均需要受 SRM 的控制。为实现该功能，需要向 2D-DMA 对应通道的 DMA2D_IN/OUT_DSCR_PORT_EN_CHx 寄存器域写 1 以使能描述符接口（DSCR-PORT 模式），从而允许 PPA 实时控制下一个像素块的起始地址。

在数据输入方向，SRM 必须每次均能接收到一个完整的像素块。当像素块选择到达图像块边界时，需要 2D-DMA 将像素块超出边界部分自动补齐，可通过向 2D-DMA 对应通道的寄存器域 DMA2D_OUT_DSCR_PORT_BLK_V/H_CHx 配置为像素块大小实现。配置值根据图像格式而不同，具体如下表 18.5-5。

表 18.5-5. 自动补全块大小配置

像素格式	DMA2D_OUT_DSCR_PORT_BLK_H_CHx	DMA2D_OUT_DSCR_PORT_BLK_V_CHx
ARGB8888 RGB888 RGB565 GRAY	34	34
YUV422	36	34
YUV420	36	36

18.5.3 缩放 - 旋转 - 镜像 (SRM)

18.5.3.1 基本功能

SRM 实现缩放、旋转、镜像三个功能，先将图像基于坐标原点进行缩放，再基于中心点（X_{middle}，Y_{middle}）进行旋转，最后基于中心点（X_{target}，Y_{target}）进行水平、垂直方向镜像，如图 18.5-4 所示。缩放、旋转、镜像参数均通过寄存器进行配置：

- PPA_SRM_SCAL_X_INT 水平方向缩放系数，整数部分
- PPA_SRM_SCAL_X_FRAG 水平方向缩放系数，小数部分
- PPA_SRM_SCAL_Y_INT 垂直方向缩放系数，整数部分

- PPA_SRM_SCAL_Y_FRAG 垂直方向缩放系数，小数部分
- PPA_SRM_ROTATE_ANGLE 逆时针旋转角度
- PPA_SRM_MIRROR_X 水平方向镜像使能
- PPA_SRM_MIRROR_Y 垂直方向镜像使能

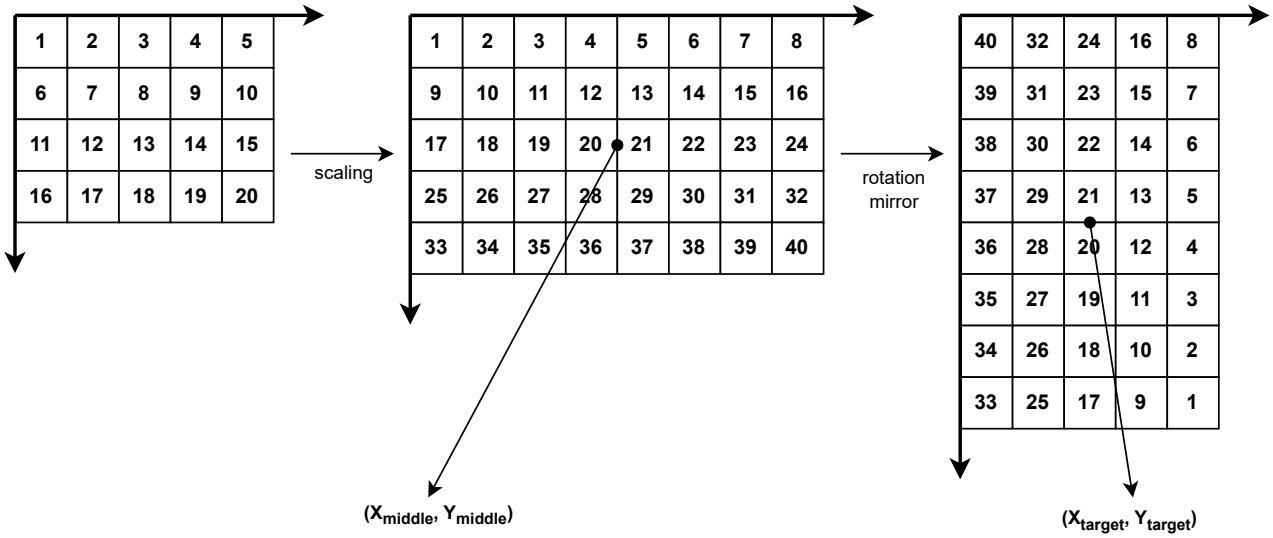


图 18.5-4. SRM 步骤示例

18.5.3.2 像素块重排

当输出像素块较小时，如果每一个像素块都发起一次 2D-DMA 传输，会导致总线带宽利用率较低。为了提高总线带宽利用率，SRM 能对输出像素块进行缓存拼接，拼接为一个更大的像素块后再一次性输出。该功能默认打开，可以通过将 PPA_SRM_MACRO_BK_RO_BYPASS 置 1 关闭。

像素块重排功能开启后，当输出像素块尺寸满足一定条件，会自动进行重排拼接，数量如表 18.5-6 所示。

- 当输出像素块为 ARGB8888/RGB888 格式时，若输出像素块垂直高度小于等于 32，且水平宽度小于等于 16 时，像素块重排功能会自动开启。
- 当输出像素块为 RGB565/YUV422/GRAY 格式时，若输出像素块垂直高度小于等于 32，且水平宽度小于等于 32 时，像素块重排功能会自动开启。
- 当输出像素块为 YUV420 格式时，若输出像素块垂直高度小于等于 32，且水平宽度小于等于 32 时，像素块重排功能会自动开启。

表 18.5-6. SRM 输出像素块重排数量

ARGB8888/RGB888		RGB565/YUV422/GRAY		YUV420	
水平尺寸	重排个数	水平尺寸	重排个数	水平尺寸（偶数）	重排个数
1	32	1	64	2	32
2	16	2	32	4	16
3	10	3	20	6	10
4	8	4	16	8	8

见下页

表 18.5-6 – 接上页

ARGB8888/RGB888		RGB565/YUV422/GRAY		YUV420	
水平尺寸	重排个数	水平尺寸	重排个数	水平尺寸（偶数）	重排个数
5	6	5	12	10	6
6	5	6	10	12	5
7	4	7	9	14	4
8	4	8	8	16	4
9	3	9	7	18	3
10	3	10	6	20-32	2
11	2	11	5		
12	2	12	5		
13	2	13	4		
14	2	14	4		
15	2	15	4		
16	2	16	4		
		17	3		
		18	3		
		19-32	2		

18.5.4 图层混合 (BLEND)

18.5.4.1 基本功能

BLEND 把两幅同尺寸图像块通过 Alpha 通道进行混合，再输出。混合公式如下，其中 Ab 为背景图层 Alpha 通道，Af 为前景图层 Alpha 通道，Cb 对应背景图层 R、G、B 分量，Cf 对应前景图层 R、G、B 分量：

- $Aout = Ab + Af - Ab * Af$
- $Cout = (Cb * Ab * (1 - Af) + Cf * Af)/(Ab + Af - Ab * Af)$

同时，BLEND 支持基于像素颜色的 color-key 抠图，通过 PPA_CK_FG/BG_HIGH/LOW_REG 配置颜色范围，可将图像在逻辑上分为 4 个不同选区，并对每个选区执行不同的操作：

- 当前景像素在 color-key 范围内，背景像素在 color-key 范围外，此时输出背景像素。
- 当背景像素在 color-key 范围内，前景像素在 color-key 范围外，此时若 PPA_COLORKEY_FG_BG_REVERSE 为 0，则输出背景像素；为 1，则输出前景像素。
- 当背景、前景像素均在 color-key 范围内，此时输出 PPA_COLORKEY_DEFAULT_R/G/B 设置的颜色。
- 当背景、前景像素均不在 color-key 范围内，此时按照正常 Alpha Blending 流程输出。

当 PPA_BLEND_BYPASS 置为 1 后，BLEND 会直接输出背景图层的数据；若此时前景输入数据类型为 A4/A8，则前景输入数据会替换背景输入数据的 Alpha 通道再输出。

18.5.4.2 填充图像输出

BLEND 还支持填充图像输出。该功能开启后，BLEND 只有输出端会工作，连续不断输出填充图像。填充图像的尺寸以及数据可分别通过 PPA_BLEND_TX_SIZE_REG 和 PPA_BLEND_TX_FIX_PIXEL 设置。

18.6 中断

ESP32-S31 PPA 包含一个中断信号，该中断会输出到 9中断矩阵 *[to be added later]*。

- PPA_INTR

PPA_INTR 中断由内部多个中断源产生，包括：

- PPA_SRM_PARAM_CFG_ERR_INT：当 SRM 配置了错误参数触发。用户可通过 PPA_SRM_PARAM_ERR_ST_REG 获取具体错误类型，如表 18.6-1 所示。
- PPA_BLEND_PARAM_CFG_ERR_INT：当 BLEND 配置了错误参数触发。用户可通过 PPA_BLEND_ST_REG 获取具体错误类型，如表 18.6-2 所示。
- PPA_SRM_EOF_INT：当 SRM 完成一次图像块处理触发。
- PPA_BLEND_EOF_INT：当 BLEND 完成一次图像块处理触发。

表 18.6-1. SRM 参数错误类型

错误类型	详情
PPA_TX_DSCR_VB_ERR_ST	SRM 输出图像块垂直尺寸加上输出链表配置的垂直偏移 Y 超出输出链表配置的整幅图像垂直尺寸 VA
PPA_TX_DSCR_HB_ERR_ST	SRM 输出图像块水平尺寸加上输出链表配置的水平偏移 X 超出输出链表配置的整幅图像水平尺寸 HA
PPA_Y_RX_SCAL_EQUAL_0_ERR_ST	SRM 输入图像块垂直方向缩放系数为 0
PPA_RX_DSCR_VB_ERR_ST	SRM 输入链表配置的垂直方向图像块尺寸 VB 加上垂直方向偏移 Y 大于输入链表配置的整幅图像垂直尺寸 VA
PPA_YDST_LEN_TOO_SAMLL_ERR_ST	SRM 缩放后的图像块垂直方向尺寸为 0。例如，图像块垂直尺寸为 14，垂直缩放系数为 1/16，不进行旋转，此时缩放后的图像块垂直尺寸为 0
PPA_YDST_LEN_TOO_LARGE_ERR_ST	SRM 缩放后的图像块垂直方向尺寸大于 8191
PPA_X_RX_SCAL_EQUAL_0_ERR_ST	SRM 输入图像块水平方向缩放系数为 0
PPA_RX_DSCR_HB_ERR_ST	SRM 输入链表配置的水平方向图像块尺寸 HB 加上水平方向偏移 X 大于输入链表配置的整幅图像水平尺寸 HA
PPA_XDST_LEN_TOO_SAMLL_ERR_ST	SRM 缩放后的图像块水平方向尺寸为 0
PPA_XDST_LEN_TOO_LARGE_ERR_ST	SRM 缩放后的图像块水平方向尺寸大于 8191
PPA_X_YUV_RX_SCALE_ERR_ST	当 SRM 输入为 YUV422 或 YUV420 时，输入链表的 HA/hb/X 参数为奇数
PPA_Y_YUV_RX_SCALE_ERR_ST	当 SRM 输入为 YUV420 时，输入链表的 VA/vb/Y 参数为奇数
PPA_X_YUV_TX_SCALE_ERR_ST	当 SRM 输出为 YUV422 或 YUV420 时，输出链表的 HA/hb/X 参数为奇数
PPA_Y_YUV_TX_SCALE_ERR_ST	当 SRM 输出为 YUV420 时，输出链表的 VA/vb/Y 参数为奇数

表 18.6-2. BLEND 参数错误类型

错误类型	详情
PPA_BLEND_SIZE_DIFF_ST	BLEND 前景和背景尺寸不一致

见下页

表 18.6-2 – 接上页

错误类型	详情
PPA_BLEND_YUV_X_SCALE_ERR_ST	BLEND 背景图像格式为 YUV422 或 YUV420 时，水平尺寸为奇数
PPA_BLEND_YUV_Y_SCALE_ERR_ST	BLEND 背景图像格式为 YUV420 时，垂直尺寸为奇数

18.7 配置流程

18.7.1 PPA 时钟复位配置流程

1. 打开 PPA、2D-DMA 模块的时钟：
   - 向 HP_SYS_CLKRST_REG_PPA_SYS_CLK_EN 写 1 打开 PPA 时钟
   - 向 HP_SYS_CLKRST_REG_DMA2D_SYS_CLK_EN 写 1 打开 2D-DMA 时钟
2. 复位 PPA、2D-DMA 模块：
   - 向 HP_SYS_CLKRST_REG_PPA_RST_EN 写 1 再写 0 复位 PPA
   - 向 HP_SYS_CLKRST_REG_DMA2D_RST_EN 写 1 再写 0 复位 2D-DMA

18.7.2 SRM 配置流程

1. 参考 18.7.1 打开 PPA、2D-DMA 时钟，复位 PPA、2D-DMA。
2. 配置 2D-DMA 输入输出链表，具体参数可参考表 18.5-4。
3. 选择 2D-DMA 输入输出通道：
   - 向 DMA2D_OUT_PERI_SEL_CHx 写 0x1，将一条 2D-DMA 输出通道分配给 PPA SRM 输入。
   - 向 DMA2D_IN_PERI_SEL_CHx 写 0x1，将一条 2D-DMA 输入通道分配给 PPA SRM 输出。
4. 对 PPA 使用的 2D-DMA 通道使能描述符接口：
   - 向 DMA2D_OUT_DSCR_PORT_EN_CHx 写 1 使能描述符接口。
   - 向 DMA2D_IN_DSCR_PORT_EN_CHx 写 1 使能描述符接口。
5. 配置 2D-DMA 描述符接口补全参数：
   - 配置 DMA2D_OUT_DSCR_PORT_BLK_H_CHx 水平补全参数，详见表 18.5-5。
   - 配置 DMA2D_OUT_DSCR_PORT_BLK_V_CHx 垂直补全参数，详见表 18.5-5。
6. 开启 2D-DMA 输入输出通道：
   - 向 DMA2D_OUTLINK_START_CHx 写 1 开启 2D-DMA 输出通道。
   - 向 DMA2D_INLINK_START_CHx 写 1 开启 2D-DMA 输入通道。
7. 配置 SRM 参数：
   - 配置 PPA_SRM_BK_SIZE_SEL 设置处理图像块尺寸。
   - 配置 PPA_SRM_RX_CM 设置 SRM 输入数据类型。
   - 配置 PPA_SRM_TX_CM 设置 SRM 输出数据类型。

- 配置 PPA_SRM_SCAL_X_INT 设置水平方向缩放系数的整数部分。
- 配置 PPA_SRM_SCAL_X_FRAG 设置水平方向缩放系数的小数部分。
- 配置 PPA_SRM_SCAL_Y_INT 设置垂直方向缩放系数，整数部分。
- 配置 PPA_SRM_SCAL_Y_FRAG 设置垂直方向缩放系数，小数部分。
- 配置 PPA_SRM_ROTATE_ANGLE 设置逆时针旋转角度。
- 配置 PPA_SRM_MIRROR_X 设置水平方向镜像使能。
- 配置 PPA_SRM_MIRROR_Y 设置垂直方向镜像使能。

8. 向 PPA_SCAL_ROTATE_START 写 1 开启 SRM 工作。

18.7.3 BLEND CLUT 配置流程

1. FIFO 模式：
   - 向 PPA_APB_FIFO_MASK 写 0 进入 FIFO 模式
   - 写 PPA_RDWR_WORD_BLEND0_CLUT 初始化背景图层 CLUT
   - 写 PPA_RDWR_WORD_BLEND1_CLUT 初始化前景图层 CLUT
2. MEM 模式：
   - 向 PPA_APB_FIFO_MASK 写 1 进入 MEM 模式。
   - 直接向对应地址写数据初始化 CLUT，可参考表 18.5-3。

18.7.4 BLEND 配置流程

1. 参考 18.7.1 打开 PPA、2D-DMA 时钟，复位 PPA、2D-DMA。
2. 配置 2D-DMA 输入输出链表，具体参数可参考表 18.5-4。
3. 选择 2D-DMA 输入输出通道：
   - 向 DMA2D_OUT_PERI_SEL_CHx 写 0x2，将一条 2D-DMA 输出通道分配给 PPA BLEND 背景层输入。
   - 向 DMA2D_OUT_PERI_SEL_CHx 写 0x3，将一条 2D-DMA 输出通道分配给 PPA BLEND 前景层输入。
   - 向 DMA2D_IN_PERI_SEL_CHx 写 0x2，将一条 2D-DMA 输入通道分配给 PPA BLEND 输出。
4. 开启 2D-DMA 输入输出通道。其中，x 为分配给 BLEND 前景、背景的两条通道：
   - 向 DMA2D_OUTLINK_START_CHx 写 1 开启 2D-DMA 输出通道
   - 向 DMA2D_INLINK_START_CHx 写 1 开启 2D-DMA 输入通道
5. 配置 BLEND 参数：
   - 配置 PPA_BLEND0_RX_CM 设置 BLEND 背景图层输入数据类型。若为 L4/L8，需要先按照 18.7.3 初始化 CLUT
   - 配置 PPA_BLEND1_RX_CM 设置 BLEND 前景图层输入数据类型。若为 L4/L8，需要先按照 18.7.3 初始化 CLUT
   - 配置 PPA_BLEND_TX_CM 设置 BLEND 输出数据类型

- 配置 PPA_CK_BG_HIGH_REG、PPA_CK_BG_LOW_REG 分别配置背景图层 color-key 上下限
- 配置 PPA_CK_FG_HIGH_REG、PPA_CK_FG_LOW_REG 分别配置前景图层 color-key 上下限
- 配置 PPA_CK_DEFAULT_REG 分别配置 color-key 默认值以及行为
- 向 PPA_BLEND_EN 写 1 使能 BLEND

6. 向 PPA_BLEND_TRANS_MODE_UPDATE 写 1 开启 BLEND 工作。

18.7.5 BLEND 图像填充配置流程

1. 参考 18.7.1 打开 PPA、2D-DMA 时钟，复位 PPA、2D-DMA。
2. 配置 2D-DMA 输入（PPA BLEND 输出）链表，具体参数可参考表 18.5-4。
3. 向 DMA2D_IN_PERI_SEL_CHx 写 0x2，将一条 2D-DMA 输入通道分配给 PPA BLEND 输出。
4. 向 DMA2D_INLINK_START_CHx 写 1 开启 2D-DMA 输入通道。
5. 配置 BLEND 参数：
   - 配置 PPA_BLEND_TX_CM 设置 BLEND 输出数据类型
   - 向 PPA_BLEND_FIX_PIXEL_FILL_EN 写 1 使能图像填充功能
   - 配置 PPA_BLEND_TX_FIX_PIXEL 设置填充图像像素值
   - 配置 PPA_BLEND_HB、PPA_BLEND_VB 设置填充图像尺寸
6. 向 PPA_BLEND_TRANS_MODE_UPDATE 写 1 开启 BLEND 工作。

18.7.6 错误处理流程

- 参考 18.7.1 复位整个 PPA，再重新配置
- 或者向 PPA_SCAL_ROTATE_RST、PPA_BLEND_RST 写 1 再写 0 分别复位 SRM、BLEND，再重新配置

说明：

- 本章节中所有 2D-DMA 配置只涉及一些关键步骤，详细流程可参考 *4 2D-DMA 控制器 (2D-DMA)*。
- 本章节中，DMA2D_OUT 方向对应 PPA 输入方向，DMA2D_IN 方向对应 PPA 输出方向。

18.8 寄存器列表

本小节的所有地址均为相对于像素处理加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问权限
配置寄存器			
PPA_BLEND0_CLUT_DATA_REG	BLEND 背景图层 CLUT SRAM 读写寄存器	0x0000	R/W
PPA_BLEND1_CLUT_DATA_REG	BLEND 前景图层 CLUT SRAM 读写寄存器	0x0004	R/W
PPA_CLUT_CONF_REG	BLEND CLUT 配置寄存器	0x000C	R/W
PPA_SRM_COLOR_MODE_REG	SRM 颜色空间配置寄存器	0x0020	R/W
PPA_BLEND_COLOR_MODE_REG	BLEND 颜色空间配置寄存器	0x0024	R/W
PPA_SRM_BYTE_ORDER_REG	SRM 字节顺序控制寄存器	0x0028	R/W
PPA_BLEND_BYTE_ORDER_REG	BLEND 字节顺序控制寄存器	0x002C	R/W
PPA_BLEND_TRANS_MODE_REG	BLEND 控制寄存器	0x0034	varies
PPA_SRM_FIX_ALPHA_REG	SRM 透明度通道配置寄存器	0x0038	R/W
PPA_BLEND_TX_SIZE_REG	BLEND 填充图像尺寸配置寄存器	0x003C	R/W
PPA_BLEND_FIX_ALPHA_REG	BLEND 透明度通道配置寄存器	0x0040	R/W
PPA_BLEND_RGB_REG	BLEND RGB 颜色配置寄存器	0x0048	R/W
PPA_BLEND_FIX_PIXEL_REG	BLEND 填充图像像素配置寄存器	0x004C	R/W
PPA_CK_FG_LOW_REG	BLEND 前景图层 color-key 下阈值配置寄存器	0x0050	R/W
PPA_CK_FG_HIGH_REG	BLEND 前景图层 color-key 上阈值配置寄存器	0x0054	R/W
PPA_CK_BG_LOW_REG	BLEND 背景图层 color-key 下阈值配置寄存器	0x0058	R/W
PPA_CK_BG_HIGH_REG	BLEND 背景图层 color-key 上阈值配置寄存器	0x005C	R/W
PPA_CK_DEFAULT_REG	BLEND color-key 默认值配置寄存器	0x0060	R/W
PPA_SRM_SCAL_ROTATE_REG	SRM 配置寄存器	0x0064	varies
PPA_REG_CONF_REG	寄存器时钟门控控制	0x006C	R/W
PPA_RGB2GRAY_REG	RGB 转 GRAY 系数寄存器	0x0098	R/W
中断寄存器			
PPA_INT_RAW_REG	中断源状态寄存器	0x0010	R/WTC/SS
PPA_INT_ST_REG	可屏蔽中断状态寄存器	0x0014	RO
PPA_INT_ENA_REG	中断使能寄存器	0x0018	R/W
PPA_INT_CLR_REG	中断清除寄存器	0x001C	WT
状态寄存器			
PPA_CLUT_CNT_REG	BLEND CLUT 写计数器状态寄存器	0x0070	RO
PPA_BLEND_ST_REG	BLEND 状态寄存器	0x0074	RO
PPA_SRM_PARAM_ERR_ST_REG	SRM 配置错误状态寄存器	0x0078	RO
调试寄存器			
PPA_DEBUG_CTRL0_REG	调试寄存器	0x0090	R/W
PPA_DEBUG_CTRL1_REG	调试寄存器	0x0094	R/W

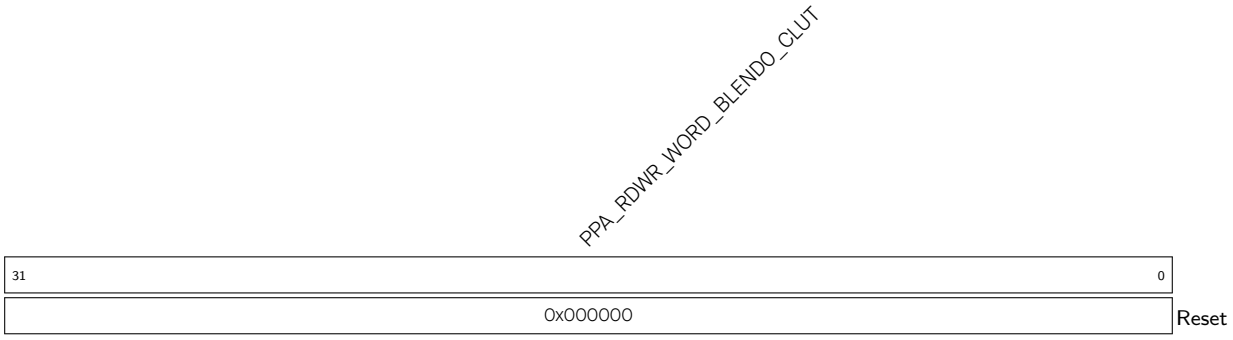
名称	描述	地址	访问权限
版本寄存器			
PPA_DATE_REG	PPA 版本寄存器	0x0100	R/W

18.9 寄存器

本小节的所有地址均为相对于像素处理加速器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

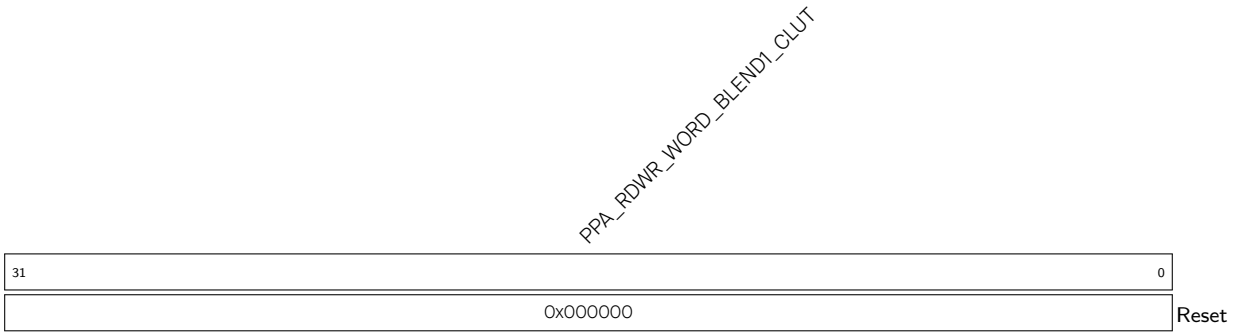
关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 18.1. PPA_BLEND0_CLUT_DATA_REG (0x0000)



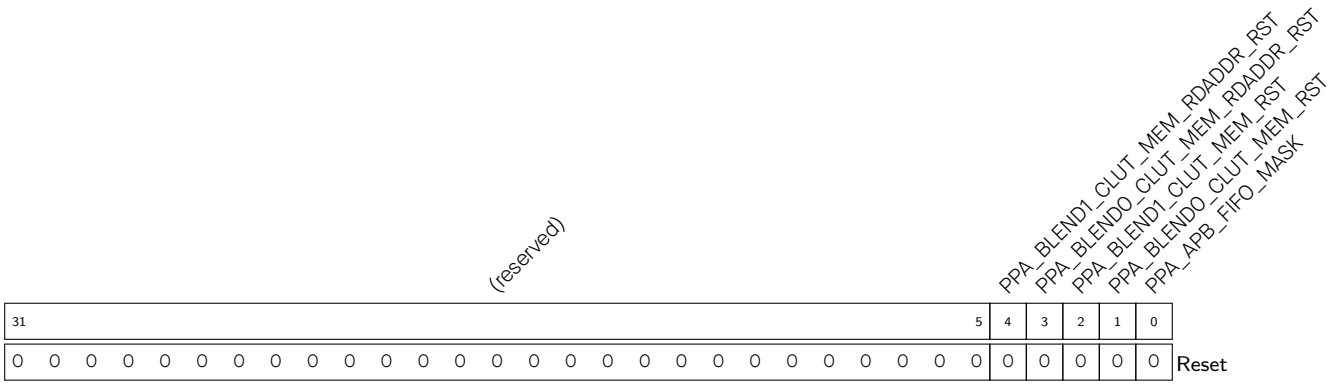
PPA_RDWR_WORD_BLEND0_CLUT FIFO 模式下 BLEND 背景图层 CLUT 访问寄存器。(R/W)

Register 18.2. PPA_BLEND1_CLUT_DATA_REG (0x0004)



PPA_RDWR_WORD_BLEND1_CLUT FIFO 模式下 BLEND 前景图层 CLUT 访问寄存器。(R/W)

Register 18.3. PPA_CLUT_CONF_REG (0x000C)



PPA_APB_FIFO_MASK 配置 BLEND CLUT 的访问方式。

1'b0: FIFO 模式访问 CLUT，具体可见 18.5.1.2。

1'b1: MEM 模式访问 CLUT，具体可见 18.5.1.2。

(R/W)

PPA_BLENDO_CLUT_MEM_RST 配置 BLENDO CLUT 逻辑的复位。

0：释放复位

1：复位

(R/W)

PPA_BLEND1_CLUT_MEM_RST 配置 BLEND1 CLUT 逻辑的复位。

0：释放复位

1：复位

(R/W)

PPA_BLENDO_CLUT_MEM_RDADDR_RST 配置 FIFO 模式下 BLENDO CLUT 的读地址复位。

0：释放复位

1：复位

(R/W)

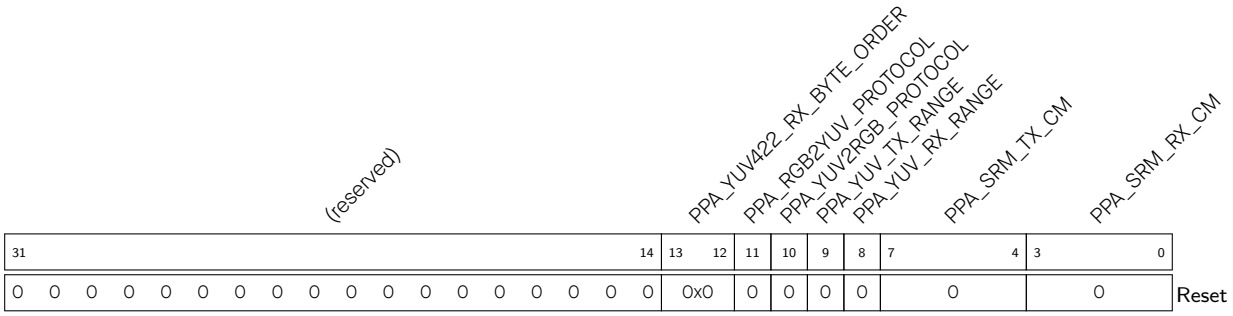
PPA_BLEND1_CLUT_MEM_RDADDR_RST 配置 FIFO 模式下 BLEND1 CLUT 的读地址复位。

0：释放复位

1：复位

(R/W)

Register 18.4. PPA_SRM_COLOR_MODE_REG (0x0020)



PPA_SRM_RX_CM 配置 SRM 输入图像颜色格式。

0: ARGB8888

1: RGB888

2: RGB565

8: YUV420

9: YUV422

12: GRAY

其他：保留

(R/W)

PPA_SRM_TX_CM 配置 SRM 输出图像颜色格式。

0: ARGB8888

1: RGB888

2: RGB565

8: YUV420

9: YUV422

12: GRAY

其他：保留

(R/W)

PPA_YUV_RX_RANGE 配置 SRM 输入为 YUV 时的 YUV 范围。

0：限制范围

1：全范围

(R/W)

PPA_YUV_TX_RANGE 配置 SRM 输出为 YUV 时的 YUV 范围。

0：限制范围

1：全范围

(R/W)

PPA_YUV2RGB_PROTOCOL 配置 SRM 输入方向 YUV 转 RGB 使用协议。

0: BT601

1: BT709

(R/W)

PPA_RGB2YUV_PROTOCOL 配置 SRM 输出方向 RGB 转 YUV 使用协议。

0: BT601

1: BT709

(R/W)

Register 18.4. PPA_SRM_COLOR_MODE_REG (0x0020)

接上页...

PPA_YUV422_RX_BYTE_ORDER 配置当 SRM 输入方向为 YUV422 时的排列格式。

0：YVYU

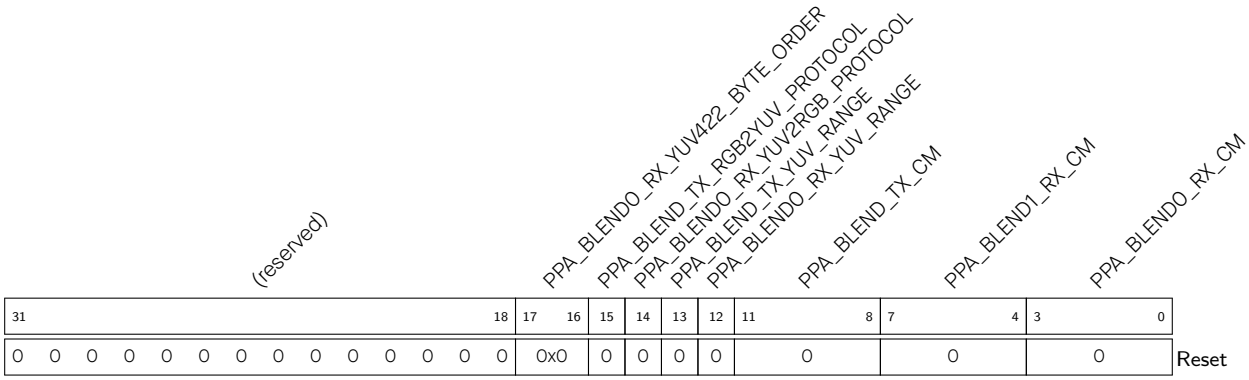
1：YUYV

2：VYUY

3：UYVY

(R/W)

Register 18.5. PPA_BLEND_COLOR_MODE_REG (0x0024)



PPA_BLEND0_RX_CM 配置 BLEND 背景图层输入颜色格式。

0: ARGB8888
1: RGB888
2: RGB565
4: L8
5: L4
8: YUV420
9: YUV422
12: GRAY
其他：保留
(R/W)

PPA_BLEND1_RX_CM 配置 BLEND 前景图层输入颜色格式。

0: ARGB8888
1: RGB888
2: RGB565
4: L8
5: L4
6: A8
7: A4
其他：保留
(R/W)

PPA_BLEND_TX_CM 配置 BLEND 输出颜色格式。

0: ARGB8888
1: RGB888
2: RGB565
8: YUV420
9: YUV422
12: GRAY
其他：保留
(R/W)

见下页...

Register 18.5. PPA_BLEND_COLOR_MODE_REG (0x0024)

接上页...

PPA_BLEND0_RX_YUV_RANGE 配置 BLEND 背景输入为 YUV 时的 YUV 范围。

0：限制范围

1：全范围

(R/W)

PPA_BLEND_TX_YUV_RANGE 配置 BLEND 输出为 YUV 时的 YUV 范围。

0：限制范围

1：全范围

(R/W)

PPA_BLEND0_RX_YUV2RGB_PROTOCOL 配置 BLEND 背景输入方向 YUV 转 RGB 使用协议。

0: BT601

1: BT709

(R/W)

PPA_BLEND_TX_RGB2YUV_PROTOCOL 配置 BLEND 输出方向 RGB 转 YUV 使用协议。

0: BT601

1: BT709

(R/W)

PPA_BLEND0_RX_YUV422_BYTE_ORDER 配置当 BLEND 背景输入方向为 YUV422 时的排列格式。

0：YVYU

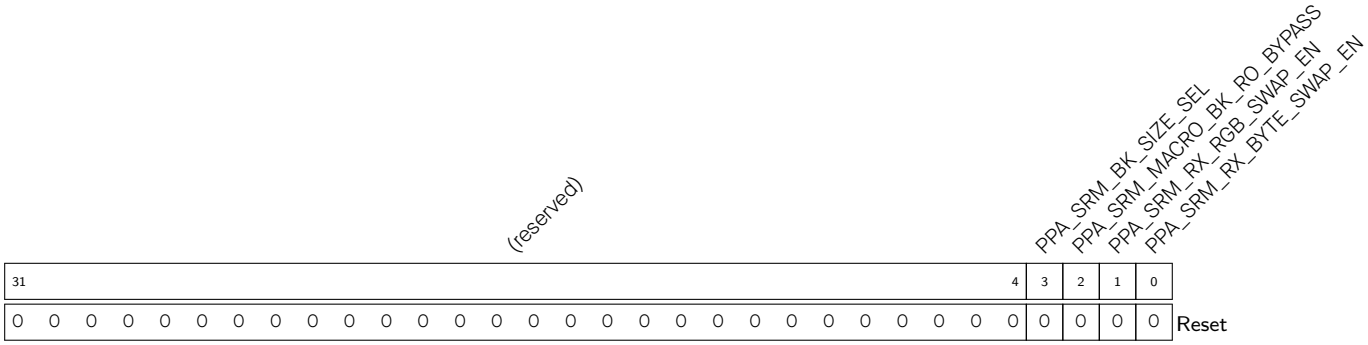
1：YUYV

2：VYUY

3：UYVY

(R/W)

Register 18.6. PPA_SRM_BYTE_ORDER_REG (0x0028)



PPA_SRM_RX_BYTE_SWAP_EN 配置 SRM 有效数据输入是否按字节进行翻转。

0：不翻转

1：翻转

(R/W)

PPA_SRM_RX_RGB_SWAP_EN 配置是否在 SRM 输入为 RGB 或 ARGB 时翻转各通道排列。

0：不翻转

1：RGB 翻转为 BGR，ARGB 翻转为 BGRA

(R/W)

PPA_SRM_MACRO_BK_RO_BYPASS 配置像素块重排功能。

0：打开像素块重排功能，自动计算是否需要重排

1：关闭像素块重排功能

(R/W)

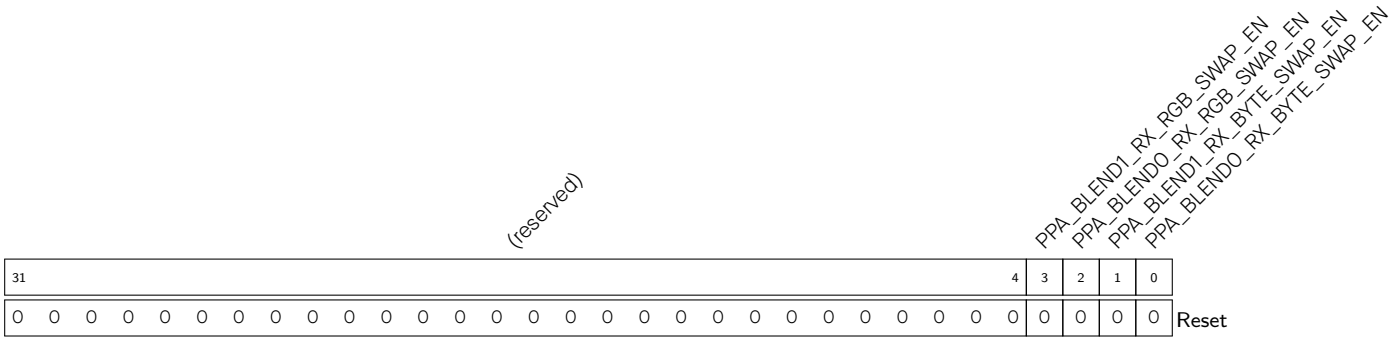
PPA_SRM_BK_SIZE_SEL 配置 SRM 像素块尺寸。

0：32x32

1：16x16

(R/W)

Register 18.7. PPA_BLEND_BYTE_ORDER_REG (0x002C)



PPA_BLEND0_RX_BYTE_SWAP_EN 配置 BLEND 背景图层有效数据输入是否按字节进行翻转。

0：不翻转

1：翻转

(R/W)

PPA_BLEND1_RX_BYTE_SWAP_EN 配置 BLEND 前景图层有效数据输入是否按字节进行翻转。

0：不翻转

1：翻转

(R/W)

PPA_BLEND0_RX_RGB_SWAP_EN 配置是否在 BLEND 背景图层输入为 RGB 或 ARGB 时翻转通道排列。

0：不翻转 RGB

1：RGB 翻转为 BGR，ARGB 翻转为 BGRA

(R/W)

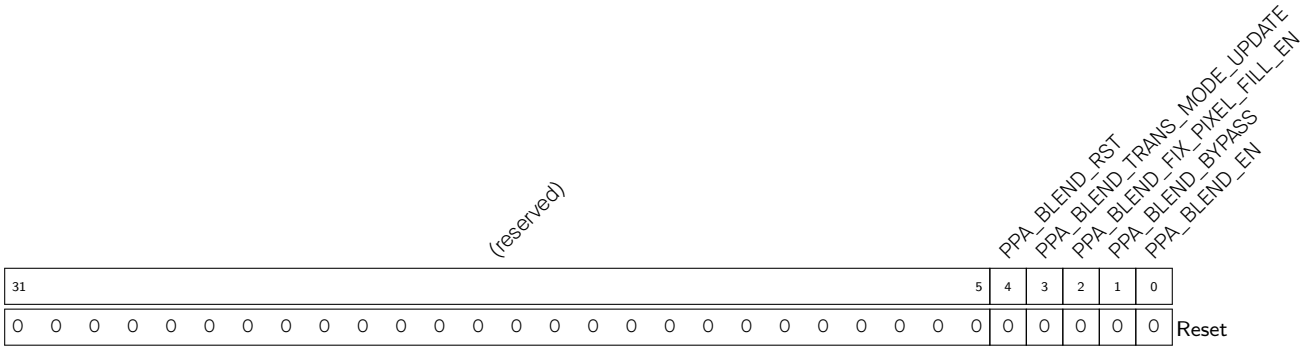
PPA_BLEND1_RX_RGB_SWAP_EN 配置是否在 BLEND 前景图层输入为 RGB 或 ARGB 时翻转通道排列。

0：不翻转 RGB

1：RGB 翻转为 BGR，ARGB 翻转为 BGRA

(R/W)

Register 18.8. PPA_BLEND_TRANS_MODE_REG (0x0034)



PPA_BLEND_EN 配置 BLEND 工作开关。

0：关闭 BLEND

1：打开 BLEND

(R/W)

PPA_BLEND_BYPASS 配置是否跳过 BLEND 直接输出背景图层。

0：不跳过 BLEND

1：跳过 BLEND

(R/W)

PPA_BLEND_FIX_PIXEL_FILL_EN 配置填充图像输出功能。

0：关闭填充图像输出功能

1：打开填充图像输出功能，此时只有 BLEND 输出工作，输出图像象素以及尺寸由 PPA_BLEND_TX_SIZE_REG 和 PPA_BLEND_TX_FIX_PIXEL 控制

(R/W)

PPA_BLEND_TRANS_MODE_UPDATE 写 1 更新传输模式，使 PPA_BLEND_FIX_PIXEL_FILL_EN 生效，并开启 BLEND 工作。(WT)

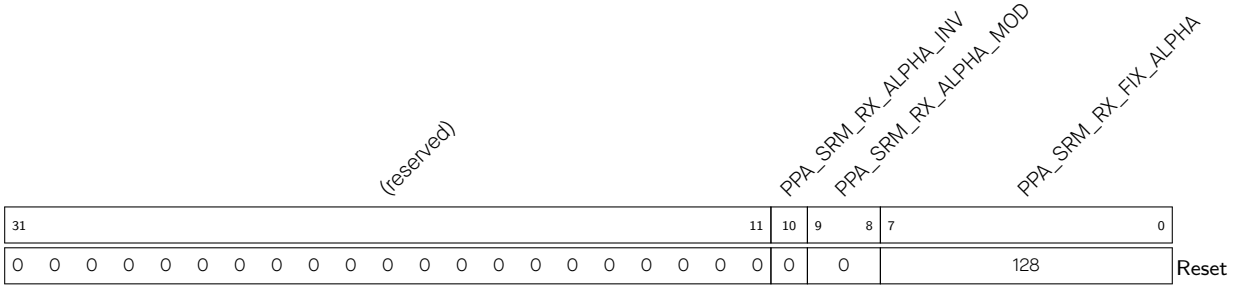
PPA_BLEND_RST 配置 BLEND 逻辑的复位。

0：释放复位

1：复位

(R/W)

Register 18.9. PPA_SRM_FIX_ALPHA_REG (0x0038)



PPA_SRM_RX_FIX_ALPHA 配置 SRM Alpha 通道值。(R/W)

PPA_SRM_RX_ALPHA_MOD 配置 SRM 输入数据的 Alpha 通道替换模式。

1：用 PPA_SRM_RX_FIX_ALPHA 作为最终 Alpha 值

2：原始 Alpha 值与 PPA_SRM_RX_FIX_ALPHA 相乘的高 8 位为最终 Alpha 值

其他：原始 Alpha 值为最终 Alpha 值

(R/W)

PPA_SRM_RX_ALPHA_INV 配置是否反相 SRM 输入数据的原始 Alpha 值。

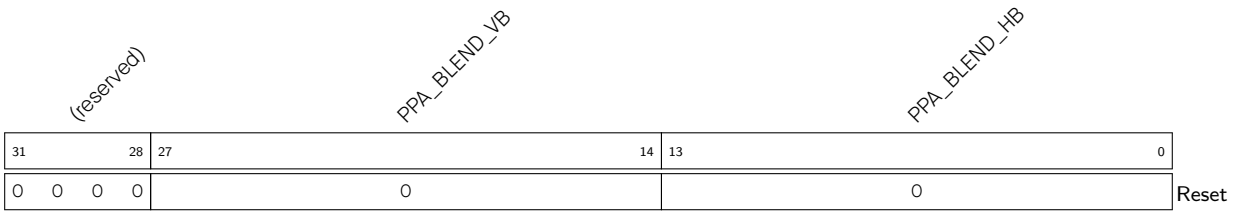
0：原始 Alpha 值不变

1：原始 Alpha 值反相，变为 255 减去原始 Alpha 值

对于不带 Alpha 通道的数据类型，原始 Alpha 值为 255

(R/W)

Register 18.10. PPA_BLEND_TX_SIZE_REG (0x003C)



PPA_BLEND_HB 配置填充图像输出功能下的水平方向尺寸，单位：像素。(R/W)

PPA_BLEND_VB 配置填充图像输出功能下的垂直方向尺寸，单位：像素。(R/W)

Register 18.11. PPA_BLEND_FIX_ALPHA_REG (0x0040)



PPA_BLEND0_RX_FIX_ALPHA 配置 BLEND 背景图层的 Alpha 通道值。(R/W)

PPA_BLEND1_RX_FIX_ALPHA 配置 BLEND 前景图层的 Alpha 通道值。(R/W)

PPA_BLEND0_RX_ALPHA_MOD 配置 BLEND 背景图层数据的 Alpha 通道替换模式。

1：用 PPA_BLEND0_RX_FIX_ALPHA 作为最终 Alpha 值

2：原始 Alpha 值与 PPA_BLEND0_RX_FIX_ALPHA 相乘的高 8 位为最终 Alpha 值

其他：原始 Alpha 值为最终 Alpha 值

(R/W)

PPA_BLEND1_RX_ALPHA_MOD 配置 BLEND 前景图层数据的 Alpha 通道替换模式。

1：用PPA_BLEND1_RX_FIX_ALPHA 作为最终 Alpha 值

2：原始 Alpha 值与 PPA_BLEND1_RX_FIX_ALPHA 相乘的高 8 位为最终 Alpha 值

其他：原始 Alpha 值为最终 Alpha 值

(R/W)

PPA_BLEND0_RX_ALPHA_INV 配置是否反相 BLEND 背景图层数据的原始 Alpha 值。

0：原始 Alpha 值不变

1：原始 Alpha 值反相，变为 255 减去原始 Alpha 值

对于不带 Alpha 通道的数据类型，原始 Alpha 值为 255。

(R/W)

PPA_BLEND1_RX_ALPHA_INV 配置是否反相 BLEND 前景图层数据的原始 Alpha 值。

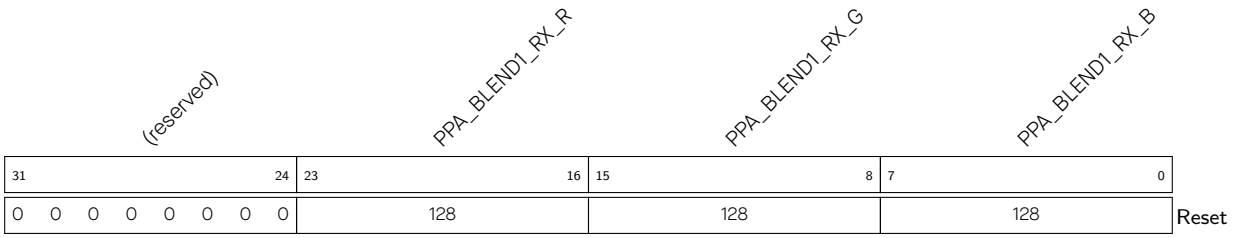
0：原始 Alpha 值不变

1：原始 Alpha 值反相，变为 255 减去原始 Alpha 值

对于不带 Alpha 通道的数据类型，原始 Alpha 值为 255。

(R/W)

Register 18.12. PPA_BLEND_RGB_REG (0x0048)

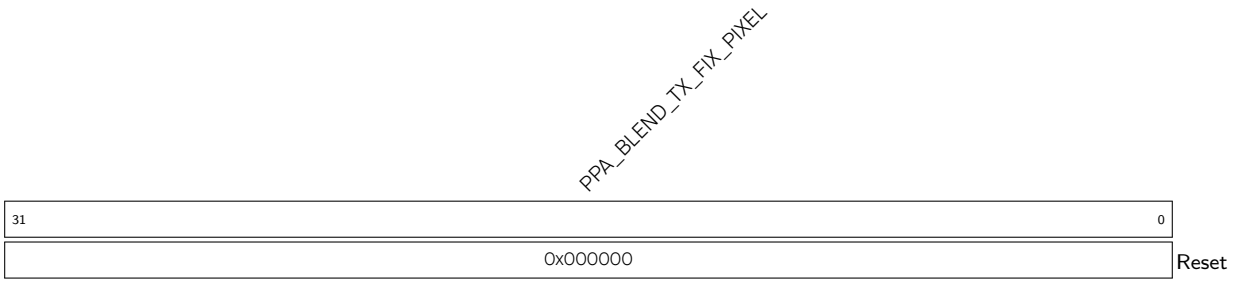


PPA_BLEND1_RX_B 配置 BLEND 前景图层为 A4/A8 时的 B 通道值。(R/W)

PPA_BLEND1_RX_G 配置 BLEND 前景图层为 A4/A8 时的 G 通道值。(R/W)

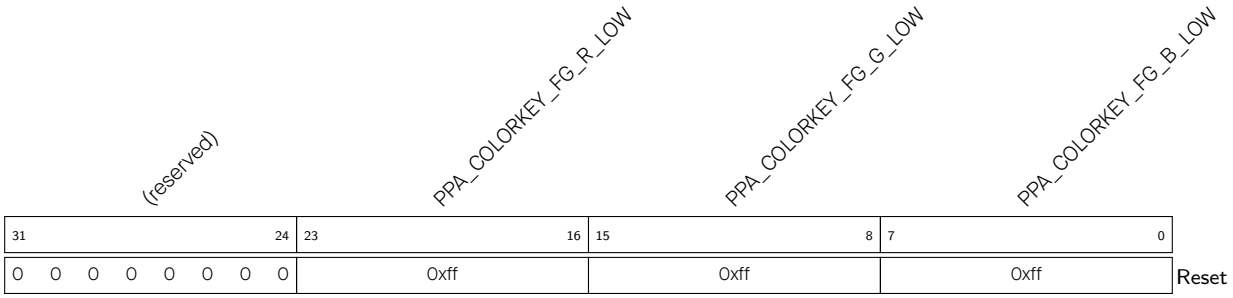
PPA_BLEND1_RX_R 配置 BLEND 前景图层为 A4/A8 时的 R 通道值。(R/W)

Register 18.13. PPA_BLEND_FIX_PIXEL_REG (0x004C)



PPA_BLEND_TX_FIX_PIXEL 配置 BLEND 填充图像输出模式下的像素值。
[31:24]：A，[23:16]：R，[15:8]：G，[7:0]：B。
(R/W)

Register 18.14. PPA_CK_FG_LOW_REG (0x0050)

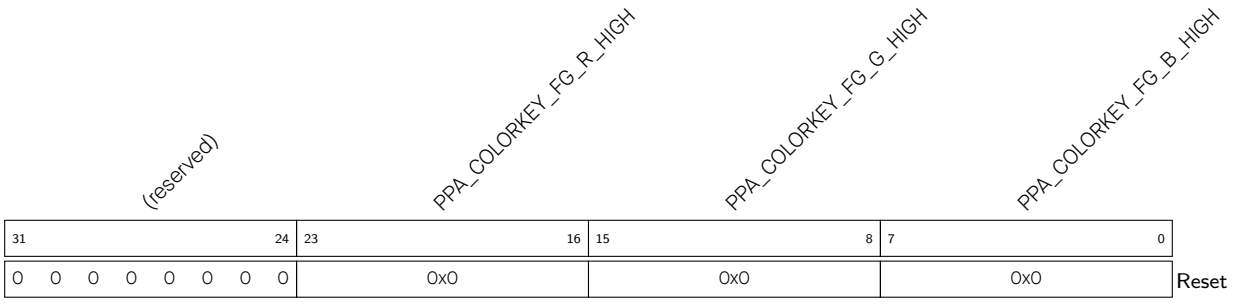


PPA_COLORKEY_FG_B_LOW 配置 BLEND 前景图层 B 通道 color-key 下阈值。(R/W)

PPA_COLORKEY_FG_G_LOW 配置 BLEND 前景图层 G 通道 color-key 下阈值。(R/W)

PPA_COLORKEY_FG_R_LOW 配置 BLEND 前景图层 R 通道 color-key 下阈值。(R/W)

Register 18.15. PPA_CK_FG_HIGH_REG (0x0054)

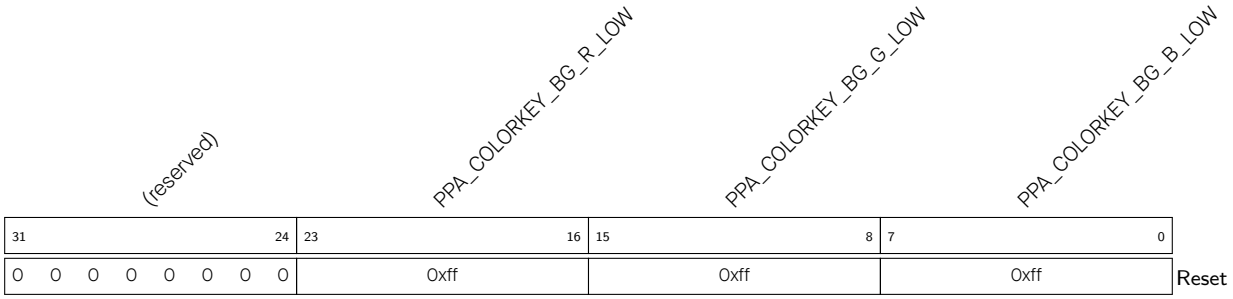


PPA_COLORKEY_FG_B_HIGH 配置 BLEND 前景图层 B 通道 color-key 上阈值。(R/W)

PPA_COLORKEY_FG_G_HIGH 配置 BLEND 前景图层 G 通道 color-key 上阈值。(R/W)

PPA_COLORKEY_FG_R_HIGH 配置 BLEND 前景图层 R 通道 color-key 上阈值。(R/W)

Register 18.16. PPA_CK_BG_LOW_REG (0x0058)

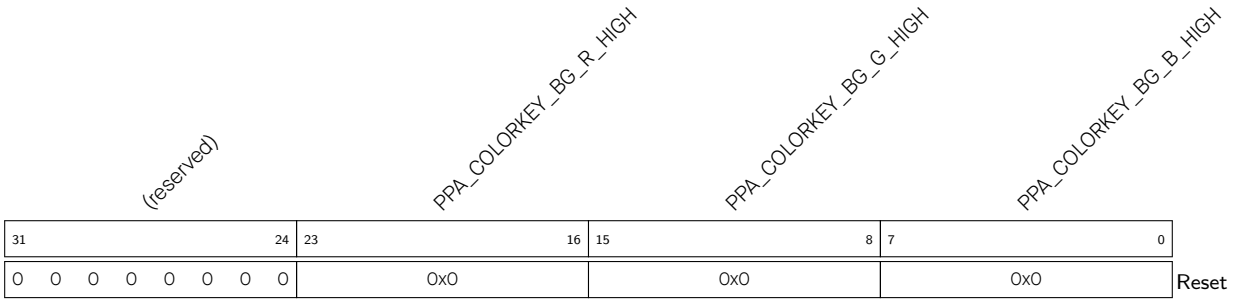


PPA_COLORKEY_BG_B_LOW 配置 BLEND 背景图层 B 通道 color-key 下阈值。(R/W)

PPA_COLORKEY_BG_G_LOW 配置 BLEND 背景图层 G 通道 color-key 下阈值。(R/W)

PPA_COLORKEY_BG_R_LOW 配置 BLEND 背景图层 R 通道 color-key 下阈值。(R/W)

Register 18.17. PPA_CK_BG_HIGH_REG (0x005C)



PPA_COLORKEY_BG_B_HIGH 配置 BLEND 背景图层 B 通道 color-key 上阈值。(R/W)

PPA_COLORKEY_BG_G_HIGH 配置 BLEND 背景图层 G 通道 color-key 上阈值。(R/W)

PPA_COLORKEY_BG_R_HIGH 配置 BLEND 背景图层 R 通道 color-key 上阈值。(R/W)

Register 18.18. PPA_CK_DEFAULT_REG (0x0060)



PPA_COLORKEY_DEFAULT_B 配置 BLEND color-key B 通道默认值。(R/W)

PPA_COLORKEY_DEFAULT_G 配置 BLEND color-key G 通道默认值。(R/W)

PPA_COLORKEY_DEFAULT_R 配置 BLEND color-key R 通道默认值。(R/W)

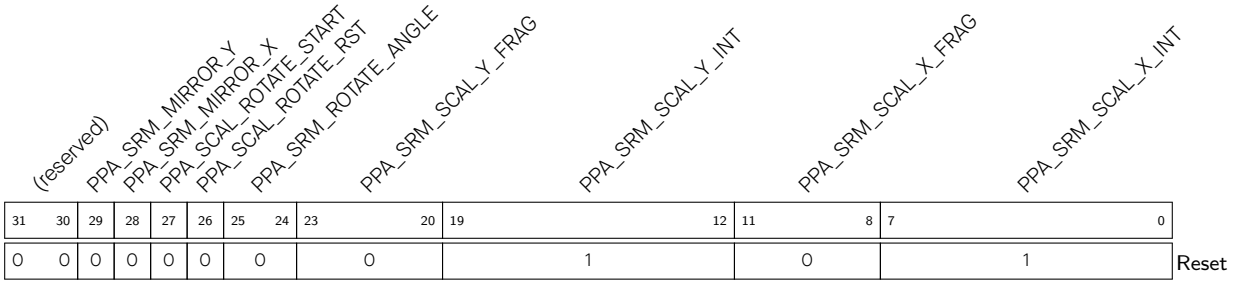
PPA_COLORKEY_FG_BG_REVERSE 配置当像素处于背景 color-key 范围但不在前景 color-key 范围时的处理方式

0：输出背景像素

1：输出前景像素

(R/W)

Register 18.19. PPA_SRM_SCAL_ROTATE_REG (0x0064)



PPA_SRM_SCAL_X_INT 配置 SRM 水平方向缩放系数整数部分。(R/W)

PPA_SRM_SCAL_X_FRAG 配置 SRM 水平方向缩放系数小数部分。(R/W)

PPA_SRM_SCAL_Y_INT 配置 SRM 垂直方向缩放系数整数部分。(R/W)

PPA_SRM_SCAL_Y_FRAG 配置 SRM 垂直方向缩放系数小数部分。(R/W)

PPA_SRM_ROTATE_ANGLE 配置 SRM 逆时针旋转角度

0: 0°
1: 90°
2: 180°
3: 270°
(R/W)

PPA_SCAL_ROTATE_RST 配置 SRM 逻辑的复位

0：释放复位
1：复位
(R/W)

PPA_SCAL_ROTATE_START 写 1 启动 SRM。(WT)

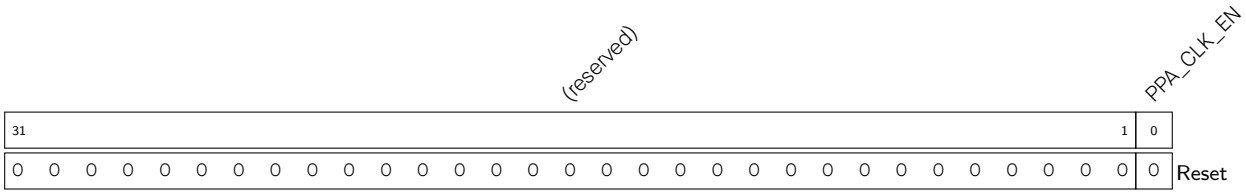
PPA_SRM_MIRROR_X 配置 SRM 水平方向镜像

0: 关闭水平镜像
1: 打开水平镜像
(R/W)

PPA_SRM_MIRROR_Y 配置 SRM 垂直方向镜像

0: 关闭垂直镜像
1: 打开垂直镜像
(R/W)

Register 18.20. PPA_REG_CONF_REG (0x006C)



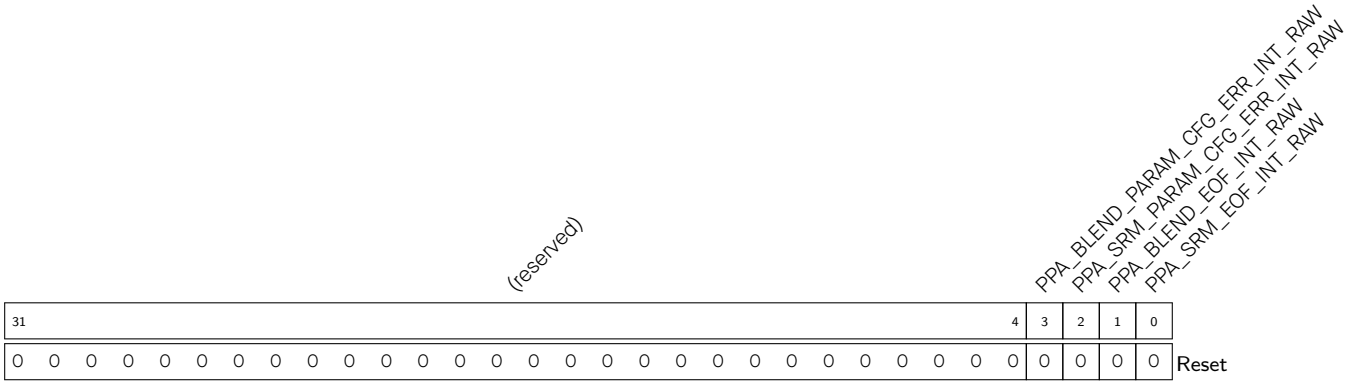
PPA_CLK_EN 配置是否常开 PPA 寄存器时钟。

0：时钟只会在发生寄存器访问时打开

1：时钟常开

(R/W)

Register 18.21. PPA_INT_RAW_REG (0x0010)



PPA_SRM_EOF_INT_RAW 表示 PPA_SRM_EOF_INT 的原始中断状态。

(R/WTC/SS)

PPA_BLEND_EOF_INT_RAW 表示 PPA_BLEND_EOF_INT 的原始中断状态。

(R/WTC/SS)

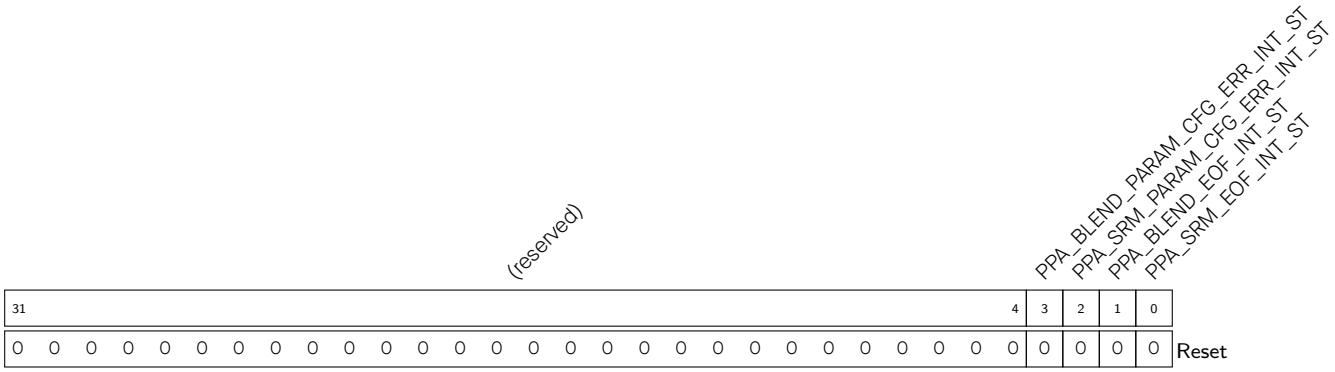
PPA_SRM_PARAM_CFG_ERR_INT_RAW 表示 PPA_SRM_PARAM_CFG_ERR_INT 的原始中断状态。可通过 PPA_SRM_PARAM_ERR_ST_REG 查看具体错误。

(R/WTC/SS)

PPA_BLEND_PARAM_CFG_ERR_INT_RAW 表示 PPA_BLEND_PARAM_CFG_ERR_INT 的原始中断状态。可通过 PPA_BLEND_ST_REG 查看具体错误。

(R/WTC/SS)

Register 18.22. PPA_INT_ST_REG (0x0014)



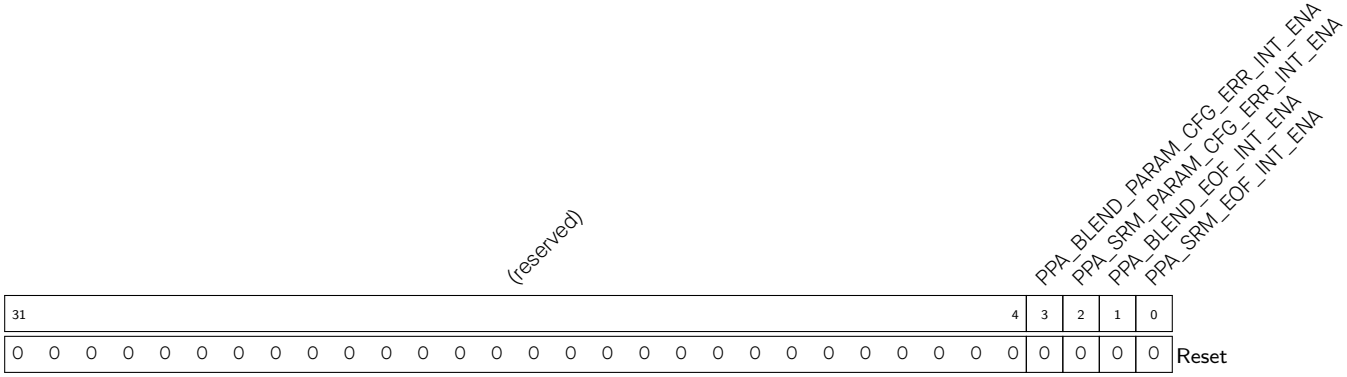
PPA_SRM_EOF_INT_ST 表示 PPA_SRM_EOF_INT 的屏蔽中断状态。(RO)

PPA_BLEND_EOF_INT_ST 表示 PPA_BLEND_EOF_INT 的屏蔽中断状态。(RO)

PPA_SRM_PARAM_CFG_ERR_INT_ST 表示 PPA_SRM_PARAM_CFG_ERR_INT 的屏蔽中断状态。(RO)

PPA_BLEND_PARAM_CFG_ERR_INT_ST 表示 PPA_BLEND_PARAM_CFG_ERR_INT 的屏蔽中断状态。(RO)

Register 18.23. PPA_INT_ENA_REG (0x0018)



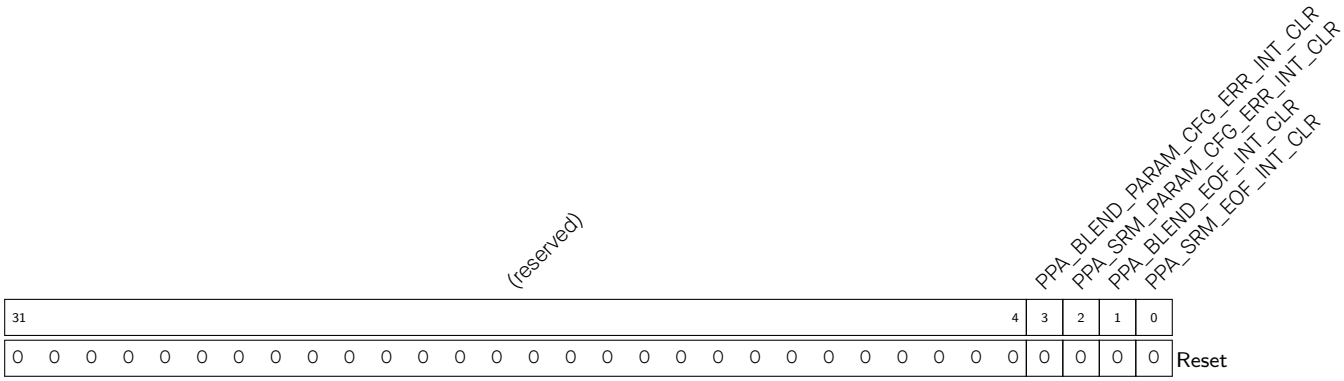
PPA_SRM_EOF_INT_ENA 写 1 使能 PPA_SRM_EOF_INT。(R/W)

PPA_BLEND_EOF_INT_ENA 写 1 使能 PPA_BLEND_EOF_INT。(R/W)

PPA_SRM_PARAM_CFG_ERR_INT_ENA 写 1 使能 PPA_SRM_PARAM_CFG_ERR_INT。(R/W)

PPA_BLEND_PARAM_CFG_ERR_INT_ENA 写 1 使能 PPA_BLEND_PARAM_CFG_ERR_INT。(R/W)

Register 18.24. PPA_INT_CLR_REG (0x001C)



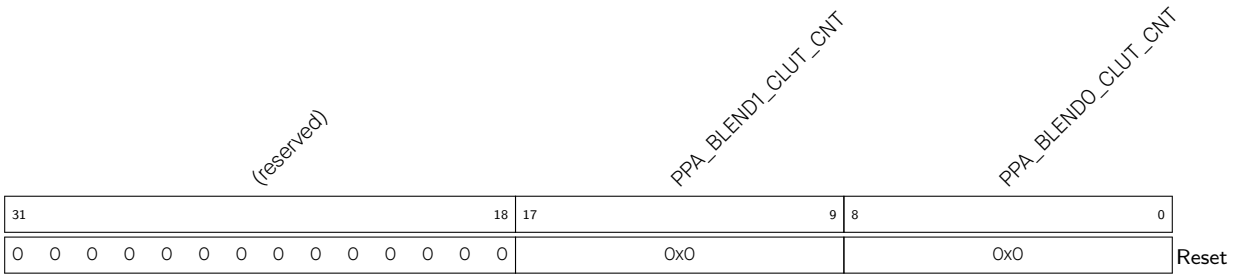
PPA_SRM_EOF_INT_CLR 写 1 清空 PPA_SRM_EOF_INT。(WT)

PPA_BLEND_EOF_INT_CLR 写 1 清空 PPA_BLEND_EOF_INT。(WT)

PPA_SRM_PARAM_CFG_ERR_INT_CLR 写 1 清空 PPA_SRM_PARAM_CFG_ERR_INT。(WT)

PPA_BLEND_PARAM_CFG_ERR_INT_CLR 写 1 清空 PPA_BLEND_PARAM_CFG_ERR_INT。(WT)

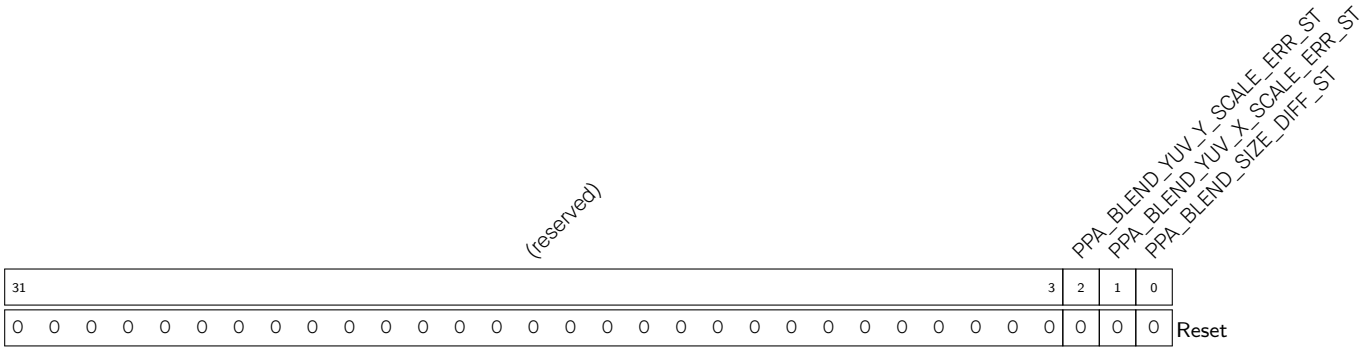
Register 18.25. PPA_CLUT_CNT_REG (0x0070)



PPA_BLEND0_CLUT_CNT 表示 BLEND 背景图层 CLUT 在 FIFO 模式下的当前写地址。(RO)

PPA_BLEND1_CLUT_CNT 表示 BLEND 前景图层 CLUT 在 FIFO 模式下的当前写地址。(RO)

Register 18.26. PPA_BLEND_ST_REG (0x0074)



PPA_BLEND_SIZE_DIFF_ST 表示 BLEND 前景、背景尺寸是否一致。

0：BLEND 前景、背景尺寸一致

1：BLEND 前景、背景尺寸不一致

(RO)

PPA_BLEND_YUV_X_SCALE_ERR_ST 表示图像格式为 YUV422 或 YUV420 时 X 方向尺寸为奇数。(RO)

PPA_BLEND_YUV_Y_SCALE_ERR_ST 表示图像格式为 YUV420 时 Y 方向尺寸为奇数。(RO)

Register 18.27. PPA_SRM_PARAM_ERR_ST_REG (0x0078)



PPA_TX_DSCR_VB_ERR_ST 表示 SRM 输出图像块垂直尺寸加上输出链表配置的垂直偏移 Y 是否超出输出链表配置的整幅图像垂直尺寸 VA。

0：未发生该错误

1：发生该错误

(RO)

PPA_TX_DSCR_HB_ERR_ST 表示 SRM 输出图像块水平尺寸加上输出链表配置的水平偏移 X 是否超出输出链表配置的整幅图像水平尺寸 HA。

0：未发生该错误

1：发生该错误

(RO)

PPA_Y_RX_SCAL_EQUAL_0_ERR_ST 表示 SRM 输入图像块垂直方向缩放系数是否为 0。

0：未发生该错误

1：发生该错误

(RO)

PPA_RX_DSCR_VB_ERR_ST 表示 SRM 输入链表配置的垂直方向图像块尺寸 VB 加上垂直方向偏移 Y 是否大于整幅图像垂直尺寸 VA。

0：未发生该错误

1：发生该错误

(RO)

PPA_YDST_LEN_TOO_SAMLL_ERR_ST 表示 SRM 缩放后的图像块垂直方向尺寸是否为 0。

0：未发生该错误

1：发生该错误

(RO)

PPA_YDST_LEN_TOO_LARGE_ERR_ST 表示 SRM 缩放后的图像块垂直方向尺寸是否大于 8191。

0：未发生该错误

1：发生该错误

(RO)

PPA_X_RX_SCAL_EQUAL_0_ERR_ST 表示 SRM 输入图像块水平方向缩放系数是否为 0。

0：未发生该错误

1：发生该错误

(RO)

见下页...

Register 18.27. PPA_SRM_PARAM_ERR_ST_REG (0x0078)

接上页...

PPA_RX_DSCR_HB_ERR_ST 表示 SRM 输入链表配置的水平方向图像块尺寸 HB 加上水平方向偏移 X 是否大于整幅图像水平尺寸 HA。

0：未发生该错误

1：发生该错误

(RO)

PPA_XDST_LEN_TOO_SAMLL_ERR_ST 表示 SRM 缩放后的图像块水平方向尺寸是否为 0。

0：未发生该错误

1：发生该错误

(RO)

PPA_XDST_LEN_TOO_LARGE_ERR_ST 表示 SRM 缩放后的图像块水平方向尺寸是否大于 8191。

0：未发生该错误

1：发生该错误

(RO)

PPA_X_YUV_RX_SCALE_ERR_ST 表示 SRM 输入链表的 HA/HB/X 参数在输入类型为 YUV420/YUV422 时是否为奇数。

0：未发生该错误

1：发生该错误

(RO)

PPA_Y_YUV_RX_SCALE_ERR_ST 表示 SRM 输入链表的 VA/VB/Y 参数在输入类型为 YUV420 时是否为奇数。

0：未发生该错误

1：发生该错误

(RO)

PPA_X_YUV_TX_SCALE_ERR_ST 表示 SRM 输出链表的 HA/HB/X 参数在输出类型为 YUV420/YUV422 时是否为奇数。

0：未发生该错误

1：发生该错误

(RO)

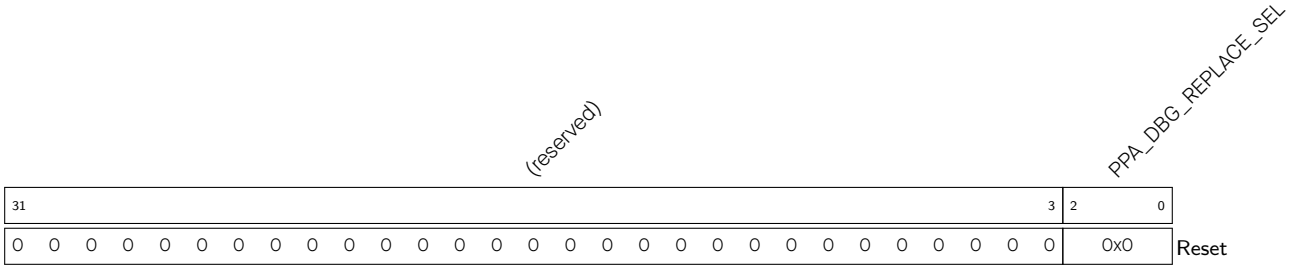
PPA_Y_YUV_TX_SCALE_ERR_ST 表示 SRM 输出链表的 VA/VB/Y 参数在输出类型为 YUV420 时是否为奇数。

0：未发生该错误

1：发生该错误

(RO)

Register 18.28. PPA_DEBUG_CTRL0_REG (0x0090)



PPA_DBG_REPLACE_SEL 配置替换像素的位置。

0：不替换

1：SRM RX 输入端

2：SRM 插值输出端

3: SRM TX 输出端

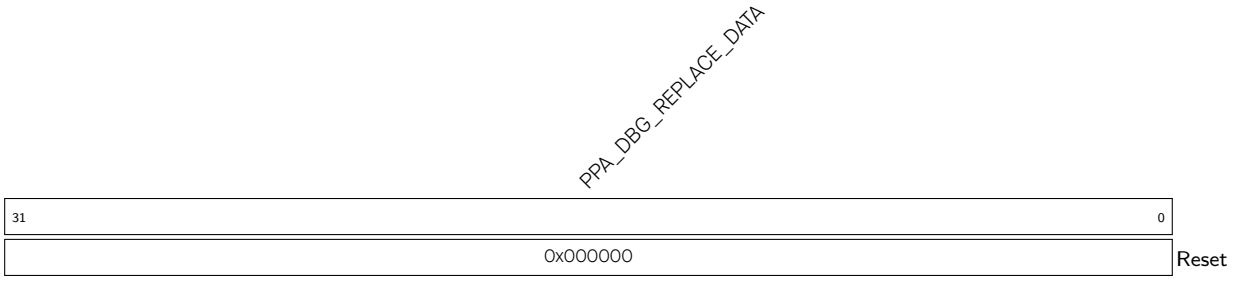
4：BLEND 前景输入端

5：BLEND 背景输入端

6：BLEND 输出端

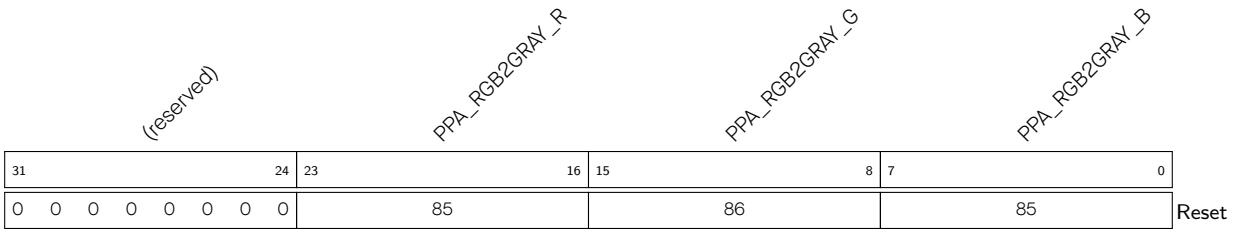
(R/W)

Register 18.29. PPA_DEBUG_CTRL1_REG (0x0094)



PPA_DBG_REPLACE_DATA 配置替换像素的值。(R/W)

Register 18.30. PPA_RGB2GRAY_REG (0x0098)

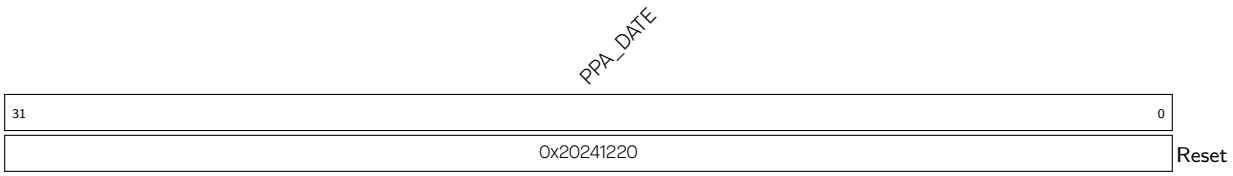


PPA_RGB2GRAY_B 配置 RGB 转 GRAY 时 B 的系数。(R/W)

PPA_RGB2GRAY_G 配置 RGB 转 GRAY 时 G 的系数。(R/W)

PPA_RGB2GRAY_R 配置 RGB 转 GRAY 时 R 的系数。(R/W)

Register 18.31. PPA_DATE_REG (0x0100)



PPA_DATE 版本控制寄存器。(R/W)

第 19 章

音频采样率转换器

19.1 概述

音频采样率转换器（Audio Sample Rate Converter，简称 ASRC）是一种加速器类音频处理模块，用于在不同采样率之间高质量地转换音频信号，以支持不同采样标准设备之间的协同工作。该模块通常通过 DMA 接口与片上或片外存储器进行数据交互，实现音频数据的高效传输与处理。

19.2 术语

为了更清晰地说明 ASRC 的功能，本章使用以下术语：

音频采样率转换器 (Audio Sample Rate Converter, ASRC)	用于实现不同采样率音频信号之间的转换
整数倍重采样器 (Integer Resampler)	实现整数倍的采样率转换
分数采样率转换器 (Fractional Sample Rate Converter, FRC)	实现非整数倍的采样率转换
插值 (Interpolation)	在原始采样点之间插入新采样点，以提高采样率
抽取 (Decimation)	以固定比例丢弃采样点，以降低采样率
上采样 (Upsampling)	提高采样率的过程，通常包括插值和滤波
下采样 (Downsampling)	降低采样率的过程，通常包括抽取和滤波
滤波 (Filtering)	在插值或抽取前后抑制混叠或插值噪声的过程
采样点 (Sample)	音频数据的基本单位，通常占用两个字节
帧结束 (End of Frame, EOF)	表示一帧音频数据处理完成的状态标志或中断信号

19.3 主要特性

ESP32-S31 中的 ASRC 模块具备以下特性：

- 支持常见采样率之间的任意互转，包括：8 kHz、16 kHz、32 kHz、44.1 kHz 和 48 kHz
- 输入和输出信号宽度为 16 bit，采用 Q1.15 格式
- 采样率转换通过两个整数倍重采样器与一个分数采样率转换器（FRC）级联实现，级联方式可通过软件灵活配置：
  - 整数倍重采样器的转换因子可配置为 2 或 $\frac{1}{2}$
  - 分数采样率转换器（FRC）的转换因子可在 0 到 2 之间配置（不包括 0 和 2）
- 支持多种声道模式，包括：
  - 单声道接收 (Rx) / 单声道发送 (Tx)

  - 双声道接收 (Rx) / 双声道发送 (Tx)，两个声道同时处理
  - 单声道接收 / 双声道发送
  - 双声道接收 / 单声道发送
- 支持任意长度输入数据，并可根据输出数据长度产生 EOF (End of Frame) 标志
- 当两个通道独立使用时，每个通道支持如下模式：
  - 输入单声道，输出单声道
  - 输入单声道，输出双声道（两个声道数据相同）
  - 输入双声道，输出单声道（可选择某一声道进行处理）
- 当两个通道协同使用时，每个通道均可配置为输入或输出双声道

19.4 结构概览

ESP32-S31 ASRC 模块的系统结构如图 19.4-1 所示。

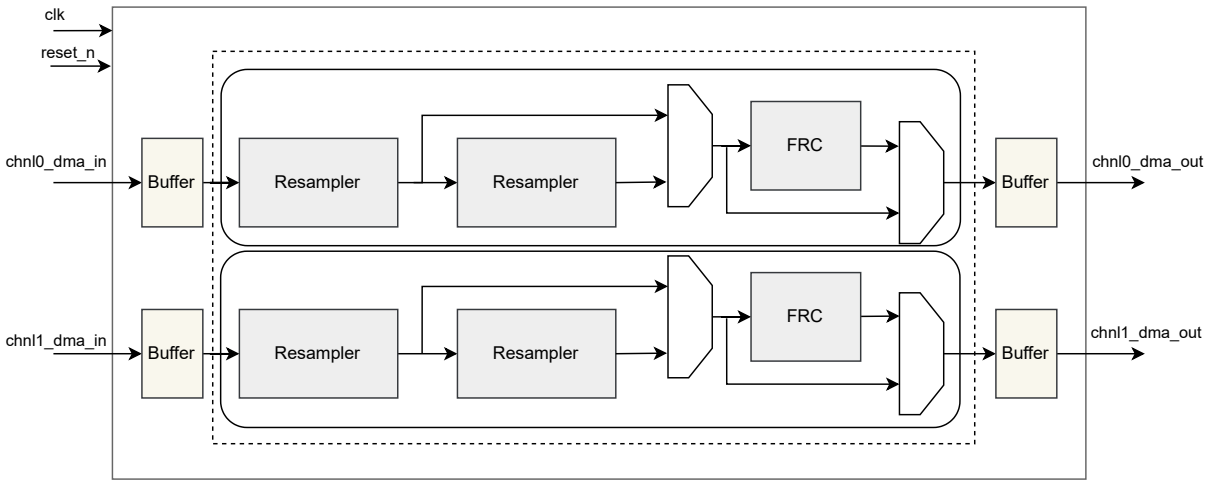


图 19.4-1. ASRC 模块结构

- 数据输入通路：ch0_dma_in 和 ch1_dma_in 分别接收来自 DMA 模块两个 TX 通道的输入数据。
- 数据输出通路：ch0_dma_out 和 ch1_dma_out 分别将处理结果输出至 DMA 模块的两个 RX 通道。
- 时钟结构：模块仅包含一个时钟 clk。
- DMA 传输过程：
  - 完成相关模式参数配置并启动对应通道后，采样率转换模块内的各级处理单元进入就绪状态。
  - 待 DMA 的 TX 通道和 RX 通道的链表配置完成后，第一级处理单元开始从 DMA 读取输入数据，进行处理，并将结果写回 DMA。
  - 整个过程中，若 TX 通道中数据不足，或 RX 通道中可用空间不足，采样率转换流程将自动暂停，直至资源恢复可用。
- 采样率转换结构：
  - 模块包含两个采样率转换通道 CHNLO 和 CHNL1，可同时处理两个声道的音频数据。

  - 每组结构包含两个整数倍重采样器（Resampler）和一个分数采样率转换器（FRC）。
  - Resampler 支持配置为 2×（上采样）或 $\frac{1}{2}$×（下采样）。
  - FRC 支持任意非整数倍的采样率转换。
  - 通过配置寄存器，可灵活设置两个 Resampler 和一个 FRC 的级联方式。
- 通道工作模式：
  - 独立模式：两个通道的配置可以不同，每个通道使用独立的 DMA 接口。
  - 协同模式：两个通道需使用相同配置，共用 CHNL0 的 DMA 接口，仅由 CHNL0 产生中断。

19.5 功能描述

19.5.1 时钟

如图 19.4-1 所示，音频采样率转换器所使用的时钟信号 clk 同时用于寄存器配置以及数据处理与传输。

有关时钟的更多信息，请查看章节 7 复位和时钟 *[to be added later]*。

时钟的详细配置流程，请查看章节 19.7 配置流程。

19.5.2 复位

音频采样率转换器支持完全复位和部分复位：

- 完全复位：复位整个 ASRC 模块，包括寄存器以及 CHNL0 和 CHNL1 两个数据处理通道。
- 部分复位：复位模块中的特定组件。详见 19.7.1 使用限制与提醒。

19.5.3 重采样器

为提高滤波处理的并行度并减少数据延迟，ASRC 模块采用由两个乘累加器和三个加法器组成的结构。该模块支持 2× 上采样和 $\frac{1}{2}$× 下采样，具体模式可通过寄存器进行灵活配置。

通过级联两个具有相同配置的重采样器，可分别实现 4× 上采样或 $\frac{1}{4}$× 下采样功能。但需注意，禁止将两个级联模块配置为混合模式（例如 “2× 上采样 + $\frac{1}{2}$× 下采样” 或 “$\frac{1}{2}$× 下采样 + 2× 上采样”），以避免引发不确定的行为或性能异常。

19.5.4 分数采样率转换器

分数采样率转换器支持任意采样率转换，其转换因子可表示为 $\frac{L}{M} \in (0, 2)$，其中 L 为插值因子，M 为抽取因子。为确保转换精度和性能，L 与 M 必须为正整数且互质；若两者不互质，将会影响转换效果。

在模块运行前，需进行如下寄存器配置：

- 向寄存器 ASRC_CHNL*n*_FRAC_L 写入插值因子 L
- 向寄存器 ASRC_CHNL*n*_FRAC_M 写入抽取因子 M
- 向寄存器 ASRC_CHNL*n*_FRAC_RECIPL 写入值 $\left\lfloor \frac{2^{20} + L}{2L} \right\rfloor$，该值用于近似表示 $1/L$

注意: 文中的 *n* 表示通道编号，可取值为 0 或 1，后续内容中不再重复说明。

19.5.5 输入/输出数据长度

ASRC 模块用于实现采样率转换功能，可支持采样率升高、降低以及保持不变三种模式。其中，采样率不变的情况应通过将选择器配置为 bypass 模式来实现。采样率变化将导致输出数据长度的相应变化：

- 当采样率升高时，输出数据长度增加
- 当采样率降低时，输出数据长度减少
- 当采样率不变时，输出数据长度保持不变

设输入采样率为 F_i，输出采样率为 F_o，输入数据长度为 N_i，则输出数据长度 N_o 可按以下公式计算：

$$N_o = \lfloor xN_i \rfloor, \quad \text{其中} x = \frac{F_o}{F_i}, \quad x \in \mathbb{Q}^+$$

例如，当输入采样率 $F_i = 8\,\text{kHz}$，输出采样率 $F_o = 44.1\,\text{kHz}$，输入数据长度为 128（单位：sample）时，输出数据长度为：

$$N_o = \left\lfloor \frac{F_o}{F_i} \cdot N_i \right\rfloor = \left\lfloor \frac{44.1}{8} \cdot 128 \right\rfloor = 705 \text{ (samples)}$$

注意：

- 在单声道模式下，输入数据长度 N_i 应为 1 个 sample（即 2 字节）的整数倍
- 在双声道模式下，N_i 应为 2 个 sample（即 4 字节）的整数倍，且两个声道的数据长度必须相等
- 文中所有输入/输出数据长度均以 sample（采样点）为单位

19.5.5.1 数据无剩余

ASRC 模块由三级子模块级联构成，每一级的输出数据长度由其输入数据长度和采样率转换系数决定。各级支持的采样率转换系数包括：有理数 $\frac{L}{M}$、2 或者 $\frac{1}{2}$。

对于单级结构，若采样率转换系数 $\frac{L}{M} < 1$ 或为 $\frac{1}{2}$，则输入数据长度不得小于 $\lceil \frac{M}{L} \rceil$ 或 2 samples，以确保处理过程中无数据剩余。

对于级联结构，输入数据长度 N_i 应不小于 $4 \times \lceil \frac{M}{L} \rceil$，以确保各子模块处理过程中无数据剩余。

19.5.5.2 数据有剩余

当输入频率大于输出频率时，需要特别考虑输入数据剩余问题。以下以两倍下采样为例说明：

在两倍下采样中，系统每接收 2 个输入 samples 生成 1 个输出 sample。若输入数据个数为奇数（如 1），将无法产生输出，导致最后一个 sample 残留在系统中。这种残留可能会影响上层应用对数据包的组织和解析。

若无法满足每级模块输入数据无剩余的条件，可以启用硬件输出数据补偿功能，方法为配置寄存器 ASRC_CHNL*n*_OUT_LEN_COMP 为 1。当硬件检测到输出数据长度已达到配置寄存器 ASRC_CHNL*n*_OUT_LEN（单位为字节）后仍继续产生新数据，系统将在下次传输时在该长度基础上增加 2 字节（单声道）或 4 字节（双声道）。

启用补偿功能时，DMA 的接收 buffer 长度至少需增加 2 或 4 字节以防止溢出。

此外，启用硬件输出数据补偿功能后，需通知 ASRC 控制器当前数据是否已全部发送完毕，可通过以下任一方式实现：

- 启用 DMA 的 EOF 功能
- 设置 ASRC 的接收数据长度 ASRC_CHNL*n*_DIN_CFG_REG

若未正确配置，可能导致补偿异常，影响后续数据传输的稳定性与正确性。

19.5.6 DMA 接口

ASRC 模块的输入/输出数据路径均为 DMA 接口。

待输出达到一定长度后，会产生信号 EOF 并发送给 DMA。这里输出长度由软件按照 19.5.5 输入/输出数据长度 描述的公式配置。产生 EOF 后，系统仍然会处于等待输入状态。

选择软件配置输出长度产生 EOF 时，当输出计数达到指定值后，会将 EOF 写入 DMA 。

19.6 中断

每个 ASRC 通道可以产生一个电平中断，该中断在输出数据长度计数达到设定值后触发。

如果 ASRC 模块输出输入频率比为 x，接收到 EOF 时数据长度为 N_i，那么当输出数据总长度为 $\lfloor xN_i \rfloor$ 时，则触发中断。

ESP32-S31 的音频采样率转换器可以生成以下中断信号并发送给 中断矩阵。

- ASRC_CHNLO_INT
- ASRC_CHNL1_INT

这些中断信号由音频采样率转换器的内部中断源生成。表 19.6-1 列出了音频采样率转换器的内部中断源、中断触发条件以及生成的中断信号。

表 19.6-1. ASRC 的内部中断源

内部中断源	触发条件	中断信号
ASRC_CHNLO_INT	输出计数器达到 CHNLO 的预设值	ASRC_CHNLO_INTR
ASRC_CHNL1_INT	输出计数器达到 CHNL1 的预设值	ASRC_CHNL1_INTR

说明：
中断、中断源、中断信号等术语的解释以及它们之间的关系，请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 19.9 寄存器列表。

19.7 配置流程

ASRC 模块的配置流程如下：

1. 配置以下寄存器，设置采样率转换相关参数：
   - 级联方式：ASRC_CHNL*n*_CTRL_CFG_REG
   - 声道模式：ASRC_CHNL*n*_CTRL_CFG_REG
   - 采样率转换因子和通道模式选择：ASRC_CHNL*n*_CTRL_CFG_REG、ASRC_CHNL*n*_FRAC_COEFF_REG、ASRC_CHNL*n*_FRAC_RECIPL_REG

- 输出数据长度 EOF 产生模式：ASRC_CHNL*n*_DOUT_CFG_REG

2. 启动模块：设置 ASRC_CHNL*n*_START_REG 以启动模块。模块将进入等待数据状态。
3. 按需清除缓存 ASRC_CHNL*n*_RESET_REG。清除缓存将删除模块内部的历史处理信息。建议如下：
   - 若每帧数据表示一段完整的音频信号，可清除模块缓存
   - 若一段完整的音频信号由多帧数据组成，则每帧之间不应清除缓存
4. 启用硬件输出数据补偿功能后，需通知 ASRC 控制器当前数据是否已全部发送完毕，可通过以下任一方式实现：
   - 启用 DMA 的 EOF 功能
   - 设置 ASRC 的接收数据长度 ASRC_CHNL*n*_DIN_CFG_REG

   详细说明参见 19.5.5 输入/输出数据长度 小节。
5. 通过以下寄存器监控输出数据状态：
   - ASRC_CHNL*n*_OUT_CNT：自通道启动以来累计向 DMA 写入的总字节数
   - ASRC_CHNL*n*_OUT_INC：自上次 EOF 发生后向 DMA 写入的字节数

19.7.1 使用限制与提醒

1. 在使用 ASRC 模块之前，需先完成模式选择、插值系数、抽取系数、通道模式等相关寄存器的配置。
2. DMA 通道与 ASRC 通道的启动顺序没有严格要求。
3. 模块运行过程中，禁止修改相关配置寄存器，以避免状态不一致或数据异常。
4. ASRC 中涉及的数据计数器单位为 **字节**，每个采样点占用 2 字节。
5. 当两个通道独立工作时，每个通道的寄存器可分别配置，支持以下模式：
   - 输入单声道，输出单声道
   - 输入单声道，输出双声道（两个声道数据相同）
   - 输入双声道，输出单声道（选择一个声道进行处理）

   当两个通道协同工作时，两个通道的配置需保持一致，支持：
   - 输入双声道，输出双声道
6. 需根据实际通道的接收寄存器 ASRC_CHNL*n*_IN_LEN 与发送寄存器 ASRC_CHNL*n*_OUT_LEN 的数据长度进行配置。配置规则如下：
   - 通道独立工作时：
     - 若处理单声道数据，仅配置对应通道的 1 个声道数据长度（单位：字节）
     - 若处理双声道数据，需配置该通道上 2 个声道的总数据长度（单位：字节）
   - 通道协同工作时：
     - 每个通道仅处理一个声道数据，配置该声道的数据长度（单位：字节）
7. 在双声道输入或者双声道输出模式下，只能使用 CHNLO 通道，并且仅会产生 CHNLO 中断信号。
8. 若需处理新的数据，建议执行模块复位流程，或在上一次传输完成后按如下顺序清除内部状态：

- 清除 ASRC_CHNL*n*_START
- 置位 ASRC_CHNL*n*_INFIFO_RESET
- 置位 ASRC_CHNL*n*_RESET
- 置位 ASRC_CHNL*n*_OUTFIFO_RESET

然后再进行配置并启动。

9. 分数阶滤波器结构的选择依据如下：
   - 若输入采样率 < 输出采样率，使用后置结构
   - 若输入采样率 > 输出采样率，使用前置结构
   - 若输入采样率与输出采样率相等，则所有滤波器 bypass
10. 使用过程中需关注输入输出数据长度的配比与限制，具体规则详见 19.5.5 输入/输出数据长度。

19.8 应用示例

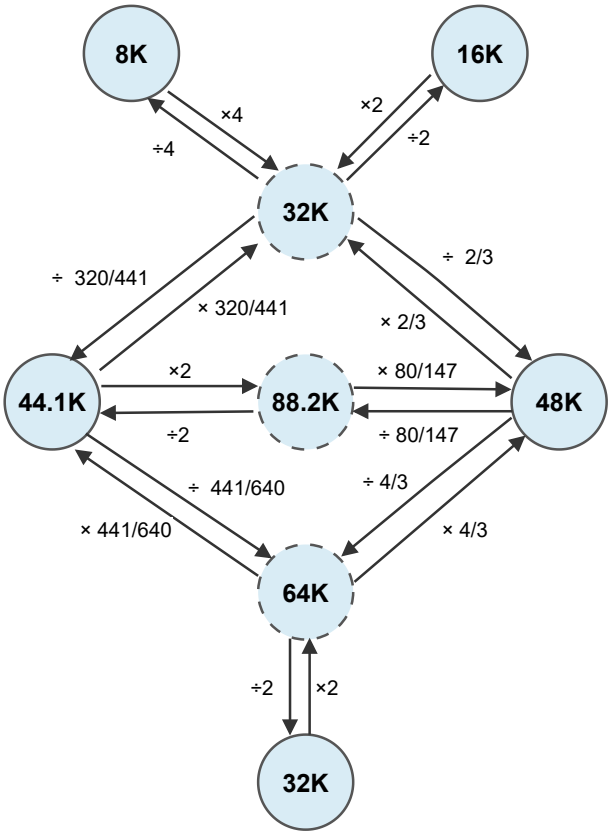


图 19.8-1. 采样率转换关系

为了实现高质量的采样率转换，建议采用图示方法进行配置。各类采样率转换的处理流程如下：

- 8 kHz / 16 kHz 转 44.1 kHz / 48 kHz
  输入信号先插值至 32 kHz，再通过分数采样率转换器转换为目标采样率。
- 32 kHz 转 44.1 kHz / 48 kHz
  输入信号先插值至 64 kHz，再通过分数采样率转换器转换为目标采样率。
- 44.1 kHz / 48 kHz 转 8 kHz / 16 kHz
  输入信号先通过分数采样率转换器转换为 32 kHz，再进行下采样得到 8 kHz 或 16 kHz 输出。
- 44.1 kHz / 48 kHz 转 32 kHz
  输入信号先通过分数采样率转换器转换为 64 kHz，再下采样至 32 kHz。
- 44.1 kHz 与 48 kHz 互转
  需通过中间频率 88.2 kHz 进行两级转换：
  - 44.1 kHz 转 48 kHz：先上采样至 88.2 kHz，再转换为 48 kHz
  - 48 kHz 转 44.1 kHz：先上采样至 88.2 kHz，再转换为 44.1 kHz

19.9 寄存器列表

本小节的所有地址均为相对于音频采样率转换器基地址的地址偏移量（相对地址），具体基地址请见章节 3 系统和存储器 *[to be added later]* 中的表 3。

请查看章节 寄存器的访问类型，了解“访问”列缩写的含义。

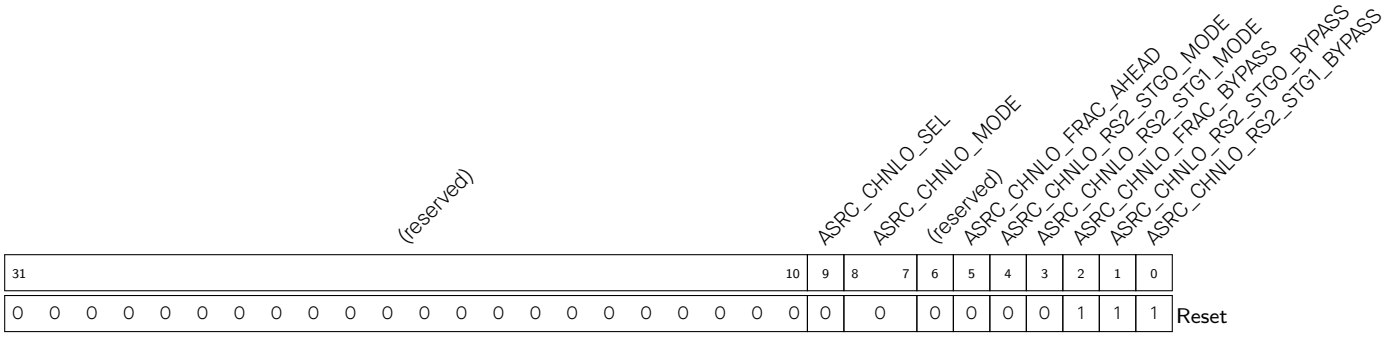
名称	描述	地址	访问
控制和配置寄存器			
ASRC_CHNL0_CTRL_CFG_REG	CHNL0 控制寄存器	0x0000	R/W
ASRC_CHNL0_FRAC_COEFF_REG	CHNL0 分数采样率转换器配置寄存器	0x0004	R/W
ASRC_CHNL0_FRAC_RECIPL_REG	CHNL0 分数采样率转换器配置寄存器	0x0008	R/W
ASRC_CHNL0_RESET_REG	CHNL0 复位寄存器	0x000C	WT
ASRC_CHNL0_START_REG	CHNL0 启动寄存器	0x0010	R/W
ASRC_CHNL0_DIN_CFG_REG	CHNL0 输入控制和配置寄存器	0x0014	varies
ASRC_CHNL0_DOUT_CFG_REG	CHNL0 输出控制和配置寄存器	0x0018	varies
ASRC_CHNL0_FIFO_CTRL_REG	CHNL0 FIFO 复位控制寄存器	0x001C	WT
ASRC_CHNL1_CTRL_CFG_REG	CHNL1 控制寄存器	0x003C	R/W
ASRC_CHNL1_FRAC_COEFF_REG	CHNL1 分数采样率转换器配置寄存器	0x0040	R/W
ASRC_CHNL1_FRAC_RECIPL_REG	CHNL1 分数采样率转换器配置寄存器	0x0044	R/W
ASRC_CHNL1_RESET_REG	CHNL1 复位寄存器	0x0048	WT
ASRC_CHNL1_START_REG	CHNL1 启动寄存器	0x004C	R/W
ASRC_CHNL1_DIN_CFG_REG	CHNL1 输入控制和配置寄存器	0x0050	varies
ASRC_CHNL1_DOUT_CFG_REG	CHNL1 输出控制和配置寄存器	0x0054	varies
ASRC_CHNL1_FIFO_CTRL_REG	CHNL1 FIFO 复位控制寄存器	0x0058	WT
中断寄存器			
ASRC_CHNL0_INT_RAW_REG	CHNL0 原始中断状态	0x0020	R/WTC/SS
ASRC_CHNL0_INT_ST_REG	CHNL0 屏蔽中断状态	0x0024	RO
ASRC_CHNL0_INT_ENA_REG	CHNL0 中断使能位	0x0028	R/W
ASRC_CHNL0_INT_CLR_REG	CHNL0 中断清除位	0x002C	WT
ASRC_CHNL1_INT_RAW_REG	CHNL1 原始中断状态	0x005C	R/WTC/SS
ASRC_CHNL1_INT_ST_REG	CHNL1 屏蔽中断状态	0x0060	RO
ASRC_CHNL1_INT_ENA_REG	CHNL1 中断使能位	0x0064	R/W
ASRC_CHNL1_INT_CLR_REG	CHNL1 中断清除位	0x0068	WT
状态寄存器			
ASRC_CHNL0_OUT_CNT_REG	CHNL0 状态寄存器	0x0030	RO
ASRC_CHNL1_OUT_CNT_REG	CHNL1 状态寄存器	0x006C	RO
调试寄存器			
ASRC_CHNL0_TRACE1_REG	CHNL0 调试寄存器 1	0x0038	RO
ASRC_CHNL1_TRACE1_REG	CHNL1 调试寄存器 1	0x0074	RO
配置寄存器			
ASRC_SYS_REG	时钟控制和配置寄存器	0x00F8	R/W
版本寄存器			
ASRC_DATE_REG	版本控制寄存器	0x00FC	R/W

19.10 寄存器

本小节的所有地址均为相对于音频采样率转换器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 19.1. ASRC_CHNL0_CTRL_CFG_REG (0x0000)



ASRC_CHNL0_RS2_STG1_BYPASS 设置此位以绕过通道 0 中的第 1 阶重采样器。(R/W)

ASRC_CHNL0_RS2_STG0_BYPASS 设置此位以绕过通道 0 中的第 0 阶重采样器。(R/W)

ASRC_CHNL0_FRAC_BYPASS 设置此位以绕过通道 0 中的分数重采样器。(R/W)

ASRC_CHNL0_RS2_STG1_MODE 写入此位以配置通道 0 中第 1 阶重采样器的模式：

0：2 倍插值

1：2 倍抽取

(R/W)

ASRC_CHNL0_RS2_STG0_MODE 写入此位以配置通道 0 中第 0 阶重采样器的模式：

0：2 倍插值

1：2 倍抽取

(R/W)

ASRC_CHNL0_FRAC_AHEAD 设置此位以将分数重采样器移至通道 0 中的第 1 阶位置；当输入频率高于输出频率时置 1。(R/W)

ASRC_CHNL0_MODE 写入该字段以配置通道模式：

0：输入和输出均为单声道

1：输入和输出均为双声道

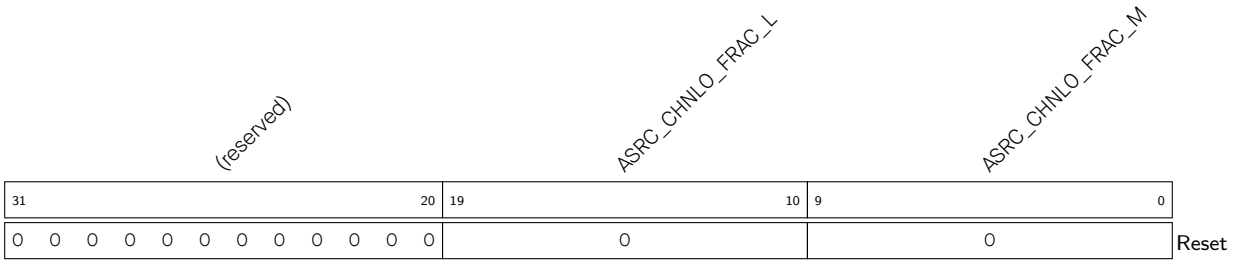
2：输入为单声道，输出为双声道

3：输入为双声道，输出为单声道

(R/W)

ASRC_CHNL0_SEL 写入此位以选择需要处理的 16 位数据。(R/W)

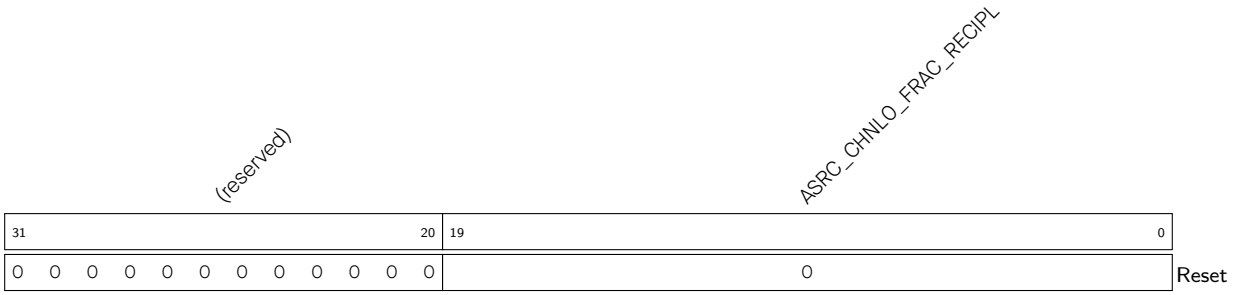
Register 19.2. ASRC_CHNLO_FRAC_COEFF_REG (0x0004)



ASRC_CHNLO_FRAC_M 写入该字段以指定通道 0 中分数重采样因子的分母，与 ASRC_CHNLO_FRAC_L 互质。(R/W)

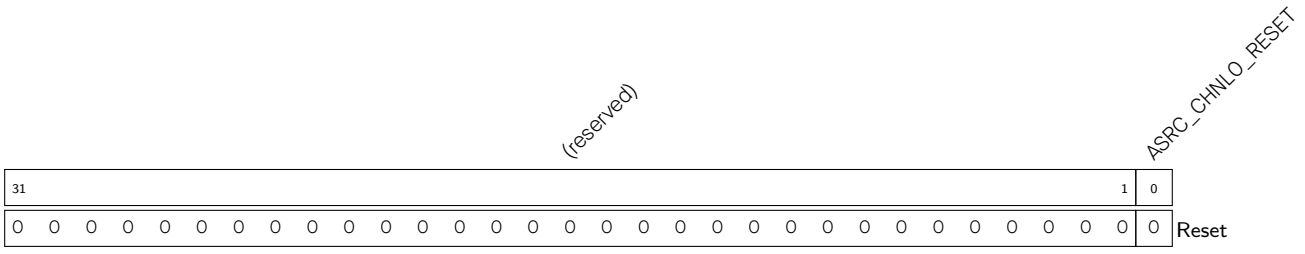
ASRC_CHNLO_FRAC_L 写入该字段以指定通道 0 中分数重采样因子的分子，与 ASRC_CHNLO_FRAC_M 互质。(R/W)

Register 19.3. ASRC_CHNLO_FRAC_RECIPL_REG (0x0008)



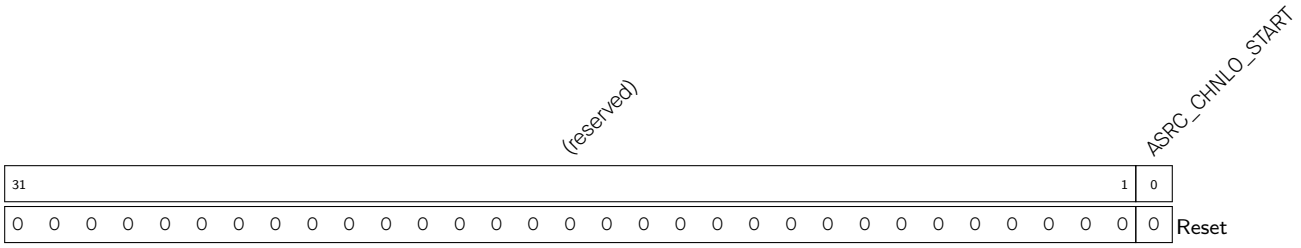
ASRC_CHNLO_FRAC_RECIPL 写入该字段以设置通道 0 中的值，计算公式为 $\left\lfloor \frac{2^{19}+L}{2L} \right\rfloor$。(R/W)

Register 19.4. ASRC_CHNLO_RESET_REG (0x000C)



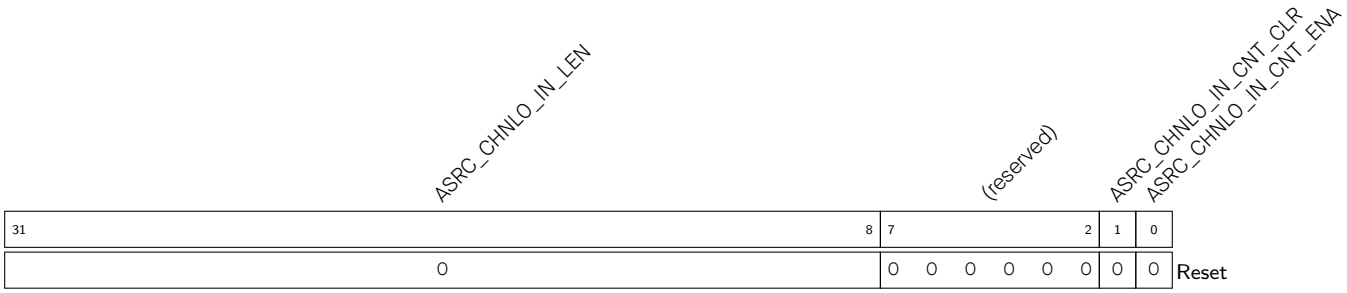
ASRC_CHNLO_RESET 设置此位以复位通道 0。(WT)

Register 19.5. ASRC_CHNLO_START_REG (0x0010)



ASRC_CHNLO_START 设置此位以启动通道 0。(R/W)

Register 19.6. ASRC_CHNLO_DIN_CFG_REG (0x0014)

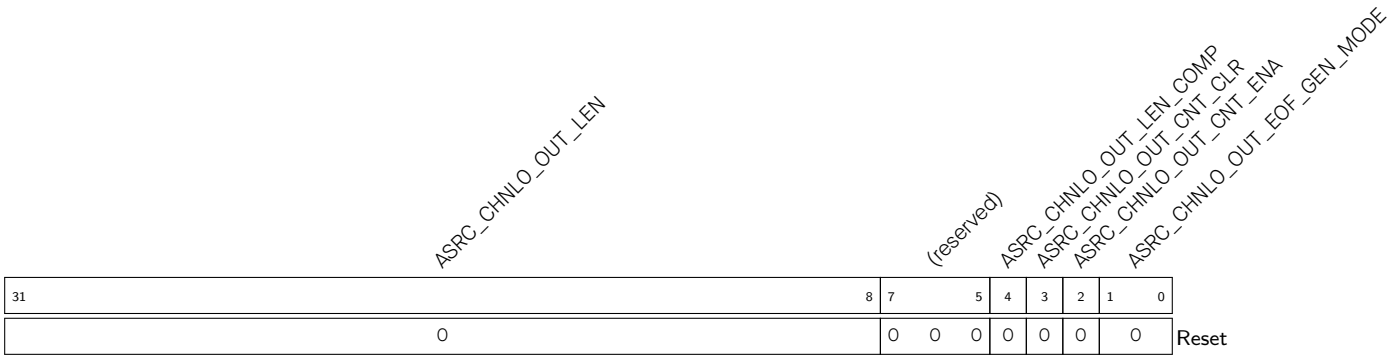


ASRC_CHNLO_IN_CNT_ENA 设置此位以启用输入数据字节计数器。(R/W)

ASRC_CHNLO_IN_CNT_CLR 写入此位以清除输入数据字节计数器。(WT)

ASRC_CHNLO_IN_LEN 写入该字段以指定从 DMA 接收的数据字节数。(R/W)

Register 19.7. ASRC_CHNLO_DOUT_CFG_REG (0x0018)



ASRC_CHNLO_OUT_EOF_GEN_MODE 写入该字段以指定写入 DMA 的 EOF 类型：

0：计数器 EOF

1：DMA 输入 EOF

2：计数器 EOF 和 DMA 输入 EOF 均写入

3：无 EOF 写入

(R/W)

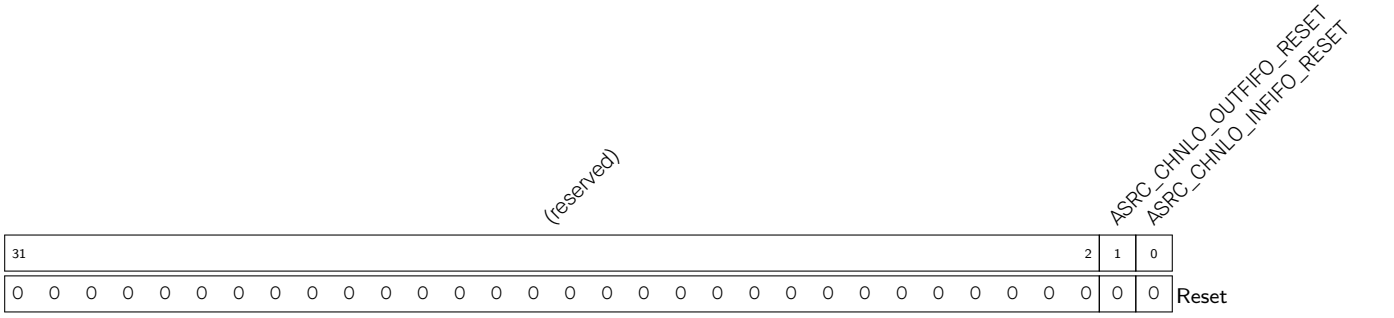
ASRC_CHNLO_OUT_CNT_ENA 设置此位以启用输出数据字节计数器。(R/W)

ASRC_CHNLO_OUT_CNT_CLR 写入此位以清除输出数据字节计数器。(WT)

ASRC_CHNLO_OUT_LEN_COMP 设置此位以启用输出数据字节计数器补偿，当使用分数重采样器并进行 2 倍抽取且 ASRC_CHNLO_OUT_CNT ≥ ASRC_CHNLO_OUT_LEN 时生效。(R/W)

ASRC_CHNLO_OUT_LEN 写入该字段以指定传输到 DMA 的数据字节数，当计数器到达末尾时，EOF（结束标志）将被置位。(R/W)

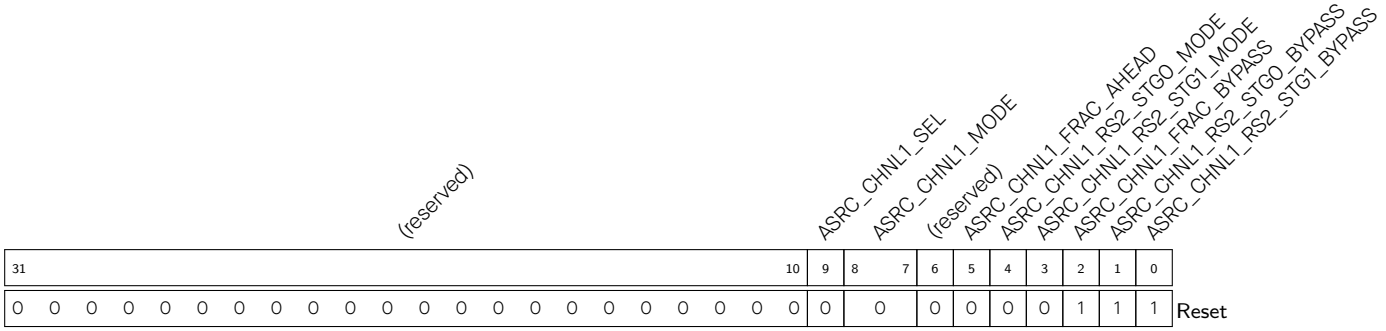
Register 19.8. ASRC_CHNLO_FIFO_CTRL_REG (0x001C)



ASRC_CHNLO_INFIFO_RESET 写入此位以复位输入 FIFO。(WT)

ASRC_CHNLO_OUTFIFO_RESET 写入此位以复位输出 FIFO。(WT)

Register 19.9. ASRC_CHNL1_CTRL_CFG_REG (0x003C)



ASRC_CHNL1_RS2_STG1_BYPASS 设置此位以绕过通道 1 中的第 1 阶重采样器。(R/W)

ASRC_CHNL1_RS2_STG0_BYPASS 设置此位以绕过通道 1 中的第 0 阶重采样器。(R/W)

ASRC_CHNL1_FRAC_BYPASS 设置此位以绕过通道 1 中的分数重采样器。(R/W)

ASRC_CHNL1_RS2_STG1_MODE 写入此位以配置通道 1 中第 1 阶重采样器的模式：

0：2 倍插值

1：2 倍抽取

(R/W)

ASRC_CHNL1_RS2_STG0_MODE 写入此位以配置通道 1 中第 0 阶重采样器的模式：

0：2 倍插值

1：2 倍抽取

(R/W)

ASRC_CHNL1_FRAC_AHEAD 设置此位以将分数重采样器移至通道 1 中的第 1 阶位置；当输入频率高于输出频率时置 1。(R/W)

ASRC_CHNL1_MODE 写入该字段以配置通道模式：

0：输入和输出均为单声道

1：输入和输出均为双声道

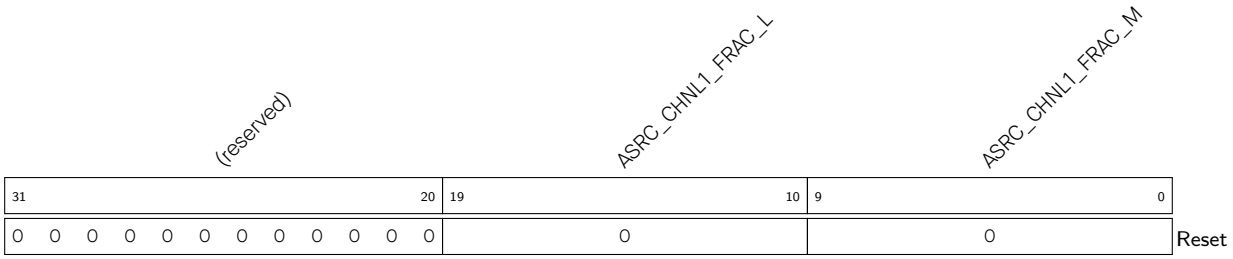
2：输入为单声道，输出为双声道

3：输入为双声道，输出为单声道

(R/W)

ASRC_CHNL1_SEL 写入此位以选择需要处理的 16 位数据。(R/W)

Register 19.10. ASRC_CHNL1_FRAC_COEFF_REG (0x0040)



ASRC_CHNL1_FRAC_M 写入该字段以指定通道 1 中分数重采样因子的分母，与 ASRC_CHNL1_FRAC_L 互质。(R/W)

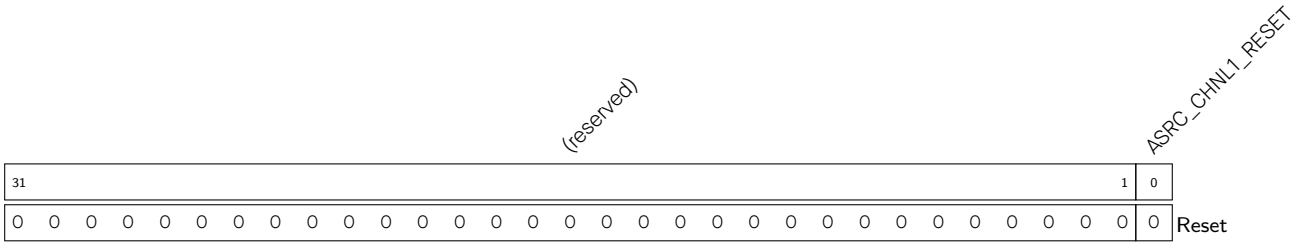
ASRC_CHNL1_FRAC_L 写入该字段以指定通道 1 中分数重采样因子的分子，与 ASRC_CHNL1_FRAC_M 互质。(R/W)

Register 19.11. ASRC_CHNL1_FRAC_RECIPL_REG (0x0044)



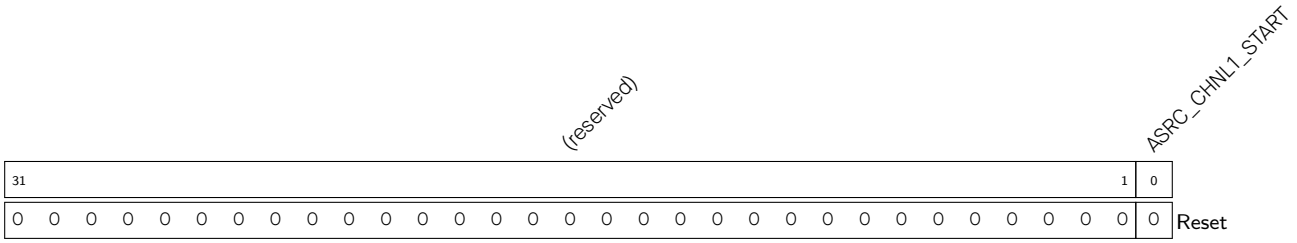
ASRC_CHNL1_FRAC_RECIPL 写入该字段以设置通道 1 中的值，计算公式为 $\left\lfloor \frac{2^{19}+L}{2L} \right\rfloor$。(R/W)

Register 19.12. ASRC_CHNL1_RESET_REG (0x0048)



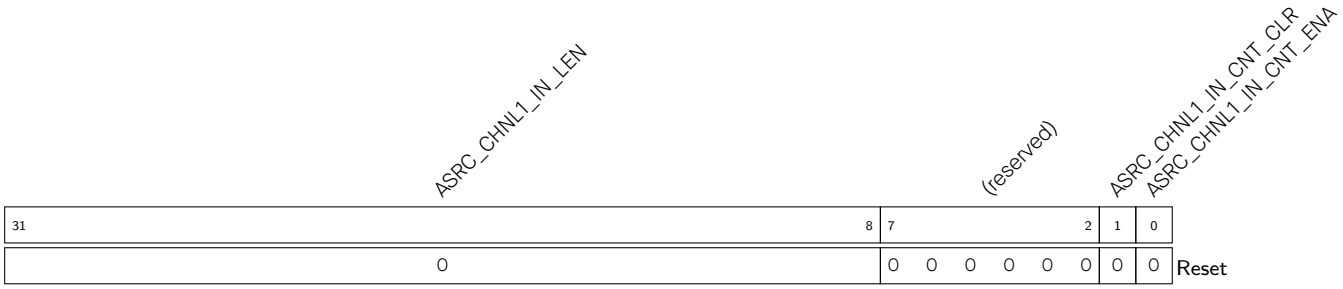
ASRC_CHNL1_RESET 设置此位以复位通道 1。(WT)

Register 19.13. ASRC_CHNL1_START_REG (0x004C)



ASRC_CHNL1_START 设置此位以启动通道 1。(R/W)

Register 19.14. ASRC_CHNL1_DIN_CFG_REG (0x0050)

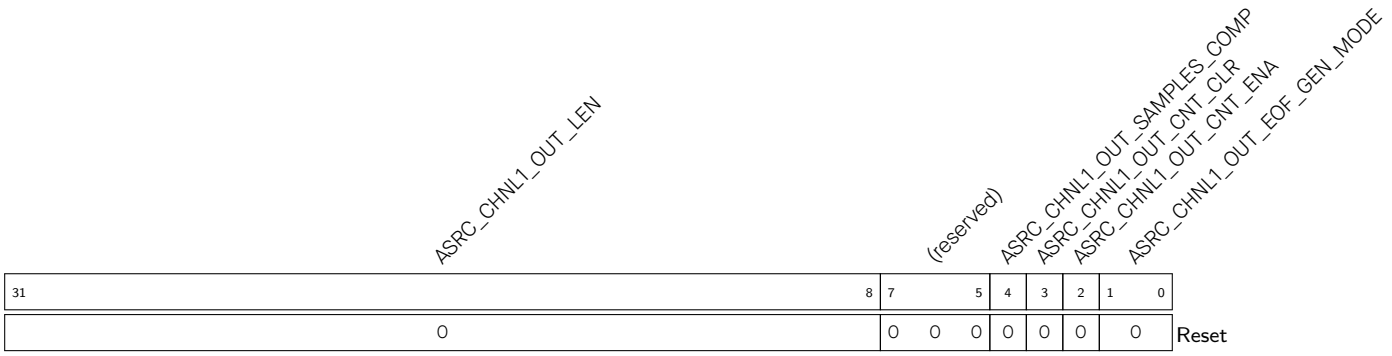


ASRC_CHNL1_IN_CNT_ENA 设置此位以使能输入数据字节计数器。(R/W)

ASRC_CHNL1_IN_CNT_CLR 设置此位以清除输入数据字节计数器。(WT)

ASRC_CHNL1_IN_LEN 写入该字段以指定从 DMA 接收的数据字节数。(R/W)

Register 19.15. ASRC_CHNL1_DOUT_CFG_REG (0x0054)



ASRC_CHNL1_OUT_EOF_GEN_MODE 写入该字段以配置写入 DMA 的 EOF 类型：

0：计数器 EOF 事件

1：DMA 输入 EOF 信号

2：计数器 EOF 和 DMA 输入 EOF 均写入

3：无 EOF 写入

(R/W)

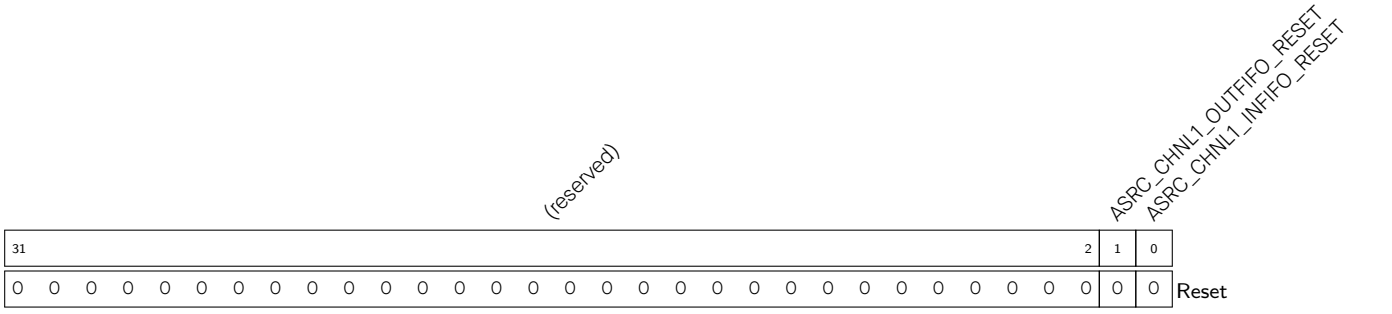
ASRC_CHNL1_OUT_CNT_ENA 设置此位以启用输出数据字节计数器。(R/W)

ASRC_CHNL1_OUT_CNT_CLR 写入此位以清除输出数据字节计数器。(WT)

ASRC_CHNL1_OUT_SAMPLES_COMP 设置此位以启用输出数据字节计数器补偿，当使用分数重采样器并进行 2 倍抽取且 ASRC_CHNL1_OUT_CNT ≥ ASRC_CHNL1_OUT_LEN 时生效。(R/W)

ASRC_CHNL1_OUT_LEN 写入该字段以指定传输到 DMA 的数据字节数，当计数器到达末尾时，EOF（结束标志）将被置位。(R/W)

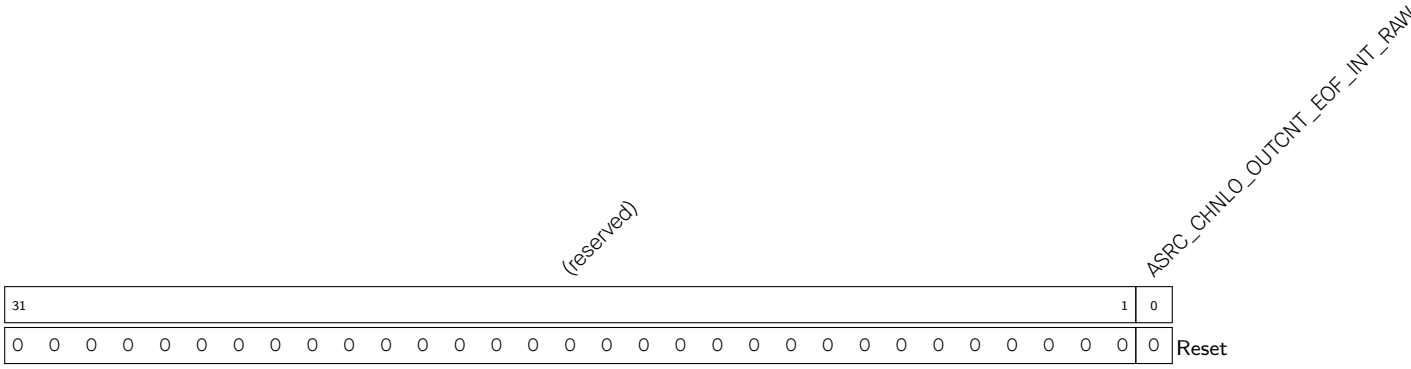
Register 19.16. ASRC_CHNL1_FIFO_CTRL_REG (0x0058)



ASRC_CHNL1_INFIFO_RESET 写入此位以复位输入 FIFO。(WT)

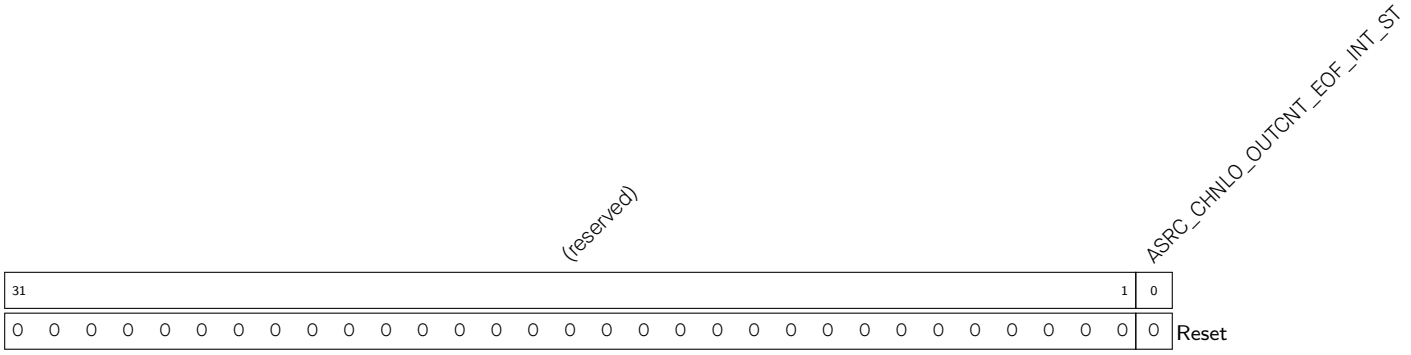
ASRC_CHNL1_OUTFIFO_RESET 写入此位以复位输出 FIFO。(WT)

Register 19.17. ASRC_CHNLO_INT_RAW_REG (0x0020)



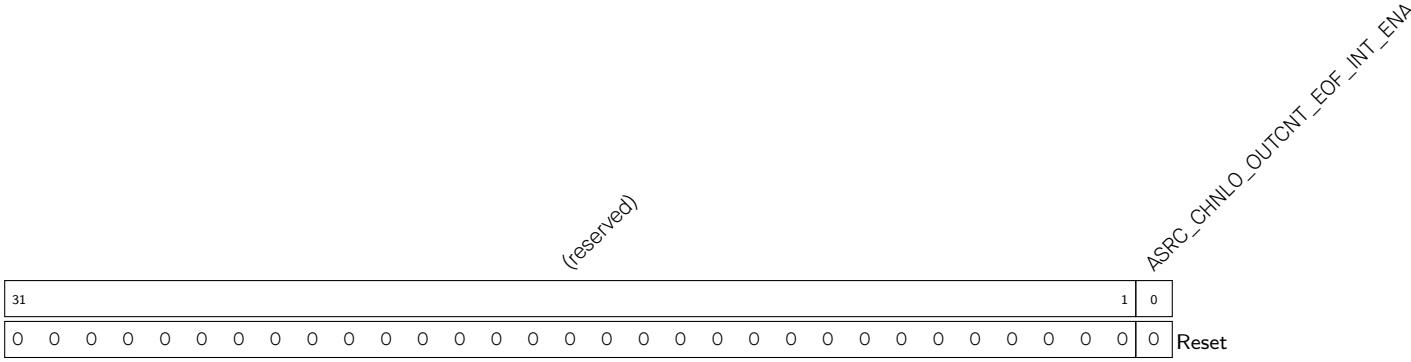
ASRC_CHNLO_OUTCNT_EOF_INT_RAW ASRC_CHNLO_INT 中断的原始中断状态。(R/WTC/SS)

Register 19.18. ASRC_CHNLO_INT_ST_REG (0x0024)



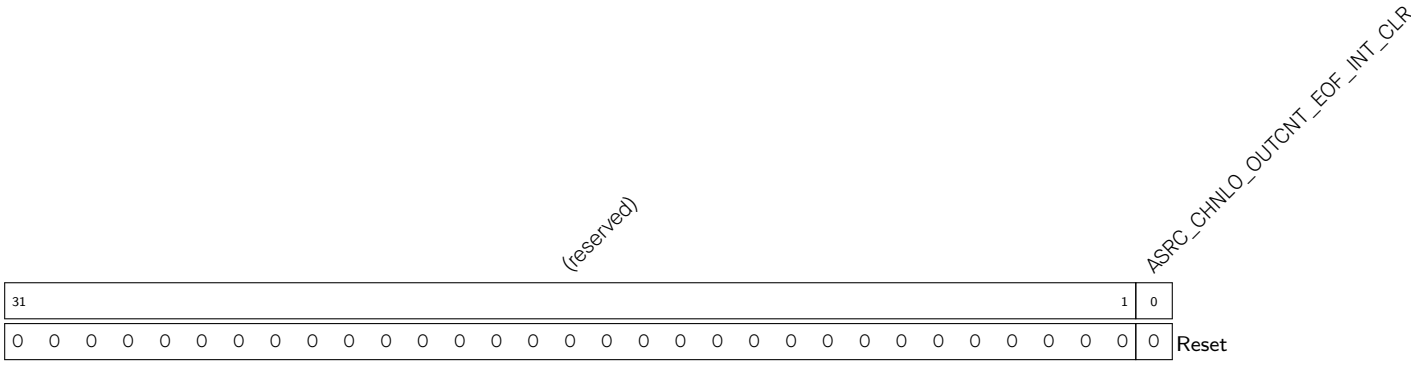
ASRC_CHNLO_OUTCNT_EOF_INT_ST ASRC_CHNLO_INT 中断的屏蔽中断状态。(RO)

Register 19.19. ASRC_CHNLO_INT_ENA_REG (0x0028)



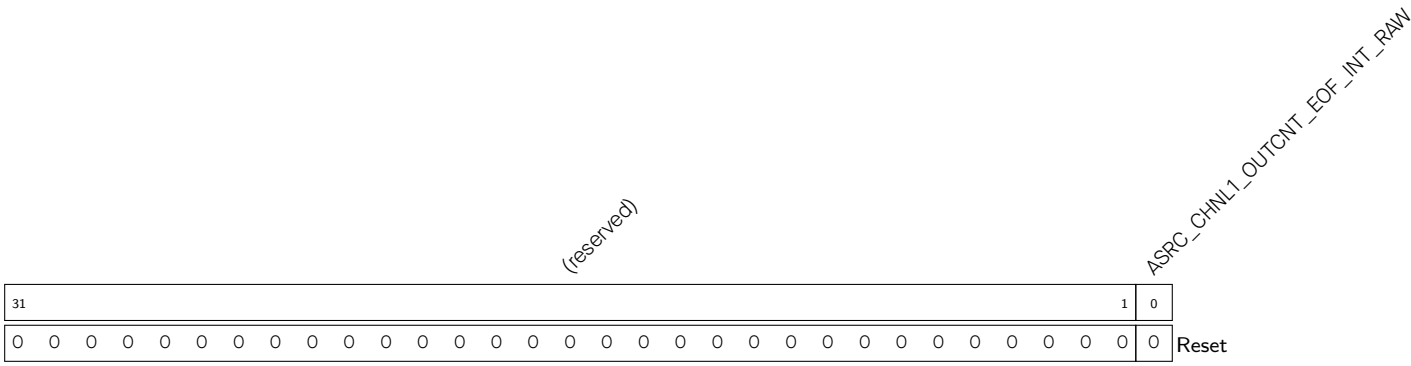
ASRC_CHNLO_OUTCNT_EOF_INT_ENA 写 1 使能 ASRC_CHNLO_INT 中断。(R/W)

Register 19.20. ASRC_CHNL0_INT_CLR_REG (0x002C)



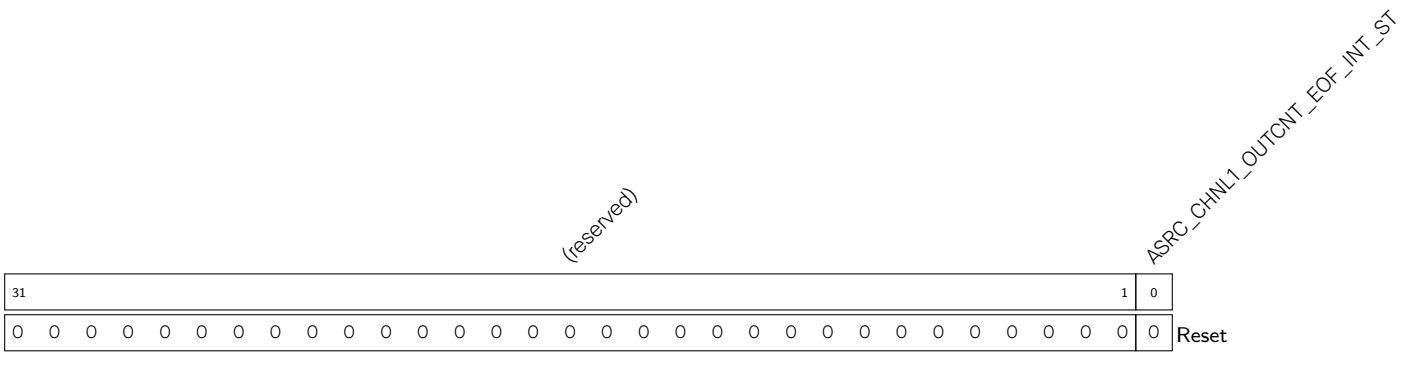
ASRC_CHNL0_OUTCNT_EOF_INT_CLR 写 1 清除 ASRC_CHNL0_INT 中断。(WT)

Register 19.21. ASRC_CHNL1_INT_RAW_REG (0x005C)



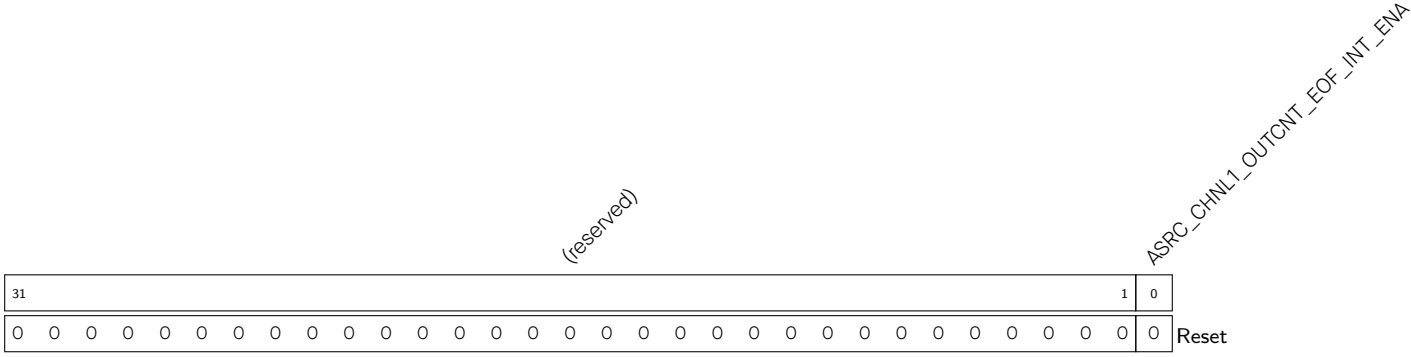
ASRC_CHNL1_OUTCNT_EOF_INT_RAW ASRC_CHNL1_INT 中断的原始中断状态。(R/WTC/SS)

Register 19.22. ASRC_CHNL1_INT_ST_REG (0x0060)



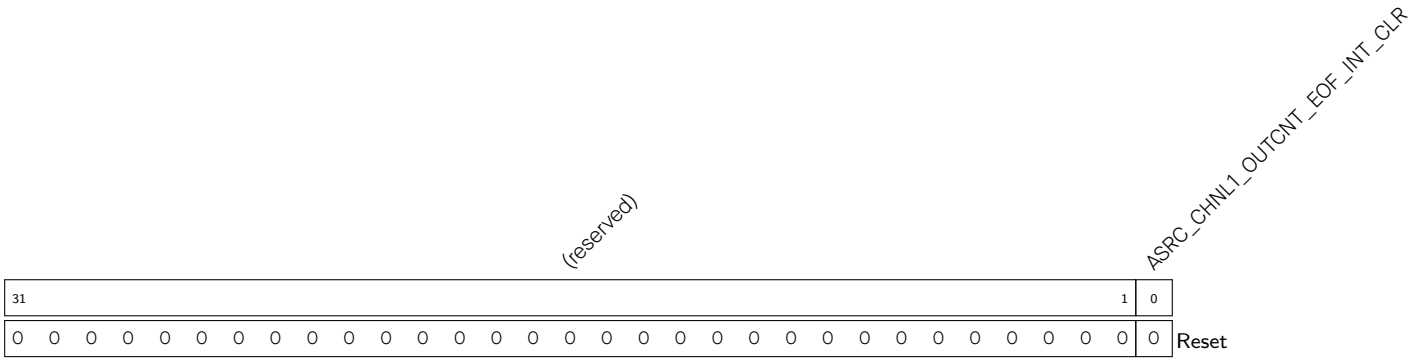
ASRC_CHNL1_OUTCNT_EOF_INT_ST ASRC_CHNL1_INT 中断的屏蔽中断状态。(RO)

Register 19.23. ASRC_CHNL1_INT_ENA_REG (0x0064)



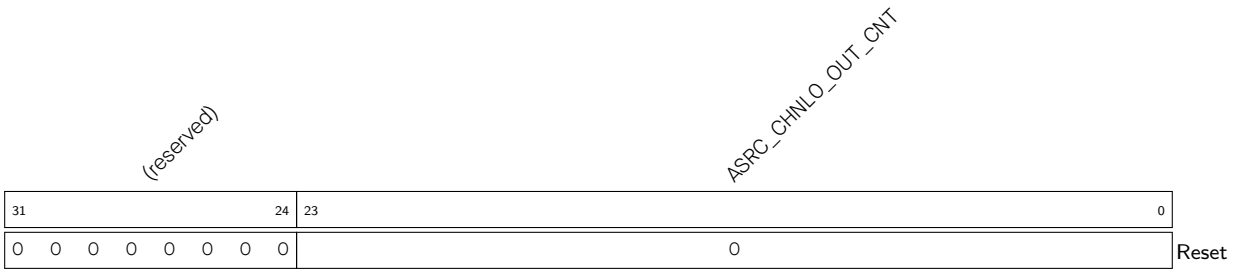
ASRC_CHNL1_OUTCNT_EOF_INT_ENA 写 1 使能 ASRC_CHNL1_INT 中断。(R/W)

Register 19.24. ASRC_CHNL1_INT_CLR_REG (0x0068)



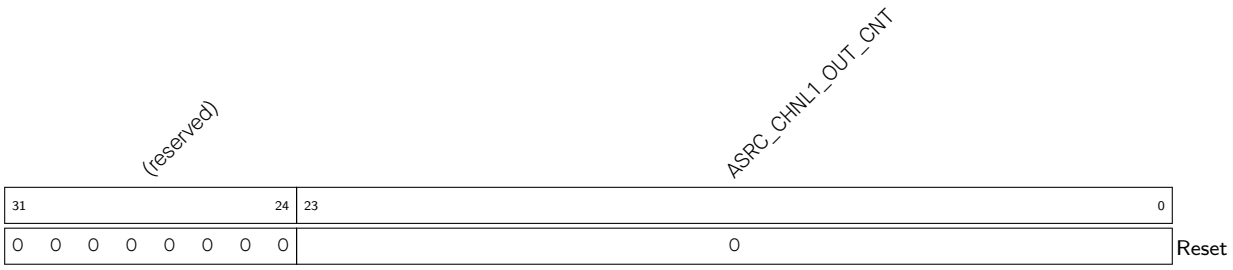
ASRC_CHNL1_OUTCNT_EOF_INT_CLR 写 1 清除 ASRC_CHNL1_INT 中断。(WT)

Register 19.25. ASRC_CHNL0_OUT_CNT_REG (0x0030)



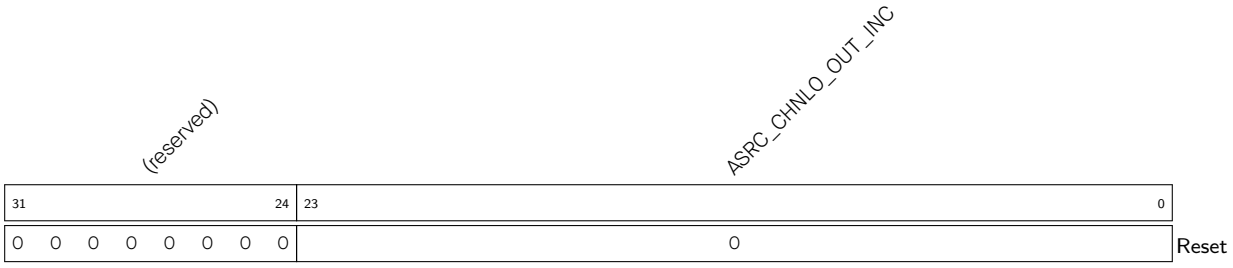
ASRC_CHNL0_OUT_CNT 累计向 DMA 写入的总字节数。(RO)

Register 19.26. ASRC_CHNL1_OUT_CNT_REG (0x006C)



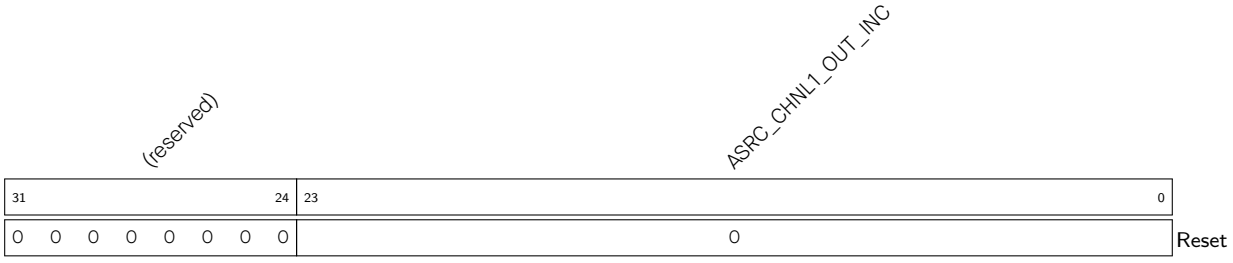
ASRC_CHNL1_OUT_CNT 累计向 DMA 写入的总字节数。(RO)

Register 19.27. ASRC_CHNL0_TRACE1_REG (0x0038)



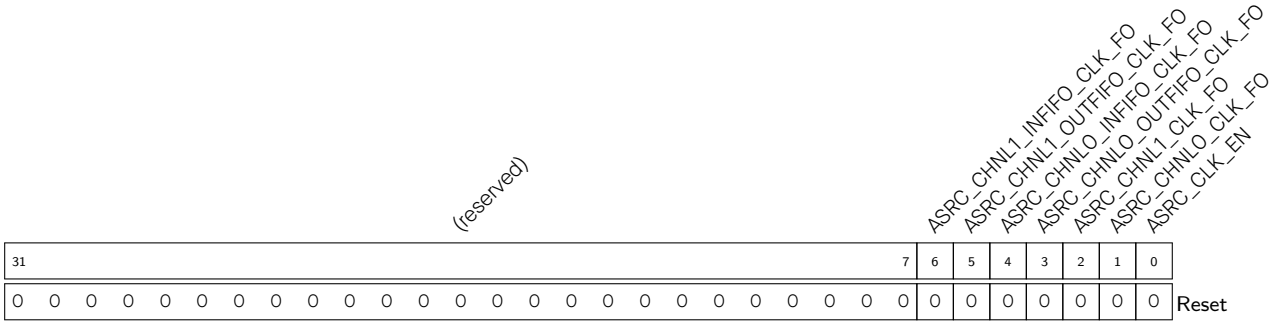
ASRC_CHNL0_OUT_INC 自上次 EOF 发生后向 DMA 写入的字节数。(RO)

Register 19.28. ASRC_CHNL1_TRACE1_REG (0x0074)



ASRC_CHNL1_OUT_INC 自上次 EOF 发生后向 DMA 写入的字节数。(RO)

Register 19.29. ASRC_SYS_REG (0x00F8)



ASRC_CLK_EN 配置是否启用 ASRC 模块的时钟。

0: 禁用

1: 启用

(R/W)

ASRC_CHNL0_CLK_FO 设置此位使通道 0 时钟自由运行。(R/W)

ASRC_CHNL1_CLK_FO 设置此位使通道 1 时钟自由运行。(R/W)

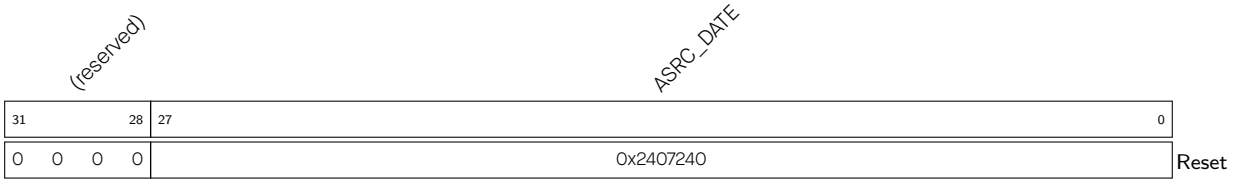
ASRC_CHNL0_OUTFIFO_CLK_FO 设置此位使通道 0 输出 FIFO 时钟自由运行。(R/W)

ASRC_CHNL0_INFIFO_CLK_FO 设置此位使通道 0 输入 FIFO 时钟自由运行。(R/W)

ASRC_CHNL1_OUTFIFO_CLK_FO 设置此位使通道 1 输出 FIFO 时钟自由运行。(R/W)

ASRC_CHNL1_INFIFO_CLK_FO 设置此位使通道 1 输入 FIFO 时钟自由运行。(R/W)

Register 19.30. ASRC_DATE_REG (0x00FC)



ASRC_DATE 版本控制寄存器。(R/W)

第 20 章

LCD 与 Camera 控制器 (LCD_CAM)

20.1 概述

ESP32-S31 的 LCD_CAM 控制器包含一个独立的 LCD 控制模块和 Camera（摄像头）控制模块，可以外接 LCD 和摄像头设备，功能灵活多样。

LCD 模块提供了一个 LCD 接口，可以连接兼容 RGB、Motorola 6800 (MOTO6800) 和 Intel 8080 (I8080) 的 LCD 设备。LCD 模块负责将视频数据发送到 LCD 设备。

Camera 模块提供一个并行摄像头接口，用于连接 DVP 摄像头，支持 8/16 位模式。Camera 模块可以管理与摄像头的通信，接收图像或视频帧。

ESP32-S31 的 LCD_CAM 模块特别适用于需要屏幕显示和视频处理的开发项目。

20.2 特性

LCD_CAM 模块具有以下主要特性：

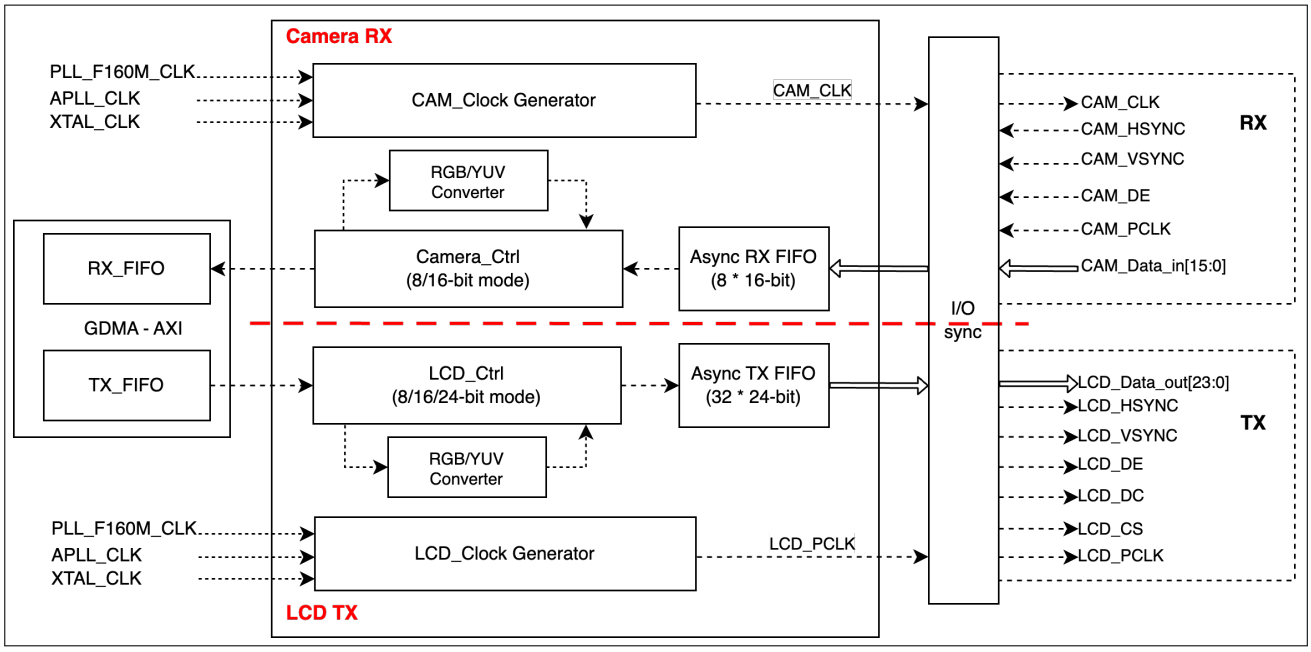
- 支持以下工作模式：
  - LCD 主机发送模式
  - Camera 从机接收模式
  - Camera 主机接收模式
- 支持同时外接 LCD 和摄像头设备
- 当外接 LCD 设备时，支持：
  - 8/16/24 位并行输出模式
  - RGB、MOTO6800、I8080 多种 LCD 模式
  - LCD 数据可由 GDMA-AXI 取自内部或外部存储器
- 当外接摄像头设备（即 DVP 图像传感器）时，支持：
  - 8/16 位并行输入模式
  - 视频数据可由 GDMA-AXI 存入内部或外部存储器
- 支持 LCD_CAM 接口中断

20.3 功能描述

20.3.1 功能框图

图 20.3-1 是 ESP32-S31 LCD_CAM 模块的结构框图，包含：

- 一个发送控制单元 (LCD_Ctrl)
- 一个接收控制单元 (Camera_Ctrl)
- 一个发送异步 FIFO (Async TX FIFO)，用于与外部设备如 LCD 进行交互
- 一个接收异步 FIFO (Async RX FIFO)，用于与外部设备如摄像头进行交互
- 两个时钟生成模块（LCD_Clock Generator 和 CAM_Clock Generator），用于生成对应模块的时钟
- 两个格式转换模块 (RGB/YUV Converter)，用于各种格式的视频数据互相转换



-→: 控制信号 =>: 数据流

图 20.3-1. LCD_CAM 功能框图

20.3.2 信号描述

表 20.3-1 列出了 LCD_CAM 工作所需的输入和输出信号。

三种工作模式的解释如下：

- **Camera 从机接收模式：**在此模式下，Camera 模块作为通信接收方输入控制信号。
- **Camera 主机接收模式：**在此模式下，Camera 模块向从机发送时钟信号，并作为通信接收方接收数据。
- **LCD 主机发送模式：**在此模式下，LCD 模块作为通信发起方向从机发送控制信号。

表 20.3-1. 信号描述

工作模式	信号[1]	方向	功能
Camera 从机接收模式	CAM_PCLK	输入	像素时钟输入信号
	CAM_VSYNC	输入	帧同步输入信号 (VSYNC)[3]

见下页

表 20.3-1 – 接上页

工作模式	信号[1]	方向	功能
	CAM_HSYNC	输入	行同步输入信号 (HSYNC)[3]
	CAM_DE	输入	数据有效输入信号 (DE)[3]
	CAM_Data_in[*N*:0][2]	输入	并行输入数据总线，支持 8/16 位并行数据输入
Camera 主机接收模式	CAM_PCLK	输入	像素时钟输入信号
	CAM_CLK	输出	主机时钟输出信号
	CAM_VSYNC	输入	帧同步输入信号[3]
	CAM_HSYNC	输入	行同步输入信号[3]
	CAM_DE	输入	数据有效输入信号[3]
	CAM_Data_in[*N*:0][2]	输入	并行输入数据总线，支持 8/16 位并行数据输入
LCD 主机发送模式	LCD_PCLK	输出	LCD 像素时钟输出信号
	LCD_HSYNC	输出	RGB 格式下，用作行同步信号
	LCD_VSYNC	输出	RGB 格式下，用作帧同步信号
	LCD_DE	输出	RGB 格式下，用作数据有效信号
	LCD_DC	输出	I8080 格式下，用作数据和命令信号 (DC)
	LCD_CS	输出	I8080/MOTO6800 格式下，用作片选信号 (CS)
	LCD_Data_out[*M*:0][2]	输出	并行输出数据总线，支持 8/16/24 位并行数据输出

[1] LCD_CAM 的所有信号均需要经过 GPIO 交换矩阵映射到芯片管脚。更多信息请参考章节 *6 IO MUX 和 GPIO 交换矩阵 [to be added later]*。

[2] 输入信号位宽 8 或 16 时，则 *N* 分别为 7 或 15。输出信号位宽 8、16 或 24 时，*M* 分别为 7、15 或 23。

[3] LCD_CAM_CAM_VH_DE_MODE_EN 为 1（VSYNC + HSYNC 模式）时：VSYNC、HSYNC、DE 线同时控制数据有效，因此用户需要接 VSYNC、HSYNC、DE 信号线。LCD_CAM_CAM_VH_DE_MODE_EN 为 0（DE 模式）时：VSYNC、DE 线同时控制数据有效，HSYNC 线可以不接。但是在此模式下，Camera 的 YUV-RGB 转换功能将不可用。

20.3.3 LCD_CAM 模块时钟

20.3.3.1 LCD 时钟

时钟源经时钟生成模块处理后生成 LCD 模块所需的时钟，如图 20.3-2 所示。其中，

- LCD_CLK 为 LCD 模块的主时钟，由时钟源分频获得；
- LCD_PCLK 为像素时钟，由 LCD_CLK 分频获得。

HP_SYS_CLKRST_LCD_CLK_EN 用于使能 LCD_CLK_SRC（时钟源），HP_SYS_CLKRST_LCD_CLK_SRC_SEL 用于选择时钟源：

- 0：选择 XTAL_CLK
- 1：选择 PLL_F160M_CLK
- 2：选择 APLL_CLK
- 3：关闭 LCD 时钟源

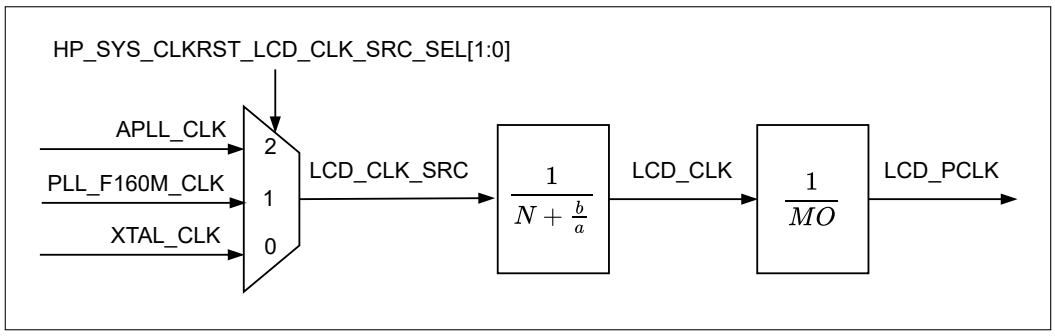


图 20.3-2. LCD 时钟

LCD_CLK 的频率 f_{LCD_CLK} 与分频器时钟源频率 $f_{LCD_CLK_SRC}$ 间的关系如下：

$$f_{LCD_CLK} = \frac{f_{LCD_CLK_SRC}}{N + \frac{b}{a}}$$

其中，*N*、*a*、*b* 与 HP_SYS_CLKRST_LCD_CLK_DIV_NUM、HP_SYS_CLKRST_LCD_CLK_DIV_NUMERATOR、HP_SYS_CLKRST_LCD_CLK_DIV_DENOMINATOR 的值相关，具体为：

- HP_SYS_CLKRST_LCD_CLK_DIV_NUM = 0 时，*N* = 1，*a* 和 *b* 的值无需关注，始终为 1 分频。
- HP_SYS_CLKRST_LCD_CLK_DIV_NUM >=1 时，*N* = HP_SYS_CLKRST_LCD_CLK_DIV_NUM + 1。
  - 对于整数分频，请将 HP_SYS_CLKRST_LCD_CLK_DIV_NUMERATOR 和 HP_SYS_CLKRST_LCD_CLK_DIV_DENOMINATOR 清零。
  - 对于小数分频，*b* = HP_SYS_CLKRST_LCD_CLK_DIV_NUMERATOR，*a* = HP_SYS_CLKRST_LCD_CLK_DIV_DENOMINATOR，请保证 HP_SYS_CLKRST_LCD_CLK_DIV_NUMERATOR 的值小于 HP_SYS_CLKRST_LCD_CLK_DIV_DENOMINATOR 的值。

LCD 模块的像素时钟 LCD_PCLK 由 LCD_CLK 分频获得。即：

$$f_{LCD_PCLK} = \frac{f_{LCD_CLK}}{MO}$$

其中，MO 由 LCD_CAM_LCD_CLK_EQU_SYSCLK 和 LCD_CAM_LCD_CLKCNT_N 决定，具体为：

- LCD_CAM_LCD_CLK_EQU_SYSCLK = 1 时，MO = 1。
- LCD_CAM_LCD_CLK_EQU_SYSCLK = 0 时，MO = LCD_CAM_LCD_CLKCNT_N + 1。

注意：

- LCD_CAM_LCD_CLKCNT_N 不可配置为 0。
- 使用小数分频功能会产生时钟抖动。当 LCD_CLK 和 LCD_PCLK 无法通过 PLL_F160M_CLK 整数分频产生时，可使用 APLL_CLK 作为时钟源，详情请参考章节 7 *复位和时钟 [to be added later]*。

20.3.3.2 Camera 时钟

时钟源经时钟生成模块处理后生成 Camera 模块所需的时钟，如图 20.3-3 所示。其中，

- CAM_CLK 为 Camera 模块的主机时钟输出，由时钟源分频获得。
- CAM_PCLK 为像素时钟，从 Camera 从机输入获得。

HP_SYS_CLKRST_CAM_CLK_EN 用于使能 CAM_CLK_SRC（时钟源），HP_SYS_CLKRST_CAM_CLK_SRC_SEL 用于选择时钟源：

- 0：XTAL_CLK
- 1：PLL_F160M_CLK
- 2：APLL_CLK
- 3：关闭 Camera 时钟源

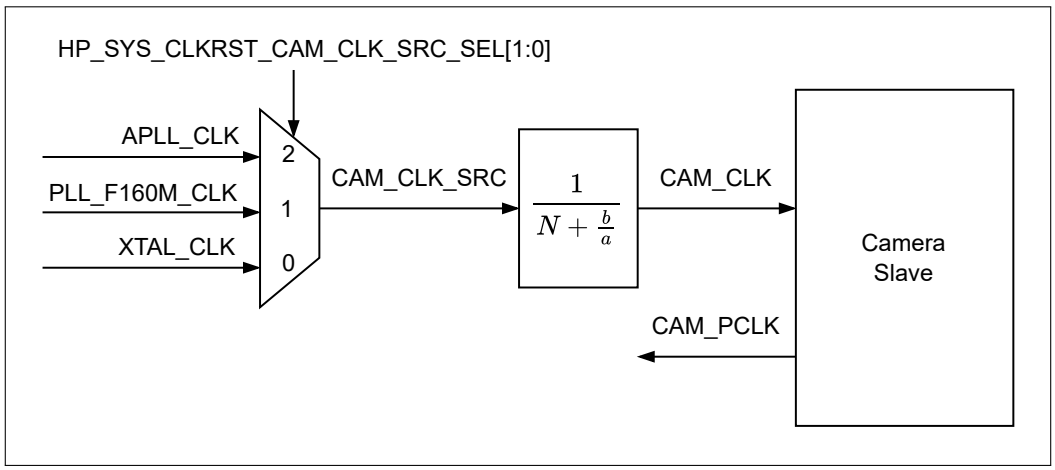


图 20.3-3. Camera 时钟

CAM_CLK 的频率 f_{CAM_CLK} 与分频器时钟源频率 $f_{CAM_CLK_SRC}$ 间的关系如下：

$$f_{CAM_CLK} = \frac{f_{CAM_CLK_SRC}}{N + \frac{b}{a}}$$

其中，2 <= *N* <= 256，*N* 对应为 HP_SYS_CLKRST_CAM_CLK_DIV_NUM 的值，具体为：

- HP_SYS_CLKRST_CAM_CLK_DIV_NUM = 0 时，*N* = 256。
- HP_SYS_CLKRST_CAM_CLK_DIV_NUM = 1 时，*N* = 2。
- HP_SYS_CLKRST_CAM_CLK_DIV_NUM 为其它值时，*N* = HP_SYS_CLKRST_CAM_CLK_DIV_NUM 的值。

b 对应 HP_SYS_CLKRST_CAM_CLK_DIV_NUMERATOR 的值，*a* 对应 HP_SYS_CLKRST_CAM_CLK_DIV_DENOMINATOR 的值。

- 对于整数分频，HP_SYS_CLKRST_CAM_CLK_DIV_NUMERATOR 和 HP_SYS_CLKRST_CAM_CLK_DIV_DENOMINATOR 都清零。
- 对于小数分频，HP_SYS_CLKRST_CAM_CLK_DIV_NUMERATOR 的值应小于 HP_SYS_CLKRST_CAM_CLK_DIV_DENOMINATOR 的值。

20.3.4 LCD_CAM 模块复位

LCD_CAM 模块共包含下列可应用于不同范围的复位寄存器：

- HP_SYS_CLKRST_LCDCAM_RST_EN：配置该复位寄存器，可以复位 LCD_CAM 模块中的所有子模块。
- LCD_CAM_LCD_RESET：配置该寄存器可复位所有 LCD 通路部分，包括发送控制单元 (LCD_Ctrl)、LCD 格式转换模块 (RGB/YUV Converter) 和 Async TX FIFO。配置该寄存器不会影响 Camera 部分的正常运行。

- LCD_CAM_CAM_RESET：配置该寄存器可复位所有 Camera 通路部分，包括接收控制单元 (Camera_Ctrl)、Camera 格式转换模块 (RGB/YUV Converter) 和 Async RX FIFO。配置该寄存器不会影响 LCD 通路的正常运行。
- LCD_CAM_LCD_AFIFO_RESET：配置该寄存器可以复位 Async TX FIFO。例如，可以在 LCD 每次发送完一帧时配置该复位寄存器，以防止前一帧的数据传输错误影响到下一帧的发送。
- LCD_CAM_CAM_AFIFO_RESET：配置该寄存器可以复位 Async RX FIFO。例如，可以在 Camera 每次接收完一帧时配置该复位寄存器，以防止前一帧的数据传输错误影响到下一帧的接收。

注意：

- 复位寄存器 HP_SYS_CLKRST_LCDCAM_RST_EN 不是硬件自清，即写 1 使能复位，写 0 释放复位。
- 上述复位寄存器除 HP_SYS_CLKRST_LCDCAM_RST_EN 以外均为硬件自清，即写 1 后，硬件会生成一个复位脉冲，并自动清零该寄存器。
- 在模块和 FIFO 复位之前，需要先配置模块时钟 LCD_CLK 和 CAM_CLK。
- Async TX FIFO 复位需要先配置 LCD_CAM_LCD_RESET，再配置 LCD_CAM_LCD_AFIFO_RESET。
- Async RX FIFO 复位需要先配置 LCD_CAM_CAM_RESET，再配置 LCD_CAM_CAM_AFIFO_RESET。

20.3.5 LCD_CAM 数据格式控制

20.3.5.1 LCD 数据格式控制

在使用 LCD 发送数据时，数据会经过三个阶段的处理过程：

1. 阶段一：数据预处理
   从 GDMA-AXI 获取输入数据，并进行数据预处理，处理完成后将数据输出给 YUV-RGB 格式转换模块。
2. 阶段二：数据色彩空间转换
   在 YUV-RGB 格式转换模块中进行数据处理（如果不开启 YUV-RGB 格式转换模块，这一阶段的数据将不发生任何改变）。
3. 阶段三：数据后处理
   LCD 模块从 YUV-RGB 格式转换模块取得数据，并进行数据后处理，处理完成后发送到 GPIO。

> **说明：**
> 每一阶段的输出数据，均为下一阶段的输入数据。

以下详细介绍每个阶段的配置。

阶段一：数据预处理

在阶段一中，配置以下寄存器可以调整来自 GDMA-AXI 的数据的位宽、数据位顺序和字节序：

- LCD_CAM_LCD_BYTE_MODE
  - 0：LCD 从 GDMA-AXI 输入的数据位宽为 8 位
  - 1：LCD 从 GDMA-AXI 输入的数据位宽为 16 位
  - 2：LCD 从 GDMA-AXI 输入的数据位宽为 24 位
- LCD_CAM_LCD_BIT_ORDER

  - 0：不反转
  - 1：反转输入数据的数据位顺序
- LCD_CAM_LCD_BYTE_ORDER
  - 0：不反转
  - 1：反转数据字节顺序。仅在输入数据位宽为 16 位或 24 位时有效。

具体配置方式见下表。

表 20.3-2. LCD 数据格式控制 - 阶段一

GDMA-AXI 数据字节序列[1]	LCD_CAM_LCD_BYTE_MODE	LCD_CAM_LCD_BIT_ORDER	LCD_CAM_LCD_BYTE_ORDER[2]	阶段一输出数据序列[3,4]
B0,B1,B2,B3,B4,B5	0	0	0	{B0}{B1}{B2}{B3}{B4}{B5}
		1	0	{B0'}{B1'}{B2'}{B3'}{B4'}{B5'}
	1	0	0	{B1,B0}{B3,B2}{B5,B4}
			1	{B0,B1}{B2,B3}{B4,B5}
		1	0	{B1',B0'}{B3',B2'}{B5',B4'}
			1	{B0',B1'}{B2',B3'}{B4',B5'}
	2	0	0	{B2,B1,B0}{B5,B4,B3}
			1	{B0,B1,B2}{B3,B4,B5}
		1	0	{B2',B1',B0'}{B5',B4',B3'}
			1	{B0',B1',B2'}{B3',B4',B5'}

[1] GDMA-AXI 数据字节序列中的 B0 ~ B5 每一个代表一个字节的数据，靠左的数据对应低地址，即 B0 ~ B5 的地址为从低到高。

[2] 只有表格中的配置组合是有效的，所有表格所示之外的配置组合可能会导致不可预期的数据错误。

[3] 输出数据序列中，每个 {} 中的数据为大端并行数据，各个 {} 中的数据为串行关系，输出顺序为从左至右。以 {B0}{B1}{B2}{B3} 为例，{B0} 中的各个数据位之间为并行关系，而 {B0} 中的数据与 {B1}、{B2}、{B3} 的数据为串行关系。输出顺序为先输出 {B0}。

[4] 输出数据序列中，Bn'[7:0] = Bn[0:7]，其中 n = 0,1,2,3,4,5。

阶段二：数据色彩空间转换

在阶段二中，数据进行色彩空间转换，配置方式请参考章节 20.3.6。

> **说明：**
> 用户可以通过将 LCD_CAM_LCD_CONV_ENABLE 配置为 0 跳过阶段二，这种情况下，阶段一的输出将会直接成为阶段三的输入。

阶段三：数据后处理

在阶段三中，配置以下寄存器可以对将要输出的数据进行数据位顺序调整和字节重排列：

- LCD_CAM_LCD_WIRE_MODE
  - 0：LCD 输出到 GPIO 的数据位宽为 8 位
  - 1：LCD 输出到 GPIO 的数据位宽为 16 位
  - 2：LCD 输出到 GPIO 的数据位宽为 24 位

- LCD_CAM_LCD_DOUT_BIT_ORDER
  - 0：不反转
  - 1：反转输出数据每个字节的数据位顺序
- LCD_CAM_LCD_DOUT_BYTE_SWIZZLE_ENABLE
  - 0：不开启 LCD 输出数据字节重排列功能。
  - 1：开启 LCD 输出数据字节重排列功能，配置寄存器 LCD_CAM_LCD_DOUT_BYTE_SWIZZLE_MODE 以调整重排列模式，各个重排列模式的具体效果请参考表格 20.3-3。

表 20.3-3. LCD 输出数据重排列功能模式

阶段三输入数据字节序列[1]	LCD_CAM_LCD_DOUT_BYTE_SWIZZLE_MODE[2]	阶段三输出数据字节序列[3]
B0,B1,B2,B3,B4,B5	0	B1,B0,B3,B2,B5,B4
	1	B0,B2,B1,B3,B5,B4
	2	B1,B0,B2,B4,B3,B5
	3	B1,B2,B0,B4,B5,B3
	4	B2,B0,B1,B5,B3,B4
	5	B2,B1,B0,B5,B4,B3

[1] 输入数据字节序列中的 B0 ~ B5 每一个代表一个字节的数据，这表示把输入数据按照小端排列为字节序列。例如输入数据有效数据位宽为 16 位时，原始输入数据序列 {B1,B0}{B3,B2}{B5,B4} 转换为输入数据字节序列后为 {B0,B1,B2,B3,B4,B5}。

[2] 只有表格中的配置值是有效的，所有表格所示之外的配置值可能会导致不可预期的数据错误。

[3] 最终发送到 GPIO 的发送数据序列可以由阶段三输出数据字节序列和发送数据位宽转换得到，例如阶段三输出数据字节序列为 B1,B0,B2,B4,B3,B5 时，若发送数据位宽为 16 位，则发送数据序列为 {B0,B1}{B4,B2}{B5,B3}。其中，每个 {} 中的数据为大端并行数据，各个 {} 中的数据为串行关系，发送顺序为从左至右。在本例的发送数据序列中，{B0,B1} 中的各个数据位之间为并行关系，而 {B0,B1} 中的数据与 {B4,B2}、{B5,B3} 的数据为串行关系。发送顺序为先发送 {B0,B1}。

20.3.5.2 Camera 数据格式控制

在使用 Camera 接收数据时，配置以下寄存器可以调整送往 GDMA-AXI 的数据的数据位顺序和字节顺序：

- LCD_CAM_CAM_2BYTE_EN
  - 0：接收数据的位宽为 8 位
  - 1：接收数据的位宽为 16 位
- LCD_CAM_CAM_BIT_ORDER
  - 0：不反转
  - 1：反转数据位顺序：
    - 在 8 位模式下，CAM_DATA_in[7:0] 反转为 CAM_DATA_in[0:7]；
    - 在 16 位模式下，CAM_DATA_in[15:0] 反转为 CAM_DATA_in[0:15]。
- LCD_CAM_CAM_BYTE_ORDER
  - 0：不反转

– 1：反转数据字节顺序。仅在 16 位模式下有效。

具体配置方式见下表。

表 20.3-4. Camera 数据格式控制

接收数据序列[1]	LCD_CAM_CAM_2BYTE_EN	LCD_CAM_CAM_BIT_ORDER[2]	LCD_CAM_CAM_BYTE_ORDER[2]	GDMA-AXI 数据序列[3,4]
{B0}{B1}{B2}{B3}	0	0	0	B0,B1,B2,B3
		1	0	B0',B1',B2',B3'
{B1,B0}{B3,B2}	1	0	0	B0,B1,B2,B3
			1	B1,B0,B3,B2
		1	0	B0',B1',B2',B3'
			1	B1',B0',B3',B2'

[1] 接收数据序列中，每个 {} 中的数据为大端并行数据，各个 {} 中的数据为串行关系，接收顺序为从左至右。以 {B0}{B1}{B2}{B3} 为例，{B0} 中的各个数据位之间为并行关系，而 {B0} 中的数据与 {B1}、{B2}、{B3} 的数据为串行关系。接收顺序为先接收 {B0}。

[2] 只有表格中的配置组合是有效的，所有表格所示之外的配置组合可能会导致不可预期的数据错误。

[3] GDMA-AXI 数据序列中的 B0 ~ B3 每一个代表一个字节的数据，靠左的数据对应低地址，即 B0 ~ B3 的地址为从低到高。

[4] GDMA-AXI 数据序列中，Bn'[7:0] = Bn[0:7]，其中 n = 0,1,2,3。

说明：
当接收序列为每次接收 1 个字节数据时，CAM_Data_in[7:0] 为有效数据，因此用户需要将 CAM_Data_in[7:0] 与主机相连。

20.3.6 YUV-RGB 数据格式转换

ESP32-S31 的 LCD_CAM 模块支持 YUV-RGB 数据格式互相转换。LCD 模块以及 Camera 模块各有一个数据格式转换模块，其功能包括：

- 支持在 BT601 和 BT709 两种标准下进行格式转换
- RGB565 格式到 YUV422/420/411 (full/limited range) 格式的转换
- YUV422/420/411 (full/limited range) 格式到 RGB565 格式的转换
- YUV422/420/411 (full/limited range) 各种格式的转换

LCD 模块的 YUV-RGB 数据格式转换模块还支持:

- YUV422/420/411 (full/limited range) 格式到 RGB888 (full/limited range) 格式的转换
- RGB565 格式到 YUV444 (full/limited range) 格式的转换
- RGB888 (full/limited range) 格式与 RGB565 格式的转换
- YUV422/420/411 (full/limited range) 格式到 YUV444 (full/limited range) 格式的转换
- 当不启用 YUV420 上采样时，可以使能 LCD_CAM_LCD_TRANS_BUFFER_ENA，使用 YUV420 上采样 SRAM 作为数据 buffer，提高鲁棒性

20.3.6.1 YUV 数据流格式

在 ESP32-S31 LCD_CAM 模块中约定如下：

假设 8 个依次待传输像素点对应的 YUV 数据为 $[Y_i, U_i, V_i]$ (i = 1 ~ 8)，则在：

- YUV422 模式下，LCD 依次发送或 Camera 依次接收如下数据：

Y_1	U_1	Y_2	V_1	Y_3	U_3	Y_4	V_3	Y_5	U_5	Y_6	V_5	Y_7	U_7	Y_8	V_7

- YUV420 模式下，LCD 依次发送或 Camera 依次接收如下数据。
  - 奇数行

U_1	Y_1	Y_2	U_3	Y_3	Y_4

  - 偶数行

V_1	Y_1	Y_2	V_3	Y_3	Y_4

- YUV411 模式下，LCD 依次发送或 Camera 依次接收如下数据：

Y_1	U_1	Y_2	Y_3	V_3	Y_4	Y_5	U_5	Y_6	Y_7	V_7	Y_8

20.3.6.2 格式转换模块配置流程

Camera 模块中的格式转换模块与 LCD 模块中的格式转换模块几乎完全相同。因此，下文以 LCD 模块中的格式转换模块为例说明配置流程：

1. 置位 LCD_CAM_LCD_CONV_ENABLE 使能 YUV-RGB 格式转换模块；
2. 通过 LCD_CAM_LCD_CONV_MODE_8BITS_ON 配置输入数据的有效位宽：
   - 0：输入数据的有效数据位宽为 16 位
   - 1：输入数据的有效数据位宽为 8 位
3. 配置输出数据有效位宽：
   - 对于 Camera 模块中的格式转换模块：输出数据的有效数据位宽始终等于输入数据的有效数据位宽
   - 对于 LCD 模块中的格式转换模块，当 LCD_CAM_LCD_WIRE_MODE 的值为：
     - 2 时，格式转换模块的输出数据的有效数据位宽为 24 位
     - 其他时，格式转换模块的输出数据的有效数据位宽等于输入数据的有效数据位宽
4. 通过 LCD_CAM_LCD_CONV_PROTOCOL_MODE 配置标准：
   - 0：使用 BT601 标准
   - 1：使用 BT709 标准
5. 配置转换模式：

- 在 24 位输出时：

表 20.3-5. 24 位输出时的转换模式配置

转换模式	TRANS_MODE[1]	YUV_MODE[2]	YUV2YUV_MODE[3]	RGB2RGB_MODE[4]
RGB565 → YUV444	1	-	3	-
RGB565 → RGB888	0	-	-	1
YUV422 → RGB888	0	0	3	-
YUV420 → RGB888	0	1	3	-
YUV411 → RGB888	0	2	3	-
YUV422 → YUV444	1	0	0/1/2	-
YUV420 → YUV444	1	1	0/1/2	-
YUV411 → YUV444	1	2	0/1/2	-

[1] 对应 LCD_CAM_LCD_CONV_TRANS_MODE 的值
[2] 对应 LCD_CAM_LCD_CONV_YUV_MODE 的值
[3] 对应 LCD_CAM_LCD_CONV_YUV2YUV_MODE 的值
[4] 对应 LCD_CAM_LCD_CONV_RGB2RGB_MODE 的值

- 在非 24 位输出时：

表 20.3-6. 非 24 位输出时的转换模式配置

转换模式	TRANS_MODE[1]	YUV_MODE[2]	YUV2YUV_MODE[3]	RGB2RGB_MODE[4]
RGB565 → YUV422	1	0	3	-
RGB565 → YUV420	1	1	3	-
RGB565 → YUV411	1	2	3	-
RGB888 → RGB565	0	-	-	0
YUV422 → RGB565	0	0	3	-
YUV420 → RGB565	0	1	3	-
YUV411 → RGB565	0	2	3	-
YUV422 → YUV420	1	0	1	-
YUV422 → YUV411	1	0	2	-
YUV420 → YUV422	1	1	0	-
YUV420 → YUV420	1	1	1	-
YUV411 → YUV422	1	2	0	-
YUV411 → YUV420	1	2	1	-

[1] 对应 LCD_CAM_LCD_CONV_TRANS_MODE 的值
[2] 对应 LCD_CAM_LCD_CONV_YUV_MODE 的值
[3] 对应 LCD_CAM_LCD_CONV_YUV2YUV_MODE 的值
[4] 对应 LCD_CAM_LCD_CONV_RGB2RGB_MODE 的值

6. 配置输入数据色彩空间：配置 LCD_CAM_LCD_CONV_DATA_IN_MODE 为：
   - 0：有限色彩空间 (limited range)[1]
   - 1：全色彩空间 (full range)[2]
7. 配置输出数据色彩空间：配置 LCD_CAM_LCD_CONV_DATA_OUT_MODE 为：

- 0：有限色彩空间 (limited range)[1]
- 1：全色彩空间 (full range)[2]

说明：

1. 如果选择有限色彩空间，则
   - RGB 的取值范围为：16 ~ 235；
   - YUV 的取值范围为：
     - Y：16 ~ 235；
     - U-V：16 ~ 240。
2. 如果选择全色彩空间，则 RGB 或 YUV 的取值范围为：0 ~ 255；

20.3.7 LCD_CAM 时序

20.3.7.1 LCD 时序（RGB 格式）

图 20.3-4 所示为 LCD RGB 格式的视频帧结构：

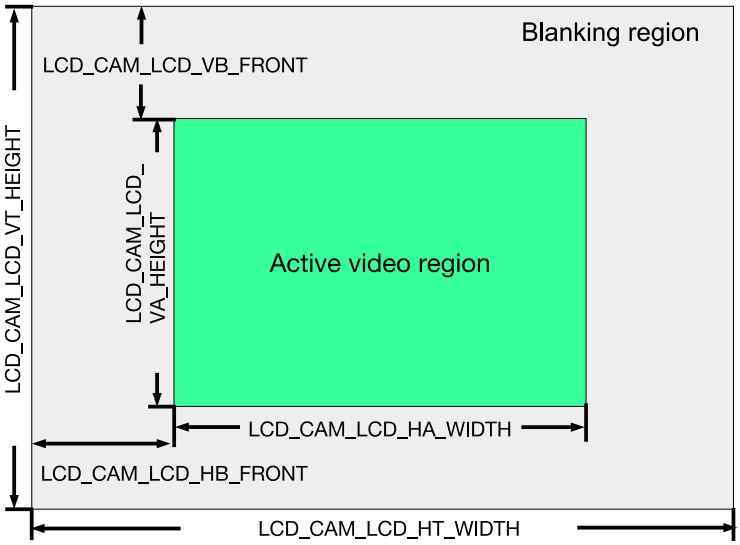


图 20.3-4. LCD 视频帧结构

如图所示，视频帧结构由以下寄存器配置：

- LCD_CAM_LCD_VT_HEIGHT
- LCD_CAM_LCD_VA_HEIGHT
- LCD_CAM_LCD_HB_FRONT
- LCD_CAM_LCD_HT_WIDTH
- LCD_CAM_LCD_HA_WIDTH
- LCD_CAM_LCD_VB_FRONT

图 20.3-5 显示了一个完整帧的时序。

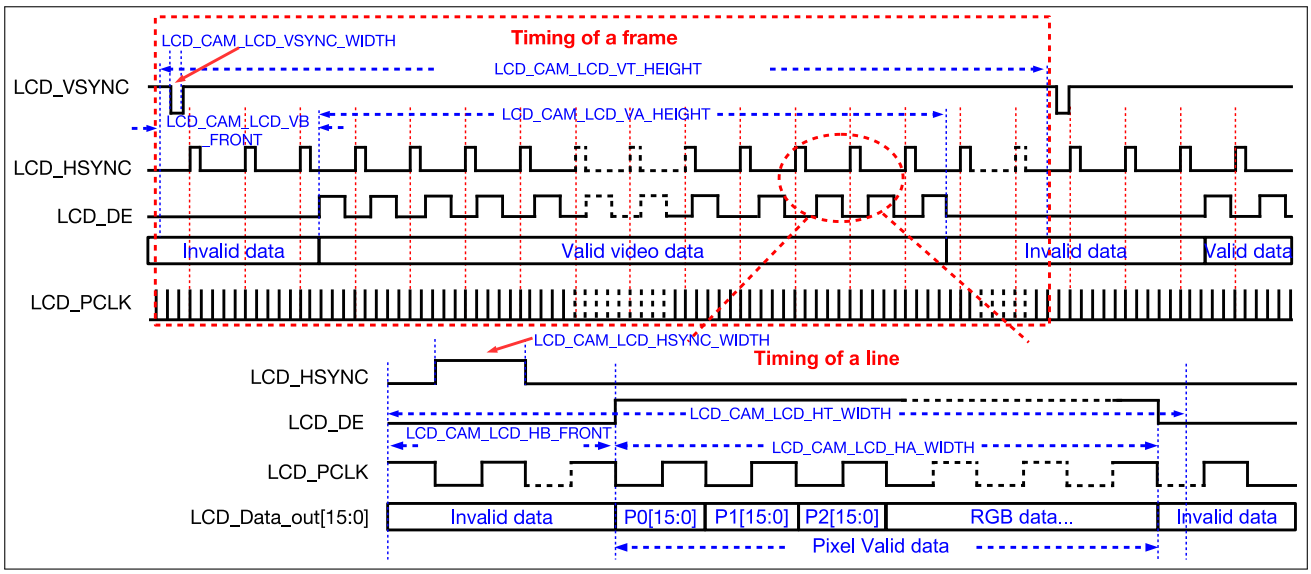


图 20.3-5. LCD 时序（RGB 格式）

说明：

配置上图所示的时序参数时，请特别注意参数值等于寄存器值 + 1。例如，如果希望 VSYNC 宽度为 1，则需要配置 LCD_CAM_LCD_VSYNC_WIDTH 为 0。更多信息见寄存器描述部分。

20.3.7.2 LCD 时序（I8080/MOTO6800 格式）

图 20.3-6 为 LCD I8080/MOTO6800 格式的时序图。

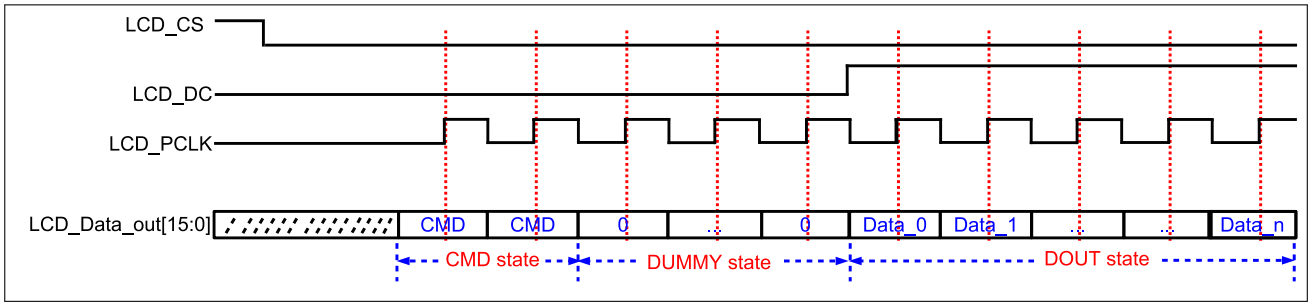


图 20.3-6. LCD 时序（I8080/MOTO6800 格式）

20.4 中断

ESP32-S31 的 LCD_CAM 可以生成以下中断信号并发送给中断矩阵。

- LCD_CAM_INTR

这个中断信号可由 LCD_CAM 的以下内部中断源生成：

- LCD_CAM_CAM_HS_INT：当 Camera 模块接收行数大于等于 LCD_CAM_CAM_LINE_INT_NUM + 1 时触发。
- LCD_CAM_CAM_VSYNC_INT：当 Camera 模块接收完一帧数据时触发。

- LCD_CAM_LCD_TRANS_DONE_INT：当 LCD 模块完成数据发送时触发。
- LCD_CAM_LCD_VSYNC_INT：当 LCD 模块发送完一帧数据时触发。

说明：
中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 20.6 寄存器列表。

20.5 软件配置流程

说明：
LCD 模块和 Camera 模块的寄存器相关配置，均需要通过置位对应的 LCD_CAM_LCD_UPDATE 和 LCD_CAM_CAM_UPDATE 的方式来进行更新，从而将 LCD/Camera 寄存器数据从 APB 时钟域分别同步到 LCD/Camera 时钟域。见下文示例。

20.5.1 软件配置 LCD（RGB 格式）发送流程

软件配置 LCD 以 RGB 格式发送的流程如下：

1. 配置时钟，详见章节 20.3.3。
2. 配置信号管脚，详见表 20.3-1 和章节 6 *IO MUX 和 GPIO 交换矩阵 [to be added later]*。
3. 使能相应的中断，详见章节 20.4。
4. 置位 LCD_CAM_LCD_RGB_MODE_EN，使能 RGB 格式；
5. 配置帧结构，详见章节 20.3.7.1。
6. 置位 LCD_CAM_LCD_UPDATE 使配置生效。
7. 复位发送控制单元 LCD_Ctrl 和 Async TX FIFO，详见章节 20.3.4。
8. 配置 GDMA-AXI 发送链表，详见章节 3 通用 *DMA 控制器 (GDMA-AHB, GDMA-AXI)*。
9. 开始发送数据：
    - 等待 LCD 从机配置完成；
    - 置位 LCD_CAM_LCD_START 开始发送数据。
10. 等待步骤 3 设置的中断信号；
11. 在数据发送的帧间隔，检查 LCD_CAM_LCD_NEXT_FRAME_EN 是否被置位：
    - 如果被置位，则继续发送下一帧数据；
    - 如果没有被置位，则 LCD 停止发送数据。
12. 若数据发送完成，则清零 LCD_CAM_LCD_START。

20.5.2 软件配置 LCD（I8080/MOTO6800 格式）发送流程

软件配置 LCD 以 I8080/MOTO6800 格式发送的流程如下：

1. 配置时钟，详见章节 20.3.3。
2. 配置信号管脚，详见表 20.3-1 和章节 6 *IO MUX 和 GPIO 交换矩阵 [to be added later]*。
3. 使能相应的中断，详见章节 20.4。
4. 清零 LCD_CAM_LCD_RGB_MODE_EN 关闭 RGB 格式。
5. 配置 CMD 阶段，包括 LCD_CAM_LCD_CMD、LCD_CAM_LCD_CMD_2_CYCLE_EN、LCD_CAM_LCD_FIRST_CMD_VALUE 和 LCD_CAM_LCD_LATTER_CMD_VALUE。
6. 配置 DUMMY 阶段，包括 LCD_CAM_LCD_DUMMY、LCD_CAM_LCD_DUMMY_CYCLELEN。
7. 配置 DOUT 阶段，包括 LCD_CAM_LCD_DOUT，根据输出模式不同进行以下配置：
   - 定长输出模式[a]，在 LCD_CAM_LCD_DOUT_CYCLELEN 中配置数据长度。
   - 持续输出模式[b]，则需置位 LCD_CAM_LCD_ALWAYS_OUT_EN 使能持续输出模式，不需要配置 LCD_CAM_LCD_DOUT_CYCLELEN。

   说明：

   (a) 定长输出模式是指：LCD 发送完成 LCD_CAM_LCD_DOUT_CYCLELEN 中配置的数据长度即结束 LCD 发送。

   (b) 持续输出模式是指：LCD 持续发送数据，直到

   i. LCD_CAM_LCD_START 被清零；

   ii. 或 LCD_CAM_LCD_RESET 被置位；

   iii. 或 GDMA-AXI 中所有数据被发送完。
8. 配置 DC 信号模式，包括 DC 信号的默认值以及在各阶段的值，详见寄存器 LCD_CAM_LCD_MISC_REG 描述。
9. 置位 LCD_CAM_LCD_UPDATE 使配置生效。
10. 复位发送控制单元 (LCD_Ctrl) 和 Async TX FIFO，详见章节 20.3.4。
11. 配置 GDMA-AXI 发送链表，详见章节 3 *通用 DMA 控制器 (GDMA-AHB, GDMA-AXI)*。
12. 开始发送数据：
    - 等待 LCD 从机配置完成；
    - 置位 LCD_CAM_LCD_START 开始发送数据。
13. 等待步骤 3 设置的中断信号；
14. 若数据发送完成，则清零 LCD_CAM_LCD_START。

注意：

无论 LCD 配置为 RGB 格式，还是 I8080/MOTO6800 格式，访问内部和外部存储器时均应注意以下限制：

- 如果 LCD 数据总线设置为 8 位并行输出时，则
  - 像素时钟 LCD_PCLK 频率需小于 80 MHz；
  - 如果同时使用了 YUV-RGB 格式转换，则 LCD_PCLK 频率需小于 60 MHz。
- 如果 LCD 数据总线设置为 16 位并行输出时，则

- 像素时钟频率需小于 40 MHz；
- 如果同时使用了 YUV-RGB 格式转换，则 LCD_PCLK 频率需小于 30 MHz。

20.5.3 软件配置 Camera 接收流程

软件配置 Camera 接收模式的流程如下：

1. 根据章节 20.3.3 的描述，配置时钟。注意，在从机模式下，模块时钟频率需要大于 2 倍的图像传感器的 PCLK 频率。
2. 配置信号管脚，详见表 20.3-1 和章节 6 *IO MUX 和 GPIO 交换矩阵 [to be added later]*。
3. 根据控制信号 HSYNC 置位或清零 LCD_CAM_CAM_VH_DE_MODE_EN。
4. 配置接收数据格式，详见章节 20.3.5.2。
5. 置位 LCD_CAM_CAM_UPDATE 使配置生效。
6. 复位接收控制单元 Camera_Ctrl 和 Async Rx FIFO，详见章节 20.3.4。
7. 使能相应的中断，详见章节 20.4。
8. 配置 GDMA-AXI 接收链表，并在 LCD_CAM_CAM_REC_DATA_BYTELEN 中配置接收数据长度。
9. 开始接收数据：
   - 在主机模式下，等待从机准备好后，置位 LCD_CAM_CAM_START 开始接收数据；
   - 在从机模式下，置位 LCD_CAM_CAM_START，等待主机提供时钟和控制信号后开始接收数据。
10. 接收数据，并存到 GDMA-AXI。最终产生步骤 7 中设置的中断。

注意：

- 无论 Camera 模块配置为主机模式还是从机模式，均应注意以下限制：
  - 如果选择 8 位并行输入，则
    * 像素时钟 CAM_PCLK 频率需小于 80 MHz；
    * 如果同时使用了 YUV-RGB 格式转换，则 CAM_PCLK 频率需小于 60 MHz。
  - 如果选择 16 位并行输入，则
    * CAM_PCLK 频率需小于 40 MHz；
    * 如果同时使用了 YUV-RGB 格式转换，则 CAM_PCLK 频率需小于 30 MHz。

说明：
对于 LCD_CAM 模块，如果同时外接了 LCD 和摄像头，则访问内部/外部存储器时，需保证接口上最大的数据吞吐率小于 GDMA-AXI 总数据带宽。此处默认 AXI_CLK 频率为 80 MHz，更多有关 AXI_CLK 的信息请见章节 7 *复位和时钟 [to be added later]*。

20.6 寄存器列表

本小节的所有地址均为相对于 LCD_CAM 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

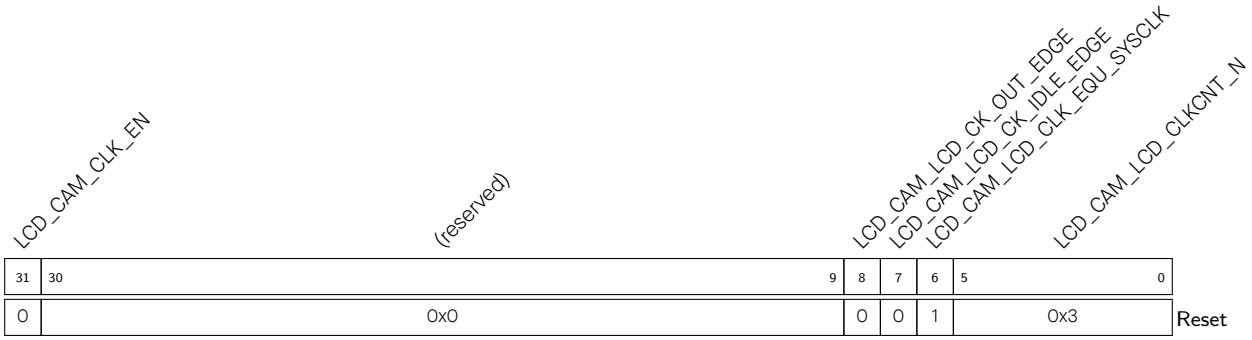
名称	描述	地址	访问
LCD 配置寄存器			
LCD_CAM_LCD_CLOCK_REG	LCD 时钟配置寄存器	0x0000	R/W
LCD_CAM_LCD_RGB_YUV_REG	LCD 数据格式转换配置寄存器	0x0010	R/W
LCD_CAM_LCD_USER_REG	LCD 用户配置寄存器	0x0014	varies
LCD_CAM_LCD_MISC_REG	LCD MISC 配置寄存器	0x0018	varies
LCD_CAM_LCD_RGB_VERTICAL_REG	LCD 垂直参数信号配置寄存器	0x001C	R/W
LCD_CAM_LCD_RGB_HORIZONTAL_REG	LCD 水平参数信号配置寄存器	0x0020	R/W
LCD_CAM_LCD_RGB_BLANK_REG	LCD blank 参数信号配置寄存器	0x0024	R/W
LCD_CAM_LCD_RGB_CTRL_REG	LCD 时序参数信号配置寄存器	0x0028	R/W
LCD_CAM_LCD_FIRST_CMD_VAL_REG	LCD CMD 值配置寄存器	0x002C	R/W
LCD_CAM_LCD_LATTER_CMD_VAL_REG	LCD CMD 值配置寄存器	0x0030	R/W
LCD_CAM_LCD_DLY_MODE_CFG1_REG	LCD 数据/信号延迟模式配置寄存器	0x0034	R/W
LCD_CAM_LCD_DLY_MODE_CFG2_REG	LCD 数据延迟模式配置寄存器	0x0038	R/W
LCD_CAM_LCD_TRANS_BUFF_CFG_REG	LCD 传输 buffer 配置寄存器	0x003C	R/W
Camera 配置寄存器			
LCD_CAM_CAM_CTRL_REG	Camera 时钟配置寄存器	0x0004	R/W
LCD_CAM_CAM_CTRL1_REG	Camera 控制寄存器	0x0008	varies
LCD_CAM_CAM_RGB_YUV_REG	Camera 数据格式转换配置寄存器	0x000C	R/W
中断寄存器			
LCD_CAM_LC_DMA_INT_ENA_REG	LCD_CAM GDMA-AXI 中断使能寄存器	0x0064	R/W
LCD_CAM_LC_DMA_INT_RAW_REG	LCD_CAM GDMA-AXI 原始中断寄存器	0x0068	RO
LCD_CAM_LC_DMA_INT_ST_REG	LCD_CAM GDMA-AXI 屏蔽中断状态寄存器	0x006C	RO
LCD_CAM_LC_DMA_INT_CLR_REG	LCD_CAM GDMA-AXI 中断清除寄存器	0x0070	WO
版本寄存器			
LCD_CAM_LC_DATE_REG	版本控制寄存器	0x00FC	R/W

20.7 寄存器

本小节的所有地址均为相对于 LCD_CAM 基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 20.1. LCD_CAM_LCD_CLOCK_REG (0x0000)



LCD_CAM_LCD_CLKCNT_N 与 LCD_CAM_LCD_CLK_EQU_SYSCLK 共同决定 LCD 模块的像素时钟 LCD_PCLK 的频率。

LCD_CAM_LCD_CLK_EQU_SYSCLK = 0 时，

$f_{LCD_PCLK} = f_{LCD_CLK}/(LCD_CAM_LCD_CLKCNT_N + 1)$。

注意，该字段不可配置为 0。

(R/W)

LCD_CAM_LCD_CLK_EQU_SYSCLK 与 LCD_CAM_LCD_CLKCNT_N 共同决定 LCD 模块的像素时钟 LCD_PCLK 的频率。

0：$f_{LCD_PCLK} = f_{LCD_CLK}/(LCD_CAM_LCD_CLKCNT_N + 1)$。

1：$f_{LCD_PCLK} = f_{LCD_CLK}$

(R/W)

LCD_CAM_LCD_CK_IDLE_EDGE 配置空闲状态下，LCD_PCLK 的电平。

0：低电平

1：高电平

(R/W)

LCD_CAM_LCD_CK_OUT_EDGE 配置前半个时钟周期内，LCD_PCLK 的电平。

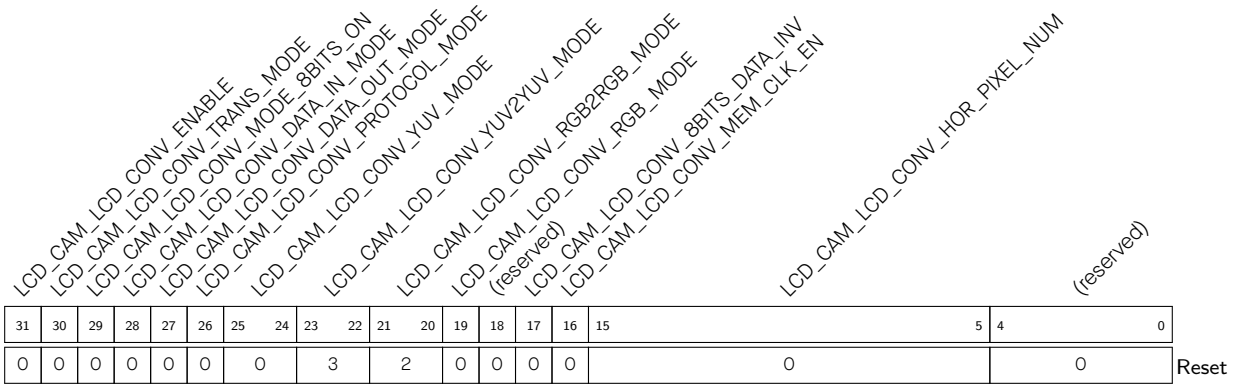
0：低电平

1：高电平

(R/W)

LCD_CAM_CLK_EN 置位此位，则强制打开所有配置寄存器的时钟，不再使用时钟门控。(R/W)

Register 20.2. LCD_CAM_LCD_RGB_YUV_REG (0x0010)



LCD_CAM_LCD_CONV_HOR_PIXEL_NUM 配置 YUV420 模式下的水平像素个数。(R/W)

LCD_CAM_LCD_CONV_MEM_CLK_EN 配置是否打开色彩转换 MEM CLOCK GATE。

0：打开 CLOCK GATE

1：关闭 CLOCK GATE

(R/W)

LCD_CAM_LCD_CONV_8BITS_DATA_INV 配置是否交换两个字节的 GDMA-AXI 输入数据。

0：不交换

1：交换

(R/W)

LCD_CAM_LCD_CONV_RGB_MODE 配置输出 RGB 的数据格式。

0：数据被转换为 RGB565

1：数据被转换为 RGB888

(R/W)

LCD_CAM_LCD_CONV_RGB2RGB_MODE 配置 RGB-to-RGB 模式下的数据转换格式。

0：数据被转换为 RGB565 格式

1：数据被转换为 RGB888 格式

2/3：不启用格式转换

(R/W)

LCD_CAM_LCD_CONV_YUV2YUV_MODE 配置 YUV-to-YUV 模式下的数据转换格式。

0：数据被转换为 YUV422 格式

1：数据被转换为 YUV420 格式

2：数据被转换为 YUV411 格式

3：不启用格式转换

仅在 LCD_CAM_LCD_CONV_TRANS_MODE = 1 时有效。

(R/W)

LCD_CAM_LCD_CONV_YUV_MODE 配置 YUV-to-YUV 模式或 YUV-to-RGB 模式下，输入数据具体的 YUV 格式。

0：输入数据为 YUV422 格式

1：输入数据为 YUV420 格式

2：输入数据为 YUV411 格式

(R/W)

见下页...

Register 20.2. LCD_CAM_LCD_RGB_YUV_REG (0x0010)

接上页...

LCD_CAM_LCD_CONV_PROTOCOL_MODE 配置格式转换的标准。

0：选择 BT601

1：选择 BT709

(R/W)

LCD_CAM_LCD_CONV_DATA_OUT_MODE 配置输出数据的色彩空间。

0：有限色彩空间

1：全色彩空间

(R/W)

LCD_CAM_LCD_CONV_DATA_IN_MODE 配置输入数据的色彩空间。

0：有限色彩空间

1：全色彩空间

(R/W)

LCD_CAM_LCD_CONV_MODE_8BITS_ON 配置 YUV-RGB 格式转换模块输入数据的有效位宽。

0：16 位模式

1：8 位模式

(R/W)

LCD_CAM_LCD_CONV_TRANS_MODE 配置格式转换模式。

0：转换成 RGB 格式

1：转换成 YUV 格式

(R/W)

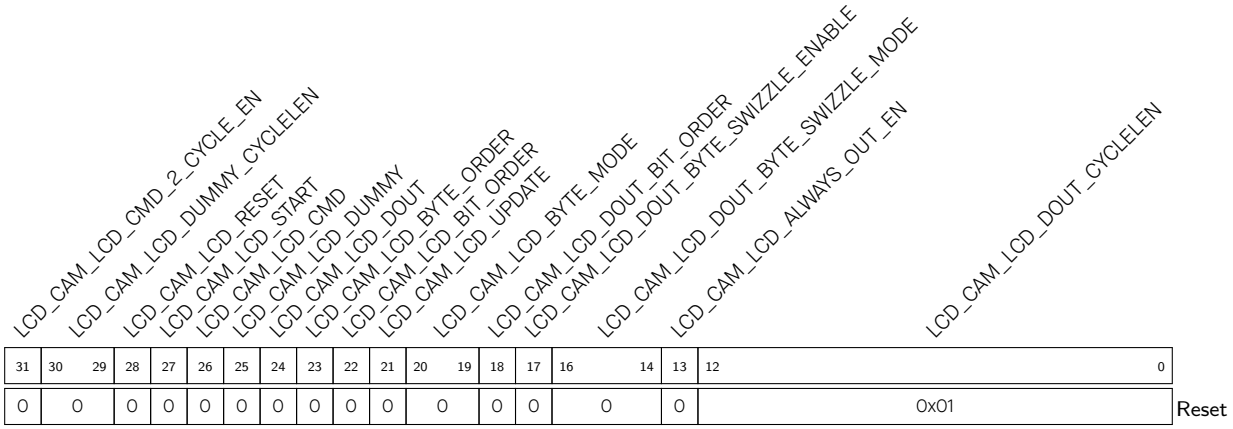
LCD_CAM_LCD_CONV_ENABLE 配置是否使能 YUV-RGB 格式转换模块。

0：旁路格式转换模块

1：使能格式转换模块

(R/W)

Register 20.3. LCD_CAM_LCD_USER_REG (0x0014)



LCD_CAM_LCD_DOUT_CYCLELEN 配置 LCD 模块输出数据阶段 (DOUT) 的周期时长。实际周期时长 = 本字段的值 + 1。(R/W)

LCD_CAM_LCD_ALWAYS_OUT_EN 使能 LCD 模块的持续输出模式。在该模式下，LCD 模块处于 DOUT 阶段时，持续输出数据，直至 LCD_CAM_LCD_START 被清除或 LCD_CAM_LCD_RESET 被置位。(R/W)

LCD_CAM_LCD_DOUT_BYTE_SWIZZLE_MODE 配置 LCD 数据字节重排列模式，具体重排列方式请参考表格 20.3-3。(R/W)

LCD_CAM_LCD_DOUT_BYTE_SWIZZLE_ENABLE 配置是否开启 LCD 输出数据字节重排列功能。

0：不开启

1：开启

(R/W)

LCD_CAM_LCD_DOUT_BIT_ORDER 配置是否反转 LCD 输出数据的数据位顺序。

0：不反转

1：反转数据位顺序。在 n 位模式下，LCD_DATA_out[n-1:0] 反转为 LCD_DATA_out[0:n-1]。

(R/W)

LCD_CAM_LCD_BYTE_MODE 配置来自 GDMA-AXI 的数据的位宽。

0：8 位

1: 16 位

2: 24 位

(R/W)

LCD_CAM_LCD_UPDATE 配置是否更新 LCD 寄存器配置。

0：不更新

1：更新。将由硬件清除。

(R/W)

LCD_CAM_LCD_BIT_ORDER 配置是否反转 LCD 输入数据的数据位顺序。

0：不反转

1：反转数据位顺序。在 n 位模式下，LCD_DATA_in[n-1:0] 反转为 LCD_DATA_in[0:n-1]。

(R/W)

见下页...

Register 20.3. LCD_CAM_LCD_USER_REG (0x0014)

接上页...

LCD_CAM_LCD_BYTE_ORDER 配置是否反转 LCD 输入数据的字节序。

0：不反转

1：反转字节序。仅在 16 位和 24 位模式下有效。

(R/W)

LCD_CAM_LCD_DOUT 配置 LCD 发送数据的 DOUT 阶段。

0：禁用该功能

1：LCD 启动后，开始以 LCD RGB/I8080 模式发送数据

(R/W)

LCD_CAM_LCD_DUMMY 配置 LCD 发送数据的 DUMMY 阶段。

0：禁用该功能

1：LCD 启动后，使能 LCD 模式下的 DUMMY 阶段

(R/W)

LCD_CAM_LCD_CMD 配置 LCD 发送数据的 CMD 阶段。

0：禁用该功能

1：LCD 启动后，开始以 LCD 模式发送命令

(R/W)

LCD_CAM_LCD_START 置位则 LCD 开始发送数据。(R/W)

LCD_CAM_LCD_RESET LCD 模块复位寄存器。(WO)

LCD_CAM_LCD_DUMMY_CYCLELEN 设置 DUMMY 阶段的周期。实际周期 = 该字段的值 + 1。(R/W)

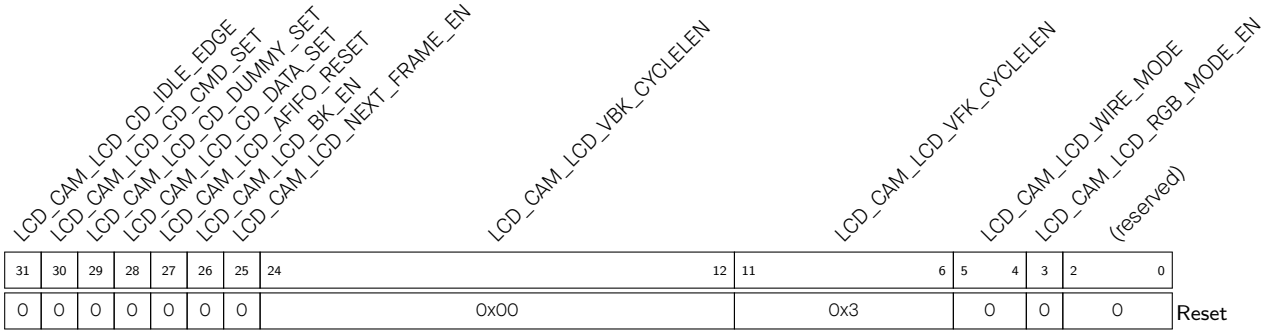
LCD_CAM_LCD_CMD_2_CYCLE_EN 设置 Command 阶段的周期。

0：一个周期

1：两个周期

(R/W)

Register 20.4. LCD_CAM_LCD_MISC_REG (0x0018)



LCD_CAM_LCD_RGB_MODE_EN 配置 LCD 输出模式。

0: 非 RGB 模式

1: RGB 模式

(R/W)

LCD_CAM_LCD_WIRE_MODE 配置 LCD 输出到 GPIO 的数据位宽。

0: 8 位

1: 16 位

2: 24 位

(R/W)

LCD_CAM_LCD_VFK_CYCLELEN 在非 LCD RGB 模式下，设置建立 (Setup) 时钟周期。实际周期 = 该字段的值 + 1。(R/W)

LCD_CAM_LCD_VBK_CYCLELEN 在非 LCD RGB 模式下，设置保持 (Hold) 时钟周期。实际周期 = 该字段的值 + 1。(R/W)

LCD_CAM_LCD_NEXT_FRAME_EN 配置是否继续发送下一帧数据。

0：当前一帧数据发送完成后，LCD 停止工作。

1：当前一帧数据发送完成后，继续发送下一帧数据。

(R/W)

LCD_CAM_LCD_BK_EN 配置 LCD 发送数据时，是否允许有消隐区。

0：无消隐区

1：允许有消隐区

(R/W)

LCD_CAM_LCD_AFIFO_RESET 置位则复位 Async TX FIFO。(WO)

LCD_CAM_LCD_CD_DATA_SET 配置 LCD I8080 模式下，处于 DOUT 阶段时，CD 线上的值。

0：LCD_CD = LCD_CAM_LCD_CD_IDLE_EDGE

1：LCD_CD = !LCD_CAM_LCD_CD_IDLE_EDGE

(R/W)

LCD_CAM_LCD_CD_DUMMY_SET 配置 LCD I8080 模式下，处于 DUMMY 阶段时，CD 线上的值。

0：LCD_CD = LCD_CAM_LCD_CD_IDLE_EDGE

1：LCD_CD = !LCD_CAM_LCD_CD_IDLE_EDGE

(R/W)

见下页...

Register 20.4. LCD_CAM_LCD_MISC_REG (0x0018)

接上页...

LCD_CAM_LCD_CD_CMD_SET 配置 LCD I8080 模式下，处于 CMD 阶段时，CD 线上的值。

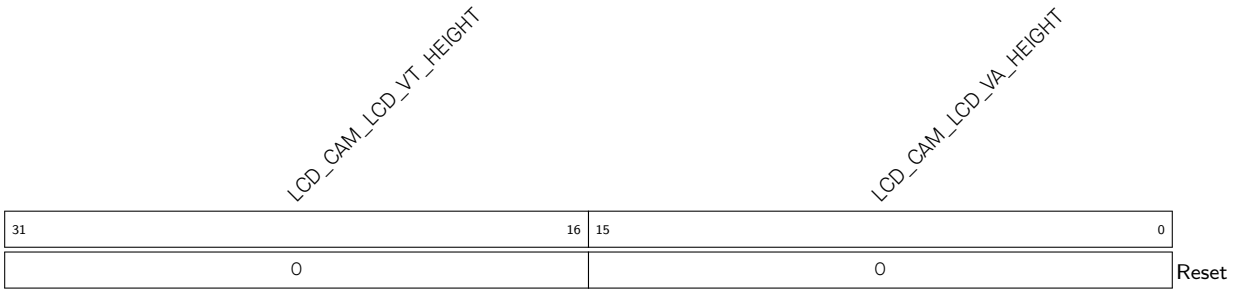
0：LCD_CD = LCD_CAM_LCD_CD_IDLE_EDGE

1：LCD_CD = !LCD_CAM_LCD_CD_IDLE_EDGE

(R/W)

LCD_CAM_LCD_CD_IDLE_EDGE LCD_DC 的默认值。(R/W)

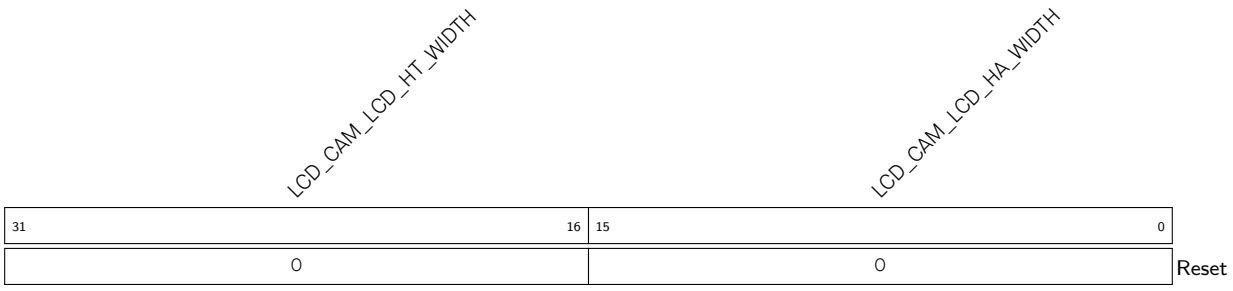
Register 20.5. LCD_CAM_LCD_RGB_VERTICAL_REG (0x001C)



LCD_CAM_LCD_VA_HEIGHT 配置帧有效垂直高度。(R/W)

LCD_CAM_LCD_VT_HEIGHT 配置帧总垂直高度。(R/W)

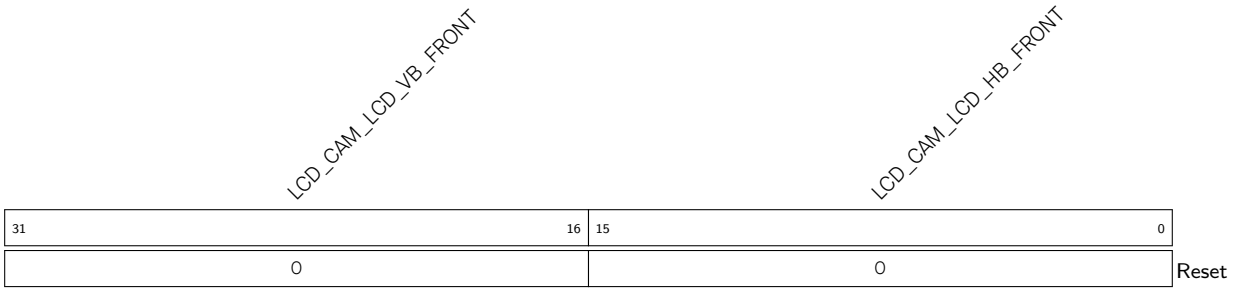
Register 20.6. LCD_CAM_LCD_RGB_HORIZONTAL_REG (0x0020)



LCD_CAM_LCD_HA_WIDTH 配置帧有效水平宽度。(R/W)

LCD_CAM_LCD_HT_WIDTH 配置帧总水平宽度。(R/W)

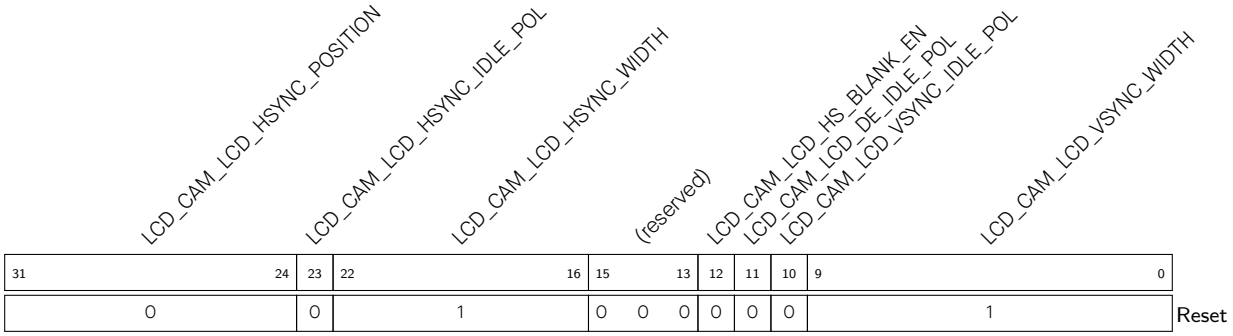
Register 20.7. LCD_CAM_LCD_RGB_BLANK_REG (0x0024)



LCD_CAM_LCD_VB_FRONT 配置 (VSYNC_WIDTH + 垂直后肩) 的值。 (R/W)

LCD_CAM_LCD_HB_FRONT 配置 (HSYNC_POSITION + HSYNC_WIDTH + 水平后肩) 的值。 (R/W)

Register 20.8. LCD_CAM_LCD_RGB_CTRL_REG (0x0028)



LCD_CAM_LCD_VSYNC_WIDTH 配置 LCD_VSYNC 的有效脉冲宽度。参数值 = 寄存器值 + 1。(R/W)

LCD_CAM_LCD_VSYNC_IDLE_POL 配置 LCD_VSYNC 空闲值。

0：空闲值为低电平

1：空闲值为高电平

(R/W)

LCD_CAM_LCD_DE_IDLE_POL 配置 LCD_DE 空闲值。

0：空闲值为低电平

1：空闲值为高电平

(R/W)

LCD_CAM_LCD_HS_BLANK_EN 配置 RGB 模式下，LCD_HSYNC 的输出。

0：LCD_HSYNC 仅在有效的视频行输出。

1：LCD_HSYNC 可在场消隐行输出。

(R/W)

LCD_CAM_LCD_HSYNC_WIDTH 配置 LCD_HSYNC 有效脉冲宽度。参数值 = 寄存器值 + 1。(R/W)

LCD_CAM_LCD_HSYNC_IDLE_POL 配置 LCD_HSYNC 空闲值。

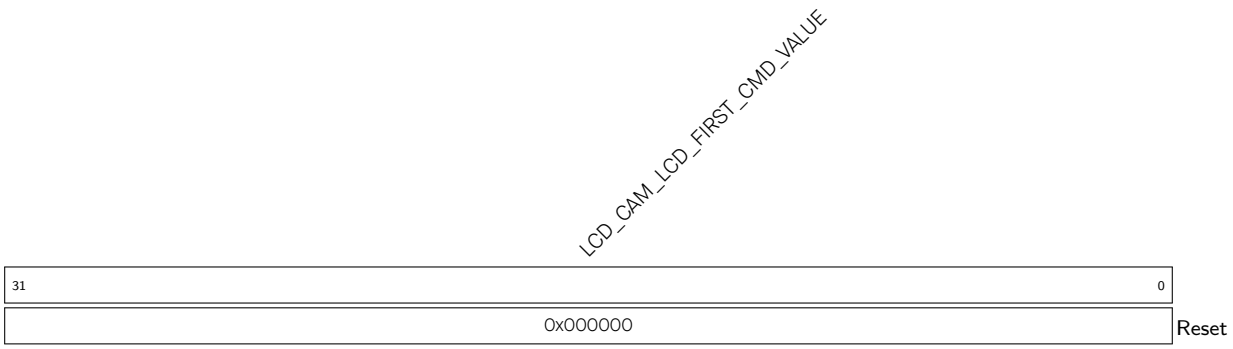
0：空闲值为低电平

1：空闲值为高电平

(R/W)

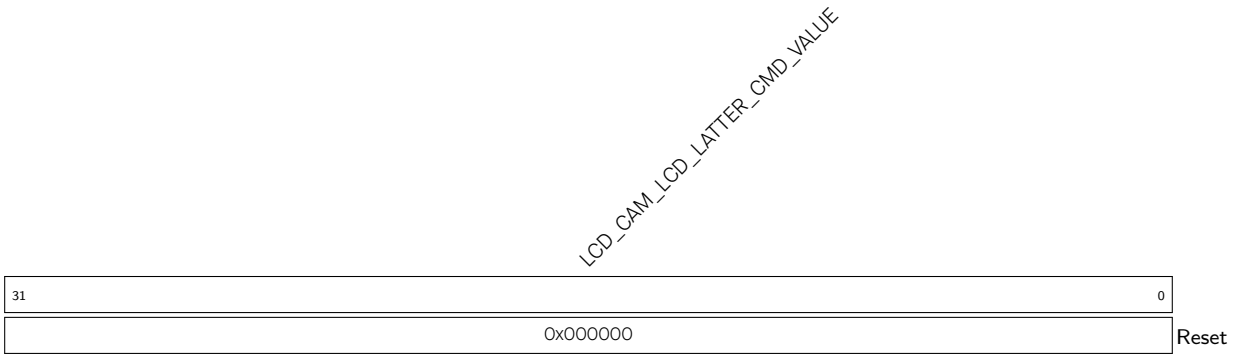
LCD_CAM_LCD_HSYNC_POSITION 配置 LCD_HSYNC 有效脉冲位置。参数值 = 寄存器值 + 1。单位为像素。(R/W)

Register 20.9. LCD_CAM_LCD_FIRST_CMD_VAL_REG (0x002C)



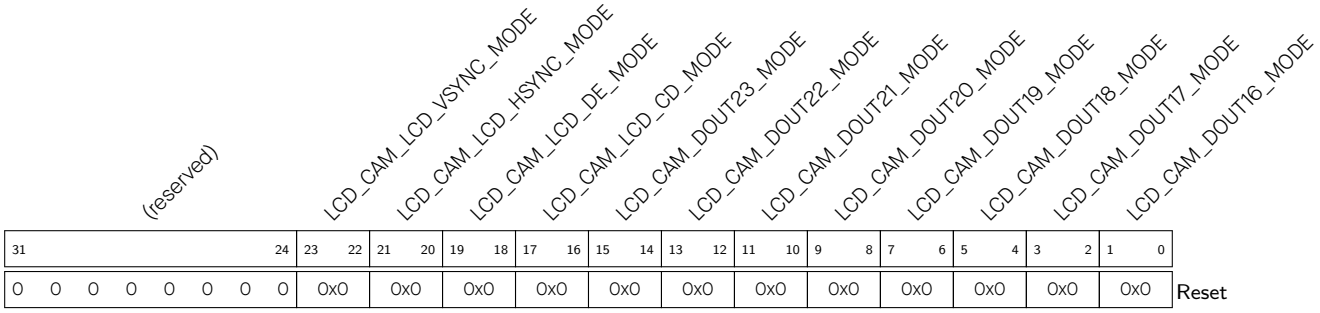
LCD_CAM_LCD_FIRST_CMD_VALUE 配置 LCD 处于 CMD 阶段第一个周期时，发送的命令值。(R/W)

Register 20.10. LCD_CAM_LCD_LATTER_CMD_VAL_REG (0x0030)



LCD_CAM_LCD_LATTER_CMD_VALUE 配置 LCD 处于 CMD 阶段第二个周期时，发送的命令值。(R/W)

Register 20.11. LCD_CAM_LCD_DLY_MODE_CFG1_REG (0x0034)



LCD_CAM_DOUT*n*_MODE (*n*: 16 - 23) 配置输出数据位 (*n*) 的延迟模式。

0：无延迟

1：在 LCD_CLK 上升沿延迟

2：在 LCD_CLK 下降沿延迟

(R/W)

LCD_CAM_LCD_CD_MODE 配置 LCD_DC 输出信号的延迟模式。

0：无延迟

1：在 LCD_CLK 上升沿延迟

2：在 LCD_CLK 下降沿延迟

(R/W)

LCD_CAM_LCD_DE_MODE 配置 LCD_DE 输出信号的延迟模式。

0：无延迟

1：在 LCD_CLK 上升沿延迟

2：在 LCD_CLK 下降沿延迟

(R/W)

LCD_CAM_LCD_HSYNC_MODE 配置 LCD_HSYNC 输出信号的延迟模式。

0：无延迟

1：在 LCD_CLK 上升沿延迟

2：在 LCD_CLK 下降沿延迟

(R/W)

LCD_CAM_LCD_VSYNC_MODE 配置 LCD_VSYNC 输出信号的延迟模式。

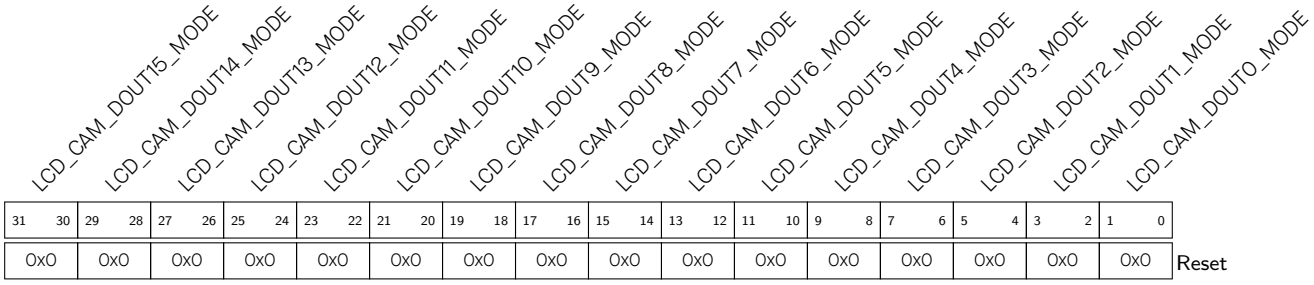
0：无延迟

1：在 LCD_CLK 上升沿延迟

2：在 LCD_CLK 下降沿延迟

(R/W)

Register 20.12. LCD_CAM_LCD_DLY_MODE_CFG2_REG (0x0038)



LCD_CAM_DOUT*n*_MODE (*n*: 0 - 15) 配置输出数据位 (*n*) 的延迟模式。

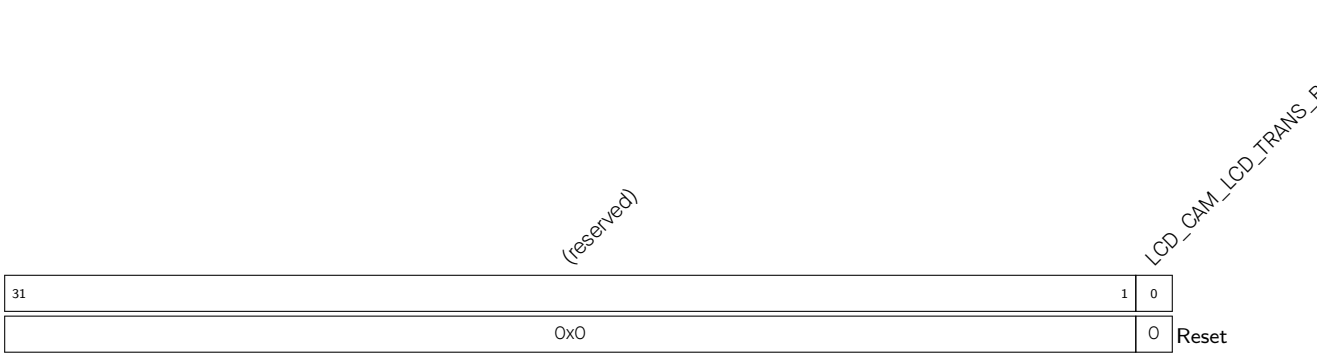
0：无延迟

1：在 LCD_CLK 上升沿延迟

2：在 LCD_CLK 下降沿延迟

(R/W)

Register 20.13. LCD_CAM_LCD_TRANS_BUFF_CFG_REG (0x003C)



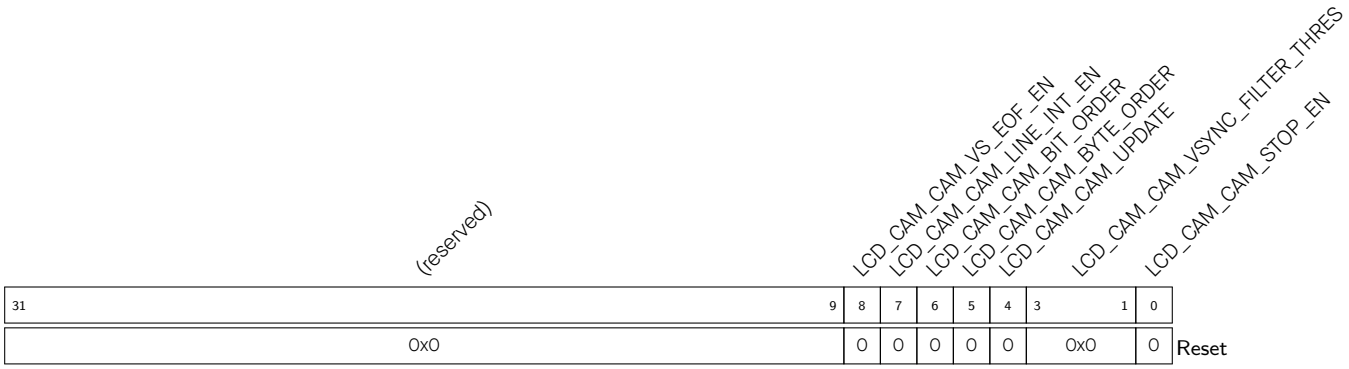
LCD_CAM_LCD_TRANS_BUFFER_ENA 配置是否启用 SRAM 作为数据缓冲器。

0：不启用

1：启用（需要不使能 YUV420 进行上采样色彩转换功能）

(R/W)

Register 20.14. LCD_CAM_CAM_CTRL_REG (0x0004)



LCD_CAM_CAM_STOP_EN 配置是否使能 Camera 停止信号。

0：Camera 不停止工作。

1：GDMA-AXI RX FIFO 存满后，Camera 停止工作。

(R/W)

LCD_CAM_CAM_VSYNC_FILTER_THRES 设置 CAM_VSYNC 信号的滤波阈值。(R/W)

LCD_CAM_CAM_UPDATE 配置是否更新 Camera 寄存器配置。

0：不更新

1：更新。将由硬件清除。

LCD_CAM_CAM_BYTE_ORDER 配置在 Camera 接收数据时是否反转数据字节顺序。

0：不反转

1：反转数据字节顺序。仅在 16 位模式下有效。

(R/W)

LCD_CAM_CAM_BIT_ORDER 配置在 Camera 接收数据时是否反转数据位顺序。

0：不反转

1：反转数据位顺序。在 n 位模式下，CAM_DATA_in[n-1:0] 反转为 CAM_DATA_in[0:n-1]。n = 8 或 16。

(R/W)

LCD_CAM_CAM_LINE_INT_EN 控制 LCD_CAM_CAM_HS_INT 中断原始值的生成。

0：不使能中断，即 LCD_CAM_CAM_HS_INT_RAW 无法生成。

1：使能中断，即 LCD_CAM_CAM_HS_INT_RAW 可以生成。

(R/W)

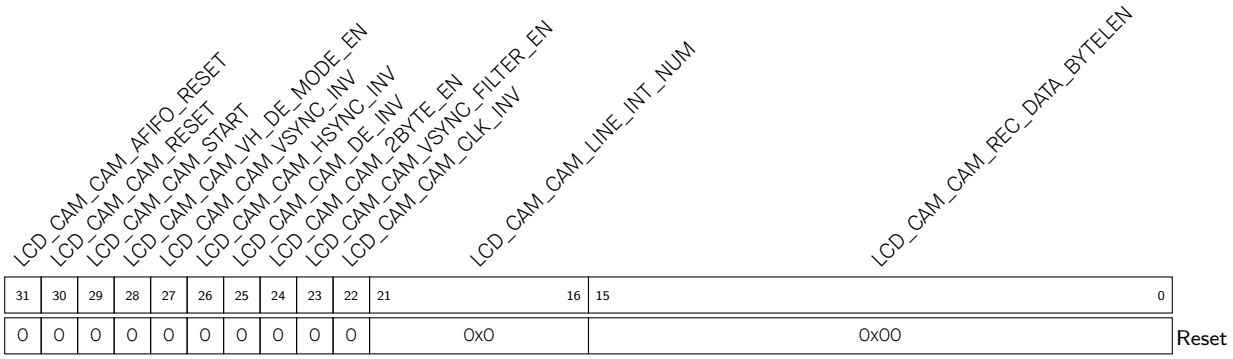
LCD_CAM_CAM_VS_EOF_EN 配置是否由 CAM_VSYNC 控制生成 in_suc_eof。

0：in_suc_eof 由 LCD_CAM_CAM_REC_DATA_BYTELEN 控制。

1：in_suc_eof 由 CAM_VSYNC 控制。

(R/W)

Register 20.15. LCD_CAM_CAM_CTRL1_REG (0x0008)



LCD_CAM_CAM_REC_DATA_BYTELEN 设置 Camera 触发 GDMA-AXI in_suc_eof_int 中断的接收数据字节长度。接收的数据长度 = 该字段的值 + 1。(R/W)

LCD_CAM_CAM_LINE_INT_NUM 设置触发 LCD_CAM_CAM_HS_INT 中断的视频行数。视频行数 = 该字段 + 1。(R/W)

LCD_CAM_CAM_CLK_INV 配置是否反转 CAM_PCLK 输入信号。

0：不反转

1：反转

(R/W)

LCD_CAM_CAM_VSYNC_FILTER_EN 配置是否使能 CAM_VSYNC 滤波功能。

0：不使用该功能

1：使能 CAM_VSYNC 滤波功能

(R/W)

LCD_CAM_CAM_2BYTE_EN 配置接收数据的位宽。

0：8 位

1：16 位

(R/W)

LCD_CAM_CAM_DE_INV 配置是否反转 CAM_DE 输入信号。

0：不反转

1：反转

(R/W)

LCD_CAM_CAM_HSYNC_INV 配置是否反转 CAM_HSYNC 输入信号。

0：不反转

1：反转

(R/W)

LCD_CAM_CAM_VSYNC_INV 配置是否反转 CAM_VSYNC 输入信号。

0：不反转

1：反转

(R/W)

见下页...

Register 20.15. LCD_CAM_CAM_CTRL1_REG (0x0008)

接上页...

LCD_CAM_CAM_VH_DE_MODE_EN 配置输入控制信号。

0：VSYNC、DE 线同时控制数据有效，HSYNC 线可以不接。但是在此模式下，Camera 的 YUV-RGB 转换功能将不可用。

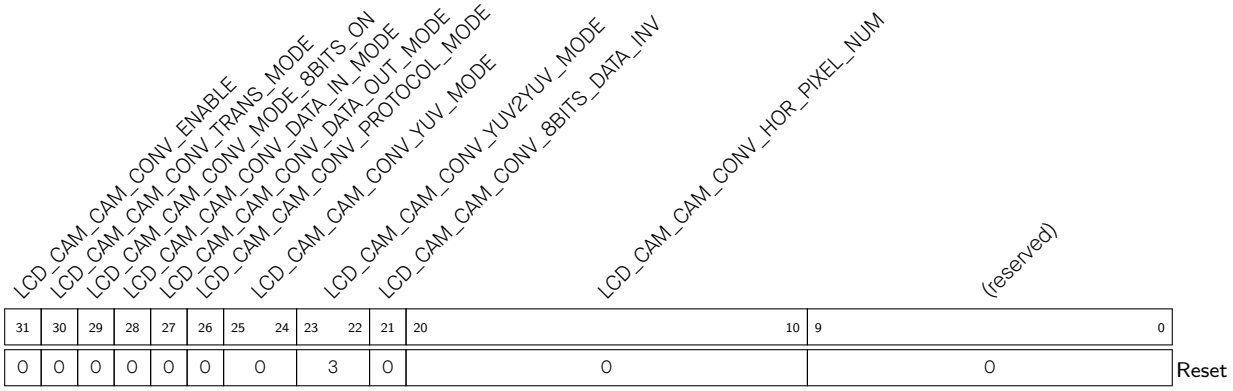
1：VSYNC、HSYNC、DE 线同时控制数据有效，因此用户需要接 VSYNC、HSYNC、DE 信号线。(R/W)

LCD_CAM_CAM_START Camera 模块使能信号。(R/W)

LCD_CAM_CAM_RESET 置位则复位 Camera 模块。(WO)

LCD_CAM_CAM_AFIFO_RESET Camera Async RX FIFO 复位信号。(WO)

Register 20.16. LCD_CAM_CAM_RGB_YUV_REG (0x000C)



LCD_CAM_CAM_CONV_HOR_PIXEL_NUM 配置 RGB-to-YUV420 模式下的图像水平像素个数。

LCD_CAM_CAM_CONV_8BITS_DATA_INV 配置是否交换两个字节的输入数据。

0：不交换

1：交换

(R/W)

LCD_CAM_CAM_CONV_YUV2YUV_MODE 配置 YUV-to-YUV 模式下的数据转换格式。

0：数据被转换为 YUV422 格式

1：数据被转换为 YUV420 格式

2：数据被转换为 YUV411 格式

3：不启用格式转换。

仅在 LCD_CAM_CAM_CONV_TRANS_MODE = 1 时有效。

(R/W)

LCD_CAM_CAM_CONV_YUV_MODE 配置在 RGB-to-YUV 模式、YUV-to-YUV 模式或 YUV-to-RGB 模式下，数据的 YUV 格式。RGB-to-YUV 模式下，指的是转换后的 YUV 格式；YUV-to-YUV 模式或 YUV-to-RGB 模式下，指的是 Camera 输入的 YUV 格式。

0：YUV422 格式

1：YUV420 格式（启用色彩空间转换时，Camera 输入不支持该模式）

2：YUV411 格式

(R/W)

LCD_CAM_CAM_CONV_PROTOCOL_MODE 配置格式转换的标准。

0：选择 BT601

1：选择 BT709

(R/W)

LCD_CAM_CAM_CONV_DATA_OUT_MODE 配置 YUV-RGB 格式转换模块输出数据的色彩空间。

0：有限色彩空间

1：全色彩空间

(R/W)

LCD_CAM_CAM_CONV_DATA_IN_MODE 配置 YUV-RGB 格式转换模块输入数据的色彩空间。

0：有限色彩空间

1：全色彩空间

(R/W)

见下页...

Register 20.16. LCD_CAM_CAM_RGB_YUV_REG (0x000C)

接上页...

LCD_CAM_CAM_CONV_MODE_8BITS_ON 配置输入数据位宽。

0：16 位模式

1：8 位模式

(R/W)

LCD_CAM_CAM_CONV_TRANS_MODE 配置格式转换模式。

0：转换成 RGB 格式

1：转换成 YUV 格式

(R/W)

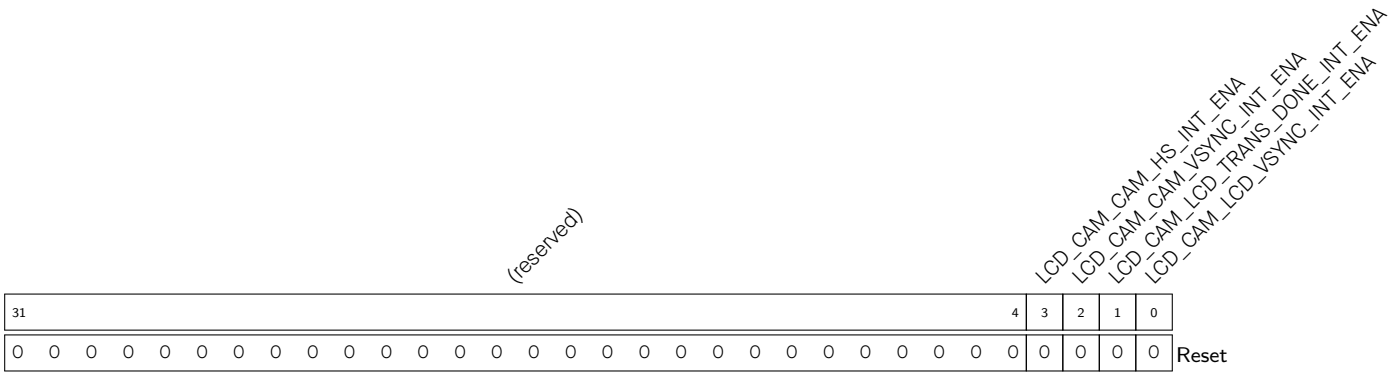
LCD_CAM_CAM_CONV_ENABLE 配置是否使能 YUV-RGB 格式转换模块。

0：旁路格式转换模块

1：使能格式转换模块

(R/W)

Register 20.17. LCD_CAM_LC_DMA_INT_ENA_REG (0x0064)



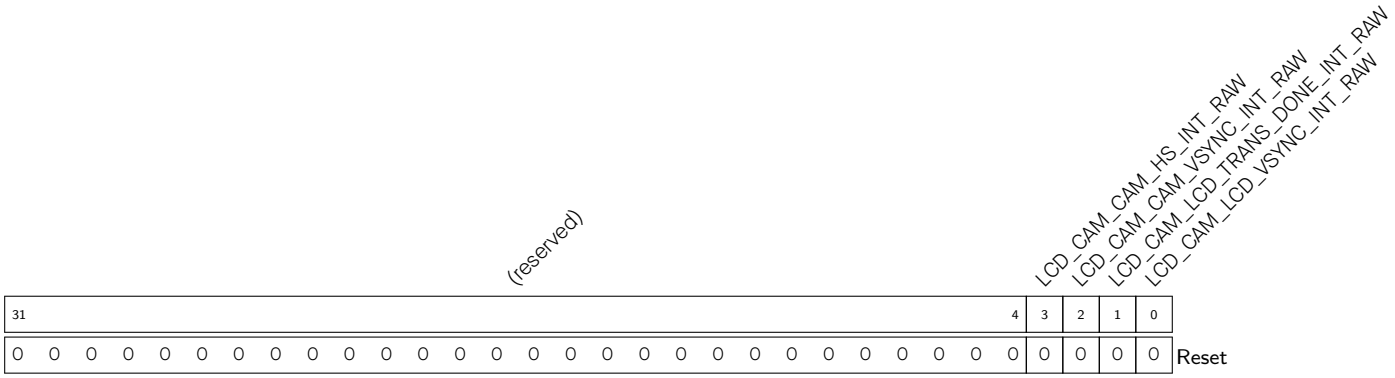
LCD_CAM_LCD_VSYNC_INT_ENA 写 1 使能 LCD_CAM_LCD_VSYNC_INT。(R/W)

LCD_CAM_LCD_TRANS_DONE_INT_ENA 写 1 使能 LCD_CAM_LCD_TRANS_DONE_INT。(R/W)

LCD_CAM_CAM_VSYNC_INT_ENA 写 1 使能 LCD_CAM_CAM_VSYNC_INT。(R/W)

LCD_CAM_CAM_HS_INT_ENA 写 1 使能 LCD_CAM_CAM_HS_INT。(R/W)

Register 20.18. LCD_CAM_LC_DMA_INT_RAW_REG (0x0068)



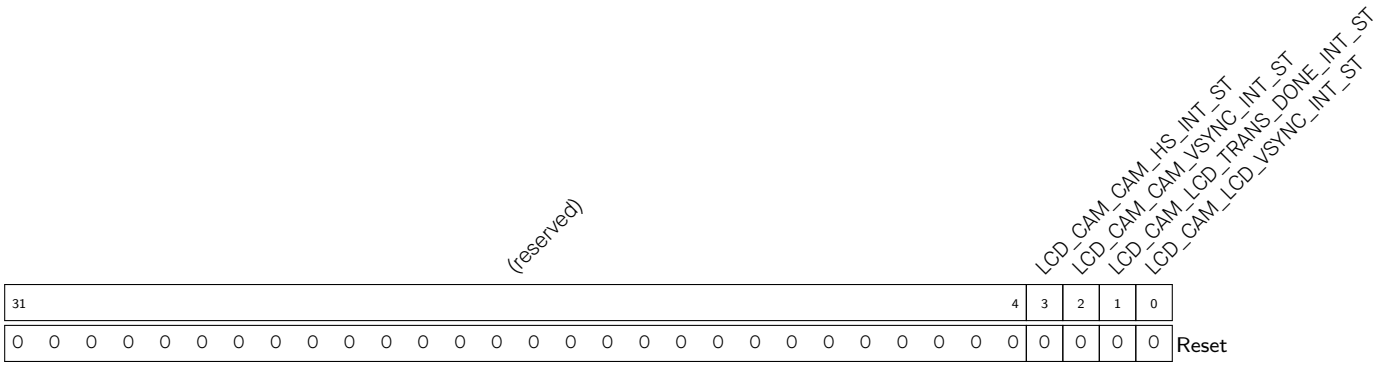
LCD_CAM_LCD_VSYNC_INT_RAW LCD_CAM_LCD_VSYNC_INT 的原始中断状态。(RO)

LCD_CAM_LCD_TRANS_DONE_INT_RAW LCD_CAM_LCD_TRANS_DONE_INT 的原始中断状态。(RO)

LCD_CAM_CAM_VSYNC_INT_RAW LCD_CAM_CAM_VSYNC_INT 的原始中断状态。(RO)

LCD_CAM_CAM_HS_INT_RAW LCD_CAM_CAM_HS_INT 的原始中断状态。(RO)

Register 20.19. LCD_CAM_LC_DMA_INT_ST_REG (0x006C)



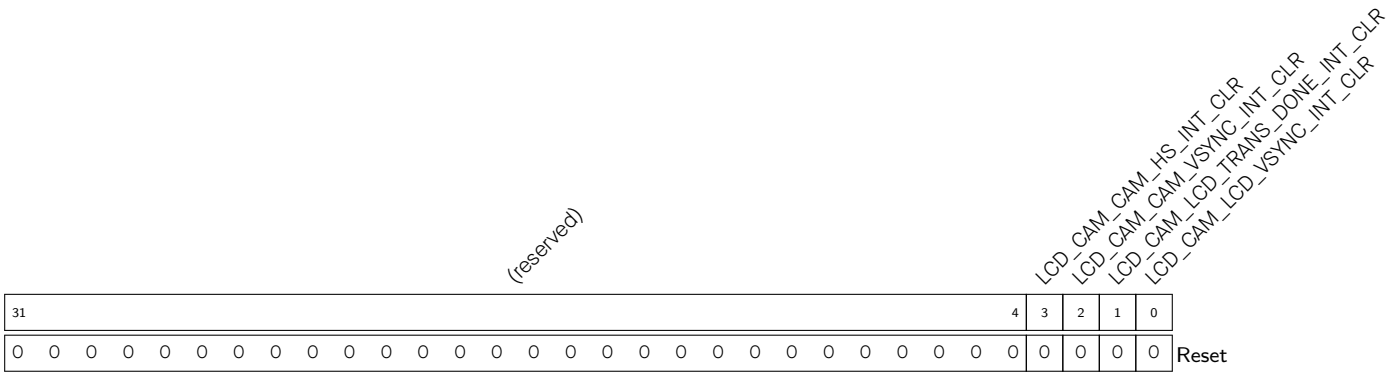
LCD_CAM_LCD_VSYNC_INT_ST LCD_CAM_LCD_VSYNC_INT 的屏蔽中断状态。(RO)

LCD_CAM_LCD_TRANS_DONE_INT_ST LCD_CAM_LCD_TRANS_DONE_INT 的屏蔽中断状态。(RO)

LCD_CAM_CAM_VSYNC_INT_ST LCD_CAM_CAM_VSYNC_INT 的屏蔽中断状态。(RO)

LCD_CAM_CAM_HS_INT_ST LCD_CAM_CAM_HS_INT 的屏蔽中断状态。(RO)

Register 20.20. LCD_CAM_LC_DMA_INT_CLR_REG (0x0070)



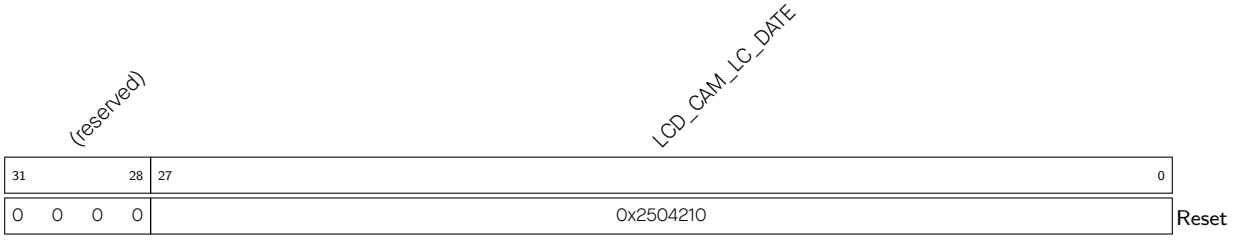
LCD_CAM_LCD_VSYNC_INT_CLR 写 1 清除 LCD_CAM_LCD_VSYNC_INT。(WO)

LCD_CAM_LCD_TRANS_DONE_INT_CLR 写 1 清除 LCD_CAM_LCD_TRANS_DONE_INT。(WO)

LCD_CAM_CAM_VSYNC_INT_CLR 写 1 清除 LCD_CAM_CAM_VSYNC_INT。(WO)

LCD_CAM_CAM_HS_INT_CLR 写 1 清除 LCD_CAM_CAM_HS_INT。(WO)

Register 20.21. LCD_CAM_LC_DATE_REG (0x00FC)



LCD_CAM_LC_DATE 版本控制寄存器。(R/W)

第 V 卷

通讯接口

该部分涉及系统的连接性，介绍各种带有通讯接口的模块，如 I2C、I2S、SPI、UART、USB 等，同时还包括其他用于遥控、电机控制、LED 控制等功能的信号生成接口。

第 21 章

UART 控制器 (UART)

21.1 概述

UART 是一种以字符为导向、用于实现设备间通信的异步传输接口。异步传输的意思是不需要在发送数据上添加时钟信息。这也要求发送端和接收端的速率、停止位、奇偶校验位等都要相同，通信才能成功。

一个典型的 UART 帧开始于一个起始位，紧接着是有效数据，然后是奇偶校验位（可有可无），最后是停止位。ESP32-S31 上的 UART 控制器支持多种字符长度和停止位。另外，控制器还支持软硬件流控和 GDMA，可以实现高速的数据传输。开发者可以使用多个 UART 端口，同时又能保证很少的软件开销。

ESP32-S31 芯片中共有五个 UART 控制器，包含四个正常功能的 UART 和一个满足低功耗需求的 LP UART。UART 控制器可以兼容不同的 UART 设备。另外，UART 还可以用于红外数据交换 (IrDA) 或 RS485 调制解调器。本文以 UART*n* 指代四个 UART 控制器，*n* 为 0、1、2、3。LP UART 是 UART 的功能剪裁版本，且单独控制一组寄存器，通过表 21.2-1 可以区分 UART 和 LP UART 的特性。

21.2 主要特性

UART 控制器的特性以及与 LP UART 的特性区分见下表：

表 21.2-1. UART 和 LP UART 特性区分

UART 特性	LP UART 特性
可编程收发波特率，最大为 5 MBaud	
一个 UART 的发送通道和接收通道分别有 128 x 8 bit RAM 存储空间	LP UART 的发送 FIFO 和接收 FIFO 共用 20 x 8 bit 存储空间
全双工异步通信	
数据位（5 到 8 位）	
停止位（1、1.5 或 2 位）	
奇偶校验位	
AT_CMD 特殊字符检测	
RS485 协议	—
IrDA 协议	—
GDMA 高速数据通信	—
接收超时	
UART 唤醒模式	
软件流控和硬件流控	

见下页

表 21.2-1 – 接上页

UART 特性	LP_UART 特性
三个可分频的时钟源： 1. XTAL_CLK 2. RC_FAST_CLK 3. PLL_F80M_CLK	三个可分频的时钟源： 1. RC_FAST_CLK 2. XTAL_DIV_CLK 3. PLL_F8M_CLK

下文主要针对正常功能的 UART 进行叙述。

21.3 UART 架构

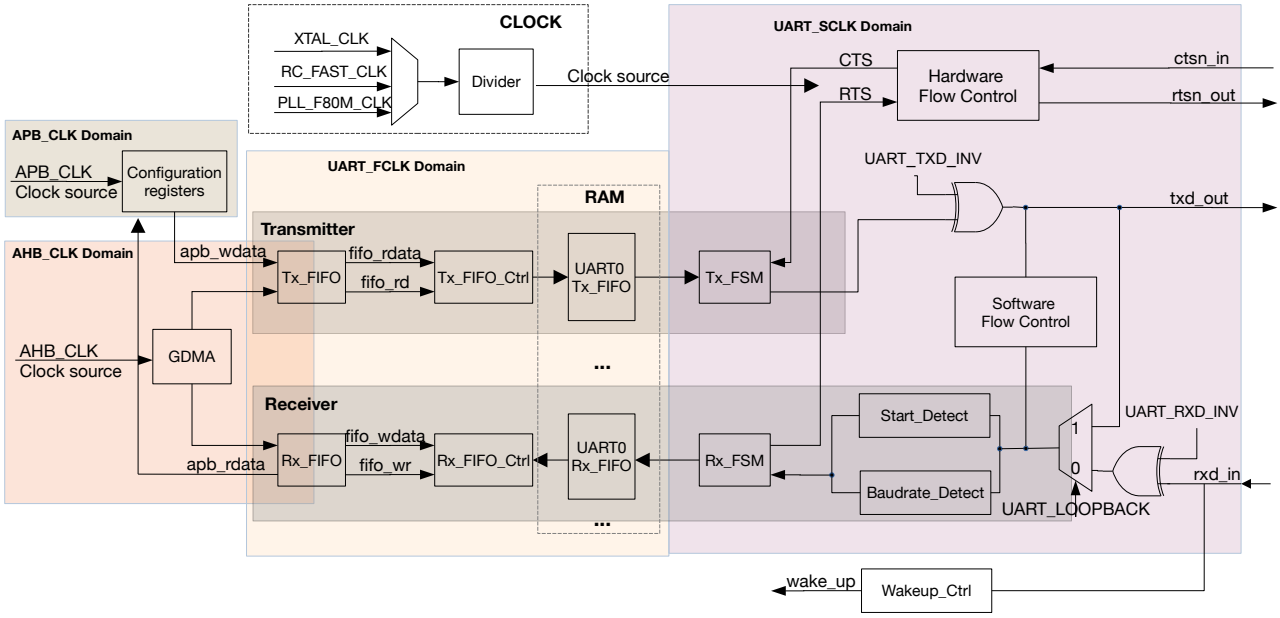


图 21.3-1. UART 基本架构图

图 21.3-1 为 UART 基本架构图。UART 模块工作在 4 个时钟域，分别为 APB_CLK、AHB_CLK、UART_SCLK、UART_FCLK。其中 APB_CLK 与 AHB_CLK 时钟关系为同步不同频（APB_CLK 由 AHB_CLK 分频得到），UART_SCLK 与 UART_FCLK 时钟关系为同步不同频（UART_SCLK 由 UART_FCLK 分频得到）。UART_FCLK 有三个时钟源：PLL_F80M_CLK、RC_FAST_CLK 以及晶振时钟 XTAL_CLK（详情请参考章节 7 复位和时钟 *[to be added later]*）。可以通过配置 PCR_UART*n*_SCLK_SEL 来选择 UART_FCLK 时钟源，之后使用分频器对时钟源进行分频，然后产生时钟信号 UART_SCLK。分频系数的整数部分由 PCR_UART*n*_SCLK_DIV_NUM 配置，小数部分的分母由 PCR_UART*n*_SCLK_DIV_A 配置，小数部分的分子由 PCR_UART*n*_SCLK_DIV_B 配置。支持的分频范围为 1 ~ 256。只有正常功能的 UART 包含此分频器，LP UART 不包含。

UART 控制器可以分为两个功能块：发送块和接收块。

发送块包含一个发送 FIFO 用于缓存待发送的数据。软件可以通过 APB 总线向 Tx_FIFO 写数据，也可以通过 GDMA 将数据搬入 Tx_FIFO。Tx_FIFO_Ctrl 用于控制 Tx_FIFO 的读写过程，当 Tx_FIFO 非空时，Tx_FSM 通过 Tx_FIFO_Ctrl 读取数据，并将数据按照配置的帧格式转化成比特流。比特流输出信号 txd_out 可以通过配置 UART_TXD_INV 寄存器实现取反功能。

接收块包含一个接收 FIFO 用于缓存待处理的数据。输入比特流 rxd_in 可以输入到 UART 控制器。可以通过

UART_RXD_INV 寄存器实现取反。Baudrate_Detect 通过检测最小比特流输入信号的脉宽来测量输入信号的波特率。Start_Detect 用于检测数据的起始位，当检测到起始位之后，Rx_FSM 通过 Rx_FIFO_Ctrl 将帧解析后的数据存入 Rx_FIFO 中。软件可以通过 APB 总线读取 Rx_FIFO 中的数据，也可以使用 GDMA 进行数据接收。

HW_Flow_Ctrl 通过标准 UART RTS 和 CTS（rtsn_out 和 ctsn_in）流控信号来控制 rxd_in 和 txd_out 的数据流。SW_Flow_Ctrl 通过在发送数据流中插入特殊字符以及在接收数据流中检测特殊字符来进行数据流的控制。当 UART 处于 Light-sleep 模式（详情请参考章节 11 *低功耗管理 [to be added later]*）时，可以通过四种不同的方式产生 wake_up 信号给 PMU 模块，由 PMU 来唤醒 ESP32-S31 芯片。具体唤醒方式请参考章节 21.4.8。

21.4 功能描述

21.4.1 时钟与复位

UART 为异步外设。其寄存器配置模块工作在 APB_CLK 时钟域，TX/RX FIFO 作为跨时钟域模块工作在 AHB_CLK 和 UART_FCLK 时钟域。UART RAM 控制单元工作在 UART_FCLK 时钟域，控制 UART 发送与接收的模块工作在 UART_SCLK 时钟域，即 UART Core 模块的时钟域。

在 UART_SCLK 时钟频率满足生成波特率的需求下，可通过预分频使 UART Core 模块工作在较小的时钟频率，从而减小 UART 外设的功耗。通常情况下，UART Core 模块时钟小于 APB_CLK 时钟，并且在满足 UART 波特率的情况下，UART Core 时钟分频系数可以配置到最大值。UART 也支持 UART Core 模块时钟大于 APB_CLK 时钟，此时，UART Core 模块时钟最大为 APB_CLK 的 3 倍。另外，UART Core 的 TX/RX 时钟可以被单独控制。置位 UART_TX_SCLK_EN 使能 UART Core 的 TX 时钟；置位 UART_RX_SCLK_EN 使能 UART Core 的 RX 时钟。

为确保配置寄存器的值成功从 APB_CLK 时钟域同步到 UART_SCLK 时钟域，寄存器配置需要遵循一定的流程，详情请参考章节 21.6。

对整个 UART 的复位，需要遵循如下配置流程：

- 将 PCR_UART*n*_CLK_EN 置 1 打开 UART*n* Core 时钟。
- 向 PCR_UART*n*_RST_EN 位写 1。
- 将 PCR_UART*n*_RST_EN 位清 0。

21.4.2 UART FIFO

UART 控制器的发送块和接收块分别使用一块 128 x 8 位 RAM 存储空间，并且分别通过一个 4 x 8 位的异步 FIFO 接口访问对应的 RAM。发送块和接收块的存储空间和异步 FIFO 接口相互独立，不可复用。

UART 的 Tx_FIFO 可以通过置位 UART_TXFIFO_RST 来复位，Rx_FIFO 可以通过置位 UART_RXFIFO_RST 来复位。

对于 TX FIFO，可以通过 APB 总线或 GDMA 向其写入数据，硬件 Tx_FSM 自动从其中读取数据，数据将按照配置的帧格式转换成比特流；对于 RX FIFO，可以通过 APB 总线或 GDMA 读取其中的数据，并存储到内存，硬件 Rx_FSM 将接收到的比特流转换成字节并写入 RX FIFO。UART 控制器共享同一个 GDMA 通道。

配置 UART_TXFIFO_EMPTY_THRHD 可以设置 Tx_FIFO 空信号阈值，当存储在 Tx_FIFO 中的数据量小于 UART_TXFIFO_EMPTY_THRHD 时会产生中断 UART_TXFIFO_EMPTY_INT；配置 UART_RXFIFO_FULL_THRHD 可以设置 Rx_FIFO 满信号阈值，当储存在 Rx_FIFO 中的数据量大于或等于 UART_RXFIFO_FULL_THRHD 会产生中断 UART_RXFIFO_FULL_INT。另外，当 Rx_FIFO 中储存的数据量超过其能存储的最大值时，会产生 UART_RXFIFO_OVF_INT 中断。

UART*n* 可以通过寄存器 UART_FIFO_REG 访问 FIFO。您可以写 UART_RXFIFO_RD_BYTE 将数据存入 TX FIFO，也可以读 UART_RXFIFO_RD_BYTE 获取 RX FIFO 中的数据。

21.4.3 波特率产生与检测

21.4.3.1 波特率产生

在 UART 发送或接收数据之前，需要配置寄存器来设置波特率。波特率发生器主要通过对输入时钟源的分频来实现，支持小数分频。UART_CLKDIV_SYNC_REG 将分频系数分成两个部分：UART_CLKDIV 用于配置整数部分，UART_CLKDIV_FRAG 用于配置小数部分。在输入时钟为 80 MHz 的情况下，UART 能支持的最大波特率为 5 MBaud。

波特率分频器系数为

$$UART_CLKDIV + \frac{UART_CLKDIV_FRAG}{16}$$

也就是说，最终波特率为

$$\frac{INPUT_FREQ}{UART_CLKDIV + \frac{UART_CLKDIV_FRAG}{16}}$$

其中，INPUT_FREQ 为 UART Core 时钟。例如，若 UART_CLKDIV = 694，UART_CLKDIV_FRAG = 7，则分频系数为

$$694 + \frac{7}{16} = 694.4375$$

UART_CLKDIV_FRAG 为 0 时，分频器为整数分频，每 UART_CLKDIV 个输入脉冲都会产生一个输出脉冲。

UART_CLKDIV_FRAG 不为 0 时，分频器为小数分频，输出波特率脉冲不完全统一。如图 21.4-1 所示，每 16 个输出脉冲，波特率发生器分频 (UART_CLKDIV + 1) 个输入脉冲或 UART_CLKDIV 个输入脉冲。分频 (UART_CLKDIV + 1) 个输入脉冲产生 UART_CLKDIV_FRAG 个输出脉冲，分频 UART_CLKDIV 个输入脉冲产生剩余的 (16 - UART_CLKDIV_FRAG) 个输出脉冲。

如图 21.4-1 所示，输出脉冲相互交错，使得输出时序更加统一。

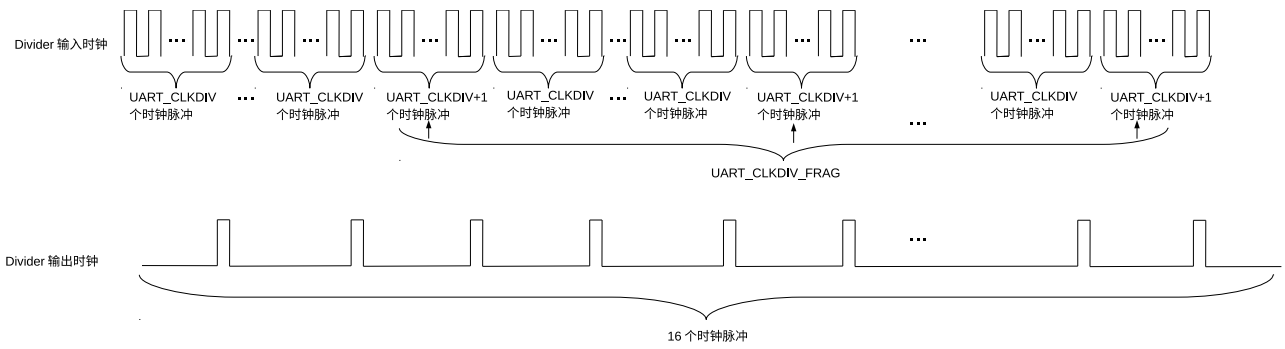


图 21.4-1. UART 控制器分频

为了支持 IrDA（详情见章节 21.4.7），IrDA 小数分频器会产生 16 × UART_CLKDIV_SYNC_REG 分频的时钟用于 IrDA 数据传输。产生 IrDA 数据传输时钟的小数分频器原理与上述小数分频器一样，取UART_CLKDIV/16 作为分频值的整数部分，取 UART_CLKDIV 的低 4 比特作为小数部分。

21.4.3.2 波特率检测

置位 UART_AUTOBAUD_EN 可以开启 UART 波特率自检测功能。图 21.4-2 中的 Baudrate_Detect 可以滤除信号脉宽小于 UART_GLITCH_FILT 的噪声。

在 UART 双方进行通信之前，可以通过发送随机数据，让具有波特率检测功能的数据接收方进行波特率分析。UART_LOWPULSE_MIN_CNT 存储了最小低电平脉冲宽度，UART_HIGHPULSE_MIN_CNT 存储了最小高电平脉冲宽度，UART_POSEDGE_MIN_CNT 存储了两个上升沿之间的最小脉冲宽度，UART_NEGEDGE_MIN_CNT 存储了两个下降沿之间最小的脉冲宽度。软件可以通过读取这四个寄存器获取发送方的波特率。

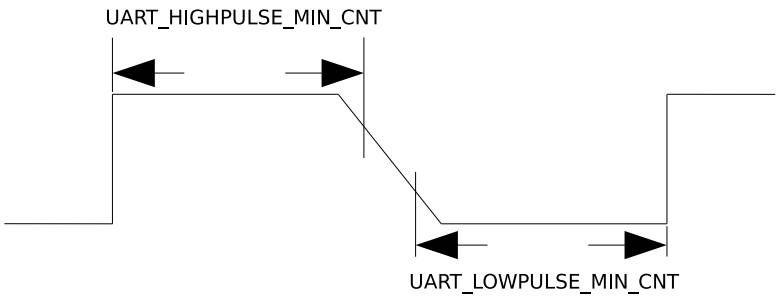


图 21.4-2. UART 信号下降沿较差时序图

波特率的计算分为三种情况：

1. 正常情况下，为防止因亚稳态在上升沿或下降沿附近采样数据错误而导致 UART_LOWPULSE_MIN_CNT 或者 UART_HIGHPULSE_MIN_CNT 不准确，单比特脉冲宽度可以通过将这两个值相加取平均消除误差。计算公式如下：

$$B_{\text{uart}} = \frac{f_{\text{clk}}}{(\text{UART_LOWPULSE_MIN_CNT} + \text{UART_HIGHPULSE_MIN_CNT} + 2)/2}$$

2. 对于 UART 信号的下降沿信号比较差的情况，如图21.4-2 所示，这时通过取 UART_LOWPULSE_MIN_CNT 与 UART_HIGHPULSE_MIN_CNT 的和平均得到的值不准确，可以通过 UART_POSEDGE_MIN_CNT 获取发送方波特率。计算公式如下：

$$B_{\text{uart}} = \frac{f_{\text{clk}}}{(\text{UART_POSEDGE_MIN_CNT} + 1)/2}$$

3. 对于 UART 信号的上升沿信号比较差的情况，可以通过 UART_NEGEDGE_MIN_CNT 获取发送方波特率。计算公式如下：

$$B_{\text{uart}} = \frac{f_{\text{clk}}}{(\text{UART_NEGEDGE_MIN_CNT} + 1)/2}$$

21.4.4 UART 数据帧

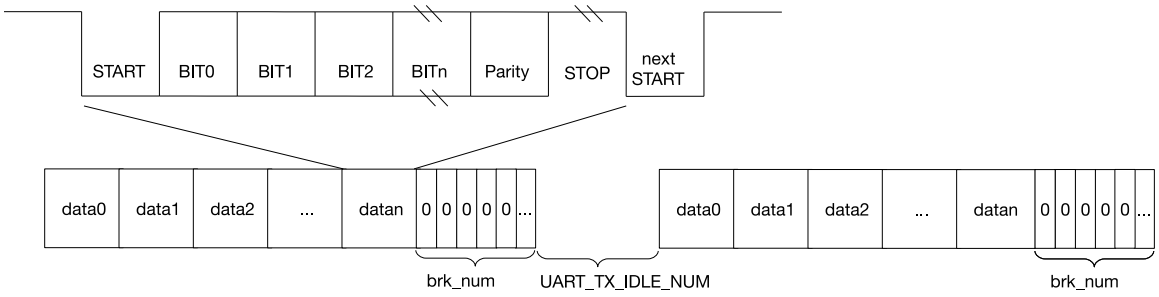


图 21.4-3. UART 数据帧结构

图 21.4-3 所示为基本数据帧格式，数据帧从起始位开始，以停止位结束。起始位占用 1 位，停止位可以通过配置 UART_STOP_BIT_NUM 实现 1、1.5、2 位宽（RS485 模式下可增加转换延时，详见章节 21.4.6.2）。起始位为

低电平，停止位为高电平。

数据位宽 (BITO ~ BITn) 为 5 ~ 8 位，可以通过 UART_BIT_NUM 进行配置。当置位 UART_PARITY_EN 时，数据帧会在数据之后添加一位奇偶校验位。UART_PARITY 用于选择奇校验或是偶校验。当接收器检测到输入数据的校验位错误时会产生 UART_PARITY_ERR_INT 中断，输入数据仍会存入 Rx_FIFO。当接收器检测到数据数据帧格式错误时会产生 UART_FRM_ERR_INT 中断，默认情况下，输入数据会被存入 Rx_FIFO。

Tx_FIFO 中数据都发送完成后会产生 UART_TX_DONE_INT 中断。置位 UART_TXD_BRK 时，Tx_FIFO 中数据发送完成后，发送端会进入终止状态 (break condition)，继续发送几个连续的特殊数据帧 NULL。在 NULL 数据帧发送完成后，TX 数据线输出为低电平。NULL 数据帧的数量可由 UART_TX_BRK_NUM 进行配置。发送器发送完所有的 NULL 数据帧之后会产生 UART_TX_BRK_DONE_INT 中断。数据帧之间可以通过配置 UART_TX_IDLE_NUM 保持最小间隔时间。当一帧数据之后的空闲时间大于等于 UART_TX_IDLE_NUM 寄存器的配置值时则产生 UART_TX_BRK_IDLE_DONE_INT 中断。

在传输一个 NULL 数据帧所需的时间内，接收端 RX 数据线若一直检测到低电平，接收端会检测为终止状态，并触发 UART_BRK_DET_INT 中断表示终止状态已结束。

接收端通过 UART_RXFIFO_TOUT_INT 中断检测总线状态。接收端接收到至少一个字节数据后，总线处于空闲状态超过 UART_RX_TOUT_THRHD 位时间时，触发 UART_RXFIFO_TOUT_INT 中断。您可用此中断检测发送端是否已经发送所有数据。

21.4.5 AT_CMD 字符格式

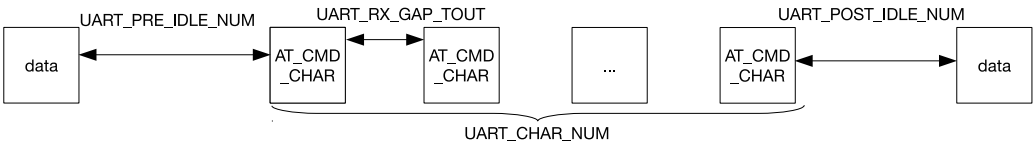


图 21.4-4. AT_CMD 字符格式

图 21.4-4 为一种特殊的 AT_CMD 字符格式。当接收器连续收到 AT_CMD_CHAR 字符且字符之间满足如下条件时将会产生 UART_AT_CMD_CHAR_DET_INT 中断。

- 接收到的第一个 AT_CMD_CHAR 与上一个非 AT_CMD_CHAR 之间间隔至少 UART_PRE_IDLE_NUM 个波特率周期。
- AT_CMD_CHAR 字符之间间隔小于 UART_RX_GAP_TOUT 个波特率周期。
- 接收的 AT_CMD_CHAR 字符个数必须大于等于 UART_CHAR_NUM。
- 接收到的最后一个 AT_CMD_CHAR 字符与下一个非 AT_CMD_CHAR 之间间隔至少 UART_POST_IDLE_NUM 个波特率周期。

注意：由于 AT_CMD_CHAR 字符之间间隔小于 UART_RX_GAP_TOUT 个波特率周期的限制，建议用户使用该功能时，配置的 APB 时钟频率不要低于 8 MHz。

21.4.6 RS485

UART 控制器支持 RS485 通讯模式，RS485 因使用差分信号传输数据，相比于 RS232 具有更远的传输距离及更高的传输速率。RS485 有两线半双工及四线全双工两种选择，UART 模块采用两线半双工模式，并支持侦听总线的功能。

21.4.6.1 驱动控制

如图 21.4-5 所示，RS485 两线 multidrop 系统中，需要一个外部 RS485 收发器实现单端信号与差分信号的转换。RS485 收发器包括一个驱动器与一个接收器。当 UART 不作为发送器时，通过关闭驱动器来断开与差分传输线的连接。DE 为 1 时，使能驱动器；DE 为 0 关闭驱动器。

UART 接收端通过外部接收器将差分信号转为单端信号。RE 作为接收器的使能控制信号，RE 为 0，使能接收器；RE 为 1，关闭接收器。如果 RE 被配置为 0，从而允许 UART 保持侦听总线上的数据，包括 UART 发送的数据。

DE 信号的控制分为软件控制和硬件控制两种方法。为减少软件的开销，DE 信号采用硬件来控制。图 21.4-5 所示，DE 与 UART 的 dtrn_out 相连（详见 21.4.9.1 小节）。

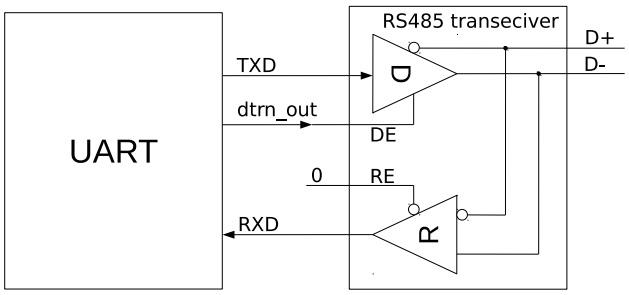


图 21.4-5. RS485 模式驱动控制结构图

21.4.6.2 转换延时

默认情况下，UART 处于接收状态。当从发送转为接收状态时，为保证发送数据被稳定接收，RS485 协议要求在发送停止位之后增加一个波特率的转换延时。UART 发送模块支持在起始位之前或在停止位之后增加一个波特率的延时。置位 UART_DLO_EN，在起始位之前增加一个波特率周期延时；置位 UART_DL1_EN，在停止位之后增加一个波特率周期延时。

21.4.6.3 总线侦听

RS485 两线 multidrop 系统中，当外部 RS485 收发器的 RE 被配置为 0 时，UART 支持侦听总线。默认情况下，不允许 UART 在发送数据时接收数据。置位 UART_RS485TX_RX_EN，允许在发送数据时接收数据，配合图 21.4-5 中外部 RS485 收发器的配置，UART 保持侦听传输总线。另外，默认情况下，不允许 UART 在接收数据时发送数据。置位 UART_RS485RXBY_TX_EN，允许在接收数据时发送数据。

UART 支持侦听 UART 发送的数据。UART 处于发送状态下，当侦听到 UART 发送的数据与 UART 接收的数据冲突时，触发 UART_RS485_CLASH_INT 中断；侦听到发送的数据帧错误时，触发 UART_RS485_FRM_ERR_INT 中断；侦听到发送数据校验位错误时，触发 UART_RS485_PARITY_ERR_INT 中断。

21.4.7 IrDA

IrDA 数据协议由物理层，链路接入层和链路管理层三个基本层协议组成。UART 实现了其物理层协议。在 IrDA 编码模式下，支持最大信号速率到 115.2 Kbit/s，即 SIR 模式。如图 21.4-6 所示，IrDA 编码器将来自 UART 的非归零编码 (NRZ) 信号采用反向归零编码 (RZI) 并输出给外部驱动和红外 LED，用 3/16 Bit Time 的脉宽调制信号表示逻辑 “0”，用低电平表示逻辑 “1”。IrDA 解码器接收来自红外接收器的信号并输出为 UART 的 NRZ 编码。一般情况下，接收端信号空闲时为高电平，编码器输出校验位与解码器输入校验位相反。当检测到低脉冲表示接收到开始信号。

IrDA 使能时，一个位被划分为 16 个时钟周期，在其第 9、10、11 个时钟周期中，当需要发送的比特为 0 时，IrDA 输出为高。

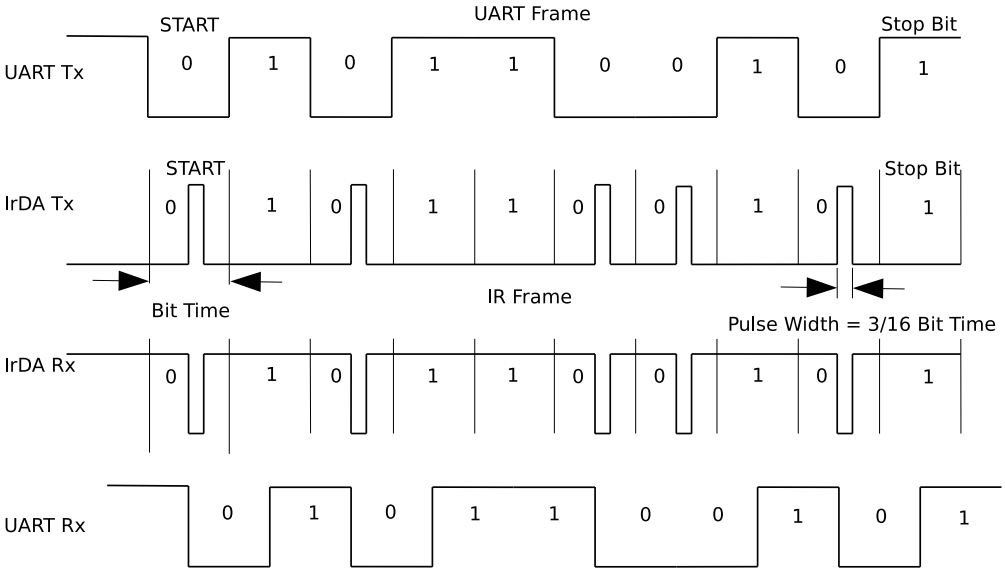


图 21.4-6. SIR 模式编解码时序图

IrDA 为半双工传输协议，不允许同时进行收发。如图21.4-7 所示，置位 UART_IRDA_EN 使能 IrDA 功能。置位 UART_IRDA_TX_EN（置 1）使能 IrDA 发送数据，这时不允许 IrDA 接收数据；复位 UART_IRDA_TX_EN（清 0）使能 IrDA 接收数据，这时不允许 IrDA 发送数据。

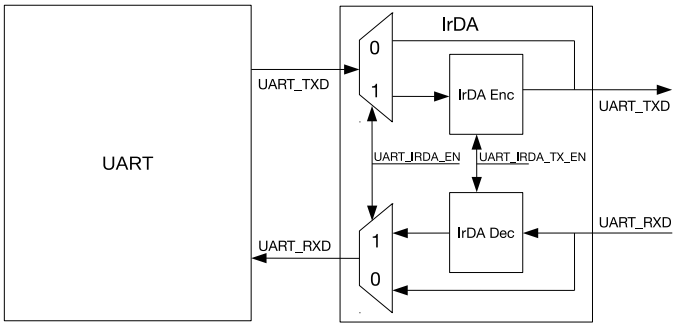


图 21.4-7. IrDA 编解码结构图

21.4.8 唤醒

UART 支持唤醒功能。当 UART 处于 Light-sleep 模式时，可以通过四种不同的方式产生 wake_up 信号给 PMU 模块，由 PMU 来唤醒芯片。

- UART_WK_MODE_SEL=0，当所有的时钟都关闭时，此时可以通过使 RXD 翻转若干周期，当上升沿个数大于等于 UART_ACTIVE_THRESHOLD +3 时唤醒芯片。
- UART_WK_MODE_SEL=1，由于 UART Core 时钟保持工作，因此 UART RX 仍然可以接收数据并将数据暂存在 Rx FIFO 中。当 Rx FIFO 中的数据量超过 UART_RX_WAKE_UP_THRHD 时，可以将芯片从 Light-sleep 中唤醒。
- UART_WK_MODE_SEL=2，当 UART RX 监测到起始位后，唤醒芯片。

- UART_WK_MODE_SEL=3，当 UART RX 接收到特定字符序列后，唤醒芯片。用户通过配置 UART_WK_CHAR0、UART_WK_CHAR1、UART_WK_CHAR2、UART_WK_CHAR3、UART_WK_CHAR4 自定义唤醒字符。UART 监测到由 CHAR0/CHAR1/CHAR2/CHAR3/CHAR4 组成的字符序列流之后唤醒芯片。可以通过配置 UART_CHAR_NUM 和 UART_WK_CHAR_MASK 来设置不同的字符序列。如下表所示，对于最后一个配置，UART 将依次检测 CHAR0 ~ CHAR4。

表 21.4-1. UART_CHAR_WAKEUP 模式配置

UART_CHAR_NAME	UART_WP_CHAR_MASK	检测字符序列
1	0xF	CHAR4
2	0x7	CHAR3/CHAR4
3	0x3	CHAR2/CHAR3/CHAR4
4	0x1	CHAR1/CHAR2/CHAR3/CHAR4
5	0x0	CHAR0/CHAR1/CHAR2/CHAR3/CHAR4

使用 UART 唤醒之后，需要通过在 Active 模式下向 UART 传输数据或是复位整个 UART 模块清除 wake_up 信号，否则下一次唤醒所需的上升沿个数将减少。

21.4.9 流控

UART 控制器有两种数据流控方式：硬件流控和软件流控。硬件流控主要通过输出信号 rtsn_out 以及输入信号 ctsn_in 进行数据流控制。软件流控主要通过在发送数据流中插入特殊字符以及在接收数据流中检测特殊字符来实现数据流控功能。

21.4.9.1 硬件流控

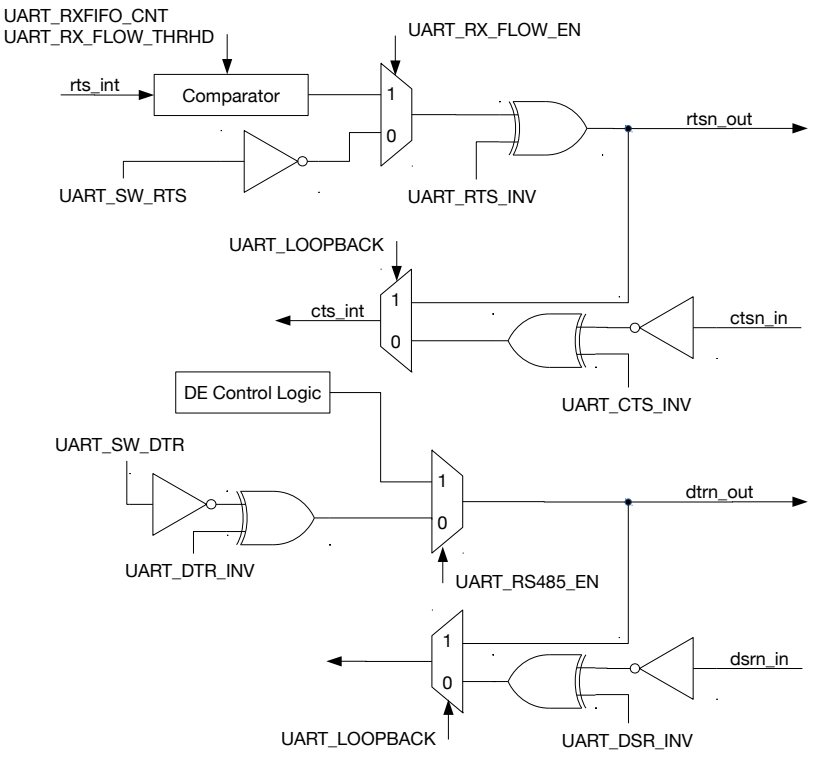


图 21.4-8. 硬件流控图

图 21.4-8 为 UART 硬件流控图。硬件流控的控制信号为输出信号 rtsn_out 及输入信号 ctsn_in。图 21.4-9 为两个 UART 之间硬件流控信号连接图。记 ESP32-S31 UART 为 IUO，External UART 为 EUO，下文将使用这两个标记来区分两个 UART。输出信号 rtsn_out (IUO) 为低电平表示允许对方 (EUO) 发送数据，rtsn_out (IUO) 为高电平表示通知对方 (EUO) 中止数据发送直到 rtsn_out (IUO) 恢复低电平。rtsn_out 输出信号的控制有两种方式。

- 软件控制：将 UART_RX_FLOW_EN 置 0 进入该模式。该模式下通过软件配置 UART_SW_RTS 改变 rtsn_out 的电平。
- 硬件控制：将 UART_RX_FLOW_EN 置 1 进入该模式。该模式下硬件会当 Rx_FIFO 中的数据大于 UART_RX_FLOW_THRHD 时拉高 rtsn_out 的电平。

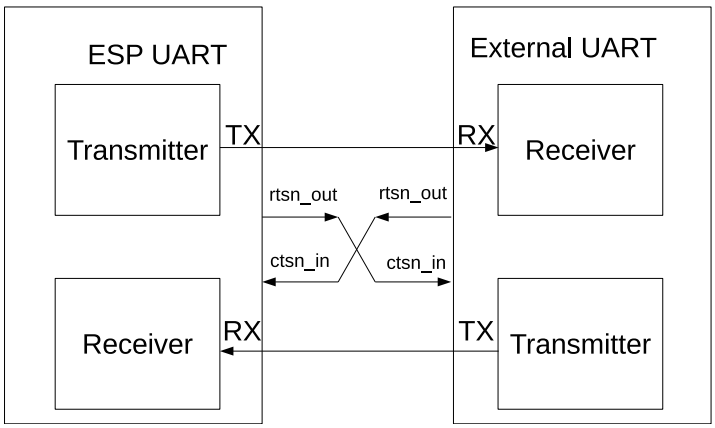


图 21.4-9. 硬件流控信号连接图

输入信号 ctsn_in (IUO) 为低电平表示允许发送端 (IUO) 发送数据；ctsn_in (IUO) 为高电平表示禁止发送端 (IUO) 发送数据。当 UART 检测到输入信号 ctsn_in (IUO) 的沿变化时会产生 UART_CTS_CHG_INT 中断。

UART 发送设备 (IUO) 输出信号 dtrn_out 为高电平表示发送数据已经准备完毕，处于可用状态。dtrn_out 通过配置寄存器 UART_SW_DTR 产生。UART 接收设备 (IUO) 在检测到输入信号 dsrn_in 的沿变化时会产生 UART_DSR_CHG_INT 中断。软件在检测到中断后，通过读取 UART_DSRN 可以获取 dsrn_in 的输入信号电平，UART_DSRN 为高电平时，表示对方设备 (EUO) 处于可用状态。

对于 RS485 两线 multidrop 系统，使用 dtrn_out 来收发转换。置位 UART_RS485_EN 使能 RS485 功能，dtrn_out 由硬件产生。数据开始发送时，dtrn_out 拉高，使能外部驱动器；数据最后一位发送完成后，dtrn_out 拉低，关闭外部驱动器。注意，当使能停止位之后增加一个波特率延时时，dtrn_out 会在延时结束后才拉低。

置位 UART_LOOPBACK 即开启 UART 的回环测试功能。此时 UART 的输出信号 txd_out 和其输入信号 rxd_in 相连，rtsn_out 和 ctsn_in 相连，dtrn_out 和 dsrn_out 相连。当接收的数据与发送的数据相同时表明 UART 能够正常发送和接收数据。

21.4.9.2 软件流控

软件流控不使用硬件的 CTS/RTS 控制线，而是在发送数据流中嵌入 XON/XOFF 字符来通知对方是否可以使用数据发送来实现流控。将 UART_SW_FLOW_CON_EN 置 1 使能软件流控。

在使用软件流控后，硬件会自动检测接收数据流中是否有 XON/XOFF 字符，在检测到相应的字符后会产生 UART_SW_XOFF_INT 或 UART_SW_XON_INT 中断。在检测到接收数据流中有 XOFF 字符后，发送器将会在发送完当前数据后停止发送；在检测到接收数据流中有 XON 字符后，将会使能发送器发送数据。另外，软件可以

通过置位 UART_FORCE_XOFF 来强制发送器停止发送数据，发送器会在发送完当前字节后停止发送；也可以通过置位 UART_FORCE_XON 来使能发送器发送数据。

软件可以根据 Rx_FIFO 中剩余空间大小决定流控字符的发送。置位 UART_SEND_XOFF，发送器会在发送完当前数据之后插入一个 XOFF 字符，该字符通过寄存器 UART_XOFF_CHAR 配置；置位 UART_SEND_XON，发送器会在发送完当前数据之后插入一个 XON 字符，该字符通过寄存器 UART_XON_CHAR 配置。另外，当 UART 接收 FIFO 中的数据量超过 UART_XOFF_THRESHOLD 时，硬件会置位 UART_SEND_XOFF，UART 发送器会在发送完当前数据之后插入一个 XOFF 字符，该字符通过寄存器 UART_XOFF_CHAR 配置。当 UART 接收 FIFO 中的数据量小于 UART_XON_THRESHOLD 时，硬件会置位 UART_SEND_XON，UART 发送器会在发送完当前数据之后插入一个 XON 字符，该字符通过寄存器 UART_XON_CHAR 配置。

当 UART 工作在全双工模式时，如果 UART RX 接收到 XOFF，此时尽管 UART RX 接受的数据超过阈值，UART TX 将被禁止发送包括 XOFF 在内的任何数据。为了避免该种情况导致的软流控死锁或者 overflow，可以使能 UART_XON_XOFF_STILL_SEND，此时允许 UART TX 在被禁止发送时，仍然可以发送一个 XOFF 字符。

21.4.10 GDMA 模式

ESP32-S31 中的 UART 接口通过通用主机控制器接口 (UHCI) 共用 1 组 GDMA TX/RX 通道。在 GDMA 模式下，支持对 HCI 协议数据包的解析 (decoder) 及数据包封装 (encoder)。UHCI_UART_SEL 字段用于选择哪个串口占用 GDMA 通道。

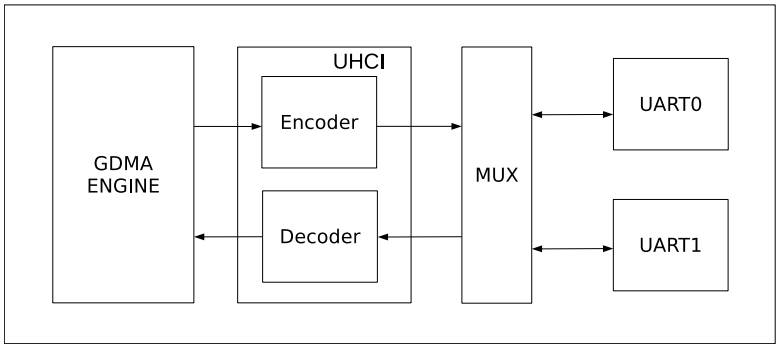


图 21.4-10. GDMA 模式数据传输

图 21.4-10 为 GDMA 方式数据传输图。在 GDMA Rx 通道接收数据前，软件将接收链表准备好。AHB_DMA_INLINK_ADDR_CH*n* 用于指向第一个接收链表描述符。置位 AHB_DMA_INLINK_START_CH*n* 之后，通用主机控制器接口 (UHCI) 会将 UART 接收到的数据传送给 decoder。经过 decoder 解析之后的数据在 GDMA 通道的控制下存入接收链表指定的 RAM 空间。

在 GDMA Tx 通道发送数据前，软件需要将发送链表和发送数据准备好，AHB_DMA_OUTLINK_ADDR_CH*n* 用于指向第一个发送链表描述符。置位 AHB_DMA_OUTLINK_START_CH*n* 之后，GDMA 引擎即从链表中指定的 RAM 地址读取数据，并通过 encoder 进行数据包封装，然后经 UART 的发送模块串行发送出去。

HCI 的数据包格式为（分隔符 + 数据 + 分隔符）。encoder 用于在数据前后加上分隔符，并将数据中和分隔符一样的数据替换为特殊字符。decoder 用于去除数据包前后分隔符，并将数据中的特殊字符替换为分隔符。数据前后的分隔符可以有连续多个。分隔符可由 UHCI_SEPER_CHAR 进行配置，默认值为 0xC0。数据中与分隔符一样的数据可以用 UHCI_ESC_SEQ0_CHAR0（默认为 0xDB）和 UHCI_ESC_SEQ0_CHAR1（默认为 0xDD）进行替换。当数据全部发送完成后，会产生 GDMA_OUT_TOTAL_EOF_CH*n*_INT 中断。当数据接收完成后，会产生 GDMA_IN_SUC_EOF_CH*n*_INT 中断。

21.5 中断

ESP32-S31 的 UART*n* 和 UHCI 可以生成以下中断信号并发送给中断矩阵。

- UART*n*_INTR
- UHCI_INTR

这些中断信号分别由 UART*n* 和 UHCI 的内部中断源生成。

UART*n* 中断源如下所列：

- UART_AT_CMD_CHAR_DET_INT：当接收器检测到 AT_CMD 字符时触发此中断。
- UART_RS485_CLASH_INT：在 RS485 模式下检测到发送器和接收器之间的冲突时触发此中断。
- UART_RS485_FRM_ERR_INT：在 RS485 模式下检测到发送块发送的数据帧错误时触发此中断。
- UART_RS485_PARITY_ERR_INT：在 RS485 模式下检测到发送块发送的数据校验位错误时触发此中断。
- UART_TX_DONE_INT：当发送器发送完 FIFO 中的所有数据时触发此中断。
- UART_TX_BRK_IDLE_DONE_INT：当发送器在最后一个数据发送后保持了最短的间隔时间时触发此中断。
- UART_TX_BRK_DONE_INT：当发送 FIFO 中的数据发送完之后发送器完成了发送 NULL 则触发此中断。
- UART_GLITCH_DET_INT：当接收器在起始位的中点处检测到毛刺时触发此中断。
- UART_SW_XOFF_INT：UART_SW_FLOW_CON_EN 置位时，当接收器接收到 XOFF 字符时触发此中断。
- UART_SW_XON_INT：UART_SW_FLOW_CON_EN 置位时，当接收器接收到 XON 字符时触发此中断。
- UART_RXFIFO_TOUT_INT：当接收器至少已经接收一个字节的数据，且之后在 UART_RX_TOUT_THRHD 位的时间内，总线一直处于 idle 状态将触发此中断。
- UART_BRK_DET_INT：当接收器在停止位之后检测到一个 NULL（即传输一个 NULL 的时间内保持逻辑低电平）时触发此中断。
- UART_CTS_CHG_INT：当接收器检测到 CTSn 信号的沿变化时触发此中断。
- UART_DSR_CHG_INT：当接收器检测到 DSRn 信号的沿变化时触发此中断。
- UART_RXFIFO_OVF_INT：当接收器接收到的数据量多于 FIFO 的存储量时触发此中断。
- UART_FRM_ERR_INT：当接收器检测到数据帧错误时触发此中断。
- UART_PARITY_ERR_INT：当接收器检测到校验位错误时触发此中断。
- UART_TXFIFO_EMPTY_INT：当发送 FIFO 中的数据量少于 UART_TXFIFO_EMPTY_THRHD 所指定的值时触发此中断。
- UART_RXFIFO_FULL_INT：当接收器接收到的数据多于 UART_RXFIFO_FULL_THRHD 所指定的值时触发此中断。
- UART_WAKEUP_INT：UART 被唤醒时产生此中断。

UHCI 中断源如下所列：

- UHCI_APP_CTRL1_INT：软件置位 UHCI_APP_CTRL1_INT_RAW 时触发此中断。
- UHCI_APP_CTRL0_INT：软件置位 UHCI_APP_CTRL0_INT_RAW 时触发此中断。
- UHCI_OUTLINK_EOF_ERR_INT：当检测到发送链表描述符中的 EOF 有错误时触发此中断。

- UHCI_SEND_A_REG_Q_INT：当使用 always_send 发送一串短包，UHCI 发送了短包后触发此中断。
- UHCI_SEND_S_REG_Q_INT：当使用 single_send 发送一串短包，UHCI 发送了短包后触发此中断。
- UHCI_TX_HUNG_INT：当 UHCI 利用 GDMA Tx 通道从 RAM 中读取数据的时间过长时触发此中断。
- UHCI_RX_HUNG_INT：当 UHCI 利用 GDMA Rx 通道接收数据的时间过长时触发此中断。
- UHCI_TX_START_INT：当检测到分隔符时触发此中断。
- UHCI_RX_START_INT：当分隔符已发送时触发此中断。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 21.7 寄存器列表。

21.6 编程流程

21.6.1 寄存器类型

UART 的所有寄存器都处于 APB_CLK 时钟域。

UART 上的配置寄存器分为两类：第一类寄存器作用在 APB/AHB 时钟域，该类寄存器配置后不需要额外的操作；第二类寄存器作用在 UART_SCLK 时钟域，因此需要进行跨时钟域处理。在配置完该类寄存器后，需要通过写寄存器 UART_REG_UPDATE 使配置的值同步到 Core 时钟域。当同步完成后，硬件会将 UART_REG_UPDATE 自动清零。推荐用户在配置需要同步的寄存器之后，检查 UART_REG_UPDATE 是否为 0，确保上一次的同步操作完成之后再执行接下来的寄存器配置。

为了方便用户区分这两类寄存器，章节 21.7 中将需要做跨时钟域操作的寄存器放在一起，该类寄存器名称附带 _SYNC 后缀。不需要做跨时钟域操作的寄存器放在一起，该类寄存器名称不带 _SYNC 后缀。

21.6.2 具体步骤

图21.6-1 显示了 UART 模块的编程流程。主要包括：初始化、寄存器配置、启动 UART TX/RX 和数据传输结束。

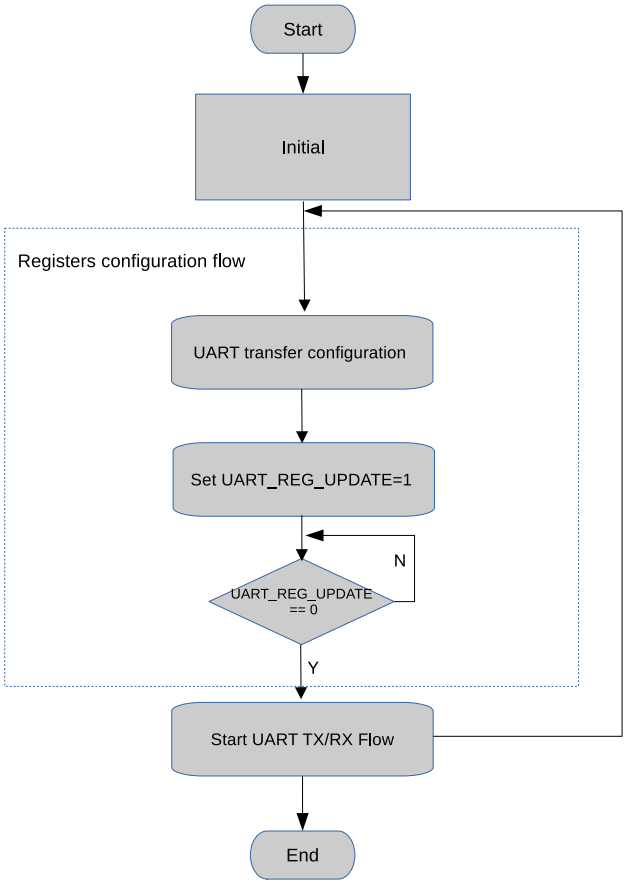


图 21.6-1. UART 编程流程

21.6.2.1 UART*n* 模块初始化

UART*n* 模块初始化流程如下：

- 将 PCR_UART*n*_CLK_EN 置 1，打开 UART*n* 的 APB_CLK；
- 向 PCR_UART*n*_RST_EN 写 1；
- 将 PCR_UART*n*_RST_EN 清 0。

21.6.2.2 UART*n* 通信配置

UART*n* 通信配置流程如下：

- 等待 UART_REG_UPDATE 为 0，确保上一次同步已经完成；
- 配置 PCR_UART*n*_SCLK_SEL 选择时钟源；
- 配置 PCR_UART*n*_SCLK_DIV_NUM、PCR_UART*n*_SCLK_DIV_A、PCR_UART*n*_SCLK_DIV_B 设置预分频器系数；
- 配置 UART_CLKDIV、UART_CLKDIV_FRAG 设置发送波特率；

- 配置 UART_BIT_NUM 设置数据长度；
- 配置 UART_PARITY_EN、UART_PARITY 设置奇偶校验；
- 可选步骤，根据应用不同存在差异；
- 向 UART_REG_UPDATE 写 1，将配置的值同步到 Core 时钟域。

21.6.2.3 启动 UART*n*

启动 UART*n* TX 发送数据：

- 配置 UART_TXFIFO_EMPTY_THRHD，设置 TX FIFO 空阈值；
- 对 UART_TXFIFO_EMPTY_INT_ENA 置 0，关闭 UART_TXFIFO_EMPTY_INT 中断；
- 向 UART_RXFIFO_RD_BYTE 写入需要发送的数据；
- 置位 UART_TXFIFO_EMPTY_INT_CLR，清除 UART_TXFIFO_EMPTY_INT 中断；
- 置位 UART_TXFIFO_EMPTY_INT_ENA，使能 UART_TXFIFO_EMPTY_INT 中断；
- 检测 UART_TXFIFO_EMPTY_INT_ST，等待发送数据结束。

启动 UART*n* RX 数据接收：

- 配置 UART_RXFIFO_FULL_THRHD，设置 RX FIFO 满阈值；
- 置位 UART_RXFIFO_FULL_INT_ENA，使能 UART_RXFIFO_FULL_INT 中断；
- 检测 UART_RXFIFO_FULL_INT_ST，等待 RX FIFO 接收数据满；
- 通过读 UART_RXFIFO_RD_BYTE，从 RX FIFO 中读出数据，并可通过 UART_RXFIFO_CNT 获得当前 RX FIFO 中的接收数据量。

21.7 寄存器列表

21.7.1 UART 寄存器列表

本小节的所有地址均为相对于 UART 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
FIFO 配置寄存器			
UART_FIFO_REG	FIFO 数据寄存器	0x0000	RO
UART_TOUT_CONF_SYNC_REG	UART 阈值和分配配置	0x0064	R/W
UART 中断寄存器			
UART_INT_RAW_REG	原始中断状态	0x0004	R/WTC/SS
UART_INT_ST_REG	屏蔽中断状态	0x0008	RO
UART_INT_ENA_REG	中断使能位	0x000C	R/W
UART_INT_CLR_REG	中断清除位	0x0010	WT
配置寄存器			
UART_CLKDIV_SYNC_REG	时钟分频配置	0x0014	R/W
UART_RX_FILT_REG	RX 滤波器配置	0x0018	R/W
UART_CONF0_SYNC_REG	配置寄存器 0	0x0020	R/W
UART_CONF1_REG	配置寄存器 1	0x0024	R/W
UART_HWFC_CONF_SYNC_REG	硬件流控配置	0x002C	R/W
UART_SLEEP_CONF0_REG	UART 睡眠配置寄存器 0	0x0030	R/W
UART_SLEEP_CONF1_REG	UART 睡眠配置寄存器 1	0x0034	R/W
UART_SLEEP_CONF2_REG	UART 睡眠配置寄存器 2	0x0038	R/W
UART_SWFC_CONF0_SYNC_REG	软件流控字符配置	0x003C	varies
UART_SWFC_CONF1_REG	软件流控字符配置	0x0040	R/W
UART_TXBRK_CONF_SYNC_REG	TX 断开字符配置	0x0044	R/W
UART_IDLE_CONF_SYNC_REG	帧结束空闲配置	0x0048	R/W
UART_RS485_CONF_SYNC_REG	RS485 模式配置	0x004C	R/W
UART_CLK_CONF_REG	UART core 时钟配置	0x0088	R/W
UART_REG_UPDATE_REG	UART 寄存器配置更新	0x0098	R/W/SC
UART_ID_REG	UART ID 寄存器	0x009C	R/W
状态寄存器			
UART_STATUS_REG	UART 状态寄存器	0x001C	RO
UART_MEM_TX_STATUS_REG	TX FIFO 写入、读取偏移地址	0x0068	RO
UART_MEM_RX_STATUS_REG	RX FIFO 写入、读取偏移地址	0x006C	RO
UART_FSM_STATUS_REG	UART 发送和接收状态	0x0070	RO
UART_AFIFO_STATUS_REG	UART 异步 FIFO 状态	0x0090	RO
AT 转义序列检测配置寄存器			
UART_AT_CMD_PRECNT_SYNC_REG	序列发送前的时序配置	0x0050	R/W
UART_AT_CMD_POSTCNT_SYNC_REG	序列发送后的时序配置	0x0054	R/W
UART_AT_CMD_GAPTOUT_SYNC_REG	超时配置	0x0058	R/W

名称	描述	地址	访问
UART_AT_CMD_CHAR_SYNC_REG	AT 转义序列检测配置	0x005C	R/W
自动波特率检测寄存器			
UART_POSPULSE_REG	自动波特率检测高电平脉冲寄存器	0x0074	RO
UART_NEGPULSE_REG	自动波特率检测低电平脉冲寄存器	0x0078	RO
UART_LOWPULSE_REG	自动波特率检测最短低电平脉冲持续时间寄存器	0x007C	RO
UART_HIGHPULSE_REG	自动波特率检测最短高电平脉冲持续时间寄存器	0x0080	RO
UART_RXD_CNT_REG	自动波特率检测沿变化计数寄存器	0x0084	RO
版本寄存器			
UART_DATE_REG	UART 版本控制寄存器	0x008C	R/W

21.7.2 LP UART 寄存器列表

本小节的所有地址均为相对于 LP UART 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
FIFO 配置寄存器			
LP_UART_FIFO_REG	FIFO 数据寄存器	0x0000	RO
LP_UART_TOUT_CONF_SYNC_REG	LP UART 阈值和分配配置	0x0064	R/W
LP UART 中断寄存器			
LP_UART_INT_RAW_REG	原始中断状态	0x0004	R/WTC/SS
LP_UART_INT_ST_REG	屏蔽中断状态	0x0008	RO
LP_UART_INT_ENA_REG	中断使能位	0x000C	R/W
LP_UART_INT_CLR_REG	中断清除位	0x0010	WT
配置寄存器			
LP_UART_CLKDIV_SYNC_REG	时钟分频配置	0x0014	R/W
LP_UART_RX_FILT_REG	RX 滤波器配置	0x0018	R/W
LP_UART_CONF0_SYNC_REG	配置寄存器 0	0x0020	R/W
LP_UART_CONF1_REG	配置寄存器 1	0x0024	R/W
LP_UART_HWFC_CONF_SYNC_REG	硬件流控配置	0x002C	R/W
LP_UART_SLEEP_CONF0_REG	LP UART 睡眠配置寄存器 0	0x0030	R/W
LP_UART_SLEEP_CONF1_REG	LP UART 睡眠配置寄存器 1	0x0034	R/W
LP_UART_SLEEP_CONF2_REG	LP UART 睡眠配置寄存器 2	0x0038	R/W
LP_UART_SWFC_CONF0_SYNC_REG	软件流控字符配置	0x003C	varies
LP_UART_SWFC_CONF1_REG	软件流控字符配置	0x0040	R/W
LP_UART_TXBRK_CONF_SYNC_REG	TX 断开字符配置	0x0044	R/W
LP_UART_IDLE_CONF_SYNC_REG	帧结束空闲配置	0x0048	R/W
LP_UART_DELAY_CONF_SYNC_REG	RS485 模式配置	0x004C	R/W
LP_UART_CLK_CONF_REG	LP UART core 时钟配置	0x0088	R/W
LP_UART_REG_UPDATE_REG	LP UART 寄存器配置更新	0x0098	R/W/SC

名称	描述	地址	访问
LP_UART_ID_REG	LP UART ID 寄存器	0x009C	R/W
状态寄存器			
LP_UART_STATUS_REG	LP UART 状态寄存器	0x001C	RO
LP_UART_MEM_TX_STATUS_REG	TX FIFO 写入、读取偏移地址	0x0068	RO
LP_UART_MEM_RX_STATUS_REG	RX FIFO 写入、读取偏移地址	0x006C	RO
LP_UART_FSM_STATUS_REG	LP UART 发送和接收状态	0x0070	RO
LP_UART_AFIFO_STATUS_REG	LP UART 异步 FIFO 状态	0x0090	RO
AT 转义序列检测配置寄存器			
LP_UART_AT_CMD_PRECNT_SYNC_REG	序列发送前的时序配置	0x0050	R/W
LP_UART_AT_CMD_POSTCNT_SYNC_REG	序列发送前后的时序配置	0x0054	R/W
LP_UART_AT_CMD_GAPTOUT_SYNC_REG	超时配置	0x0058	R/W
LP_UART_AT_CMD_CHAR_SYNC_REG	AT 转义序列检测配置	0x005C	R/W
版本寄存器			
LP_UART_DATE_REG	LP UART 版本控制寄存器	0x008C	R/W

21.7.3 UHCI 寄存器列表

本小节的所有地址均为相对于 UHCI 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
配置寄存器			
UHCI_CONF0_REG	UHCI 配置寄存器	0x0000	R/W
UHCI_CONF1_REG	UHCI 配置寄存器	0x0014	varies
UHCI_ESCAPE_CONF_REG	转义符配置	0x0020	R/W
UHCI_HUNG_CONF_REG	超时配置	0x0024	R/W
UHCI_ACK_NUM_REG	配置 UHCI ACK 值	0x0028	varies
UHCI_QUICK_SENT_REG	UHCI 快速发送配置寄存器	0x0030	varies
UHCI_REG_Q0_WORD0_REG	Q0 WORD0 快速发送寄存器	0x0034	R/W
UHCI_REG_Q0_WORD1_REG	Q0 WORD1 快速发送寄存器	0x0038	R/W
UHCI_REG_Q1_WORD0_REG	Q1 WORD0 快速发送寄存器	0x003C	R/W
UHCI_REG_Q1_WORD1_REG	Q1 WORD1 快速发送寄存器	0x0040	R/W
UHCI_REG_Q2_WORD0_REG	Q2 WORD0 快速发送寄存器	0x0044	R/W
UHCI_REG_Q2_WORD1_REG	Q2 WORD1 快速发送寄存器	0x0048	R/W
UHCI_REG_Q3_WORD0_REG	Q3 WORD0 快速发送寄存器	0x004C	R/W
UHCI_REG_Q3_WORD1_REG	Q3 WORD1 快速发送寄存器	0x0050	R/W
UHCI_REG_Q4_WORD0_REG	Q4 WORD0 快速发送寄存器	0x0054	R/W
UHCI_REG_Q4_WORD1_REG	Q4 WORD1 快速发送寄存器	0x0058	R/W
UHCI_REG_Q5_WORD0_REG	Q5 WORD0 快速发送寄存器	0x005C	R/W
UHCI_REG_Q5_WORD1_REG	Q5 WORD1 快速发送寄存器	0x0060	R/W
UHCI_REG_Q6_WORD0_REG	Q6 WORD0 快速发送寄存器	0x0064	R/W
UHCI_REG_Q6_WORD1_REG	Q6 WORD1 快速发送寄存器	0x0068	R/W

名称	描述	地址	访问
UHCI_ESC_CONF0_REG	转义序列配置寄存器 0	0x006C	R/W
UHCI_ESC_CONF1_REG	转义序列配置寄存器 1	0x0070	R/W
UHCI_ESC_CONF2_REG	转义序列配置寄存器 2	0x0074	R/W
UHCI_ESC_CONF3_REG	转义序列配置寄存器 3	0x0078	R/W
UHCI_PKT_THRES_REG	包长度配置寄存器	0x007C	R/W
UHCI 中断寄存器			
UHCI_INT_RAW_REG	原始中断状态	0x0004	varies
UHCI_INT_ST_REG	屏蔽中断状态	0x0008	RO
UHCI_INT_ENA_REG	中断使能位	0x000C	R/W
UHCI_INT_CLR_REG	中断清除位	0x0010	WT
UHCI 状态寄存器			
UHCI_STATE0_REG	UHCI 接收状态	0x0018	RO
UHCI_STATE1_REG	UHCI 发送状态	0x001C	RO
UHCI_RX_HEAD_REG	UHCI 包报头寄存器	0x002C	RO
版本寄存器			
UHCI_DATE_REG	UHCI 版本控制寄存器	0x0080	R/W

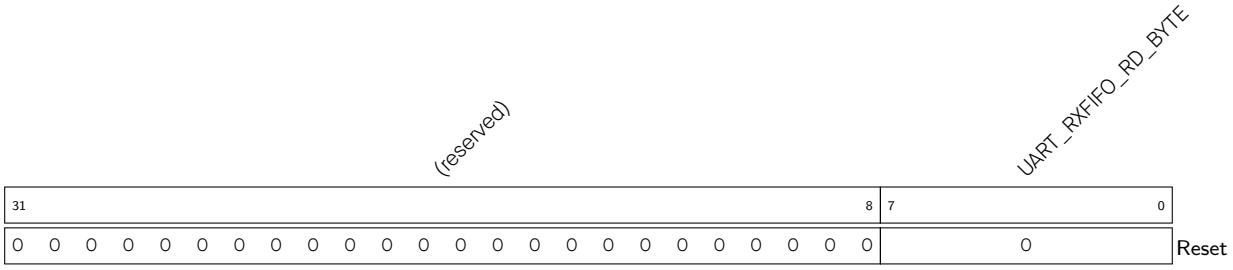
21.8 寄存器

21.8.1 UART 寄存器

本小节的所有地址均为相对于 UART 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

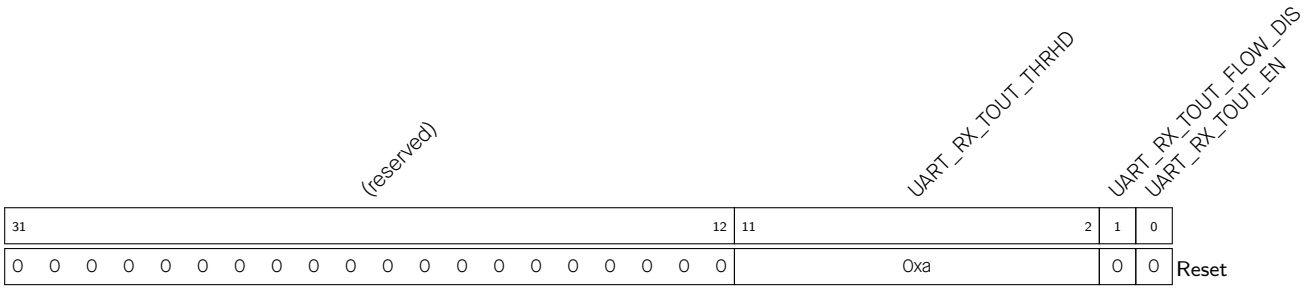
Register 21.1. UART_FIFO_REG (0x0000)



UART_RXFIFO_RD_BYTE 表示 UART*n* 从 FIFO 读取的数据。

单位：字节。(RO)

Register 21.2. UART_TOUT_CONF_SYNC_REG (0x0064)



UART_RX_TOUT_EN 配置是否开启 UART 接收器的超时功能。

0：关闭

1：开启

(R/W)

UART_RX_TOUT_FLOW_DIS 配置开启硬件流控时是否关闭 idle 状态计数器计数。

0：开启

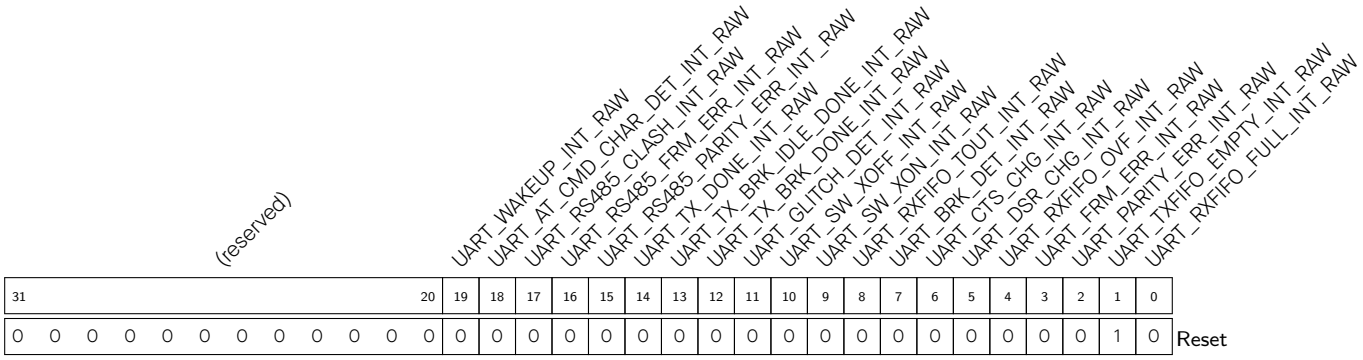
1：关闭

(R/W)

UART_RX_TOUT_THRHD 配置触发超时前总线可保持空闲状态的时间。

单位：位时间（传输一位所需的时间）。(R/W)

Register 21.3. UART_INT_RAW_REG (0x0004)



UART_RXFIFO_FULL_INT_RAW UART_RXFIFO_FULL_INT 的原始中断状态。(R/WTC/SS)

UART_TXFIFO_EMPTY_INT_RAW UART_TXFIFO_EMPTY_INT 的原始中断状态。(R/WTC/SS)

UART_PARITY_ERR_INT_RAW UART_PARITY_ERR_INT 的原始中断状态。(R/WTC/SS)

UART_FRM_ERR_INT_RAW UART_FRM_ERR_INT 的原始中断状态。(R/WTC/SS)

UART_RXFIFO_OVF_INT_RAW UART_RXFIFO_OVF_INT 的原始中断状态。(R/WTC/SS)

UART_DSR_CHG_INT_RAW UART_DSR_CHG_INT 的原始中断状态。(R/WTC/SS)

UART_CTS_CHG_INT_RAW UART_CTS_CHG_INT 的原始中断状态。(R/WTC/SS)

UART_BRK_DET_INT_RAW UART_BRK_DET_INT 的原始中断状态。(R/WTC/SS)

UART_RXFIFO_TOUT_INT_RAW UART_RXFIFO_TOUT_INT 的原始中断状态。(R/WTC/SS)

UART_SW_XON_INT_RAW UART_SW_XON_INT 的原始中断状态。(R/WTC/SS)

UART_SW_XOFF_INT_RAW UART_SW_XOFF_INT 的原始中断状态。(R/WTC/SS)

UART_GLITCH_DET_INT_RAW UART_GLITCH_DET_INT 的原始中断状态。(R/WTC/SS)

UART_TX_BRK_DONE_INT_RAW UART_TX_BRK_DONE_INT 的原始中断状态。(R/WTC/SS)

UART_TX_BRK_IDLE_DONE_INT_RAW UART_TX_BRK_IDLE_DONE_INT 的原始中断状态。(R/WTC/SS)

UART_TX_DONE_INT_RAW UART_TX_DONE_INT 的原始中断状态。(R/WTC/SS)

UART_RS485_PARITY_ERR_INT_RAW UART_RS485_PARITY_ERR_INT 的原始中断状态。(R/WTC/SS)

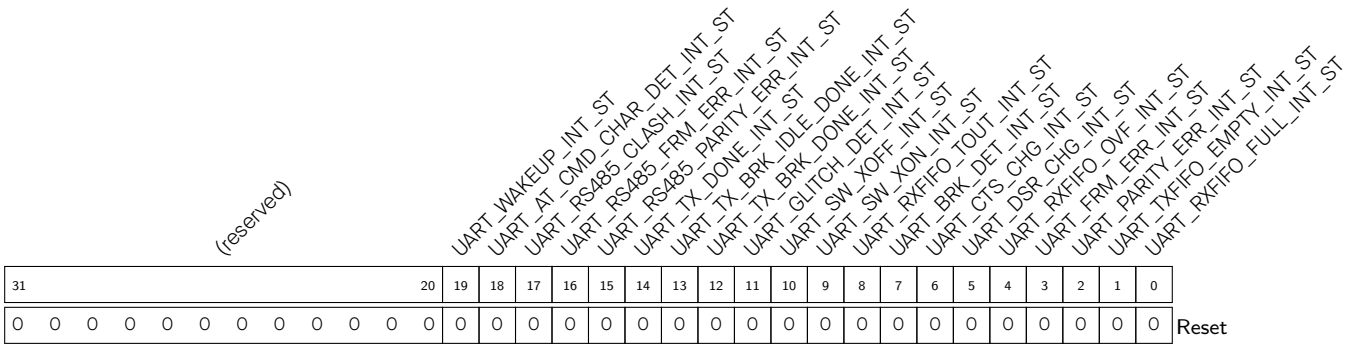
UART_RS485_FRM_ERR_INT_RAW UART_RS485_FRM_ERR_INT 的原始中断状态。(R/WTC/SS)

UART_RS485_CLASH_INT_RAW UART_RS485_CLASH_INT 的原始中断状态。(R/WTC/SS)

UART_AT_CMD_CHAR_DET_INT_RAW UART_AT_CMD_CHAR_DET_INT 的原始中断状态。(R/WTC/SS)

UART_WAKEUP_INT_RAW UART_WAKEUP_INT 的原始中断状态。(R/WTC/SS)

Register 21.4. UART_INT_ST_REG (0x0008)



UART_RXFIFO_FULL_INT_ST UART_RXFIFO_FULL_INT 的屏蔽中断状态。(RO)

UART_TXFIFO_EMPTY_INT_ST UART_TXFIFO_EMPTY_INT 的屏蔽中断状态。(RO)

UART_PARITY_ERR_INT_ST UART_PARITY_ERR_INT 的屏蔽中断状态。(RO)

UART_FRM_ERR_INT_ST UART_FRM_ERR_INT 的屏蔽中断状态。(RO)

UART_RXFIFO_OVF_INT_ST UART_RXFIFO_OVF_INT 的屏蔽中断状态。(RO)

UART_DSR_CHG_INT_ST UART_DSR_CHG_INT 的屏蔽中断状态。(RO)

UART_CTS_CHG_INT_ST UART_CTS_CHG_INT 的屏蔽中断状态。(RO)

UART_BRK_DET_INT_ST UART_BRK_DET_INT 的屏蔽中断状态。(RO)

UART_RXFIFO_TOUT_INT_ST UART_RXFIFO_TOUT_INT 的屏蔽中断状态。(RO)

UART_SW_XON_INT_ST UART_SW_XON_INT 的屏蔽中断状态。(RO)

UART_SW_XOFF_INT_ST UART_SW_XOFF_INT 的屏蔽中断状态。(RO)

UART_GLITCH_DET_INT_ST UART_GLITCH_DET_INT 的屏蔽中断状态。(RO)

UART_TX_BRK_DONE_INT_ST UART_TX_BRK_DONE_INT 的屏蔽中断状态。(RO)

UART_TX_BRK_IDLE_DONE_INT_ST UART_TX_BRK_IDLE_DONE_INT 的屏蔽中断状态。(RO)

UART_TX_DONE_INT_ST UART_TX_DONE_INT 的屏蔽中断状态。(RO)

UART_RS485_PARITY_ERR_INT_ST UART_RS485_PARITY_ERR_INT 的屏蔽中断状态。(RO)

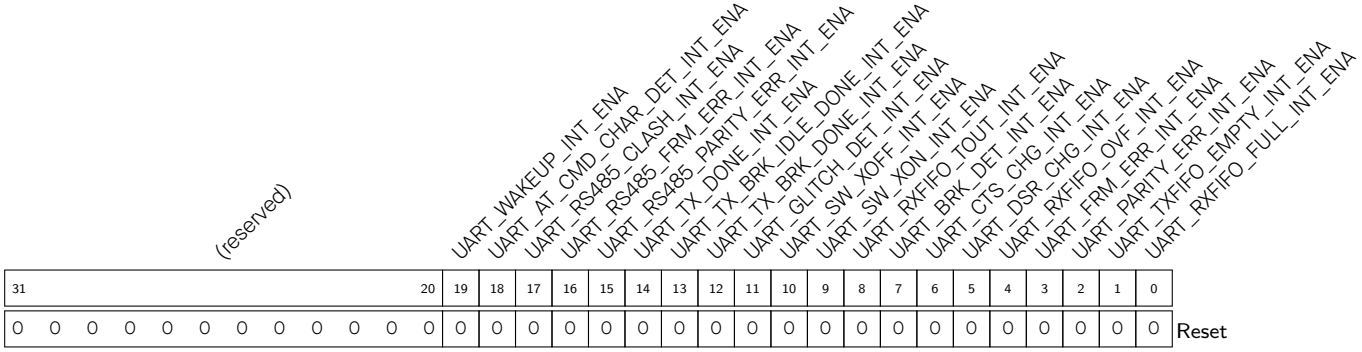
UART_RS485_FRM_ERR_INT_ST UART_RS485_FRM_ERR_INT 的屏蔽中断状态。(RO)

UART_RS485_CLASH_INT_ST UART_RS485_CLASH_INT 的屏蔽中断状态。(RO)

UART_AT_CMD_CHAR_DET_INT_ST UART_AT_CMD_CHAR_DET_INT 的屏蔽中断状态。(RO)

UART_WAKEUP_INT_ST UART_WAKEUP_INT 的屏蔽中断状态。(RO)

Register 21.5. UART_INT_ENA_REG (0x000C)



UART_RXFIFO_FULL_INT_ENA 写 1 使能 UART_RXFIFO_FULL_INT。(R/W)

UART_TXFIFO_EMPTY_INT_ENA 写 1 使能 UART_TXFIFO_EMPTY_INT。(R/W)

UART_PARITY_ERR_INT_ENA 写 1 使能 UART_PARITY_ERR_INT。(R/W)

UART_FRM_ERR_INT_ENA 写 1 使能 UART_FRM_ERR_INT。(R/W)

UART_RXFIFO_OVF_INT_ENA 写 1 使能 UART_RXFIFO_OVF_INT。(R/W)

UART_DSR_CHG_INT_ENA 写 1 使能 UART_DSR_CHG_INT。(R/W)

UART_CTS_CHG_INT_ENA 写 1 使能 UART_CTS_CHG_INT。(R/W)

UART_BRK_DET_INT_ENA 写 1 使能 UART_BRK_DET_INT。(R/W)

UART_RXFIFO_TOUT_INT_ENA 写 1 使能 UART_RXFIFO_TOUT_INT。(R/W)

UART_SW_XON_INT_ENA 写 1 使能 UART_SW_XON_INT。(R/W)

UART_SW_XOFF_INT_ENA 写 1 使能 UART_SW_XOFF_INT。(R/W)

UART_GLITCH_DET_INT_ENA 写 1 使能 UART_GLITCH_DET_INT。(R/W)

UART_TX_BRK_DONE_INT_ENA 写 1 使能 UART_TX_BRK_DONE_INT。(R/W)

UART_TX_BRK_IDLE_DONE_INT_ENA 写 1 使能 UART_TX_BRK_IDLE_DONE_INT。(R/W)

UART_TX_DONE_INT_ENA 写 1 使能 UART_TX_DONE_INT。(R/W)

UART_RS485_PARITY_ERR_INT_ENA 写 1 使能 UART_RS485_PARITY_ERR_INT。(R/W)

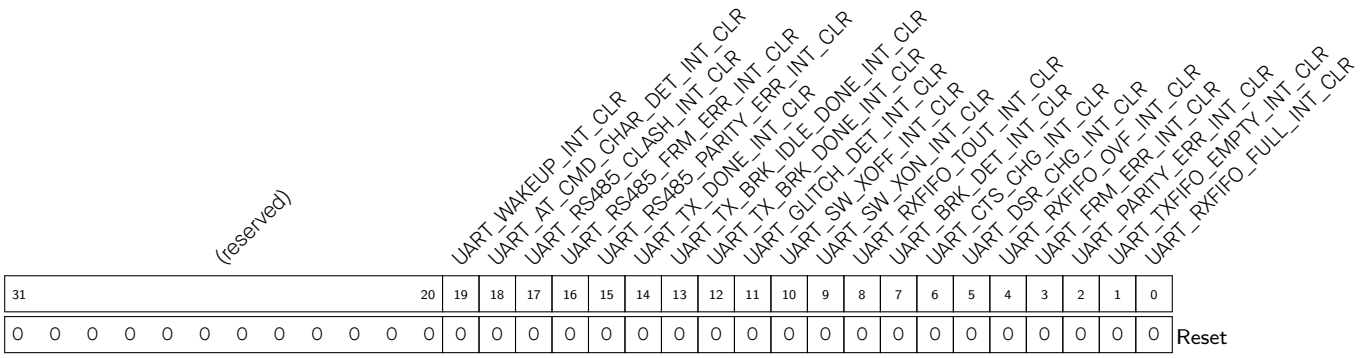
UART_RS485_FRM_ERR_INT_ENA 写 1 使能 UART_RS485_FRM_ERR_INT。(R/W)

UART_RS485_CLASH_INT_ENA 写 1 使能 UART_RS485_CLASH_INT。(R/W)

UART_AT_CMD_CHAR_DET_INT_ENA 写 1 使能 UART_AT_CMD_CHAR_DET_INT。(R/W)

UART_WAKEUP_INT_ENA 写 1 使能 UART_WAKEUP_INT。(R/W)

Register 21.6. UART_INT_CLR_REG (0x0010)



UART_RXFIFO_FULL_INT_CLR 写 1 清除 UART_RXFIFO_FULL_INT。(WT)

UART_TXFIFO_EMPTY_INT_CLR 写 1 清除 UART_TXFIFO_EMPTY_INT。(WT)

UART_PARITY_ERR_INT_CLR 写 1 清除 UART_PARITY_ERR_INT。(WT)

UART_FRM_ERR_INT_CLR 写 1 清除 UART_FRM_ERR_INT。(WT)

UART_RXFIFO_OVF_INT_CLR 写 1 清除 UART_RXFIFO_OVF_INT。(WT)

UART_DSR_CHG_INT_CLR 写 1 清除 UART_DSR_CHG_INT。(WT)

UART_CTS_CHG_INT_CLR 写 1 清除 UART_CTS_CHG_INT。(WT)

UART_BRK_DET_INT_CLR 写 1 清除 UART_BRK_DET_INT。(WT)

UART_RXFIFO_TOUT_INT_CLR 写 1 清除 UART_RXFIFO_TOUT_INT。(WT)

UART_SW_XON_INT_CLR 写 1 清除 UART_SW_XON_INT。(WT)

UART_SW_XOFF_INT_CLR 写 1 清除 UART_SW_XOFF_INT。(WT)

UART_GLITCH_DET_INT_CLR 写 1 清除 UART_GLITCH_DET_INT。(WT)

UART_TX_BRK_DONE_INT_CLR 写 1 清除 UART_TX_BRK_DONE_INT。(WT)

UART_TX_BRK_IDLE_DONE_INT_CLR 写 1 清除 UART_TX_BRK_IDLE_DONE_INT。(WT)

UART_TX_DONE_INT_CLR 写 1 清除 UART_TX_DONE_INT。(WT)

UART_RS485_PARITY_ERR_INT_CLR 写 1 清除 UART_RS485_PARITY_ERR_INT。(WT)

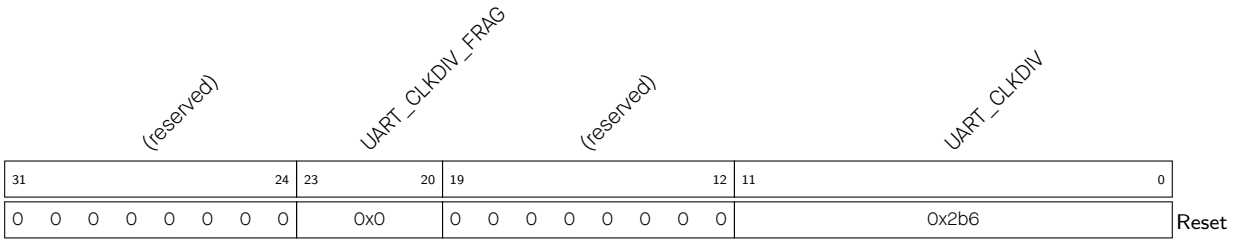
UART_RS485_FRM_ERR_INT_CLR 写 1 清除 UART_RS485_FRM_ERR_INT。(WT)

UART_RS485_CLASH_INT_CLR 写 1 清除 UART_RS485_CLASH_INT。(WT)

UART_AT_CMD_CHAR_DET_INT_CLR 写 1 清除 UART_AT_CMD_CHAR_DET_INT。(WT)

UART_WAKEUP_INT_CLR 写 1 清除 UART_WAKEUP_INT。(WT)

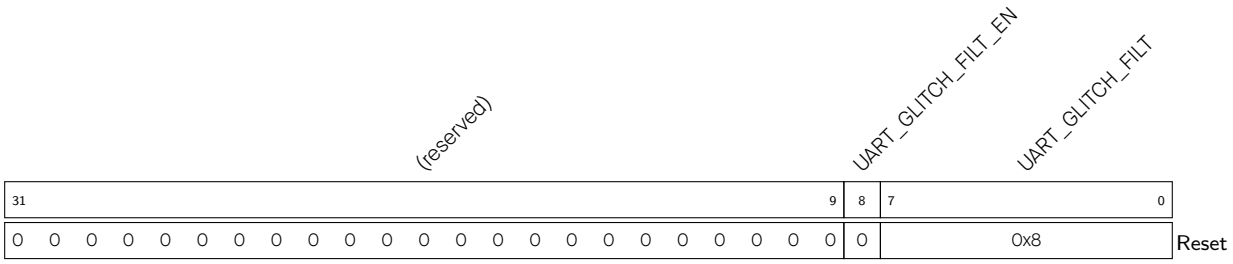
Register 21.7. UART_CLKDIV_SYNC_REG (0x0014)



UART_CLKDIV 配置波特率分频系数的整数部分。(R/W)

UART_CLKDIV_FRAG 配置波特率分频系数的小数部分。(R/W)

Register 21.8. UART_RX_FILT_REG (0x0018)



UART_GLITCH_FILT 配置滤波长度。

单位：UART Core 时钟周期。

宽度小于该数值的输入脉冲会被忽略。(R/W)

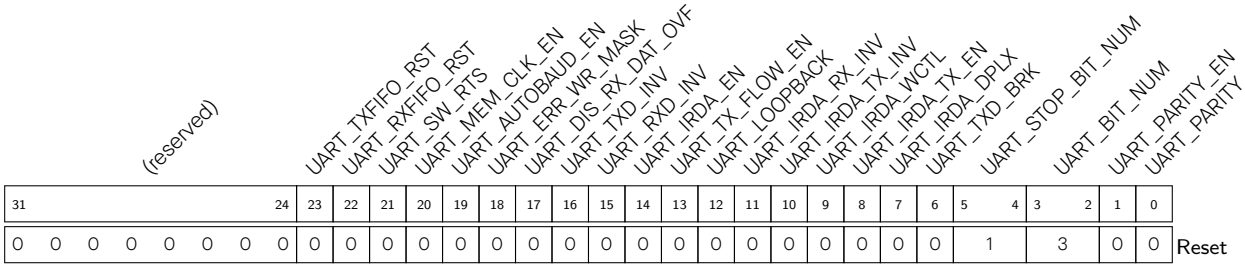
UART_GLITCH_FILT_EN 配置是否开启接收信号滤波器。

0：关闭

1：开启

(R/W)

Register 21.9. UART_CONF0_SYNC_REG (0x0020)



UART_PARITY 配置奇偶检验方式。

0：偶校验

1：奇校验

(R/W)

UART_PARITY_EN 配置是否开启 UART 奇偶校验。

0：关闭

1：开启

(R/W)

UART_BIT_NUM 配置数据位长度。

0：5 位

1：6 位

2：7 位

3：8 位

(R/W)

UART_STOP_BIT_NUM 配置停止位长度。

0: 无效值，没有作用

1: 1 位

2: 1.5 位

3: 2 位

(R/W)

UART_TXD_BRK 配置完成数据发送后，发送器是否发送 NULL。

0：不发送

1：发送

(R/W)

UART_IRDA_DPLX 配置是否开启 IrDA 回环测试模式。

0：关闭

1：开启

(R/W)

UART_IRDA_TX_EN 配置是否开启 IrDA 发送器。

0：关闭

1：开启

(R/W)

见下页...

Register 21.9. UART_CONF0_SYNC_REG (0x0020)

接上页...

UART_IRDA_WCTL 配置 IrDA 发送器的第 11 位。

0：该位为 0

1：该位与第 10 位相同

(R/W)

UART_IRDA_TX_INV 配置是否翻转 IrDA 发送器的电平。

0：不翻转

1：翻转

(R/W)

UART_IRDA_RX_INV 配置是否翻转 IrDA 接收器的电平。

0：不翻转

1：翻转

(R/W)

UART_LOOPBACK 配置是否开启 UART 回环测试模式。

0：关闭

1：开启

(R/W)

UART_TX_FLOW_EN 配置是否开启发送器的流控功能。

0：关闭

1：开启

(R/W)

UART_IRDA_EN 配置是否开启 IrDA。

0：关闭

1：开启

(R/W)

UART_RXD_INV 配置是否翻转 UART RXD 信号电平。

0：不翻转

1：翻转

(R/W)

UART_TXD_INV 配置是否翻转 UART TXD 信号电平。

0：不翻转

1：翻转

(R/W)

UART_DIS_RX_DAT_OVF 配置是否关闭 UART 接收器数据溢出检测。

0：开启

1：关闭

(R/W)

见下页...

Register 21.9. UART_CONF0_SYNC_REG (0x0020)

接上页...

UART_ERR_WR_MASK 配置接收数据有错误时，是否仍将其存储至 FIFO。

0：存储

1：不存储

(R/W)

UART_AUTOBAUD_EN 配置是否开启波特率检测。

0：关闭

1：开启

(R/W)

UART_MEM_CLK_EN 配置是否开启 UART 存储器门控。

0：关闭

1：开启

(R/W)

UART_SW_RTS 配置软件流控使用的 RTS 信号。

0: 禁止发送端发送数据

1: 允许发送端发送数据

(R/W)

UART_RXFIFO_RST 配置是否复位 UART RX FIFO。

0：不复位

1：复位

(R/W)

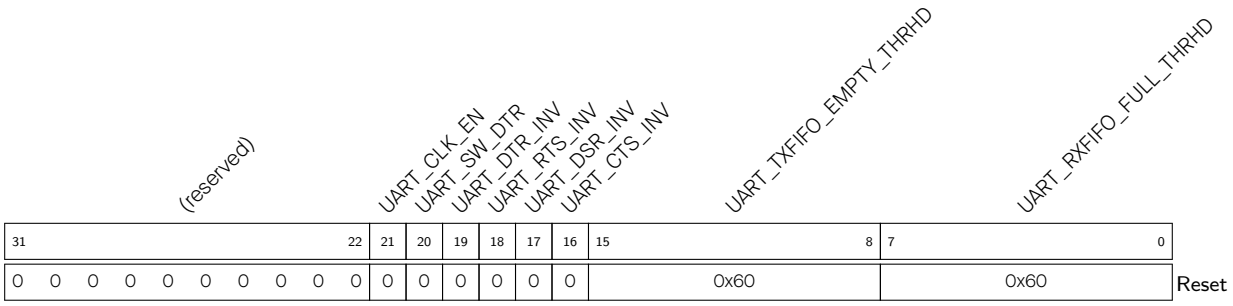
UART_TXFIFO_RST 配置是否复位 UART TX FIFO。

0：不复位

1：复位

(R/W)

Register 21.10. UART_CONF1_REG (0x0024)



UART_RXFIFO_FULL_THRHD 配置 RX FIFO 为满的阈值。

单位：字节。(R/W)

UART_TXFIFO_EMPTY_THRHD 配置 TX FIFO 为空的阈值。

单位：字节。(R/W)

UART_CTS_INV 配置是否翻转 UART CTS 信号电平。

0：不翻转

1：翻转

(R/W)

UART_DSR_INV 配置是否翻转 UART DSR 信号电平。

0：不翻转

1：翻转

(R/W)

UART_RTS_INV 配置是否翻转 UART RTS 信号电平。

0：不翻转

1：翻转

(R/W)

UART_DTR_INV 配置是否翻转 UART DTR 信号电平。

0：不翻转

1：翻转

(R/W)

UART_SW_DTR 配置软件流控使用的 DTR 信号。

0: 发送数据未准备完毕，处于不可用状态

1: 发送数据准备完毕，处于可用状态

(R/W)

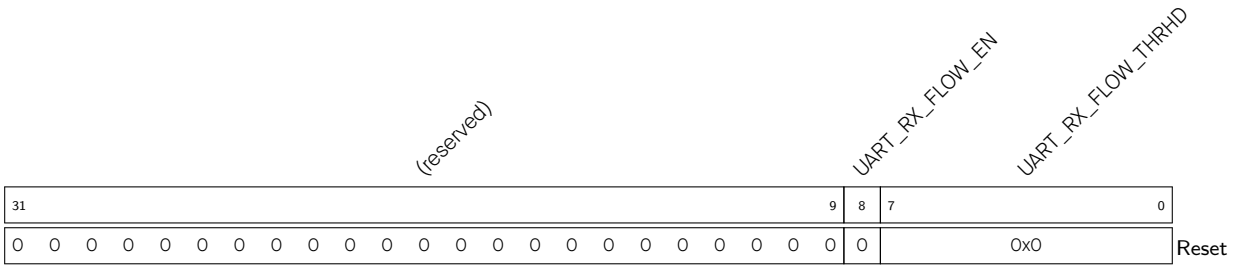
UART_CLK_EN 配置时钟门控。

0：仅在应用写寄存器时开启时钟

1：一直强制为寄存器开启时钟

(R/W)

Register 21.11. UART_HWFC_CONF_SYNC_REG (0x002C)



UART_RX_FLOW_THRHD 配置使用硬件流控时接收数据的最大数。

单位：字节。(R/W)

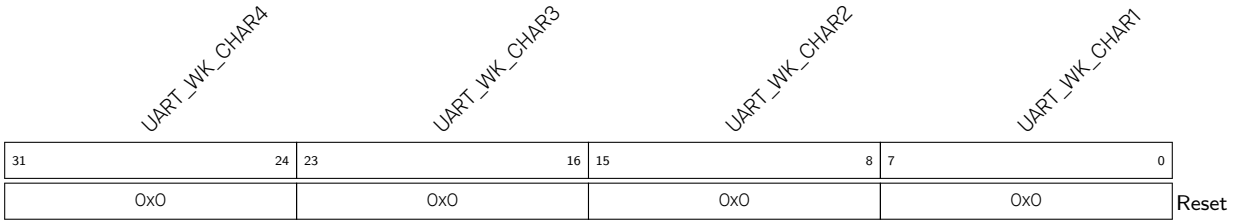
UART_RX_FLOW_EN 配置是否开启 UART 接收器。

0：关闭

1：开启

(R/W)

Register 21.12. UART_SLEEP_CONF0_REG (0x0030)



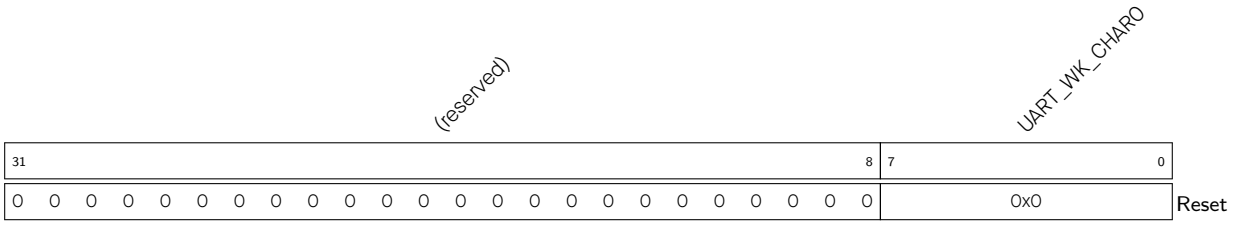
UART_WK_CHAR1 配置唤醒芯片的特定字符 1。(R/W)

UART_WK_CHAR2 配置唤醒芯片的特定字符 2。(R/W)

UART_WK_CHAR3 配置唤醒芯片的特定字符 3。(R/W)

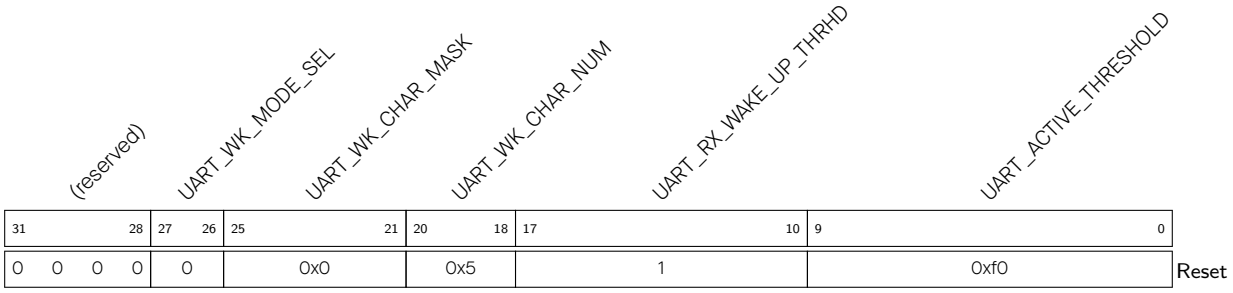
UART_WK_CHAR4 配置唤醒芯片的特定字符 4。(R/W)

Register 21.13. UART_SLEEP_CONF1_REG (0x0034)



UART_WK_CHAR0 配置唤醒芯片的特定字符 0。(R/W)

Register 21.14. UART_SLEEP_CONF2_REG (0x0038)



UART_ACTIVE_THRESHOLD 配置唤醒模式 0 唤醒芯片所需的 RXD 沿变化次数。(R/W)

UART_RX_WAKE_UP_THRHD 配置唤醒模式 1 唤醒芯片所需接收的数据量。

单位：字节。(R/W)

UART_WK_CHAR_NUM 配置选取多少个唤醒字符。(R/W)

UART_WK_CHAR_MASK 配置是否屏蔽唤醒字符。

0：不屏蔽

1：屏蔽

(R/W)

UART_WK_MODE_SEL 配置选取哪个唤醒模式。不同模式的含义详见 21.4.8 唤醒 小节。

0：唤醒模式 0

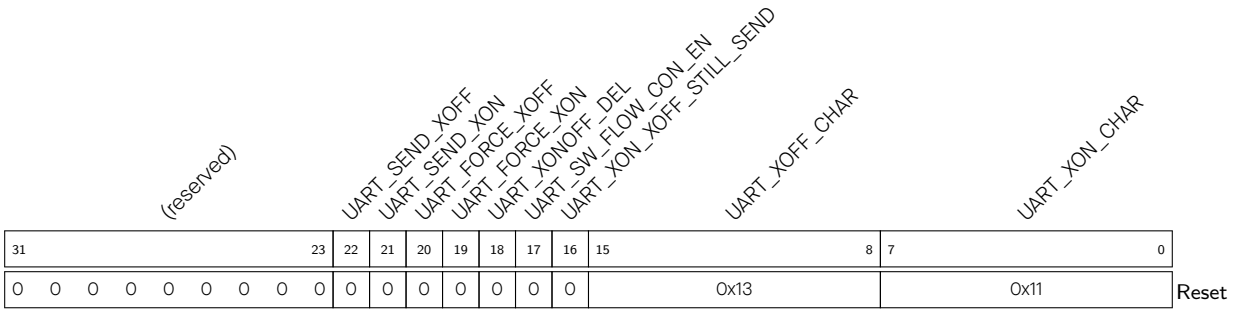
1：唤醒模式 1

2：唤醒模式 2

3：唤醒模式 3

(R/W)

Register 21.15. UART_SWFC_CONF0_SYNC_REG (0x003C)



UART_XON_CHAR 配置 XON 流控字符。(R/W)

UART_XOFF_CHAR 配置 XOFF 流控字符。(R/W)

UART_XON_XOFF_STILL_SEND 配置 UART 发送器关闭时，是否仍能发送 XON 或 XOFF 字符。

0：不能发送

1：可以发送

(R/W)

UART_SW_FLOW_CON_EN 配置是否开启软件流控。

0：关闭

1：开启

(R/W)

UART_XONOFF_DEL 配置是否移除接收数据中的流控字符。

0：不移除

1：移除

(R/W)

UART_FORCE_XON 配置是否让发送器继续发送数据。

0：不发送

1：继续发送

(R/W)

UART_FORCE_XOFF 配置是否让发送器停止发送数据。

0：不停止

1：停止

(R/W)

UART_SEND_XON 配置是否发送 XON 字符。

0：不发送

1：发送

(R/W/SS/SC)

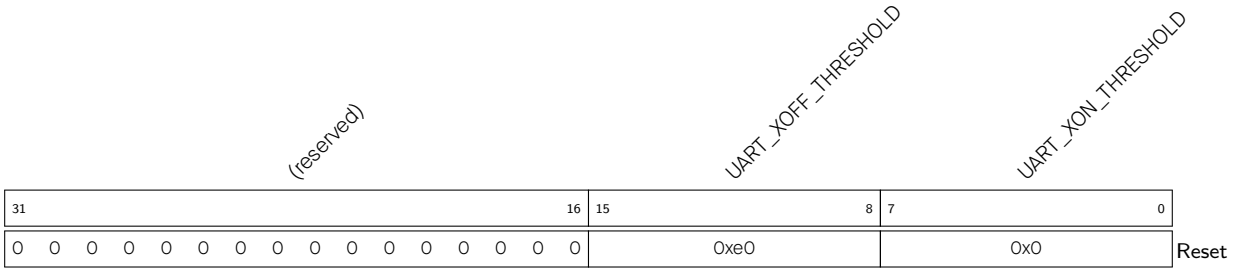
UART_SEND_XOFF 配置是否发送 XOFF 字符。

0：不发送

1：发送

(R/W/SS/SC)

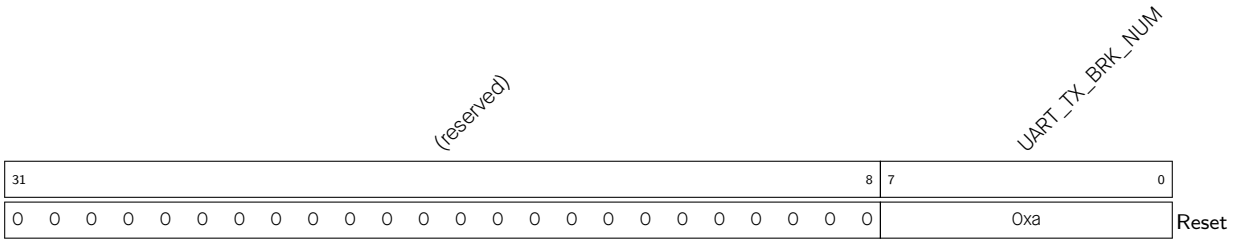
Register 21.16. UART_SWFC_CONF1_REG (0x0040)



UART_XON_THRESHOLD 配置软件流控时，发送 XON 字符所需的 RX FIFO 数据阈值。
单位：字节。(R/W)

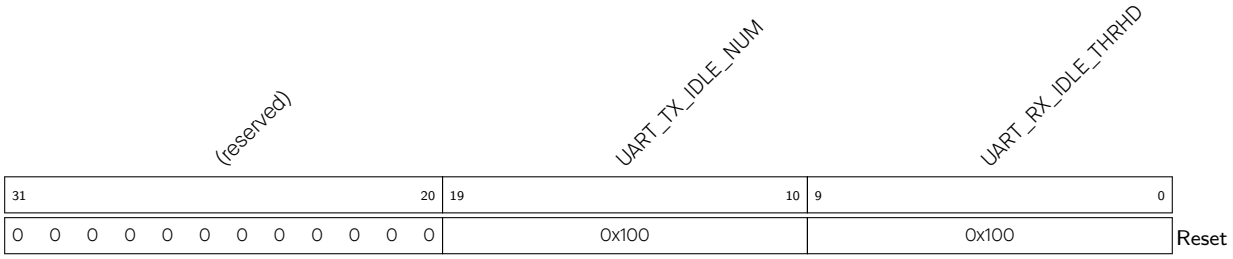
UART_XOFF_THRESHOLD 配置软件流控时，发送 XOFF 字符所需的 RX FIFO 数据阈值。
单位：字节。(R/W)

Register 21.17. UART_TXBRK_CONF_SYNC_REG (0x0044)



UART_TX_BRK_NUM 配置完成数据发送后待发 NULL 字符的数量。
仅在 UART_TXD_BRK 为 1 时有效。(R/W)

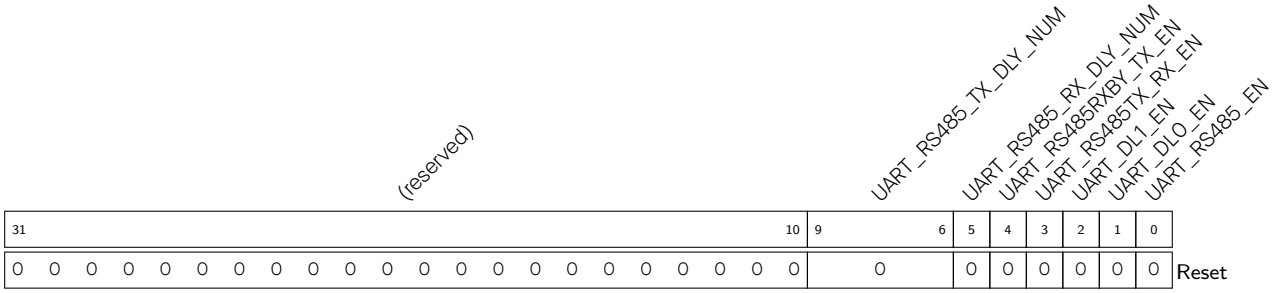
Register 21.18. UART_IDLE_CONF_SYNC_REG (0x0048)



UART_RX_IDLE_THRHD 配置接收一字节数据时间过长、产生帧结束信号的阈值。
单位：位时间（传输一位所需的时间）。(R/W)

UART_TX_IDLE_NUM 配置两次数据传输的间隔时间。
单位：位时间（传输一位所需的时间）。(R/W)

Register 21.19. UART_RS485_CONF_SYNC_REG (0x004C)



UART_RS485_EN 配置是否开启 RS485 模式。

0：关闭

1：开启

(R/W)

UART_DLO_EN 配置是否在起始位之前增加一位延时。

0：不增加

1：增加

(R/W)

UART_DL1_EN 配置是否在停止位之后增加一位延时。

0：不增加

1：增加

(R/W)

UART_RS485TX_RX_EN 配置发送器在 RS485 模式下发送数据时，是否开启接收器接收数据。

0：关闭

1：开启

(R/W)

UART_RS485RXBY_TX_EN 配置 RS485 接收器线路繁忙时，是否开启 RS485 发送器发送数据。

0：关闭

1：开启

(R/W)

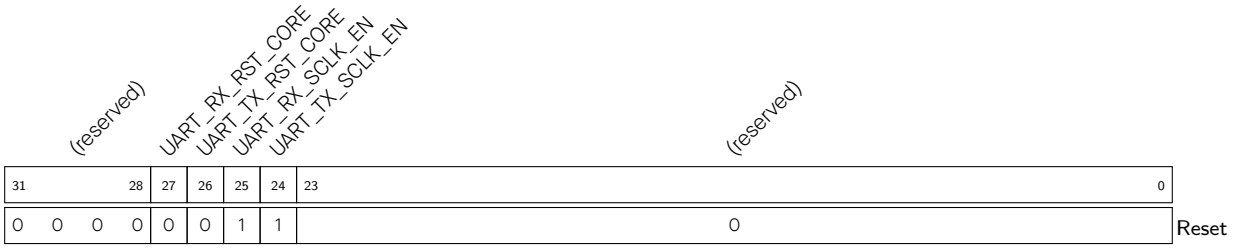
UART_RS485_RX_DLY_NUM 配置接收器内部数据信号的延迟长度。

单位：位时间（传输一位所需的时间）。(R/W)

UART_RS485_TX_DLY_NUM 配置发送器内部数据信号的延迟长度。

单位：位时间（传输一位所需的时间）。(R/W)

Register 21.20. UART_CLK_CONF_REG (0x0088)



UART_TX_SCLK_EN 配置是否开启 UART TX 时钟。

0：关闭

1：开启

(R/W)

UART_RX_SCLK_EN 配置是否开启 UART RX 时钟。

0：关闭

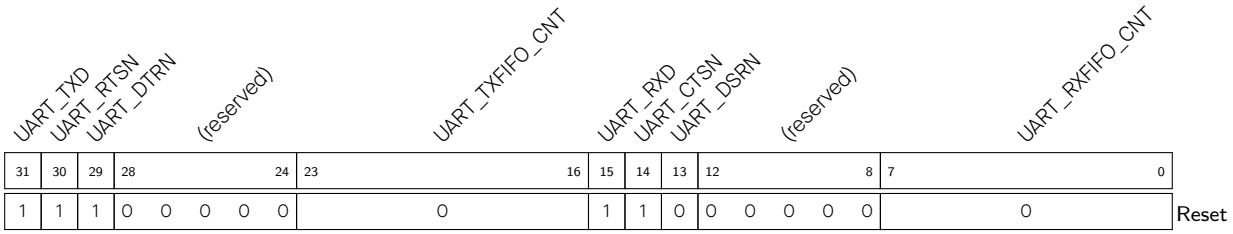
1：开启

(R/W)

UART_TX_RST_CORE 向此位先写 1 后写 0，复位 UART TX。(R/W)

UART_RX_RST_CORE 向此位先写 1 后写 0，复位 UART RX。(R/W)

Register 21.21. UART_STATUS_REG (0x001C)



UART_RXFIFO_CNT 表示 RX FIFO 中有效数据量。

单位：字节。(RO)

UART_DSRN 表示内部 UART DSR 信号的电平值。(RO)

UART_CTSN 表示内部 UART CTS 信号的电平值。(RO)

UART_RXD 表示内部 UART RXD 信号的电平值。(RO)

UART_TXFIFO_CNT 表示 TX FIFO 中的数据量。

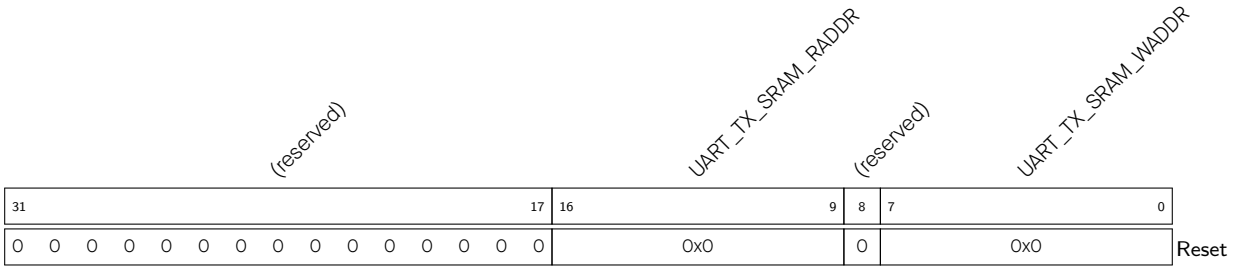
单位：字节。(RO)

UART_DTRN 表示内部 UART DTR 信号的电平。(RO)

UART_RTSN 表示内部 UART RTS 信号的电平。(RO)

UART_TXD 表示内部 UART TXD 信号的电平。(RO)

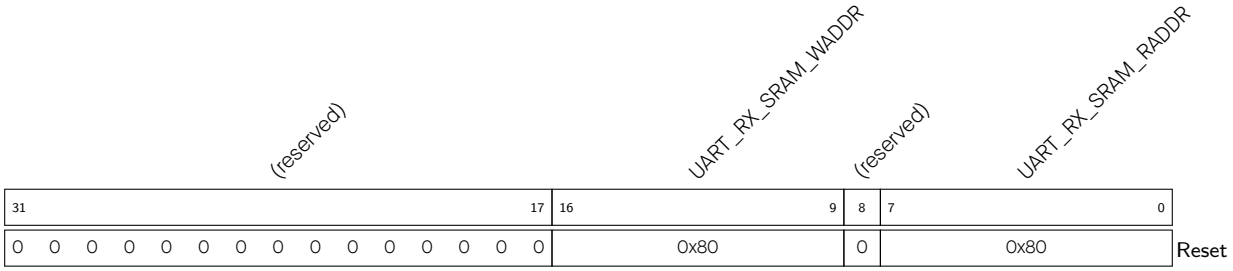
Register 21.22. UART_MEM_TX_STATUS_REG (0x0068)



UART_TX_SRAM_WADDR 表示写 TX FIFO 的偏移地址。(RO)

UART_TX_SRAM_RADDR 表示读 TX FIFO 的偏移地址。(RO)

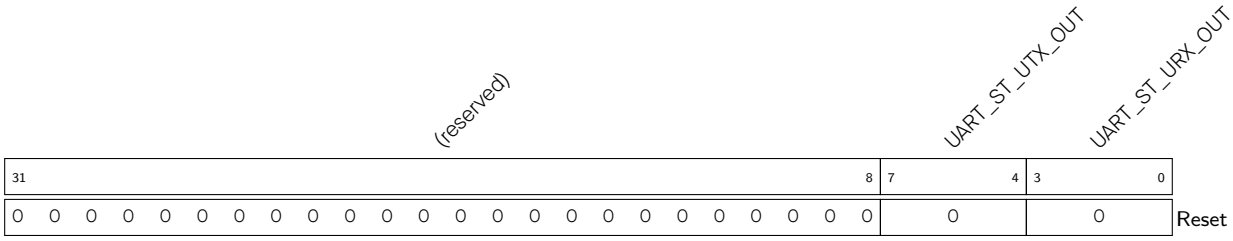
Register 21.23. UART_MEM_RX_STATUS_REG (0x006C)



UART_RX_SRAM_RADDR 表示读 RX FIFO 的偏移地址。(RO)

UART_RX_SRAM_WADDR 表示写 RX FIFO 的偏移地址。(RO)

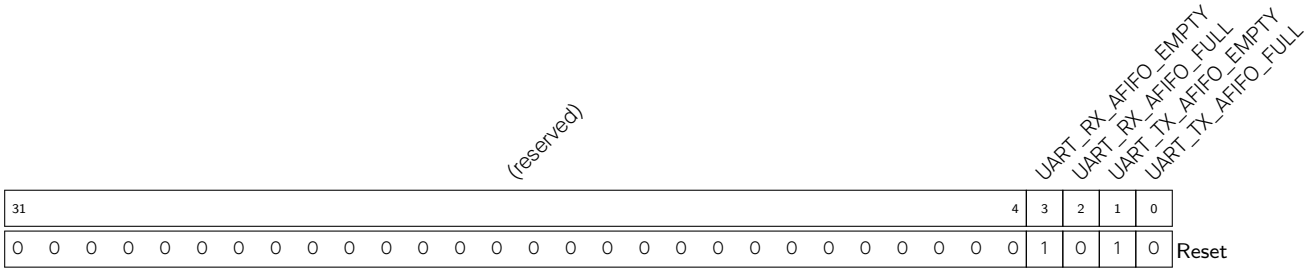
Register 21.24. UART_FSM_STATUS_REG (0x0070)



UART_ST_URX_OUT 表示 UART 接收器的状态。(RO)

UART_ST_UTX_OUT 表示 UART 发送器的状态。(RO)

Register 21.25. UART_AFIFO_STATUS_REG (0x0090)



UART_TX_AFIFO_FULL 表示 APB TX 异步 FIFO 是否已满。

0：未满

1：已满

(RO)

UART_TX_AFIFO_EMPTY 表示 APB TX 异步 FIFO 是否为空。

0：不为空

1：为空

(RO)

UART_RX_AFIFO_FULL 表示 APB RX 异步 FIFO 是否已满。

0：未满

1：已满

(RO)

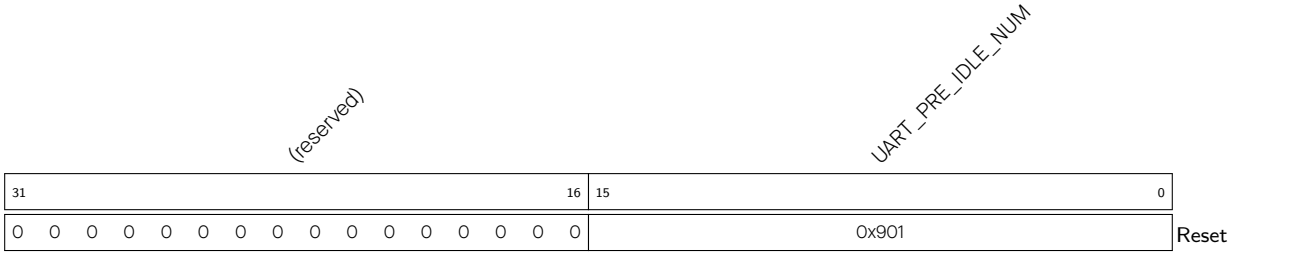
UART_RX_AFIFO_EMPTY 表示 APB RX 异步 FIFO 是否为空。

0：不为空

1：为空

(RO)

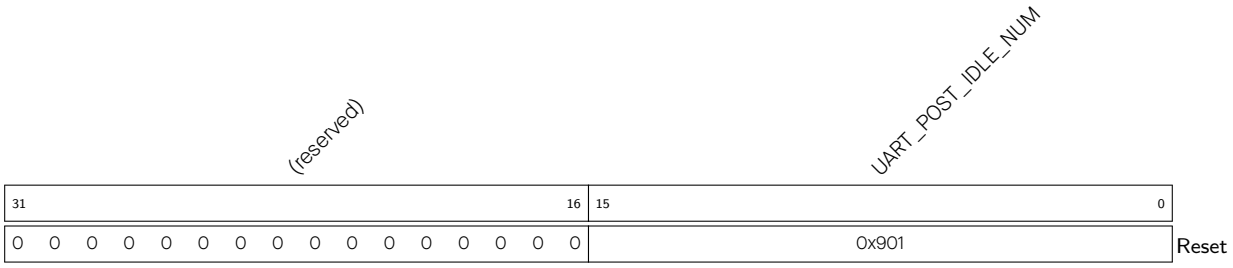
Register 21.26. UART_AT_CMD_PRECNT_SYNC_REG (0x0050)



UART_PRE_IDLE_NUM 配置接收器接收到第一个 AT_CMD 之前的空闲时间。

单位：位时间（传输一位所需的时间）。(R/W)

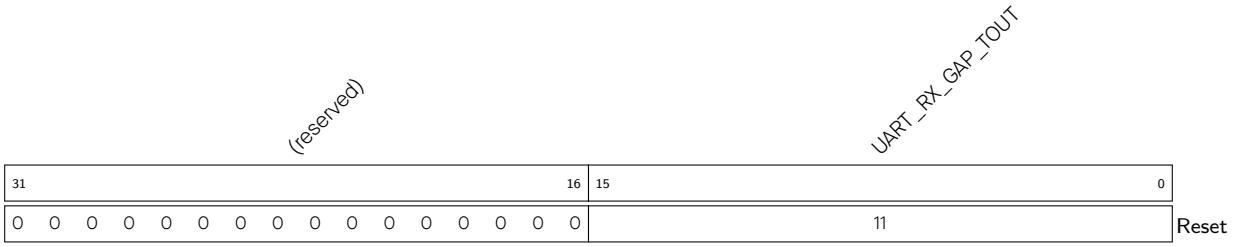
Register 21.27. UART_AT_CMD_POSTCNT_SYNC_REG (0x0054)



UART_POST_IDLE_NUM 配置最后一个 AT_CMD 和后续数据之间的间隔时间。

单位：位时间（传输一位所需的时间）。(R/W)

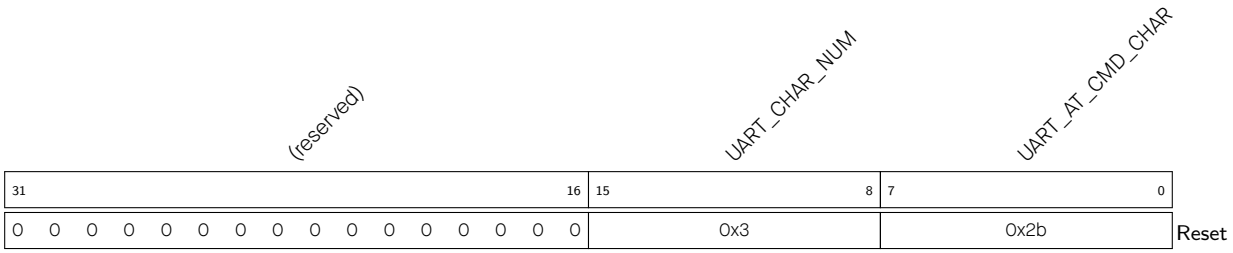
Register 21.28. UART_AT_CMD_GAPTOUT_SYNC_REG (0x0058)



UART_RX_GAP_TOUT 配置 AT_CMD 之间的间隔时间。

单位：位时间（传输一位所需的时间）。(R/W)

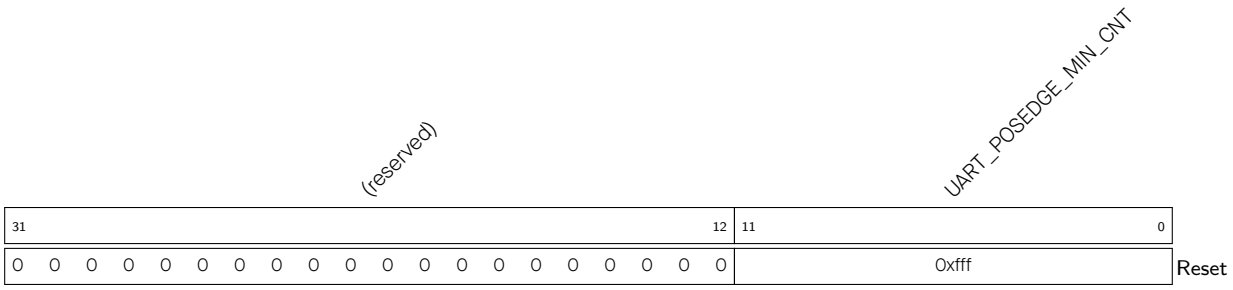
Register 21.29. UART_AT_CMD_CHAR_SYNC_REG (0x005C)



UART_AT_CMD_CHAR 配置 AT_CMD 字符。(R/W)

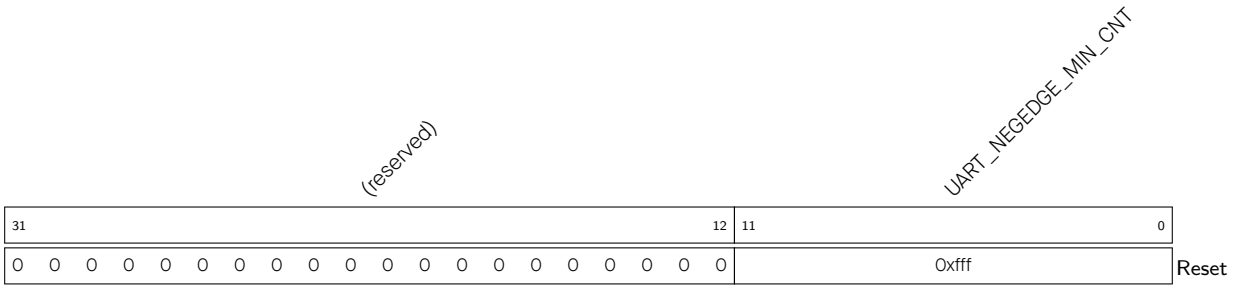
UART_CHAR_NUM 配置接收器可连续接收的 AT_CMD 字符数量。(R/W)

Register 21.30. UART_POSPULSE_REG (0x0074)



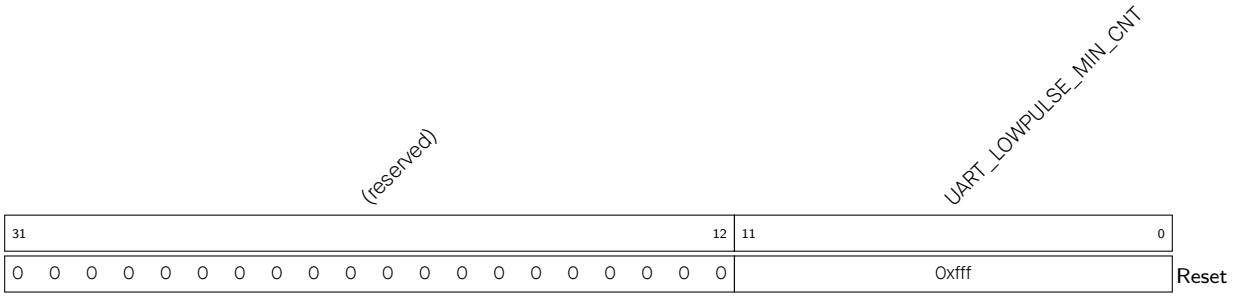
UART_POSEDGE_MIN_CNT 表示两个上升沿之间的最小输入时钟计数值。用于波特率检测。(RO)

Register 21.31. UART_NEGPULSE_REG (0x0078)



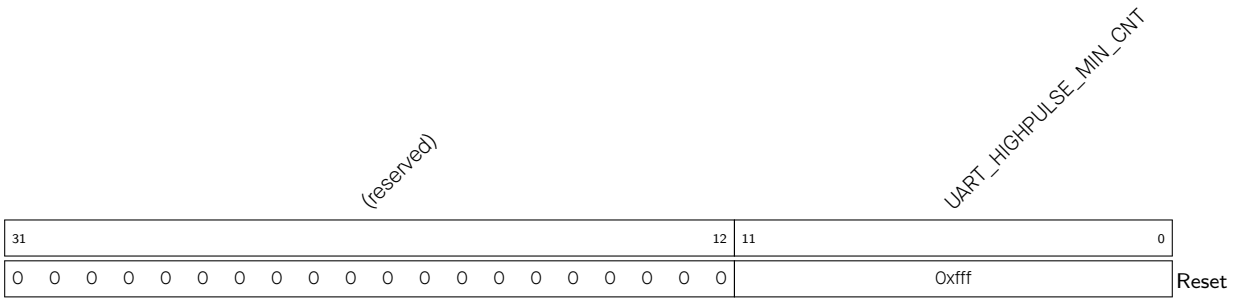
UART_NEGEDGE_MIN_CNT 两个下降沿之间的最小输入时钟计数值。用于波特率检测。(RO)

Register 21.32. UART_LOWPULSE_REG (0x007C)



UART_LOWPULSE_MIN_CNT 表示低电平脉冲的最短持续时间，用于波特率检测。
单位：APB_CLK 时钟周期。(RO)

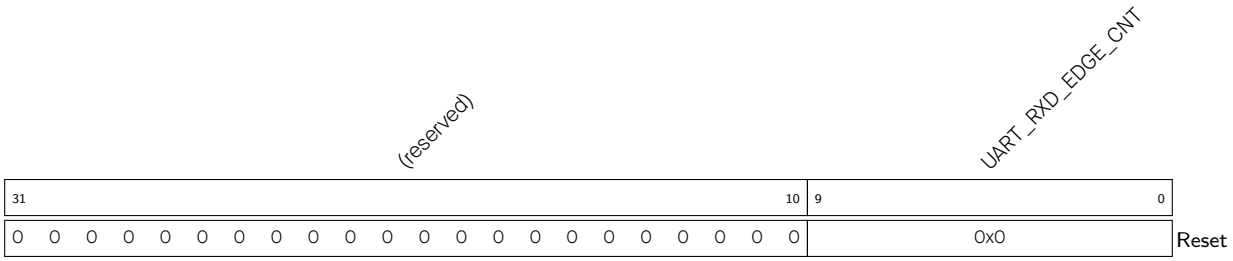
Register 21.33. UART_HIGHPULSE_REG (0x0080)



UART_HIGHPULSE_MIN_CNT 表示高电平脉冲最长持续时间，用于波特率检测。

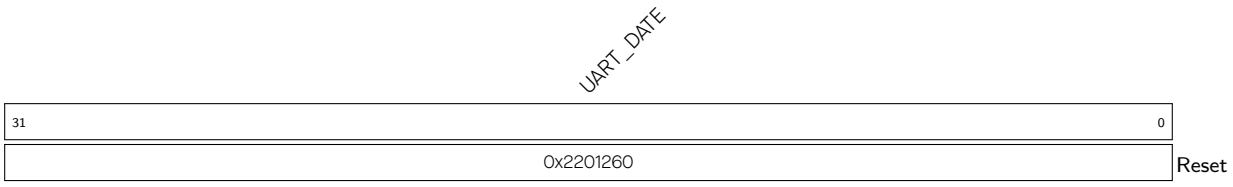
单位：APB_CLK 时钟周期。(RO)

Register 21.34. UART_RXD_CNT_REG (0x0084)



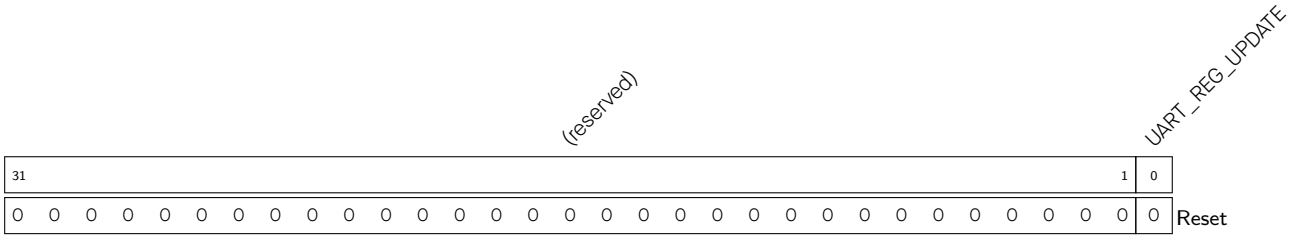
UART_RXD_EDGE_CNT 表示 RXD 沿变化的次数。用于波特率检测。(RO)

Register 21.35. UART_DATE_REG (0x008C)



UART_DATE 版本控制寄存器。(R/W)

Register 21.36. UART_REG_UPDATE_REG (0x0098)



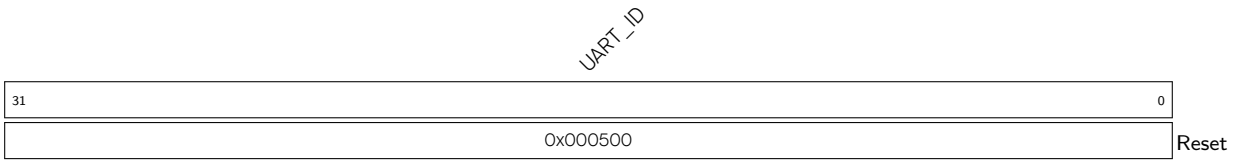
UART_REG_UPDATE 配置是否同步寄存器

0：不同步

1：同步

(R/W/SC)

Register 21.37. UART_ID_REG (0x009C)



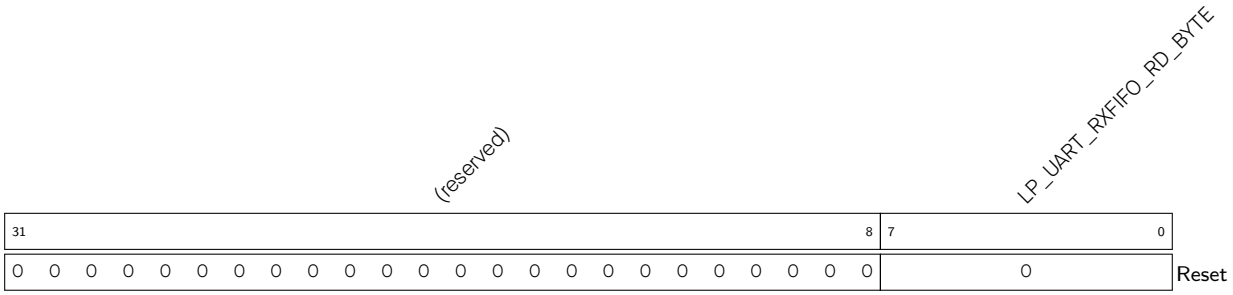
UART_ID 配置 UART ID。(R/W)

21.8.2 LP UART 寄存器

本小节的所有地址均为相对于 LP UART 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

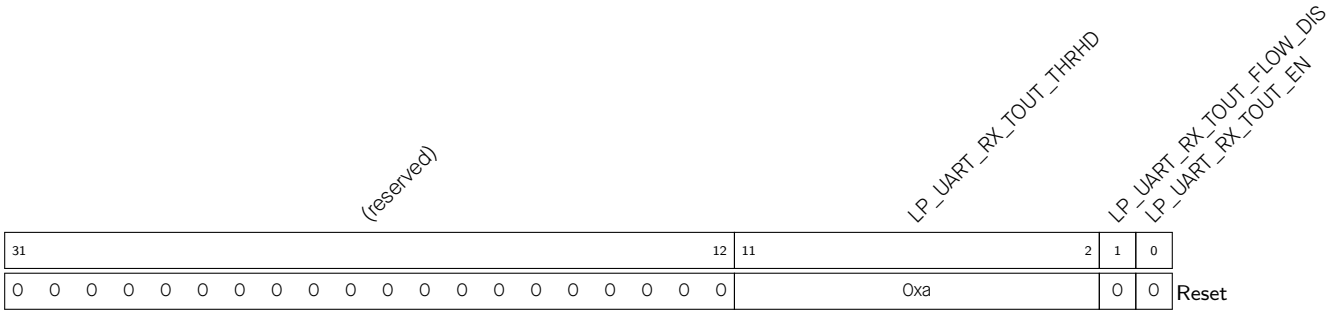
Register 21.38. LP_UART_FIFO_REG (0x0000)



LP_UART_RXFIFO_RD_BYTE 表示 LP UART *n* 从 FIFO 读取的数据。

单位：字节。(RO)

Register 21.39. LP_UART_TOUT_CONF_SYNC_REG (0x0064)



LP_UART_RX_TOUT_EN 配置是否开启 LP UART 接收器的超时功能。

0：关闭

1：开启

(R/W)

LP_UART_RX_TOUT_FLOW_DIS 配置开启硬件流控时是否关闭 idle 状态计数器计数。

0：无效值，没有作用

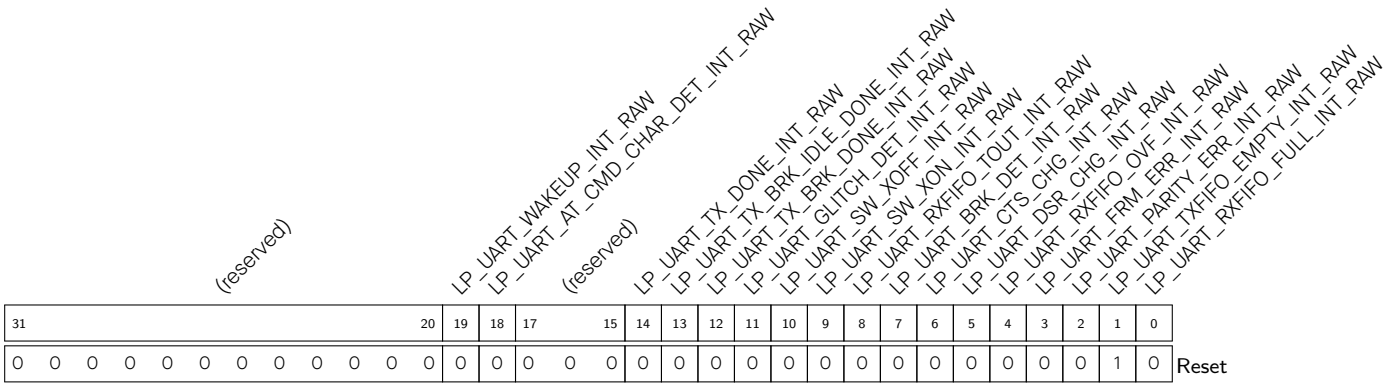
1：关闭

(R/W)

LP_UART_RX_TOUT_THRHD 配置触发超时前总线可保持空闲状态的时间。

单位：位时间（传输一位所需的时间）。(R/W)

Register 21.40. LP_UART_INT_RAW_REG (0x0004)



LP_UART_RXFIFO_FULL_INT_RAW LP_UART_RXFIFO_FULL_INT 的原始中断状态。(R/WTC/SS)

LP_UART_TXFIFO_EMPTY_INT_RAW LP_UART_TXFIFO_EMPTY_INT 的原始中断状态。(R/ WTC/SS)

LP_UART_PARITY_ERR_INT_RAW LP_UART_PARITY_ERR_INT 的原始中断状态。(R/WTC/SS)

LP_UART_FRM_ERR_INT_RAW LP_UART_FRM_ERR_INT 的原始中断状态。(R/WTC/SS)

LP_UART_RXFIFO_OVF_INT_RAW LP_UART_RXFIFO_OVF_INT 的原始中断状态。(R/WTC/SS)

LP_UART_DSR_CHG_INT_RAW LP_UART_DSR_CHG_INT 的原始中断状态。(R/WTC/SS)

LP_UART_CTS_CHG_INT_RAW LP_UART_CTS_CHG_INT 的原始中断状态。(R/WTC/SS)

LP_UART_BRK_DET_INT_RAW LP_UART_BRK_DET_INT 的原始中断状态。(R/WTC/SS)

LP_UART_RXFIFO_TOUT_INT_RAW UART_RXFIFO_TOUT_INT 的原始中断状态。(R/WTC/SS)

LP_UART_SW_XON_INT_RAW LP_UART_SW_XON_INT 的原始中断状态。(R/WTC/SS)

LP_UART_SW_XOFF_INT_RAW LP_UART_SW_XOFF_INT 的原始中断状态。(R/WTC/SS)

LP_UART_GLITCH_DET_INT_RAW LP_UART_GLITCH_DET_INT 的原始中断状态。(R/WTC/SS)

LP_UART_TX_BRK_DONE_INT_RAW UART_TX_BRK_DONE_INT 的原始中断状态。(R/WTC/SS)

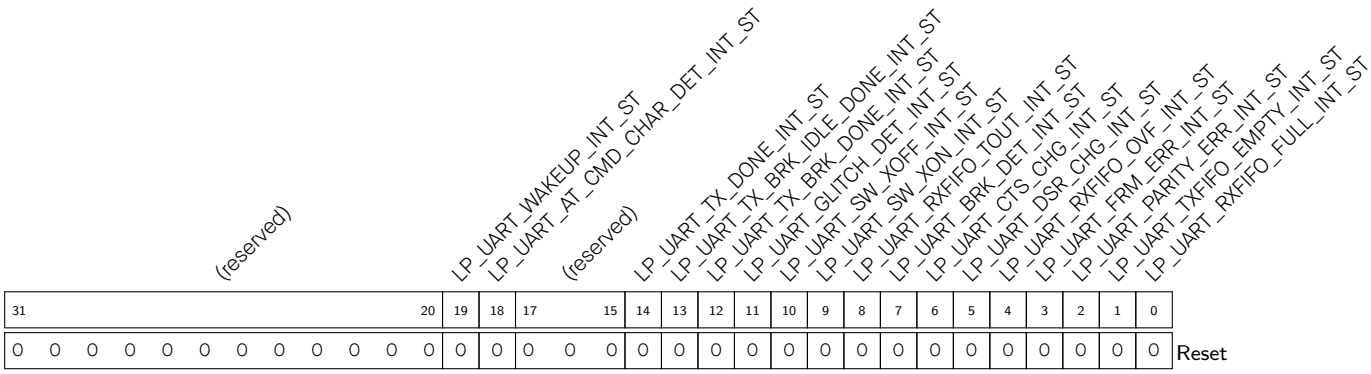
LP_UART_TX_BRK_IDLE_DONE_INT_RAW LP_UART_TX_BRK_IDLE_DONE_INT 的原始中断状态。(R/WTC/SS)

LP_UART_TX_DONE_INT_RAW LP_UART_TX_DONE_INT 的原始中断状态。(R/WTC/SS)

LP_UART_AT_CMD_CHAR_DET_INT_RAW LP_UART_AT_CMD_CHAR_DET_INT 的原始中断状态。(R/WTC/SS)

LP_UART_WAKEUP_INT_RAW LP_UART_WAKEUP_INT 的原始中断状态。(R/WTC/SS)

Register 21.41. LP_UART_INT_ST_REG (0x0008)



LP_UART_RXFIFO_FULL_INT_ST LP_UART_RXFIFO_FULL_INT 的屏蔽中断状态。(RO)

LP_UART_TXFIFO_EMPTY_INT_ST LP_UART_TXFIFO_EMPTY_INT 的屏蔽中断状态。(RO)

LP_UART_PARITY_ERR_INT_ST LP_UART_PARITY_ERR_INT 的屏蔽中断状态。(RO)

LP_UART_FRM_ERR_INT_ST LP_UART_FRM_ERR_INT 的屏蔽中断状态。(RO)

LP_UART_RXFIFO_OVF_INT_ST LP_UART_RXFIFO_OVF_INT 的屏蔽中断状态。(RO)

LP_UART_DSR_CHG_INT_ST LP_UART_DSR_CHG_INT 的屏蔽中断状态。(RO)

LP_UART_CTS_CHG_INT_ST LP_UART_CTS_CHG_INT 的屏蔽中断状态。(RO)

LP_UART_BRK_DET_INT_ST LP_UART_BRK_DET_INT 的屏蔽中断状态。(RO)

LP_UART_RXFIFO_TOUT_INT_ST LP_UART_RXFIFO_TOUT_INT 的屏蔽中断状态。(RO)

LP_UART_SW_XON_INT_ST LP_UART_SW_XON_INT 的屏蔽中断状态。(RO)

LP_UART_SW_XOFF_INT_ST LP_UART_SW_XOFF_INT 的屏蔽中断状态。(RO)

LP_UART_GLITCH_DET_INT_ST LP_UART_GLITCH_DET_INT 的屏蔽中断状态。(RO)

LP_UART_TX_BRK_DONE_INT_ST LP_UART_TX_BRK_DONE_INT 的屏蔽中断状态。(RO)

LP_UART_TX_BRK_IDLE_DONE_INT_ST LP_UART_TX_BRK_IDLE_DONE_INT 的屏蔽中断状态。(RO)

LP_UART_TX_DONE_INT_ST LP_UART_TX_DONE_INT 的屏蔽中断状态。(RO)

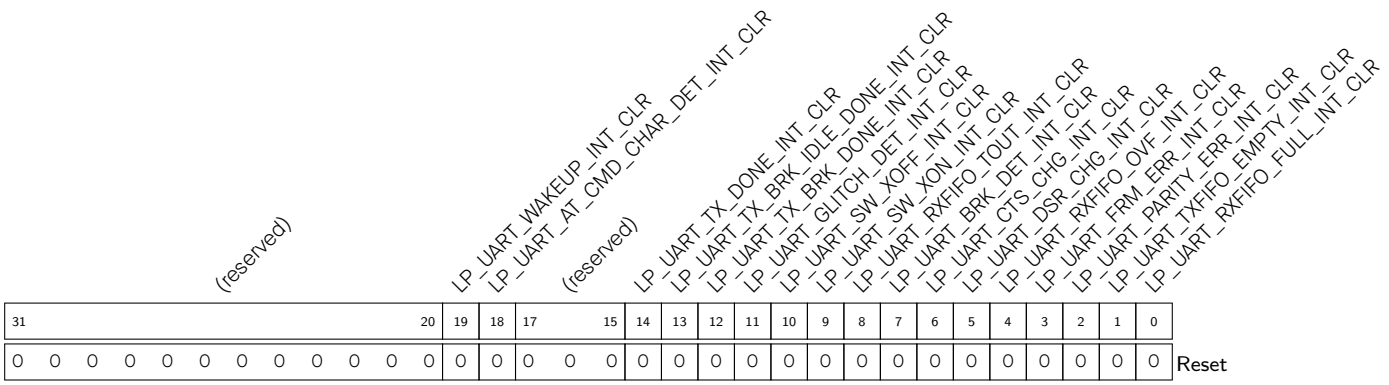
LP_UART_AT_CMD_CHAR_DET_INT_ST LP_UART_AT_CMD_CHAR_DET_INT 的屏蔽中断状态。(RO)

LP_UART_WAKEUP_INT_ST LP_UART_WAKEUP_INT 的屏蔽中断状态。(RO)

Register 21.42. LP_UART_INT_ENA_REG (0x000C)


(reserved)
LP_UART_WAKEUP_INT_ENA
LP_UART_AT_CMD_CHAR_DET_INT_ENA
(reserved)
LP_UART_TX_DONE_INT_ENA
LP_UART_TX_BRK_IDLE_DONE_INT_ENA
LP_UART_TX_BRK_DONE_INT_ENA
LP_UART_GLITCH_DET_INT_ENA
LP_UART_SW_XOFF_INT_ENA
LP_UART_SW_XON_INT_ENA
LP_UART_RXFIFO_TOUT_INT_ENA
LP_UART_BRK_DET_INT_ENA
LP_UART_CTS_CHG_INT_ENA
LP_UART_DSR_CHG_INT_ENA
LP_UART_RXFIFO_OVF_INT_ENA
LP_UART_FRM_ERR_INT_ENA
LP_UART_PARITY_ERR_INT_ENA
LP_UART_TXFIFO_EMPTY_INT_ENA
LP_UART_RXFIFO_FULL_INT_ENA
31 20 19 18 17 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0 0 0

Register 21.43. LP_UART_INT_CLR_REG (0x0010)



LP_UART_RXFIFO_FULL_INT_CLR 写 1 清除 LP_UART_RXFIFO_FULL_INT。(WT)

LP_UART_TXFIFO_EMPTY_INT_CLR 写 1 清除 LP_UART_TXFIFO_EMPTY_INT。(WT)

LP_UART_PARITY_ERR_INT_CLR 写 1 清除 LP_UART_PARITY_ERR_INT。(WT)

LP_UART_FRM_ERR_INT_CLR 写 1 清除 LP_UART_FRM_ERR_INT。(WT)

LP_UART_RXFIFO_OVF_INT_CLR 写 1 清除 LP_UART_RXFIFO_OVF_INT。(WT)

LP_UART_DSR_CHG_INT_CLR 写 1 清除 LP_UART_DSR_CHG_INT。(WT)

LP_UART_CTS_CHG_INT_CLR 写 1 清除 LP_UART_CTS_CHG_INT。(WT)

LP_UART_BRK_DET_INT_CLR 写 1 清除 LP_UART_BRK_DET_INT。(WT)

LP_UART_RXFIFO_TOUT_INT_CLR 写 1 清除 LP_UART_RXFIFO_TOUT_INT。(WT)

LP_UART_SW_XON_INT_CLR 写 1 清除 LP_UART_SW_XON_INT。(WT)

LP_UART_SW_XOFF_INT_CLR 写 1 清除 LP_UART_SW_XOFF_INT。(WT)

LP_UART_GLITCH_DET_INT_CLR 写 1 清除 LP_UART_GLITCH_DET_INT。(WT)

LP_UART_TX_BRK_DONE_INT_CLR 写 1 清除 LP_UART_TX_BRK_DONE_INT。(WT)

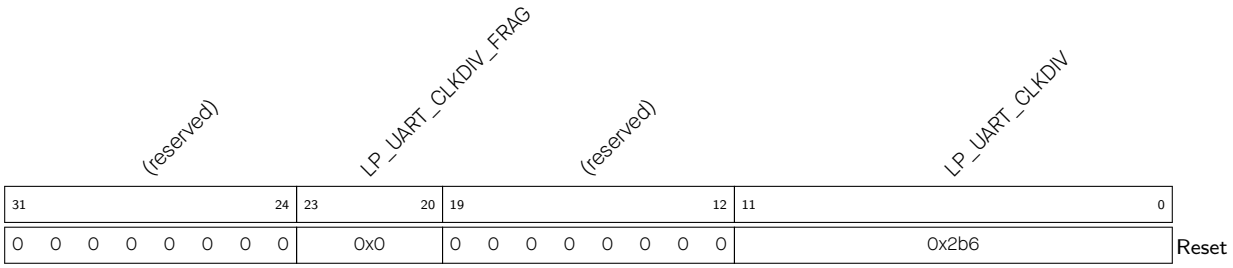
LP_UART_TX_BRK_IDLE_DONE_INT_CLR 写 1 清除 LP_UART_TX_BRK_IDLE_DONE_INT。(WT)

LP_UART_TX_DONE_INT_CLR 写 1 清除 LP_UART_TX_DONE_INT。(WT)

LP_UART_AT_CMD_CHAR_DET_INT_CLR 写 1 清除 LP_UART_AT_CMD_CHAR_DET_INT。(WT)

LP_UART_WAKEUP_INT_CLR 写 1 清除 LP_UART_WAKEUP_INT。(WT)

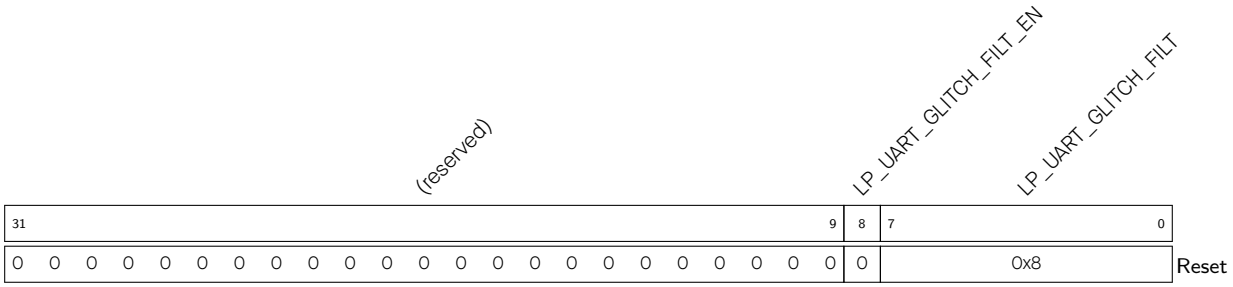
Register 21.44. LP_UART_CLKDIV_SYNC_REG (0x0014)



LP_UART_CLKDIV 配置波特率分频系数的整数部分。(R/W)

LP_UART_CLKDIV_FRAG 配置波特率分频系数的小数部分。(R/W)

Register 21.45. LP_UART_RX_FILT_REG (0x0018)



LP_UART_GLITCH_FILT 配置滤波长度。

单位：UART Core 时钟周期。

宽度小于该数值的输入脉冲会被忽略。(R/W)

LP_UART_GLITCH_FILT_EN 配置是否开启接收信号滤波器。

0：关闭

1：开启

(R/W)

Register 21.46. LP_UART_CONF0_SYNC_REG (0x0020)


(reserved)
LP_UART_TXFIFO_RST
LP_UART_RXFIFO_RST
LP_UART_SW_RTS
LP_UART_MEM_CLK_EN
(reserved)
LP_UART_ERR_WR_MASK
LP_UART_DIS_RX_DAT_OVF
LP_UART_TXD_INV
LP_UART_RXD_INV
(reserved)
LP_UART_TX_FLOW_EN
LP_UART_LOOPBACK
(reserved)
LP_UART_TXD_BRK
LP_UART_STOP_BIT_NUM
LP_UART_BIT_NUM
LP_UART_PARITY_EN
LP_UART_PARITY
31 24 23 22 21 20 19 18 17 16 15 14 13 12 11 7 6 5 4 3 2 1 0
0 0 0

Register 21.46. LP_UART_CONF0_SYNC_REG (0x0020)

接上页...

LP_UART_RXD_INV 配置是否翻转 LP UART RXD 信号电平。

0：不翻转

1：翻转

(R/W)

LP_UART_TXD_INV 配置是否翻转 LP UART TXD 信号电平。

0：不翻转

1：翻转

(R/W)

LP_UART_DIS_RX_DAT_OVF 配置是否关闭 LP UART 接收器数据溢出检测。

0：开启

1：关闭

(R/W)

LP_UART_ERR_WR_MASK 配置接收数据有错误时，是否扔将其存储至 FIFO。

0：存储

1：不存储

(R/W)

LP_UART_MEM_CLK_EN 配置是否开启 LP UART 存储器门控。

0：关闭

1：开启

(R/W)

LP_UART_SW_RTS 配置软件流控使用的 RTS 信号。

0: 禁止发送端发送数据

1: 允许发送端发送数据

(R/W)

LP_UART_RXFIFO_RST 配置是否复位 LP UART RX FIFO。

0：不复位

1：复位

(R/W)

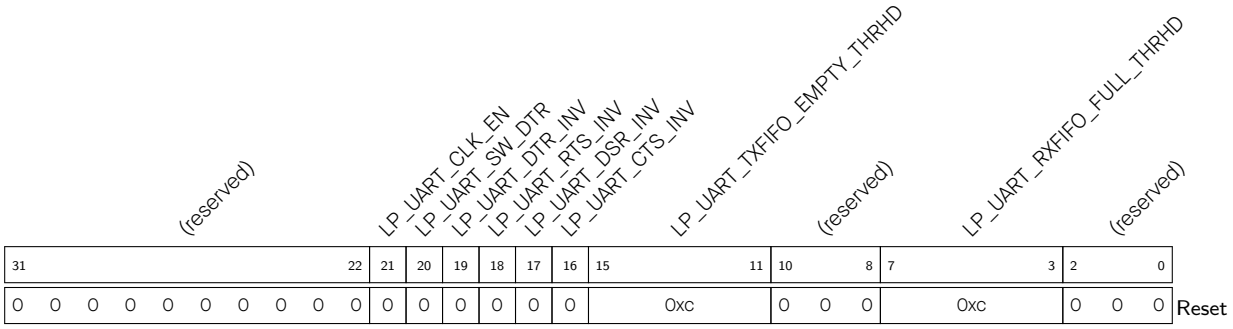
LP_UART_TXFIFO_RST 配置是否复位 LP UART TX FIFO。

0：不复位

1：复位

(R/W)

Register 21.47. LP_UART_CONF1_REG (0x0024)



LP_UART_RXFIFO_FULL_THRHD 配置 RX FIFO 为满的阈值。

单位：字节。(R/W)

LP_UART_TXFIFO_EMPTY_THRHD 配置 TX FIFO 为空的阈值。

单位：字节。(R/W)

LP_UART_CTS_INV 配置是否翻转 LP UART CTS 信号电平。

0：不翻转

1：翻转

(R/W)

LP_UART_DSR_INV 配置是否翻转 LP UART DSR 信号电平。

0：不翻转

1：翻转

(R/W)

LP_UART_RTS_INV 配置是否翻转 LP UART RTS 信号电平。

0：不翻转

1：翻转

(R/W)

LP_UART_DTR_INV 配置是否翻转 LP UART DTR 信号电平。

0：不翻转

1：翻转

(R/W)

LP_UART_SW_DTR 配置软件流控使用的 DTR 信号。

0: 发送数据未准备完毕，处于不可用状态

1: 发送数据准备完毕，处于可用状态

(R/W)

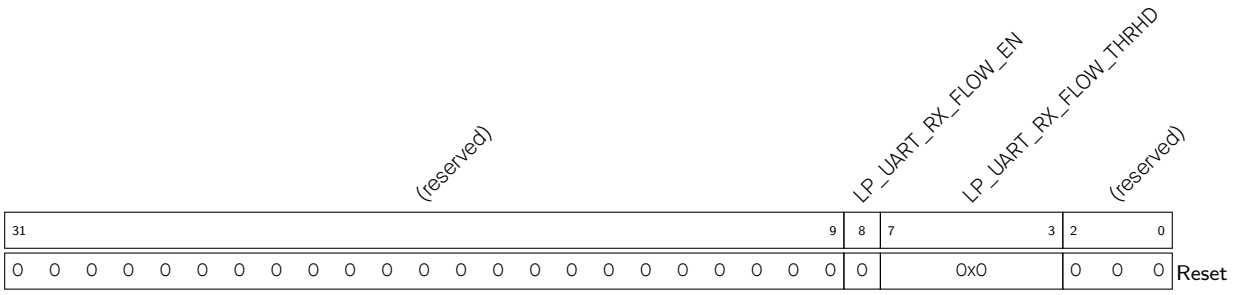
LP_UART_CLK_EN 配置时钟门控。

0：仅在应用写寄存器时开启时钟

1：一直强制为寄存器开启时钟

(R/W)

Register 21.48. LP_UART_HWFC_CONF_SYNC_REG (0x002C)



LP_UART_RX_FLOW_THRHD 配置使用硬件流控时接收数据的最大数。

单位：字节。(R/W)

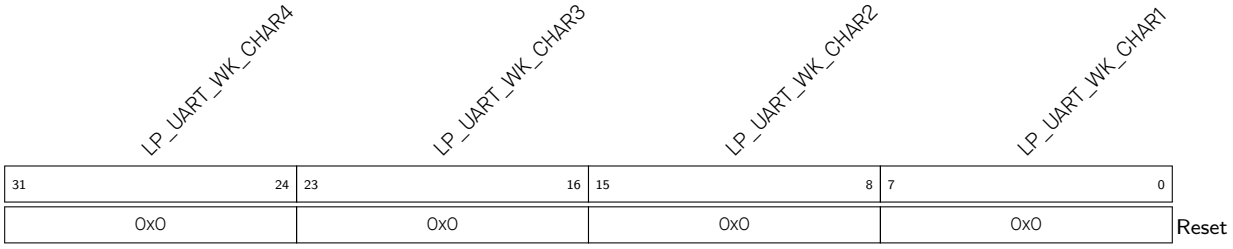
LP_UART_RX_FLOW_EN 配置是否开启 LP UART 接收器。

0：关闭

1：开启

(R/W)

Register 21.49. LP_UART_SLEEP_CONF0_REG (0x0030)



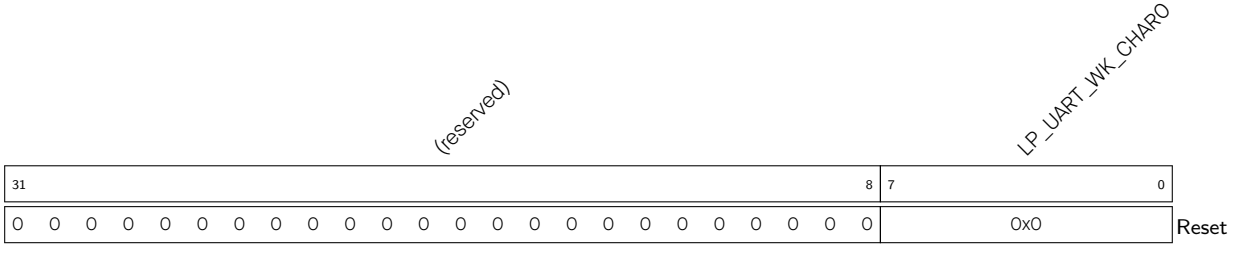
LP_UART_WK_CHAR1 配置唤醒芯片的特定字符 1。(R/W)

LP_UART_WK_CHAR2 配置唤醒芯片的特定字符 2。(R/W)

LP_UART_WK_CHAR3 配置唤醒芯片的特定字符 3。(R/W)

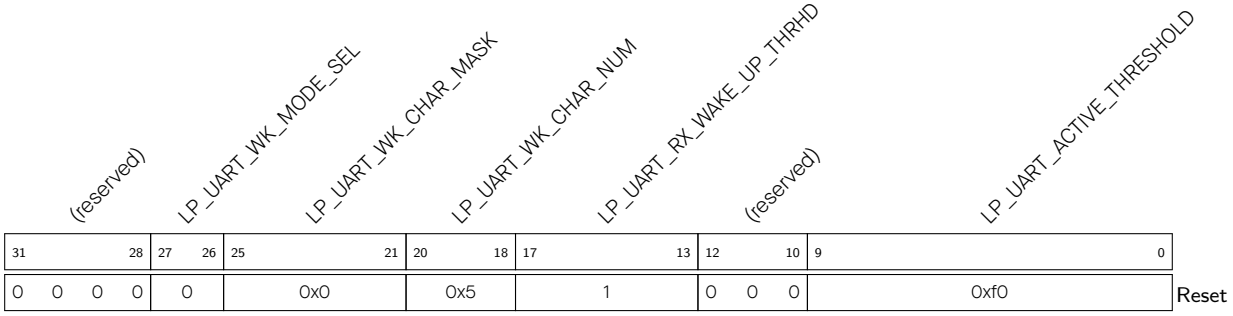
LP_UART_WK_CHAR4 配置唤醒芯片的特定字符 4。(R/W)

Register 21.50. LP_UART_SLEEP_CONF1_REG (0x0034)



LP_UART_WK_CHAR0 配置唤醒芯片的特定字符 0。(R/W)

Register 21.51. LP_UART_SLEEP_CONF2_REG (0x0038)



LP_UART_ACTIVE_THRESHOLD 配置唤醒模式 0 唤醒芯片所需的 RXD 沿变化次数。(R/W)

LP_UART_RX_WAKE_UP_THRHD 配置唤醒模式 1 唤醒芯片所需接收的数据量。

单位：字节。(R/W)

LP_UART_WK_CHAR_NUM 配置选取多少个唤醒字符。(R/W)

LP_UART_WK_CHAR_MASK 配置是否屏蔽唤醒字符。

0：不屏蔽

1：屏蔽

(R/W)

LP_UART_WK_MODE_SEL 配置选取哪个唤醒模式。

0：唤醒模式 0

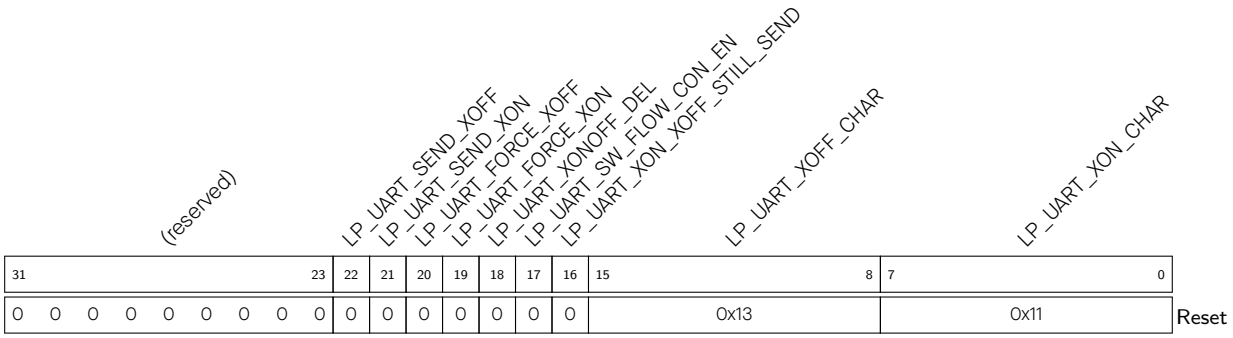
1：唤醒模式 1

2：唤醒模式 2

3：唤醒模式 3

(R/W)

Register 21.52. LP_UART_SWFC_CONF0_SYNC_REG (0x003C)



LP_UART_XON_CHAR 配置 XON 流控字符。(R/W)

LP_UART_XOFF_CHAR 配置 XOFF 流控字符。(R/W)

LP_UART_XON_XOFF_STILL_SEND 配置 LP UART 发送器关闭时，是否仍能发送 XON 或 XOFF 字符。

0：不能发送

1：可以发送

(R/W)

LP_UART_SW_FLOW_CON_EN 配置是否开启软件流控。

0：关闭

1：开启

(R/W)

LP_UART_XONOFF_DEL 配置是否移除接收数据中的流控字符。

0：不移除

1：移除

(R/W)

LP_UART_FORCE_XON 配置是否让发送器继续发送数据。

0：不发送

1：继续发送

(R/W)

LP_UART_FORCE_XOFF 配置是否让发送器停止发送数据。

0：不停止

1：停止

(R/W)

LP_UART_SEND_XON 配置是否发送 XON 字符。

0：不发送

1：发送

(R/W/SS/SC)

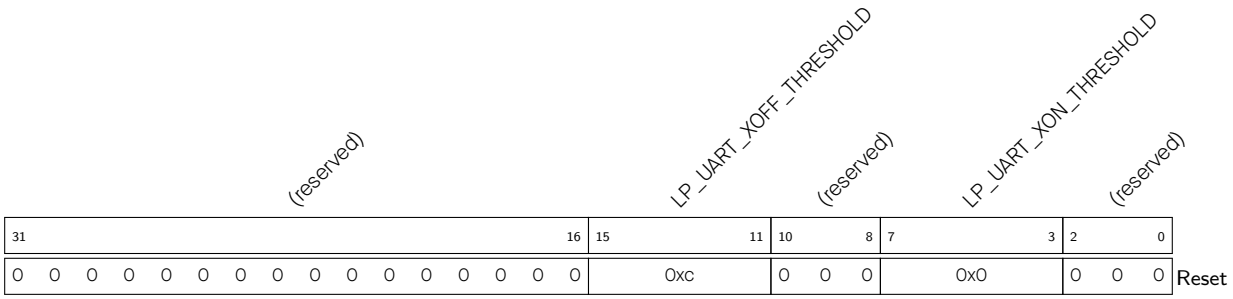
LP_UART_SEND_XOFF 配置是否发送 XOFF 字符。

0：不发送

1：发送

(R/W/SS/SC)

Register 21.53. LP_UART_SWFC_CONF1_REG (0x0040)



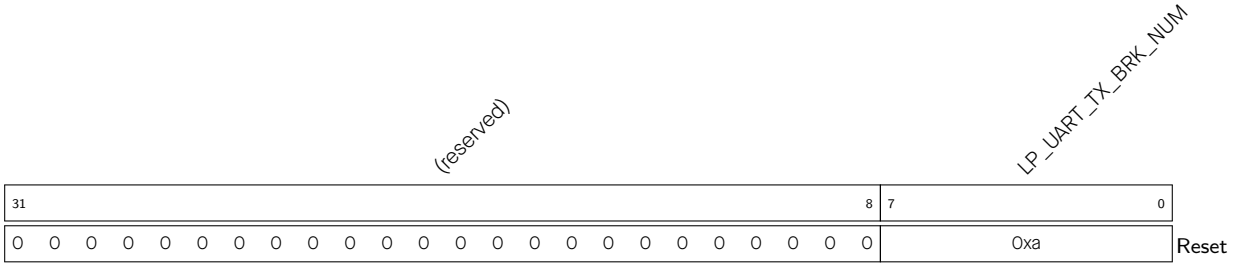
LP_UART_XON_THRESHOLD 配置软件流控时，发送 XON 字符所需的 RX FIFO 数据阈值。

单位：字节。(R/W)

LP_UART_XOFF_THRESHOLD 配置软件流控时，发送 XOFF 字符所需的 RX FIFO 数据阈值。

单位：字节。(R/W)

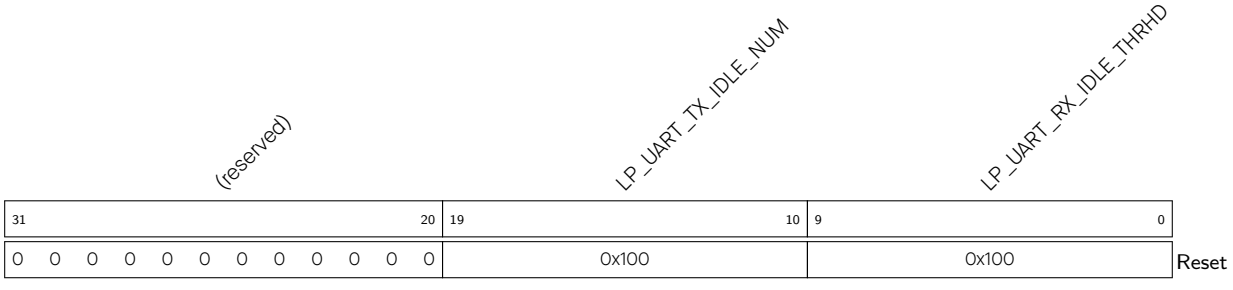
Register 21.54. LP_UART_TXBRK_CONF_SYNC_REG (0x0044)



LP_UART_TX_BRK_NUM 配置完成数据发送后待发 NULL 字符的数量。

仅在 LP_UART_TXD_BRK 为 1 时有效。(R/W)

Register 21.55. LP_UART_IDLE_CONF_SYNC_REG (0x0048)



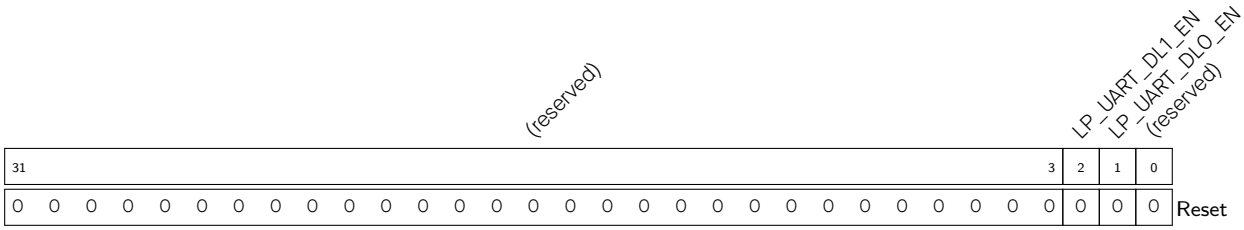
LP_UART_RX_IDLE_THRHD 配置接收一字节数据时间过长、产生帧结束信号的阈值。

单位：位时间（传输一位所需的时间）。(R/W)

LP_UART_TX_IDLE_NUM 配置两次数据传输的间隔时间。

单位：位时间（传输一位所需的时间）。(R/W)

Register 21.56. LP_UART_DELAY_CONF_SYNC_REG (0x004C)



LP_UART_DL0_EN 配置是否在起始位之前增加一位停止位。

0: 不增加

1: 增加

(R/W)

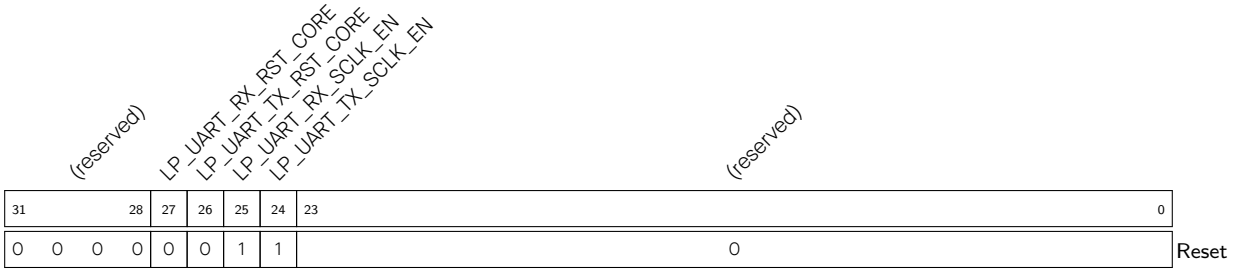
LP_UART_DL1_EN 配置是否在停止位之后增加一位停止位。

0: 不增加

1: 增加

(R/W)

Register 21.57. LP_UART_CLK_CONF_REG (0x0088)



LP_UART_TX_SCLK_EN 配置是否开启 LP UART TX 时钟。

0：关闭

1：开启

(R/W)

LP_UART_RX_SCLK_EN 配置是否开启 LP UART RX 时钟。

0：关闭

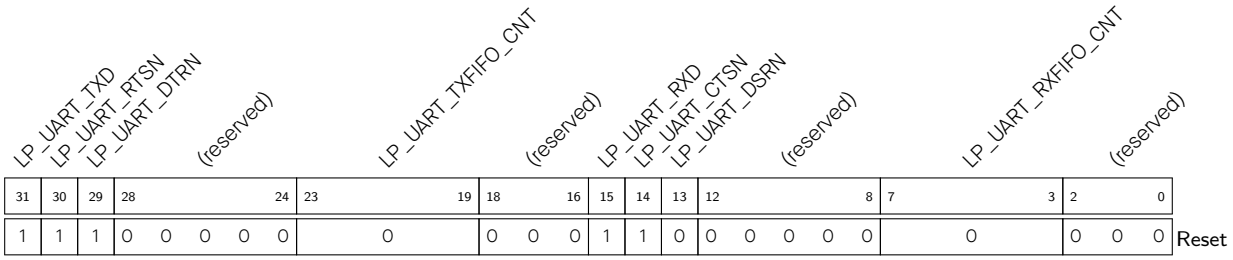
1：开启

(R/W)

LP_UART_TX_RST_CORE 向此位先写 1 后写 0，复位 LP UART TX。(R/W)

LP_UART_RX_RST_CORE 向此位先写 1 后写 0，复位 LP UART RX。(R/W)

Register 21.58. LP_UART_STATUS_REG (0x001C)



LP_UART_RXFIFO_CNT 表示 RX FIFO 中有效数据量。

单位：字节。(RO)

LP_UART_DSRN 表示内部 LP UART DSR 信号的电平值。(RO)

LP_UART_CTSN 表示内部 LP UART CTS 信号的电平值。(RO)

LP_UART_RXD 表示内部 LP UART RXD 信号的电平值。(RO)

LP_UART_TXFIFO_CNT 表示 TX FIFO 中的数据量。

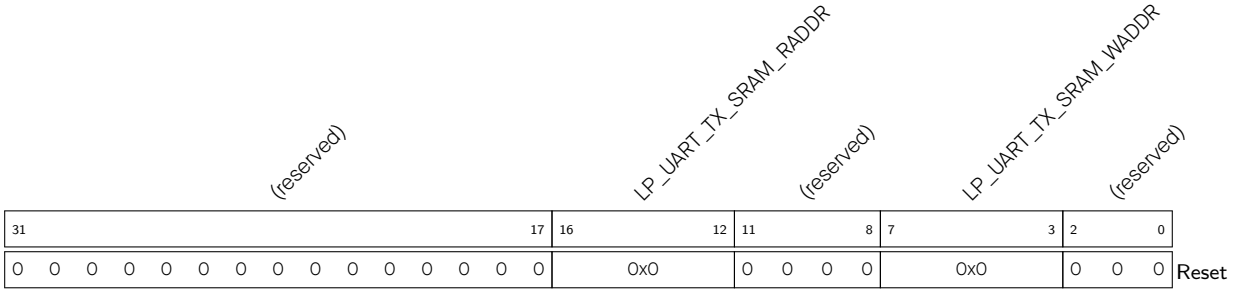
单位：字节。(RO)

LP_UART_DTRN 表示内部 LP UART DTR 信号的电平。(RO)

LP_UART_RTSN 表示内部 LP UART RTS 信号的电平。(RO)

LP_UART_TXD 表示内部 LP UART TXD 信号的电平。(RO)

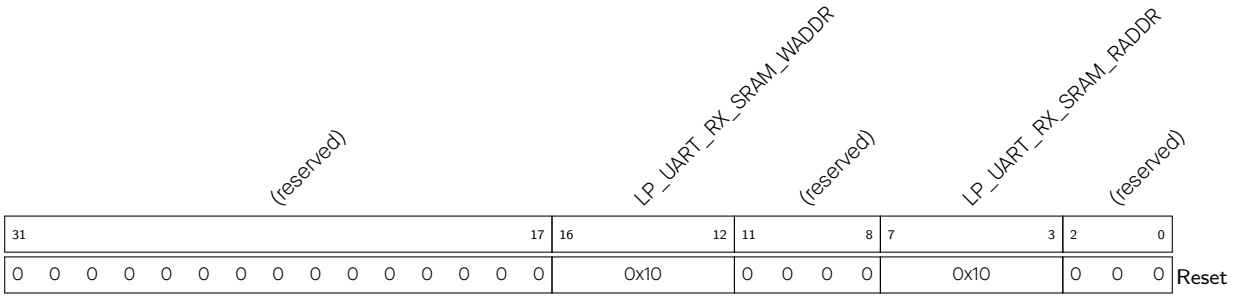
Register 21.59. LP_UART_MEM_TX_STATUS_REG (0x0068)



LP_UART_TX_SRAM_WADDR 表示写 TX FIFO 的偏移地址。(RO)

LP_UART_TX_SRAM_RADDR 表示读 TX FIFO 的偏移地址。(RO)

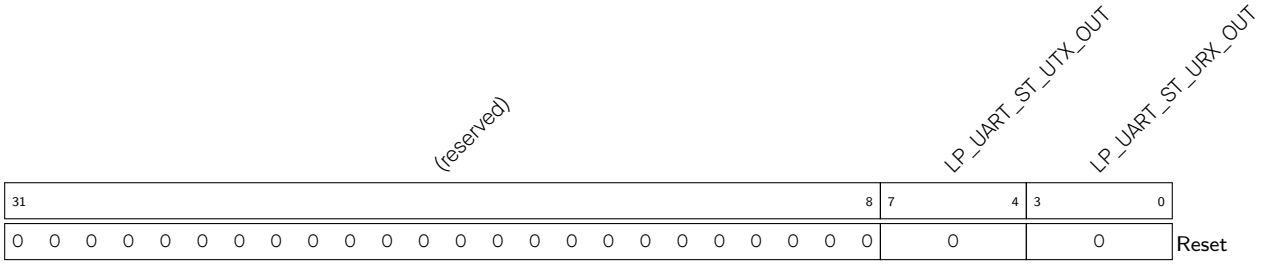
Register 21.60. LP_UART_MEM_RX_STATUS_REG (0x006C)



LP_UART_RX_SRAM_RADDR 表示读 RX FIFO 的偏移地址。(RO)

LP_UART_RX_SRAM_WADDR 表示写 RX FIFO 的偏移地址。(RO)

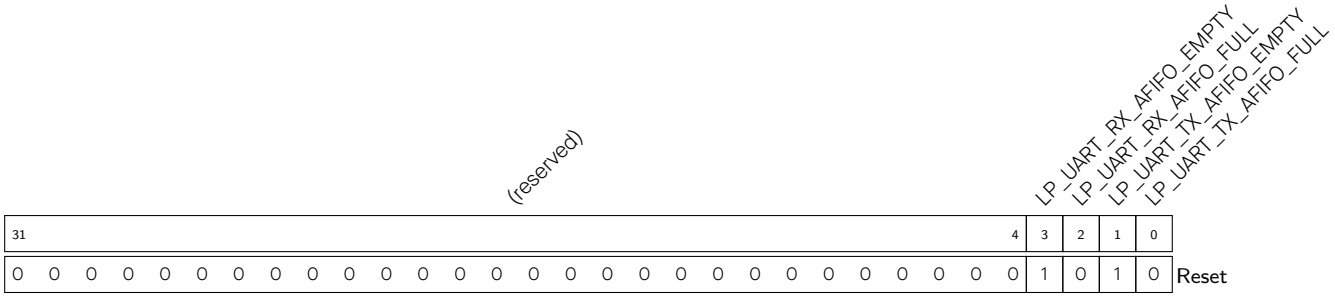
Register 21.61. LP_UART_FSM_STATUS_REG (0x0070)



LP_UART_ST_URX_OUT 表示 LP UART 接收器的状态。(RO)

LP_UART_ST_UTX_OUT 表示 LP UART 发送器的状态。(RO)

Register 21.62. LP_UART_AFIFO_STATUS_REG (0x0090)



LP_UART_TX_AFIFO_FULL 表示 APB TX 异步 FIFO 是否已满。

0：未满

1：已满

(RO)

LP_UART_TX_AFIFO_EMPTY 表示 APB TX 异步 FIFO 是否为空。

0：不为空

1：为空

(RO)

LP_UART_RX_AFIFO_FULL 表示 APB RX 异步 FIFO 是否已满。

0：未满

1：已满

(RO)

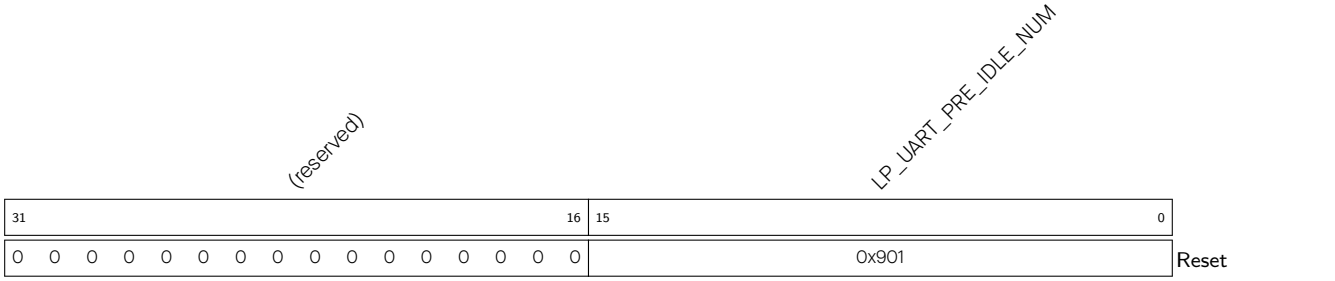
LP_UART_RX_AFIFO_EMPTY 表示 APB RX 异步 FIFO 是否为空。

0：不为空

1：为空

(RO)

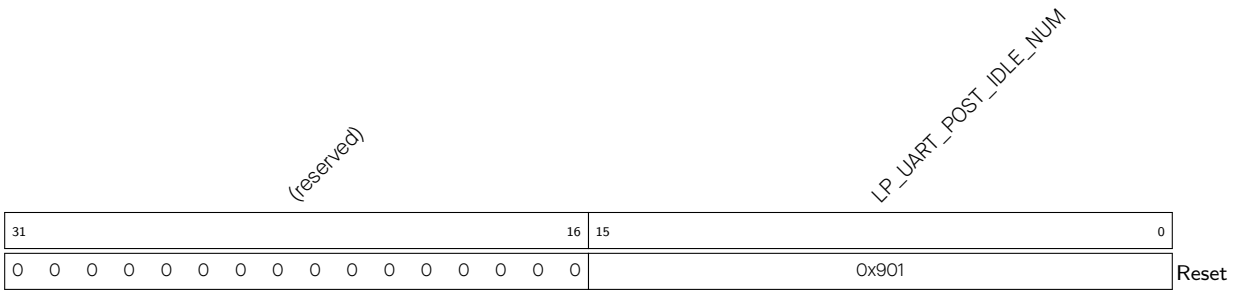
Register 21.63. LP_UART_AT_CMD_PRECNT_SYNC_REG (0x0050)



LP_UART_PRE_IDLE_NUM 配置接收器接收到第一个 AT_CMD 之前的空闲时间。

单位：位时间（传输一位所需的时间）。(R/W)

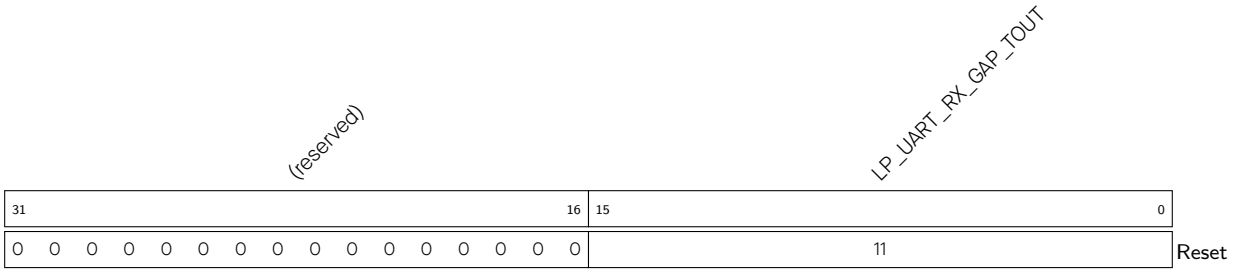
Register 21.64. LP_UART_AT_CMD_POSTCNT_SYNC_REG (0x0054)



LP_UART_POST_IDLE_NUM 配置最后一个 AT_CMD 和后续数据之间的间隔时间。

单位：位时间（传输一位所需的时间）。(R/W)

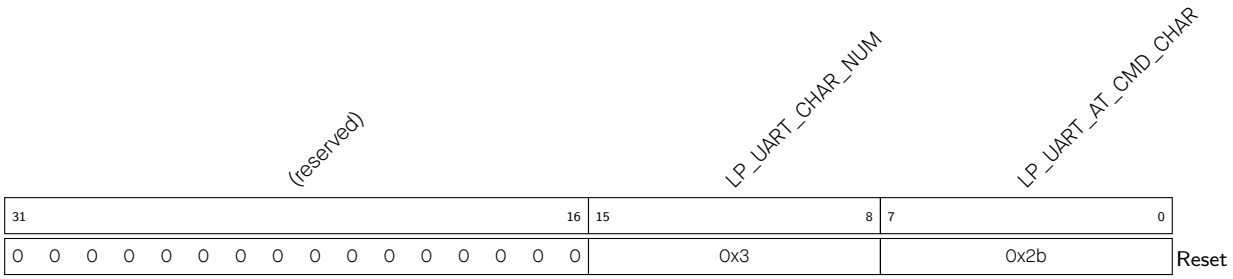
Register 21.65. LP_UART_AT_CMD_GAPTOUT_SYNC_REG (0x0058)



LP_UART_RX_GAP_TOUT 配置 AT_CMD 之间的间隔时间。

单位：位时间（传输一位所需的时间）。(R/W)

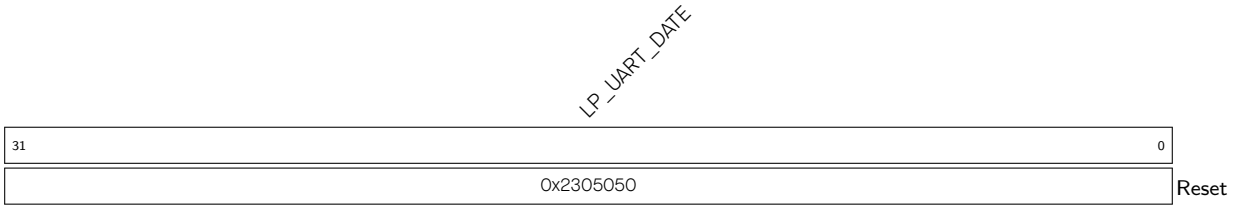
Register 21.66. LP_UART_AT_CMD_CHAR_SYNC_REG (0x005C)



LP_UART_AT_CMD_CHAR 配置 AT_CMD 字符。(R/W)

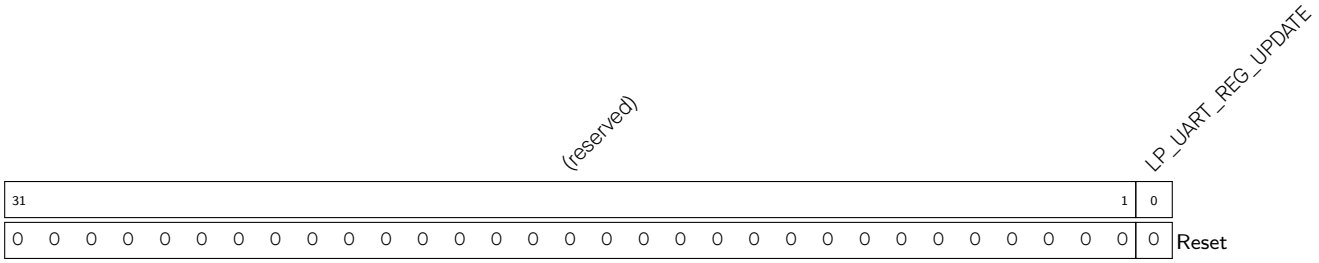
LP_UART_CHAR_NUM 配置接收器可连续接收的 AT_CMD 字符数量。(R/W)

Register 21.67. LP_UART_DATE_REG (0x008C)



LP_UART_DATE 版本控制寄存器。(R/W)

Register 21.68. LP_UART_REG_UPDATE_REG (0x0098)



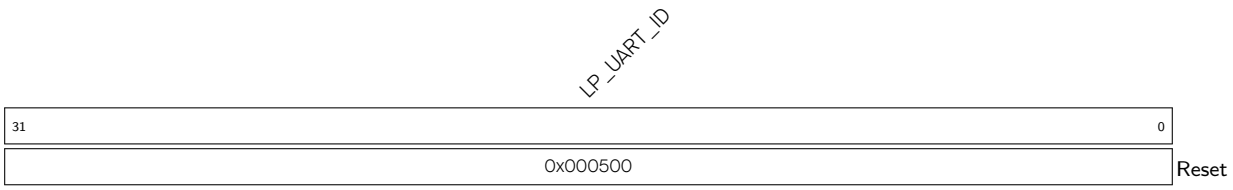
LP_UART_REG_UPDATE 配置是否同步寄存器

0：不同步

1：同步

(R/W/SC)

Register 21.69. LP_UART_ID_REG (0x009C)



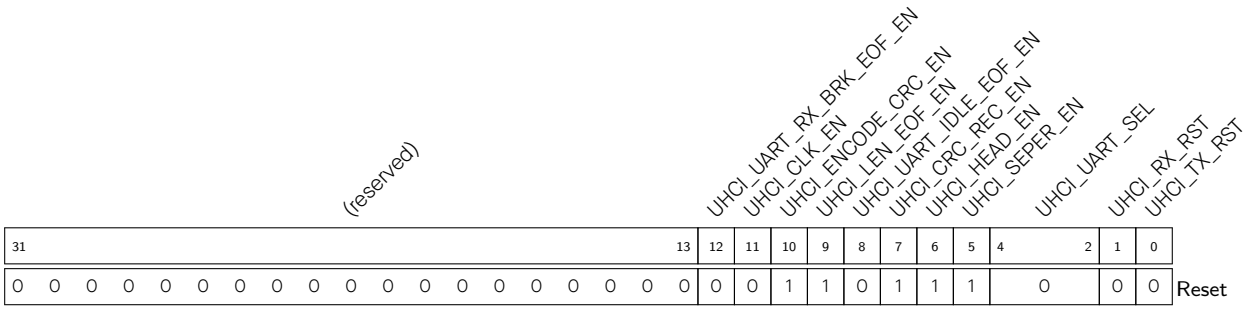
LP_UART_ID 配置 LP UART ID。(R/W)

21.8.3 UHCI 寄存器

本小节的所有地址均为相对于 UHCI 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 21.70. UHCI_CONF0_REG (0x0000)



UHCI_TX_RST 先写 1 再写 0 复位解码器状态机。(R/W)

UHCI_RX_RST 先写 1 再写 0 复位编码器状态机。(R/W)

UHCI_UART_SEL 从 UART 0 ~ 3 中选择其一连接到 UHCI。

0：选择 UART0

1：选择 UART1

2：选择 UART2

3：选择 UART3

4 ~ 7：无效值，没有 UART 连接到 UHCI

(R/W)

UHCI_SEPER_EN 配置是否使用特殊字符分隔数据帧。

0：不分隔

1：分隔

(R/W)

UHCI_HEAD_EN 配置是否用格式报头编码数据包。

0：不使用报头

1：使用报头

(R/W)

UHCI_CRC_REC_EN 配置 UHCI 是否开启接收 16 位 CRC。

0：关闭

1：开启

(R/W)

见下页...

Register 21.70. UHCI_CONF0_REG (0x0000)

接上页...

UHCI_UART_IDLE_EOF_EN 配置 UART 空闲时 UHCI 是否停止接收数据。

0：不停止

1：停止

(R/W)

UHCI_LEN_EOF_EN 配置 UHCI 解码器停止接收数据的条件。

0：接收到 0xC0 后停止

1：接收字节数达到指定值时停止。UHCI_HEAD_EN 为 1 时，指定值是 UCHI 数据包报头指定的数据长度。UHCI_HEAD_EN 为 0 时，指定值为配置值。

(R/W)

UHCI_ENCODE_CRC_EN 配置是否开启数据完整性检测，在数据末尾加 16 位 CCITT-CRC。

0：关闭

1：开启

(R/W)

UHCI_CLK_EN 配置时钟门控。

0：仅在应用写寄存器时开启时钟

1：一直强制为寄存器开启时钟

(R/W)

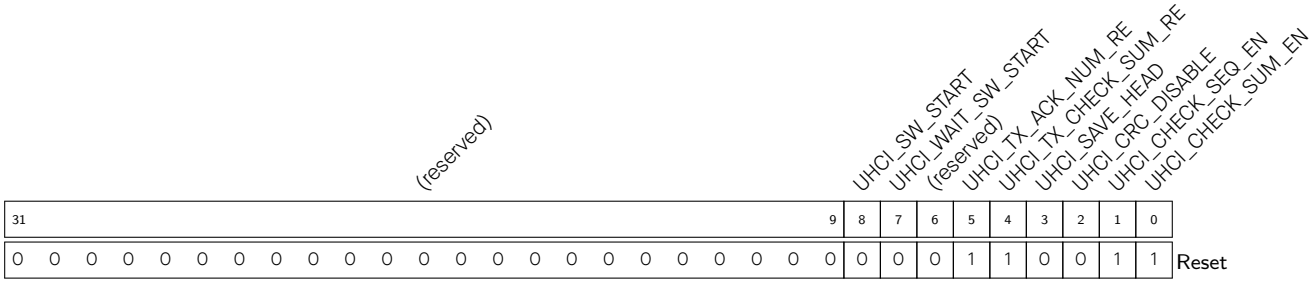
UHCI_UART_RX_BRK_EOF_EN 配置 UART 收到 NULL 帧后，UHCI 是否停止接收数据。

0：不停止

1：停止

(R/W)

Register 21.71. UHCI_CONF1_REG (0x0014)



UHCI_CHECK_SUM_EN 配置 UHCI 接收数据包时，是否开启报头校验和检查。

0：关闭

1：开启

(R/W)

UHCI_CHECK_SEQ_EN 配置 UHCI 接收数据包时，是否开启序列号检查。

0：关闭

1：开启

(R/W)

UHCI_CRC_DISABLE 配置是否开启 CRC 计算。

0：关闭

1：开启

仅在 UHCI 包中的数据完整性检测位为 1 时有效。

(R/W)

UHCI_SAVE_HEAD 配置 UHCI 接收数据包时，是否保存数据包报头。

0：不保存

1：保存

(R/W)

UHCI_TX_CHECK_SUM_RE 配置用校验和编码数据包。

0：不使用

1：使用

(R/W)

UHCI_TX_ACK_NUM_RE 配置准备发送可靠数据包时，是否用 ACK 编码该数据包。

0：不使用

1：使用

(R/W)

UHCI_WAIT_SW_START 配置是否将 UHCI 编码器的状态机调至 ST_SW_WAIT 状态。

0：否

1：是

(R/W)

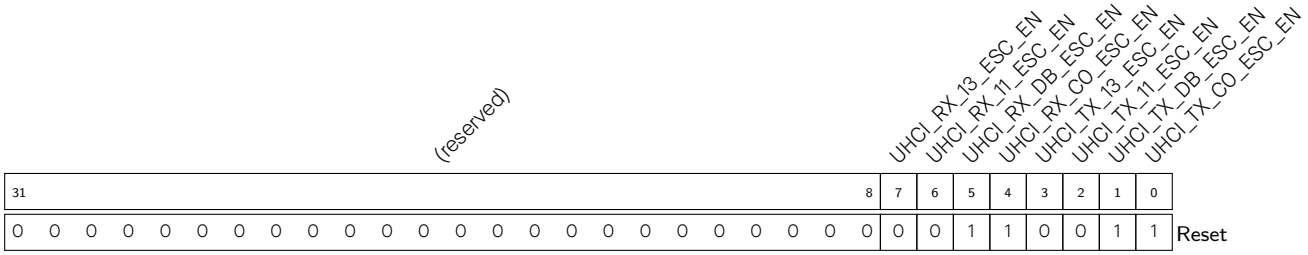
UHCI_SW_START 编码器的状态机为 ST_SW_WAIT 状态时，配置 UHCI 是否开始发送数据包。

0：不发送

1：开始发送

(R/W/SC)

Register 21.72. UHCI_ESCAPE_CONF_REG (0x0020)



UHCI_TX_C0_ESC_EN 配置 DMA 接收数据时，是否开启 0xC0 字符解码。

0：关闭

1：开启

(R/W)

UHCI_TX_DB_ESC_EN 配置 DMA 接收数据时，是否开启 0xDB 字符解码。

0：关闭

1：开启

(R/W)

UHCI_TX_11_ESC_EN 配置 DMA 接收数据时，是否开启流控字符 0x11 解码。

0：关闭

1：开启

(R/W)

UHCI_TX_13_ESC_EN 配置 DMA 接收数据时，是否开启流控字符 0x13 解码。

0：关闭

1：开启

(R/W)

UHCI_RX_C0_ESC_EN 配置 DMA 接收数据时，是否用特殊字符替换 0xC0。

0：不替换

1：替换

(R/W)

UHCI_RX_DB_ESC_EN 配置 DMA 接收数据时，是否用特殊字符替换 0xDB。

0：不替换

1：替换

(R/W)

UHCI_RX_11_ESC_EN 配置 DMA 接收数据时，是否用特殊字符替换流控字符 0x11。

0：不替换

1：替换

(R/W)

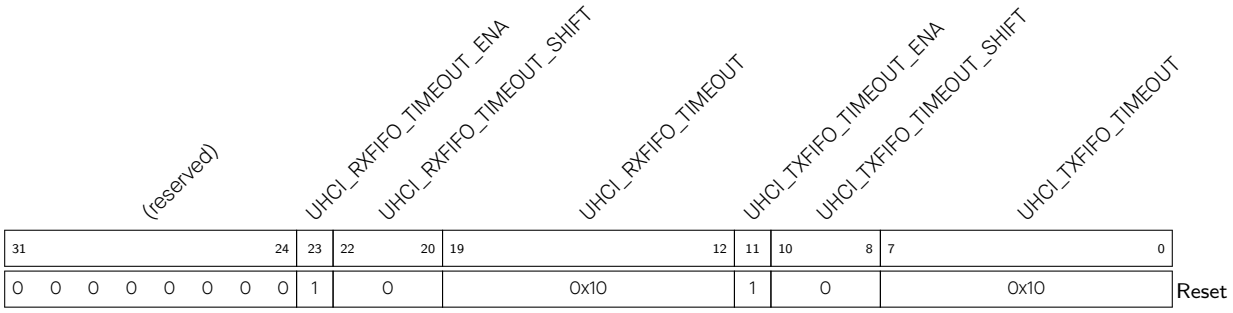
UHCI_RX_13_ESC_EN 配置 DMA 接收数据时，是否用特殊字符替换流控字符 0x13。

0：不替换

1：替换

(R/W)

Register 21.73. UHCI_HUNG_CONF_REG (0x0024)



UHCI_TXFIFO_TIMEOUT 配置 DMA 接收数据的超时值。

单位：毫秒。(R/W)

UHCI_TXFIFO_TIMEOUT_SHIFT 配置 TX FIFO 超时计数器的阈值上限。(R/W)

UHCI_TXFIFO_TIMEOUT_ENA 配置是否开启 TX FIFO 接收数据超时。

0：关闭

1：开启

(R/W)

UHCI_RXFIFO_TIMEOUT 配置 DMA 从 RAM 读取数据的超时值。单位：毫秒。(R/W)

UHCI_RXFIFO_TIMEOUT_SHIFT 配置 RX FIFO 超时计数器的阈值上限。(R/W)

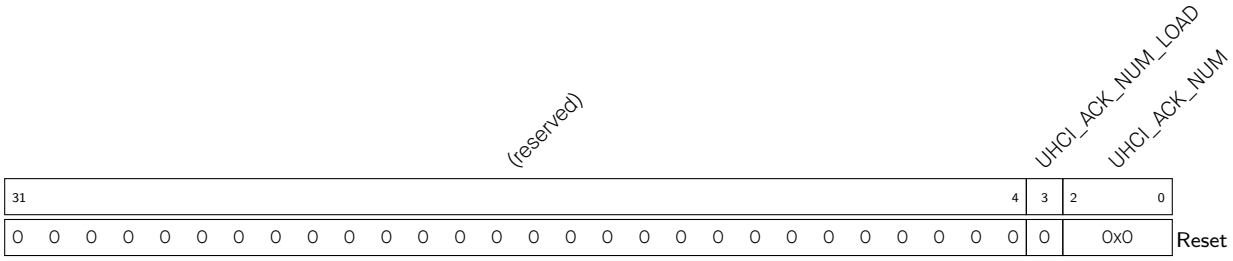
UHCI_RXFIFO_TIMEOUT_ENA 配置是否开启 DMA 发送数据超时。

0：关闭

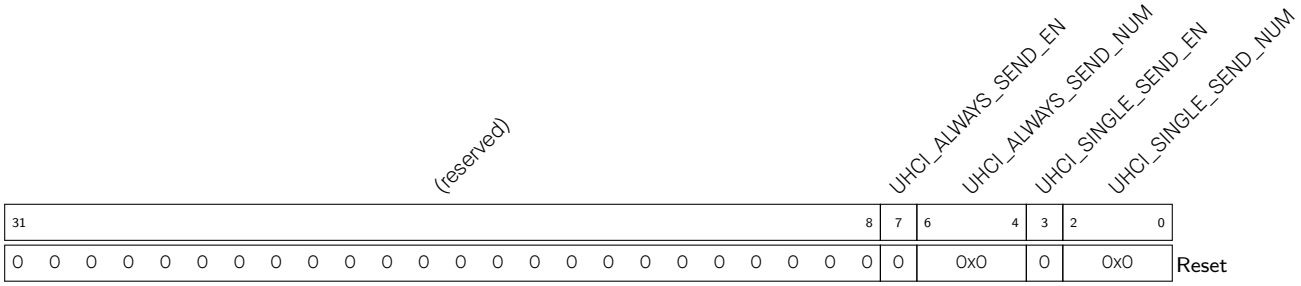
1：开启

(R/W)

Register 21.74. UHCI_ACK_NUM_REG (0x0028)


(reserved)
UHCI_ACK_NUM_LOAD
UHCI_ACK_NUM
31 4 3 2 0
0 0 0 0 0 0 0 0

Register 21.75. UHCI_QUICK_SENT_REG (0x0030)



UHCI_SINGLE_SEND_NUM 配置 single_send 模式下发送哪个寄存器中的数据。

0：Q0 寄存器

1：Q1 寄存器

2：Q2 寄存器

3：Q3 寄存器

4：Q4 寄存器

5：Q5 寄存器

6：Q6 寄存器

7：无效值，没有作用

(R/W)

UHCI_SINGLE_SEND_EN 配置是否开启 single_send 模式。

0：关闭

1：开启

(R/W/SC)

UHCI_ALWAYS_SEND_NUM 配置 always_send 模式下发送哪个寄存器中的数据。

0：Q0 寄存器

1：Q1 寄存器

2：Q2 寄存器

3：Q3 寄存器

4：Q4 寄存器

5：Q5 寄存器

6：Q6 寄存器

7：无效值，没有作用

(R/W)

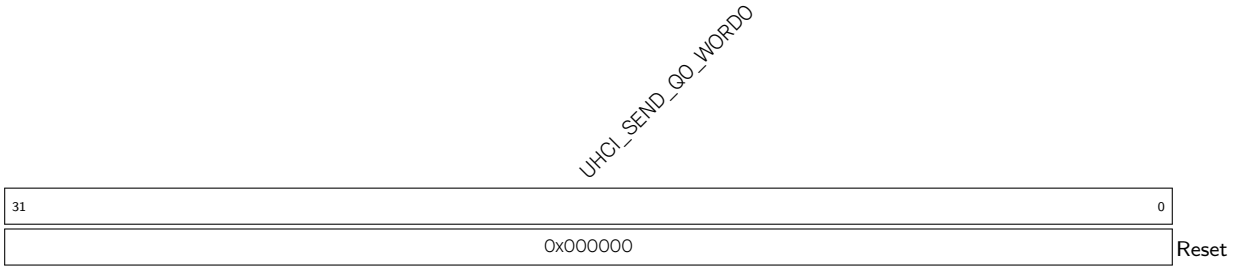
UHCI_ALWAYS_SEND_EN 配置是否开启 always_send 模式。

0：关闭

1：开启

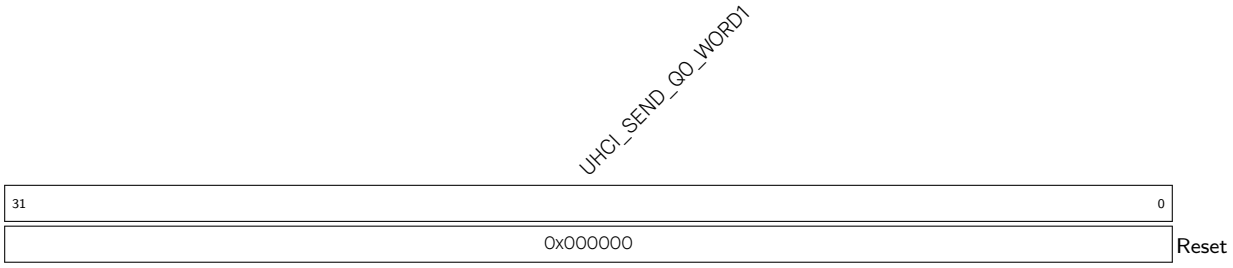
(R/W)

Register 21.76. UHCI_REG_Q0_WORD0_REG (0x0034)



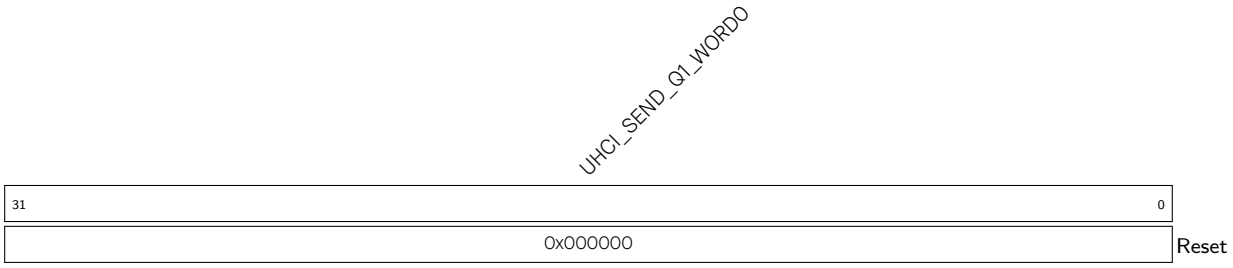
UHCI_SEND_Q0_WORD0 Q0 寄存器待发送数据。(R/W)

Register 21.77. UHCI_REG_Q0_WORD1_REG (0x0038)



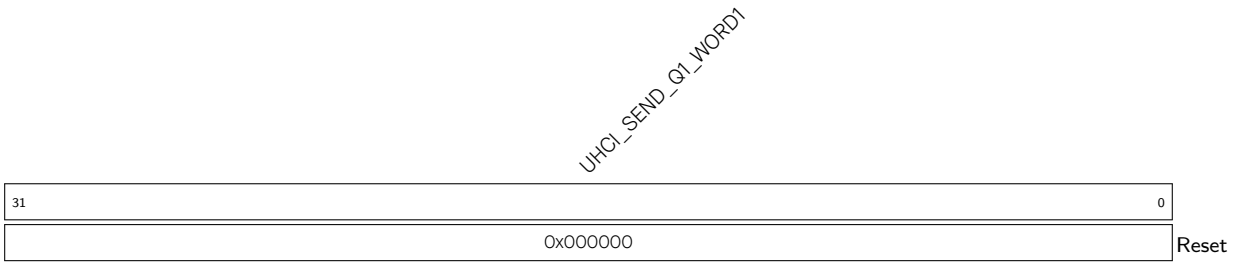
UHCI_SEND_Q0_WORD1 Q0 寄存器待发送数据。(R/W)

Register 21.78. UHCI_REG_Q1_WORD0_REG (0x003C)



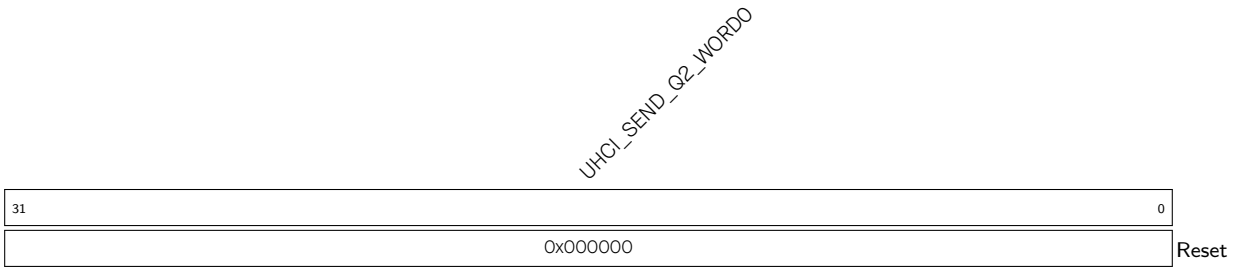
UHCI_SEND_Q1_WORD0 Q1 寄存器待发送数据。(R/W)

Register 21.79. UHCI_REG_Q1_WORD1_REG (0x0040)



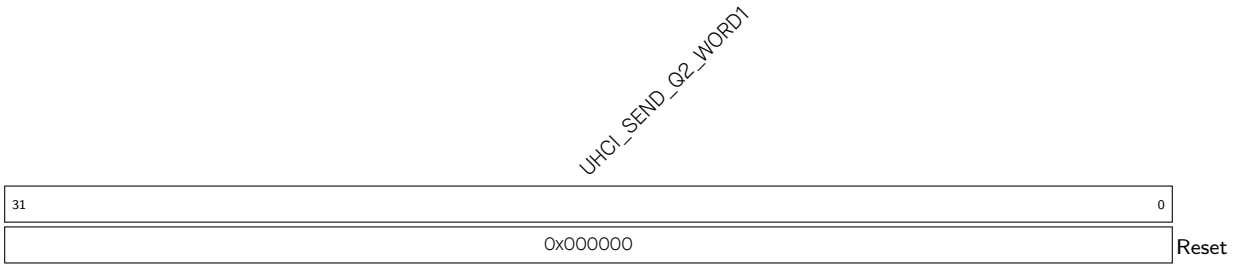
UHCI_SEND_Q1_WORD1 Q1 寄存器待发送数据。(R/W)

Register 21.80. UHCI_REG_Q2_WORD0_REG (0x0044)



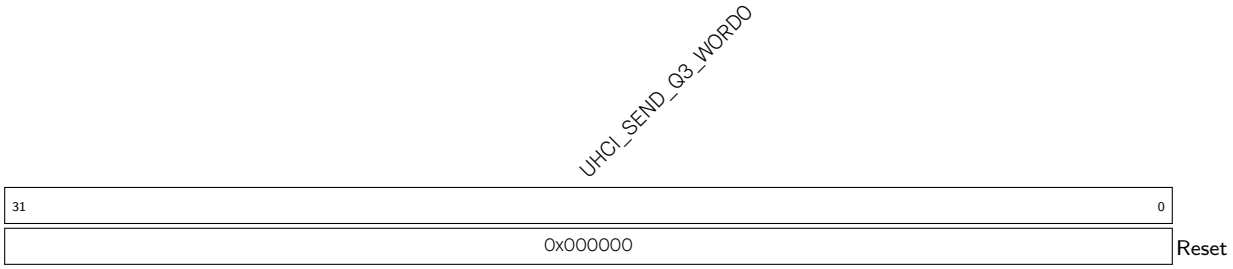
UHCI_SEND_Q2_WORD0 Q2 寄存器待发送数据。(R/W)

Register 21.81. UHCI_REG_Q2_WORD1_REG (0x0048)



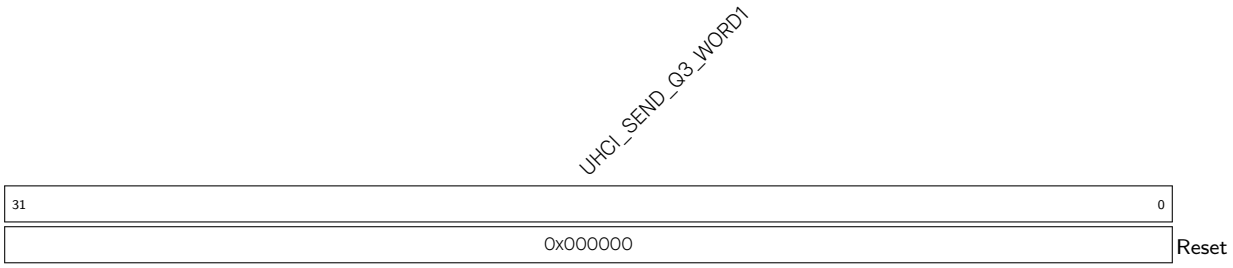
UHCI_SEND_Q2_WORD1 Q2 寄存器待发送数据。(R/W)

Register 21.82. UHCI_REG_Q3_WORD0_REG (0x004C)



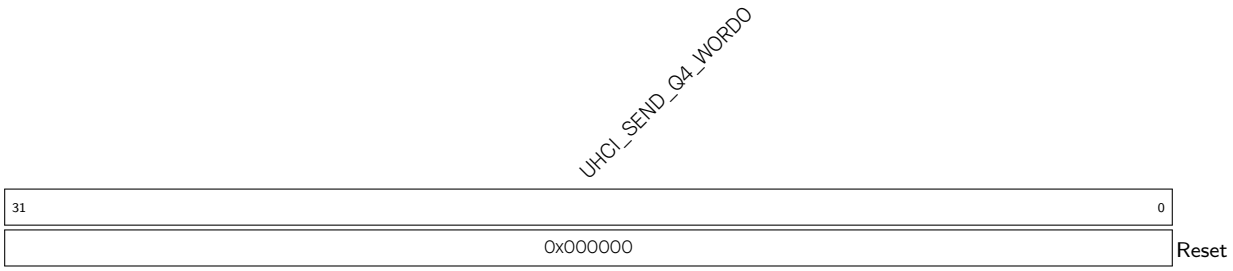
UHCI_SEND_Q3_WORD0 Q3 寄存器待发送数据。(R/W)

Register 21.83. UHCI_REG_Q3_WORD1_REG (0x0050)



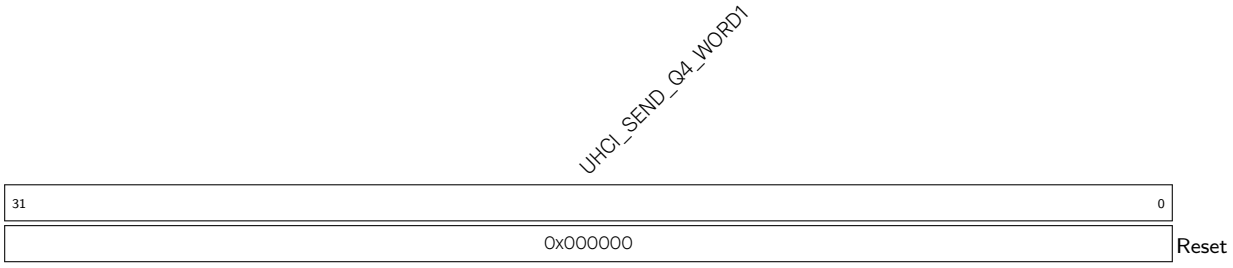
UHCI_SEND_Q3_WORD1 Q3 寄存器待发送数据。(R/W)

Register 21.84. UHCI_REG_Q4_WORD0_REG (0x0054)



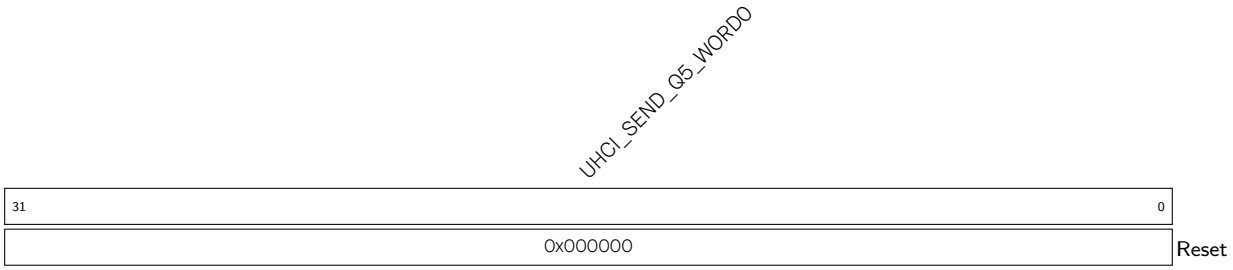
UHCI_SEND_Q4_WORD0 Q4 寄存器待发送数据。(R/W)

Register 21.85. UHCI_REG_Q4_WORD1_REG (0x0058)



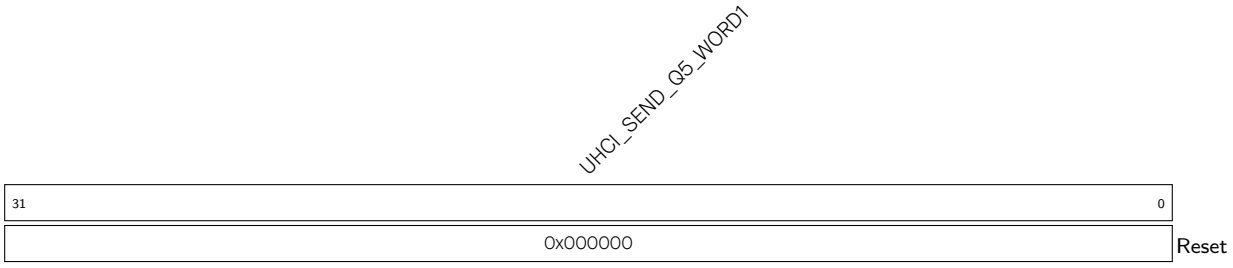
UHCI_SEND_Q4_WORD1 Q4 寄存器待发送数据。(R/W)

Register 21.86. UHCI_REG_Q5_WORD0_REG (0x005C)



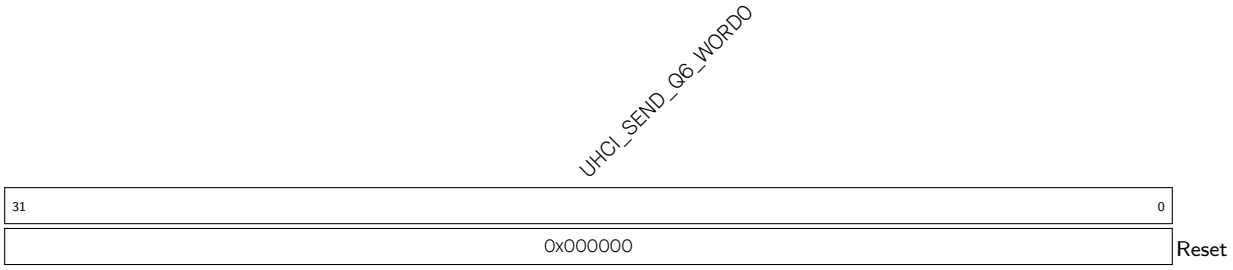
UHCI_SEND_Q5_WORD0 Q5 寄存器待发送数据。(R/W)

Register 21.87. UHCI_REG_Q5_WORD1_REG (0x0060)



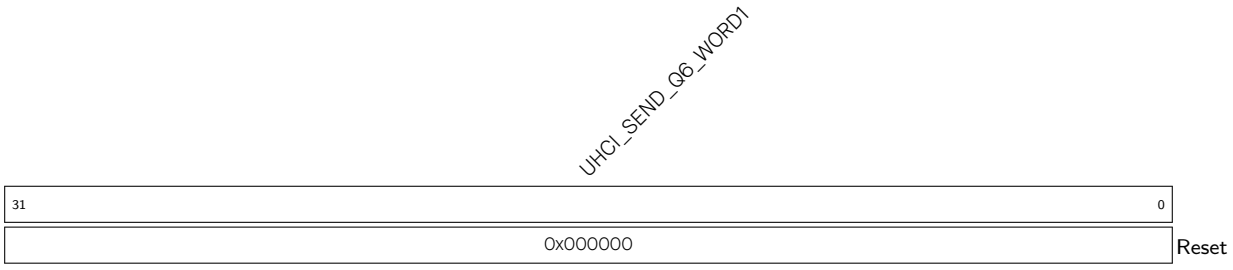
UHCI_SEND_Q5_WORD1 Q5 寄存器待发送数据。(R/W)

Register 21.88. UHCI_REG_Q6_WORD0_REG (0x0064)



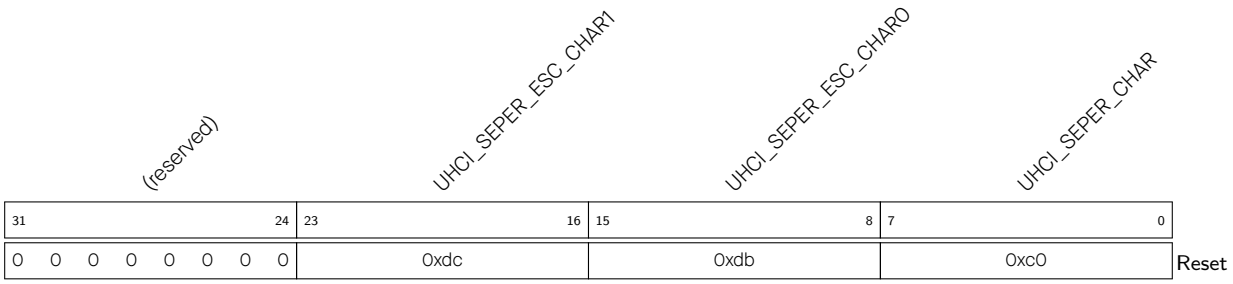
UHCI_SEND_Q6_WORD0 Q6 寄存器待发送数据。(R/W)

Register 21.89. UHCI_REG_Q6_WORD1_REG (0x0068)



UHCI_SEND_Q6_WORD1 Q6 寄存器待发送数据。(R/W)

Register 21.90. UHCI_ESC_CONF0_REG (0x006C)

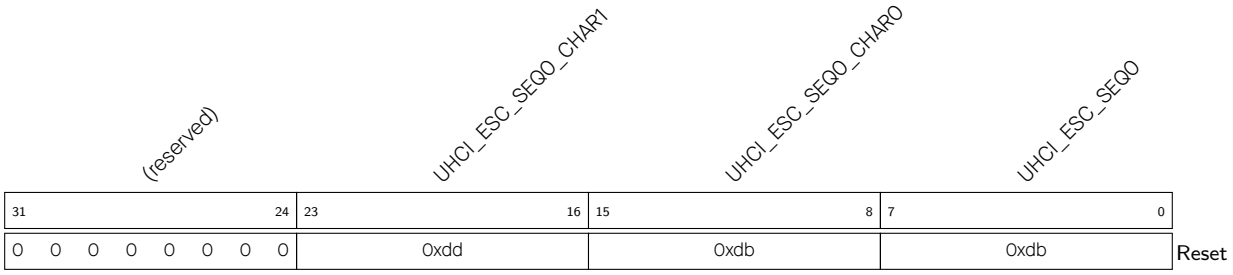


UHCI_SEPER_CHAR 配置编码数据包的分隔符，默认为 0xC0。(R/W)

UHCI_SEPER_ESC_CHAR0 配置 SLIP 转义序列的第一个字符，默认为 0xDB。(R/W)

UHCI_SEPER_ESC_CHAR1 配置 SLIP 转义序列的第二个字符，默认为 0xDC。(R/W)

Register 21.91. UHCI_ESC_CONF1_REG (0x0070)

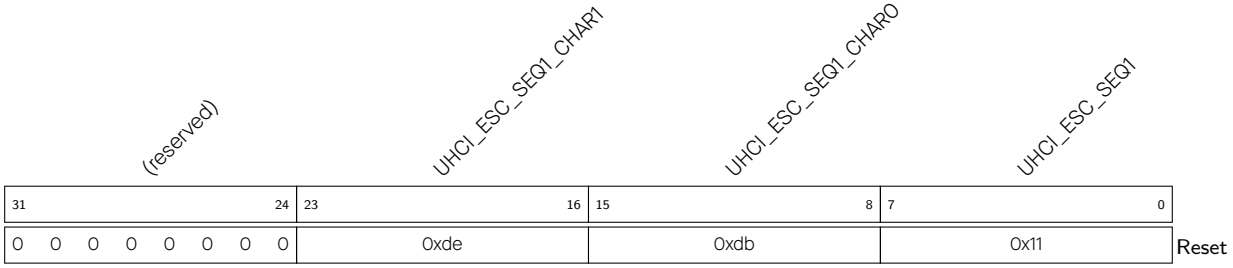


UHCI_ESC_SEQ0 配置需编码的字符，默认为用作 SLIP 转义序列第一个字符的 0xDB。(R/W)

UHCI_ESC_SEQ0_CHAR0 配置 SLIP 转义序列的第一个字符，UHCI_ESC_SEQ0 默认为 0xDB。(R/W)

UHCI_ESC_SEQ0_CHAR1 配置 SLIP 转义序列的第二个字符，UHCI_ESC_SEQ0 默认为 0xDD。(R/W)

Register 21.92. UHCI_ESC_CONF2_REG (0x0074)

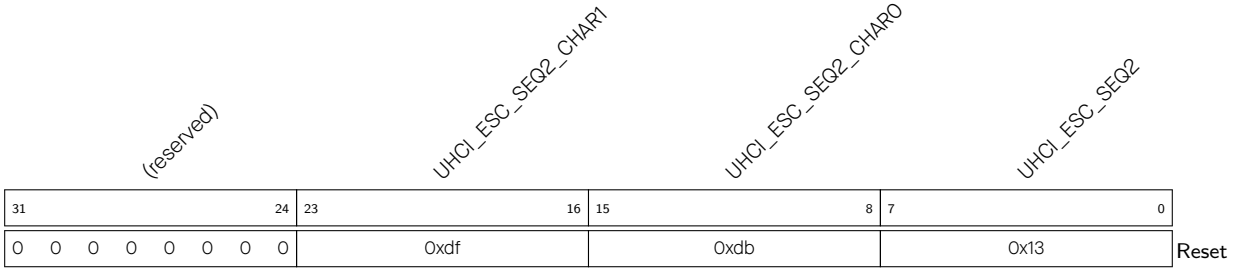


UHCI_ESC_SEQ1 配置需编码的字符，默认为流控字符的 0x11。(R/W)

UHCI_ESC_SEQ1_CHAR0 配置 SLIP 转义序列的第一个字符，UHCI_ESC_SEQ1 默认为 0xDB。(R/W)

UHCI_ESC_SEQ1_CHAR1 配置 SLIP 转义序列的第二个字符，UHCI_ESC_SEQ1 默认为 0xDE。(R/W)

Register 21.93. UHCI_ESC_CONF3_REG (0x0078)

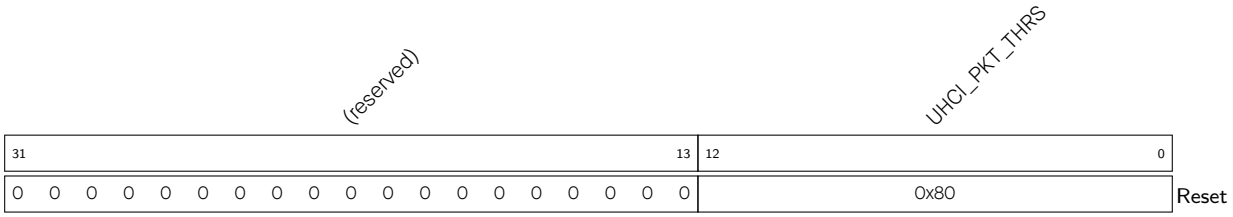


UHCI_ESC_SEQ2 配置需编码的字符，默认为流控字符的 0x13。(R/W)

UHCI_ESC_SEQ2_CHAR0 配置 SLIP 转义序列的第一个字符，UHCI_ESC_SEQ2 默认为 0xDB。(R/W)

UHCI_ESC_SEQ2_CHAR1 配置 SLIP 转义序列的第二个字符，UHCI_ESC_SEQ2 默认为 0xDF。(R/W)

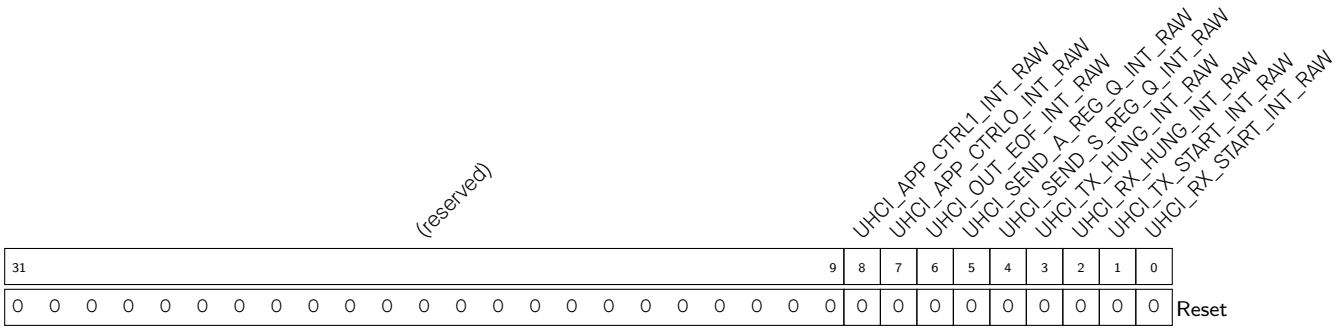
Register 21.94. UHCI_PKT_THRES_REG (0x007C)



UHCI_PKT_THRS 配置数据包的最大长度。

单位：字节。仅在 UHCI_HEAD_EN 为 0 时有效。(R/W)

Register 21.95. UHCI_INT_RAW_REG (0x0004)



UHCI_RX_START_INT_RAW UHCI_RX_START_INT 的原始中断状态。(R/WTC/SS)

UHCI_TX_START_INT_RAW UHCI_TX_START_INT 的原始中断状态。(R/WTC/SS)

UHCI_RX_HUNG_INT_RAW UHCI_RX_HUNG_INT 的原始中断状态。(R/WTC/SS)

UHCI_TX_HUNG_INT_RAW UHCI_TX_HUNG_INT 的原始中断状态。(R/WTC/SS)

UHCI_SEND_S_REG_Q_INT_RAW UHCI_SEND_S_REG_Q_INT 的原始中断状态。(R/WTC/SS)

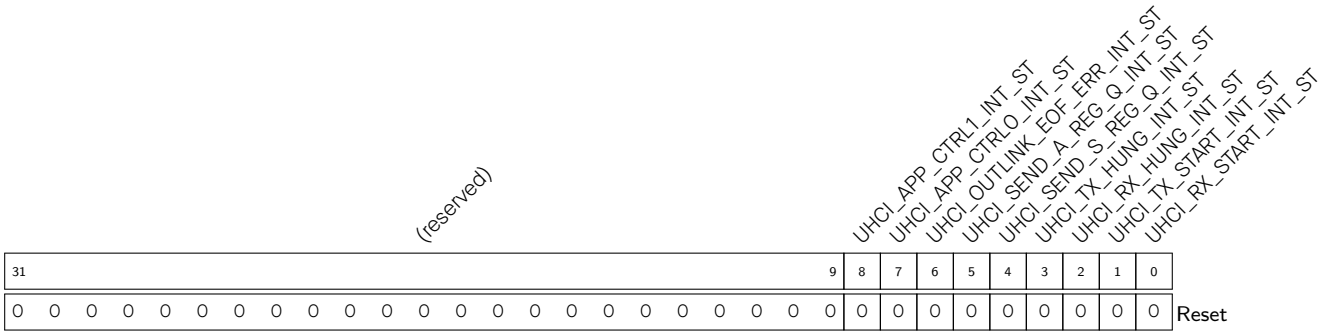
UHCI_SEND_A_REG_Q_INT_RAW UHCI_SEND_A_REG_Q_INT 的原始中断状态。(R/WTC/SS)

UHCI_OUT_EOF_INT_RAW UHCI_OUT_EOF_INT 的原始中断状态。(R/WTC/SS)

UHCI_APP_CTRL0_INT_RAW UHCI_APP_CTRL0_INT 的原始中断状态。(R/W)

UHCI_APP_CTRL1_INT_RAW UHCI_APP_CTRL1_INT 的原始中断状态。(R/W)

Register 21.96. UHCI_INT_ST_REG (0x0008)



UHCI_RX_START_INT_ST UHCI_RX_START_INT 的屏蔽中断状态。(RO)

UHCI_TX_START_INT_ST UHCI_TX_START_INT 的屏蔽中断状态。(RO)

UHCI_RX_HUNG_INT_ST UHCI_RX_HUNG_INT 的屏蔽中断状态。(RO)

UHCI_TX_HUNG_INT_ST UHCI_TX_HUNG_INT 的屏蔽中断状态。(RO)

UHCI_SEND_S_REG_Q_INT_ST UHCI_SEND_S_REG_Q_INT 的屏蔽中断状态。(RO)

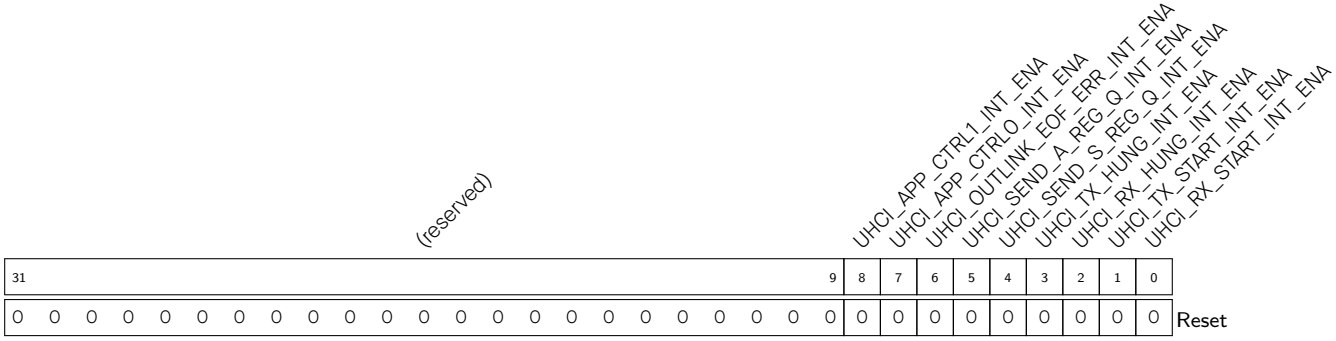
UHCI_SEND_A_REG_Q_INT_ST UHCI_SEND_A_REG_Q_INT 的屏蔽中断状态。(RO)

UHCI_OUTLINK_EOF_ERR_INT_ST UHCI_OUTLINK_EOF_ERR_INT 的屏蔽中断状态。(RO)

UHCI_APP_CTRL0_INT_ST UHCI_APP_CTRL0_INT 的屏蔽中断状态。(RO)

UHCI_APP_CTRL1_INT_ST UHCI_APP_CTRL1_INT 的屏蔽中断状态。(RO)

Register 21.97. UHCI_INT_ENA_REG (0x000C)



UHCI_RX_START_INT_ENA 写 1 使能 UHCI_RX_START_INT。(R/W)

UHCI_TX_START_INT_ENA 写 1 使能 UHCI_TX_START_INT。(R/W)

UHCI_RX_HUNG_INT_ENA 写 1 使能 UHCI_RX_HUNG_INT。(R/W)

UHCI_TX_HUNG_INT_ENA 写 1 使能 UHCI_TX_HUNG_INT。(R/W)

UHCI_SEND_S_REG_Q_INT_ENA 写 1 使能 UHCI_SEND_S_REG_Q_INT。(R/W)

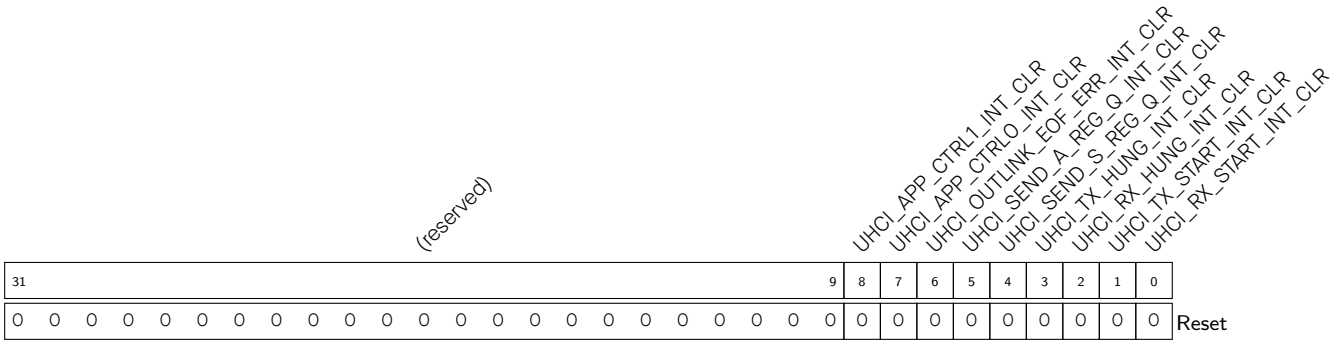
UHCI_SEND_A_REG_Q_INT_ENA 写 1 使能 UHCI_SEND_A_REG_Q_INT。(R/W)

UHCI_OUTLINK_EOF_ERR_INT_ENA 写 1 使能 UHCI_OUTLINK_EOF_ERR_INT。(R/W)

UHCI_APP_CTRL0_INT_ENA 写 1 使能 UHCI_APP_CTRL0_INT。(R/W)

UHCI_APP_CTRL1_INT_ENA 写 1 使能 UHCI_APP_CTRL1_INT。(R/W)

Register 21.98. UHCI_INT_CLR_REG (0x0010)



UHCI_RX_START_INT_CLR 写 1 清除 UHCI_RX_START_INT。(WT)

UHCI_TX_START_INT_CLR 写 1 清除 UHCI_TX_START_INT。(WT)

UHCI_RX_HUNG_INT_CLR 写 1 清除 UHCI_RX_HUNG_INT。(WT)

UHCI_TX_HUNG_INT_CLR 写 1 清除 UHCI_TX_HUNG_INT。(WT)

UHCI_SEND_S_REG_Q_INT_CLR 写 1 清除 UHCI_SEND_S_REG_Q_INT。(WT)

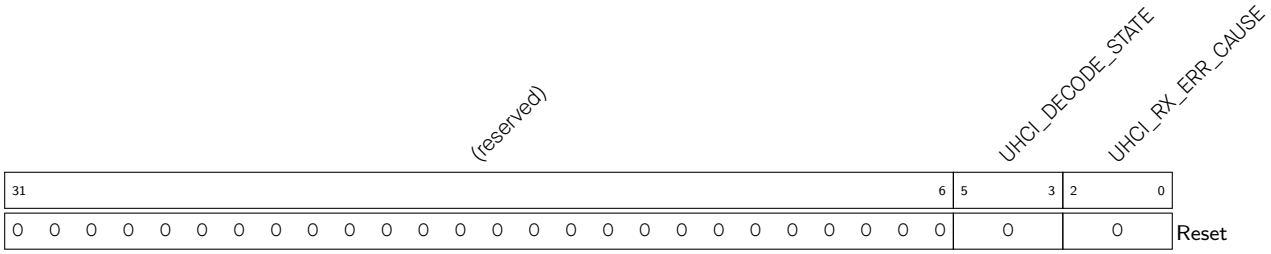
UHCI_SEND_A_REG_Q_INT_CLR 写 1 清除 UHCI_SEND_A_REG_Q_INT。(WT)

UHCI_OUTLINK_EOF_ERR_INT_CLR 写 1 清除 UHCI_OUTLINK_EOF_ERR_INT。(WT)

UHCI_APP_CTRL0_INT_CLR 写 1 清除 UHCI_APP_CTRL0_INT。(WT)

UHCI_APP_CTRL1_INT_CLR 写 1 清除 UHCI_APP_CTRL1_INT。(WT)

Register 21.99. UHCI_STATE0_REG (0x0018)

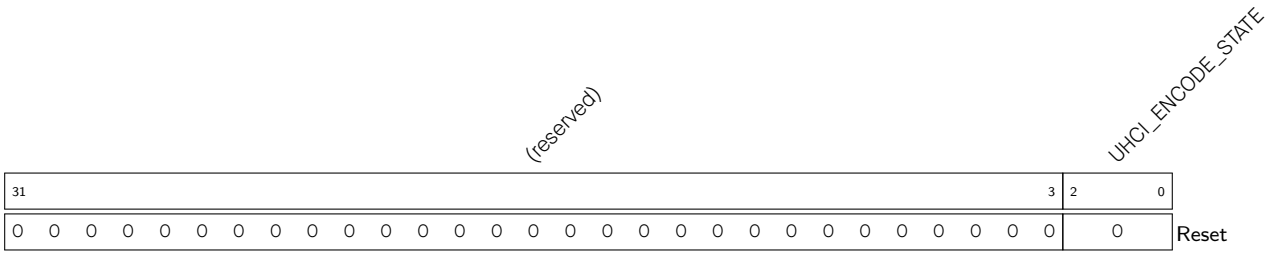


UHCI_RX_ERR_CAUSE 表示 DMA 接收数据错误的原因。

0：无效，没有作用
1：HCI 包校验和错误
2：HCI 包序列号错误
3：HCI 包 CRC 位错误
4：找到 0xC0 但接收的 HCI 包不完整
5：未找到 0xC0 但接收的 HCI 包完整
6：CRC 检测错误
7：无效，没有作用
(RO)

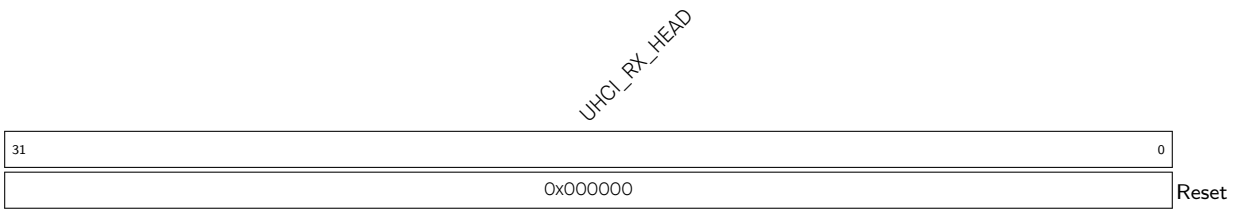
UHCI_DECODE_STATE 表示 UHCI 解码器状态。(RO)

Register 21.100. UHCI_STATE1_REG (0x001C)



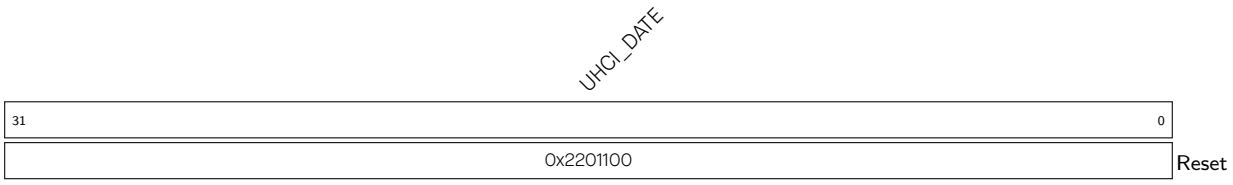
UHCI_ENCODE_STATE 表示 UHCI 编码器状态。(RO)

Register 21.101. UHCI_RX_HEAD_REG (0x002C)



UHCI_RX_HEAD 表示当前接收数据包的报头。(RO)

Register 21.102. UHCI_DATE_REG (0x0080)



UHCI_DATE 版本控制寄存器。(R/W)

第 22 章

USB 串口/JTAG 控制器

ESP32-S31 中包含一个 USB 串口/JTAG 控制器，可用于烧录芯片的外部 flash、读取程序输出的数据以及将调试器连接到正在运行的程序中。任何带有 USB 主机（下文将简称为“主机”）的计算机都可以实现上述功能，无需其他外部组件辅助。

22.1 概述

使用串口通信和 JTAG 调试端口通信来编程和调试 ESP32-S31 项目都是可行的，但也有其限制。首先，串口通信和 JTAG 调试端口通信都占用 IO 管脚，减少了软件中可用于控制外部信号的管脚数量。此外，串口通信和 JTAG 调试端口通信都需借助外部芯片或适配器与主机连接，因此，如果需要整合这两个功能，则需额外使用外部芯片或调试适配器。

为解决上述问题，ESP32-S31 提供了一个 USB 串口/JTAG 控制器，同时集成 USB-串口转换器和 USB-JTAG 适配器功能。由于该模块仅使用 USB 2.0 所需的两条数据线直接连接外部 USB 主机，因此 ESP32-S31 仅需占用两个管脚用于调试。

22.2 主要特性

USB 串口/JTAG 控制器具有如下特性：

- 支持 USB 全速标准；集成 CDC-ACM （通信设备类抽象控制模型）和 JTAG 适配器固定功能
- CDC-ACM：
  - 配置虚拟串行功能，在大多数现代操作系统上可实现即插即用
  - 支持主机控制芯片复位和进入下载模式
- JTAG 适配器：
  - 支持使用紧凑的 JTAG 指令实现与 CPU 调试内核的快速通信
- 共配置一个控制端点、一个虚拟中断端点、两个批量输入端点和两个批量输出端点，可实现最大 64 字节的数据载荷
- 集成内部 PHY：基本无需其他外部组件连接主机计算机

说明：

SDIO 从机控制器在单线模式下可以与 USB 串口/JTAG 控制器同时使用，但在四线模式下不能与 USB 串口/JTAG 控制器同时使用。

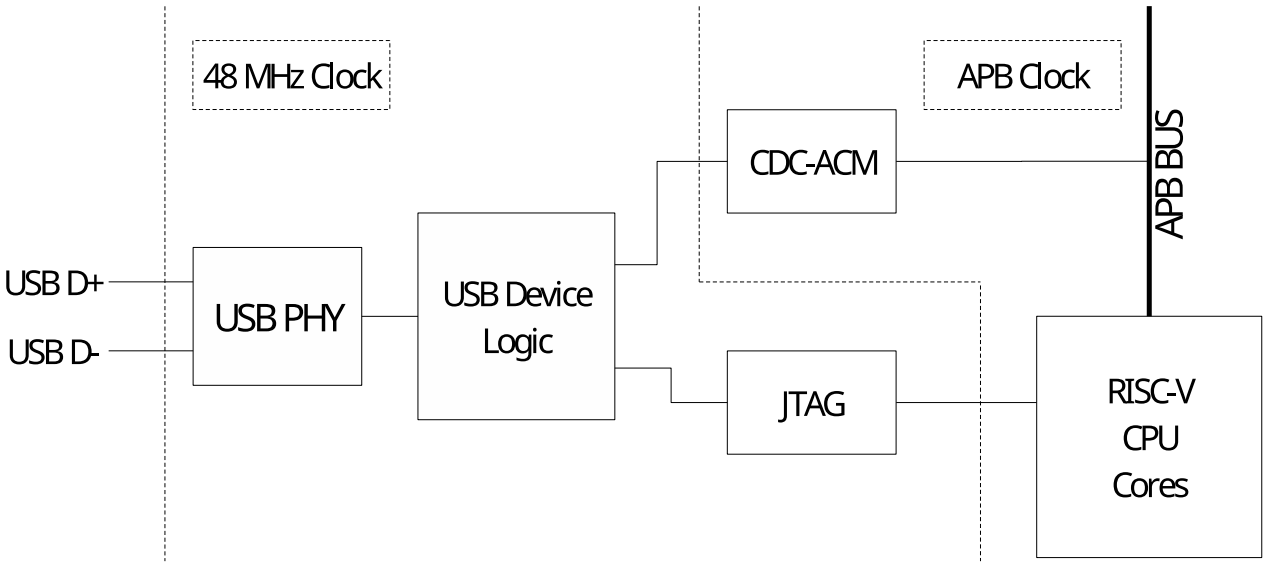


图 22.2-1. USB 串口/JTAG 控制器高层框图

如图 22.2-1 所示，USB 串口/JTAG 控制器包含 USB PHY、USB 设备接口、JTAG 命令处理器、响应捕捉单元以及若干个 CDC-ACM 寄存器。PHY 和设备接口使用主 PLL (BBPLL) 时钟产生的 48 MHz 时钟作为时钟源，CDC-ACM 块中软件可访问的部分则使用 APB_CLK 作为时钟源。JTAG 命令处理器与 ESP32-S31 主处理器中的 JTAG 调试单元相连；CDC-ACM 寄存器则连接至 APB 总线，主 CPU 上运行的软件可对其进行读写访问。

请注意，USB 串口/JTAG 控制器仅支持 USB 2.0 全速标准 (12 Mbps)，不支持 USB 2.0 标准的其他模式（如 480 Mbps 的高速模式）。

图 22.2-2 展示了 USB 串口/JTAG 控制器中 USB 部分的内部详细结构。USB 串口/JTAG 控制器由一个 USB 2.0 全速设备组成，包含一个控制端点、一个虚拟中断端点、两个批量输入端点和两个批量输出端点。这些端点共同组成该复合型 USB 设备，具体可分为 CDC-ACM USB 设备和实现 JTAG 接口功能的供应商特定设备。JTAG 接口直接与芯片的 RISC-V CPU 的调试接口相连，可对运行在该 CPU 上的程序进行调试。同时，CDC-ACM 中包含一组寄存器，CPU 上运行的程序可对其进行读写操作。此外，芯片上的 ROM 启动代码可允许用户通过使用该接口重新烧录 flash。

> **说明：**
> 如需确定某个功能使用的具体端点，请解析设备返回的相关 USB 描述符。这样可以保证与功能相同但实现方式不同的设备兼容。

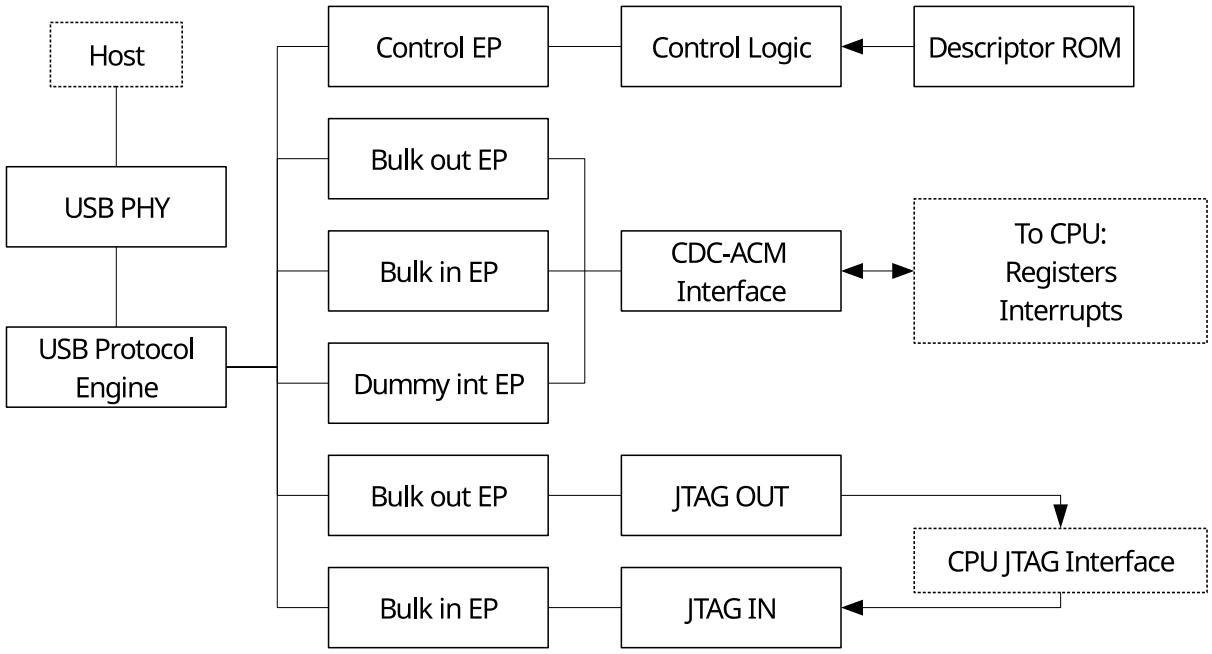


图 22.2-2. USB 串口/JTAG 控制器框图

22.3 功能描述

USB 串口/JTAG 控制器一边与 USB 主机处理器连接，另一边与 CPU 调试硬件以及通过 CDC-ACM 端口通信的软件连接。

22.3.1 CDC-ACM USB 接口描述

CDC-ACM 接口遵循标准 USB CDC-ACM 类别进行虚拟串口通信，包含一个虚拟中断端点（不会发送任何事件，无使用需求）以及一个批量输入端点 (Bulk IN) 和一个批量输出端点 (Bulk OUT) 进行数据接收和发送。这些端点一次可以处理最高 64 字节的数据包，实现高吞吐量。CDC-ACM 为标准的 USB 设备类型，主机一般无需任何特殊安装程序就能正常工作，也就是说，当 USB 调试设备正确连接至主机时，操作系统应能在片刻后显示新的串口信息。

CDC-ACM 接口可以接收以下标准 CDC-ACM 控制请求：

表 22.3-1. 标准 CDC-ACM 控制请求

命令	操作
SEND_BREAK	接收但忽略（虚拟命令）
SET_LINE_CODING	接收，发送值在软件中可读
GET_LINE_CODING	默认返回 9600 baud，无奇偶校验，8 个数据位，1 个停止位（可通过软件更改）
SET_CONTROL_LINE_STATE	设置 RTS/DTR 线的状态，如表 22.3-2 所示

除了通用的通信之外，CDC-ACM 接口还可以复位 ESP32-S31 并选择使其进入下载模式，从而烧录新的固件。这一功能可通过设置虚拟串口的 RTS 和 DTR 线来实现。

表 22.3-2. CDC-ACM 中 RTS 和 DTR 的设置

RTS	DTR	操作
0	0	清除下载模式标志
0	1	置位下载模式标志
1	0	复位 ESP32-S31
1	1	无操作

请注意，当 ESP32-S31 复位时，如果下载模式标志已置位，则 ESP32-S31 重启时将直接进入下载模式；如果下载模式标志已清除，则 ESP32-S31 将从 flash 启动。具体操作流程，请参见章节 22.5。除此之外，也可以通过烧写相应 eFuse 来禁用上述功能，详细信息请参见章节 *4 eFuse 控制器 (eFuse) [to be added later]*。

22.3.2 CDC-ACM 固件接口描述

由于 USB 串口/JTAG 控制器与 ESP32-S31 的内部 APB 总线相连，因此 CPU 可直接与该模块交互，主要用于对连接主机上的虚拟串口进行读写操作。

CPU 向主机发送并从主机接收 0 ~ 64 字节大小的 USB CDC-ACM 串口数据包。主机已积累足够多的 CDC-ACM 数据时，将向 CDC-ACM 的接收端点发送一个数据包。如果 USB 串行/JTAG 控制器中有空闲缓冲区，该缓冲区将接收这一数据包。反之，主机会定期检查 USB 串口/JTAG 控制器内是否有待向主机发送的数据包，如果有，主机将接收这个数据包。

固件可通过以下两种方式之一获知是否有来自主机的新数据：第一，只要缓冲区中还有来自主机的未读数据，USB_SERIAL_JTAG_SERIAL_OUT_EP_DATA_AVAIL 位将保持为 1；第二，如果有新的未读数据，USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT 中断将被触发。当新数据可用时，固件可通过重复从 USB_SERIAL_JTAG_EP1_REG 读取字节来获取该数据。读取每个字节后，可通过检查 USB_SERIAL_JTAG_SERIAL_OUT_EP_DATA_AVAIL 位来看是否还有其他可读取的数据，从而确定需要读取的总字节数。读取完所有数据后，USB 设备会自动做好准备，以接收来自主机的新数据包。

一般来说，发送数据后，连接的主机会阻塞，直到固件通过上述方法之一读取数据。然而，也可能存在某些固件根本不读取数据的情况。通常来说，用于与独立 USB 串口转换器通信的主机程序不会出现这种情况，例如，程序会停止执行直到数据被读取。ESP32-S31 的 USB 串口/JTAG 转换器可以模拟独立 USB 串口/JTAG 转换器的行为：通过向 USB_SERIAL_JTAG_SERIAL_TIMEOUT_MAX 写入超时时间并向 USB_SERIAL_JTAG_SERIAL_TIMEOUT_EN 写入 1，任何未读取的数据将在设定的超时时间后自动刷新。

当固件需要发送数据时，可将待发送数据置于发送缓冲区并触发刷写，从而使主机以 USB 数据包的形式接收该数据。在此之前，需确保发送缓冲区有可用空间以存储待发送数据。固件可通过读取 USB_REG_SERIAL_IN_EP_DATA_FREE 位检查发送缓冲区是否有可用空间；当该值为 1 时，发送缓冲区有可用空间。此时，固件可通过向 USB_SERIAL_JTAG_EP1_REG 寄存器中写入字节，从而向缓冲区中写入数据。但是，数据写入后并不会立即触发向主机发送数据，需首先对缓冲区执行刷写操作。刷写操作后，整个缓冲区将准备好被 USB 主机立即接收。可通过两种方式触发刷写：1) 将第 64 个字节写入缓冲区后，USB 硬件会自动将缓冲区刷写到主机；2) 固件可通过向 USB_REG_SERIAL_WR_DONE 写入 1 来触发刷写。

不论以何种方式触发刷写操作，在此期间固件都无法向缓冲区写入数据，直到缓冲区中的所有数据都已被主机读取完成。主机读取完成后，将触发 USB_SERIAL_JTAG_SERIAL_IN_EMPTY_INT 中断，此时可向缓冲区内写入新的 64 字节数据。

在软件中可以处理一些带外串行请求，例如主机设置 DTR 和 RTS，以及改变线路状态。如果 CDC-ACM 接口收到 SET_LINE_CODING 请求，可将外设配置为触发 USB_SERIAL_JTAG_SET_LINE_CODE_INT 中断，此时可以从 USB_SERIAL_JTAG_SET_LINE_CODE_WO_REG 寄存器读取线路编码。类似的，如果

SET_CONTROL_LINE_STATE 请求导致线路状态改变，将触发 USB_SERIAL_JTAG_RTS_CHG_INT 和 USB_SERIAL_JTAG_DTR_CHG_INT 中断。此后，软件可以通过 USB_SERIAL_JTAG_RTS 和 USB_SERIAL_JTAG_DTR 位来读取具体状态。注意，如前所述，某些 RTS/DTR 序列会导致 ESP32-S31 硬件复位。软件可以通过置位 USB_SERIAL_JTAG_USB_UART_CHIP_RST_DIS 位来禁止硬件识别这些 DTR/RTS 序列，从而允许软件自由解读信号。

最后，主机可以通过 GET_LINE_CODING 读取当前线路状态。此事件将回送 USB_SERIAL_JTAG_GET_LINE_CODE_WO_REG 寄存器中的数据，并触发 USB_SERIAL_JTAG_GET_LINE_CODE_INT 中断。

22.3.3 USB-JTAG 接口：JTAG 命令处理器

USB-JTAG 接口使用一种供应商特定接口类型实现命令解析的功能。它由两个端点组成：一个用于接收命令，一个用于发送响应。此外，一些对时效性要求不高的命令也可以作为控制请求发出。

JTAG 命令处理器负责解析从主机到 JTAG 接口的命令。JTAG 命令处理器内部包含一个全四线 JTAG 总线，包括发送信号到 RISC-V CPU 的 TCK、TMS 和 TDI 输出线，以及从 CPU 返回信号至 JTAG 响应捕捉单元的 TDO 线。这些信号都符合 IEEE 1149.1 JTAG 标准。此外，还有一条 SRST 线用于复位 ESP32-S31。

另外，软件也可以置位 USB_SERIAL_JTAG_USB_JTAG_BRIDGE_EN，将这些信号通过 GPIO 交换矩阵连接至 ESP32-S31 的 IO 焊盘上，此操作也将允许通过 USB 串口/JTAG 控制器对外部设备进行调试。

JTAG 命令处理器会将每个接收到的半字节（4 位）解析为一条命令。由于 USB 以 8 位为一个字节接收数据，这就意味着每个字节中都包含两条命令。USB 命令处理器将先解析高 4 位字节，然后再解析低 4 位字节。这些命令用于控制内部 JTAG 总线的 TCK、TMS、TDI、SRST 线，以及向 JTAG 响应捕捉单元发出信号，说明需要捕捉 TDO 线（由 CPU 调试逻辑驱动）的状态。

JTAG 总线中，TCK、TMS、TDI 和 TDO 线直接与 RISC-V CPU 的 JTAG 调试逻辑相连。上文提到的 SRST 线则与 ESP32-S31 数字电路中的复位逻辑相连，该线电平拉高，芯片将进行芯片复位。请注意，SRST 线并不会对 USB 串口/JTAG 控制器模块产生影响。

1 个半字节中可包含以下命令：

表 22.3-3. 半字节中的命令

位	3	2	1	0
CMD_CLK	0	cap	tms	tdi
CMD_RST	1	0	0	srst
CMD_FLUSH	1	0	1	0
CMD_RSV	1	0	1	1
CMD_REP	1	1	R1	R0

- CMD_CLK：将 TDI 和 TMS 设置为指示值，并在 TCK 上发出一个时钟脉冲。如果 CAP 位为 1，将指示 JTAG 响应捕捉单元捕捉 TDO 线的状态。该指令构成了 JTAG 通信的基础。
- CMD_RST：将 SRST 线的状态设置为指示值。该命令可用于复位 ESP32-S31。
- CMD_FLUSH：指示 JTAG 响应捕捉单元对接收到的所有位的缓冲区进行刷写操作，以便主机读取这些位。请注意在某些情况下，一次 JTAG 通信会结束于第奇数个命令，即结束于第奇数个半字节。此时，可重复执行该命令直到获得偶数个半字节，使其组成整数个字节。
- CMD_RSV：该版本中保留。ESP32-S31 在接收到该命令时会自动忽略。

- CMD_REP：将上一条指令（非 CMD_REP）重复一定次数，以压缩多次重复 CMD_CLK 的命令流。因此，CMD_CLK 命令后可能跟随着多个 CMD_REP。一次 CMD_REP 命令产生的重复次数可表示为 $repetition_count = (R1 \times 2 + R0) \times (4^{cmd_rep_count})$，其中 cmd_rep_count 表示该命令之前的 CMD_REP 数量。请注意，CMD_REP 仅用于重复 CMD_CLK 命令。也就是说，如果在 CMD_FLUSH 后使用该命令，USB 设备将无法响应，需进行 USB 复位后才可恢复正常。

22.3.4 USB-JTAG 接口：CMD_REP 使用示例

下列命令用于演示如何使用 CMD_REP 命令。请注意，该示例中每个命令为半字节，命令流的每个字节为 0x0D 0x5E 0xCF。

1. 0x0 (CMD_CLK: cap=0, tdi=0, tms=0)
2. 0xD (CMD_REP: R1=0, R0=1)
3. 0x5 (CMD_CLK: cap=1, tdi=0, tms=1)
4. 0xE (CMD_REP: R1=1, R0=0)
5. 0xC (CMD_REP: R1=0, R0=0)
6. 0xF (CMD_REP: R1=1, R0=1)

每一步骤的具体操作为：

1. TCK 上发出一个时钟脉冲，TDI 和 TMS 设置为 0。未捕捉到任何数据。
2. TCK 上再次发出 $(0 \times 2 + 1) \times (4^0) = 1$ 个时钟脉冲，其他设置与步骤 1 相同。
3. TCK 上发出一个时钟脉冲，TDI 设置为 0，TMS 设置为 1。捕捉到 TDO 线上的数据。
4. TCK 上再次发出 $(1 \times 2 + 0) \times (4^0) = 2$ 个时钟脉冲，其他设置与步骤 3 相同。
5. 未发生任何动作：$(0 \times 2 + 0) \times (4^1) = 0$。请注意，该操作将增加下一步骤中的 cmd_rep_count 数值。
6. TCK 上再次发出 $(1 \times 2 + 1) \times (4^2) = 48$ 个时钟脉冲，其他设置与步骤 3 相同。

换言之，该命令流示例的操作结果等同于执行两次命令 1，然后执行 51 次命令 3 的效果。

22.3.5 USB-JTAG 接口：响应捕捉单元

响应捕捉单元首先读取内部 JTAG 总线的 TDO 线，并在命令处理器执行 CMD_CLK (cap=1) 命令时捕捉 TDO 线的值，然后将这个值放入内部移位寄存器中，且在接收到 8 位时向 USB 缓冲区写入 1 个字节。这 8 位中的最低有效位即为最先从 TDO 线读取的值。

一旦接收到 64 字节 (512 位) 数据或执行 CMD_FLUSH 命令后，响应捕捉单元将使缓冲区可被主机接收。请注意，USB 逻辑的接口为双缓冲。因此，只要 USB 的吞吐量充足，响应捕捉单元就可以随时接收更多数据，即当一个缓冲区等待发送给主机时，另一个缓冲区可以继续接收数据。当主机从缓冲区成功接收数据且响应捕捉单元对缓冲区执行刷写操作后，这两个缓冲区便可以交换位置。

同时，这也意味着一个命令流可导致最多 128 字节（若该命令流中有刷写命令，则该数字会减小）的捕捉数据生成，而无需主机主动接收这些数据。如果还是生成了超过该阈值数量的捕捉数据，则命令流将暂停，且设备在这些数据被读取之前不会接收其他命令。

另需注意，一般情况下，响应捕捉单元会尽量不发送 0 字节响应。例如，当发送一系列 CMD_FLUSH 命令后，并不会产生一系列 0 字节 USB 响应。但在当前版本中，一些特殊情况下也可能产生 0 字节响应。建议直接忽略这些响应信息。

22.3.6 USB-JTAG 接口：控制传输请求

除命令处理器和响应捕捉单元之外，USB-JTAG 接口也可接收一些控制请求，具体如下表所示：

表 22.3-4. USB-JTAG 控制请求

bmRequestType	bRequest	wValue	wIndex	wLength	Data
01000000b	0 (VEND_JTAG_SETDIV)	[divider]	接口	0	None
01000000b	1 (VEND_JTAG_SETIO)	[iobits]	接口	0	None
11000000b	2 (VEND_JTAG_GETTDO)	0	接口	1	[iostate]
10000000b	6 (GET_DESCRIPTOR)	0x2000	0	256	[jtag cap desc]

- VEND_JTAG_SETDIV：设置使用的分频器。该命令直接影响 TCK 时钟脉冲的持续时间。TCK 时钟脉冲来自于由内部分频器向下分频得到的 48 MHz 基准时钟。该控制请求允许主机设置此内部分频器。请注意，该分频器在启动时的初始值为 2，即 TCK 时钟速率一般为 24 MHz。
- VEND_JTAG_SETIO：跳过 JTAG 命令处理器，直接将内部 TDI、TDO、TMS 和 SRST 线设置为指定值。这些指定值在 wValue 字段中以 11'b0, srst, trst, tck, tms, tdi 的格式编码。
- VEND_JTAG_GETTDO：跳过 JTAG 响应捕捉单元，直接读取内部 TDO 信号。该请求返回 1 个字节，其中的最低有效位代表 TDO 线的状态。
- GET_DESCRIPTOR：为标准 USB 请求，该请求也可使用供应商专用的 0x2000 wValue 获取 JTAG 功能描述符。该请求返回一定字节，具体字节数所代表的 USB-JTAG 适配器功能如表 22.3-5 所示。这一固定结构允许主机软件自动支持未来的硬件新版本，无需再次更新。

ESP32-S31 包含的 JTAG 功能描述符如下表所示。请注意，所有 16 位值都为小端序存储。

表 22.3-5. JTAG 功能描述符

字节	数值	描述
0	1	JTAG 协议功能结构的版本
1	10	JTAG 协议功能长度
2	1	结构类型：1 代表高速功能结构类型
3	8	该高速功能结构长度
4 ~ 5	4800	以 10 kHz 为增量的 JTAG 基准时钟速度，其最大速度为该值的一半
6 ~ 7	1	最小分频系数，可通过 VEND_JTAG_SETDIV 设置
8 ~ 9	255	最大分频系数，可通过 VEND_JTAG_SETDIV 设置

22.4 中断

ESP32-S31 的 USB 串口/JTAG 控制器的下列内部中断源可生成中断信号 USB_SERIAL_JTAG_INTR，并发送给中断矩阵。

- USB_SERIAL_JTAG_JTAG_IN_FLUSH_INT：JTAG 批量输出端点接收到刷写命令时触发。
- USB_SERIAL_JTAG_SOF_INT：接收到 SOF 帧时触发。
- USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT：串口输出端点接收到 1 个数据包时触发。
- USB_SERIAL_JTAG_SERIAL_IN_EMPTY_INT：串口输入端点为空时触发。

- USB_SERIAL_JTAG_PID_ERR_INT：检测到 PID 错误时触发。
- USB_SERIAL_JTAG_CRC5_ERR_INT：检测到 CRC5 错误时触发。
- USB_SERIAL_JTAG_CRC16_ERR_INT：检测到 CRC16 错误时触发。
- USB_SERIAL_JTAG_STUFF_ERR_INT：检测到位填充错误时触发。
- USB_SERIAL_JTAG_IN_TOKEN_REC_IN_EP1_INT：输入端点 1 接收到一个 IN 令牌时触发。
- USB_SERIAL_JTAG_USB_BUS_RESET_INT：检测到 USB 总线复位时触发。
- USB_SERIAL_JTAG_OUT_EP1_ZERO_PAYLOAD_INT：输出端点 1 接收到有效载荷为 0 的数据包时触发。
- USB_SERIAL_JTAG_OUT_EP2_ZERO_PAYLOAD_INT：输出端点 2 接收到有效载荷为 0 的数据包时触发。
- USB_SERIAL_JTAG_RTS_CHG_INT：USB 串行通道的 RTS 电平改变时触发。
- USB_SERIAL_JTAG_DTR_CHG_INT：USB 串行通道的 DTR 电平改变时触发。
- USB_SERIAL_JTAG_GET_LINE_CODE_INT：收到 GET_LINE_CODING 请求的电平时触发。
- USB_SERIAL_JTAG_SET_LINE_CODE_INT：收到 SET_LINE_CODING 请求的电平时触发。

22.5 配置流程

使用 USB 串口/JTAG 控制器之前，几乎不需要多余的配置。除了主机操作系统已经完成的标准 USB 初始化之外，USB-JTAG 硬件本身不需要进行任何配置。此外，CDC-ACM 虚拟串口在主机端也支持即插即用。

固件方面也几乎不需要初始化。USB 硬件是自初始化的，在其启动后，如果固件连接了一台主机并在 CDC-ACM 接口上监听，除非固件设置了中断处理程序，否则无需任何特定设置就可以实现前文所述的数据交换。

需要注意的是，可能会出现主机未连接或 CDC-ACM 虚拟串口未打开的情况。在这种情况下，发送至主机的数据包永远无法被接收，发送缓冲区也永远不会为空。因而，必须对这种情况进行检测并执行超时操作，以便清楚地检测主机侧的端口是否关闭。

其次，需要注意 USB 设备依赖于产生 48 MHz USB PHY 时钟的 BBPLL。如果此 PLL 被禁用，USB 通信将停止。

上述情况可能发生在 Deep-sleep 期间。在 Deep-sleep 模式下，USB 串口/JTAG 控制器（及其连接的 RISC-V CPU）将完全断电。如果有设备需要在这两种模式下进行调试，最好使用外部 JTAG 调试器和串行接口。

CDC-ACM 接口还可用于复位芯片，使其进入或退出下载模式。产生正确的握手信号序列则有些复杂，因为大多数操作系统仅支持分别设置或重置 DTR 和 RTS，无法同时进行。此外，一些驱动程序（如 Windows 系统上的标准 CDC-ACM 驱动程序）须先设置 RTS 后才可设置 DTR，此时用户必须先明确设置 RTS，才能传播 DTR 的值。推荐遵循以下步骤进行设置。

复位芯片使其进入下载模式：

表 22.5-1. 复位芯片进入下载模式

操作	内部状态	备注
清除 DTR	RTS=?, DTR=0	初始化以获取数值
清除 RTS	RTS=0, DTR=0	-
设置 DTR	RTS=0, DTR=1	设置下载模式标志
清除 RTS	RTS=0, DTR=1	传播 DTR
设置 RTS	RTS=1, DTR=1	-
清除 DTR	RTS=1, DTR=0	复位芯片
设置 RTS	RTS=1, DTR=0	传播 DTR
清除 RTS	RTS=0, DTR=0	清除下载标志

复位 SoC 使其从 flash 启动：

表 22.5-2. 复位 SoC 从 flash 启动

操作	内部状态	备注
清除 DTR	RTS=?, DTR=0	-
清除 RTS	RTS=0, DTR=0	清除下载标志
设置 RTS	RTS=1, DTR=0	复位 SoC
清除 RTS	RTS=0, DTR=0	退出复位

22.6 寄存器列表

本小节的所有地址均为相对于 USB 串口/JTAG 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
配置寄存器			
USB_SERIAL_JTAG_EP1_REG	CDC-ACM 输入/输出端点 FIFO 访问配置寄存器	0x0000	R/W
USB_SERIAL_JTAG_EP1_CONF_REG	CDC-ACM FIFO 配置寄存器	0x0004	varies
USB_SERIAL_JTAG_CONF0_REG	PHY 硬件配置寄存器	0x0018	R/W
USB_SERIAL_JTAG_TEST_REG	PHY 调试寄存器	0x001C	varies
USB_SERIAL_JTAG_MISC_CONF_REG	时钟使能控制寄存器	0x0044	R/W
USB_SERIAL_JTAG_MEM_CONF_REG	存储器配置寄存器	0x0048	R/W
USB_SERIAL_JTAG_CHIP_RST_REG	CDC-ACM 芯片重置寄存器	0x004C	varies
USB_SERIAL_JTAG_GET_LINE_CODE_W0_REG	GET_LINE_CODING 命令的 W0	0x0058	R/W
USB_SERIAL_JTAG_GET_LINE_CODE_W1_REG	GET_LINE_CODING 命令的 W1	0x005C	R/W
USB_SERIAL_JTAG_CONFIG_UPDATE_REG	配置寄存器更新寄存器	0x0060	WT
USB_SERIAL_JTAG_SER_AFIFO_CONFIG_REG	串口 AFIFO 配置寄存器	0x0064	varies
USB_SERIAL_JTAG_SERIAL_EP_TIMEOUT0_REG	USB UART 输出端点超时配置	0x006C	varies
USB_SERIAL_JTAG_SERIAL_EP_TIMEOUT1_REG	USB UART 输出端点超时配置	0x0070	R/W
中断寄存器			
USB_SERIAL_JTAG_INT_RAW_REG	中断原始状态寄存器	0x0008	R/WTC/SS
USB_SERIAL_JTAG_INT_ST_REG	中断状态寄存器	0x000C	RO
USB_SERIAL_JTAG_INT_ENA_REG	中断使能状态寄存器	0x0010	R/W
USB_SERIAL_JTAG_INT_CLR_REG	中断清除状态寄存器	0x0014	WT
状态寄存器			
USB_SERIAL_JTAG_JFIFO_ST_REG	JTAG FIFO 状态与控制寄存器	0x0020	varies
USB_SERIAL_JTAG_FRAM_NUM_REG	接收 SOF 帧索引寄存器	0x0024	RO
USB_SERIAL_JTAG_IN_EP0_ST_REG	输入端点状态信息寄存器	0x0028	RO
USB_SERIAL_JTAG_IN_EP1_ST_REG	CDC-ACM 输入端点状态信息寄存器	0x002C	RO
USB_SERIAL_JTAG_IN_EP2_ST_REG	CDC-ACM 中断输入端点状态信息寄存器	0x0030	RO
USB_SERIAL_JTAG_IN_EP3_ST_REG	JTAG 输入端点状态信息寄存器	0x0034	RO
USB_SERIAL_JTAG_OUT_EP0_ST_REG	输出端点状态信息寄存器	0x0038	RO
USB_SERIAL_JTAG_OUT_EP1_ST_REG	CDC-ACM 输出端点状态信息寄存器	0x003C	RO
USB_SERIAL_JTAG_OUT_EP2_ST_REG	JTAG 输出端点状态信息寄存器	0x0040	RO
USB_SERIAL_JTAG_SET_LINE_CODE_W0_REG	SET_LINE_CODING 命令的 W0	0x0050	RO
USB_SERIAL_JTAG_SET_LINE_CODE_W1_REG	SET_LINE_CODING 命令的 W1	0x0054	RO
USB_SERIAL_JTAG_BUS_RESET_ST_REG	USB 总线重置状态寄存器	0x0068	RO
版本寄存器			

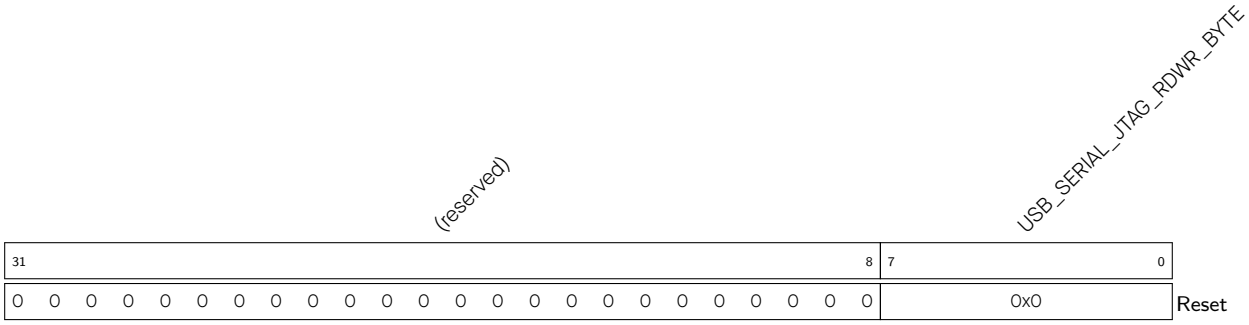
名称	描述	地址	访问
USB_SERIAL_JTAG_DATE_REG	版本寄存器	0x0080	R/W

22.7 寄存器

本小节的所有地址均为相对于 USB 串口/JTAG 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 22.1. USB_SERIAL_JTAG_EP1_REG (0x0000)



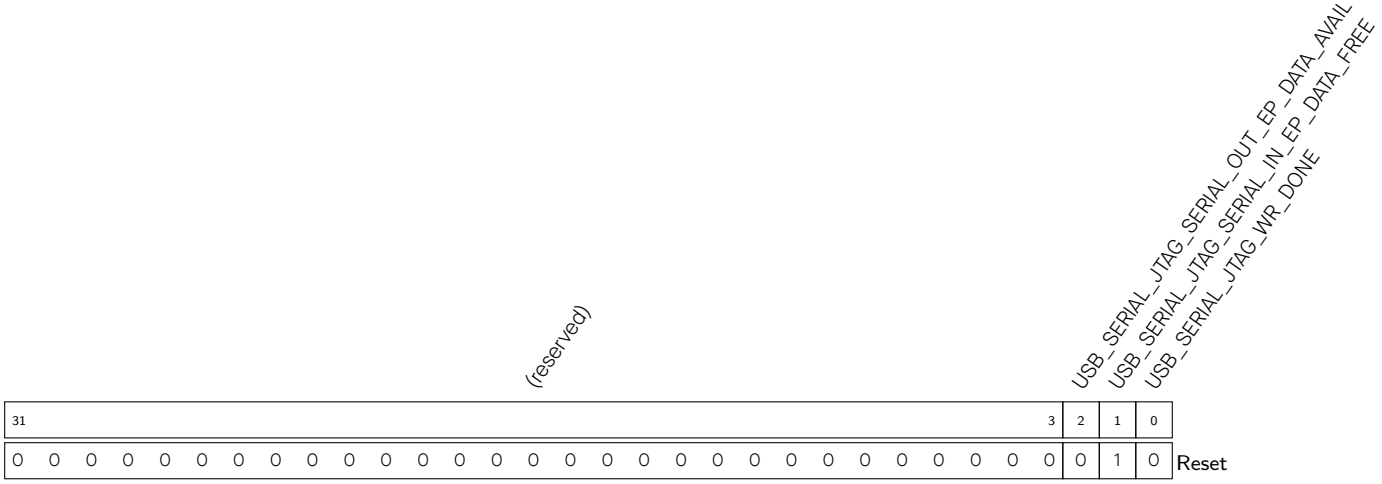
USB_SERIAL_JTAG_RDWR_BYTE 写入该位即为将数据写入 CDC-ACM TX FIFO；读取该位即为从 CDC-ACM RX FIFO 读取一个字节并返回该字节。

USB_SERIAL_JTAG_SERIAL_IN_EMPTY_INT 置位时，用户可向 CDC-ACM TX FIFO 中写入数据（最大 64 字节）。

USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT 置位时，用户可通过查看 USB_SERIAL_JTAG_OUT_EP1_WR_ADDR 的值获知接收到的数据量，然后从 CDC-ACM RX FIFO 中读取这些数据。

(R/W)

Register 22.2. USB_SERIAL_JTAG_EP1_CONF_REG (0x0004)



USB_SERIAL_JTAG_WR_DONE 配置是否表明已完成向 CDC-ACM TX FIFO 写入字节数据。

0: 无效

1: 表明已完成向 CDC-ACM TX FIFO 写入字节数据

该位将保持为 0，直到 USB 主机读取 CDC-ACM TX FIFO 中的数据。

(WT)

USB_SERIAL_JTAG_SERIAL_IN_EP_DATA_FREE 表示 CDC-ACM TX FIFO 是否可以写入数据。

0：CDC-ACM TX FIFO 已写满，不可写入数据

1：CDC-ACM TX FIFO 未写满且可以写入数据

写入 USB_SERIAL_JTAG_WR_DONE 后，该位将置为 0，直到其中的数据已发送至 USB 主机。

(RO)

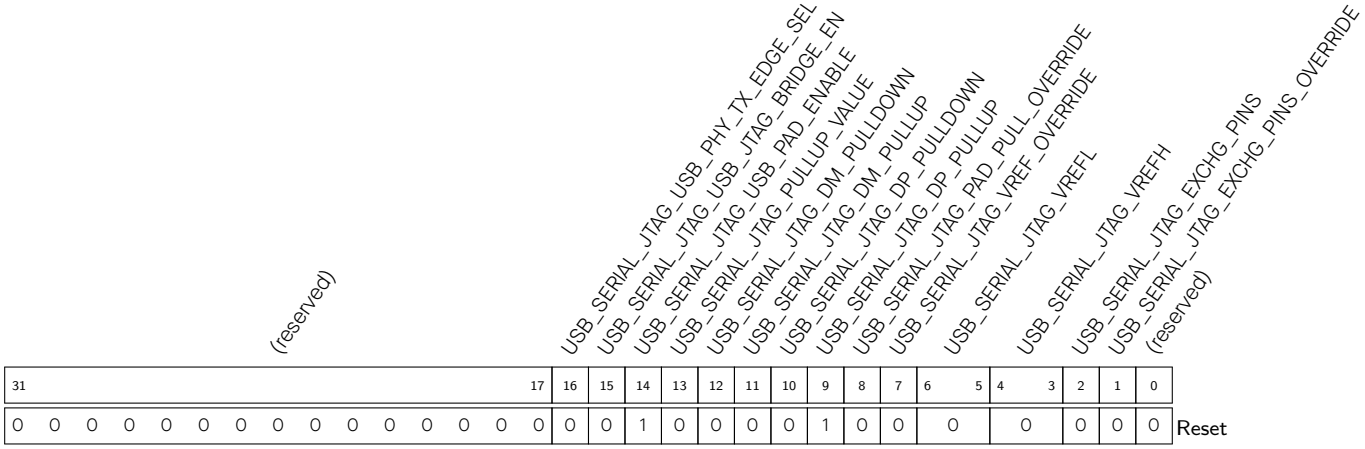
USB_SERIAL_JTAG_SERIAL_OUT_EP_DATA_AVAIL 表示 CDC-ACM RX FIFO 中是否有数据。

0: CDC-ACM RX FIFO 中没有数据

1: CDC-ACM RX FIFO 中有数据

(RO)

Register 22.3. USB_SERIAL_JTAG_CONF0_REG (0x0018)



USB_SERIAL_JTAG_EXCHG_PINS_OVERRIDE 配置是否使能软件控制 USB D+ 和 D- 管脚交换。

0：不使能

1：使能

(R/W)

USB_SERIAL_JTAG_EXCHG_PINS 配置是否交换 USB D+ 和 D- 管脚。

0：无效

1：交换

(R/W)

USB_SERIAL_JTAG_VREFH 配置单端输入高阈值。

0：1.76 V

1：1.84 V

2：1.92 V

3：2.00 V

(R/W)

USB_SERIAL_JTAG_VREFL 配置单端输入低阈值。

0：0.80 V

1：0.88 V

2：0.96 V

3：1.04 V

(R/W)

USB_SERIAL_JTAG_VREF_OVERRIDE 配置是否使能软件控制输入阈值。

0：不使能

1：使能

(R/W)

见下页...

Register 22.3. USB_SERIAL_JTAG_CONF0_REG (0x0018)

接上页...

USB_SERIAL_JTAG_PAD_PULL_OVERRIDE 配置是否使能软件控制 USB D+ 和 D- 管脚的上下拉电阻。

0：不使能

1：使能

(R/W)

USB_SERIAL_JTAG_DP_PULLUP 配置当 USB_SERIAL_JTAG_PAD_PULL_OVERRIDE 为 1 时，是否使能 USB D+ 管脚的上拉电阻。

0：不使能

1：使能

(R/W)

USB_SERIAL_JTAG_DP_PULLDOWN 配置当 USB_SERIAL_JTAG_PAD_PULL_OVERRIDE 为 1 时，是否使能 USB D+ 管脚的下拉电阻。

0：不使能

1：使能

(R/W)

USB_SERIAL_JTAG_DM_PULLUP 配置当 USB_SERIAL_JTAG_PAD_PULL_OVERRIDE 为 1 时，是否使能 USB D- 管脚的上拉电阻。

0：不使能

1：使能

(R/W)

USB_SERIAL_JTAG_DM_PULLDOWN 配置当 USB_SERIAL_JTAG_PAD_PULL_OVERRIDE 为 1 时，是否使能 USB D- 管脚的下拉电阻。

0：不使能

1：使能

(R/W)

USB_SERIAL_JTAG_PULLUP_VALUE 配置当 USB_SERIAL_JTAG_PAD_PULL_OVERRIDE 为 1 时的上拉数值。

0：2.2 K

1：1.1 K

(R/W)

USB_SERIAL_JTAG_USB_PAD_ENABLE 配置是否使能 USB 管脚。

0：不使能

1：使能

(R/W)

见下页...

Register 22.3. USB_SERIAL_JTAG_CONF0_REG (0x0018)

接上页...

USB_SERIAL_JTAG_USB_JTAG_BRIDGE_EN 配置是否断开 USB_JTAG 和内部 JTAG 的连接。

0：USB_JTAG 连接到 CPU 的内部 JTAG 端口

1：USB_JTAG 和内部 JTAG 之间断开连接，MTMS、MTDI 和 MTCK 为通过 GPIO 交换矩阵的输出，MTDO 为通过 GPIO 交换矩阵的输入

(R/W)

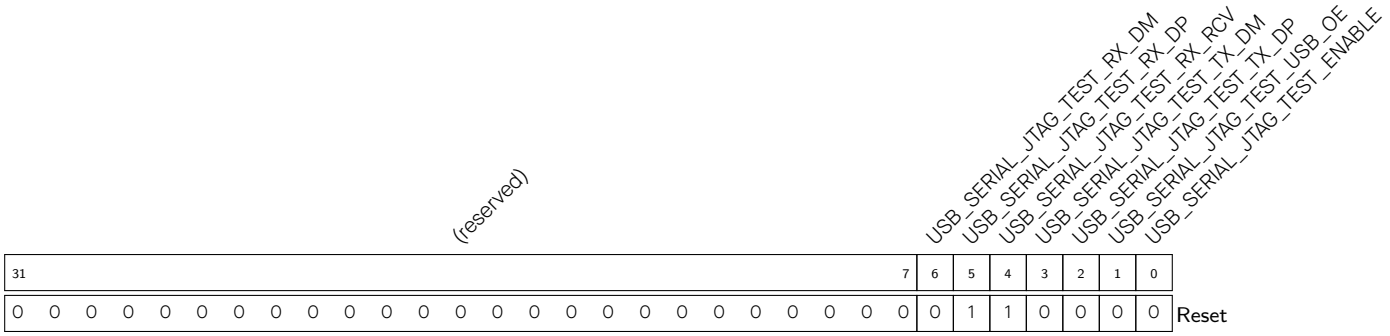
USB_SERIAL_JTAG_USB_PHY_TX_EDGE_SEL 配置将 DP 和 DM 信号发送到 USB PHY 的时钟沿。

0：在时钟下降沿发送

1：在时钟上升沿发送

(R/W)

Register 22.4. USB_SERIAL_JTAG_TEST_REG (0x001C)



USB_SERIAL_JTAG_TEST_ENABLE 配置是否使能 USB 管脚测试模式。

0：不使能，恢复正常模式

1：使能 USB 管脚测试模式

使能 USB 管脚测试模式将允许使用该寄存器的其他位来控制/读取 USB 管脚。

(R/W)

USB_SERIAL_JTAG_TEST_USB_OE 配置是否使能 USB 管脚输出。

0：将 D+ 和 D- 设为高阻抗

1：在 D+ 和 D- 管脚上输出 USB_SERIAL_JTAG_TEST_TX_DP 和 USB_SERIAL_JTAG_TEST_TX_DM 中设置的值

(R/W)

USB_SERIAL_JTAG_TEST_TX_DP 当 USB_SERIAL_JTAG_TEST_USB_OE 为 1 时，设置测试模式下 USB D+ 管脚的值。(R/W)

USB_SERIAL_JTAG_TEST_TX_DM 当 USB_SERIAL_JTAG_TEST_USB_OE 为 1 时，设置测试模式下 USB D- 管脚的值。(R/W)

USB_SERIAL_JTAG_TEST_RX_RCV 表示在测试模式下 USB D- 和 USB D+ 管脚之间电压差的当前逻辑水平。

0：USB D- 电压高于 USB D+

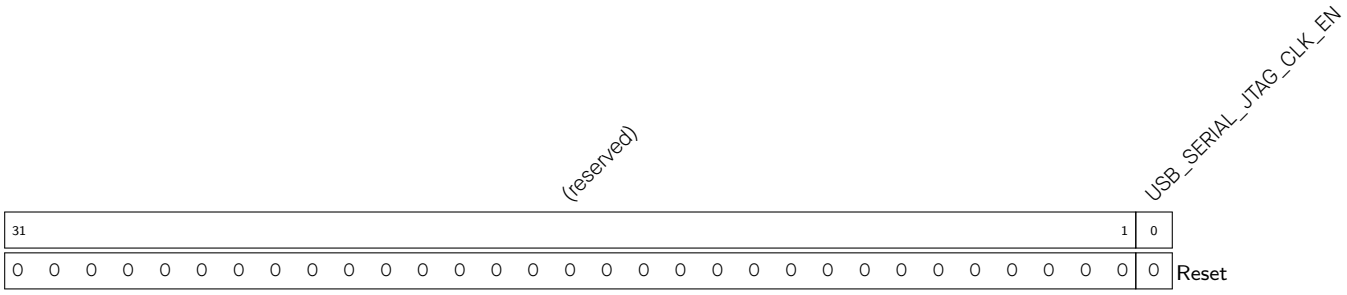
1：USB D- 电压低于 USB D+

(RO)

USB_SERIAL_JTAG_TEST_RX_DP 表示在测试模式下 USB D+ 管脚的逻辑电平。(RO)

USB_SERIAL_JTAG_TEST_RX_DM 表示在测试模式下 USB D- 管脚的逻辑电平。(RO)

Register 22.5. USB_SERIAL_JTAG_MISC_CONF_REG (0x0044)



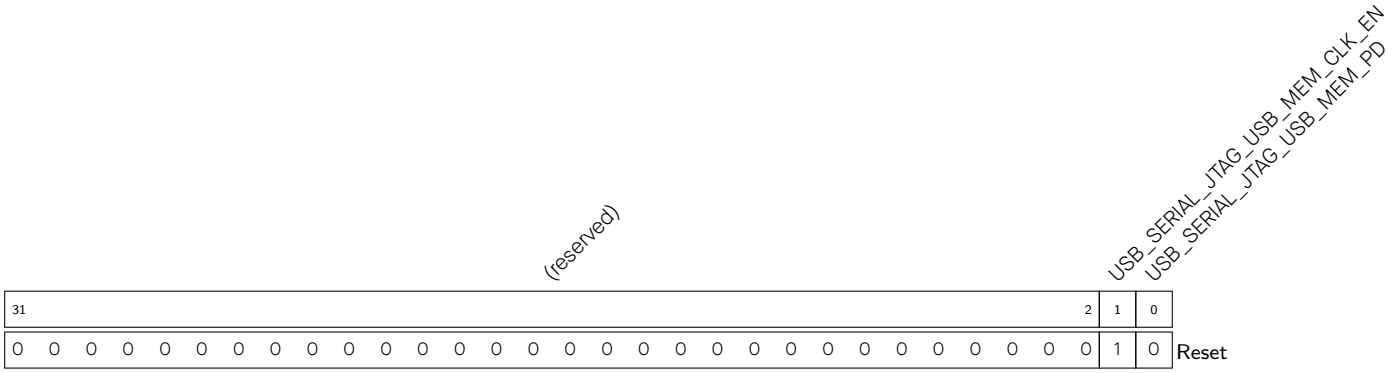
USB_SERIAL_JTAG_CLK_EN 配置是否强制打开寄存器的时钟。

0：支持仅当应用程序向寄存器写入数据时打开时钟

1：强制打开寄存器的时钟

(R/W)

Register 22.6. USB_SERIAL_JTAG_MEM_CONF_REG (0x0048)



USB_SERIAL_JTAG_USB_MEM_PD 配置是否关闭 USB 存储器。

0：无效

1：关闭

(R/W)

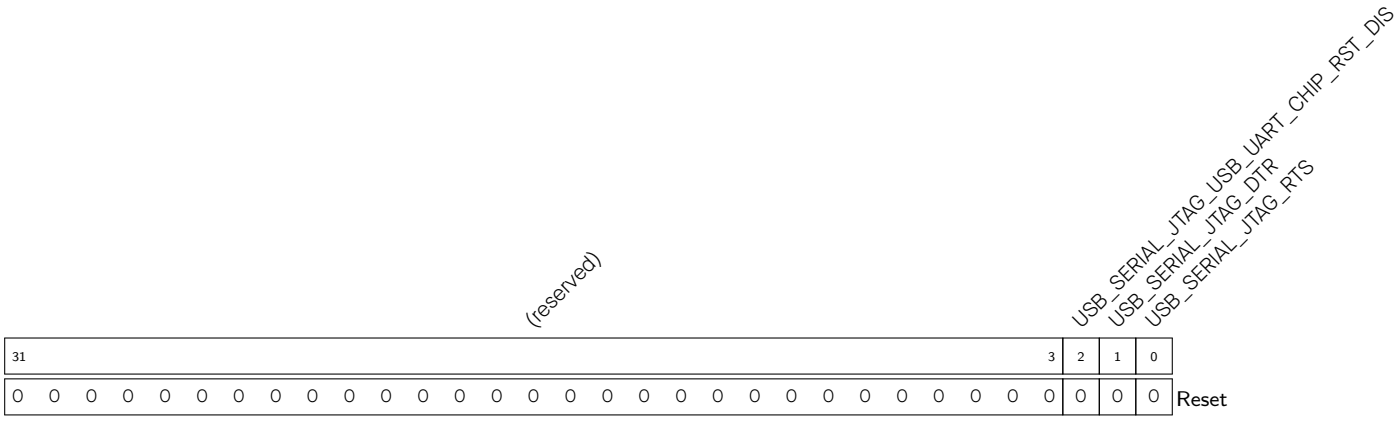
USB_SERIAL_JTAG_USB_MEM_CLK_EN 配置是否强制对 USB 存储器进行时钟分频。

0：无效

1：强制

(R/W)

Register 22.7. USB_SERIAL_JTAG_CHIP_RST_REG (0x004C)



USB_SERIAL_JTAG_RTS 表示最近的 SET_LINE_CODING 命令所设定的 RTS 信号的状态。(RO)

USB_SERIAL_JTAG_DTR 表示最近的 SET_LINE_CODING 命令所设定的 DTR 信号的状态。(RO)

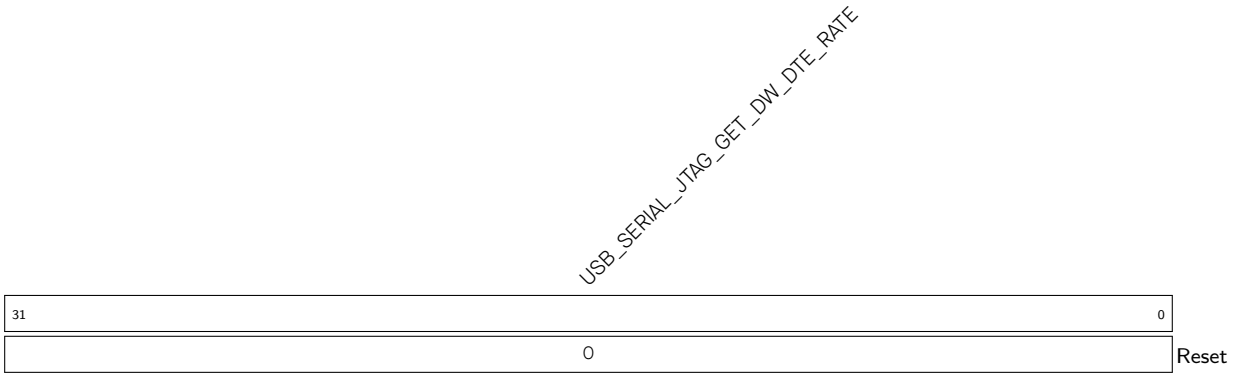
USB_SERIAL_JTAG_USB_UART_CHIP_RST_DIS 配置是否禁用来自 USB 串行通道的芯片复位。

0: 无效

1: 禁用

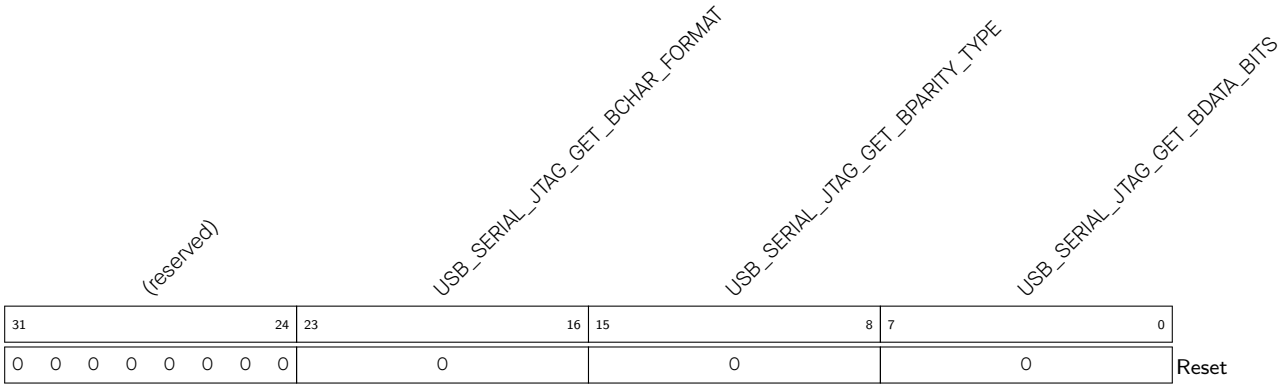
(R/W)

Register 22.8. USB_SERIAL_JTAG_GET_LINE_CODE_W0_REG (0x0058)



USB_SERIAL_JTAG_GET_DW_DTE_RATE 配置软件设置的 dwDTERate 值，该值由 GET_LINE_CODING 命令请求。(R/W)

Register 22.9. USB_SERIAL_JTAG_GET_LINE_CODE_W1_REG (0x005C)

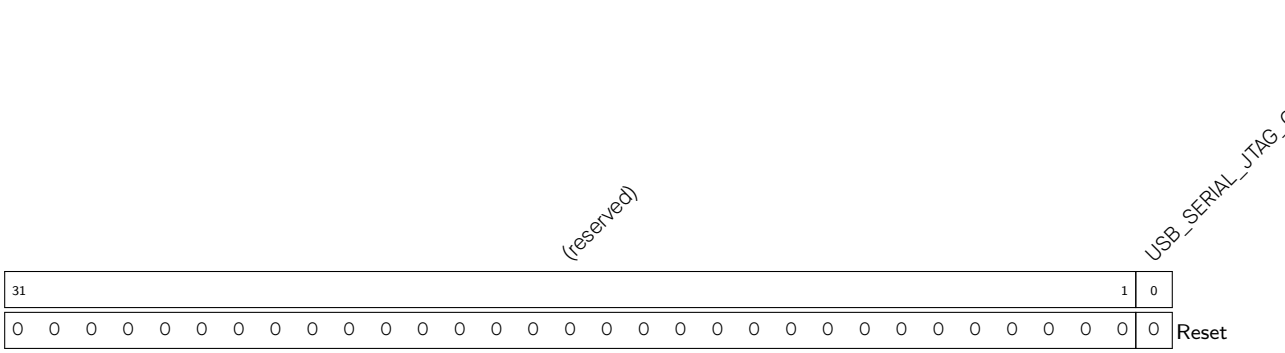


USB_SERIAL_JTAG_GET_BDATA_BITS 配置软件设置的 bDataBits 值，该值由 GET_LINE_CODING 命令请求。(R/W)

USB_SERIAL_JTAG_GET_BPARITY_TYPE 配置软件设置的 bParityType 值，该值由 GET_LINE_CODING 命令请求。(R/W)

USB_SERIAL_JTAG_GET_BCHAR_FORMAT 配置软件设置的 bCharFormat 值，该值由 GET_LINE_CODING 命令请求。(R/W)

Register 22.10. USB_SERIAL_JTAG_CONFIG_UPDATE_REG (0x0060)



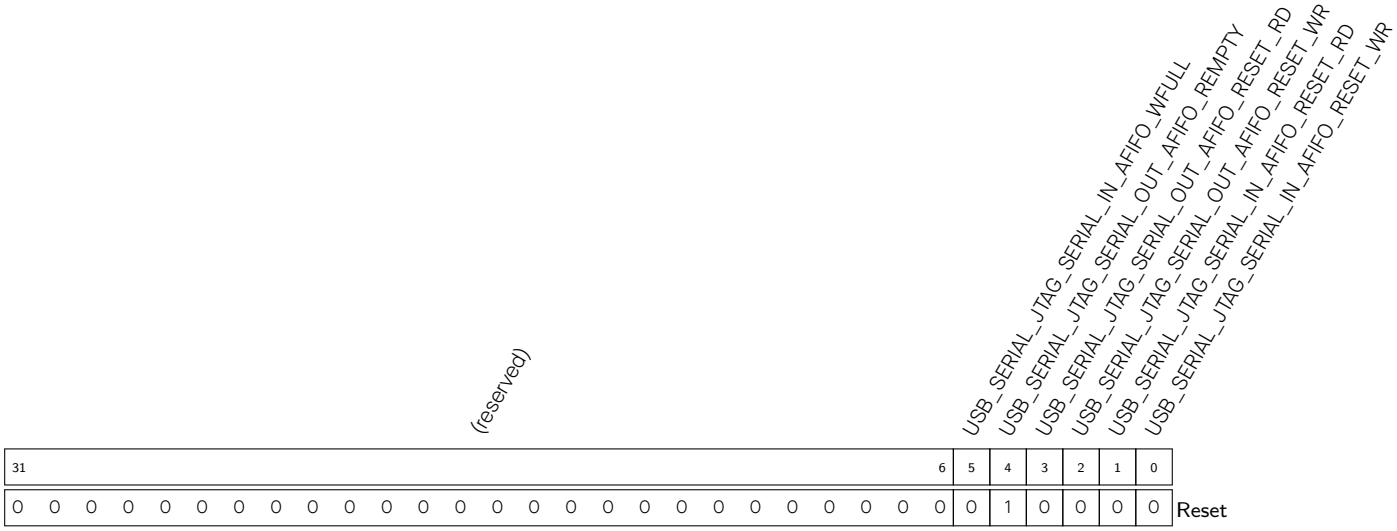
USB_SERIAL_JTAG_CONFIG_UPDATE 配置是否将配置寄存器的值从 APB 时钟域更新到 48 MHz 时钟域。

0: 无效

1: 更新

(WT)

Register 22.11. USB_SERIAL_JTAG_SER_AFIFO_CONFIG_REG (0x0064)



USB_SERIAL_JTAG_SERIAL_IN_AFIFO_RESET_WR 配置是否复位 CDC_ACM IN 异步 FIFO 写入时钟域。

0: 无效

1: 复位

(R/W)

USB_SERIAL_JTAG_SERIAL_IN_AFIFO_RESET_RD 配置是否复位 CDC_ACM IN 异步 FIFO 读取时钟域。

0: 无效

1: 复位

(R/W)

USB_SERIAL_JTAG_SERIAL_OUT_AFIFO_RESET_WR 配置是否复位 CDC_ACM OUT 异步 FIFO 写入时钟域。

0: 无效

1: 复位

(R/W)

USB_SERIAL_JTAG_SERIAL_OUT_AFIFO_RESET_RD 配置是否复位 CDC_ACM OUT 异步 FIFO 读取时钟域。

0: 无效

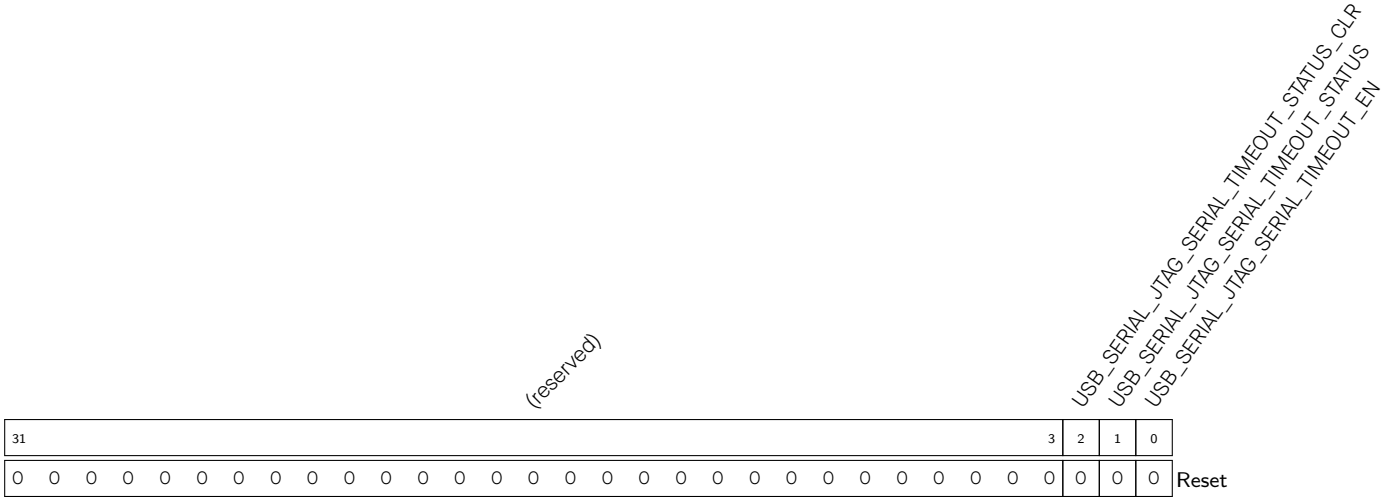
1: 复位

(R/W)

USB_SERIAL_JTAG_SERIAL_OUT_AFIFO_REMPTY 表示 CDC_ACM OUT 读取时钟域的异步 FIFO 空信号。(RO)

USB_SERIAL_JTAG_SERIAL_IN_AFIFO_WFULL 表示 CDC_ACM IN 写入时钟域的异步 FIFO 满信号。(RO)

Register 22.12. USB_SERIAL_JTAG_SERIAL_EP_TIMEOUT0_REG (0x006C)



USB_SERIAL_JTAG_SERIAL_TIMEOUT_EN 使能 USB 串口输出端点超时功能。超时事件发生时，串口输出端点缓冲区自动清除，USB_SERIAL_JTAG_SERIAL_TIMEOUT_STATUS 被置位。

0：禁用超时，缓冲区不会自动清除

1：使能超时

(R/W)

USB_SERIAL_JTAG_SERIAL_TIMEOUT_STATUS 超时状态。

0：未发生超时

1：串口输出端点触发了超时事件

(R/WTC/SS)

USB_SERIAL_JTAG_SERIAL_TIMEOUT_STATUS_CLR 写 1 清除 USB_SERIAL_JTAG_SERIAL_TIMEOUT_STATUS。

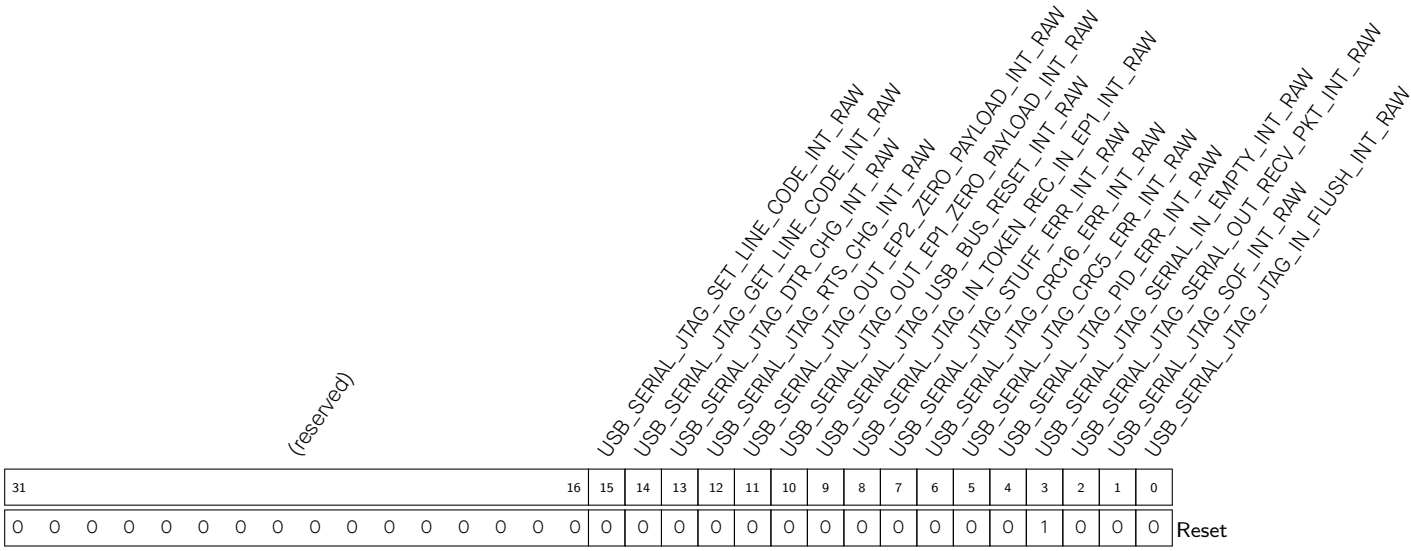
(WT)

Register 22.13. USB_SERIAL_JTAG_SERIAL_EP_TIMEOUT1_REG (0x0070)



USB_SERIAL_JTAG_SERIAL_TIMEOUT_MAX USB 串口输出端点的最大超时数值，表示等待芯片读取数据的最长等待时间，单位为 20.83 ns。(R/W)

Register 22.14. USB_SERIAL_JTAG_INT_RAW_REG (0x0008)



USB_SERIAL_JTAG_JTAG_IN_FLUSH_INT_RAW USB_SERIAL_JTAG_JTAG_IN_FLUSH_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_SOF_INT_RAW USB_SERIAL_JTAG_SOF_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT_RAW USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_SERIAL_IN_EMPTY_INT_RAW USB_SERIAL_JTAG_SERIAL_IN_EMPTY_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_PID_ERR_INT_RAW USB_SERIAL_JTAG_PID_ERR_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_CRC5_ERR_INT_RAW USB_SERIAL_JTAG_CRC5_ERR_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_CRC16_ERR_INT_RAW USB_SERIAL_JTAG_CRC16_ERR_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_STUFF_ERR_INT_RAW USB_SERIAL_JTAG_STUFF_ERR_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_IN_TOKEN_REC_IN_EP1_INT_RAW USB_SERIAL_JTAG_IN_TOKEN_REC_IN_EP1_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_USB_BUS_RESET_INT_RAW USB_SERIAL_JTAG_USB_BUS_RESET_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_OUT_EP1_ZERO_PAYLOAD_INT_RAW USB_SERIAL_JTAG_OUT_EP1_ZERO_PAYLOAD_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_OUT_EP2_ZERO_PAYLOAD_INT_RAW USB_SERIAL_JTAG_OUT_EP2_ZERO_PAYLOAD_INT 的原始中断状态。(R/WTC/SS)

见下页...

Register 22.14. USB_SERIAL_JTAG_INT_RAW_REG (0x0008)

接上页...

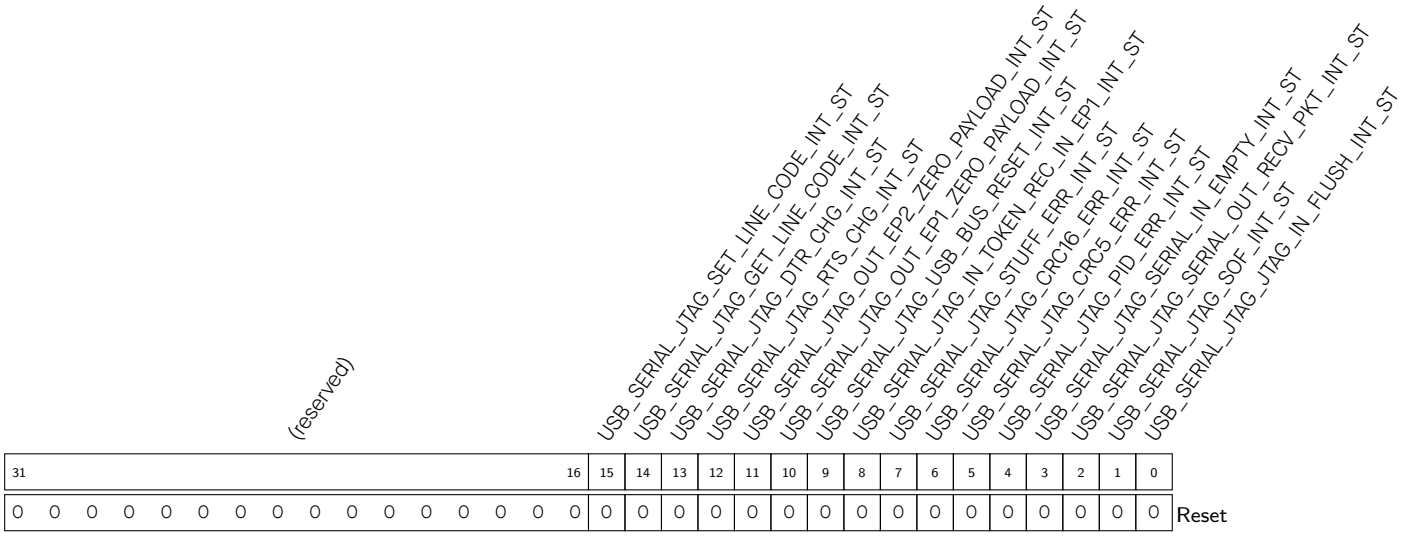
USB_SERIAL_JTAG_RTS_CHG_INT_RAW USB_SERIAL_JTAG_RTS_CHG_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_DTR_CHG_INT_RAW USB_SERIAL_JTAG_DTR_CHG_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_GET_LINE_CODE_INT_RAW USB_SERIAL_JTAG_GET_LINE_CODE_INT 的原始中断状态。(R/WTC/SS)

USB_SERIAL_JTAG_SET_LINE_CODE_INT_RAW USB_SERIAL_JTAG_SET_LINE_CODE_INT 的原始中断状态。(R/WTC/SS)

Register 22.15. USB_SERIAL_JTAG_INT_ST_REG (0x000C)



USB_SERIAL_JTAG_JTAG_IN_FLUSH_INT_ST USB_SERIAL_JTAG_JTAG_IN_FLUSH_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_SOF_INT_ST USB_SERIAL_JTAG_SOF_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT_ST USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_SERIAL_IN_EMPTY_INT_ST USB_SERIAL_JTAG_SERIAL_IN_EMPTY_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_PID_ERR_INT_ST USB_SERIAL_JTAG_PID_ERR_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_CRC5_ERR_INT_ST USB_SERIAL_JTAG_CRC5_ERR_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_CRC16_ERR_INT_ST USB_SERIAL_JTAG_CRC16_ERR_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_STUFF_ERR_INT_ST USB_SERIAL_JTAG_STUFF_ERR_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_IN_TOKEN_REC_IN_EP1_INT_ST USB_SERIAL_JTAG_IN_TOKEN_REC_IN_EP1_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_USB_BUS_RESET_INT_ST USB_SERIAL_JTAG_USB_BUS_RESET_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_OUT_EP1_ZERO_PAYLOAD_INT_ST USB_SERIAL_JTAG_OUT_EP1_ZERO_PAYLOAD_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_OUT_EP2_ZERO_PAYLOAD_INT_ST USB_SERIAL_JTAG_OUT_EP2_ZERO_PAYLOAD_INT 的屏蔽中断状态。(RO)

见下页...

Register 22.15. USB_SERIAL_JTAG_INT_ST_REG (0x000C)

接上页...

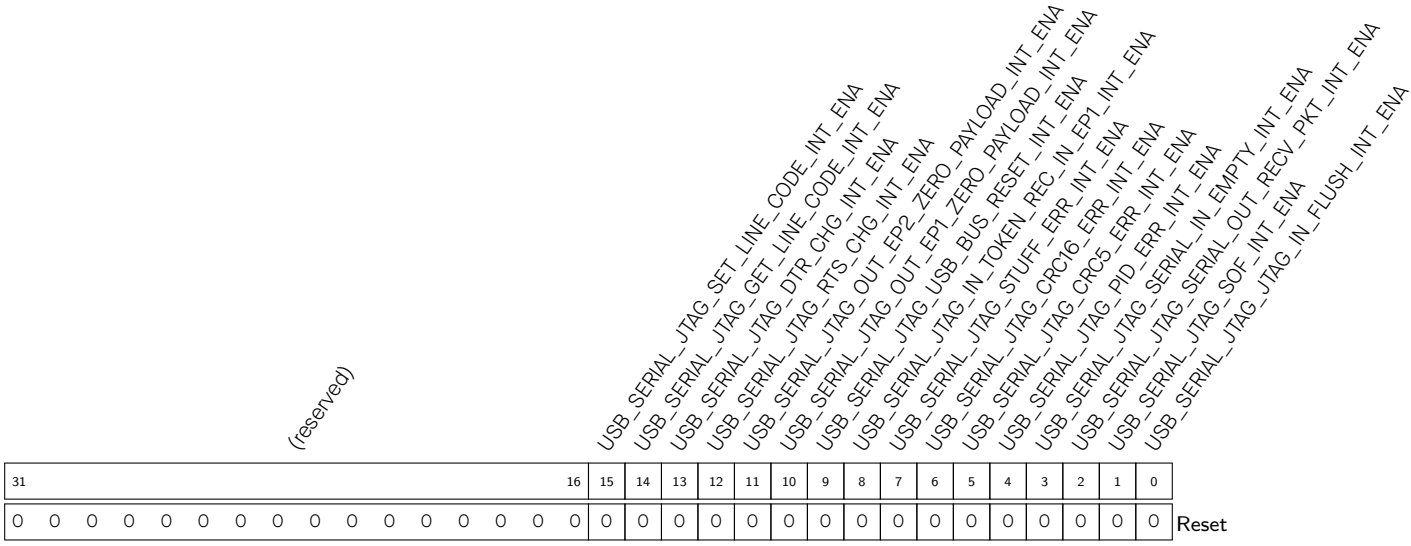
USB_SERIAL_JTAG_RTS_CHG_INT_ST USB_SERIAL_JTAG_RTS_CHG_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_DTR_CHG_INT_ST USB_SERIAL_JTAG_DTR_CHG_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_GET_LINE_CODE_INT_ST USB_SERIAL_JTAG_GET_LINE_CODE_INT 的屏蔽中断状态。(RO)

USB_SERIAL_JTAG_SET_LINE_CODE_INT_ST USB_SERIAL_JTAG_SET_LINE_CODE_INT 的屏蔽中断状态。(RO)

Register 22.16. USB_SERIAL_JTAG_INT_ENA_REG (0x0010)



USB_SERIAL_JTAG_JTAG_IN_FLUSH_INT_ENA 写 1 使能 USB_SERIAL_JTAG_JTAG_IN_FLUSH_INT 中断。(R/W)

USB_SERIAL_JTAG_SOF_INT_ENA 写 1 使能 USB_SERIAL_JTAG_SOF_INT 中断。(R/W)

USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT_ENA 写 1 使能 USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT 中断。(R/W)

USB_SERIAL_JTAG_SERIAL_IN_EMPTY_INT_ENA 写 1 使能 USB_SERIAL_JTAG_SERIAL_IN_EMPTY_INT 中断。(R/W)

USB_SERIAL_JTAG_PID_ERR_INT_ENA 写 1 使能 USB_SERIAL_JTAG_PID_ERR_INT 中断。(R/W)

USB_SERIAL_JTAG_CRC5_ERR_INT_ENA 写 1 使能 USB_SERIAL_JTAG_CRC5_ERR_INT 中断。(R/W)

USB_SERIAL_JTAG_CRC16_ERR_INT_ENA 写 1 使能 USB_SERIAL_JTAG_CRC16_ERR_INT 中断。(R/W)

USB_SERIAL_JTAG_STUFF_ERR_INT_ENA 写 1 使能 USB_SERIAL_JTAG_STUFF_ERR_INT 中断。(R/W)

USB_SERIAL_JTAG_IN_TOKEN_REC_IN_EP1_INT_ENA 写 1 使能 USB_SERIAL_JTAG_IN_TOKEN_REC_IN_EP1_INT 中断。(R/W)

USB_SERIAL_JTAG_USB_BUS_RESET_INT_ENA 写 1 使能 USB_SERIAL_JTAG_USB_BUS_RESET_INT 中断。(R/W)

USB_SERIAL_JTAG_OUT_EP1_ZERO_PAYLOAD_INT_ENA 写 1 使能 USB_SERIAL_JTAG_OUT_EP1_ZERO_PAYLOAD_INT 中断。(R/W)

USB_SERIAL_JTAG_OUT_EP2_ZERO_PAYLOAD_INT_ENA 写 1 使能 USB_SERIAL_JTAG_OUT_EP2_ZERO_PAYLOAD_INT 中断。(R/W)

见下页...

Register 22.16. USB_SERIAL_JTAG_INT_ENA_REG (0x0010)

接上页...

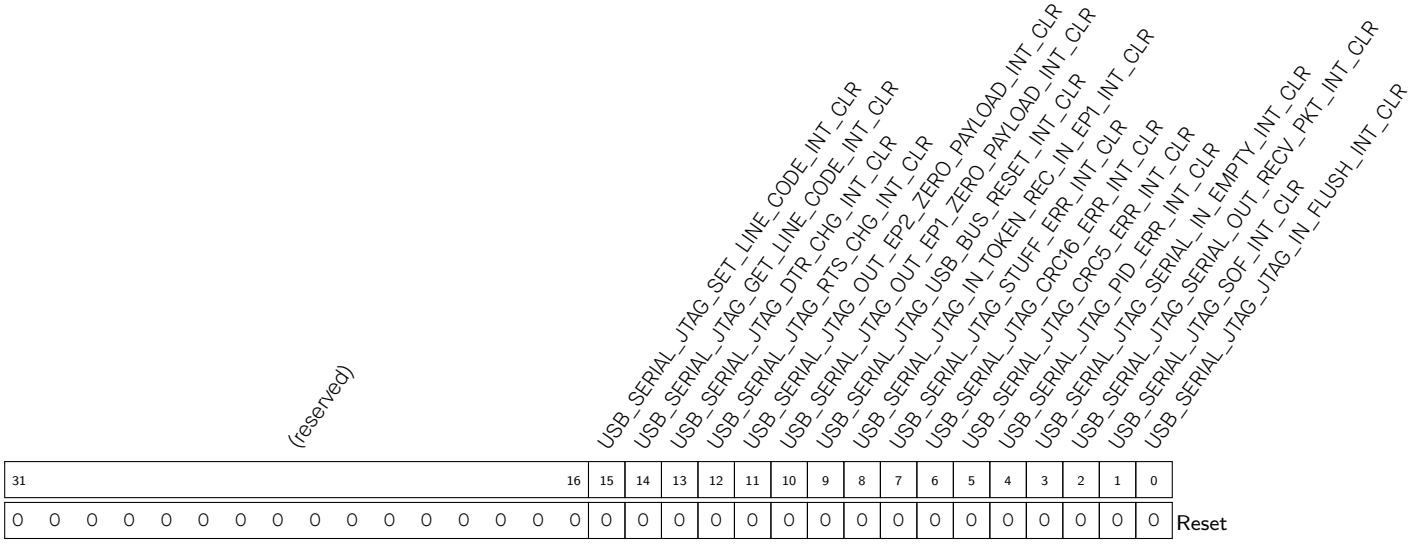
USB_SERIAL_JTAG_RTS_CHG_INT_ENA 写 1 使能 USB_SERIAL_JTAG_RTS_CHG_INT 中断。(R/W)

USB_SERIAL_JTAG_DTR_CHG_INT_ENA 写 1 使能 USB_SERIAL_JTAG_DTR_CHG_INT 中断。(R/W)

USB_SERIAL_JTAG_GET_LINE_CODE_INT_ENA 写 1 使能 USB_SERIAL_JTAG_GET_LINE_CODE_INT 中断。(R/W)

USB_SERIAL_JTAG_SET_LINE_CODE_INT_ENA 写 1 使能 USB_SERIAL_JTAG_SET_LINE_CODE_INT 中断。(R/W)

Register 22.17. USB_SERIAL_JTAG_INT_CLR_REG (0x0014)



USB_SERIAL_JTAG_JTAG_IN_FLUSH_INT_CLR 写 1 清除 USB_SERIAL_JTAG_JTAG_IN_FLUSH_INT 中断。(WT)

USB_SERIAL_JTAG_SOF_INT_CLR 写 1 清除 USB_SERIAL_JTAG_SOF_INT 中断。(WT)

USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT_CLR 写 1 清除 USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT 中断。(WT)

USB_SERIAL_JTAG_SERIAL_IN_EMPTY_INT_CLR 写 1 清除 USB_SERIAL_JTAG_SERIAL_IN_EMPTY_INT 中断。(WT)

USB_SERIAL_JTAG_PID_ERR_INT_CLR 写 1 清除 USB_SERIAL_JTAG_PID_ERR_INT 中断。(WT)

USB_SERIAL_JTAG_CRC5_ERR_INT_CLR 写 1 清除 USB_SERIAL_JTAG_CRC5_ERR_INT 中断。(WT)

USB_SERIAL_JTAG_CRC16_ERR_INT_CLR 写 1 清除 USB_SERIAL_JTAG_CRC16_ERR_INT 中断。(WT)

USB_SERIAL_JTAG_STUFF_ERR_INT_CLR 写 1 清除 USB_SERIAL_JTAG_STUFF_ERR_INT 中断。(WT)

USB_SERIAL_JTAG_IN_TOKEN_REC_IN_EP1_INT_CLR 写 1 清除 USB_SERIAL_JTAG_IN_TOKEN_REC_IN_EP1_INT 中断。(WT)

USB_SERIAL_JTAG_USB_BUS_RESET_INT_CLR 写 1 清除 USB_SERIAL_JTAG_USB_BUS_RESET_INT 中断。(WT)

USB_SERIAL_JTAG_OUT_EP1_ZERO_PAYLOAD_INT_CLR 写 1 清除 USB_SERIAL_JTAG_OUT_EP1_ZERO_PAYLOAD_INT 中断。(WT)

USB_SERIAL_JTAG_OUT_EP2_ZERO_PAYLOAD_INT_CLR 写 1 清除 USB_SERIAL_JTAG_OUT_EP2_ZERO_PAYLOAD_INT 中断。(WT)

见下页...

Register 22.17. USB_SERIAL_JTAG_INT_CLR_REG (0x0014)

接上页...

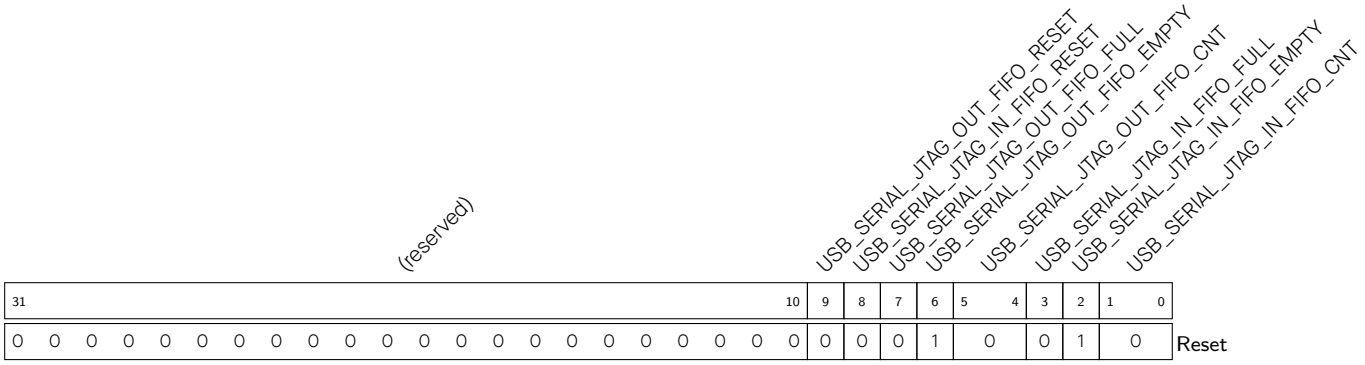
USB_SERIAL_JTAG_RTS_CHG_INT_CLR 写 1 清除 USB_SERIAL_JTAG_RTS_CHG_INT 中断。(WT)

USB_SERIAL_JTAG_DTR_CHG_INT_CLR 写 1 清除 USB_SERIAL_JTAG_DTR_CHG_INT 中断。(WT)

USB_SERIAL_JTAG_GET_LINE_CODE_INT_CLR 写 1 清除 USB_SERIAL_JTAG_GET_LINE_CODE_INT 中断。(WT)

USB_SERIAL_JTAG_SET_LINE_CODE_INT_CLR 写 1 清除 USB_SERIAL_JTAG_SET_LINE_CODE_INT 中断。(WT)

Register 22.18. USB_SERIAL_JTAG_JFIFO_ST_REG (0x0020)



USB_SERIAL_JTAG_IN_FIFO_CNT 表示 JTAG IN FIFO 计数器。(RO)

USB_SERIAL_JTAG_IN_FIFO_EMPTY 表示 JTAG IN FIFO 是否为空。

0：JTAG IN FIFO 不为空

1：JTAG IN FIFO 为空

(RO)

USB_SERIAL_JTAG_IN_FIFO_FULL 表示 JTAG IN FIFO 是否为满。

0：JTAG IN FIFO 不为满

1：JTAG IN FIFO 为满

(RO)

USB_SERIAL_JTAG_OUT_FIFO_CNT 表示 JTAG OUT FIFO 计数器。(RO)

USB_SERIAL_JTAG_OUT_FIFO_EMPTY 表示 JTAG OUT FIFO 是否为空。

0：JTAG OUT FIFO 不为空

1：JTAG OUT FIFO 为空

(RO)

USB_SERIAL_JTAG_OUT_FIFO_FULL 表示 JTAG OUT FIFO 是否为满。

0：JTAG OUT FIFO 不为满

1：JTAG OUT FIFO 为满

(RO)

USB_SERIAL_JTAG_IN_FIFO_RESET 配置是否复位 JTAG IN FIFO。

0：无效

1：复位

(R/W)

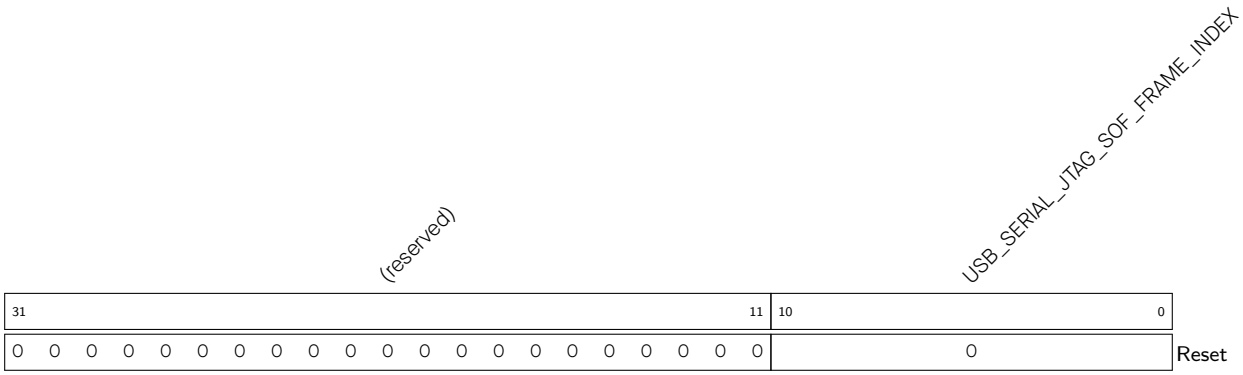
USB_SERIAL_JTAG_OUT_FIFO_RESET 配置是否复位 JTAG OUT FIFO。

0：无效

1：复位

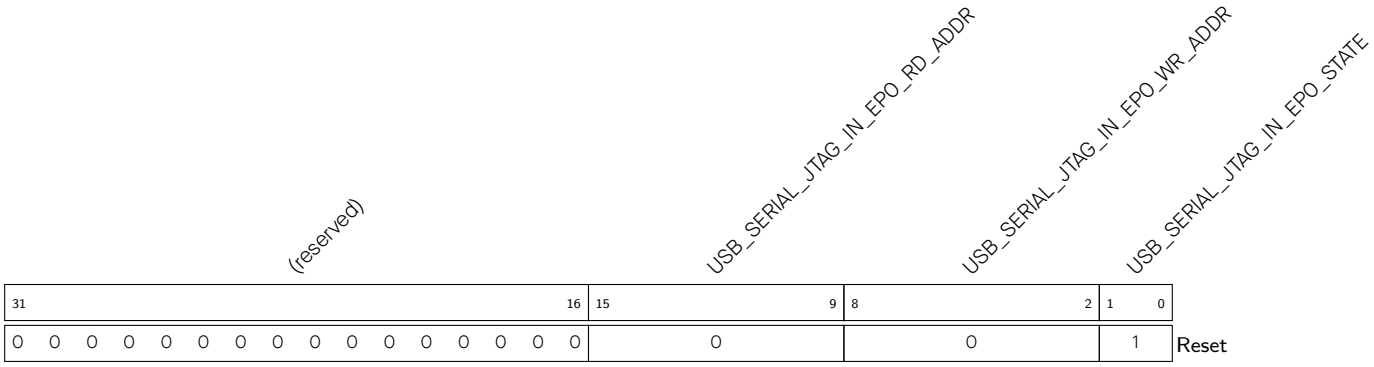
(R/W)

Register 22.19. USB_SERIAL_JTAG_FRAM_NUM_REG (0x0024)



USB_SERIAL_JTAG_SOF_FRAME_INDEX 接收 SOF 帧的帧索引。(RO)

Register 22.20. USB_SERIAL_JTAG_IN_EP0_ST_REG (0x0028)

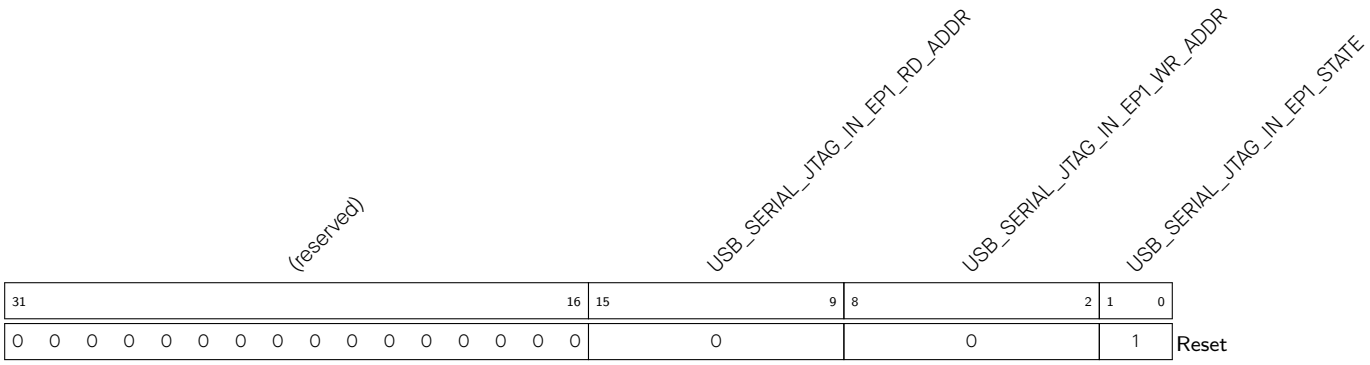


USB_SERIAL_JTAG_IN_EP0_STATE 表示输入端点 0 的状态。(RO)

USB_SERIAL_JTAG_IN_EP0_WR_ADDR 表示输入端点 0 的写入数据地址。(RO)

USB_SERIAL_JTAG_IN_EP0_RD_ADDR 表示输入端点 0 的读取数据地址。(RO)

Register 22.21. USB_SERIAL_JTAG_IN_EP1_ST_REG (0x002C)

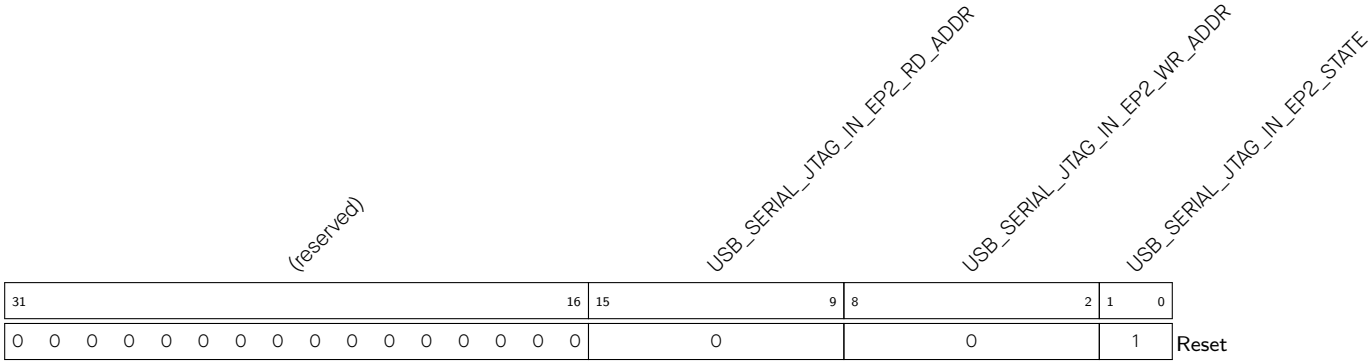


USB_SERIAL_JTAG_IN_EP1_STATE 表示输入端点 1 的状态。(RO)

USB_SERIAL_JTAG_IN_EP1_WR_ADDR 表示输入端点 1 的写入数据地址。(RO)

USB_SERIAL_JTAG_IN_EP1_RD_ADDR 表示输入端点 1 的读取数据地址。(RO)

Register 22.22. USB_SERIAL_JTAG_IN_EP2_ST_REG (0x0030)

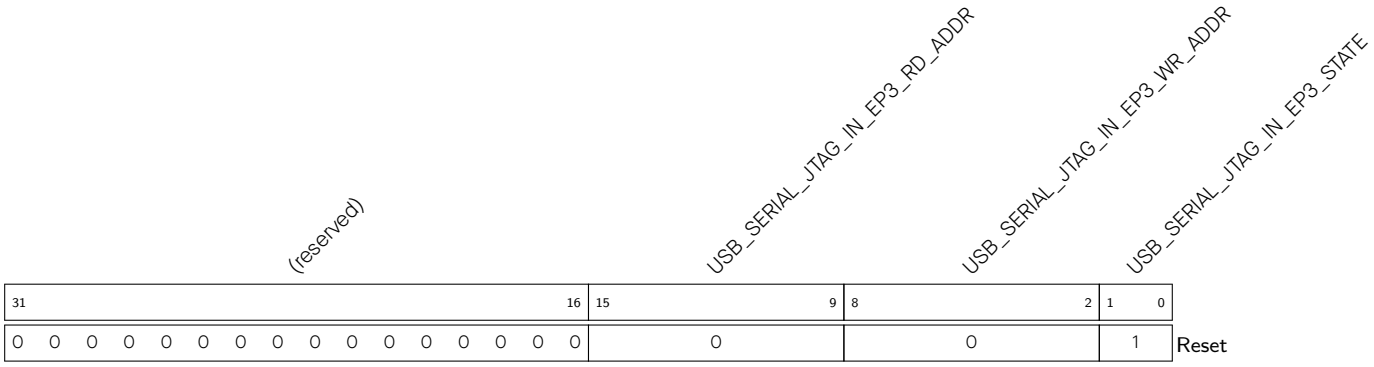


USB_SERIAL_JTAG_IN_EP2_STATE 表示输入端点 2 的状态。(RO)

USB_SERIAL_JTAG_IN_EP2_WR_ADDR 表示输入端点 2 的写入数据地址。(RO)

USB_SERIAL_JTAG_IN_EP2_RD_ADDR 表示输入端点 2 的读取数据地址。(RO)

Register 22.23. USB_SERIAL_JTAG_IN_EP3_ST_REG (0x0034)

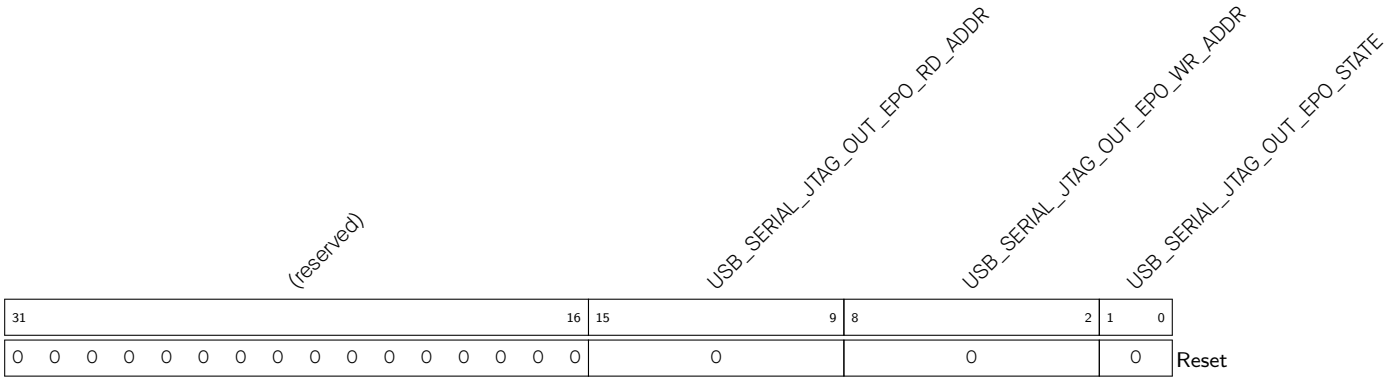


USB_SERIAL_JTAG_IN_EP3_STATE 表示输入端点 3 的状态。(RO)

USB_SERIAL_JTAG_IN_EP3_WR_ADDR 表示输入端点 3 的写入数据地址。(RO)

USB_SERIAL_JTAG_IN_EP3_RD_ADDR 表示输入端点 3 的读取数据地址。(RO)

Register 22.24. USB_SERIAL_JTAG_OUT_EP0_ST_REG (0x0038)



USB_SERIAL_JTAG_OUT_EP0_STATE 表示输出端点 0 的状态。(RO)

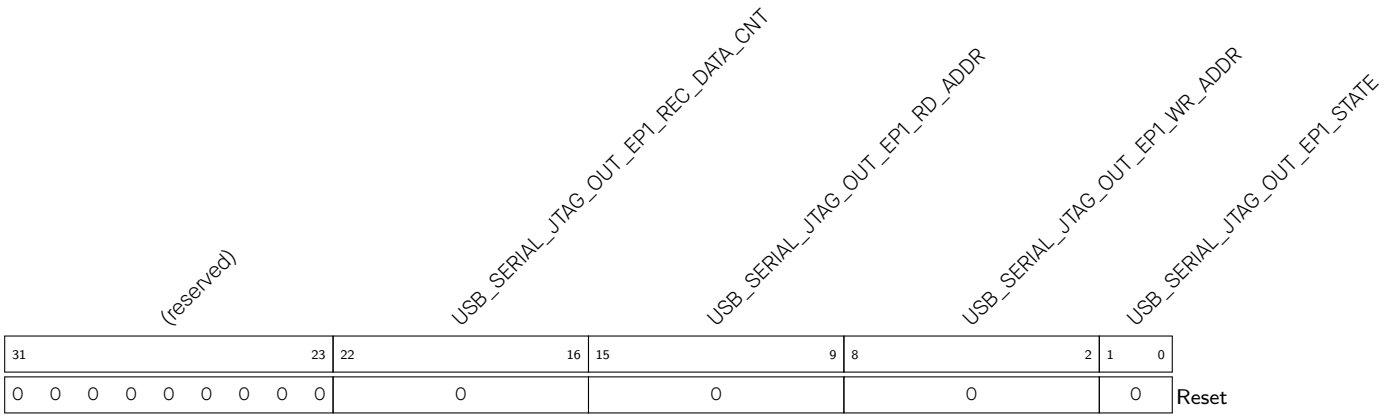
USB_SERIAL_JTAG_OUT_EP0_WR_ADDR 表示输出端点 0 的写入数据地址。

当检测到 USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT 中断时，输出端点 0 中有 (USB_SERIAL_JTAG_OUT_EP0_WR_ADDR − 2) 个字节数据。

(RO)

USB_SERIAL_JTAG_OUT_EP0_RD_ADDR 表示输出端点 0 的读取数据地址。(RO)

Register 22.25. USB_SERIAL_JTAG_OUT_EP1_ST_REG (0x003C)



USB_SERIAL_JTAG_OUT_EP1_STATE 表示输出端点 1 的状态。(RO)

USB_SERIAL_JTAG_OUT_EP1_WR_ADDR 表示输出端点 1 的写入数据地址。

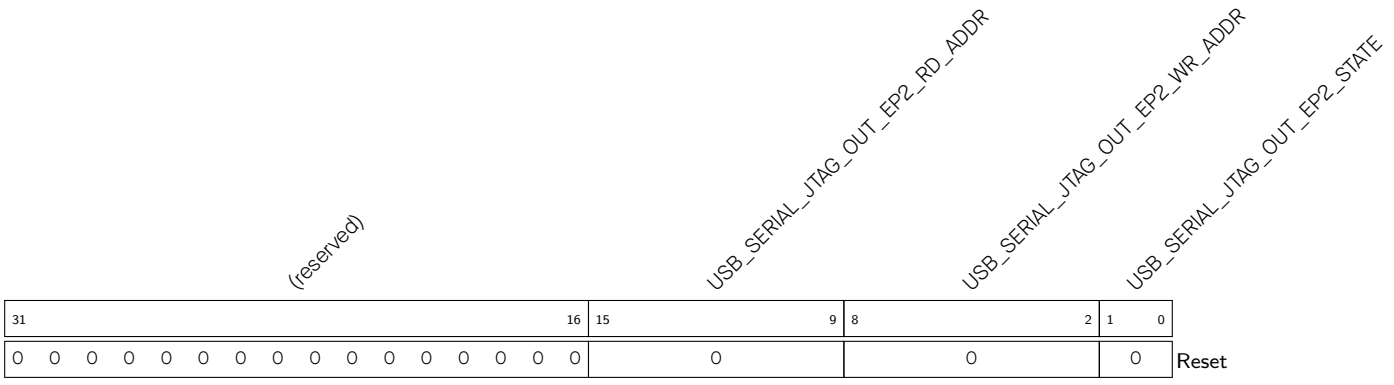
当检测到 USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT 中断时，输出端点 1 中有 (USB_SERIAL_JTAG_OUT_EP1_WR_ADDR – 2) 个字节数据。

(RO)

USB_SERIAL_JTAG_OUT_EP1_RD_ADDR 表示输出端点 1 的读取数据地址。(RO)

USB_SERIAL_JTAG_OUT_EP1_REC_DATA_CNT 当接收到一个数据包时输出端点 1 中的数据计数器。(RO)

Register 22.26. USB_SERIAL_JTAG_OUT_EP2_ST_REG (0x0040)



USB_SERIAL_JTAG_OUT_EP2_STATE 表示输出端点 2 的状态。(RO)

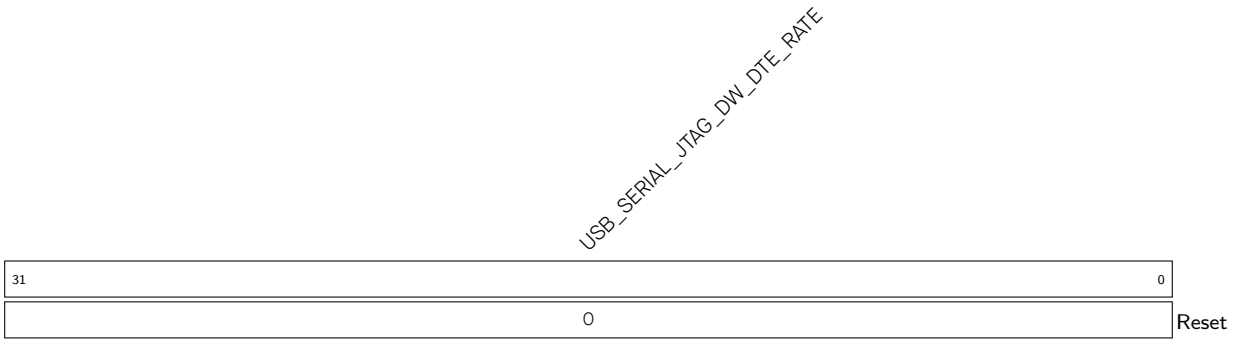
USB_SERIAL_JTAG_OUT_EP2_WR_ADDR 表示输出端点 2 的写入数据地址。

当检测到 USB_SERIAL_JTAG_SERIAL_OUT_RECV_PKT_INT 中断时，输出端点 2 中有 (USB_SERIAL_JTAG_OUT_EP2_WR_ADDR – 2) 个字节数据。

(RO)

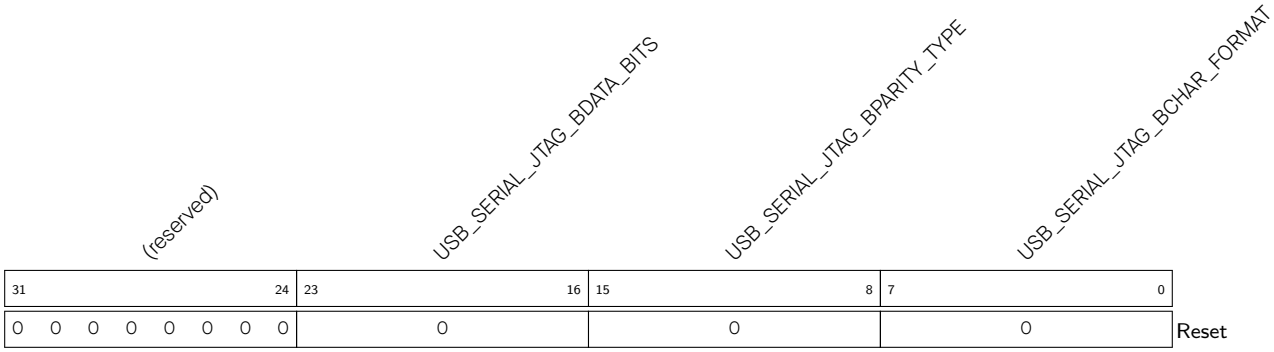
USB_SERIAL_JTAG_OUT_EP2_RD_ADDR 表示输出端点 2 的读取数据地址。(RO)

Register 22.27. USB_SERIAL_JTAG_SET_LINE_CODE_W0_REG (0x0050)



USB_SERIAL_JTAG_DW_DTE_RATE 表示主机通过 SET_LINE_CODING 命令设置的 dwDTERate 值。(RO)

Register 22.28. USB_SERIAL_JTAG_SET_LINE_CODE_W1_REG (0x0054)

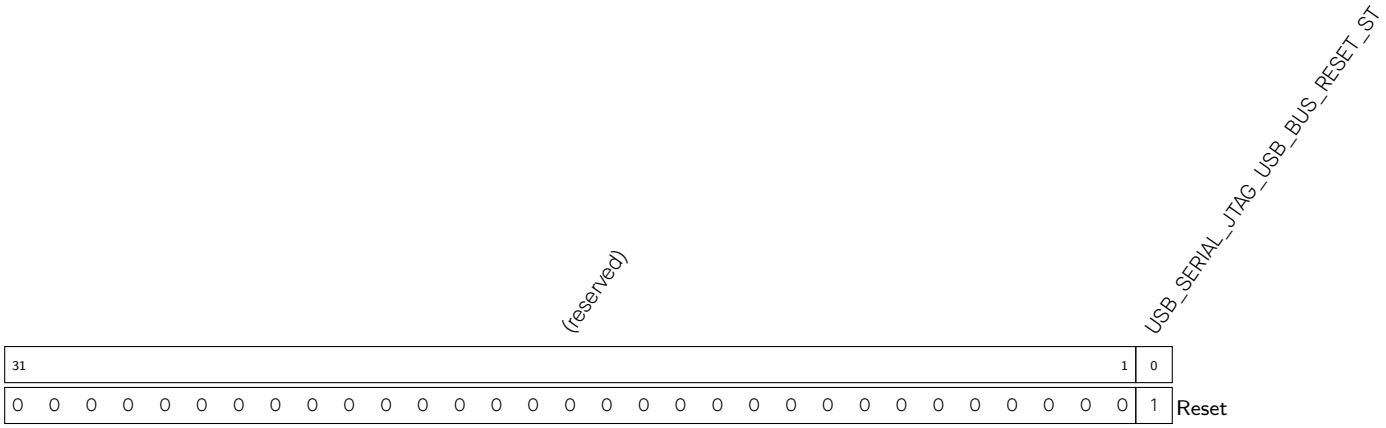


USB_SERIAL_JTAG_BCHAR_FORMAT 表示主机通过 SET_LINE_CODING 命令设置的 bCharFormat 值。(RO)

USB_SERIAL_JTAG_BPARITY_TYPE 表示主机通过 SET_LINE_CODING 命令设置的 bParityTpye 值。(RO)

USB_SERIAL_JTAG_BDATA_BITS 表示主机通过 SET_LINE_CODING 命令设置的 bDataBits 值。(RO)

Register 22.29. USB_SERIAL_JTAG_BUS_RESET_ST_REG (0x0068)



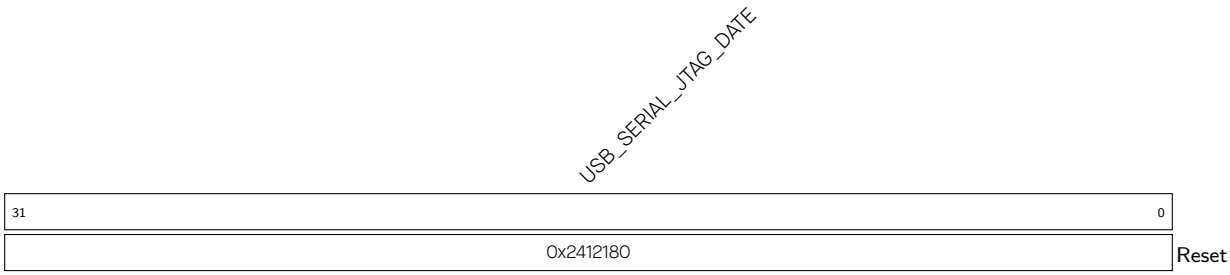
USB_SERIAL_JTAG_USB_BUS_RESET_ST 表示 USB 总线复位是否被释放。

0：USB 串口/JTAG 控制器处于 USB 总线复位状态

1：USB 总线复位被释放

(RO)

Register 22.30. USB_SERIAL_JTAG_DATE_REG (0x0080)



USB_SERIAL_JTAG_DATE 版本控制寄存器。(R/W)

第 23 章

LED PWM 控制器 (LEDC)

23.1 概述

LED PWM 控制器用于生成控制 LED 的脉冲宽度调制信号 (PWM)，具有占空比自动渐变等功能。该外设也可生成 PWM 信号用作其他用途。

23.2 特性

LED PWM 控制器具有如下特性:

- 两个独立的 LED PWM 控制器（LEDC0、LEDC1），每个包含 8 个独立的 PWM 生成通道（共计 16 个通道）
- PWM 占空比最大精度为 20 位
- 每个 LED PWM 控制器包含 4 个独立的定时器，可实现小数分频
- PWM 输出信号相位可调节
- PWM 占空比微调
- 占空比自动渐变，即 PWM 信号占空比可逐渐增加或减小，无须处理器干预，渐变完成时产生中断
- 每个 PWM 生成器包含 16 个占空比渐变区间，用于生成占空比伽马曲线渐变的信号。每个区间都可以独立配置占空比变化方向、变化步长、变化次数以及变化频率
- 低功耗模式 (Light-sleep mode) 下可输出 PWM 信号
- 支持通过事件任务矩阵 (ETM) 生成事件并响应任务

两个独立的 LED PWM 控制器完全相同，下文中以其中一个为例进行介绍。

由于四个定时器具有相同的功能和运行方式，下文将四个定时器统称为定时器*x*（*x* 的范围是 0 到 3）。同样的，八个 PWM 生成器的功能和运行方式也相同，因此下文将统称为 PWM*n*（*n* 的范围是 0 到 7）。

23.3 架构概览

图 23.3-1 为 LED PWM 控制器的架构。

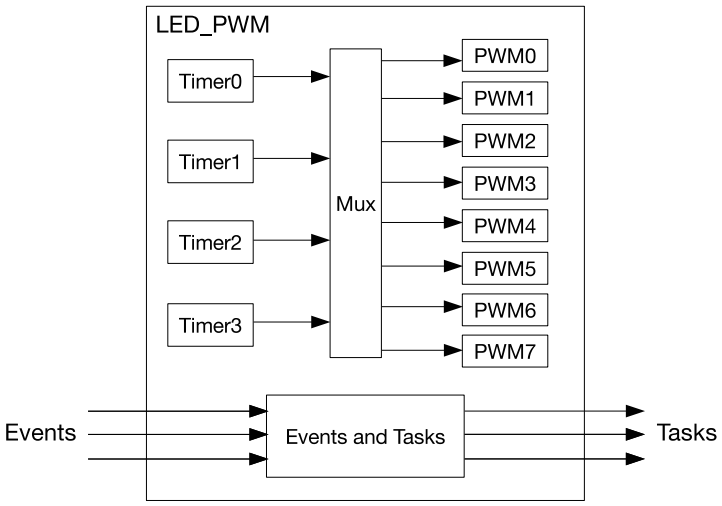


图 23.3-1. LED PWM 控制器架构

四个定时器每个内部都有一个时基计数器（即基于基准时钟周期计数的计数器），因此每个定时器都可以独立配置（可配置时钟分频器和计数器最大值）。每个 PWM 生成器通过配置 LEDC_TIMER_SEL_CH*n* 可以在四个定时器中择一，并以该定时器的计数值 timer*x*_cnt 为基准生成 PWM 信号。

图 23.3-2 为定时器和 PWM 生成器的主要功能块。

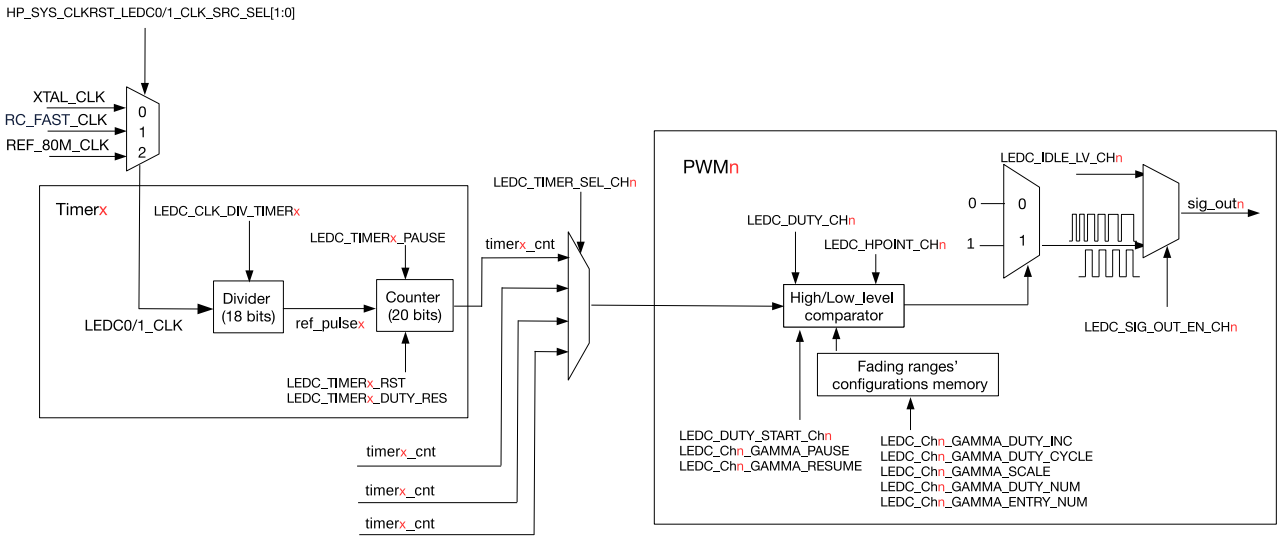


图 23.3-2. 定时器和 PWM 生成器功能框图

23.4 功能描述

23.4.1 时钟

LED PWM 控制器有两种时钟：

- APB_CLK：用于配置 LED PWM 寄存器的总线时钟
- LEDC0/1_CLK：LED PWM 控制器工作使用的功能时钟

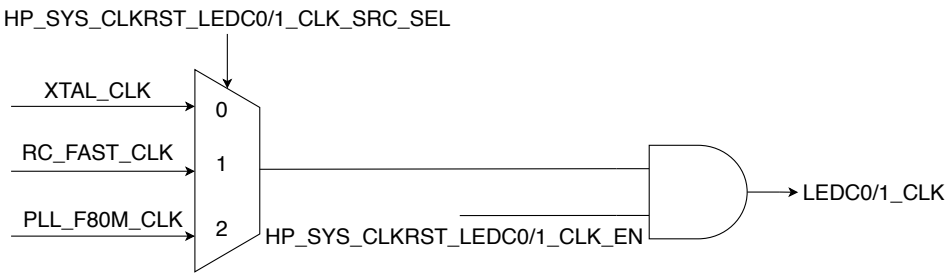


图 23.4-1. LEDC_CLK 时钟图

如图 23.4-1 所示，功能时钟 LEDC_CLK 有以下时钟源：

- XTAL_CLK
- RC_FAST_CLK
- PLL_F80M_CLK

有关时钟的更多信息，请查看章节 7 *复位和时钟 [to be added later]*。

有关具体的时钟配置流程，请查看章节 23.7 配置流程。

23.4.2 复位

LED PWM 可通过置位 HP_SYS_CLKRST_LEDC_CTRL0_REG 寄存器的 HP_SYS_CLKRST_LEDC0_RST_EN 或 HP_SYS_CLKRST_LEDC1_RST_EN 字段进行完全复位。

23.4.3 定时器

LED PWM 控制器的每个定时器内部都有一个时基计数器。图 23.3-2 中时基计数器使用的时钟信号称为 ref_pulse*x*。所有定时器使用同一个时钟源信号 LEDC_CLK，该时钟源信号经分频器分频后产生 ref_pulse*x* 供计数器使用。每个定时器工作状态独立可控，通过置位对应的 LEDC_TIMER*x*_POWER_UP 进行使能，通过清零对应的 LEDC_TIMER*x*_POWER_UP 停止工作。

23.4.3.1 时钟分频器配置

LEDC_CLK 信号传输到时钟分频器，产生 ref_pulse*x* 信号供计数器使用。ref_pulse*x* 的频率等于 LEDC_CLK 的频率经分频系数 LEDC_CLK_DIV 分频后的结果（见图 23.3-2）。

由于时钟分频器为小数分频，因此分频系数 LEDC_CLK_DIV 可为非整数，其可根据下列等式配置：

$$\text{LEDC_CLK_DIV} = A + \frac{B}{256}$$

- 整数部分 *A* 为 LEDC_CLK_DIV_TIMER*x* 字段的高 10 位（即 LEDC_TIMER*x*_CONF_REG[22:13]）
- 小数部分 *B* 为 LEDC_CLK_DIV_TIMER*x* 字段的低 8 位（即 LEDC_TIMER*x*_CONF_REG[12:5]）

小数部分 *B* 为 0 时，LEDC_CLK_DIV 的值为整数（整数分频）。也就是说，每 *A* 个 LEDC_CLK 时钟周期产生一个 ref_pulse*x* 时钟脉冲。

小数部分 *B* 不为 0 时，LEDC_CLK_DIV 的值为非整数。时钟分频器按照 *A* 个 LEDC_CLK 时钟周期和 (*A*+1) 个 LEDC_CLK 时钟周期轮流产生 ref_pulse*x* 时钟脉冲，实现非整数分频。这样一来，ref_pulse*x* 时钟脉冲的平均频率便会是理想值（非整数分频的频率）。每 256 个 ref_pulse*x* 时钟脉冲中：

- 有 *B* 个 ref_pulse*x* 时钟脉冲以 (*A*+1) 个 LEDC_CLK 时钟周期分频
- 有 (256-*B*) 个 ref_pulse*x* 时钟脉冲以 *A* 个 LEDC_CLK 时钟周期分频
- 以 (*A*+1) 个 LEDC_CLK 时钟周期分频的 ref_pulse*x* 时钟脉冲均匀分布在以 *A* 个 LEDC_CLK 时钟周期分频的 ref_pulse*x* 时钟脉冲中

图 23.4-2 展示了分频系数 LEDC_CLK_DIV 非整数时，LEDC_CLK 时钟脉冲和 ref_pulse*x* 时钟脉冲的关系。

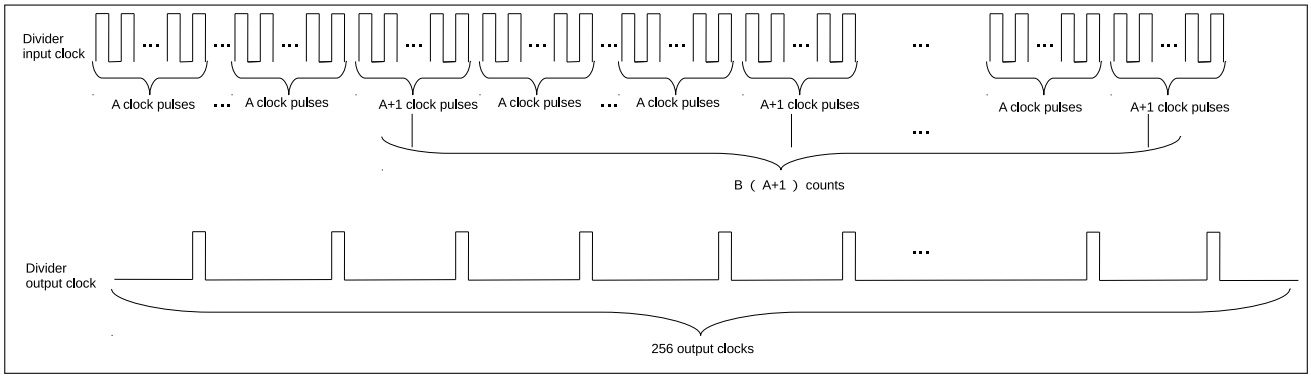


图 23.4-2. LEDC_CLK_DIV 为非整数时的分频

在运行时改变定时器时钟的分频系数，需先配置 LEDC_CLK_DIV_TIMER*x* 字段，然后置位 LEDC_TIMER*x*_PARA_UP 字段应用新配置。新配置会在计数器下次溢出时生效。LEDC_TIMER*x*_PARA_UP 字段由硬件自动清除。

23.4.3.2 20 位计数器

每个定时器有一个以 ref_pulse*x* 为基准时钟的 20 位时基计数器（见图 23.3-2）。LEDC_TIMER*x*_DUTY_RES 字段用于配置 20 位计数器的实际使用位宽。因此，PWM 信号的最大精度为 20 位。计数器最大可计数至 $2^{\text{LEDC_TIMER}x\text{_DUTY_RES}} - 1$，然后溢出并重新从 0 开始计数。软件可以读取、复位、暂停计数器。计数器和 PWM 信号精度的关系如图 23.4-3 所示。

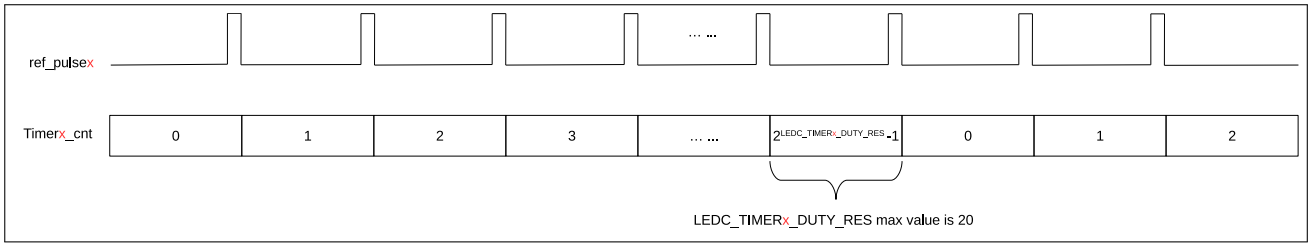


图 23.4-3. 计数器和 PWM 信号精度关系

计数器可在每次溢出时触发 LEDC_TIMER*x*_OVF_INT 中断（这个中断为硬件自动产生，不需要配置）。计数器也可配置为在溢出 LEDC_OVF_NUM_CH*n* + 1 次时触发 LEDC_OVF_CNT_CH*n*_INT 中断，该中断配置步骤如下：

1. 配置 LEDC_TIMER_SEL_CH*n*，为 PWM 生成器选择该定时器
2. 置位 LEDC_OVF_CNT_EN_CH*n*，使能溢出次数计数器
3. 把 LEDC_OVF_NUM_CH*n* 的值设为触发中断的计数器溢出次数减 1

4. 置位 LEDC_OVF_CNT_CH*n*_INT_ENA，使能溢出中断
5. 配置 LEDC_TIMER*x*_DUTY_RES，为所选的定时器 Timer*x* 指定计数器的位宽，等待 LEDC_OVF_CNT_CH*n*_INT 中断产生

若想在运行时改变计数器的最大值，需要先配置 LEDC_TIMER*x*_DUTY_RES 字段，然后置位 LEDC_TIMER*x*_PARA_UP 字段。新的配置在计数器下一次溢出时生效。如果重新配置 LEDC_OVF_CNT_EN_CH*n* 字段，需置位 LEDC_PARA_UP_CH*n* 应用新配置。总之，更改配置时需置位 LEDC_TIMER*x*_PARA_UP 或 LEDC_PARA_UP_CH*n* 应用新配置。
LEDC_TIMER*x*_PARA_UP 和 LEDC_PARA_UP_CH*n* 字段由硬件自动清除。

如图 23.3-2 所示，PWM 生成器输出信号 sig_out*n* 的频率取决于定时器时钟源 LEDC_CLK 的频率、时钟分频系数 LEDC_CLK_DIV 以及占空比精度（计数器位宽）LEDC_TIMER*x*_DUTY_RES：

$$f_{\text{PWM}} = \frac{f_{\text{LEDC_CLK}}}{\text{LEDC_CLK_DIV} \cdot 2^{\text{LEDC_TIMER}x\text{_DUTY_RES}}}$$

上述公式变形后，可得到以下公式计算预期的占空比精度：

$$\text{LEDC_TIMER}x\text{_DUTY_RES} = \log_2\left(\frac{f_{\text{LEDC_CLK}}}{f_{\text{PWM}} \cdot \text{LEDC_CLK_DIV}}\right)$$

表 23.4-1 列出了常用配置频率及其对应精度。

表 23.4-1. 常用配置频率及精度

LEDC_CLK	PWM 频率	最高精度（位）[1]	最低精度（位）[2]
REF_80M_CLK (80 MHz)	1 kHz	16	7
REF_80M_CLK (80 MHz)	5 kHz	13	4
REF_80M_CLK (80 MHz)	10 kHz	12	3
XTAL_40M_CLK (40 MHz)	1 kHz	15	6
XTAL_40M_CLK (40 MHz)	4 kHz	13	4
FOSC_20M_CLK (20 MHz)	1 kHz	14	5
FOSC_20M_CLK (20 MHz)	2 kHz	13	4

[1] 最高精度指时钟分频系数 LEDC_CLK_DIV 为 1 时的精度。如果经公式计算出的最高精度超过了计数器位宽 20 位，则最高精度为 20。
[2] 最低精度指时钟分频系数 LEDC_CLK_DIV 为 $1023 + \frac{255}{256}$ 时的精度。如果经公式计算出的最低精度小于 0，则最低精度为 1。

23.4.3.3 定时器配置限制

对于定时器的时钟分频器的分频系数 LEDC_CLK_DIV 和 20 位计数器的占空比精度 LEDC_TIMER*x*_DUTY_RES，其需要恒满足：

$$\text{LEDC_CLK_DIV} \cdot 2^{\text{LEDC_TIMER}x\text{_DUTY_RES}} \geq 8 \tag{23.1}$$

23.4.4 PWM 生成器

每个 LED PWM 生成器工作状态独立可控，通过置位对应的 LEDC_CH*n*_POWER_UP 进行使能。

要生成 PWM 信号，PWM 生成器 (PWM*n*) 需选择一个定时器 (Timer*x*)。每个 PWM 生成器均可通过置位 LEDC_TIMER_SEL_CH*n* 单独配置，在四个定时器中选择一个用于生成 PWM 信号。

如图 23.3-2 所示，每个 PWM 生成器主要包括一个高低电平比较器和两个选择器。PWM 生成器将定时器的 20 位计数值 (Timer*x*_cnt) 与高低电平比较器的值 Hpoint*n* 和 Lpoint*n* 比较。如果定时器的计数值等于 Hpoint*n* 或 Lpoint*n*，PWM 信号可以输出高低电平：

- 如果 Timer*x*_cnt == Hpoint*n*，则 sig_out*n* 为 1。
- 如果 Timer*x*_cnt == Lpoint*n*，则 sig_out*n* 为 0。

图 23.4-4 展示了如何使用 Hpoint*n* 和 Lpoint*n* 生成占空比固定的 PWM 信号。

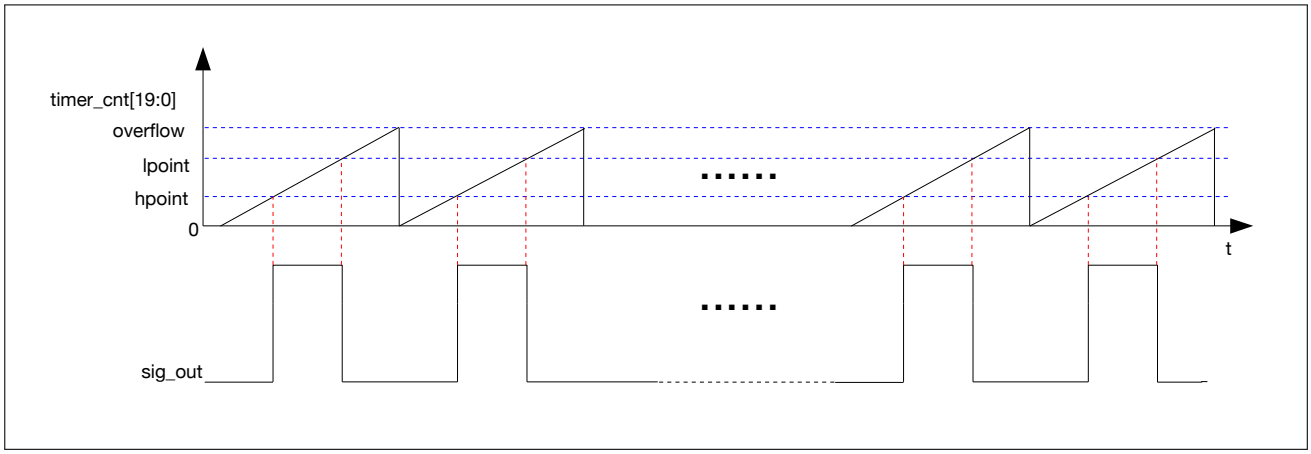


图 23.4-4. LED PWM 输出信号图

每当所选定时器的计数器溢出时，PWM 生成器 (PWM*n*) 的 Hpoint*n* 值更新为 LEDC_HPOINT_CH*n*。Lpoint*n* 的值同样在计数器每次溢出时更新，为 LEDC_DUTY_CH*n*[24:4] 和 LEDC_HPOINT_CH*n* 的和。通过配置 LEDC_DUTY_CH*n*[24:4] 和 LEDC_HPOINT_CH*n* 两个字段可对 Hpoint*n* 和 Lpoint*n* 进行设置，进而可设置 PWM 输出信号的相对相位和占空比。

置位 LEDC_SIG_OUT_EN_CH*n*，开启 PWM 信号 (sig_out*n*) 输出；清除 LEDC_SIG_OUT_EN_CH*n*，关闭 PWM 信号输出，输出信号 (sig_out*n*) 输出恒定电平，电平值为 LEDC_IDLE_LV_CH*n*。

23.4.4.1 PWM 输出信号微调

LEDC_DUTY_CH*n*[3:0] 通过周期性改变 PWM 输出信号 sig_out*n* 的占空比实现微调。如 LEDC_DUTY_CH*n*[3:0] 不为 0，那么 sig_out*n* 每 16 个周期中，有 LEDC_DUTY_CH*n*[3:0] 个周期的 PWM 脉冲占空比要比 (16 - LEDC_DUTY_CH*n*[3:0]) 个周期的脉冲占空比多一个定时器的计数周期。比如，如果 LEDC_DUTY_CH*n*[24:4] 设为 10，LEDC_DUTY_CH*n*[3:0] 设为 5，则 16 个周期中，有 5 个周期的 PWM 脉冲占空比为 11，剩余 11 个周期的 PWM 脉冲占空比为 10。16 个周期的平均占空比为 10.3125。

23.4.4.2 应用新配置

如果重新配置 LEDC_TIMER_SEL_CH*n*、LEDC_HPOINT_CH*n*、LEDC_DUTY_CH*n*[24:4] 和 LEDC_SIG_OUT_EN_CH*n* 字段，需置位 LEDC_PARA_UP_CH*n* 应用新配置。新配置在计数器下次溢出时生效。LEDC_PARA_UP_CH*n* 字段由硬件自动清除。

如果停止 LED PWM 生成器的工作，则清空对应的 LEDC_CH*n*_POWER_UP。

23.4.5 占空比渐变

PWM 生成器可以渐变 PWM 输出信号的占空比（即由一种占空比逐渐变为另一种占空比）。每个 PWM 生成器最多可以包含 16 个占空比渐变区间，每个区间可以独立配置占空比渐变方向（增加或减少）、渐变步长、渐变次数和渐变频率。如果开启占空比渐变功能，每个区间的 Lpoint*n* 的值会根据该区间的渐变配置进行变化。

23.4.5.1 线性占空比渐变

通过配置第一个占空比渐变区间的渐变方向、渐变步长、渐变次数和渐变频率，可以产生线性占空比渐变 PWM 信号。

配置步骤请见章节 23.7 配置流程。

当上述配置步骤完成后，每当计数器溢出 LEDC_CH*n*_GAMMA_DUTY_CYCLE 次，PWM 生成器将 PWM 信号的占空比渐变一次。PWM 信号每次渐变时，其 Lpoint*n* 会增加或减少（由 LEDC_CH*n*_GAMMA_DUTY_INC 配置）LEDC_CH*n*_GAMMA_SCALE；相应地，占空比会增加或减少 ΔDC（由 LEDC_CH*n*_GAMMA_DUTY_INC 配置）。

$$\Delta DC = \frac{\text{LEDC_CH}n\text{_GAMMA_SCALE}}{\text{LEDC_TIMER}x\text{_DUTY_RES}} \tag{23.2}$$

PWM 信号的占空比会渐变 LEDC_CH*n*_GAMMA_DUTY_NUM 次。完成该次数的渐变后，PWM 信号的占空比停止变化并保持以该占空比输出。由于每次渐变时占空比增加/减少的值都相同，所以产生的 PWM 信号为线性占空比渐变信号。

图 23.4-5 展示了一个 PWM 信号占空比线性渐变的例子。

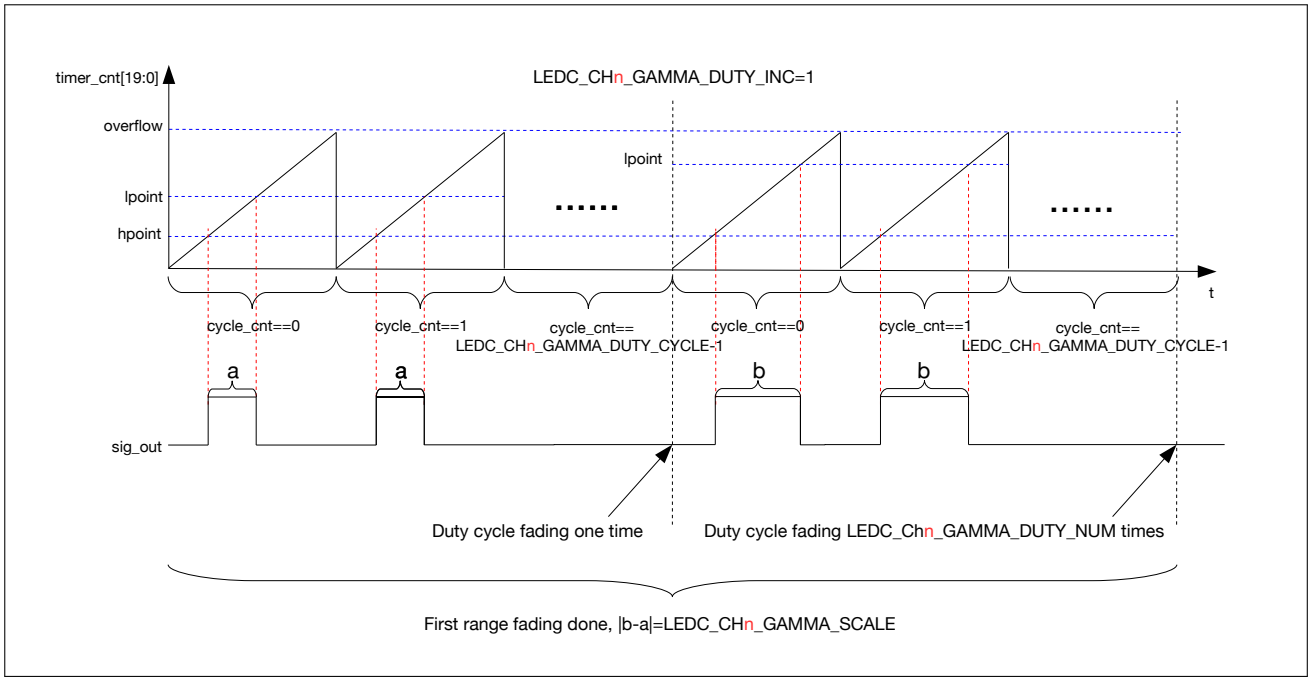


图 23.4-5. 输出信号占空比线性渐变图

23.4.5.2 伽马曲线渐变

通过配置多个占空比渐变区间的占空比渐变方向、渐变步长、渐变次数和渐变频率，PWM 生成器可以产生伽马曲线渐变的 PWM 信号。

配置步骤请见章节 23.7 配置流程。

经过上述所有的配置步骤，PWM 生成器会产生包含 LEDC_CHn_GAMMA_ENTRY_NUM 个占空比渐变区间的 PWM 信号。首先，PWM 信号会根据序号为 0 的占空比渐变区间的配置进行占空比渐变，渐变完成后，按照序号为 1 的占空比渐变区间的配置进行占空比渐变，直到序号为 (LEDC_CHn_GAMMA_ENTRY_NUM – 1) 的占空比渐变区间（即最后一个区间）完成渐变。在每个区间中，PWM 信号都是独立变化的。对于每个区间，PWM 信号会在计数器每溢出 LEDC_CHn_GAMMA_DUTY_CYCLE 次后，进行一次占空比渐变。PWM 信号每次渐变时，其 Lpointn 会增加或减少（由 LEDC_CHn_GAMMA_DUTY_INC 配置）LEDC_CHn_GAMMA_SCALE；相应地，占空比会增加或减少 ΔDC（由 LEDC_CHn_GAMMA_DUTY_INC 配置）。

$$\Delta DC = \frac{\text{LEDC_CH}n\text{_GAMMA_SCALE}}{\text{LEDC_TIMER}x\text{_DUTY_RES}} \tag{23.3}$$

在该区间内，占空比会渐变 LEDC_CHn_GAMMA_DUTY_NUM 次，当指定次数的占空比渐变完成后，视为当前区间的占空比渐变完成。

当 LEDC_CHn_GAMMA_ENTRY_NUM 个区间的占空比渐变都完成后，PWM 信号会停止占空比渐变，其占空比会保持最后一次渐变的占空比。由于每个占空比渐变区间的渐变过程都是独立不同的线性渐变，因此多个区间的线性占空比渐变就可以拟合成一个特定的伽马曲线占空比渐变。

图 23.4-6 展示了一个伽马曲线渐变 PWM 信号。

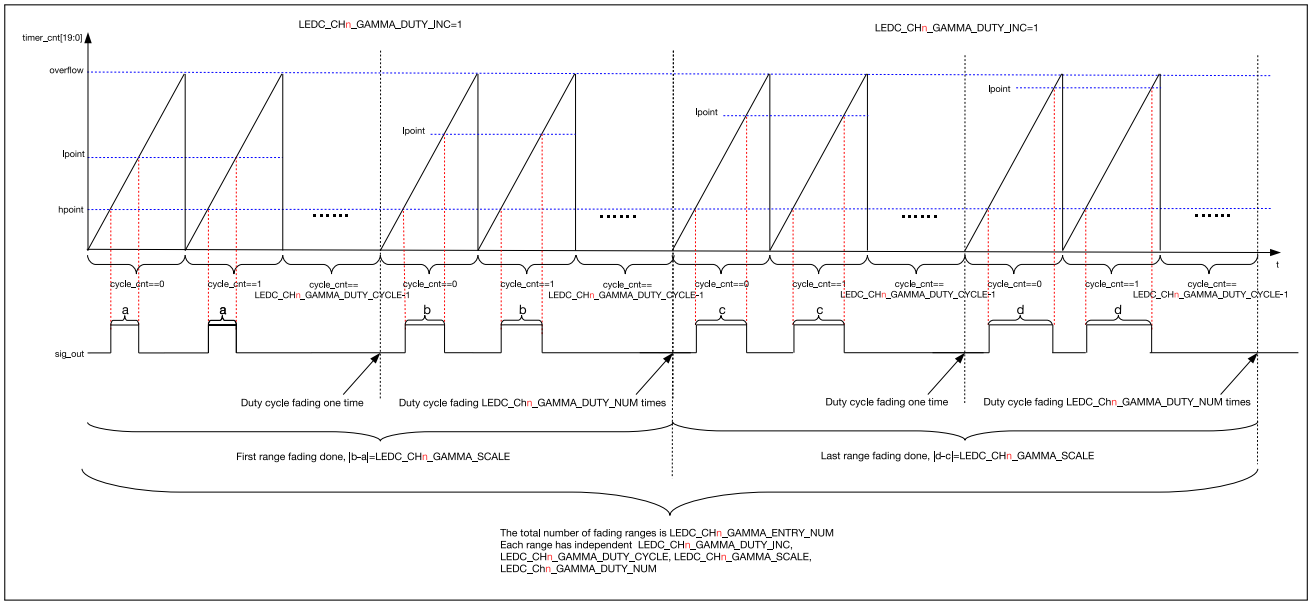


图 23.4-6. 输出信号占空比伽马曲线渐变图

23.4.5.3 占空比渐变暂停和恢复

向 LEDC_CHn_GAMMA_CONF_REG 寄存器的 LEDC_CHn_GAMMA_PAUSE 字段写 1 可以暂停已经开始的占空比渐变过程。一旦将 LEDC_CHn_GAMMA_PAUSE 字段置 1，PWM 信号将会保持最近一次渐变后的占空比。

向 LEDC_CHn_GAMMA_CONF_REG 寄存器的 LEDC_CHn_GAMMA_RESUME 字段写 1 可以恢复被暂停的占空比渐变过程。此时，PWM 信号将从暂停状态开始，按照原有的区间配置继续渐变。
LEDC_CHn_GAMMA_RESUME 字段置 1 后，硬件会自动清除 LEDC_CHn_GAMMA_PAUSE 字段。

23.5 事件任务矩阵功能

在 ESP32-S31 中，LEDC 支持 ETM 功能，即可以通过任意外设的 ETM 事件触发 LEDC 的 ETM 任务，或者通过 LEDC 的 ETM 事件触发任意外设的 ETM 任务。关于 ETM 更多详细信息，请参考章节 10 事件任务矩阵 (ETM) [to be added later]。这里仅介绍与 LEDC 相关的 ETM 任务和 ETM 事件。

ETM 外设相关的事件和任务的使能由寄存器 LEDC_EVT_TASK_EN0_REG、LEDC_EVT_TASK_EN1_REG 和 LEDC_EVT_TASK_EN2_REG 的相关字段所配置。事件和任务与寄存器 LEDC_EVT_TASK_EN0_REG、LEDC_EVT_TASK_EN1_REG、LEDC_EVT_TASK_EN2_REG 的每个字段的对应关系参照章节 23.9。

本节中，*n* 表示 PWM 通道号（范围是 0 到 7），*x* 表示定时器号（范围是 0 到 3）。

LEDC 可接收的 ETM 任务有：

- LEDC_TASK_DUTY_SCALE_UPDATE_CH*n*：如果 LEDC_TASK_DUTY_SCALE_UPDATE_CH*n*_EN 字段使能，当接收到 LEDC_TASK_DUTY_SCALE_UPDATE_CH*n* 任务后，相应的 LEDC 通道（PWM*n*）将会根据最新配置的 LEDC_CH*n*_GAMMA_SCALE 的值，重新开始生成占空比渐变 PWM 信号。
- LEDC_TASK_TIMER*x*_RES_UPDATE：如果 LEDC_TASK_TIMER*x*_RES_UPDATE_EN 字段使能，当接收到 LEDC_TASK_TIMER*x*_RES_UPDATE 任务后，相应的定时器（Timer*x*）将会根据最新配置的 LEDC_TIMER*x*_DUTY_RES 字段的值，在下一次计数器溢出时，更新计数器的最大计数值为该值。
- LEDC_TASK_TIMER*x*_CAP：如果 LEDC_TASK_TIMER*x*_CAP_EN 字段使能，当接收到 LEDC_TASK_TIMER*x*_CAP 任务时，相应的定时器（Timer*x*）将会抓取当前计数器的计数值。该计数值将会存储在寄存器 LEDC_TIMER*x*_CNT_CAP_REG 的 LEDC_TIMER*x*_CNT_CAP 字段。
- LEDC_TASK_SIG_OUT_DIS_CH*n*：如果 LEDC_TASK_SIG_OUT_DIS_CH*n*_EN 字段使能，当接收到 LEDC_TASK_SIG_OUT_DIS_CH*n* 任务时，相应的 LEDC 通道（PWM*n*）的 PWM 信号将停止输出，该通道的输出信号 (sig_out*n*) 将会变为 LEDC_IDLE_LV_CH*n* 字段所配置电平的固定电平信号，如图 23.3-2 所示。
- LEDC_TASK_OVF_CNT_RST_CH*n*：如果 LEDC_TASK_OVF_CNT_RST_CH*n*_EN 字段使能，当接收到 LEDC_TASK_OVF_CNT_RST_CH*n* 任务时，相应的 LEDC 通道（PWM*n*）定时器的计数器溢出次数的计数器值将会被复位为 0。
- LEDC_TASK_TIMER*x*_RST：如果 LEDC_TASK_TIMER*x*_RST_EN 字段使能，当接收到 LEDC_TASK_TIMER*x*_RST 任务时，相应的定时器（Timer*x*）的计数器值将会被复位为 0。
- LEDC_TASK_TIMER*x*_PAUSE：如果 LEDC_TASK_TIMER*x*_PAUSE_RESUME_EN 字段使能，当接收到 LEDC_TASK_TIMER*x*_PAUSE 任务时，相应的定时器（Timer*x*）将会被暂停。
- LEDC_TASK_TIMER*x*_RESUME：如果 LEDC_TASK_TIMER*x*_PAUSE_RESUME_EN 字段使能，当接收到 LEDC_TASK_TIMER*x*_RESUME 任务时，若相应的定时器（Timer*x*）此时为暂停状态，则将被恢复。
- LEDC_TASK_GAMMA_RESTART_CH*n*：如果 LEDC_TASK_GAMMA_RESTART_CH*n*_EN 字段使能，当接收到 LEDC_TASK_GAMMA_RESTART_CH*n* 任务时，相应的 LEDC 通道（PWM*n*）将会重新开始产生占空比渐变 PWM 信号。
- LEDC_TASK_GAMMA_PAUSE_CH*n*：如果 LEDC_TASK_GAMMA_PAUSE_CH*n*_EN 字段使能，当接收到 LEDC_TASK_GAMMA_PAUSE_CH*n* 任务时，相应的 LEDC 通道（PWM*n*）将会在计数器下一次溢出时停止占空比的渐变。即，当该任务被接收后，相应的 LEDC 通道（PWM*n*）的 PWM 信号的占空比将不会再进行渐变。

- LEDC_TASK_GAMMA_RESUME_CH*n*：如果 LEDC_TASK_GAMMA_RESUME_CH*n*_EN 字段使能，当接收到 LEDC_TASK_GAMMA_RESUME_CH*n* 任务时，相应的 LEDC 通道（PWM*n*）将会在计数器下一次溢出时恢复占空比的渐变。即，当该任务被接收后，相应的 LEDC 通道（PWM*n*）的 PWM 信号将会按照暂停时的占空比渐变配置继续进行占空比的渐变。

LEDC 可产生的 ETM 事件有：

- LEDC_EVT_DUTY_CHNG_END_CH*n*：如果 LEDC_EVT_DUTY_CHNG_END_CH*n*_EN 字段使能，当相应的 LEDC 通道（PWM*n*）占空比渐变完成后，将会产生 LEDC_EVT_DUTY_CHNG_END_CH*n* 事件。
- LEDC_EVT_OVF_CNT_PLS_CH*n*：如果 LEDC_EVT_OVF_CNT_PLS_CH*n*_EN 字段使能，当相应的 LEDC 通道（PWM*n*）的定时器的计数器溢出 LEDC_OVF_NUM_CH*n* + 1 次后，将会产生 LEDC_EVT_OVF_CNT_PLS_CH*n* 事件。
- LEDC_EVT_TIME_OVF_TIMER*x*：如果 LEDC_EVT_TIME_OVF_TIMER*x*_EN 字段使能，当相应的定时器（Timer*x*）的计数器溢出时，将会产生 LEDC_EVT_TIME_OVF_TIMER*x* 事件。
- LEDC_EVT_TIMER*x*_CMP：如果 LEDC_EVT_TIMER*x*_CMP_EN 字段使能，当相应的定时器（Timer*x*）的计数器值等于寄存器 LEDC_TIMER*x*_CMP_REG 的 LEDC_TIMER*x*_CMP 字段的值时，将会产生 LEDC_EVT_TIMER*x*_CMP 事件。

在具体应用中，LEDC 的 ETM 事件可以用来触发 LEDC 的 ETM 任务，例如，LEDC_EVT_DUTY_CHNG_END_CH*n* 事件可以触发 LEDC_TASK_GAMMA_RESTART_CH*n* 任务，从而在一次渐变完成后直接开始下一次渐变。

23.6 中断

ESP32-S31 的 LEDC 模块会产生 LEDC_INT 中断信号，该信号会被发送到 中断矩阵。

下列内部中断源可以产生 LEDC_INT 中断信号：

- LEDC_OVF_CNT_CH*n*_INT：定时器计数器溢出 LEDC_OVF_NUM_CH*n* + 1 次且寄存器 LEDC_OVF_CNT_EN_CH*n* 置 1 时触发中断。
- LEDC_DUTY_CHNG_END_CH*n*_INT：PWM*n* 渐变完成后触发中断。
- LEDC_TIMER*x*_OVF_INT：定时器达到最大计数值时触发中断。

只有当中断使能寄存器 *XXX*_ENA 置位为 1 时，才能触发上述中断源。置位 *XXX*_CLR 可清除中断（*XXX* 为中断源）。

23.7 配置流程

LED PWM 控制器生成 PWM 信号的配置流程为：

1. 配置时钟
   (a) 将 HP_SYS_CLKRST_LEDC0_APB_CLK_EN 字段置 1 使能 LEDC0 APB 时钟；或者将 HP_SYS_CLKRST_LEDC1_APB_CLK_EN 字段置 1 使能 LEDC1 APB 时钟。
   (b) 配置 HP_SYS_CLKRST_LEDC0_CLK_SRC_SEL 字段选择 LEDC0_CLK 的时钟源或配置 HP_SYS_CLKRST_LEDC1_CLK_SRC_SEL 字段选择 LEDC1_CLK 的时钟源。

- XTAL_CLK：将 HP_SYS_CLKRST_LEDC0_CLK_SRC_SEL/HP_SYS_CLKRST_LEDC1_CLK_SRC_SEL 置 0
- RC_FAST_CLK：将 HP_SYS_CLKRST_LEDC0_CLK_SRC_SEL/HP_SYS_CLKRST_LEDC1_CLK_SRC_SEL 置 1
- PLL_80M_CLK：将 HP_SYS_CLKRST_LEDC0_CLK_SRC_SEL/HP_SYS_CLKRST_LEDC1_CLK_SRC_SEL 置 2

(c) 将 HP_SYS_CLKRST_LEDC0_CLK_EN 字段置 1 使能 LEDC0_CLK；或者将 HP_SYS_CLKRST_LEDC1_CLK_EN 字段置 1 使能 LEDC1_CLK。

2. 参照 23.4.3 对定时器 Timer*x* 进行配置。配置完成后，时钟源、分频器与计数器就配置完成了。
3. 配置 HP_SYS_LEDC0_MEM_LP_CTRL_REG 或 HP_SYS_LEDC1_MEM_LP_CTRL_REG，为 LEDC 内存上电。该内存默认处于下电状态，未上电时无法进行后续的 PWM*n* 生成器配置。
4. 参照 23.4.4 对 PWM*n* 生成器进行配置。配置完成后，PWM*n* 从四个定时器中选择其中一个，并设置 PWM*n* 输出的相位和占空比。
5. 根据不同的占空比渐变类型选择相应的配置过程：

   (a) 线性占空比渐变：

   i. 配置寄存器字段 LEDC_DUTY_CH*n*。该字段用来设置 Lpoint*n* 的初始值。

   ii. 置位寄存器字段 LEDC_DUTY_START_CH*n*。当该字段置位/清除，占空比渐变会使能/关闭。

   iii. 将线性占空比渐变配置写入寄存器（LEDC_CH*n*_MEM 起始地址 + 占空比区间值 * 4）。LEDC_CH*n*_MEM 起始地址包含在表 23.8-1 中。

   请注意以下的字段名称只是为了方便叙述，寄存器中并不存在这些字段。

   A. 第 0 位（为方便叙述，以下称为 LEDC_CH*n*_GAMMA_DUTY_INC）用于配置方向。当该字段置位或清零时，当前所配置的占空比渐变区间的 Lpoint*n* 会增加/减少。

   B. 第 1~10 位（为方便叙述，以下称为 LEDC_CH*n*_GAMMA_DUTY_CYCLE）用来设置当前所配置的占空比渐变区间的 Lpoint*n* 每增加/减少一次，计数器溢出的次数。即当计数器溢出次数为配置次数，Lpoint*n* 将会增加/减少一次。

   C. 第 11~20 位（为方便叙述，以下称为 LEDC_CH*n*_GAMMA_SCALE）用来设置当前所配置的占空比渐变区间的 Lpoint*n* 每次增加/减少的量。

   D. 第 21~30 位（为方便叙述，以下称为 LEDC_CH*n*_GAMMA_DUTY_NUM）用来设置当前所配置的占空比渐变区间的渐变次数。

   E. 占空比范围值（0~15）用于指定上述配置的属性属于哪个占空比渐变区间。由于线性占空比渐变只需要配置第一个占空比渐变区间即可，所以占空比范围值应配置为 0。

   iv. 将寄存器 LEDC_CH*n*_GAMMA_CONF_REG 的 LEDC_CH*n*_GAMMA_ENTRY_NUM 字段配置为 1。该字段用来指定一次渐变过程中有效的占空比渐变区间的个数（当指定数目的区间都渐变完成后，PWM 信号会停止占空比渐变，PWM 生成器会触发 LEDC_DUTY_CHNG_END_CH*n*_INT 中断）。由于线性占空比渐变只包含一个渐变区间（第一个占空比渐变区间），所以 LEDC_CH*n*_GAMMA_ENTRY_NUM 字段配置为 1。

   v. 置位寄存器字段 LEDC_PARA_UP_CH*n* 应用上述步骤的配置。当该字段被置位后，上述的占空比渐变配置会在下一次计数器溢出时生效，PWM 生成器会输出指定配置的线性占空比渐变 PWM 信号（LEDC_PARA_UP_CH*n* 字段会被硬件自动清除）。

(b) 伽马曲线渐变：

i. 与线性占空比渐变的步骤 1 相同。

ii. 与线性占空比渐变的步骤 2 相同。

iii. 配置多个占空比渐变区间：将伽马曲线渐变配置写入寄存器（LEDC_CH*n*_MEM 起始地址 + 占空比区间值 * 4），LEDC_CH*n*_MEM 起始地址包含在表 23.8-1 中。

A. 第 0 位（为方便叙述，以下称为 LEDC_CH*n*_GAMMA_DUTY_INC）用于配置方向。当该字段置位或清零时，当前所配置的占空比渐变区间的 Lpoint*n* 会增加/减少。

B. 第 1~10 位（为方便叙述，以下称为 LEDC_CH*n*_GAMMA_DUTY_CYCLE）用来设置当前所配置的占空比渐变区间的 Lpoint*n* 每增加/减少一次，计数器溢出的次数。即当计数器溢出次数为配置次数，Lpoint*n* 将会增加/减少一次。

C. 第 11~20 位（为方便叙述，以下称为 LEDC_CH*n*_GAMMA_SCALE）用来设置当前所配置的占空比渐变区间的 Lpoint*n* 每次增加/减少的量。

D. 第 21~30 位（为方便叙述，以下称为 LEDC_CH*n*_GAMMA_DUTY_NUM）用来设置当前所配置的占空比渐变区间的渐变次数。

E. 占空比范围值 (0~15) 用于指定上述配置的属性属于哪个占空比渐变区间。对于伽马曲线渐变，所配置的占空比渐变区间的序号必须从 0 开始，每配置完一个占空比渐变区间，下一个要配置的占空比渐变区间的序号需要加 1。

F. 上述步骤配置完毕后，一个占空比渐变区间的配置视为完成，然后可以重复上述的步骤配置其他的占空比渐变区间的参数。每个占空比渐变区间的配置都是独立的，拥有有效配置的区间个数 (0~15) 可以是任意的。

iv. 当所需的占空比渐变区间都配置完毕后，将 LEDC_CH*n*_GAMMA_CONF_REG 寄存器的 LEDC_CH*n*_GAMMA_ENTRY_NUM 字段配置为步骤 3 中配置过的占空比渐变区间的总个数。

v. 置位寄存器的 LEDC_PARA_UP_CH*n* 字段，应用以上步骤的配置。当该字段被置位后，上述步骤的占空比渐变配置会在下一次计数器溢出时生效，PWM 生成器会按照配置输出伽马曲线渐变的 PWM 信号（LEDC_PARA_UP_CH*n* 字段会自动被硬件清除）。

按照上述步骤配置完成后，LED PWM 控制器会生成所需的 PWM 信号。

占空比渐变随时都可以暂停或恢复，详情请参考 23.4.5.3。如果需要在 PWM 信号停止渐变之前（即 LEDC_DUTY_CHNG_END_CH*n*_INT 尚未触发）更改占空比渐变配置，则需要在更改配置之前暂停占空比渐变。但是，如果需要在 PWM 信号停止渐变后（即 LEDC_DUTY_CHNG_END_CH*n*_INT 已被触发）更改占空比渐变配置，则可以直接进行更改。

软件随时都可以启用或禁用 PWM 信号输出，也可对输出信号的电平进行更改。详情请参考 23.4.4。

23.8 存储空间

每个 LEDC 通道都有一块存储空间用于存放占空比渐变的配置数据。每个存储空间最多可储存 16 个区间的配置数据。本小节的所有地址均为相对于 LED PWM 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

表 23.8-1. LEDC 存储空间

名称	描述	大小	起始地址	终止地址	访问权限
LEDC_CH*n*_MEM	PWM*n* 内存	64 B	0x400 + PWM*n* * 64	0x400 + (PWM*n* + 1) * 64	WR

23.9 寄存器列表

本小节的所有地址均为相对于 LED PWM 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
配置寄存器			
LEDC_CH0_CONF0_REG	通道 0 的配置寄存器 0	0x0000	varies
LEDC_CH0_HPOINT_REG	通道 0 的 Hpoint 寄存器	0x0004	R/W
LEDC_CH0_DUTY_REG	通道 0 的初始占空比	0x0008	R/W
LEDC_CH0_CONF1_REG	通道 0 的配置寄存器 1	0x000C	R/W/SC
LEDC_CH1_CONF0_REG	通道 1 的配置寄存器 0	0x0014	varies
LEDC_CH1_HPOINT_REG	通道 1 的 Hpoint 寄存器	0x0018	R/W
LEDC_CH1_DUTY_REG	通道 1 的初始占空比	0x001C	R/W
LEDC_CH1_CONF1_REG	通道 1 的配置寄存器 1	0x0020	R/W/SC
LEDC_CH2_CONF0_REG	通道 2 的配置寄存器 0	0x0028	varies
LEDC_CH2_HPOINT_REG	通道 2 的 Hpoint 寄存器	0x002C	R/W
LEDC_CH2_DUTY_REG	通道 2 的初始占空比	0x0030	R/W
LEDC_CH2_CONF1_REG	通道 2 的配置寄存器 1	0x0034	R/W/SC
LEDC_CH3_CONF0_REG	通道 3 的配置寄存器 0	0x003C	varies
LEDC_CH3_HPOINT_REG	通道 3 的 Hpoint 寄存器	0x0040	R/W
LEDC_CH3_DUTY_REG	通道 3 的初始占空比	0x0044	R/W
LEDC_CH3_CONF1_REG	通道 3 的配置寄存器 1	0x0048	R/W/SC
LEDC_CH4_CONF0_REG	通道 4 的配置寄存器 0	0x0050	varies
LEDC_CH4_HPOINT_REG	通道 4 的 Hpoint 寄存器	0x0054	R/W
LEDC_CH4_DUTY_REG	通道 4 的初始占空比	0x0058	R/W
LEDC_CH4_CONF1_REG	通道 4 的配置寄存器 1	0x005C	R/W/SC
LEDC_CH5_CONF0_REG	通道 5 的配置寄存器 0	0x0064	varies
LEDC_CH5_HPOINT_REG	通道 5 的 Hpoint 寄存器	0x0068	R/W
LEDC_CH5_DUTY_REG	通道 5 的初始占空比	0x006C	R/W
LEDC_CH5_CONF1_REG	通道 5 的配置寄存器 1	0x0070	R/W/SC
LEDC_CH6_CONF0_REG	通道 6 的配置寄存器 0	0x0078	varies
LEDC_CH6_HPOINT_REG	通道 6 的 Hpoint 寄存器	0x007C	R/W
LEDC_CH6_DUTY_REG	通道 6 的初始占空比	0x0080	R/W
LEDC_CH6_CONF1_REG	通道 6 的配置寄存器 1	0x0084	R/W/SC
LEDC_CH7_CONF0_REG	通道 7 的配置寄存器 0	0x008C	varies
LEDC_CH7_HPOINT_REG	通道 7 的 Hpoint 寄存器	0x0090	R/W
LEDC_CH7_DUTY_REG	通道 7 的初始占空比	0x0094	R/W
LEDC_CH7_CONF1_REG	通道 7 的配置寄存器 1	0x0098	R/W/SC
LEDC_TIMER0_CONF_REG	配置定时器 0	0x00A0	varies
LEDC_TIMER1_CONF_REG	配置定时器 1	0x00A8	varies
LEDC_TIMER2_CONF_REG	配置定时器 2	0x00B0	varies
LEDC_TIMER3_CONF_REG	配置定时器 3	0x00B8	varies

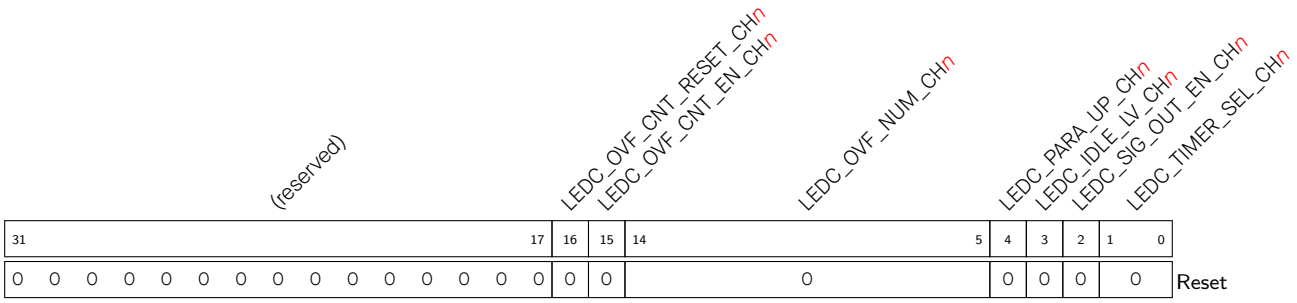
名称	描述	地址	访问
LEDC_CH0_GAMMA_CONF_REG	通道 0 伽马配置寄存器	0x0100	varies
LEDC_CH1_GAMMA_CONF_REG	通道 1 伽马配置寄存器	0x0104	varies
LEDC_CH2_GAMMA_CONF_REG	通道 2 伽马配置寄存器	0x0108	varies
LEDC_CH3_GAMMA_CONF_REG	通道 3 伽马配置寄存器	0x010C	varies
LEDC_CH4_GAMMA_CONF_REG	通道 4 伽马配置寄存器	0x0110	varies
LEDC_CH5_GAMMA_CONF_REG	通道 5 伽马配置寄存器	0x0114	varies
LEDC_CH6_GAMMA_CONF_REG	通道 6 伽马配置寄存器	0x0118	varies
LEDC_CH7_GAMMA_CONF_REG	通道 7 伽马配置寄存器	0x011C	varies
LEDC_EVT_TASK_EN0_REG	LEDC 事件任务使能寄存器 0	0x0120	R/W
LEDC_EVT_TASK_EN1_REG	LEDC 事件任务使能寄存器 1	0x0124	R/W
LEDC_EVT_TASK_EN2_REG	LEDC 事件任务使能寄存器 2	0x0128	R/W
LEDC_TIMER0_CMP_REG	定时器 0 的比较值	0x0140	R/W
LEDC_TIMER1_CMP_REG	定时器 1 的比较值	0x0144	R/W
LEDC_TIMER2_CMP_REG	定时器 2 的比较值	0x0148	R/W
LEDC_TIMER3_CMP_REG	定时器 3 的比较值	0x014C	R/W
LEDC_CONF_REG	LEDC 全局配置寄存器	0x0170	R/W
LEDC_CH_POWER_UP_CONF_REG	通道上电配置寄存器	0x0174	R/W
LEDC_TIMER_POWER_UP_CONF_REG	定时器上电配置寄存器	0x0178	R/W
状态寄存器			
LEDC_CH0_DUTY_R_REG	通道 0 的当前占空比	0x0010	RO
LEDC_CH1_DUTY_R_REG	通道 1 的当前占空比	0x0024	RO
LEDC_CH2_DUTY_R_REG	通道 2 的当前占空比	0x0038	RO
LEDC_CH3_DUTY_R_REG	通道 3 的当前占空比	0x004C	RO
LEDC_CH4_DUTY_R_REG	通道 4 的当前占空比	0x0060	RO
LEDC_CH5_DUTY_R_REG	通道 5 的当前占空比	0x0074	RO
LEDC_CH6_DUTY_R_REG	通道 6 的当前占空比	0x0088	RO
LEDC_CH7_DUTY_R_REG	通道 7 的当前占空比	0x009C	RO
LEDC_TIMER0_VALUE_REG	配置定时器 0 的当前计数值	0x00A4	RO
LEDC_TIMER1_VALUE_REG	配置定时器 1 的当前计数值	0x00AC	RO
LEDC_TIMER2_VALUE_REG	配置定时器 2 的当前计数值	0x00B4	RO
LEDC_TIMER3_VALUE_REG	配置定时器 3 的当前计数值	0x00BC	RO
LEDC_TIMER0_CNT_CAP_REG	抓取定时器 0 的计数值	0x0150	RO
LEDC_TIMER1_CNT_CAP_REG	抓取定时器 1 的计数值	0x0154	RO
LEDC_TIMER2_CNT_CAP_REG	抓取定时器 2 的计数值	0x0158	RO
LEDC_TIMER3_CNT_CAP_REG	抓取定时器 3 的计数值	0x015C	RO
中断寄存器			
LEDC_INT_RAW_REG	原始中断寄存器	0x00C0	R/WTC/SS
LEDC_INT_ST_REG	屏蔽中断寄存器	0x00C4	RO
LEDC_INT_ENA_REG	中断使能寄存器	0x00C8	R/W
LEDC_INT_CLR_REG	中断清除寄存器	0x00CC	WT
版本寄存器			
LEDC_DATE_REG	版本控制寄存器	0x017C	R/W

23.10 寄存器

本小节的所有地址均为相对于 LED PWM 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 23.1. LEDC_CH*n*_CONF0_REG (*n*: 0-7) (0x0000+0x14**n*)



LEDC_TIMER_SEL_CH*n* 配置通道 *n* 选取的定时器。

0：定时器 0

1：定时器 1

2：定时器 2

3：定时器 3

(R/W)

LEDC_SIG_OUT_EN_CH*n* 配置是否开启通道 *n* 的信号输出。

0：关闭

1：开启

(R/W)

LEDC_IDLE_LV_CH*n* 配置通道 *n* 关闭（即 LEDC_SIG_OUT_EN_CH*n* 为 0）时的输出电平值。

0：低电平

1：高电平

(R/W)

LEDC_PARA_UP_CH*n* 配置是否更新以下字段：LEDC_HPOINT_CH*n*、LEDC_DUTY_START_CH*n*、LEDC_SIG_OUT_EN_CH*n*、LEDC_TIMER_SEL_CH*n*、LEDC_OVF_CNT_EN_CH*n* 和占空比区间配置

0：无效值，没有作用

1：更新

该字段由硬件自清

(WT)

LEDC_OVF_NUM_CH*n* 配置定时器的最大溢出次数减 1。

通道 *n* 的定时器溢出次数达到 (LEDC_OVF_NUM_CH*n* + 1) 次时，触发 LEDC_OVF_CNT_CH*n*_INT 中断。(R/W)

LEDC_OVF_CNT_EN_CH*n* 配置是否开启通道 *n* 的溢出次数计数器。

0：关闭

1：开启

(R/W)

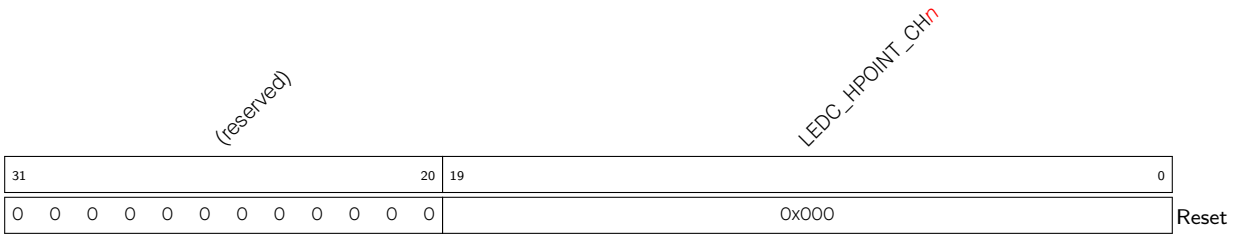
LEDC_OVF_CNT_RESET_CH*n* 配置是否复位通道 *n* 的溢出次数计数器。

0：无效值，没有作用

1：复位

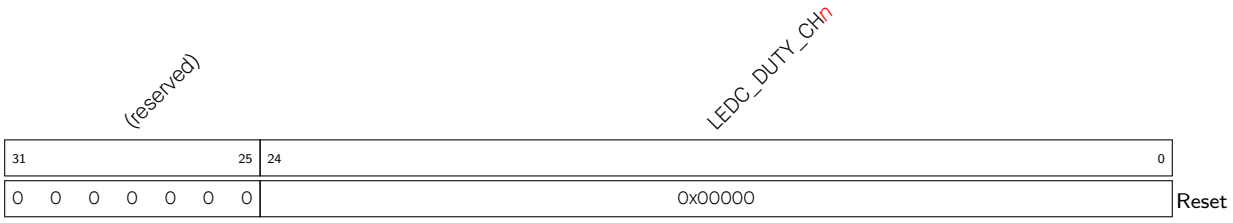
(WT)

Register 23.2. LEDC_CH*n*_HPOINT_REG (*n*: 0-7) (0x0004+0x14*n*)



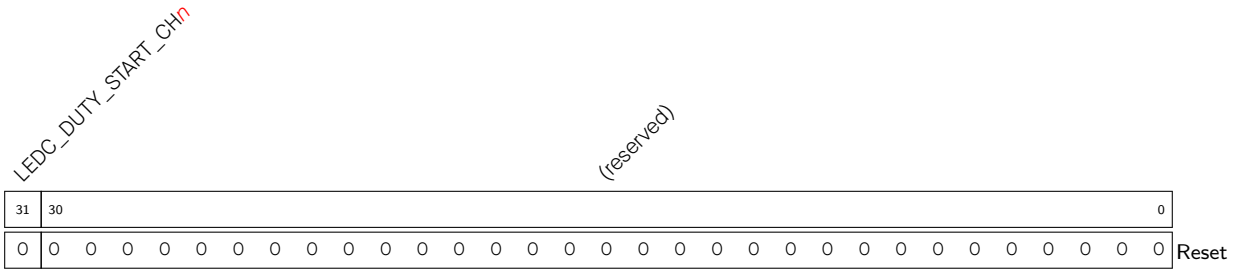
LEDC_HPOINT_CH*n* 配置通道 *n* 输出的 Hpoint 值。当定时器达到该寄存器的规定值时，输出值变为高电平。(R/W)

Register 23.3. LEDC_CH*n*_DUTY_REG (*n*: 0-7) (0x0008+0x14*n*)



LEDC_DUTY_CH*n* 配置通道 *n* 输出信号的占空比。(R/W)

Register 23.4. LEDC_CH*n*_CONF1_REG (*n*: 0-7) (0x000C+0x14*n*)



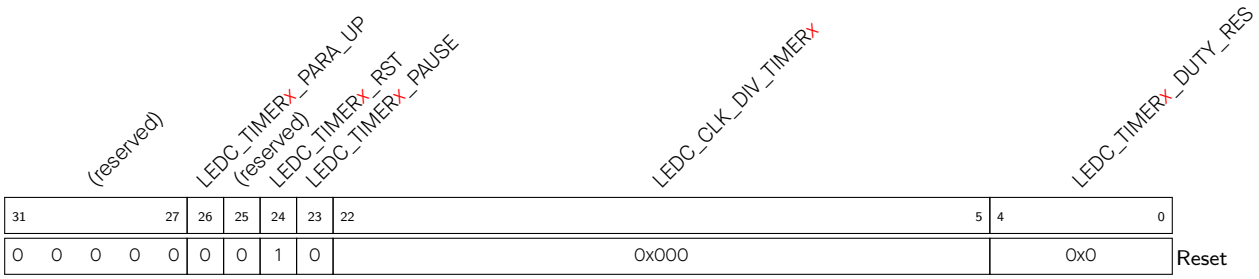
LEDC_DUTY_START_CH*n* 配置占空比渐变配置是否生效。

0：不生效

1：生效

(R/W/SC)

Register 23.5. LEDC_TIMERx_CONF_REG (x: 0-3) (0x00A0+0x8*x)



LEDC_TIMERx_DUTY_RES 配置定时器 *x* 计数器的位宽，有效值为 1 到 20。(R/W)

LEDC_CLK_DIV_TIMERx 配置定时器 *x* 中分频器的分频系数。

低 8 位为小数部分，高 10 位为整数部分。(R/W)

LEDC_TIMERx_PAUSE 配置是否暂停定时器 *x* 的计数器。

0：不暂停

1：暂停

(R/W)

LEDC_TIMERx_RST 配置是否复位定时器 *x* 的计数器。复位后，计数器的值为 0。

0：不复位

1：复位

(R/W)

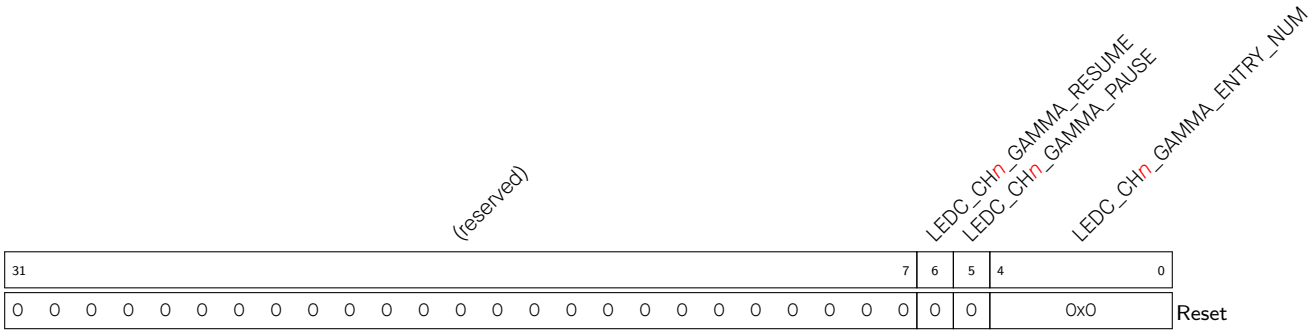
LEDC_TIMERx_PARA_UP 配置是否更新 LEDC_CLK_DIV_TIMERx 和 LEDC_TIMERx_DUTY_RES 字段。

0：无效值，没有作用

1：更新

(WT)

Register 23.6. LEDC_CH*n*_GAMMA_CONF_REG (*n*: 0-7) (0x0100+0x4**n*)



LEDC_CH*n*_GAMMA_ENTRY_NUM 配置通道 *n* 的占空比渐变区间数量。(R/W)

LEDC_CH*n*_GAMMA_PAUSE 配置是否暂停占空比渐变。

0：无效值，没有作用

1：暂停

(WT)

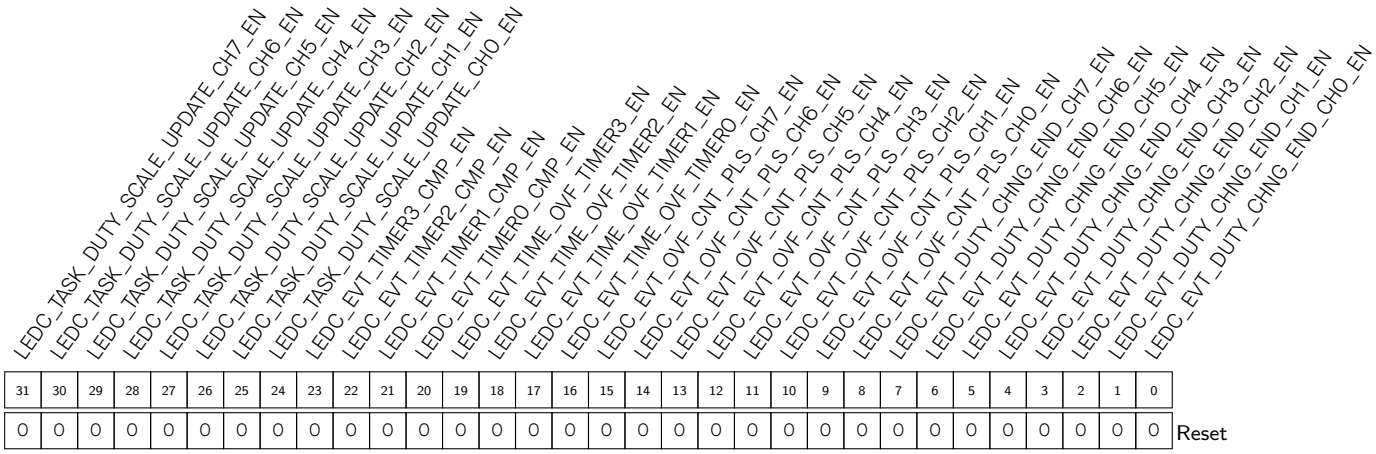
LEDC_CH*n*_GAMMA_RESUME 配置是否恢复已暂停的占空比渐变。

0：无效值，没有作用

1：恢复

(WT)

Register 23.7. LEDC_EVT_TASK_EN0_REG (0x0120)



LEDC_EVT_DUTY_CHNG_END_CH*n*_EN (*n*: 0-7) 配置是否使能 LEDC_EVT_DUTY_CHNG_END_CH*n* 事件。

0：关闭

1：使能

(R/W)

LEDC_EVT_OVF_CNT_PLS_CH*n*_EN (*n*: 0-7) 配置是否使能 LEDC_EVT_OVF_CNT_PLS_CH*n* 事件。

0：关闭

1：使能

(R/W)

LEDC_EVT_TIME_OVF_TIMER*x*_EN (*x*: 0-3) 配置是否使能 LEDC_EVT_TIME_OVF_TIMER*x* 事件。

0：关闭

1：使能

(R/W)

LEDC_EVT_TIMER*x*_CMP_EN (*x*: 0-3) 配置是否使能 LEDC_EVT_TIMER*x*_CMP 事件。

0：关闭

1：使能

(R/W)

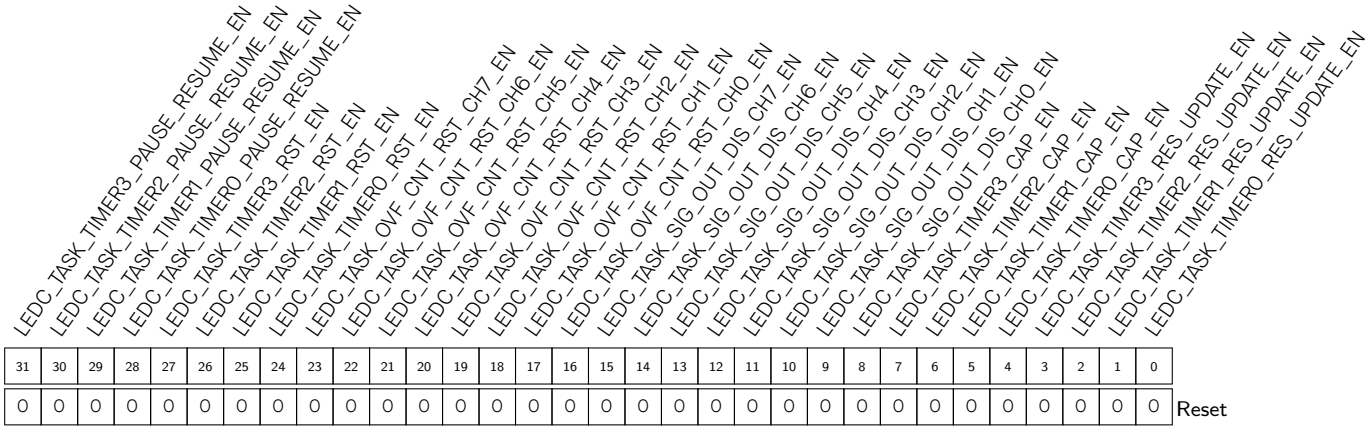
LEDC_TASK_DUTY_SCALE_UPDATE_CH*n*_EN (*n*: 0-7) 配置是否使能 LEDC_TASK_DUTY_SCALE_UPDATE_CH*n* 事件。

0：关闭

1：使能

(R/W)

Register 23.8. LEDC_EVT_TASK_EN1_REG (0x0124)



LEDC_TASK_TIMER*x*_RES_UPDATE_EN (*x*: 0-3) 配置是否使能 LEDC_TASK_TIMER*x*_RES_UPDATE 任务。

0：关闭

1：使能

(R/W)

LEDC_TASK_TIMER*x*_CAP_EN (*x*: 0-3) 配置是否使能 LEDC_TASK_TIMER*x*_CAP 任务。

0：关闭

1：使能

(R/W)

LEDC_TASK_SIG_OUT_DIS_CH*n*_EN (*n*: 0-7) 配置是否使能 LEDC_TASK_SIG_OUT_DIS_CH*n* 任务。

0：关闭

1：使能

(R/W)

LEDC_TASK_OVF_CNT_RST_CH*n*_EN (*n*: 0-7) 配置是否使能 LEDC_TASK_OVF_CNT_RST_CH*n* 任务。

0：关闭

1：使能

(R/W)

LEDC_TASK_TIMER*x*_RST_EN (*x*: 0-3) 配置是否使能 LEDC_TASK_TIMER*x*_RST 任务。

0：关闭

1：使能

(R/W)

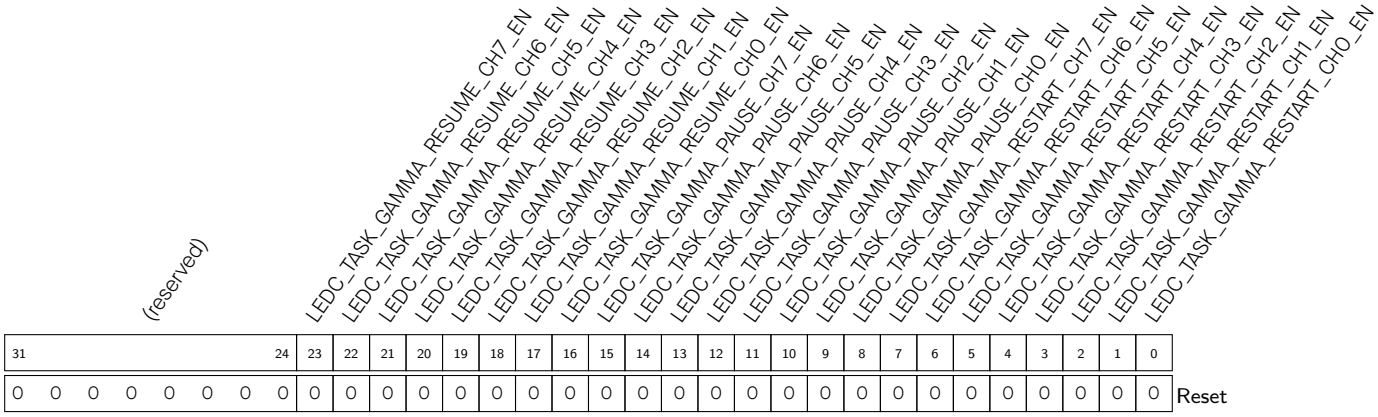
LEDC_TASK_TIMER*x*_PAUSE_RESUME_EN (*x*: 0-3) 配置是否使能 LEDC_TASK_TIMER*x*_PAUSE 和 LEDC_TASK_TIMER*x*_RESUME 任务。

0：关闭

1：使能

(R/W)

Register 23.9. LEDC_EVT_TASK_EN2_REG (0x0128)



LEDC_TASK_GAMMA_RESTART_CH*n*_EN (*n*: 0-7) 配置是否使能 LEDC_TASK_GAMMA_RESTART_CH*n* 任务。

0：关闭

1：使能

(R/W)

LEDC_TASK_GAMMA_PAUSE_CH*n*_EN (*n*: 0-7) 配置是否使能 LEDC_TASK_GAMMA_PAUSE_CH*n* 任务。

0：关闭

1：使能

(R/W)

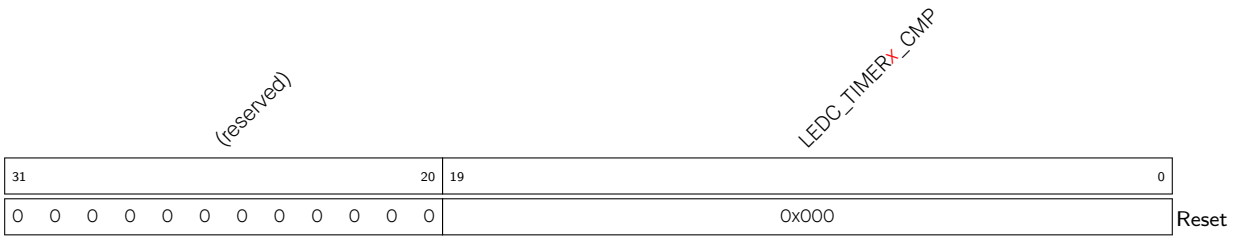
LEDC_TASK_GAMMA_RESUME_CH*n*_EN (*n*: 0-7) 配置是否使能 LEDC_TASK_GAMMA_RESUME_CH*n* 任务。

0：关闭

1：使能

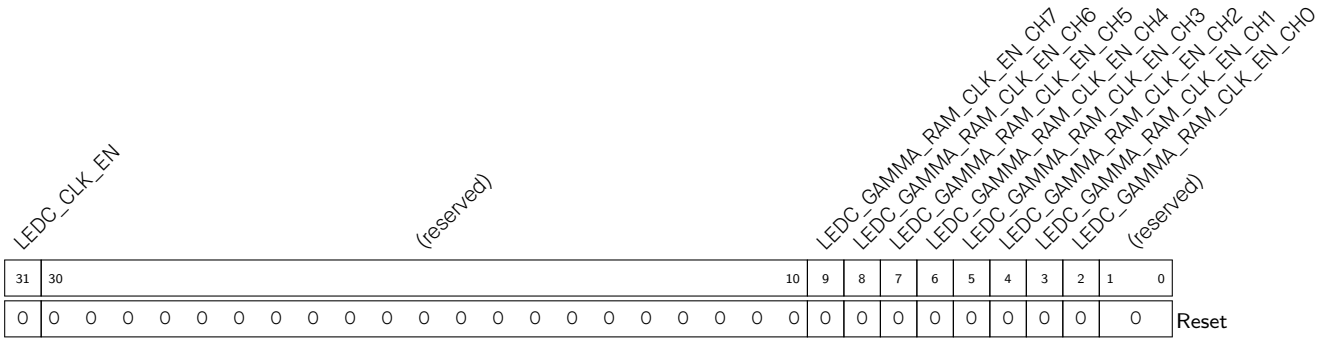
(R/W)

Register 23.10. LEDC_TIMER*x*_CMP_REG (*x*: 0-3) (0x0140+0x4**x*)

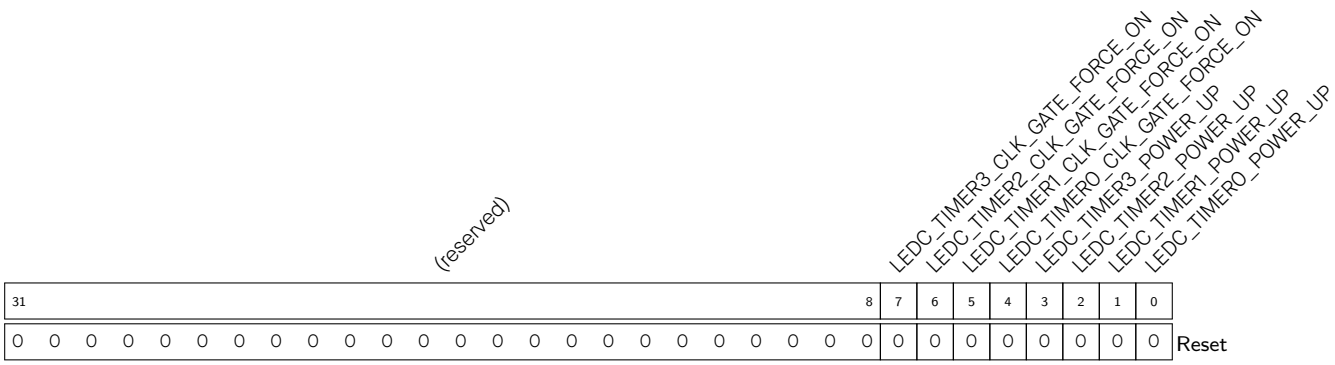


LEDC_TIMER*x*_CMP 配置 LEDC 定时器 *x* 的比较值。(R/W)

Register 23.11. LEDC_CONF_REG (0x0170)


LEDC_CLK_EN
(reserved)
LEDC_GAMMA_RAM_CLK_EN_CH7
LEDC_GAMMA_RAM_CLK_EN_CH6
LEDC_GAMMA_RAM_CLK_EN_CH5
LEDC_GAMMA_RAM_CLK_EN_CH4
LEDC_GAMMA_RAM_CLK_EN_CH3
LEDC_GAMMA_RAM_CLK_EN_CH2
LEDC_GAMMA_RAM_CLK_EN_CH1
LEDC_GAMMA_RAM_CLK_EN_CH0
(reserved)
31 30 10 9 8 7 6 5 4 3 2 1 0
0 0 0

Register 23.13. LEDC_TIMER_POWER_UP_CONF_REG (0x0178)



LEDC_TIMER*x*_POWER_UP (*x*: 0-3) 配置定时器 *x* 是否上电。

0：掉电

1：上电

(R/W)

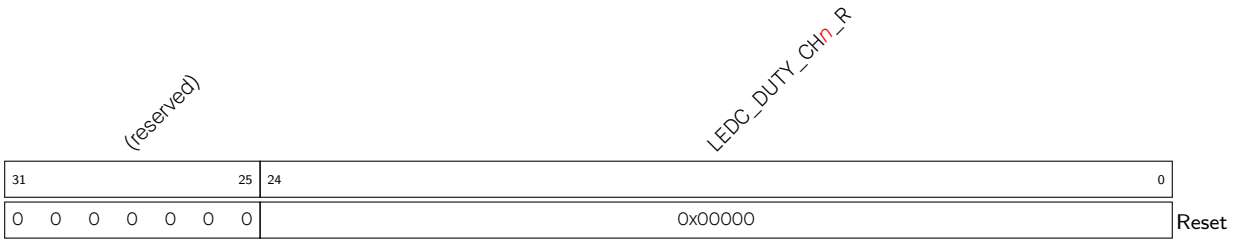
LEDC_TIMER*x*_CLK_GATE_FORCE_ON (*x*: 0-3) 配置是否开启定时器 *x* 的时钟门控。

0：仅在定时器 *x* 上电时开启时钟门控

1：强制为定时器 *x* 开启时钟门控

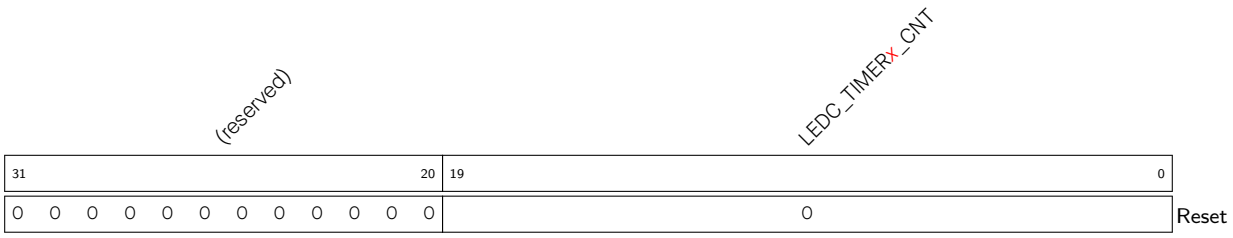
(R/W)

Register 23.14. LEDC_CH*n*_DUTY_R_REG (*n*: 0-7) (0x0010+0x14**n*)



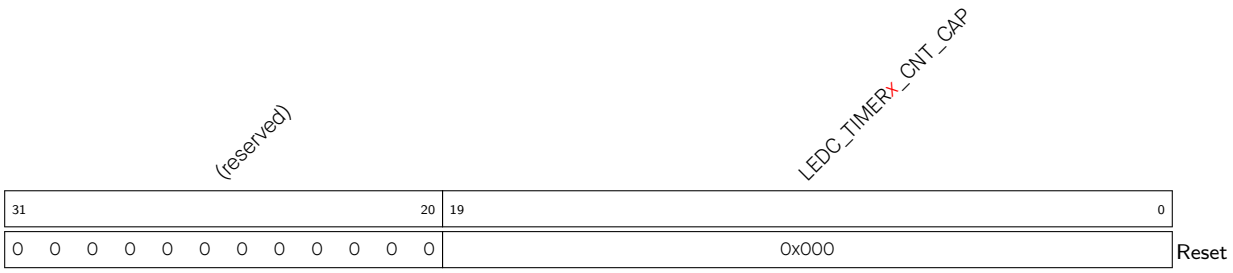
LEDC_DUTY_CH*n*_R 通道 *n* 输出信号的当前占空比。(RO)

Register 23.15. LEDC_TIMER*x*_VALUE_REG (*x*: 0-3) (0x00A4+0x8**x*)



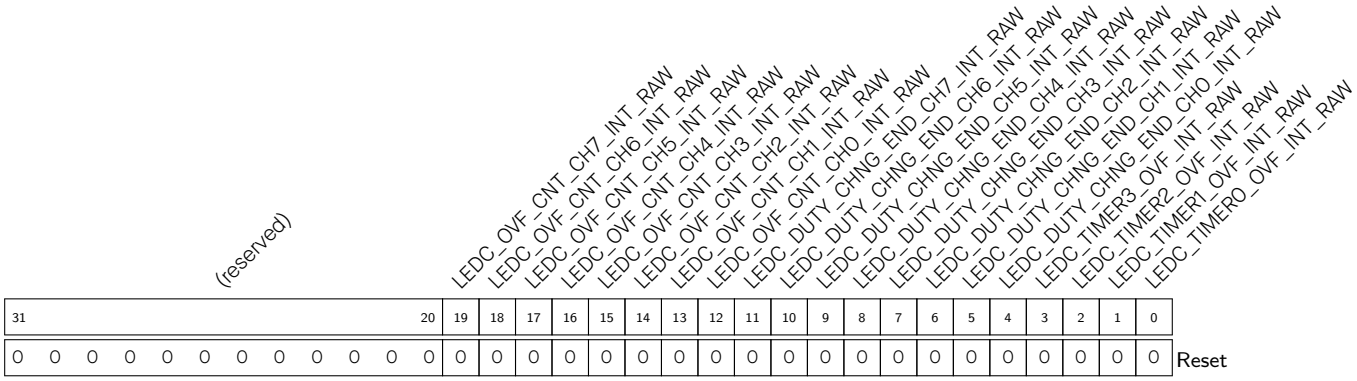
LEDC_TIMER*x*_CNT 定时器 *x* 计数器的当前数值。(RO)

Register 23.16. LEDC_TIMERx_CNT_CAP_REG (x: 0-3) (0x0150+0x4*x)



LEDC_TIMERx_CNT_CAP 抓取的定时器 *x* 的计数器数值。(RO)

Register 23.17. LEDC_INT_RAW_REG (0x00C0)

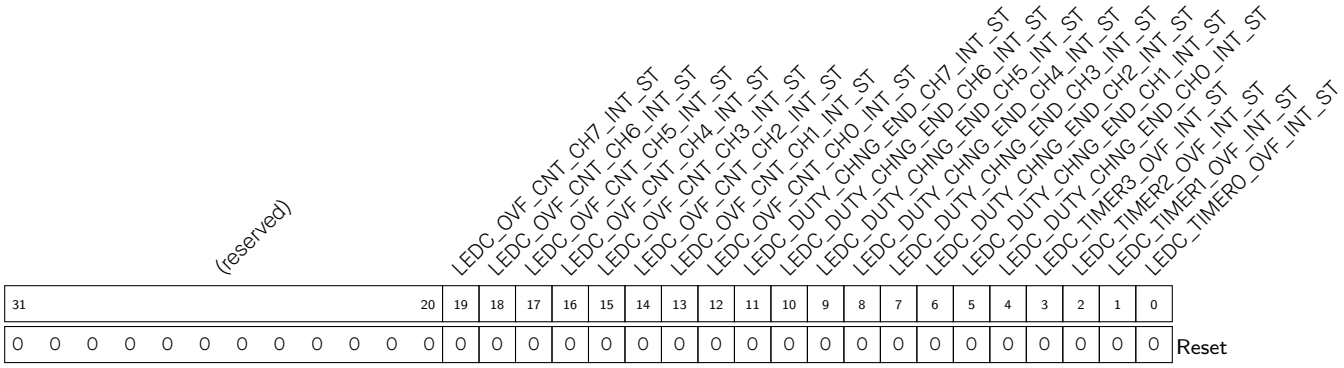


LEDC_TIMERx_OVF_INT_RAW (x: 0-3) 原始状态位：LEDC_TIMERx_OVF_INT 的原始中断状态。定时器 *x* 的计数器到达最大值时，触发该中断。(R/WTC/SS)

LEDC_DUTY_CHNG_END_CHn_INT_RAW (n: 0-7) 原始状态位：LEDC_DUTY_CHNG_END_CHn_INT 的原始中断状态。占空比渐变结束时会触发该中断。(R/WTC/SS)

LEDC_OVF_CNT_CHn_INT_RAW (n: 0-7) 原始状态位：LEDC_OVF_CNT_CHn_INT 的原始中断状态。当 ovf_cnt 达到 LEDC_OVF_NUM_CHn 的设定值时，触发该中断。(R/WTC/SS)

Register 23.18. LEDC_INT_ST_REG (0x00C4)

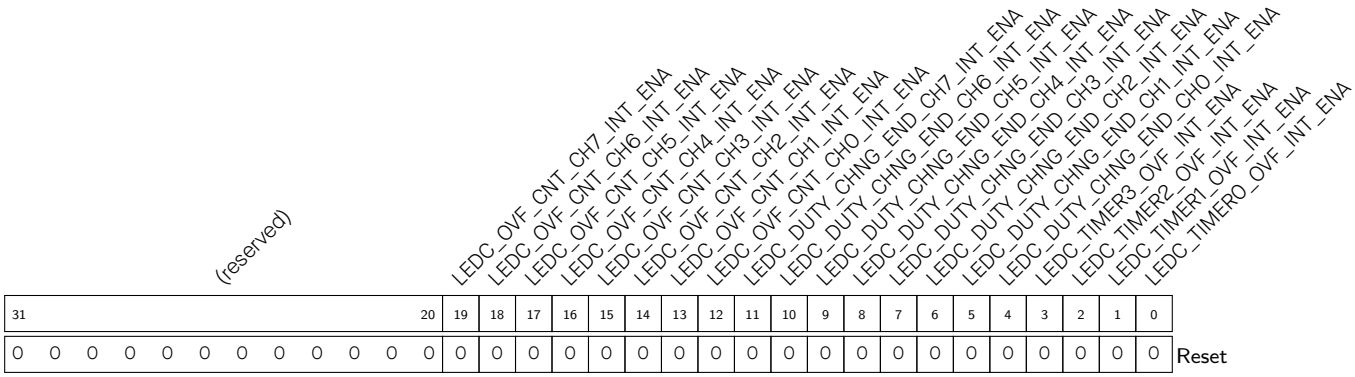


LEDC_TIMER*x*_OVF_INT_ST (*x*: 0-3) 屏蔽状态位：LEDC_TIMER*x*_OVF_INT 的屏蔽中断状态。仅在 LEDC_TIMER*x*_OVF_INT_ENA 为 1 时有效。(RO)

LEDC_DUTY_CHNG_END_CH*n*_INT_ST (*n*: 0-7) 屏蔽状态位：LEDC_DUTY_CHNG_END_CH*n*_INT 的屏蔽中断状态。仅在 LEDC_DUTY_CHNG_END_CH*n*_INT_ENA 为 1 时有效。(RO)

LEDC_OVF_CNT_CH*n*_INT_ST (*n*: 0-7) 屏蔽状态位：LEDC_OVF_CNT_CH*n*_INT 的屏蔽中断状态。仅在 LEDC_OVF_CNT_CH*n*_INT_ENA 为 1 时有效。(RO)

Register 23.19. LEDC_INT_ENA_REG (0x00C8)

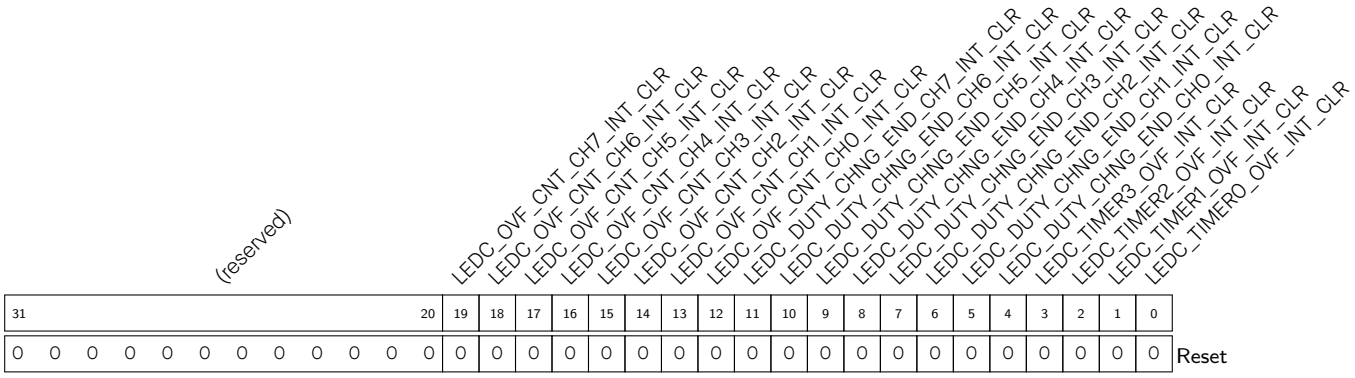


LEDC_TIMER*x*_OVF_INT_ENA (*x*: 0-3) 使能位：写 1 使能 LEDC_TIMER*x*_OVF_INT。(R/W)

LEDC_DUTY_CHNG_END_CH*n*_INT_ENA (*n*: 0-7) 使能位：写 1 使能 LEDC_DUTY_CHNG_END_CH*n*_INT。(R/W)

LEDC_OVF_CNT_CH*n*_INT_ENA (*n*: 0-7) 使能位：写 1 使能 LEDC_OVF_CNT_CH*n*_INT。(R/W)

Register 23.20. LEDC_INT_CLR_REG (0x00CC)

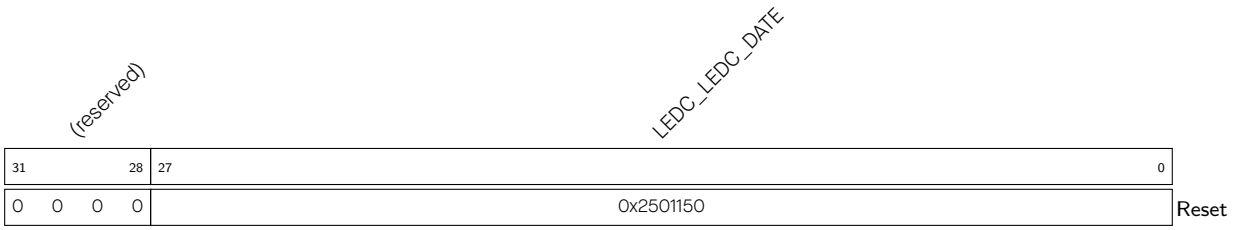


LEDC_TIMER*x*_OVF_INT_CLR (*x*: 0-3) 清零位：写 1 清零 LEDC_TIMER*x*_OVF_INT。(WT)

LEDC_DUTY_CHNG_END_CH*n*_INT_CLR (*n*: 0-7) 清零位：写 1 清零 LEDC_DUTY_CHNG_END_CH*n*_INT。(WT)

LEDC_OVF_CNT_CH*n*_INT_CLR (*n*: 0-7) 清零位：写 1 清零 LEDC_OVF_CNT_CH*n*_INT。(WT)

Register 23.21. LEDC_DATE_REG (0x017C)



LEDC_LEDC_DATE 版本控制寄存器。(R/W)

第 24 章

电机控制脉宽调制器 (MCPWM)

24.1 概述

电机控制脉宽调制器 (MCPWM) 外设用于电机和电源控制。该外设提供了 6 个 PWM 输出，可在几种拓扑结构中运行。常见的拓扑结构之一是用一对 PWM 输出来驱动 H 桥以控制电机旋转速度和旋转方向。

MCPWM 外设主要分为五个子模块：PWM 定时器、PWM 操作器、捕获模块、事件矩阵 (Event Task Matrix, ETM) 模块、故障检测模块。PWM 定时器提供参考时序，可以自由运行，或同步到其他定时器或外部源。每个 PWM 操作器具有生成一个 PWM 通道波形对所需的所有控制资源。捕获模块用于需要精确定时外部事件的系统。ETM 模块对输入到 MCPWM 的任务做出相应的响应动作，根据不同的运动状态产生相应的事件。故障检测模块用于捕获外部故障，使得系统可以选择做出响应。

ESP32-S31 有一个 MCPWM 外设，即 MCPWM0。

24.2 主要特性

MCPWM 外设有一个时钟分频器（预分频器）、三个 PWM 定时器、三个 PWM 操作器、一个捕获模块、一个 ETM 模块和一个故障检测模块。MCPWM 的主时钟可以从三个时钟源中任选其一，分别为：PLL_F160M_CLK, XTAL_CLK 和 RC_FAST_CLK（通过 PCR_PWM_CLK_CONF_REG 寄存器的 PCR_PWM_CLKM_SEL 字段配置）。图 24.2-1 描述了 MCPWM 内的模块和接口上的信号。PWM 定时器用于生成定时参考，PWM 操作器将根据生成的定时参考生成所需的波形。通过配置，任一 PWM 操作器可以使用任一 PWM 定时器的定时参考。不同的 PWM 操作器可以使用相同的 PWM 定时器的定时参考产生 PWM 信号，也可以使用不同的 PWM 定时器的值来生成单独的 PWM 信号。不同的 PWM 定时器可以进行同步。

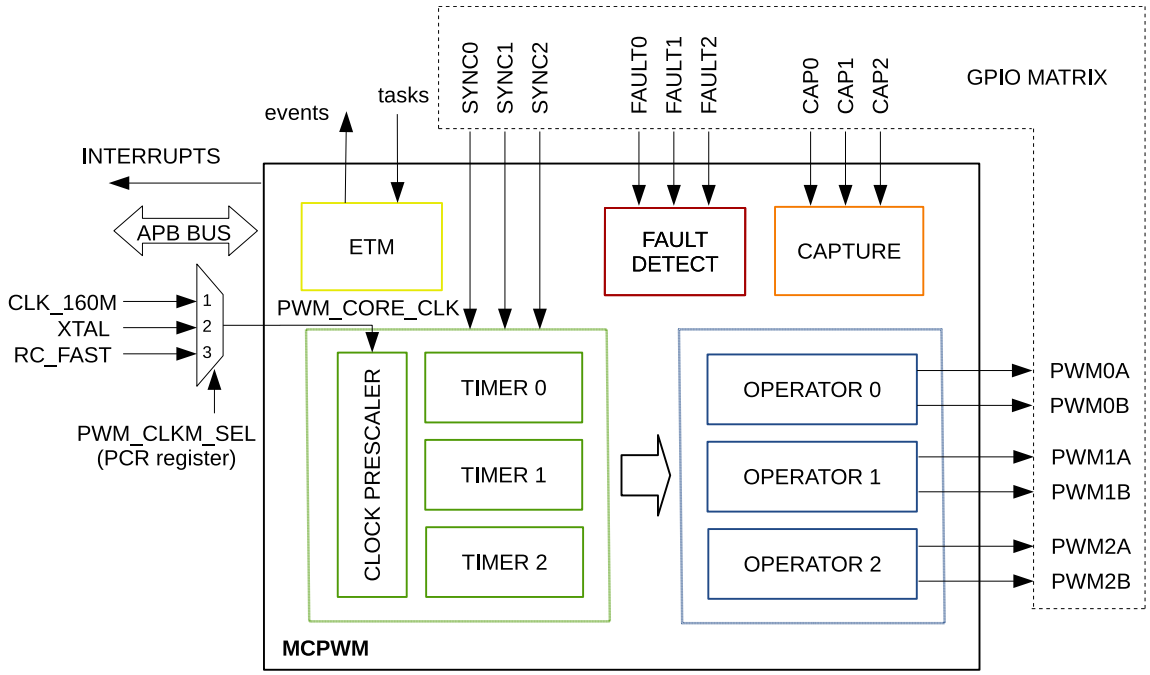


图 24.2-1. MCPWM 外设概览

以下是图 24.2-1 中模块的功能概述：

- PWM 定时器 0、1 和 2：
  - 每个 PWM 定时器都有一个专用的 8 位时钟预分频器。
  - PWM 定时器中的 16 位计数器的工作模式包括：递增计数模式，递减计数模式，递增递减循环计数模式。
  - 硬件/软件同步可以触发 PWM 定时器重载，重载值位于相位寄存器中；同时触发预分频的重启，从而同步定时器的时钟。硬件同步源可以来自任何 GPIO 或任何其他 PWM 定时器的 sync_out 信号。软件同步源来自于向 MCPWM_TIMER*n*_SYNC_SW 位写入取反值。
- PWM 操作器 0、1 和 2：
  - 每个 PWM 操作器有两个 PWM 输出（PWM*x*A 和 PWM*x*B），可以在对称和非对称配置中独立工作。
  - 可以通过异步方式更新对 PWM 信号的控制。
  - 死区时间在上升沿和下降沿可配置，并可分别设置。
  - 所有事件都可触发 CPU 中断。
  - 通过高频载波信号调制 PWM 输出，在使用变压器隔离栅极驱动器时可发挥巨大作用。
  - 周期、时间戳寄存器和其他主要的控制寄存器有影子寄存器，具有灵活的更新方式。
- 故障检测模块：
  - 出现故障时，可选择在逐周期模式或一次性模式下处理。
  - 出现故障时可强制 PWM 输出高或低电平。
- 捕获模块：
  - 捕获模块的时钟为 MCPWM 的主时钟。

  - 旋转电机的速度测量。
  - 位置传感器脉冲之间的间隔时间测量。
  - 脉冲序列信号的周期和占空比测量。
  - 从电流/电压传感器的占空比编码信号导出的解码电流或电压振幅。
  - 3 个独立的捕获通道，各具备一个 32 位的时间戳寄存器。
  - 输入捕获信号可以预分频，边沿极性可选。
  - 捕获定时器可以与 PWM 定时器或外部信号同步。
  - 3 个捕获通道上都可以产生中断。
- ETM 模块：
  - 根据每个定时器和操作器不同的运行状态，产生不同的事件。
  - 每个定时器和操作器响应其对应的任务，自动进行相应的操作。
  - 每个事件和任务都可以独立使能。某个事件不使能时，不产生对应的事件；某个任务不使能时，不响应对应的任务。

24.3 模块

24.3.1 概述

每个 MCPWM 包含的关键模块为定时器模块、操作器模块、故障检测模块、捕获模块和 ETM 模块，关键模块的功能详见下文。

24.3.1.1 定时器模块

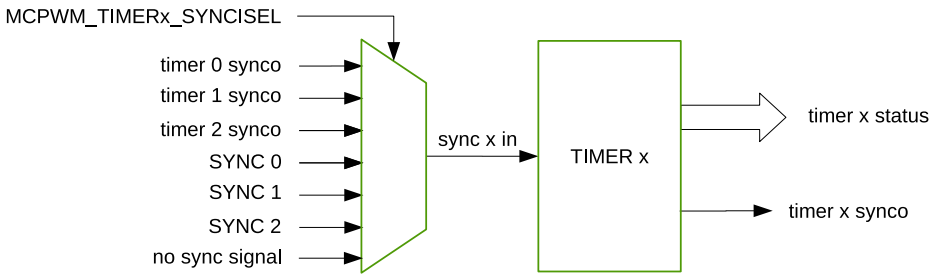


图 24.3-1. 定时器模块

该模块包含一个定时器，它可以按照指定的周期进行计数，它可以工作在递增计数模式，递减计数模式或递增递减循环计数模式下，它可以选择不同的同步输入源（共 7 个可选输入源）来进行计数值和方向的重载，从而使多个定时器彼此同步，它还可以输出一个同步输出源（共 4 个可选输出源）供其他定时器使用。

图中的 timer*x* status 指定时器模块输出的定时器状态，如计数模式，计数值等于 0 或周期等。

24.3.1.2 操作器模块

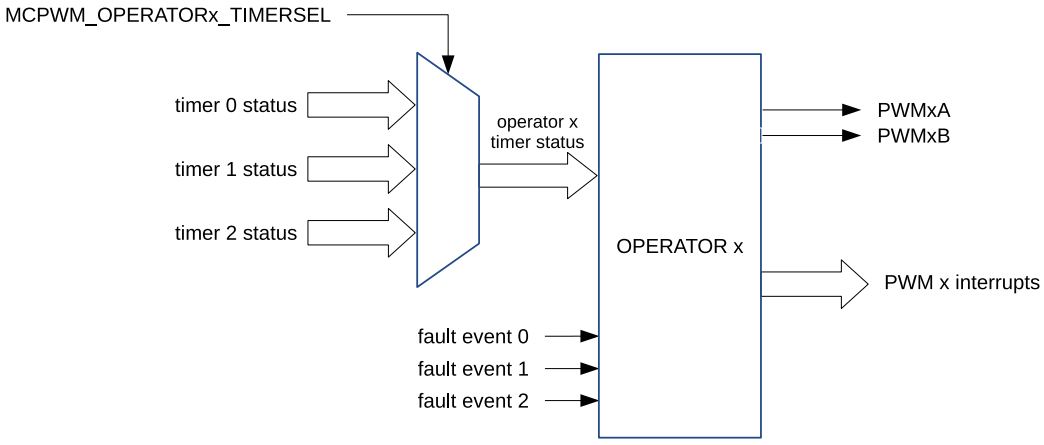


图 24.3-2. 操作器模块

一个 PWM 操作器包含一个 PWM 生成器、一个死区生成器、一个故障检测模块以及一个 PWM 载波模块，它们的功能如表 24.3-1 所示：

表 24.3-1. 操作器模块包含的各个子模块功能

子模块	功能
PWM 生成器	按照指定的占空比和波形，生成 PWM 信号
死区生成器	可以选择对 PWM 生成器产生的 PWM 信号的上升沿和下降沿添加死区（即对上升沿和下降沿添加延迟）以及进行反相等操作
PWM 载波	可以选择对死区生成器产生的 PWM 信号添加载波，载波的频率可以配置，并且添加载波后的 PWM 波形的第一个脉冲持续时间可以配置
故障检测	检测故障并生成对应的故障事件，按照指定的间隔模式（一次性模式或逐周期模式）响应故障事件以及发生故障事件时，按照指定的方式对 PWM 载波模块产生的 PWM 信号采取操作

24.3.1.3 捕获模块

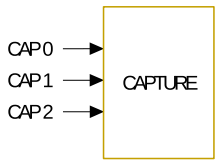


图 24.3-3. 捕获模块

该模块包含一个可以进行同步的定时器，当输入的捕获信号有效时，将捕获定时器的计数值。

24.3.1.4 ETM 模块

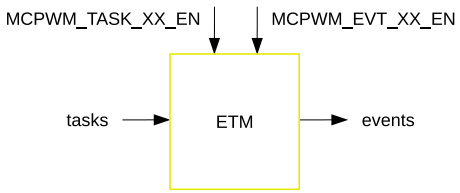


图 24.3-4. ETM 模块

该模块对接收到的任务进行响应，或产生事件并输出，可以独立配置每个事件和任务的使能。

24.3.2 PWM 定时器模块

MCPWM 外设有三个 PWM 定时器模块。它们中的任何一个都可以决定三个 PWM 操作器模块中任意一个的必要事件时序。通过使用 GPIO 交换矩阵的同步信号，内置同步逻辑允许一个或多个 MCPWM 外设中的多个 PWM 定时器模块作为一个系统协同工作。

24.3.2.1 PWM 定时器模块的配置

用户可配置 PWM 定时器模块的以下功能：

- 通过指定 PWM 定时器频率或周期来控制事件发生的频率

- 配置某个 PWM 定时器与其他 PWM 定时器或模块进行相位同步
- 配置定时器计数模式：递增，递减，或递增递减循环计数模式
- 使用预分频器更改 PWM 定时器时钟 (PT_CLK) 的速率。每个定时器都有自己的预分频器，通过寄存器 MCPWM_TIMER0_CFG0_REG 的 MCPWM_TIMER*n*_PRESCALE 配置。PWM 定时器根据该寄存器的设置以较慢的速度递增或递减。当定时器停止后并再次开始计数时，新的MCPWM_TIMER*n*_PRESCALE 配置值才会生效。

24.3.2.2 PWM 定时器工作模式和定时事件生成

PWM 定时器有三种工作模式，由 PWM*x* 定时器模式字段配置：

- 递增计数模式：
  定时器从零增加到周期字段中配置的值。达到周期值后，PWM 定时器清零，并再次开始递增。PWM 周期 = 周期寄存器中的周期值 + 1。
  说明：周期寄存器为 MCPWM_TIMER*n*_PERIOD (*x* = 0, 1, 2)，对应 MCPWM_TIMER0_PERIOD、MCPWM_TIMER1_PERIOD、MCPWM_TIMER2_PERIOD。
- 递减计数模式：
  PWM 定时器从周期寄存器中的值开始递减到零。达到零后，将恢复为周期值，并再次开始递减。在这种情况下，PWM 周期 = 周期寄存器中的周期值 + 1。
- 递增-递减循环模式：
  此模式结合了上述两种模式。PWM 定时器从零开始递增，直到达到周期值，再次递减为零。PWM 定时器按照此模式循环递增递减。PWM 周期 = 周期寄存器的周期值 × 2。

图 24.3-5 至 24.3-8 显示不同模式下的 PWM 定时器波形，包括同步事件期间的定时器行为。递增计数模式中，同步后永远保持递增计数；递减计数模式中，同步后永远保持递减计数；递增-递减循环模式中，同步后由
MCPWM_TIMER*n*_PHASE_DIRECTION 配置计数方向。

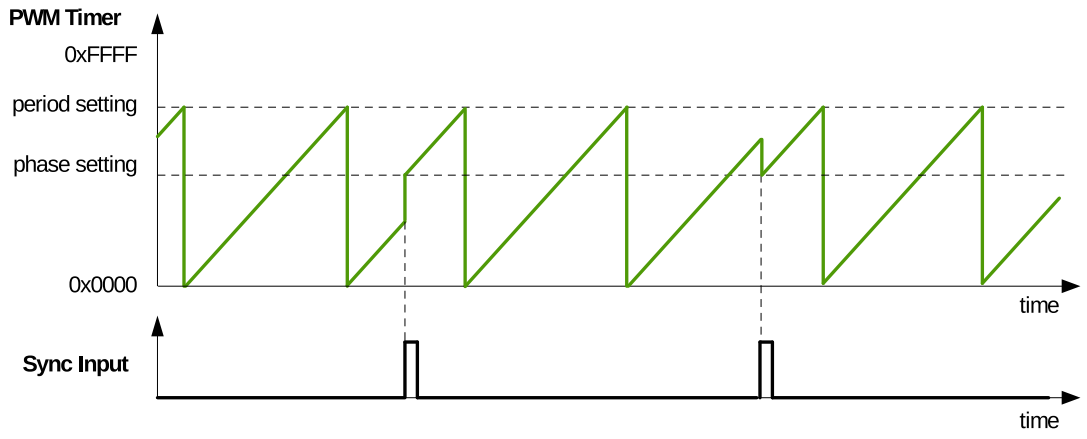


图 24.3-5. 递增计数模式波形

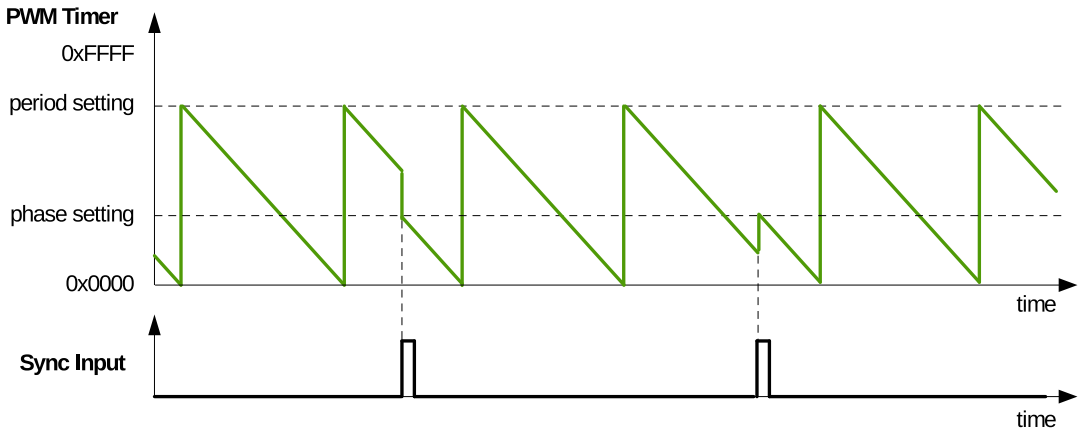


图 24.3-6. 递减计数模式波形

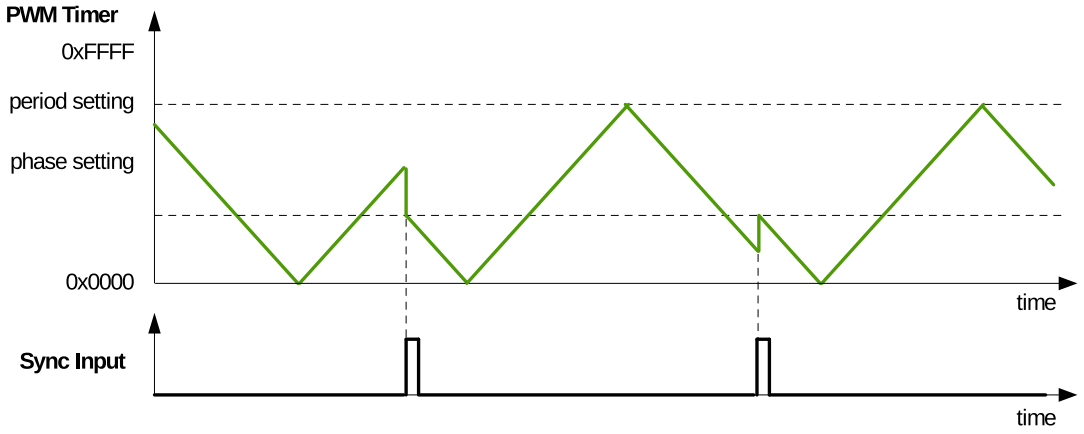


图 24.3-7. 递增递减循环模式波形，同步事件后递减

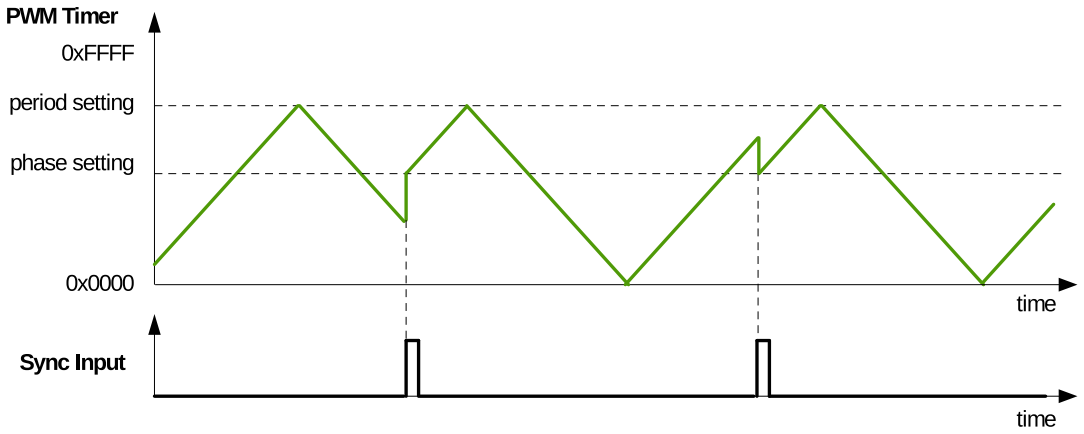


图 24.3-8. 递增递减循环模式波形，同步事件后递增

PWM 定时器运行时，定期自动生成以下定时事件：

- UTEP：当 PWM 定时器等于周期字段的值 (MCPWM_TIMER*n*_PERIOD) 且 PWM 定时器递增计数时，生成的定时事件。
- UTEZ：当 PWM 定时器等于零且 PWM 定时器递增计数时，生成的定时事件。
- DTEP：当 PWM 定时器等于周期字段的值 (MCPWM_TIMER*n*_PERIOD) 且 PWM 定时器递减时，生成的定时事件。
- DTEZ：当 PWM 定时器等于零且 PWM 定时器递减时，生成的定时事件。

图 24.3-9 至 24.3-11 为 U/DTEP 和 U/DTEZ 定时事件的时序波形。

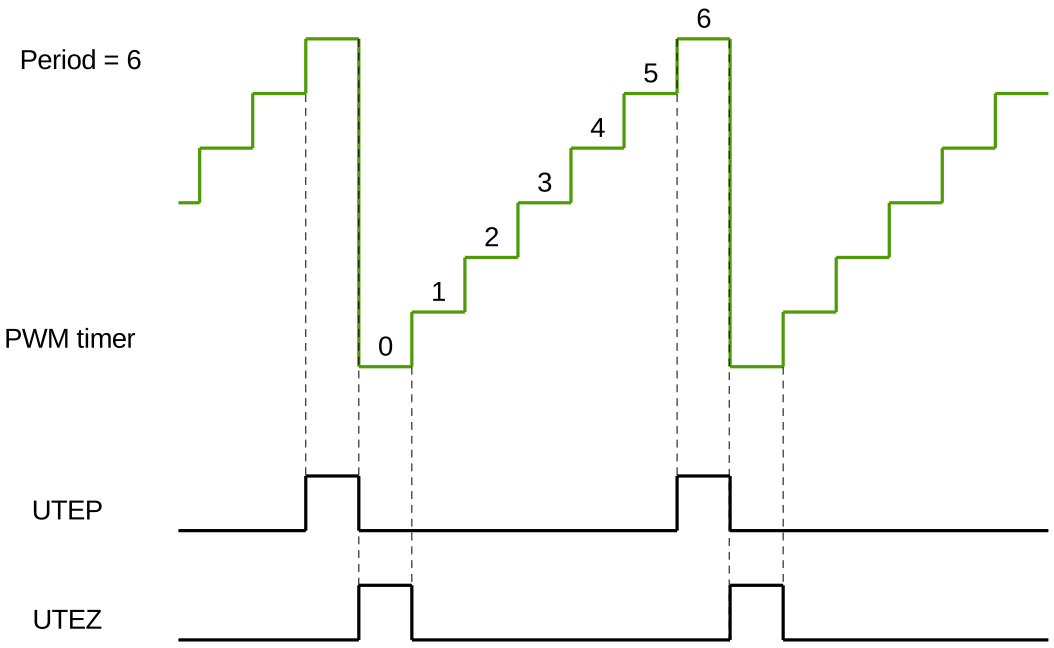


图 24.3-9. 递增模式中生成的 UTEP 和 UTEZ

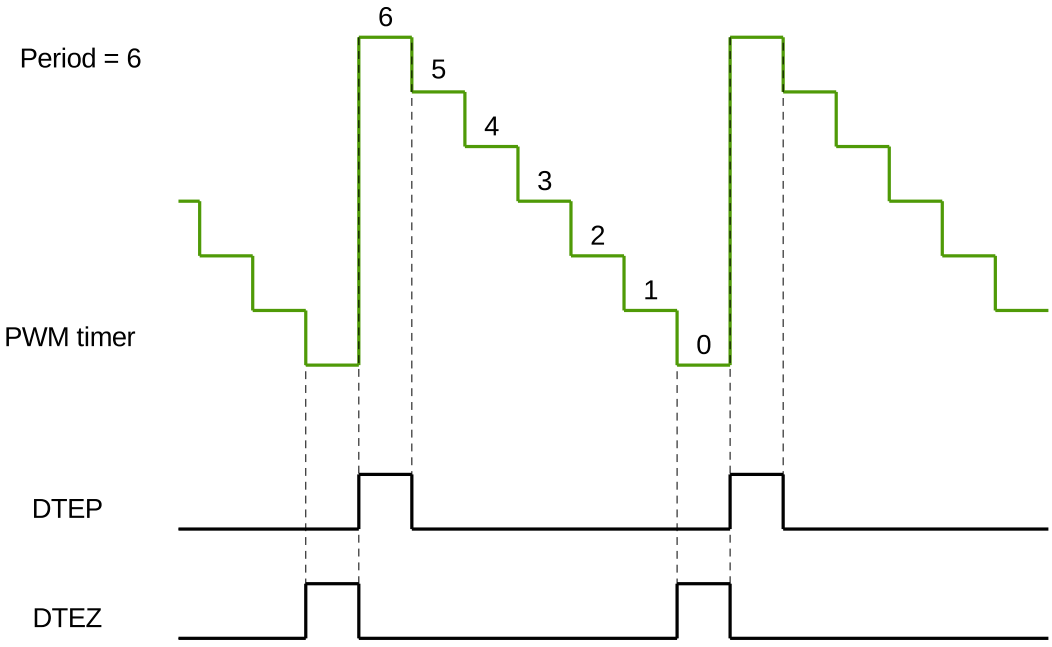


图 24.3-10. 递减模式中生成的 DTEP 和 DTEZ

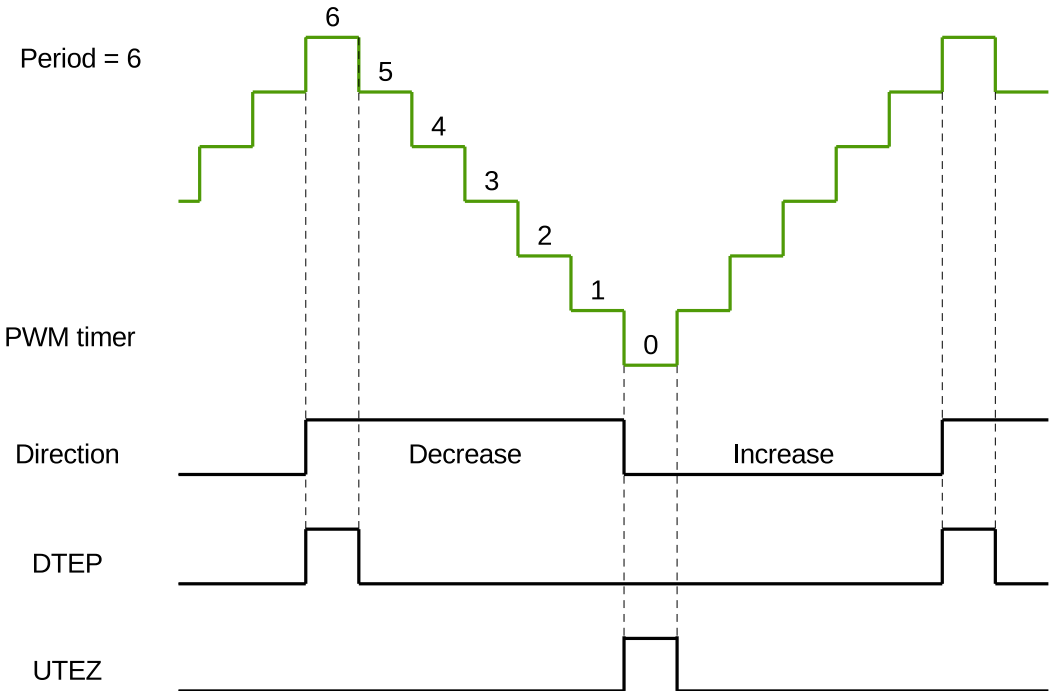


图 24.3-11. 递增递减模式中生成的 DTEP 和 UTEZ

需注意，在递增递减模式下，当定时器递增计数时，定时器范围为 [0，周期值 - 1]；递减计数时，定时器范围为 [周期值，1]。也就是说，在该模式下，将定时器同步为 0 时，则不支持递减计数，即 MCPWM_TIMER*n*_PHASE_DIRECTION 不能为 1；定时器同步为周期值时，也不支持递增计数，即 MCPWM_TIMER*n*_PHASE_DIRECTION 不能为 0。所以，当定时器同步为 0 时，计数方向只能为递增，即将 MCPWM_TIMER*n*_PHASE_DIRECTION 配置为 0；定时器同步为周期值时，计数方向只能为递减，即将 MCPWM_TIMER*n*_PHASE_DIRECTION 配置为 1。

24.3.2.3 PWM 定时器影子寄存器

PWM 定时器周期寄存器和 PWM 定时器时钟预分频器寄存器配有影子寄存器。影子寄存器能够备份即将写入有效寄存器的值，并在硬件同步的特定时刻写入有效寄存器。有效寄存器和影子寄存器的功能如下：

- 有效寄存器：直接控制硬件执行的所有操作。
- 影子寄存器：作为临时缓冲区，存储即将写入有效寄存器的值。在用户配置的某个时间点，影子寄存器中的值被写入有效寄存器。在此之前，影子寄存器的内容对受控硬件没有任何直接影响，防止寄存器由软件异步修改时可能发生的错误硬件操作。影子寄存器和有效寄存器具有相同的存储器地址。软件始终在影子寄存器中进行读写操作。

  当定时器开始工作时，时钟预分频器的有效寄存器更新。当 MCPWM_GLOBAL_UP_EN 置 1 时，可通过以下方式选择更新有效寄存器的时间点：

  - 将 MCPWM_TIMER*n*_PERIOD_UPMETHOD 置 0，立即更新。
  - 将 MCPWM_TIMER*n*_PERIOD_UPMETHOD 置 1，当 PWM 定时器值为 0 时，开始更新。
  - 将 MCPWM_TIMER*n*_PERIOD_UPMETHOD 置 2，当 PWM 定时器同步时，开始更新。
  - 将 MCPWM_TIMER*n*_PERIOD_UPMETHOD 置 3，当 PWM 定时器值为 0 时或同步时，开始更新。
  - 软件也可以触发全局强制更新位 MCPWM_GLOBAL_FORCE_UP，该位将触发模块中的所有寄存器根据影子寄存器进行更新。

24.3.2.4 PWM 定时器同步和锁相

PWM 模块采用灵活的同步方法。每个 PWM 定时器都有一个同步输入和一个同步输出。同步输入可以从 GPIO 矩阵的三个同步输出和三个 PWM 定时器的三个同步输出信号中选择。同步输出可以在 PWM 定时器等于周期、PWM 定时器等于零、同步输入有效或软件同步时生成。因此，每个 PWM 定时器都可以灵活的选择同步源，从而实现多种相位同步方式。在同步时，PWM 定时器的预分频器会复位其计数器，并重新开始对时钟的分频，以保证定时器的正确性。

24.3.3 PWM 操作器模块

PWM 操作器模块具备以下功能：

- 根据相应 PWM 定时器的定时参考生成 PWM 信号对。
- PWM 信号对的每个信号都可以独立设置特定的死区（即对 PWM 信号的上升沿和下降沿添加延迟）。
- 可通过配置将载波叠加到 PWM 信号上。
- 故障条件下处理响应。

图 24.3-12 为 PWM 操作器的框图。

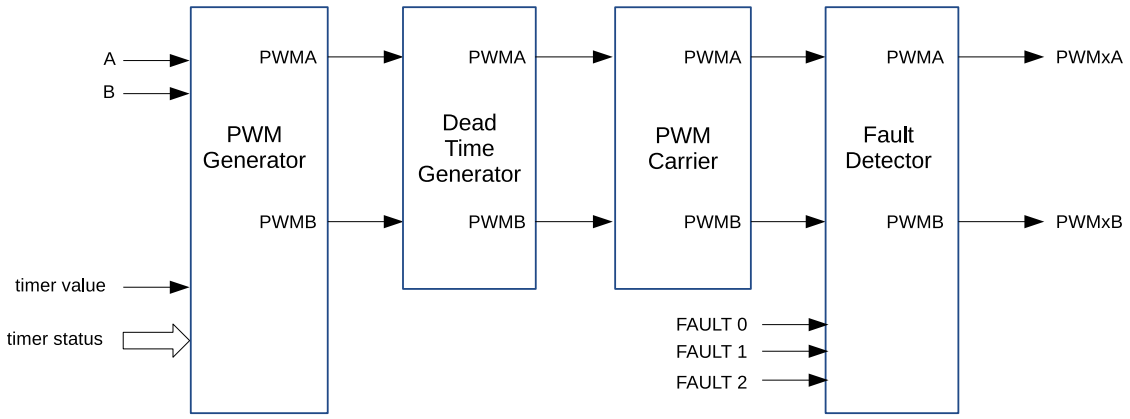


图 24.3-12. PWM 操作器的子模块

24.3.3.1 PWM 生成器模块

PWM 生成器模块的作用

此模块可以生成或导入重要的定时事件，转化为特定操作后，在 PWM*x*A 和 PWM*x*B 输出处生成所需的波形。

PWM 生成器模块执行以下操作：

- 基于使用寄存器 A 和 B 配置的时间戳生成定时事件。满足以下条件时发生定时事件（寄存器 A 和 B 的配置方法在章节 24.3.3.1 描述）：
  - UTEA：PWM 定时器递增计数并且其值等于寄存器 A。
  - UTEB：PWM 定时器递增计数并且其值等于寄存器 B。
  - DTEA：PWM 定时器递减计数并且其值等于寄存器 A。
  - DTEB：PWM 定时器递减计数并且其值等于寄存器 B。
- 基于故障或同步事件生成 U/DTO、U/DT1 定时事件。
  - UTO：定时器递增计数并且检测到 FAULTO 事件（字段 MCPWM_GEN*n*_TO_SEL 置 0）或检测到 FAULT1 事件（字段 MCPWM_GEN*n*_TO_SEL 置 1）或检测到 FAULT2 事件（字段 MCPWM_GEN*n*_TO_SEL 置 2）或同步（字段 MCPWM_GEN*n*_TO_SEL 置 3）。
  - UT1：定时器递增计数并且检测到 FAULTO 事件（字段 MCPWM_GEN*n*_T1_SEL 置 0）或检测到 FAULT1 事件（字段 MCPWM_GEN*n*_T1_SEL 置 1）或检测到 FAULT2 事件（字段 MCPWM_GEN*n*_T1_SEL 置 2）或同步（字段 MCPWM_GEN*n*_T1_SEL 置 3）。
  - DTO：定时器递减计数并且检测到 FAULTO 事件（字段 MCPWM_GEN*n*_TO_SEL 置 0）或检测到 FAULT1 事件（字段 MCPWM_GEN*n*_TO_SEL 置 1）或检测到 FAULT2 事件（字段 MCPWM_GEN*n*_TO_SEL 置 2）或同步（字段 MCPWM_GEN*n*_TO_SEL 置 3）。
  - DT1：定时器递减计数并且检测到 FAULTO 事件（字段 MCPWM_GEN*n*_T1_SEL 置 0）或检测到 FAULT1 事件（字段 MCPWM_GEN*n*_T1_SEL 置 1）或检测到 FAULT2 事件（字段 MCPWM_GEN*n*_T1_SEL 置 2）或同步（字段 MCPWM_GEN*n*_T1_SEL 置 3）。
- 当这些定时事件同时发生时管理优先级。
- 基于定时事件产生置 1、置 0 和取反操作。

- 根据 PWM 生成器模块的配置来控制 PWM 占空比。
- 使用影子寄存器处理新的时间戳值，以防止 PWM 波形中的毛刺干扰。

PWM 生成器影子寄存器

硬件使用的时间戳寄存器 A 和 B 都配有影子寄存器，为 MCPWM_GEN*n*_A_REG 和 MCPWM_GEN*n*_B_REG。影子寄存器提供了一种与硬件同步更新寄存器的方法。

当 MCPWM_GLOBAL_UP_EN 置 1 时，影子寄存器中的值可在某个特定时间写入有效寄存器中。MCPWM_GEN*n*_A_REG 和 MCPWM_GEN*n*_B_REG 的更新方式字段为 MCPWM_GEN*n*_CFG_UPMETHOD。软件也可以触发全局强制更新位 MCPWM_GLOBAL_FORCE_UP，该位将触发模块中的所有寄存器根据影子寄存器进行更新。更多关于影子寄存器的描述，请参考章节 24.3.2.3。

定时事件

表 24.3-2 概括了所有定时信号和事件。

表 24.3-2. PWM 生成器中的所有定时事件

信号	事件描述	PWM 定时器操作
DTEP	PWM 定时器的值等于周期寄存器的值	PWM 定时器递减计数
DTEZ	PWM 定时器的值等于 0	
DTEA	PWM 定时器的值等于寄存器 A	
DTEB	PWM 定时器的值等于寄存器 B	
DTO 事件	基于故障或同步事件	
DT1 事件	基于故障或同步事件	
UTEP	PWM 定时器的值等于周期寄存器的值	PWM 定时器递增计数
UTEZ	定时器的值等于 0	
UTEA	PWM 定时器的值等于寄存器 A	
UTEB	PWM 定时器的值等于寄存器 B	
UTO 事件	基于故障或同步事件	
UT1 事件	基于故障或同步事件	
软件强制事件	软件触发的异步事件	-

软件强制事件用于在 PWM*x*A 和 PWM*x*B 的输出上施加非连续或连续的强制电平。此更改是异步完成的。软件强制事件由寄存器 MCPWM_GEN*n*_FORCE_REG 控制。

PWM 生成器模块中 TO/T1 的选择和配置独立于故障处理模块中的故障事件的配置，某个特定的跳闸事件可以不被配置为在故障处理器模块中引起跳闸动作，而是用于由 PWM 生成器用于触发 TO/T1 以控制 PWM 波形。

需要注意的是，当 PWM 定时器处于递增递减循环计数模式时，它将在 TEP 事件后递减，在 TEZ 事件后递增。因此，在此模式下，将出现 DTEP 和 UTEZ，但不会出现 UTEP 和 DTEZ。

PWM 生成器可以同时处理多个事件。事件优先级由硬件决定，详见表 24.3-3 和表 24.3-4。优先级从 1（最高）到 7（最低）排列。需要注意的是，TEP 和 TEZ 事件的优先级取决于 PWM 定时器的计数模式。

如果 A 或 B 的值设置为大于周期，则 U/DTEA 和 U/DTEB 将永远不会发生。

表 24.3-3. PWM 定时器递增计数时，定时事件的优先级

优先级	事件
1（最高）	软件强制事件
2	UTEP
3	UTO
4	UT1
5	UTEB
6	UTEA
7（最低）	UTEZ

表 24.3-4. PWM 定时器递减计数时，定时事件的优先级

优先级	事件
1（最高）	软件强制事件
2	DTEZ
3	DTO
4	DT1
5	DTEB
6	DTEA
7（最低）	DTEP

说明：

1. UTEP 和 UTEZ 不同时发生。当 PWM 定时器处于递增计数模式，UTEP 将始终比 UTEZ 提前一个周期发生，如图 24.3-9 所示。因此，它们对 PWM 信号的作用不会彼此干扰。当 PWM 定时器处于递增递减循环模式时，UTEP 不会发生。
2. DTEP 和 DTEZ 不同时发生。当 PWM 定时器处于递减计数模式时，DTEZ 始终比 DTEP 早一个周期发生，如图 24.3-10 所示。因此，它们对 PWM 信号的作用不会彼此干扰。当 PWM 定时器处于递增递减循环模式时，DTEZ 不会发生。

PWM 信号生成

当某个定时事件发生时，PWM 生成器控制输出 PWM*x*A 和 PWM*x*B 的电平。定时事件通过 PWM 定时器计数模式（递增或递减）进一步限定。根据定时器计数方向，模块可以对 PWM 定时器递增或递减计数的阶段执行不同的操作。

可以在 PWM*x*A 和 PWM*x*B 输出上配置以下操作：

- 置为高电平：将 PWM*x*A 或 PWM*x*B 的输出配置为高电平。
- 置为低电平：通过将 PWM*x*A 或 PWM*x*B 的输出配置为低电平，来清除 PWM*x*A 或 PWM*x*B 的输出。
- 取反：将 PWM*x*A 或 PWM*x*B 的当前输出电平配置为相反的值。如果它当前被拉高，则拉低，或反之。
- 不进行操作：保持 PWM*x*A 和 PWM*x*B 输出电平不变。在这种状态下，仍然可以触发中断。

输出上的操作通过寄存器 MCPWM_GEN*n*_A_REG 和 MCPWM_GEN*n*_B_REG 配置。每一次输出的操作都独立配置。此外，可以根据事件，灵活地在输出上执行不同操作。表 24.3-2 中列举的任何事件都可以作用于 PWM*x*A 或 PWM*x*B 输出上。关于生成器 0，1 或 2 的寄存器信息，请参考章节 24.5。

常见配置的波形

在下面各种波形的配置中，period 指 PWM 操作器所选 PWM 定时器的周期寄存器的配置值（即 MCPWM_TIMER*n*_PERIOD (*x* = 0, 1, 2)），配置方法参考章节 24.3.2。A 和 B 指硬件使用的时间戳寄存器 A 和 B，配置方法参考章节 24.3.3.1。UTEA/B、UTEZ/P、DTEA/B、DTEZ/P、DTO/1 和 UTO/1 指 PWM 生成器的各种定时事件，其定义参考章节 24.3.3.1。向上的箭头、向下的箭头、T 标识分别代表定时事件发生时将 PWM 波形置为高电平、置为低电平、取反，配置方法参考章节 24.3.3.1。PWM*x*A 和 PWM*x*B 是 PWM 生成器 *x* 的输出信号。

图 24.3-13 为 PWM 定时器在递增递减循环计数时生成的对称 PWM 波形。该模式下的直流 0%-100% 调制可由以下公式获得：

$$Duty = (Period - A) \div Period$$

如果 A 的值等于 PWM 定时器的值，并且 PWM 定时器递增，则 PWM 输出被拉高。如果 A 的值等于 PWM 定时器的值，并且 PWM 定时器递减，则 PWM 输出被拉低。

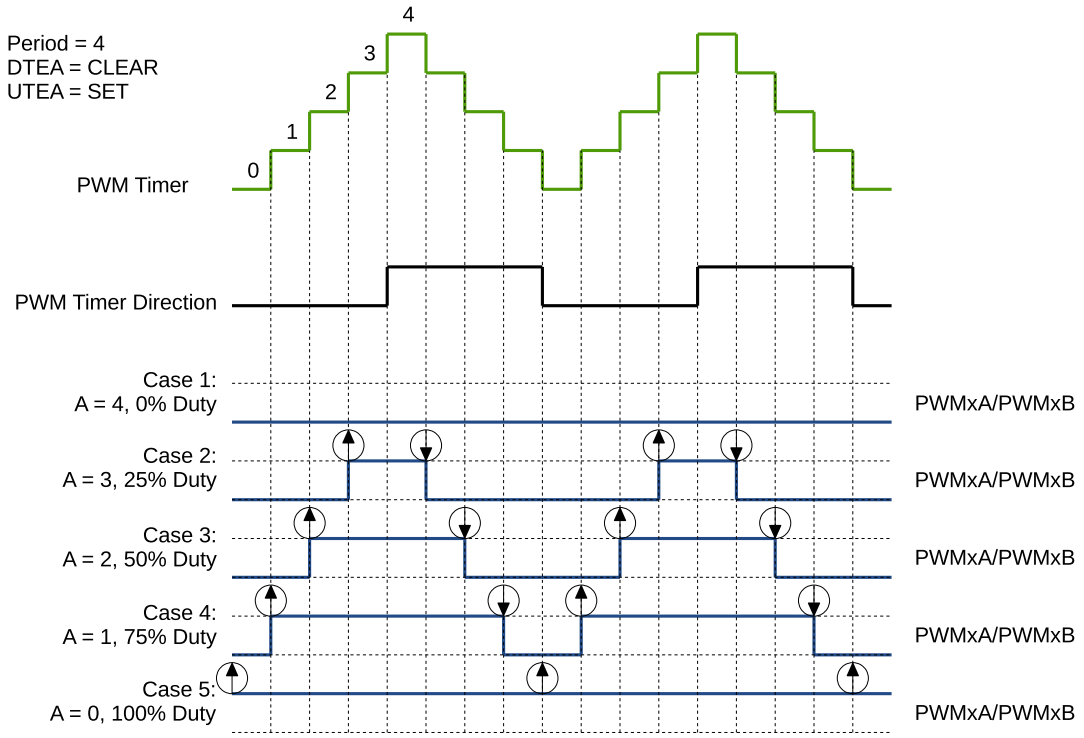


图 24.3-13. 递增递减模式下的对称波形

图 24.3-14 至图 24.3-17 的 PWM 波形描述了常见的 PWM 生成器配置。

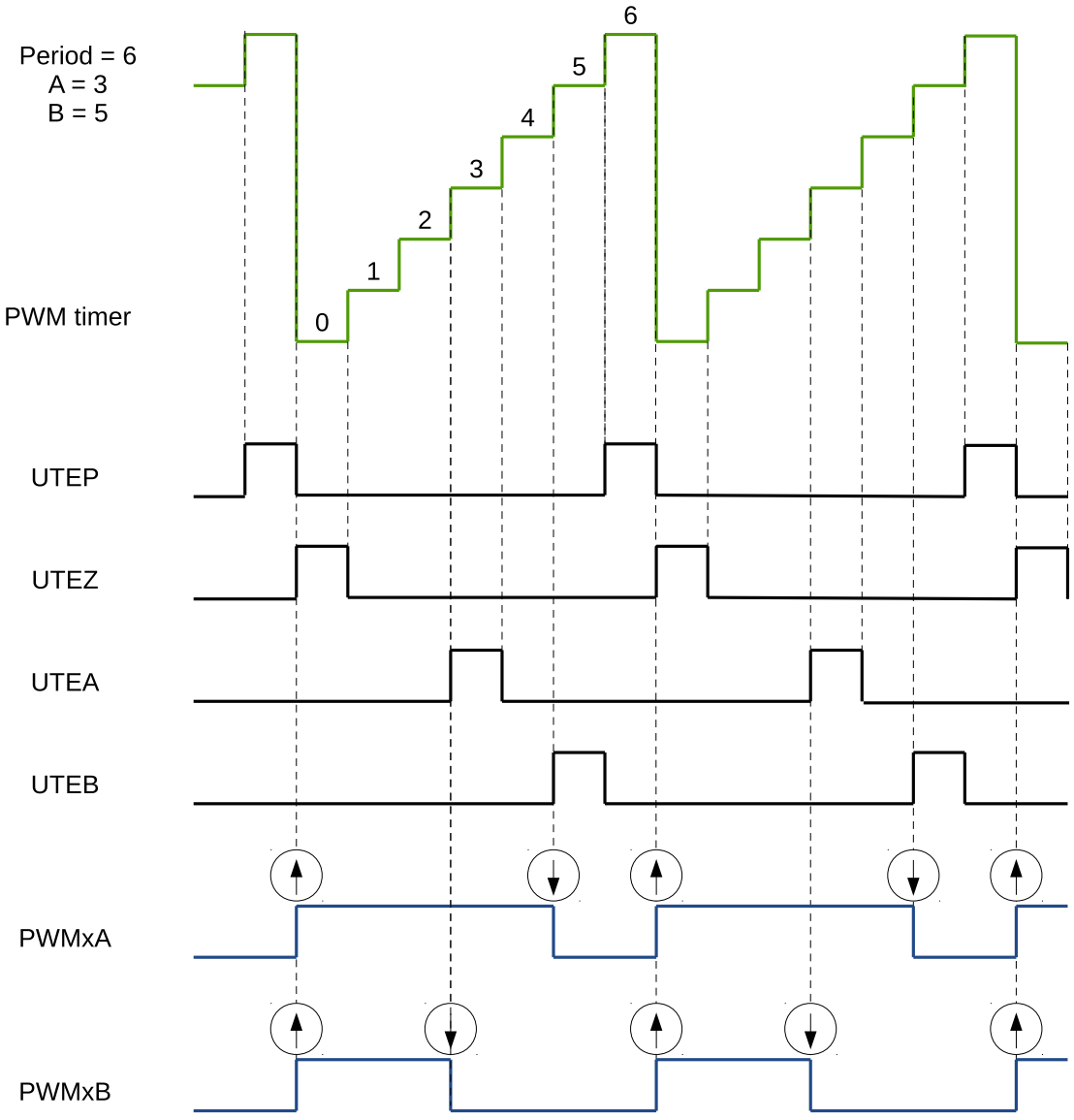


图 24.3-14. 递增计数模式，单边不对称波形，PWM*x*A 和 PWM*x*B 独立调制–高电平

PWM*x*A 的占空比调制由 B 设置，高电平有效，与 B 成正比。
PWM*x*B 的占空比调制由 A 设置，高电平有效，与 A 成正比。

$$Period = (MCPWM_TIMERn_PERIOD + 1) \times T_{PT_clk}$$

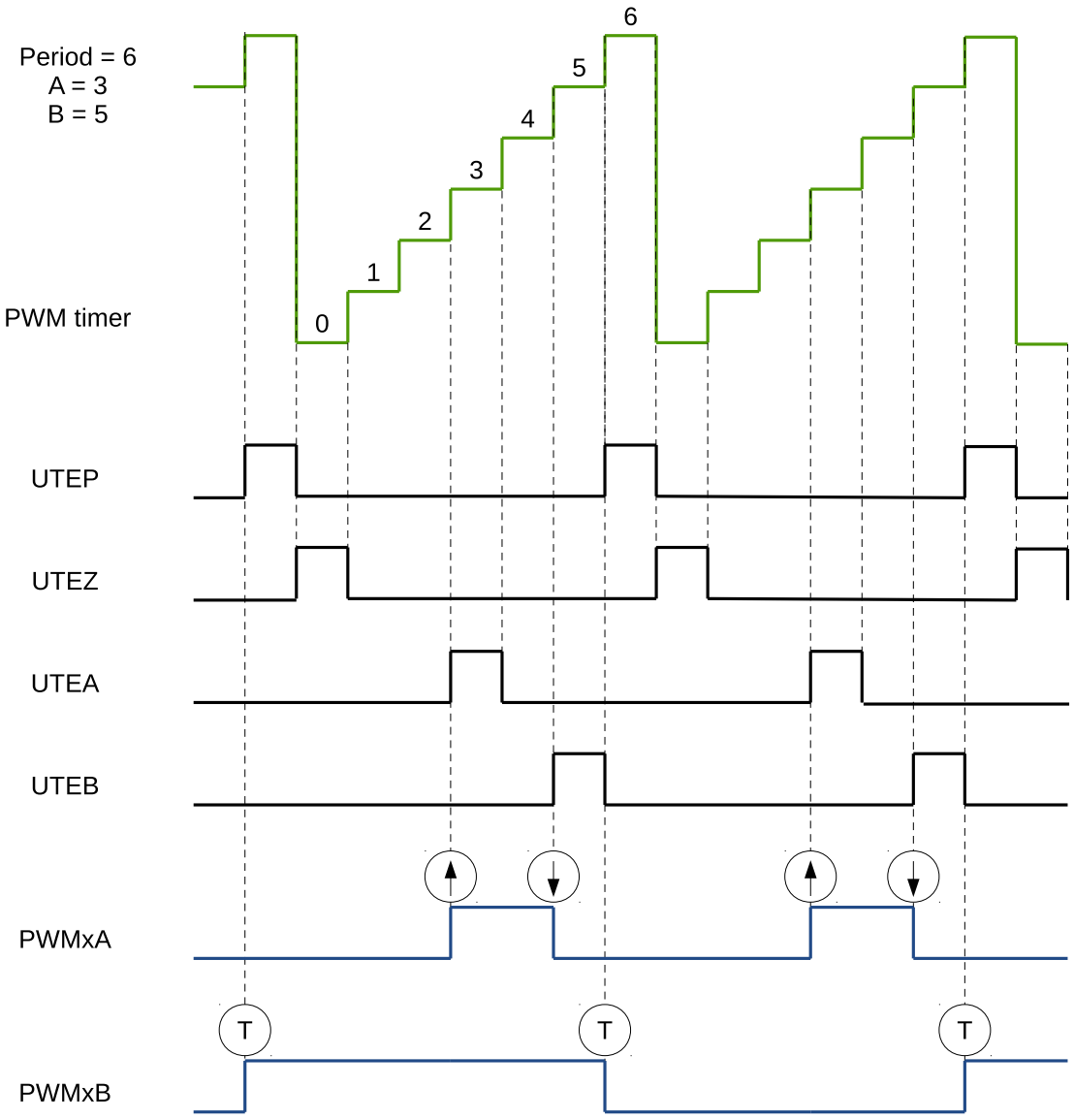


图 24.3-15. 递增计数模式，脉冲位置不对称波形，PWM*x*A 独立调制

脉冲可以在 PWM 波形内（零至周期值之间）的任何地方生成。

PWM*x*A 占空比与 (B – A) 成正比。

$$Period = (MCPWM_TIMERn_PERIOD + 1) \times T_{PT_clk}$$

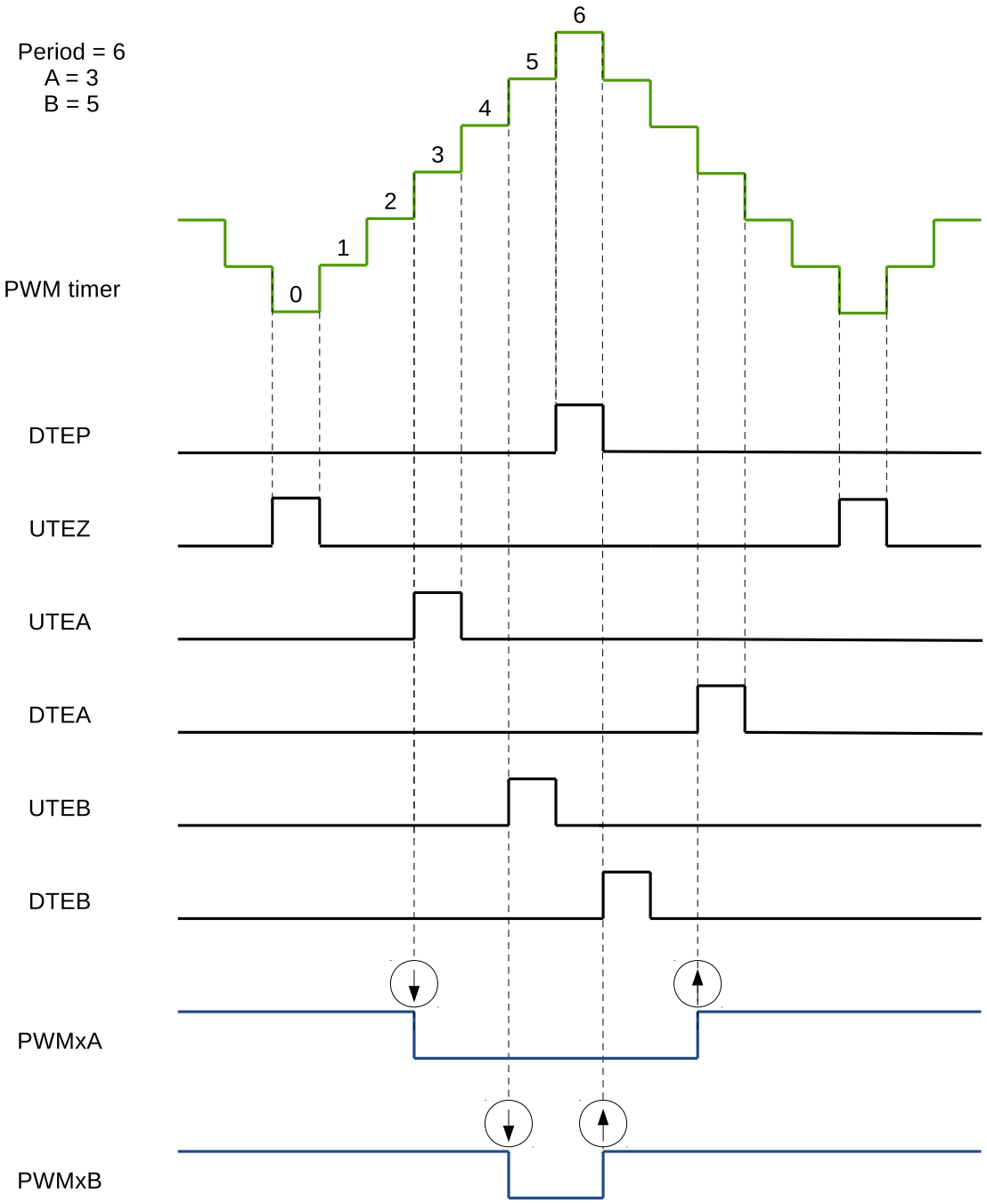


图 24.3-16. 递增递减循环计数模式，双沿对称波形，在 PWM*x*A 和 PWM*x*B 上独立调制–高电平有效

PWM*x*A 的占空比调制由 A 设置，高电平有效，与 A 成正比。
PWM*x*B 的占空比调制由 B 设置，高电平有效，与 B 成正比。
输出 PWM*x*A 和 PWM*x*B 可驱动不同开关。

$$Period = (2 \times MCPWM_TIMERn_PERIOD) \times T_{PT_clk}$$

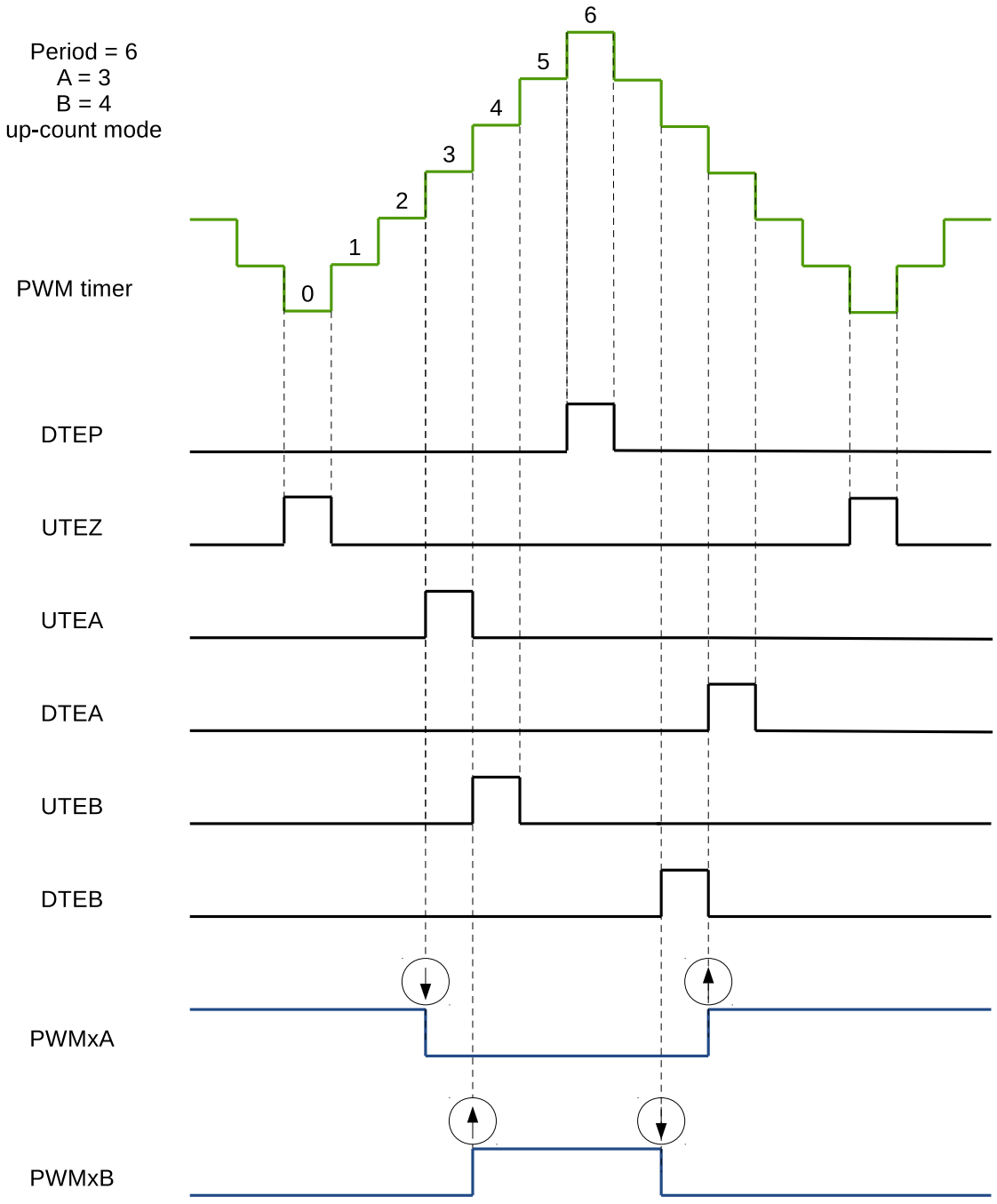


图 24.3-17. 递增递减循环计数模式，双沿对称波形，在 PWM*x*A 和 PWM*x*B 上独立调制–互补

PWM*x*A 的占空比调制由 A 设置，高电平有效，与 A 成正比。
PWM*x*B 的占空比调制由 B 设置，高电平有效，与 B 成正比。
PWM*x*A/B 输出可驱动上/下（互补）开关。
死区 = B – A，边沿位置完全可由软件配置。必要时，可使用死区生成器模块设置其他边沿延迟方式。

$$Period = (2 \times MCPWM_TIMERn_PERIOD) \times T_{PT_clk}$$

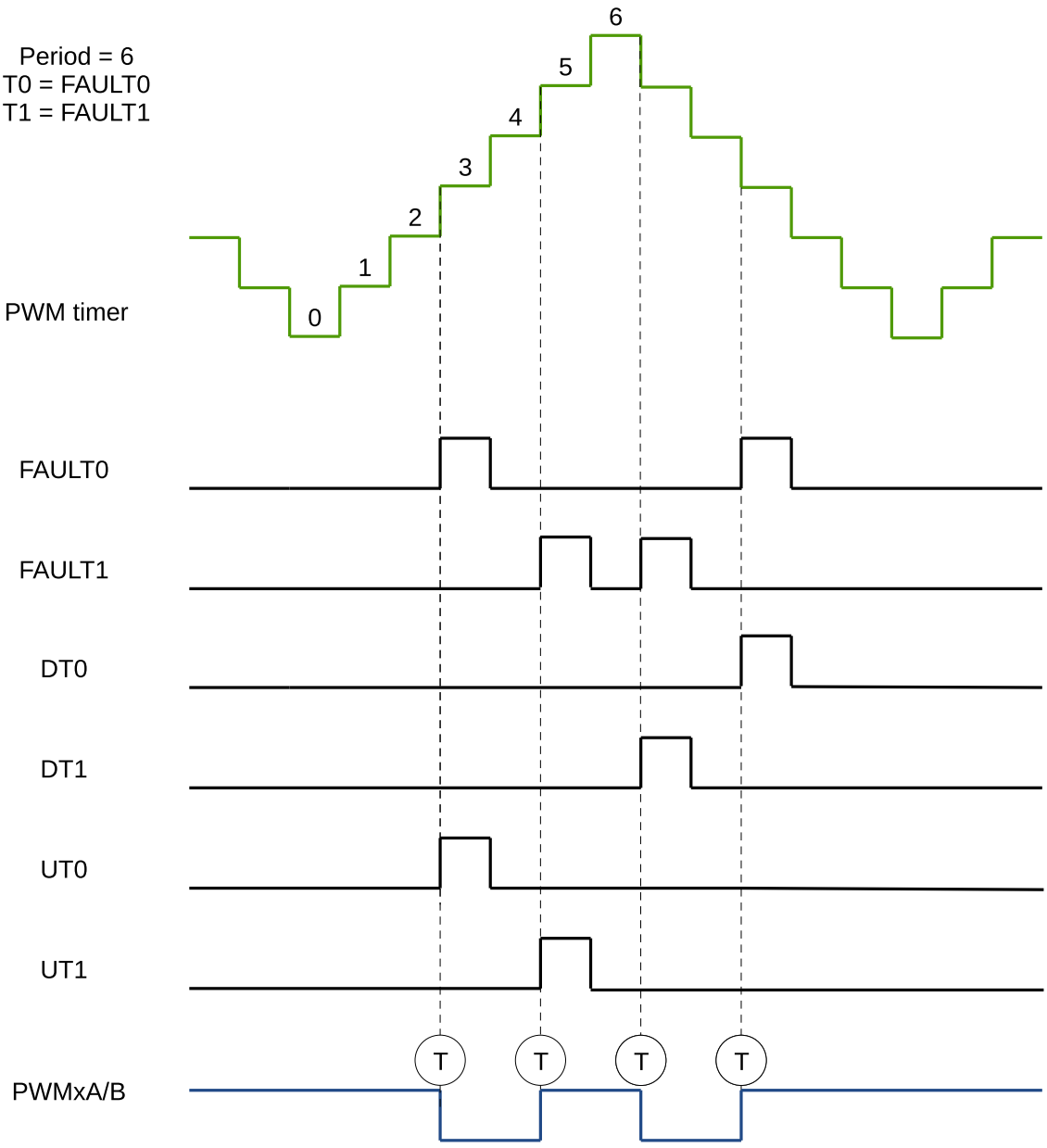


图 24.3-18. 递增递减循环计数模式，错误或同步事件，在 PWM*x*A 和 PWM*x*B 上相同调制

图 24.3-18 为当 UTO/1 和 DTO/1 事件产生时的波形。在本例中，TO 选择 FAULTO ，T1 选择 FAULT1。TO 和 T1 所选择的事件可以独立配置，这些事件可以是 FAULTO、FAULT1、FAULT2 或同步事件。具体配置方法见章节 24.3.3.1。

软件强制事件

在 PWM 生成器内有 2 种软件强制事件：

- 非连续即时 (NCI) 软件强制事件：当由软件触发时，这类事件在 PWM 输出上立即生效。强制是不连续的，意味着下一个激活的定时事件能够改变 PWM 输出。
- 连续 (CNTU) 软件强制事件：这类事件是连续的。强制 PWM 将持续输出，直到通过软件释放。事件触发器可配置。这类事件可配置为定时或即时发生。

对于 NCI 软件强制事件，向寄存器 MCPWM_GEN*n*_A_NCIFORCE 或 MCPWM_GEN*n*_B_NCIFORCE 写 1 再写

0，触发 PWM*x*A 或 PWM*x*B 的 NCI 软件强制事件。配置寄存器 MCPWM_GEN*n*_A_NCIFORCE_MODE 或 MCPWM_GEN*n*_B_NCIFORCE_MODE 指定当 NCI 软件强制事件发生时，对 PWM*x*A 或 PWM*x*B 波形的操作，配置为 0 或 3 表示无操作，配置为 1 表示置为低电平，配置为 2 表示置为高电平。

对于 CNTU 软件强制事件，配置寄存器 MCPWM_GEN*n*_CNTUFORCE_UPMETHOD，指定 CNTU 软件强制事件的触发方式，所有 bit 配置为 0，表示立即触发 CNTU 软件强制事件，bit 0 配置为 1，表示 TEZ 事件触发 CNTU 软件强制事件，bit 1 配置为 1，表示 TEP 事件触发 CNTU 软件强制事件，bit 2 配置为 1，表示 TEA 事件触发 CNTU 软件强制事件，bit 3 配置为 1，表示 TEB 事件触发 CNTU 软件强制事件，bit 4 配置为 1，表示 PWM 生成器所选的 PWM 定时器同步时触发 CNTU 软件强制事件，bit 5 配置为 1，表示不触发 CNTU 软件强制事件。配置寄存器 MCPWM_GEN*n*_A_CNTUFORCE_MODE 或 MCPWM_GEN*n*_B_CNTUFORCE_MODE 指定当 CNTU 软件强制事件发生时，对 PWM*x*A 或 PWM*x*B 波形的操作，配置为 0 或 3 表示无操作，配置为 1 表示置为低电平，配置为 2 表示置为高电平。

图 24.3-19 为 NCI 软件强制事件的一种波形。NCI 用于单独强制 PWM*x*A 输出为低电平，PWM*x*B 不受强制事件控制。

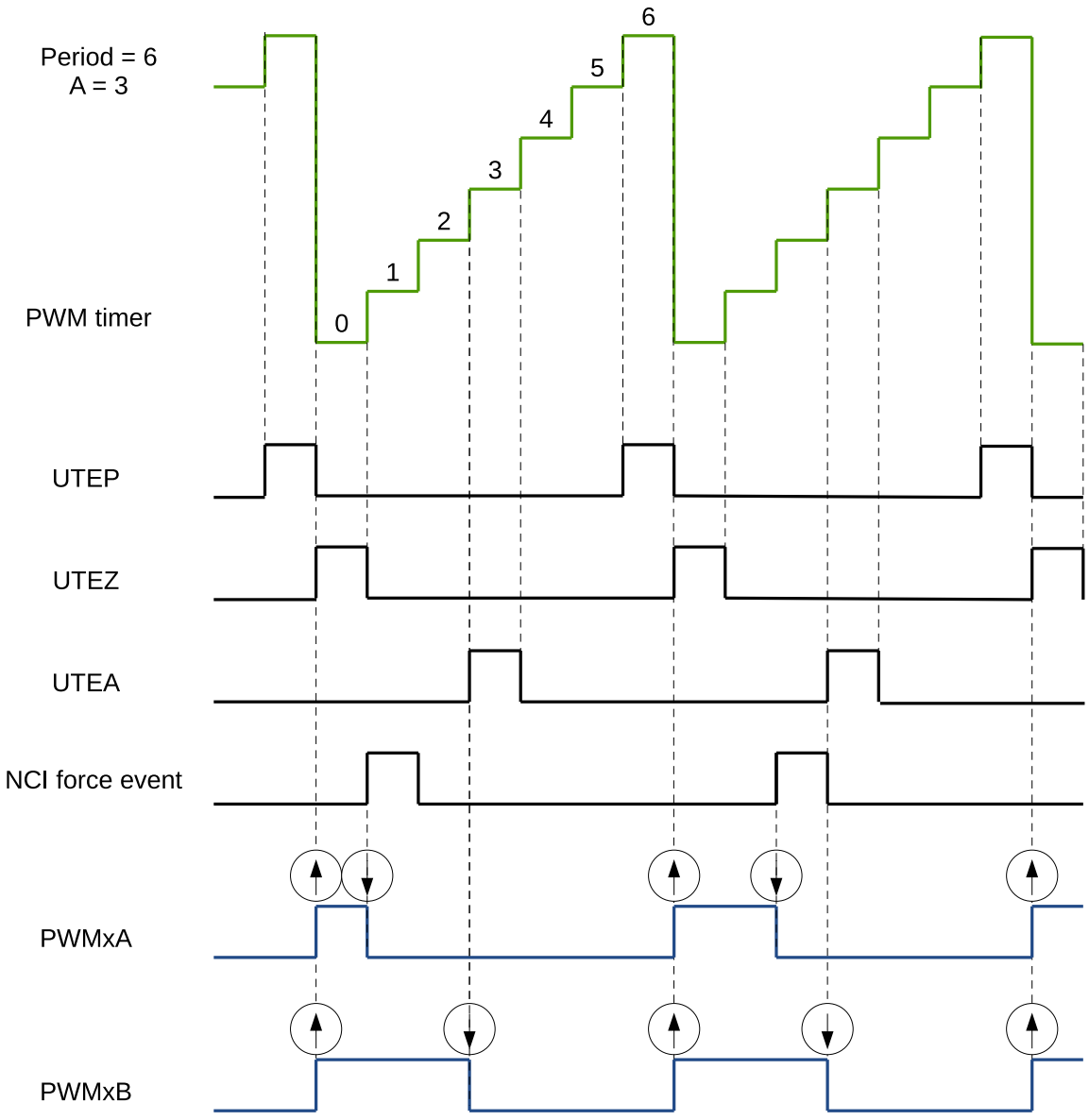


图 24.3-19. NCI 在 PWM*x*A 输出上软件强制事件示例

图 24.3-20 为 CNTU 软件强制事件的波形。TEZ 事件被选为 CNTU 软件强制事件的触发器。CNTU 用于单独强制 PWM*x*B 输出为低电平，但 PWM*x*A 不受强制。

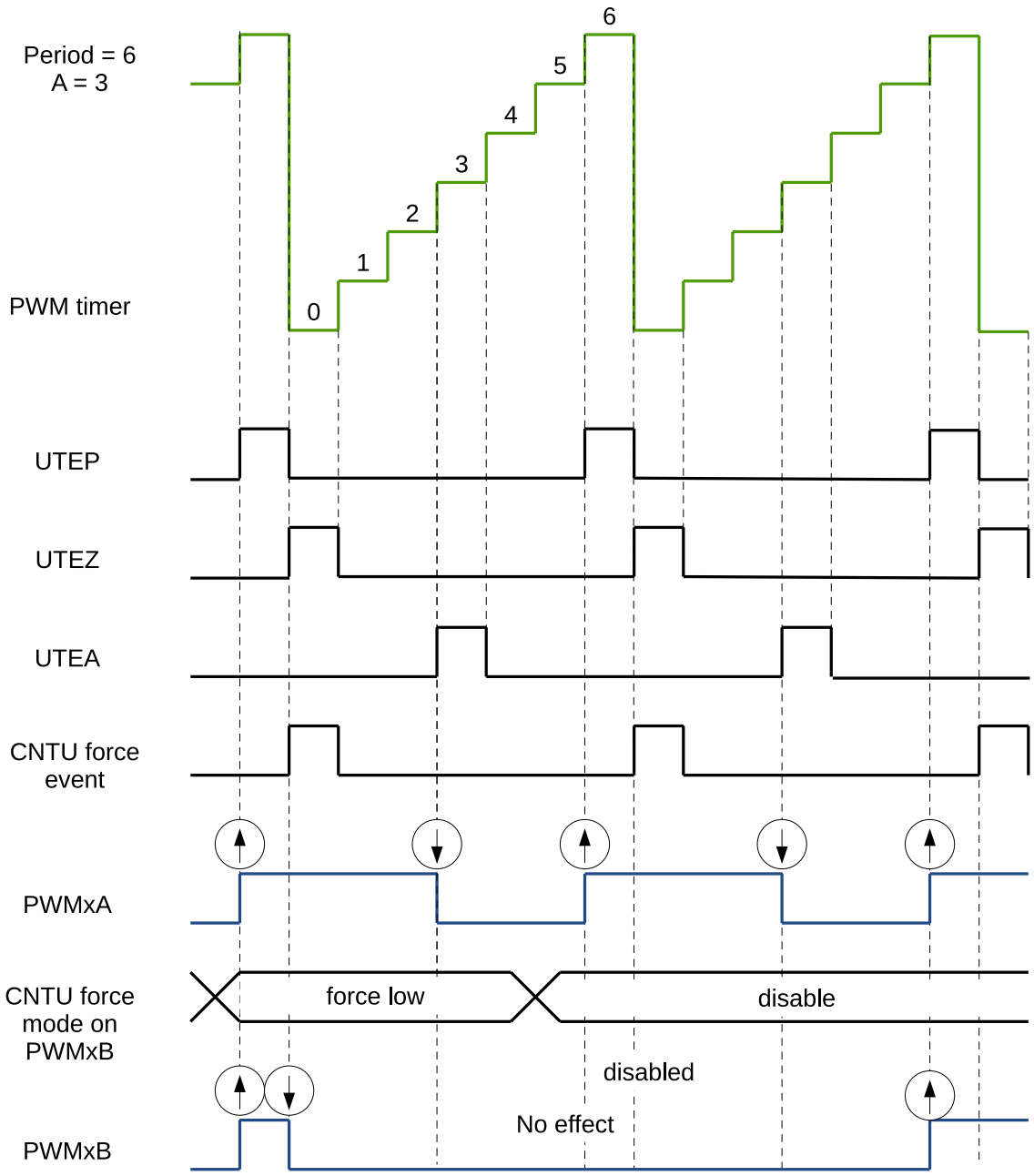


图 24.3-20. CNTU 在 PWM*x*B 输出上软件强制事件示例

24.3.3.2 死区生成器模块

死区生成器模块作用

章节 24.3.3.1 介绍了在 PWM*x*A 和 PWM*x*B 输出上生成信号的几种方式/事件，包括信号边沿的特定位置。通过改变信号之间的边沿位置以及设置信号的占空比，可获得所需的死区。此外，也可以使用专门的死区生成器模块控制死区。

死区生成器模块的主要功能如下：

- 可以将输入的 PWM*x*A 或 PWM*x*B 信号作为公共源来生成具有死区的 PWM*x*A 和 PWM*x*B 信号对输出
- 通过在信号边沿增加延迟来生成死区：
  - 上升沿延迟 (RED)
  - 下降沿延迟 (FED)
- 可以生成多种 PWM 波形，经典的死区配置为：
  - 高电平有效互补 (AHC)
  - 低电平有效互补 (ALC)
  - 高电平有效 (AH)
  - 低电平有效 (AL)
- 如果死区已经在生成器中配置，死区发生器也可以不进行配置。

死区模块生成器影子寄存器

延迟寄存器 RED 和 FED 的影子寄存器为 MCPWM_DT*n*_RED_CFG_REG 和 MCPWM_DT*n*_FED_CFG_REG。
MCPWM_GLOBAL_UP_EN 置 1 时，影子寄存器中的值可在某个特定时间写入有效寄存器中。
MCPWM_DT*n*_RED_CFG_REG 的更新方式寄存器为 MCPWM_DB*n*_RED_UPMETHOD；
MCPWM_DT*n*_FED_CFG_REG 的更新方式寄存器为 MCPWM_DB*n*_FED_UPMETHOD。软件也可以触发全局强制更新位 MCPWM_GLOBAL_FORCE_UP，该位将触发模块中的所有寄存器根据影子寄存器进行更新。寄存器描述详见章节 24.3.2.3。

死区生成器模块的操作要点

图 24.3-21 描述了创建死区模块的开关拓扑。

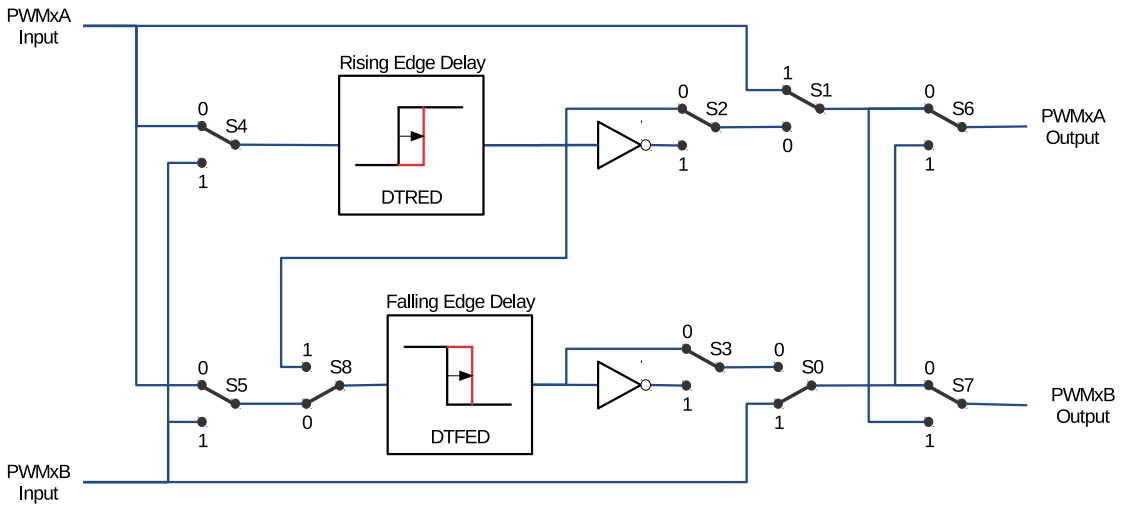


图 24.3-21. 死区模块的开关拓扑

上图中的 S0 - S8 开关由 MCPWM_DT*n*_CFG_REG 寄存器中的字段控制，详细信息见表 24.3-5。

表 24.3-5. 控制死区时间生成器开关的字段

开关	字段
S0	MCPWM_DB*n*_B_OUTBYPASS
S1	MCPWM_DB*n*_A_OUTBYPASS
S2	MCPWM_DB*n*_RED_OUTINVERT
S3	MCPWM_DB*n*_FED_OUTINVERT
S4	MCPWM_DB*n*_RED_INSEL
S5	MCPWM_DB*n*_FED_INSEL
S6	MCPWM_DB*n*_A_OUTSWAP
S7	MCPWM_DB*n*_B_OUTSWAP
S8	MCPWM_DB*n*_DEB_MODE

所有开关组合皆受支持，但不是所有的开关模式都是典型的使用模式。表 24.3-6 列举了一些典型的死区配置。在这些配置中，S4 和 S5 的开关位置将 PWM*x*A 设置为下降沿和上升沿延迟的公共源。表 24.3-6 中的模式可分为以下几类：

- **模式 1：绕过下降沿 (FED) 和上升沿 (RED) 的延迟**
  该模式下，死区模块被关闭。PWM*x*A 和 PWM*x*B 信号的波形无变化。
- **模式 2-5：经典死区极性设置**
  这些模式为典型极性配置，涵盖工业电源栅极驱动器中的高 / 低电平有效模式。图 24.3-22 至 24.3-25为典型波形。
- **模式 6 和 7：绕过下降沿 (FED) 或上升沿 (RED) 的延迟**
  这两种模式下，绕过上升沿延迟 (RED) 或下降沿延迟 (FED) 。因此，无需使用对应延迟。

表 24.3-6. 死区生成器的典型操作模式

模式	描述	S0	S1	S2	S3
1	PWM*x*A 和 PWM*x*B 波形无变化	1	1	X	X
2	高电平有效互补 (AHC)，参见图 24.3-22	0	0	0	1
3	低电平有效互补 (ALC)，参见图 24.3-23	0	0	1	0
4	高电平有效 (AH)，参见图 24.3-24	0	0	0	0
5	低电平有效 (AL)，参见图 24.3-25	0	0	1	1
6	PWM*x*A 输出 = PWM*x*A 输入（无延迟） PWM*x*B 输出 = PWM*x*A 输入，下降沿延迟	0	1	0 或 1	0 或 1
7	PWM*x*A 输出 = PWM*x*A 输入，上升沿延迟 PWM*x*B 输出 = PWM*x*B 输入（无延迟）	1	0	0 或 1	0 或 1

说明：

以上所有模式中，S4 - S8 的开关位置都置 0。

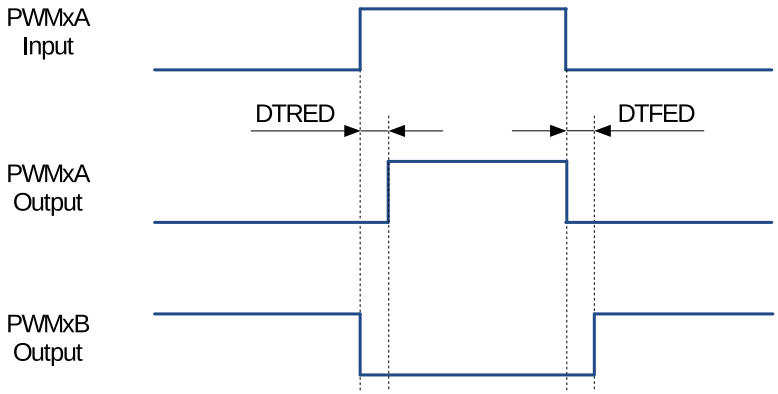


图 24.3-22. 高电平有效互补 (AHC) 死区波形

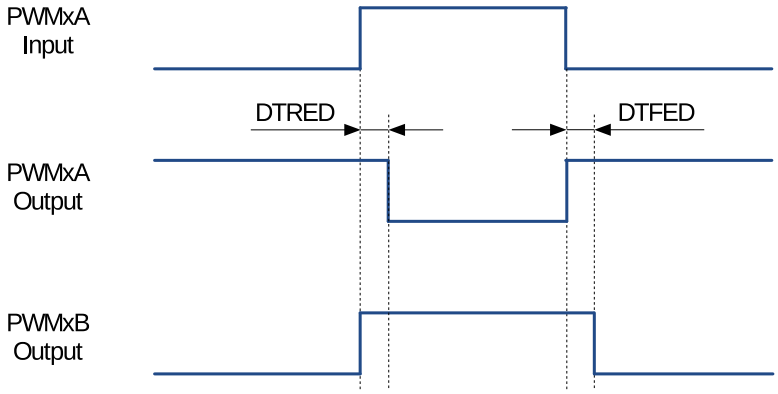


图 24.3-23. 低电平有效互补 (ALC) 死区波形

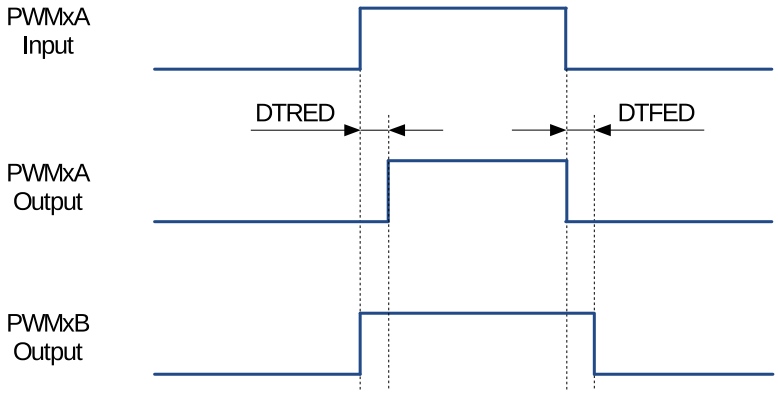


图 24.3-24. 高电平有效 (AH) 死区波形

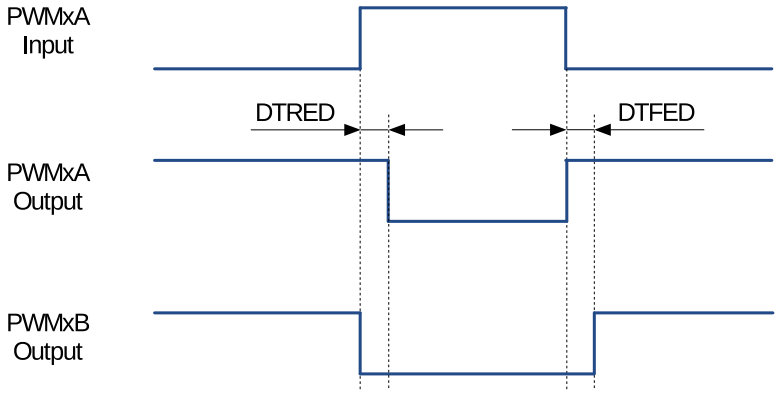


图 24.3-25. 低电平有效 (AL) 死区波形

上升沿延迟 (RED) 和下降沿延迟 (FED) 可分别设置。延迟的值通过 16 位寄存器 MCPWM_DT*x*_*RED* 和 MCPWM_DT*x*_*FED* 配置。寄存器值表示一个信号边沿可以延迟的 DT_clk 时钟周期值。DT_clk 可通过寄存器 MCPWM_DB*n*_CLK_SEL 从 PWM_clk 或 PT_clk 中选择。

通过以下公式计算下降沿延迟 (FED) 和上升沿延迟 (RED) 的值:

$$FED = MCPWM_DTn_FED \times T_{DT_clk}$$

$$RED = MCPWM_DTn_RED \times T_{DT_clk}$$

24.3.3.3 PWM 载波模块

将 PWM 输出耦合到电机驱动器可能需要使用变压器隔离。变压器只提供交流信号，而 PWM 信号的占空比可能在 0% 到 100% 之间变化。PWM 载波模块可以通过使用高频载波对其进行调制，将该信号传递给变压器。

功能概述

此模块的以下关键功能可配置:

- 载波频率
- 第一个脉冲的脉宽
- 第二个以及之后的脉冲的占空比
- 开启/关闭载波

操作要点

PWM 载波时钟 (PC_clk) 来自于 PWM_CLK。通过寄存器 MCPWM_CARRIER*n*_CFG_REG 的 MCPWM_CHOPPER*n*_PRESCALE 和 MCPWM_CHOPPER*n*_DUTY 位配置频率和占空比。一次性脉冲的功能在于提供高能量脉冲以接通电源开关。随后的脉冲用于保持上电的状态。一次性脉冲宽度可通过 MCPWM_CHOPPER*n*_OSHTWTH 位进行配置。通过 MCPWM_CHOPPER*n*_EN 位来使能/禁用载波模块。

载波示例

图 24.3-26 描述了载波叠加在原始 PWM 脉冲上的示例波形。图中未显示第一个脉冲和占空比控制，相关详细信息将在后两节中介绍。

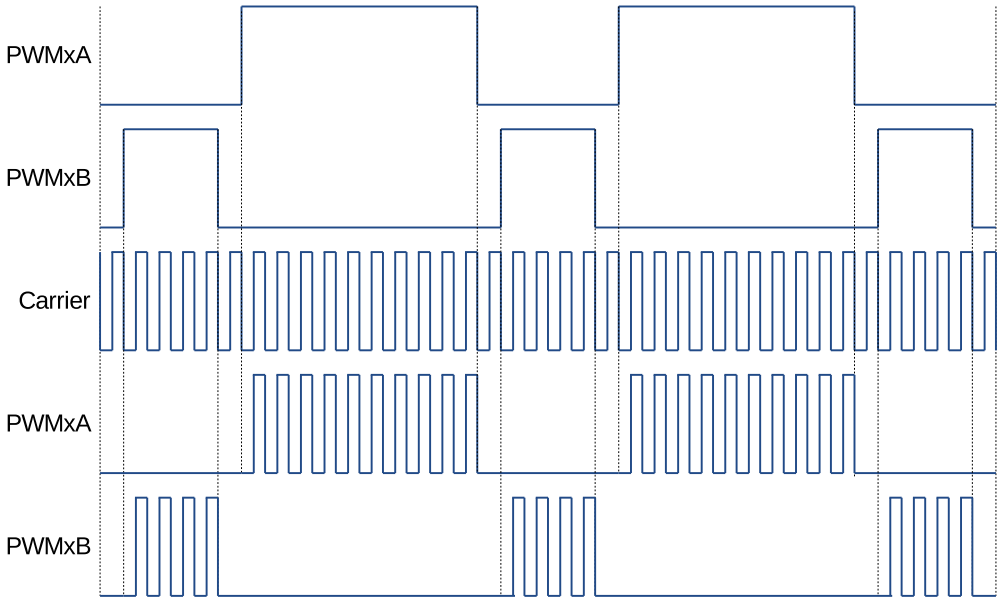


图 24.3-26. PWM 载波操作的波形示例

第一个脉冲

第一个脉冲的宽度可配置，其值有 16 种可能，可通过下公式计算：

$$T_{1stpulse} = T_{PWM_clk} \times 8 \times (MCPWM_CARRIERn_PRESCALE + 1) \times (MCPWM_CARRIERn_OSHTWTH + 1)$$

其中：

- T_{PMW_clk} 为 PWM 时钟周期 (PWM_clk)
- $(MCPWM_CARRIERn_OSHTWTH + 1)$ 为一次性脉冲宽度值（取值范围：1-16）
- $(MCPWM_CARRIERn_PRESCALE + 1)$ PWM 载波时钟 (PC_clk) 预分频值

图 24.3-27 展示了第一个脉冲和之后持续的脉冲。

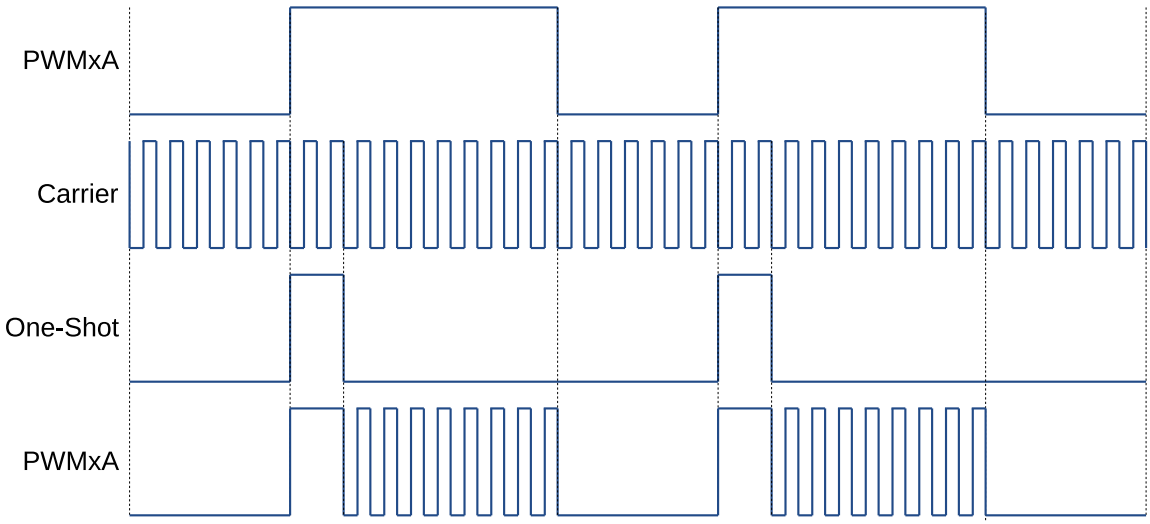


图 24.3-27. 载波模块的第一个脉冲和之后持续的脉冲示例

占空比控制

在发出第一个一次性脉冲之后，根据载波频率调制剩余的 PWM 信号。用户可配置该信号的占空比。在一定情况下，调整占空比可使信号通过隔离变压器后仍然可以开启或关闭电动机驱动器，改变电机旋转速度和方向。

占空比通过 MCPWM_CHOPPER*n*_DUTY 或寄存器 MCPWM_CARRIER*n*_CFG_REG 的 [7:5] 位设置，其值有 7 种可能性。

占空比的值可通过以下方式计算：

$$Duty = MCPWM_CARRIERn_DUTY \div 8$$

图 24.3-28 为所有 7 种占空比设置。

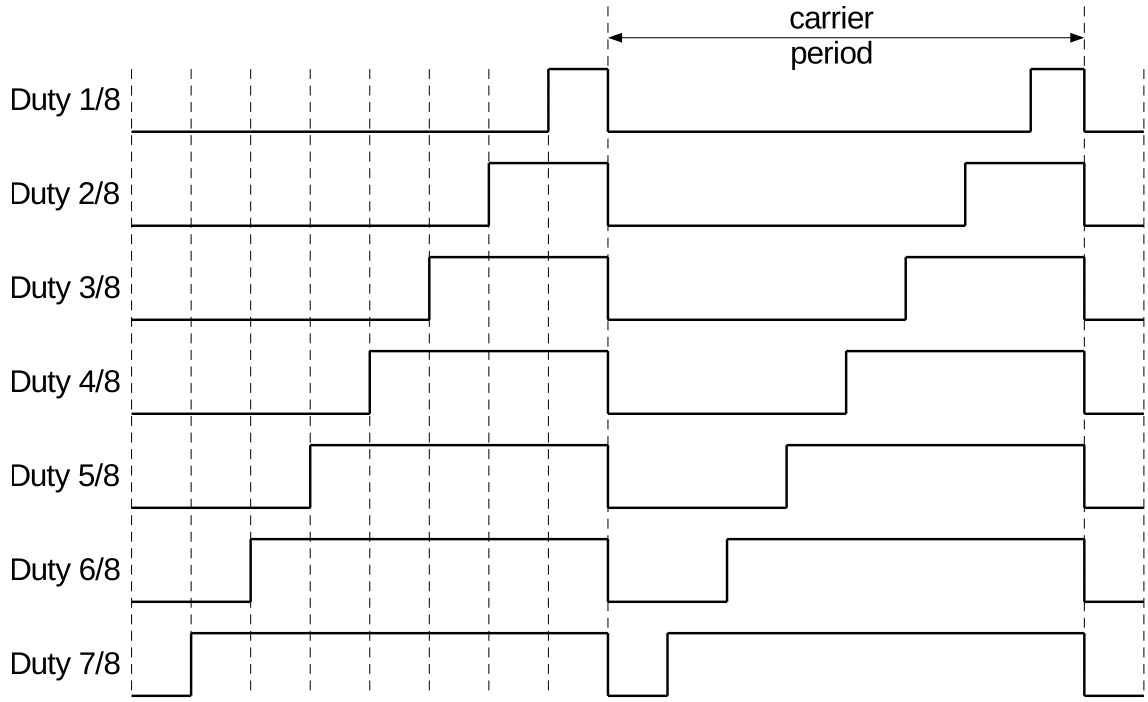


图 24.3-28. PWM 载波模块中持续脉冲的 7 种占空比设置

24.3.3.4 故障检测模块

每个 MCPWM 外设都连接来自 GPIO 矩阵的 3 个故障信号（FAULT0、FAULT1 和 FAULT2）。这些信号用于指示外部故障状况，并且可由故障检测模块预处理后生成故障事件。故障事件通过执行用户代码，针对特定故障调整 MCPWM 输出。

故障检测模块功能

故障检测模块的主要功能为：

- 在检测到故障时强制 PWM*x*A 和 PWM*x*B 输出信号进入以下状态之一：
  - 高
  - 低
  - 取反
  - 无
- 在检测到过电流/短路时执行一次性跳闸（OST）。
- 逐周期跳闸（CBC）以提供限流操作。
- 每个故障信号单独分配一次性或逐周期操作。
- 每个故障输入都生成中断。
- 支持软件强制跳闸。
- 根据需要开启或关闭模块功能。

操作与配置要点

本节介绍故障处理模块的操作要点和配置选项。

来自管脚的故障信号在 GPIO 矩阵中采样和同步。为了保证故障脉冲成功采样，每个脉冲持续时间必须至少为 2 个 APB 时钟周期。故障检测模块使用 PWM_clk 对故障信号进行采样，因此，来自 GPIO 矩阵的故障脉冲持续时间必须至少为 1 个 PWM_clk 周期。换句话说，无论 APB 时钟周期和 PWM_clk 周期的大小关系如何，管脚上的故障信号脉冲的宽度必须至少等于两个 APB 时钟周期与一个 PWM_clk 周期的和。

故障处理器模块可以使用故障信号 FAULT0 至 FAULT2 中的高电平或低电平来生成故障事件 fault_event0 至 fault_event2。每个故障事件可以单独配置为进行 CBC 操作、OST 操作或无操作。

- **逐周期 (CBC) 操作：**
  触发 CBC 操作时，PWM*x*A 和 PWM*x*B 的状态立即根据字段 MCPWM_TZ*n*_A_CBC_U/D 和 MCPWM_TZ*n*_B_CBC_U/D 改变。PWM 定时器递增或递减计数时，可配置不同的操作。不同的故障事件可触发不同的 CBC 操作中断。通过状态字段 MCPWM_TZ*n*_CBC_ON 开启或关闭 CBC 操作。在没有故障事件时，将在指定时间点，即发生 D/UTEP 或 D/UTEZ 事件时，清除 PWM*x*A/B 上的 CBC 操作。由字段 MCPWM_TZ*n*_CBCPULSE 决定 PWM*x*A 和 PWM*x*B 恢复正常的事件。因此，在此模式下，每个 PWM 循环后，CBC 操作都将清除或刷新。
- **一次性（OST）操作：**
  触发 OST 操作时，PWM*x*A 和 PWM*x*B 的状态立即根据字段 MCPWM_TZ*n*_A_OST_U/D 和 MCPWM_TZ*n*_B_OST_U/D 改变。PWM 定时器递增或递减计数时，可配置不同的操作。不同的故障事件可触发不同的 OST 操作中断。通过状态字段 MCPWM_TZ*n*_OST_ON 开启或关闭 OST 操作。在没有故障事件时，PWM*x*A/B 上的 OST 操作无法自动清除。须通过将 MCPWM_TZ*n*_CLR_OST 位置 1 来清除一次性操作。

24.3.4 捕获模块

24.3.4.1 介绍

捕获模块包含 3 个完整的捕获通道。通道输入信号 CAP0、CAP1 和 CAP2 来自于 GPIO 矩阵。由于 GPIO 矩阵的灵活性，CAP0、CAP1 和 CAP2 可以通过任一管脚输入配置。多个捕获通道可以来自同一管脚输入，每个通道的预分频可以分别设置。同时，每个捕获通道也可以来自不同的管脚输入。因此，可以通过后台硬件用多种方式处理捕获信号，而不直接由 CPU 处理。

捕捉模块有以下独立资源：

- 一个 32 位定时器（计数器），可与 PWM 定时器、另一个模块或软件同步。
- 三个捕获通道，每个通道配有一个 32 位时间戳和一个捕获预分频器。
- 任何捕获通道的边沿极性（上升/下降沿）可独立选择。
- 输入捕获信号预分频（分频取值范围：1 ~ 256）。
- 三个捕获事件都有中断功能。

24.3.4.2 捕获定时器

捕获定时器是一个 32 位计数器，使能时不断递增计数。将 MCPWM_CAP_TIMER_EN 置 1 使能捕获寄存器。其操作时钟源为 MCPWM 模块的主时钟。配置 MCPWM_CAP_SYNCI_EN 后，发生同步事件时，存储在

MCPWM_CAP_TIMER_PHASE_REG 寄存器中的相位将被加载至计数器中。同步事件可来自 PWM 定时器同步输出或 PWM 模块同步输入，通过 MCPWM_CAP_SYNCI_SEL 进行配置。也可将 MCPWM_CAP_SYNC_SW 置 1 生成同步事件。捕获定时器为所有三个捕获通道提供定时参考。

24.3.4.3 捕获通道

必要时，到达捕获通道的捕获信号可先被反相，然后预分频。每个捕获通道都有一个预分频寄存器 MCPWM_CAP*n*_PRESCALE。最后，预处理后的捕获信号的指定边沿将触发捕获事件。将 MCPWM_CAP*n*_EN 置 1 使能捕获通道。捕获事件将在 MCPWM_CAP*n*_MODE 配置的时间发生。在捕获事件发生时，捕获定时器的值存储在时间戳寄存器 MCPWM_CAP_CH*n*_REG 中。捕获事件中的不同捕获通道可生成不同的中断。触发捕获事件的边沿储存在寄存器 MCPWM_CAP*n*_EDGE 中。捕获事件可通过将 MCPWM_CAP*n*_SW 置 1 由软件强制发生。

24.3.5 ETM 模块

24.3.5.1 介绍

在 ESP32-S31 中，MCPWM 支持 ETM 功能，即可以通过任意外设的 ETM 事件触发 MCPWM 的 ETM 任务，或者通过 MCPWM 的 ETM 事件触发任意外设的 ETM 任务。MCPWM 的捕获模块、故障检测模块、三个定时器以及三个操作器都可以独立产生事件或响应任务。关于 ETM 更多详细信息，请参考章节 10 事件任务矩阵 (ETM) [to be added later]。这里仅介绍与 MCPWM 相关的 ETM 任务和 ETM 事件。

24.3.5.2 MCPWM 可产生的 ETM 事件

将使能字段置 1 后，满足对应生成条件时，将产生对应 ETM 事件，详见表 24.3-7：

表 24.3-7. MCPWM 可产生的 ETM 事件

使能字段	生成条件	产生事件
MCPWM_EVT_CAP*n*_EN	发生 CAP*n* 捕获事件	MCPWM*0*_EVT_CAP*n*
MCPWM_EVT_TZ*n*_OST_EN	PWM 操作器 *n* 执行一次性跳闸 (OST) 操作	MCPWM*0*_EVT_TZ*n*_OST
MCPWM_EVT_TZ*n*_CBC_EN	PWM 操作器 *n* 执行逐周期跳闸 (CBC) 操作	MCPWM*0*_EVT_TZ*n*_CBC
MCPWM_EVT_F*n*_CLR_EN	清除故障事件 fault_event*n*	MCPWM*0*_EVT_F*n*_CLR
MCPWM_EVT_F*n*_EN	产生故障事件 fault_event*n*	MCPWM*0*_EVT_F*n*
MCPWM_EVT_OP*n*_TEB_EN	PWM 操作器 *n* 选择的定时器计数值等于时间戳 B 的值	MCPWM*0*_EVT_OP*n*_TEB
MCPWM_EVT_OP*n*_TEA_EN	PWM 操作器 *n* 选择的定时器计数值等于时间戳 A 的值[1]	MCPWM*0*_EVT_OP*n*_TEA
MCPWM_EVT_OP*n*_TEE1_EN	PWM 操作器 *n* 选择的定时器计数值等于寄存器 MCPWM_OP*n*_TSTMP_E1_REG 的值	MCPWM*0*_EVT_OP*n*_TEE1
MCPWM_EVT_OP*n*_TEE2_EN	PWM 操作器 *n* 选择的定时器计数值等于寄存器 MCPWM_OP*n*_TSTMP_E2_REG 的值	MCPWM*0*_EVT_OP*n*_TEE2
MCPWM_EVT_TIMER*n*_TEP_EN	定时器 *n* 的计数值等于周期值 MCPWM_TIMER*n*_PERIOD	MCPWM*0*_EVT_TIMER*n*_TEP
MCPWM_EVT_TIMER*n*_TEZ_EN	定时器 *n* 的计数值等于 0	MCPWM*0*_EVT_TIMER*n*_TEZ
MCPWM_EVT_TIMER*n*_STOP_EN	定时器 *n* 停止计数	MCPWM*0*_EVT_TIMER*n*_STOP

[1] 有关时间戳 A 和时间戳 B 的描述，详见章节 24.3.3.1。

24.3.5.3 MCPWM 可接收的 ETM 任务

将使能字段置 1 后，接收对应有效任务后，将产生对应的响应操作，详见表 24.3-8：

表 24.3-8. MCPWM 可接收的 ETM 任务

使能字段	接收的有效任务	响应操作
MCPWM_TASK_CAP*n*_EN	MCPWM0_TASK_CAP*n*	CAP*n* 通道进行捕获操作
MCPWM_TASK_CLR*n*_OST_EN	MCPWM0_TASK_CLR*n*_OST	清除 PWM 操作器 *n* 的一次性跳闸 (OST) 操作
MCPWM_TASK_TZ*n*_OST_EN	MCPWM0_TASK_TZ*n*_OST	PWM 操作器 *n* 执行一次性跳闸 (OST) 操作
MCPWM_TASK_TIMER*n*_PERIOD_UP_EN	MCPWM0_TASK_TIMER*n*_PERIOD_UP	定时器 *n* 的周期更新为周期寄存器 MCPWM_TIMER*n*_PERIOD 配置的值
MCPWM_TASK_TIMER*n*_SYNC_EN	MCPWM0_TASK_TIMER*n*_SYN	定时器 *n* 执行一次同步操作
MCPWM_TASK_GEN_STOP_EN	MCPWM0_TASK_GEN_STOP	所有定时器停止计数，所有的 PWM 操作器输出的 PWM 信号保持不变
MCPWM_TASK_CMPR*n*_B_UP_EN	MCPWM0_TASK_CMPR*n*_B_UP	将 PWM 操作器 *n* 的时间戳 B 更新为时间戳 B 影子寄存器 MCPWM_GEN*n*_B 的值
MCPWM_TASK_CMPR*n*_A_UP_EN	MCPWM0_TASK_CMPR*n*_A_UP	将 PWM 操作器 *n* 的时间戳 A 更新为时间戳 A 影子寄存器 MCPWM_GEN*n*_A 的值

24.4 中断

ESP32-S31 的 MCPWM0 会分别生成 PWM0_INTR 中断信号，并发送给中断矩阵。

PWM0_INTR 中断信号由以下内部中断源生成：

- MCPWM_TIMER*x*_STOP_INT：定时器*x* 停止后触发。
- MCPWM_TIMER*x*_TEZ_INT：由 PWM 定时器*x* TEZ 事件触发。
- MCPWM_TIMER*x*_TEP_INT：由 PWM 定时器*x* TEP 事件触发。
- MCPWM_FAULT*x*_INT：fault_event*x* 开始后触发。
- MCPWM_FAULT*x*_CLR_INT：fault_event*x* 结束后触发。
- MCPWM_CMPR*x*_TEA_INT：由 PWM 操作器*x* TEA 事件触发。
- MCPWM_CMPR*x*_TEB_INT：由 PWM 操作器*x* TEB 事件触发。
- MCPWM_TZ*x*_CBC_INT：由 PWM*x* 逐周期操作触发。
- MCPWM_TZ*x*_OST_INT：由 PWM*x* 一次性操作触发。
- MCPWM_CAP*x*_INT：由信道*x* 上捕获事件触发。

上述中断源仅在对应的中断寄存器使能字段 *interrupt_source_name*_ENA 置 1 时才能触发，将 *interrupt_source_name*_CLR 字段置 1 即清除对应中断。

说明：
中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 24.5 寄存器列表。

24.5 寄存器列表

本小节的所有地址均为相对于电机控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
预分频器配置			
MCPWM_CLK_CFG_REG	配置预分频器	0x0000	R/W
MCPWM 定时器 0 配置与状态			
MCPWM_TIMER0_CFG0_REG	定时器周期与更新方法	0x0004	R/W
MCPWM_TIMER0_CFG1_REG	工作模式与开始/停止控制	0x0008	varies
MCPWM_TIMER0_SYNC_REG	PWM 定时器 0 同步功能配置寄存器	0x000C	R/W
MCPWM_TIMER0_STATUS_REG	PWM 定时器 0 状态	0x0010	RO
MCPWM 定时器 1 配置与状态			
MCPWM_TIMER1_CFG0_REG	定时器更新方式与周期	0x0014	R/W
MCPWM_TIMER1_CFG1_REG	工作模式与开始/停止控制	0x0018	varies
MCPWM_TIMER1_SYNC_REG	同步设置	0x001C	R/W
MCPWM_TIMER1_STATUS_REG	PWM 定时器 1 状态	0x0020	RO
MCPWM 定时器 2 配置与状态			
MCPWM_TIMER2_CFG0_REG	定时器方式与周期	0x0024	R/W
MCPWM_TIMER2_CFG1_REG	工作模式与开始/停止控制	0x0028	varies
MCPWM_TIMER2_SYNC_REG	同步设置	0x002C	R/W
MCPWM_TIMER2_STATUS_REG	PWM 定时器 2 状态	0x0030	RO
MCPWM 定时器常见配置			
MCPWM_TIMER_SYNCI_CFG_REG	定时器同步输入选择	0x0034	R/W
MCPWM_OPERATOR_TIMERSEL_REG	为 PWM 操作器选择特定的计时器	0x0038	R/W
MCPWM 操作器 0 配置与状态			
MCPWM_GEN0_STMP_CFG_REG	时间戳寄存器 A 和 B 的传输状态和更新方式	0x003C	varies
MCPWM_GEN0_TSTMP_A_REG	寄存器 A 的影子寄存器	0x0040	R/W
MCPWM_GEN0_TSTMP_B_REG	寄存器 B 的影子寄存器	0x0044	R/W
MCPWM_GEN0_CFG0_REG	故障事件 T0 和 T1 处理	0x0048	R/W
MCPWM_GEN0_FORCE_REG	软件强制 PWM0A 和 PWM0B 输出	0x004C	R/W
MCPWM_GEN0_A_REG	PWM0A 输出上事件触发的操作	0x0050	R/W
MCPWM_GEN0_B_REG	PWM0B 输出上事件触发的操作	0x0054	R/W
MCPWM_DT0_CFG_REG	死区配置	0x0058	R/W
MCPWM_DT0_FED_CFG_REG	FED 的影子寄存器	0x005C	R/W
MCPWM_DT0_RED_CFG_REG	RED 的影子寄存器	0x0060	R/W
MCPWM_CARRIER0_CFG_REG	载波使能与配置	0x0064	R/W
MCPWM_FH0_CFG0_REG	故障事件中 PWM0A 和 PWM0B 上的操作	0x0068	R/W
MCPWM_FH0_CFG1_REG	故障处理的软件触发	0x006C	R/W
MCPWM_FH0_STATUS_REG	故障事件状态	0x0070	RO
MCPWM 操作器 1 配置与状态			
MCPWM_GEN1_STMP_CFG_REG	时间戳寄存器 A 和 B 的传输状态和更新方式	0x0074	varies

名称	描述	地址	访问
MCPWM_GEN1_TSTMP_A_REG	寄存器 A 的影子寄存器	0x0078	R/W
MCPWM_GEN1_TSTMP_B_REG	寄存器 B 的影子寄存器	0x007C	R/W
MCPWM_GEN1_CFG0_REG	故障事件 T0 和 T1 处理	0x0080	R/W
MCPWM_GEN1_FORCE_REG	软件强制 PWM1A 和 PWM1B 输出	0x0084	R/W
MCPWM_GEN1_A_REG	PWM1A 输出上的事件触发的操作	0x0088	R/W
MCPWM_GEN1_B_REG	PWM1B 输出上的事件触发的操作	0x008C	R/W
MCPWM_DT1_CFG_REG	死区配置	0x0090	R/W
MCPWM_DT1_FED_CFG_REG	FED 的影子寄存器	0x0094	R/W
MCPWM_DT1_RED_CFG_REG	RED 的影子寄存器	0x0098	R/W
MCPWM_CARRIER1_CFG_REG	使能与配置载波	0x009C	R/W
MCPWM_FH1_CFG0_REG	故障事件中 PWM1A 和 PWM1B 输出上的操作	0x00A0	R/W
MCPWM_FH1_CFG1_REG	故障处理的软件触发	0x00A4	R/W
MCPWM_FH1_STATUS_REG	故障事件状态	0x00A8	RO
MCPWM 操作器 2 的配置与状态			
MCPWM_GEN2_STMP_CFG_REG	时间戳寄存器 A 和 B 的传输状态和更新方式	0x00AC	varies
MCPWM_GEN2_TSTMP_A_REG	寄存器 A 的影子寄存器	0x00B0	R/W
MCPWM_GEN2_TSTMP_B_REG	寄存器 B 的影子寄存器	0x00B4	R/W
MCPWM_GEN2_CFG0_REG	故障事件 T0 和 T1 处理	0x00B8	R/W
MCPWM_GEN2_FORCE_REG	软件强制 PWM2A 和 PWM2B 输出	0x00BC	R/W
MCPWM_GEN2_A_REG	PWM2A 输出上的事件触发的操作	0x00C0	R/W
MCPWM_GEN2_B_REG	PWM2B 输出上的事件触发的操作	0x00C4	R/W
MCPWM_DT2_CFG_REG	死区类型的选择与配置	0x00C8	R/W
MCPWM_DT2_FED_CFG_REG	FED 影子寄存器	0x00CC	R/W
MCPWM_DT2_RED_CFG_REG	RED 影子寄存器	0x00D0	R/W
MCPWM_CARRIER2_CFG_REG	使能与配置载波	0x00D4	R/W
MCPWM_FH2_CFG0_REG	故障事件中 PWM2A 和 PWM2B 输出上的操作	0x00D8	R/W
MCPWM_FH2_CFG1_REG	故障处理的软件触发	0x00DC	R/W
MCPWM_FH2_STATUS_REG	故障事件状态	0x00E0	RO
故障检测与配置			
MCPWM_FAULT_DETECT_REG	故障检测与配置	0x00E4	varies
捕获配置与状态			
MCPWM_CAP_TIMER_CFG_REG	配置捕获定时器	0x00E8	varies
MCPWM_CAP_TIMER_PHASE_REG	捕获定时器同步相位	0x00EC	R/W
MCPWM_CAP_CH0_CFG_REG	捕获通道 0 的配置与使能	0x00F0	varies
MCPWM_CAP_CH1_CFG_REG	捕获通道 1 的配置与使能	0x00F4	varies
MCPWM_CAP_CH2_CFG_REG	捕获通道 2 的配置与使能	0x00F8	varies
MCPWM_CAP_CH0_REG	捕获通道 0 值的状态	0x00FC	RO
MCPWM_CAP_CH1_REG	捕获通道 1 值的状态	0x0100	RO
MCPWM_CAP_CH2_REG	捕获通道 2 值的状态	0x0104	RO
MCPWM_CAP_STATUS_REG	上一次捕获触发器的边沿	0x0108	RO
使能有效寄存器的更新			
MCPWM_UPDATE_CFG_REG	使能更新	0x010C	R/W
管理中断			

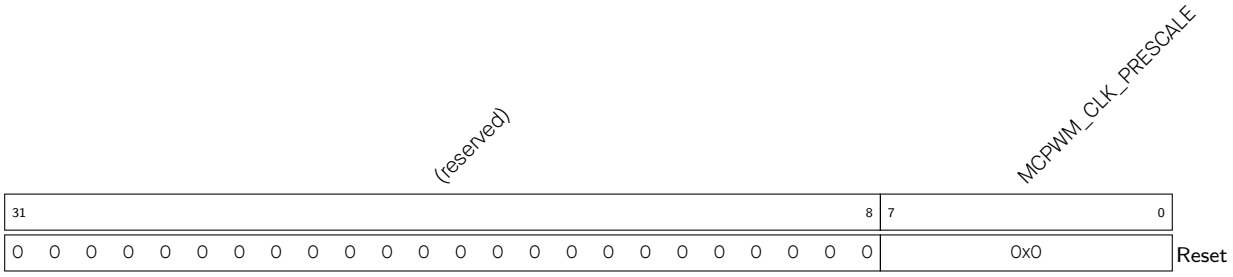
名称	描述	地址	访问
MCPWM_INT_ENA_REG	中断使能位	0x0110	R/W
MCPWM_INT_RAW_REG	原始中断状态	0x0114	R/WTC/SS
MCPWM_INT_ST_REG	屏蔽中断状态	0x0118	RO
MCPWM_INT_CLR_REG	中断清除位	0x011C	WT
使能 MCPWM 事件			
MCPWM_EVT_EN_REG	使能 MCPWM 事件	0x0120	R/W
MCPWM_EVT_EN2_REG	使能 MCPWM 事件	0x0128	R/W
使能 MCPWM 任务			
MCPWM_TASK_EN_REG	使能 MCPWM 任务	0x0124	R/W
MCPWM 生成器配置			
MCPWM_OP0_TSTMP_E1_REG	配置生成器 0 时间戳 E1 值	0x012C	R/W
MCPWM_OP0_TSTMP_E2_REG	配置生成器 0 时间戳 E2 值	0x0130	R/W
MCPWM_OP1_TSTMP_E1_REG	配置生成器 1 时间戳 E1 值	0x0134	R/W
MCPWM_OP1_TSTMP_E2_REG	配置生成器 1 时间戳 E2 值	0x0138	R/W
MCPWM_OP2_TSTMP_E1_REG	配置生成器 2 时间戳 E1 值	0x013C	R/W
MCPWM_OP2_TSTMP_E2_REG	配置生成器 2 时间戳 E2 值	0x0140	R/W
MCPWM APB 配置			
MCPWM_CLK_REG	MCPWM APB 配置	0x0144	R/W
版本寄存器			
MCPWM_VERSION_REG	版本控制寄存器	0x0148	R/W

24.6 寄存器

本小节的所有地址均为相对于电机控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

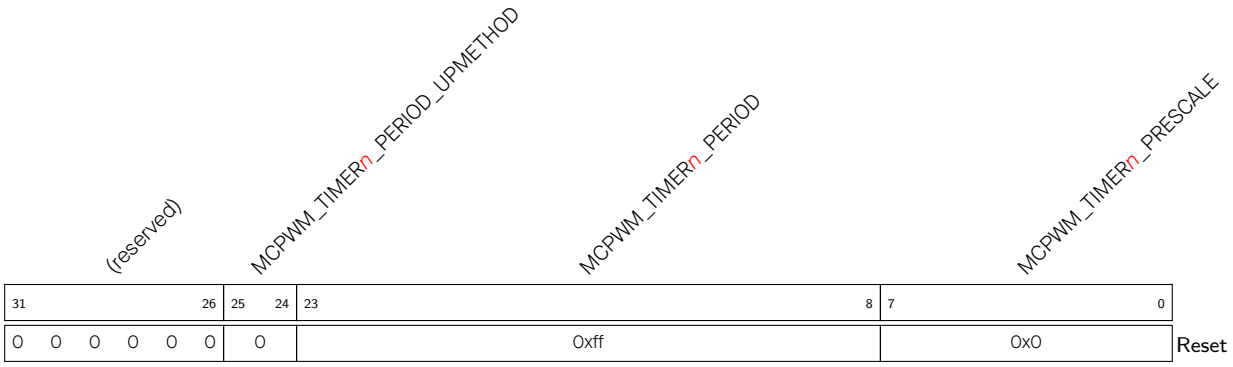
关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 24.1. MCPWM_CLK_CFG_REG (0x0000)



MCPWM_CLK_PRESCALE 配置时钟预分频系数，使得 PWM_CLK 的周期为 6.25 ns * (PWM_CLK_PRESCALE + 1)。(R/W)

Register 24.2. MCPWM_TIMER*n*_CFG0_REG (*n*: 0-2) (0x0004+0x10n*)**

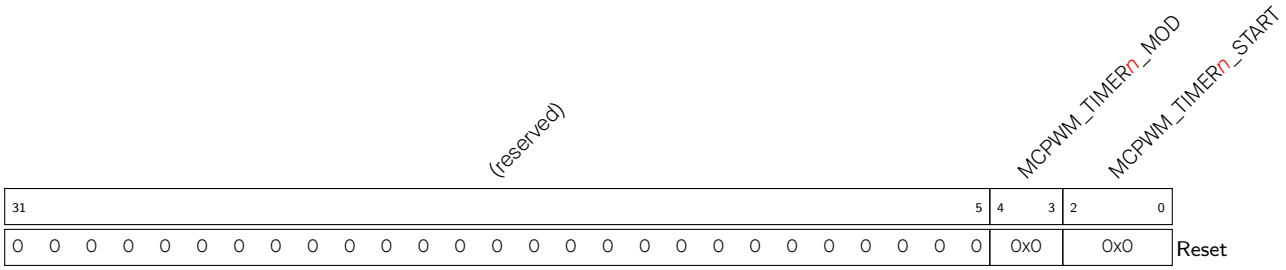


MCPWM_TIMER*n*_PRESCALE 配置定时器*n* 的预分频系数，使得 PTO_CLK 的周期 = PWM_CLK 的周期 * (PWM_TIMER*n*_PRESCALE + 1)。
(R/W)

MCPWM_TIMER*n*_PERIOD 配置定时器*n* 的影子周期。
(R/W)

MCPWM_TIMER*n*_PERIOD_UPMETHOD 配置 PWM 定时器*n* 周期有效寄存器的更新方式。
0：立即更新
1：发生 TEZ 事件时更新
2：同步时更新
3：发生 TEZ 事件或同步时更新
本文档中，TEZ 指定时器为 0 时的事件。
(R/W)

Register 24.3. MCPWM_TIMER*n*_CFG1_REG(*n*: 0-2) (0x0008+0x10**n*)



MCPWM_TIMER*n*_START 控制 PWM 定时器的开启与关闭。

0：如果开启，在 TEZ 事件发生时停止

1：如果开启，在 TEP 事件发生时停止

2：开启

3：开启，并在下一个 TEZ 事件发生时停止

4：开启，并在下一个 TEP 事件发生时停止

其他值：无效值，不产生影响

本文档中，TEP 指定时器为周期值时发生的事件。

(R/W/SC)

MCPWM_TIMER*n*_MOD 配置 PWM 定时器 0 的工作模式。

0：暂停

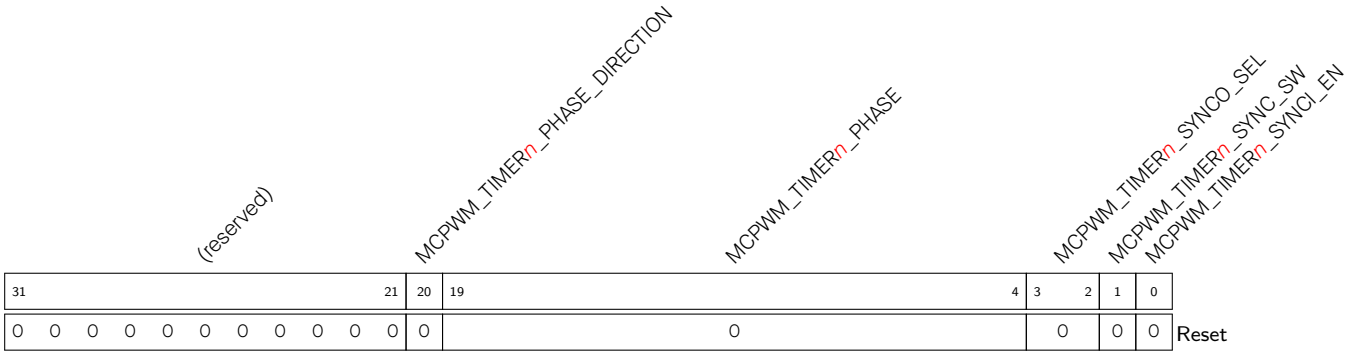
1：递增模式

2：递减模式

3：递增递减循环模式

(R/W)

Register 24.4. MCPWM_TIMER*n*_SYNC_REG (*n*: 0-2) (0x000C+0x10**n*)



MCPWM_TIMER*n*_SYNCI_EN 配置是否开启同步输入事件发生时的定时器*n* 相位重载。

0：关闭

1：开启

(R/W)

MCPWM_TIMER*n*_SYNC_SW 配置是否触发软件同步。

0：无效

1：触发软件同步

(R/W)

MCPWM_TIMER*n*_SYNCO_SEL 选择 PWM 定时器*n* 的同步输出来源。

0：同步。取反 MCPWM_TIMER*n*_SYNC_SW 位时始终生成同步输出。

1：TEZ

2：TEP

3：无效

(R/W)

MCPWM_TIMER*n*_PHASE 同步事件中定时器*n* 重载的相位。

(R/W)

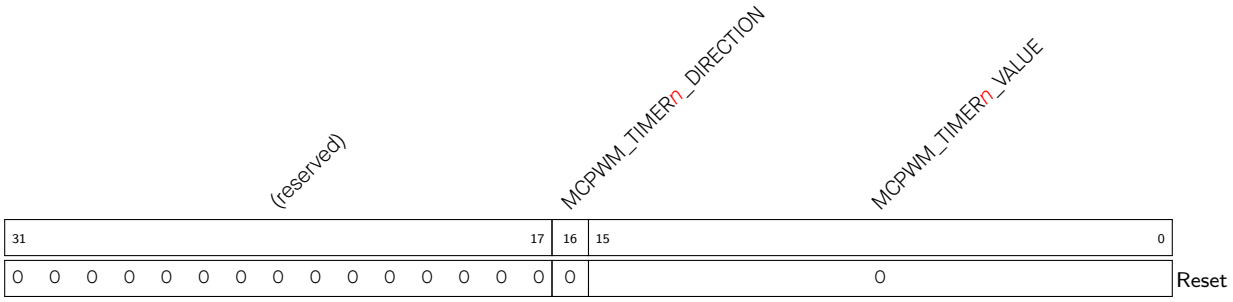
MCPWM_TIMER*n*_PHASE_DIRECTION 当定时器*n* 为递增-递减循环模式时，配置该定时器方向。

0：递增

1：递减

(R/W)

Register 24.5. MCPWM_TIMER*n*_STATUS_REG(*n*: 0-2) (0x0010+0x10**n*)



MCPWM_TIMER*n*_VALUE 表示当前 PWM 定时器*n* 计数器的值。(RO)

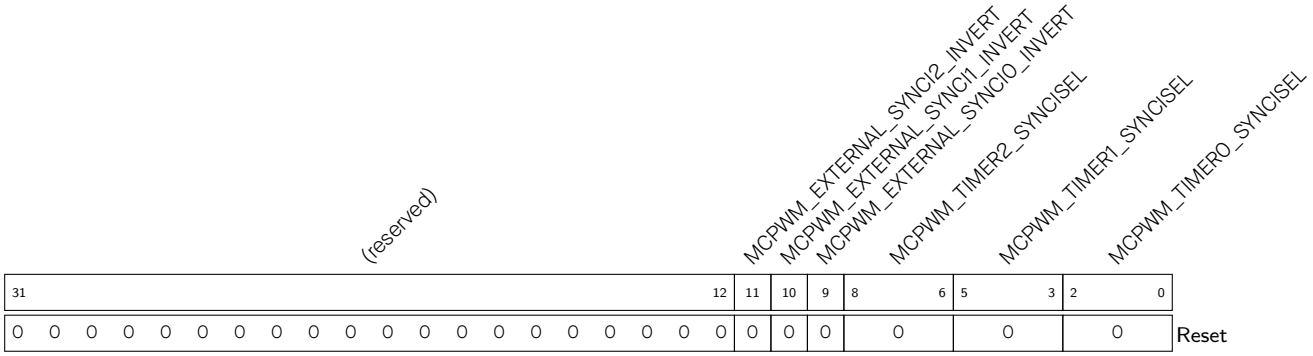
MCPWM_TIMER*n*_DIRECTION 配置当前 PWM 定时器*n* 的计数器模式。

0：递增模式

1：递减模式

(RO)

Register 24.6. MCPWM_TIMER_SYNCI_CFG_REG (0x0034)



MCPWM_TIMER*n*_SYNCISEL 选择 PWM 定时器*n* 的同步输入来源。

1：PWM 定时器 0 同步输出

2：PWM 定时器 1 同步输出

3：PWM 定时器 2 同步输出

4：来自 GPIO 矩阵的 SYNC0

5：来自 GPIO 矩阵的 SYNC1

6：来自 GPIO 矩阵的 SYNC2

其他值：未选择任何同步输入

(R/W)

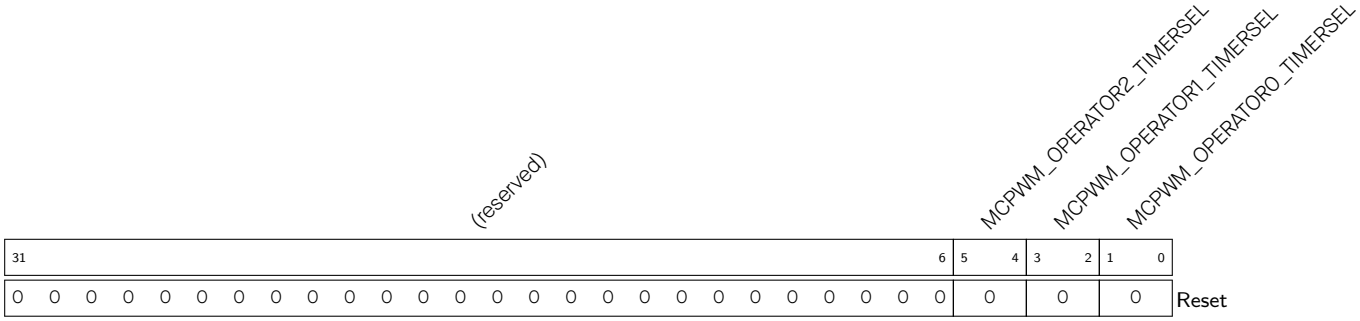
MCPWM_EXTERNAL_SYNCI*n*_INVERT 配置是否将来自 GPIO 矩阵的 SYNC*n* 反相。

0：不反相

1：反相

(R/W)

Register 24.7. MCPWM_OPERATOR_TIMERSEL_REG (0x0038)



MCPWM_OPERATOR*n*_TIMERSEL 选择 PWM 操作器*n* 的定时参考来源。

0：定时器 0

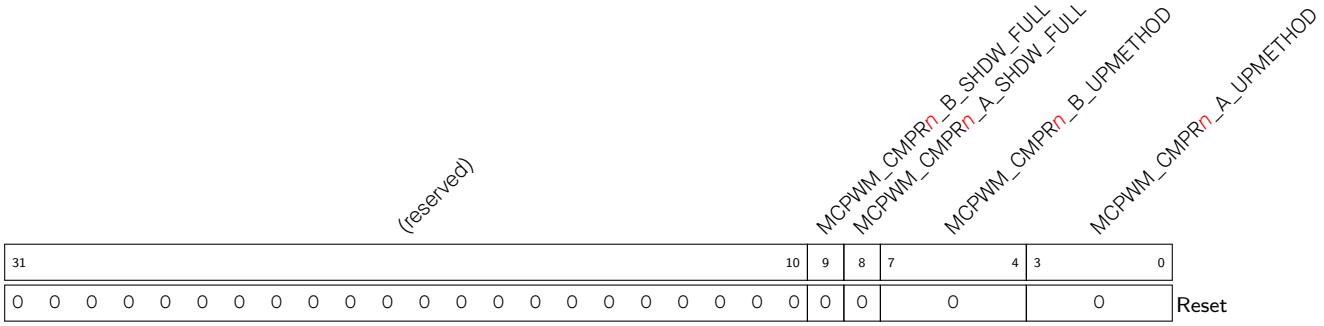
1：定时器 1

2：定时器 2

3：无效，将选择定时器 2

(R/W)

Register 24.8. MCPWM_GEN*n*_STMP_CFG_REG (*n*: 0-2) (0x003C+0x38**n*)



MCPWM_CMPR*n*_A_UPMETHOD 配置 PWM 生成器*n* 时间戳寄存器 A 的有效寄存器的更新方式。

所有位值为 0：立即更新

bit0 为 1：发生 TEZ 事件时更新

bit1 为 1：发生 TEP 事件时更新

bit2 为 1：发生同步时间时更新

bit3 为 1：关闭更新

(R/W)

MCPWM_CMPR*n*_B_UPMETHOD 配置 PWM 生成器*n* 时间戳寄存器 B 的有效寄存器的更新方式。

所有位值为 0：立即更新

bit0 为 1：发生 TEZ 事件时更新

bit1 为 1：发生 TEP 事件时更新

bit2 为 1：发生同步时间时更新

bit3 为 1：关闭更新

(R/W)

MCPWM_CMPR*n*_A_SHDW_FULL 配置是否在某个特定时间将影子寄存器中的值写入对应的有效寄存器中。由硬件置 1 或复位。

0：将影子寄存器中最新的值写入有效寄存器 A 中

1：将值写入 PWM 生成器*n* 时间戳寄存器 A 的影子寄存器中，等待传输给有效寄存器 A。

(R/SC/WTC)

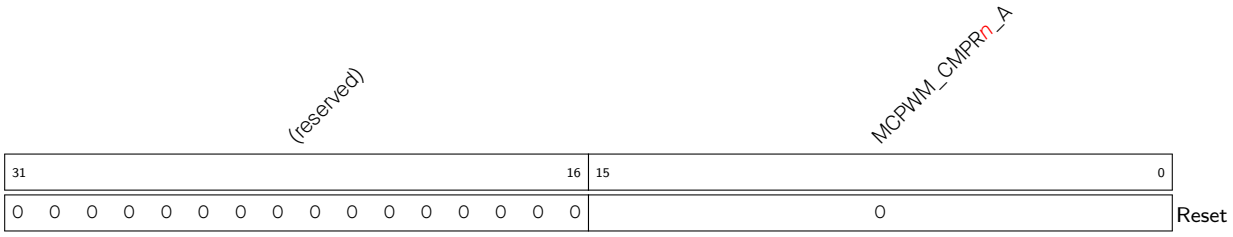
MCPWM_CMPR*n*_B_SHDW_FULL 配置是否在某个特定时间将影子寄存器中的值写入对应的有效寄存器中。由硬件置 1 或复位。

0：将影子寄存器中最新的值写入有效寄存器 B 中

1：将值写入 PWM 生成器*n* 时间戳寄存器 B 的影子寄存器中，等待传输给有效寄存器 B。

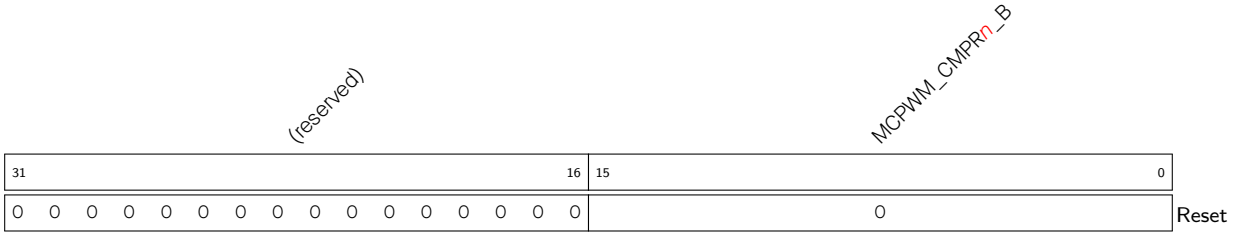
(R/SC/WTC)

Register 24.9. MCPWM_GEN*n*_TSTMP_A_REG(*n*: 0-2) (0x0040+0x38**n*)



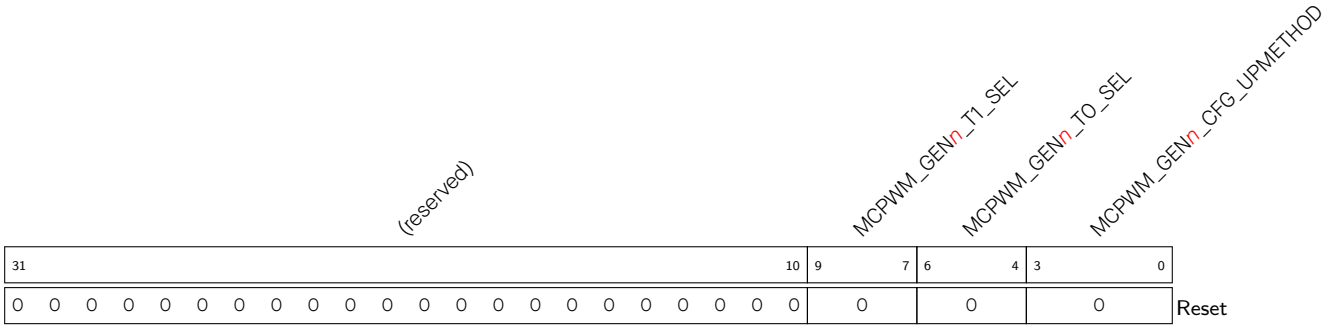
MCPWM_CMPR*n*_A 配置 PWM 生成器*n* 时间戳寄存器 A 的影子寄存器。 (R/W)

Register 24.10. MCPWM_GEN*n*_TSTMP_B_REG(*n*: 0-2) (0x0044+0x38**n*)



MCPWM_CMPR*n*_B 配置 PWM 生成器*n* 时间戳寄存器 B 的影子寄存器。 (R/W)

Register 24.11. MCPWM_GEN*n*_CFG0_REG(*n*: 0-2) (0x0048+0x38**n*)



MCPWM_GEN*n*_CFG_UPMETHOD 配置 PWM 生成器*n* 有效配置寄存器的更新方式。

所有 bit 值为 0：立即更新

bit0 为 1：发生 TEZ 事件时更新

bit1 为 1：发生 TEP 事件时更新

bit2 为 1：发生同步时间时更新

bit3 为 1：关闭更新

(R/W)

MCPWM_GEN*n*_T0_SEL 选择 PWM 生成器*n* event_t0 的信号源，立即生效。

0：fault_event0

1：fault_event1

2：fault_event2

3：sync_taken

4：无

(R/W)

MCPWM_GEN*n*_T1_SEL 选择 PWM 生成器*n* event_t1 的信号源，立即生效。

0：fault_event0

1：fault_event1

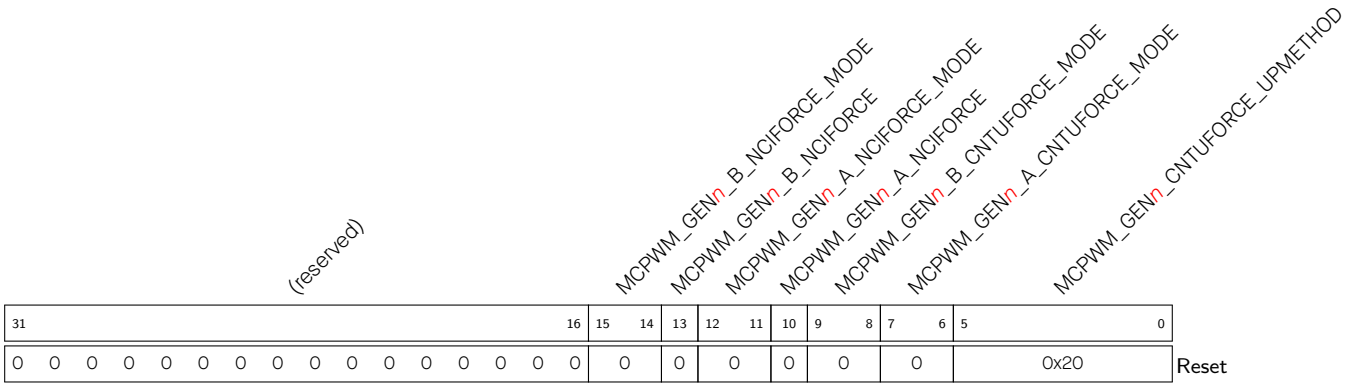
2：fault_event2

3：sync_taken

4：无

(R/W)

Register 24.12. MCPWM_GEN*n*_FORCE_REG(*n*: 0-2) (0x004C+0x38**n*)



MCPWM_GEN*n*_CNTUFORCE_UPMETHOD 配置生成器*n* 的连续软件强制事件更新方式。

所有 bit 为 0 时：立即更新

bit0 为 1：发生 TEZ 事件时更新

bit1 为 1：发生 TEP 事件时更新

bit2 为 1：发生 TEA 事件时更新

bit3 为 1：发生 TEB 事件时更新

bit4：发生同步事件时更新

bit5 为 1：关闭更新

本文档中，TEA/B 指定时器值为寄存器 A/B 的值时生成的事件。

(R/W)

MCPWM_GEN*n*_A_CNTUFORCE_MODE 配置 PWM*n* A 的连续软件强制模式。

0：关闭

1：低电平

2：高电平

3：关闭

(R/W)

MCPWM_GEN*n*_B_CNTUFORCE_MODE 配置 PWM*n* B 的连续软件强制模式。

0：关闭

1：低电平

2：高电平

3：关闭

(R/W)

MCPWM_GEN*n*_A_NCIFORCE 配置是否触发 PWM*n* A 上的非连续即时软件强制事件。

0：无效

1：触发 PWM*n* A 上的非连续即时软件强制事件

(R/W)

见下页...

Register 24.12. MCPWM_GEN*n*_FORCE_REG(*n*: 0-2) (0x004C+0x38**n*)

... 接上页

MCPWM_GEN*n*_A_NCIFORCE_MODE 配置用于 PWM*n* A 的非连续即时软件强制模式。

0：关闭

1：低电平

2：高电平

3：关闭

(R/W)

MCPWM_GEN*n*_B_NCIFORCE 配置是否触发 PWM*n* B 上的非连续即时软件强制事件。

0：无效

1：触发 PWM*n* B 上的非连续即时软件强制事件

(R/W)

MCPWM_GEN*n*_B_NCIFORCE_MODE 配置用于 PWM*n* B 的非连续即时软件强制模式。

0：关闭

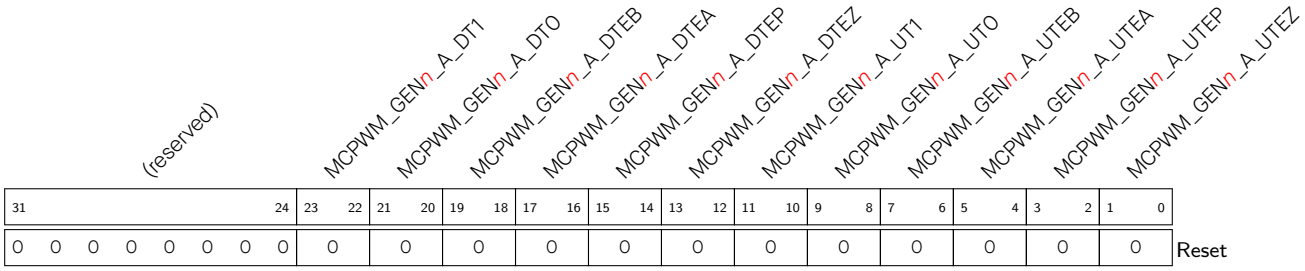
1：低电平

2：高电平

3：关闭

(R/W)

Register 24.13. MCPWM_GEN*n*_A_REG (*n*: 0-2) (0x0050+0x38**n*)



MCPWM_GEN*n*_A_UTEZ 配置定时器递增时，TEZ 事件在 PWM*n*A 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

MCPWM_GEN*n*_A_UTEP 配置定时器递增时，TEP 事件在 PWM*n*A 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

MCPWM_GEN*n*_A_UTEA 配置定时器递增时，TEA 事件在 PWM*n*A 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

MCPWM_GEN*n*_A_UTEB 配置定时器递增时，TEB 事件在 PWM*n*A 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

见下页...

Register 24.13. MCPWM_GEN*n*_A_REG(*n*: 0-2) (0x0050+0x38**n*)

... 接上页

MCPWM_GEN*n*_A_UT0 配置定时器递增时，event_t0 在 PWM*n*A 上触发的操作。

0：波形无改变
1：拉低
2：拉高
3：取反
(R/W)

MCPWM_GEN*n*_A_UT1 配置定时器递增时，event_t1 在 PWM*n*A 上触发的操作。

0：波形无改变
1：拉低
2：拉高
3：取反
(R/W)

MCPWM_GEN*n*_A_DTEZ 配置定时器递减时，TEZ 事件在 PWM*n*A 上触发的操作。

0：波形无改变
1：拉低
2：拉高
3：取反
(R/W)

MCPWM_GEN*n*_A_DTEP 配置定时器递减时，TEP 事件在 PWM*n*A 上触发的操作。

0：波形无改变
1：拉低
2：拉高
3：取反
(R/W)

MCPWM_GEN*n*_A_DTEA 配置定时器递减时，TEA 事件在 PWM*n*A 上触发的操作。

0：波形无改变
1：拉低
2：拉高
3：取反
(R/W)

见下页...

Register 24.13. MCPWM_GEN*n*_A_REG(*n*: 0-2) (0x0050+0x38**n*)

... 接上页

MCPWM_GEN*n*_A_DTEB 配置定时器递减时，TEB 事件在 PWM*n*A 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

MCPWM_GEN*n*_A_DT0 配置定时器递减时，event_t0 在 PWM*n*A 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

MCPWM_GEN*n*_A_DT1 配置定时器递减时，event_t1 在 PWM*n*A 上触发的操作。

0：波形无改变

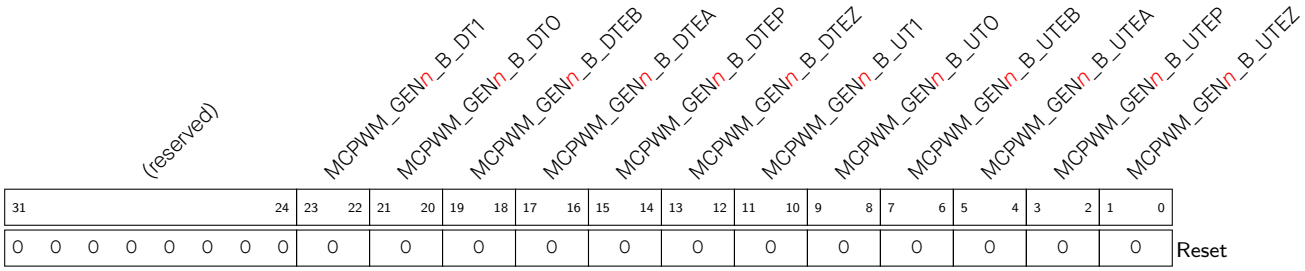
1：拉低

2：拉高

3：取反

(R/W)

Register 24.14. MCPWM_GEN*n*_B_REG (*n*: 0-2) (0x0054+0x38**n*)



MCPWM_GEN*n*_B_UTEZ 定时器递增时，TEZ 在 PWM*n*B 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

MCPWM_GEN*n*_B_UTEP 定时器递增时，TEP 在 PWM*n*B 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

MCPWM_GEN*n*_B_UTEA 定时器递增时，TEA 在 PWM*n*B 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

MCPWM_GEN*n*_B_UTEB 定时器递增时，TEB 在 PWM*n*B 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

见下页...

Register 24.14. MCPWM_GEN*n*_B_REG(*n*: 0-2) (0x0054+0x38**n*)

... 接上页

MCPWM_GEN*n*_B_UT0 定时器递增时，event_t0 在 PWM*n*B 上触发的操作。

0：波形无改变
1：拉低
2：拉高
3：取反
(R/W)

MCPWM_GEN*n*_B_UT1 定时器递增时，event_t1 在 PWM*n*B 上触发的操作。

0：波形无改变
1：拉低
2：拉高
3：取反
(R/W)

MCPWM_GEN*n*_B_DTEZ 定时器递减时，TEZ 事件在 PWM*n*B 上触发的操作。

0：波形无改变
1：拉低
2：拉高
3：取反
(R/W)

MCPWM_GEN*n*_B_DTEP 定时器递减时，TEP 事件在 PWM*n*B 上触发的操作。

0：波形无改变
1：拉低
2：拉高
3：取反
(R/W)

MCPWM_GEN*n*_B_DTEA 定时器递减时，TEA 事件在 PWM*n*B 上触发的操作。

0：波形无改变
1：拉低
2：拉高
3：取反
(R/W)

见下页...

Register 24.14. MCPWM_GEN*n*_B_REG(*n*: 0-2) (0x0054+0x38**n*)

... 接上页

MCPWM_GEN*n*_B_DTEB 定时器递减时，TEB 事件在 PWM*n*B 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

MCPWM_GEN*n*_B_DT0 定时器递减时，event_t0 事件在 PWM*n*B 上触发的操作。

0：波形无改变

1：拉低

2：拉高

3：取反

(R/W)

MCPWM_GEN*n*_B_DT1 定时器递减时，event_t1 在 PWM*n*B 上触发的操作。

0：波形无改变

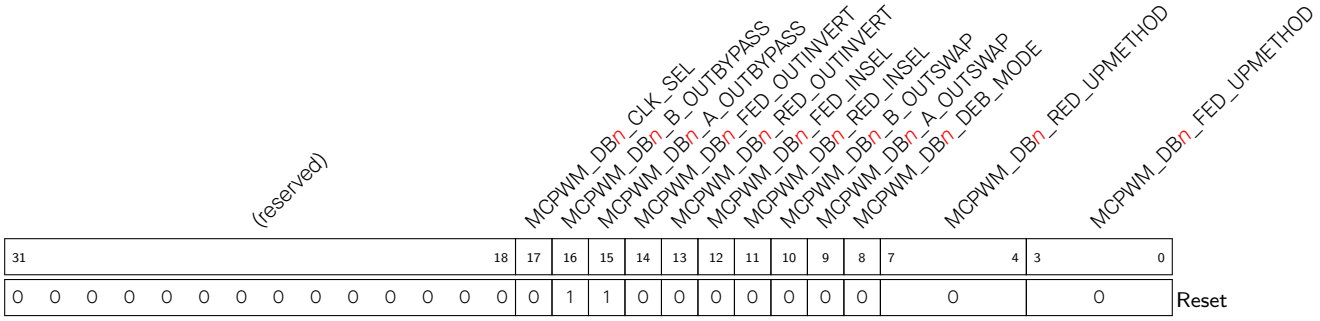
1：拉低

2：拉高

3：取反

(R/W)

Register 24.15. MCPWM_DT*n*_CFG_REG(*n*: 0-2) (0x0058+0x38**n*)



MCPWM_DB*n*_FED_UPMETHOD 配置下降沿延迟有效寄存器的更新方式。

所有位值为 0：立即更新

bit0 为 1：发生 TEZ 事件时更新

bit1 为 1：发生 TEP 事件时更新

bit2 为 1：发生同步时间时更新

bit3 为 1：关闭更新

(R/W)

MCPWM_DB*n*_RED_UPMETHOD 配置上升沿延迟有效寄存器的更新方式。

所有位值为 0：立即更新

bit0 为 1：发生 TEZ 事件时更新

bit1 为 1：发生 TEP 事件时更新

bit2 为 1：发生同步时间时更新

bit3 为 1：关闭更新

(R/W)

MCPWM_DB*n*_DEB_MODE 配置表 24.3-5中的 S8 开关。详细配置信息请参考表 24.3-6。

0：FED/RED 分别在不同路径上生效

1：FED/RED 在 B 路径上生效，A 输出为旁路或 dulpB 模式

(R/W)

MCPWM_DB*n*_A_OUTSWAP 配置表 24.3-5中的 S6 开关。详细配置信息请参考表 24.3-6。(R/W)

MCPWM_DB*n*_B_OUTSWAP 配置表 24.3-5中的 S7 开关。详细配置信息请参考表 24.3-6。(R/W)

MCPWM_DB*n*_RED_INSEL 配置表 24.3-5中的 S4 开关。详细配置信息请参考表 24.3-6。(R/W)

MCPWM_DB*n*_FED_INSEL 配置表 24.3-5中的 S5 开关。详细配置信息请参考表 24.3-6。(R/W)

MCPWM_DB*n*_RED_OUTINVERT 配置表 24.3-5中的 S2 开关。详细配置信息请参考表 24.3-6。(R/W)

MCPWM_DB*n*_FED_OUTINVERT 配置表 24.3-5中的 S3 开关。详细配置信息请参考表 24.3-6。(R/W)

MCPWM_DB*n*_A_OUTBYPASS 配置表 24.3-5中的 S1 开关。详细配置信息请参考表 24.3-6。(R/W)

MCPWM_DB*n*_B_OUTBYPASS 配置表 24.3-5中的 S0 开关。详细配置信息请参考表 24.3-6。(R/W)

见下页...

Register 24.15. MCPWM_DT*n*_CFG_REG(*n*: 0-2) (0x0058+0x38**n*)

... 接上页

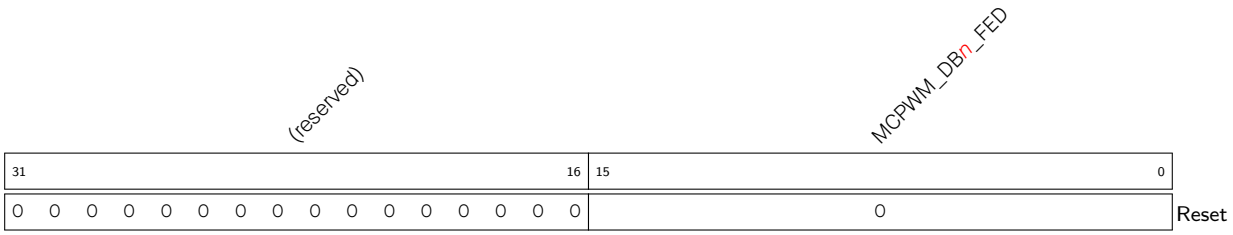
MCPWM_DB*n*_CLK_SEL 选择死区时间生成器 0 的时钟。

0：PWM_CLK

1：PT_CLK

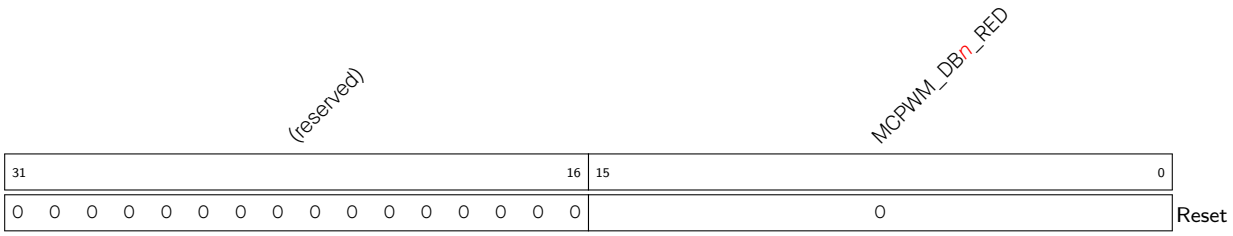
(R/W)

Register 24.16. MCPWM_DT*n*_FED_CFG_REG(*n*: 0-2) (0x005C+0x38**n*)



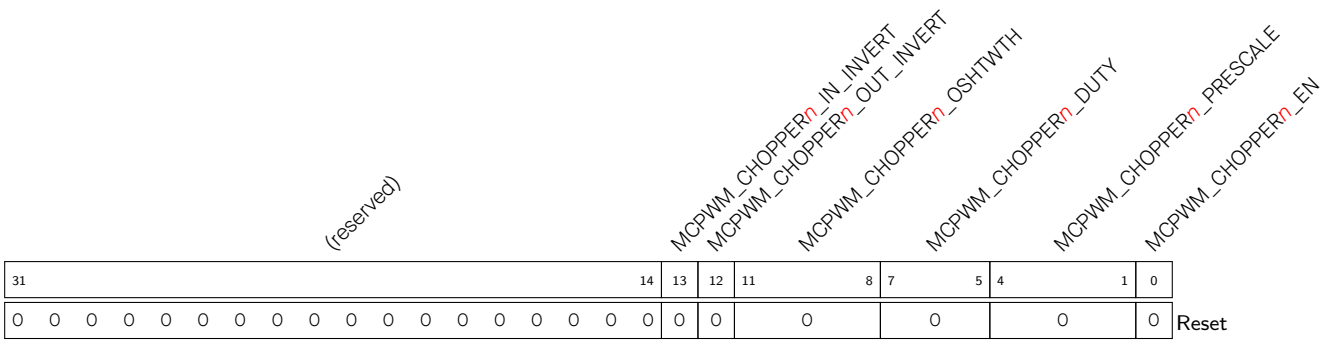
MCPWM_DB*n*_FED 配置 FED 的影子寄存器。(R/W)

Register 24.17. MCPWM_DT*n*_RED_CFG_REG(*n*: 0-2) (0x0060+0x38**n*)



MCPWM_DB*n*_RED 配置 RED 的影子寄存器。(R/W)

Register 24.18. MCPWM_CARRIER*n*_CFG_REG(*n*: 0-2) (0x0064+0x38**n*)



MCPWM_CHOPPER*n*_EN 配置是否使能载波*n*。

0：绕过

1：使能

(R/W)

MCPWM_CHOPPER*n*_PRESCALE 配置 PWM 载波*n* 时钟 (PC_CLK) 的预分频值。PC_CLK 周期 = PWM_CLK 周期 * (PWM_CARRIER*n*_PRESCALE + 1)。(R/W)

MCPWM_CHOPPER*n*_DUTY 配置载波占空比。占空比 = PWM_CARRIER*n*_DUTY/8。(R/W)

MCPWM_CHOPPER*n*_OSHTWTH 配置载波第一个脉冲的宽度，单位为载波周期。(R/W)

MCPWM_CHOPPER*n*_OUT_INVERT 配置是否将此模块的 PWM*n*A 和 PWM*n*B 输出反相。

0：正常输出

1：反相

(R/W)

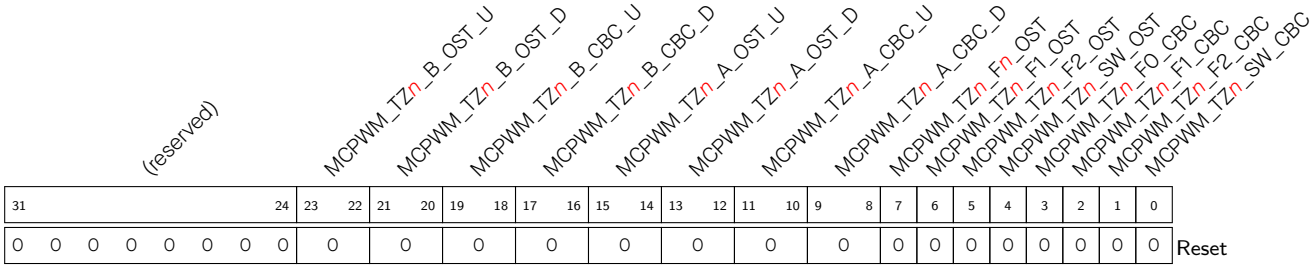
MCPWM_CHOPPER*n*_IN_INVERT 配置是否将此模块的 PWM*n*A 和 PWM*n*B 输入反相。

0：正常输入

1：反相

(R/W)

Register 24.19. MCPWM_FH*n*_CFG0_REG(*n*: 0-2) (0x0068+0x38**n*)



MCPWM_TZ*n*_SW_CBC 配置是否使能软件强制逐周期模式操作。

0：关闭

1：使能

(R/W)

MCPWM_TZ*n*_F2_CBC 配置 fault_event2 是否触发逐周期模式操作。

0：关闭

1：使能

(R/W)

MCPWM_TZ*n*_F1_CBC 配置 fault_event1 是否触发逐周期模式操作。

0：关闭

1：使能

(R/W)

MCPWM_TZ*n*_F0_CBC 配置 fault_event0 是否触发逐周期模式操作。

0：关闭

1：使能

(R/W)

MCPWM_TZ*n*_SW_OST 配置是否使能软件强制一次性模式操作。

0：关闭

1：使能

(R/W)

MCPWM_TZ*n*_F2_OST 配置 fault_event2 是否触发一次性模式操作。

0：关闭

1：使能

(R/W)

见下页...

Register 24.19. MCPWM_FH*n*_CFG0_REG(*n*: 0-2) (0x0068+0x38**n*)

... 接上页

MCPWM_TZ*n*_F1_OST 配置 fault_event1 是否触发一次性模式操作。

0：关闭
1：使能
(R/W)

MCPWM_TZ*n*_F*n*_OST 配置 fault_event0 是否触发一次性模式操作。

0：关闭
1：使能
(R/W)

MCPWM_TZ*n*_A_CBC_D 配置定时器递减计数并且发生故障事件时，PWM*n*A 上的逐周期模式操作。

0：无
1：强制拉低
2：强制拉高
3：取反
(R/W)

MCPWM_TZ*n*_A_CBC_U 配置定时器递增计数并且发生故障事件时，PWM*n*A 上的逐周期模式操作。

0：无
1：强制拉低
2：强制拉高
3：取反
(R/W)

MCPWM_TZ*n*_A_OST_D 配置定时器递减计数并且发生故障事件时，PWM*n*A 上的一次性模式操作。

0：无
1：强制拉低
2：强制拉高
3：取反
(R/W)

见下页...

Register 24.19. MCPWM_FH*n*_CFG0_REG(*n*: 0-2) (0x0068+0x38**n*)

... 接上页

MCPWM_TZ*n*_A_OST_U 配置定时器递增计数并且发生故障事件时，PWM*n*A 上的一次性模式操作。

0：无
1：强制拉低
2：强制拉高
3：取反
(R/W)

MCPWM_TZ*n*_B_CBC_D 配置定时器递减计数并且发生故障事件时，PWM*n*B 上的逐周期模式操作。

0：无
1：强制拉低
2：强制拉高
3：取反
(R/W)

MCPWM_TZ*n*_B_CBC_U 配置定时器递增计数并且发生故障事件时，PWM*n*B 上的逐周期模式操作。

0：无
1：强制拉低
2：强制拉高
3：取反
(R/W)

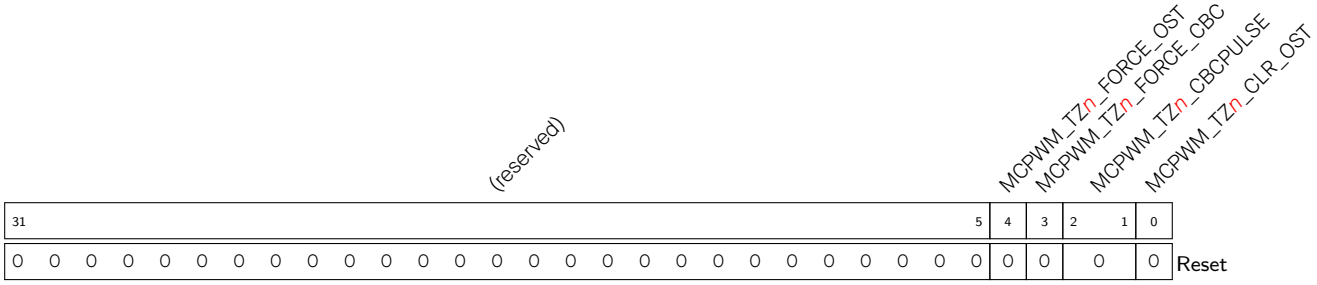
MCPWM_TZ*n*_B_OST_D 配置定时器递减计数并且发生故障事件时，PWM*n*B 上的一次性模式操作。

0：无
1：强制拉低
2：强制拉高
3：取反
(R/W)

MCPWM_TZ*n*_B_OST_U 配置定时器递增计数并且发生故障事件时，PWM*n*B 上的一次性模式操作。

0：无
1：强制拉低
2：强制拉高
3：取反
(R/W)

Register 24.20. MCPWM_FH*n*_CFG1_REG(*n*: 0-2) (0x006C+0x38**n*)



MCPWM_TZ*n*_CLR_OST 配置是否通过软件清除持续一次性模式操作。

0：无效

1：通过软件清除持续一次性模式操作

(R/W)

MCPWM_TZ*n*_CBCPULSE 配置逐周期模式操作更新的时间点。

bit0 为 1：发生 TEZ 事件时

bit1 为 1：发生 TEP 事件时

(R/W)

MCPWM_TZ*n*_FORCE_CBC 配置是否触发逐周期模式的操作。

0：无效

1：触发

(R/W)

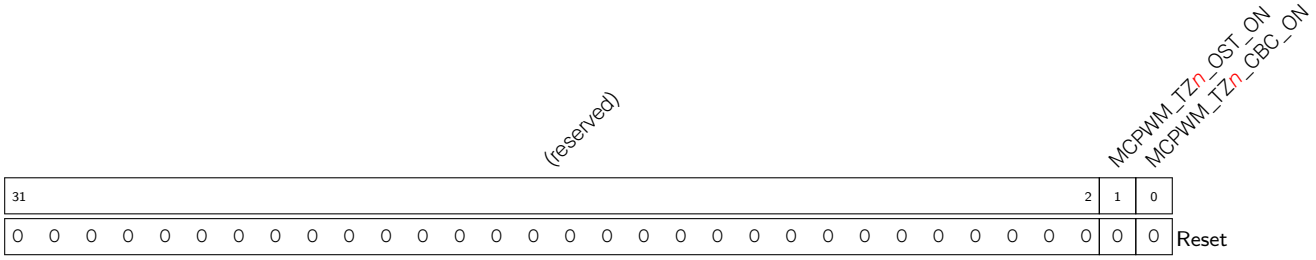
MCPWM_TZ*n*_FORCE_OST 配置是否触发一次性模式的操作。

0：无效

1：触发

(R/W)

Register 24.21. MCPWM_FH*n*_STATUS_REG(*n*: 0-2) (0x0070+0x38**n*)



MCPWM_TZ*n*_CBC_ON 表示是否正在进行逐周期模式的操作。

0：不存在正在进行的逐周期模式操作

1：逐周期模式的操作正在进行

(RO)

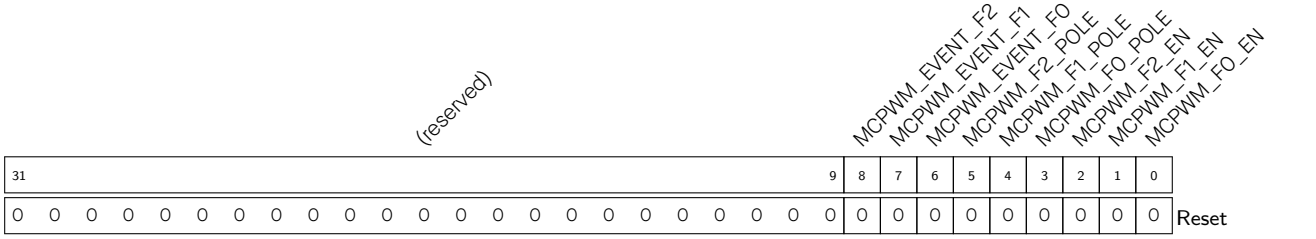
MCPWM_TZ*n*_OST_ON 表示是否正在进行一次性模式的操作。

0：不存在正在进行的一次性模式操作

1：一次性模式的操作正在进行

(RO)

Register 24.22. MCPWM_FAULT_DETECT_REG (0x00E4)



MCPWM_F*n*_EN 配置是否使能生成 fault_event*n*。

0：无效

1：使能

(R/W)

MCPWM_F*n*_POLE 配置来自 GPIO 矩阵的 FAULT*n* 信号源触发 fault_event*n* 时极性。

0：低电平触发

1：高电平触发

(R/W)

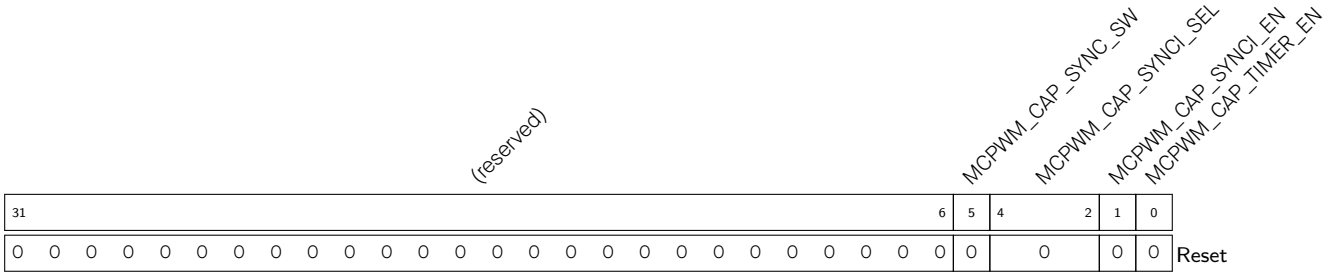
MCPWM_EVENT_F*n* 表示 fault_event*n* 事件的状态。此字段由硬件置 1 和清零。

0：fault_event*n* 事件停止

1：fault_event*n* 事件持续

(RO)

Register 24.23. MCPWM_CAP_TIMER_CFG_REG (0x00E8)



MCPWM_CAP_TIMER_EN 配置是否使能捕获定时器在 APB_CLK 下的递增。

0：无效

1：使能捕获定时器在 APB_CLK 下的递增

(R/W)

MCPWM_CAP_SYNCI_EN 配置是否使能捕获定时器同步。

0：无效

1：使能捕获定时器同步

(R/W)

MCPWM_CAP_SYNCI_SEL 配置捕获模块的同步输入。

0：无

1：定时器 0 的同步输出

2：定时器 1 的同步输出

3：定时器 2 的同步输出

4：来自 GPIO 矩阵的 SYNC0

5：来自 GPIO 矩阵的 SYNC1

6：来自 GPIO 矩阵的 SYNC2

7：无

(R/W)

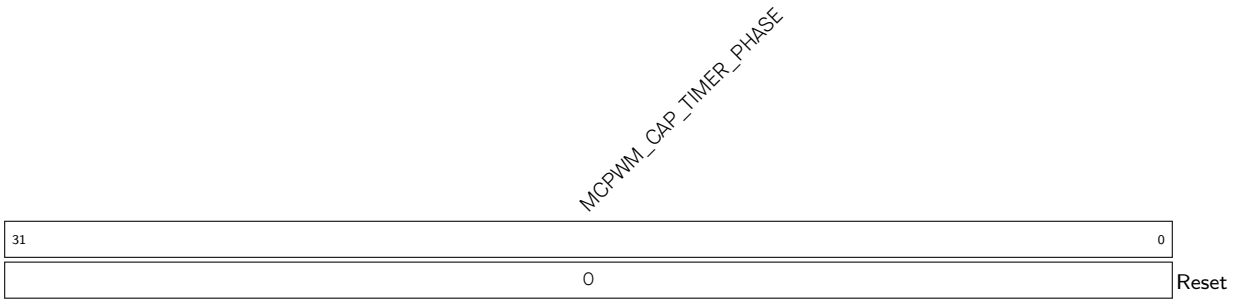
MCPWM_CAP_SYNC_SW 当 MCPWM_CAP_SYNCI_EN 置位时，配置是否同步捕获定时器，在捕获定时器中写入相位寄存器的值。

0：无效

1：同步捕获定时器

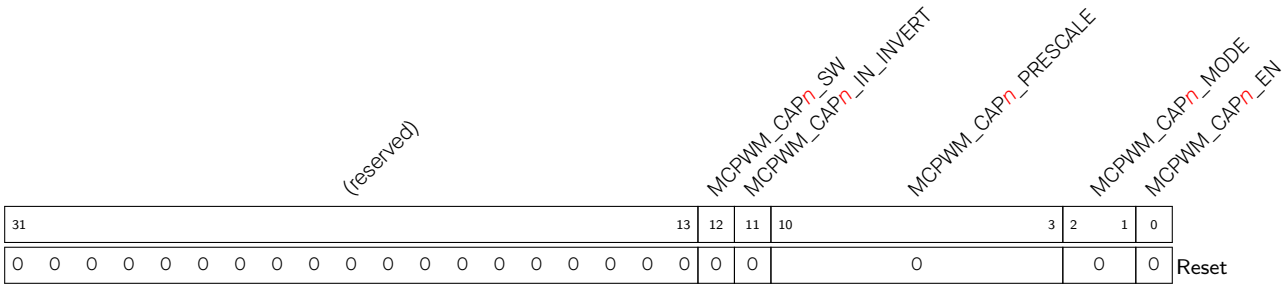
(WT)

Register 24.24. MCPWM_CAP_TIMER_PHASE_REG (0x00EC)



MCPWM_CAP_TIMER_PHASE 配置捕获定时器同步操作的相位值。(R/W)

Register 24.25. MCPWM_CAP_CH*n*_CFG_REG (*n*: 0-2) (0x00F0+0x4**n*)



MCPWM_CAP*n*_EN 配置是否使能信道 *n* 上的捕获事件。

0：不使能

1：使能

(R/W)

MCPWM_CAP*n*_MODE 配置预分频后信道 *n* 上的捕获沿。

0：无

bit0 为 1：使能下降沿捕获。

bit1 为 1：使能上升沿捕获。

(R/W)

MCPWM_CAP*n*_PRESCALE 配置 CAP*n* 上升沿的预分频值。预分频值 = PWM_CAP*n*_PRESCALE + 1。(R/W)

MCPWM_CAP*n*_IN_INVERT 配置是否在预分频前反相来自 GPIO 矩阵的 CAP*n*。

0：无效

1：在预分频前反相来自 GPIO 矩阵的 CAP*n*

(R/W)

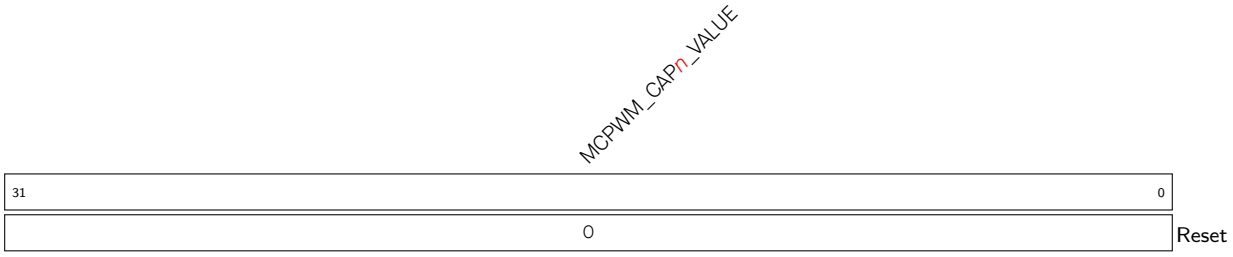
MCPWM_CAP*n*_SW 配置是否触发信道 0 上的软件强制捕获事件。

0：不触发

1：触发

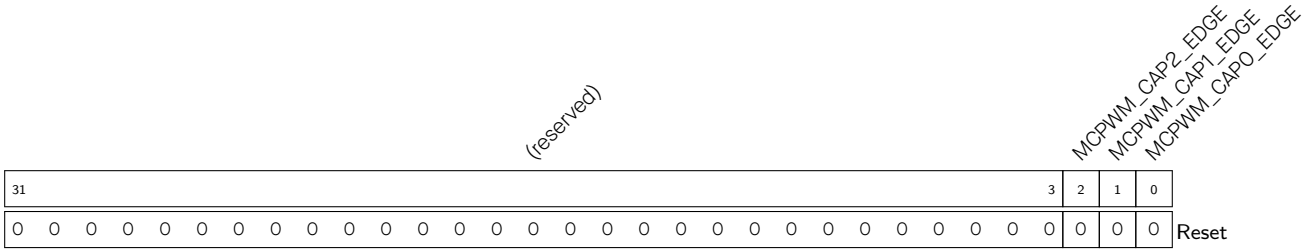
(WT)

Register 24.26. MCPWM_CAP_CH*n*_REG (*n*: 0-2) (0x00FC+0x4**n*)



MCPWM_CAP*n*_VALUE 表示信道 *n* 上一次捕获的值。(RO)

Register 24.27. MCPWM_CAP_STATUS_REG (0x0108)



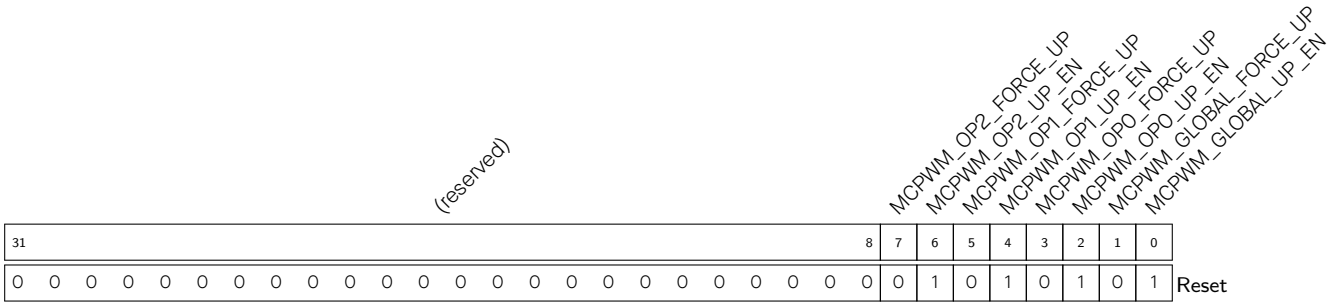
MCPWM_CAP*n*_EDGE 信道*n* 上一次捕获触发事件的边沿。

0：上升沿

1：下降沿

(RO)

Register 24.28. MCPWM_UPDATE_CFG_REG (0x010C)



MCPWM_GLOBAL_UP_EN 配置是否更新 MCPWM 模块的所有有效寄存器。

0：无效

1：更新 MCPWM 模块的所有有效寄存器

(R/W)

MCPWM_GLOBAL_FORCE_UP 配置是否强制更新 MCPWM 模块的所有有效寄存器。

0：无效

1：强制更新 MCPWM 模块的所有有效寄存器

(R/W)

MCPWM_OP0_UP_EN 当 MCPWM_GLOBAL_UP_EN 置 1 时，配置是否更新 PWM 操作器 0 中的有效寄存器。

0：无效

1：更新 PWM 操作器 0 中的有效寄存器

(R/W)

MCPWM_OP0_FORCE_UP 配置是否强制更新 PWM 操作器 0 中的有效寄存器。

0：无效

1：强制更新

(R/W)

MCPWM_OP1_UP_EN 当 MCPWM_GLOBAL_UP_EN 置 1 时，配置是否更新 PWM 操作器 1 中的有效寄存器。

0：无效

1：更新 PWM 操作器 1 中的有效寄存器

(R/W)

MCPWM_OP1_FORCE_UP 配置是否强制更新 PWM 操作器 1 中的有效寄存器。

0：无效

1：强制更新

(R/W)

MCPWM_OP2_UP_EN 当 MCPWM_GLOBAL_UP_EN 置 1 时，配置是否更新 PWM 操作器 2 中的有效寄存器。

0：无效

1：更新 PWM 操作器 2 中的有效寄存器

(R/W)

见下页...

Register 24.28. MCPWM_UPDATE_CFG_REG (0x010C)

... 接上页

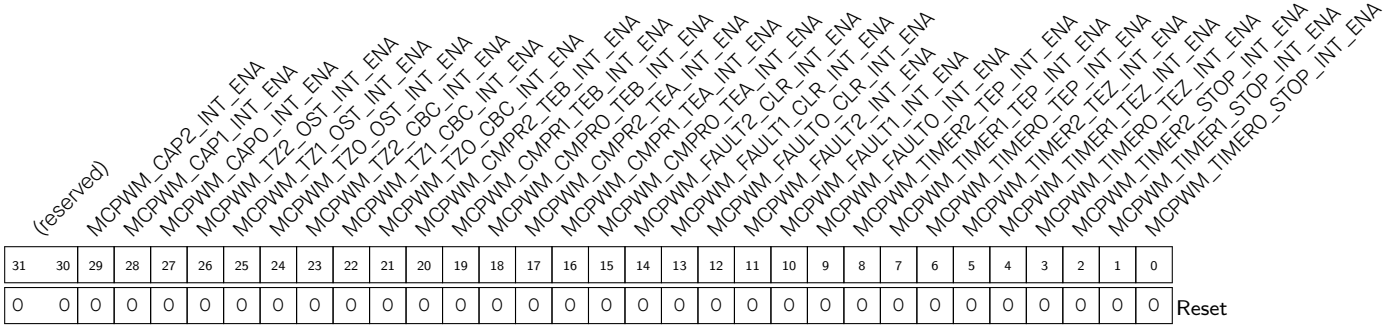
MCPWM_OP2_FORCE_UP 配置是否强制更新 PWM 操作器 2 中的有效寄存器。

0：无效

1：强制更新

(R/W)

Register 24.29. MCPWM_INT_ENA_REG (0x0110)



31 30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0 0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

MCPWM_TIMER0_STOP_INT_ENA 写 1 使能 MCPWM_TIMER0_STOP_INT。(R/W)

MCPWM_TIMER1_STOP_INT_ENA 写 1 使能 MCPWM_TIMER1_STOP_INT。(R/W)

MCPWM_TIMER2_STOP_INT_ENA 写 1 使能 MCPWM_TIMER2_STOP_INT。(R/W)

MCPWM_TIMER0_TEZ_INT_ENA 写 1 使能 MCPWM_TIMER0_TEZ_INT。(R/W)

MCPWM_TIMER1_TEZ_INT_ENA 写 1 使能 MCPWM_TIMER1_TEZ_INT。(R/W)

MCPWM_TIMER2_TEZ_INT_ENA 写 1 使能 MCPWM_TIMER2_TEZ_INT。(R/W)

MCPWM_TIMER0_TEP_INT_ENA 写 1 使能 MCPWM_TIMER0_TEP_INT。(R/W)

MCPWM_TIMER1_TEP_INT_ENA 写 1 使能 MCPWM_TIMER1_TEP_INT。(R/W)

MCPWM_TIMER2_TEP_INT_ENA 写 1 使能 MCPWM_TIMER2_TEP_INT。(R/W)

MCPWM_FAULT0_INT_ENA 写 1 使能 MCPWM_FAULT0_INT。(R/W)

MCPWM_FAULT1_INT_ENA 写 1 使能 MCPWM_FAULT1_INT。(R/W)

MCPWM_FAULT2_INT_ENA 写 1 使能 MCPWM_FAULT2_INT。(R/W)

MCPWM_FAULT0_CLR_INT_ENA 写 1 使能 MCPWM_FAULT0_CLR_INT。(R/W)

MCPWM_FAULT1_CLR_INT_ENA 写 1 使能 MCPWM_FAULT1_CLR_INT。(R/W)

MCPWM_FAULT2_CLR_INT_ENA 写 1 使能 MCPWM_FAULT2_CLR_INT。(R/W)

MCPWM_CMPR0_TEA_INT_ENA 写 1 使能 MCPWM_CMPR0_TEA_INT。(R/W)

MCPWM_CMPR1_TEA_INT_ENA 写 1 使能 MCPWM_CMPR1_TEA_INT。(R/W)

MCPWM_CMPR2_TEA_INT_ENA 写 1 使能 MCPWM_CMPR2_TEA_INT。(R/W)

见下页...

Register 24.29. MCPWM_INT_ENA_REG (0x0110)

... 接上页

MCPWM_CMPR0_TEB_INT_ENA 写 1 使能 MCPWM_CMPR0_TEB_INT。(R/W)

MCPWM_CMPR1_TEB_INT_ENA 写 1 使能 MCPWM_CMPR1_TEB_INT。(R/W)

MCPWM_CMPR2_TEB_INT_ENA 写 1 使能 MCPWM_CMPR2_TEB_INT。(R/W)

MCPWM_TZ0_CBC_INT_ENA 写 1 使能 MCPWM_TZ0_CBC_INT。(R/W)

MCPWM_TZ1_CBC_INT_ENA 写 1 使能 MCPWM_TZ1_CBC_INT。(R/W)

MCPWM_TZ2_CBC_INT_ENA 写 1 使能 MCPWM_TZ2_CBC_INT。(R/W)

MCPWM_TZ0_OST_INT_ENA 写 1 使能 MCPWM_TZ0_OST_INT。(R/W)

MCPWM_TZ1_OST_INT_ENA 写 1 使能 MCPWM_TZ1_OST_INT。(R/W)

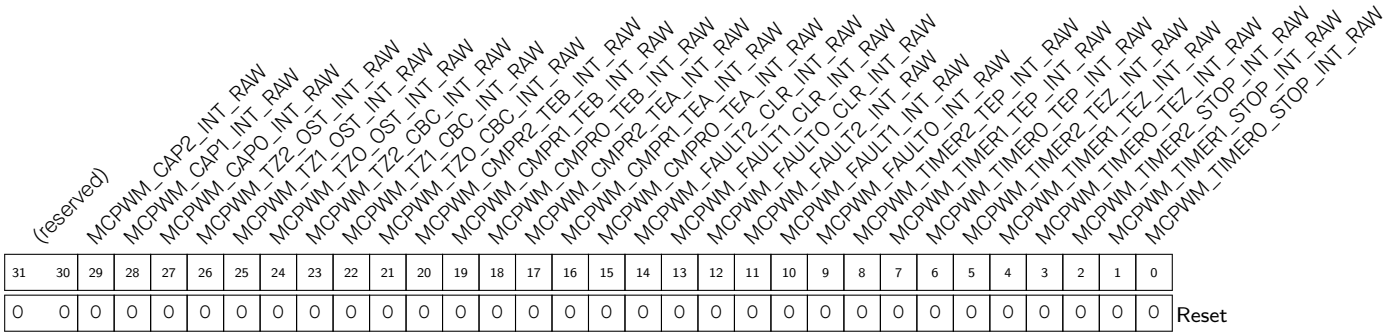
MCPWM_TZ2_OST_INT_ENA 写 1 使能 MCPWM_TZ2_OST_INT。(R/W)

MCPWM_CAP0_INT_ENA 写 1 使能 MCPWM_CAP0_INT。(R/W)

MCPWM_CAP1_INT_ENA 写 1 使能 MCPWM_CAP1_INT。(R/W)

MCPWM_CAP2_INT_ENA 写 1 使能 MCPWM_CAP2_INT。(R/W)

Register 24.30. MCPWM_INT_RAW_REG (0x0114)



MCPWM_TIMER0_STOP_INT_RAW MCPWM_TIMER0_STOP_INT 的原始状态位。(R/WTC/SS)

MCPWM_TIMER1_STOP_INT_RAW MCPWM_TIMER1_STOP_INT 的原始状态位。(R/WTC/SS)

MCPWM_TIMER2_STOP_INT_RAW MCPWM_TIMER2_STOP_INT 的原始状态位。(R/WTC/SS)

MCPWM_TIMER0_TEZ_INT_RAW MCPWM_TIMER0_TEZ_INT 的原始状态位。(R/WTC/SS)

MCPWM_TIMER1_TEZ_INT_RAW MCPWM_TIMER1_TEZ_INT 的原始状态位。(R/WTC/SS)

MCPWM_TIMER2_TEZ_INT_RAW MCPWM_TIMER2_TEZ_INT 的原始状态位。(R/WTC/SS)

MCPWM_TIMER0_TEP_INT_RAW MCPWM_TIMER0_TEP_INT 的原始状态位。(R/WTC/SS)

MCPWM_TIMER1_TEP_INT_RAW MCPWM_TIMER1_TEP_INT 的原始状态位。(R/WTC/SS)

MCPWM_TIMER2_TEP_INT_RAW MCPWM_TIMER2_TEP_INT 的原始状态位。(R/WTC/SS)

MCPWM_FAULT0_INT_RAW MCPWM_FAULT0_INT 的原始状态位。(R/WTC/SS)

MCPWM_FAULT1_INT_RAW MCPWM_FAULT1_INT 的原始状态位。(R/WTC/SS)

MCPWM_FAULT2_INT_RAW MCPWM_FAULT2_INT 的原始状态位。(R/WTC/SS)

MCPWM_FAULT0_CLR_INT_RAW MCPWM_FAULT0_CLR_INT 的原始状态位。(R/WTC/SS)

MCPWM_FAULT1_CLR_INT_RAW MCPWM_FAULT1_CLR_INT 的原始状态位。(R/WTC/SS)

MCPWM_FAULT2_CLR_INT_RAW MCPWM_FAULT2_CLR_INT 的原始状态位。(R/WTC/SS)

见下页...

Register 24.30. MCPWM_INT_RAW_REG (0x0114)

... 接上页

MCPWM_CMPR0_TEA_INT_RAW MCPWM_CMPR0_TEA_INT 的原始状态位。(R/WTC/SS)

MCPWM_CMPR1_TEA_INT_RAW MCPWM_CMPR1_TEA_INT 的原始状态位。(R/WTC/SS)

MCPWM_CMPR2_TEA_INT_RAW MCPWM_CMPR2_TEA_INT 的原始状态位。(R/WTC/SS)

MCPWM_CMPR0_TEB_INT_RAW MCPWM_CMPR0_TEB_INT 的原始状态位。(R/WTC/SS)

MCPWM_CMPR1_TEB_INT_RAW MCPWM_CMPR1_TEB_INT 的原始状态位。(R/WTC/SS)

MCPWM_CMPR2_TEB_INT_RAW MCPWM_CMPR2_TEB_INT 的原始状态位。(R/WTC/SS)

MCPWM_TZ0_CBC_INT_RAW MCPWM_TZ0_CBC_INT 的原始状态位。(R/WTC/SS)

MCPWM_TZ1_CBC_INT_RAW MCPWM_TZ1_CBC_INT 的原始状态位。(R/WTC/SS)

MCPWM_TZ2_CBC_INT_RAW MCPWM_TZ2_CBC_INT 的原始状态位。(R/WTC/SS)

MCPWM_TZ0_OST_INT_RAW MCPWM_TZ0_OST_INT 的原始状态位。(R/WTC/SS)

MCPWM_TZ1_OST_INT_RAW MCPWM_TZ1_OST_INT 的原始状态位。(R/WTC/SS)

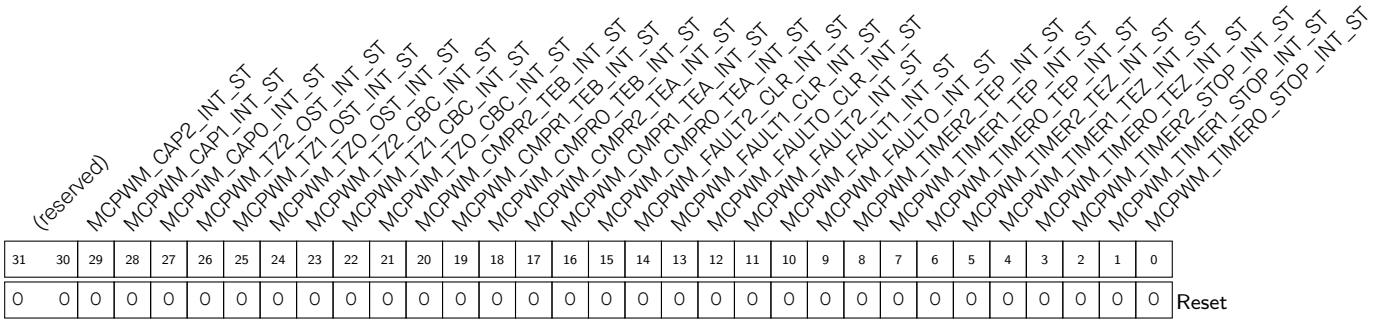
MCPWM_TZ2_OST_INT_RAW MCPWM_TZ2_OST_INT 的原始状态位。(R/WTC/SS)

MCPWM_CAP0_INT_RAW MCPWM_CAP0_INT 的原始状态位。(R/WTC/SS)

MCPWM_CAP1_INT_RAW MCPWM_CAP1_INT 的原始状态位。(R/WTC/SS)

MCPWM_CAP2_INT_RAW MCPWM_CAP2_INT 的原始状态位。(R/WTC/SS)

Register 24.31. MCPWM_INT_ST_REG (0x0118)



MCPWM_TIMER0_STOP_INT_ST MCPWM_TIMER0_STOP_INT 的屏蔽状态位。(RO)

MCPWM_TIMER1_STOP_INT_ST MCPWM_TIMER1_STOP_INT 的屏蔽状态位。(RO)

MCPWM_TIMER2_STOP_INT_ST MCPWM_TIMER2_STOP_INT 的屏蔽状态位。(RO)

MCPWM_TIMER0_TEZ_INT_ST MCPWM_TIMER0_TEZ_INT 的屏蔽状态位。(RO)

MCPWM_TIMER1_TEZ_INT_ST MCPWM_TIMER1_TEZ_INT 的屏蔽状态位。(RO)

MCPWM_TIMER2_TEZ_INT_ST MCPWM_TIMER2_TEZ_INT 的屏蔽状态位。(RO)

MCPWM_TIMER0_TEP_INT_ST MCPWM_TIMER0_TEP_INT 的屏蔽状态位。(RO)

MCPWM_TIMER1_TEP_INT_ST MCPWM_TIMER1_TEP_INT 的屏蔽状态位。(RO)

MCPWM_TIMER2_TEP_INT_ST MCPWM_TIMER2_TEP_INT 的屏蔽状态位。(RO)

MCPWM_FAULT0_INT_ST MCPWM_FAULT0_INT_INT 的屏蔽状态位。(RO)

MCPWM_FAULT1_INT_ST MCPWM_FAULT1_INT_INT 的屏蔽状态位。(RO)

MCPWM_FAULT2_INT_ST MCPWM_FAULT2_INT_INT 的屏蔽状态位。(RO)

MCPWM_FAULT0_CLR_INT_ST MCPWM_FAULT0_CLR_INT 的屏蔽状态位。(RO)

MCPWM_FAULT1_CLR_INT_ST MCPWM_FAULT1_CLR_INT 的屏蔽状态位。(RO)

MCPWM_FAULT2_CLR_INT_ST MCPWM_FAULT2_CLR_INT 的屏蔽状态位。(RO)

MCPWM_CMPR0_TEA_INT_ST MCPWM_CMPR0_TEA_INT 的屏蔽状态位。(RO)

MCPWM_CMPR1_TEA_INT_ST MCPWM_CMPR1_TEA_INT 的屏蔽状态位。(RO)

MCPWM_CMPR2_TEA_INT_ST MCPWM_CMPR2_TEA_INT 的屏蔽状态位。(RO)

见下页...

Register 24.31. MCPWM_INT_ST_REG (0x0118)

... 接上页

MCPWM_CMPR0_TEB_INT_ST MCPWM_CMPR0_TEB_INT 的屏蔽状态位。(RO)

MCPWM_CMPR1_TEB_INT_ST MCPWM_CMPR1_TEB_INT 的屏蔽状态位。(RO)

MCPWM_CMPR2_TEB_INT_ST MCPWM_CMPR2_TEB_INT 的屏蔽状态位。(RO)

MCPWM_TZ0_CBC_INT_ST MCPWM_TZ0_CBC_INT 的屏蔽状态位。(RO)

MCPWM_TZ1_CBC_INT_ST MCPWM_TZ1_CBC_INT 的屏蔽状态位。(RO)

MCPWM_TZ2_CBC_INT_ST MCPWM_TZ2_CBC_INT 的屏蔽状态位。(RO)

MCPWM_TZ0_OST_INT_ST MCPWM_TZ0_OST_INT 的屏蔽状态位。(RO)

MCPWM_TZ1_OST_INT_ST MCPWM_TZ1_OST_INT 的屏蔽状态位。(RO)

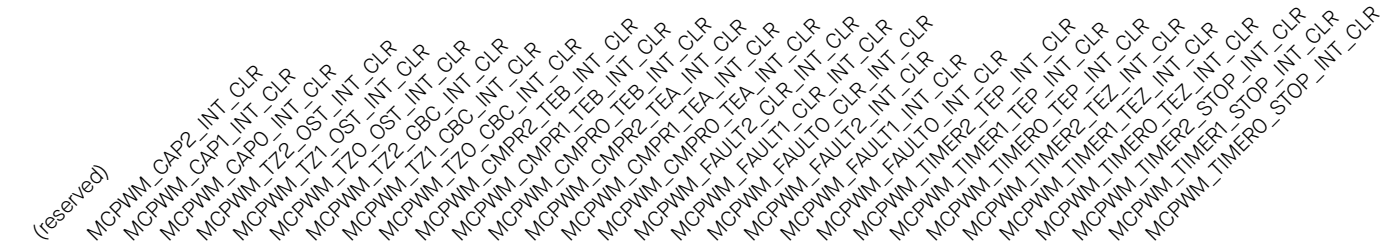
MCPWM_TZ2_OST_INT_ST MCPWM_TZ2_OST_INT 的屏蔽状态位。(RO)

MCPWM_CAP0_INT_ST MCPWM_CAP0_INT 的屏蔽状态位。(RO)

MCPWM_CAP1_INT_ST MCPWM_CAP1_INT 的屏蔽状态位。(RO)

MCPWM_CAP2_INT_ST MCPWM_CAP2_INT 的屏蔽状态位。(RO)

Register 24.32. MCPWM_INT_CLR_REG (0x011C)



MCPWM_TIMER0_STOP_INT_CLR 写 1 清除 MCPWM_TIMER0_STOP_INT。(WT)

MCPWM_TIMER1_STOP_INT_CLR 写 1 清除 MCPWM_TIMER1_STOP_INT。(WT)

MCPWM_TIMER2_STOP_INT_CLR 写 1 清除 MCPWM_TIMER2_STOP_INT。(WT)

MCPWM_TIMER0_TEZ_INT_CLR 写 1 清除 MCPWM_TIMER0_TEZ_INT。(WT)

MCPWM_TIMER1_TEZ_INT_CLR 写 1 清除 MCPWM_TIMER1_TEZ_INT。(WT)

MCPWM_TIMER2_TEZ_INT_CLR 写 1 清除 MCPWM_TIMER2_TEZ_INT。(WT)

MCPWM_TIMER0_TEP_INT_CLR 写 1 清除 MCPWM_TIMER0_TEP_INT。(WT)

MCPWM_TIMER1_TEP_INT_CLR 写 1 清除 MCPWM_TIMER1_TEP_INT。(WT)

MCPWM_TIMER2_TEP_INT_CLR 写 1 清除 MCPWM_TIMER2_TEP_INT。(WT)

MCPWM_FAULT0_INT_CLR 写 1 清除 MCPWM_FAULT0_INT。(WT)

MCPWM_FAULT1_INT_CLR 写 1 清除 MCPWM_FAULT1_INT。(WT)

MCPWM_FAULT2_INT_CLR 写 1 清除 MCPWM_FAULT2_INT。(WT)

MCPWM_FAULT0_CLR_INT_CLR 写 1 清除 MCPWM_FAULT0_CLR_INT。(WT)

MCPWM_FAULT1_CLR_INT_CLR 写 1 清除 MCPWM_FAULT1_CLR_INT。(WT)

MCPWM_FAULT2_CLR_INT_CLR 写 1 清除 MCPWM_FAULT2_CLR_INT。(WT)

见下页...

Register 24.32. MCPWM_INT_CLR_REG (0x011C)

... 接上页

MCPWM_CMPR0_TEA_INT_CLR 写 1 清除 MCPWM_CMPR0_TEA_INT。(WT)

MCPWM_CMPR1_TEA_INT_CLR 写 1 清除 MCPWM_CMPR1_TEA_INT。(WT)

MCPWM_CMPR2_TEA_INT_CLR 写 1 清除 MCPWM_CMPR2_TEA_INT。(WT)

MCPWM_CMPR0_TEB_INT_CLR 写 1 清除 MCPWM_CMPR0_TEB_INT。(WT)

MCPWM_CMPR1_TEB_INT_CLR 写 1 清除 MCPWM_CMPR1_TEB_INT。(WT)

MCPWM_CMPR2_TEB_INT_CLR 写 1 清除 MCPWM_CMPR2_TEB_INT。(WT)

MCPWM_TZ0_CBC_INT_CLR 写 1 清除 MCPWM_TZ0_CBC_INT。(WT)

MCPWM_TZ1_CBC_INT_CLR 写 1 清除 MCPWM_TZ1_CBC_INT。(WT)

MCPWM_TZ2_CBC_INT_CLR 写 1 清除 MCPWM_TZ2_CBC_INT。(WT)

MCPWM_TZ0_OST_INT_CLR 写 1 清除 MCPWM_TZ0_OST_INT。(WT)

MCPWM_TZ1_OST_INT_CLR 写 1 清除 MCPWM_TZ1_OST_INT。(WT)

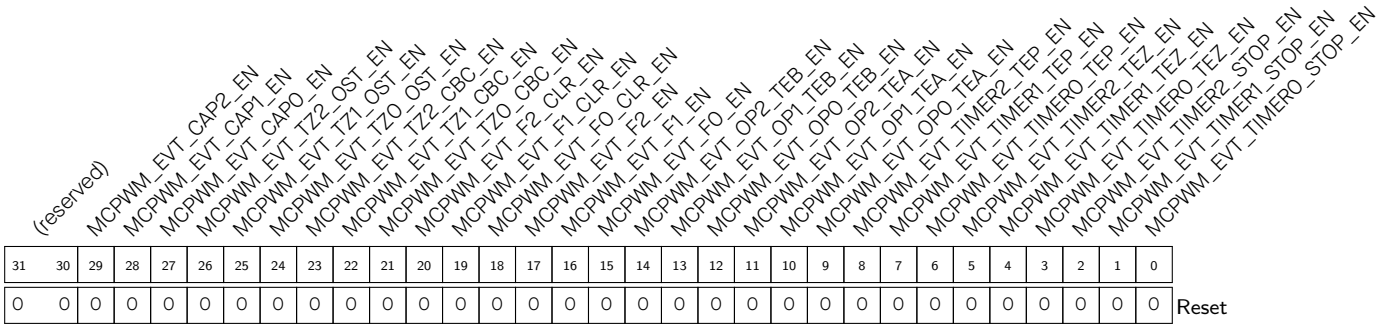
MCPWM_TZ2_OST_INT_CLR 写 1 清除 MCPWM_TZ2_OST_INT。(WT)

MCPWM_CAP0_INT_CLR 写 1 清除 MCPWM_CAP0_INT。(WT)

MCPWM_CAP1_INT_CLR 写 1 清除 MCPWM_CAP1_INT。(WT)

MCPWM_CAP2_INT_CLR 写 1 清除 MCPWM_CAP2_INT。(WT)

Register 24.33. MCPWM_EVT_EN_REG (0x0120)



MCPWM_EVT_TIMER0_STOP_EN 配置是否使能当定时器 0 停止时的事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TIMER1_STOP_EN 配置是否使能当定时器 1 停止时的事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TIMER2_STOP_EN 配置是否使能当定时器 2 停止时的事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TIMER0_TEZ_EN 配置是否使能当定时器 0 等于零时的事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TIMER1_TEZ_EN 配置是否使能当定时器 1 等于零时的事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TIMER2_TEZ_EN 配置是否使能当定时器 2 等于零时的事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TIMER0_TEP_EN 配置是否使能当定时器 0 等于周期时的事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TIMER1_TEP_EN 配置是否使能当定时器 1 等于周期时的事件生成。

0：关闭

1：使能

(R/W)

见下页...

Register 24.33. MCPWM_EVT_EN_REG (0x0120)

... 接上页

MCPWM_EVT_TIMER2_TEP_EN 配置是否使能当定时器 2 等于周期时的事件生成。

0：关闭
1：使能
(R/W)

MCPWM_EVT_OP0_TEA_EN 配置是否使能当生成器 0 等于时间戳 A 时的事件生成。

0：关闭
1：使能
(R/W)

MCPWM_EVT_OP1_TEA_EN 配置是否使能当生成器 1 等于时间戳 A 时的事件生成。

0：关闭
1：使能
(R/W)

MCPWM_EVT_OP2_TEA_EN 配置是否使能当生成器 2 等于时间戳 A 时的事件生成。

0：关闭
1：使能
(R/W)

MCPWM_EVT_OP0_TEB_EN 配置是否使能当生成器 0 等于时间戳 B 时的事件生成。

0：关闭
1：使能
(R/W)

MCPWM_EVT_OP1_TEB_EN 配置是否使能当生成器 1 等于时间戳 B 时的事件生成。

0：关闭
1：使能
(R/W)

MCPWM_EVT_OP2_TEB_EN 配置是否使能当生成器 2 等于时间戳 B 时的事件生成。

0：关闭
1：使能
(R/W)

MCPWM_EVT_F0_EN 配置是否使能 FAULT0 事件生成。

0：关闭
1：使能
(R/W)

MCPWM_EVT_F1_EN 配置是否使能 FAULT1 事件生成。

0：关闭
1：使能
(R/W)

见下页...

Register 24.33. MCPWM_EVT_EN_REG (0x0120)

... 接上页

MCPWM_EVT_F2_EN 配置是否使能 FAULT2 事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_F0_CLR_EN 配置是否使能 FAULT0 清除事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_F1_CLR_EN 配置是否使能 FAULT0 清除事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_F2_CLR_EN 配置是否使能 FAULT0 清除事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TZ0_CBC_EN 配置是否使能 CBC0 事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TZ1_CBC_EN 配置是否使能 CBC1 事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TZ2_CBC_EN 配置是否使能 CBC2 事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TZ0_OST_EN 配置是否使能 OST0 事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_TZ1_OST_EN 配置是否使能 OST1 事件生成。

0：关闭

1：使能

(R/W)

见下页...

Register 24.33. MCPWM_EVT_EN_REG (0x0120)

... 接上页

MCPWM_EVT_TZ2_OST_EN 配置是否使能 OST2 事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_CAP0_EN 配置是否使能 capture0 事件生成。

0：关闭

1：使能

(R/W)

MCPWM_EVT_CAP1_EN 配置是否使能 capture1 事件生成。

0：关闭

1：使能

(R/W)

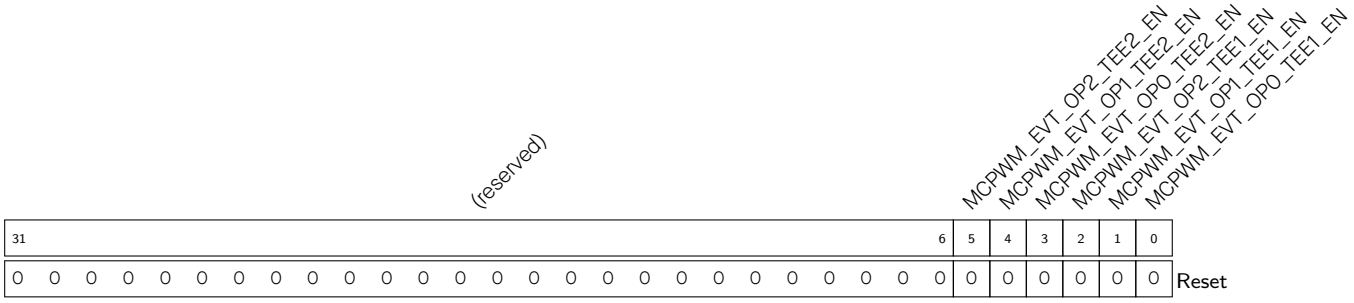
MCPWM_EVT_CAP2_EN 配置是否使能 capture2 事件生成。

0：关闭

1：使能

(R/W)

Register 24.34. MCPWM_EVT_EN2_REG (0x0128)



MCPWM_EVT_OP*n*_TEE1_EN 配置是否在 PWM 生成器计数器的值等于 MCPWM_OP*n*_TSTMP_E1_REG 寄存器的值时，生成 MCPWM_EVT_OP*n*_TEE1 事件。

0：不生成

1：生成

(R/W)

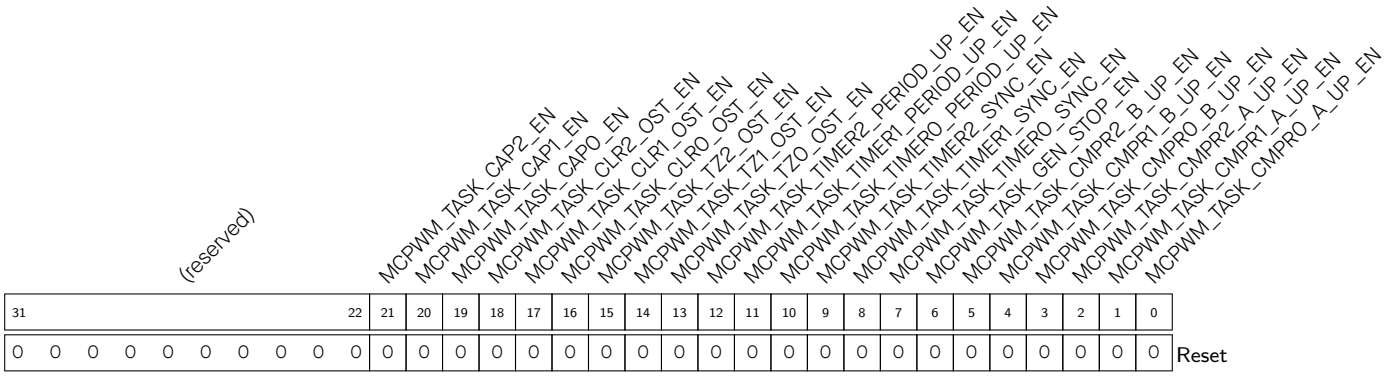
MCPWM_EVT_OP*n*_TEE2_EN 配置是否在 PWM 生成器计数器的值等于 MCPWM_OP*n*_TSTMP_E2_REG 寄存器的值时，生成 MCPWM_EVT_OP*n*_TEE2 事件。

0：不生成

1：生成

(R/W)

Register 24.35. MCPWM_TASK_EN_REG (0x0124)



MCPWM_TASK_CMPR0_A_UP_EN 配置是否接收 PWM 生成器 0 时间戳 A 的影子寄存器的更新任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_CMPR1_A_UP_EN 配置是否接收 PWM 生成器 1 时间戳 A 的影子寄存器的更新任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_CMPR2_A_UP_EN 配置是否接收 PWM 生成器 2 时间戳 A 的影子寄存器的更新任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_CMPR0_B_UP_EN 配置是否接收 PWM 生成器 0 时间戳 B 的影子寄存器的更新任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_CMPR1_B_UP_EN 配置是否接收 PWM 生成器 1 时间戳 B 的影子寄存器的更新任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_CMPR2_B_UP_EN 配置是否接收 PWM 生成器 2 时间戳 B 的影子寄存器的更新任务。

0：无效

1：接收

(R/W)

见下页...

Register 24.35. MCPWM_TASK_EN_REG (0x0124)

... 接上页

MCPWM_TASK_GEN_STOP_EN 配置是否接收所有 PWM 生成器的停止任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_TIMER0_SYNC_EN 配置是否接收定时器 0 的同步任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_TIMER1_SYNC_EN 配置是否接收定时器 1 的同步任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_TIMER2_SYNC_EN 配置是否接收定时器 2 的同步任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_TIMER0_PERIOD_UP_EN 配置是否接收定时器 0 的周期更新任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_TIMER1_PERIOD_UP_EN 配置是否接收定时器 1 的周期更新任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_TIMER2_PERIOD_UP_EN 配置是否接收定时器 2 的周期更新任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_TZ0_OST_EN 配置是否接收 OST0 任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_TZ1_OST_EN 配置是否接收 OST1 任务。

0：无效

1：接收

(R/W)

见下页...

Register 24.35. MCPWM_TASK_EN_REG (0x0124)

... 接上页

MCPWM_TASK_TZ2_OST_EN 配置是否接收 OST2 任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_CLR0_OST_EN 配置是否接收 OST0 清除任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_CLR1_OST_EN 配置是否接收 OST1 清除任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_CLR2_OST_EN 配置是否接收 OST2 清除任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_CAP0_EN 配置是否接收 capture0 任务。

0：无效

1：接收

(R/W)

MCPWM_TASK_CAP1_EN 配置是否接收 capture1 任务。

0：无效

1：接收

(R/W)

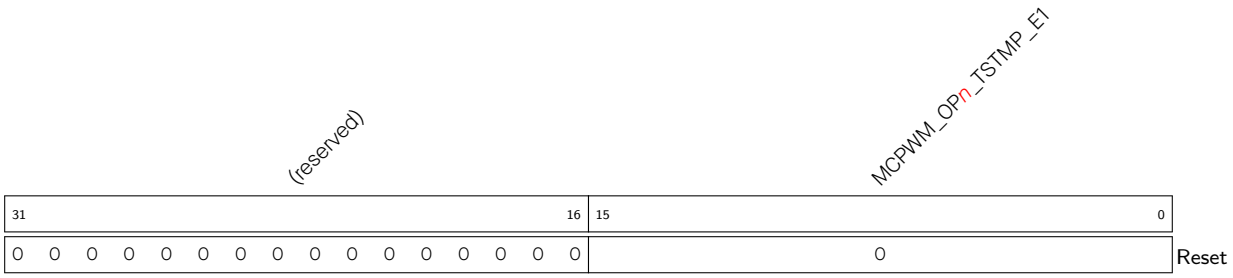
MCPWM_TASK_CAP2_EN 配置是否接收 capture2 任务。

0：无效

1：接收

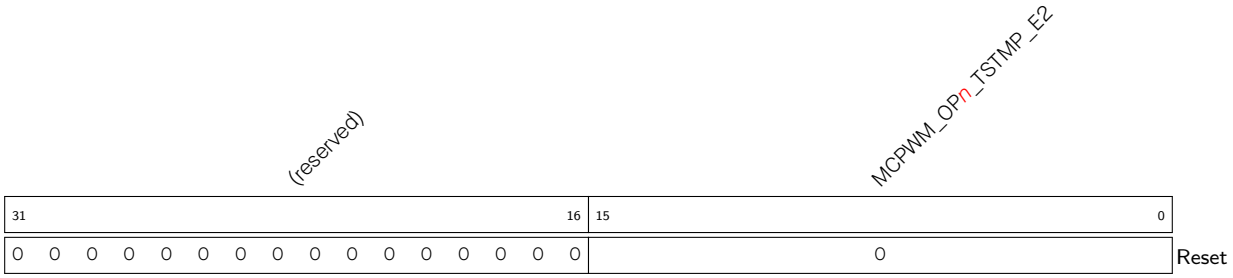
(R/W)

Register 24.36. MCPWM_OP*n*_TSTMP_E1_REG (*n*: 0-2) (0x012C+0x8**n*)



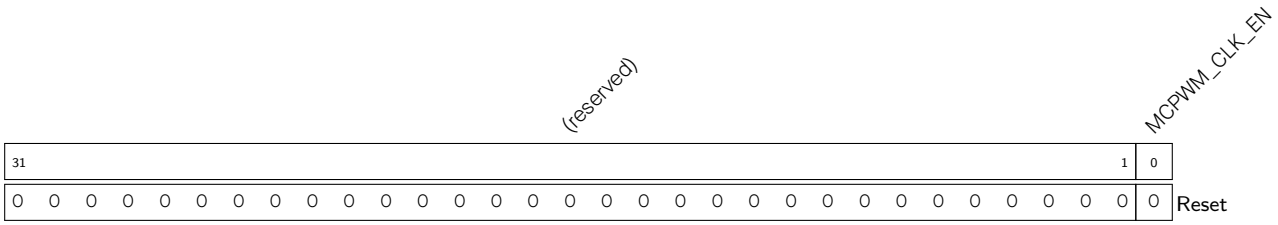
MCPWM_OP*n*_TSTMP_E1 配置生成器*n* 时间戳 E1 值。(R/W)

Register 24.37. MCPWM_OP*n*_TSTMP_E2_REG(*n*: 0-2) (0x0130+0x8**n*)



MCPWM_OP*n*_TSTMP_E2 配置生成器*n* 时间戳 E2 值。(R/W)

Register 24.38. MCPWM_CLK_REG (0x0144)



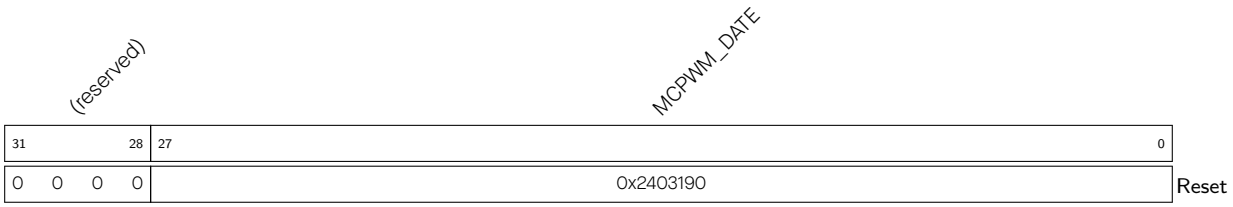
MCPWM_CLK_EN 配置是否强制使能时钟。

0：无效

1：强制使能时钟

(R/W)

Register 24.39. MCPWM_VERSION_REG (0x0148)



MCPWM_DATE 版本控制寄存器。(R/W)

第 25 章

红外遥控 (RMT)

25.1 概述

红外遥控 (RMT) 是一个红外发送和接收控制器，可通过软件编解码多种红外协议。RMT 模块可以实现将模块内置 RAM 中的脉冲编码转换为信号输出，或将模块的输入信号转换为脉冲编码存入 RAM 中。此外，RMT 模块也支持对输出信号进行载波调制或对输入信号进行解调和滤波处理。

RMT 共有八个通道，编码为 0 ~ 7，各通道可独立用于发送或接收信号：

- 0 ~ 3 通道专门用于发送信号；
- 4 ~ 7 通道专门用于接收信号。

每个发送通道和接收通道分别有一组功能相同的寄存器。另外，发送通道 3 和接收通道 7 对应的 RAM 支持 GDMA 访问，因此还有 GDMA 相关的控制和状态寄存器。为了方便叙述，以 *n* 表示各个发送通道，以 *m* 表示各个接收通道。

25.2 主要特性

RMT 控制器具有如下特性：

- 共配置八个通道：
  - 0 ~ 3 通道支持发送
  - 4 ~ 7 通道支持接收
  - 八个通道共享 384 x 32 位的 RAM
- 发射器支持以下模式：
  - 普通发送模式
  - 乒乓发送模式
  - 持续发送模式
  - 载波调制
  - 多通道同时发送
  - 发送通道 3 支持 GDMA 访问
- 接收器支持以下模式：
  - 普通接收模式
  - 乒乓接收模式

- 接收滤波
- 载波解调
- 接收通道 7 支持 GDMA 访问

25.3 功能描述

25.3.1 RMT 架构

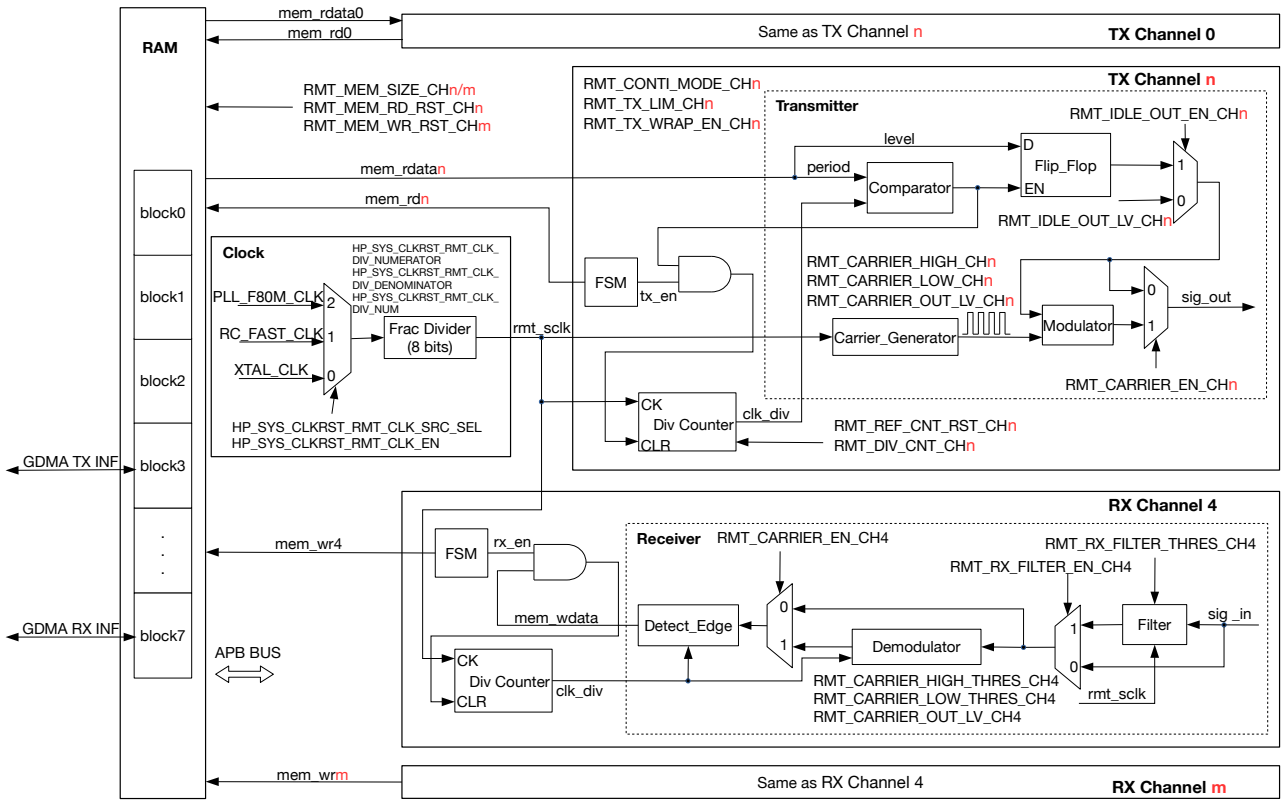


图 25.3-1. RMT 结构框图

如图 25.3-1 所示，每个发送通道内部各有：

- 一个时钟分频计数器 (Div Counter)
- 一个状态机 (FSM)
- 一个发射器 (Transmitter)

每个接收通道内部也各有：

- 一个时钟分频计数器 (Div Counter)
- 一个状态机 (FSM)
- 一个接收器 (Receiver)

八个通道共享一块 384 x 32 位的 RAM。

25.3.2 RAM

25.3.2.1 RAM 结构

RAM 中脉冲编码结构如图 25.3-2 所示。每个脉冲编码为 16 位，由 level 与 period 两部分组成。其中 level 表示输入或输出信号的逻辑电平值（0 或 1），period 表示该电平信号持续的时钟（见图 25.3-1 中的 clk_div）周期数。

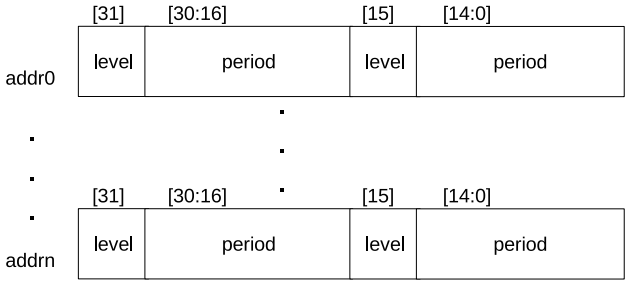


图 25.3-2. RAM 中脉冲编码结构

period 的最小值为 0，period 为 0 是一次传输的结束标志。对于非 0 的 period，即非结束标志的 period，其值需要满足与 APB 时钟和 RMT 时钟相关的限制条件，具体请参考以下公式：

$$5 \times T_{apb_clk} + 6 \times T_{rmt_sclk} < period \times T_{clk_div} \tag{25.1}$$

说明：

RMT 时钟与 APB 时钟需满足以下关系：

$$1.5 \times T_{apb_clk} < 9 \times T_{rmt_sclk} \tag{25.2}$$

RMT 可捕获的脉冲宽度，即 $period \times T_{clk_div}$ 受上述公式以及 rmt_sclk 时钟频率的限制。具体如下：

- 最小脉冲长度应大于 $(5 \times T_{apb_clk} + 6 \times T_{rmt_sclk})$；
- 最大脉冲长度应小于或等于（最大 period × 最大 T_{clk_div}），即 $((2^{15} - 1) \times$ 最大 $T_{rmt_sclk} \times 256)$。

更多有关 rmt_sclk 时钟频率，或 APB_CLK 时钟频率，请参考小节 25.3.3，或章节 7 *复位和时钟 [to be added later]*。

25.3.2.2 RAM 使用说明

RAM 按照 48 x 32 位分成八个 block。默认情况下每个通道只能使用一个 block（固定为通道 0 使用 block 0，通道 1 使用 block 1，以此类推）。

当发送通道 *n* 或接收通道 *m* 单次传输的脉冲编码数大于一个 block 时，可以：

- 置位 RMT_MEM_TX/RX_WRAP_EN_CH*n*/*m* 使能乒乓操作；
- 或通过配置 RMT_MEM_SIZE_CH*n*/*m* 寄存器，允许该通道占用多个 block。

当设置 RMT_MEM_SIZE_CH*n*/*m* > 1 时，通道 *n*/*m* 将占用 block (*n*/*m*) ~ block (*n*/*m* + RMT_MEM_SIZE_CH*n*/*m* -1) 的存储空间。通道 *n*/*m* + 1 ~ *n*/*m* + RMT_MEM_SIZE_CH*n*/*m* - 1 因为对应的 RAM block 被占用而无法使用。例如，如果通道 0 配置使用了 block 0 和 block 1，则通道 1 无法使用，通道 2 和通道 3 不受影响。

注意，每个通道使用 RAM 的空间是根据地址从低到高进行映射的，因此通道 0 可以通过配置 RMT_MEM_SIZE_CH0 寄存器来使用通道 1、2、3、...、7 的 RAM 空间，但是通道 7 不能使用通道 0、1、2、...、6 的 RAM 空间。因此 RMT_MEM_SIZE_CH*n* 不应超过 (8 - *n*)，RMT_MEM_SIZE_CH*m* 不应超过 (8 - *m*)。

RAM 可被 APB 总线及通道的发射器或接收器访问，为了防止接收通道访问 RAM 和 APB 访问时发生冲突，例如 APB 读 RAM 时，通道向 RAM 发起写操作，用户可以通过配置 RMT_MEM_OWNER_CH*m* 来决定当前 RAM 的使用权。当接收通道发生越权访问时会产生 RMT_MEM_OWNER_ERR_CH*m* 标志信号。

25.3.2.3 RAM 访问方式

APB 总线访问 RAM 有 FIFO 和 NONFIFO（直接地址）两种模式：

- RMT_APB_FIFO_MASK 置 0 时，选择 FIFO 模式；
- RMT_APB_FIFO_MASK 置 1 时，选择 NONFIFO 模式。

另外，通道 3 和通道 7 还分别支持 GDMA 访问方式。

FIFO 模式

在 FIFO 模式下，APB 通过固定地址 RMT_CH*n*/*m*DATA_REG 向 RAM 写数据或从 RAM 读数据。

NONFIFO 模式

在 NONFIFO 模式下，APB 向连续地址段写入数据或从连续地址段读出数据：

- 发送通道 *n* 对应的写地址段的首地址是：RMT 基地址 + 0x800 + *n*x48，第 2 个数据的访问地址是 RMT 基地址 + 0x800 + *n*x48 + 0x4，以此类推，后面的地址依次加上 0x4。
- 接收通道 *m* 对应的读地址段的首地址是：RMT 基地址 + 0x8C0 + *m*x48，第 2 个数据的访问地址是 RMT 基地址 + 0x8C0 + *m*x48 + 0x4，以此类推，后面的地址依次加上 0x4。

GDMA 模式

不同于其它几个发送通道，通道 3 还支持 GDMA 访问方式。如果将 RMT_DMA_ACCESS_EN_CH3 置 1，则通道 3 对应的 RAM 仅支持从 GDMA 取数。APB 对通道 3 的 FIFO 方式访问将被忽略，同时也不允许 APB 通过 NONFIFO 方式访问通道 3 的 RAM 区域，**否则会出现不可预计的后果**。

为保证传输数据的正确性，需要先启动 GDMA，在 GDMA 通道已经接收到数据后，再启动 RMT 发送。普通发送模式下，GDMA 写满通道 3 的 RAM 后会触发 RMT_APB_MEM_WR_ERR_CH3。使能 RMT_MEM_TX_WRAP_EN_CH3 后，不需要软件进行乒乓操作就可以连续发送大于一个 block 的数据，但要保证通道准备发送数据前 GDMA 已经将待发送数据准备好，否则可能会发出与期望不符的数据。

不同于其它几个接收通道，通道 7 还支持 GDMA 访问方式。如果将 RMT_DMA_ACCESS_EN_CH7 置 1，通道 7 对应的 RAM 支持向 GDMA 发送数据，同时也支持 APB 的 NONFIFO 方式访问。

普通接收模式下，GDMA 读到通道 7 RAM 大小的数据后就会触发 RMT_APB_MEM_RD_ERR_CH7，之后收到的数会被丢弃。使能 RMT_MEM_TX_WRAP_EN_CH7 后，不需要软件进行乒乓操作就可以连续接收大于一个 block 的数据。如果通道 7 的 RAM 接收满后 GDMA 还没有接收，新收到的数据会替换上一个数据。

> **说明：**
> 当通道 7 接收到结束标志时，会产生 GDMA 的 in_suc_eof 中断。如果是 period[14:0] 为 0，则会向 GDMA 写两个字节；如果是 period[30:16] 为 0，则会向 GDMA 写四个字节。

25.3.3 时钟

用户可以通过配置 HP_SYS_CLKRST_RMT_CLK_SRC_SEL 选择 RMT 的时钟源：PLL_F80M_CLK、RC_FAST_CLK 或 XTAL_CLK，配置 HP_SYS_CLKRST_RMT_CLK_EN 为高电平来打开 RMT 的时钟。选择后的时钟经过小数分频得到 RMT 的工作时钟（图 25.3-1 rmt_sclk），分频系数为 HP_SYS_CLKRST_RMT_CLK_DIV_NUM + 1 + HP_SYS_CLKRST_RMT_CLK_DIV_NUMERATOR/HP_SYS_CLKRST_RMT_CLK_DIV_DENOMINATOR。

更多信息，请参考章节 7 *复位和时钟 [to be added later]*。RMT_DIV_CNT_CH*n*/*m* 用于配置 RMT 通道内部的时钟分频器的分频系数，除 0 表示 256 分频外，其他分频数等同于 RMT_DIV_CNT_CH*n*/*m* 的值。时钟分频器可以通过配置 RMT_REF_CNT_RST_CH*n*/*m* 进行复位。时钟分频器的分频时钟可供计数器使用，见图 25.3-1。

25.3.4 发射器

说明：
本小节以及后续小节所述的配置，均需要通过置位 RMT_CONF_UPDATE_CH*n*/*m* 的方式来进行更新，详情见第 25.3.6 小节。

25.3.4.1 普通发送模式

当 RMT_TX_START_CH*n* 置为 1 时，通道 *n* 的发射器开始从通道对应 RAM block 的起始地址，按照地址从低到高依次读取脉冲编码进行发送。当遇到结束标志（period 等于 0）时，发射器将结束发送返回空闲状态，并产生 RMT_CH*n*_TX_END_INT 中断。配置 RMT_TX_STOP_CH*n* 可以使发射器立刻停止发送并进入空闲状态。发射器空闲状态发送的电平由结束标志中的 level 段或者是 RMT_IDLE_OUT_LV_CH*n* 决定。用户可以配置 RMT_IDLE_OUT_EN_CH*n* 来选择这两种方式：

- 清除 RMT_IDLE_OUT_EN_CH*n*，发射器空闲状态发送的电平由结束标志中的 level 段决定；
- 置位 RMT_IDLE_OUT_EN_CH*n*，发射器空闲状态发送的电平由 RMT_IDLE_OUT_LV_CH*n* 决定。

25.3.4.2 乒乓发送模式

当发送的脉冲编码较多时，可通过置位 RMT_MEM_TX_WRAP_EN_CH*n* 使能通道 *n* 的乒乓模式。在乒乓操作模式下，发射器会循环从通道对应的 RAM 区域取出脉冲编码进行发送，直到遇到结束标识为止。例如，当 RMT_MEM_SIZE_CH*n* = 1 时，发射器将从 48 * *n* 地址开始发送，然后对应 RAM 的地址递增。发完 (48 * (*n* + 1) - 1) 地址的数据后，下次继续从 48 * *n* 地址开始递增发送数据，依此类推，遇到结束标识时停止发送。RMT_MEM_SIZE_CH*n* > 1 的情形下，乒乓操作同样适用。

每当发射器发送的脉冲编码数大于等于 RMT_TX_LIM_CH*n* 时，会产生 RMT_CH*n*_TX_THR_EVENT_INT 中断。在乒乓模式下，可以设置 RMT_TX_LIM_CH*n* 为每个通道对应 RAM 空间的一半或几分之一。软件在检测到 RMT_CH*n*_TX_THR_EVENT_INT 中断之后，可以更新已使用过的 RAM 区域的脉冲编码，从而实现乒乓操作。

说明：
当 RAM 采用 GDMA 方式访问时，不需要额外操作就能支持多于一个 RAM 的脉冲编码发送。而采用 APB 方式访问时，需要软件进行乒乓操作。

25.3.4.3 发送加载波

此外，发射器还可以对输出信号进行载波调制，置位 RMT_CARRIER_EN_CH*n* 可以使能该功能。载波的波形可配置。一个载波周期中高电平持续时间为 (RMT_CARRIER_HIGH_CH*n* + 1) 个 rmt_sclk 时钟周期，低电平持续的时间为 (RMT_CARRIER_LOW_CH*n* + 1) 个 rmt_sclk 时钟周期。置位 RMT_CARRIER_OUT_LV_CH*n* 时在输出信号高电平上加载波信号，清零 RMT_CARRIER_OUT_LV_CH*n* 时在输出信号低电平上加载波信号。同时，在进行载波调制时，载波可以一直加载在输出信号上，也可以仅加载在有效的脉冲编码（RAM 中的数据）上。通过配置 RMT_CARRIER_EFF_EN_CH*n*，可以选择这两种模式。RMT_CARRIER_EFF_EN_CH*n* 设置为 0 时在所有信号上加载波，设置为 1 时在有效信号上加载波。

25.3.4.4 持续发送模式

置位 RMT_TX_CONTI_MODE_CH*n* 可以使能发射器的持续发送功能。置位后，发射器会循环发送 RAM 中的脉冲编码。

持续发送模式下，

- 如果遇到结束标志，则重新从该通道 RAM 中的第一个数据开始发送；
- 如果没有结束标志，会在发送到最后一个数据处回卷，重新开始发送第一个数据。

置位 RMT_TX_LOOP_CNT_EN_CH*n* 后，发射器每遇到一次结束标志，循环发送的次数会加 1。当该次数达到 RMT_TX _LOOP_NUM_CH*n* 设定的值时，会产生 RMT_CH*n*_TX_LOOP_INT 中断。如果置位 RMT_LOOP_STOP_EN_CH*n*，则发送会在产生 RMT_CH*n*_TX_LOOP_INT 中断后立即停止，否则会继续发送。持续发送模式下，如果遇到的结束标志类型是 period[14:0] 为 0，那么这个结束标志前一个数据的 period 需要满足：

$$10 \times T_{apb_clk} + 19 \times T_{rmt_sclk} < period \times T_{clk_div} \tag{25.3}$$

而其它数据的 period，满足非 0 period 关系式 即可。

25.3.4.5 多通道同时发送

RMT 模块支持多通道同时发送，具体配置步骤如下所示：

- 首先配置 RMT_TX_SIM_CH*n* 用于选择同步发送的通道；
- 然后置位 RMT_TX_SIM_EN 使能发射器多个通道同步发送的功能；
- 最后将所选同步发送通道的 RMT_TX_START_CH*n* 置为 1。

当最后一个通道完成配置时，此时多个通道会同时启动发送。另外，RMT 模块还支持通道 0 ~ 2 的 RAM 采用 APB 访问和通道 3 的 RAM 采用 GDMA 方式访问下的多通道同时发送。

25.3.5 接收器

25.3.5.1 普通接收模式

RMT_RX_EN_CH*m* 置为 1 时接收器开始工作，置为 0 时会停止接收。接收器会从信号的第一个跳变沿开始计数，并检测信号电平及其持续的时钟周期数，将其按照脉冲编码的格式存入 RAM 中。当信号在一个电平下持续的时钟周期数超过 RMT_IDLE_THRES_CH*m* 时，接收器结束接收过程，返回空闲状态，并产生 RMT_CH*m*_RX_END_INT 中断。RMT_IDLE_THRES_CH*m* 的值需要大于输入信号的高电平或低电平的最大时钟

周期，否则接收器会将该信号误判为空闲信号而进入空闲状态。当接收数据存满了接收通道设置的 RAM 空间时，会停止接收，并产生 RMT_CH*m*_ERR_INT 中断（由 RAM 满事件触发）。

25.3.5.2 乒乓接收模式

当接收的脉冲编码较多时，可通过置位 RMT_MEM_RX_WRAP_EN_CH*m* 使能通道 *m* 的乒乓模式。当 RAM 采用 GDMA 方式访问时，不需要额外操作就能支持多于一个 RAM 的脉冲编码接收。而采用 APB 方式访问时，需要软件进行乒乓操作。在乒乓操作模式下，接收器会将接收到的脉冲编码循环存入通道对应的 RAM 区域，当信号在一个电平下持续的时钟周期数超过 RMT_IDLE_THRES_CH*m* 时，接收器结束接收过程，返回空闲状态，并产生 RMT_CH*m*_RX_END_INT 中断。例如，当 RMT_MEM_SIZE_CH*m* = 1 时，接收器将从 48 * *m* 地址开始接收，然后对应 RAM 的地址递增。收完 (48 * (*m* +1) - 1) 地址的数据后，下次继续从 48 * *m* 地址开始递增接收数据，以此类推，遇到信号在一个电平下持续的时钟周期数超过 RMT_IDLE_THRES_CH*m* 时停止接收。RMT_MEM_SIZE_CH*m* > 1 的情形下，乒乓操作同样适用。

每当接收器接收的脉冲编码数大于或等于 RMT_CH*m*_RX_LIM_REG 时，会产生 RMT_CH*m*_RX_THR_EVENT_INT 中断。在乒乓模式下，可以设置 RMT_CH*m*_RX_LIM_REG 为每个通道对应 RAM 空间的一半或几分之一。软件在检测到 RMT_CH*m*_RX_THR_EVENT_INT 中断之后，可以回收已使用过的 RAM 区域的脉冲编码，从而实现乒乓操作。

25.3.5.3 接收滤波

置位 RMT_RX_FILTER_EN_CH*m* 可使能通道 *m* 的接收器对输入信号进行滤波。滤波器的功能为连续采样输入信号，如果输入信号在连续 RMT_RX_FILTER _THRES_CH*m* 个 rmt_sclk 时钟周期内保持不变，则输入信号有效，否则输入信号无效。只有有效的输入信号才能通过滤波器。因此，滤波器会滤除脉冲宽度小于 RMT_RX_FILTER_THRES_CH*m* 个 rmt_sclk 时钟周期的线路毛刺。

25.3.5.4 接收去载波

此外，接收器还可以对输入信号或滤波后的信号进行去载波调制，置位 RMT_CARRIER_EN_CH*m* 可以使能该功能。去载波可以对高电平调制或低电平调制的载波进行滤除：

- 清零 RMT_CARRIER_OUT_LV_CH*m*，配置滤除低电平载波；
- 置位 RMT_CARRIER_OUT_LV_CH*m*，配置滤除高电平载波。

配置 RMT_CARRIER_HIGH_THRES_CH*m* 和 RMT_CARRIER_LOW_THRES_CH*m* 可设置去载波的高电平阈值和低电平阈值。当信号的高电平持续时间小于 RMT_CARRIER_HIGH_THRES_CH*m* 个 clk_div 分频时钟周期，或者信号的低电平持续时间小于 RMT_CARRIER _LOW_THRES_CH*m* 个 clk_div 分频时钟周期，则信号会被认为是载波而被滤除。

25.3.6 配置参数更新

RMT 的配置寄存器均需要配置各自通道的 RMT_CONF_UPDATE_CH*n*/*m* 位来更新进入各自通道，更新方法是向 RMT_CONF_UPDATE_CH*n*/*m* 写入 1。发送通道和接收通道需要通过这种方法更新的配置参数见表 25.3-1。

表 25.3-1. 更新配置参数

配置寄存器	配置参数
发送通道	

见下页

表 25.3-1 – 接上页

配置寄存器	配置参数
RMT_CH*n*CONFO_REG	RMT_CARRIER_OUT_LV_CH*n*
	RMT_CARRIER_EN_CH*n*
	RMT_CARRIER_EFF_EN_CH*n*
	RMT_DIV_CNT_CH*n*
	RMT_TX_STOP_CH*n*
	RMT_IDLE_OUT_EN_CH*n*
	RMT_IDLE_OUT_LV_CH*n*
	RMT_TX_CONTI_MODE_CH*n*
RMT_CH*n*CARRIER_DUTY_REG	RMT_CARRIER_HIGH_CH*n*
	RMT_CARRIER_LOW_CH*n*
RMT_CH*n*_TX_LIM_REG	RMT_TX_LOOP_CNT_EN_CH*n*
	RMT_TX_LOOP_NUM_CH*n*
	RMT_TX_LIM_CH*n*
RMT_TX_SIM_REG	RMT_TX_SIM_EN
接收通道	
RMT_CH*m*CONFO_REG	RMT_CARRIER_OUT_LV_CH*m*
	RMT_CARRIER_EN_CH*m*
	RMT_IDLE_THRES_CH*m*
	RMT_DIV_CNT_CH*m*
RMT_CH*m*CONF1_REG	RMT_RX_FILTER_THRES_CH*m*
	RMT_RX_EN_CH*m*
RMT_CH*m*_RX_CARRIER_RM_REG	RMT_CARRIER_HIGH_THRES_CH*m*
	RMT_CARRIER_LOW_THRES_CH*m*
RMT_CH*m*_RX_LIM_REG	RMT_RX_LIM_CH*m*
RMT_REF_CNT_RST_REG	RMT_REF_CNT_RST_CH*m*

25.4 中断

ESP32-S31 的 RMT 控制器可以生成 RMT_INTR 中断信号并发送给 中断矩阵，该中断信号由 RMT 的内部中断源生成。

以下中断源生成 RMT_INTR 中断信号：

- RMT_CH*n*_ERR_INT：当通道 *n* 发生读写数据不正确，或内存空满错误时，即触发此中断。
- RMT_CH*m*_ERR_INT：当通道 *m* 发生读写数据不正确，或内存空满错误时，即触发此中断。
- RMT_CH*n*_TX_THR_EVENT_INT：发射器每发送 RMT_CH*n*_TX_LIM_REG 的数据，即触发一次此中断。
- RMT_CH*m*_RX_THR_EVENT_INT：接收器每接收 RMT_CH*m*_RX_LIM_REG 的数据，即触发一次此中断。
- RMT_CH*n*_TX_END_INT：当发射器停止发送信号时，即触发此中断。
- RMT_CH*m*_RX_END_INT：当接收器停止接收信号时，即触发此中断。
- RMT_CH*n*_TX_LOOP_INT：发射器处于循环发送模式时，当循环次数达到 RMT_TX_LOOP_NUM_CH*n* 的值后，会产生此中断。

- RMT_CH3_DMA_ACCESS_FAIL_INT：当 GDMA 向通道 3 RAM 区域写入数据个数减去通道 3 实际发送个数超过通道 3 RAM 大小，且 GDMA 又写入数据，会产生此中断。
- RMT_CH7_DMA_ACCESS_FAIL_INT：当通道 7 RAM 接收数据个数减去 GDMA 取走的数据超过通道 7 RAM 大小，且通道 7 又接收到数据，会产生此中断。

说明：

中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 25.5 寄存器列表。

25.5 寄存器列表

本小节的所有地址均为相对于红外遥控基地址的地址偏移量（相对地址），具体基地址请见章节 3 系统和存储器 *[to be added later]* 中的表 3。

请查看章节 寄存器的访问类型，了解“访问”列缩写的含义。

名称	描述	地址	访问
FIFO 读/写寄存器			
RMT_CH0DATA_REG	通道 0 通过 APB FIFO 进行读写操作时用到的数据寄存器	0x0000	HRO
RMT_CH1DATA_REG	通道 1 通过 APB FIFO 进行读写操作时用到的数据寄存器	0x0004	HRO
RMT_CH2DATA_REG	通道 2 通过 APB FIFO 进行读写操作时用到的数据寄存器	0x0008	HRO
RMT_CH3DATA_REG	通道 3 通过 APB FIFO 进行读写操作时用到的数据寄存器	0x000C	HRO
RMT_CH4DATA_REG	通道 4 通过 APB FIFO 进行读写操作时用到的数据寄存器	0x0010	HRO
RMT_CH5DATA_REG	通道 5 通过 APB FIFO 进行读写操作时用到的数据寄存器	0x0014	HRO
RMT_CH6DATA_REG	通道 6 通过 APB FIFO 进行读写操作时用到的数据寄存器	0x0018	HRO
RMT_CH7DATA_REG	通道 7 通过 APB FIFO 进行读写操作时用到的数据寄存器	0x001C	HRO
配置寄存器			
RMT_CH0CONF0_REG	通道 0 的配置寄存器 0	0x0020	varies
RMT_CH1CONF0_REG	通道 1 的配置寄存器 0	0x0024	varies
RMT_CH2CONF0_REG	通道 2 的配置寄存器 0	0x0028	varies
RMT_CH3CONF0_REG	通道 3 的配置寄存器 0	0x002C	varies
RMT_CH4CONF0_REG	通道 4 的配置寄存器 0	0x0030	R/W
RMT_CH4CONF1_REG	通道 4 的配置寄存器 1	0x0034	varies
RMT_CH5CONF0_REG	通道 5 的配置寄存器 0	0x0038	R/W
RMT_CH5CONF1_REG	通道 5 的配置寄存器 1	0x003C	varies
RMT_CH6CONF0_REG	通道 6 的配置寄存器 0	0x0040	R/W
RMT_CH6CONF1_REG	通道 6 的配置寄存器 1	0x0044	varies
RMT_CH7CONF0_REG	通道 7 的配置寄存器 0	0x0048	R/W
RMT_CH7CONF1_REG	通道 7 的配置寄存器 1	0x004C	varies
RMT_CH4_RX_CARRIER_RM_REG	通道 4 的载波滤除寄存器	0x0090	R/W
RMT_CH5_RX_CARRIER_RM_REG	通道 5 的载波滤除寄存器	0x0094	R/W
RMT_CH6_RX_CARRIER_RM_REG	通道 6 的载波滤除寄存器	0x0098	R/W
RMT_CH7_RX_CARRIER_RM_REG	通道 7 的载波滤除寄存器	0x009C	R/W
RMT_SYS_CONF_REG	RMT APB 配置寄存器	0x00C0	R/W
RMT_REF_CNT_RST_REG	RMT 时钟分频器复位寄存器	0x00C8	WT
状态寄存器			

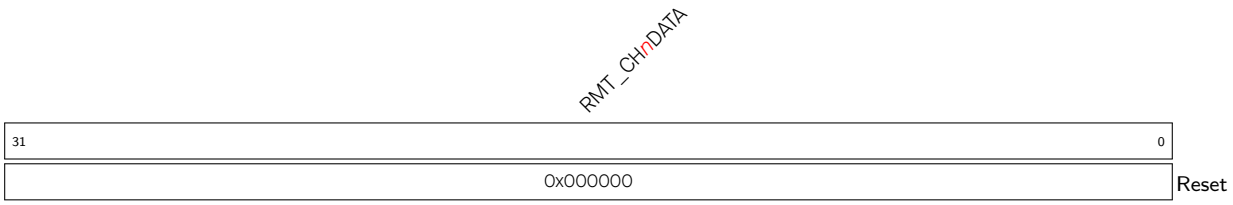
名称	描述	地址	访问
RMT_CH0STATUS_REG	通道 0 的状态寄存器	0x0050	RO
RMT_CH1STATUS_REG	通道 1 的状态寄存器	0x0054	RO
RMT_CH2STATUS_REG	通道 2 的状态寄存器	0x0058	RO
RMT_CH3STATUS_REG	通道 3 的状态寄存器	0x005C	RO
RMT_CH4STATUS_REG	通道 4 的状态寄存器	0x0060	RO
RMT_CH5STATUS_REG	通道 5 的状态寄存器	0x0064	RO
RMT_CH6STATUS_REG	通道 6 的状态寄存器	0x0068	RO
RMT_CH7STATUS_REG	通道 7 的状态寄存器	0x006C	RO
中断寄存器			
RMT_INT_RAW_REG	原始中断状态寄存器	0x0070	R/WTC/SS
RMT_INT_ST_REG	屏蔽中断状态寄存器	0x0074	RO
RMT_INT_ENA_REG	中断使能寄存器	0x0078	R/W
RMT_INT_CLR_REG	中断清零寄存器	0x007C	WT
载波占空比寄存器			
RMT_CH0CARRIER_DUTY_REG	通道 0 的占空比配置寄存器	0x0080	R/W
RMT_CH1CARRIER_DUTY_REG	通道 1 的占空比配置寄存器	0x0084	R/W
RMT_CH2CARRIER_DUTY_REG	通道 2 的占空比配置寄存器	0x0088	R/W
RMT_CH3CARRIER_DUTY_REG	通道 3 的占空比配置寄存器	0x008C	R/W
TX 事件配置寄存器			
RMT_CH0_TX_LIM_REG	通道 0 的 TX 事件配置寄存器	0x00A0	varies
RMT_CH1_TX_LIM_REG	通道 1 的 TX 事件配置寄存器	0x00A4	varies
RMT_CH2_TX_LIM_REG	通道 2 的 TX 事件配置寄存器	0x00A8	varies
RMT_CH3_TX_LIM_REG	通道 3 的 TX 事件配置寄存器	0x00AC	varies
RMT_TX_SIM_REG	RMT TX 同步发送寄存器	0x00C4	R/W
RX 事件配置寄存器			
RMT_CH4_RX_LIM_REG	通道 4 RX 事件配置寄存器	0x00B0	R/W
RMT_CH5_RX_LIM_REG	通道 5 RX 事件配置寄存器	0x00B4	R/W
RMT_CH6_RX_LIM_REG	通道 6 RX 事件配置寄存器	0x00B8	R/W
RMT_CH7_RX_LIM_REG	通道 7 RX 事件配置寄存器	0x00BC	R/W
版本寄存器			
RMT_DATE_REG	版本控制寄存器	0x00CC	R/W

25.6 寄存器

本小节的所有地址均为相对于红外遥控基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 25.1. RMT_CH*n*DATA_REG (*n*: 0-3) (0x0000+0x4n*)**



RMT_CH*n*DATA 通道 *n* 通过 APB FIFO 进行读写操作时用到的数据寄存器。(HRO)

Register 25.2. RMT_CH*m*DATA_REG (*m*: 4-7) (0x0000+0x4m*)**



RMT_CH*m*DATA 通道 *m* 通过 APB FIFO 进行读写操作时用到的数据寄存器。(HRO)

Register 25.3. RMT_CH*n*CONF0_REG (*n*: 0-3) (0x0020+0x4**n*)



RMT_TX_START_CH*n* 配置是否使能通道 *n* 开始发送数据。

0：无效

1：使能

(WT)

RMT_MEM_RD_RST_CH*n* 配置是否复位通道 *n* 通过发射器访问的 RAM 读地址。

0：无效

1：复位

(WT)

RMT_APB_MEM_RST_CH*n* 配置是否复位通道 *n* 通过 APB FIFO 访问的读/写 RAM 地址。

0：无效

1：复位

(WT)

RMT_TX_CONTI_MODE_CH*n* 配置是否使能通道 *n* 的持续发送模式。

0：无效

1：使能

在这种模式下，发射器从第一个数据开始发送，如果遇到结束标志，则再次发送第一个数据；如果没有遇到结束标志，发送完最后一个数据后，回卷到第一个数据再次继续发送。

(R/W)

RMT_MEM_TX_WRAP_EN_CH*n* 配置是否使能通道 *n* 的乒乓发送模式。

0：无效

1：使能

在这种模式下，如果待发送的数据长度大于该通道的 RAM Block 长度，则发射器将继续从第一个数据开始循环发送。

(R/W)

RMT_IDLE_OUT_LV_CH*n* 配置通道 *n* 在空闲模式下的输出信号电平。(R/W)

RMT_IDLE_OUT_EN_CH*n* 配置是否使能通道 *n* 在空闲状态下的输出模式。

0：无效

1：使能

(R/W)

见下页...

Register 25.3. RMT_CH*n*CONF0_REG (*n*: 0-3) (0x0020+0x4**n*)

接上页...

RMT_TX_STOP_CH*n* 配置通道 *n* 的发射器是否停止发送数据。

0：无效

1：停止

(R/W/SC)

RMT_DIV_CNT_CH*n* 配置通道 *n* 的时钟分频系数。

测量单位：rmt_sclk

(R/W)

RMT_MEM_SIZE_CH*n* 配置通道 *n* 可用的最大 RAM Block 数量。(R/W)

RMT_CARRIER_EFF_EN_CH*n* 配置是否仅在发送数据状态下对输出信号载波调制。

0：配置通道 *n* 对发送数据状态和空闲状态均加载载波

1：配置通道 *n* 仅在发送数据状态下对输出信号载波调制

仅在 RMT_CARRIER_EN_CH*n* 为 1 时有效。

(R/W)

RMT_CARRIER_EN_CH*n* 配置是否使能通道 *n* 对输出信号进行载波调制。

0：不使能

1：使能

(R/W)

RMT_CARRIER_OUT_LV_CH*n* 配置通道 *n* 的载波调制方式。

0：载波加载在低电平上

1：载波加载在高电平上

(R/W)

RMT_CONF_UPDATE_CH*n* 通道 *n* 的同步位。(WT)

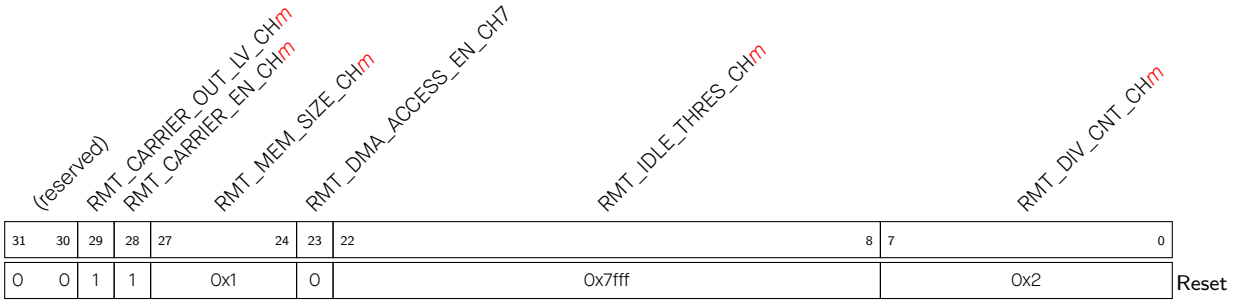
RMT_DMA_ACCESS_EN_CH3 配置是否启用通道 3 的 DMA 访问功能。通道 0 ~ 2 中该位保留。

0: 不启用

1: 启用

(R/W)

Register 25.4. RMT_CH*m*CONF0_REG (*m*: 4-7) (0x0030, 0x0038, 0x0040, 0x0048)



RMT_DIV_CNT_CH*m* 配置通道 *m* 的时钟分频时钟。

测量单位：rmt_sclk

(R/W)

RMT_IDLE_THRES_CH*m* 配置接收阈值。接收器长时间检测不到信号变化，且持续的时间大于 RMT_IDLE_THRES_CH*m* 的值，则接收器停止接收过程。

测量单位：clk_div

(R/W)

RMT_DMA_ACCESS_EN_CH7 配置是否启用通道 7 的 DMA 访问功能。通道 4 ~ 6 中该位保留。

0：不启用

1：启用

(R/W)

RMT_MEM_SIZE_CH*m* 配置通道 *m* 可用的最大 RAM Block 数量。(R/W)

RMT_CARRIER_EN_CH*m* 配置是否使能通道 *m* 对输入信号进行载波解调。

0：不使能

1：使能

(R/W)

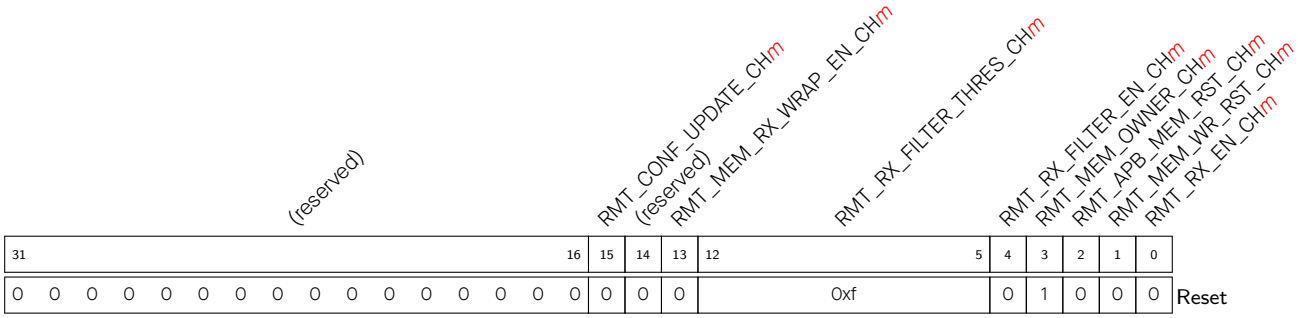
RMT_CARRIER_OUT_LV_CH*m* 配置通道 *m* 的载波解调方式。

0：在低电平上进行载波解调

1：在高电平上进行载波解调

(R/W)

Register 25.5. RMT_CHmCONF1_REG (m: 4-7) (0x0034, 0x003C, 0x0044, 0x004C)



RMT_RX_EN_CHm 配置是否使能通道 *m* 的接收器，使接收器开始接收数据。

0：不使能

1：使能

(R/W)

RMT_MEM_WR_RST_CHm 配置是否复位通道 *m* 通过接收器访问的 RAM 写地址。

0：无效

1：复位

(WT)

RMT_APB_MEM_RST_CHm 配置是否复位通道 *m* 通过 APB FIFO 访问的读/写 RAM 地址。

0：无效

1：复位

(WT)

RMT_MEM_OWNER_CHm 配置通道 *m* 的 RAM 使用权。

0：APB 总线有权使用该通道的 RAM

1：接收器有权使用该通道的 RAM

(R/W/SC)

RMT_RX_FILTER_EN_CHm 配置是否使能通道 *m* 的接收器滤波功能。

0：不使能

1：使能

(R/W)

RMT_RX_FILTER_THRES_CHm 配置是否在接收模式下，忽略宽度小于 RMT_RX_FILTER_THRES_CH*m* 个 rmt_sclk 周期的输入脉冲。

0：无效

1：忽略

(R/W)

RMT_MEM_RX_WRAP_EN_CHm 配置是否使能通道 *m* 的乒乓接收模式。

0：不使能

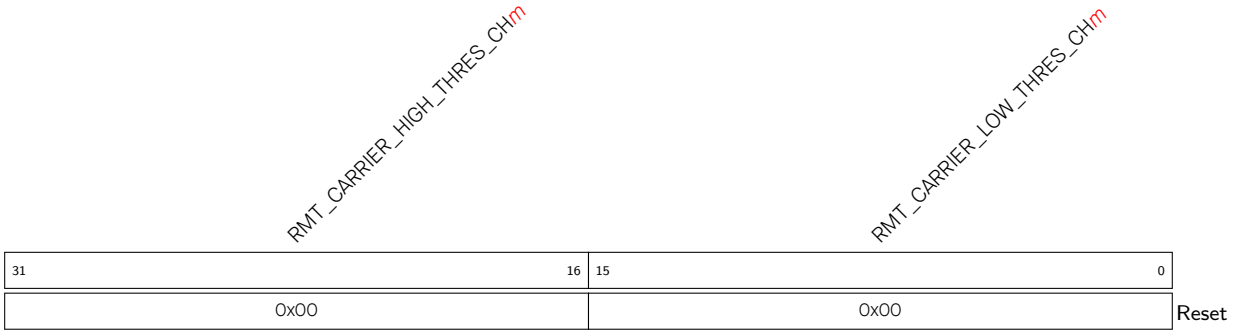
1：使能

在这种模式下，如果待接收的数据长度大于该通道的 RAM Block 长度，则接收器将继续把接收数据存入 RAM 的第一个地址，依次循环。

(R/W)

RMT_CONF_UPDATE_CHm 通道 *m* 的同步位。(WT)

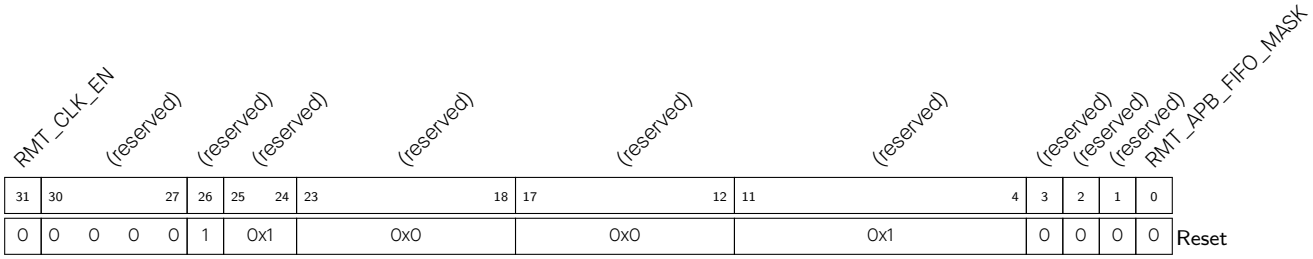
Register 25.6. RMT_CH*m*_RX_CARRIER_RM_REG (*m*: 4-7) (0x0090, 0x0094, 0x0098, 0x009C)



RMT_CARRIER_LOW_THRES_CH*m* 配置通道 *m* 载波调制模式下的低电平周期，该周期为 (RMT_CARRIER_LOW_THRES_CH*m* + 1)。(R/W)

RMT_CARRIER_HIGH_THRES_CH*m* 配置通道 *m* 载波调制模式下的高电平周期，该周期为 (RMT_CARRIER_HIGH_THRES_CH*m* + 1)。(R/W)

Register 25.7. RMT_SYS_CONF_REG (0x00C0)



RMT_APB_FIFO_MASK 配置 RAM 的访问模式。

0：通过 FIFO 访问 RAM

1：直接访问 RAM

(R/W)

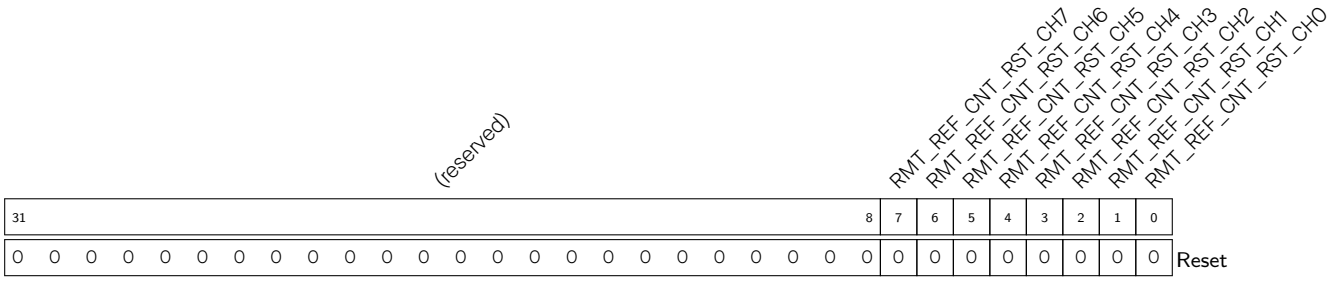
RMT_CLK_EN 配置是否使能 RMT 寄存器的时钟门控。

0：关闭寄存器的驱动时钟

1：打开寄存器的驱动时钟

(R/W)

Register 25.8. RMT_REF_CNT_RST_REG (0x00C8)



RMT_REF_CNT_RST_CH*n* (*n*: 0-3) 配置是否复位通道 *n* 的时钟分频器。

0：无效

1：复位

(WT)

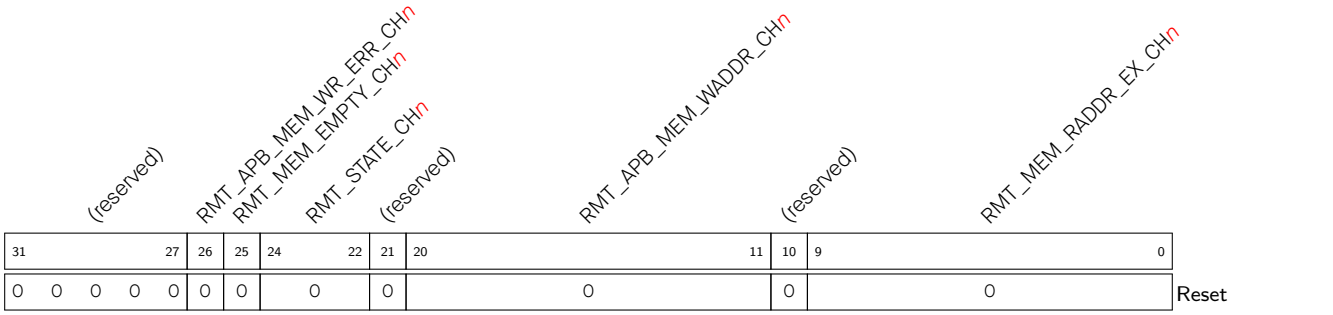
RMT_REF_CNT_RST_CH*m* (*m*: 4-7) 配置是否复位通道 *m* 的时钟分频器。

0：无效

1：复位

(WT)

Register 25.9. RMT_CH*n*STATUS_REG (*n*: 0-3) (0x0050+0x4**n*)



RMT_MEM_RADDR_EX_CH*n* 表示通道 *n* 发射器使用 RAM 时的地址偏移量。(RO)

RMT_APB_MEM_WADDR_CH*n* 表示 RMT 使用 APB 总线访问 RAM 时的地址偏移量。(RO)

RMT_STATE_CH*n* 表示通道 *n* 的 FSM 状态。(RO)

RMT_MEM_EMPTY_CH*n* 表示发送的数据长度是否溢出 RAM Block，且乒乓模式未启用。

0：未溢出

1：溢出

(RO)

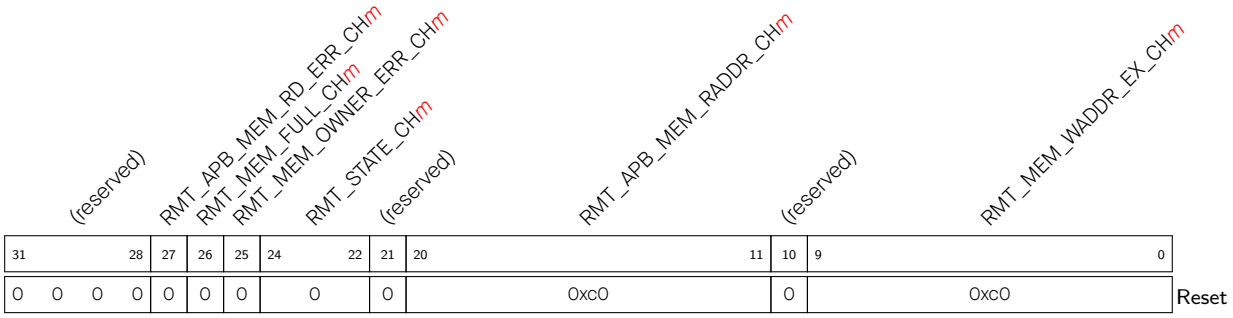
RMT_APB_MEM_WR_ERR_CH*n* 表示 RMT 使用 APB 总线进行写 RAM 操作时，偏移地址是否溢出 RAM Block。

0：未溢出

1：溢出

(RO)

Register 25.10. RMT_CH*m*STATUS_REG (*m*: 4-7) (0x0050+0x4**m*)



RMT_MEM_WADDR_EX_CH*m* 表示通道 *m* 接收器使用 RAM 时的地址偏移量。(RO)

RMT_APB_MEM_RADDR_CH*m* 表示 RMT 使用 APB 总线访问 RAM 时的地址偏移量。(RO)

RMT_STATE_CH*m* 表示通道 *m* 的 FSM 状态。(RO)

RMT_MEM_OWNER_ERR_CH*m* 表示 RAM Block 使用权是否发生错误。

0：无错误

1：发生错误

(RO)

RMT_MEM_FULL_CH*m* 表示接收器接收的数据长度是否溢出 RAM Block。

0：未溢出

1：溢出

(RO)

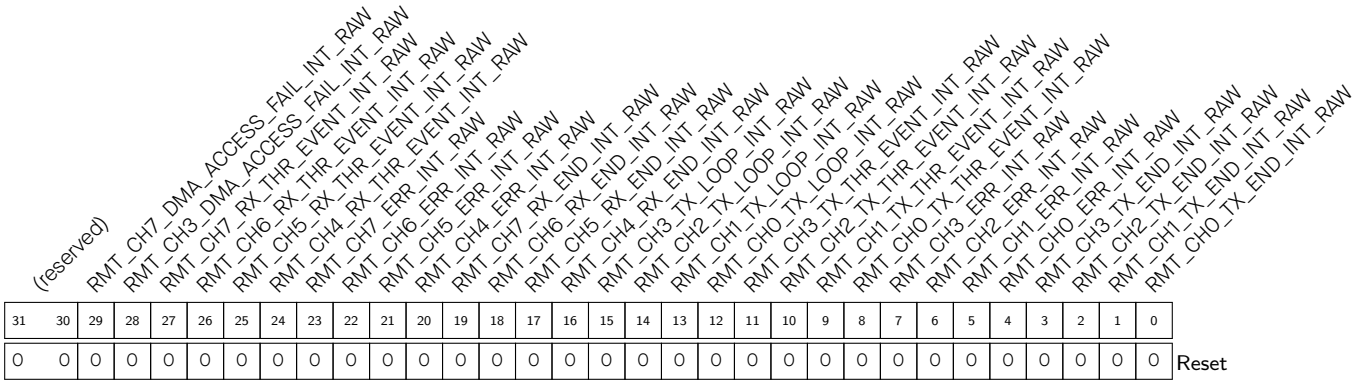
RMT_APB_MEM_RD_ERR_CH*m* 表示 RMT 使用 APB 总线执行 RAM 读操作时，偏移地址是否溢出 RAM Block。

0：未溢出

1：溢出

(RO)

Register 25.11. RMT_INT_RAW_REG (0x0070)



RMT_CH*n*_TX_END_INT_RAW (*n*: 0-3) RMT_CH*n*_TX_END_INT 的原始中断状态。(R/WTC/SS)

RMT_CH*n*_ERR_INT_RAW (*n*: 0-3) RMT_CH*n*_ERR_INT 的原始中断状态。(R/WTC/SS)

RMT_CH*n*_TX_THR_EVENT_INT_RAW (*n*: 0-3) RMT_CH*n*_TX_THR_EVENT_INT 的原始中断状态。(R/WTC/SS)

RMT_CH*n*_TX_LOOP_INT_RAW (*n*: 0-3) RMT_CH*n*_TX_LOOP_INT 的原始中断状态。(R/WTC/SS)

RMT_CH*m*_RX_END_INT_RAW (*m*: 4-7) RMT_CH*m*_RX_END_INT 的原始中断状态。(R/WTC/SS)

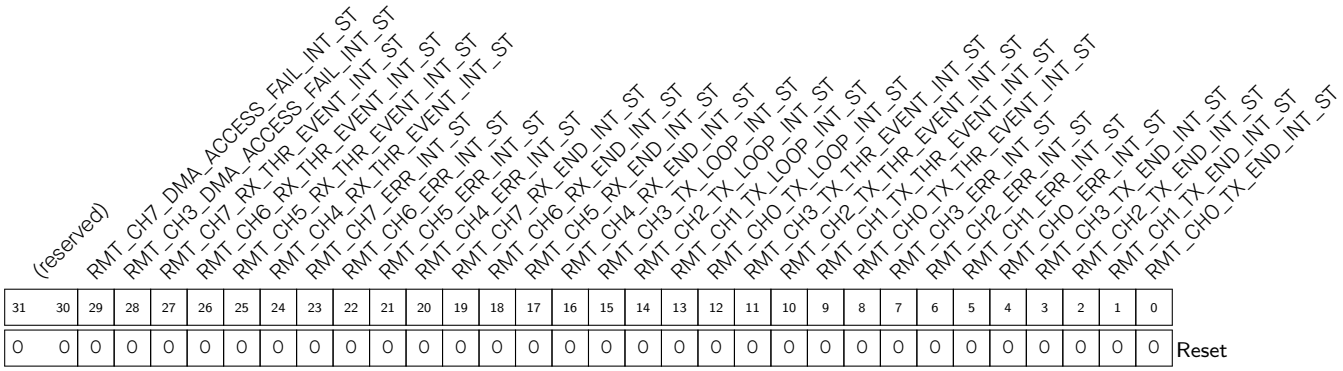
RMT_CH*m*_ERR_INT_RAW (*m*: 4-7) RMT_CH*m*_ERR_INT 的原始中断状态。(R/WTC/SS)

RMT_CH*m*_RX_THR_EVENT_INT_RAW (*m*: 4-7) RMT_CH*m*_RX_THR_EVENT_INT 的原始中断状态。(R/WTC/SS)

RMT_CH*n*_DMA_ACCESS_FAIL_INT_RAW (*n*: 3) RMT_CH*n*_DMA_ACCESS_FAIL_INT 的原始中断状态。(R/WTC/SS)

RMT_CH*m*_DMA_ACCESS_FAIL_INT_RAW (*m*: 7) RMT_CH*m*_DMA_ACCESS_FAIL_INT 的原始中断状态。(R/WTC/SS)

Register 25.12. RMT_INT_ST_REG (0x0074)



RMT_CH*n*_TX_END_INT_ST (*n*: 0-3) RMT_CH*n*_TX_END_INT 的屏蔽中断状态。(RO)

RMT_CH*n*_ERR_INT_ST (*n*: 0-3) RMT_CH*n*_ERR_INT 的屏蔽中断状态。(RO)

RMT_CH*n*_TX_THR_EVENT_INT_ST (*n*: 0-3) RMT_CH*n*_TX_THR_EVENT_INT 的屏蔽中断状态。(RO)

RMT_CH*n*_TX_LOOP_INT_ST (*n*: 0-3) RMT_CH*n*_TX_LOOP_INT 的屏蔽中断状态。(RO)

RMT_CH*m*_RX_END_INT_ST (*m*: 4-7) RMT_CH*m*_RX_END_INT 的屏蔽中断状态。(RO)

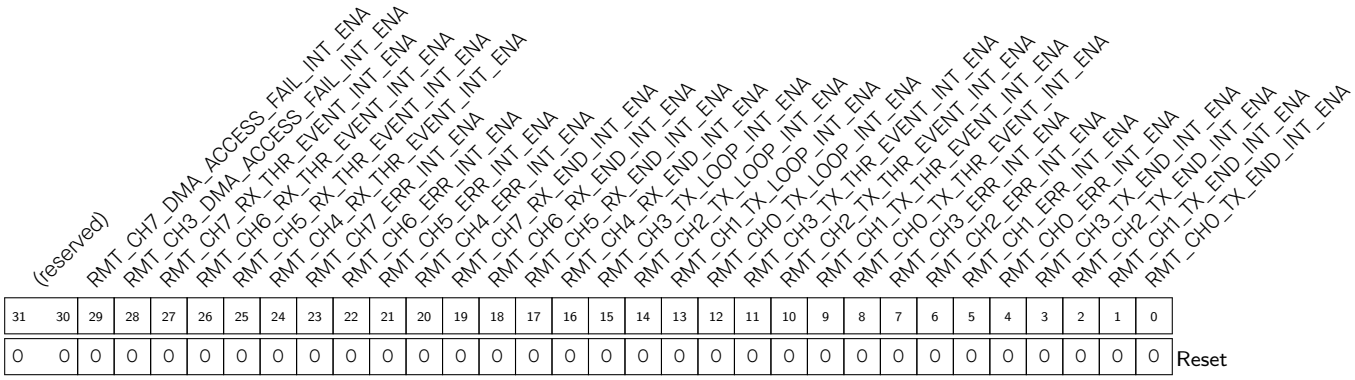
RMT_CH*m*_ERR_INT_ST (*m*: 4-7) RMT_CH*m*_ERR_INT 的屏蔽中断状态。(RO)

RMT_CH*m*_RX_THR_EVENT_INT_ST (*m*: 4-7) RMT_CH*m*_RX_THR_EVENT_INT 的屏蔽中断状态。(RO)

RMT_CH*n*_DMA_ACCESS_FAIL_INT_ST (*n*: 3) RMT_CH*n*_DMA_ACCESS_FAIL_INT 的屏蔽中断状态。(RO)

RMT_CH*m*_DMA_ACCESS_FAIL_INT_ST (*m*: 7) RMT_CH*m*_DMA_ACCESS_FAIL_INT 的屏蔽中断状态。(RO)

Register 25.13. RMT_INT_ENA_REG (0x0078)



RMT_CH*n*_TX_END_INT_ENA (*n*: 0-3) 写 1 使能 RMT_CH*n*_TX_END_INT。(R/W)

RMT_CH*n*_ERR_INT_ENA (*n*: 0-3) 写 1 使能 RMT_CH*n*_ERR_INT。(R/W)

RMT_CH*n*_TX_THR_EVENT_INT_ENA (*n*: 0-3) 写 1 使能 RMT_CH*n*_TX_THR_EVENT_INT。(R/W)

RMT_CH*n*_TX_LOOP_INT_ENA (*n*: 0-3) 写 1 使能 RMT_CH*n*_TX_LOOP_INT。(R/W)

RMT_CH*m*_RX_END_INT_ENA (*m*: 4-7) 写 1 使能 RMT_CH*m*_RX_END_INT。(R/W)

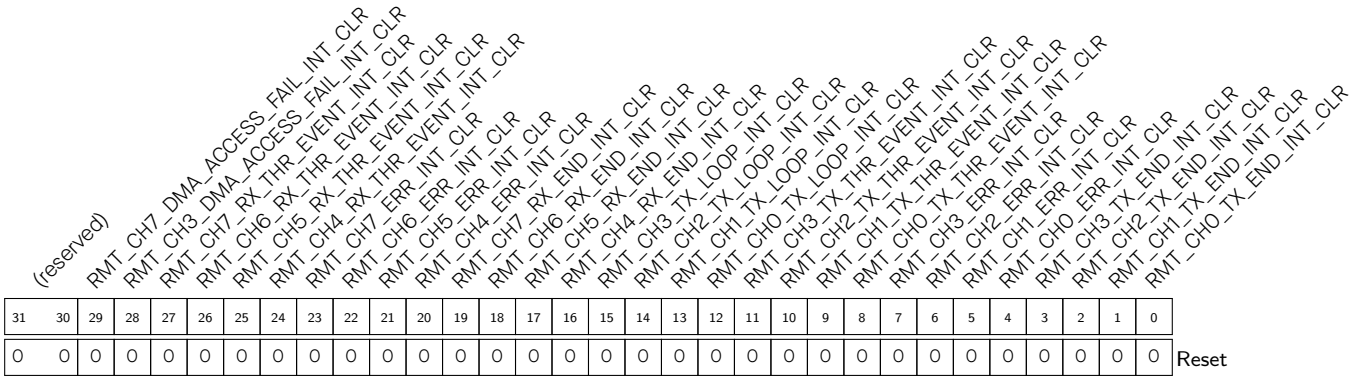
RMT_CH*m*_ERR_INT_ENA (*m*: 4-7) 写 1 使能 RMT_CH*m*_ERR_INT。(R/W)

RMT_CH*m*_RX_THR_EVENT_INT_ENA (*m*: 4-7) 写 1 使能 RMT_CH*m*_RX_THR_EVENT_INT。(R/W)

RMT_CH*n*_DMA_ACCESS_FAIL_INT_ENA (*n*: 3) 写 1 使能 RMT_CH*n*_DMA_ACCESS_FAIL_INT。(R/W)

RMT_CH*m*_DMA_ACCESS_FAIL_INT_ENA (*m*: 7) 写 1 使能 RMT_CH*m*_DMA_ACCESS_FAIL_INT。(R/W)

Register 25.14. RMT_INT_CLR_REG (0x007C)



RMT_CH*n*_TX_END_INT_CLR (*n*: 0-3) 写 1 清除 RMT_CH*n*_TX_END_INT。(WT)

RMT_CH*n*_ERR_INT_CLR (*n*: 0-3) 写 1 清除 RMT_CH*n*_ERR_INT。(WT)

RMT_CH*n*_TX_THR_EVENT_INT_CLR (*n*: 0-3) 写 1 清除 RMT_CH*n*_TX_THR_EVENT_INT。(WT)

RMT_CH*n*_TX_LOOP_INT_CLR (*n*: 0-3) 写 1 清除 RMT_CH*n*_TX_LOOP_INT。(WT)

RMT_CH*m*_RX_END_INT_CLR (*m*: 4-7) 写 1 清除 RMT_CH*m*_RX_END_INT。(WT)

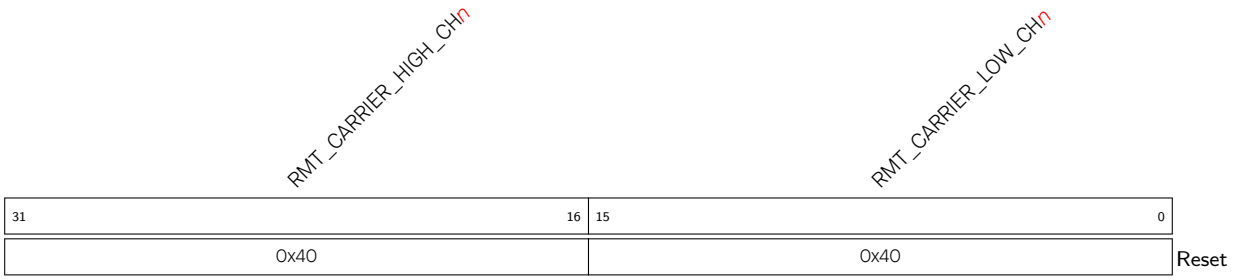
RMT_CH*m*_ERR_INT_CLR (*m*: 4-7) 写 1 清除 RMT_CH*m*_ERR_INT。(WT)

RMT_CH*m*_RX_THR_EVENT_INT_CLR (*m*: 4-7) 写 1 清除 RMT_CH*m*_RX_THR_EVENT_INT。(WT)

RMT_CH*n*_DMA_ACCESS_FAIL_INT_CLR (*n*: 3) 写 1 清除 RMT_CH*n*_DMA_ACCESS_FAIL_INT。(WT)

RMT_CH*m*_DMA_ACCESS_FAIL_INT_CLR (*m*: 7) 写 1 清除 RMT_CH*m*_DMA_ACCESS_FAIL_INT。(WT)

Register 25.15. RMT_CH*n*CARRIER_DUTY_REG (*n*: 0-3) (0x0080+0x4**n*)



RMT_CARRIER_LOW_CH*n* 配置通道 *n* 载波的低电平时钟周期。

测量单位：rmt_sclk

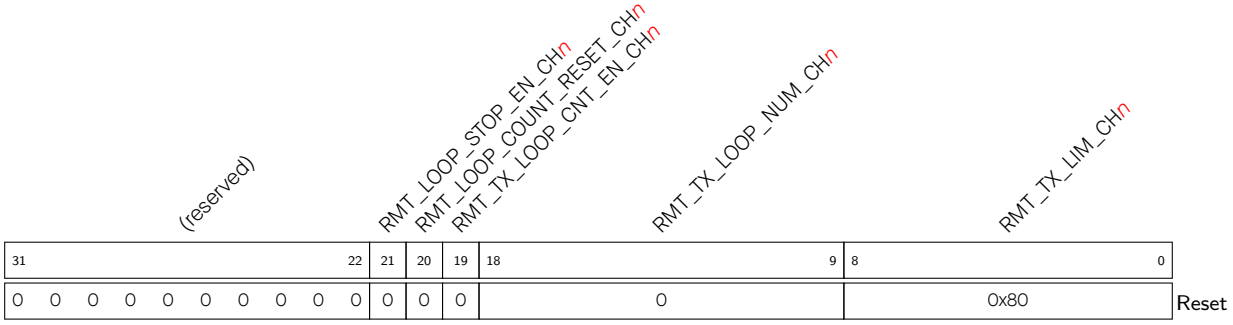
(R/W)

RMT_CARRIER_HIGH_CH*n* 配置通道 *n* 载波的高电平时钟周期。

测量单位：rmt_sclk

(R/W)

Register 25.16. RMT_CH*n*_TX_LIM_REG (*n*: 0-3) (0x00A0+0x4**n*)



RMT_TX_LIM_CH*n* 配置通道 *n* 发送脉冲编码数量的上限值。(R/W)

RMT_TX_LOOP_NUM_CH*n* 配置持续发送模式下最大循环发送次数。(R/W)

RMT_TX_LOOP_CNT_EN_CH*n* 配置是否使能循环次数计数。

0：无效

1：使能

(R/W)

RMT_LOOP_COUNT_RESET_CH*n* 配置是否重置持续发送模式下的循环计数器。

0：无效

1：重置

(WT)

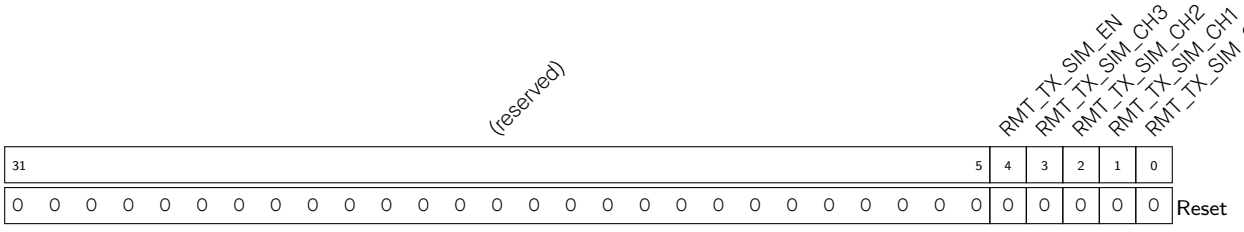
RMT_LOOP_STOP_EN_CH*n* 配置是否在通道 *n* 的循环计数器记到循环数后使能循环发送停止功能。

0：无效

1：使能

(R/W)

Register 25.17. RMT_TX_SIM_REG (0x00C4)



RMT_TX_SIM_CH*n* (*n*: 0-3) 配置是否使能通道 *n* 与其它启用的通道同步开始发送数据。

0: 不使能

1: 使能

(R/W)

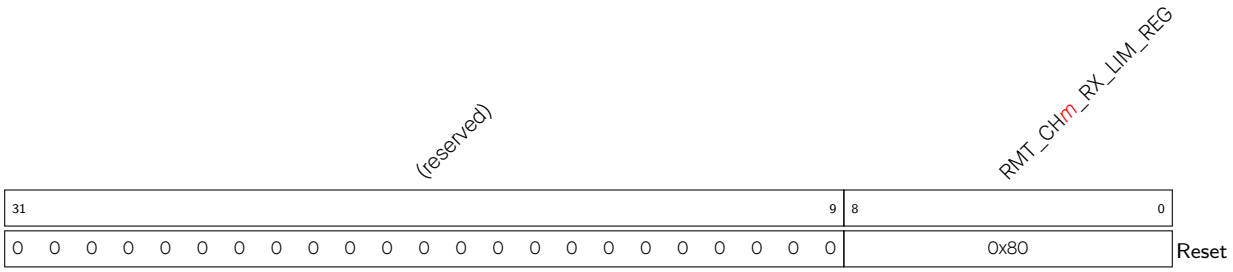
RMT_TX_SIM_EN 配置是否使能多个通道开始同步发送数据。

0: 不使能

1: 使能

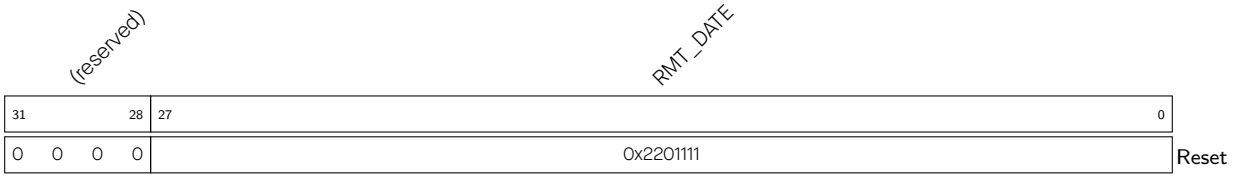
(R/W)

Register 25.18. RMT_CH*m*_RX_LIM_REG (*m*: 4-7) (0x00B0, 0x00B4, 0x00B8, 0x00BC)



RMT_CH*m*_RX_LIM_REG 配置通道 *m* 最大可接收的脉冲编码数量。(R/W)

Register 25.19. RMT_DATE_REG (0x00CC)



RMT_DATE 版本控制寄存器。(R/W)

第 26 章

并行 IO 控制器 (PARLIO)

26.1 概况

ESP32-S31 包含一个并行 IO 控制器 (PARLIO)，支持通过通用直接存储访问 (GDMA) 在并行总线上实现外部设备和内部存储器之间的数据通信。它由 TX 和 RX 两个子模块组成，分别作为发送器和接收器。PARLIO 支持全双工通信。

鉴于 IO 数据的灵活性，PARLIO 可用作通用接口，连接各种外围设备。例如可以把 SPI 作为主机，PARLIO 作为从机，实现点对点传输。详细的应用实例，请参考小节 26.8。

26.2 术语

为了更好地说明 PARLIO 的功能，本章使用了以下术语。

RX 模块	PARLIO 子模块，从外部并行总线接收数据并将其存储在内部存储器中。
TX 模块	PARLIO 子模块，将数据从内部存储器传输到外部并行总线上。
RXD	从 RX 模块的 IO 接口接收的并行数据。
TXD	从 TX 模块的 IO 接口发送的并行数据。
帧	传输数据的单位，从设置 START 信号时起，到收到 EOF 信号止。
自由运行时钟	持续翻转的时钟。若非自由运行时钟，则时钟只在收到有效数据的期间翻转，其余时间停止翻转。
GDMA SUC EOF	GDMA 帧传输成功结束的信号。GDMA 接收到此信号时，触发一个 GDMA 中断，表明当前帧正确，且数据接收结束。
GDMA ERR EOF	GDMA 帧传输错误结束的信号。GDMA 接收到此信号时，触发一个 GDMA 中断，表明当前帧错误，且数据接收结束。
CDC	跨时钟域处理 (Clock Domain Crossing)。

26.3 特性

PARLIO 模块具备以下特性：

- 支持多种时钟源可选：
  - 包括外部 IO 时钟 PAD_CLK_TX/RX、内部系统时钟 XTAL_CLK、PLL_F160M_CLK 和 RC_FAST_CLK
  - 最大支持 40 MHz 的 IO 时钟频率
  - 时钟支持整数和小数分频
- 支持将传输数据总线位宽配置为 1/2/4/8/16 位

- 支持 16 位全双工传输
- 总线位宽为 1/2/4 位时，支持在一个字节范围内对比特数据顺序进行翻转
- 包含用于接收 IO 并行数据的 RX 子模块：
  - 支持对输出时钟进行门控
  - 支持 RX 子模块输入时钟和输出时钟分别取反
  - 支持多种接收模式
  - 支持配置 GDMA SUC EOF 信号生成模式
  - 支持配置外部使能信号的 IO 管脚
- 包含用于发送 IO 并行数据的 TX 子模块：
  - 支持片选功能
  - 支持对输出时钟进行门控
  - 支持 TX 子模块输入时钟和输出时钟分别取反
  - 支持配置 TX EOF 信号生成模式
  - 支持配置总线空闲值

26.4 系统架构

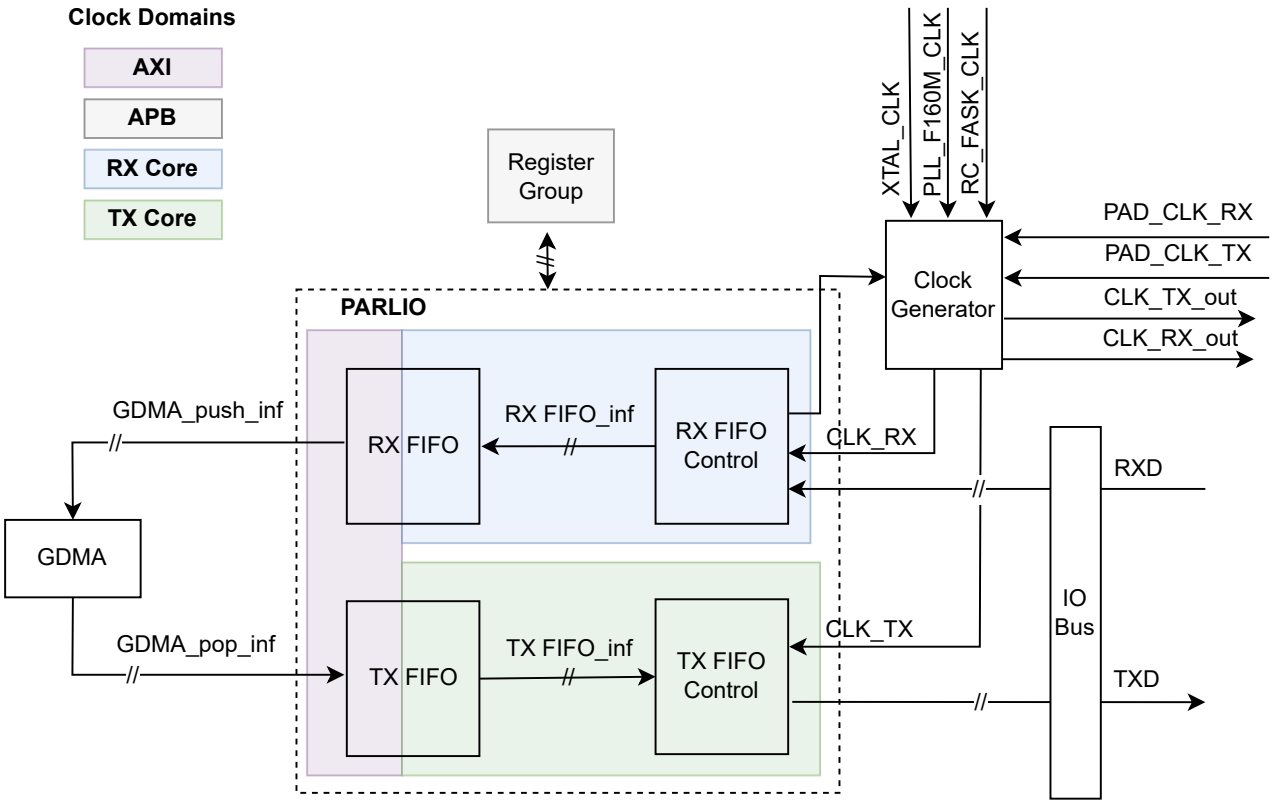


图 26.4-1. PARLIO 系统框图

图 26.4-1 展示了 PARLIO 的系统内部架构。除了 RX 模块和 TX 模块外，PARLIO 还包括一组状态配置寄存器。RX 模块接收 RXD 信号，并在完成串并转换后将其存入异步 FIFO，然后将这些数据通过 GDMA 存到内部存储器。

TX 模块通过 GDMA 从内部存储器获取数据并存入异步 FIFO，将这些数据进行并串转换后，通过 TXD 信号从 IO 总线输出。

26.5 功能描述

26.5.1 时钟生成器

PARLIO 模块有四个输入时钟域，即 RX Core、TX Core、AXI 和 APB，如图 26.4-1 所示。

状态配置寄存器组位于 APB 时钟域内。

GDMA 接口逻辑位于 AXI 时钟域内。

RX Core 有四个时钟源可供选择，分别为内部系统时钟源 XTAL_CLK、RC_FAST_CLK、PLL_F160M_CLK 和外部时钟源 (PAD_CLK_RX)，如图 26.5-1 所示。配置 HP_SYS_CLKRST_PARLIO_RX_CLK_SRC_SEL 选择时钟源，配置 HP_SYS_CLKRST_PARLIO_RX_CLK_DIV_NUM、HP_SYS_CLKRST_PARLIO_RX_CLK_DIV_DENOMINATOR 和 HP_SYS_CLKRST_PARLIO_RX_CLK_DIV_NUMERATOR 对时钟进行分频，这三个寄存器最大值均为 $(2^8 - 1)$。

TX Core 有四个时钟源可供选择，分别为内部系统时钟源 XTAL_CLK、RC_FAST_CLK、PLL_F160M_CLK 和外部时钟源 (PAD_CLK_TX)，如图 26.5-1 所示。配置 HP_SYS_CLKRST_PARLIO_TX_CLK_SRC_SEL 选择时钟源，配置 HP_SYS_CLKRST_PARLIO_TX_CLK_DIV_NUM、HP_SYS_CLKRST_PARLIO_TX_CLK_DIV_DENOMINATOR 和 HP_SYS_CLKRST_PARLIO_TX_CLK_DIV_NUMERATOR 对时钟进行分频，这三个寄存器最大值均为 $(2^8 - 1)$。

RX 模块和 TX 模块支持对输入时钟进行取反，同时其工作时钟可以输出到 IO，并在输出前支持对输出时钟进行取反。此外 RX 模块和 TX 模块还支持对输出时钟进行门控。

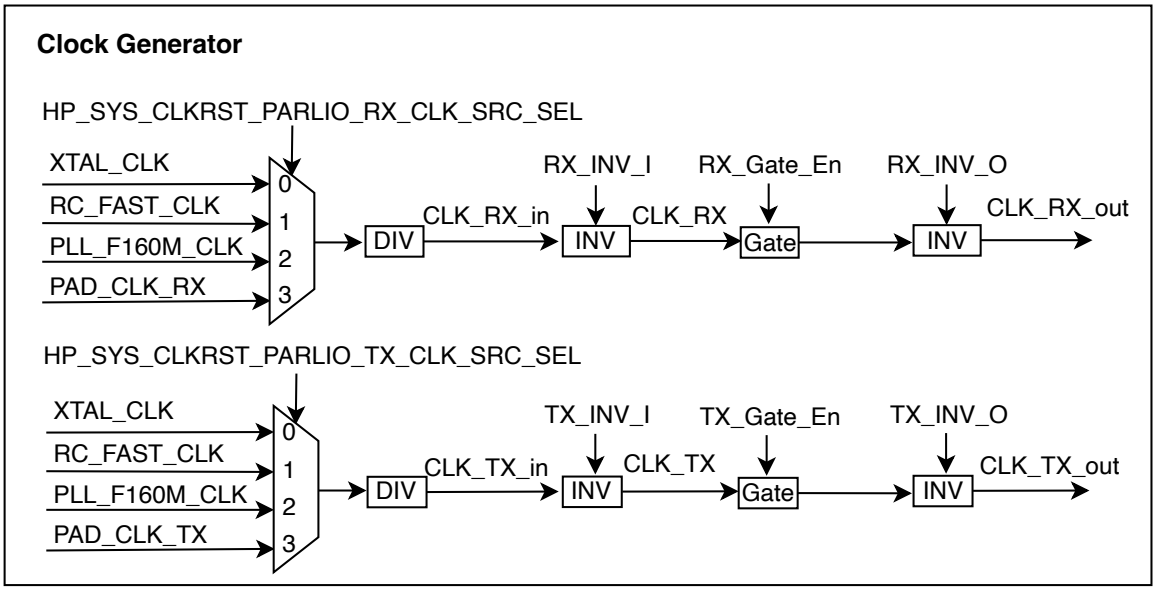


图 26.5-1. PARLIO 时钟生成

26.5.2 时钟复位使用限制

由于 PARLIO 模块的通用性，其 PAD 时钟 (PAD_CLK_TX/RX) 可能来自不同的主机（外部设备或内部时钟源），这些时钟可能是自由运行时钟，也可能不是。如果外部运行时钟不是自由运行时钟，PARLIO 内部的一些控制信号将无法进行 CDC，因此对操作过程会有一定限制。

1. 异步 FIFO 复位时，需要两个时钟周期才能从 AXI 时钟域同步到 Core 时钟域。因此如果需要在使用非自由运行时钟时复位 AXI 时钟域，需保证提前两个时钟周期进行复位同步。具体的操作步骤见下表。

表 26.5-1. 时钟限制下复位 AXI 时钟域操作步骤

时钟限制		具体操作
当前帧传输时钟	下一帧传输时钟	
自由运行时钟	非自由运行时钟	在切换到非自由运行时钟之前复位下一帧传输，并在复位完成后配置时钟切换。
非自由运行时钟	自由运行时钟	只需确保在复位和正式开始传输期间有两个时钟周期的间隔。
非自由运行时钟	非自由运行时钟	如需复位下一帧传输，需要先切换到内部的自由运行时钟，复位完成后再切换回实际时钟。

2. 由于非自由运行时钟造成的限制，PARL_IO_RX_START 和 PARL_IO_TX_START 无法进行 CDC，所以在操作时必须等到 PARL_IO_RX_START 和 PARL_IO_TX_START 稳定后再开始数据传输，否则可能会发生亚稳态的问题。

   RX 模块中的具体操作步骤为：

   - 清除 HP_SYS_CLKRST_PARLIO_RX_CLK_EN 关闭 RX Core 时钟域；
   - 向 PARL_IO_RX_START 写 1；
   - 置位 HP_SYS_CLKRST_PARLIO_RX_CLK_EN 打开 RX Core 时钟域；
   - 操作外部设备，使其开始发送数据；
   - 清除 HP_SYS_CLKRST_PARLIO_RX_CLK_EN 关闭 RX Core 时钟域；
   - 向 PARL_IO_TX_START 写 0。

   TX 模块中的具体操作步骤为：

   - 清除 HP_SYS_CLKRST_PARLIO_TX_CLK_EN 关闭 TX Core 时钟域；
   - 向 PARL_IO_TX_START 写 1；
   - 置位 HP_SYS_CLKRST_PARLIO_TX_CLK_EN 打开 TX Core 时钟域；
   - 操作外部设备，使其开始接收数据；
   - 清除 HP_SYS_CLKRST_PARLIO_TX_CLK_EN 关闭 TX Core 时钟域；
   - 向 PARL_IO_TX_START 写 0。

3. 复位需遵循以下要求：

   - 模块启动时，时钟复位应遵循以下顺序：

     (a) 复位 APB 时钟域；

     (b) 复位 GDMA 时钟域；

(c) 复位 Core 时钟域。

- 帧间传输需要复位 Core 时钟域和异步 FIFO。

26.5.3 主从机模式

RX 模块和 TX 模块均支持主从机模式。

当 TX 模块作为主机时，必须将时钟源设置为内部自由运行时钟。TX 模块在时钟的上升沿驱动 TXD。

当 TX 模块作为从机时，可能出现的三种情况及其具体要求如下表所示：

表 26.5-2. 时钟限制下 TX 模块从机模式操作要求

时钟条件		具体要求
主机发送时钟	时钟波形	
自由运行时钟	任意波形	对主机的驱动时钟采集边沿没有要求。
非自由运行时钟	正向波形（图 26.5-2）	主机需要在下降沿时采集。
非自由运行时钟	负向波形（图 26.5-3）	主机在输出前需要对原时钟进行取反，将其转换成正向波形。

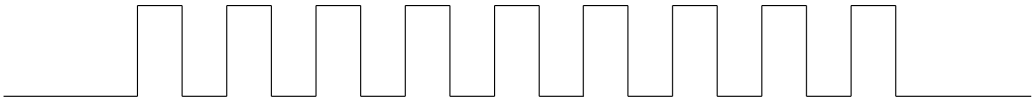


图 26.5-2. 主机时钟正向波形

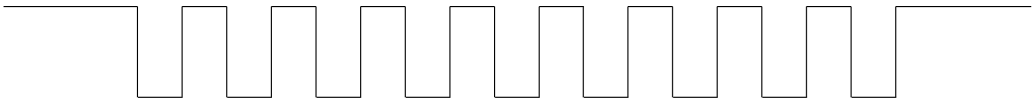


图 26.5-3. 主机时钟负向波形

当 RX 模块作为主机时，必须将时钟源设置为内部自由运行时钟。RX 模块在时钟的上升沿驱动 RXD。

当 RX 模块作为从机时，可能出现的三种情况及其具体要求如下表所示：

表 26.5-3. 时钟限制下 RX 模块从机模式操作要求

时钟条件		具体要求
主机发送时钟	时钟波形	
自由运行时钟	任意波形	对主机的驱动时钟采集边沿没有要求，采集的有效数据以外部使能信号为准。
非自由运行时钟	正向波形（图 26.5-2）	主机需在上升沿驱动数据，RX 模块需在下降沿采集数据（即对主时钟进行翻转）。
非自由运行时钟	负向波形（图 26.5-3）	主机需在下降沿驱动数据，RX 模块需在上升沿采集数据（即使用原始主时钟）

26.5.4 RX 模块接收模式

PARLIO 的 RX 模块有八种接收模式，可根据使能信号规格分为三大类：

- 电平使能模式：接收的数据由外部信号电平使能；
- 脉冲使能模式：接收的数据由外部信号脉冲使能；
- 软件使能模式：数据接收的使能信号可由用户直接配置。

RX 模块还支持对外部使能信号进行翻转。如果外部使能信号为低电平或负脉冲，可通过置位 PARL_IO_RX_EXT_EN_INV 使能该功能，转换为以下相应的接收模式。

26.5.4.1 电平使能模式

电平使能模式如图 26.5-4 所示。在这种模式下，外部使能信号的有效电平必须与有效数据对齐。由于外部电平使能信号需要占用一个 IO 管脚，RXD 可用的 IO 管脚最多为 15 个。

Mode	Sub-mode	Description
		signal level high
LEVEL_ENABLE	\	← Valid data →

图 26.5-4. RX 模块电平使能模式

26.5.4.2 脉冲使能模式

根据脉冲有效电平及其与有效数据的对应性，脉冲使能模式可分为六种子模式，详细分类可参见图 26.5-5。

子模式 1 ~ 4 都包含起始脉冲和结束脉冲，不同之处在于起始脉冲和结束脉冲是否与有效数据对齐。

子模式 5 ~ 6 只包含起始脉冲，通过配置 PARL_IO_RX_BITLEN 可表示有效数据传输结束。

由于外部脉冲使能信号需要占用一个 IO 管脚，RXD 可用的 IO 管脚最多为 15 个。但在子模式 4 和子模式 6 下，数据在脉冲的第一个边沿后和最后一个边沿前被认为有效，可以实现使能信号的 IO 管脚与数据 IO 管脚的复用，此时 RXD 可使用 16 个 IO 管脚。

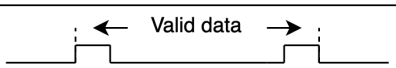
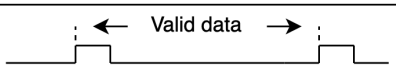
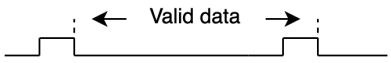
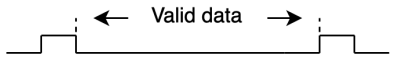
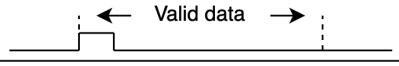
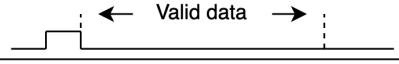
Mode	Sub-mode	Description
PULSE_ENABLE	sub-mode1	*pulse start(data bit include) && pulse end(data bit include)* 
	sub-mode2	*pulse start(data bit include) && pulse end(data bit exclude)* 
	sub-mode3	*pulse start(data bit exclude) && pulse end(data bit include)* 
	sub-mode4	*pulse start(data bit exclude) && pulse end(data bit exclude)* 
	sub-mode5	*pulse start(data bit include) && length end* 
	sub-mode6	*pulse start(data bit exclude) && length end* 

图 26.5-5. RX 模块脉冲使能子模式

26.5.4.3 软件使能模式

软件使能模式下的使能信号由内部配置寄存器决定。用户切换到此模式时，只有同时写 1 到 PARL_IO_RX_SW_EN 和 PARL_IO_RX_START，才能开始接收数据。

由于使能信号不占用接口 IO 管脚，RXD 在接口上的最大可用 IO 管脚数量为 16。由于时钟域不同，使能信号无法与有效数据对齐，因此只能通过有效时钟沿来判定有效数据。此时，RX Core 时钟需要与有效数据一一对应。

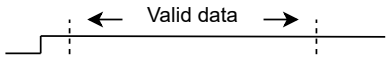
Mode	Sub-mode	Description
SW_ENABLE	/	/ 

图 26.5-6. RX 模块软件使能模式

26.5.5 RX 模块 GDMA SUC EOF 信号生成

RX 模块产生一个 GDMA SUC EOF 信号，表示当前帧传输结束，并将其发送到 GDMA 接口。GDMA SUC EOF 信号可由外部使能信号生成，也可基于内部配置的比特长度触发。

- 当 GDMA SUC EOF 信号由内部配置的比特长度生成时，接收模式的选择不受限制。但在这种情况下必须配置 PARL_IO_RX_BITLEN。如果 PARL_IO_RX_BITLEN 的配置值小于实际接收的数据，将提前触发 GDMA SUC EOF，此时 RX 模块停止从 FIFO 读取数据，但 FIFO 会继续接收外部数据，直到 RX_FIFO_WOVF_INT 中断被触发。
- 当 GDMA SUC EOF 信号由外部使能信号产生时，只可选脉冲使能模式的子模式 1、3。在这种模式下，传输不受 PARL_IO_RX_BITLEN 值的影响，并且传输帧的数据长度不受限制。

26.5.6 TX 模块 EOF 信号生成

TX 模块产生一个 EOF 信号，并生成 TX_EOF_INT 中断表示当前帧传输结束。TX 模块 EOF 信号可由 GDMA 配置的 EOF 信号生成，也可基于内部配置的比特长度触发。

- 当 TX EOF 信号由内部配置的比特长度生成时，但在这种情况下必须配置 PARL_IO_TX_BITLEN。如果 PARL_IO_TX_BITLEN 的配置值小于实际发送的数据，将提前触发 TX EOF。
- 当 TX EOF 信号由 GDMA 配置的 EOF 信号产生时，传输不受 PARL_IO_TX_BITLEN 值的影响，并且传输帧的数据长度不受限制，直到接收到 GDMA 配置的 EOF 信号时，传输才会结束。

26.5.7 RX 模块超时

RX 模块支持接收超时功能。当 RX 模块 FIFO 长时间未接收到有效数据，超时将被触发，产生一个 ERR EOF 信号，并发送至 GDMA 接口以表示结束数据接收。配置 PARL_IO_RX_TIMEOUT_THRES 可设置超时阈值。

默认情况下启用超时功能，用户也可以选择禁用该功能。超时阈值的配置上限为 GDMA 时钟域的 $(2^{16}-1)$ 周期，下限取决于 GDMA 时钟域和 RX Core 时钟域之间的相对频率关系。建议将 PARL_IO_RX_TIMEOUT_THRES 设置为较大的数值，以避免生成非期望的 GDMA ERR EOF 信号。

26.5.8 TX 模块无限制长度数据传输

默认情况下，TX 模块的传输结束标志由 PARL_IO_TX_BITLEN 的配置值决定。当发送的数据达到配置值时，系统会生成一个 TX EOF 中断信号，标志着该数据帧的传输结束。此时，单帧传输的最大值受到 PARL_IO_TX_BITLEN 的位宽限制，使其无法实现无限长度的数据传输。

为了实现更大长度的单帧传输，可以将 PARL_IO_TX_EOF_GEN_SEL 设置为 1。在这种配置下，硬件将使用 GDMA 发送链表配置的 EOF 信号作为单帧传输结束的标志。此时，数据传输不再受限于 PARL_IO_TX_BITLEN 的位宽，从而能够支持更大长度的数据传输。

26.5.9 TX 片选功能

为模拟 LCD 的 I8080 协议，TX 模块配备了一个可输出的片选信号，在拉低时有效。用户可通过配置 PARL_IO_TX_CS_START_DELAY 和 PARL_IO_TX_CS_STOP_DELAY 来控制 DMA 输出数据和片选信号的相位关系。需要注意的是，在开启时钟门控、且使用片选信号作为时钟门控控制源的情况下，对上述两个寄存器的操作在实现基础功能的同时，还可以控制输出时钟和片选信号的相位关系。

26.5.10 TX 模块输出时钟门控

TX 模块支持对输出时钟进行门控。时钟门控功能默认关闭，可通过软件置位 PARL_IO_TX_GATING_EN 使能该功能。

目前 TXD 最高位有两个控制源可配置，分别为 DMA 输出数据和片选信号 (CS)。将 PARL_IO_TX_VALID_OUTPUT_EN 配置为 0 即选择 DMA 输出数据为控制源，此时必须将 TX 模块配置为最大位宽，因为当控制源为 GDMA 输出数据时，门控信号固定为 TXD 最高位，该位为高电平时，时钟信号可以翻转。需要注意的是，在选择 GDMA 输出数据作为控制源时，如果外设处于空闲状态，则此时时钟门控将被用户配置的总线空闲值的最高位控制，应参照小节 26.5.11 *TX 模块总线空闲值* 中的注意事项进行配置；若选择 DMA 输出数据作为门控功能控制源时启用时钟门控功能，门控信号将占用一个 IO，此时 TXD 可以使用的 IO 管脚最多为 15 个。

将 PARL_IO_TX_VALID_OUTPUT_EN 配置为 1 即选择片选信号为控制源，此时对位宽无限制。

26.5.11 TX 模块总线空闲值

TX 模块不传输数据时被认为保持在空闲状态。TX 模块支持总线空闲值可配置。

总线空闲值默认为 0x0，最大可配置值为 0xFF。注意，应避免配置的空闲值与其他已开启的功能产生冲突，例如当使用 TXD 的最高有效位作为有效信号时，应避免将空闲值的最高有效位配置为 1。

26.5.12 单帧数据传输

RX/TX 模块以比特为单位进行传输，即单帧至少传输 1 个比特的数据。

当 RX 模块通过比特长度生成 GDMA EOF 信号时，单帧传输的最大长度是 $(2^{19}-1)$ 比特。当 RX 模块通过外部设备的使能信号产生 GDMA EOF 信号时，单帧传输的比特数没有限制。

当 TX 模块通过比特长度生成 EOF 信号时，单帧传输的最大长度为 $(2^{19}-1)$ 比特。当 TX 模块通过 GDMA 配置的 EOF 生成 TX EOF 信号，单帧传输的比特数不受限制。

由于 PARLIO 在 GDMA 一侧以字节为单位进行传输，因此当 IO 一侧的数据不与字节对齐时，PARLIO 会对数据进行特殊处理。

- 在接收状态下，通过 GDMA 存入内存的数据将自动在高位补 0，从而与字节对齐。
- 在发送状态下，从内存中取出的数据将根据配置的 PARL_IO_TX_BITLEN 值被截断，超过该配置值的数据不会被发送。

当配置的数据总线宽度为 2/4/8/16 位时，比特长度必须配置为对应总线宽度的倍数。

26.5.13 字节范围内数据顺序翻转

总线位宽为 1/2/4 位时，可以对一个字节范围内的数据顺序进行翻转。以 RX 模块为例，当配置总线位宽为 2 比特时，数据在被写入 RX FIFO 之前需要被打包成一个字节。

假设原始的比特数据排列顺序为：

$$\{\{b_0,b_1\},\{b_2,b_3\},\{b_4,b_5\},\{b_6,b_7\}\}$$

则启用该功能后，数据顺序将更改为：

$$\{\{b_6,b_7\},\{b_4,b_5\},\{b_2,b_3\},\{b_0,b_1\}\}$$

26.6 中断

ESP32-S31 的 PARLIO 模块可以生成以下中断信号并发送给中断矩阵。

- PARL_IO_TX_INTR
- PARL_IO_RX_INTR

这些中断信号由 PARLIO 模块的内部中断源生成。表 26.6-1 列出了 PARLIO 的内部中断源、中断触发条件、以及生成的中断信号。

表 26.6-1. PARLIO 的内部中断源

内部中断源	触发条件	中断信号
TX_FIFO_REMPTY_INT	TX 模块 FIFO 空（此时 TX 发送的数据可能出现错误）	PARL_IO_TX_INTR

RX_FIFO_WOVF_INT	RX 模块 FIFO 满（此时 RX 接收的数据可能出现错误）	PARL_IO_RX_INTR
TX_EOF_INT	TX 模块发送完成完整的一帧数据	PARL_IO_TX_INTR

说明：
中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 26.9 寄存器列表。

26.7 配置流程

26.7.1 数据接收操作流程

本节介绍 RX 模块接收数据的配置流程。参照以下流程，从连接到外部设备的 IO 管脚接收并行数据，并存储在内部存储器中。关于 RX 模块时钟和复位的详细操作限制，请参考小节 26.5.2。

1. 复位 RX 模块。具体复位方法与顺序，请参考小节 26.5.2。
2. 置位 PARL_IO_RX_FIFO_WOVF_INT_CLR 和 PARL_IO_RX_FIFO_WOVF_INT_ENA。
3. 选择 RXD IO 管脚。如果用户使用 PAD 时钟，则也需配置时钟 IO 管脚。
4. 配置时钟寄存器，选择时钟源并对时钟进行分频。
5. 关闭 RX 模块 Core 时钟域的时钟。
6. 按照小节 26.3 和 26.5 中的描述，选择所需接收模式并启用相关功能。
7. 配置 GDMA 接收链表。
8. 置位 PARL_IO_RX_REG_UPDATE，同步寄存器信号。
9. 置位 PARL_IO_RX_START。
10. 打开 RX 模块 Core 时钟域的时钟。
11. 操作外部设备，使其发送数据。
12. 轮询 GDMA SUC EOF 中断信号。
13. 清除 GDMA SUC EOF 中断信号。
14. 关闭 RX 模块 Core 时钟域的时钟。
15. 清除 PARL_IO_RX_START。

26.7.2 数据发送操作流程

本节介绍 TX 模块发送数据的编程流程。参照以下流程，将内部存储器中的并行数据传输到与外部设备相连的 IO 管脚。关于 TX 模块时钟和复位的详细操作限制，请参考小节 26.5.2。

1. 复位 TX 模块。具体复位方法与顺序，请参考小节 26.5.2。

2. 依次置位 PARL_IO_TX_FIFO_REMPTY_INT_CLR、PARL_IO_TX_EOF_INT_CLR、PARL_IO_TX_FIFO_REMPTY_INT_ENA 和 PARL_IO_TX_EOF_INT_ENA。
3. 选择 TXD IO 管脚。如果用户使用 PAD 时钟，则也需配置时钟 IO 管脚。
4. 配置时钟寄存器，选择时钟源并对时钟进行分频。
5. 关闭 TX 模块 Core 时钟域的时钟。
6. 按照小节 26.5 中的描述，选择开启所需功能。
7. 配置 GDMA 发送链表。
8. 轮询 PARL_IO_TX_READY。
9. 置位 PARL_IO_TX_START。
10. 打开 TX 模块 Core 时钟域的时钟。
11. 操作外部设备，使其接收数据。
12. 轮询 PARL_IO_TX_EOF_INT_ST 中断信号。
13. 置位 PARL_IO_TX_EOF_INT_CLR 中断信号。
14. 关闭 TX 模块 Core 时钟域的时钟。
15. 清除 PARL_IO_TX_START。

26.8 应用示例

本节介绍 PARLIO 模块的应用示例及其具体操作过程。示例中使用的外围设备都来自 ESP 系列芯片，这些外设均可以和 PARLIO 模块组成完整的数据通路。

说明：
示例中所构建的数据通路可能不是最佳方案，例如，在实际应用中，用户可使用两颗相同 ESP 芯片上的 SPI 外设来完成点对点传输，而无需使用 PARLIO。但本小节中的示例在一定程度上证明了 PARLIO 接口的灵活性。

26.8.1 与 SPI 进行数据传输

在本示例中，外部 SPI 作为主机发送数据，PARLIO RX 模块作为从机接收数据，与此同时，PARLIO TX 模块作为主机发送数据，外部 SPI 作为从机接收数据，从而实现点对点串行数据传输。

- 实现 SPI 发送、PARLIO 接收，请参考以下操作过程：
  1. 配置 SPI 时钟。
  2. 将 SPI 配置为主机。
  3. 配置信号管脚，将 FSPICLK 管脚连接至 PAD_CLK_RX，FSPICS0 管脚连接至 RXD[15]，FSPID 管脚连接至 RXD[0]。
  4. 将发送的数据写入 SPI 缓冲器中，并配置发送数据的位长。
  5. 置位 SPI_UPDATE，更新配置的寄存器值。
  6. 复位 PARLIO RX 模块。

7. 配置 PARLIO RX 模块时钟。
8. 关闭 PARLIO RX 模块 Core 时钟域。
9. 将 PARLIO 接收模式配置为电平使能模式的子模式 1，配置 RX 模块数据总线宽度为 1 位，然后根据 SPI 的发送数据长度配置 PARL_IO_RX_BITLEN，最后置位 PARL_IO_RX_REG_UPDATE。
10. 配置 PARLIO GDMA 接收链表。
11. 置位 PARL_IO_RX_START。
12. 打开 PARLIO RX 模块 Core 时钟域。
13. 置位 SPI_USR，开始传输 SPI 的数据。
14. 轮询 GDMA SUC EOF 中断信号。
15. 清除 PARL_IO_RX_START。

- 实现 PARLIO 发送、SPI 接收，请参考以下操作过程：
  1. 配置 SPI 时钟。
  2. 将 SPI 配置为从机。
  3. 配置信号管脚，将 FSPICLK 管脚连接至 PAD_CLK_TX，FSPICSO 管脚连接至 TXD[15]，FSPID 管脚连接至 TXD[0]。
  4. 置位 SPI_RD_BIT_ORDER，翻转比特数据顺序。
  5. 置位 SPI_UPDATE，更新配置的寄存器值。
  6. 复位 PARLIO TX 模块。
  7. 置位 PARL_IO_TX_EOF_INT_CLR 和 PARL_IO_TX_EOF_INT_ENA。
  8. 配置 PARLIO TX 模块时钟。
  9. 关闭 PARLIO TX 模块 Core 时钟域。
  10. 配置数据总线宽度为 1 位，写 1 至 PARL_IO_TX_VALID_OUTPUT_EN，然后配置 PARL_IO_TX_BITLEN。
  11. 配置 PARLIO GDMA 发送链表。
  12. 轮询 PARL_IO_TX_READY。
  13. 写 1 至 PARL_IO_TX_START。
  14. 打开 PARLIO TX 模块 Core 时钟域。
  15. 开始数据传输。
  16. 轮询 PARL_IO_TX_EOF_INT_ST。
  17. 置位 PARL_IO_TX_EOF_INT_CLR。
  18. 关闭 PARLIO TX 模块 Core 时钟域。
  19. 清除 PARL_IO_TX_START。

26.8.2 与 I2S 进行数据传输

在本示例中，外部 I2S 作为主机发送数据，PARLIO RX 模块作为从机接收数据。PARLIO 支持 I2S TDM MSB 对齐标准模式和 TDM PCM 标准模式下的传输。当 I2S 传输协议为 TDM MSB 对齐标准模式时，需将 PARLIO 的接收模式配置为电平使能模式。当 I2S 传输协议为 TDM PCM 标准模式时，需将 PARLIO 的接收模式配置为脉冲使能模式的子模式 4，并置位 PARL_IO_RX_EXT_EN_INV。

本节以 TDM PCM 标准模式为例，具体操作过程如下：

1. 配置 I2S 时钟。
2. 配置信号管脚，将 I2SO_BCK_out 连接至 PAD_CLK_RX，I2SO_WS_out 连接至 RXD[15]，I2SO_Data_out 连接至 RXD[0]。
3. 将 I2S 配置为主机。
4. 配置所需 I2S TX 数据和通道模式，并置位 I2S_TX_UPDATE。
5. 复位 I2S TX 模块和 TX FIFO。
6. 使能 I2S_TX_DONE_INT。
7. 配置 I2S GDMA 发送链表。
8. 置位 I2S_TX_STOP_EN。
9. 复位 PARLIO RX 模块。
10. 配置 PARLIO RX 模块时钟。
11. 关闭 PARLIO RX 模块 Core 时钟域。
12. 将 PARLIO 接收模式配置为脉冲使能模式的子模式 4，并置位 PARL_IO_RX_EXT_EN_INV，配置 RX 模块数据总线宽度为 1 位，然后根据 I2S 的发送数据长度配置 PARL_IO_RX_BITLEN，最后置位 PARL_IO_RX_REG_UPDATE。
13. 配置 PARLIO GDMA 接收链表。
14. 置位 PARL_IO_RX_START。
15. 打开 PARLIO RX 模块 Core 时钟域。
16. 置位 I2S_TX_START，开始发送数据。
17. 轮询 I2S_TX_DONE_INT。
18. 轮询 GDMA SUC EOF 中断信号。
19. 清除 I2S_TX_START。
20. 清除 PARL_IO_RX_START。

26.8.3 与 LCD 进行数据传输

在本示例中，PARLIO TX 模块作为主机向外部 LCD 发送数据（使用 I8080/MOTO6800 格式），设备间传输 8 位并行数据。具体操作过程如下：

1. 配置信号管脚。将 CLK_TX_out 连接至 LCD 的像素时钟 PCLK，TXD[7:0] 连接至 LCD 数据输入脚，TXD[8] 连接至 LCD 的 CS 脚，TXD[9] 连接至 LCD 的 CD 脚。

2. 复位 PARLIO TX 模块。
3. 依次置位 PARL_IO_TX_FIFO_REMPTY_INT_CLR、PARL_IO_TX_EOF_INT_CLR、PARL_IO_TX_FIFO_REMPTY_INT_ENA 和 PARL_IO_TX_EOF_INT_ENA。
4. 配置 PARLIO TX 模块时钟。
5. 关闭 PARLIO TX 模块 Core 时钟域。
6. 配置数据总线宽度为 16 位，然后配置 PARL_IO_TX_BITLEN。
7. 配置 GDMA 发送链表。注意，在链表中发送的数据应符合 I8080/MOTO6800 格式。低 8 位是有效的并行数据，第 9、10 位分别是 CS 和 CD，最高有效位是常数 1，其余位为任意值。
8. 轮询 PARL_IO_TX_READY。
9. 写 1 至 PARL_IO_TX_START。
10. 打开 PARLIO TX 模块 Core 时钟域。
11. 开始数据传输。
12. 轮询 PARL_IO_TX_EOF_INT_ST。
13. 置位 PARL_IO_TX_EOF_INT_CLR。
14. 关闭 PARLIO TX 模块 Core 时钟域。
15. 清除 PARL_IO_TX_START。

26.9 寄存器列表

本小节的所有地址均为相对于并行 IO 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

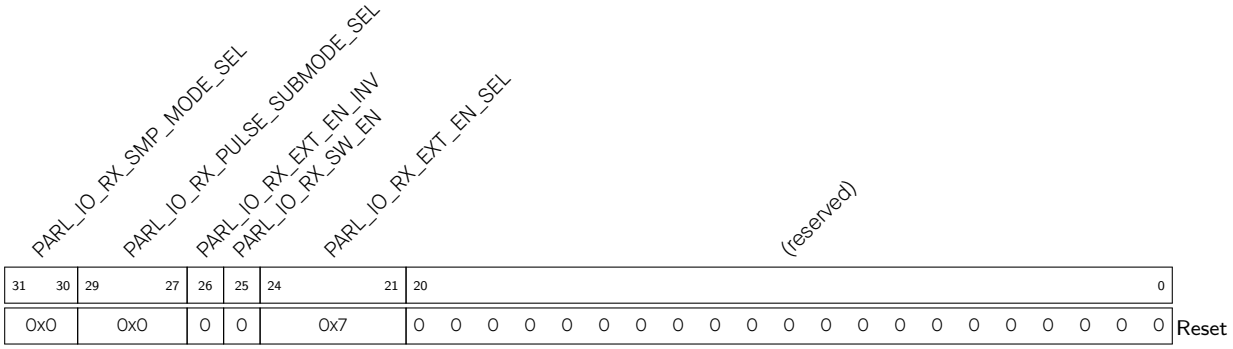
名称	描述	地址	访问
PARLIO RX 配置寄存器			
PARL_IO_RX_MODE_CFG_REG	PARLIO RX 采样模式配置寄存器	0x0000	R/W
PARL_IO_RX_DATA_CFG_REG	PARLIO RX 数据配置寄存器	0x0004	R/W
PARL_IO_RX_GENRL_CFG_REG	PARLIO RX 通用配置寄存器	0x0008	R/W
PARL_IO_RX_START_CFG_REG	PARLIO RX 开始配置寄存器	0x000C	R/W
PARLIO TX 配置寄存器			
PARL_IO_TX_DATA_CFG_REG	PARLIO TX 数据配置寄存器	0x0010	R/W
PARL_IO_TX_START_CFG_REG	PARLIO TX 开始配置寄存器	0x0014	R/W
PARL_IO_TX_GENRL_CFG_REG	PARLIO TX 通用配置寄存器	0x0018	R/W
PARLIO 配置与状态寄存器			
PARL_IO_FIFO_CFG_REG	PARLIO FIFO 配置寄存器	0x001C	R/W
PARL_IO_REG_UPDATE_REG	PARLIO 寄存器更新配置寄存器	0x0020	WT
PARL_IO_ST_REG	PARLIO 状态寄存器	0x0024	RO
PARLIO 中断配置与状态寄存器			
PARL_IO_INT_ENA_REG	PARLIO 中断使能寄存器	0x0028	R/W
PARL_IO_INT_RAW_REG	PARLIO 原始中断寄存器	0x002C	R/SS/WTC
PARL_IO_INT_ST_REG	PARLIO 中断状态寄存器	0x0030	RO
PARL_IO_INT_CLR_REG	PARLIO 中断清除寄存器	0x0034	WT
PARLIO RX/TX 状态寄存器			
PARL_IO_RX_ST0_REG	PARLIO RX 状态寄存器 0	0x0038	RO
PARL_IO_RX_ST1_REG	PARLIO RX 状态寄存器 1	0x003C	RO
PARL_IO_TX_ST0_REG	PARLIO TX 状态寄存器 0	0x0040	RO
PARLIO 时钟配置寄存器			
PARL_IO_RX_CLK_CFG_REG	PARLIO RX 时钟配置寄存器	0x0044	R/W
PARL_IO_TX_CLK_CFG_REG	PARLIO TX 时钟配置寄存器	0x0048	R/W
PARL_IO_CLK_REG	PARLIO 时钟配置寄存器	0x0120	R/W
PARLIO TX 片选配置寄存器			
PARL_IO_TX_CS_CFG_REG	PARLIO TX 片选配置寄存器	0x004C	R/W
PARLIO 版本寄存器			
PARL_IO_VERSION_REG	版本控制寄存器	0x03FC	R/W

26.10 寄存器

本小节的所有地址均为相对于并行 IO 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 如何配置寄存器的保留域。

Register 26.1. PARL_IO_RX_MODE_CFG_REG (0x0000)



PARL_IO_RX_EXT_EN_SEL 配置 RX 外部使能信号的 IO 管脚。(R/W)

PARL_IO_RX_SW_EN 配置是否使能软件数据采集。

0：不使能

1：使能

(R/W)

PARL_IO_RX_EXT_EN_INV 配置是否翻转外部使能信号。

0：无效

1：翻转

(R/W)

PARL_IO_RX_PULSE_SUBMODE_SEL 配置 RXD 脉冲使能子模式。

0：正脉冲起始（包括数据位）& 正脉冲结束（包括数据位）

1：正脉冲起始（包括数据位）& 正脉冲结束（不包括数据位）

2：正脉冲起始（不包括数据位）& 正脉冲结束（包括数据位）

3：正脉冲起始（不包括数据位）& 正脉冲结束（不包括数据位）

4：正脉冲起始（包括数据位）& 长度结束

5：正脉冲起始（不包括数据位）& 长度结束

(R/W)

PARL_IO_RX_SMP_MODE_SEL 配置 RXD 采集模式。

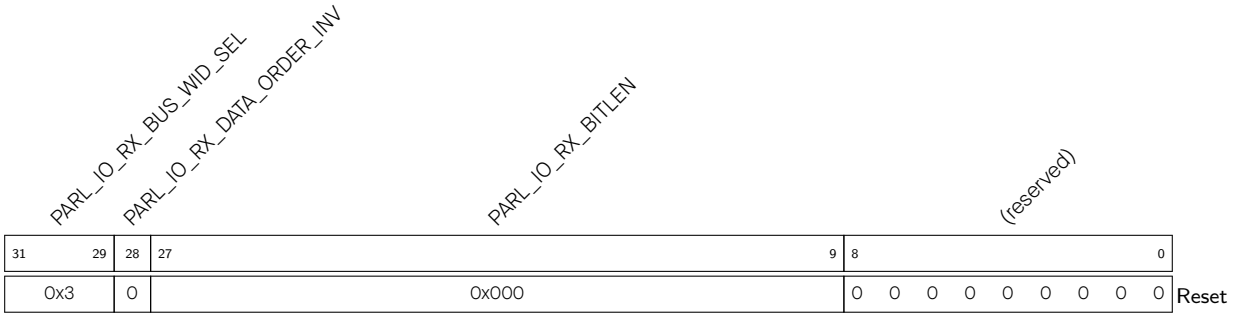
0：外部电平使能模式

1：外部脉冲使能模式

2：内部软件使能模式

(R/W)

Register 26.2. PARL_IO_RX_DATA_CFG_REG (0x0004)



PARL_IO_RX_BITLEN 配置 RXD 的比特长度。(R/W)

PARL_IO_RX_DATA_ORDER_INV 配置是否翻转从 RX_FIFO 发送到 DMA 的数据的字节范围内比特顺序。

0：无效

1：翻转数据顺序

(R/W)

PARL_IO_RX_BUS_WID_SEL 配置 RXD 总线位宽。

0：1 位

1：2 位

2：4 位

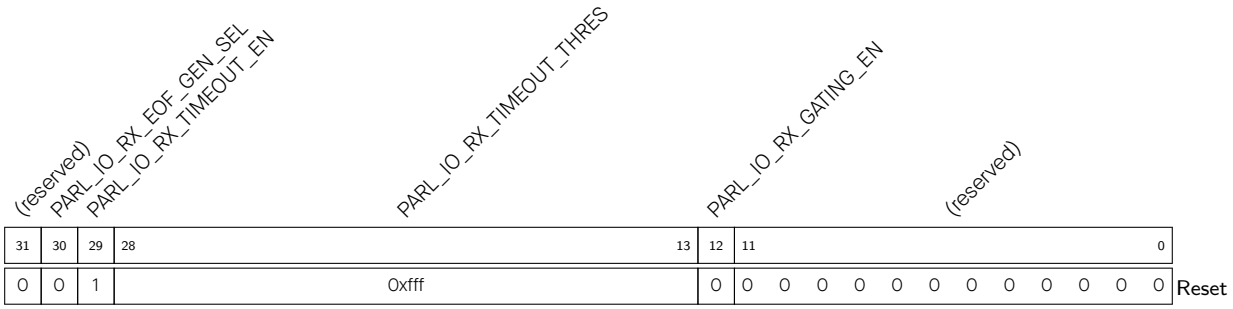
3：8 位

4：16 位

5 ~ 15：无效值，会引发错误

(R/W)

Register 26.3. PARL_IO_RX_GENRL_CFG_REG (0x0008)



PARL_IO_RX_GATING_EN 配置是否使能 RX 输出时钟的时钟门控。

0：不使能

1：使能

(R/W)

PARL_IO_RX_TIMEOUT_THRES 配置 RX 超时计数器的阈值。(R/W)

PARL_IO_RX_TIMEOUT_EN 配置是否使能超时功能来生成 DMA ERR EOF。

0：不使能

1：使能

(R/W)

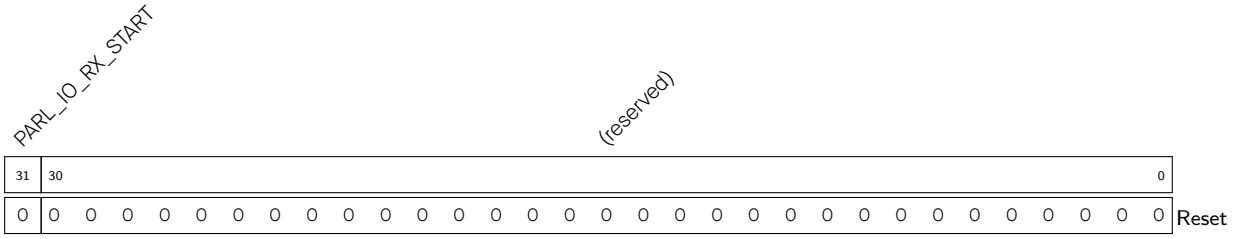
PARL_IO_RX_EOF_GEN_SEL 配置 DMA SUC EOF 的生成模式。

0：通过配置的数据比特长度生成 DMA SUC EOF

1：通过外部使能信号生成 DMA SUC EOF

(R/W)

Register 26.4. PARL_IO_RX_START_CFG_REG (0x000C)



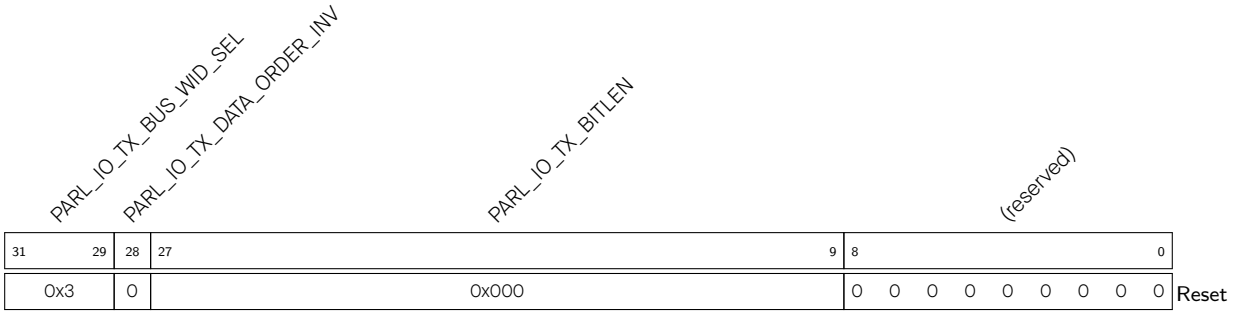
PARL_IO_RX_START 配置是否开始 RX 数据采集。

0：无效

1：开始

(R/W)

Register 26.5. PARL_IO_TX_DATA_CFG_REG (0x0010)



PARL_IO_TX_BITLEN 配置 TXD 的比特长度。(R/W)

PARL_IO_TX_DATA_ORDER_INV 配置是否翻转从 TX_FIFO 发送到 IO 的数据的字节范围内比特顺序。

0：无效

1：翻转数据顺序

(R/W)

PARL_IO_TX_BUS_WID_SEL 配置 TXD 总线位宽。

0：1 位

1：2 位

2：4 位

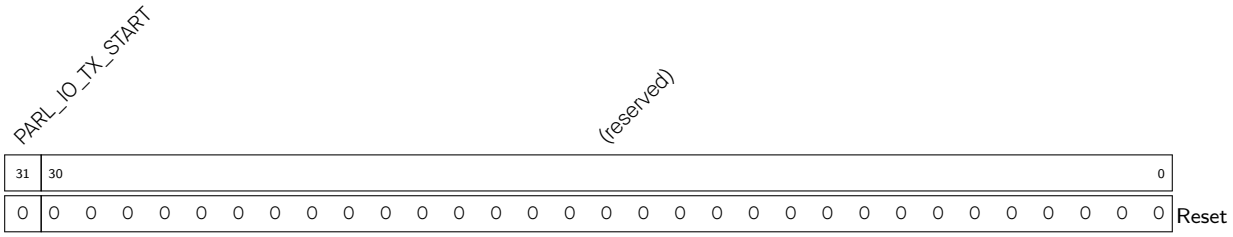
3：8 位

4：16 位

5 ~ 15：无效值，会引发错误

(R/W)

Register 26.6. PARL_IO_TX_START_CFG_REG (0x0014)



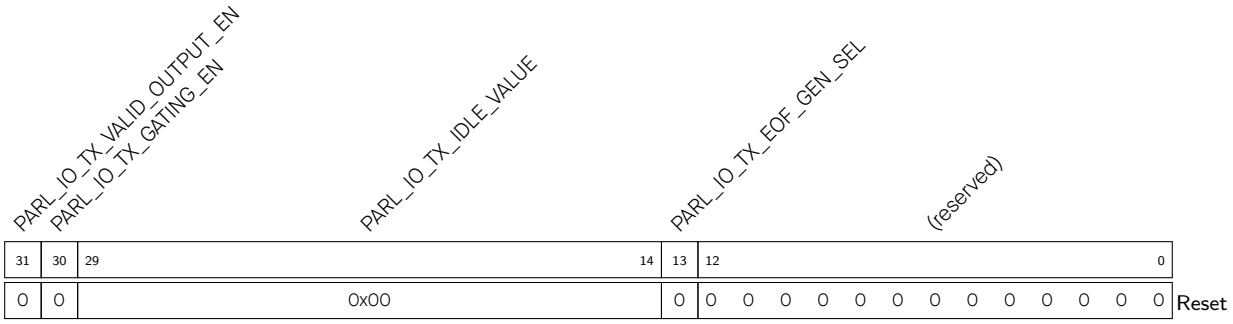
PARL_IO_TX_START 配置是否开始 TX 数据输出。

0：无效

1：开始

(R/W)

Register 26.7. PARL_IO_TX_GENRL_CFG_REG (0x0018)



PARL_IO_TX_EOF_GEN_SEL 配置 TX EOF 的生成模式。

0：通过配置的数据比特长度生成 TX EOF

1：通过 DMA EOF 信号生成 TX EOF

(R/W)

PARL_IO_TX_IDLE_VALUE 配置 TX 总线上在未发送状态下的数据值。(R/W)

PARL_IO_TX_GATING_EN 配置是否使能 TX 输出时钟的时钟门控。

0：不使能

1：使能

(R/W)

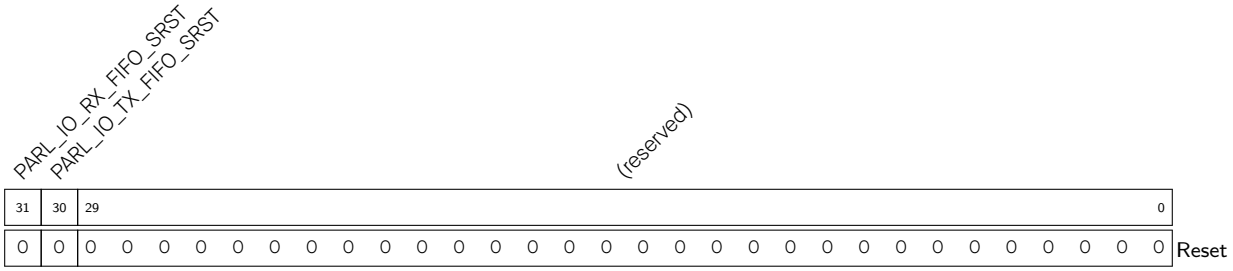
PARL_IO_TX_VALID_OUTPUT_EN 配置使能 DMA 输出数据或者片选信号作为 TXD 最高位控制源。

0：选择 DMA 输出数据为控制源

1：选择片选信号为控制源

(R/W)

Register 26.8. PARL_IO_FIFO_CFG_REG (0x001C)



PARL_IO_TX_FIFO_SRST 配置是否复位 TX 异步 FIFO。

0：无效

1：复位

(R/W)

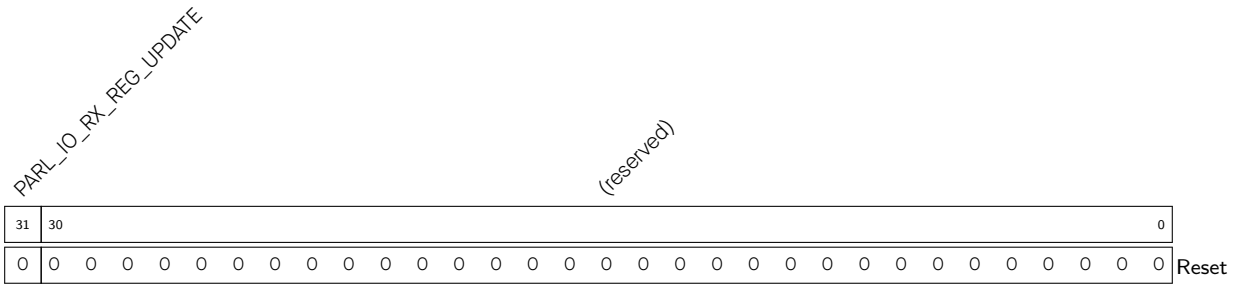
PARL_IO_RX_FIFO_SRST 配置是否复位 RX 异步 FIFO。

0：无效

1：复位

(R/W)

Register 26.9. PARL_IO_REG_UPDATE_REG (0x0020)



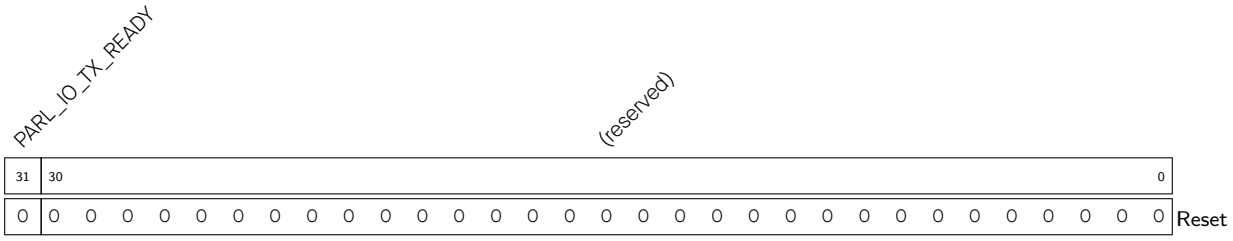
PARL_IO_RX_REG_UPDATE 配置是否更新 RX 寄存器配置。

0：无效

1：更新

(WT)

Register 26.10. PARL_IO_ST_REG (0x0024)



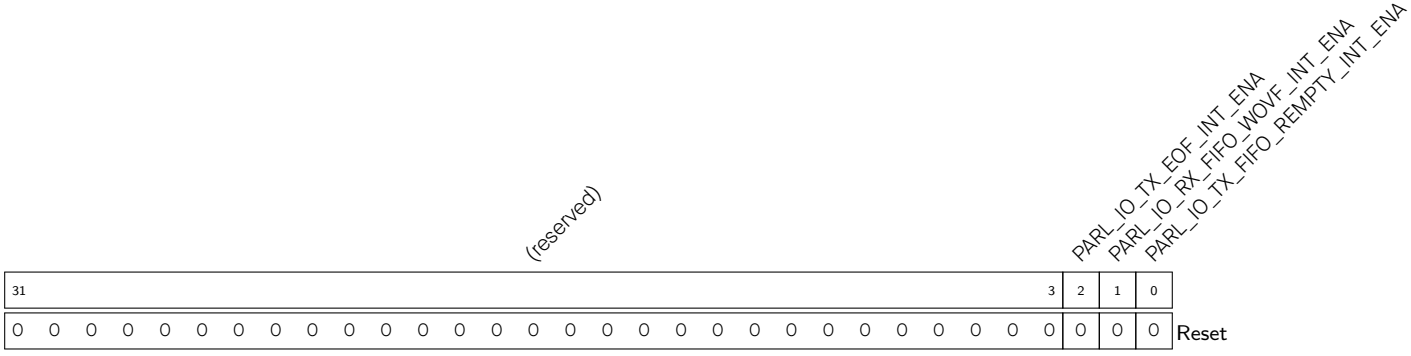
PARL_IO_TX_READY 表示 TX 模块是否已准备好传输数据。

0：未准备好

1：已准备好

(RO)

Register 26.11. PARL_IO_INT_ENA_REG (0x0028)

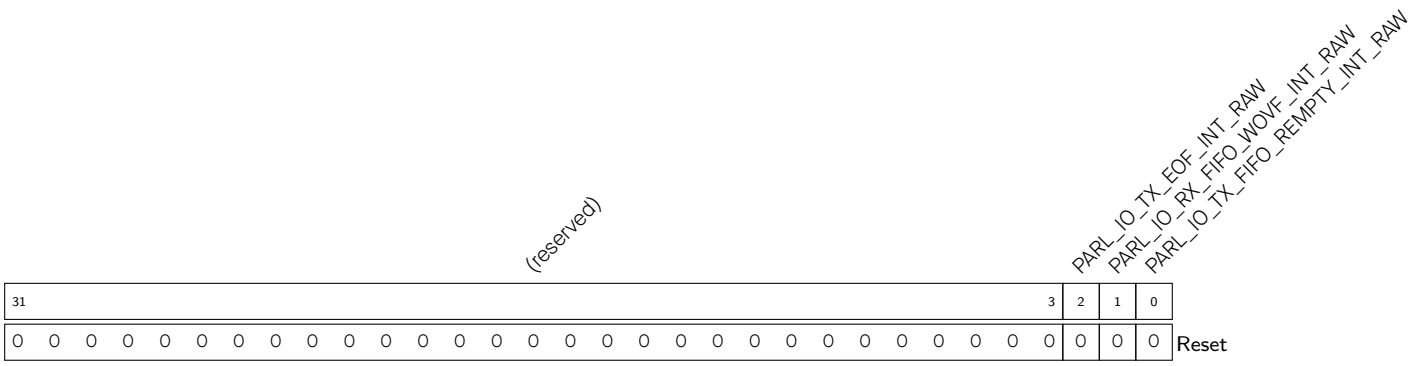


PARL_IO_TX_FIFO_REMPTY_INT_ENA 写 1 使能 TX_FIFO_REMPTY_INT 的屏蔽中断状态。(R/W)

PARL_IO_RX_FIFO_WOVF_INT_ENA 写 1 使能 RX_FIFO_WOVF_INT 的屏蔽中断状态。(R/W)

PARL_IO_TX_EOF_INT_ENA 写 1 使能 TX_EOF_INT 的屏蔽中断状态。(R/W)

Register 26.12. PARL_IO_INT_RAW_REG (0x002C)

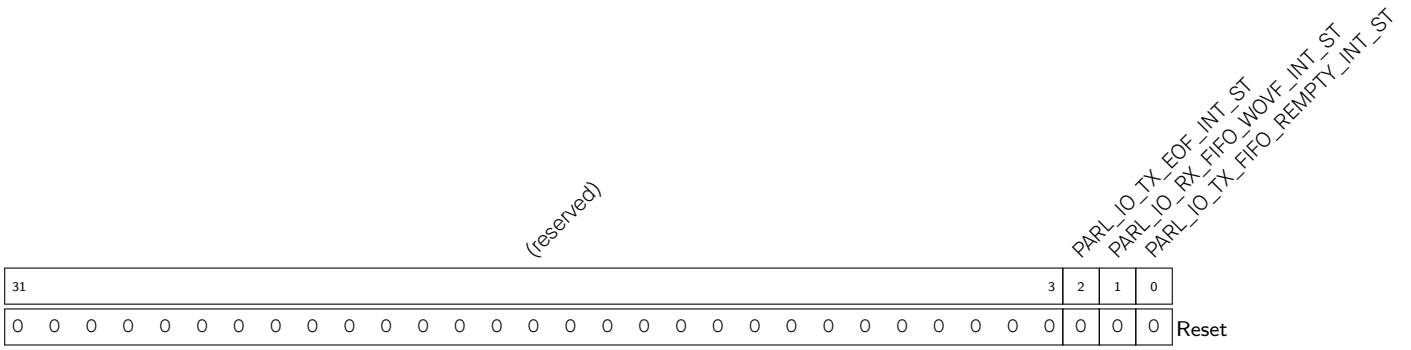


PARL_IO_TX_FIFO_REMPTY_INT_RAW TX_FIFO_REMPTY_INT 的原始中断状态。(R/WTC/SS)

PARL_IO_RX_FIFO_WOVF_INT_RAW RX_FIFO_WOVF_INT 的原始中断状态。(R/WTC/SS)

PARL_IO_TX_EOF_INT_RAW TX_EOF_INT 的原始中断状态。(R/WTC/SS)

Register 26.13. PARL_IO_INT_ST_REG (0x0030)

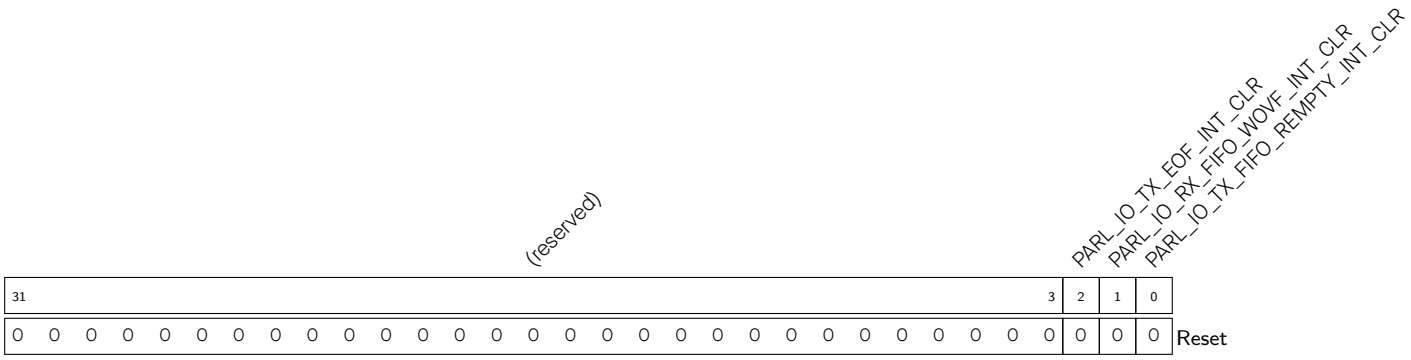


PARL_IO_TX_FIFO_REMPTY_INT_ST TX_FIFO_REMPTY_INT 的屏蔽中断状态。(RO)

PARL_IO_RX_FIFO_WOVF_INT_ST RX_FIFO_WOVF_INT 的屏蔽中断状态。(RO)

PARL_IO_TX_EOF_INT_ST TX_EOF_INT 的屏蔽中断状态。(RO)

Register 26.14. PARL_IO_INT_CLR_REG (0x0034)

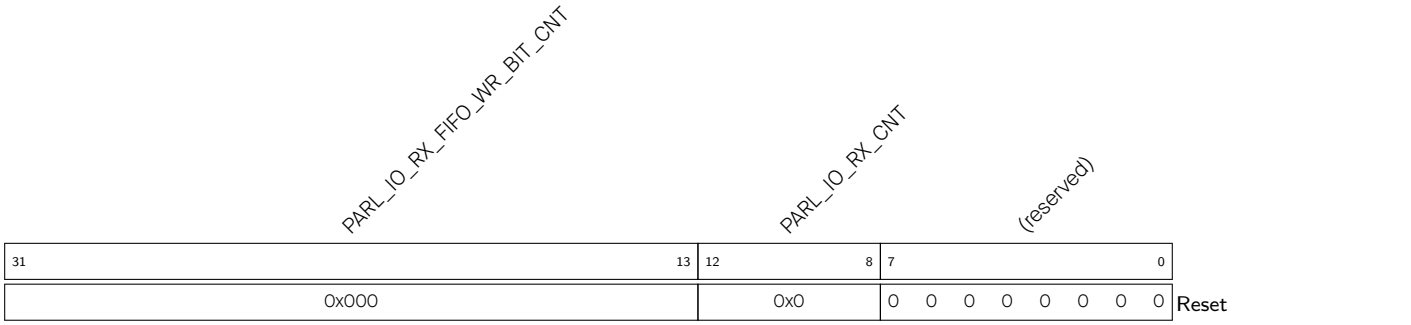


PARL_IO_TX_FIFO_REMPTY_INT_CLR 写 1 清除 TX_FIFO_REMPTY_INT。(WT)

PARL_IO_RX_FIFO_WOVF_INT_CLR 写 1 清除 RX_FIFO_WOVF_INT。(WT)

PARL_IO_TX_EOF_INT_CLR 写 1 清除 TX_EOF_INT。(WT)

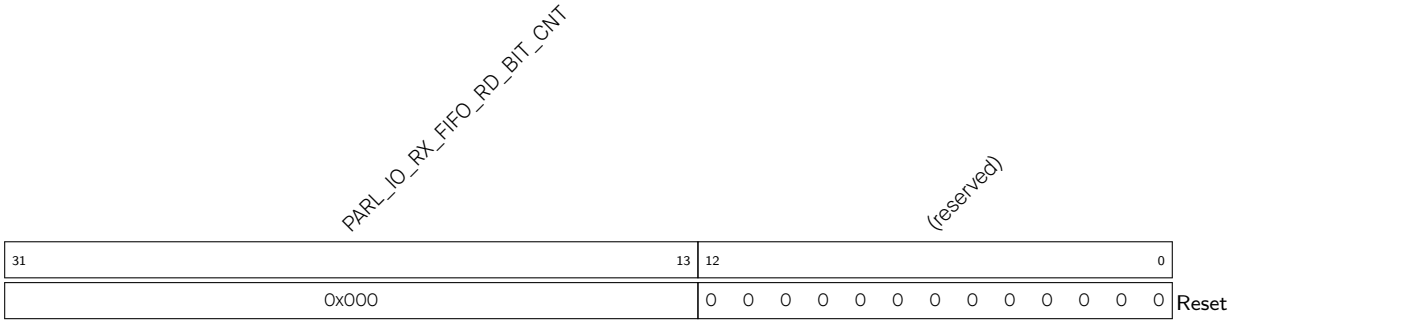
Register 26.15. PARL_IO_RX_ST0_REG (0x0038)



PARL_IO_RX_CNT 表示读取 RX FIFO 的时钟周期数。(RO)

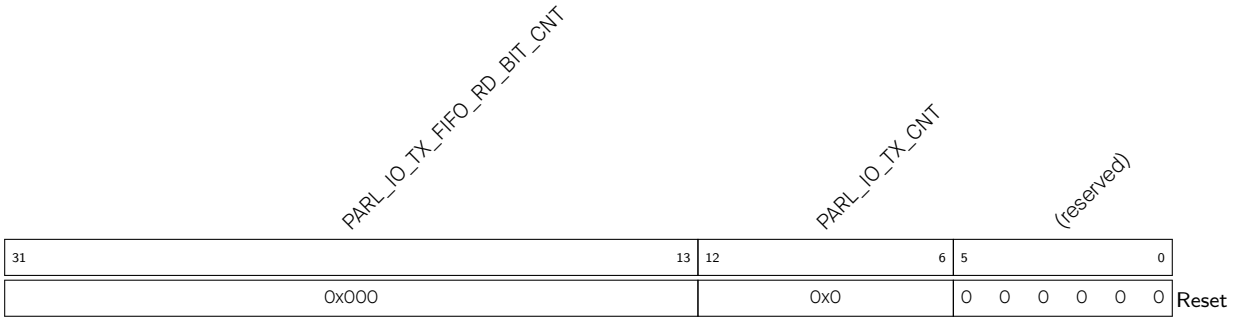
PARL_IO_RX_FIFO_WR_BIT_CNT 表示当前写入 RX FIFO 的位数。(RO)

Register 26.16. PARL_IO_RX_ST1_REG (0x003C)



PARL_IO_RX_FIFO_RD_BIT_CNT 表示当前从 RX FIFO 中读取的位数。(RO)

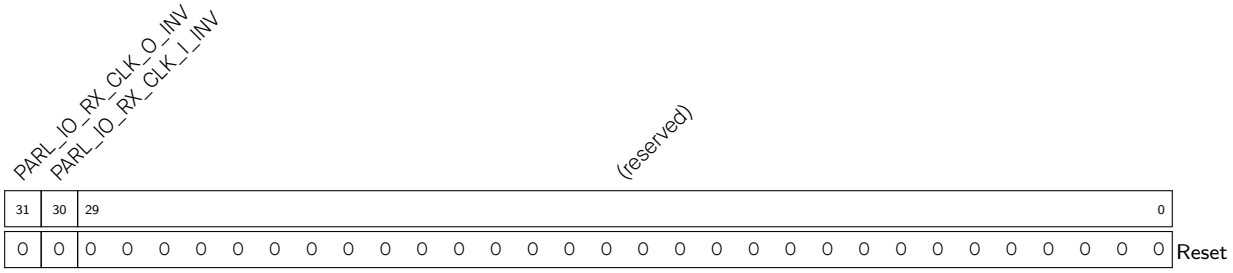
Register 26.17. PARL_IO_TX_ST0_REG (0x0040)



PARL_IO_TX_CNT 表示读取 TX FIFO 的周期数。(RO)

PARL_IO_TX_FIFO_RD_BIT_CNT 表示当前从 TX FIFO 中读取的位数。(RO)

Register 26.18. PARL_IO_RX_CLK_CFG_REG (0x0044)



PARL_IO_RX_CLK_I_INV 配置是否翻转 RX 的输入 Core 时钟。

0：无效

1：翻转

(R/W)

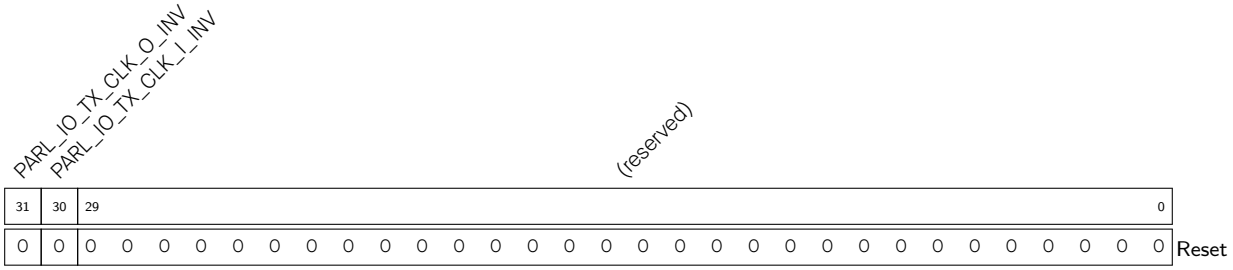
PARL_IO_RX_CLK_O_INV 配置是否翻转 RX 的输出 Core 时钟。

0：无效

1：翻转

(R/W)

Register 26.19. PARL_IO_TX_CLK_CFG_REG (0x0048)



PARL_IO_TX_CLK_I_INV 配置是否翻转 TX 的输入 Core 时钟。

0：无效

1：翻转

(R/W)

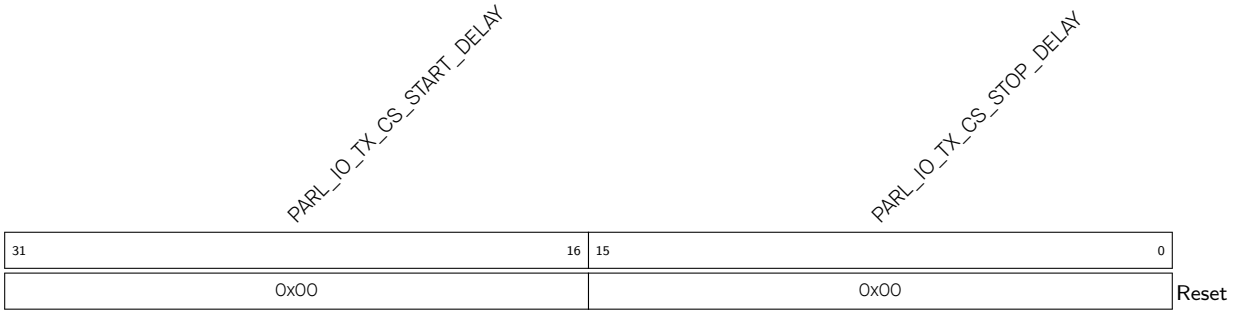
PARL_IO_TX_CLK_O_INV 配置是否翻转 TX 的输出 Core 时钟。

0：无效

1：翻转

(R/W)

Register 26.20. PARL_IO_TX_CS_CFG_REG (0x004C)



PARL_IO_TX_CS_STOP_DELAY 配置 TX 数据发送结束与 TX_CS_O 上升沿之间的延迟。

单位：TX Core 时钟周期

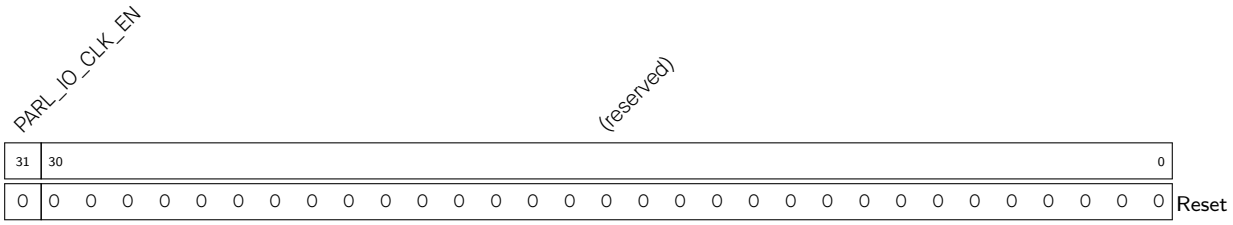
(R/W)

PARL_IO_TX_CS_START_DELAY 配置 TX_CS_O 下降沿与 TX 数据开始传输之间的延迟。

单位：TX Core 时钟周期

(R/W)

Register 26.21. PARL_IO_CLK_REG (0x0120)



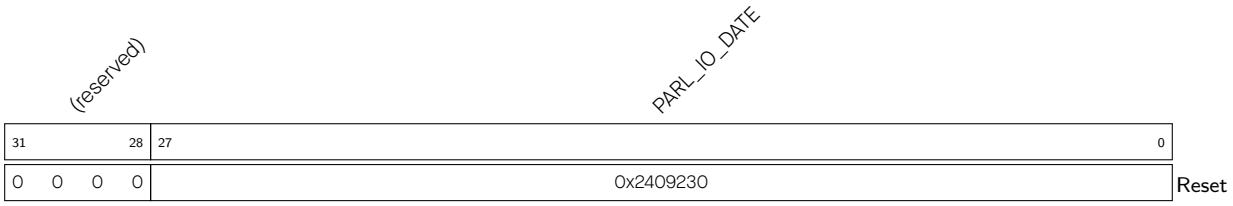
PARL_IO_CLK_EN 寄存器时钟门控。

0：只在读写寄存器时开启寄存器时钟

1：始终开启寄存器时钟

(R/W)

Register 26.22. PARL_IO_VERSION_REG (0x03FC)



PARL_IO_DATE 版本控制寄存器。(R/W)

第 VI 卷

模拟和混合信号处理

该部分描述了与模数转换、电压比较、片上传感器以及温度和触摸感应等功能相关的模块，展示了系统在处理模拟信号方面的性能。

第 27 章

触摸传感器 (TOUCH)

ESP32-S31 搭载了 14 个电容式触摸传感器单元，每个单元可通过一个触摸管脚外接触摸板，构成触摸传感器系统。ESP32-S31 触摸传感器系统主要应用于人机交互场景中，检测手指触碰或接近，此外还支持跳频采样、防潮功能和遇水保护功能。

27.1 术语

为了更好地对电容式触摸传感器进行说明，本章及后续小节使用了以下术语。

触摸传感器 (Touch Sensor)	实现触摸传感功能的内部电路。
触摸管脚 (Touch Pin)	具有触摸传感功能的芯片管脚。
触摸板 (Touch Panel)	芯片外部连接的用于接收触摸动作的设备。
触摸传感器系统 (Touch Sensor System)	触摸管脚、触摸传感器、触摸板、以及连接各部分的电路共同构成触摸传感器系统。

说明：
为方便描述，对触摸板进行采样、扫描、测量、充放电等操作，即表示对触摸管脚进行采样、扫描、测量、充放电等操作。

27.2 特性

ESP32-S31 的触摸传感器模块具有以下特性：

- 支持 14 个电容触摸管脚的检测
- 可由软件或专用硬件定时器触发采样操作
- 支持两种采样方式：
  - 将来自触摸管脚的脉冲序列信号作为时钟信号处理，利用该时钟来计数采样周期
  - 将来自触摸管脚的脉冲序列信号作为数字信号处理，利用系统时钟采样该数字信号的上升沿来计数采样周期
- 支持扫描模式，可配置 Touch FSM 按照固定顺序对多个触摸管脚进行采样
- 支持超时机制，监测通道异常
- 支持跳频采样，增加检测的抗干扰性
- 支持接近感应模式，最多可配置三个通道
- 支持配置单个触摸传感器在休眠模式时正常工作
- 支持触摸传感器用作唤醒源

- 支持防潮功能
- 支持遇水保护功能

27.3 系统架构

27.3.1 触摸板

ESP32-S31 触摸传感器的触摸板主要由三部分组成：

- 电极 (Electrode)：手指触碰时，电极电容发生变化。
- 基片 (Substrate)：基础材料，承载平面保护层、电极及接入相应通道需要的连接部件。
- 平面保护层 (Protective cover)：将其余部件与外界环境隔开。

用户需自行设计触摸板，确定触摸板用材及排列方式等，具体信息可参考 触摸传感器应用方案简介。触摸板经触摸管脚（通道）连接至 ESP32-S31 提供的触摸传感器，如下图 27.3-1 所示。

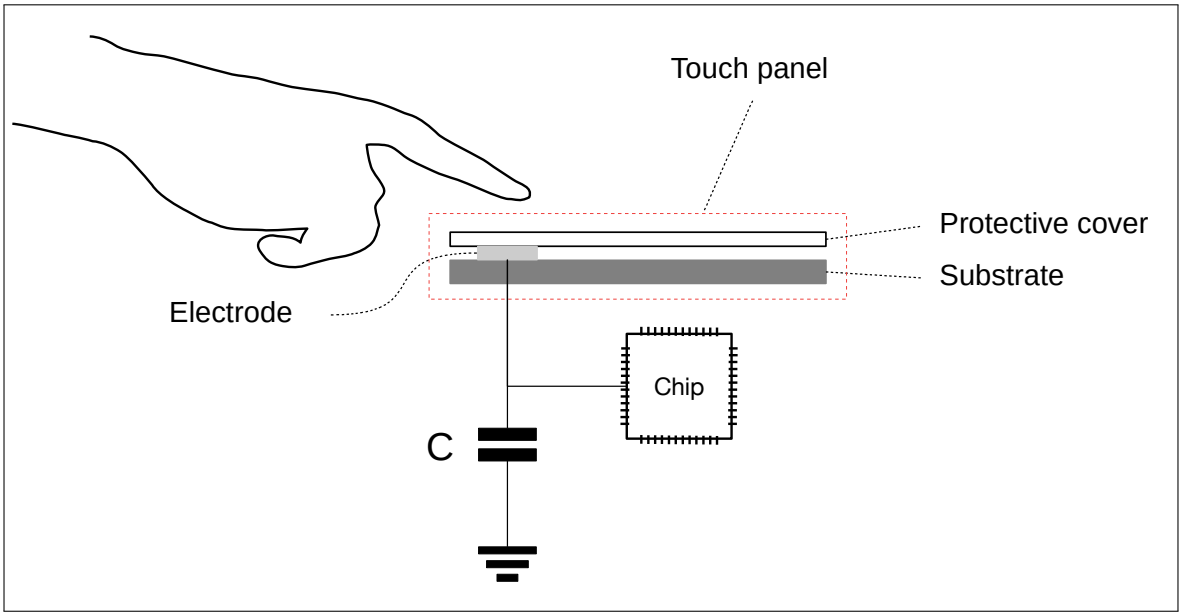


图 27.3-1. 触摸传感器系统

用户触碰保护层时，电极电容增加。如果该电极通过通道已连接至 ESP32-S31 触摸传感器，则传感器将检测到电极的电容变化。电容变化超过阈值（可配置），传感器将触发中断。

27.3.2 电容触摸管脚

ESP32-S31 搭载 14 个电容触摸管脚 (T0 ~ T13)，通过芯片电容触摸管脚与外部触摸板连接，用于检测外部的触摸情况。对应的连接关系见表 27.3-1。

表 27.3-1. ESP32-S31 电容式触摸传感器的管脚

电容触摸管脚	对应 GPIO 管脚
T0	GPIO6
T1	GPIO7

电容触摸管脚	对应 GPIO 管脚
T2	GPIO8
T3	GPIO9
T4	GPIO10
T5	GPIO11
T6	GPIO12
T7	GPIO13
T8	GPIO14
T9	GPIO15
T10	GPIO16
T11	GPIO17
T12	GPIO18
T13	GPIO19

27.3.3 触摸传感器

触摸传感器使用固定电源对触摸管脚进行充放电，每次充放电会生成一个脉冲信号，即 TOUCH_OUT 信号。触摸传感器可检测到触摸管脚上的电容变化，当手指触碰或靠近触摸管脚时，管脚电容增大，充放电变慢，因而产生脉冲信号的频率变慢。通过测量产生固定次数个脉冲信号所需的时间变化，即可判断管脚是否被触碰。

图 27.3-2 展示了触摸传感器的内部结构。每个触摸传感器均有一组输入输出信号，其中一些信号连接至 Touch FSM，见小节 27.4.1。表 27.3-2 展示了触摸传感器的各个输出信号及其功能。

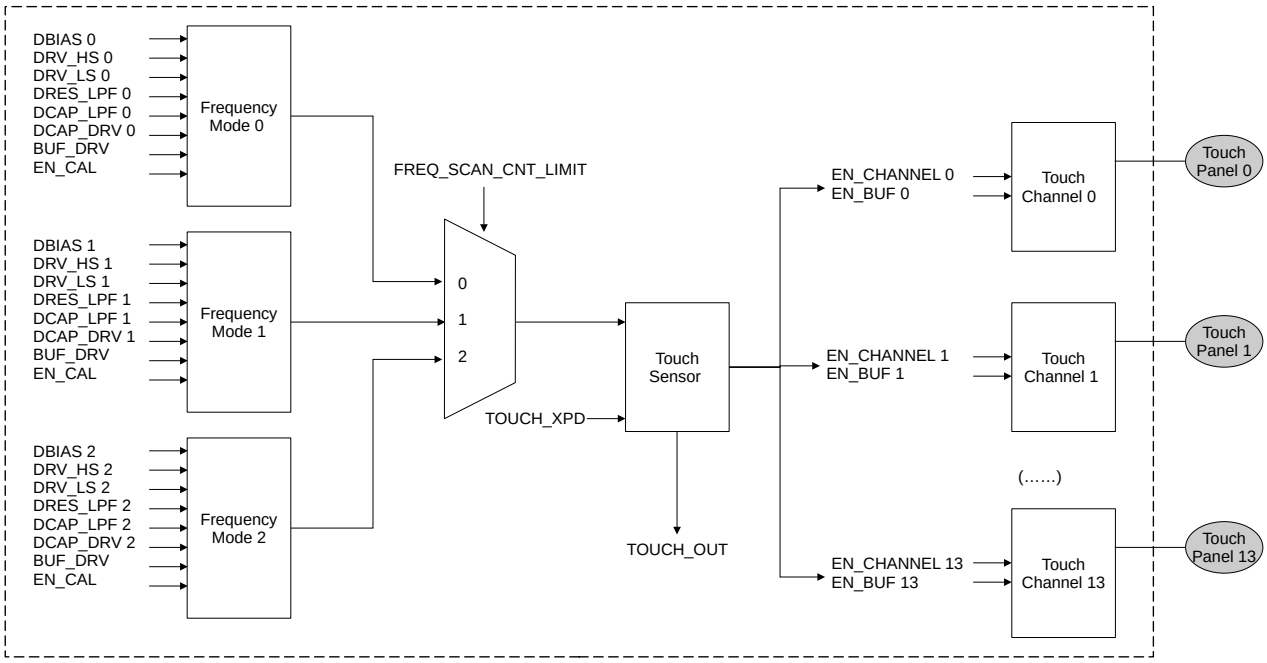


图 27.3-2. 触摸传感器内部结构

表 27.3-2. 电容式触摸传感器输出信号

信号	信号源	功能
TOUCH_XPD	Touch FSM	控制触摸传感器的上下电
TOUCH_START	Touch FSM	控制触摸传感器发起一次测量。Touch FSM 拉低 TOUCH_START 信号，测量结束
DBIAS	Touch FSM	FSM 会根据不同频率模式选择不同的 DBIAS 参数，用于调节触摸传感器内部 LDO 的电压，从而影响触摸传感器的充放电速度
DRV_HS	Touch FSM	FSM 会根据不同频率模式选择不同的 DRV_HS 参数，用于调节触摸传感器的高速驱动能力
DRV_LS	Touch FSM	FSM 会根据不同频率模式选择不同的 DRV_LS 参数，用于调节触摸传感器的低速驱动能力
DRES_LPF	Touch FSM	FSM 会根据不同频率模式选择不同的 DRES_LPF 参数，用于调节 RC 低通滤波器，调节档位分为 0 kΩ、3 kΩ、4.5 kΩ 和 7.5 kΩ。
DCAP_LPF	Touch FSM	FSM 会根据不同频率模式选择不同的 DCAP_LPF 参数，用于调节 RC 低通滤波器，调节范围为 0 ~ 2.54 pF，调节精度为 20 fF
DCAP_DRV	Touch FSM	用于在测量模式下调节连接触摸通道的内部电容值，通常设置为 0
EN_CAL	Touch FSM	用于断开和触摸管脚的连接，可以测试触摸传感器内部的电容（需要关闭触摸板）
BUF_DRV	Touch FSM	调节 TOUCH BUFF 的驱动能力
EN_BUF	Touch FSM	使能 TOUCH BUFF（软件可配置的一块触摸板）用于防潮模式，缓和小水滴对传感器的影响
TOUCH_OUT	触摸传感器	Touch FSM 对 TOUCH_OUT 信号中的脉冲进行计数，测量产生 N 个脉冲需要的时间

27.4 功能描述

27.4.1 Touch FSM

触摸传感器的 Touch FSM 负责执行测量操作，包括选择要测量的触摸传感器、为触摸传感器生成需要的信号、以及处理触摸传感器输出的信号等。Touch FSM 的内部结构及工作原理见图 27.4-1，时钟源为 LP_DYN_FAST_CLK。关于 LP_DYN_FAST_CLK 更多信息，请参考章节 7 复位和时钟 *[to be added later]*。

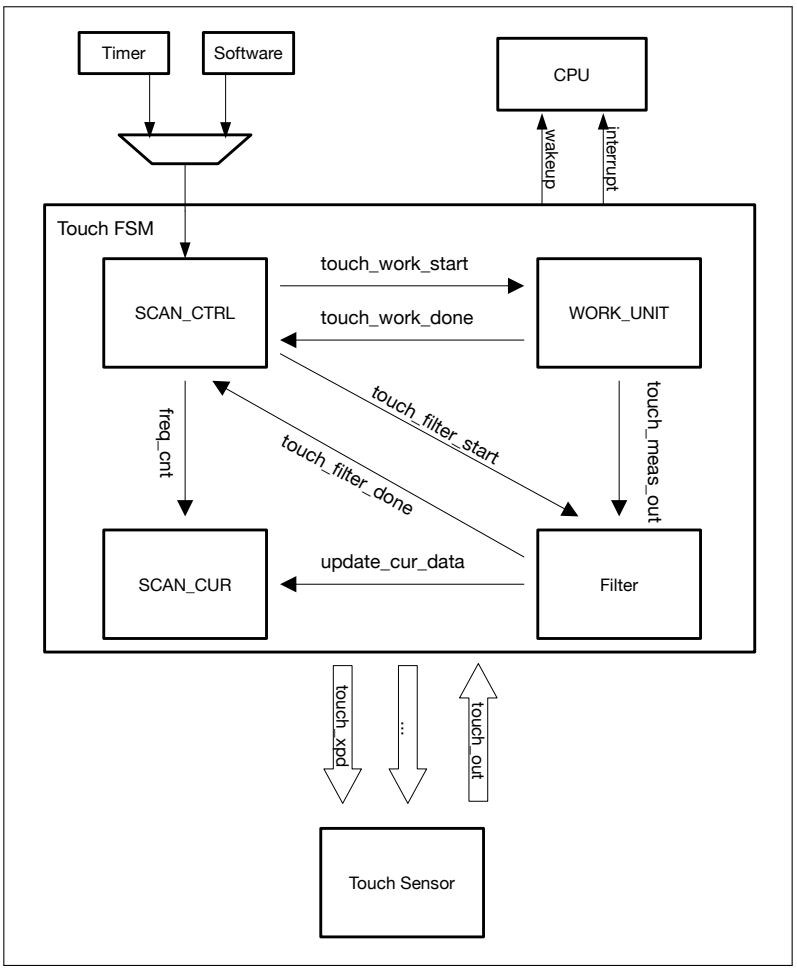


图 27.4-1. Touch FSM 内部结构和工作原理

Touch FSM 各个模块的功能如下：

- SCAN_CTRL 模块：扫描模式下，选择要测量的触摸传感器以及跳频等参数，控制每个触摸传感器的测量启动和结束。
- WORK_UNIT 模块：测量过程中，负责采样已选择的触摸传感器。
- Filter 模块：使能此模块，则触摸传感器在测量时将使用 IIR 滤波器，返回的滤波值即为采样值。详细信息见小节 27.4.3.1。
- SCAN_CUR 模块：测量过程中，用于缓存不同触摸传感器在不同频率模式下的参数，用于下一次测量时的计算。

27.4.2 采样信号预处理

触摸传感器的核心是通过检测触摸板上的电容变化，输出对应的脉冲序列（TOUCH_OUT 信号），通过计算触摸和非触摸状态下 TOUCH_OUT 信号的变化，来反映触摸板是否被触摸，因此，对 TOUCH_OUT 信号的采样十分关键。图 27.4-2 展示了对 TOUCH_OUT 信号预处理的过程。

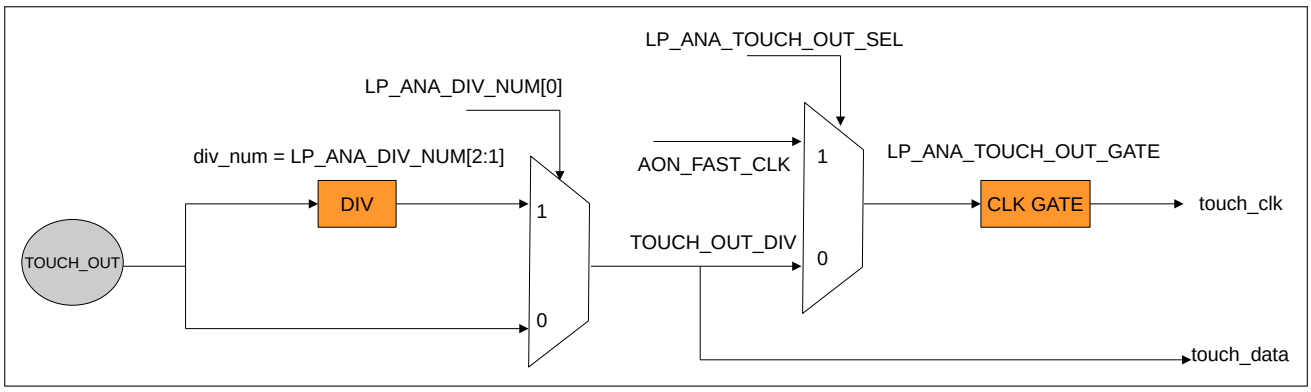


图 27.4-2. TOUCH_OUT 信号预处理

当 TOUCH_OUT 信号的频率远高于采样频率时，为了提高信号采样的准确性，可以通过配置 TOUCH_AON_DIV_NUM*n* 对 TOUCH_OUT 信号进行分频处理。*n* 代表在跳频模式（参考小节 27.4.2.4）下的不同频率模式，取值为 0 ~ 2。该寄存器的最低位为是否开启分频的标志位，若最低位配置为 0，则表示不对 TOUCH_OUT 信号进行分频处理，若最低位配置为 1，则表示需要对 TOUCH_OUT 信号进行分频处理。该寄存器的高两位表示分频系数，取值范围为 0、1、2、3，对应的分频范围为 0、2、4、6 分频。

若分频处理后，频率依然高于采样频率的两倍，可以配置 TOUCH_AON_TOUCH_OUT_SEL 为 1，将分频处理后的 TOUCH_OUT 信号直接当作时钟使用。配置 TOUCH_AON_TOUCH_OUT_GATE 可控制开启或关闭该时钟。

27.4.2.1 测量过程

根据是否开启跳频，触摸传感器的一次测量操作可以分为以下两种流程：

- 非跳频采样：
  1. Touch FSM 选择需要测量的触摸传感器，将相关配置信号发送到该传感器。
  2. Touch FSM 发送 START 信号到触摸传感器，初始化测量操作。Touch FSM 启动内部触摸计数器 (touch counter) 记录测量持续时长。
  3. Touch FSM 内部的脉冲计数器 (pulse counter) 开始对 TOUCH_OUT 信号的脉冲数量进行计数。
  4. 脉冲计数器达到 TOUCH_AON_TOUCH_MEAS_NUM0 中设定的计数阈值，则测量结束。START 信号拉低，触摸计数器停止计数。[触摸计数器的值 × LP_DYN_SLOW_CLK 周期] 即为对触摸管脚进行 TOUCH_AON_TOUCH_MEAS_NUM0 次充放电所需的时间。
- 跳频采样：
  1. Touch FSM 选择需要测量的触摸传感器，将相关配置信号发送到该传感器。
  2. 置位 TOUCH_AON_FREQ_SCAN_EN 开启跳频模式，配置 TOUCH_AON_FREQ_SCAN_CNT_LIMIT 选择所支持跳频的频率模式个数，最大支持三种采样频率模式，并配置各频率模式所需的触摸传感器参数以及采样 TOUCH_OUT 信号的脉冲次数。
  3. Touch FSM 发送 START 信号到触摸传感器，初始化测量操作。Touch FSM 启动内部触摸计数器 (touch counter) 记录测量持续时长。
  4. Touch FSM 内部的脉冲计数器 (pulse counter) 开始对 TOUCH_OUT 信号的脉冲数量进行计数。

5. 脉冲计数器达到对应频率模式 TOUCH_AON_TOUCH_MEAS_NUM*n* 中设定的计数阈值，则当前频率测量结束，触摸计数器的值即为当前频率模式下对触摸管脚进行 TOUCH_AON_TOUCH_MEAS_NUM*n* 次充放电所需时间。
6. 采样频率计数器加 1，触摸计数器清零，等待一小段延时后开始下一频率模式的采样。当所有频率模式采样结束后，START 信号拉低，触摸计数器清零。

采样值（测量值）为触摸计数器的值，表示为原始触摸数据 (touch_raw_data)。用户可从 TOUCH_PAD*n*_DATA 中读取原始触摸数据的值。注意，TOUCH_PAD*n*_DATA 返回的值也可能是滤波后的原始触摸数据，即平滑触摸数据 (touch_smooth_data) 和基准数据 (benchmark)。用户可配置 TOUCH_AON_TOUCH_DATA_SEL 和 TOUCH_AON_TOUCH_FREQ_SEL 选择具体的返回值类型。有关采样值返回类型的详细信息，以及如何使用这些数据来判断触摸动作，请参考小节 27.4.3.1。

说明：
注意，如果脉冲计数器长时间未达到设置的计数阈值，而触摸计数器已达到 TOUCH_AON_TOUCH_TIMEOUT_NUM 中设定的超时阈值，且触摸计数器超时中断使能寄存器 TOUCH_AON_TOUCH_TIMEOUT_EN 置位，则会触发 TOUCH_TIMEOUT_INT_RAW 超时中断，提示电路异常。

27.4.2.2 测量操作的触发源

Touch FSM 通过发送 START 信号发起一次测量操作。START 信号可由软件触发，或由触摸定时器 (touch timer) 触发。使用触摸定时器可发起周期性测量，而无需软件介入。

触摸定时器的时钟源为 LP_DYN_SLOW_CLK，时长可设置为若干个 LP_DYN_SLOW_CLK 周期。定时器到期则生成 START 信号。测量操作执行完毕后，定时器计数复位，开始重新计数，直至下次过期。

- 配置 START 信号由软件触发：
  - 置位 TOUCH_AON_TOUCH_START_FORCE，选择 START 信号由软件触发。
  - 软件置位 TOUCH_AON_TOUCH_START_EN，产生 START 信号，初始化一次测量操作。
- 配置 START 信号由触摸定时器触发：
  - 清零 TOUCH_AON_TOUCH_START_FORCE。
  - 通过 PMU_TOUCH_WAIT_CYCLES 配置定时器发送 START 信号前的等待时间（单位：LP_DYN_SLOW_CLK 周期）。
  - 通过 PMU_TOUCH_SLEEP_CYCLES 配置定时器接收到触摸传感器采样计算完成的 DONE 信号后的等待时间（单位：LP_DYN_SLOW_CLK 周期）。
  - 置位 PMU_TOUCH_SLEEP_TIMER_EN 使能定时器。
- 软件配置强制退出触摸定时器当前测量周期，可以采用两种配置方式：
  1. 采用软件产生触摸传感器的测量结束信号，进而退出触摸定时器当前测量周期。
     - 置位 TOUCH_AON_TOUCH_DONE_FORCE，选择 DONE 信号由软件触发。
     - 软件置位 TOUCH_AON_TOUCH_DONE_EN，产生 DONE 信号，结束一次测量操作。
  2. 采用软件直接强制退出触摸定时器当前测量周期。
     - 置位 PMU_TOUCH_FORCE_DONE，产生 DONE 强制退出触摸定时器当前测量周期。

说明：
触摸定时器在产生 START 信号之前和接收到 DONE 信号之后都会等待一段时间，因此连续测量两个触摸传感器之间的间隔时间为 (PMU_TOUCH_WAIT_CYCLES + PMU_TOUCH_SLEEP_CYCLES)，计时时钟为 LP_DYN_SLOW_CLK。

27.4.2.3 扫描模式

在扫描模式下，Touch FSM 对多个触摸传感器按照一定顺序进行测量。每生成一个 START 信号，就选择一个触摸传感器进行测量。因此，利用多个 START 信号即可测量多个触摸传感器。具体的扫描过程见图 27.4-3。

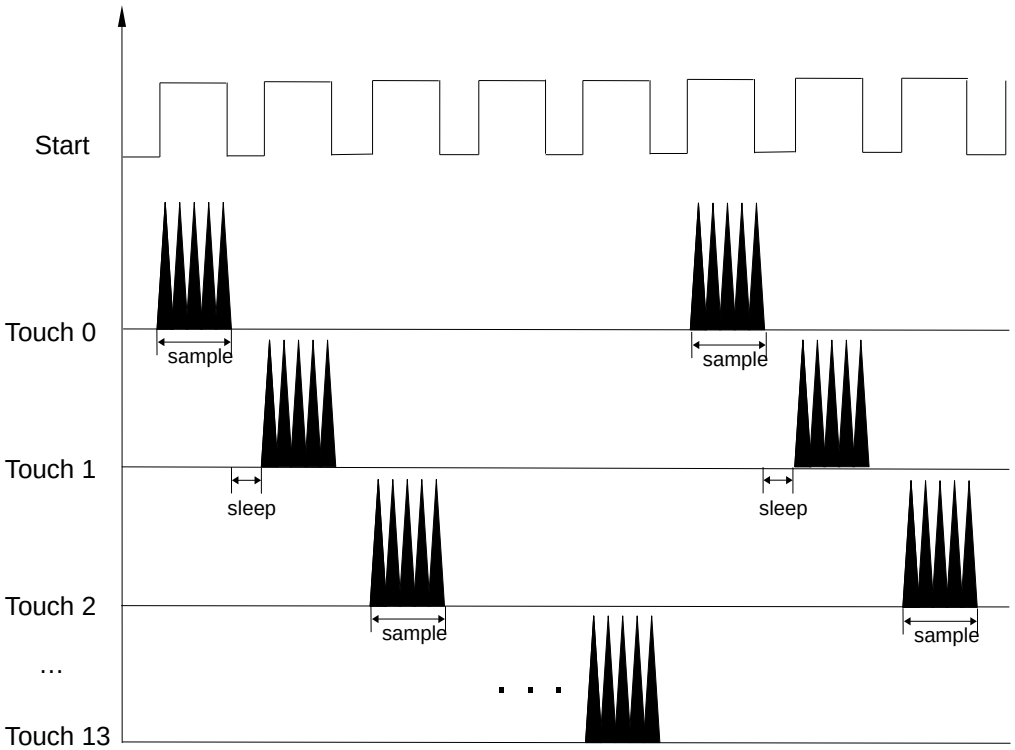


图 27.4-3. 扫描模式时序图

通过 TOUCH_AON_TOUCH_SCAN_PAD_MAP 指定已使能触摸传感器的位图，以使能扫描模式。对于每个 START 信号，Touch FSM 将从位图中依次选择一个触摸传感器进行测量。一个 START 信号仅执行一次测量操作。经过多个 START 信号，Touch FSM 按照顺序依次对使能的触摸传感器进行测量。

对不同触摸传感器进行测量时，两次测量之间的休眠时间间隔为 PMU_TOUCH_WAIT_CYCLES + PMU_TOUCH_SLEEP_CYCLES，时钟周期为 LP_DYN_SLOW_CLK。

说明：

- 触摸传感器在接收到来自触摸定时器的 START 信号后，为了保证采样信号的稳定，还会再等待一小段时间才开始测量，等待时间可通过 TOUCH_AON_TOUCH_XPD_WAIT 配置，计时时钟为 AON_FAST_CLK。
- 如果使用触摸定时器生成 START 信号，整个扫描过程无需软件介入。

27.4.2.4 跳频模式

在跳频模式下，Touch FSM 可以在不同频率模式下对触摸传感器进行测量。跳频模式最多支持三种频率模式，每种频率模式都分别对应一套参数配置。每生成一个 START 信号，即会按照频率模式依次进行测量，当所有频率模式都测量结束时，才生成 DONE 信号。跳频模式和扫描模式可以一起使用，因此利用多个 START 信号即可监测多个触摸传感器在不同频率模式下的触摸情况。具体的扫描过程见图 27.4-4。

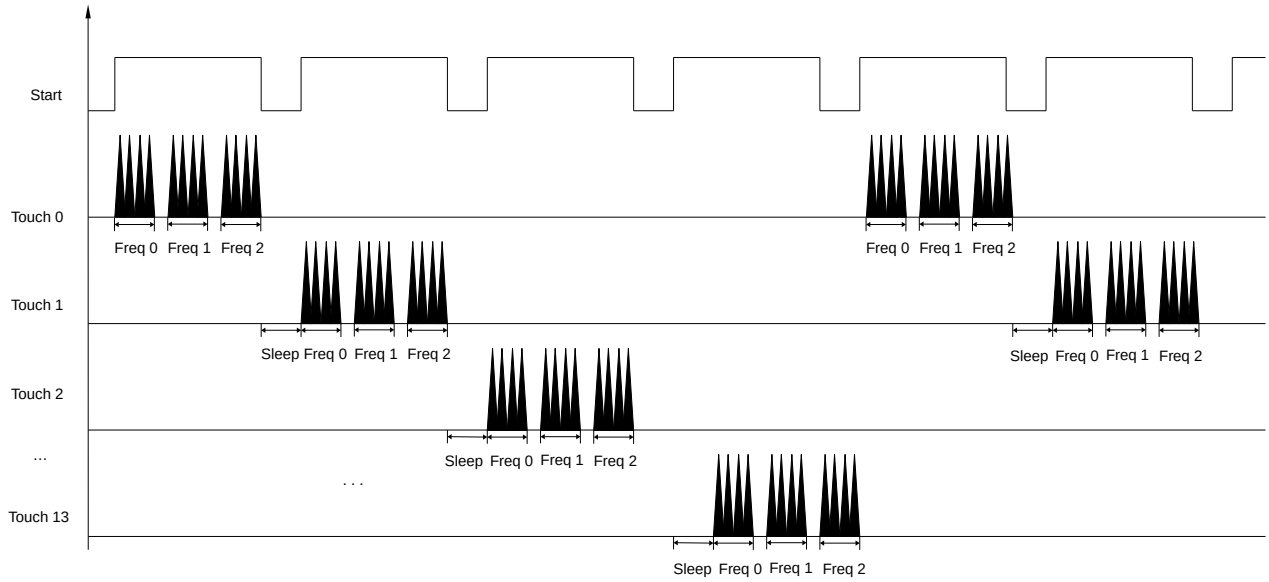


图 27.4-4. 触摸传感器跳频时序图

置位 TOUCH_AON_FREQ_SCAN_EN 可开启跳频模式，配置 TOUCH_AON_FREQ_SCAN_CNT_LIMIT 选择所支持的最大跳频数量，另外还需配置不同频率模式下调节触摸传感器采样性能的模拟参数寄存器组，具体如下：

- TOUCH_AON_TOUCH_FREQ*n*_DBIAS (*n* = 0 ~ 2)
- TOUCH_AON_TOUCH_FREQ*n*_DRV_HS (*n* = 0 ~ 2)
- TOUCH_AON_TOUCH_FREQ*n*_DRV_LS (*n* = 0 ~ 2)
- TOUCH_AON_TOUCH_FREQ*n*_DRES_LPF (*n* = 0 ~ 2)
- TOUCH_AON_TOUCH_FREQ*n*_DCAP_LPF (*n* = 0 ~ 2)

对于每个 START 信号，Touch FSM 将按照频率模式 0、1、2 的顺序依次进行测量。一个 START 信号最多触发三次测量操作。当所有频率模式都测量结束，START 信号拉低，等待下一个触摸传感器的测量。经过多个 START 信号，Touch FSM 依次对使能的触摸传感器进行测量。同一触摸传感器只有在大于等于 TOUCH_AON_FREQ_SCAN_CNT_RISE 个频点均被触发时，才能判定为触摸。

说明：

- 同一个触摸传感器在不同频率模式下测量时，两次测量之间的等待时间可通过 TOUCH_AON_TOUCH_XPD_WAIT 配置，计数时钟是 AON_FAST_CLK。
- 触摸传感器在接收到来自触摸定时器的 START 信号后，为了保证采样信号的稳定，会等待一小段时间才开始测量，等待时间可通过 TOUCH_AON_TOUCH_XPD_WAIT 配置，计数时钟为 AON_FAST_CLK。
- 如果使用触摸定时器生成 START 信号，整个扫描过程无需软件介入。

27.4.3 触摸检测

27.4.3.1 采样值

如小节 27.4.2.1 所述，用户可从 TOUCH_PAD*n*_DATA 中读取测量所得采样值。但 TOUCH_PAD*n*_DATA 中所存储的具体采样值类型由 TOUCH_AON_TOUCH_DATA_SEL 和 TOUCH_AON_TOUCH_FREQ_SEL 共同决定。支持以下三种类型的采样值：

1. **原始触摸数据** (touch_raw_data)：测量结束时，触摸计数器的计数值表示对触摸传感器进行固定次数的充放电所需的时间，单位为 LP_DYN_SLOW_CLK 周期。开启 IIR 滤波器后，该数据为滤波后的数据。
2. **平滑触摸数据** (touch_smooth_data)：一系列来自某个触摸传感器的原始触摸数据经 IIR 滤波生成的移动平均线。

   平滑触摸数据不易被噪声或采样异常值干扰，因此常用于硬件触摸检测。若开启跳频模式，每个触摸管脚在不同频率模式下分别对应一套平滑触摸数据，各自维护，互不影响。生成平滑触摸数据的 IIR 滤波设置可在 TOUCH_AON_TOUCH_SMOOTH_LVL 中配置。具体信息见表 27.4-1。

表 27.4-1. 平滑触摸数据计算

TOUCH_AON_TOUCH_SMOOTH_LVL	类型	公式
0	—	touch_raw_data
1	IIR 1/2	1/2 touch_raw_data + 1/2 touch_smooth_data
2	IIR 1/4	1/4 touch_raw_data + 3/4 touch_smooth_data
3	IIR 1/8	1/8 touch_raw_data + 7/8 touch_smooth_data

3. **基准数据** (benchmark)：同样由原始触摸数据经 IIR 滤波处理而生成，但其移动平均线窗口更广。

   基准数据常用于表示触摸传感器的稳定读数，不受触摸动作的影响。触摸检测的判断基于基准数据进行。每个触摸传感器在不同频率模式下都有一套独立的基准数据，其初始默认值为 0xFFFF。生成基准数据的 IIR 滤波可通过 TOUCH_AON_TOUCH_FILTER_MODE 配置，具体信息见表 27.4-2。

   除了通过 IIR 滤波的方式更新基准数据以外，还可以通过软件的方式直接更新。软件更新基准数据的步骤如下：

   - 通过 TOUCH_AON_TOUCH_UPDATE_BENCHMARK_FREQ_SEL 配置需要更新基准数据的目标频点。
   - 通过 TOUCH_AON_TOUCH_UPDATE_BENCHMARK_PAD_SEL 配置需要更新基准数据的目标通道。
   - 通过 TOUCH_AON_TOUCH_BENCHMARK_SW 配置基准数据值。
   - 配置 TOUCH_AON_TOUCH_UPDATE_BENCHMARK_SW，即可通过软件触发基准数据更新。

   该方式一般用于软件调试。当目标频点和通道的基准数据完成更新后，会产生中断 TOUCH_BENCHMARK_UPDATE_INT。

表 27.4-2. 基准数据算法

TOUCH_AON_TOUCH_FILTER_MODE	类型	公式
0	IIR 1/4	1/4 touch_raw_data + 3/4 benchmark
1	IIR 1/8	1/8 touch_raw_data + 7/8 benchmark
2	IIR 1/16	1/16 touch_raw_data + 15/16 benchmark
3	IIR 1/32	1/32 touch_raw_data + 31/32 benchmark

表 27.4-2. 基准数据算法

TOUCH_AON_TOUCH_FILTER_MODE	类型	公式
4	IIR 1/64	1/64 touch_raw_data + 63/64 benchmark
5	IIR 1/128	1/128 touch_raw_data + 127/128 benchmark
6	IIR 1/256	1/256 touch_raw_data + 255/256 benchmark
7	JITTER	touch_raw_data +/– TOUCH_AON_TOUCH_JITTER_STEP

27.4.3.2 硬件触摸检测

硬件触摸检测可检测到触摸动作或触摸释放动作，然后触发中断。使用硬件触摸检测，需定义如下几个关键参数：

- **触碰阈值** (finger_threshold)：用于判定触摸或触摸中断的比较值。触碰阈值由寄存器 TOUCH_AON_TOUCH_PAD*x*_TH*y* 配置，*x* 代表触摸管脚编号，取值为 0 ~ 13，分别对应 T0 ~ T13，*y* 代表跳频频率模式，取值为 0 ~ 2。
- **噪声阈值** (noise_threshold)：用于表示平滑触摸数据上下波动的最大值。噪声阈值由 TOUCH_AON_TOUCH_NOISE_THRES 配置。
- **迟滞** (hysteresis)：用于避免平滑触摸数据在触碰阈值附近来回波动导致的触摸误判。迟滞由 TOUCH_AON_TOUCH_HYSTERESIS 配置。

触碰阈值和噪声阈值均为基准数据的偏移，而不是绝对阈值，迟滞为触碰阈值的偏移。采用这种方法可以避免将原始触摸数据的渐变（缓慢变化）误测为触摸。多种环境因素，比如温度、电源、噪声，均有可能造成原始触摸数据的渐变。

触摸传感器基于上述参数进行一系列计算和比较之后，最终会将触摸管脚上检测到的触碰结果通过中断的方式输出。每个触摸传感器都对应一个控制寄存器 TOUCH_AON_TOUCH_OUTEN 来配置是否输出检测结果。

27.4.4 接近模式

物体靠近，但未接触触摸管脚时，触摸管脚电容将发生轻微变动，远小于物理触摸引起的电容变化。接近模式可用于检测电容轻微变动。

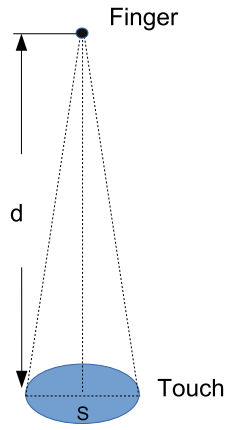


图 27.4-5. 感测面积示意图

- 检测距离 “d” 与感应面积 “S” 呈正相关关系，即在其他条件不变时，感应面积越大，可支持的检测距离越远。如图 27.4-5 所示，当前指标为最大检测距离 16 cm、感应面积 20 cm^2。

- 最多可以同时将三个触摸管脚配置为接近模式。
- 接近模式同样支持跳频检测。

在接近模式下，触摸传感器进行固定次数的采样并累计采样值。如果最终累计采样值超出设定的阈值，则认为检测到了物体靠近，并触发中断。注意，接近模式会对采样值进行累计，所以触摸传感器不会生成小节 27.4.3.1 所述的数值，即原始触摸数据、平滑触摸数据和基准数据。

接近模式操作流程如下：

- 通过配置 TOUCH_AON_TOUCH_APPROACH_PAD0、TOUCH_AON_TOUCH_APPROACH_PAD1 或 TOUCH_AON_TOUCH_APPROACH_PAD2，配置触摸传感器为接近模式。
- 通过配置 TOUCH_AON_TOUCH_APPROACH_LIMIT 设置采样次数，用于生成累计值。触摸传感器内置采样计数器，用于记录采样次数。
- 配置 TOUCH_AON_TOUCH_PAD*x*_TH*y* 设置阈值，*x* 代表触摸管脚编号，取值为 0 ~ 13，分别对应 T0 ~ T13，*y* 代表跳频频率模式，取值为 0 ~ 2。
- 采样计数器达到 TOUCH_AON_TOUCH_APPROACH_MEAS_NUM*n* 设置的采样次数：
  - 如果累计值大于设定的阈值，则触发中断。
  - 采样计数器和累计值复位归零。触摸传感器重新开始累计采样值。

27.4.5 休眠模式

为了降低功耗，触摸传感器支持休眠模式。在休眠状态下，可以指定一个触摸传感器处于活跃状态（以下简称为休眠触摸传感器），实时监测是否有触摸发生，其余 13 个触摸传感器全部掉电并处于休眠状态。休眠触摸传感器一旦检测到触摸或触摸释放，会立即产生对应中断，唤醒其余 13 个触摸传感器。休眠触摸传感器和正常模式下的触摸传感器一样，支持跳频模式和接近感应模式。休眠触摸传感器对应一套独立的阈值参数，其配置流程如下：

- 配置 TOUCH_AON_TOUCH_SLP_PAD 选择某一个触摸传感器作为休眠触摸传感器。
- 配置 TOUCH_AON_TOUCH_SLP_TH0、TOUCH_AON_TOUCH_SLP_TH1、TOUCH_AON_TOUCH_SLP_TH2，设置休眠触摸传感器在不同频率模式下的触碰阈值。
- 配置 TOUCH_AON_TOUCH_SLP_APPROACH_EN，设置休眠触摸传感器开启接近模式。
- 配置 TOUCH_AON_TOUCH_SLP_CHANNEL_CLR，清零休眠触摸传感器的基准数据、平滑触摸数据以及采样次数等中间数据。

说明：
休眠模式下，休眠触摸传感器的噪声阈值、迟滞、平滑触摸数据、基准数据与正常模式下的配置相同。

27.4.6 防潮功能

如果触摸板上有小水珠，且面积大到足以连通两个或更多触摸板，则可能导致临近触摸板出现电耦合现象。耦合现象会导致电容变化，此时触摸板可能会误测触摸动作。当小水珠跨接在触摸板和接地之间时也会引入较大的寄生电容，导致误触发。防潮设计可缓和小水珠带来的影响。

用户可配置使用防潮功能。具体步骤如下：

- 配置 TOUCH_AON_TOUCH_SHIELD_PAD_EN 开启防潮功能；

- 配置 TOUCH_AON_TOUCH_BUFSEL 选择用于防潮功能的触摸管脚。

说明：
防潮功能仅在频率低于 10 MHz 时有效。因此，当触摸传感器输出的采样频率高于 10 MHz 时，需要禁用防潮功能。

27.4.7 遇水保护功能

如果传感器触摸板阵列变潮，即大部分触摸板已浸水，全部或大部分触摸管脚因存在误测触摸动作的可能，触摸传感器将无法继续检测触摸动作。因此，ESP32-S31 触摸传感器支持遇水保护功能，该设计可检测到阵列已浸水，可关停传感器阵列。

用户可配置 TOUCH_AON_TOUCH_OUT_RING 选择用于遇水保护功能的触摸管脚。

27.5 中断

ESP32-S31 的触摸传感器可以生成 LP_TOUCH_INTR 中断信号并发送给 中断矩阵。

LP_TOUCH_INTR 中断信号由触摸传感器的内部中断源生成。表 27.5-1 列出了触摸传感器的内部中断源、中断触发条件、以及生成的中断信号。

表 27.5-1. 触摸传感器的内部中断源

内部中断源	触发条件	中断信号
TOUCH_APPROACH_LOOP_DONE_INT	接近模式累计采样结束	LP_TOUCH_INTR
TOUCH_TIMEOUT_INT	对 TOUCH_OUT 信号采样超时	
TOUCH_INACTIVE_INT	检测到触摸释放	
TOUCH_ACTIVE_INT	检测到触摸	
TOUCH_DONE_INT	对单个触摸管脚在某个频率模式下的采样比较完成	
TOUCH_SCAN_DONE_INT	对所有选中的触摸管脚在所有频率模式下都采样比较完成	
TOUCH_BENCHMARK_UPDATE_INT	软件更新基准数据完成	

说明：
中断、中断源、中断信号等术语的解释以及它们之间的联系请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 27.6 寄存器列表。

27.6 寄存器列表

27.6.1 中断与状态寄存器列表

本小节的所有地址均为相对于触摸传感器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
中断寄存器			
TOUCH_INT_RAW_REG	触摸传感器原始中断寄存器	0x0000	R/SS/WTC
TOUCH_INT_ST_REG	触摸传感器中断状态寄存器	0x0004	RO
TOUCH_INT_ENA_REG	触摸传感器中断使能寄存器	0x0008	R/W
TOUCH_INT_CLR_REG	触摸传感器中断清零寄存器	0x000C	WT
状态寄存器			
TOUCH_CHN_STATUS_REG	扫描模式状态寄存器	0x0010	RO
TOUCH_STATUS_0_REG	触摸管脚 0 状态寄存器	0x0014	RO
TOUCH_STATUS_1_REG	触摸管脚 1 状态寄存器	0x0018	RO
TOUCH_STATUS_2_REG	触摸管脚 2 状态寄存器	0x001C	RO
TOUCH_STATUS_3_REG	触摸管脚 3 状态寄存器	0x0020	RO
TOUCH_STATUS_4_REG	触摸管脚 4 状态寄存器	0x0024	RO
TOUCH_STATUS_5_REG	触摸管脚 5 状态寄存器	0x0028	RO
TOUCH_STATUS_6_REG	触摸管脚 6 状态寄存器	0x002C	RO
TOUCH_STATUS_7_REG	触摸管脚 7 状态寄存器	0x0030	RO
TOUCH_STATUS_8_REG	触摸管脚 8 状态寄存器	0x0034	RO
TOUCH_STATUS_9_REG	触摸管脚 9 状态寄存器	0x0038	RO
TOUCH_STATUS_10_REG	触摸管脚 10 状态寄存器	0x003C	RO
TOUCH_STATUS_11_REG	触摸管脚 11 状态寄存器	0x0040	RO
TOUCH_STATUS_12_REG	触摸管脚 12 状态寄存器	0x0044	RO
TOUCH_STATUS_13_REG	触摸管脚 13 状态寄存器	0x0048	RO
TOUCH_STATUS_15_REG	休眠模式的触摸管脚状态寄存器	0x0050	RO

GoBack

名称	描述	地址	访问
TOUCH_STATUS_16_REG	接近模式状态寄存器	0x0054	RO
TOUCH_STATUS_17_REG	跳频模式状态寄存器	0x0058	RO
TOUCH_CHN_TMP_STATUS_REG	触摸状态寄存器	0x005C	RO
版本寄存器			
TOUCH_DATE_REG	版本控制寄存器	0x0100	R/W

27.6.2 配置寄存器列表

本小节的所有地址均为相对于 LP 模拟外设基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

名称	描述	地址	访问
配置寄存器			
TOUCH_AON_APPROACH_WORK_MEAS_NUM_REG	接近模式下测量次数配置寄存器	0x0000	R/W
TOUCH_AON_SCAN_CTRL1_REG	扫描模式配置寄存器 1	0x0004	R/W
TOUCH_AON_SCAN_CTRL2_REG	扫描模式配置寄存器 2	0x0008	R/W
TOUCH_AON_WORK_REG	采样预处理寄存器	0x000C	varies
TOUCH_AON_WORK_MEAS_NUM_REG	不同频率模式下测量次数配置寄存器	0x0010	R/W
TOUCH_AON_FILTER1_REG	触摸检测配置寄存器 1	0x0014	R/W
TOUCH_AON_FILTER2_REG	触摸检测配置寄存器 2	0x0018	R/W
TOUCH_AON_FILTER3_REG	触摸检测配置寄存器 3	0x001C	varies
TOUCH_AON_SLP0_REG	休眠模式配置寄存器 1	0x0020	varies
TOUCH_AON_SLP1_REG	休眠模式配置寄存器 2	0x0024	R/W
TOUCH_AON_CLR_REG	状态清零寄存器	0x0028	WT
TOUCH_AON_APPROACH_REG	接近模式配置寄存器	0x002C	R/W
TOUCH_AON_FREQ0_SCAN_PARA_REG	触摸传感器实现采样频率模式 0 的模拟参数配置寄存器	0x0030	R/W
TOUCH_AON_FREQ1_SCAN_PARA_REG	触摸传感器实现采样频率模式 1 的模拟参数配置寄存器	0x0034	R/W
TOUCH_AON_FREQ2_SCAN_PARA_REG	触摸传感器实现采样频率模式 2 的模拟参数配置寄存器	0x0038	R/W

名称	描述	地址	访问
TOUCH_AON_ANA_PARA_REG	触摸传感器模拟参数配置寄存器	0x003C	R/W
TOUCH_AON_MUX0_REG	DEBUG 模式选择配置寄存器	0x0040	R/W
TOUCH_AON_MUX1_REG	DEBUG 模式功能配置寄存器	0x0044	R/W
TOUCH_AON_PAD0_TH0_REG	触摸管脚 0 在采样频率模式 0 下的触碰阈值配置寄存器	0x0048	R/W
TOUCH_AON_PAD0_TH1_REG	触摸管脚 0 在采样频率模式 1 下的触碰阈值配置寄存器	0x004C	R/W
TOUCH_AON_PAD0_TH2_REG	触摸管脚 0 在采样频率模式 2 下的触碰阈值配置寄存器	0x0050	R/W
TOUCH_AON_PAD1_TH0_REG	触摸管脚 1 在采样频率模式 0 下的触碰阈值配置寄存器	0x0054	R/W
TOUCH_AON_PAD1_TH1_REG	触摸管脚 1 在采样频率模式 1 下的触碰阈值配置寄存器	0x0058	R/W
TOUCH_AON_PAD1_TH2_REG	触摸管脚 1 在采样频率模式 2 下的触碰阈值配置寄存器	0x005C	R/W
TOUCH_AON_PAD2_TH0_REG	触摸管脚 2 在采样频率模式 0 下的触碰阈值配置寄存器	0x0060	R/W
TOUCH_AON_PAD2_TH1_REG	触摸管脚 2 在采样频率模式 1 下的触碰阈值配置寄存器	0x0064	R/W
TOUCH_AON_PAD2_TH2_REG	触摸管脚 2 在采样频率模式 2 下的触碰阈值配置寄存器	0x0068	R/W
TOUCH_AON_PAD3_TH0_REG	触摸管脚 3 在采样频率模式 0 下的触碰阈值配置寄存器	0x006C	R/W
TOUCH_AON_PAD3_TH1_REG	触摸管脚 3 在采样频率模式 1 下的触碰阈值配置寄存器	0x0070	R/W
TOUCH_AON_PAD3_TH2_REG	触摸管脚 3 在采样频率模式 2 下的触碰阈值配置寄存器	0x0074	R/W
TOUCH_AON_PAD4_TH0_REG	触摸管脚 4 在采样频率模式 0 下的触碰阈值配置寄存器	0x0078	R/W
TOUCH_AON_PAD4_TH1_REG	触摸管脚 4 在采样频率模式 1 下的触碰阈值配置寄存器	0x007C	R/W
TOUCH_AON_PAD4_TH2_REG	触摸管脚 4 在采样频率模式 2 下的触碰阈值配置寄存器	0x0080	R/W
TOUCH_AON_PAD5_TH0_REG	触摸管脚 5 在采样频率模式 0 下的触碰阈值配置寄存器	0x0084	R/W
TOUCH_AON_PAD5_TH1_REG	触摸管脚 5 在采样频率模式 1 下的触碰阈值配置寄存器	0x0088	R/W
TOUCH_AON_PAD5_TH2_REG	触摸管脚 5 在采样频率模式 2 下的触碰阈值配置寄存器	0x008C	R/W
TOUCH_AON_PAD6_TH0_REG	触摸管脚 6 在采样频率模式 0 下的触碰阈值配置寄存器	0x0090	R/W
TOUCH_AON_PAD6_TH1_REG	触摸管脚 6 在采样频率模式 1 下的触碰阈值配置寄存器	0x0094	R/W
TOUCH_AON_PAD6_TH2_REG	触摸管脚 6 在采样频率模式 2 下的触碰阈值配置寄存器	0x0098	R/W
TOUCH_AON_PAD7_TH0_REG	触摸管脚 7 在采样频率模式 0 下的触碰阈值配置寄存器	0x009C	R/W
TOUCH_AON_PAD7_TH1_REG	触摸管脚 7 在采样频率模式 1 下的触碰阈值配置寄存器	0x00A0	R/W
TOUCH_AON_PAD7_TH2_REG	触摸管脚 7 在采样频率模式 2 下的触碰阈值配置寄存器	0x00A4	R/W
TOUCH_AON_PAD8_TH0_REG	触摸管脚 8 在采样频率模式 0 下的触碰阈值配置寄存器	0x00A8	R/W
TOUCH_AON_PAD8_TH1_REG	触摸管脚 8 在采样频率模式 1 下的触碰阈值配置寄存器	0x00AC	R/W

GoBack

GoBack

名称	描述	地址	访问
TOUCH_AON_PAD8_TH2_REG	触摸管脚 8 在采样频率模式 2 下的触碰阈值配置寄存器	0x00B0	R/W
TOUCH_AON_PAD9_TH0_REG	触摸管脚 9 在采样频率模式 0 下的触碰阈值配置寄存器	0x00B4	R/W
TOUCH_AON_PAD9_TH1_REG	触摸管脚 9 在采样频率模式 1 下的触碰阈值配置寄存器	0x00B8	R/W
TOUCH_AON_PAD9_TH2_REG	触摸管脚 9 在采样频率模式 2 下的触碰阈值配置寄存器	0x00BC	R/W
TOUCH_AON_PAD10_TH0_REG	触摸管脚 10 在采样频率模式 0 下的触碰阈值配置寄存器	0x00C0	R/W
TOUCH_AON_PAD10_TH1_REG	触摸管脚 10 在采样频率模式 1 下的触碰阈值配置寄存器	0x00C4	R/W
TOUCH_AON_PAD10_TH2_REG	触摸管脚 10 在采样频率模式 2 下的触碰阈值配置寄存器	0x00C8	R/W
TOUCH_AON_PAD11_TH0_REG	触摸管脚 11 在采样频率模式 0 下的触碰阈值配置寄存器	0x00CC	R/W
TOUCH_AON_PAD11_TH1_REG	触摸管脚 11 在采样频率模式 1 下的触碰阈值配置寄存器	0x00D0	R/W
TOUCH_AON_PAD11_TH2_REG	触摸管脚 11 在采样频率模式 2 下的触碰阈值配置寄存器	0x00D4	R/W
TOUCH_AON_PAD12_TH0_REG	触摸管脚 12 在采样频率模式 0 下的触碰阈值配置寄存器	0x00D8	R/W
TOUCH_AON_PAD12_TH1_REG	触摸管脚 12 在采样频率模式 1 下的触碰阈值配置寄存器	0x00DC	R/W
TOUCH_AON_PAD12_TH2_REG	触摸管脚 12 在采样频率模式 2 下的触碰阈值配置寄存器	0x00E0	R/W
TOUCH_AON_PAD13_TH0_REG	触摸管脚 13 在采样频率模式 0 下的触碰阈值配置寄存器	0x00E4	R/W
TOUCH_AON_PAD13_TH1_REG	触摸管脚 13 在采样频率模式 1 下的触碰阈值配置寄存器	0x00E8	R/W
TOUCH_AON_PAD13_TH2_REG	触摸管脚 13 在采样频率模式 2 下的触碰阈值配置寄存器	0x00EC	R/W
TOUCH_AON_DATE_REG	版本控制寄存器	0x00FC	R/W

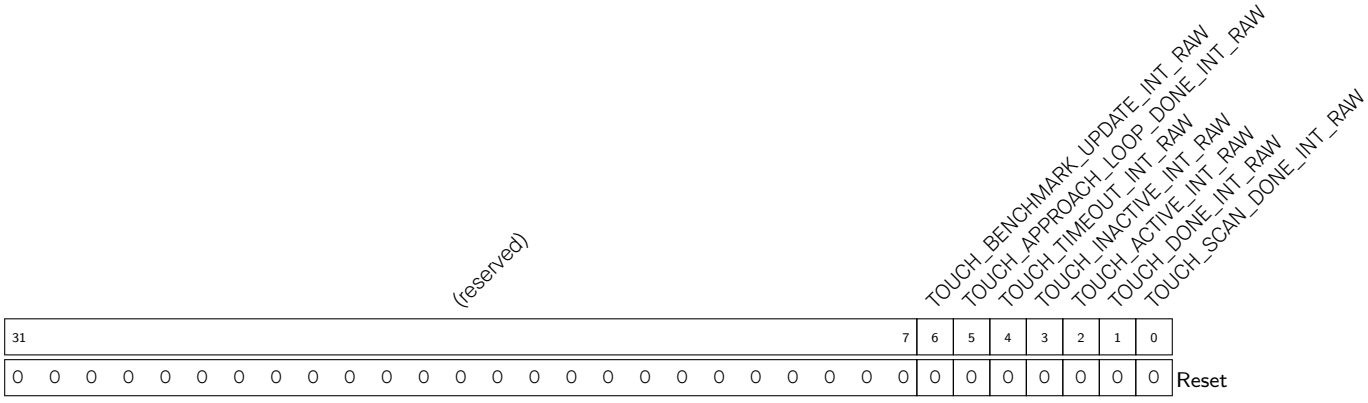
27.7 寄存器

27.7.1 中断与状态寄存器

本小节的所有地址均为相对于触摸传感器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 27.1. TOUCH_INT_RAW_REG (0x0000)



TOUCH_SCAN_DONE_INT_RAW TOUCH_SCAN_DONE_INT 中断的原始中断状态。(R/SS/WTC)

TOUCH_DONE_INT_RAW TOUCH_DONE_INT 中断的原始中断状态。(R/SS/WTC)

TOUCH_ACTIVE_INT_RAW TOUCH_ACTIVE_INT 中断的原始中断状态。(R/SS/WTC)

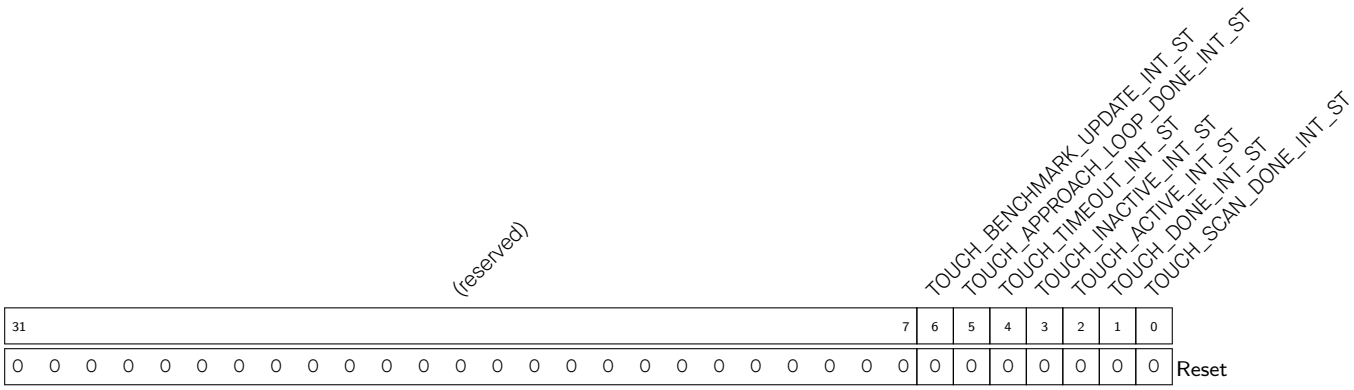
TOUCH_INACTIVE_INT_RAW TOUCH_INACTIVE_INT 中断的原始中断状态。(R/SS/WTC)

TOUCH_TIMEOUT_INT_RAW TOUCH_TIMEOUT_INT 中断的原始中断状态。(R/SS/WTC)

TOUCH_APPROACH_LOOP_DONE_INT_RAW TOUCH_APPROACH_LOOP_DONE_INT 中断的原始中断状态。(R/SS/WTC)

TOUCH_BENCHMARK_UPDATE_INT_RAW TOUCH_BENCHMARK_UPDATE_INT 中断的原始中断状态。(R/SS/WTC)

Register 27.2. TOUCH_INT_ST_REG (0x0004)



TOUCH_SCAN_DONE_INT_ST TOUCH_SCAN_DONE_INT 中断的屏蔽中断状态。(RO)

TOUCH_DONE_INT_ST TOUCH_DONE_INT 中断的屏蔽中断状态。(RO)

TOUCH_ACTIVE_INT_ST TOUCH_ACTIVE_INT 中断的屏蔽中断状态。(RO)

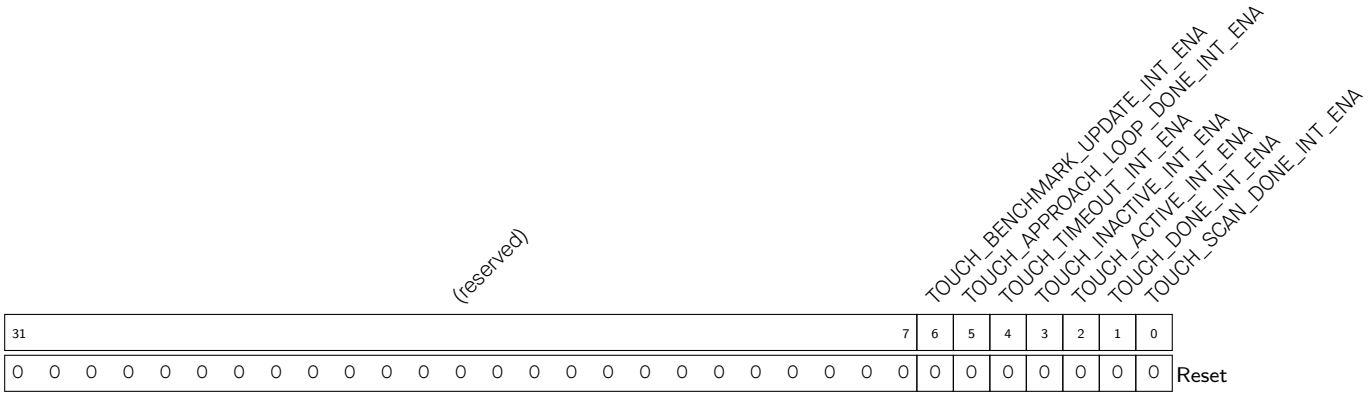
TOUCH_INACTIVE_INT_ST TOUCH_INACTIVE_INT 中断的屏蔽中断状态。(RO)

TOUCH_TIMEOUT_INT_ST TOUCH_TIMEOUT_INT 中断的屏蔽中断状态。(RO)

TOUCH_APPROACH_LOOP_DONE_INT_ST TOUCH_APPROACH_LOOP_DONE_INT 中断的屏蔽中断状态。(RO)

TOUCH_BENCHMARK_UPDATE_INT_ST TOUCH_BENCHMARK_UPDATE_INT 中断的屏蔽中断状态。(RO)

Register 27.3. TOUCH_INT_ENA_REG (0x0008)



TOUCH_SCAN_DONE_INT_ENA 写 1 使能 TOUCH_SCAN_DONE_INT 中断。(R/W)

TOUCH_DONE_INT_ENA 写 1 使能 TOUCH_DONE_INT 中断。(R/W)

TOUCH_ACTIVE_INT_ENA 写 1 使能 TOUCH_ACTIVE_INT 中断。(R/W)

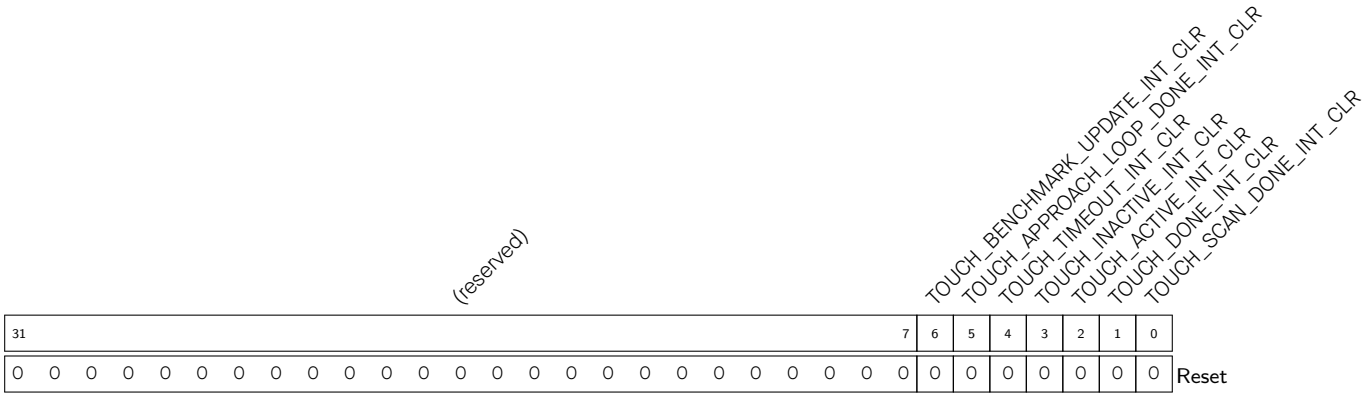
TOUCH_INACTIVE_INT_ENA 写 1 使能 TOUCH_INACTIVE_INT 中断。(R/W)

TOUCH_TIMEOUT_INT_ENA 写 1 使能 TOUCH_TIMEOUT_INT 中断。(R/W)

TOUCH_APPROACH_LOOP_DONE_INT_ENA 写 1 使能 TOUCH_APPROACH_LOOP_DONE_INT 中断。(R/W)

TOUCH_BENCHMARK_UPDATE_INT_ENA 写 1 使能 TOUCH_BENCHMARK_UPDATE_INT 中断。(R/W)

Register 27.4. TOUCH_INT_CLR_REG (0x000C)



TOUCH_SCAN_DONE_INT_CLR 写 1 清除 TOUCH_SCAN_DONE_INT 中断。(WT)

TOUCH_DONE_INT_CLR 写 1 清除 TOUCH_DONE_INT 中断。(WT)

TOUCH_ACTIVE_INT_CLR 写 1 清除 TOUCH_ACTIVE_INT 中断。(WT)

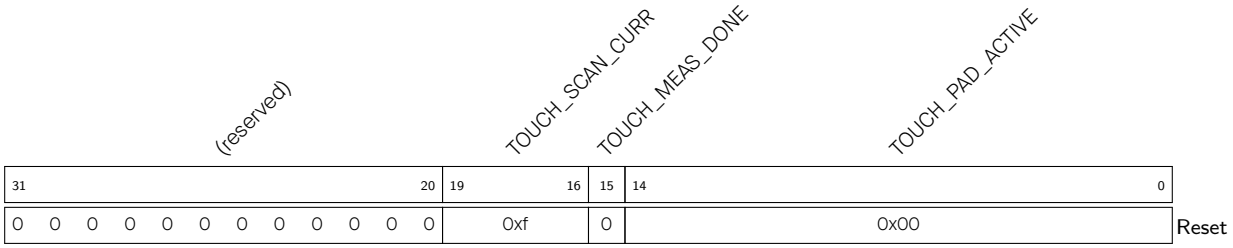
TOUCH_INACTIVE_INT_CLR 写 1 清除 TOUCH_INACTIVE_INT 中断。(WT)

TOUCH_TIMEOUT_INT_CLR 写 1 清除 TOUCH_TIMEOUT_INT 中断。(WT)

TOUCH_APPROACH_LOOP_DONE_INT_CLR 写 1 清除 TOUCH_APPROACH_LOOP_DONE_INT 中断。(WT)

TOUCH_BENCHMARK_UPDATE_INT_CLR 写 1 清除 TOUCH_BENCHMARK_UPDATE_INT 中断。(WT)

Register 27.5. TOUCH_CHN_STATUS_REG (0x0010)



TOUCH_PAD_ACTIVE 表示触摸管脚是否检测到触摸状态，每一位均代表一个触摸管脚的触摸状态，位 0 ~ 13 分别对应触摸管脚 0 ~ 13，其他位无效。

0：未检测到触摸状态

1：检测到触摸状态

(RO)

TOUCH_MEAS_DONE 表示触摸管脚是否在所有频率模式下都测量完成。

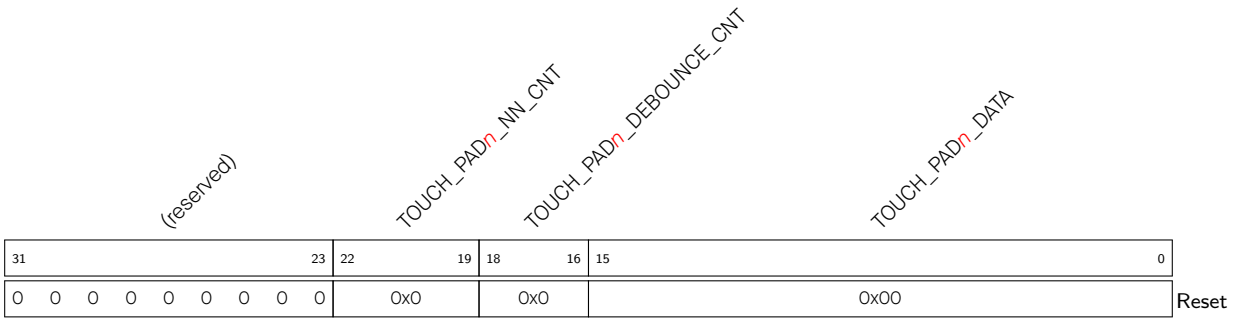
0：未完成

1：完成

(RO)

TOUCH_SCAN_CURR 表示当前被扫描的触摸管脚的序号。(RO)

Register 27.6. TOUCH_STATUS_*n*_REG (*n*: 0-13) (0x0014+0x4**n*)

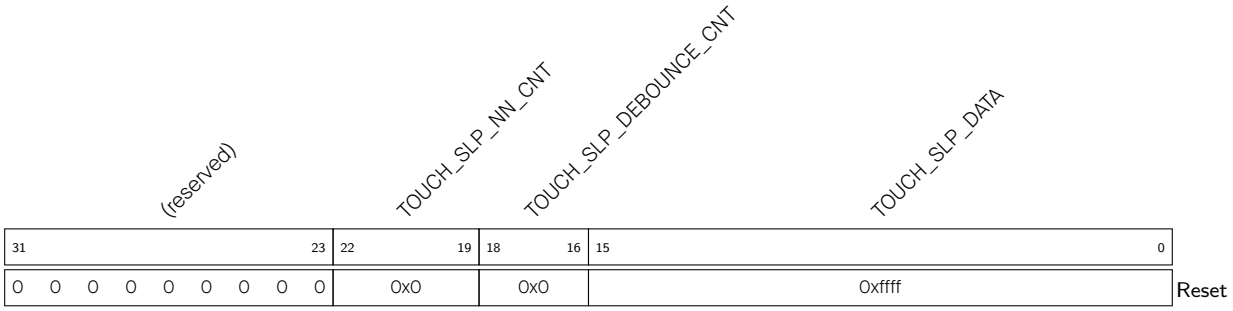


TOUCH_PAD*n*_DATA 表示触摸管脚 *n* 采样的原始数据或平滑触摸数据或基准数据。(RO)

TOUCH_PAD*n*_DEBOUNCE_CNT 表示触摸管脚 *n* 连续检测到触摸或触摸释放的累计次数。(RO)

TOUCH_PAD*n*_NN_CNT 表示触摸管脚 *n* 连续检测到平滑触摸数据小于参考阈值的累计次数。(RO)

Register 27.7. TOUCH_STATUS_15_REG (0x0050)

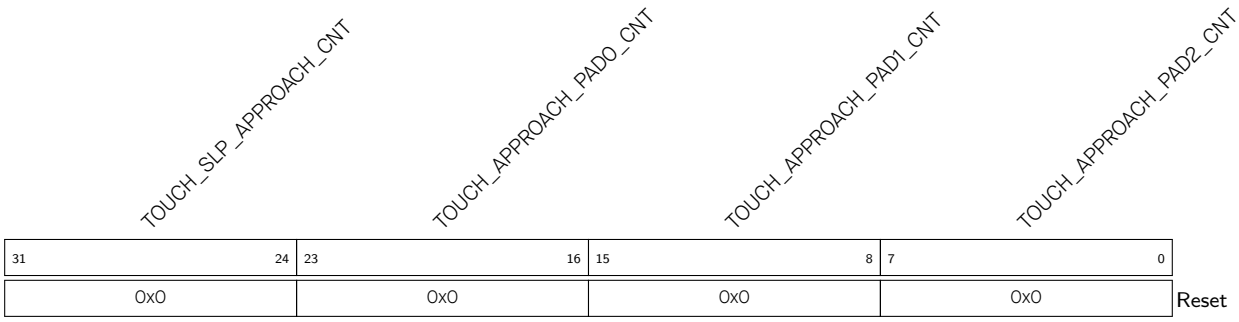


TOUCH_SLP_DATA 表示休眠模式下触摸管脚采样的平滑触摸数据或基准数据。(RO)

TOUCH_SLP_DEBOUNCE_CNT 表示休眠模式下触摸管脚连续检测到触摸或触摸释放的累计次数。(RO)

TOUCH_SLP_NN_CNT 表示休眠模式下触摸管脚连续检测到平滑触摸数据小于参考阈值的累计次数。(RO)

Register 27.8. TOUCH_STATUS_16_REG (0x0054)



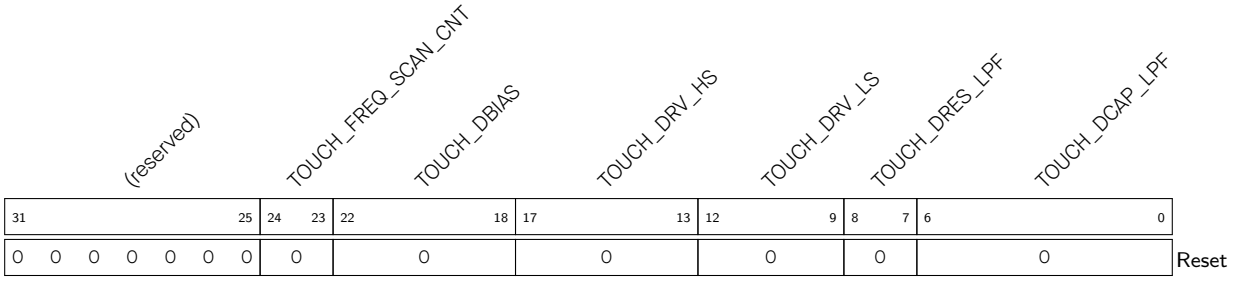
TOUCH_APPROACH_PAD2_CNT 表示接近模式下触摸管脚 2 采样的累计次数。(RO)

TOUCH_APPROACH_PAD1_CNT 表示接近模式下触摸管脚 1 采样的累计次数。(RO)

TOUCH_APPROACH_PAD0_CNT 表示接近模式下触摸管脚 0 采样的累计次数。(RO)

TOUCH_SLP_APPROACH_CNT 表示接近模式下休眠管脚采样的累计次数。(RO)

Register 27.9. TOUCH_STATUS_17_REG (0x0058)



TOUCH_DCAP_LPF 表示当前触摸管脚的 DCAP_LPF 值。(RO)

TOUCH_DRES_LPF 表示当前触摸管脚的 DRES_LPF 值。(RO)

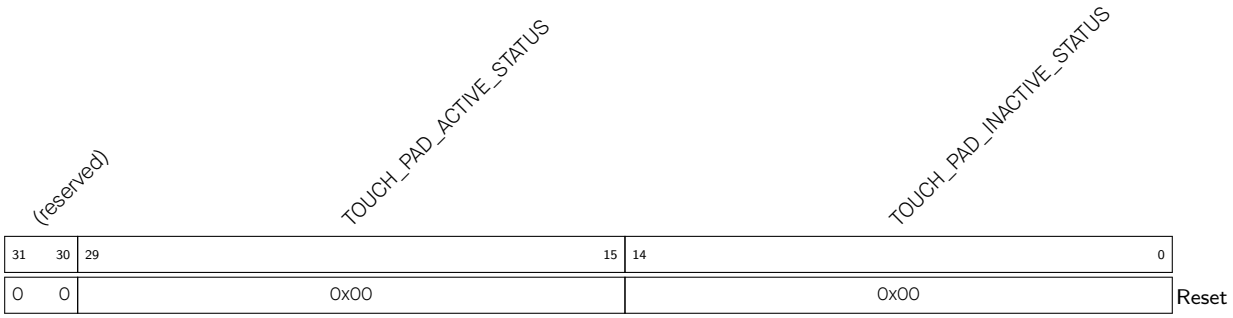
TOUCH_DRV_LS 表示当前触摸管脚的 DRV_LS 值。(RO)

TOUCH_DRV_HS 表示当前触摸管脚的 DRV_HS 值。(RO)

TOUCH_DBIAS 表示当前触摸管脚的 DBIAS 值。(RO)

TOUCH_FREQ_SCAN_CNT 表示当前触摸管脚的 SCAN_CNT 值。(RO)

Register 27.10. TOUCH_CHN_TMP_STATUS_REG (0x005C)



TOUCH_PAD_INACTIVE_STATUS 表示触摸管脚是否检测到触摸板从触摸状态到触摸释放状态的过程，位 0 ~ 13 分别对应触摸管脚 0 ~ 13，其他位无效。

0：未检测到触摸释放

1：检测到触摸释放状态

(RO)

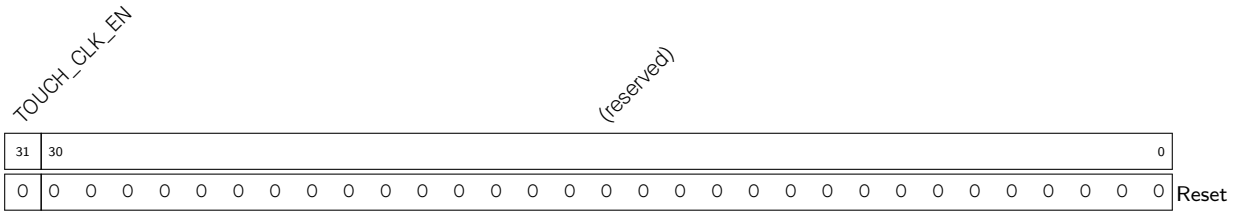
TOUCH_PAD_ACTIVE_STATUS 表示触摸管脚是否检测到被触摸的过程，位 0 ~ 13 分别对应触摸管脚 0 ~ 13，其他位无效。

0：未检测到被触摸

1：检测到触摸管脚被触摸

(RO)

Register 27.11. TOUCH_DATE_REG (0x0100)



TOUCH_CLK_EN 配置是否使能寄存器读写时钟。

0：不使能

1：使能

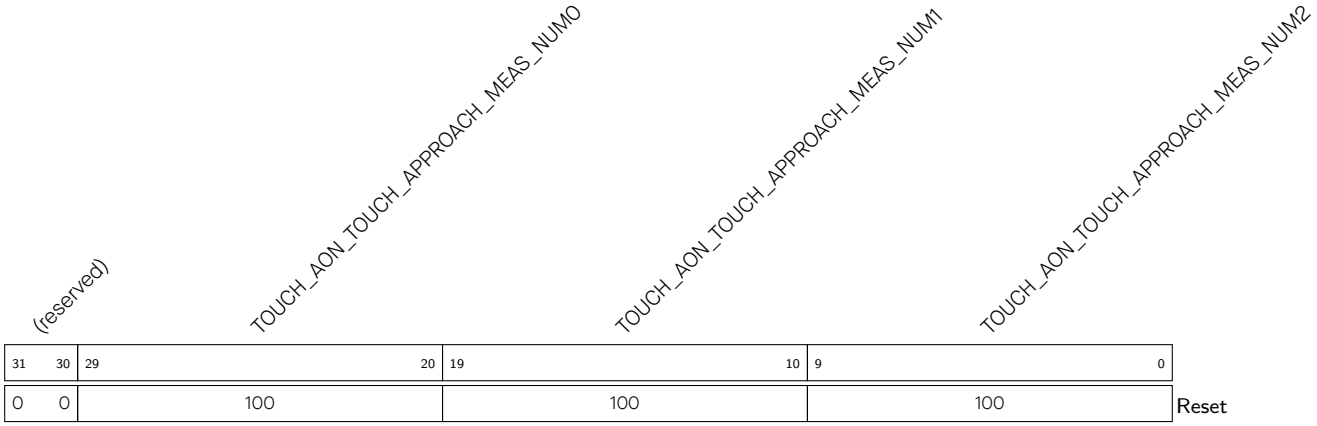
(R/W)

27.7.2 配置寄存器

本小节的所有地址均为相对于 LP 模拟外设基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 27.12. TOUCH_AON_APPROACH_WORK_MEAS_NUM_REG (0x0000)

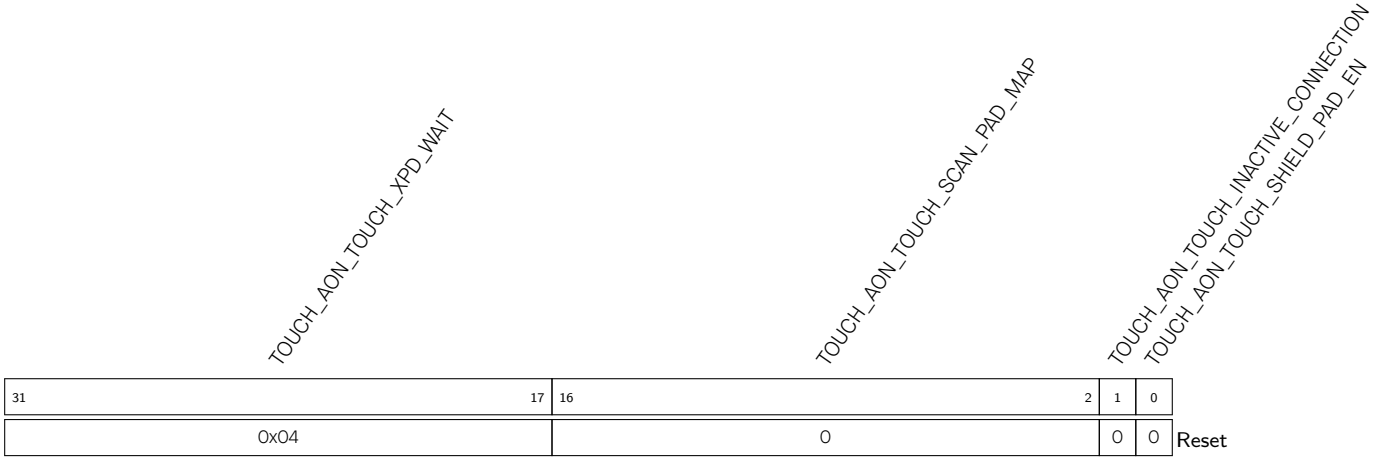


TOUCH_AON_TOUCH_APPROACH_MEAS_NUM2 配置接近模式在采样频率模式为 2 时的测量次数。(R/W)

TOUCH_AON_TOUCH_APPROACH_MEAS_NUM1 配置接近模式在采样频率模式为 1 时的测量次数。(R/W)

TOUCH_AON_TOUCH_APPROACH_MEAS_NUM0 配置接近模式在采样频率模式为 0 时的测量次数。(R/W)

Register 27.13. TOUCH_AON_SCAN_CTRL1_REG (0x0004)



TOUCH_AON_TOUCH_SHIELD_PAD_EN 配置是否开启防潮功能。

0：不开启

1：开启

(R/W)

TOUCH_AON_TOUCH_INACTIVE_CONNECTION 配置是否上电在扫描模式下参与测量的触摸管脚。

0：不上电

1：上电

(R/W)

TOUCH_AON_TOUCH_SCAN_PAD_MAP 配置扫描模式下参与测量的触摸管脚，位 0 ~ 13 分别对应触摸管脚 0 ~ 13，其他位无效。

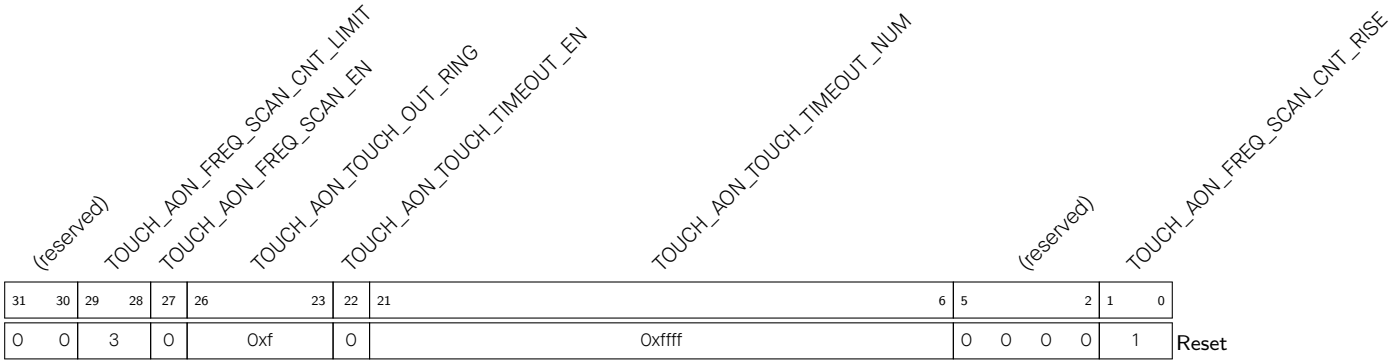
0：无效

1：使能该管脚

(R/W)

TOUCH_AON_TOUCH_XPD_WAIT 配置启动测量后的上电等待时间。(R/W)

Register 27.14. TOUCH_AON_SCAN_CTRL2_REG (0x0008)



TOUCH_AON_FREQ_SCAN_CNT_RISE 配置在跳频模式下用于触摸检测的触发频率点数量。(R/W)

TOUCH_AON_TOUCH_TIMEOUT_NUM 配置测量超时次数。(R/W)

TOUCH_AON_TOUCH_TIMEOUT_EN 配置是否使能测量超时。

0：不使能

1：使能

(R/W)

TOUCH_AON_TOUCH_OUT_RING 配置选择用于遇水保护功能的触摸管脚。可以配置为 0 ~ 13，其他值无效。(R/W)

TOUCH_AON_FREQ_SCAN_EN 配置是否使能跳频模式。

0：不使能

1：使能

(R/W)

TOUCH_AON_FREQ_SCAN_CNT_LIMIT 配置跳频次数。

0：1

1：2

2：3

3：无效

(R/W)

Register 27.15. TOUCH_AON_WORK_REG (0x000C)



TOUCH_AON_DIV_NUM2 配置采样频率模式为 2 时是否分频及对应的 TOUCH_OUT 信号分频系数。

bit[0]：配置是否分频。

0：不分频

1：分频

bit[1-2]：配置 TOUCH_OUT 信号分频系数。

0：0

1：2

2：4

3：6

(R/W)

TOUCH_AON_DIV_NUM1 配置采样频率模式为 1 时是否分频及对应的 TOUCH_OUT 信号分频系数。具体配置参考 TOUCH_AON_DIV_NUM2。(R/W)

TOUCH_AON_DIV_NUM0 配置采样频率模式为 0 时是否分频及对应的 TOUCH_OUT 信号分频系数。具体配置参考 TOUCH_AON_DIV_NUM2。(R/W)

TOUCH_AON_TOUCH_OUT_SEL 配置 TOUCH_OUT 信号作为时钟或数据使用。

0：TOUCH_OUT 信号作为数据使用

1：TOUCH_OUT 信号作为时钟使用

(R/W)

TOUCH_AON_TOUCH_OUT_RESET 写 1 复位分频后的 TOUCH_OUT 信号。(WT)

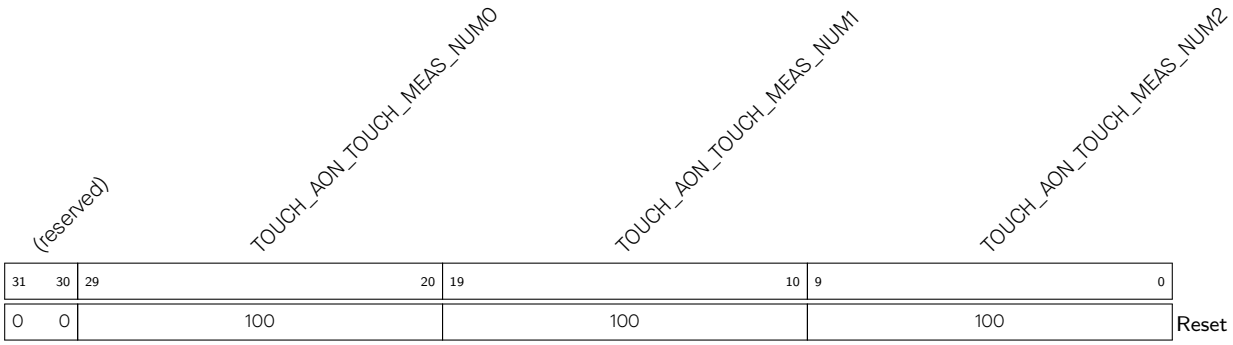
TOUCH_AON_TOUCH_OUT_GATE 配置是否开启 TOUCH_OUT 信号分频后的门控。

0：不开启

1：开启

(R/W)

Register 27.16. TOUCH_AON_WORK_MEAS_NUM_REG (0x0010)

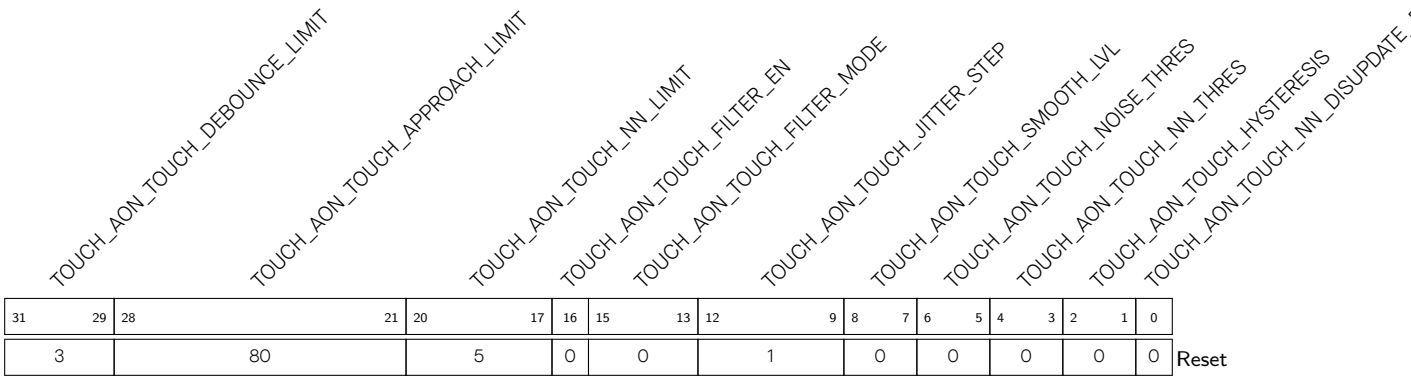


TOUCH_AON_TOUCH_MEAS_NUM2 配置正常模式下采样频率模式为 2 时的测量次数。(R/W)

TOUCH_AON_TOUCH_MEAS_NUM1 配置正常模式下采样频率模式为 1 时的测量次数。(R/W)

TOUCH_AON_TOUCH_MEAS_NUM0 配置正常模式下采样频率模式为 0 时的测量次数。(R/W)

Register 27.17. TOUCH_AON_FILTER1_REG (0x0014)



TOUCH_AON_TOUCH_NN_DISUPDATE_BENCHMARK_EN 配置是否关闭基准数据修正功能。

0：不关闭

1：关闭

(R/W)

TOUCH_AON_TOUCH_HYSTERESIS 配置采样迟滞。(R/W)

TOUCH_AON_TOUCH_NN_THRES 配置反相噪声阈值。(R/W)

TOUCH_AON_TOUCH_NOISE_THRES 配置噪声阈值。(R/W)

TOUCH_AON_TOUCH_SMOOTH_LVL 配置平滑触摸数据的 IIR 滤波模式。(R/W)

TOUCH_AON_TOUCH_JITTER_STEP 配置基准数据滤波模式下的抖动参数。(R/W)

TOUCH_AON_TOUCH_FILTER_MODE 配置基准数据滤波模式。(R/W)

TOUCH_AON_TOUCH_FILTER_EN 配置是否使能基准数据滤波模式。

0：不使能

1：使能

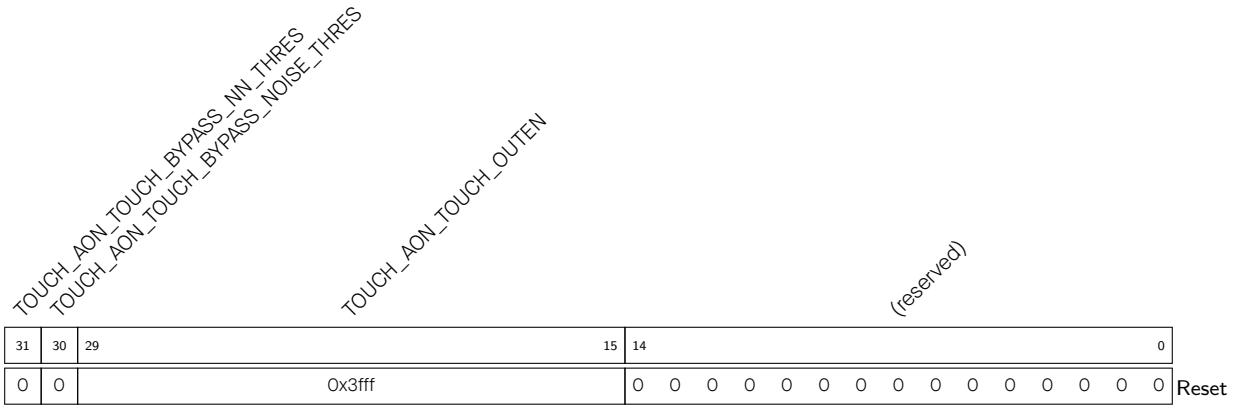
(R/W)

TOUCH_AON_TOUCH_NN_LIMIT 配置超过反相噪声阈值的连续采样次数。(R/W)

TOUCH_AON_TOUCH_APPROACH_LIMIT 配置接近模式采样次数。(R/W)

TOUCH_AON_TOUCH_DEBOUNCE_LIMIT 配置触摸状态改变连续采样次数。(R/W)

Register 27.18. TOUCH_AON_FILTER2_REG (0x0018)



TOUCH_AON_TOUCH_OUTEN 配置是否使能输出 14 个触摸管脚检测结果，位 0 ~ 13 分别对应触摸管脚 0 ~ 13，其他位无效。

0：不使能

1：使能

(R/W)

TOUCH_AON_TOUCH_BYPASS_NOISE_THRES 配置是否使能平滑数据 bypass 模式。

0：不使能

1：使能

(R/W)

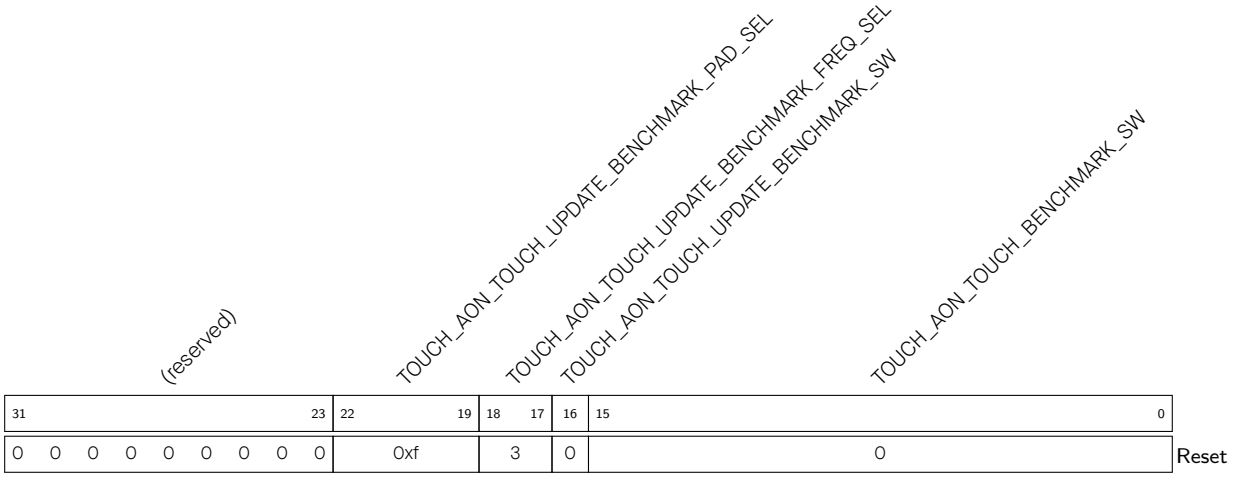
TOUCH_AON_TOUCH_BYPASS_NN_THRES 配置是否使能反相噪声的 bypass 模式。

0：不使能

1：使能

(R/W)

Register 27.19. TOUCH_AON_FILTER3_REG (0x001C)



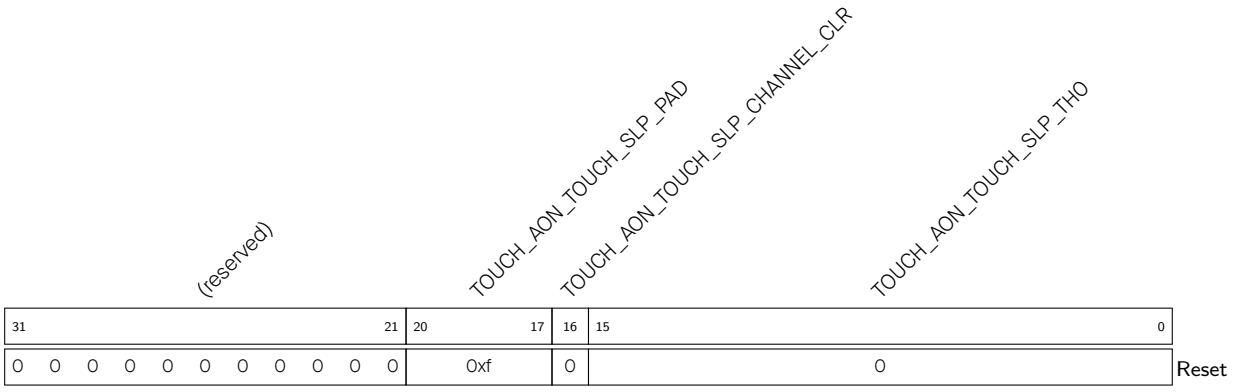
TOUCH_AON_TOUCH_BENCHMARK_SW 配置软件写入基准数据。(R/W)

TOUCH_AON_TOUCH_UPDATE_BENCHMARK_SW 写 1 使能软件写入基准数据。(WT)

TOUCH_AON_TOUCH_UPDATE_BENCHMARK_FREQ_SEL 配置软件用于更新基准数据的频率点。(R/W)

TOUCH_AON_TOUCH_UPDATE_BENCHMARK_PAD_SEL 配置软件用于更新基准数据的通道。(R/W)

Register 27.20. TOUCH_AON_SLP0_REG (0x0020)

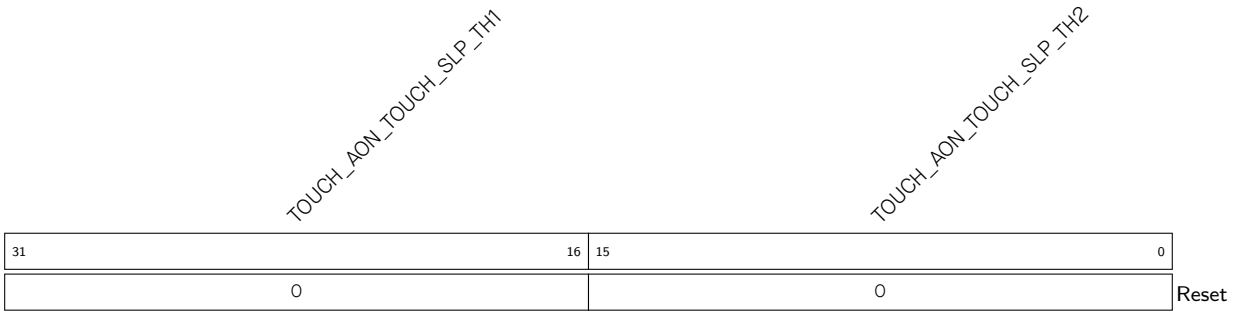


TOUCH_AON_TOUCH_SLP_TH0 配置休眠模式下在采样频率模式为 0 时的触碰阈值。(R/W)

TOUCH_AON_TOUCH_SLP_CHANNEL_CLR 写 1 清理休眠模式下基准数据。(WT)

TOUCH_AON_TOUCH_SLP_PAD 配置选择进入休眠模式的触摸传感器编号，可配置为 0 ~ 13，分别对应触摸管脚 0 ~ 13，其他值无效。(R/W)

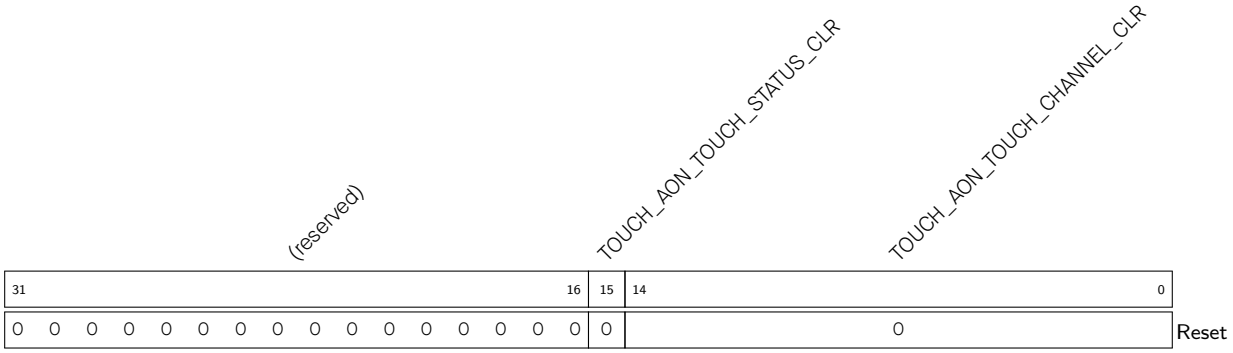
Register 27.21. TOUCH_AON_SLP1_REG (0x0024)



TOUCH_AON_TOUCH_SLP_TH2 配置休眠模式下在采样频率模式为 2 时的触碰阈值。(R/W)

TOUCH_AON_TOUCH_SLP_TH1 配置休眠模式下在采样频率模式为 1 时的触碰阈值。(R/W)

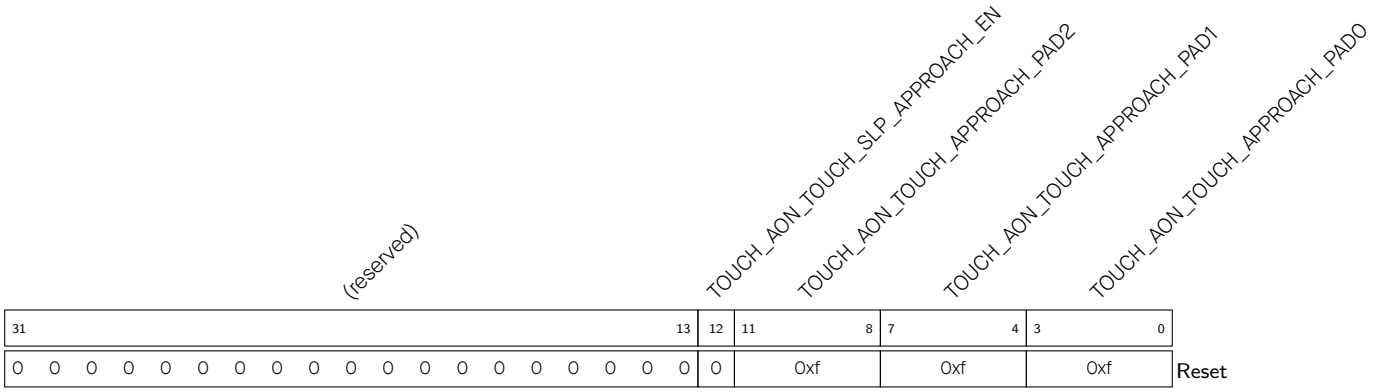
Register 27.22. TOUCH_AON_CLR_REG (0x0028)



TOUCH_AON_TOUCH_CHANNEL_CLR 写 1 独立清零 14 个触摸传感器的触摸状态，位 0 ~ 13 分别对应触摸管脚 0 ~ 13，其他位无效。(WT)

TOUCH_AON_TOUCH_STATUS_CLR 写 1 清零触摸状态。(WT)

Register 27.23. TOUCH_AON_APPROACH_REG (0x002C)



TOUCH_AON_TOUCH_APPROACH_PAD0 配置接近模式使用的第一个传感器触摸管脚，可以配置为 0 ~ 13，其他值无效。(R/W)

TOUCH_AON_TOUCH_APPROACH_PAD1 配置接近模式使用的第二个传感器触摸管脚，可以配置为 0 ~ 13，其他值无效。(R/W)

TOUCH_AON_TOUCH_APPROACH_PAD2 配置接近模式使用的第三个传感器触摸管脚，可以配置为 0 ~ 13，其他值无效。(R/W)

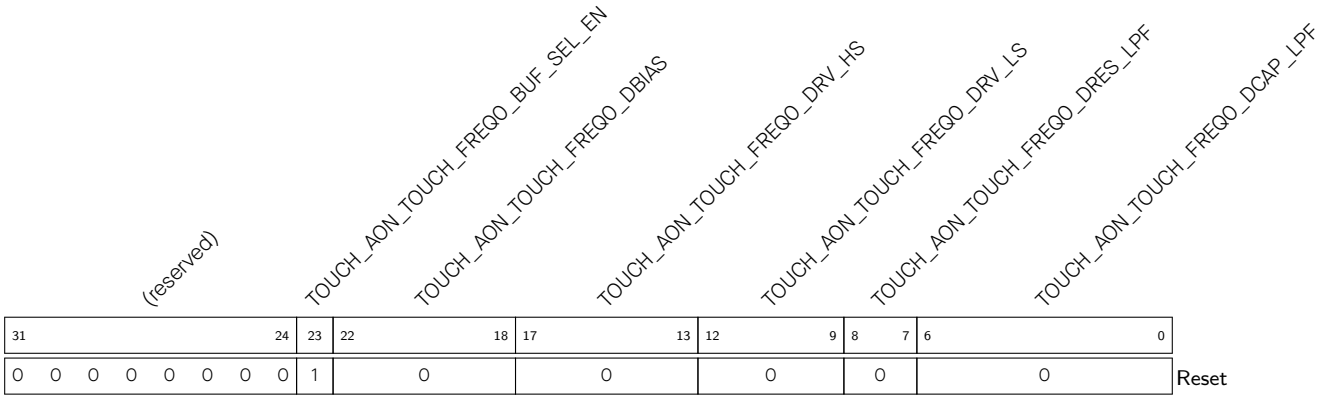
TOUCH_AON_TOUCH_SLP_APPROACH_EN 配置是否使能接近模式。

0：不使能

1：使能

(R/W)

Register 27.24. TOUCH_AON_FREQ0_SCAN_PARA_REG (0x0030)



TOUCH_AON_TOUCH_FREQ0_DCAP_LPF 配置采样频率模式为 0 时触摸传感器低通滤波器参数，调节范围为 0 ~ 2.54 pF，调节精度为 20 fF。(R/W)

TOUCH_AON_TOUCH_FREQ0_DRES_LPF 配置采样频率模式为 0 时触摸传感器低通滤波器参数。

0: 0 kΩ

1: 3 kΩ

2: 4.5 kΩ

3: 7.5 kΩ

(R/W)

TOUCH_AON_TOUCH_FREQ0_DRV_LS 配置采样频率模式为 0 时触摸传感器的 DRV_LS 参数。(R/W)

TOUCH_AON_TOUCH_FREQ0_DRV_HS 配置采样频率模式为 0 时触摸传感器的 DRV_HS 参数。(R/W)

TOUCH_AON_TOUCH_FREQ0_DBIAS 配置采样频率模式为 0 时是否开启防潮功能及配置触摸传感器内部电压。

bit[0]：配置在采样频率模式为 0 时是否开启防潮功能。

0：不开启

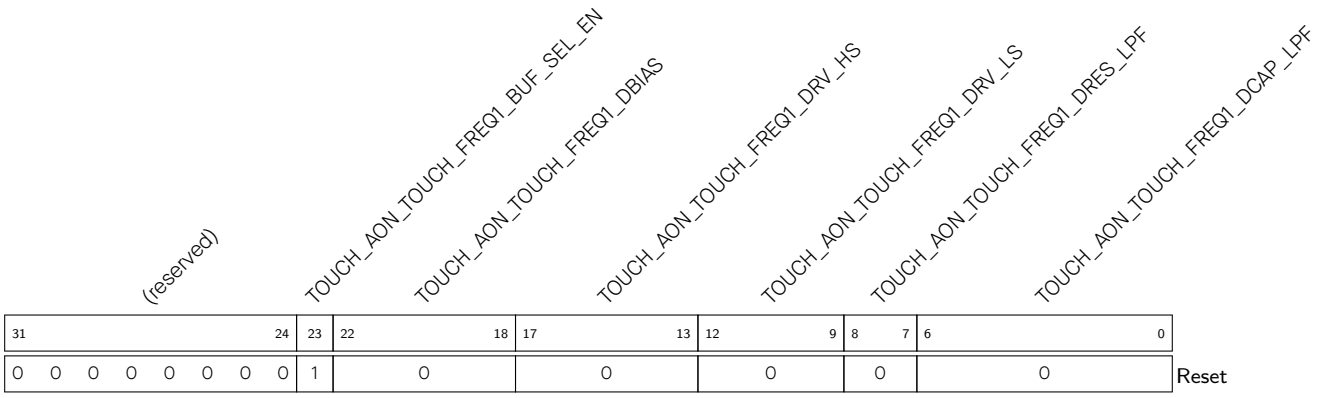
1：开启

bit[1-4]：配置采样频率模式为 0 时触摸传感器的内部电压。

(R/W)

TOUCH_AON_TOUCH_FREQ0_BUF_SEL_EN 配置在频率模式 0 下是否使能对应的 BUF_SEL。(R/W)

Register 27.25. TOUCH_AON_FREQ1_SCAN_PARA_REG (0x0034)



TOUCH_AON_TOUCH_FREQ1_DCAP_LPF 配置采样频率模式为 1 时触摸传感器低通滤波器参数，调节范围为 0 ~ 2.54 pF，调节精度为 20 fF。(R/W)

TOUCH_AON_TOUCH_FREQ1_DRES_LPF 配置采样频率模式为 1 时触摸传感器低通滤波器参数。

0: 0 kΩ

1: 3 kΩ

2: 4.5 kΩ

3: 7.5 kΩ

(R/W)

TOUCH_AON_TOUCH_FREQ1_DRV_LS 配置采样频率模式为 1 时触摸传感器的 DRV_LS 参数。(R/W)

TOUCH_AON_TOUCH_FREQ1_DRV_HS 配置采样频率模式为 1 时触摸传感器的 DRV_HS 参数。(R/W)

TOUCH_AON_TOUCH_FREQ1_DBIAS 配置采样频率模式为 1 时是否开启防潮功能及配置触摸传感器内部电压。

bit[0]：配置在采样频率模式为 1 时是否开启防潮功能。

0：不开启

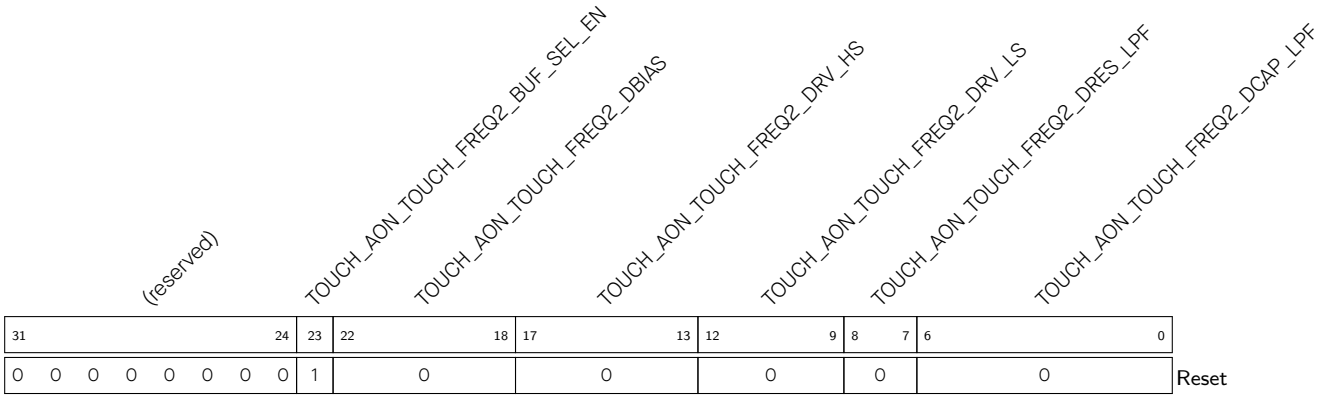
1：开启

bit[1-4]：配置采样频率模式为 1 时触摸传感器的内部电压。

(R/W)

TOUCH_AON_TOUCH_FREQ1_BUF_SEL_EN 配置在频率模式 1 下是否使能对应的 BUF_SEL。(R/W)

Register 27.26. TOUCH_AON_FREQ2_SCAN_PARA_REG (0x0038)



TOUCH_AON_TOUCH_FREQ2_DCAP_LPF 配置采样频率模式为 2 时触摸传感器低通滤波器参数，调节范围为 0 ~ 2.54 pF，调节精度为 20 fF。(R/W)

TOUCH_AON_TOUCH_FREQ2_DRES_LPF 配置采样频率模式为 2 时触摸传感器低通滤波器参数。

0: 0 kΩ

1: 3 kΩ

2: 4.5 kΩ

3: 7.5 kΩ

(R/W)

TOUCH_AON_TOUCH_FREQ2_DRV_LS 配置采样频率模式为 2 时触摸传感器的 DRV_LS 参数。(R/W)

TOUCH_AON_TOUCH_FREQ2_DRV_HS 配置采样频率模式为 2 时触摸传感器的 DRV_HS 参数。(R/W)

TOUCH_AON_TOUCH_FREQ2_DBIAS 配置采样频率模式为 2 时是否开启防潮功能及配置触摸传感器内部电压。

bit[0]：配置在采样频率模式为 2 时是否开启防潮功能。

0：不开启

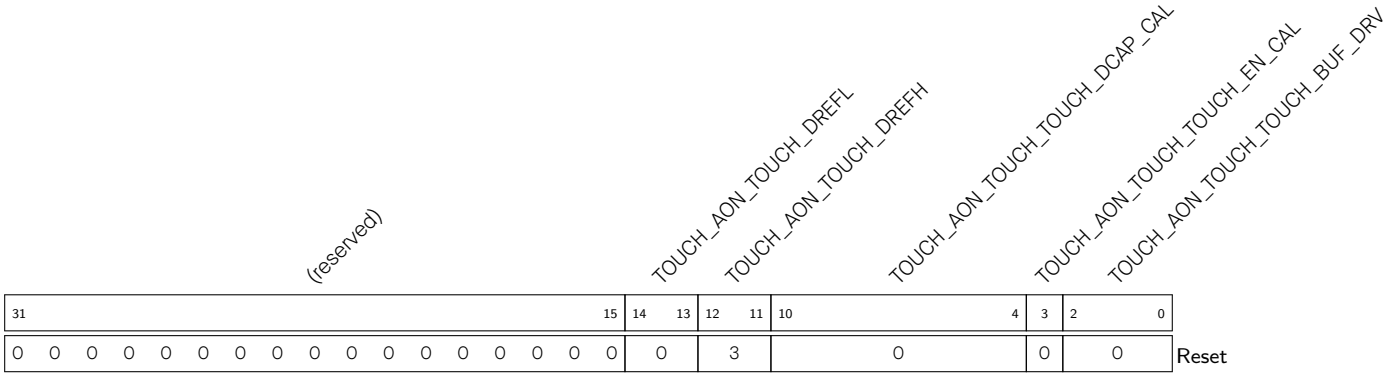
1：开启

bit[1-4]：配置采样频率模式为 2 时触摸传感器的内部电压。

(R/W)

TOUCH_AON_TOUCH_FREQ2_BUF_SEL_EN 配置在频率模式 2 下是否使能对应的 BUF_SEL。(R/W)

Register 27.27. TOUCH_AON_ANA_PARA_REG (0x003C)



TOUCH_AON_TOUCH_TOUCH_BUF_DRV 配置 BUF_DRV 的值。(R/W)

TOUCH_AON_TOUCH_TOUCH_EN_CAL 配置是否使能触摸传感器内部电容测试模式。

0：不使能

1：使能

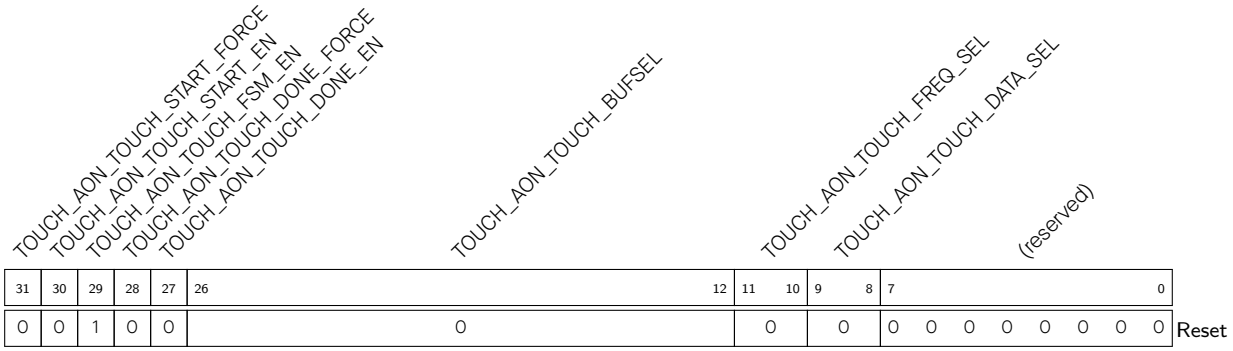
(R/W)

TOUCH_AON_TOUCH_TOUCH_DCAP_CAL 配置连接到触摸管脚上的内部电容，默认值为 0，调节范围为 0 ~ 127 pF，调节步长为 1。(R/W)

TOUCH_AON_TOUCH_DREFH 配置触摸传感器的上参考电压。(R/W)

TOUCH_AON_TOUCH_DREFL 配置触摸传感器的下参考电压。(R/W)

Register 27.28. TOUCH_AON_MUX0_REG (0x0040)



TOUCH_AON_TOUCH_DATA_SEL 配置返回值类型。

0/1：原始触摸数据

2：基准数据

3：平滑触摸数据

(R/W)

TOUCH_AON_TOUCH_FREQ_SEL 配置返回值所对应的采样频率模式。

0：0

1：1

2：2

3：无效

(R/W)

TOUCH_AON_TOUCH_BUFSEL 配置是否使能某一触摸管脚用于防潮功能，位 0 ~ 13 分别对应触摸管脚 0 ~ 13，其他位无效。

0：不使能

1：使能

(R/W)

TOUCH_AON_TOUCH_DONE_EN 配置是否使能测量的 DONE 信号由软件触发。

0：不使能

1：使能

(R/W)

TOUCH_AON_TOUCH_DONE_FORCE 配置是否产生 DONE 信号，结束当前测量操作。

0：不产生

1：产生

(R/W)

TOUCH_AON_TOUCH_FSM_EN 配置触摸管脚上下电和启动的控制源。

0：由软件控制

1：由硬件控制

(R/W)

见下页...

Register 27.28. TOUCH_AON_MUX0_REG (0x0040)

接上页...

TOUCH_AON_TOUCH_START_EN 配置是否选择测量的 START 信号由软件触发。

0：不使能

1：使能

(R/W)

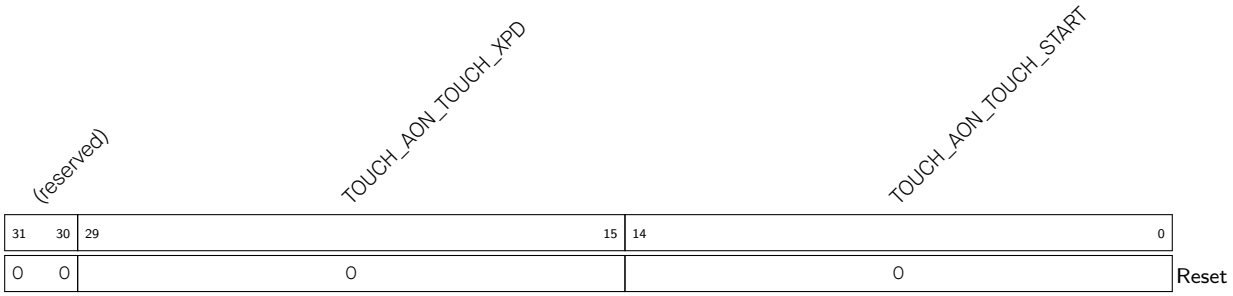
TOUCH_AON_TOUCH_START_FORCE 配置是否产生 START 信号，开启一次测量操作。

0：不产生

1：产生

(R/W)

Register 27.29. TOUCH_AON_MUX1_REG (0x0044)



TOUCH_AON_TOUCH_START 配置是否启动 14 个触摸管脚，位 0 ~ 13 分别对应触摸管脚 0 ~ 13，其他位无效。

0：无效

1：启动

(R/W)

TOUCH_AON_TOUCH_XPD 配置 14 个触摸管脚的上下电，位 0 ~ 13 分别对应触摸管脚 0 ~ 13，其他位无效。

0：不上电

1：上电

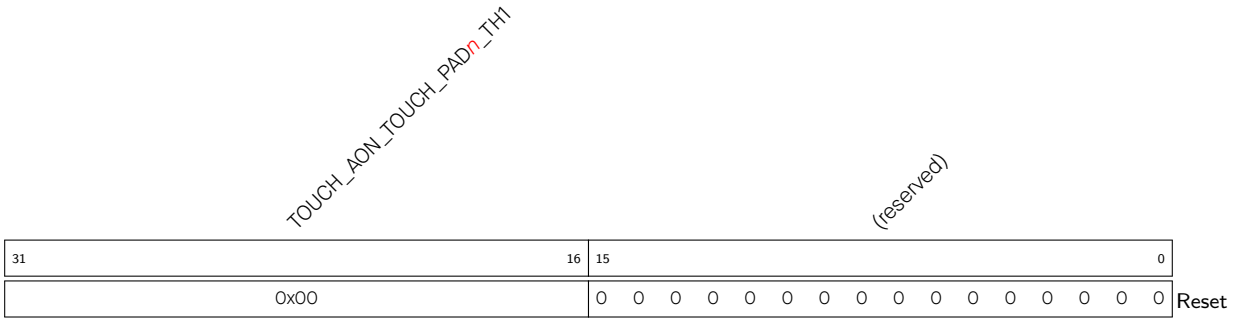
(R/W)

Register 27.30. TOUCH_AON_PADn_TH0_REG (n: 0-13) (0x0048+0xC*n)



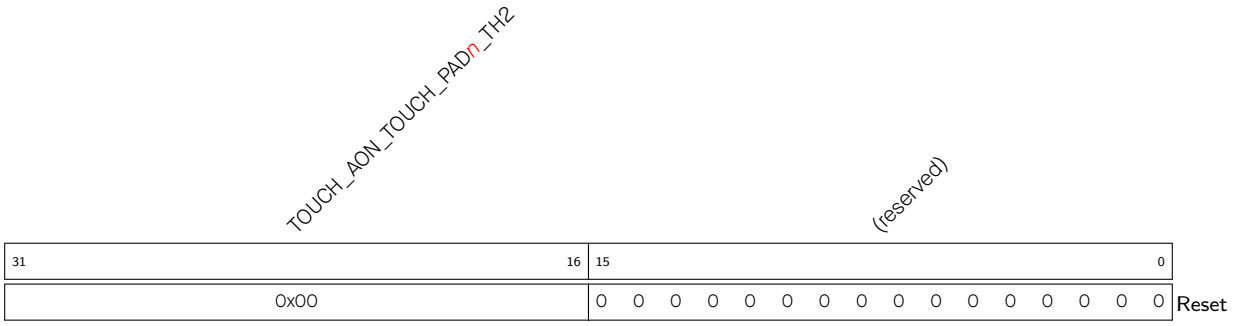
TOUCH_AON_TOUCH_PADn_TH0 配置触摸传感器 n 在采样频率模式为 0 时的触碰阈值。(R/W)

Register 27.31. TOUCH_AON_PADn_TH1_REG (n: 0-13) (0x004C+0xC*n)



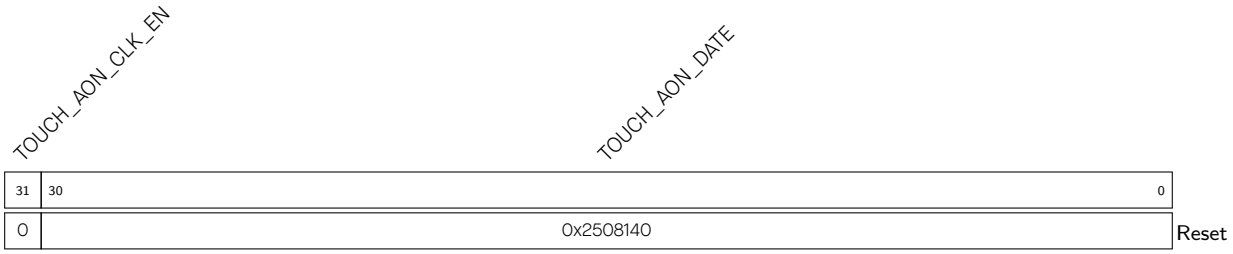
TOUCH_AON_TOUCH_PADn_TH1 配置触摸传感器 n 在采样频率模式为 1 时的触碰阈值。(R/W)

Register 27.32. TOUCH_AON_PADn_TH2_REG (n: 0-13) (0x0050+0xC*n)



TOUCH_AON_TOUCH_PADn_TH2 配置触摸传感器 n 在采样频率模式为 2 时的触碰阈值。(R/W)

Register 27.33. TOUCH_AON_DATE_REG (0x00FC)



TOUCH_AON_DATE 版本控制寄存器。(R/W)

TOUCH_AON_CLK_EN 配置是否使能访问寄存器的时钟。

0：不使能

1：使能

(R/W)

第 28 章

SAR ADC 数字控制器 (ADC)

28.1 概述

ESP32-S31 SAR ADC 数字控制器是 HP ADC 子系统的数字控制模块，负责管理两路独立 SAR ADC（SAR1 和 SAR2）的采样触发、时序控制、数据滤波、阈值监控以及 DMA 传输。模块通过 APB 总线向 CPU 提供寄存器接口，支持软件触发、定时扫描和 ETM 事件触发三种触发源，并通过异步 FIFO 将采样结果发送到系统总线。

模块输出 1 路汇聚中断 `adc_reg_int`，由 7 个中断源聚合而来，软件通过标准 RAW/ST/ENA/CLR 四寄存器流程访问。

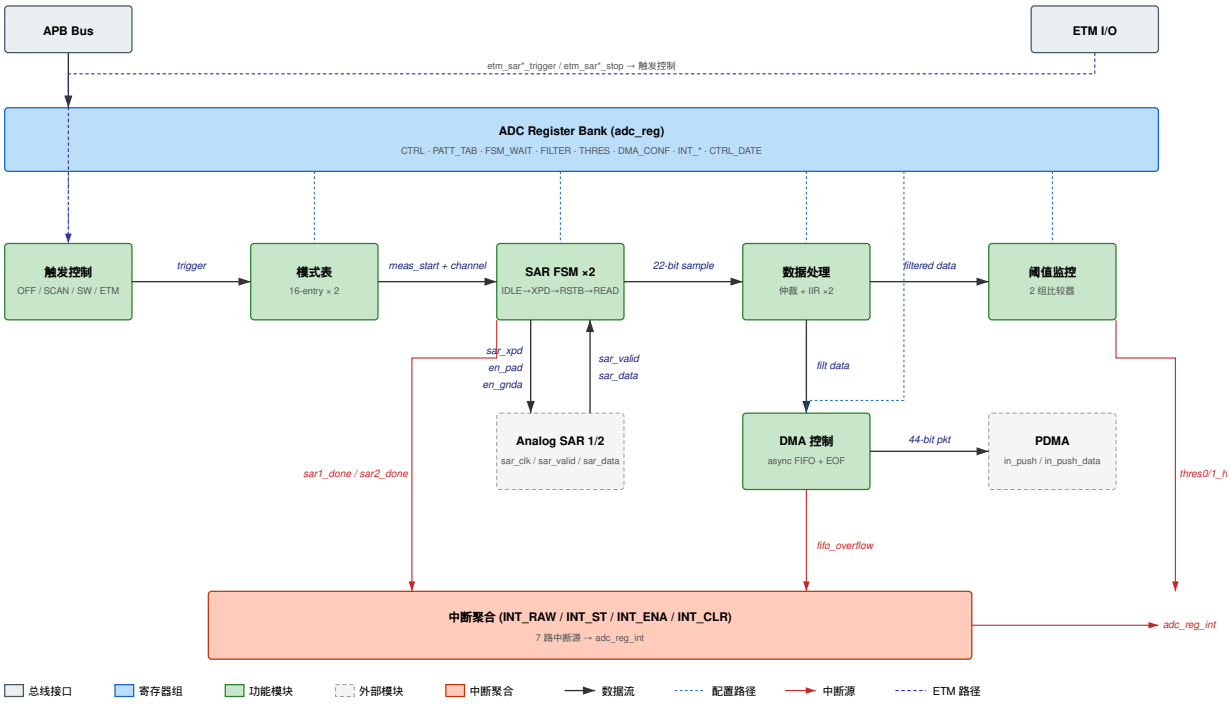


图 28.1-1. SAR ADC 数字控制器系统框图

28.2 特性

SAR ADC 数字控制器具有以下特性：

- 支持两路独立 SAR ADC (SAR1 / SAR2) 并行工作
- 每路支持 3 种触发模式：SCAN（定时器）、SW（软件写寄存器）、ETM
- 每路独立的模式表包含 16 个样式，可按顺序循环扫描 16 个通道（支持单端和差分）

- 每路独立的四状态 SAR 控制 FSM（IDLE → XPD → RSTB → READ），支持连续采样模式
- 2 路 IIR 低通滤波器，滤波系数可选 1/2/4/8/16/32/64
- 2 组阈值比较器，支持单通道监控和 thres0 全通道监控
- DMA 传输通路（异步 FIFO + EOF 帧管理），其中异步 FIFO 深度为 4
- 7 路中断聚合输出：SAR 转换完成、阈值越限、FIFO 溢出
- ETM 事件输入/输出
- 寄存器时钟门控

28.2.1 ADC 通道

SAR ADC 具有 8 组差分通道，连接芯片的 16 个管脚。ADC 需首先通过内部多路器选择待测量的模拟管脚，然后才能采样模拟信号。

表 28.2-1 列出了用作 ADC 通道的芯片管脚。

表 28.2-1. SAR ADC 通道

管脚序号	管脚名称	ADC 通道编号
55	GPIO42	ADC1_CHANNEL0_N
56	GPIO43	ADC1_CHANNEL0_P
57	GPIO44	ADC1_CHANNEL1_N
58	GPIO45	ADC1_CHANNEL1_P
59	GPIO46	ADC1_CHANNEL2_N
60	GPIO47	ADC1_CHANNEL2_P
61	GPIO48	ADC1_CHANNEL3_N
62	GPIO49	ADC1_CHANNEL3_P
65	GPIO50	ADC2_CHANNEL0_N
66	GPIO51	ADC2_CHANNEL0_P
67	GPIO52	ADC2_CHANNEL1_N
68	GPIO53	ADC2_CHANNEL1_P
69	GPIO54	ADC2_CHANNEL2_N
70	GPIO55	ADC2_CHANNEL2_P
71	GPIO56	ADC2_CHANNEL3_N
72	GPIO57	ADC2_CHANNEL3_P

28.2.2 ADC 时钟

SAR ADC 的系统时钟为 ADC_CTRL_CLK，它是 DIG ADC FSM 和其他逻辑电路的工作时钟（APB 接口除外）。它是源时钟经过 LP_PERICLKRST_LP_ADC_DIV_NUM + 1 的分频所得。

ADC_CTRL_CLK 可选择两个时钟源：XTAL_CLK 和 RC_FAST_CLK，通过寄存器 LP_PERICLKRST_LP_ADC_CLK_SEL 配置。

更多有关时钟的信息请参考章节 7 *复位和时钟 [to be added later]*。

28.3 触发控制

28.3.1 特性

- SAR1 / SAR2 独立配置触发模式
- 4 种触发模式：OFF / SCAN / SW / ETM
- 内置定时器生成 SCAN 触发脉冲
- 软件和 ETM 均可随时停止转换并返回 IDLE

28.3.2 功能描述

28.3.2.1 触发模式选择

通过 ADC_SAR1_TRIGGER_MODE 选择 SAR1 的触发模式：

- 0：OFF，关闭触发，不产生任何采样启动脉冲
- 1：SCAN，定时器周期性自动触发
- 2：SW，软件写 ADC_SAR1_TRIGGER_START = 1 单次触发
- 3：ETM，由外部 ETM 信号 `etm_sar1_trigger` 触发，详细内容请参考 28.9 *ETM 事件*

SAR2 通过 ADC_SAR2_TRIGGER_MODE 独立配置，行为相同。

28.3.2.2 定时器触发

当触发模式为 SCAN 时，内部定时器以 ADC 工作时钟计数：

1. 软件写 ADC_TIMER_EN = 1 使能定时器
2. 软件写 ADC_TIMER_TARGET 设置计数周期
3. 定时器计数到目标值后自动产生一次采样触发脉冲，并重新开始计数；采样周期 = 2 × ADC_TIMER_TARGET 个 ADC 工作时钟
4. 写 ADC_TIMER_EN = 0 关闭定时器并复位计数器

28.3.2.3 软件触发与停止

- **启动**：向 ADC_SAR1_TRIGGER_START (WT) 写 1，在 SW 模式下立即产生一次触发脉冲；SAR2 对应 ADC_SAR2_TRIGGER_START
- **停止**：向 ADC_SAR1_TRIGGER_STOP (WT) 写 1，无论 FSM 当前状态立即回到 IDLE；SAR2 对应 ADC_SAR2_TRIGGER_STOP
- 软件停止命令与 ETM 的 `etm_sar1_stop` / `etm_sar2_stop` 停止信号会进行逻辑或运算，任意信号有效时均可使 FSM 返回 IDLE

28.4 模式表

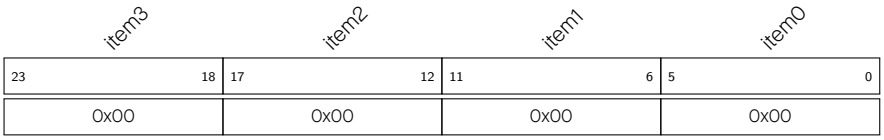
28.4.1 特性

- 每路 SAR 独立的 16 个样式
- 每个样式包含通道号和单端/差分模式
- 支持循环扫描，指针自动回卷
- 模式表更新时机可配置（测量完成 / 测量开始）

28.4.2 功能描述

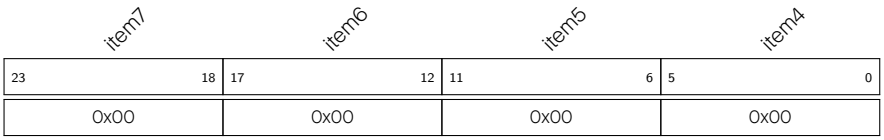
28.4.2.1 模式表结构

模式表最多包含 16 个样式，每个样式为 6-bit 的采样配置。每路 SAR 通过 4 个 24-bit 寄存器承载 16 个样式：ADC_SAR1_PATT_TAB1_REG ~ ADC_SAR1_PATT_TAB4_REG。每个寄存器内 4 个 item 位域从高位到低位排列，分别对应 4 个样式。



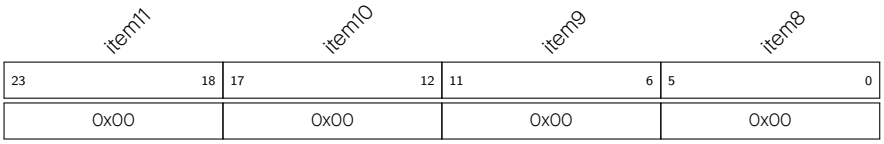
item *x* 表示样式 0 ~ 样式 3。

图 28.4-1. ADC_SAR1_PATT_TAB1_REG 包含样式 0 ~ 3



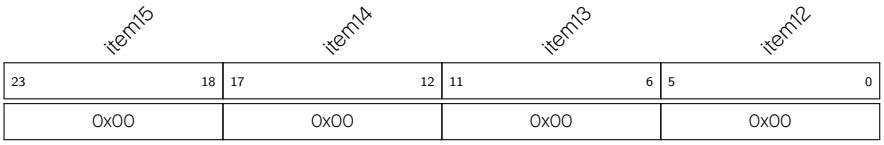
item *x* 表示样式 4 ~ 样式 7。

图 28.4-2. ADC_SAR1_PATT_TAB2_REG 包含样式 4 ~ 7



item *x* 表示样式 8 ~ 样式 11。

图 28.4-3. ADC_SAR1_PATT_TAB3_REG 包含样式 8 ~ 11



item *x* 表示样式 12 ~ 样式 15。

图 28.4-4. ADC_SAR1_PATT_TAB4_REG 包含样式 12 ~ 15

16 个样式按以下顺序排列：ADC_SAR1_PATT_TAB1_REG 包含样式 0~3，ADC_SAR1_PATT_TAB2_REG 包含样

式 4~7，ADC_SAR1_PATT_TAB3_REG 包含样式 8~11，ADC_SAR1_PATT_TAB4_REG 包含样式 12~15。每个寄存器内 item 位域从高位到低位排列（[23:18] 为 item3，[17:12] 为 item2，[11:6] 为 item1，[5:0] 为 item0）。

每个样式占 6 bit，包含 mode、reserved、channel 三个子字段。图 28.4-5 所示为样式结构，子字段说明见下文。

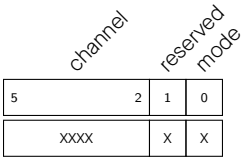


图 28.4-5. 样式结构（6-bit）

mode 配置模式：

0：单端模式

1：差分模式

详见章节 28.4.2.4 通道与模式选择。

reserved 保留字段

channel 配置采样通道号：

0：CHANNEL_0_N

1：CHANNEL_0_P

2：CHANNEL_1_N

3：CHANNEL_1_P

4：CHANNEL_2_N

5：CHANNEL_2_P

6：CHANNEL_3_N

7：CHANNEL_3_P

8–15：reserved

SAR2 使用 ADC_SAR2_PATT_TAB1_REG ~ ADC_SAR2_PATT_TAB4_REG，编码相同。

28.4.2.2 扫描长度与指针

ADC_SAR1_PATT_LEN（4 bits）配置模式表长度，值 0~15 对应长度 1~16。模式表指针从 0 开始，每次更新请求后递增 1，达到 ADC_SAR1_PATT_LEN 后回卷到 0。

写 ADC_SAR1_PATT_P_CLEAR (WT) = 1 可立即清零模式表指针。

28.4.2.3 更新时机

ADC_SAR1_PATT_TYPE 控制模式表指针的更新时机：

- 0：当前采样完成后切换到下一个样式—适用于每次采样完成后都需切换样式的场景
- 1：下一次采样启动时切换到下一个样式—适用于仅在定时器或 ETM 触发时才切换样式的场景

28.4.2.4 通道与模式选择

根据当前样式的 mode 位：

- **单端模式**（mode = 0）：使能 channel 指定的单个 pad，同时使能对应的 GND 参考

- **差分模式**（mode = 1）：使能 channel[3:1] 指定的相邻偶奇 pad 对，GND 参考关闭

28.5 SAR 控制 FSM

28.5.1 特性

- 四状态 FSM：IDLE → XPD → RSTB → READ
- 上电等待和复位等待周期独立可配置
- 支持连续采样模式
- SAR 电源三态控制（自动 / 强制关闭 / 强制开启）
- SAR1 和 SAR2 各自独立的 FSM 实例

28.5.2 功能描述

28.5.2.1 状态机流程

- **IDLE**：空闲，等待采样触发
- **XPD**：使能 SAR 电源，等待 ADC_XPD_WAIT 次 SAR 转换完成
- **RSTB**：纯等待状态，等待 ADC_RSTB_WAIT 次 SAR 转换完成，为模拟侧预留建立时间
- **READ**：等待模拟 SAR 送出下一次转换结果，锁存后回到 IDLE

ADC_XPD_WAIT 和 ADC_RSTB_WAIT 通过 ADC_FSM_WAIT_REG 配置，各 4 bit。

模拟 SAR 上电后自行连续转换，每 24 个 ADC 工作时钟（T_{conv}）完成一次并给出结果，与 FSM 之间无握手。一次转换分两段：**前 5 clk 为采样窗**，模拟输入被采样到电容阵列上；其后 19 clk 为逐位逼近，模拟输入不再影响结果。因此模拟通道需在采样窗打开前建立完成。

两个等待值以**完成的 SAR 转换次数**为单位，一个计数即一整个 T_{conv}。配 0 是特例：该状态只停留 1 个 ADC 工作时钟，因此 ADC_RSTB_WAIT 从 0 改到 1 会跳变一整个 T_{conv}。ADC_RSTB_WAIT 推荐配置为 2。

28.5.2.2 连续采样模式

ADC_SAR1_CONTINUE_MODE_EN = 1 时，READ 完成后不回 IDLE：收到新触发则跳 RSTB，否则**保持在 READ**。

FSM 停在 READ 期间持续采样：每 24 clk 输出一个样本，刷新 ADC_APB_SARADC1_DATA，产生一次 ADC_SAR1_DONE_INT 中断、一次 DMA 推送和一个 ETM 事件，并把模式表指针推进一格。因此连续模式的用法是**触发一次后自由运行**——启动后即以每 24 clk 一个样本持续输出，通道按模式表循环。SAR2 对应 ADC_SAR2_CONTINUE_MODE_EN。

再次触发会使 FSM 跳回 RSTB，该期间无样本输出。因此连续模式下的触发不是「发起一次采样」，而是**在样本流中插入一段 RSTB**。

采样耗时：

- **启动延迟**（触发 → 第一个样本）≈ (ADC_XPD_WAIT + ADC_RSTB_WAIT + 1) × 24 clk；ADC_XPD_WAIT = 8、ADC_RSTB_WAIT = 2 时约 264 clk
- **稳态采样周期** = 24 clk，吞吐 1 / 24 clk，与两个等待值无关——它们只在启动时各走一次

下面三张图的 T_{conv} 按比例压缩以便画出 clk，等待值取 ADC_XPD_WAIT = 2、ADC_RSTB_WAIT = 1（场景 2 取 0），分别如图 28.5-1、图 28.5-2 与图 28.5-3 所示。

场景 1：多通道自由扫描（ADC_SAR1_PATT_TYPE = 0，done 切换）——指针每出一个样本推进一次，通道逐样本轮转。

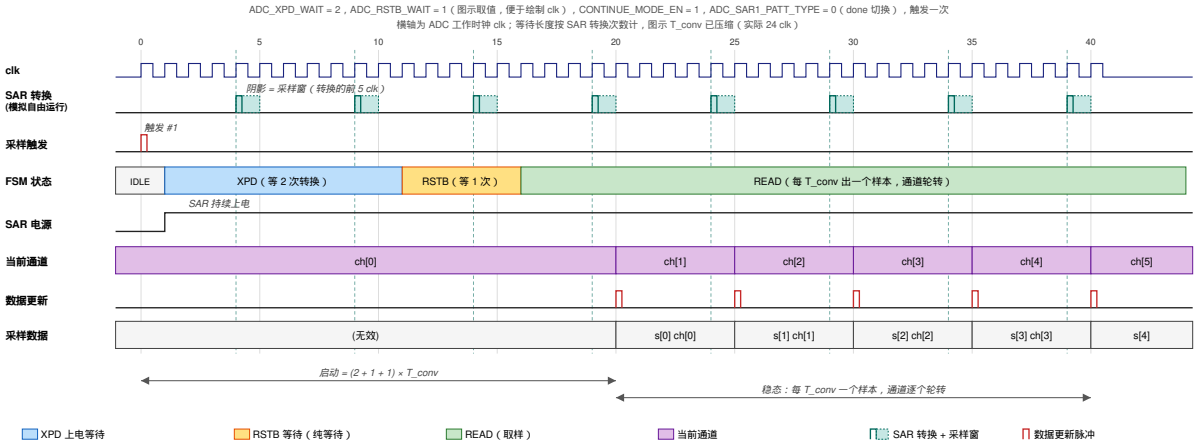


图 28.5-1. 连续采样与通道切换时序图

场景 2：通道不切换（ADC_SAR1_PATT_TYPE = 1，start 切换）——指针改为在触发时推进，自由运行期间无触发，通道保持不变。

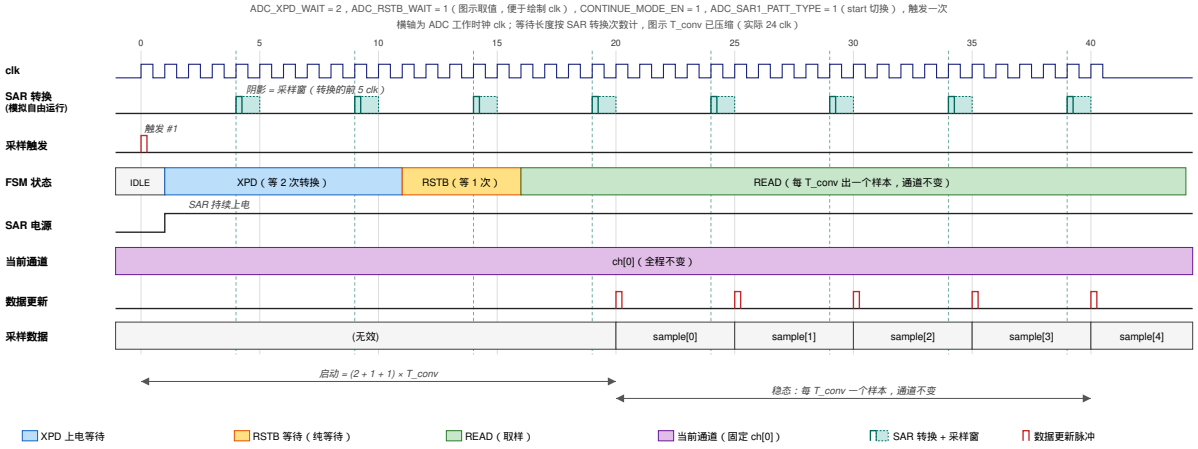


图 28.5-2. 连续采样时序图（PATT_TYPE = 1，通道不切换）

场景 3：非连续模式（ADC_SAR1_CONTINUE_MODE_EN = 0）软件触发——每次触发走完 IDLE → XPD → RSTB → READ 再回 IDLE，采样间隔期间 SAR 断电。

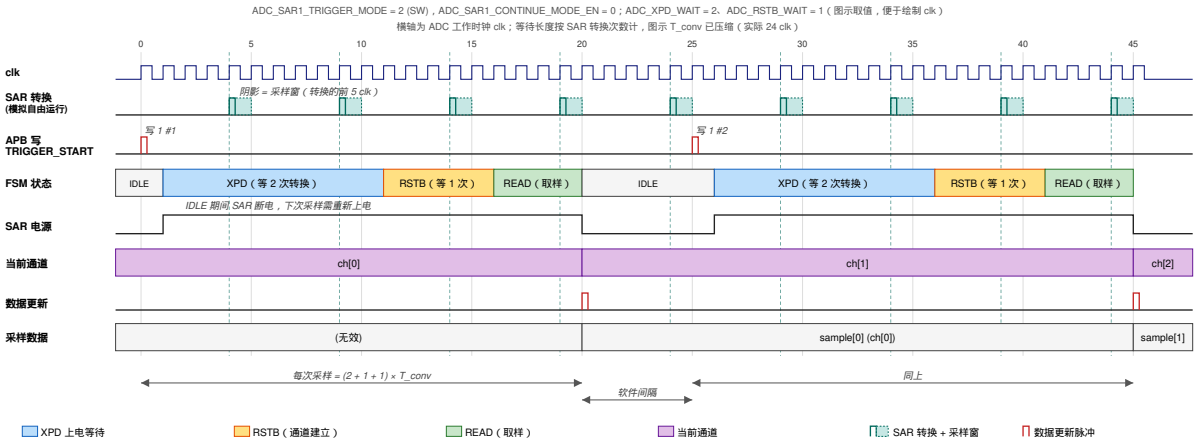


图 28.5-3. 非连续模式软件触发采样时序图

通道切换位置：模式表指针在样本输出的同时推进，通道在其后 1 ~ 2 个 clk 切换，而采样窗是每次转换开头的 5 个 clk。因此切换落在采样窗内：窗内前 1 ~ 2 clk 仍处在上一通道，其余 3 ~ 4 clk 处在新通道。多通道自由扫描的样本是否有效，取决于模拟 mux 能否在这 3 ~ 4 clk 内建立完成。

单次触发后自由运行时 ADC_RSTB_WAIT 只走一次，其建立时间只作用于第一个样本。**多通道扫描应按采样节奏周期性触发**：每次触发使 FSM 经过 RSTB，为刚切换的通道提供 ADC_RSTB_WAIT × 24 clk 建立时间，此后取的样本采样窗完整落在新通道上。若需要更长的建立时间，可改用 ADC_SAR1_CONTINUE_MODE_EN = 0（场景 3）。

三种用法的取舍：

用法	采样周期	通道建立时间	适用
连续模式 + 单次触发	24 clk	采样窗内约 3 ~ 4 clk	通道不切换的高速采样
连续模式 + 周期触发	(RSTB_WAIT + 1) × 24 clk，推荐 2 为 72 clk	RSTB_WAIT × 24 clk，推荐 2 为约 46 clk	**多通道扫描**
CONTINUE_MODE_EN = 0	(XPD_WAIT + RSTB_WAIT + 1) × 24 clk，推荐 8 / 2 为 264 clk	(XPD_WAIT + RSTB_WAIT) × 24 clk，推荐 8 / 2 为 240 clk	需要更长建立时间时

SCAN 模式下定时器触发周期应不短于上表的采样周期（触发周期 = 2 × ADC_TIMER_TARGET 个 ADC 工作时钟）。

28.5.2.3 SAR 电源控制

ADC_XPD_SAR1_FORCE[1:0] 控制 SAR1 电源：

- 0~1：自动控制—非 IDLE 状态时开启，IDLE 时关闭
- 2：强制关闭 SAR 电源
- 3：强制开启 SAR 电源

SAR2 通过 ADC_XPD_SAR2_FORCE 独立控制。

28.5.2.4 状态查询

软件可读取 ADC_SAR1_STATE (RO) 或 ADC_SAR2_STATE (RO) 查询 FSM 当前状态：0=IDLE, 1=XPD, 2=RSTB, 3=READ。

28.6 数据处理

28.6.1 特性

- SAR1 / SAR2 双路数据仲裁
- 2 个独立 IIR 低通滤波器
- 滤波系数为 2 的幂次（1~64）
- 不匹配滤波通道的数据直接透传

28.6.2 功能描述

28.6.2.1 数据仲裁

SAR1 和 SAR2 的转换完成信号到达后，数据处理模块按以下规则选择数据：

- 仅 SAR1 数据到达：处理 SAR1 数据
- 仅 SAR2 数据到达：处理 SAR2 数据
- 两路同时到达：先处理 SAR1，再处理 SAR2（顺序完成后结果进入后级）

28.6.2.2 IIR 滤波器

模块提供 2 个独立的 IIR 滤波通道。ADC_FILTER_CHANNELO 与 ADC_FILTER_CHANNEL1 分别指定各滤波通道要滤波的通道号。

滤波系数通过 ADC_FILTER_FACTORO / ADC_FILTER_FACTOR1（3 bits）配置：

- 0：k=1（无滤波，直通）
- 1：k=2
- 2：k=4
- 3：k=8
- 4：k=16
- 5：k=32
- 6：k=64

滤波算法为一阶 IIR：

$$sum_n = sum_{n-1} - \frac{sum_{n-1}}{k} + adc_din_n$$

$$filtered_data_n = \frac{sum_n}{k}$$

其中 sum 为内部的积分器，adc_din 为输入的 ADC 采样原始值，filtered_data 为经过滤波器的 ADC 数据。

不匹配滤波通道号的样点不经滤波，直接透传到后续处理。写 ADC_FILTER_RESET (WT) = 1 可清零两个滤波器的内部状态。

28.6.2.3 采样数据读取

最近一次转换的数据可通过 ADC_APB_SARADC1_DATA（RO，22 bits）和 ADC_APB_SARADC2_DATA（RO，22 bits）读取，[21:17] 为通道号，[16:0] 为 17 bits ADC 转换结果。

28.7 阈值监控

28.7.1 特性

- 2 组独立阈值比较器：阈值 0 (thres0)、阈值 1 (thres1)
- 每组独立配置监控通道、上阈值和下阈值
- thres0 支持全通道监控模式
- 越限时置位对应中断

28.7.2 功能描述

28.7.2.1 阈值配置

每组阈值比较器对应的寄存器字段：

- 使能：ADC_THRES0_EN（写 1 使能）
- 通道：ADC_THRES0_CHANNEL（5 bits）
- 高阈值：ADC_THRES0_HIGH（17 bits）
- 低阈值：ADC_THRES0_LOW（17 bits）

thres1 使用 ADC_THRES1_EN / ADC_THRES1_CHANNEL / ADC_THRES1_HIGH / ADC_THRES1_LOW。

28.7.2.2 比较逻辑

滤波后的采样数据到达时，如果通道号匹配且比较器已使能：

- 若 adc_din > ADC_THRES0_HIGH：产生 THRES0_HIGH 中断脉冲
- 若 adc_din < ADC_THRES0_LOW：产生 THRES0_LOW 中断脉冲

每次越限事件置位一次中断；软件通过写 ADC_INT_CLR_REG 清除后，后续新的越限事件仍会继续触发。

28.7.2.3 全通道监控模式

设置 ADC_THRES_ALL_EN = 1 时，thres0 不再限制于 ADC_THRES0_CHANNEL 指定的通道，而是监控所有通道的采样数据。该功能仅对 thres0 有效，thres1 始终按指定通道监控。

28.8 DMA 控制

28.8.1 特性

- 通过异步 FIFO 跨 ADC 时钟域 → DMA 时钟域传输数据
- 支持 EOF 帧标记，方便软件分帧处理
- FIFO 深度 4，溢出保护与中断

28.8.2 功能描述

28.8.2.1 DMA 使能

设置 ADC_APB_ADC_TRANS = 1 使能 DMA 传输通路。使能后，每次经数据处理完成的样点都会写入异步 FIFO，由 DMA 控制器搬运到系统内存。

28.8.2.2 EOF 帧管理

ADC_APB_ADC_EOF_NUM（16 bits）配置每帧样点数。DMA 控制器维护一个 EOF 计数器：

1. 每传输一个样点递增 1
2. 计数达到 ADC_APB_ADC_EOF_NUM –1 时在 DMA 数据中插入 EOF 标志位，并清零计数器
3. 软件可通过写 ADC_DMA_EOF_NUM_CLR (WT) = 1 手动清零计数器

28.8.2.3 DMA 数据格式

软件视角下，每次 DMA 传输得到一个 32 位采样字（4 字节），组织如下：

- byte 3：保留
- byte 2：`channel`+ `data_msb` —采样通道号 + ADC 采样数据的 MSB
- byte 1：`data_high` —ADC 采样数据高 8 位
- byte 0：`data_low` —ADC 采样数据低 8 位

完整 ADC 采样值由 `data_msb`、`data_high` 与 `data_low` 拼接：
`sample = (data_msb << 16) | (data_high << 8) | data_low`。软件按 4 字节读取 DMA 数据即可；硬件内部对齐与打包细节由 DMA 控制器处理。

28.8.2.4 FIFO 溢出

当异步 FIFO 写满后继续写入会触发一次 ADC_FIFO_OVERFLOW 中断。中断仅在 FIFO 进入溢出状态的瞬间产生一次，若持续溢出不会重复触发；软件清中断并排空 FIFO 后，下一次新的溢出事件才会再次触发。

28.9 ETM 事件

28.9.1 特性

- 支持 ETM 触发输入启动/停止 ADC 转换

- 提供多种事件输出供系统其他模块使用
- 输出独立的 ETM 参考时钟

28.9.2 功能描述

28.9.2.1 ETM 触发输入

- `etm_sar1_trigger` —SAR1 ETM 触发启动（ADC_SAR1_TRIGGER_MODE = 3 时有效）
- `etm_sar2_trigger` —SAR2 ETM 触发启动（ADC_SAR2_TRIGGER_MODE = 3 时有效）
- `etm_sar1_stop` —SAR1 ETM 停止，与软件 ADC_SAR1_TRIGGER_STOP 逻辑或后生效
- `etm_sar2_stop` —SAR2 ETM 停止，与软件 ADC_SAR2_TRIGGER_STOP 逻辑或后生效

28.9.2.2 ETM 事件输出

- `etm_sar1_done` / `etm_sar2_done` —单次转换完成事件
- `etm_event_sar1_start` / `etm_event_sar2_start` —采样开始事件（FSM 由 RSTB 进入 READ）
- `etm_event_thres0_high_int_etm` / `etm_event_thres1_high_int_etm` —阈值高越限事件
- `etm_event_thres0_low_int_etm` / `etm_event_thres1_low_int_etm` —阈值低越限事件

28.9.2.3 ETM 时钟

`clk_adc_etm` 输出 ADC 工作时钟，供 ETM 系统作为参考时钟使用。

28.10 中断

SAR ADC 模块共提供 7 路中断源，聚合成 1 路 `adc_reg_int` 输出：

表 28.10-1. SAR ADC 中断源

中断源	描述
ADC_SAR1_DONE_INT	SAR1 单次转换完成
ADC_SAR2_DONE_INT	SAR2 单次转换完成
ADC_THRES0_HIGH_INT	滤波后的数据超过高阈值 ADC_THRES0_HIGH
ADC_THRES1_HIGH_INT	滤波后的数据超过高阈值 ADC_THRES1_HIGH
ADC_THRES0_LOW_INT	滤波后的数据低于低阈值 ADC_THRES0_LOW
ADC_THRES1_LOW_INT	滤波后的数据低于低阈值 ADC_THRES1_LOW
ADC_FIFO_OVERFLOW_INT	DMA FIFO 溢出

中断寄存器组：ADC_INT_ENA_REG（0x064）、ADC_INT_RAW_REG（0x068）、ADC_INT_ST_REG（0x06C）、ADC_INT_CLR_REG（0x070）。

中断处理流程：

1. 硬件事件发生后 ADC_INT_RAW_REG 对应 bit 自动置位

2. ADC_INT_ST_REG = ADC_INT_RAW_REG & ADC_INT_ENA_REG
3. ADC_INT_ST_REG 任一 bit 非零时 `adc_reg_int` 输出拉高
4. CPU 读 ADC_INT_ST_REG 判断中断源
5. 处理完成后写 ADC_INT_CLR_REG 对应 bit = 1 清除
6. ADC_INT_RAW_REG 不会自动清除，必须软件写 ADC_INT_CLR_REG 清除

28.11 寄存器列表

本小节的所有地址均为相对于 SAR ADC 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

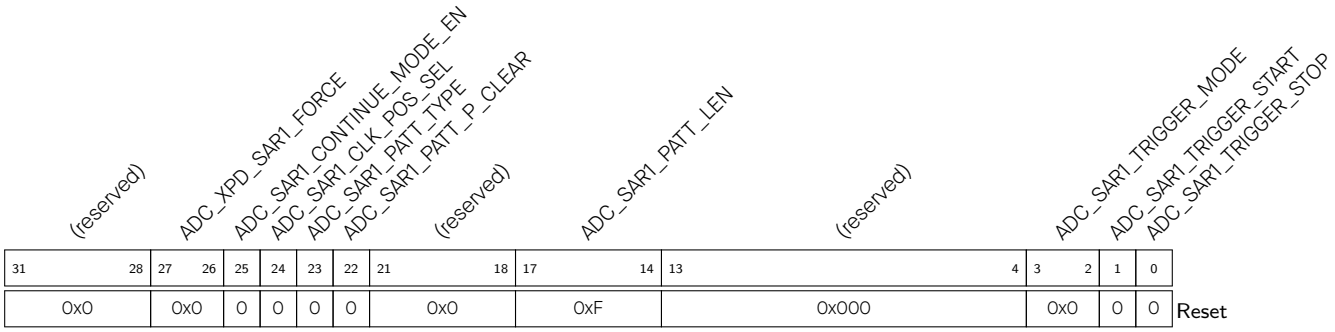
名称	描述	地址	访问
配置寄存器			
ADC_CTRL0_REG	SAR1 控制寄存器	0x000	R/W
ADC_CTRL1_REG	SAR2 控制寄存器	0x004	R/W
ADC_CTRL2_REG	定时器配置寄存器	0x008	R/W
ADC_FILTER_CTRL1_REG	滤波系数配置寄存器	0x00C	R/W
ADC_FSM_WAIT_REG	FSM 等待时间配置寄存器	0x010	R/W
ADC_SAR1_STATUS_REG	SAR1 状态寄存器	0x014	RO
ADC_SAR2_STATUS_REG	SAR2 状态寄存器	0x018	RO
ADC_SAR1_PATT_TAB1_REG	SAR1 模式表样式 0~3 寄存器	0x01C	R/W
ADC_SAR1_PATT_TAB2_REG	SAR1 模式表样式 4~7 寄存器	0x020	R/W
ADC_SAR1_PATT_TAB3_REG	SAR1 模式表样式 8~11 寄存器	0x024	R/W
ADC_SAR1_PATT_TAB4_REG	SAR1 模式表样式 12~15 寄存器	0x028	R/W
ADC_SAR2_PATT_TAB1_REG	SAR2 模式表样式 0~3 寄存器	0x02C	R/W
ADC_SAR2_PATT_TAB2_REG	SAR2 模式表样式 4~7 寄存器	0x030	R/W
ADC_SAR2_PATT_TAB3_REG	SAR2 模式表样式 8~11 寄存器	0x034	R/W
ADC_SAR2_PATT_TAB4_REG	SAR2 模式表样式 12~15 寄存器	0x038	R/W
ADC_FILTER_CTRL0_REG	滤波控制寄存器	0x03C	R/W
ADC_SAR1_DATA_STATUS_REG	SAR1 转换数据状态寄存器	0x040	RO
ADC_SAR2_DATA_STATUS_REG	SAR2 转换数据状态寄存器	0x044	RO
ADC_THRES0_HIGH_THRES_REG	阈值 0 高阈值寄存器	0x048	R/W
ADC_THRES0_LOW_THRES_REG	阈值 0 低阈值寄存器	0x04C	R/W
ADC_THRES0_CTRL_REG	阈值 0 控制寄存器	0x050	R/W
ADC_THRES1_HIGH_THRES_REG	阈值 1 高阈值寄存器	0x054	R/W
ADC_THRES1_LOW_THRES_REG	阈值 1 低阈值寄存器	0x058	R/W
ADC_THRES1_CTRL_REG	阈值 1 控制寄存器	0x05C	R/W
ADC_THRES_CTRL_REG	阈值 0 全通道监控寄存器	0x060	R/W
ADC_INT_ENA_REG	中断使能寄存器	0x064	R/W
ADC_INT_RAW_REG	原始中断状态寄存器	0x068	R/WTC/SS
ADC_INT_ST_REG	屏蔽中断状态寄存器	0x06C	RO
ADC_INT_CLR_REG	中断清除寄存器	0x070	WT

名称	描述	地址	访问
ADC_DMA_CONF_REG	DMA 配置寄存器	0x074	R/W
ADC_CTRL_DATE_REG	时钟门控与版本控制寄存器	0x3FC	R/W

28.12 寄存器

本小节的所有地址均为相对于 SAR ADC 控制器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

Register 28.1. ADC_CTRL0_REG (0x000)



ADC_XPD_SAR1_FORCE 配置 SAR1 电源强制控制。

0~1：自动控制（非 IDLE 时开启，IDLE 时关闭）

2：强制关闭

3：强制开启

(R/W)

ADC_SAR1_CONTINUE_MODE_EN 配置是否使能 SAR1 连续采样模式。

0：禁用

1：使能

(R/W)

ADC_SAR1_CLK_POS_SEL 配置 SAR1 采样数据时钟相位。

0：时钟下降沿

1：时钟上升沿

(R/W)

ADC_SAR1_PATT_TYPE 配置 SAR1 模式表指针更新时机。

0：测量完成时更新

1：测量开始时更新

(R/W)

ADC_SAR1_PATT_P_CLEAR 写 1 清零 SAR1 模式表指针。(WT)

ADC_SAR1_PATT_LEN 配置 SAR1 模式表长度。

0~15：对应长度 1~16

(R/W)

ADC_SAR1_TRIGGER_MODE 配置 SAR1 触发模式。

0：OFF

1：SCAN

2：SW

3：ETM

(R/W)

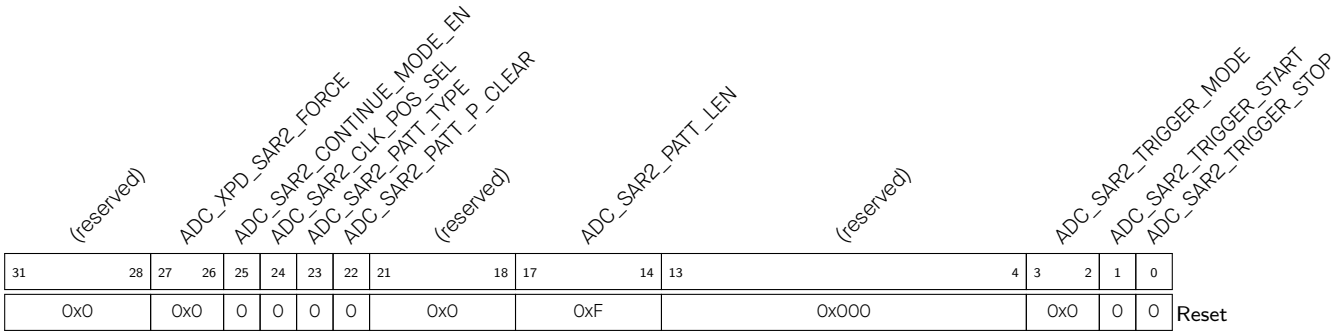
ADC_SAR1_TRIGGER_START 写 1 触发 SAR1 启动一次转换。

仅在 ADC_SAR1_TRIGGER_MODE 为 2 时有效。

(WT)

ADC_SAR1_TRIGGER_STOP 写 1 停止 SAR1 转换并使 FSM 返回 IDLE。(WT)

Register 28.2. ADC_CTRL1_REG (0x004)



ADC_XPD_SAR2_FORCE 配置 SAR2 电源强制控制。

0~1：自动控制（非 IDLE 时开启，IDLE 时关闭）

2：强制关闭

3：强制开启

(R/W)

ADC_SAR2_CONTINUE_MODE_EN 配置是否使能 SAR2 连续采样模式。

0：禁用

1：使能

(R/W)

ADC_SAR2_CLK_POS_SEL 配置 SAR2 采样数据时钟相位。

0：时钟下降沿

1：时钟上升沿

(R/W)

ADC_SAR2_PATT_TYPE 配置 SAR2 模式表指针更新时机。

0：测量完成时更新

1：测量开始时更新

(R/W)

ADC_SAR2_PATT_P_CLEAR 写 1 清零 SAR2 模式表指针。(WT)

ADC_SAR2_PATT_LEN 配置 SAR2 模式表长度。

0~15：对应长度 1~16

(R/W)

ADC_SAR2_TRIGGER_MODE 配置 SAR2 触发模式。

0：OFF

1：SCAN

2：SW

3：ETM

(R/W)

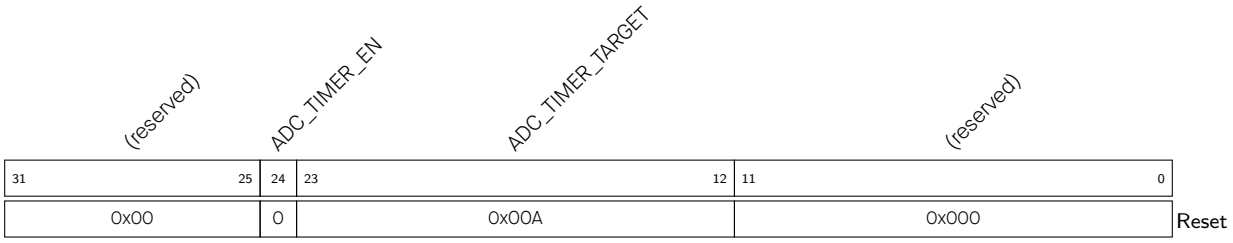
ADC_SAR2_TRIGGER_START 写 1 触发 SAR2 启动一次转换。

仅在 ADC_SAR2_TRIGGER_MODE 为 2 时有效。

(WT)

ADC_SAR2_TRIGGER_STOP 写 1 停止 SAR2 转换并使 FSM 返回 IDLE。(WT)

Register 28.3. ADC_CTRL2_REG (0x008)



ADC_TIMER_EN 配置是否使能定时器。

0：禁用

1：使能

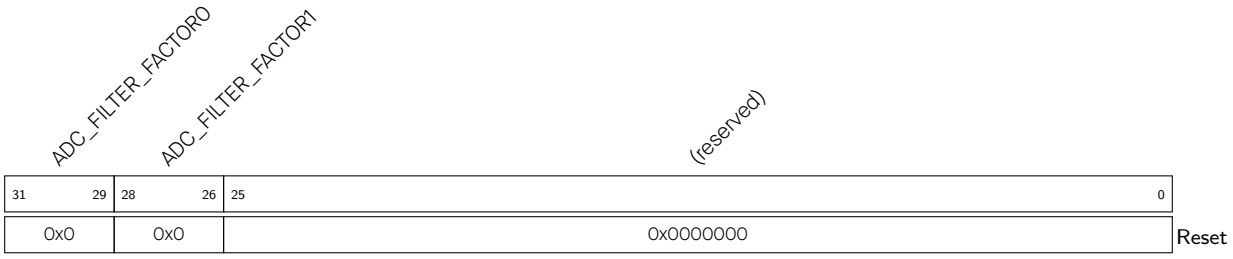
(R/W)

ADC_TIMER_TARGET 配置定时器产生一次采样触发脉冲的半周期计数值。

单位：ADC 工作时钟周期。

(R/W)

Register 28.4. ADC_FILTER_CTRL1_REG (0x00C)



ADC_FILTER_FACTOR0 配置滤波通道 0 的滤波系数。

0：k=1（无滤波）

1：k=2

2：k=4

3：k=8

4：k=16

5：k=32

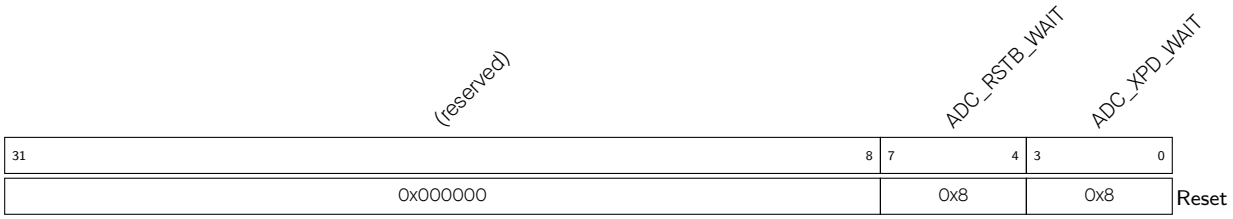
6：k=64

7：reserved

(R/W)

ADC_FILTER_FACTOR1 配置滤波通道 1 的滤波系数。配置同 ADC_FILTER_FACTOR0。(R/W)

Register 28.5. ADC_FSM_WAIT_REG (0x010)



ADC_RSTB_WAIT 配置 SAR 通道复位等待时间。

单位：完成的 SAR 转换次数（每次对应一个 T_{conv} = 24 个 ADC 工作时钟）。

配置为 0 时，该状态仅停留 1 个 ADC 工作时钟。

(R/W)

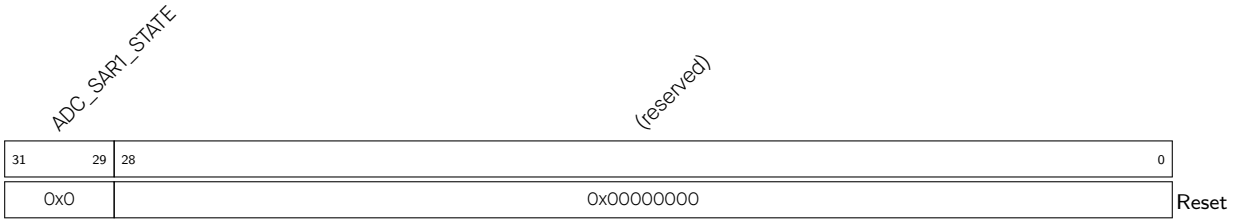
ADC_XPD_WAIT 配置 SAR 上电等待时间。

单位：完成的 SAR 转换次数（每次对应一个 T_{conv} = 24 个 ADC 工作时钟）。

配置为 0 时，该状态仅停留 1 个 ADC 工作时钟。

(R/W)

Register 28.6. ADC_SAR1_STATUS_REG (0x014)



ADC_SAR1_STATE 表示 SAR1 FSM 当前状态。

0：IDLE

1：XPD

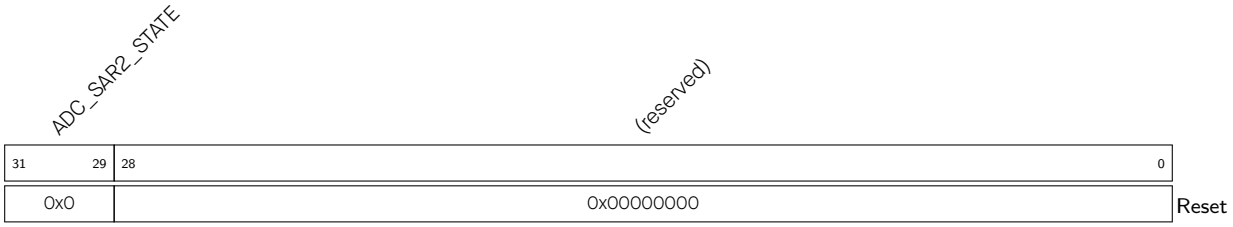
2：RSTB

3：READ

4-7：reserved

(RO)

Register 28.7. ADC_SAR2_STATUS_REG (0x018)



ADC_SAR2_STATE 表示 SAR2 FSM 当前状态。

0：IDLE

1：XPD

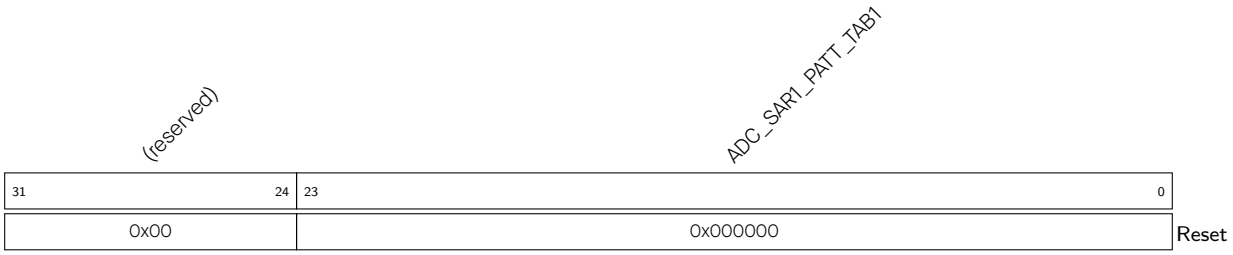
2：RSTB

3：READ

4-7：reserved

(RO)

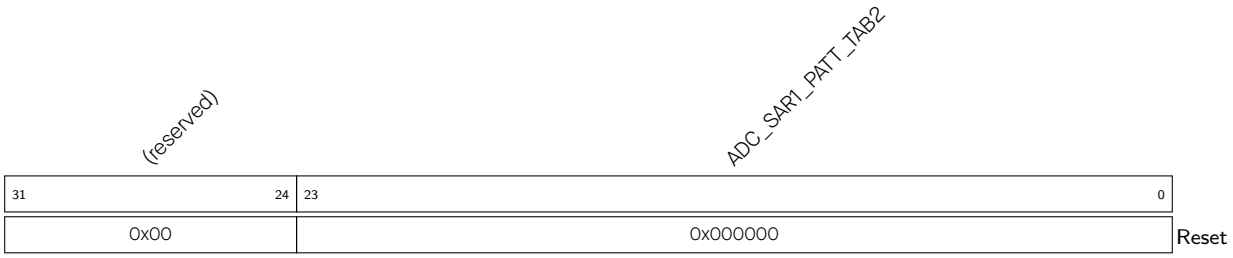
Register 28.8. ADC_SAR1_PATT_TAB1_REG (0x01C)



ADC_SAR1_PATT_TAB1 配置 SAR1 模式表样式 0~3。每个样式占 6 位。详见小节 28.4.2.1 模式表结构。

(R/W)

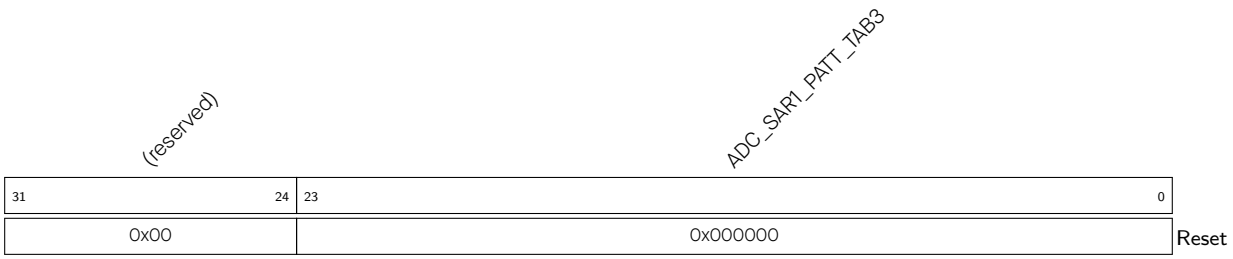
Register 28.9. ADC_SAR1_PATT_TAB2_REG (0x020)



ADC_SAR1_PATT_TAB2 配置 SAR1 模式表样式 4~7。详见小节 28.4.2.1 模式表结构。

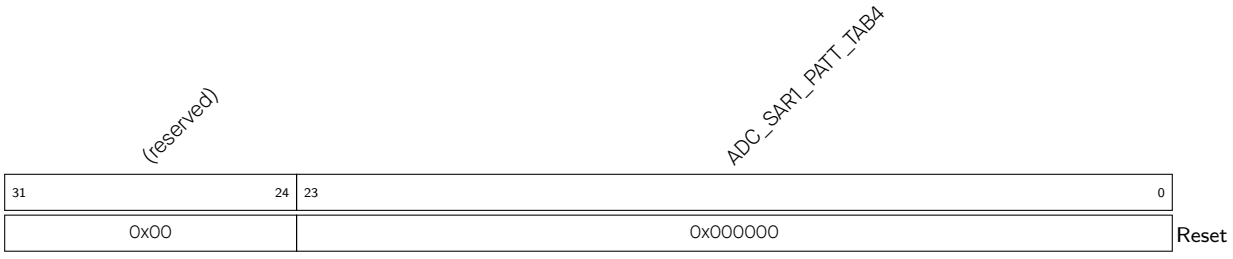
(R/W)

Register 28.10. ADC_SAR1_PATT_TAB3_REG (0x024)



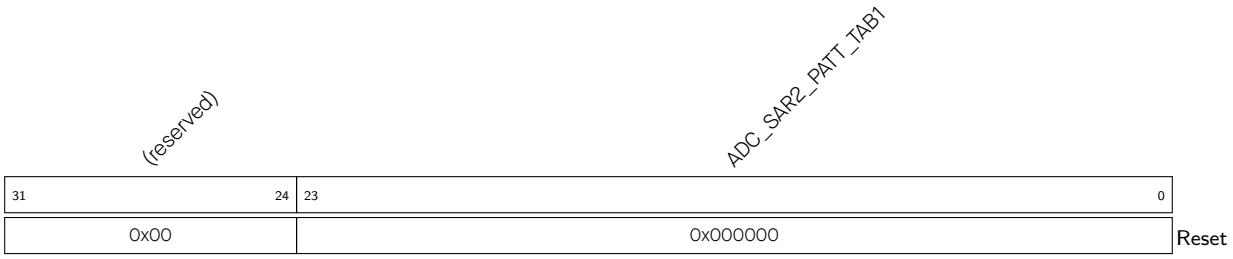
ADC_SAR1_PATT_TAB3 配置 SAR1 模式表样式 8~11。详见小节 28.4.2.1 模式表结构。 (R/W)

Register 28.11. ADC_SAR1_PATT_TAB4_REG (0x028)



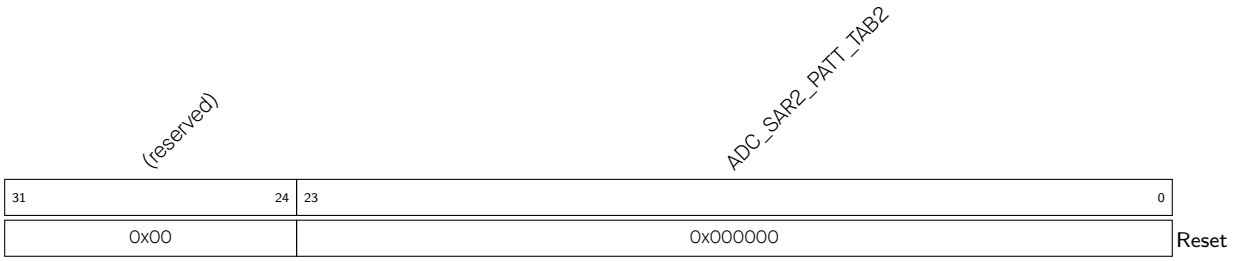
ADC_SAR1_PATT_TAB4 配置 SAR1 模式表样式 12~15。详见小节 28.4.2.1 模式表结构。 (R/W)

Register 28.12. ADC_SAR2_PATT_TAB1_REG (0x02C)



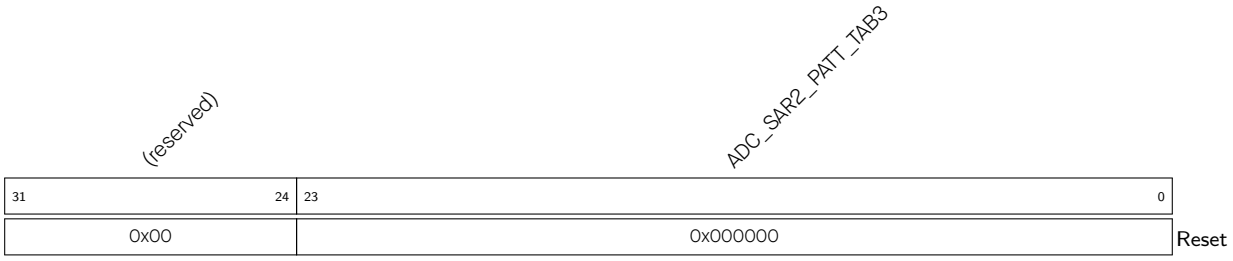
ADC_SAR2_PATT_TAB1 配置 SAR2 模式表样式 0~3。每个样式占 6 位。详见小节 28.4.2.1 模式表结构。 (R/W)

Register 28.13. ADC_SAR2_PATT_TAB2_REG (0x030)



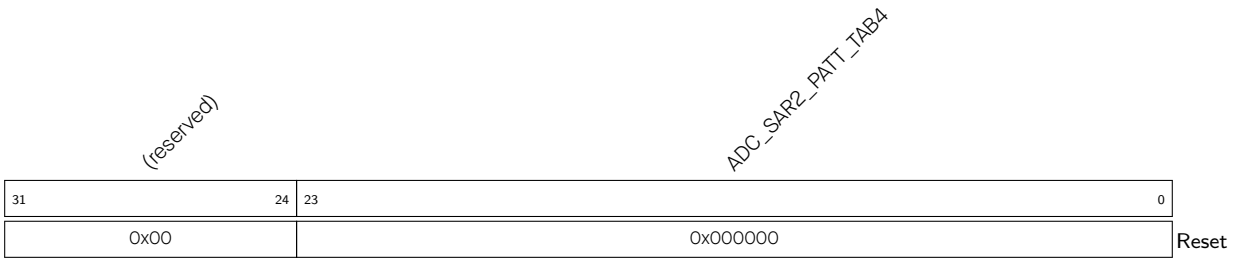
ADC_SAR2_PATT_TAB2 配置 SAR2 模式表样式 4~7。详见小节 28.4.2.1 模式表结构。 (R/W)

Register 28.14. ADC_SAR2_PATT_TAB3_REG (0x034)



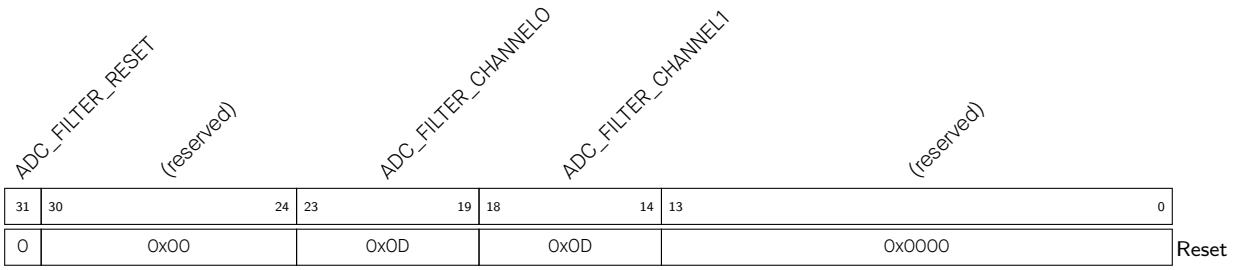
ADC_SAR2_PATT_TAB3 配置 SAR2 模式表样式 8~11。详见小节 28.4.2.1 模式表结构。 (R/W)

Register 28.15. ADC_SAR2_PATT_TAB4_REG (0x038)



ADC_SAR2_PATT_TAB4 配置 SAR2 模式表样式 12~15。详见小节 28.4.2.1 模式表结构。 (R/W)

Register 28.16. ADC_FILTER_CTRL0_REG (0x03C)



ADC_FILTER_RESET 写 1 清零两个滤波通道的内部状态。(WT)

ADC_FILTER_CHANNEL0 配置滤波通道 0 的通道号。

bit[4]：区分 SAR1/SAR2

0：SAR1

1：SAR2

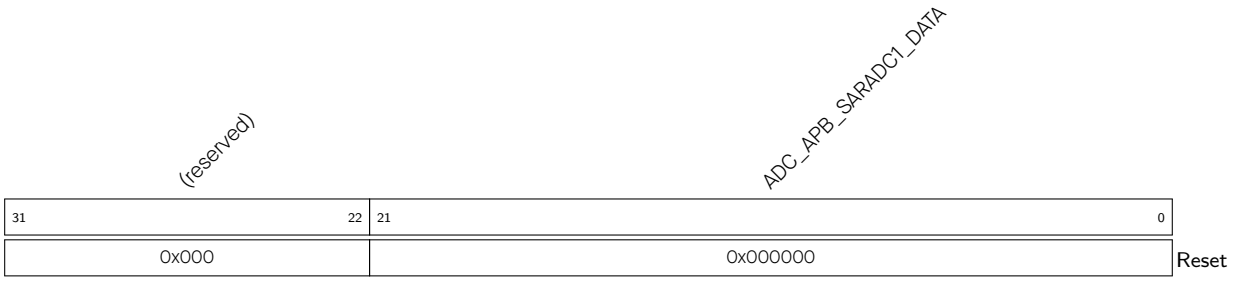
bit[3:0]：通道

(R/W)

ADC_FILTER_CHANNEL1 配置滤波通道 1 的通道号。配置同 ADC_FILTER_CHANNEL0 的 bit[4] 与 bit[3:0]。

(R/W)

Register 28.17. ADC_SAR1_DATA_STATUS_REG (0x040)



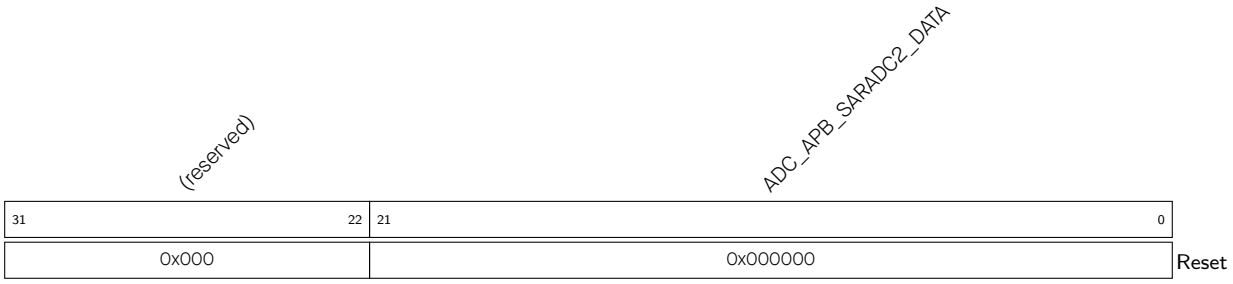
ADC_APB_SARADC1_DATA 表示 SAR1 最近一次转换数据。

bit[21:17]：通道号

bit[16:0]：17 bit ADC 数据

(RO)

Register 28.18. ADC_SAR2_DATA_STATUS_REG (0x044)



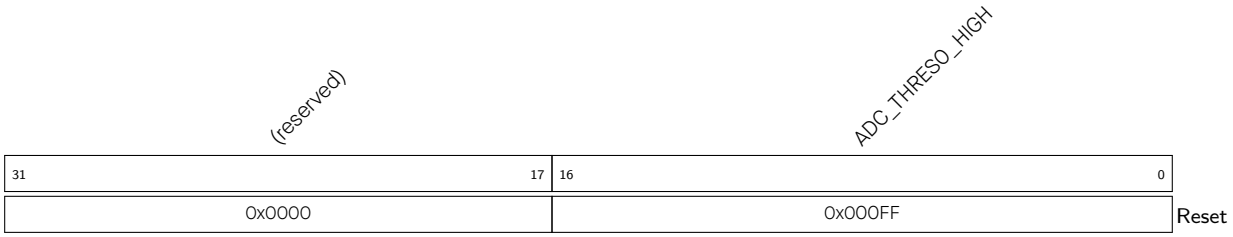
ADC_APB_SARADC2_DATA 表示 SAR2 最近一次转换数据。

bit[21:17]：通道号

bit[16:0]：17 bit ADC 数据

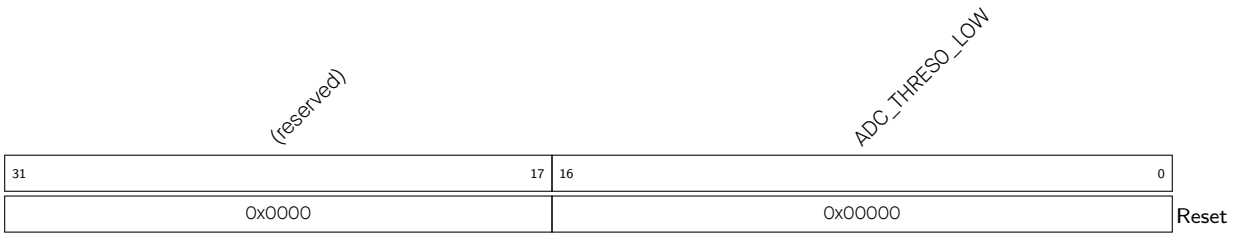
(RO)

Register 28.19. ADC_THRES0_HIGH_THRES_REG (0x048)



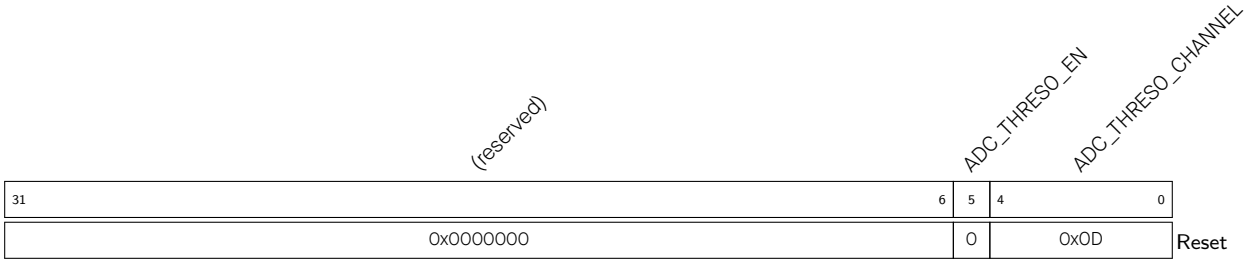
ADC_THRES0_HIGH 配置阈值 0 高阈值。(R/W)

Register 28.20. ADC_THRES0_LOW_THRES_REG (0x04C)



ADC_THRES0_LOW 配置阈值 0 低阈值。(R/W)

Register 28.21. ADC_THRES0_CTRL_REG (0x050)



ADC_THRES0_EN 配置是否使能阈值 0 比较器。

0：禁用

1：使能

(R/W)

ADC_THRES0_CHANNEL 配置阈值 0 监控的通道号。

bit[4]：区分 SAR1/SAR2

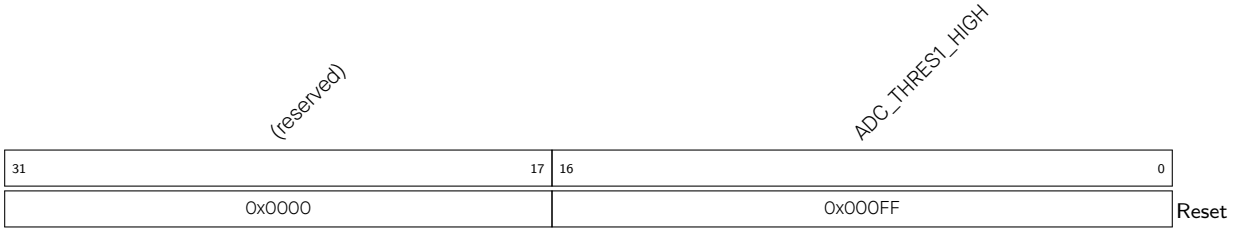
0：SAR1

1：SAR2

bit[3:0]：通道

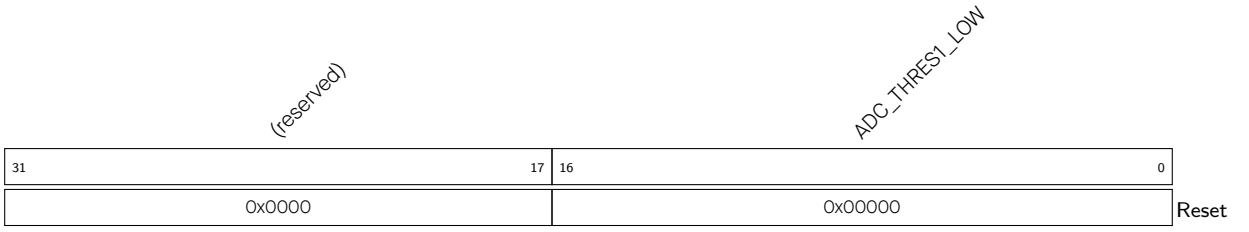
(R/W)

Register 28.22. ADC_THRES1_HIGH_THRES_REG (0x054)



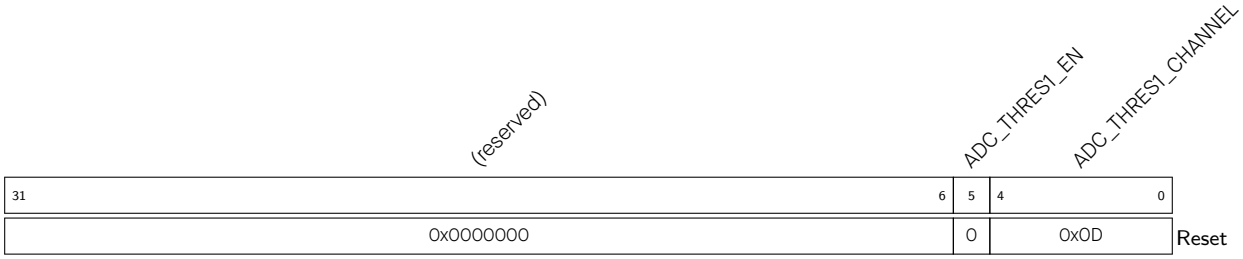
ADC_THRES1_HIGH 配置阈值 1 高阈值。(R/W)

Register 28.23. ADC_THRES1_LOW_THRES_REG (0x058)



ADC_THRES1_LOW 配置阈值 1 低阈值。(R/W)

Register 28.24. ADC_THRES1_CTRL_REG (0x05C)



ADC_THRES1_EN 配置是否使能阈值 1 比较器。

0：禁用

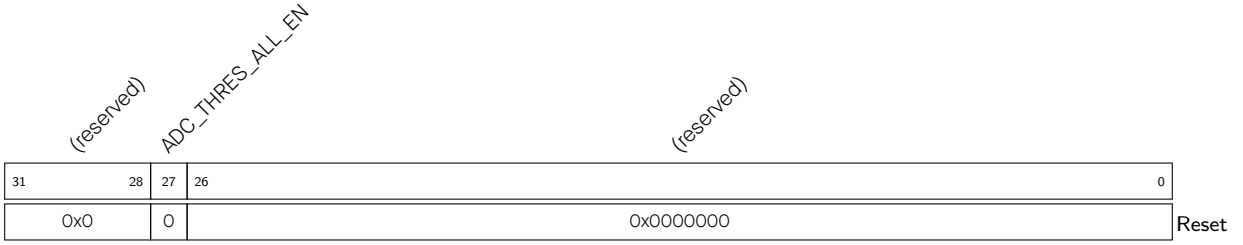
1：使能

(R/W)

ADC_THRES1_CHANNEL 配置阈值 1 监控的通道号。配置同 ADC_THRESO_CHANNEL 的 bit[4] 与 bit[3:0]。

(R/W)

Register 28.25. ADC_THRES_CTRL_REG (0x060)



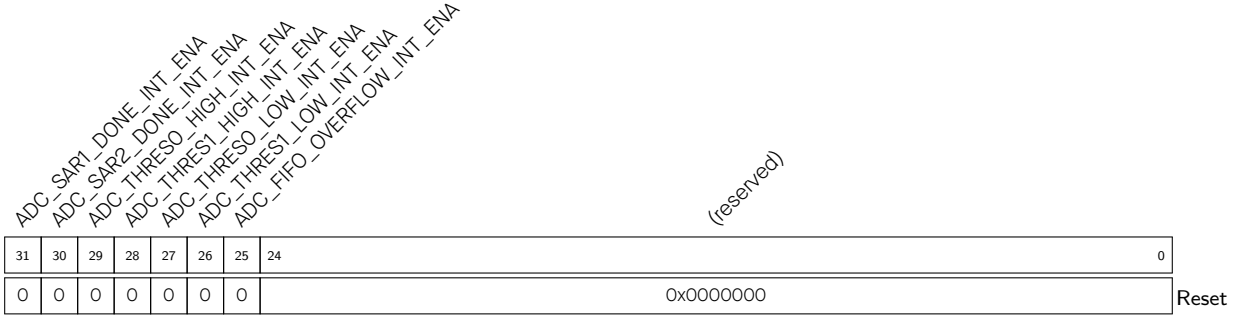
ADC_THRES_ALL_EN 配置是否使能阈值 0 全通道监控。

0：仅监控 ADC_THRESO_CHANNEL 指定的通道

1：监控所有通道（阈值 1 不受影响）

(R/W)

Register 28.26. ADC_INT_ENA_REG (0x064)



ADC_SAR1_DONE_INT_ENA 写 1 使能 ADC_SAR1_DONE_INT 中断。(R/W)

ADC_SAR2_DONE_INT_ENA 写 1 使能 ADC_SAR2_DONE_INT 中断。(R/W)

ADC_THRES0_HIGH_INT_ENA 写 1 使能 ADC_THRES0_HIGH_INT 中断。(R/W)

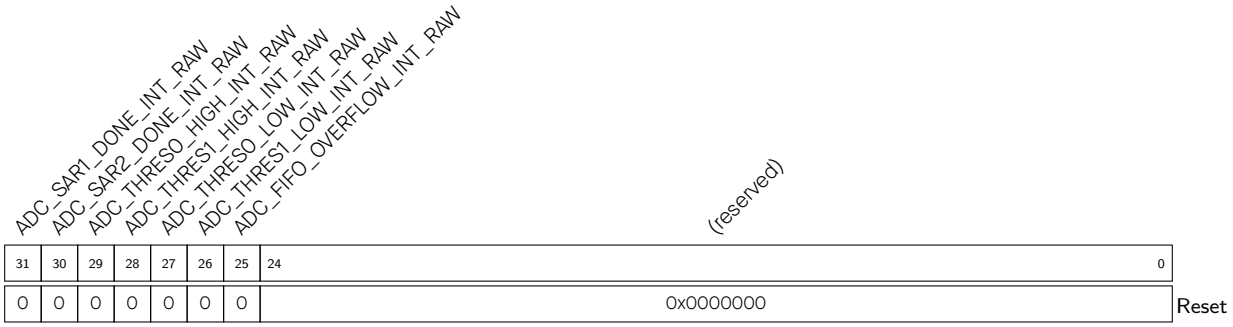
ADC_THRES1_HIGH_INT_ENA 写 1 使能 ADC_THRES1_HIGH_INT 中断。(R/W)

ADC_THRES0_LOW_INT_ENA 写 1 使能 ADC_THRES0_LOW_INT 中断。(R/W)

ADC_THRES1_LOW_INT_ENA 写 1 使能 ADC_THRES1_LOW_INT 中断。(R/W)

ADC_FIFO_OVERFLOW_INT_ENA 写 1 使能 ADC_FIFO_OVERFLOW_INT 中断。(R/W)

Register 28.27. ADC_INT_RAW_REG (0x068)



ADC_SAR1_DONE_INT_RAW ADC_SAR1_DONE_INT 的原始中断状态。(R/WTC/SS)

ADC_SAR2_DONE_INT_RAW ADC_SAR2_DONE_INT 的原始中断状态。(R/WTC/SS)

ADC_THRES0_HIGH_INT_RAW ADC_THRES0_HIGH_INT 的原始中断状态。(R/WTC/SS)

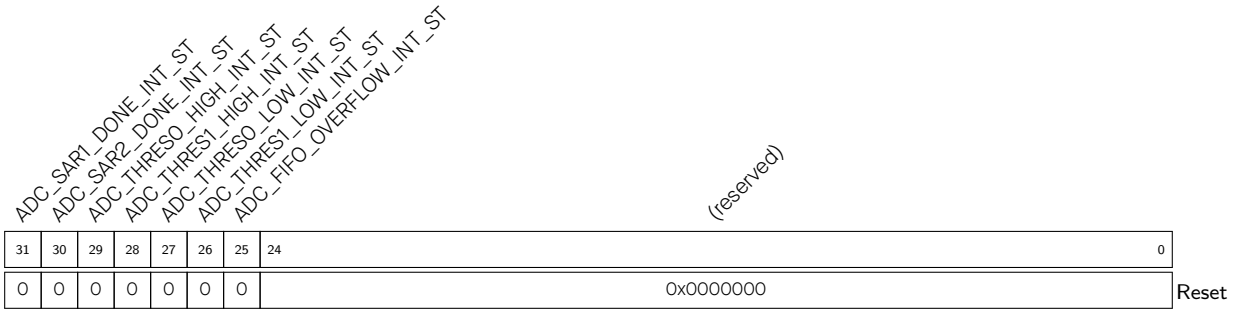
ADC_THRES1_HIGH_INT_RAW ADC_THRES1_HIGH_INT 的原始中断状态。(R/WTC/SS)

ADC_THRES0_LOW_INT_RAW ADC_THRES0_LOW_INT 的原始中断状态。(R/WTC/SS)

ADC_THRES1_LOW_INT_RAW ADC_THRES1_LOW_INT 的原始中断状态。(R/WTC/SS)

ADC_FIFO_OVERFLOW_INT_RAW ADC_FIFO_OVERFLOW_INT 的原始中断状态。(R/WTC/SS)

Register 28.28. ADC_INT_ST_REG (0x06C)



ADC_SAR1_DONE_INT_ST ADC_SAR1_DONE_INT 的屏蔽中断状态。(RO)

ADC_SAR2_DONE_INT_ST ADC_SAR2_DONE_INT 的屏蔽中断状态。(RO)

ADC_THRES0_HIGH_INT_ST ADC_THRES0_HIGH_INT 的屏蔽中断状态。(RO)

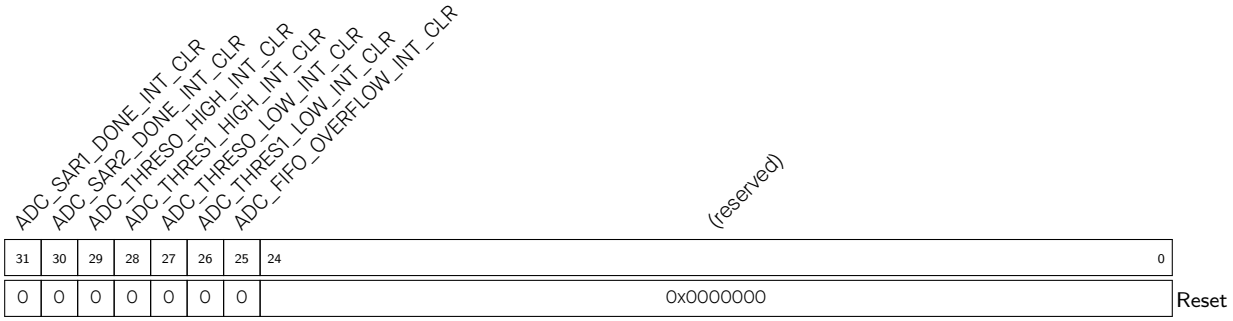
ADC_THRES1_HIGH_INT_ST ADC_THRES1_HIGH_INT 的屏蔽中断状态。(RO)

ADC_THRES0_LOW_INT_ST ADC_THRES0_LOW_INT 的屏蔽中断状态。(RO)

ADC_THRES1_LOW_INT_ST ADC_THRES1_LOW_INT 的屏蔽中断状态。(RO)

ADC_FIFO_OVERFLOW_INT_ST ADC_FIFO_OVERFLOW_INT 的屏蔽中断状态。(RO)

Register 28.29. ADC_INT_CLR_REG (0x070)



ADC_SAR1_DONE_INT_CLR 写 1 清除 ADC_SAR1_DONE_INT 中断。(WT)

ADC_SAR2_DONE_INT_CLR 写 1 清除 ADC_SAR2_DONE_INT 中断。(WT)

ADC_THRES0_HIGH_INT_CLR 写 1 清除 ADC_THRES0_HIGH_INT 中断。(WT)

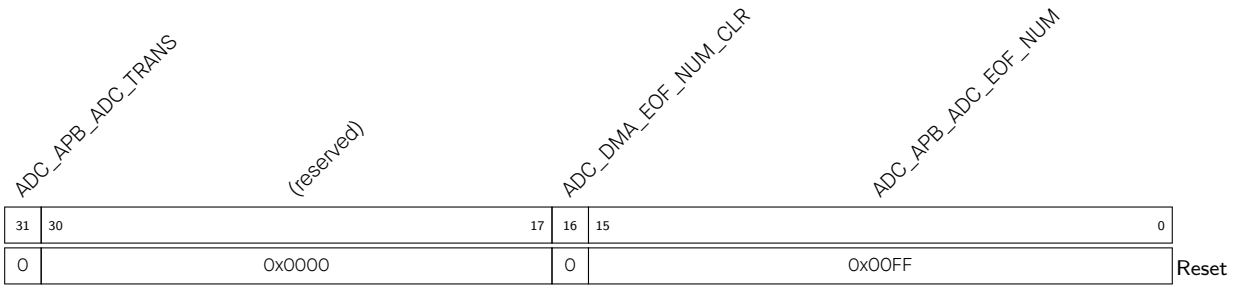
ADC_THRES1_HIGH_INT_CLR 写 1 清除 ADC_THRES1_HIGH_INT 中断。(WT)

ADC_THRES0_LOW_INT_CLR 写 1 清除 ADC_THRES0_LOW_INT 中断。(WT)

ADC_THRES1_LOW_INT_CLR 写 1 清除 ADC_THRES1_LOW_INT 中断。(WT)

ADC_FIFO_OVERFLOW_INT_CLR 写 1 清除 ADC_FIFO_OVERFLOW_INT 中断。(WT)

Register 28.30. ADC_DMA_CONF_REG (0x074)



ADC_APB_ADC_TRANS 配置是否使能 DMA 传输。

0：禁用

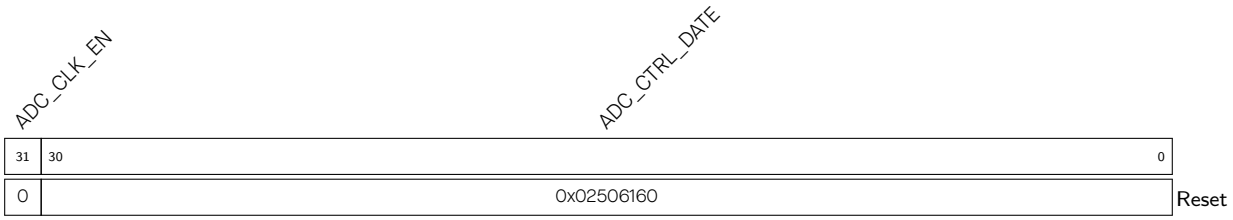
1：使能

(R/W)

ADC_DMA_EOF_NUM_CLR 写 1 清零 DMA EOF 计数器。(WT)

ADC_APB_ADC_EOF_NUM 配置 DMA 帧长度（样点数）。(R/W)

Register 28.31. ADC_CTRL_DATE_REG (0x3FC)



ADC_CLK_EN 配置寄存器时钟门控。

0：自动时钟门控

1：强制时钟常开

(R/W)

ADC_CTRL_DATE 版本控制寄存器。(R/W)

第 29 章

模拟电压比较器

29.1 概述

ESP32-S31 芯片集成了一个模拟电压比较器，通过比较输入电压与参考电压来监测电压变化。比较器提供多个芯片焊盘 (PAD) 用于接入电压信号，支持多通道比较。比较结果可生成 ETM（事件任务矩阵）事件或触发中断。

29.2 主要特性

模拟电压比较器具有以下主要特性：

- 提供 4 个芯片焊盘 (PAD) 接口，用于接入电压信号
- 支持内部或外部参考电压，内部参考电压支持迟滞功能
- 支持多通道电压比较：
  - 最多支持 3 个通道
  - 通过轮询扫描方式依次比较不同 PAD 的输入电压与参考电压
  - 扫描可由软件启动或由 ETM 任务触发
- 每个通道提供以下功能：
  - 毛刺过滤
  - 状态变化可产生 ETM 事件或触发中断
  - 通道计时，用于记录中断间隔

29.3 系统架构

图 29.3-1 展示了模拟电压比较器的系统架构。

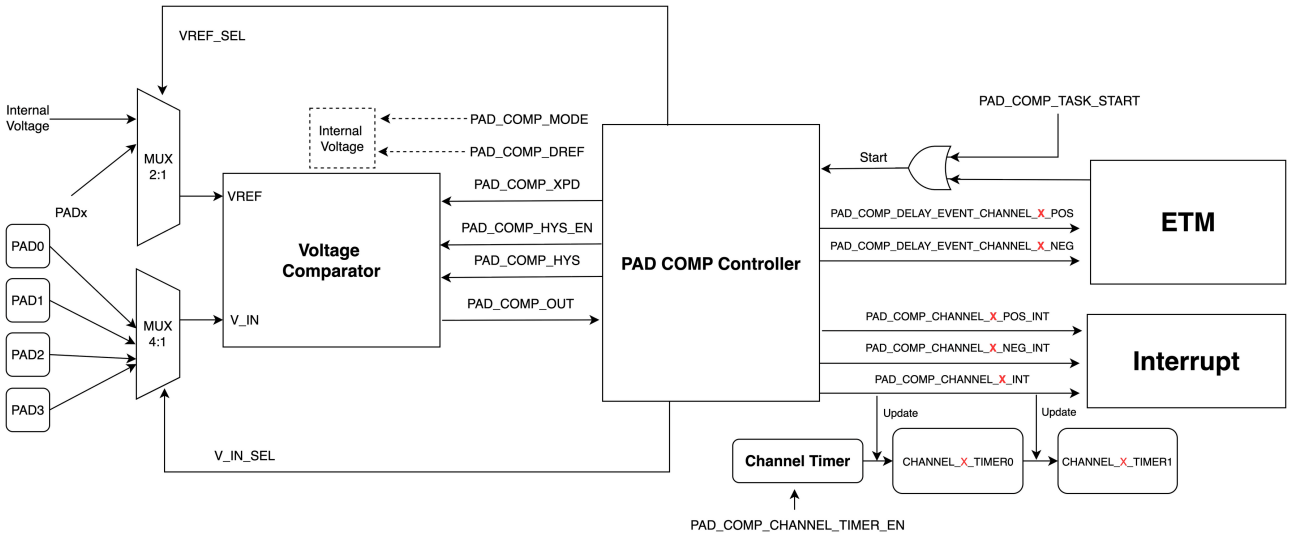


图 29.3-1. 模拟电压比较器系统框图

如上图所示，模拟电压比较器的结构如下：

- 四个 PAD 接口：PAD0~PAD3，用于接入电压信号，可作为通道输入或外部参考电压源。
- Voltage Comparator（电压比较器）：比较输入电压 (V_IN) 与参考电压 (VREF)，并通过 PAD_COMP_OUT 输出比较结果。参考电压可选内部产生或由外部 PAD 提供。
- PAD COMP Controller（模拟电压比较器控制器）：控制通道扫描，处理比较结果并生成中断及 ETM 事件。
- Channel Timer（通道计时器）：为各通道中断间隔提供计时参考。

表 29.3-1 列出了 PAD 与 GPIO 管脚之间的映射关系。

表 29.3-1. 模拟电压比较器 PAD 和 GPIO 之间的映射关系

	PAD0	PAD1	PAD2	PAD3
GPIO 管脚	GPIO37	GPIO38	GPIO39	GPIO40

29.4 功能描述

29.4.1 参考电压配置

模拟电压比较器的参考电压可选内部参考电压或者外部参考电压。根据参考电压来源不同，比较模式分为以下两种：

- **内部参考电压模式**：设置 PAD_COMP_MODE = 0，将输入电压与内部参考电压进行比较。

  在此模式下，4 个 PAD 均可分配给通道作为输入，但由于硬件仅支持 3 个通道，因此最多使用 3 个 PAD。

  内部参考电压由 PAD_COMP_DREF 配置：

  - 计算公式：VREF = PAD_COMP_DREF × 0.1 × VDDPST_3 (V)
  - 范围：0 ~ (0.7 × VDDPST_3) V
  - 步长：(0.1 × VDDPST_3) V

其中 VDDPST_3 为比较器 PAD 所在数字 HP IO 电源域的供电电压，由 VDDPST_3 管脚输入，通常为 3.3 V。内部参考电压支持迟滞功能，可提高比较器在临界区域的稳定性。当 PAD_COMP_HYS_EN = 1 时启用迟滞。PAD_COMP_HYS 配置迟滞电压 V_HYS。

当 PAD_COMP_OUT 为低电平时，内部参考电压为 VREF + V_HYS；当 PAD_COMP_OUT 为高电平时，内部参考电压为 VREF。表 29.4-1 列出了 V_HYS 与 PAD_COMP_HYS 的对应关系。

表 29.4-1. V_HYS 取值

PAD_COMP_HYS	V_HYS
0	0 V
1	0.04 V
2	0.08 V
4	0.12 V
3、5、6、7	保留

- **外部参考电压模式**：设置 PAD_COMP_MODE = 1，将输入电压与外部参考电压进行比较。

  在此模式下，需要从 4 个 PAD 中选择 1 个作为外部参考电压源，剩余 3 个 PAD 可分配给 3 个通道作为输入。

  通过 PAD_COMP_VREF_CHANNEL_SEL 选择外部参考电压源 PAD。该字段为 4-bit bitmap，由高到低依次对应 PAD3、PAD2、PAD1、PAD0，且仅允许一个 bit 为 1。

  > **说明：**
  > 用作外部参考电压源的 PAD 不应同时分配给任何通道作为输入 PAD。

29.4.2 控制器

控制器 (PAD COMP Controller) 支持轮询扫描，按通道编号顺序（1 → 2 → 3）依次扫描已启用的通道。

- **输入 PAD 分配**：通过 PAD_COMP_CHANNEL_X_SEL (X = 1, 2, 3) 为每个通道选择输入 PAD。

  该字段为 4-bit bitmap，由高到低依次对应 PAD3、PAD2、PAD1、PAD0，且仅允许一个 bit 为 1。

  > **说明：**
  > 各通道必须选择不同的 PAD。在外部参考电压模式下，用作外部参考电压源的 PAD 不可同时分配给任何通道。

- **启用通道**：通过 PAD_COMP_POLL_MASK 选择哪些通道参与扫描。

  该字段为 3-bit bitmap，由高到低依次对应通道 3、通道 2、通道 1。对应 bit 为 1 表示启用该通道，未启用的通道将在扫描过程中被跳过。

- **扫描触发与模式**：扫描可由软件通过 PAD_COMP_START 触发，或由 ETM 任务 PAD_COMP_TASK_START 触发。扫描行为由 PAD_COMP_POLL_MODE 控制：
  - 0：全扫描模式。每次触发时，依次扫描所有已启用通道。
  - 1：单步轮询模式。每次触发时，仅扫描下一个已启用通道。

- **通道切换等待时间**：由于 Voltage Comparator 为复用模块，切换输入通道时需要一定的稳定时间，由 PAD_COMP_POLL_PERIOD 控制，单位为 PAD_COMP_CLK 时钟周期，通常为 1 μs。
- **通道状态与毛刺过滤**：为避免输入通道处于参考电压临界区时产生毛刺，模块支持毛刺过滤。稳定时间结束后，根据通道状态 PAD_COMP_CHANNEL_X_STATUS 对 PAD_COMP_OUT 进行毛刺过滤。

  若当前状态为高电平，则需要连续采集到 PAD_COMP_LIMIT_CNT 个低电平，才会切换为低电平。反之，若当前状态为低电平，则需连续采集到 PAD_COMP_LIMIT_CNT 个高电平，通道状态才会切换为高电平。

  PAD_COMP_OUT 发生一次跳变并触发中断后，PAD_COMP_FILTER_CNT 配置一段以 PAD_COMP_CLK 时钟周期为单位的去抖时间，在此期间不再触发新的中断，从而抑制输入电压在参考电压附近抖动导致的重复中断。PAD_COMP_LIMIT_CNT 与 PAD_COMP_FILTER_CNT 配合使用：前者在采样阶段过滤毛刺，后者在中断触发后抑制重复中断。
- **通道中断**：中断由通道状态的变化触发：
  - 当 PAD_COMP_CHANNEL_X_STATUS 发生低到高的变化时，触发 PAD_COMP_CHANNEL_X_INT 和 PAD_COMP_CHANNEL_X_POS_INT
  - 当 PAD_COMP_CHANNEL_X_STATUS 发生高到低的变化时，触发 PAD_COMP_CHANNEL_X_INT 和 PAD_COMP_CHANNEL_X_NEG_INT
- **通道 ETM 事件**：当 PAD_COMP_CHANNEL_X_EVENT_TIMER_EN = 1 时，ETM 事件在中断触发后延迟 PAD_COMP_DELAY_EVENT_TIME 个 PAD_COMP_CLK 时钟周期产生：
  - PAD_COMP_DELAY_EVENT_CHANNEL_X_POS：对应 PAD_COMP_CHANNEL_X_POS_INT
  - PAD_COMP_DELAY_EVENT_CHANNEL_X_NEG：对应 PAD_COMP_CHANNEL_X_NEG_INT
- **通道计时器 (Channel Timer)**：当 PAD_COMP_CHANNEL_TIMER_EN = 1 时，通道计时器按照 PAD_COMP_CLK 持续计数。每当 PAD_COMP_CHANNEL_X_INT 产生时，当前计数值会保存到 PAD_COMP_CHANNEL_X_TIMER0，同时将之前的 TIMER0 值保存到 PAD_COMP_CHANNEL_X_TIMER1，从而记录两次中断的时间间隔。

图 29.4-1 展示了全扫描模式下的多通道扫描时序。本例配置为：PAD_COMP_POLL_MODE = 0，PAD_COMP_POLL_MASK = 3'b111（3 个通道全部启用），PAD_COMP_POLL_PERIOD = 4，PAD_COMP_LIMIT_CNT = 5，PAD_COMP_CHANNEL_1_SEL = 4'b0001，PAD_COMP_CHANNEL_2_SEL = 4'b0010，PAD_COMP_CHANNEL_3_SEL = 4'b0100。

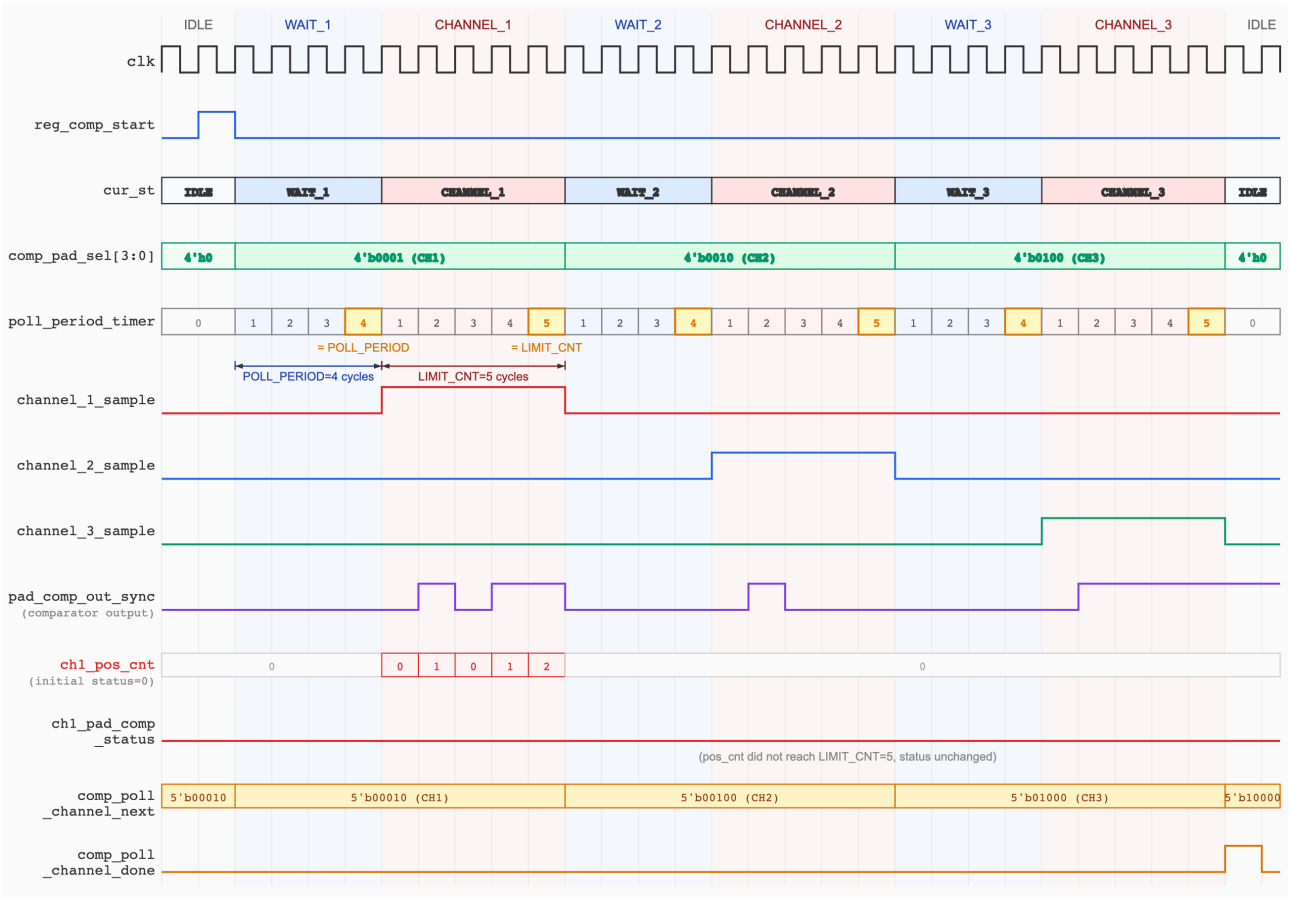


图 29.4-1. 模拟电压比较器全扫描模式时序图

- 图中蓝色底色区域为 WAIT_x 阶段：PAD 通道已切换，等待 PAD_COMP_POLL_PERIOD 个周期让比较器输出稳定。
- 红色底色区域为 CHANNEL_x 阶段：采样比较器输出，持续 PAD_COMP_LIMIT_CNT 个周期，正/负方向独立计数。
- 黄色高亮值表示定时器匹配值（触发状态转换）。

全扫描模式下，状态机依次遍历所有 PAD_COMP_POLL_MASK 使能的通道（WAIT → CHANNEL），全部完成后返回 IDLE。单步轮询模式（PAD_COMP_POLL_MODE = 1）下，每次触发仅完成一个通道后即返回 IDLE，下次触发继续轮询下一个使能通道。

29.4.3 时钟

模拟电压比较器有两种时钟：

- APB_CLK：用于寄存器访问的总线时钟，由 HP_SYS_CLKRST_PAD_COMP_APB_CLK_EN 使能
- PAD_COMP_CLK：用于模块功能运行的时钟，由 HP_SYS_CLKRST_PAD_COMP_CLK_EN 使能

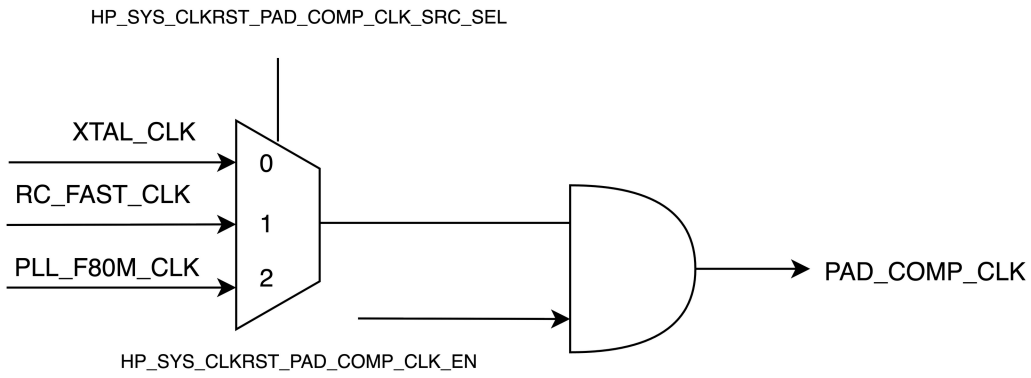


图 29.4-2. 模拟电压比较器时钟图

如图 29.4-2 模拟电压比较器时钟图 所示，PAD_COMP_CLK 的时钟源通过 HP_SYS_CLKRST_PAD_COMP_CLK_SRC_SEL 选择，可选时钟源包括：

- XTAL_CLK
- RC_FAST_CLK
- PLL_F80M_CLK

有关时钟的更多信息，请查看章节 7 复位和时钟 *[to be added later]*。

29.4.4 复位

模拟电压比较器有两种复位控制。向相应字段写 1 发起复位，写 0 解除复位：

- APB 复位：HP_SYS_CLKRST_PAD_COMP_APB_RST_EN
- 内核复位：HP_SYS_CLKRST_PAD_COMP_CORE_RST_EN

有关复位的更多信息，请查看章节 7 复位和时钟 *[to be added later]*。

29.5 事件任务矩阵

在 ESP32-S31 中，模拟电压比较器支持 ETM 功能。该模块可产生 ETM 事件，用于触发其他外设的 ETM 任务。关于 ETM 更多详细信息，请参考章节 10 事件任务矩阵 (ETM) [to be added later]。这里仅介绍与模拟电压比较器相关的 ETM 事件与任务。

模拟电压比较器可产生的 ETM 事件包括：

- PAD_COMP_DELAY_EVENT_CHANNEL_X_POS：通道 X 的 PAD_COMP_OUT（经毛刺过滤）由低电平变为高电平，即输入电压从低于参考电压变为高于参考电压。
- PAD_COMP_DELAY_EVENT_CHANNEL_X_NEG：通道 X 的 PAD_COMP_OUT（经毛刺过滤）由高电平变为低电平，即输入电压从高于参考电压变为低于参考电压。

模拟电压比较器可接收的 ETM 任务包括：

- PAD_COMP_TASK_START：触发通道扫描。根据 PAD_COMP_POLL_MODE 配置：当其为 0 时，每次触发对所有已启用通道完成一次扫描；当其为 1 时，每次触发仅对下一个已启用通道完成一次扫描。

29.6 中断

ESP32-S31 的模拟电压比较器可生成 PAD_COMP_INT 中断信号，并通过中断矩阵输出。

该中断由模拟电压比较器内部中断源产生。表 29.6-1 列出了内部中断源、触发条件以及对应的中断信号。

表 29.6-1. 模拟电压比较器的内部中断源

内部中断源	触发条件	中断信号
PAD_COMP_CHANNEL_X_INT	经过毛刺过滤的 PAD_COMP_OUT 信号发生跳变	PAD_COMP_INT
PAD_COMP_CHANNEL_X_NEG_INT	经过毛刺过滤的 PAD_COMP_OUT 信号由高电平变为低电平	PAD_COMP_INT
PAD_COMP_CHANNEL_X_POS_INT	经过毛刺过滤的 PAD_COMP_OUT 信号由低电平变为高电平	PAD_COMP_INT

说明：
有关“中断、中断源、中断信号”等术语的解释及其关系，请参考章节 9 中断矩阵 *[to be added later]* > 9.1 中断术语。

每个中断源都有一组通用的配置寄存器，详见章节 中断配置寄存器。具体寄存器名称请查看 29.8 寄存器列表。

29.7 配置流程

软件配置模拟电压比较器功能的流程如下：

1. 将 HP_SYS_CLKRST_PAD_COMP_APB_CLK_EN 设置为 1，使能总线时钟 APB_CLK。
2. 将 HP_SYS_CLKRST_PAD_COMP_CLK_EN 设置为 1，使能功能时钟 PAD_COMP_CLK。
3. 将 PAD_COMP_XPD 设置为 1，上电并启用电压比较器。
4. 配置参考电压：选择内部或外部模式，并设置相应参数（参考 29.4.1 参考电压配置）。
5. 如果选择外部参考电压模式，需要禁用用作外部参考电压源的 PAD 的数字功能，包括 IE、OE、WPU 和 WPD。具体配置方式请参考章节 6 *IO MUX 和 GPIO 交换矩阵 [to be added later]* 中的 34.1 模拟功能描述 *[to be added later]* 小节。
6. 配置控制器：分配通道输入 PAD、启用通道、设置扫描模式等（参考 29.4.2 控制器）。
7. 通过 PAD_COMP_START 或 PAD_COMP_TASK_START 触发扫描并获取比较结果。

29.8 寄存器列表

本小节的所有地址均为相对于模拟电压比较器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

请查看章节 *寄存器的访问类型*，了解“访问”列缩写的含义。

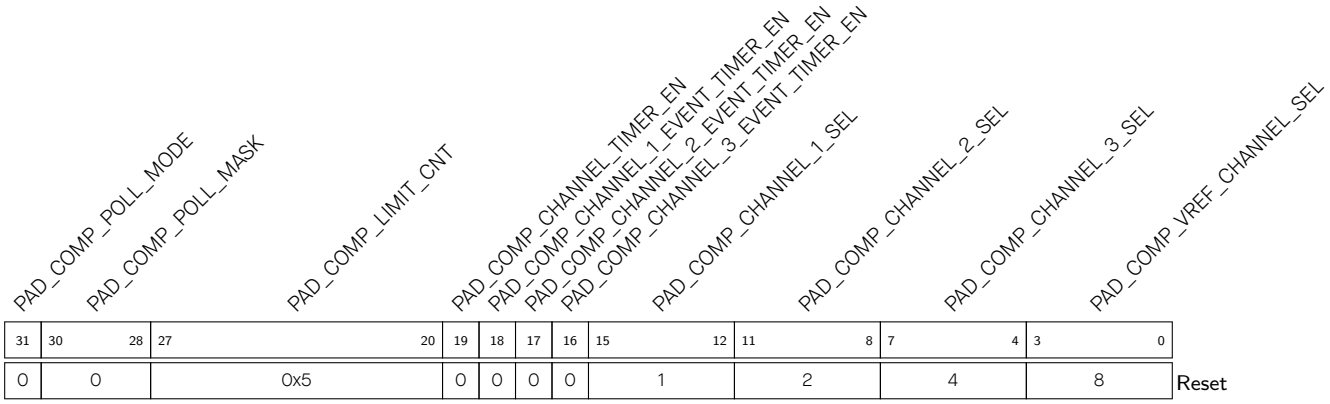
名称	描述	地址	访问权限
配置寄存器			
PAD_COMP_CONF_REG	通道及扫描配置寄存器	0x0000	R/W
PAD_COMP_FILTER_CNT_REG	去抖时间寄存器	0x0004	R/W
PAD_COMP_POLL_PERIOD_REG	通道切换周期寄存器	0x0008	R/W
PAD_COMP_DELAY_EVENT_TIME_REG	ETM 事件延迟时间寄存器	0x000C	R/W
PAD_COMP_INT_ENA_REG	中断使能寄存器	0x0010	R/W
PAD_COMP_INT_RAW_REG	原始中断状态寄存器	0x0014	R/WTC/SS
PAD_COMP_INT_CLR_REG	中断清除寄存器	0x0018	WT
PAD_COMP_INT_ST_REG	屏蔽中断状态寄存器	0x001C	RO
PAD_COMP_CHANNEL_1_TIMER0_REG	通道 1 计时器 0 寄存器	0x0020	RO
PAD_COMP_CHANNEL_1_TIMER1_REG	通道 1 计时器 1 寄存器	0x0024	RO
PAD_COMP_CHANNEL_2_TIMER0_REG	通道 2 计时器 0 寄存器	0x0028	RO
PAD_COMP_CHANNEL_2_TIMER1_REG	通道 2 计时器 1 寄存器	0x002C	RO
PAD_COMP_CHANNEL_3_TIMER0_REG	通道 3 计时器 0 寄存器	0x0030	RO
PAD_COMP_CHANNEL_3_TIMER1_REG	通道 3 计时器 1 寄存器	0x0034	RO
PAD_COMP_CHANNEL_STATUS_REG	通道状态寄存器	0x0038	RO
PAD_COMP_CFG_REG	比较器配置寄存器	0x003C	R/W
PAD_COMP_START_REG	模拟电压比较器启动配置寄存器	0x0040	WT
版本寄存器			
PAD_COMP_DATE_REG	版本寄存器	0x03FC	R/W

29.9 寄存器

本小节的所有地址均为相对于模拟电压比较器基地址的地址偏移量（相对地址），具体基地址请见章节 3 *系统和存储器 [to be added later]* 中的表 3。

关于保留 (reserved) 域的处理，请查看章节 *如何配置寄存器的保留域*。

Register 29.1. PAD_COMP_CONF_REG (0x0000)



PAD_COMP_VREF_CHANNEL_SEL 配置用作外部参考电压源的 PAD。

4-bit bitmap，由高到低依次对应 PAD3~PAD0，仅允许一个 bit 为 1。

仅在 PAD_COMP_MODE = 1 时有效。

(R/W)

PAD_COMP_CHANNEL_3_SEL 配置通道 3 的输入 PAD。

4-bit bitmap，由高到低依次对应 PAD3~PAD0，仅允许一个 bit 为 1。

注意：各通道必须选择不同的 PAD，且不可选择已分配给 PAD_COMP_VREF_CHANNEL_SEL 的 PAD。

(R/W)

PAD_COMP_CHANNEL_2_SEL 配置通道 2 的输入 PAD。

4-bit bitmap，由高到低依次对应 PAD3~PAD0，仅允许一个 bit 为 1。

注意：各通道必须选择不同的 PAD，且不可选择已分配给 PAD_COMP_VREF_CHANNEL_SEL 的 PAD。

(R/W)

PAD_COMP_CHANNEL_1_SEL 配置通道 1 的输入 PAD。

4-bit bitmap，由高到低依次对应 PAD3~PAD0，仅允许一个 bit 为 1。

注意：各通道必须选择不同的 PAD，且不可选择已分配给 PAD_COMP_VREF_CHANNEL_SEL 的 PAD。

(R/W)

见下页...

Register 29.1. PAD_COMP_CONF_REG (0x0000)

接上页...

PAD_COMP_CHANNEL_3_EVENT_TIMER_EN 配置是否使能通道 3 的 ETM 事件延迟功能。

0: 禁用

1: 使能

(R/W)

PAD_COMP_CHANNEL_2_EVENT_TIMER_EN 配置是否使能通道 2 的 ETM 事件延迟功能。

0: 禁用

1: 使能

(R/W)

PAD_COMP_CHANNEL_1_EVENT_TIMER_EN 配置是否使能通道 1 的 ETM 事件延迟功能。

0: 禁用

1: 使能

(R/W)

PAD_COMP_CHANNEL_TIMER_EN 配置是否使能通道计时器。

0: 禁用

1: 使能

(R/W)

PAD_COMP_LIMIT_CNT 配置毛刺过滤所需的连续采样次数。(R/W)

PAD_COMP_POLL_MASK 配置参与扫描的通道。

3-bit bitmap，由高到低依次对应通道 3~1。对应 bit 为 1 表示启用该通道。

(R/W)

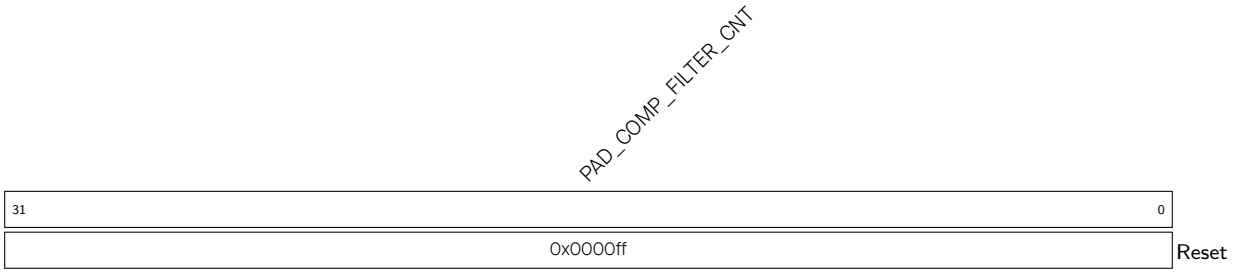
PAD_COMP_POLL_MODE 配置通道扫描模式。

0: 全扫描模式

1: 单步轮询模式

(R/W)

Register 29.2. PAD_COMP_FILTER_CNT_REG (0x0004)



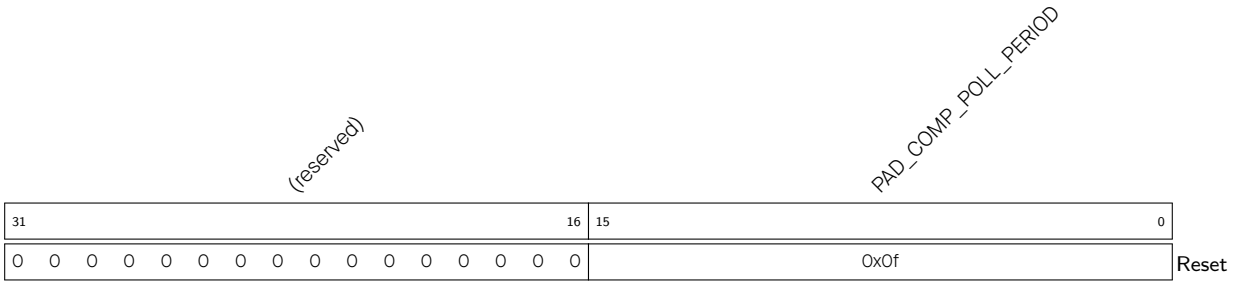
PAD_COMP_FILTER_CNT 配置中断触发后的去抖时间。在此期间不再触发新的中断。

单位：PAD_COMP_CLK 时钟周期。

该字段必须大于或等于 1。

(R/W)

Register 29.3. PAD_COMP_POLL_PERIOD_REG (0x0008)

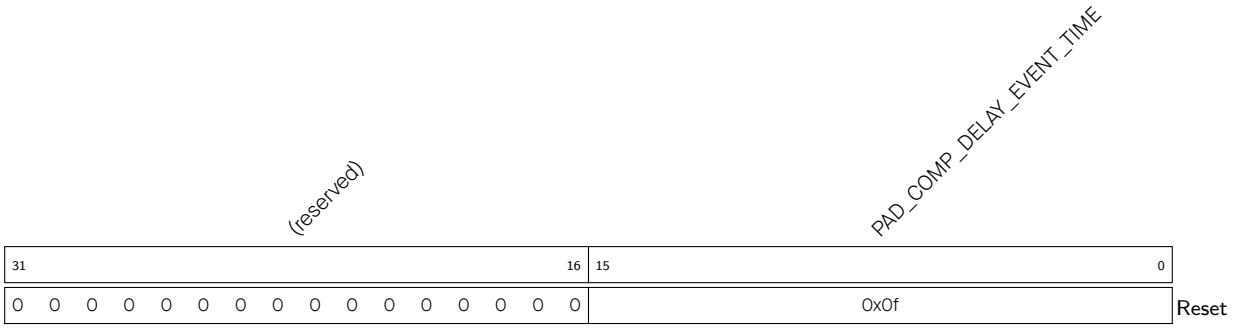


PAD_COMP_POLL_PERIOD 配置通道切换稳定时间。

单位：PAD_COMP_CLK 时钟周期。

(R/W)

Register 29.4. PAD_COMP_DELAY_EVENT_TIME_REG (0x000C)



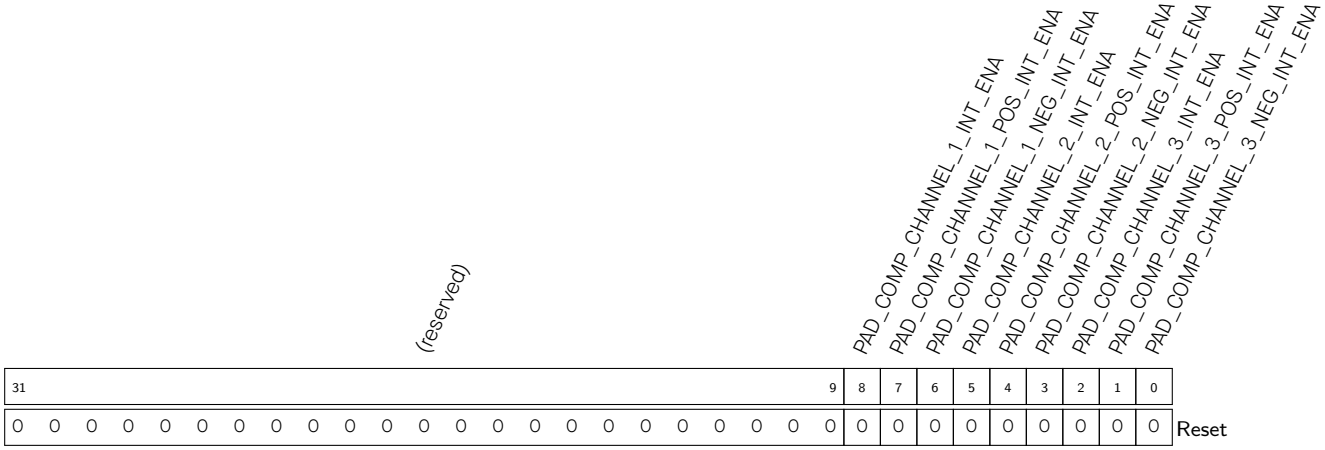
PAD_COMP_DELAY_EVENT_TIME 配置 ETM 事件延迟时间。

单位：PAD_COMP_CLK 时钟周期。

仅在 PAD_COMP_CHANNEL_X_EVENT_TIMER_EN = 1 时有效。

(R/W)

Register 29.5. PAD_COMP_INT_ENA_REG (0x0010)



PAD_COMP_CHANNEL_3_NEG_INT_ENA 写 1 使能 PAD_COMP_CHANNEL_3_NEG_INT。(R/W)

PAD_COMP_CHANNEL_3_POS_INT_ENA 写 1 使能 PAD_COMP_CHANNEL_3_POS_INT。(R/W)

PAD_COMP_CHANNEL_3_INT_ENA 写 1 使能 PAD_COMP_CHANNEL_3_INT。(R/W)

PAD_COMP_CHANNEL_2_NEG_INT_ENA 写 1 使能 PAD_COMP_CHANNEL_2_NEG_INT。(R/W)

PAD_COMP_CHANNEL_2_POS_INT_ENA 写 1 使能 PAD_COMP_CHANNEL_2_POS_INT。(R/W)

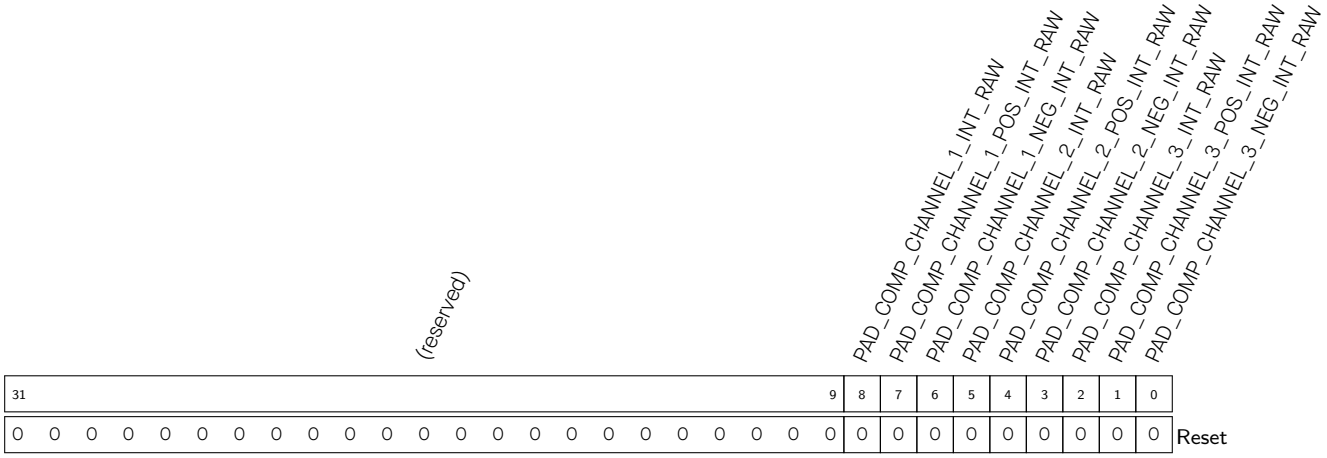
PAD_COMP_CHANNEL_2_INT_ENA 写 1 使能 PAD_COMP_CHANNEL_2_INT。(R/W)

PAD_COMP_CHANNEL_1_NEG_INT_ENA 写 1 使能 PAD_COMP_CHANNEL_1_NEG_INT。(R/W)

PAD_COMP_CHANNEL_1_POS_INT_ENA 写 1 使能 PAD_COMP_CHANNEL_1_POS_INT。(R/W)

PAD_COMP_CHANNEL_1_INT_ENA 写 1 使能 PAD_COMP_CHANNEL_1_INT。(R/W)

Register 29.6. PAD_COMP_INT_RAW_REG (0x0014)



PAD_COMP_CHANNEL_3_NEG_INT_RAW PAD_COMP_CHANNEL_3_NEG_INT 的原始中断状态。(R/WTC/SS)

PAD_COMP_CHANNEL_3_POS_INT_RAW PAD_COMP_CHANNEL_3_POS_INT 的原始中断状态。(R/WTC/SS)

PAD_COMP_CHANNEL_3_INT_RAW PAD_COMP_CHANNEL_3_INT 的原始中断状态。(R/WTC/SS)

PAD_COMP_CHANNEL_2_NEG_INT_RAW PAD_COMP_CHANNEL_2_NEG_INT 的原始中断状态。(R/WTC/SS)

PAD_COMP_CHANNEL_2_POS_INT_RAW PAD_COMP_CHANNEL_2_POS_INT 的原始中断状态。(R/WTC/SS)

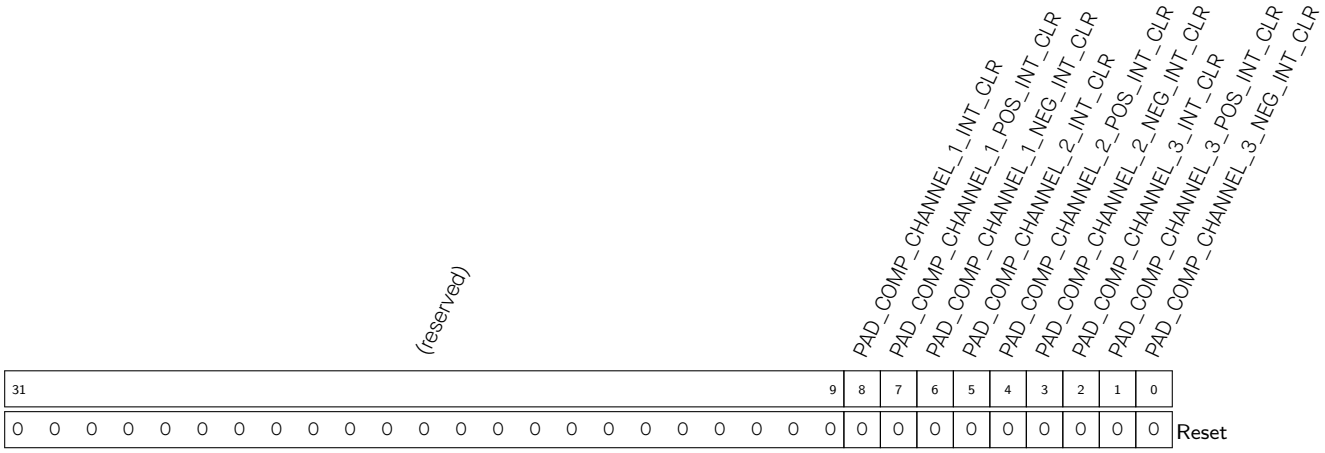
PAD_COMP_CHANNEL_2_INT_RAW PAD_COMP_CHANNEL_2_INT 的原始中断状态。(R/WTC/SS)

PAD_COMP_CHANNEL_1_NEG_INT_RAW PAD_COMP_CHANNEL_1_NEG_INT 的原始中断状态。(R/WTC/SS)

PAD_COMP_CHANNEL_1_POS_INT_RAW PAD_COMP_CHANNEL_1_POS_INT 的原始中断状态。(R/WTC/SS)

PAD_COMP_CHANNEL_1_INT_RAW PAD_COMP_CHANNEL_1_INT 的原始中断状态。(R/WTC/SS)

Register 29.7. PAD_COMP_INT_CLR_REG (0x0018)



PAD_COMP_CHANNEL_3_NEG_INT_CLR 写 1 清除 PAD_COMP_CHANNEL_3_NEG_INT。(WT)

PAD_COMP_CHANNEL_3_POS_INT_CLR 写 1 清除 PAD_COMP_CHANNEL_3_POS_INT。(WT)

PAD_COMP_CHANNEL_3_INT_CLR 写 1 清除 PAD_COMP_CHANNEL_3_INT。(WT)

PAD_COMP_CHANNEL_2_NEG_INT_CLR 写 1 清除 PAD_COMP_CHANNEL_2_NEG_INT。(WT)

PAD_COMP_CHANNEL_2_POS_INT_CLR 写 1 清除 PAD_COMP_CHANNEL_2_POS_INT。(WT)

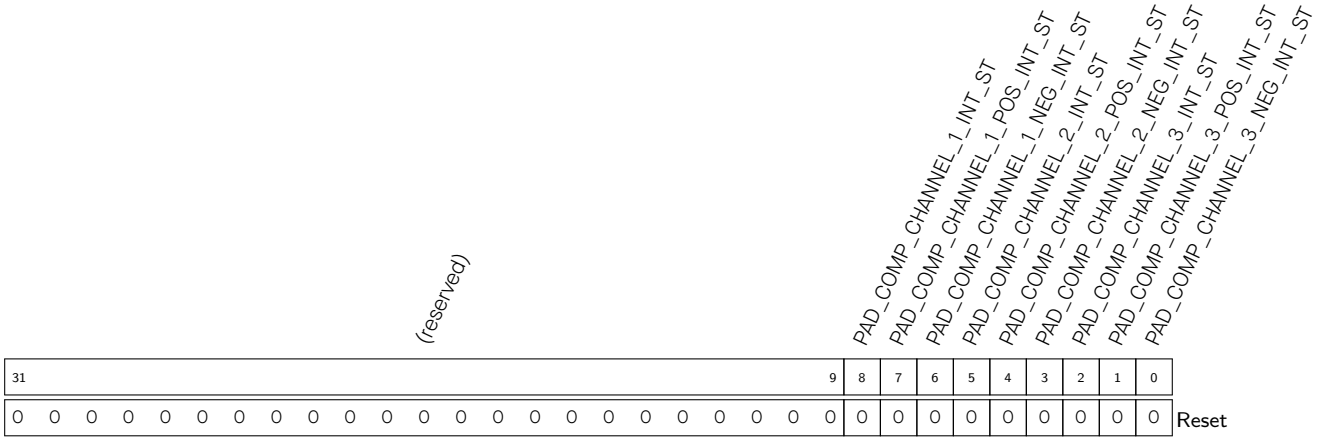
PAD_COMP_CHANNEL_2_INT_CLR 写 1 清除 PAD_COMP_CHANNEL_2_INT。(WT)

PAD_COMP_CHANNEL_1_NEG_INT_CLR 写 1 清除 PAD_COMP_CHANNEL_1_NEG_INT。(WT)

PAD_COMP_CHANNEL_1_POS_INT_CLR 写 1 清除 PAD_COMP_CHANNEL_1_POS_INT。(WT)

PAD_COMP_CHANNEL_1_INT_CLR 写 1 清除 PAD_COMP_CHANNEL_1_INT。(WT)

Register 29.8. PAD_COMP_INT_ST_REG (0x001C)



PAD_COMP_CHANNEL_3_NEG_INT_ST PAD_COMP_CHANNEL_3_NEG_INT 的屏蔽中断状态。(RO)

PAD_COMP_CHANNEL_3_POS_INT_ST PAD_COMP_CHANNEL_3_POS_INT 的屏蔽中断状态。(RO)

PAD_COMP_CHANNEL_3_INT_ST PAD_COMP_CHANNEL_3_INT 的屏蔽中断状态。(RO)

PAD_COMP_CHANNEL_2_NEG_INT_ST PAD_COMP_CHANNEL_2_NEG_INT 的屏蔽中断状态。(RO)

PAD_COMP_CHANNEL_2_POS_INT_ST PAD_COMP_CHANNEL_2_POS_INT 的屏蔽中断状态。(RO)

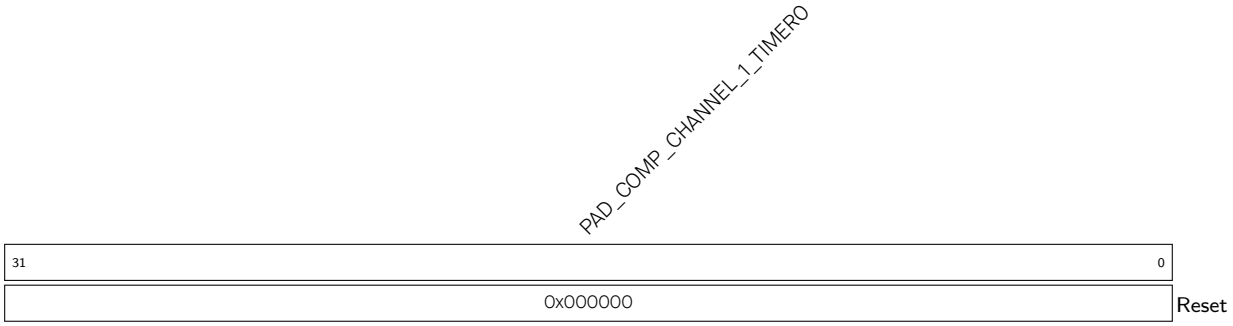
PAD_COMP_CHANNEL_2_INT_ST PAD_COMP_CHANNEL_2_INT 的屏蔽中断状态。(RO)

PAD_COMP_CHANNEL_1_NEG_INT_ST PAD_COMP_CHANNEL_1_NEG_INT 的屏蔽中断状态。(RO)

PAD_COMP_CHANNEL_1_POS_INT_ST PAD_COMP_CHANNEL_1_POS_INT 的屏蔽中断状态。(RO)

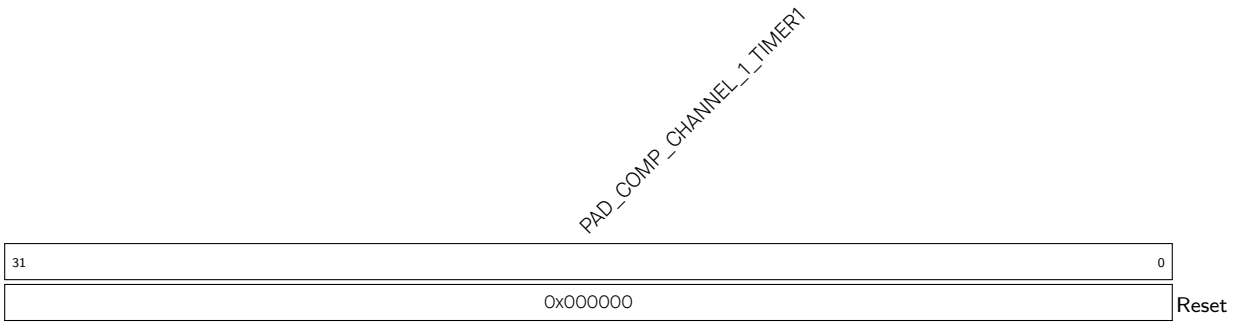
PAD_COMP_CHANNEL_1_INT_ST PAD_COMP_CHANNEL_1_INT 的屏蔽中断状态。(RO)

Register 29.9. PAD_COMP_CHANNEL_1_TIMER0_REG (0x0020)



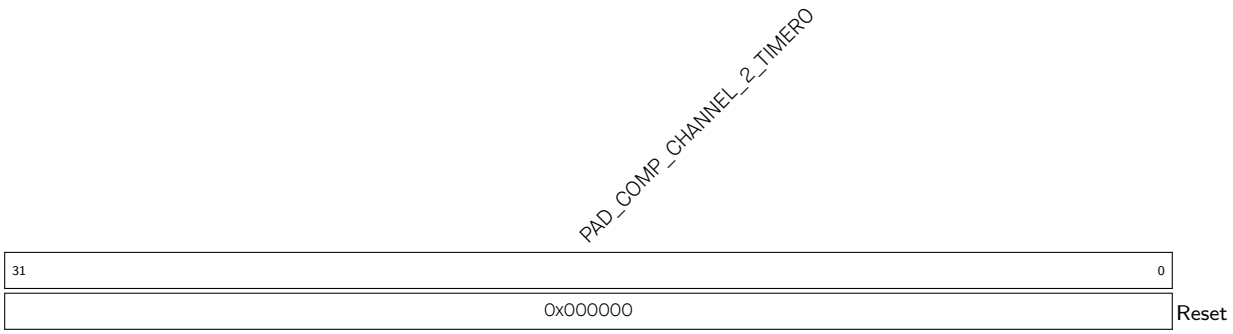
PAD_COMP_CHANNEL_1_TIMER0 表示通道 1 当前中断时的计时器值。(RO)

Register 29.10. PAD_COMP_CHANNEL_1_TIMER1_REG (0x0024)



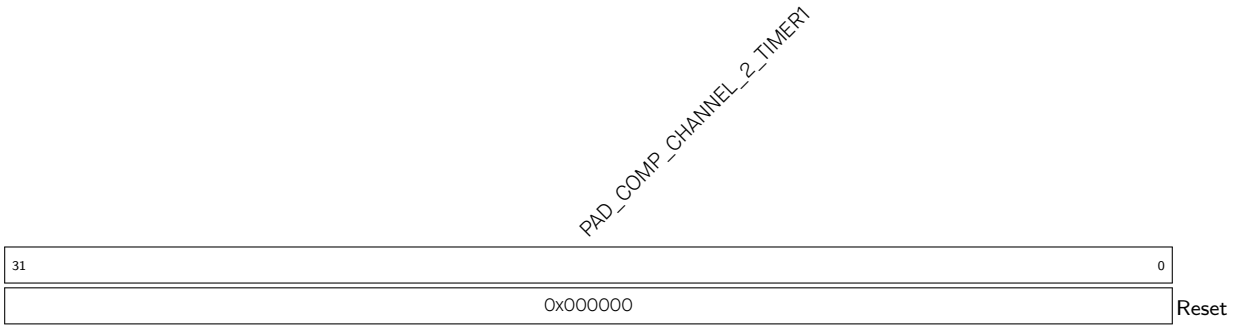
PAD_COMP_CHANNEL_1_TIMER1 表示通道 1 上一次中断时的计时器值。(RO)

Register 29.11. PAD_COMP_CHANNEL_2_TIMER0_REG (0x0028)



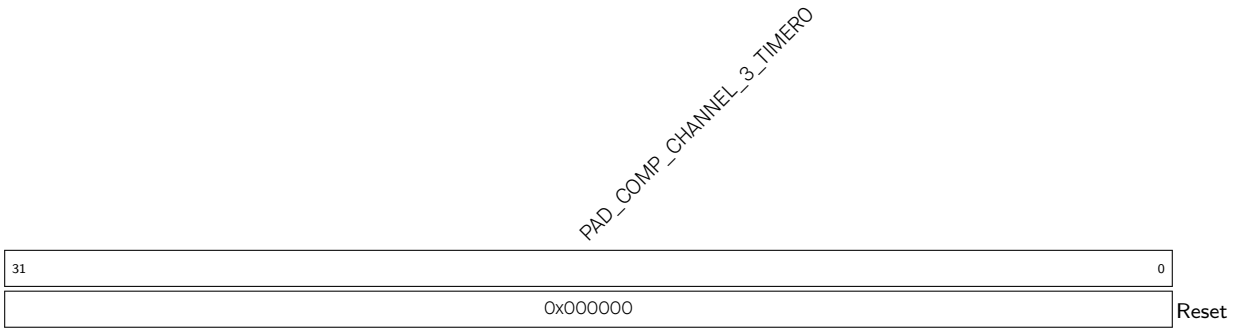
PAD_COMP_CHANNEL_2_TIMER0 表示通道 2 当前中断时的计时器值。(RO)

Register 29.12. PAD_COMP_CHANNEL_2_TIMER1_REG (0x002C)



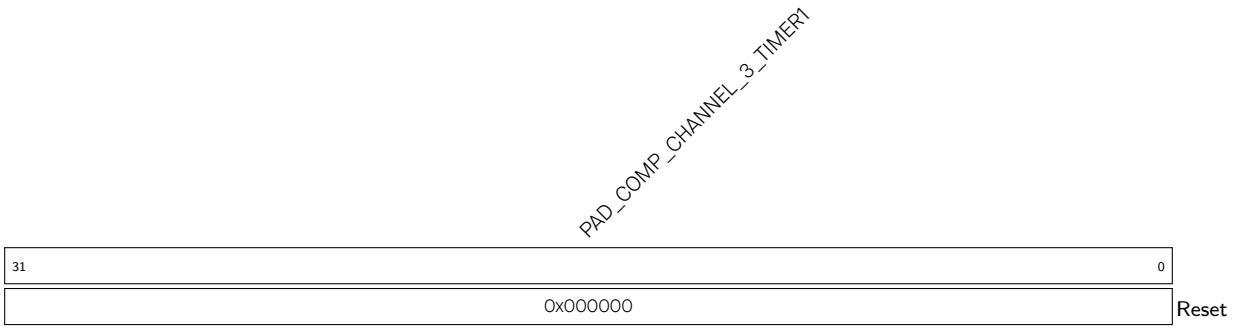
PAD_COMP_CHANNEL_2_TIMER1 表示通道 2 上一次中断时的计时器值。(RO)

Register 29.13. PAD_COMP_CHANNEL_3_TIMER0_REG (0x0030)



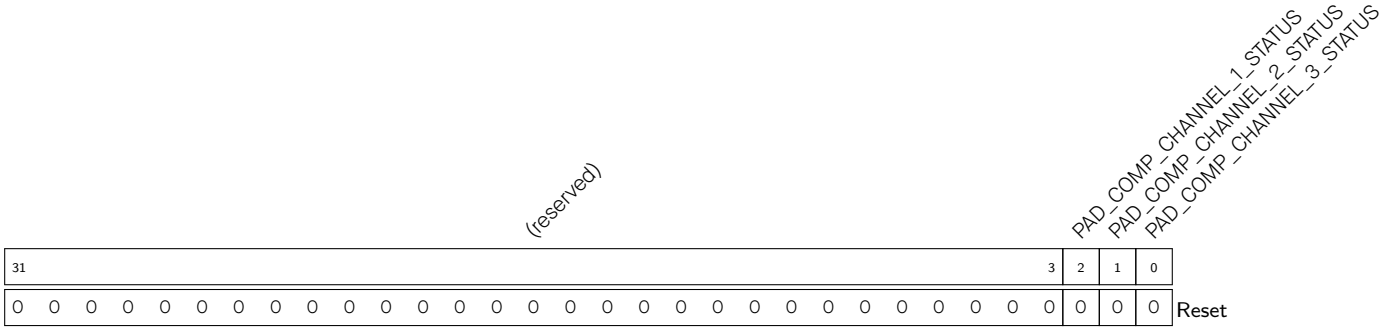
PAD_COMP_CHANNEL_3_TIMER0 表示通道 3 当前中断时的计时器值。(RO)

Register 29.14. PAD_COMP_CHANNEL_3_TIMER1_REG (0x0034)



PAD_COMP_CHANNEL_3_TIMER1 表示通道 3 上一次中断时的计时器值。(RO)

Register 29.15. PAD_COMP_CHANNEL_STATUS_REG (0x0038)



PAD_COMP_CHANNEL_3_STATUS 表示通道 3 的比较器输出状态。

0: 输入电压 < 参考电压

1: 输入电压 > 参考电压

(RO)

PAD_COMP_CHANNEL_2_STATUS 表示通道 2 的比较器输出状态。

0: 输入电压 < 参考电压

1: 输入电压 > 参考电压

(RO)

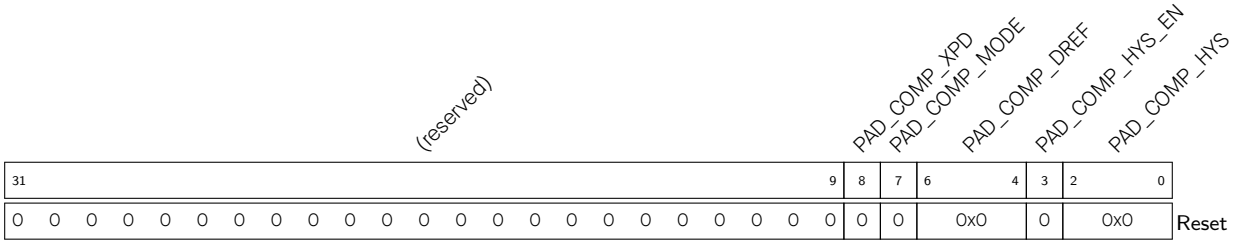
PAD_COMP_CHANNEL_1_STATUS 表示通道 1 的比较器输出状态。

0: 输入电压 < 参考电压

1: 输入电压 > 参考电压

(RO)

Register 29.16. PAD_COMP_CFG_REG (0x003C)



PAD_COMP_HYS 配置迟滞电压。

0: 0 V

1: 0.04 V

2: 0.08 V

4: 0.12 V

3、5、6、7: 保留

(R/W)

PAD_COMP_HYS_EN 配置是否使能迟滞功能。

0: 禁用

1: 使能

仅在 PAD_COMP_MODE = 0 时有效。

(R/W)

PAD_COMP_DREF 配置内部参考电压。

内部参考电压 VREF = PAD_COMP_DREF × 0.1 × VDDPST_3 (V)。

仅在 PAD_COMP_MODE = 0 时有效。

(R/W)

PAD_COMP_MODE 配置比较器模式。

0: 内部参考电压模式

1: 外部参考电压模式

(R/W)

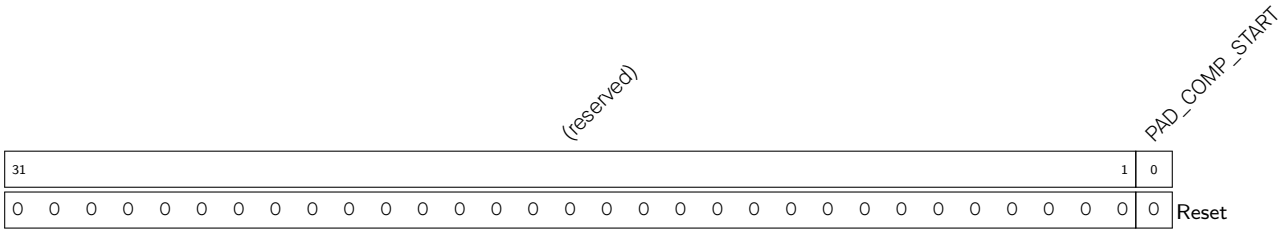
PAD_COMP_XPD 配置比较器电源。

0: 断电

1: 上电

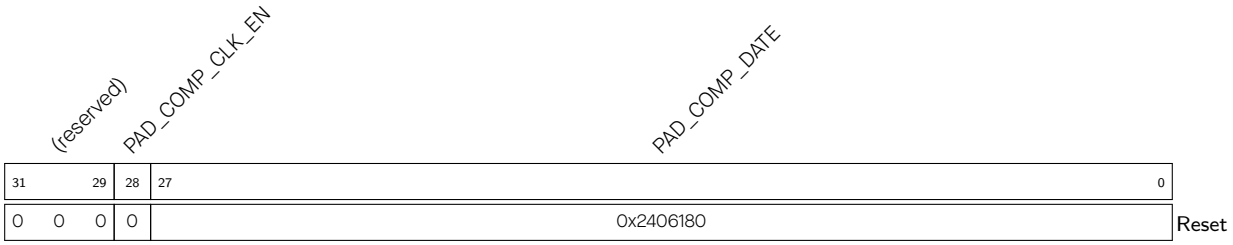
(R/W)

Register 29.17. PAD_COMP_START_REG (0x0040)



PAD_COMP_START 通过软件启动模拟电压比较器任务。(WT)

Register 29.18. PAD_COMP_DATE_REG (0x03FC)



PAD_COMP_DATE 版本控制寄存器。(R/W)

PAD_COMP_CLK_EN 配置是否使能寄存器访问时钟 APB_CLK。(R/W)

第 VII 卷

附录

附录部分包括以下内容：

- 相关文档和资源
- 词汇列表
- 如何配置寄存器的保留域
- 中断配置寄存器
- 修订历史

相关文档和资源

相关文档

- 《ESP32-S31 系列芯片技术规格书》 – 提供 ESP32-S31 芯片的硬件技术规格。
- 《ESP32-S31 硬件设计指南》 – 提供基于 ESP32-S31 芯片的产品设计规范。
- 技术参考手册 LaTeX 源码 – 乐鑫芯片技术参考手册 LaTeX 源码的公开仓库。
  https://github.com/espressif/esp-technical-reference-manual-latex
- 证书
  https://documentation.espressif.com/zh/documentList?t= 证书 &eol=false
- ESP32-S31 产品/工艺变更通知 (PCN)
  https://documentation.espressif.com/zh/documentList?t=PCN&s=ESP32-S31
- ESP32-S31 公告 – 提供有关安全、bug、兼容性、器件可靠性的信息
  https://documentation.espressif.com/zh/documentList?t= 公告 &s=ESP32-S31
- 文档更新和订阅通知
  https://documentation.espressif.com/zh/documentList?eol=false

开发者社区

- 《ESP32-S31 ESP-IDF 编程指南》 – ESP-IDF 开发框架的文档中心。
- ESP-IDF 及 GitHub 上的其它开发框架
  https://github.com/espressif
- ESP32 论坛 – 工程师对工程师 (E2E) 的社区，您可以在这里提出问题、解决问题、分享知识、探索观点。
  https://esp32.com/
- *ESP-FAQ* – 由乐鑫官方推出的针对常见问题的总结。
  https://espressif.com/projects/esp-faq/zh_CN/latest/index.html
- *The ESP Journal* – 分享乐鑫工程师的最佳实践、技术文章和工作随笔。
  https://blog.espressif.com/
- SDK 和演示、App、工具、AT 等下载资源
  https://espressif.com/zh-hans/support/download/sdks-demos

产品

- ESP32-S31 系列芯片 – ESP32-S31 全系列芯片。
  https://espressif.com/zh-hans/products/socs?id=ESP32-S31
- ESP32-S31 系列模组 – ESP32-S31 全系列模组。
  https://espressif.com/zh-hans/products/modules?id=ESP32-S31
- ESP32-S31 系列开发板 – ESP32-S31 全系列开发板。
  https://espressif.com/zh-hans/products/devkits?id=ESP32-S31
- ESP Product Selector（乐鑫产品选型工具）– 通过筛选性能参数、进行产品对比快速定位您所需要的产品。
  https://products.espressif.com/#/product-selector?language=zh

联系我们

- 商务问题、技术支持、电路原理图 & PCB 设计审阅、购买样品（线上商店）、成为供应商、意见与建议
  https://espressif.com/zh-hans/contact-us/sales-questions

词汇列表

外设相关词汇

2D-DMA	2D-DMA 控制器
ADC	ADC 控制器
AES	AES 加速器
ASRC	音频采样率转换器
CORDIC	CORDIC 加速器
DAC	DAC 控制器
DMA	DMA 控制器
ECC	ECC 加速器
ECDSA_DS	ECDSA 数字签名外设
eFuse	eFuse 控制器
EMAC	以太网介质访问控制器
ETM	事件任务矩阵
HMAC	HMAC 加速器
I2C	I2C 控制器
I2S	I2S 控制器
LCD_CAM	LCD 与 Camera 控制器
LEDC	LED PWM 控制器
MCPWM	电机控制脉宽调制器
PARLIO	并行 IO 控制器
PCNT	脉冲计数器控制器
PMS	权限控制
PPA	像素处理加速器
RMT	红外遥控
RNG	随机数发生器
RSA	RSA 加速器
RSA_DS	RSA 数字签名外设
SDC	安全调试控制器
SDHOST	SD/MMC 主机控制器
SHA	SHA 加速器
SPI	SPI 控制器
TIMG	定时器组
TOUCH	触摸传感器
TRACE	RISC-V Trace 编码器
TSENS	温度传感器
TWAI	双线汽车接口
UART	UART 控制器
USB OTG	USB On-The-Go
USB_SERIAL_JTAG	USB 串口/JTAG 控制器
WDT	看门狗定时器
XTS_AES	片外存储器加密与解密

寄存器相关缩写

REG **寄存器**。

SYSREG **系统寄存器**是一组控制系统复位、存储器、时钟、软件中断、电源管理、时钟门控等的寄存器。

ISO **隔离**。如果外设或其他芯片组件断电，其输出信号的管脚（若有）将会浮空。ISO 寄存器会隔离上述引脚并令其保持在某个确定值，以使连接到这些引脚的其他非断电外设/设备免受影响。

NMI **非屏蔽中断**是一种 CPU 指令无法禁用或忽略的硬件中断。出现此类中断说明发生严重错误。

W1TS 添加到寄存器/字段名称中的缩写，表示此类寄存器/字段用于置位名称相似寄存器中的相应字段。例如，寄存器 GPIO_ENABLE_W1TS_REG 用于置位寄存器 GPIO_ENABLE_REG 中的相应字段。

W1TC 与 *W1TS* 相同，但用于清除相应寄存器中的字段。

寄存器的访问类型

TRM 章节 寄存器列表 和 寄存器 详述了寄存器及其字段的访问类型。

常用访问类型及组合如下：

- RO
- WO
- WT
- R/W
- R/W1
- WL
- R/W/SC
- R/W/SS
- R/W/SS/SC
- R/WC/SS
- R/WC/SC
- R/WC/SS/SC
- R/WS/SC
- R/WS/SS
- R/WS/SS/SC
- R/SS/WTC
- R/SC/WTC
- R/SS/SC/WTC
- RF/WF
- R/SS/RC
- varies

下文提供了所有访问类型的具体描述。

R	**软件可读**。用户软件可以读取此寄存器/字段；通常与其他访问类型结合使用。
RO	**软件只读**。用户软件只可读取此寄存器/字段。
HRO	**硬件只读**。仅硬件可以读取此寄存器/字段；用于存储变量参数的默认设置。
W	**软件可写**。用户软件可以写入此寄存器/字段；通常与其他访问类型结合使用。
WO	**软件只写**。用户软件只可写入此寄存器/字段。
W1	**软件只写一次**。用户软件只可写入一次此寄存器/字段；只允许写入 1，写入 0 无效。
SS	**硬件置位**。在指定事件中，硬件自动将 1 写入此寄存器/字段；与一位字段一同使用。
SC	**硬件清零**。在指定事件中，硬件自动将 0 写入此寄存器/字段；与一位和多位字段一同使用。
SM	**硬件修改**。在指定事件中，硬件自动将指定值写入此寄存器/字段；与多位字段一同使用。
SU	**硬件更新**。在指定事件中，硬件自动更新此寄存器/字段；与多位字段一同使用。
RS	**软件读置位**。如果用户软件读取此寄存器/字段，硬件会自动写 1。
RC	**软件读清零**。如果用户软件读取此寄存器/字段，硬件会自动写 0。
RF	**软件读 FIFO**。如果用户软件将新数据写入 FIFO，寄存器/字段会自动读取。
WF	**软件写 FIFO**。如果用户软件将新数据写入此寄存器/字段，寄存器/字段会自动通过 APB 总线将数据传递到 FIFO。
WS	**软件写置位**。如果用户软件写入此寄存器/字段，硬件会自动置位此寄存器/字段。
W1S	**软件写 1 置位**。如果用户软件将 1 写入此寄存器/字段，硬件会自动置位此寄存器/字段。
WOS	**软件写 0 置位**。如果用户软件将 0 写入此寄存器/字段，硬件会自动置位此寄存器/字段。
WC	**软件写清零**。如果用户软件写入此寄存器/字段，硬件会自动清零此寄存器/字段。
W1C	**软件写 1 清零**。如果用户软件将 1 写入此寄存器/字段，硬件会自动清零此寄存器/字段。
WOC	**软件写 0 清零**。如果用户软件将 0 写入此寄存器/字段，硬件会自动清零此寄存器/字段。
WT	**软件写产生边沿触发信号**。如果用户软件将 1 写入此字段，将会产生边沿触发信号（APB 总线中的脉冲）或清除相应的 WTC 字段（详见 WTC）。

WTC **软件写其他寄存器位清零本寄存器位**。如果用户软件将 1 写入相应的 WT 字段，硬件会自动清除此字段（详见 WT）。

W1T **软件写 1 取反**。如果用户软件将 1 写入此字段，硬件会自动取反相应字段，否则不会取反。

W0T **软件写 0 取反**。如果用户软件将 0 写入此字段，硬件会自动取反相应字段，否则不会取反。

WL **软件仅在锁禁用时写**。如果锁被禁用，用户软件可以写入此寄存器/字段。

varies **访问类型不定**。此寄存器中的不同字段访问类型可能不同。

如何配置寄存器的保留域

概述

寄存器的保留域指的是寄存器中不对用户开放、或者配置为非默认值时会导致不可预测的结果的域。

如何配置保留域

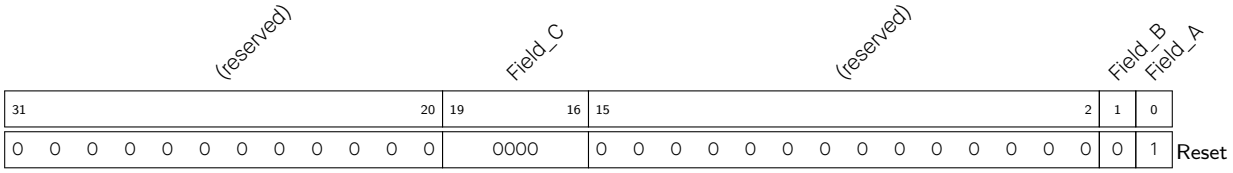
保留域的值不能修改。由于写寄存器时必须整体写，不能只写部分域，因此，在写带有保留域的寄存器时需要特别注意，只能采用以下两种方式：

1. 读取寄存器的值，仅修改需要配置的域，然后将修改后的值以及其他未修改的值一起写回寄存器，这样保留域的值就会保持不变。

   或者

2. 仅修改需要配置的域，然后将保留域的默认值写回寄存器。默认值即寄存器图表中的“Reset”值。例如，寄存器 X 中 Field_A 的默认值为 1。

Register 29.19. 寄存器 X (地址)



假设您要将 寄存器 X 的 Field_A、Field_B 和 Field_C 设置为 0x0、0x1 和 0x2，您可以：

- 使用第 1 种方式，修改这三个域的值，然后把读到的值写回寄存器。假设寄存器读取的值为 0x0000_0003，修改三个域的值后，将值 0x0002_0002 写入寄存器。
- 使用第 2 种方式，修改这三个域的值，然后把保留域的默认值写回寄存器，即把 0x0002_0002 写入寄存器。

中断配置寄存器

大部分外设的内部中断源都有以下配置寄存器：

- **RAW**（原始状态）寄存器：该寄存器指示原始中断状态，每个位对应一个内部中断源。当中断源触发时，其 RAW 位为 1。
- **ENA**（使能）寄存器：该寄存器用于启用或禁用内部中断源，每个位对应一个内部中断源。

  通过操作 ENA 寄存器，可以根据需要屏蔽或取消屏蔽某个内部中断源。当中断源被屏蔽（禁用）时，它不会生成中断信号，但仍可以从 RAW 寄存器中读取其值。
- **ST**（状态）寄存器：该寄存器指示中断源的屏蔽状态，每个位对应一个内部中断源。ST 位为 1 代表 RAW 位和 ENA 位都为 1，即中断源已生成且未被屏蔽。RAW 位和 ENA 位的值为其他组合时，ST 位为 0。

  ENA/RAW/ST 寄存器的配置见表 29.9-4。
- **CLR**（清除）寄存器：CLR 寄存器负责清除内部中断源。写 1 将清除该位对应的中断源。

表 29.9-4. ENA/RAW/ST 寄存器的配置

ENA 位的值	RAW 位的值	ST 位的值
0	忽略	0
1	0	0
	1	1

修订历史

日期	版本	发布说明
2026-09-21	v0.1	首次发布



免责声明和版权公告

本文档中的信息，包括供参考的 URL 地址，如有变更，恕不另行通知。

本文档可能引用了第三方的信息，所有引用的信息均为“按现状”提供，乐鑫不对信息的准确性、真实性做任何保证。

乐鑫不对本文档的内容做任何保证，包括内容的适销性、是否适用于特定用途，也不提供任何其他乐鑫提案、规格书或样品在他处提到的任何保证。

乐鑫不对本文档是否侵犯第三方权利做任何保证，也不对使用本文档内信息导致的任何侵犯知识产权的行为负责。本文档在此未以禁止反言或其他方式授予任何知识产权许可，不管是明示许可还是暗示许可。

Wi-Fi 联盟成员标志归 Wi-Fi 联盟所有。蓝牙标志是 Bluetooth SIG 的注册商标。

文档中提到的所有商标名称、商标和注册商标均属其各自所有者的财产，特此声明。

版权归 © 2026 乐鑫信息科技（上海）股份有限公司。保留所有权利。

www.espressif.com